ARM assembly programozás C-fejlesztőknek: AArch64-alkalmazásszinttől Cortex-M bare-metalig

Tartalomjegyzék

I. RÉSZ – AArch64: alkalmazásszintű ARM assembly Linux alatt **5**

1 Olvasói útmutató: kinek szól a könyv, és hogyan olvasd **6**
1.1 A könyv használata: C-előismeret, tanulási kimenetek, belépési pontok 6

2 Bevezetés: hardver, ISA és a gép nyelve **16**
2.1 Miért assembly: amit a C elrejt 16
2.2 Számrendszerek és bitszintű reprezentáció 24
2.3 Számítógép-architektúra fogalmai: ISA, mikroarchitektúra, regiszter, busz 34
2.4 Az ARM család áttekintése: profilok, verziók és a könyv két része 43
2.5 C fordítási láncolat és hova kerül az assembly 52

3 Fejlesztői környezet és labor: eszközlánc, QEMU, build-munkafolyamat **63**
3.1 Labor-alapkörnyezet: eszközlánc telepítése és verifikáció 63
3.2 Assemble-link-run munkafolyamat natív és cross környezetben 76
3.3 Opcionális fizikai hardver: Raspberry Pi 64-bit módban 84
3.4 Szerkesztő és debug-eszközök: GDB, objdump, readelf első áttekintése 92

4 Az első program: regiszterek, ELF és a Linux syscall-felszín **101**
4.1 Az A64 regiszterkészlet áttekintése 101
4.2 Hello world tisztán assemblyben: szekciók, `_start`, syscall hívás 110
4.3 Az ELF formátum alapjai és a program életciklusa 118
4.4 Linux syscall-konvenció AArch64-en 127

5 Adatmozgatás, aritmetika és állapotjelzők **136**
5.1 MOV-variánsok és immediate-kódolás 136
5.2 Egész aritmetika: ADD/SUB/MUL/DIV és a kettes komplemens viselkedése . . 144
5.3 Az NZCV állapotjelzők és eredetük 151
5.4 Feltételes végrehajtás: CSET/CSEL és feltételkódok 161

6 Vezérlésátadás: elágazás és ciklusok C-szemmel **169**
6.1 Feltétel nélküli és feltételes elágazás 169
6.2 C if/else és switch leképezése assemblyre 176
6.3 Ciklusszervezés: while/for/do-while mintázatok assemblyben 186
6.4 Hívás és visszatérés: BL/RET bevezető mintája 194

7 Memóriacímzés és adatszerkezetek **203**
7.1 Load/store címzési módok 203
7.2 Tömbök és indexelt elérés assemblyben 211
7.3 Struktúrák, igazítás és mezőelérés 218
7.4 Bájtsorrend: endianness és miért releváns a határon 228
7.5 Statikus és dinamikus memória: .bss, .data és egyszerű allokáció 235

8 Függvények és az AAPCS64 hívási konvenció **244**
8.1 A stack frame felépítése és a verem iránya 244

8.2 Az AAPCS64 regiszterszerződés . . . 252
8.3 Prológus és epilógus: regisztermentés és keretkezelés . . . 261
8.4 Rekurzív függvények assemblyben . . . 278
8.5 Több visszatérési érték és struktúra-visszaadás szerződése . . . 286

9 Bitmanipuláció és logikai műveletek 295
9.1 Logikai alapműveletek és bitmaszkolás . . . 295
9.2 Shift és rotate: LSL/LSR/ASR/ROR gyakorlati mintázatok . . . 302
9.3 Bitfield-utasítások: BFI/UBFX/SBFX és csomagolt adatszerkezetek . . . 310
9.4 Gyakorlati bitmanipulációs feladatok: popcount, parity, bitfordítás . . . 319

10 C-assembly interoperabilitás és fordítóprogram-elemzés 328
10.1 Inline assembly GCC-ben . . . 328
10.2 Assembly függvény hívása C-ből és C hívása assemblyből . . . 338
10.3 Fordítói optimalizálási szintek hatása a generált kódra . . . 347
10.4 Tipikus fordítói mintázatok felismerése . . . 356

11 Debugging, disassembly és instrumentáció 364
11.1 GDB assembly szinten . . . 364
11.2 Disassembly objdump/readelf-fel . . . 372
11.3 Hibakeresési esettanulmány: szegmentálási hiba gyökérokának megtalálása . . . 380
11.4 Egyszerű instrumentáció: ciklusszámlálás és időmérés . . . 388

12 Lebegőpontos aritmetika és NEON/SIMD vektorizáció 397
12.1 A lebegőpontos regiszterkészlet és az IEEE 754 ábrázolás . . . 397
12.2 Skaláris lebegőpontos utasítások . . . 406
12.3 A NEON regiszterkészlet és adatelrendezés . . . 415
12.4 Vektorizált összeadás/szorzás: egy skaláris ciklus NEON-átirata . . . 423
12.5 NEON-gyorsulás mérése és korlátai . . . 432

13 Teljesítmény-tudatos kód: cache, pipeline és mérés 444
13.1 Cache-hierarchia és a memóriahozzáférési minták hatása . . . 444
13.2 Pipeline alapfogalmak és adatfüggőségi akadályok . . . 454
13.3 Branch prediction és elágazás-szervezés hatása . . . 465
13.4 Mikrobenchmark készítése és eredmények értelmezése . . . 474

14 Biztonsági szemlélet: verem, védelem és ROP 484
14.1 Verem-túlcsordulás mechanizmusa assembly szinten . . . 484
14.2 Védelmi mechanizmusok: canary, ASLR, NX/DEP, PAC . . . 492
14.3 ROP-lánc felismerése és miértje . . . 505
14.4 Biztonságos assembly- és C-interop gyakorlat . . . 515

15 Záróprojekt I: teljesítménykritikus assembly modul C programban 526
15.1 Projektspecifikáció: feladat, interfész és mérési terv . . . 526
15.2 Megvalósítás: az assembly modul implementációja . . . 540
15.3 Mérés és validáció . . . 553
15.4 Visszacsatolás és auditmódszer . . . 572

II. RÉSZ – Cortex-M: bare-metal mikrokontroller-programozás 587

16 Thumb-2, a Cortex-M futási modell és a minimal runtime 588
16.1 A-profile és M-profile különbségei: miért más a Cortex-M . . . 588
16.2 Thumb-2 ISA áttekintés . . . 606

16.3 Memóriatérkép és a bare-metal futási modell . . . 613
16.4 Reset és boot-folyamat: vektortábla . . . 621
16.5 Linker script alapjai: szekciók memóriába helyezése . . . 633
16.6 Startup kód: .data másolása, .bss törlése . . . 644
16.7 Firmware build és QEMU-betöltés munkafolyamat . . . 652

17 Kivételek és megszakítások: NVIC **660**
17.1 A Cortex-M kivételmodell: prioritás és vektortábla-bejegyzések . . . 660
17.2 NVIC konfiguráció: engedélyezés és prioritás . . . 668
17.3 ISR írása: C és assembly vegyesen . . . 678
17.4 Megszakítás-latencia mérése és korlátai . . . 687

18 Memory-mapped perifériák és GPIO **700**
18.1 Memory-mapped I/O alapelve . . . 700
18.2 GPIO közvetlen vezérlése: bitbanging . . . 709
18.3 Periféria-regiszter absztrakció C struktúrával . . . 720

19 Megszakítás-vezérelt I/O és driver minta **730**
19.1 Polling versus interrupt-vezérelt I/O . . . 730
19.2 Egyszerű UART driver: inicializálás, küldés/vétel megszakítással . . . 738
19.3 Timer-alapú időzítés és periodikus feladatok . . . 750

20 Záróprojekt II: interrupt-vezérelt Cortex-M firmware-modul **765**
20.1 Projektspecifikáció: periféria, megszakítás-logika és mérési terv . . . 765
20.2 Megvalósítás: driver és ISR integrálása a minimal runtime-ba . . . 774
20.3 Mérés, validáció és a QEMU-alapú teszt reprodukálhatósága . . . 791
20.4 Auditszempontok bare-metal firmware-hez . . . 809

I. RÉSZ – AArch64: alkalmazásszintű ARM assembly Linux alatt

1 Olvasói útmutató: kinek szól a könyv, és hogyan olvasd

1.1 A könyv használata: C-előismeret, tanulási kimenetek, belépési pontok

1.1.1 Motiváció

Mielőtt bármelyik fejezetbe belekezdenél, két kérdésre érdemes néhány perc alatt válaszolni: *van-e meg a szükséges C-alapom*, és *hol érdemes belépnem a könyvbe*. Ha ezt kihagyod, két tipikus veszteség ér: vagy olyan alfejezeteken időzöl fölöslegesen, amelyeket már tudsz (például a számrendszereket), vagy olyan szerződéseket (hívási konvenció, NVIC-regiszterek) ugrasz át „ismerősnek tűnik” alapon, amelyeket a könyv később mindenhol feltételez. Ez az alfejezet ezért nem technikai tananyag, hanem útvonalterv: önellenőrző listát, konkrét tanulási kimeneteket és belépési pontokat ad.

1.1.2 Definíció: előismeret-küszöb és belépési pont

Előismeret-küszöb: a minimális, cselekvésben mérhető C-tudás (nem „közepes szintű C”, mert az nem mérhető), amellyel a könyv technikai alfejezetei a C nyelv újratanítása nélkül értelmezhetők.

Belépési pont: a könyvtérkép alfejezete, ahonnan az olvasó a meglévő tudásához igazítva kezdheti a tanulást — a küszöb mindenkire érvényes, a belépési pont csak a kezdőpozíciót választja meg.

1.1.3 Önellenőrző lista: a szükséges C-tudás

A következő állításoknak kell igaznak lennie rád. Ha valamelyik nem az, azt előbb pótolni kell; a könyv ezeket nem tanítja újra, csak használja.

- **Mutató és tömb:** meg tudsz írni egy függvényt, amely `int` tömböt és annak hosszát kapja, és mutatóaritmetikával (nem indexeléssel) járja be. Érted a különbséget `int *p[4]` és `int (*p)[4]` között.
- **Struktúra és memórialayout:** tudod, miért nem feltétlenül `sizeof(struct)` a mezők méreteinek összege, és miért számít a mezők sorrendje.
- **Többfájlos build és Makefile:** létre tudsz hozni legalább két `.c` és egy `.h` fájlból álló programot saját `Makefile`-lal, `all` és `clean` céllal; a fejlécbe deklarációt, egyetlen fordítási egységbe definíciót teszel, és érted, miért nem elég a `#include`: a linkernek a definíciót tartalmazó objektumra is szüksége van.
- **Fordítási lánc fogalmi szinten:** meg tudod mondani, mit csinál a preprocesszor, a fordító és a linker, és miért kapcsolódik össze két `.o` fájl egy futtatható állományba.

- **Parancssor:** otthonosan mozogsz shellben, fájlt másolsz, szövegszerkesztőt használsz, és nem riadsz meg egy nem nulla kilépési kódtól.

Ami **nem** része a küszöbnek: assembly, regiszterek, ISA, NEON/SIMD, megszakításkezelés, bare-metal fejlesztés, QEMU. Ezeket a könyv nulláról építi fel.

1.1.3.1 Az önellenőrző lista rövid válaszai

- **Mutató és tömb:** `int *p[4]` négy `int*` elemet tartalmazó tömb; `int (*p)[4]` egy négy `int`-ből álló tömb pointere. Utóbbinál `p + 1` négy `int`-nyivel (16 bájttal) lép, előbbinél egy pointerrel.
- **Struktúra és memórialayout:** a mezők közé igazítási hézag kerülhet, ezért `sizeof(struct)` nagyobb lehet a mezőméretek összegénél; a sorrend a hézagok miatt számít.
- **Többfájlos build:** a deklaráció a fejlécbe, a definíció pontosan egy `.c` fájlba kerül; a linker a definíciót tartalmazó objektumot keresi, a `#include` önmagában nem elég.
- **Fordítási lánc:** a preprocesszor szöveget cserél, a fordító assemblyt és objektumot állít elő, a linker kapcsolja össze az objektumokat és a könyvtárakat.
- **Parancssor:** a nem nulla kilépési kód hibát jelez; `echo $?` mutatja az utolsó parancs állapotát.

Ajánlott C-források a hiány pótlására: Brian W. Kernighan és Dennis M. Ritchie, *The C Programming Language*; Jens Gustedt, *Modern C* (ingyenesen elérhető).

1.1.4 Tanulási kimenetek

Rész	A rész végére önállóan…
I. rész – AArch64	A64 assembly függvényt írsz, assemblálsz (fordítasz), linkelsz és GDB-vel hibakeresel; a függvényt C-ből hívod az AAPCS64 szerződés szerint; felismered a fordító tipikus optimalizálási mintázatait; egy skaláris ciklust NEON-nal vektorizálsz, és a gyorsulást Amdahl-korláttal értelmezed; felismered a memóriabiztonsági védelmi mechanizmusokat (verem-canary, ASLR, NX, PAC).
II. rész – Cortex-M	Bare-metal startup kódot és linker scriptet írsz; vektortáblát és NVIC-vezérelt megszakításkezelőt állítasz be; memory-mapped perifériadrivert (GPIO, UART, timer) implementálsz; a firmware-t `qemu-system-arm` alatt reprodukálhatóan futtatod és mérőponttal igazolod.

1.1.5 Belépési pontok és átugorható szakaszok

Olvasóprofil	Belépési pont	Átugorható	**Nem** átugorható
Assembly-kezdő, C-tudás megvan	Miért assembly: amit a C elrejt	—	semmi
ISA/architektúra-alapokat ismer, assemblyben kezdő	Az A64 regiszterkészlet áttekintése	Számrendszerek és bitszintű reprezentáció (a 2.2 belépőtesztje alapján), részben Számítógép-architektúra fogalmai	I. rész: Az AAPCS64 regiszterszerződés; II. rész: az AAPCS32 hívási szabályai (A-profile és M-profile különbségei) és NVIC konfiguráció: engedélyezés és prioritás
AArch64-et ismer, csak Cortex-M kell	Thumb-2, a Cortex-M futási modell és a minimal runtime, az A-profile és M-profile különbségei áthidaló példájával	I. RÉSZ – AArch64 nagy része; Az AAPCS64 regiszterszerződés csak összehasonlításként ajánlott	az AAPCS32 hívási szabályai (alkalmazásuk: A-profile és M-profile különbségei, ISR írása: C és assembly vegyesen), Memóriatérkép és a bare-metal futási modell, NVIC konfiguráció: engedélyezés és prioritás

Kötelező környezetlépés útvonalanként. Minden belépési út előtt kell környezet; a belépési pont nem jelent előkészítés nélküli indulást:

Útvonal	Kötelező előzetes környezetlépés	Nem tételezünk fel előre elkészített fájlt
Assembly-kezdő	Labor-alapkörnyezet (eszközlánc, QEMU, könyvtárrend); az 2.1 futtatható bemutatója *előzetes demonstráció*, a telepítést az 3.1 adja	az 2.1 példái saját `mkdir`-rel indulnak
ISA-t ismer	ugyanaz az 3.1 telepítés, utána 4.1	igen, minden másolás forrása korábbi, megnevezett lépés
Csak Cortex-M	3.1 M-profile részének smoke-tesztje (`cm/a006`), és a kísérőcsomag starter fájljai (`lab-mps2.ld`, `lab-startup.s`: a Linker script és a Startup kód kanonikus példánya, a csomagban `tools/cm-labs/canon/`)	a kanonikus pár a csomagból másolható; működését az 16.5–16.6 később bontja ki

Az átugorható szakaszok is *alap-ISA-ismeretek*: ha a kettes komplemens, a bináris jelölés

vagy az ISA/mikroarchitektúra megkülönböztetése már biztos, a Számrendszerek és bitszintű reprezentáció és a Számítógép-architektúra fogalmai valóban kihagyható.

Az át nem ugorható szerződések más okból kötelezők, és **útvonalanként** különböznek:

Útvonal	Át nem ugorható szerződés	Nem előfeltétel
AArch64 (I. rész)	Az AAPCS64 regiszterszerződés	az NVIC és az AAPCS32
Cortex-M (II. rész)	az AAPCS32 hívási szabályai (A-profile és M-profile különbségei, ISR írása: C és assembly vegyesen), Memóriatérkép és a bare-metal futási modell, NVIC konfiguráció: engedélyezés és prioritás	az AAPCS64 (csak összehasonlításként ajánlott)

Egyik sem a könyv saját konvenciója:

- Az **AAPCS64** az Arm 64 bites architektúrájának **eljáráshívási szabványa** (*Procedure Call Standard*, PCS). Az argumentumok és eredmények átadását, valamint a hívásokon át megőrzendő gépállapotot rögzíti. A PCS a tágabb **bináris alkalmazásinterfész** (ABI) hívási szabályokat rögzítő része; az ABI ezen túl például az objektumformátumot és az adatábrázolást is szabályozza (Az AAPCS64 regiszterszerződés). A könyv AArch64-célján (Linux, little-endian, LP64) az alap AAPCS64 érvényes. Más platformok, például más operációs rendszerek ABI-jai erre építve saját kiegészítéseket és eltéréseket rögzíthetnek.
- Az **NVIC** az M-profile architektúra része, de hozzá implementáció- és panelspecifikus beállítások társulnak: a ténylegesen megvalósított megszakításvonalak száma, a prioritásmezők kihasznált bitjei és a perifériák IRQ-számai a konkrét magtól, SoC-tól és paneltől függenek.

Ezeket el kell választani a választott labor *saját* szerződésétől — például a QEMU-célgéptől (`mps2-an385`) és az azon érvényes perifériacímektől —, amely csak a könyv példáira érvényes. A választott útvonal szerződését azért kell tapasztalt olvasóként is végigvenni, mert az útvonal későbbi példái rá épülnek, nem azért, mert a könyv találta ki: az I. rész C-ből hívható assembly függvényei az AAPCS64-re, a II. rész megszakításos laborjai az AAPCS32-re és az NVIC-re.

A **kizárólag Cortex-M-et tanuló olvasó** kötelező PCS-belépője az **AAPCS32** (*Procedure Call Standard for the Arm Architecture*), a 32 bites Arm architektúra, köztük az M-profile eljáráshívási szabványa. Az AAPCS64 regiszterszerződés ilyenkor hasznos összehasonlítás, de nem helyettesíti az AAPCS32-t: eltér a regiszterszélesség (32 vs. 64 bit), az argumentumregiszterek köre (R0–R3 vs. X0–X7), a hívott által megőrzendő regiszterek halmaza (R4–R8, R10, R11, az R9 platformfüggő vs. X19–X29 és a V8–V15 alsó 64 bitje) és a verem igazítási követelménye (nyilvános interfészen 8 vs. 16 bájt).

1.1.5.1 Hogyan használd a feladatokat és a megoldásokat

Ez az útmutató a könyv **minden** feladatsorára és megoldására érvényes; az alfejezetek nem ismétlik meg.

- **Címkék.** Minden feladat három címkét kap: **típus** (magyarázó, implementációs, hibakeresési/mérési, integráló), **nehézség** (alapszint, középszint, elmélyült) és **cél** (megértésellenőrzés, rutinépítés, integráció). A forma egységes: **N. (típus, szint — cél: …)**.

- **Kétlépcsős megoldások.** Ahol a megoldás két részre oszlik, előbb egy rövid **segítség** áll, amely irányt ad, de nem árulja el a választ, utána a **teljes megoldás**. Érdemes a segítségnél megállni, és csak akkor továbbolvasni, ha önállóan elakadtál.
- **Futtatási feltételek.** A laborok munkakönyvtára, build- és futtatóparancsa, valamint elvárt kimenete mindig az adott alfejezetben áll; ezeket az útmutató nem helyettesíti.

1.1.6 0. labor (opcionális): előismeret-önellenőrző build

Ez a labor a küszöböt ellenőrzi; ha az önellenőrző lista minden pontja biztos, kihagyható, és a 3.1 telepítés után közvetlenül a 3. fejezet példáival lehet folytatni.

Munkakönyvtár: `~/arm-labor/a64/a000/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a000
cd ~/arm-labor/a64/a000
```

Az alábbi háromfájlos program a küszöb elemeit (mutató, struktúra, többfájlos build) gyakorolja: két külön fordítási egységből (`lab-main.c`, `lab-sum.c`) és egy közös fejlécből (`lab-sum.h`) áll. A fejléc a `struct pair` típust és a `sum_values` **deklarációját** adja, a függvény **definíciója** egyetlen fordítási egységben, a `lab-sum.c`-ben van. Célja nem a funkcionalitás, hanem hogy *te* tudd összeilleszteni és lefordítani.

```
/* lab-sum.h */
#ifndef LAB_SUM_H
#define LAB_SUM_H

struct pair { int key; int value; };

int sum_values(const struct pair *p, int n);   /* deklaráció */

#endif

/* lab-sum.c */
#include "lab-sum.h"

int sum_values(const struct pair *p, int n) {  /* definíció */
    int s = 0;
    for (const struct pair *q = p; q < p + n; ++q) s += q->value;
    return s;
}

/* lab-main.c */
#include <stdio.h>
#include "lab-sum.h"

int main(void) {
    struct pair v[3] = {{1,10},{2,20},{3,30}};
    printf("%d %zu\n", sum_values(v, 3), sizeof(struct pair));
    return 0;
}
```

A `Makefile` mindkét objektumot linkeli, és mindkét objektum függőségei között szerepel a fejléc:

```
# Makefile
CC = gcc
CFLAGS = -Wall -Wextra -O2 -std=c11
OBJS = lab-main.o lab-sum.o
```

```
.PHONY: all clean
all: lab-a000

lab-a000: $(OBJS)
        $(CC) $(CFLAGS) -o $@ $^

lab-main.o: lab-main.c lab-sum.h
        $(CC) $(CFLAGS) -c -o $@ $<

lab-sum.o: lab-sum.c lab-sum.h
        $(CC) $(CFLAGS) -c -o $@ $<

clean:
        rm -f lab-a000 $(OBJS)
```

Futtatás (target: x86_64 host, natív `gcc`; a kód hordozható, AArch64-en azonos kimenet várható):

```
make
./lab-a000
echo $?
```

Várt kimenet: `60 8`, majd a kilépési állapot `0`. Az `60` a három `value` összege; a `8` a `struct pair` mérete ezen az ABI-n (két 4 bájtos `int`, igazítás miatt hézag nélkül). A pontos méret ABI-függő; a `sizeof` kiíratása éppen ezért része az ellenőrzésnek.

Elfogadási próbák. A futó program önmagában még nem bizonyítja, hogy a többfájlos build szerkezete helyes. Két próbának is teljesülnie kell:

1. *A definíció valóban a másik objektumban van.* Linkeld a programot a `lab-sum.o` nélkül. Ez egy **negatív próba**: itt a **nem nulla** kilépési állapot az elvárt eredmény.

   ```
   make -B OBJS=lab-main.o
   echo "make kilépési állapot: $?"    # elvárt: nem nulla (GNU Make: 2)
   ```

   A `-B` kapcsoló kötelező. Egy sikeres `make` után a `lab-a000` újabb, mint minden függősége, és a Make az időbélyegek alapján dönt: az `OBJS` lista megváltozása önmagában nem kényszerít új linkelést. A `-B` nélkül a parancs `Nothing to be done for 'all'.` üzenettel, 0-s állapottal érne véget, és a próba semmit sem bizonyítana. Ugyanezt a próbát a Make megkerülésével is elvégezheted: `gcc -o lab-missing lab-main.o`.

   Valódi linkerhibát kell kapnod, nem fordítási hibát: a `lab-main.c` a fejléc deklarációja miatt hibátlanul lefordul, csak a definíció hiányzik.

   ```
   gcc -Wall -Wextra -O2 -std=c11 -c -o lab-main.o lab-main.c
   gcc -Wall -Wextra -O2 -std=c11 -o lab-a000 lab-main.o
   /usr/bin/ld: lab-main.o: in function `main':
   lab-main.c:(.text.startup+0x39): undefined reference to `sum_values'
   collect2: error: ld returned 1 exit status
   make: *** [Makefile:10: lab-a000] Error 1
   make kilépési állapot: 2
   ```

   (A `+0x39` eltolás fordítóverziótól és kapcsolóktól függ.) Ezután térj vissza a helyes objektumlistához, és ellenőrizd újra a kimenetet:

   ```
   make -B && ./lab-a000                # elvárt: 60 8
   ```

2. *A fejléc módosítása az érintett objektumokat újrafordítja.* Egy sikeres `make` után:

   ```
   touch lab-sum.h
   make
   ```

Mindkét `gcc -c` sornak (`lab-main.o` és `lab-sum.o`) meg kell jelennie, majd a linkelésnek. Ellenpróbaként `touch lab-sum.c` után csak a `lab-sum.o` fordul újra.

A két eltéréstípust érdemes külön kezelni. Ha az összeg nem `60`, vagy bármelyik elfogadási próba eltér a fentitől, a **build szerkezete** vagy a küszöb valamelyik eleme bizonytalan; azt pótolni kell a továbblépés előtt. Ha viszont csak a második szám tér el a `8`-tól, az nem build-hiba, hanem a cél-**ABI** tulajdonsága. Ilyenkor azt kell ellenőrizni, hogy a kapott méret megfelel-e az adott ABI `int`-méretének és igazítási szabályának (két `int` mező, hézag nélkül: `2 * sizeof(int)`).

1.1.7 Tipikus hibák és ellenpróbák

- **Tünet:** „értem a mutatókat”, de a `q < p + n` feltétel bizonytalan. **Hipotézis:** a mutatóaritmetika léptéke nem tiszta. **Adatgyűjtés:** írasd ki `(char*)q - (char*)p` értékét a ciklusban. **Egy változtatás:** cseréld a ciklust indexelésre, és hasonlítsd össze. **Ellenőrzés:** a két változat azonos eredményt adjon.
- **Tünet:** a Makefile „működik”, de egy forrás- vagy fejlécmódosítás után nem épül újra. **Hipotézis:** hiányzó vagy hibás függőségi sor. **Ellenőrzés:** `make -n` száraz futtatással nézd meg, mit tenne.
- **Tünet:** a fejléc módosítása után a program régi viselkedést mutat, vagy a két objektum eltérő `struct pair`-t lát. **Hipotézis:** a `.h` hiányzik az objektumok függőségi sorából. **Ellenőrzés:** `touch lab-sum.h`, majd `make -n` — mindkét objektum fordítását ki kell írnia.
- **Tünet (AArch64-útvonal):** átugrottad az AAPCS64 regiszterszerződés alfejezetet, mert „a hívási konvenciót úgyis ismerem”. **Ellenpróba:** próbáld meg megmondani, mely regisztereket kell mentenie egy assembly függvénynek, mielőtt a Függvények és az AAPCS64 hívási konvenció fejezethez érsz. Ha nem megy, nem volt átugorható. Cortex-M-útvonalon ugyanezt a kérdést az AAPCS32-re tedd fel.

1.1.8 Rövid összefoglaló

Az **előismeret-küszöb** az a cselekvésben mérhető C-tudáshalmaz (mutató, struktúra, Makefile, fordítási lánc), amely nélkül a könyv technikai fejezetei nem értelmezhetők. A **belépési pont** a könyvtérkép azon alfejezete, ahonnan a küszöböt teljesítő olvasó a tudásához igazítva indul: assembly-kezdőnek Miért assembly: amit a C elrejt, ISA-alapokat ismerőnek Az A64 regiszterkészlet áttekintése, AArch64-ismerőnek Thumb-2, a Cortex-M futási modell és a minimal runtime. Az **át nem ugorható szerződések** útvonalanként különböznek. Az AArch64-útvonalon ez Az AAPCS64 regiszterszerződés, az NVIC itt nem előfeltétel. A Cortex-M-útvonalon az AAPCS32, a memóriatérkép és az NVIC konfiguráció: engedélyezés és prioritás; az AAPCS64 itt csak összehasonlításként hasznos, és nem helyettesíti az M-profile hívási szabályait. Egyik sem a könyv saját konvenciója: az AAPCS64 és az AAPCS32 eljáráshívási szabvány (PCS), vagyis az ABI hívási szabályokat rögzítő része; az NVIC architekturális komponens, amelyhez implementáció- és panelspecifikus beállítások társulnak. Ezeket el kell választani a választott labor saját szerződésétől.

1.1.9 Források és ellenőrzési hiányok

A könyv egészében hivatkozott elsődleges források élő linkje és aktuális verziója:

- *Arm Architecture Reference Manual for A-profile architecture* (Arm DDI0487) és *Armv7-M Architecture Reference Manual* (Arm DDI0403E) — a `developer.arm.com/documentation/...` linkek élnek (a `support.arm.com`-ra irányított átirányítás megerősített), de az oldal JavaScript-renderelésű, ezért a pontos fejezetszám/kiadásbetű automatikus lekéréssel nem volt kinyerhető.
- *Procedure Call Standard for the Arm 64-bit Architecture* (AAPCS64) — github.com/ARM-software/abi-aa, az AAPCS64 regiszterszerződés elsődleges forrása. **Ellenőrizve**: aktuális kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-vonatkozású részek Beta).
- *Procedure Call Standard for the Arm Architecture* (AAPCS32) — github.com/ARM-software/abi-aa, a Cortex-M-útvonal hívási szabályainak elsődleges forrása. Az aktuális kiadás **nincs ellenőrizve**. Az NVIC architekturális leírásának forrása az *Armv7-M Architecture Reference Manual* (Arm DDI0403E); a megvalósított IRQ-k száma és prioritásbitjei a konkrét mag és panel dokumentációjából ellenőrizendők.
- GNU Binutils, GCC és GDB kézikönyv — három **különálló** projekt, három különálló dokumentációval: sourceware.org/binutils/docs/ (**ellenőrizve**: binutils 2.47), gcc.gnu.org/onlinedocs/ (**ellenőrizve**: legújabb kiadás GCC 16.2), sourceware.org/gdb/current/onlinedocs/gdb.html/ (**ellenőrizve**, de a „current” egy fejlesztői git-pillanatkép, nem konkrét stabil kiadás).
- A könyv kísérőcsomagja (laborforrások, helyettesítő futtatók, próbák és naplók) — `dsk-arm-assembly-tools-1.0.tar.gz`, github.com/gellistvan/dsk-arm-assembly-tools, v1.0; ugyanez a csomag a könyv HTML-kiadása mellett is letölthető: `tar.gz`, `.sha256`; tartalmát, ellenőrzőösszegét és futtatását az A-profile és M-profile különbségei alfejezet Források-szakasza írja le. A könyvben `tools/…` alakban hivatkozott fájlok ennek a csomagnak a fájljai.
- QEMU System és User Mode Emulation dokumentáció — qemu.org/docs/master/. **Ellenőrizve**: a `master` ág dokumentációja, nem egy konkrét kiadott verzióé; a ténylegesen telepített QEMU ettől eltérhet.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: (1) a telepített Ubuntu 24.04 LTS csomagolt GCC/binutils/GDB/QEMU pontos verziószáma a laborban ellenőrizendő; (2) a `qemu-system-arm -machine mps2-an385` memóriatérkép mezőnevei a QEMU forráskódjából (`hw/arm/mps2.c`) ellenőrizendők, ezt a fenti dokumentációs linkek önmagukban nem igazolják; (3) a `lab-a000` példa kimenete (`60 8`), a `lab-sum.o` elhagyásakor kapott `undefined reference` hiba és a fejlécmódosítás utáni újrafordítás x86_64 hoston, GCC 13.3.0-val (Ubuntu 24.04) **mért**; más fordítón és AArch64-en nem mértük, és a `sizeof(struct pair)` értéke az adott ABI-tól függ, ezért a hallgató laborjában validálandó.

1.1.9.1 Függelék: ellenőrzési jegyzőkönyv

Ez a szakasz az olvasói útmutató függeléke; a tanuláshoz nem szükséges elolvasni, csak a forrás- és mérésállapot jelöléseit rögzíti. Az alfejezetek „Források és ellenőrzési hiányok” szakaszai ezekre a jelölésekre hivatkoznak, és csak a saját példájuk feltételeit sorolják fel. A **forrásállapot** a fenti listára épül (ellenőrzés: 2026-10):

Forrás	Állapot	Közös korlát
Arm-dokumentáció (`developer.arm.com`: DDI0487, DDI0403E, architektúra-oldalak)	a link él	JavaScript-renderelés miatt a fejezetszám és a kiadásbetű nem ellenőrzött; a konkrét fejezetet a laborban kell visszakeresni

Forrás	Állapot	Közös korlát
AAPCS64	ellenőrizve: 2025Q4 (2026-01-23), Release	az SME-részek Beta minőségűek
AAPCS32	a link él	az aktuális kiadás nincs ellenőrizve
GNU binutils, GCC, GDB kézikönyv	ellenőrizve: binutils 2.47, GCC 16.2; GDB „current”	a GDB „current” fejlesztői pillanatkép; a telepített eszközök verziója eltérhet
QEMU dokumentáció (`docs/master`)	ellenőrizve	a `master` ág dokumentációja, nem egy kiadott verzióé; a telepített (Ubuntu 24.04: QEMU 8.2.2) változat eltérhet
„dokumentációs kiindulópont”	megnevezett forrás	tartalmi állítás nem épül rá ellenőrzöttként

A fenti források **ajánlott továbbolvasás** és kiindulópont, nem a konkrét állítások bizonyítéka. Kritikus állításnál az adott alfejezet „Források és ellenőrzési hiányok” szakasza adja meg a pontos dokumentumot, fejezetet/utasítást/regisztert és az ellenőrzési státuszt (ellenőrzött / nem ellenőrzött); a felhasznált bizonyítékot (például forráskód, mérés, reprodukció) ott külön választjuk az ajánlott olvasmányoktól, és a „nem ellenőrzött” jelölést nem töröljük, hanem ellenőrzés után frissítjük.

A **mérésállapot** jelölései eszközverzióhoz és bizonyítékhoz kötöttek:

Jelölés a szövegben	Eszközök és verzió	Bizonyíték
„helyettesítő eszközlánccal mérve” (I. rész)	`zig cc` 0.17.0 (Clang + `ld.lld`, `-target aarch64-linux-musl`), `lab-run.py` 1.1 (unicorn 2.1.4)	kísérőcsomag v1.0, `tests/run.sh substitute`, `logs/substitute-1.0-20261028.log`
„helyettesítő eszközlánccal mérve” (II. rész)	`zig cc` 0.17.0 (`-target thumb-freestanding-eabi -mcpu=cortex_m3`), `cm-run.py` 1.2 (unicorn 2.1.4, capstone 5.0.7, pyelftools 0.33)	ugyanott; a II. rész laboronkénti állapota az A-profile és M-profile különbségei közös táblázatában
host-mérés („host `gcc`”, „GNU ld/readelf 2.42”)	x86_64, Ubuntu 24.04, gcc 13.3, GNU binutils 2.42	a helyben közölt kimenet; a csomag hostpróbái ugyanabban a naplóban
fő út (GNU eszközlánc + QEMU)	`aarch64-linux-gnu-*`, `arm-none-eabi-*`, `qemu-aarch64`, `qemu-system-arm -machine mps2-an385`	csak ott állítjuk, ahol a szöveg kifejezetten így jelöli; a kísérőcsomag `tests/run.sh gnu-qemu` naplója; a v1.0 csomagban ilyen napló még nincs
„levezetett”, „illusztratív”, „hipotetikus”	—	a levezetés maga; nem mérés

A tanulónak ajánlott fő út és a bemutatott bizonyíték viszonya. A laborok parancsai a GNU eszközláncot és a QEMU-t használják (fő út), a közölt kimenetek viszont, ahol a

szöveg „helyettesítő eszközlánccal mérve” jelölést ad, Clang/`ld.lld` és saját emulátor futásából származnak. A két út assembler-szintaxisa, ELF-részletei, perifériamodellje és hibaviselkedése eltérhet, és a helyettesítő PASS nem igazolja a GNU/QEMU útvonalat. Ezért laboronként így olvasd az állapotot:

Út	Mit igazol	Állapot a v1.0 kiadásban
Helyettesítő út (Clang + saját emulátor)	a közölt kimenetek és a próbák (`tests/run.sh substitute`)	lefutott, napló a kísérőcsomagban; kiegészítő bizonyíték, nem helyettesíti a fő utat
Fő út (GNU + QEMU)	a parancsok tiszta Ubuntu 24.04 környezetben	**részben mért**: a fejezetekben `mérve (GNU + QEMU 8.2.2)` jelöléssel szereplő értékek a fő út mérései; a teljes `tests/run.sh gnu-qemu` napló még hiányzik, ezért a kiadás **béta**
Csak statikusan ellenőrzött / kihagyott (SKIP)	szövegolvasás, kódolvasás	a szövegben ennek megfelelően címkézve

A teljes fő úti napló elkészültéig a kiadvány **béta** állapotú: ahol a fejezet `mérve (GNU + QEMU 8.2.2)` jelölést ad, ott a kimenet a fő út mérése; ahol nem, ott a kimenet továbbra is elvárt vagy illusztratív. Ha a saját géped kimenete eltér, előbb az eszköz- és emulátorverziót vesd össze ezzel a táblával, és ne a saját hibádat feltételezd.

2 Bevezetés: hardver, ISA és a gép nyelve

2.1 Miért assembly: amit a C elrejt

2.1.1 Mérnöki kérdés: mit hajt végre valójában a processzor?

Egy C-függvény helyes eredményt ad, mégis túl lassú. Egy külső könyvtár meghívása után látszólag sértetlen változók változnak meg. Egy mikrokontroller perifériája nem reagál a programra. Egy hibás memóriahozzáférés pedig nem ott okoz összeomlást, ahol a forráskód alapján várnád.

Ezekben a helyzetekben ugyanaz a kérdés merül fel: **milyen gépi műveletek valósítják meg a C-programot, és milyen szerződésekre támaszkodnak?**

Az assembly ismerete nem azt jelenti, hogy ezentúl minden függvényt kézzel kell megírnod. Gyakrabban fogsz fordító által előállított kódot olvasni, mint saját assemblyt készíteni. A cél annak felismerése, hogy egy teljesítményprobléma, hibás függvényhívás vagy biztonsági hiba melyik rétegben keletkezik.

Ehhez az alfejezethez nincs szükség korábbi assemblyismeretre. A laborban egy rövid C-programból indulunk, és csak annyi gépi részletet vezetünk be, amennyi a megfigyelés értelmezéséhez szükséges.

2.1.2 Az assembly helye az absztrakciós rétegek között

Az **assembly nyelv** az adott utasításkészlet gépi utasításainak szimbolikus leírására szolgáló nyelv. Az **utasításkészlet-architektúra** (*instruction set architecture*, ISA) meghatározza többek között az elérhető utasításokat, a programból látható regisztereket és az utasítások előírt hatását. A regiszter itt a processzor utasításaiban közvetlenül megnevezhető adattároló hely.

Az assemblyforrás nem kizárólag utasításokat tartalmazhat: vannak benne szimbolikus nevek és a fordítóeszköznek szóló direktívák is. Ezért nem minden assemblyforrás-sor jelent egy végrehajtandó gépi utasítást.

A C-program útja leegyszerűsítve:

Réteg	Feladata	Mit vizsgálunk benne?
C-forrás	A számítás leírása a C szabályaival	Típusok, algoritmus, megengedett műveletek
C-fordító	Gépi megvalósítás kiválasztása, optimalizálás	Milyen műveleteket hagyott el vagy alakított át?
Assemblyforrás	A célgépi műveletek szimbolikus alakja	Milyen utasítások és regiszterek szerepelnek?
Assembler	Tárgyfájl előállítása	Utasításkódok, szimbólumok, később feloldandó hivatkozások

Réteg	Feladata	Mit vizsgálunk benne?
Linker	Tárgyfájlok és könyvtárak összekapcsolása	Hol találhatók a hívott függvények és adatok?
Betöltés és végrehajtás	A program futtatása a célkörnyezetben	Milyen állapotváltozások történnek?

A gyakorlatban a köztes assemblyszöveget nem feltétlenül mentjük külön fájlba. A rétegek ettől még hasznos gondolkodási modellt alkotnak. A részletes munkafolyamatot a C fordítási láncolat és hova kerül az assembly bontja ki.

Az ISA azt írja elő, **mit** jelentenek az utasítások. A **mikroarchitektúra** (*microarchitecture*) az adott processzor belső megvalósítása: hogyan szervezi meg például az utasítások és memóriaelérések végrehajtását. Azonos ISA-t támogató processzorokon ugyanaz a program eltérő sebességgel futhat.

2.1.3 Négy helyzet, ahol a C-forrás önmagában kevés

Teljesítmény. Egy rövid C-kifejezésből több utasítás keletkezhet, egy hosszabb ciklust pedig a fordító jelentősen átalakíthat. A gépi kód megmutatja, történt-e vektorizálás, maradt-e ismételt memóriaolvasás, vagy keletkezett-e költségesnek bizonyuló elágazás. Ebből azonban még nem következik futási idő: a kevesebb utasítás nem automatikusan gyorsabb. A kódelemzés hipotézist ad, a mérés ellenőrzi.

Interoperabilitás. Különböző nyelveken vagy külön fordított modulokban megírt függvényeknek meg kell egyezniük az argumentumok és eredmények átadásában. Ezt részben a **bináris alkalmazásinterfész** (*application binary interface*, ABI) szabályozza. Egy hibás assemblyfüggvény helyes számítást végezhet, mégis tönkreteheti a hívó állapotát, ha felülír egy megőrzendő regisztert. AArch64-en ezt az AAPCS64 regiszterszerződés alapján fogjuk ellenőrizni.

Hardverelérés. A C nyelv önmagában nem határoz meg mikrokontroller-perifériákat, megszakításvezérlőt vagy processzorspecifikus vezérlőutasításokat. Sok hardverkezelés megírható C-ben, de a helyes működéshez ismerni kell az alatta lévő architektúrát és az eszköz dokumentációját. A `volatile` sem általános megoldás minden sorrendezési vagy párhuzamossági problémára.

Biztonsági hibák elemzése. Egy érvénytelen memóriaírás forráskódbeli helye és a későbbi összeomlás helye eltérhet. A regiszterek, a memória és a gépi utasítások vizsgálata segít feltárni, mi sérült meg. A könyv ezt védelmi célból használja: hibakeresésre, ellenőrzésre és a védelmi mechanizmusok megértésére, nem támadási receptek készítésére.

Ezekhez nem mindig szükséges kézzel assemblyt írni. Az ISA-szintű olvasási és következtetési képesség viszont sokszor nélkülözhetetlen.

2.1.4 A C szemantikája nem azonos a gép számtanával

A C **nem definiált viselkedése** (*undefined behavior*, UB) olyan eset, amelyre a nyelvi szabvány nem ír elő követelményt. Ez nem azt jelenti, hogy a program kötelezően hibajelzést ad, vagy hogy a processzor „véletlenszerűen” működik.

Fontos példa az egész aritmetika:

- Az előjel nélküli aritmetika a megfelelő típusban modulo 2^N működik, ahol N a típus szélessége (`uint32_t` esetén 2^{32}). A labor AArch64-profiljában a `uint32_t` értékeinek tartománya 0–4294967295.
- Az előjeles egész összeadás túlcsordulása C-ben nem definiált viselkedés.
- A fordító optimalizáláskor támaszkodhat arra, hogy a helyesen működő program nem hajt végre ilyen túlcsordulást.

Nézzük az `x + 1 > x` kifejezést, ahol `x` típusa `int`. Minden olyan bemenetre igaz, amelynél az összeadás definiált. Az `INT_MAX` bemenetnél viszont maga az összeadás nem definiált (UB), így erre a bemenetre a szabvány semmilyen eredményt nem ír elő. A fordító ezért a teljes kifejezést állandó igaz értékre egyszerűsítheti.

Nem helyes így érvelni: „A processzor összeadója körbefordul, tehát a C-kifejezés eredménye is ebből következik.” Lehetséges, hogy a fordító egyáltalán nem állít elő összeadó utasítást.

Az assembly az adott bináris megvalósítást teszi láthatóvá. **Nem ad visszamenőleg definiált C-jelentést egy UB-t tartalmazó programnak.** Más optimalizálás, fordító vagy környezet más kódot eredményezhet.

2.1.5 Kidolgozott példa: egy összeadás megmarad, egy másik eltűnhet

Munkakönyvtár: `~/arm-labor/a64/a001/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

> **Futtatási feltétel.** A labor a 3.1-ben telepített `aarch64-linux-gnu-*` eszközláncot és `qemu-aarch64`-et használja. Ha ezek még nincsenek a gépen, előbb végezd el a 3.1 telepítését (gyors parancs: `sudo apt install gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu gdb-multiarch qemu-user`); addig a labort csak olvasd el.

```
mkdir -p ~/arm-labor/a64/a001
cd ~/arm-labor/a64/a001
```

A fejezet végigvitt projektje. Az itt elkészülő `lab-hidden` program a Bevezetés: hardver, ISA és a gép nyelve fejezet közös, végigvitt projektje. A Számítógép-architektúra fogalmai, Az ARM család áttekintése és a C fordítási láncolat alfejezet végén egy-egy rövid „Végigvitt projekt” lépés erre a programra alkalmazza az ott tanultakat: utasításkódolás, célplatform-azonosítás, a fordítási lánc lépései. Az alfejezetek saját, kisebb buildjei *célzott kiegészítések*; első olvasáskor kihagyhatók, a projekt lépései nem. A `lab-hidden` fájljai ebben a könyvtárban maradnak, a későbbi lépések átmásolják őket.

Célprofil: A64 ISA, AArch64 Linux, Armv8-A alapprofil, little-endian; futtatás `qemu-aarch64` user-mode emulációban.

A gazdagép az Ubuntu 24.04 LTS x86_64 alapkörnyezet. A parancsokhoz a `gcc-aarch64-linux-gnu`, `binutils-aarch64-linux-gnu` és `qemu-user` csomagok szükségesek. Telepítésüket és ellenőrzésüket a Labor-alapkörnyezet: eszközlánc telepítése és verifikáció részletezi.

Mentsd a következő teljes programot `lab-hidden.c` néven:

```
#include <inttypes.h>
#include <limits.h>
#include <stdint.h>
#include <stdio.h>

uint32_t lab_next_u(uint32_t x)
{
    return x + UINT32_C(1);
```

```c
}

int lab_rises_s(int x)
{
    return x + 1 > x;
}

int main(void)
{
    uint32_t u = lab_next_u(UINT32_MAX);
    int s = lab_rises_s(INT_MAX - 1);

    printf("unsigned_next=%" PRIu32 "\n", u);
    printf("signed_rises=%d\n", s);

    return (u == UINT32_C(0) && s == 1) ? 0 : 1;
}
```

A teszt szándékosan **nem** hívja meg a `lab_rises_s` függvényt `INT_MAX` argumentummal. A ténylegesen vizsgált előjeles összeadás eredménye `INT_MAX`, tehát ábrázolható.

2.1.5.1 Fordítás, összeállítás, linkelés és futtatás

Az alábbi parancsok a gazdagépen futnak; az elkészült program célja végig **AArch64, qemu-user**.

```
# C -> A64 assemblyforrás.
aarch64-linux-gnu-gcc \
    -std=c11 -O2 -fno-inline -Wall -Wextra \
    -march=armv8-a -mlittle-endian \
    -S lab-hidden.c -o lab-hidden.s

# A64 assemblyforrás -> AArch64 tárgyfájl.
aarch64-linux-gnu-as \
    -march=armv8-a -EL \
    lab-hidden.s -o lab-hidden.o

# Tárgyfájl -> statikusan linkelt AArch64 Linux-program.
aarch64-linux-gnu-gcc \
    -march=armv8-a -mlittle-endian -static \
    lab-hidden.o -o lab-hidden

# Futtatás: AArch64 qemu-user cél.
qemu-aarch64 ./lab-hidden
printf 'exit=%s\n' "$?"
```

A `-S` assemblyforrásnál állítja meg a C-fordítást. A `-fno-inline` megakadályozza a függvények szokásos beágyazását (*inlining*), így könnyebb külön megvizsgálni őket; az összes többi optimalizálást nem tiltja le. Az interprocedurális optimalizálás (IPA-CP, konstans-propagáció, `.constprop` klónok) ettől még átalakíthatja a hívást; a robusztus megoldás a `__attribute__((noipa))`, amely a klónozást is tiltja. A statikus linkelés ebben a laborban elkerüli a külön célrendszeri dinamikus betöltő beállítását.

Minden buildlépés várt kilépési állapota 0. Ha valamelyik hibával leáll, ne futtasd tovább a lépéssort egy esetleg korábbról megmaradt binárissal.

Levezetett, nem ténylegesen futtatott eredmény:

Ellenőrzés	Várt érték
`unsigned_next` kimeneti mező	0
`signed_rises` kimeneti mező	1
`exit` kimeneti mező	0

Az előjel nélküli eredmény levezetése:

$$(4294967295 + 1) \bmod 2^{32} = 0.$$

Itt a feltétel a labor 32 bites előjel nélküli számítása. A visszaadott 0 tehát nem hiba, hanem a C által előírt eredmény.

2.1.5.2 Mit keress a gépi kódban?

A **visszafejtés** (*disassembly*) a gépi utasításkódokat olvasható assemblyalakban jeleníti meg. A következő parancsok a gazdagépen vizsgálják a **qemu-user számára készített AArch64 binárist**; nem futtatják azt:

```
aarch64-linux-gnu-objdump \
    -d --disassemble=lab_next_u lab-hidden

aarch64-linux-gnu-objdump \
    -d --disassemble=lab_rises_s lab-hidden
```

A `lab_next_u` esetében várható, leegyszerűsített utasításminta:

```
// A64 ISA (AArch64, I. rész).
// Illusztratív függvénytörzs, nem különálló teljes program.
lab_next_u:
    add w0, w0, #1
    ret
```

Az A64 `W0` egy 32 bites regiszternézet. Ennél az AAPCS64 szerinti függvényinterfésznél ezen érkezik az argumentum, és ezen távozik az eredmény. Az A64 `ADD` itt hozzáad 1-et, és a 32 bites eredményt tartja meg. Az utasítás ezen alakjának közvetlen konstansa (*immediate*) 12 bites előjel nélküli érték lehet, opcionálisan 12 bittel balra tolva; az 1 eltolás nélkül kódolható.

Az A64 `RET` visszatérést végez, alapértelmezésben az A64 `X30` 64 bites regiszterben tárolt címre. A hívás részleteit később tanuljuk meg.

A `lab_rises_s` függvénynél optimalizált fordításban várhatóan az állandó 1 előállítását és visszatérést látod, nem az eredeti összeadást és összehasonlítást. Ez megengedett optimalizálás, nem kötelező utasítássorozat. A konkrét alakot a laborban kell ellenőrizni.

2.1.6 Tipikus hibák és ellenpróbák

A vizsgálat menete legyen mindig: **tünet → hipotézis → adatgyűjtés → egy változtatás → új ellenőrzés**.

Tünet	Lehetséges ok	Ellenőrzés vagy javítás
Közvetlen indításkor végrehajtási formátumhiba jelenik meg	Az x86_64 gazdagép AArch64 programot kapott	Használd a `qemu-aarch64 ./lab-hidden` parancsot
A program nem a várt eredményt adja a forrás módosítása után	Régi assemblyből vagy tárgyfájlból készült bináris fut	Ismételd meg mindhárom buildlépést; vizsgáld meg a binárist `objdump` segítségével
Nem találod az előjeles összeadást	A fordító eltüntette	Vizsgáld meg a `lab_rises_s` teljes függvénytörzsét
Hiányzó célrendszeri betöltőről kapsz hibát	Nem a bemutatott statikus program készült el	Ellenőrizd a linkelési parancsot és a `-static` kapcsolót

Ha a `binfmt_misc`-en keresztül a `qemu-user` regisztrálva van, a közvetlen indítás is működik; ezt `ls /proc/sys/fs/binfmt_misc/` mutatja.

A hiányzó utasítás is adat. Ne módosítsd rögtön a programot azért, hogy a fordító a várt mintát adja: előbb ellenőrizd, jogos volt-e az elvárásod.

Ez a labor funkcionális ellenőrzés, nem sebességmérés. A QEMU user-mode futási idejéből nem következtetünk fizikai ARM-processzor utasításkésleltetésére vagy ciklusszámára.

2.1.7 Hogyan vezet az út AArch64-től Cortex-M-ig?

A könyv két külön végrehajtási környezetet használ.

Szempont	I. rész: AArch64	II. rész: Cortex-M
ISA és profil	A64, Armv8-A alapprofil	Thumb-2, Armv7-M, Cortex-M3
Környezet	Linux-alkalmazás	Operációs rendszer nélküli firmware
Elsődleges emuláció	`qemu-aarch64`	`qemu-system-arm -machine mps2-an385`
Központi kérdés	Hogyan működik együtt a C, az ABI és a gépi kód?	Hogyan indul el és kezel hardvert a firmware?

Az első részben az operációs rendszer biztosítja a futási környezet jelentős részét. A másodikban az indulást, a memória inicializálását és a megszakítások kezelését is nekünk kell megszerveznünk.

Átvihető tudás a bitszintű gondolkodás, a gépi kód olvasása és a szerződések ellenőrzése. Nem vihetők át változtatás nélkül az utasítások, regiszternevek és indítási feltételezések.

A könyv nem teljes operációsrendszer-kernel megírását, nem valós idejű operációs rendszer (*real-time operating system*, RTOS) ütemezőjének megvalósítását és nem gyártóspecifikus fejlesztőkörnyezet használatát tanítja. A hangsúly a reprodukálható, architektúraközeli munkafolyamaton van.

2.1.8 A tananyag tanulási szerződése

Minden alfejezet konkrét mérnöki feladatból indul, majd meghatározza a szükséges fogalmakat és feltételeket. A magyarázatot fordítható vagy összeállítható példa, ellenőrzési feltétel, hibakeresési ellenpróba és fokozatos feladatsor követi. A megoldások indoklást is adnak; a rövid összefoglaló a később továbbhasznált fogalmakat emeli ki.

Háromféle állítást különítünk el:

- **Specifikáció:** mit ír elő a C, az ISA vagy az ABI?
- **Implementáció:** mit állított elő az adott fordító?
- **Mérés:** mit figyeltünk meg ténylegesen a megjelölt környezetben?

A most közölt kimenet a fő úton mért érték: Ubuntu 24.04, GCC 13.3.0, binutils 2.42, QEMU 8.2.2. A saját laborodban ugyanezt kell kapnod; ha eltér, előbb az eszközverziókat hasonlítsd össze.

2.1.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért adhat a `lab_next_u(UINT32_MAX)` definiáltan 0-t, miközben a `lab_rises_s(INT_MAX)` eredményét nem határozhatjuk meg ugyanezzel az érveléssel? Miért nem cáfolja az optimalizálás helyességét az eltűnt összeadás?

2. (implementációs, alapszint — cél: rutinépítés) Bővítsd a `lab-hidden.c` fájlt egy `uint32_t lab_double_u(uint32_t x)` függvénnyel, amely összeadással kétszerezi az argumentumot. Ellenőrizd a 0, 7 és `UINT32_MAX` bemeneteket. A `main` akkor térjen vissza 0-val, ha minden ellenőrzés sikeres.

3. (hibakeresési, középszint — cél: rutinépítés) Egy módosított változatban a `lab_next_u` már 2-t ad hozzá, de a futtatás továbbra is 0-t ír ki az előjel nélküli eredményre. A készítő csak a linkelési parancsot ismételte meg. Adj hipotézist, válassz adatgyűjtő eszközt, és írd le a javítást. Mi lesz az új helyes eredmény?

2.1.10 Megoldások

1. megoldás — Segítség. Gondold végig külön az előjel nélküli és az előjeles túlcsordulás C-beli definiáltságát — az egyik moduláris aritmetika, a másik nem azonosítható egyetlen konkrét eredménnyel.

1. megoldás — Teljes megoldás. A 32 bites előjel nélküli számítás modulo 2^{32} történik. Az előjeles `INT_MAX + 1` viszont C-ben nem definiált. A fordító az összes definiált bemenetre helyesen helyettesítheti az `x + 1 > x` kifejezést 1-gyel. Az UB-t okozó bemenet nem követel meg ettől eltérő eredményt.

2. megoldás — Segítség. Gondold végig, mi történik, ha az összeg nem fér el 32 biten: mivel egyenlő $2 \cdot (2^{32} - 1) \bmod 2^{32}$? A választ ne tizedestört közelítéssel, hanem a modulo-arimetika szabályával add meg.

2. megoldás — Teljes megoldás. A meglévő fájlba illeszthető függvény:

```
uint32_t lab_double_u(uint32_t x)
{
    return x + x;
}
```

A várt eredmények rendre 0, 14 és 4294967294. Az utolsó érték levezetése:

$$2(2^{32} - 1) \bmod 2^{32} = 2^{32} - 2.$$

A labor AArch64-profiljában az összeadás 32 bites előjel nélküli aritmetika. Bővítsd a `main` ellenőrzését ezekkel az esetekkel, majd ismételd meg a teljes C → assembly → tárgyfájl → program lépéssort. A várt kilépési állapot 0.

3. megoldás — Segítség. A tünet ("a forrás megváltozott, az eredmény nem") klasszikus jele annak, hogy a build valamelyik köztes lépése elmaradt. Nézd meg `objdump`-pal, hogy a ténylegesen linkelt `.o` tartalmazza-e az új forrás utasításait.

3. megoldás — Teljes megoldás. Valószínűleg a régi `lab-hidden.o` került ismét a programba. Az `objdump` segítségével vizsgáld meg a `lab_next_u` függvényt a végleges binárisban: az eredeti kódminta esetén még 1 hozzáadása látszik. Generáld újra az assemblyforrást, állítsd össze, majd linkeld. Az új eredmény $(4294967295 + 2) \bmod 2^{32} = 1$. A `main` elvárását is módosítani kell 1-re; különben helyesen hibát jelez a már elavult teszt.

2.1.11 Rövid összefoglaló

Az assemblytudás elsősorban betekintést ad abba, hogyan valósul meg a C-program az adott gépen. Segít teljesítményproblémák elemzésében, bináris interfészek ellenőrzésében, hardverkezelésben és biztonsági hibák feltárásában. A fordító nem köteles megőrizni a C-forrás műveleteinek látható szerkezetét: a definiált jelentést kell megtartania. A konkrét bináris megfigyelése nem teszi definiálttá a hibás C-programot, és az utasítások száma önmagában nem teljesítménymérés. A könyv ezért külön kezeli az előírást, a megvalósítást és a tényleges megfigyelést. A laborban mindig a teljes buildláncot és az ellenőrzési feltételeket követjük.

- **ISA:** a programból látható gépi műveletek és állapot szabályrendszere.
- **ABI:** a bináris együttműködés szabályrendszere.
- **UB:** olyan C-viselkedés, amelyre a szabvány nem ír elő követelményt.

2.1.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- WG14 N1570, C bizottsági tervezet: a 6.2.5 és 6.5 szakasz releváns az előjel nélküli aritmetikához és a nem ábrázolható aritmetikai eredményekhez. *Ellenőrizve*: az N1570 a C11 ratifikálása (ISO/IEC 9899:2011) előtti utolsó nyilvános WG14-tervezet — a gyakorlatban mindenhol „a C11 szabványként" hivatkozzák, mert a fizetős ISO-szöveg nem elérhető szabadon, de technikailag tervezet, nem az eladott ISO-dokumentum (a különbség túlnyomórészt szerkesztési jellegű). Nem a C17 végleges szabványszövege.
- GCC: Optimize Options: optimalizálási és beágyazási kapcsolók. *Ellenőrizve*: a GCC-dokumentáció legújabb listázott kiadása 16.2; a helyben telepített verzió ettől eltérhet.
- GNU assembler kézikönyv: az assembler szerepe és célarchitektúra-függő lehetőségei. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.
- QEMU User Mode Emulation: a felhasználói módú emuláció működési kerete. *Ellenőrizve*: a `master` ághoz tartozó dokumentáció, nem egy konkrét kiadott verzióhoz kötött; a ténylegesen telepített QEMU-verzió dokumentációja ettől eltérhet.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a programot a fő úton mértük (Ubuntu 24.04, GCC 13.3.0, binutils 2.42, QEMU 8.2.2); a konkrét GCC által előállított utasítássorozat és a telepített csomagverziók a saját laborban ellenőrzendők.

2.2 Számrendszerek és bitszintű reprezentáció

2.2.1 Belépőteszt: átugorhatod ezt az alfejezetet?

Ha a számrendszereket C-fejlesztőként már biztosan használod, ez az alfejezet nagyrészt ismétlés. A nyolc kérdést papíron, segédeszköz nélkül, legfeljebb tíz perc alatt válaszold meg, majd a megoldókulcs és az átugrási út alapján döntsd el, mit olvass el.

1. Mennyi a `0x3C` decimálisan, és mi a 8 bites bináris alakja?
2. Hány bitet ír le egy hexadecimális számjegy, és hány hexadecimális jegy kell egy 32 bites szóhoz?
3. Mi a 8 bites `0xF6` bitminta értéke előjel nélküli, illetve kettes komplemens olvasatban?
4. Hogyan írjuk fel a -1-et 32 bites kettes komplemensben, hexadecimálisan?
5. Mi lesz a 8 bites `0x80` 16 bitre kiterjesztve nullakiterjesztéssel, illetve előjelkiterjesztéssel?
6. Mennyi 8 bites előjel nélküli aritmetikában `200 + 100`?
7. Mi lesz 8 biten a `100 + 100` eredményének bitmintája, és mi ennek az előjeles olvasata?
8. C-ben `int x = INT_MAX;` és `unsigned u = UINT_MAX;` mellett mi a `x + 1`, illetve az `u + 1` státusza?

Megoldókulcs és átugrási út:

#	Helyes válasz	Ha hibáztál, olvasd el
1	60; `0b0011_1100`	Átváltás bináris, hexadecimális és decimális alak között
2	4 bit; 8 jegy	Méretek és helyi értékek; Hexadecimális jelölés a gyakorlatban
3	246, illetve -10	A kettes komplemens előjeles olvasata
4	`0xFFFFFFFF`	A kettes komplemens előjeles olvasata
5	`0x0080`, illetve `0xFF80`	Nullakiterjesztés és előjelkiterjesztés
6	44 (300 mod 256: körbefordulás)	Túlcsordulás és körbefordulás: levezetés 8 bitre
7	`0xC8`, előjelesen -56 (előjeles túlcsordulás)	Túlcsordulás és körbefordulás: levezetés 8 bitre
8	`x + 1` definiálatlan viselkedés (előjeles túlcsordulás); `u + 1` definiáltan 0	A C szemantikája külön szabályrendszer

- **Mind a nyolc helyes:** ugord át az alfejezetet, és folytasd a Számítógép-architektúra fogalmai alfejezettel. A „Kidolgozott labor: ugyanazok a bitek, két olvasat” szakaszt érdemes később, az eszközlánc telepítése után lefuttatni.

- **Egy-két hiba:** csak a táblázat harmadik oszlopában megnevezett szakaszokat olvasd el, majd lépj tovább.
- **Három vagy több hiba, vagy a 8. kérdés hibás:** olvasd végig az alfejezetet. A 8. kérdés különösen fontos: a C és a gép számtanának különválasztására a könyv egésze épít.

2.2.2 Miért kell a bitmintát és az értékét külön kezelni?

Egy bináris fájlban `0xFF` bájtot találsz. Ez 255, -1, nyolc bekapcsolt jelzőbit, vagy egy gépi utasítás része? Önmagában egyik értelmezés sem kötelező. A jelentést az adatformátum, a művelet és a választott bitszélesség határozza meg.

Ez gyakorlati probléma: egy hibás előjelkiterjesztésből rossz tömbindex, egy túlcsorduló méretszámításból túl kicsi memóriaterület lehet. Ebben az alfejezetben egész számok **bitmintája** és **számértéke** között alakítunk át. Levezetjük a kettes komplemens ábrázolást és a véges szélességű összeadás korlátait.

Profil: I. rész, AArch64, A64 ISA, Armv8-A; little-endian bájtsorrend. A matematikai példák gyakran 8 bitesek, mert így minden bit áttekinthető. Ez nem jelenti azt, hogy az A64 általános célú regiszterei 8 bitesek.

2.2.3 Méretek és helyi értékek

A **bit** egy bináris számjegy: értéke 0 vagy 1. Az alábbi méretneveket különböztetjük meg:

Elnevezés	Méret	Megjegyzés
bit	1 bit	Két lehetséges érték
félbájt (*nibble*)	4 bit	Egy hexadecimális számjegy
bájt (*byte*)	8 bit	A könyv Arm-célprofiljain
félszó (*halfword*)	16 bit	`LDRH`/`STRH`, `H` utótag
szó (*word*)	32 bit	Az itt használt Arm-elnevezés
dupla szó (*doubleword*)	64 bit	Két 32 bites szó

A „szó” nem minden architektúrán jelent ugyanannyit. A64 ISA-környezetben az általános célú **Wn** regiszternézet 32 bites, az **Xn** nézet 64 bites. Ettől az Arm terminológiájában a *word* még 32 bit marad. A C `int` vagy `long` típusának méretét pedig a megvalósítás és az ABI szabályai határozzák meg, nem a szó hétköznapi jelentése.

Egy n bites, előjel nélküli (*unsigned*) szám értéke:

$$U = \sum_{i=0}^{n-1} b_i 2^i.$$

Itt n a bitszélesség bitben, i a jobbról számozott bithely, b_i pedig 0 vagy 1; U mértékegység nélküli egész szám. A legkisebb helyi értékű bit (*least significant bit*, LSB) indexe 0, a legnagyobbé (*most significant bit*, MSB) $n-1$.

Például, 8 bites előjel nélküli olvasatban:

$$\texttt{0b00101101} = 32 + 8 + 4 + 1 = 45.$$

A számjegyek számától eltérő fogalom az érték: a `0b101` és a `0b00000101` előjel nélküli értéke egyaránt 5, de a második jelölés egy 8 bites mezőt szemléltet.

2.2.4 Átváltás bináris, hexadecimális és decimális alak között

A hexadecimális (*hexadecimal*) számrendszer alapja 16. Számjegyei 0–9, majd A–F; utóbbiak rendre a 10–15 értéket jelölik. Mivel $16 = 2^4$, minden hexadecimális számjegy pontosan négy bitnek felel meg.

Hexadecimális számjegy	Négybites alak	Előjel nélküli decimális érték
`0x2`	`0b0010`	2
`0x7`	`0b0111`	7
`0xA`	`0b1010`	10
`0xD`	`0b1101`	13
`0xF`	`0b1111`	15

Binárisból hexadecimálisra váltáskor jobbról négyes csoportokat képzünk:

- `0b0010_1101` → `0x2D`;
- `0b1101_0011` → `0xD3`.

Az aláhúzás itt csak olvasást segítő csoportosítás, nem a labor C-kódjának szintaxisa.

Hexadecimálisból decimálisra ugyanúgy helyi értékeket összegzünk:

$$\texttt{0x2D} = 2 \cdot 16 + 13 = 45.$$

Decimálisból binárisra ismételt, maradékos osztás használható:

Osztandó	2-vel osztás hányadosa	Maradék
45	22	1
22	11	0
11	5	1
5	2	1
2	1	0
1	0	1

A maradékokat **alulról felfelé** olvasva `0b101101` adódik. Nyolc bitre nullákkal kiegészítve: `0b00101101`.

Hexadecimális átváltáshoz 16-tal osztunk: 45 hányadosa 2, maradéka 13, ezért az eredmény `0x2D`. Ezek az eljárások tetszőleges nemnegatív egészre működnek; véges mezőbe tárolás előtt külön ellenőrizni kell, hogy az érték belefér-e.

2.2.5 A kettes komplemens előjeles olvasata

Ugyanannak az n bites mintának a **kettes komplemens** (*two's complement*) előjeles értéke:

$$S = \begin{cases} U, & U < 2^{n-1}, \\ U - 2^n, & U \geq 2^{n-1}. \end{cases}$$

U az előjel nélküli, S az előjeles, mértékegység nélküli érték; $n \geq 1$ a bitszélesség. Ezzel egyenértékű, ha a legfelső bit helyi értékét negatívnak vesszük:

$$S = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i.$$

Nyolc biten ezért a legfelső bit súlya -128, nem $+128$.

8 bites minta	Előjel nélküli érték	Kettes komplemens előjeles érték
`0x00` = `0b00000000`	0	0
`0x7F` = `0b01111111`	127	127
`0x80` = `0b10000000`	128	−128
`0xD3` = `0b11010011`	211	−45
`0xFF` = `0b11111111`	255	−1

A `0b` bináris literál C17-ben nem szabványos, csak GNU-kiterjesztés (C23-tól szabványos); a példák a bináris alak szemléltetésére használják.

Az ábrázolható tartományok:

Bitszélesség	Előjel nélküli tartomány	Kettes komplemens előjeles tartomány
n bit	$0 \ldots 2^n - 1$	$-2^{n-1} \ldots 2^{n-1} - 1$
8 bit	0…255	−128…127
16 bit	0…65535	−32768…32767
32 bit	0…4294967295	−2147483648…2147483647

Negatív szám kódolásakor előbb bitszélességet választunk. A -45 érték 8 bites mintája:

$$2^8 - 45 = 211 = \texttt{0xD3}.$$

Ugyanezt adja a pozitív érték bitjeinek megfordítása és 1 hozzáadása:

1. $+45$ mintája: `0b00101101`;
2. bitenkénti megfordítás: `0b11010010`;
3. hozzáadunk 1-et: `0b11010011`.

Levezetés: az n bites megfordítás $2^n - 1 - U$ értéket ad. Ehhez 1-et hozzáadva $2^n - U$ keletkezik, amelynek kettes komplemens olvasata $-U$.

A `-0x2D` matematikailag -45; a `0xD3` önmagában egy pozitív hexadecimális számjelölés. Utóbbi csak **8 bites kettes komplemens mintaként értelmezve** jelenti ugyanazt.

A tartomány aszimmetrikus: -128 ábrázolható 8 biten, $+128$ előjelesen nem. Ezért a legkisebb negatív érték negálása ugyanazon a bitszélességen nem adhat helyes, ábrázolható pozitív eredményt.

2.2.6 Nullakiterjesztés és előjelkiterjesztés

Szélesebb mezőre váltáskor meg kell választani, mivel töltjük ki az új felső biteket.

- **Nullakiterjesztés** (*zero extension*): az új bitek nullák; az előjel nélküli értéket őrzi meg.
- **Előjelkiterjesztés** (*sign extension*): az új bitek a régi legfelső bit másolatai; a kettes komplemens előjeles értéket őrzi meg.

Kiinduló 8 bites minta	16 bites eredmény	Művelet	Megőrzött érték
`0x2D`	`0x002D`	Mindkettő	45
`0xD3`	`0x00D3`	Nullakiterjesztés	Előjel nélküli 211
`0xD3`	`0xFFD3`	Előjelkiterjesztés	Előjeles -45

Levezetés: negatív szám esetén az n bites előjel nélküli minta U, előjeles értéke $U - 2^n$. Ha $m > n$ bitre terjesztünk előjellel, az új felső egyesek értéke $2^m - 2^n$. Az új előjeles olvasat tehát:

$$(U + 2^m - 2^n) - 2^m = U - 2^n.$$

Az érték változatlan. A kiterjesztés módját tehát nem a hexadecimális számjegyek „kinézete”, hanem a kívánt értelmezés dönti el.

2.2.7 Túlcsordulás és körbefordulás: levezetés 8 bitre

A **túlcsordulás** (*overflow*) azt jelenti, hogy a pontos matematikai eredmény kívül esik a választott értéktartományon. A **körbefordulás** (*wraparound*) a véges szélességű, modulo szerinti eredményképzés.

Ha egy összeadásból csak az alsó n bitet tartjuk meg, akkor:

$$R = (A + B) \bmod 2^n.$$

Itt A és B az operandusminták előjel nélküli értéke, R a tárolt minta előjel nélküli értéke; valamennyi mértékegység nélküli egész.

Nyolc biten az eredményt 256-tal vett maradékra korlátozzuk.

Operandus-minták	Pontos előjel nélküli összeg	Pontos előjeles összeg	Alsó 8 bit	Előjel nélküli túlcsordulás	Előjeles túlcsordulás
`0xFF` + `0x01`	256	$-1 + 1 = 0$	`0x00`	Igen	Nem
`0x7F` + `0x01`	128	$127 + 1 = 128$	`0x80`	Nem	Igen
`0x80` + `0xFF`	383	$-128 - 1 = -129$	`0x7F`	Igen	Igen

Az első sorban az előjel nélküli összeg kilenc bitet igényel: `0b1_00000000`. A kilencedik bit elhagyása után nulla marad.

A második sorban nincs kilencedik bit, mégis előjeles túlcsordulás történik: két nemnegatív operandusból negatívnak olvasott eredmény keletkezik.

Ebből következik az előjeles összeadás szabálya:

- azonos előjelű operandusoknál túlcsordulás van, ha az eredmény előjele eltér;
- különböző előjelű operandusok összege nem csordul túl előjelesen.

Utóbbi oka, hogy a negatív operandus csökkenti a pozitív értéket, és fordítva: az összeg nem kerülhet az operandusok által megengedett szélsőértékeken túlra.

Kivonásnál az előjeles túlcsordulás akkor lehetséges, ha az operandusok előjele különböző, és az eredmény előjele eltér a kisebbítendőétől. Például 8 biten $127 - (-1) = 128$ nem ábrázolható előjelesen.

2.2.7.1 A C szemantikája külön szabályrendszer

A gépi alsó-bites eredményből nem következik, hogy minden hasonló C-kifejezés szabályos:

- Az előjel nélküli C-aritmetika a művelet tényleges típusának tartományában modulo szerint működik.
- Az előjeles aritmetikai túlcsordulás C-ben nem definiált viselkedés (*undefined behavior*, UB).
- A keskeny egész típusok műveleteit az egészpromóció (*integer promotion*) gyakran `int` típusban végzi el.

Így két `uint8_t` összeadása az AArch64 laborprofilban nem automatikusan „8 bites összeadás”. A 255 + 1 összeg `int` típusban 256; a későbbi `uint8_t` konverzió eredménye lesz 0.

2.2.8 Hexadecimális jelölés a gyakorlatban

A hexadecimális alak tömör, de nem jelöl önmagában adattípust.

Felhasználás	Példa	Helyes olvasási kérdés
Bájtcímhez viszonyított eltolás	`0x20` bájt	32 bájttal arrébb?
8 bites bitmaszk (*bit mask*)	`0xF0`	A felső négy bitet jelöli?
A64 ISA utasításkódolás	`0xD503201F`	Ez az A64 `NOP` kódolása?

A `0x20` itt eltolás, nem cím: egy báziscímhez adva kapunk belőle címet. C-ben ilyen eltolást csak egy érvényes objektumon belül szabad alkalmazni. A maszkot nem előjeles számként, az utasításkódot pedig nem aritmetikai adatként értelmezzük.

A szám leírása és memóriabeli bájtsorrendje is különbözik. A 32 bites `0x12345678` minta little-endian tárolásakor növekvő bájteltolás mellett `0x78`, `0x56`, `0x34`, `0x12` következik. Ez nem változtatja meg a szám helyi értékeit. A részletes tárgyalás: Bájtsorrend: endianness és miért releváns a határon.

2.2.9 Kidolgozott labor: ugyanazok a bitek, két olvasat

Munkakönyvtár: `~/arm-labor/a64/a002/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

> **Futtatási feltétel.** A labor a 3.1-ben telepített `aarch64-linux-gnu-*` eszközláncot és `qemu-aarch64`-et használja. Ha ezek még nincsenek a gépen, előbb végezd el a 3.1 telepítését (gyors parancs: `sudo apt install gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu gdb-multiarch qemu-user`); addig a labort csak olvasd el.

```
mkdir -p ~/arm-labor/a64/a002
cd ~/arm-labor/a64/a002
```

A program tudatosan szélesebb típusokkal számítja ki a pontos összegeket, majd külön képezi a 8 bites eredményt. Így nem idéz elő előjeles C-túlcsordulást.

A `lab_signed8` nem támaszkodik tartományon kívüli, előjeles típusra történő konverzióra. A labor C17-et használ; a bináris `0b` alakot kiírja, a forrás konstansait azonban hexadecimálisan vagy decimálisan adja meg.

Fájl: `lab-bits.c`. C17, AArch64 célprofil; little-endian.

```
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

static int32_t lab_signed8(uint8_t x)
{
    int32_t u = x;
    return u < 128 ? u : u - 256;
}

static void lab_show8(uint8_t x)
{
    printf("0b");
    for (int bit = 7; bit >= 0; --bit)
        putchar((((unsigned)x >> bit) & 1u) ? '1' : '0');

    printf(" hex=0x%02x unsigned=%u signed=%" PRId32 "\n",
           (unsigned)x, (unsigned)x, lab_signed8(x));
}

static int lab_add8(uint8_t a, uint8_t b, uint8_t expected,
                    int expected_uo, int expected_so)
{
    unsigned wide_u = (unsigned)a + (unsigned)b;
    int32_t wide_s = lab_signed8(a) + lab_signed8(b);
    uint8_t result = (uint8_t)wide_u;

    int uo = wide_u > 255u;
```

```
    int so = wide_s < -128 || wide_s > 127;

    printf("add 0x%02x + 0x%02x -> 0x%02x UO=%d SO=%d\n",
           (unsigned)a, (unsigned)b, (unsigned)result, uo, so);

    return result == expected
        && uo == expected_uo
        && so == expected_so;
}

int main(void)
{
    lab_show8(0x2d);
    lab_show8(0xd3);
    lab_show8(0xff);

    uint8_t small = 0xd3;
    uint32_t zero = small;
    int32_t sign = lab_signed8(small);

    printf("zero=0x%08" PRIx32 " sign=0x%08" PRIx32 "\n",
           zero, (uint32_t)sign);

    int ok = zero == 211u && sign == -45;
    ok &= lab_add8(0xff, 0x01, 0x00, 1, 0);
    ok &= lab_add8(0x7f, 0x01, 0x80, 0, 1);
    ok &= lab_add8(0x80, 0xff, 0x7f, 1, 1);

    puts(ok ? "PASS" : "FAIL");
    return ok ? 0 : 1;
}
```

A negatív `sign` érték `uint32_t` típusra alakítása definiált, modulo 2^{32} szerinti konverzió; ezért alkalmas a kívánt 32 bites minta hexadecimális megjelenítésére.

2.2.9.1 Fordítás, összeállítás, linkelés, futtatás

Cél: qemu-user, qemu-aarch64; host: Ubuntu 24.04 x86_64. A parancsok a cross-eszközláncot és a statikus linkeléshez szükséges céloldali C-könyvtárat feltételezik. Telepítésük később, a Labor-alapkörnyezet: eszközlánc telepítése és verifikáció alfejezetben szerepel.

```
# C -> A64 ISA assembly; explicit little-endian célprofil.
aarch64-linux-gnu-gcc -std=c11 -O0 -g \
    -Wall -Wextra -Wconversion \
    -march=armv8-a -mlittle-endian \
    -S lab-bits.c -o lab-bits.s

# A64 ISA assembly -> objektum.
aarch64-linux-gnu-as -EL -march=armv8-a \
    lab-bits.s -o lab-bits.o

# Statikus AArch64 Linux-program.
aarch64-linux-gnu-gcc -static lab-bits.o -o lab-bits

# Futtatás: qemu-user.
qemu-aarch64 ./lab-bits
printf 'exit=%d\n' "$?"
```

Várt eredmények (mérve: Ubuntu 24.04, GCC 13.3.0, QEMU 8.2.2):

Ellenőrzés	Várt adat
`0x2D`	`0b00101101`, előjel nélkül 45, előjelesen 45
`0xD3`	`0b11010011`, előjel nélkül 211, előjelesen −45
`0xFF`	`0b11111111`, előjel nélkül 255, előjelesen −1
Kiterjesztések	`zero=0x000000d3 sign=0xffffffd3`
Első összeadás	`0x00`, `UO=1`, `SO=0`
Második összeadás	`0x80`, `UO=0`, `SO=1`
Harmadik összeadás	`0x7f`, `UO=1`, `SO=1`
Önteszt és kilépés	`PASS`, majd `exit=0`

Az `UO` és `SO` a program által kiszámított, 8 bites előjel nélküli, illetve előjeles túlcsordulási feltétel. **Nem a processzor állapotjelzőinek kiolvasása.**

2.2.10 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Ellenpróba
A várt −45 helyett 211 jelenik meg	Rossz előjeles értelmezés	A `0xD3` mintából vond ki a 256-ot
Szélesítés után `0x00D3` látható	Nullakiterjesztést alkalmaztál	Vizsgáld meg a forrás típusát és a kívánt értéket
„Nincs kilencedik bit, tehát nincs túlcsordulás”	Összekevered a két értéktartományt	Próbáld ki a `0x7F + 0x01` esetet
Azt vártad, hogy 8 bites összeadást látsz, de 32 bites `add w…` van	A várakozás hibás: az egészpromóció miatt a C `int`-ben számol	Ellenőrizd a C szabályát és a generált `add w…` utasítást

Cél: qemu-user programjának statikus vizsgálata, A64 ISA disassembly:

```
aarch64-linux-gnu-objdump -d --disassemble=lab_add8 ./lab-bits
```

Ez a parancs nem futtatja a programot. Egyelőre azt figyeld meg, hogy a C-szintű bájtos adattípus nem követel minden köztes művelethez bájtszélességű utasítást.

Hibakereséskor előbb rögzítsd a bitmintát, a bitszélességet és a kívánt olvasatot. Ezután csak egy értelmezést vagy konverziót módosíts, majd ismételd meg az öntesztet. Pusztán más kiírási formátum választása nem javítja meg a hibás számítást.

2.2.11 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Alakítsd át a 173 előjel nélküli értéket 8 bites bináris és hexadecimális alakra! Mi ugyanezen minta előjeles értéke? Add meg a −83 érték 8 bites kódját és 16 bites előjelkiterjesztését!

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a labort a `0xC8 + 0x64` és a `0x9C + 0xCE` esetekkel! Előbb kézzel határozd meg az eredménymintát és a két túlcsordulási feltételt.

3. (hibakeresési, középszint — cél: megértés-ellenőrzés) A `lab_signed8` törzsét ideiglenesen cseréld `return (int32_t)x;` sorra. Mely eredmények romlanak el? Fordítsd újra

ugyanazokkal a parancsokkal, vizsgáld meg a kimenetet és az `objdump` eredményét, majd egyetlen javítással állítsd helyre a programot.

2.2.12 Megoldások

1. megoldás — Segítség. Bontsd 173-at kettőhatványok összegére (128+…), majd a kapott 8 bites mintát olvasd le a kettes komplemens képlet szerint: ha a legfelső bit 1, vond ki 256-ot.

1. megoldás — Teljes megoldás. $173 = 128 + 32 + 8 + 4 + 1$, ezért a minta `0b10101101`, azaz `0xAD`. Nyolc bites kettes komplemens olvasata $173 - 256 = -83$. A -83 kódja tehát `0xAD`, 16 bites előjelkiterjesztése `0xFFAD`. A `0x00AD` nem -83-at, hanem 173-at jelentene.

2. megoldás — Segítség. Számítsd ki mindkét esetben külön az előjel nélküli és az előjeles pontos összeget, majd vesd össze a 8 bitbe még beférő tartománnyal (0–255, illetve -128–127) — a túlcsordulás pontosan akkor áll fenn, ha a pontos érték kilóg ebből.

2. megoldás — Teljes megoldás. Az első esetben az előjel nélküli összeg $200 + 100 = 300$, maradéka 44, vagyis `0x2C`: `UO=1`. Előjelesen $-56 + 100 = 44$, ezért `SO=0`.

A második esetben $156 + 206 = 362$, maradéka 106, vagyis `0x6A`: `UO=1`. Előjelesen $-100 - 50 = -150$, ami kisebb -128-nál, ezért `SO=1`. A `main` öntesztje így bővíthető:

```
/* C17, AArch64 célprofil; beszúrás a végső puts elé. */
ok &= lab_add8(0xc8, 0x64, 0x2c, 1, 0);
ok &= lab_add8(0x9c, 0xce, 0x6a, 1, 1);
```

3. megoldás — Segítség. A `(int32_t)x` egyszerű konverzió nem vizsgálja, hogy a 8 bites minta legfelső bitje 1-e — gondold végig, melyik bemenet (`0x2d` vagy `0xd3`/`0xff`) ad emiatt eltérő eredményt az eredeti függvényhez képest.

3. megoldás — Teljes megoldás. A hibás változat a `uint8_t` numerikus értékét őrzi meg: `0xD3` esetén 211-et, `0xFF` esetén 255-öt ad. Emiatt az előjeles kiírás, a kiterjesztés és az első túlcsordulási ellenőrzés (`0xFF + 0x01`: SO=1) hibás; a második és a harmadik eset véletlenül helyes marad. Ez didaktikailag is fontos: egy rossz implementáció is átmehet néhány teszten. Várt eredmény: `FAIL`, kilépési állapot 1. Az `objdump` a megváltozott megvalósítást mutatja, de az értelmezési hiba oka már a típusokból következik. A javítás a 128-tól kezdődő mintákból 256 kivonásának visszaállítása.

2.2.13 Rövid összefoglaló

A bitszélesség és az értelmezés együtt határozza meg egy bitminta számértékét. A hexadecimális jelölés négybites csoportokat tömörít, de önmagában nem mond semmit az előjelről vagy az adat szerepéről. Kettes komplemensben a legfelső bit negatív helyi értékű; ezért a pozitív és negatív tartomány nem szimmetrikus. Szélesítéskor a nullakiterjesztés az előjel nélküli, az előjelkiterjesztés az előjeles értéket őrzi meg. A labor szélesebb köztes típusokkal választotta külön a pontos összeget és a tárolt mintát. A gépi körbefordulásból nem következik az előjeles C-túlcsordulás megengedettsége.

- **Kettes komplemens:** előjeles értelmezés negatív legfelső helyi értékkel.
- **Túlcsordulás:** a pontos eredmény nem ábrázolható a választott tartományban.
- **Körbefordulás:** eredményképzés modulo 2^n szerint.

2.2.14 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Architecture Reference Manual for A-profile architecture, DDI0487: A64 adatszélességek és utasításkódolások. *Ellenőrizve: a link él* (`developer.arm.com` → `support.arm.com` átirányítás megerősített); a JavaScript-alapú megjelenítés miatt a pontos fejezetszám automatikus lekéréssel nem volt kinyerhető.
- WG14 N1570, C11 bizottsági tervezet: 6.2.5, 6.3.1.1, 6.3.1.3 és 6.5; előjel nélküli aritmetika, egészpromóciók, konverziók és túlcsordulás. Ez a WG14 utolsó nyilvános tervezete a C11 (ISO/IEC 9899:2011) ratifikálása előtt — a ténylegesen eladott ISO-szöveg fizetős, ezért a gyakorlatban mindenhol ezt a tervezetet idézik „a C11 szabvány” helyett; a két szöveg főleg szerkesztési részletekben tér el. Nem a C17 végleges szabványszövege.
- GNU assembler kézikönyv: AArch64 célbeállítások és objektumkészítés. **Ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- QEMU User Mode Emulation: idegen célarchitektúrájú Linux-programok futtatási modellje. **Ellenőrizve**: a hivatkozott dokumentáció a `master` ághoz tartozik, nem egy konkrét kiadott QEMU-verzióhoz; az Ubuntu 24.04 csomagolt verziója ettől eltérhet.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a helyi cross-libc statikus linkelhetősége, a ténylegesen telepített eszközláncverziók (gcc, binutils, QEMU), és a bemutatott laborkimenet tényleges, lefuttatott reprodukálása — ezek egyike sem történt meg itt.

2.3 Számítógép-architektúra fogalmai: ISA, mikroarchitektúra, regiszter, busz

2.3.1 Miért kell különválasztani a szerződést és a megvalósítást?

Egy C-ben írt tömbfeldolgozó függvény ugyanazzal az eredménnyel futhat két különböző ARM processzoron, miközben az egyik lényegesen hamarabb végez. Mi biztosítja az azonos eredményt, és mi okozza az eltérő futási időt?

A választ két szinten keressük:

- Az **utasításkészlet-architektúra** (*instruction set architecture*, ISA) meghatározza a gépi program számára látható működési szabályokat.
- A **mikroarchitektúra** (*microarchitecture*) az a konkrét belső szervezés, amellyel egy processzor ezeket a szabályokat megvalósítja.

Ez mérnöki szempontból alapvető különbség. A program helyességét nem alapozhatjuk egy adott processzor véletlen belső tulajdonságára; a teljesítményét viszont nem jósolhatjuk meg pusztán az utasítások jelentéséből.

Előfeltétel a Számrendszerek és bitszintű reprezentáció anyaga. Az alfejezet minden konkrét gépi példája **A64 ISA, AArch64, Armv8-A alapprofil**, little-endian környezetben. Cortex-M-utasítást itt nem használunk.

2.3.2 A tárolt program és a von Neumann-modell

A **tárolt program elve** (*stored-program concept*) szerint az utasítások kódolt bitmintái memóriában tárolhatók, ugyanúgy, mint a feldolgozott adatok. A von Neumann-modell közös memóriában kezeli a programot és az adatokat; a processzor ebből olvassa ki a végrehajtandó utasításokat. A Cortex-M3 ezzel szemben **módosított Harvard-architektúra**: külön I-Code és D-Code buszon éri el az utasítás- és adatterületet (a részleteket a 16.3 tárgyalja).

A „program mint adat” nem azt jelenti, hogy minden adat végrehajtható. Azt jelenti, hogy egy utasításnak is van tárolható, másolható, fájlba írható reprezentációja. Az assembler szöveges utasításnevekből ilyen gépi kódot állít elő.

Ugyanazokat a bájtokat különböző eszközök másként értelmezhetik:

- a disassembler utasításként próbálja visszafejteni;
- egy fájlellenőrző program bájtsorozatként dolgozza fel;
- a processzor utasítás-elővételezése végrehajtandó kódként kezeli.

A tényleges végrehajthatóságot a memória-hozzáférési jogosultságok is korlátozzák. Egy írható adatlap nem feltétlenül végrehajtható.

A mai processzorok gyakran külön utasítás- és adatgyorsítótárat használnak. Ez nem cáfolja a tárolt program elvét: a programozói modell lehet közös címtérre épülő, miközben a belső adatút részben szétválik. Nem következik belőle az sem, hogy minden memóriaforgalom egyetlen fizikai vezetékkötegen halad.

2.3.3 Mi tartozik az ISA-szerződéshez?

Az ISA több, mint az utasításnevek listája. Meghatározza többek között:

- az utasítások kódolását és műveleti jelentését;
- a program által elérhető regisztereket;
- az operandusok szélességét;
- a memóriaelérések és a vezérlésátadás szabályait;
- a kivételek és egyéb architekturális események releváns viselkedését.

A mikroarchitektúra ezzel szemben olyan döntéseket tartalmazhat, mint a végrehajtóegységek száma, a gyorsítótárak szervezése vagy az egyszerre feldolgozható utasítások mennyisége.

Kérdés	Elsődleges szint
Mit jelent az A64 `ADD W0, W1, #5`?	ISA
Az A64 `W0` írása hogyan hat az A64 `X0` felső bitjeire?	ISA
Hány egész összeadás indítható egy órajelciklusban?	Mikroarchitektúra
Mekkora egy adott processzor adatgyorsítótára?	Konkrét megvalósítás
Mennyi idő alatt futott le a laborprogram?	Mérés, meghatározott környezetben
Melyik szám jelöli Linux alatt a folyamatkilépést?	Operációsrendszer-interfész, nem ISA

Az azonos ISA-t megvalósító processzoroknak az előírt architekturális hatásokat kell biztosítaniuk, nem azonos belső lépéseket vagy futási időt. A bináris kompatibilitáshoz ezenfelül a használt kiterjesztések, az operációs rendszer és a bináris interfész egyezése is szükséges.

2.3.4 Regiszterfájl, ALU, busz és memóriahierarchia

A **regiszter** (*register*) a processzoron belüli, névvel elérhető tárolóhely. Az utasítás közvetlenül megnevezheti, mely regiszterek tartalmával dolgozzon. A programozó által látható regiszterek együttese az architekturális regiszterkészlet.

A **regiszterfájl** (*register file*) a regiszterértékeket tároló és kiszolgáló belső szerkezet. Egyszerű modellben megfelel a látható regiszterkészletnek; összetettebb processzorban több fizikai tárolóhely is lehet mögötte. Ennek részleteit egyelőre nem kell ismernünk.

Az **aritmetikai-logikai egység** (*arithmetic logic unit*, ALU) például összeadást és bitenkénti logikai műveleteket végez. Nem minden utasítást ugyanaz az egység hajt végre: a memóriaeléréshez vagy lebegőpontos számításhoz külön egységek tartozhatnak.

A **busz** (*bus*) kommunikációs út, amelyen címek, adatok és vezérlési információk továbbíthatók. Mai rendszerekben gyakran összetett összeköttetési hálózat helyettesíti az egyszerű közös buszt. Szélessége nem szükségképpen egyezik a processzor regiszterszélességével.

A **memóriahierarchia** (*memory hierarchy*) eltérő kapacitású és elérési költségű tárolási szinteket kapcsol össze:

Szint	Bevezető szerep
Regiszterek	Az utasítások közvetlen operandusai
Gyorsítótárak, *cache*-ek	Memóriatartalom gyorsan elérhető másolatai
Főmemória	A futó program nagyobb adat- és kódterülete
Háttértár	Tartós fájltárolás; nem közönséges regiszteroperandus-forrás

Az A64 *load/store* jellegű ISA: az itt használt egész összeadás regisztereken dolgozik. A memóriában lévő értéket előbb betöltjük, az eredményt pedig külön utasítással írjuk vissza.

2.3.5 Mit jelent a 32 és a 64 bites regiszternézet?

A **szóméret** (*word size*) kifejezés önmagában nem elég pontos: jelenthet természetes feldolgozási szélességet, de lehet dokumentációban rögzített adategység is. Az A64 terminológiájában a *word* 32 bites, a *doubleword* 64 bites mennyiség, noha az AArch64 általános célú regiszterei 64 bitesek.

A64-en az általános célú regiszterek `X0`–`X30` neve a 64 bites nézetet jelöli; `W0`–`W30` ugyanazon regiszterek alsó 32 bites nézete. Nem két független regiszterkészletről van szó.

Kiemelten fontos A64-szabály:

> Egy `Wn` célregiszter írása nullázza a megfelelő `Xn` regiszter felső 32 bitjét.

Például, előjel nélküli olvasatban:

A64-állapot	`X1` teljes tartalma	`W1` nézet
Kezdetben minden bit egy	`0xffffffffffffffff`	`0xffffffff`
A `W1` célregiszterbe 37 kerül	`0x0000000000000025`	`0x00000025`

Ez ISA-szerződés, nem egy adott processzor optimalizálása.

A 64 bites regiszter nem jelenti azt sem, hogy minden utasítás 64 bites kódolású: az **A64 utasításai 32 bites kódolásúak**. A regiszterszélességből továbbá nem következik, hogy minden lehetséges 64 bites cím használható. A C-típusok és a gépi nézetek megfeleltetésénél pedig a célplatform adatmodelljét is figyelembe kell venni.

2.3.6 Az elővételezés–dekódolás–végrehajtás modellje

Az **utasítás-elővételezés–dekódolás–végrehajtás** (*fetch–decode–execute*) ciklus fogalmi modell:

1. **Elővételezés:** a processzor a programszámláló (*program counter*, A64 `PC`) által meghatározott helyről utasításkódot kér le. Ezt gyakran az utasítás-gyorsítótár szolgáltatja.
2. **Dekódolás:** a bitmezők alapján felismeri a műveletet, a regiszteroperandusokat és az esetleges közvetlen értéket.
3. **Végrehajtás:** kiszámítja az eredményt, címet képez vagy vezérlést ad át.
4. **Állapotfrissítés:** az eredmény megjelenik a célregiszterben vagy memóriában, és kialakul a következő utasítás helye.

A64-ben szekvenciális haladáskor a következő utasítás címe 4 bájttal nagyobb; elágazás ezt megváltoztathatja.

Ez nem négy kötelező órajelciklus! Egy tényleges processzor finomabban bonthatja a munkát, és több utasítás különböző feldolgozási lépéseit átfedheti. Az architekturális eredménynek ettől még meg kell felelnie az ISA szabályainak.

2.3.7 Órajel, ciklus, késleltetés és átbocsátás

Az **órajel** (*clock*) időzítési referencia. Egy **órajelciklus** (*clock cycle*) időtartama:

$$T = \frac{1}{f}$$

ahol T másodpercben értendő, f pedig az órajelfrekvencia hertzben.

Elméleti, levezetett példa: állandó 2 GHz mellett egy ciklus 0,5 ns. Ha valamely végrehajtás pontosan 12 ciklust igényel, az ideje 6 ns. Ez nem állítás a későbbi labor futási idejéről.

Két további mennyiséget különítünk el:

- **Késleltetés, latency:** mennyi idő után használható fel egy művelet eredménye.
- **Átbocsátás, throughput:** időegységenként mennyi művelet indítható vagy fejezhető be a megadott feltételek között.

Egy hipotetikus, futószalagos egység eredménye lehet 3 ciklus után kész, miközben független műveletet minden ciklusban fogadhat. Egymás eredményére váró műveleteknél ez az átfedés nem használható ki ugyanígy.

Az „egy utasítás egy ciklus” ezért hibás általánosítás. Egy utasítás több ciklusig tarthat, több utasítás feldolgozása átfedhet, memóriaeléréskor pedig várakozás keletkezhet. A QEMU user-mode futási ideje nem alakítható át a célprocesszor hardverhű ciklusszámává.

2.3.8 Kidolgozott példa: memóriából regiszterbe, majd vissza

Munkakönyvtár: `~/arm-labor/a64/a003/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

> **Futtatási feltétel.** A labor a 3.1-ben telepített `aarch64-linux-gnu-*` eszközláncot és `qemu-aarch64`-et használja. Ha ezek még nincsenek a gépen, előbb végezd el a 3.1 telepítését (gyors parancs: `sudo apt install gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu gdb-multiarch qemu-user`); addig a labort csak olvasd el.

```
mkdir -p ~/arm-labor/a64/a003
cd ~/arm-labor/a64/a003
```

Szerepe a fejezetben: ez a példa *célzott kiegészítés*: egyetlen jelenséget mutat meg külön builddel, ezért első olvasáskor kihagyható. A fejezet végigvitt projektjének ide tartozó lépése — amely nem hagyható ki — a kidolgozott példa után, a „Végigvitt projekt” című részben található.

A labor feladata: egy memóriában tárolt, előjel nélküli 32 bites értékhez adjunk 5-öt, tároljuk az eredményt, majd adjuk vissza kilépési állapotként.

C-szinten a művelet `eredmeny = bemenet + 5u`. Most láthatóvá tesszük a mögötte álló adatmozgást is.

A példa előre használ néhány eszközlánc-elemet; telepítésüket a Labor-alapkörnyezet: eszközlánc telepítése és verifikáció tárgyalja. Nincs szükség fizikai ARM gépre.

Mentsük `lab-isa.s` néven:

```
// A64 ISA, Armv8-A; GNU as, AArch64-cél; little-endian.
.section .text
.balign 4
.global _start
.type _start, %function

_start:
    adr     x2, lab_values
    movn    x1, #0
    ldr     w1, [x2]
    add     w0, w1, #5
    str     w0, [x2, #4]

lab_before_exit:
    movz    x8, #93
    svc     #0

.size _start, . - _start

.section .data
.balign 4
lab_values:
```

```
    .word   37
    .word   0

.section .note.GNU-stack,"",%progbits
```

A ponttal kezdődő sorok **assemblerdirektívák**, nem processzorutasítások. Ebben a GNU as, AArch64-célú példában a `.word` 32 bites adatot hoz létre, a `.balign 4` pedig 4 bájtos igazítást kér. A címeket a linker rendezi el; nem használunk kézzel választott memóriacímeket.

Az A64-utasítások jelentése és releváns kódolási korlátai:

A64-utasítás	Hatás és korlát
`ADR X2, lab_values`	PC-relatív címet képez. Eltérése előjeles 21 bites bájteltérés: −1 MiB és +1 MiB − 1 bájt között.
`MOVN X1, #0`	A nullás közvetlen érték invertálásával minden bitet egyre állít. Az immediate 16 bites; 64 bites alakban eltolása 0, 16, 32 vagy 48 lehet, itt 0.
`LDR W1, [X2]`	4 bájtot tölt be. A használt unsigned-offset alak 12 bites, 4 bájttal skálázott eltérést enged; itt az eltérés 0.
`ADD W0, W1, #5`	32 bites összeadás. Az immediate 12 bites, opcionálisan 12 bites balra tolással; itt nincs eltolás.
`STR W0, [X2, #4]`	4 bájtot tárol a második adatmezőbe. Az eltérés 4 többszöröse, ebben az alakban 0–16380 bájt lehet.
`MOVZ X8, #93`	Nullázás mellett beírja a 16 bites immediate értékét; az eltolási lehetőségek azonosak a fenti 64 bites `MOVN` lehetőségeivel.
`SVC #0`	Supervisor-hívást kezdeményez; az immediate 16 bites. Linux/AArch64 alatt itt a rendszerhívási interfészt használjuk.

A Linux/AArch64 konvencióban az A64 `X8 = 93` választja az `exit` rendszerhívást, az A64 `X0` pedig az állapot argumentuma. Ez Linux-szabály: az A64 `SVC #0` önmagában nem jelent „kilépést”.

2.3.8.1 Összeállítás, linkelés, futtatás

```
# Target: qemu-user, Linux/AArch64, little-endian.
# Host baseline: Ubuntu Server 24.04 LTS x86_64.

aarch64-linux-gnu-as -march=armv8-a -g -o lab-isa.o lab-isa.s
aarch64-linux-gnu-ld -e _start -o lab-isa lab-isa.o
aarch64-linux-gnu-objdump -d ./lab-isa

if qemu-aarch64 ./lab-isa; then
    lab_status=0
else
    lab_status=$?
fi
printf 'Kilépési állapot: %d\n' "$lab_status"
```

Várt, levezetett kimenet: `Kilépési állapot: 42`. A program nem ír szöveget; ezt a shell írja ki. A 42 itt szándékos, nem hibát jelző eredmény.

A64-en a `lab_before_exit` címke elérésekor:

Megfigyelhető állapot	Várt érték, előjel nélküli olvasat
`X1`	`0x0000000000000025`
`X0`	`0x000000000000002a`
`X2`	A `lab_values` futásidejű címe
Első 32 bites adatmező	37
Második 32 bites adatmező	42

Little-endian tárolásnál a második mező bájtjai növekvő cím szerint: `0x2a`, `0x00`, `0x00`, `0x00`.

Megjegyzés az `objdump`-kimenethez. A `movn x1, #0` és `movz x8, #93` utasításokat a disassembler a preferált `mov` aliassal írja ki: `mov x1, #0xffffffffffffffff // #-1`, illetve `mov x8, #0x5d // #93`. A kódolás ugyanaz (`92800001`, `d2800ba8`); a `MOVN`/`MOVZ` forrásalak és a `MOV` disassembly-alak ugyanannak az utasításnak két nézete.

2.3.8.2 A köztes állapot ellenőrzése

Az első terminálban:

```
# Target: qemu-user, Linux/AArch64; GDB-kapcsolatra vár.
qemu-aarch64 -g 1234 ./lab-isa
```

A második terminálban:

```
# Target: az előző qemu-user folyamat AArch64 vendégállapota.
# A GDB kisbetűs nevei az A64 X0, X1, X2 regisztereket jelölik.
gdb-multiarch -q ./lab-isa \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_before_exit' \
    -ex 'continue' \
    -ex 'info registers x0 x1 x2' \
    -ex 'x/2uw $x2' \
    -ex 'continue' \
    -ex 'quit'
```

Hosszabb munkamenethez a GDB-parancsok szkriptfájlba tehetők:

```
cat > lab.gdb <<'GDBEOF'
target remote :1234
break lab_before_exit
continue
info registers x0 x1 x2
x/2uw $x2
continue
quit
GDBEOF
gdb-multiarch -q ./lab-isa -x lab.gdb
```

A `-x lab.gdb` forma újrafelhasználható, verziókövethető, és a későbbi laborokban is ezt érdemes használni a hosszú `-ex` láncok helyett.

A töréspont a kilépés előtt állítja meg a programot. Az `x/2uw` két, előjel nélküli decimális, 4 bájtos memóriaegységet jelenít meg; várt tartalmuk 37 és 42. A folytatás után a QEMU-folyamat várt

kilépési állapota ismét 42. Ha a GDB a folyamat leállásakor `exited with code 052` üzenetet ír, a szám **oktális**: `052` = 42 decimális. A decimális értéket a `p $_exitcode` mutatja. GDB 15.1 és QEMU 8.2.2 mellett megjelenhet a `BFD: warning: system-supplied DSO at … has a corrupt string table index` figyelmeztetés is; ez ártalmatlan, a debugolást nem érinti.

2.3.8.3 Végigvitt projekt: a `lab-hidden` utasításai mint ISA-szerződés

A Miért assembly alfejezetben készült `lab-hidden` programon most az ISA-szerződés két elemét azonosítjuk: a rögzített utasításkódolást és a regiszternézetet. A vizsgálathoz elég a kész bináris; ugyanebben a könyvtárban dolgozunk:

```
# Hostoldali vizsgálat; a fájl célja AArch64 Linux, qemu-user.
cp ~/arm-labor/a64/a001/lab-hidden .
aarch64-linux-gnu-objdump -d --disassemble=lab_next_u lab-hidden
# A kimenet a main mellett üres .fini szekciófejlécet is hozhat; a lab_next_u szekciója a
↪  lényeg:
#   11000400:   add w0, w0, #0x1
#   11000404:   ret
```

Levezetett, várt részlet (a címek a buildtől függenek):

```
<lab_next_u>:
  ...:        11000400        add        w0, w0, #0x1
  ...:        d65f03c0        ret
```

Mit mutat ez az alfejezet fogalmaival?

- **Utasításkódolás:** mindkét utasítás pontosan 4 bájt (`11000400`, `d65f03c0`). A 32 bites, rögzített hosszú kódolás az A64 ISA-szerződés része, nem a fordító döntése.
- **Regiszternézet:** az `add` a `w0`-t, vagyis az `X0` 32 bites nézetét használja; a `uint32_t` szélessége így jelenik meg a gépi szinten.
- **Ami nem látszik:** hogy az `add` hány órajelciklus alatt fut le, vagy hogy a mag milyen futószalagon hajtja végre — ez a mikroarchitektúra kérdése, a bináris erről nem árul el semmit.

2.3.9 Tipikus hibák és ellenpróbák

„Az A64 `X1` felső fele biztosan megmarad.”
Hipotézis: az A64 `W1` írása csak az alsó biteket módosítja. Ellenpróba: a program előbb minden bitet egyre állít, majd betölt az A64 `W1` célregiszterbe. A GDB-ben látott nullás felső fél cáfolja a hipotézist.

„A fájlban még nulla van, tehát a tárolás nem működött.”
Az `objdump` az ELF-fájlt vizsgálja, nem a futó folyamat módosított adatmemóriáját. A futásidejű állapotot a GDB-ben ellenőrizzük.

„A 42 utasításszámot vagy ciklusszámot jelent.”
Nem: ez a kiszámított eredmény, amelyet kilépési állapotként továbbítunk. A helyességi teszt nem teljesítménymérés.

„Az `ADR` minden távolságot elér.”
Nem: a relatív tartomány korlátozott. Tartományhiba esetén a linker diagnosztikáját kell vizsgálni (jellemző üzenet: `relocation truncated to fit: R_AARCH64_ADR_PREL_LO21`), és az `ADRP`+`ADD` párosra kell váltani (lásd 7.1), nem találomra más címet beírni.

2.3.10 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Sorold az ISA, mikroarchitektúra vagy mérési eredmény kategóriába: az A64 utasításhossz; az adatgyorsítótár mérete; az A64 `W1` írásának hatása az A64 `X1` felső felére; egy futás falióraideje.

2. (implementációs, alapszint — cél: rutinépítés) Csak az összeadás közvetlen értékét módosítsd úgy, hogy a program 44-et adjon vissza. Építsd újra az előbbi parancsokkal, és ellenőrizd a második adatmezőt is.

3. (hibakeresési, középszint — cél: rutinépítés) Kontrollált hibaként cseréld az A64 `ADD` utasításban az 5-öt 6-ra. Írd le a tünet–hipotézis–adatgyűjtés–javítás–ellenőrzés láncot. Használd az `objdump` és a GDB eredményét.

4. (számítási, alapszint — cél: megértés-ellenőrzés) Egy hipotetikus végrehajtás 2 GHz állandó órajel mellett 20 ciklust vesz igénybe. Mennyi az ideje? Miért nem állapítható meg ebből az utasítások száma?

2.3.11 Megoldások

1. megoldás — Segítség. Melyik tulajdonságot garantálja MINDEN azonos ISA-t megvalósító processzor, és melyik változhat gyártónként vagy akár futtatásonként?

1. megoldás — Teljes megoldás. Az A64 utasításhossz és a regiszternézetek írási szabálya ISA-tulajdonság. A gyorsítótár mérete megvalósítási tulajdonság. A falióraidő mérési eredmény, amelyhez a környezetet is rögzíteni kell.

2. megoldás — Segítség. 37-hez mennyit kell adni, hogy 44 legyen? Fér-e ez az érték a 12 bites immediate mezőbe?

2. megoldás — Teljes megoldás. A módosított sor A64-környezetben: `add w0, w1, #7`. A 7 belefér a 12 bites immediate mezőbe. A várt eredmény 44, hexadecimálisan `0x2c`; ezt várjuk a második adatmezőben és kilépési állapotként.

3. megoldás — Segítség. Melyik regiszter tartalmazza a hibás összeget — az A64 `X1` (a betöltött bemenet) vagy az A64 `X0` (az összeadás eredménye)? Innen indulj a hipotézisalkotásnál.

3. megoldás — Teljes megoldás. Tünet: 42 helyett 43. Hipotézis: hibás összeadandó. Az `objdump` mutatja a tényleges immediate értéket; a GDB-ben az A64 `X1` továbbra is 37, az A64 `X0` és a második adatmező viszont 43. Egyetlen javítás: 6 helyett 5. Újraépítés után mindkét ellenőrzésnek 42-t kell adnia.

4. megoldás — Segítség. Mekkora egy ciklus ideje 2 GHz-en? Utána szorozd meg a ciklusszámmal, és gondold végig, miért nem azonos a ciklus- és az utasításszám, ha a feldolgozás átfedhet.

4. megoldás — Teljes megoldás. Az elméleti idő $20/(2 \cdot 10^9)$ másodperc, vagyis 10 ns. Az utasításszám nem következik a ciklusszámból, mert a feldolgozás átfedhet, illetve várakozásokat tartalmazhat.

2.3.12 Rövid összefoglaló

Az ISA azt rögzíti, milyen állapotváltozást kell a programnak megfigyelnie; a mikroarchitektúra azt határozza meg, hogyan jön létre ez az eredmény. A tárolt program utasításai memóriában reprezentált bitminták. A processzor elővételezi, dekódolja és végrehajtja őket, de ezek a fogalmi lépések nem rendelhetők automatikusan külön órajelciklusokhoz. A regiszterek közvetlen operandusokat tartanak, az ALU számol, az összeköttetések adatot továbbítanak. A labor külön betöltést, összeadást és tárolást mutatott, valamint ellenőrizhetővé tette az A64 32 bites regiszterírás nullakiterjesztő hatását. A helyes eredmény önmagában nem mondja meg a hardveres futási időt.

- **ISA:** a gépi program számára látható működési szerződés.
- **Mikroarchitektúra:** az ISA konkrét belső megvalósítása.
- **Órajelciklus:** az órajel egy periódusa, nem utasításszámláló.

2.3.13 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Architecture Reference Manual for A-profile architecture, DDI0487 — A64 regiszternézetek, utasításkódolások és architekturális hatások. **Ellenőrizve**: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 szintaxis és assemblerdirektívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- QEMU User Mode Emulation dokumentáció — user-mode futtatás és hibakeresési kapcsolódás. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát). A dokumentáció nem állít semmit a mikroarchitekturális modellezésről (cache, sávszélesség, ciklusidőzítés) sem.
- GNU GDB kézikönyv — távoli kapcsolat, töréspontok, regiszter- és memóriavizsgálat. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Mérési állapot: a program a fő úton (GNU as/ld 2.42, QEMU 8.2.2, GDB 15.1) lefut, kilépési állapota 42; a `lab_before_exit` töréspontnál x0=0x2a, x1=0x25, x2 a `lab_values` címe, `x/2uw $x2` → 37 42. A kimenet mérve; mikroarchitekturális késleltetési adatot nem mértünk.

2.4 Az ARM család áttekintése: profilok, verziók és a könyv két része

2.4.1 Miért nem elég annyit mondani, hogy „ARM-on fut”?

Egy C-ben megírt adatfeldolgozó függvényt Linux alatt szeretnél gyorsítani. Egy másik feladatban egy szenzor jelére kell kiszámítható időn belül reagálnod, operációs rendszer nélkül. Mindkettőhöz választhatsz ARM-alapú processzort, mégsem ugyanazt az assembly kódot, fordítóprogram-célt vagy indítási eljárást fogod használni.

Az „ARM” egy architektúracsalád megnevezése, nem egyetlen gépi nyelvé. A megfelelő cél kiválasztásához külön kell azonosítani:

- az architektúraprofilt;
- az architektúraverziót;

- a végrehajtási állapotot és utasításkészletet;
- a konkrét processzormagot;
- a szoftveres futási környezetet.

Az előző alfejezet ISA és mikroarchitektúra közötti különbségére építve most ezeket helyezzük el. Az utasítások részletes működése helyett a cél annak felismerése, hogy **melyik program milyen gépen és milyen környezetben értelmezhető**.

2.4.2 Architektúraprofilok: A, R és M

Az **architektúraprofil** (*architecture profile*) az ARM architektúracsaládon belül egy alkalmazási területhez igazított követelmény- és szolgáltatáskészlet. Nem pusztán teljesítménykategória: meghatározza például a memóriakezelés és a kivételkezelés alapmodelljét.

Profil	Jellemző feladat	Jellemző memóriakezelés	Jellemző szoftverkörnyezet
A-profile, alkalmazási profil	Általános alkalmazások, szerverek, összetett felhasználói rendszerek	Virtuális memória, MMU	Linux és más összetett operációs rendszerek
R-profile, valós idejű profil	Határidős vezérlés, tárolóvezérlők, ipari és biztonságkritikus rendszerek	Architektúraverziótól függő; gyakran MPU, egyes újabb változatokban MMU is	Valós idejű operációs rendszer vagy célzott firmware
M-profile, mikrokontroller-profil	Beágyazott vezérlés, szenzorok, kis erőforrású perifériakezelés	A könyv Cortex-M3 célján nincs MMU; MPU alkalmazható, ha megvalósított	Operációs rendszer nélküli firmware vagy RTOS

A **memóriakezelő egység** (*Memory Management Unit, MMU*) címfordítást és hozzáférés-szabályozást tesz lehetővé. Linux alatt a folyamatok jellemzően saját virtuális címtérben működnek: a C-mutató értéke nem közvetlenül a fizikai memória címét jelenti.

A **memóriavédelmi egység** (*Memory Protection Unit, MPU*) ezzel szemben memóriatartományok hozzáférési tulajdonságait szabályozza, de nem biztosít MMU-szerű virtuális–fizikai címfordítást. Az MPU meglétét mindig a konkrét processzor megvalósításán kell ellenőrizni.

Az R-profile nevében a *real-time* nem azt jelenti, hogy „nagyon gyors”. A valós idejű rendszerben a helyes eredménynek meghatározott határidőn belül kell rendelkezésre állnia. Ehhez nem elegendő a megfelelő processzorprofil: a memória, az ütemezés, a megszakítások és a szoftver legrosszabb eseti viselkedése is számít.

A táblázat tehát tipikus felhasználást, nem kizárólagosságot mutat. A-profile rendszeren is lehet valós idejű feladatot végezni, és M-profile processzoron is futhat operációs rendszer. A könyv az R-profile részletes programozását nem tárgyalja.

Az M-profile verziói és a tudás átvihetősége. Az Armv7-M nem az egyetlen M-profile verzió; a könyv Cortex-M3-on tanult alapjai több-kevesebb kiegészítéssel átvihetők a későbbi magokra:

Verzió	Példa magok	Fő jellemző	A II. rész tudásának átvihetősége
Armv6-M	Cortex-M0, M0+	alap Thumb, szűk utasításkészlet	a Thumb-2 és NVIC anyag egy része nem elérhető
Armv7-M	Cortex-M3	Thumb-2, NVIC, opcionális MPU	a könyv referenciaútja
Armv7E-M	Cortex-M4, M7	DSP-utasítások, opcionális FPU	a Thumb-2/NVIC tudás átvihető; a DSP/FPU új
Armv8-M Baseline	Cortex-M23	TrustZone opcionális	alapvetően átvihető, de Armv6-M jellegű
Armv8-M Mainline	Cortex-M33	TrustZone, MPU	átvihető; a TrustZone új
Armv8.1-M	Cortex-M55, M85	Helium/MVE vektoros kiterjesztés	átvihető; a vektoros rész új
Armv9-A	Cortex-A5xx/A7xx	A-profile, SVE2	az I. rész A64-alapjai átvihetők, de a könyv nem tárgyalja

A QEMU-ban az `mps2-an385` Cortex-M3-at, az `mps2-an386` Cortex-M4-et, az `mps2-an505` Cortex-M33-at modellez; a könyv laborjai az elsőt használják.

2.4.3 Verzió, végrehajtási állapot és processzormag

Az elnevezések különböző kérdésekre válaszolnak:

Megnevezés	Mit azonosít?	Könyvbeli szerep
Armv8-A	Architektúraverzió és A-profile	Az I. rész architekturális alapprofilja
AArch64	64 bites végrehajtási állapot (*execution state*)	Az I. rész programjainak végrehajtási állapota
A64	Az AArch64 állapot utasításkészlete	Az I. rész assembly nyelve
Armv7-M	Architektúraverzió és M-profile	A II. rész architekturális alapja
Cortex-M3	Konkrét processzormag-típus	A II. rész referenciaprocesszora
Thumb-2	16 és 32 bites Thumb-utasításkódolásokat tartalmazó technológia	A II. rész Armv7-M utasításainak kódolási környezete

A Cortex-M3 tehát nem „az ARM harmadik verziója”: a név egy processzormag-típust jelöl, amely az Armv7-M architektúrát valósítja meg. Az Armv7-M és az Armv8-A verziószámából önmagában nem következik teljesítményarány; eltérő profilokról beszélünk.

A64 ISA, AArch64 környezetben az általános célú regiszterek 64 bites nézete `Xn`, 32 bites nézete `Wn`. Ettől függetlenül minden A64 utasítás kódolása 32 bites. A regiszterszélesség és az utasításhossz tehát külön fogalom.

Thumb-2/Armv7-M környezetben a Cortex-M3 általános célú regiszterei 32 bitesek, jelölésük `Rn`. Az utasításkódolások lehetnek 16 vagy 32 bitesek. Egy 32 bites Thumb-utasítás ettől nem válik A64 utasítássá, és egy 16 bites kódolás sem jelent szükségképpen 16 bites adatfeldolgozást.

A Cortex-M3 nem hajt végre A64 vagy A32 utasításokat. Az A-profile családban előforduló AArch32 állapot és annak A32/Thumb utasításai nem képezik a könyv I. részének célját.

Fontos további korlát: az AArch64 nem minden környezetben az A-profile szinonimája; az R-profile családnak is vannak AArch64 változatai. **Ebben a könyvben** az AArch64 cél mindig a kijelölt A-profile/Linux környezetet jelenti.

2.4.4 Miért Linux-userspace az I. rész?

A **felhasználói tér** (*userspace*) az operációs rendszer által korlátozott alkalmazásfuttatási környezet. Az I. részben a program betöltését, a folyamat kezdeti környezetét és a rendszererőforrások kezelését Linux biztosítja.

Ez pedagógiai előny: először a regiszterekkel, aritmetikával, memóriacímzéssel és függvényhívásokkal foglalkozhatunk. Nem kell rögtön a rendszerindítást vagy a soros port vezérlését is megírni. A kimenethez C-könyvtári függvényt vagy később rendszerhívást használhatunk.

Ez azonban nem közvetlen hardverhozzáférés. Egy Linux-alkalmazás nem írhat tetszőlegesen perifériaregisztereket, és nem módosíthatja szabadon a processzor privilegizált állapotát.

A labor elsődleges futtatója a `qemu-aarch64`, **felhasználói módú emulációban** (*user-mode emulation*). Ez AArch64 Linux-programokat futtat, és közvetíti a rendszerhívásaikat a gazdarendszer felé. Nem indít külön AArch64 Linux-kernelt, és nem emulál egy teljes Raspberry Pi alaplapot.

Az Ubuntu x86_64 hoston futó keresztfordító (*cross-compiler*) ettől még AArch64 gépi kódot állít elő. A fordító futtatási gépe és a létrehozott program célgépe két külön dolog.

2.4.5 Miért bare-metal a II. rész?

A II. részben a **közvetlen hardveren futó, operációs rendszer nélküli környezetet** (*bare-metal*) tanulmányozzuk. Itt nincs Linux, amely előkészítené a C-program futását vagy kiszolgálná a rendszerhívásait.

A firmware feladata lesz többek között:

- az indulási környezet és a kezdeti verem kijelölése;
- az inicializált és nullázandó adatterületek előkészítése;
- a kivételek és megszakítások belépési pontjainak megadása;
- a perifériák közvetlen kezelése.

Ezeket nem feltételezzük ismertnek: a minimal runtime fejezete építi fel őket.

A cél a Cortex-M3 referenciaprofil, QEMU-ban az `mps2-an385` gépmodell. A futtató eszköz itt `qemu-system-arm`, **rendszerszintű emulációban** (*system emulation*): a processzor mellett a kiválasztott gép memóriáját és modellezett perifériáit is biztosítja.

Az M-profile önmagában nem követeli meg a bare-metal működést; ez a könyv tudatos választása. Egy valós idejű operációs rendszer használata előtt így megérthető az a mechanizmus, amelyre az később épül.

2.4.6 Közös alapok, eltérő szerződések

Mindkét részben tovább használjuk a bináris reprezentációt, a regiszterek fogalmát, a memóriahozzáféréseket, az elágazásokat és a gépi kód vizsgálatát. Nem vihetők át változtatás nélkül a regiszternevek, az utasításkódolások, a hívási konvenciók vagy a programindítás szabályai.

Az I. rész a skaláris műveletektől a C–assembly együttműködésen át a NEON/SIMD feldolgozásig jut. A záróprojekt I egy C-programba illeszthető, teljesítménykritikus AArch64-modul: helyességét tesztekkel, hasznát mérési tervvel, működését audittal kell igazolni.

A II. részben az indulástól a megszakításokon át a perifériavezérlésig haladunk. A záróprojekt II megszakításvezérelt firmware-modul lesz, reprodukálható QEMU-teszttel és ellenőrizhető állapotkezeléssel.

A Cortex-M3 célon nincs NEON. A közösen továbbvihető tudás ezért elsősorban a gondolkodásmód és az ellenőrzési módszer, nem a konkrét utasítássor.

2.4.7 Kidolgozott példa: egy célplatform azonosítása

Munkakönyvtár: `~/arm-labor/a64/a004/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

> **Futtatási feltétel.** A labor a 3.1-ben telepített `aarch64-linux-gnu-*` eszközláncot és `qemu-aarch64`-et használja. Ha ezek még nincsenek a gépen, előbb végezd el a 3.1 telepítését (gyors parancs: `sudo apt install gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu gdb-multiarch qemu-user`); addig a labort csak olvasd el.

```
mkdir -p ~/arm-labor/a64/a004
cd ~/arm-labor/a64/a004
```

Szerepe a fejezetben: ez a példa *célzott kiegészítés*: egyetlen jelenséget mutat meg külön builddel, ezért első olvasáskor kihagyható. A fejezet végigvitt projektjének ide tartozó lépése — amely nem hagyható ki — a szakasz végén, a „Végigvitt projekt” részben található.

Feladat: készítsünk olyan Linux-programot, amely C-ből meghív egy A64 assembly függvényt, majd ellenőrizzük a célját és az eredményét.

Profil: I. rész, A64 ISA, Armv8-A, AArch64 Linux, little-endian, LP64 adatmodell. Utóbbiban a mutató 64 bites, az `unsigned int` 32 bites; ezt nem általában az „ARM” névből következtetjük.

Az eszközök telepítését a labor-alapkörnyezet részletezi. Itt már elérhetőnek tekintjük az AArch64 keresztfordítót, a célrendszer C-könyvtári fejlesztői állományait és a QEMU user-mode futtatót.

A `lab-profile.c` fájl:

```
/* I. rész: AArch64 Linux, LP64, little-endian laborprofil. */
#include <stdio.h>

#if !defined(__aarch64__) || !defined(__linux__)
#error "Ehhez a laborhoz AArch64 Linux celu fordito kell."
#endif

_Static_assert(sizeof(void *) == 8, "64 bites mutato szukseges");
_Static_assert(sizeof(unsigned int) == 4, "32 bites unsigned szukseges");

extern unsigned int lab_add(unsigned int a, unsigned int b);

int main(void)
```

```
{
    unsigned int result = lab_add(19u, 23u);

    printf("mutato=%zu; unsigned=%zu; osszeg=%u\n",
           sizeof(void *), sizeof(unsigned int), result);

    return result == 42u ? 0 : 1;
}
```

A `lab-profile.S` fájl:

```
// I. rész: A64 ISA, AArch64 Linux; GNU assembler-direktívák.
.text
.p2align 2
.global lab_add
.type lab_add, %function

lab_add:
    add w0, w0, w1
    ret

.size lab_add, .-lab_add
.section .note.GNU-stack,"",%progbits
```

Az A64 `ADD` itt regiszteroperandusokat használ, nincs kódolandó immediate konstansa. Az A64 `W0` és `W1` tartalmazza a két 32 bites, előjel nélküli argumentumot; az eredmény az A64 `W0` regiszterben tér vissza. Ez az AArch64 hívási konvenció egyik egyszerű esete, amelyet később az AAPCS64 regiszterszerződése részletez. Az A64 `RET` a hívóhoz tér vissza.

A ponttal kezdődő sorok GNU assemblerdirektívák, nem processzorutasítások. Az itt bemutatott jelentésük az AArch64 ELF-célhoz tartozik; például a `.p2align 2` $2^2 = 4$ bájtos igazítást kér.

Build és futtatás — target: qemu-user, AArch64 Linux:

```
aarch64-linux-gnu-gcc -std=c11 -O0 -Wall -Wextra \
    -march=armv8-a -mlittle-endian \
    -c lab-profile.c -o lab-profile-c.o

aarch64-linux-gnu-gcc -march=armv8-a -mlittle-endian \
    -c lab-profile.S -o lab-profile-asm.o

aarch64-linux-gnu-gcc -march=armv8-a -mlittle-endian -static \
    lab-profile-c.o lab-profile-asm.o -o lab-profile-a64

qemu-aarch64 ./lab-profile-a64
printf 'kilepes=%d\n' "$?"
```

Az első lépés a C-forrást fordítja, a második az assembly forrást szereli össze. A harmadik linkeli a programot a szükséges C-futtatókörnyezettel. A statikus linkelés ennél a példánál elkerüli a külön AArch64 dinamikus betöltő megadását; a program ettől továbbra is Linux-szolgáltatásokat igényel.

Mért eredmény (Ubuntu 24.04, GCC 13.3.0, QEMU 8.2.2):

Ellenőrzés	Várt érték
A program kiírása	`mutato=8; unsigned=4; osszeg=42`
A shell kiírása	`kilepes=0`
Visszatérési érték az A64 `RET` előtt	A64 `W0` = 42, előjel nélküli értelmezésben

A bináris célját külön is ellenőrizzük.

Binárisvizsgálat — target: ugyanaz az AArch64 Linux-program:

```
aarch64-linux-gnu-readelf -h lab-profile-a64
aarch64-linux-gnu-objdump -d --disassemble=lab_add lab-profile-a64
```

Az ELF-fejlécben `ELF64`, little-endian adatábrázolás és `AArch64` géptípus várható. A függvény visszafejtett kódjában az A64 `ADD` és `RET` utasításnak kell megjelennie.

Ez az ellenőrzés az ISA-t és a bináris formátumot azonosítja, **nem** egy konkrét processzormagot. A sikeres futásból sem a fizikai gép típusára, sem hardveres ciklusszámra nem következtethetünk.

2.4.7.1 Végigvitt projekt: a `lab-hidden` célplatformja

Ugyanezt az azonosítást a fejezet végigvitt projektjén is elvégezzük:

```
# Hostoldali vizsgálat; a fájl célja AArch64 Linux, qemu-user.
cp ~/arm-labor/a64/a001/lab-hidden .
aarch64-linux-gnu-readelf -h lab-hidden | grep -E 'Class|Data|Type|Machine'
```

Levezetett, várt eredmény:

ELF-mező	Várt érték	Mit azonosít
`Class`	`ELF64`	64 bites fájlformátum
`Data`	`2's complement, little endian`	little-endian adatábrázolás
`Type`	`EXEC`	statikusan linkelt, nem PIE futtatható fájl
`Machine`	`AArch64`	A-profile architektúra AArch64 végrehajtási állapota, A64 ISA

A `lab-hidden` tehát az I. rész célplatformjára készült: A-profile, AArch64, Linux-userspace. Egy Cortex-M firmware a II. részben ugyanitt `ELF32` osztályt és `ARM` géptípust mutat majd. Ahogy a fenti példánál, a fejléc itt sem árulja el a konkrét processzormagot.

2.4.8 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés és egyetlen célzott változtatás
A C-fordításnál a beépített ellenőrzés hibát jelez	A host `gcc` fordítóját használtuk	Ellenőrizzük a parancsot; csak a fordítót cseréljük `aarch64-linux-gnu-gcc`-re
Statikus linkeléskor hiányzó könyvtár jelenik meg	Hiányos a célrendszer C-fejlesztői környezete	Vizsgáljuk meg a linkerüzenetet; a hiányzó célcsomagot rendezzük
A program fut, de hibás összeget ad	Az assembly operandusai hibásak	`objdump` segítségével nézzük meg a tényleges utasítást; csak a hibás operandust javítsuk

Tünet	Hipotézis	Adatgyűjtés és egyetlen célzott változtatás
A programot Cortex-M firmware-ként próbáljuk használni	Összekevertük az ISA-t és a futási környezetet	Az ELF-fejléc alapján azonosítsuk a célt; ne átnevezéssel próbáljuk átalakítani

Az utolsó hiba nem javítható azzal, hogy másik QEMU-programot választunk. A Cortex-M3 célhoz más gépi kód, más indulási környezet és firmware-hez tartozó memóriakiosztás kell. Az ELF konténerformátum mindkét világban használható, de ettől a benne levő programok nem kompatibilisek.

2.4.9 Feladatok

1. **Magyarázó feladat (alapszint — cél: megértés-ellenőrzés).** Egy követelmény szerint a készülék összetett Linux-alkalmazásokat futtat; egy másik szerint szűk határidős vezérlési feladatot lát el; egy harmadik szerint kis firmware-rel szenzort kezel. Mely profilok tipikus kiindulópontok? Miért nem bizonyítja önmagában az R-profile választása a határidő betartását?
2. **Implementációs feladat (alapszint — cél: rutinépítés).** A laborban add össze a `0xffffffffu` és `1u` előjel nélküli értékeket. Módosítsd az elvárt eredményt is. Mi legyen a kiírás és a kilépési állapot? Használd ugyanazt a build–futtatás lépéssort.
3. **Hibakeresési feladat (középszint — cél: rutinépítés).** Az eredeti `19u`, `23u` bemenetek mellett módosítsd az A64 utasítást `add w0, w0, w0` alakra. Fordítsd újra a programot. A kimenetből állíts fel hipotézist, majd `objdump` segítségével ellenőrizd. Egyetlen operandus módosításával javítsd a hibát.

2.4.10 Megoldások

1. megoldás — Segítség. Gondold át, melyik profil melyik tulajdonságot (gazdag OS-felszín, garantált válaszidő, minimális erőforrás) teszi lehetővé — és hogy a "garantált" szó miért nem a profilból, hanem a teljes rendszer tervezéséből következik.

1. megoldás — Teljes megoldás. Linux-alkalmazásokhoz az A-profile, határidős vezérléshez gyakran az R-profile, kis szenzoros firmware-hez az M-profile kézenfekvő. Ezek kiindulópontok, nem automatikus döntések. A határidőt a teljes rendszer legrosszabb eseti végrehajtási és várakozási ideje határozza meg; a profil nem ad önmagában végponttól végpontig tartó időgaranciát.

2. megoldás — Segítség. Számítsd ki `0xffffffffu + 1u`-t 32 biten, moduló 2^{32} — mi lesz az eredmény bitmintája, és ez milyen C `unsigned int` értéknek felel meg?

2. megoldás — Teljes megoldás. A két módosított C-sor:

```
/* I. rész: ugyanaz az AArch64 Linux laborprofil. */
unsigned int result = lab_add(0xffffffffu, 1u);
/* A printf változatlanul marad. */
return result == 0u ? 0 : 1;
```

A levezetés: a 32 bites előjel nélküli legnagyobb érték $2^{32}-1$. Egyet hozzáadva 2^{32} keletkezik, amelynek alsó 32 bitje nulla. Az A64 `W0` eredménye ezért `0`, összhangban a 32 bites C `unsigned int` moduláris aritmetikájával. A várt kiírás vége `osszeg=0`, a kilépési állapot `0`.

3. megoldás — Segítség. Számítsd ki előre, mit adna `19+19` az elvárt `19+23` helyett — ha a kimenet ezt a hibás összeget mutatja, az operandus-cserére gyanakodj, és ezt `objdump`-pal igazold.

3. megoldás — Teljes megoldás. A várt hibás eredmény `38`, a kilépési állapot `1`. Az A64 utasítás kétszer az első argumentumot adja össze: `19 + 19`. Az `objdump` kimenetében a harmadik operandus A64 `W0`, nem A64 `W1`. Ennek javítása és újrafordítás után ismét `42`, illetve `0` várható. Ez funkcionális ellenőrzés, nem teljesítménymérés.

2.4.11 Rövid összefoglaló

Az ARM családon belül a profil, a verzió, a végrehajtási állapot és a konkrét processzormag külön fogalom. Az A-profile tipikusan összetett operációs rendszereket, az R-profile határidős rendszereket, az M-profile mikrokontrolleres feladatokat szolgál. A könyv először AArch64 Linux-alkalmazásokkal tanítja a gépi programozást, majd Cortex-M3 firmware-rel teszi láthatóvá az indulást, a megszakításokat és a perifériákat. A közös alapelvek átvihetők, a gépi kód és a futtatási szerződés nem. A laborban az ELF-fejléc a célt, a visszafejtett utasítások és a kimenet a működést ellenőrzik; egyik sem igazol hardveres teljesítményt.

- **Architektúraprofil:** alkalmazási területhez igazított architekturális követelménykészlet.
- **Végrehajtási állapot:** a processzor utasításkészletét és regisztermodelljét meghatározó állapot.
- **Bare-metal:** operációs rendszer nélküli, saját indulási környezetet biztosító programfuttatás.

2.4.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Architecture Reference Manual for A-profile architecture, DDI0487: AArch64 végrehajtási állapot és A64 ISA. *Ellenőrizve: a link él* (JS-renderelt oldal, a pontos kiadásbetű automatikus lekéréssel nem kinyerhető).
- Armv7-M Architecture Reference Manual, DDI0403: az Armv7-M programozási és végrehajtási modellje; a könyv referenciarevíziója DDI0403E. *Ellenőrizve: a link él* (JS-renderelt oldal, ugyanaz a korlát érvényes).
- GNU Binutils kézikönyvek: assemblerdirektívák, `readelf` és `objdump`. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- QEMU User Mode Emulation és QEMU System Emulation: a két emulációs mód eltérő hatásköre. **Tartalmilag ellenőrizve**: a `master` ágú dokumentáció nem egy konkrét kiadott QEMU-verzióhoz kötött; a ténylegesen telepített (pl. Ubuntu 24.04 csomagolt) verzió dokumentációja ettől eltérhet.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a példát itt nem fordítottuk és nem futtattuk. A telepített Ubuntu 24.04-csomagverziókat, a statikus célkönyvtárak elérhetőségét és a II. részhez szükséges `mps2-an385` gépmodell helyi támogatását a laborban kell ellenőrizni.

2.5 C fordítási láncolat és hova kerül az assembly

2.5.1 Miért kell látnunk a fordítási láncot?

Egy C-program helyesen lefordul, mégsem linkelhető; egy másikból elkészül a futtatható fájl, de az x86_64 fejlesztői gép nem tudja közvetlenül elindítani. Ugyanannak a függvénynek pedig más assembly megfelelőjét kapjuk optimalizálással, mint nélküle. Ezek különböző rétegek problémái, ezért különböző eszközökkel vizsgálandók.

A **fordítási eszközlánc** (*compilation toolchain*) a forrásprogramot több lépésben alakítja végrehajtható gépi programmá. Az assembly ennek nem a kiindulópontja és nem is a végállapota: a fordító egyik lehetséges szöveges kimenete, amelyből az assembler gépi kódot készít.

Az ISA és mikroarchitektúra különbségére, valamint az ARM-profilok áttekintésére építünk. A cél most a fájlok és eszközök szerepének felismerése, nem az assembly utasításkészletének megtanulása.

Profil: minden alábbi program AArch64 Linuxra készül, A64 ISA, Armv8-A alapprofil és little-endian bájtsorrend mellett. A host Ubuntu 24.04 x86_64, a futtatási cél `qemu-aarch64`, vagyis QEMU user-mode. A GNU assemblerdirektívák és az említett regiszterek ebben az AArch64-környezetben értendők; Cortex-M-kód itt nem szerepel.

2.5.2 A forrástól a futó folyamatig

A lánc fő állomásai:

Lépés	Bemenet	Kimenet	Fő feladat
Előfeldolgozó, preprocesszor (*preprocessor*)	`.c`, bevont fejlécek	előfeldolgozott C, jellemzően `.i`	Makróhelyettesítés, fájlbevonás, feltételes fordítás
Fordító (*compiler*)	előfeldolgozott C	assembly szöveg, `.s`	Elemzés, optimalizálás, célarchitektúrára történő kódgenerálás
Assembler	assembly szöveg	objektumfájl, `.o`	Utasítások kódolása, szimbólumok és relokációk létrehozása
Linker	objektumfájlok, könyvtárak	futtatható fájl vagy megosztott könyvtár	Szimbólumfeloldás, szekciók elrendezése, hivatkozások összekapcsolása
Betöltő (*loader*)	futtatható fájl, szükséges megosztott objektumok	futó folyamat memóriaképe	Programrészek leképezése, indulási környezet kialakítása

A preprocesszor például a `#define LAB_BIAS 22` után a `LAB_BIAS` makróelőfordulásokat helyettesíti. Nem végez regiszterkiosztást, és nem készít gépi utasításokat. Egy `.c` fájl az előfeldolgozás után, a bevont tartalmakkal együtt egy **fordítási egységet** (*translation unit*) alkot.

A fordító a C szemantikájából indul ki. Belső köztes reprezentációkat használhat, egyszerűsítheti a kifejezéseket, és kiválasztja a célgépi műveleteket. Nincs általános „egy C-sor = egy assembly utasítás” megfeleltetés.

Az assembler már nem C-kifejezéseket értelmez. A64 környezetben például egy összeadó utasítás szöveges alakjából annak gépi kódolását készíti el. A még nem ismert címekhez viszont később feldolgozandó információt hagyhat.

A linker összekapcsolja a külön fordított részeket. Ha az egyik objektum a `lab_add22` függvényre hivatkozik, a másik pedig definiálja azt, a linker összerendeli őket.

A betöltés már futásidejű feladat. Natív Linuxon a kernel az ELF programfejlécei alapján alakítja ki a folyamat képét; dinamikusan linkelt programnál egy felhasználói térben futó dinamikus betöltő is közreműködik. A QEMU user-mode a célprogram számára szükséges futtatási környezetet biztosítja, és emulálja az A64 utasítások végrehajtását. Nem teljes AArch64 gépet és vendégkernelt indít.

2.5.3 Mit jelentenek a GCC kapcsolói?

A `gcc` parancs a szokásos használatban **vezérlőprogram** (*compiler driver*): megszervezi a szükséges fordítási, assemblálási és linkelési lépéseket.

Kapcsoló	Hol áll meg?	Tipikus kimenet
`-E`	Előfeldolgozás után	`.i`, vagy `-o` nélkül a szabványos kimenet
`-S`	Assembly előállítása után	`.s`
`-c`	Objektumfájl előállítása után	`.o`
Egyik sincs megadva	A linkelést is elvégzi	Futtatható fájl
`-o név`	Nem választ feldolgozási lépést	A kimenet nevét adja meg

A `-o` nem azt jelenti, hogy „objektumfájlt kérek”. Ha `-S` mellett a kimenetnek `.o` végződésű nevet adunk, attól még assembly szöveg keletkezik.

A bemeneti kiterjesztés is számít:

- `.c`: előfeldolgozandó C-forrás;
- `.i`: már előfeldolgozott C;
- `.s`: közvetlenül assemblálható szöveg;
- `.S`: előbb a C-preprocesszoron átvezetendő assembly;
- `.o`: a linkeléshez felhasználható objektumfájl.

A szokásos egyparancsos fordítás nem feltétlenül hagyja meg a köztes fájlokat. A logikai lépések ettől még elkülöníthetők; az eszközök akár csővezetéken keresztül is továbbíthatnak adatot egymásnak.

2.5.4 Kidolgozott példa: két C-fájlból AArch64-program

Munkakönyvtár: `~/arm-labor/a64/a005/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

> **Futtatási feltétel.** A labor a 3.1-ben telepített `aarch64-linux-gnu-*` eszközláncot és `qemu-aarch64`-et használja. Ha ezek még nincsenek a gépen, előbb végezd el a 3.1 telepítését (gyors parancs: `sudo apt install gcc-aarch64-linux-gnu binutils-aarch64-linux-gnu gdb-multiarch qemu-user`); addig a labort csak olvasd el.

```
mkdir -p ~/arm-labor/a64/a005
cd ~/arm-labor/a64/a005
```

Szerepe a fejezetben: ez a példa *célzott kiegészítés*: egyetlen jelenséget mutat meg külön builddel, ezért első olvasáskor kihagyható. A fejezet végigvitt projektjének ide tartozó lépése — amely nem hagyható ki — a szakasz végén, a „Végigvitt projekt” részben található.

A példához a könyv cross-eszközlánca szükséges: `gcc-aarch64-linux-gnu`, `binutils-aarch64-linux-gnu`, `qemu-user`, továbbá az AArch64 glibc fejlesztői fájljai. Utóbbiakat az Ubuntu-környezetben jellemzően a `libc6-dev-arm64-cross` csomag biztosítja. A részletes telepítés a labor-alapkörnyezetben következik.

Két külön fordítási egységet készítünk. A `lab-main.c` használ egy függvényt, amelyet a `lab-math.c` definiál.

A `lab-main.c` tartalma:

```
#include <stdio.h>

int lab_add22(int x);

int main(void)
{
    int result = lab_add22(20);
    printf("eredmeny=%d\n", result);
    return result != 42;
}
```

A `lab-math.c` tartalma:

```
#define LAB_BIAS 22

int lab_add22(int x)
{
    return x + LAB_BIAS;
}
```

A `lab_add22` deklarációja alapján a fordító ellenőrizheti a hívás típusait. A függvénytörzs azonban nincs jelen a `lab-main.c` fordítási egységében. Ez nem fordítási hiba: a definíciót majd a linkernek kell megtalálnia.

2.5.4.1 1. Előfeldolgozás

```
# Build-cél: AArch64 Linux, későbbi futtatás qemu-user alatt.
aarch64-linux-gnu-gcc -E -O2 -march=armv8-a -mlittle-endian -fno-pie \
    lab-main.c -o lab-main.i
aarch64-linux-gnu-gcc -E -O2 -march=armv8-a -mlittle-endian -fno-pie \
    lab-math.c -o lab-math.i

# Hostoldali szövegvizsgálat.
sed -n '1,30p' lab-math.i
```

A `-E` lépés ugyanazokat az `-O2`, `-march` és `-fno-pie` kapcsolókat kapja, mint a fordítás, mert az optimalizálási és makrókapcsolók a preprocesszor kimenetét is befolyásolják: `-O2` mellett definiálódik a `__OPTIMIZE__`, és az Ubuntu GCC-je ekkor aktiválja a `_FORTIFY_SOURCE`-t. Ha az előfeldolgozás `-O2` nélkül fut, a `lab-main.i` és a belőle készült objektum más szimbólumot (például `printf` helyett `__printf_chk`) tartalmazhat, mint a végigvitt projekt.

A `lab-math.i` függvénytörzsében várhatóan már `return x + 22;` olvasható. A kimenet forráshelyeket jelölő sorokat is tartalmazhat. A `lab-main.i` jóval hosszabb lehet, mert a `stdio.h` és az általa bevont fejlécek tartalma is megjelenik benne.

2.5.4.2 2. Assembly előállítása

```
# Build-cél: A64 ISA, Armv8-A, little-endian; futtatási cél: qemu-user.
aarch64-linux-gnu-gcc -S -O2 -march=armv8-a -mlittle-endian \
    -fno-pie lab-main.i -o lab-main.s
aarch64-linux-gnu-gcc -S -O2 -march=armv8-a -mlittle-endian \
    -fno-pie lab-math.i -o lab-math.s
```

Az `-O2` optimalizálást kér. A `-march=armv8-a` az engedélyezett célarchitektúrát, a `-mlittle-endian` a bájtsorrendet rögzíti. A `-fno-pie`, majd a linkelésnél használt `-no-pie` ebben a laborban kikapcsolja a pozíciófüggetlen futtatható fájl készítését, hogy az eredmény típusa ne a disztribúció alapbeállításától függjön. Ez szemléltetési döntés, nem általános biztonsági ajánlás.

A `lab-math.s` egyik lehetséges, leegyszerűsített **A64-részlete**:

```
lab_add22:
    add w0, w0, #22
    ret
```

Ez nem a teljes generált fájl: a GNU assembler számára szükséges direktívákat és a metaadatokat itt elhagytuk. Az assembláláshoz az eredeti, GCC által készített `.s` fájlt használjuk.

A64 környezetben a `W0` 32 bites regiszternézet. Az AArch64 Linuxon alkalmazott AAPCS64 hívási konvenció szerint ebben a példában az első `int` argumentum és az `int` visszatérési érték is ezen keresztül halad. A `RET` alapértelmezésben az `X30` 64 bites regiszterben levő visszatérési címet használja. A részletes szerződést az AAPCS64 regiszterszerződése fejti ki.

Az A64 `ADD` itt használt immediate alakja nem fogad el tetszőleges konstanst: 12 bites előjel nélküli értéket kódol, opcionális 12 bites balra tolással. A `22` belefér a nem eltolt alakba.

2.5.4.3 3. Objektumfájlok készítése

```
# Build-cél: AArch64 Linux, qemu-user.
aarch64-linux-gnu-gcc -c -march=armv8-a -mlittle-endian \
    lab-main.s -o lab-main.o
aarch64-linux-gnu-gcc -c -march=armv8-a -mlittle-endian \
    lab-math.s -o lab-math.o
```

A `.s` bemenet és a `-c` együtt azt jelenti: az assembler dolgozzon, linkelés még ne történjen. A `.o` fájl már bináris, ezért szövegszerkesztő helyett objektumvizsgáló eszközt használunk.

2.5.4.4 4. Linkelés és futtatás

```
# Linkelési cél: statikus AArch64 Linux-program, qemu-user.
aarch64-linux-gnu-gcc -static -no-pie \
    lab-main.o lab-math.o -o lab-app

# Futtatási cél: qemu-user.
qemu-aarch64 ./lab-app
printf 'kilepesi_allapot=%d\n' "$?"
```

Várt, nem ténylegesen futtatott eredmény:

- a program kimenete: `eredmeny=42`;
- az utána kiírt állapot: `kilepesi_allapot=0`.

A `$?` közvetlenül az előző parancs állapotát tartalmazza. Ha előbb más parancsot futtatunk, már nem a QEMU-futtatás eredményét vizsgáljuk.

A linkelést szándékosan a GCC vezérlőprogramjával végezzük. Ez hozzáadja a C futtatókörnyezet szokásos induló objektumait és könyvtárait. A folyamat nem közvetlenül a `main` függvényben indul; az indulókód készíti elő annak meghívását.

A lánc rövidítve is megadható:

```
# Teljes build és futtatás: AArch64 Linux, qemu-user.
aarch64-linux-gnu-gcc -O2 -march=armv8-a -mlittle-endian \
    -fno-pie -static -no-pie \
    lab-main.c lab-math.c -o lab-app-onepass

qemu-aarch64 ./lab-app-onepass
printf 'kilepesi_allapot=%d\n' "$?"
```

A várt kimenet ugyanaz. Az egyparancsos változat önmagában nem őrzi meg a vizsgált `.i`, `.s` és `.o` köztes fájlokat.

2.5.4.5 Végigvitt projekt: a `lab-hidden` négy lépésben

A Miért assembly alfejezetben a `lab-hidden` buildje a C → assembly lépéssel indult; az előfeldolgozás rejtve maradt. Most mind a négy lépést külön fájlba mentjük:

```
# Build-cél: AArch64 Linux, qemu-user.
cp ~/arm-labor/a64/a001/lab-hidden.c .
LAB_CFLAGS="-std=c11 -O2 -fno-inline -Wall -Wextra -march=armv8-a -mlittle-endian"

aarch64-linux-gnu-gcc $LAB_CFLAGS -E lab-hidden.c -o lab-hidden.i   # 1. előfeldolgozás
aarch64-linux-gnu-gcc $LAB_CFLAGS -S lab-hidden.i -o lab-hidden.s   # 2. fordítás
aarch64-linux-gnu-gcc $LAB_CFLAGS -c lab-hidden.s -o lab-hidden.o   # 3. assemblálás
aarch64-linux-gnu-gcc -march=armv8-a -mlittle-endian -static \
    lab-hidden.o -o lab-hidden                                       # 4. linkelés

grep -n -A3 'lab_next_u(' lab-hidden.i   # a hívás több sorra tördelődhet
# vagy: grep -n '4294967295' lab-hidden.i
aarch64-linux-gnu-nm lab-hidden.o
qemu-aarch64 ./lab-hidden
printf 'exit=%s\n' "$?"
```

Mért megfigyelések (Ubuntu 24.04, GCC 13.3.0, QEMU 8.2.2):

Lépés	Mit ellenőrzünk	Várt megfigyelés
1. `.i`	a `main` hívássora	az `UINT32_MAX` makró helyén számkonstans áll (például `4294967295U`; a pontos alak a C-könyvtár fejléceitől függ)

Lépés	Mit ellenőrzünk	Várt megfigyelés
2–3. `.s`, `.o`	`nm lab-hidden.o`	`T lab_next_u`, `T lab_rises_s`, `T main` definiált, `U __printf_chk` feloldatlan (a `printf` fortifikált változata)
4. futtatás	kimenet és állapot	`unsigned_next=0`, `signed_rises=1`, `exit=0` — ugyanaz, mint a Miért assembly alfejezetben

A `__printf_chk` feloldatlan hivatkozása a `.o`-ban ugyanaz a jelenség, mint a `lab_add22` esete a fenti kiegészítő példában: a definíciót a linker a statikus C-könyvtárból emeli be. Az Ubuntu GCC-je `-O2` mellett alapértelmezésben `_FORTIFY_SOURCE=3`-at definiál, ezért a glibc fejléc a `printf` hívást `__printf_chk`-ra cseréli; ha a `printf` nyers neve kell, a build kapcsolói közé `-U_FORTIFY_SOURCE` (vagy `-D_FORTIFY_SOURCE=0`) tehető. A projekt ezzel a fejezet végére minden szinten — forrás, előfeldolgozott forrás, assembly, objektum, futtatható fájl — megvizsgált program.

2.5.5 Objektumfájl és futtatható fájl: mindkettő lehet ELF

Az **ELF** (*Executable and Linkable Format*) többféle bináris szerepet támogat. A „bináris fájl” ezért nem a „futtatható program” szinonimája.

Az objektumfájl már tartalmazhat gépi utasításokat, mégsem teljes program. Lehetnek benne feloldatlan külső hivatkozások és **relokációk** (*relocations*): olyan bejegyzések, amelyek megadják, hogyan kell egy címfüggő mezőt később a végleges elrendezéshez igazítani. Relokáció akkor is szükséges lehet, ha a hivatkozott szimbólum ugyanabban az objektumban definiált.

Vizsgáljuk meg a példát:

```
# Hostoldali vizsgálat; a fájlok célja AArch64 Linux, qemu-user.
aarch64-linux-gnu-readelf -h lab-main.o
aarch64-linux-gnu-readelf -h lab-app
aarch64-linux-gnu-nm -u lab-main.o
aarch64-linux-gnu-objdump -dr lab-main.o
```

Várt ellenőrzési pontok:

Vizsgálat	Várt megfigyelés
`lab-main.o` ELF-típusa	`REL`, relokálható objektum
`lab-app` ELF-típusa	`EXEC`, ennél a statikus, nem PIE buildnél
Mindkét fájl géptípusa	`AArch64`
Mindkét fájl adatábrázolása	little-endian
`nm -u lab-main.o`	Többek között `lab_add22` és `printf` feloldatlan hivatkozása
`objdump -dr`	Visszafejtett utasítások és a hozzájuk kapcsolódó relokációk

A `lab_add22` feloldatlan állapota itt normális: a fordítás sikerült, az összekapcsolás még hátravan. Az ELF részletes felépítése az ELF formátum alapjainál következik.

2.5.6 Statikus és dinamikus linkelés röviden

Statikus linkelésnél a szükséges könyvtári objektumok kódja bekerül a futtatható fájlba. A statikus könyvtár gyakran `.a` archívum; a linker általában a szükséges tagokat emeli be, nem automatikusan a teljes archívumot.

Dinamikus linkelésnél a program egyes szolgáltatásokat külön megosztott objektumokból, például `.so` fájlokból kap. A futtatáshoz kompatibilis céloldali dinamikus betöltő és könyvtárak szükségesek.

A meglévő objektumokból dinamikus változat is készíthető:

```
# Linkelési cél: dinamikus AArch64 Linux-program, qemu-user.
aarch64-linux-gnu-gcc -no-pie \
    lab-main.o lab-math.o -o lab-app-dynamic

# Hostoldali vizsgálat.
aarch64-linux-gnu-readelf -l lab-app-dynamic
aarch64-linux-gnu-readelf -d lab-app-dynamic

# Futtatási cél: qemu-user, AArch64 céloldali könyvtárakkal.
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab-app-dynamic
# A /usr/aarch64-linux-gnu sysroot a libc6-arm64-cross csomagból jön.
printf 'kilepesi_allapot=%d\n' "$?"
```

A megadott prefix az Ubuntu cross-környezetének szokásos céloldali könyvtárfája; meglétét a laborban ellenőrizni kell. Nem helyettesíthető a host x86_64 könyvtáraival.

A várt kimenet ismét `eredmeny=42`, majd nulla kilépési állapot. A programfejlécek között várhatóan `INTERP`, a dinamikus információk között `NEEDED` bejegyzés található. Az előbbi az ELF-interpretert, az utóbbi szükséges megosztott könyvtárat jelöl.

A statikus program sem „operációs rendszer nélküli”: továbbra is Linux-szolgáltatásokat használ. A statikus és dinamikus linkelés közötti különbség nem azonos a Linux és a bare-metal futtatás különbségével.

2.5.7 Miért változik az assembly ugyanabból a C-kódból?

A generált kódot több, egymástól elkülönítendő döntés alakítja:

- **Optimalizálás:** az `-O0` több közvetlen változó- és veremkezelést hagyhat meg; az `-O2` elhagyhat fölösleges műveleteket, összevonhat számításokat.
- **ISA:** a fordító csak az engedélyezett utasításkészletre támaszkodhat. Más célarchitektúra más utasításokat és regisztereket jelent.
- **Bináris alkalmazási interfész, ABI** (*Application Binary Interface*): megszabja többek között az argumentumátadást, a regiszterhasználatot és az adattípusok elrendezését.
- **Fordítóváltozat és további kapcsolók:** a kódgenerálás, a hibakeresési információ, a védelmi beállítások és a pozíciófüggetlenség is módosíthatja a kimenetet.
- **Fordítási egységek közötti optimalizálás:** linkeléskori optimalizálással (*link-time optimization*, LTO) a fordító olyan függvénytörzseket is együtt vizsgálhat, amelyek külön forrásfájlban vannak. A példánk nem kér LTO-t.

A helyes összehasonlítás nem a szövegazonosságot keresi, hanem a megkövetelt viselkedés megőrzését. Ez jól definiált C-programra vonatkozik: például az előjeles egész túlcsordulására nem alapozhatunk hordozható C-szemantikát. A labor `20 + 22` művelete 32 bites előjeles `int` esetén ábrázolható.

A rövidebb assembly önmagában nem bizonyít gyorsulást. Itt működést és fájlformátumot vizsgálunk, nem időt mérünk; a QEMU user-mode nem szolgáltat garantáltan hardverhű ciklusszámot.

2.5.8 Tipikus hibák és ellenpróbák

A hasznos hibakeresési sorrend: **tünet → hipotézis → adatgyűjtés → egy változtatás → új ellenőrzés**.

Tünet	Hipotézis és adatgyűjtés	Egy javítás és ellenőrzése
`undefined reference to lab_add22`	A linker nem kapta meg a definíciót; vizsgáljuk `nm`-mel az objektumokat	Adjuk hozzá a `lab-math.o` fájlt; linkeljünk újra
A fájl géptípusa x86-64	Véletlenül a host `gcc` fordított; ellenőrizzük `readelf -h` segítségével	Fordítsunk újra az `aarch64-linux-gnu-gcc` eszközzel
QEMU nem találja az ELF-interpretert	Dinamikus programhoz hiányzik vagy rossz a céloldali könyvtárfa; nézzük meg a `readelf -l` kimenetét	Javítsuk a `-L` prefixet, majd futtassunk újra
Forrásmódosítás után változatlan az eredmény	Elavult köztes fájlt használtunk	Indítsuk újra az érintett láncot a megváltozott bemenettől

Az utolsó eset különösen fontos: a `.i` nem élő kapcsolat a `.c` fájlhoz. Ha a makrót a forrásban átírjuk, de továbbra is a régi `.i` fájlt fordítjuk, a módosítás nem jut el a programig.

2.5.9 Feladatok

1. **Magyarázó feladat (alapszint — cél: megértés-ellenőrzés).** Melyik lépés helyettesíti a `LAB_BIAS` makrót, melyik kódolja az A64 utasításokat, és melyik kapcsolja össze a `lab_add22` hívását a definíciójával? Miért nem futtatható önmagában a `lab-main.o`?

2. **Implementációs feladat (középszint — cél: rutinépítés).** Állítsd a `LAB_BIAS` értékét `7`-re, és módosítsd a `main` ellenőrzését az új eredményhez. Végezd el újra a teljes lépéssort. Mi legyen a kimenet és a kilépési állapot? Mi történne, ha csak az eredeti `.o` fájlokat linkelnéd újra?

3. **Hibakeresési feladat (középszint — cél: rutinépítés).** Az eredeti forrásváltozat objektumaival hagyd el szándékosan a matematikai objektumot:

   ```
   # Szándékosan hibás linkelés; cél: AArch64 Linux, qemu-user.
   aarch64-linux-gnu-gcc -static -no-pie \
       lab-main.o -o lab-missing-math
   ```

   A várt eredmény nem nulla linkelési állapot; futtatható programra ne számíts. `-O2` mellett a `main` a `.text.startup` szekcióba kerülhet, ezért a linker `(.text.startup+0x10)` alakot írhat; ez nem hiba, a GCC a csak egyszer futó kódot ide teszi. Igazold a hiányzó definíció okát `nm` segítségével, majd egyetlen linkparancs-módosítással javítsd a hibát.

4. **Összehasonlító feladat (középszint — cél: megértés-ellenőrzés).** Készíts optimalizálatlan assemblyt is:

```
# Build-cél: AArch64 Linux, qemu-user; futtatás itt nincs.
aarch64-linux-gnu-gcc -S -O0 -march=armv8-a -mlittle-endian \
    -fno-pie lab-math.i -o lab-math-O0.s

# Hostoldali szövegösszehasonlítás.
diff -u lab-math-O0.s lab-math.s
```

Keress művelet- vagy veremkezelési különbséget. A `diff` eltérő fájloknál szokásosan `1` állapottal tér vissza; ez nem fordítási hiba. Mit nem állapíthatsz meg ebből a futási időről?

2.5.10 Megoldások

1. megoldás — Segítség. Nézd meg a fejezet elején lévő táblázatot: melyik sor bemenete/kimenete illeszkedik a makróra, melyik az utasításkódolásra, melyik a hivatkozás-összekapcsolásra?

1. megoldás — Teljes megoldás. A makrót a preprocesszor helyettesíti; az utasításokat az assembler kódolja; a külső hivatkozást a linker kapcsolja a definícióhoz. A `lab-main.o` relokálható részprogram: hiányzik belőle többek között a hivatkozott függvény definíciója és a teljes program indulási környezete.

2. megoldás — Segítség. Számold ki fejben `7 + 20`-at, és gondold át: a `.i`-fájl már a makróhelyettesítés UTÁNI állapotot tükrözi — ha csak a régi `.o`-kat linkelnéd újra, melyik lépés maradna ki?

2. megoldás — Teljes megoldás. Az eredmény `27`, ezért az ellenőrzés `return result != 27;`. A várt kimenet `eredmeny=27`, a kilépési állapot `0`. Mindkét módosított forrást újra elő kell feldolgozni, fordítani és assemblálni. A régi objektumok újralinkelése a régi viselkedést őrizné meg.

3. megoldás — Segítség. Az `nm -u` a feloldatlan (undefined), a `--defined-only` a ténylegesen definiált szimbólumokat listázza egy objektumban — melyik objektumban várod a `lab_add22`-t melyik szerepben?

3. megoldás — Teljes megoldás. A szükséges vizsgálat és javítás:

```
# Hostoldali vizsgálat, AArch64 objektumokon.
aarch64-linux-gnu-nm -u lab-main.o
aarch64-linux-gnu-nm --defined-only lab-math.o

# Javított linkelés és futtatás: AArch64 Linux, qemu-user.
aarch64-linux-gnu-gcc -static -no-pie \
    lab-main.o lab-math.o -o lab-app
qemu-aarch64 ./lab-app
printf 'kilepesi_allapot=%d\n' "$?"
```

A hívó objektumban `U`, a definiáló objektumban várhatóan `T` jelöli a `lab_add22` szimbólumot. Az eredeti változat várt eredménye `42`, kilépési állapota `0`. A hiba linkelési, nem betöltési probléma.

4. megoldás — Segítség. Hasonlítsd össze a két fájlban a `lab_add22` törzsét: van-e `sub sp/str/ldr/add sp` sor, és milyen `.cfi_*` direktívák kísérik? Az `-O0` jellemzően veremkeretet épít, az `-O2` gyakran regiszterben számol.

4. megoldás — Teljes megoldás. Az `-O0` változatban a `lab_add22` tipikusan veremkeretet épít:

```
lab_add22:
    sub  sp, sp, #16
    str  w0, [sp, 12]
    ldr  w0, [sp, 12]
    add  w0, w0, #22
    add  sp, sp, #16
    ret
```

A `-O2` változatban ugyanaz a függvény jellemzően keret nélkül, regiszterben számol:

```
lab_add22:
    add  w0, w0, #22
    ret
```

A különbség a veremkezelésben és a memóriaműveletek számában van: az `-O0` eltárolja és visszatölti az argumentumot, az `-O2` közvetlenül a regiszteren dolgozik. A `-O0` fájlban a `.cfi_def_cfa_offset` és társai a veremkeret leírását adják a kivételkezeléshez és a debuggerhez; a `-O2` fájlban ezek elmaradhatnak, mert nincs keret. A pontos utasítássor fordítóverzió- és kapcsolófüggő. **Futási időre ebből nem lehet következtetni**: az utasításszövegből hiányzik a cache-, pipeline- és elágazás-viselkedés, ezért tényleges időadat csak mérési tervvel és rögzített futtatókörnyezetben kapható.

2.5.11 Rövid összefoglaló

A C-program előfeldolgozáson, fordításon, assemblláláson és linkelésen keresztül válik betölthető programmá. Az assembly olvasható köztes reprezentáció, nem maga a futó program. A GCC vezérlőprogram `-E`, `-S` és `-c` kapcsolóival külön megállhatunk a lánc állomásainál; a `-o` csak nevet választ. Az objektumfájl gépi kódot tartalmazhat úgy is, hogy külső hivatkozásai még feloldatlanok. A linker és a betöltő eltérő feladatot végez. A generált assembly optimalizálási, ISA- és ABI-döntéseket tükröz, ezért szövege nem egyetlen kötelező fordítása a C-forrásnak. A laborban a várt működést a kimenet és a kilépési állapot együtt ellenőrzi.

- **Fordítási egység:** egy forrásfájl előfeldolgozott tartalma.
- **Objektumfájl:** linkelésre szánt bináris programrész.
- **Relokáció:** címfüggő mező későbbi kiigazítását leíró bejegyzés.

2.5.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GCC kézikönyv: Overall Options — feldolgozási lépések, kiterjesztések, `-E`, `-S`, `-c`, `-o`. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2.
- GNU assembler kézikönyv — assembly bemenet és objektumfájl-készítés. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU linker kézikönyv — objektumok, könyvtárak és szimbólumfeloldás. Ugyanaz a binutils 2.47-es kiadás.
- Linux kézikönyv: elf(5) — ELF-fájltípusok és betöltési struktúrák. **Tartalmilag ellenőrizve**: a Linux man-pages projekt 6.19-es kiadása.
- QEMU User Mode Emulation dokumentáció — felhasználói programok emulációja és céloldali futtatási környezet. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a kód nem volt futtatva; minden eredmény várt vagy szemléltető. A konkrét Ubuntu-csomagverziókat, a statikus glibc elérhetőségét, a céloldali könyvtárprefixet és a ténylegesen generált assemblyt a hallgatói laborban kell ellenőrizni.

3 Fejlesztői környezet és labor: eszközlánc, QEMU, build-munkafolyamat

3.1 Labor-alapkörnyezet: eszközlánc telepítése és verifikáció

3.1.1 Miért kell külön ellenőrizni a labor környezetét?

Egy assembly program hibája könnyen összetéveszthető egy környezeti hibával. Ha a shell nem találja az assemblert, a linker más architektúrájú objektumot kap, vagy a futtatási kísérlet rossz betöltőt használ, még nem a program algoritmusát kell javítanunk.

A mérnöki cél ezért egy **kicsi, ellenőrizhető eszközlánc** felépítése. A labor végén külön bizonyítékunk lesz arra, hogy:

- a szükséges programok telepítve vannak és elérhetők;
- a fordító és az assembler AArch64 gépi kódot készít;
- a linker megfelelő futtatható fájlt állít elő;
- a QEMU ezt a fájlt végrehajtja;
- a debugger felismeri a célarchitektúrát.

Az előfeltétel a C fordítási láncolat és hova kerül az assembly fogalmainak ismerete. Most nem ismételjük meg a fordítási lépéseket: működő laborba helyezzük őket.

Laborprofil: Ubuntu Server 24.04 LTS, x86_64 fejlesztői gép; AArch64 célarchitektúra, A64 ISA, Armv8-A alapprofil, little-endian bájtsorrend; futtatás `qemu-aarch64` user-mode emulációval. A labor eredményei a fő úton mérve: Ubuntu 24.04.5, `binutils-aarch64-linux-gnu` 2.42, `gcc-aarch64-linux-gnu` 13.2.0 (a fordító 13.3.0), `gdb-multiarch` 15.1, `qemu-user` 8.2.2; a Cortex-M oldalon `arm-none-eabi-gcc` 13.2.rel1 és `qemu-system-arm` 8.2.2.

3.1.2 Fejlesztői gép, célarchitektúra és cross-toolchain

A **fejlesztői gép** (*host*) az a rendszer, amelyen a szerkesztő, a fordító és a többi fejlesztőeszköz fut. A **célarchitektúra** (*target architecture*) az az utasításkészlet és kapcsolódó környezet, amelyhez a kész programot előállítjuk.

Keresztfejlesztésről (*cross-development*) beszélünk, amikor a fejlesztőeszköz más célrendszerhez készít programot, mint amelyen maga fut. Ebben a laborban a fordító x86_64 Linux-program, de AArch64 Linux-programot állít elő.

Szerep	A laborban	Ellenőrzés
Host kernelarchitektúrája	x86_64	`uname -m`
Host Ubuntu-csomagarchitektúrája	`amd64`	`dpkg --print-architecture`
GCC célkonfigurációja	`aarch64-linux-gnu`	`aarch64-linux-gnu-gcc -dumpmachine`

Szerep	A laborban	Ellenőrzés
Elkészült program géptípusa	AArch64	ELF-fejléc vizsgálata
Végrehajtási környezet	QEMU Linux user-mode	`qemu-aarch64`

Az `amd64` és az `x86_64` itt ugyanannak a hostarchitektúrának eltérő elnevezése. Az `arm64` ezzel szemben az AArch64 gyakori disztribúciós neve; nem szabad összekeverni az `amd64` névvel.

Az `aarch64-linux-gnu-` előtag a célkonfigurációt jelöli:

- `aarch64`: célarchitektúra;
- `linux`: cél operációs rendszer;
- `gnu`: GNU-alapú felhasználói környezethez tartozó toolchain-konfiguráció.

Az előtag után következik az eszköz szerepe: `gcc`, `as`, `ld`, `objdump` vagy `readelf`. Az előtag nélküli `gcc`, `as` és `ld` ezen a hoston rendszerint x86_64 célra dolgozik.

Két fontos kivétel van. A `gdb-multiarch` több architektúrát támogató debugger, ezért nem ilyen előtaggal nevezzük meg. A `qemu-aarch64` pedig emulátor, nem fordítóeszköz.

3.1.3 Mit biztosít a QEMU user-mode?

A **felhasználói módú emuláció** (*user-mode emulation*) egy másik architektúrára készült felhasználói program végrehajtását teszi lehetővé. A QEMU feldolgozza az AArch64 utasításokat, és a támogatott Linux-rendszerhívásokat a host környezetéhez közvetíti.

Ebben az üzemmódban:

- nem indul külön vendég Linux-kernel;
- nem kapunk teljes emulált alaplapot vagy perifériarendszert;
- nem tudunk vele Cortex-M bare-metal firmware-t futtatni;
- az AArch64 felhasználói kód működését vizsgálhatjuk, de nem kapunk garantáltan hardverhű időzítést.

A QEMU tehát kiváló funkcionális labor: ellenőrizhető vele például egy számítás eredménye vagy egy program kilépési állapota. Az emuláció alatt mért futási időből viszont nem vezethető le közvetlenül egy fizikai ARM processzor ciklusszáma, utasításkésleltetése vagy áteresztőképessége.

A későbbi Cortex-M labor `qemu-system-arm` programot használ majd. Ez külön futási modell, nem a mostani parancs másik célarchitektúra-neve, és külön eszközlánc (`arm-none-eabi-`) tartozik hozzá. Telepítését és ellenőrzését lent, a II. rész eszközláncáról szóló szakasz írja le.

3.1.4 Telepítés Ubuntu 24.04 LTS hoston

Először a hostot azonosítsuk, ne a telepített fordító nevéből következtessünk rá.

```
# Futtatás: Ubuntu host; tervezett cél: AArch64, qemu-user.
uname -m
dpkg --print-architecture
(
    . /etc/os-release
    printf '%s %s\n' "$ID" "$VERSION_ID"
)
```

A baseline szerinti várt eredmények rendre: `x86_64`, `amd64`, illetve `ubuntu 24.04`. A parancsok sikeres kilépési állapota `0`. Ha más rendszer jelenik meg, a környezet eltér a könyv referenciaprofiljától; ez nem feltétlenül hiba, de dokumentálandó eltérés.

Telepítsük a négy szükséges csomagot:

```
# Futtatás: Ubuntu 24.04 x86_64 host.
# Telepített eszközök célja: AArch64 Linux, qemu-user labor.
sudo apt-get update
sudo apt-get install \
    gcc-aarch64-linux-gnu \
    binutils-aarch64-linux-gnu \
    gdb-multiarch \
    qemu-user
```

Mindkét művelet várt kilépési állapota `0`; az `update` esetleges figyelmeztetéseit is ellenőrizni kell. A telepítés rendszergazdai jogosultságot igényel, a későbbi fordítás és emulált futtatás nem.

Csomag	Feladat
`gcc-aarch64-linux-gnu`	C-fordító és fordításvezérlő AArch64 Linux-célhoz
`binutils-aarch64-linux-gnu`	Assembler, linker, objektum- és ELF-vizsgáló eszközök
`gdb-multiarch`	Több célarchitektúrát kezelő GDB
`qemu-user`	Felhasználói módú emulátorok, köztük `qemu-aarch64`

Az APT a szükséges függőségeket is telepíti. Ha nem talál egy csomagot, előbb a csomaglisták frissítését, a disztribúció kiadását és a hivatalos tárolókomponensek elérhetőségét vizsgáljuk. Egyes csomagokhoz a `universe` komponensre is szükség lehet; a helyi tárolókonfiguráció ellenőrzendő.

A „reprodukálható baseline” nem jelent automatikusan változatlan csomagverziókat. A 24.04 LTS frissítéseket kap, ezért a laborjegyzőkönyvben a tényleges verziókat is rögzíteni kell. Bitazonos környezethez ezen túl rögzített rendszerképre vagy csomagforrás-állapotra lenne szükség.

3.1.4.1 Ha nem Ubuntu 24.04 x86_64-en dolgozol

A könyv parancsai Ubuntu 24.04 x86_64 hostra készültek, de a labor más környezetben is elvégezhető. A cél mindhárom esetben ugyanaz: AArch64 Linux-programok fordítása és futtatása, illetve a II. résznél Cortex-M firmware építése.

- **Windows:** telepíts WSL2-t és benne Ubuntu 24.04-et (`wsl --install -d Ubuntu-24.04`), majd a fejezet parancsait a WSL-terminálban futtasd. A fájlok a Windows és a WSL között a `/mnt/c/...` útvonalon érhetők el, de a laborokat a WSL saját könyvtárában (`~/arm-labor`) érdemes tartani a lassabb fájlrendszer-híd miatt.
- **macOS Intel:** Multipass vagy UTM alatt hozz létre Ubuntu 24.04 VM-et, és abban dolgozz; a parancsok változatlanok.
- **macOS Apple Silicon:** a VM arm64-es, ezért az I. rész AArch64-programjai natívan futnak, `qemu-aarch64` nélkül is. A `qemu-aarch64` ott is használható, de a natív futtatás gyorsabb; a II. részhez továbbra is kell `arm-none-eabi-gcc` és `qemu-system-arm` (a QEMU arm64 hoston is elérhető). A `-march=armv8-a` kapcsoló natív arm64 hoston is érvényes.

- **Konténer:** ha Docker vagy Podman áll rendelkezésre, Ubuntu 24.04 alapú konténerben ugyanazok a telepítési parancsok futnak; a konténerben a `qemu-user` és a `qemu-system-arm` is telepíthető. A konténer nem tartalmazza a host billentyűzetét/terminálját kezelő beállításokat, ezért a GDB-s laborokhoz a `--cap-add=SYS_PTRACE` és a `-it` kapcsoló kell.

Bármelyik utat választod, a 3.1 végén ellenőrizd a verziókat; a könyv `Ubuntu 24.04`-re megadott kimenetei más disztrón vagy macOS-en eltérhetnek, és ezt a laborjegyzőkönyvben jelölni kell.

3.1.4.2 A kísérőcsomag beszerzése és ellenőrzése

A könyv laborforrásai, próbái és segédeszközei egy verziózott kísérőcsomagban is elérhetők; a későbbi fejezetek `tools/…` hivatkozásai erre a csomagra mutatnak. A csomag neve `dsk-arm-assembly-tools-1.0.tar.gz`, letölthető a GitHub v1.0 kiadásból, illetve a könyv HTML-kiadása mellett a `tar.gz` és a `.sha256` fájlként. Ellenőrzés és kicsomagolás:

```
sha256sum -c dsk-arm-assembly-tools-1.0.tar.gz.sha256
tar -xzf dsk-arm-assembly-tools-1.0.tar.gz
cd dsk-arm-assembly-tools-1.0
```

A csomag tartalmát, a `tests/run.sh substitute` és `tests/run.sh gnu-qemu` futtatását, valamint a naplók helyét az A-profile és M-profile különbségei alfejezet Források-szakasza írja le részletesen. A laborokhoz nem kell a csomagot kicsomagolni: a fejezetekben szereplő források közvetlenül is használhatók; a csomag a teljes, ellenőrzött változatot adja.

3.1.5 Elérhetőség és verziók ellenőrzése

Laborkönyvtárak rendje. A könyv laborfájljainak nevei szándékosan rövidek, és ismétlődnek: `lab-main.c`, `lab-Makefile`, `lab-exit.s` vagy `lab-bits` több alfejezetben is előfordul, eltérő tartalommal. Közös munkakönyvtárban ezek felülírnák egymást. Ezért minden alfejezet saját könyvtárban dolgozik, a könyv két része pedig külön ágon:

```
~/arm-labor/
  a64/          I. rész: AArch64 Linux; aarch64-linux-gnu-*, qemu-aarch64
     a000/
     a006/
     ...
     zaro1/     az I. záróprojekt (15.1-15.4) közös könyvtára
  cm/           II. rész: Cortex-M bare-metal; arm-none-eabi-*, qemu-system-arm
     a006/      a Cortex-M eszközlánc smoke tesztje (lásd lent)
     a061/
     ...
     zaro2/     a II. záróprojekt (20.1-20.4) közös könyvtára
```

A szabályok:

- Minden laborpéldát tartalmazó alfejezetben egy **Munkakönyvtár** blokk mondja meg, melyik könyvtárban kell dolgozni. Az alfejezet minden fájlja oda kerül, és minden parancs onnan futtatandó.
- A két záróprojekt alfejezetei egymásra épülnek, ezért projektenként egy közös könyvtárban dolgoznak.
- Ha egy alfejezet egy korábbi alfejezet fájljára épít, a Munkakönyvtár blokk kifejezett `cp` paranccsal másolja át. A korábbi alfejezet könyvtárában nem dolgozunk tovább.
- Az `a64/` és a `cm/` ág nem keveredik: az egyikben `aarch64-linux-gnu-` (Raspberry Pi-n natív) eszközök, a másikban `arm-none-eabi-` eszközök futnak. Mivel a könyvtárak függetlenek, két labor — akár a két profilból egyszerre — párhuzamosan is felépíthető.

Ennek az alfejezetnek az AArch64-próbái a `~/arm-labor/a64/a006/` könyvtárban készülnek:

```
# Futtatás: Ubuntu host; cél: AArch64 qemu-user labor.
mkdir -p ~/arm-labor/a64/a006
cd ~/arm-labor/a64/a006
```

Siker esetén mindkét parancs állapota `0`. A további fájlokat ebben a könyvtárban készítjük.

```
# Futtatás: Ubuntu host; vizsgált cél: AArch64 Linux/qemu-user.
for tool in \
    aarch64-linux-gnu-gcc \
    aarch64-linux-gnu-as \
    aarch64-linux-gnu-ld \
    aarch64-linux-gnu-readelf \
    aarch64-linux-gnu-objdump \
    gdb-multiarch \
    qemu-aarch64
do
    command -v "$tool"
    "$tool" --version
done

aarch64-linux-gnu-gcc -dumpmachine
```

A `command -v` megmutatja, melyik programot választja a shell. Ubuntu-csomagból telepített eszközöknél rendszerint `/usr/bin/` alatti elérési utat várunk. A `--version` kiírásnak minden eszköznél meg kell jelennie; a `-dumpmachine` várt eredménye `aarch64-linux-gnu`.

A ciklus utolsó kilépési állapota önmagában nem igazolja az összes korábbi ellenőrzés sikerét: minden eszköz eredményét nézzük meg.

Rögzítsük a csomagverziókat is:

```
# Futtatás: Ubuntu host; cél: a qemu-user labor környezetének rögzítése.
dpkg-query -W -f='${binary:Package}\t${Version}\t${Architecture}\n' \
    gcc-aarch64-linux-gnu \
    binutils-aarch64-linux-gnu \
    gdb-multiarch \
    qemu-user > lab-packages.txt

cat lab-packages.txt
```

A várt eredmény négy csomagbejegyzés, konkrét verziószámokkal. A GCC-csomag lehet verziózott fordítócsomagra mutató csomag, ezért a csomaglista mellett a fordító saját `--version` kimenete is fontos.

Ha a felsorolt csomagok architektúrája `amd64`, az nem cáfolja az AArch64 célt: maguk az eszközök a hoston futnak.

3.1.6 Kidolgozott példa: a teljes út ellenőrzése

A teszt két részből áll. Először lefordítunk egy C-függvényt objektumfájllá. Ezután egy önálló assembly programmal külön ellenőrizzük az assembler–linker–emulátor láncot. A C-objektumot szándékosan nem linkeljük a második programba: így könnyebb elkülöníteni a hibaforrásokat.

3.1.6.1 C-fordítási próba

Mentsük `lab-compiler.c` néven:

```c
/* C-forrás; cél: AArch64 Linux, Armv8-A, little-endian. */
unsigned lab_probe(void)
{
    return 42u;
}
```

Ez csak fordítási próba, ezért nem kell hozzá `main` és futtatókörnyezet.

3.1.6.2 Önálló assembly futtatási próba

Mentsük `lab-exit.s` néven. A teljes példa **A64 ISA / AArch64, I. rész** kontextusú; a ponttal kezdődő sorok GNU assembler-direktívák, nem processzorutasítások.

```
.section .text                  // GNU as/ELF direktíva, AArch64 cél
.global _start                  // GNU as/ELF direktíva, AArch64 cél
.type _start, %function          // GNU as/ELF direktíva, AArch64 cél

_start:
    movz w0, #42                // A64: w0 = 42, kilépési argumentum
    movz w8, #93                // A64: w8 = 93, Linux exit hívásszáma
    svc  #0                     // A64: Linux rendszerhívás kezdeményezése

.size _start, . - _start         // GNU as/ELF direktíva, AArch64 cél
.section .note.GNU-stack,"",%progbits // GNU as/ELF: nem végrehajtható verem jelzése
```

A kód most tesztadapter; részletes tárgyalása az első assembly programnál és a Linux syscall-konvenciónál következik.

A szükséges értelmezés:

- Az A64 `MOVZ` itt 32 bites, előjel nélküli értékként beállítja a `W0`, illetve `W8` regisztert.
- Az A64 `MOVZ` immediate mezője 16 bites, előjel nélküli konstans. A 32 bites változat eltolása `0` vagy `16` bit lehet; itt az elhagyott eltolás `0`.
- Az A64 `SVC` szintén 16 bites, előjel nélküli immediate mezőt fogad; itt az érték `0`.
- A `93` nem az A64 ISA által kijelölt „kilépési utasítás”, hanem az AArch64 Linux rendszerhívási felületének `exit` hívásszáma.

A program egyetlen szála `42` állapottal kilép. Nem használ C-könyvtárat vagy dinamikus betöltőt.

3.1.6.3 Fordítás, assemblálás és linkelés

```
# Host: Ubuntu 24.04 x86_64.
# Cél: AArch64/A64, Armv8-A, little-endian; futtató: qemu-user.
aarch64-linux-gnu-gcc \
    -std=c11 -Wall -Wextra -O2 -march=armv8-a -mlittle-endian \
    -c lab-compiler.c -o lab-compiler.o &&
aarch64-linux-gnu-as \
    -march=armv8-a -EL -o lab-exit.o lab-exit.s &&
aarch64-linux-gnu-ld \
    -EL -e _start -o lab-exit lab-exit.o &&
aarch64-linux-gnu-readelf -h lab-compiler.o lab-exit &&
aarch64-linux-gnu-readelf -l lab-exit
```

A `&&` miatt egy hibás lépés után nem indul el a következő. A fordítás, assemblálás és linkelés várt állapota `0`.

Az ELF-vizsgálat ellenőrzési feltételei:

Tulajdonság	`lab-compiler.o`	`lab-exit`
Osztály	ELF64	ELF64
Géptípus	AArch64	AArch64
Bájtsorrend	little-endian	little-endian
Fájltípus	`REL`, relokálható objektum	`EXEC`, végrehajtható fájl
Dinamikus betöltő	Nem releváns	Nincs `INTERP` programfejléc

Az ELF64 önmagában nem elegendő: egy x86_64 objektum is lehet ELF64. A géptípust külön ellenőrizzük.

3.1.6.4 Futtatás és debuggerpróba

```
# Target: AArch64 Linux-program, qemu-user futtatással.
lab_status=0
qemu-aarch64 ./lab-exit </dev/null || lab_status=$?
printf 'QEMU kilépési állapota: %s\n' "$lab_status"
test "$lab_status" -eq 42
```

A vendégprogram nem ír szöveget. A shell várt kiírása: `QEMU kilépési állapota: 42`. A záró `test` sikeres állapota `0`.

A `42` itt szándékos teszteredmény, nem váratlan hiba. A feltételes állapotrögzítés olyan shellben is kezelni tudja, amely egyébként megállna egy nem nulla állapotú parancsnál.

Végül ellenőrizzük a debugger célfelismerését:

```
# Debugger: Ubuntu x86_64 hoston; vizsgált fájl: AArch64 qemu-user cél.
gdb-multiarch -nx -batch \
    -ex 'file lab-exit' \
    -ex 'show architecture' \
    -ex 'info files'
```

Várt állapot: `0`; a kimenetben AArch64 architektúra és a betöltött fájl adatai jelennek meg. Ez fájlbetöltési próba, nem emulált végrehajtás. A GDB `run` parancsa önmagában nem alakítja át az x86_64 hostot AArch64 géppé; a QEMU-hoz kapcsolt hibakeresést később építjük fel.

3.1.7 A II. rész eszközlánca: Cortex-M ellenőrzés és smoke teszt

Az eddig telepített `aarch64-linux-gnu-` eszközlánc **nem** a Cortex-M eszközlánca. AArch64 Linux-felhasználói programot készít: A64 utasításokat és ELF64 fájlt, Linux-rendszerhívásokkal és C-könyvtárral. A II. rész Cortex-M3 (Armv7-M) magon dolgozik, Thumb-2 utasításkészlettel, operációs rendszer nélkül. Ehhez külön eszközlánc (`arm-none-eabi-`) és külön emulátor (`qemu-system-arm`) kell. Az I. részhez ez a szakasz nem szükséges, de **legkésőbb a Thumb-2, a Cortex-M futási modell és a minimal runtime fejezet előtt** végre kell hajtani.

	I. rész	II. rész
Célkonfiguráció	`aarch64-linux-gnu`	`arm-none-eabi`
Utasításkészlet	A64	Thumb-2 (T32), Armv7-M

	I. rész	II. rész
ELF-osztály	ELF64	ELF32
Futtatás	`qemu-aarch64` (user-mode)	`qemu-system-arm -machine mps2-an385` (system-mode)
Operációs rendszer	Linux, rendszerhívásokkal	nincs (bare-metal)
Munkakönyvtár-ág	`~/arm-labor/a64/`	`~/arm-labor/cm/`

3.1.7.1 Telepítés és verziók

```
# Futtatás: Ubuntu 24.04 x86_64 host.
# Telepített eszközök célja: Cortex-M bare-metal, qemu-system labor.
sudo apt-get update
sudo apt-get install \
    gcc-arm-none-eabi \
    binutils-arm-none-eabi \
    qemu-system-arm
```

Csomag	Feladat
`gcc-arm-none-eabi`	C-fordító bare-metal 32 bites Arm célhoz, köztük Cortex-M-hez
`binutils-arm-none-eabi`	Assembler, linker, objektum- és ELF-vizsgáló eszközök ugyanehhez a célhoz
`qemu-system-arm`	Teljes rendszeremulátor 32 bites Arm gépekhez, köztük az `mps2-an385` kártyához

A debuggerhez nem kell új csomag: a már telepített `gdb-multiarch` a Cortex-M célt is kezeli.

A Cortex-M próbák saját könyvtárban készülnek, az AArch64-próbáktól elkülönítve:

```
# Futtatás: Ubuntu host; cél: Cortex-M bare-metal labor.
mkdir -p ~/arm-labor/cm/a006
cd ~/arm-labor/cm/a006

for tool in \
    arm-none-eabi-gcc \
    arm-none-eabi-as \
    arm-none-eabi-ld \
    arm-none-eabi-readelf \
    arm-none-eabi-objdump \
    qemu-system-arm
do
    command -v "$tool"
    "$tool" --version
done

arm-none-eabi-gcc -dumpmachine
qemu-system-arm -machine help | grep mps2-an385

dpkg-query -W -f='${binary:Package}\t${Version}\t${Architecture}\n' \
    gcc-arm-none-eabi \
    binutils-arm-none-eabi \
    qemu-system-arm > lab-packages.txt
cat lab-packages.txt
```

Várt eredmény: minden eszköz kiírja a verzióját, a `-dumpmachine` eredménye `arm-none-eabi`, a gépek listájában szerepel az `mps2-an385` sor, a `lab-packages.txt` pedig három csomagbejegyzést tartalmaz.

3.1.7.2 Ellenpróba: az AArch64 eszközlánc nem helyettesíti

Mentsük `lab-cm-probe.c` néven:

```
/* C-forrás; cél: Cortex-M3, Armv7-M, Thumb-2, bare-metal. */
unsigned lab_probe(void)
{
    return 42u;
}
```

Először szándékosan a rossz fordítóval próbálkozunk:

```
# Várt eredmény: HIBA. Az AArch64 fordító nem ismeri a Thumb/M-profile célt.
aarch64-linux-gnu-gcc -mcpu=cortex-m3 -mthumb -c lab-cm-probe.c -o lab-wrong.o
echo "állapot: $?"
```

A várt eredmény nem nulla állapot és legalább az egyik ismeretlen kapcsolóról szóló hibaüzenet; GCC 13.3.0-val mérve: `aarch64-linux-gnu-gcc: error: unrecognized command-line option '-mthumb'`, állapot 1. A pontos szöveg GCC-verziófüggő. Ha ez a parancs sikerülne, az `aarch64-linux-gnu-gcc` név valójában más fordítóra mutatna — ezt `command -v`-vel ellenőrizni kell.

3.1.7.3 Smoke teszt: Cortex-M3 program QEMU alatt

Mentsük `lab-cm-smoke.s` néven. A program egy kételemű vektortáblából (kezdeti veremmutató és Reset-cím) és egy reset-kezelőből áll, amely semihostinggal szöveget ír ki, majd leállítja a QEMU-t. A részleteket a vektortábla és a startup kód alfejezetei tárgyalják; itt csak az eszközláncot ellenőrizzük vele. A Cortex-M assemblyben a `@` a GNU as megjegyzésjele (Arm32), az A64-ben `//`; a különbséget a 16.2 részletezi.

```
/* Cél: Cortex-M3 (Armv7-M), Thumb-2; futtató: qemu-system-arm mps2-an385. */
    .syntax unified
    .cpu cortex-m3
    .thumb

    .section .text
lab_vectors:                    @ vektortábla a 0x00000000 címen
    .word 0x20004000            @ [0] kezdeti MSP: verem teteje az SRAM-ban
    .word reset_handler         @ [1] Reset; a .thumb_func miatt páratlan cím

    .thumb_func
    .global reset_handler
reset_handler:
    movs r0, #0x04              @ semihosting SYS_WRITE0: nullvégű szöveg
    ldr  r1, =lab_msg
    bkpt #0xab                  @ semihosting hívás
    movs r0, #0x18              @ semihosting SYS_EXIT
    ldr  r1, =0x20026           @ ADP_Stopped_ApplicationExit: QEMU 0-val kilép
    bkpt #0xab
1:  b    1b                     @ ide nem jut el a vezérlés

    .section .rodata
lab_msg:
    .asciz "Cortex-M smoke: OK\n"
```

Fordítás, assemblálás, linkelés és ELF-vizsgálat:

```
# Host: Ubuntu 24.04 x86_64.
# Cél: Cortex-M3, Armv7-M, Thumb-2, little-endian; futtató: qemu-system-arm.
arm-none-eabi-gcc \
    -std=c11 -Wall -Wextra -O2 -mcpu=cortex-m3 -mthumb \
    -c lab-cm-probe.c -o lab-cm-probe.o &&
arm-none-eabi-as \
    -mcpu=cortex-m3 -mthumb -o lab-cm-smoke.o lab-cm-smoke.s &&
arm-none-eabi-ld \
    -Ttext=0x00000000 -e reset_handler \
    -o lab-cm-smoke.elf lab-cm-smoke.o &&
arm-none-eabi-readelf -h lab-cm-probe.o lab-cm-smoke.elf
```

Tulajdonság	`lab-cm-probe.o`	`lab-cm-smoke.elf`
Osztály	ELF32	ELF32
Géptípus	ARM	ARM
Bájtsorrend	little-endian	little-endian
Fájltípus	`REL`	`EXEC`
Flags	`Version5 EABI`	`Version5 EABI, soft-float ABI`
Belépési pont	nem releváns	páratlan cím (`0x9`): Thumb-bit = 1

Futtatás:

```
# Target: Cortex-M3 bare-metal, qemu-system-arm, semihosting.
lab_status=0
timeout 10 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
    -nographic -semihosting \
    -kernel lab-cm-smoke.elf </dev/null || lab_status=$?
printf 'QEMU kilépési állapota: %s\n' "$lab_status"
test "$lab_status" -eq 0
```

Várt kiírás: `Cortex-M smoke: OK`, majd `QEMU kilépési állapota: 0`. A `</dev/null` azért kell, mert a `-nographic` a standard bemenetet a QEMU-monitorral multiplexeli; szkriptből vagy több parancs egy blokkban történő beillesztésekor a QEMU elnyelheti a következő parancs bemenetét. Ugyanezért kap a fenti `qemu-aarch64` hívás is `</dev/null`-t. A `timeout` védi a labort a lefagyástól: a `124`-es állapot azt jelenti, hogy a QEMU 10 másodperc alatt sem állt le. Ilyenkor először a `-semihosting` kapcsoló meglétét és a vektortábla elhelyezését kell ellenőrizni (`arm-none-eabi-objdump -s -j .text lab-cm-smoke.elf`: az első két szó `0x20004000` és `0x00000009`).

3.1.7.4 Elfogadás: tiszta környezet, két profil párhuzamosan

A labor akkor kész, ha mindkét profil tiszta állapotból felépül, és a két labor egymás zavarása nélkül, párhuzamosan is fut. Először töröljük a build-termékeket mindkét könyvtárból:

```
# Futtatás: Ubuntu host; mindkét profil build-termékeinek törlése.
rm -f ~/arm-labor/a64/a006/*.o ~/arm-labor/a64/a006/lab-exit
rm -f ~/arm-labor/cm/a006/*.o ~/arm-labor/cm/a006/lab-cm-smoke.elf
```

A két build újrafuttatásához használható a `lab-rebuild-both.sh` szkript (a kísérőcsomagban és a labor könyvtárában), amely az AArch64 és a Cortex-M buildet is lefuttatja.

Ezután a saját könyvtárában ismételjük meg az AArch64 fordítási blokkot (`~/arm-labor/a64/a006`) és a Cortex-M fordítási blokkot (`~/arm-labor/cm/a006`). A két programot

egy szkript indítja egyszerre, és a két kilépési állapotot **egyetlen PASS/FAIL eredménnyé** alakítja. Mentsük `~/arm-labor/lab-smoke-both.sh` néven:

```
#!/bin/bash
# lab-smoke-both.sh - a két profil smoke tesztje párhuzamosan, közös PASS/FAIL eredménnyel.
# Futtatás: Ubuntu host, bash lab-smoke-both.sh; kilépési állapot: 0 = PASS, 1 = FAIL.
# Mindkét QEMU stdin-je /dev/null: a háttérfolyamat nem versenyez a terminálért.
( cd ~/arm-labor/a64/a006 && exec timeout 10 qemu-aarch64 ./lab-exit ) < /dev/null &
pid_a64=$!
( cd ~/arm-labor/cm/a006 &&
  exec timeout 10 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
      -display none -monitor none -serial null -semihosting \
      -kernel lab-cm-smoke.elf ) < /dev/null &
pid_cm=$!

wait "$pid_a64"; st_a64=$?        # külön wait: mindkét állapot megmarad
wait "$pid_cm";  st_cm=$?
printf 'a64 állapot: %s (elvárt: 42)\n' "$st_a64"
printf 'cm állapot:  %s (elvárt: 0)\n' "$st_cm"

if [ "$st_a64" -eq 42 ] && [ "$st_cm" -eq 0 ]; then
    echo "SMOKE: PASS"
    exit 0
fi
echo "SMOKE: FAIL"
exit 1
```

A szkript három ponton tér el egy egyszerű `( … ) & wait` mintától. **(1)** Mindkét háttérfolyamat PID-jét elmenti, és **külön** `wait "$pid"` kérdezi le az állapotukat. A paraméter nélküli `wait` mindig 0-val tér vissza, egy záró `echo` pedig a saját sikeres állapotával elfedné a futtatóét. **(2)** Az `exec` miatt az alhéj állapota maga a `timeout`, illetve a QEMU állapota; a `timeout` 124-et ad lefagyásnál, a hiányzó program 127-et, a sikertelen `cd` nem nullát. **(3)** A semihostingos smoke tesztnek nincs szüksége terminálra. A `-nographic` helyett ezért `-display none -monitor none -serial null` áll, és mindkét QEMU bemenete `/dev/null`, így a háttérbe tett folyamat nem versenyez a terminálbemenetért. A semihosting-kimenet (`Cortex-M smoke: OK`) chardev megadása nélkül a QEMU szabványos hibakimenetére kerül, ezért továbbra is látszik.

```
cd ~/arm-labor
bash lab-smoke-both.sh; echo "kilépési állapot: $?"
```

Várt eredmény (a `Cortex-M smoke: OK` sor helye a két folyamat sorrendjétől függ):

```
Cortex-M smoke: OK
a64 állapot: 42 (elvárt: 42)
cm állapot:  0 (elvárt: 0)
SMOKE: PASS
kilépési állapot: 0
```

Minden más eset FAIL, és a szkript 1-gyel tér vissza: például ha a `lab-exit` nem 42-vel lép ki, ha a QEMU hiányzik (`127`), vagy ha a Cortex-M program nem áll le 10 másodpercen belül (`124`). Ha az egyik profil csak akkor működik, ha a másik nem fut, vagy ha egy fájlt a másik könyvtárból kellene átvenni, a két labor nincs elkülönítve.

3.1.8 Tipikus hibák és ellenpróbák

A hibakeresés sorrendje legyen: **tünet → hipotézis → adatgyűjtés → egy változtatás → új ellenőrzés**.

Tünet	Lehetséges ok	Első ellenpróba
`command not found`	Hiányzó csomag vagy hibás `PATH`	`command -v`, majd csomagvizsgálat
A csomag telepítve van, de a program nem található	A könyvtára nincs a `PATH`-ban	Teljes elérési úttal indítás
A linker inkompatibilis objektumot jelez	Host- és céleszközök keveredtek	Objektumok `Machine` mezője
`Exec format error` közvetlen indításkor	A host nem kezeli közvetlenül az AArch64 fájlt	Explicit `qemu-aarch64`
A QEMU hiányzó betöltőt említ	Dinamikusan linkelt célprogramhoz nincs megfelelő célkörnyezet	`readelf -l`, `INTERP` keresése
`arm-none-eabi-gcc: command not found`	A II. rész eszközlánca nincs telepítve; az `aarch64-linux-gnu-` nem helyettesíti	`dpkg-query -W gcc-arm-none-eabi`
A `qemu-system-arm` nem áll le (`timeout` után `124`)	Hiányzó `-semihosting`, vagy a vektortábla nem a `0x00000000` címen van	`arm-none-eabi-objdump -s -j .text`
Egy korábbi alfejezet példája hirtelen mást csinál	Közös munkakönyvtárban egy azonos nevű fájl felülíródott	`pwd`, a Munkakönyvtár blokk újraolvasása

Például egy assembler-elérési hiba vizsgálata:

```
# Futtatás: Ubuntu host; vizsgált eszköz célja: AArch64.
command -v aarch64-linux-gnu-as
dpkg-query -W binutils-aarch64-linux-gnu
printf '%s\n' "$PATH"
/usr/bin/aarch64-linux-gnu-as --version
```

Ha a csomagvizsgálat és a teljes útvonalas indítás sikeres, de a `command -v` nem találja az eszközt, erős a `PATH`-hiba hipotézise. Egyetlen parancsra korlátozott ellenpróba:

```
# Futtatás: Ubuntu host; ideiglenes PATH, AArch64 assembler ellenőrzése.
PATH="/usr/bin:/bin:$PATH" aarch64-linux-gnu-as --version
```

Várt állapot: `0`. Ez nem módosítja tartósan a shell konfigurációját. A végleges javítás előtt keressük meg, hol veszett el a megfelelő útvonal.

Közvetlen `./lab-exit` indítás egyes gépeken automatikus binárisformátum-kezelés miatt működhet. Ez nem bizonyít natív AArch64 végrehajtást. A reprodukálható labor mindig explicit `qemu-aarch64` parancsot használ.

3.1.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) A `uname -m` eredménye `x86_64`, a GCC `-dumpmachine` eredménye `aarch64-linux-gnu`, a csomagarchitektúra pedig `amd64`. Miért nincs ellentmondás?

2. (implementációs, alapszint — cél: rutinépítés) Módosítsd a `lab-exit.s` programot úgy, hogy `17` állapottal lépjen ki. Assembláld és linkeld újra, majd futtasd QEMU-val. Melyik ellenőrző feltételt kell megváltoztatni?

3. (hibakeresési, középszint — cél: rutinépítés) A `binutils-aarch64-linux-gnu` telepítve van. Az assembler rövid névvel nem indul, teljes elérési úttal igen. Írd le az adatgyűjtést, egyetlen ellenpróbát és a várt következtetést.

4. (verifikációs, középszint — cél: megértés-ellenőrzés) Egy társad szerint az ELF64 fejléc és a sikeres közvetlen futtatás elegendő az AArch64 labor igazolására. Adj két ellenérvet és két jobb ellenőrzést.

3.1.10 Megoldások

1. megoldás — Segítség. Gondold át, hogy az `amd64` és az `aarch64-linux-gnu` két különböző dolgot neveznek meg: melyik a futtatott, és melyik az előállított kód architektúrája?

1. megoldás — Teljes megoldás. A hoston x86_64 programok futnak; ezt az Ubuntu csomagkezelője `amd64` néven jelöli. A cross-GCC ilyen hostprogram, amely AArch64 célkódot készít. A host és a cél eltérő szerepet nevez meg.

2. megoldás — Segítség. Az A64 `MOVZ` 16 bites immediate mezejébe nézd meg, belefér-e a `17`; a shell oldali ellenőrzés egyetlen számértékét kell követnie a kódváltozásnak.

2. megoldás — Teljes megoldás. Az A64 `movz w0, #42` helyére A64 `movz w0, #17` kerül. A `17` belefér a 16 bites immediate mezőbe. Az assembler–linker lépéssort újra kell futtatni; különben a régi binárist vizsgáljuk. A shell feltétele `test "$lab_status" -eq 17` lesz. A Linux-hívásszámot tartalmazó A64 `W8` regiszter beállítása változatlan.

3. megoldás — Segítség. Három különálló adatforrás (`command -v`, csomaginfo, `PATH` kiírása) közül melyik mutatja meg, hogy a program telepítve van, de a shell nem találja — és melyik egyváltozós ellenpróba dönti el ezt anélkül, hogy bármit tartósan megváltoztatnánk?

3. megoldás — Teljes megoldás. A `command -v`, a csomagvizsgálat és a `PATH` kiírása együtt különíti el a hiányzó programot az elérési hibától. Az ideiglenesen kiegészített `PATH`-tal megismételt indítás az egyváltozós ellenpróba. Ha sikerül, nem indokolt első lépésként újratelepíteni a csomagot.

4. megoldás — Segítség. Kérdezd meg, mit mond pontosan az ELF64 osztályjelző a géptípusról, és mi történhet, ha a host automatikus binárisformátum-kezelést végez — ezek külön-külön vezetnek a hiányosságokhoz és a jobb ellenőrzésekhez.

4. megoldás — Teljes megoldás. Az ELF64 csak a fájlosztályt azonosítja, nem az utasításkészletet. A közvetlen futtatást automatikusan közbeiktatott emulátor is lehetővé teheti. Jobb ellenőrzés az ELF `Machine: AArch64` mezője, valamint az explicit QEMU-futtatás előre megadott kilépési állapottal.

3.1.11 Rövid összefoglaló

Az Ubuntu 24.04 LTS hoston telepített cross-toolchain AArch64 Linux-programokat készít, miközben maga x86_64 környezetben fut. A labor ellenőrzése több szintből áll: programelérhetőség, verziólekérdezés, célkonfiguráció, ELF-fejléc, majd ismert eredményű végrehajtás. A QEMU user-mode funkcionális tesztkörnyezet, nem teljes emulált gép és nem hardverhű időmérő. A debugger fájlbetöltési próbája külön igazolja a célarchitektúra felismerését. Hibánál előbb adatot gyűjtünk, majd egyetlen tényezőt változtatunk. A rögzített csomagverziók teszik később összehasonlíthatóvá a laborokat. A II. részhez külön eszközlánc (`arm-none-eabi-`) és külön emulátor (`qemu-system-arm`) tartozik, saját smoke teszttel; a két profil a `~/arm-labor/a64/` és

a `~/arm-labor/cm/` ágon, alfejezetenként külön könyvtárban dolgozik, ezért a laborok nem írják felül egymás fájljait.

- **Host:** a fejlesztőeszközöket futtató rendszer.
- **Célarchitektúra:** az elkészített gépi kód utasításkészletének architektúrája.
- **Cross-toolchain:** a saját futási környezetétől eltérő célrendszerhez programot előállító eszközlánc.

3.1.12 Források és ellenőrzési hiányok

Az alábbiak dokumentációs kiindulópontok; a forrásállapotot az Olvasói útmutató táblája rögzíti.

- GNU assembler kézikönyv: assemblerhasználat, célfüggő kapcsolók és direktívák.
- GCC kézikönyv: Invoking GCC: fordítási kapcsolók és célkonfiguráció.
- GNU GDB kézikönyv: fájlbetöltés és architektúrakezelés.
- QEMU dokumentáció: user-mode és system-mode emuláció.

A `lab-smoke-both.sh` a fő úton, valódi QEMU-val mérve: `SMOKE: PASS`, kilépési állapot 0; a semihosting-kimenet (`Cortex-M smoke: OK`) `-serial null` mellett is megjelenik. A hibás ágak ellenpróbái (lefagyás → `124`, hiányzó `qemu-aarch64` → `127`) a szkript állapotkezelését igazolják.

Ellenőrzési hiányok: a mért értékek a fenti Ubuntu 24.04.5 csomagverziókra érvényesek; más disztrón vagy QEMU-verzión a kimenet eltérhet, ezért a saját laborban a verziókat rögzíteni kell. A `current` és `master` dokumentáció eltérhet az Ubuntu 24.04 csomagolt változataitól. A `readelf`-táblázat és a vektortábla-szavak (`0x20004000`, `0x00000009`, entry `0x9`) a fő úton mért értékek.

3.2 Assemble-link-run munkafolyamat natív és cross környezetben

3.2.1 Miért kell külön összeszerelni és linkelni?

Egy C-függvény módosítása után gyakran egyetlen buildparancsot adunk ki. Assemblyprogramnál érdemes először szétválasztani azokat a lépéseket, amelyeket egy összetettebb buildrendszer később elrejt: a forrás feldolgozását, a futtatható állomány elkészítését és a végrehajtást. Így például egy hibás célarchitektúrát nem keverünk össze egy hiányzó futásidejű könyvtárral.

Ebben az alfejezetben egy olyan programot készítünk, amely nem ír a konzolra, hanem **42-es kilépési kódot** ad vissza. Ez szándékosan egyszerű, könnyen ellenőrizhető szerződés: ha a megfelelő értéket kapjuk, az összeállítás–linkelés–futtatás teljes útvonalát kipróbáltuk.

Előfeltétel a Labor-alapkörnyezet: eszközlánc telepítése és verifikáció szerinti környezet. A build automatizálásához a `make` programnak is elérhetőnek kell lennie.

Architektúraprofil: az alfejezet összes assemblypéldája A64 ISA, AArch64 Linux, Armv8-A alapprofil, little-endian bájtsorrend. A GNU assembler direktívái ebben a célkörnyezetben értendők; maguk a direktívák nem processzorutasítások. Cortex-M-kód itt nem szerepel.

3.2.2 Az assemble–link–run folyamat

Az **összeszerelés** (*assembly*) során az assembler gépi utasításokká és kapcsolódó leíró adatokká alakítja az assemblyforrást. A GNU eszközláncban ezt az `as` végzi.

A **linkelés**, más néven szerkesztés (*linking*) során a linker elhelyezi és összekapcsolja az objektumfájlok szekcióit, feloldja a szimbólumhivatkozásokat, valamint elkészíti a kért kimeneti állományt. Itt a GNU `ld` programot használjuk futtatható ELF előállítására.

Lépés	Bemenet	Kimenet	Felelősség
Összeszerelés	`lab-exit.s`	`lab-exit.o`	Utasításkódolás, szimbólumok és szükség esetén áthelyezési információk
Linkelés	`lab-exit.o`	`lab-exit`	Futtatható ELF, belépési pont, betöltési leírás
Futtatás	`lab-exit`	Kilépési állapot	A program végrehajtása és kapcsolat a Linux környezettel

Az **objektumfájl** (*object file*) tehát nem egyszerűen más kiterjesztésű futtatható program. A jelen munkafolyamatban áthelyezhető ELF-állomány, amelyből még hiányzik a végleges futtatható program elrendezése.

A **natív eszközlánc** (*native toolchain*) a saját futási környezetének célarchitektúrájára készít kódot. A **kereszteszközlánc** (*cross-toolchain*) ettől eltérő célrendszerhez dolgozik. Az Ubuntu x86_64 hoston futó `aarch64-linux-gnu-as` például x86_64 program, de AArch64 objektumfájlt állít elő.

Fontos különbség:

- az `as` és az `ld` **elkészíti** a célprogramot;
- a `qemu-aarch64` **végrehajtja** annak A64 utasításait emulációval;
- natív AArch64 Linuxon a program közvetlenül futtatható.

A QEMU user-mode nem teljes virtuális gépet indít, és nem bootol vendégkernelt. A célprogram felhasználói utasításait emulálja, Linux-rendszerhívásait pedig a host környezetével közvetíti.

3.2.3 Kidolgozott példa: kilépés ismert állapottal

Munkakönyvtár: `~/arm-labor/a64/a007/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a007
cd ~/arm-labor/a64/a007
```

Mentsük az alábbi teljes forrást `lab-exit.s` néven:

```
// A64 ISA; AArch64 Linux; Armv8-A; little-endian.
// A direktívák GNU as direktívák ehhez az ELF-célhoz.

.section .text
.global _start

_start:
    mov     x0, #42
    mov     x8, #93
    svc     #0

.section .note.GNU-stack,"",%progbits
```

A GNU assembler `.section .text` direktívája a kódszekciót választja ki. A `.global _start` a linker számára is láthatóvá teszi a `_start` szimbólumot. A forrásvégi `.note.GNU-stack` bejegyzés a GNU ELF-eszközlánc felé jelzi, hogy ez az objektum nem igényel végrehajtható vermet; nem végrehajtandó utasítás.

A program nem C-függvény, és nincs `main` függvénye. A linkernek a `_start` címkét adjuk meg belépési pontként. Nem térünk vissza egy hívóhoz, hanem közvetlen Linux-rendszerhívással fejezzük be a folyamatot.

A64 utasítás vagy regiszter	Szerep ebben az AArch64 Linux-példában
`MOV X0, #42`	A 64 bites `X0` regiszterbe kerül a kilépési állapot
`MOV X8, #93`	A 64 bites `X8` regiszterbe kerül a Linux `exit` rendszerhívás száma
`SVC #0`	A Linux-rendszerhívási belépés kezdeményezése

A 42 értéket nemnegatív kilépési állapotként, a 93 értéket előjel nélküli rendszerhívás-azonosítóként használjuk. A 93 **Linux AArch64 ABI-konstans**, nem az A64 ISA által meghatározott „kilépési utasítás”.

Az itt használt A64 `MOV` immediate alak alias: mindkét konstans elfér egy előjel nélküli, 16 bites mezőben, ezért egyetlen `MOVZ` utasítással kódolható, nulla eltolással. Ebből nem következik, hogy tetszőleges 64 bites konstans ugyanígy betölthető. A64 `SVC` esetén az immediate mező 16 bites; itt az értéke 0. A rendszerhívás számát nem ez a mező, hanem az A64 `X8` regiszter tartalmazza.

A részletes magyarázatot később a Linux syscall-konvenció AArch64-en adja meg. Most a három utasítás ellenőrizhető tesztprogramként szolgál.

3.2.3.1 Összeszerelés és linkelés cross környezetben

A következő parancsokat a forrás könyvtárában adjuk ki:

```
# Target: qemu-user, AArch64 Linux.
# Host: Ubuntu 24.04 LTS x86_64.

aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-exit.o lab-exit.s

aarch64-linux-gnu-ld -EL -m aarch64linux -static \
    -e _start -o lab-exit lab-exit.o
```

Várt eredmény: mindkét eszköz 0 állapottal fejeződik be, és létrejön a kimeneti fájlja. Hiba után ne folytassuk automatikusan a futtatást: egy korábbi, még létező bináris megtévesztő eredményt adhat.

Kapcsoló	Eszköz	Jelentés
`-EL`	AArch64 `as`, `ld`	Little-endian célkimenet
`-march=armv8-a`	AArch64 `as`	Az elfogadott architektúraprofil kiválasztása
`-o fájl`	`as`, `ld`	Kimeneti fájlnév
`-m aarch64linux`	AArch64 `ld`	A linker célmódjának, úgynevezett emulációjának kiválasztása
`-static`	`ld`	Statikus linkelési mód
`-e _start`	`ld`	A futtatható állomány belépési szimbóluma

A linker „emulációja” itt **nem processzoremuláció**: a kimeneti formátumhoz és elrendezéshez tartozó célmód. Semmi köze a QEMU utasítás-végrehajtásához.

A célarchitektúrát nem önmagában a `-march` kapcsoló teremti meg. A megfelelő célra konfigurált assembler is szükséges. Az x86_64 host közönséges `as` programja nem válik AArch64 assemblerré attól, hogy ezt a kapcsolót kapja.

3.2.3.2 Futtatás és ellenőrzés

```
# Target: qemu-user, AArch64 Linux.
# Interaktív shellben, bekapcsolt errexit (set -e) nélkül.

qemu-aarch64 ./lab-exit
echo $?
# Várt kiírás: 42
```

A shell `$?` változója a **legutóbbi parancs** kilépési állapotát tartalmazza. Emiatt az ellenőrzésnek közvetlenül a futtatást kell követnie.

A program nem ír szöveget: a konzolon megjelenő 42 az `echo` kimenete. A 42 itt sikeres funkcionális tesztet jelent, noha a shell és a buildeszközök általános megállapodása szerint a nulla jelzi a sikert. Automatizáláskor ezt az eltérést kifejezetten kezelni kell.

Normál Linux-folyamatkilépésnél a shellben megfigyelhető állapot a kilépési érték alsó 8 bitjére korlátozódik. A 42 ezért változatlanul ellenőrizhető; egy nagyobb érték nem feltétlenül lenne az.

Ellenőrzési feltétel: a lefordított program normálisan fejeződik be, a közvetlenül utána lekérdezett állapot pedig 42. Az itt közölt eredmény mérve (Ubuntu 24.04, GCC 13.3.0, binutils 2.42, QEMU 8.2.2); a kilépési állapot 42.

3.2.4 Miért hasznos a statikus linkelés QEMU alatt?

Statikus linkeléskor (*static linking*) a szükséges, könyvtárból származó programrészek a futtatható állományba kerülnek. Példánk még egyszerűbb: egyáltalán nem hivatkozik könyvtárra, így nem kell sem C runtime, sem `libc`.

Dinamikus linkeléskor (*dynamic linking*) a program megosztott könyvtárakat is igényelhet. A szokásos dinamikus Linux ELF ehhez megnevez egy **dinamikus betöltőt** (*dynamic loader*), amely futáskor betölti a szükséges célarchitektúrájú könyvtárakat.

A `qemu-aarch64` önmagában nem helyettesíti ezeket. Egy x86_64 host x86_64 könyvtárai nem használhatók AArch64 könyvtárakként.

A statikus példa előnye tehát:

1. nincs szükség AArch64 dinamikus betöltőre;
2. nincs szükség céloldali megosztott könyvtárakra;
3. kevesebb környezeti tényező okozhat futtatási hibát.

Ez nem jelenti azt, hogy QEMU user-mode alatt csak statikus program futhat. Dinamikus programhoz megfelelő céloldali fájlrendszerrészletre, például **sysrootra**, és annak megfelelő QEMU-konfigurációra lehet szükség. A QEMU `-L` kapcsolója ELF-interpreter-prefix megadására szolgál; használható értékét a helyi célkönyvtár-elrendezésből kell megállapítani.

Fontos ellenpróba: **a `-static` elhagyása ebből a könyvtármentes `ld` parancsból nem feltétlenül hoz létre dinamikus programot**. A kimenet tulajdonságát kell ellenőrizni, nem pusztán a kapcsoló jelenlétét.

```
# Vizsgálat hoston; a fájlok targetje AArch64 Linux.
aarch64-linux-gnu-readelf -h lab-exit.o
aarch64-linux-gnu-readelf -h lab-exit
aarch64-linux-gnu-readelf -lW lab-exit
```

Várt megfigyelések:

- az objektum típusa `REL`, a futtatható állományé `EXEC`;
- mindkettő géptípusa AArch64, bájtsorrendje little-endian;
- a futtatható állomány programfejlécei között nincs `INTERP` bejegyzés.

Az ELF részleteivel Az ELF formátum alapjai és a program életciklusa foglalkozik.

3.2.5 Ugyanez natív AArch64 Linuxon

Natív AArch64 Linuxon ugyanaz a forrás és ugyanaz az utasításkészlet használható, de nincs szükség a QEMU közbeiktatására.

```
# Target: natív AArch64 Linux; nem az x86_64 laborhost!
# Feltétel: telepített natív GNU binutils.

as -EL -march=armv8-a -o lab-exit.o lab-exit.s
ld -EL -m aarch64linux -static \
    -e _start -o lab-exit lab-exit.o

./lab-exit
echo $?
# Várt kiírás: 42
```

A parancsok ugyanazokat a laborfájlokat írják felül. A natív próba opcionális: a kötelező eredmény a QEMU-futtatás.

x86_64 hoston a közvetlen `./lab-exit` futtatás jellemzően végrehajtási formátumhibát okoz. Ha mégis működik, lehetséges, hogy a rendszer `binfmt_misc` segítségével automatikusan QEMU-hoz irányítja az idegen architektúrájú binárist. Ettől a végrehajtás nem válik natívvá. A laborban ezért használjuk az explicit `qemu-aarch64` parancsot.

3.2.6 Az első minimális Makefile

A `make` függőségek és időbélyegek alapján dönti el, mely állományokat kell újra előállítani. Mentsük a következőt `lab-Makefile` néven. A receptsorok elején **valódi tabulátor** legyen.

```
# Target: alapértelmezésben qemu-user, AArch64 Linux.
CROSS ?= aarch64-linux-gnu-
RUNNER ?= qemu-aarch64
EXPECTED ?= 42

AS = $(CROSS)as
LD = $(CROSS)ld
ASFLAGS = -EL -march=armv8-a
LDFLAGS = -EL -m aarch64linux -static -e _start

.PHONY: lab-all lab-run lab-clean

lab-all: lab-exit

lab-exit.o: lab-exit.s lab-Makefile
        $(AS) $(ASFLAGS) -o $@ $<

lab-exit: lab-exit.o lab-Makefile
        $(LD) $(LDFLAGS) -o $@ lab-exit.o

lab-run: lab-exit
        @status=0; $(RUNNER) ./lab-exit || status=$$?; \
        printf 'kilépés=%s; várt=%s\n' "$$status" "$(EXPECTED)"; \
        test "$$status" -eq "$(EXPECTED)"

lab-clean:
        rm -f lab-exit.o lab-exit
```

A `$@` az aktuális célt, a `$<` az első előfeltételt jelenti. A receptben `$$` szükséges ahhoz, hogy a shell egyetlen `$` karaktert kapjon. Az `AS` és `LD` változóknál szándékosan `=` szerepel: így felülírjuk a `make` beépített alapértékeit.

A futtatási recept elmenti a célprogram állapotát, majd összehasonlítja az elvárttal. Így a célprogram 42-es eredményéből a build számára 0-val jelzett siker lesz.

```
# Target: qemu-user, AArch64 Linux.
make -f lab-Makefile lab-all
make -f lab-Makefile lab-run
echo $?
# Várt ellenőrző sor: kilépés=42; várt=42
# Várt make-kilépési állapot: 0
```

A natív változat:

```
# Target: natív AArch64 Linux.
make -f lab-Makefile lab-clean
make -f lab-Makefile CROSS= RUNNER= lab-run
# Várt ellenőrző sor: kilépés=42; várt=42
# Várt make-kilépési állapot: 0
```

Az üres `CROSS` a natív `as` és `ld` neveket eredményezi; az üres `RUNNER` közvetlen programindítást ad. Környezetváltás előtt azért töröljük a kimeneteket, mert a `make` önmagában nem érzékeli a parancssori változóértékek megváltozását.

A `lab-clean` kizárólag a név szerint felsorolt két generált laborfájlt törli, a forrást és a Makefile-t nem.

3.2.7 Tipikus hibák és ellenpróbák

A hibakeresés sorrendje: **tünet → hipotézis → adatgyűjtés → egy változtatás → ellenőrzés**.

Tünet	Hipotézis	Adatgyűjtés	Egyetlen célzott változtatás
Az assembler nem ismeri az utasításokat	A host assemblere fut	Ellenőrizzük a kiadott parancsot és az eszköz verziójelentését	Használjuk az `aarch64-linux-gnu-as` programot
A linker inkompatibilis géptípust jelez	Rossz architektúrájú objektum került be	`readelf -h` az objektumon	Állítsuk elő újra a helyes assemblerrel
Közvetlen futtatáskor formátumhiba	A host nem AArch64	`uname -m`, valamint ELF-géptípus	Indítsuk explicit QEMU-val
A bináris létezik, de betöltőhiány jelenik meg	Dinamikus célbetöltő hiányzik	`readelf -lW`: van-e `INTERP`?	Ennél a labornál építsük újra a megadott könyvtármentes paranccsal
`make`: „missing separator”	Szóköz került a recept elejére	Láthatóvá tett whitespace a szerkesztőben	Cseréljük a behúzást tabulátorra
Az állapot váratlanul 0	Egy közbeiktatott parancs felülírta `$?` értékét	Nézzük meg a parancssorrendet	Mentsük el az állapotot azonnal

Ha kétséges, hogy a módosított forrás került-e a binárisba, használható az AArch64 disassembler:

```
# Vizsgálat hoston; target: A64 ISA, AArch64 Linux.
aarch64-linux-gnu-objdump -d lab-exit
```

A várt utasítások a két konstansbetöltés és az A64 `SVC`. A disassembler az aliasokat és a számokat eltérő szöveges alakban is megjelenítheti; ez önmagában nem hiba.

3.2.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért nem a `lab-exit.o` fájlt futtatjuk? Miért nincs szüksége a példának AArch64 `libc`-re? A statikus bináris vajon Linux nélkül is futna?

2. (implementációs, alapszint — cél: rutinépítés) Módosítsd a programot úgy, hogy 7 legyen a kilépési állapota. Építsd újra, ellenőrizd közvetlenül, majd futtasd a Makefile tesztjét megfelelő elvárt értékkel.

3. (hibakeresési, középszint — cél: rutinépítés) Az eredeti, 42-vel kilépő program mellett az alábbi sorozat végén 0 jelenik meg. Fogalmazz meg hipotézist, ellenőrizd a binárist, majd javítsd csak az állapotkezelést.

```
# Target: qemu-user, AArch64 Linux.
qemu-aarch64 ./lab-exit
printf 'A futtatás véget ért.\n'
echo $?
```

3.2.9 Megoldások

1. megoldás — Segítség. Gondold át, mi különbözteti meg egy áthelyezhető objektumfájlt egy teljes futtatható ELF-től — van-e a `.o`-nak ismert, végleges belépési címe?

1. megoldás — Teljes megoldás. A `lab-exit.o` áthelyezhető objektum, nem a linker által létrehozott futtatható ELF. A példában nincs külső függvényhivatkozás, ezért `libc` sem kell: közvetlen rendszerhívást használunk. A statikus linkelés nem szünteti meg az operációsrendszer-függést. A program továbbra is a Linux AArch64 rendszerhívási szerződésére épít.

2. megoldás — Segítség. Csak egyetlen immediate értéket kell cserélni; nézd meg, hogy 7 ugyanabba a kódolási korlátba esik-e, mint 42.

2. megoldás — Teljes megoldás. Az A64 `MOV X0, #42` helyére `MOV X0, #7` kerül. A 7 ugyanúgy elfér a használt 16 bites konstansmezőben.

```
# Target: qemu-user, AArch64 Linux.
make -f lab-Makefile lab-all
qemu-aarch64 ./lab-exit
echo $?
# Várt: 7

make -f lab-Makefile EXPECTED=7 lab-run
# Várt: kilépés=7; várt=7
# Várt make-kilépési állapot: 0
```

Az eredeti 42-es elvárással a teszt helyesen hibát jelezne.

3. megoldás — Segítség. A `$?` mindig a *legutóbbi* parancs állapotát tükrözi — mi fut le a `qemu-aarch64 ./lab-exit` és az `echo $?` között?

3. megoldás — Teljes megoldás. A `$?` már a `printf` sikeres végrehajtásához tartozik. Az `objdump -d lab-exit` segítségével először ellenőrizzük az A64 `X0` regiszterbe töltött értéket, majd mentsük el az állapotot közvetlenül a futtatásnál:

```
# Target: qemu-user, AArch64 Linux; eredeti, 42-es változat.
status=0
qemu-aarch64 ./lab-exit || status=$?
printf 'A futtatás véget ért.\n'
printf 'kilépés=%s\n' "$status"
# Várt utolsó sor: kilépés=42
```

A program változatlan marad; csak a megfigyelés hibáját javítottuk.

3.2.10 Rövid összefoglaló

Az assemblyforrásból először célarchitektúrához kötött objektumfájlt, majd linkerrel futtatható ELF-et készítünk. A cross és a natív út ugyanazt az A64 programot valósítja meg; az eszközök futási környezete és a végrehajtás módja különbözik. A könyvtármentes, statikus példa kizárja a céloldali dinamikus betöltő hiányából eredő hibákat, de továbbra is Linuxra épít. A kilépési állapotot azonnal kell ellenőrizni vagy elmenteni. A Makefile rögzíti a függőségeket, a parancsokat és az ellenőrzési feltételt, így az újraépítés megismételhető.

- **Objektumfájl:** összeszerelt kód és a további linkeléshez szükséges leíró adatok.
- **Linkelés:** objektumok összekapcsolása és a kimeneti állomány kialakítása.
- **Kilépési állapot:** a befejeződő folyamat eredményét jelző, a shellből lekérdezhető érték.

3.2.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GNU assembler kézikönyv — parancssori kapcsolók, AArch64-célopciók és direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU linker kézikönyv — statikus linkelés, belépési pont és linker-emulációk. Ugyanaz a binutils 2.47-es kiadás.
- QEMU dokumentáció — user-mode végrehajtás és céloldali betöltési környezet. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát).
- GNU make kézikönyv — függőségek, receptsorok és változók. A link él; a tartalom itt nem lett automatikus lekéréssel ellenőrizve.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a példák itt nem kerültek végrehajtásra. A telepített Ubuntu-csomagok pontos verzióját, a helyi linker támogatott célmódjait és egy esetleges dinamikus sysroot elrendezését a laborban kell ellenőrizni.

3.3 Opcionális fizikai hardver: Raspberry Pi 64-bit módban

3.3.1 Miért érdemes fizikai géppel is kiegészíteni a labort?

Egy assemblyben megírt függvény QEMU alatt helyesen számol. Következik ebből, hogy Raspberry Pi-n is gyors lesz? Nem: az utasítások jelentése és végrehajtásuk költsége két külön kérdés. A fizikai gép akkor ad többletinformációt, amikor már nemcsak a helyességet, hanem a cache, az elágazásbecslés, az órajel vagy az operációs rendszer ütemezésének hatását vizsgáljuk.

Natív végrehajtásnak (native execution) nevezzük, amikor a program gépi utasításait közvetlenül a célarchitektúrát megvalósító processzor hajtja végre, nem pedig egy másik program emulálja őket. Ebben az alfejezetben ez A64 utasításokat végrehajtó Raspberry Pi 3, 4, 5 vagy Zero 2 W gépet jelent, 64 bites Linux-felhasználói környezettel.

A fizikai gép **opcionális kiegészítés**. A kötelező út továbbra is az Ubuntu 24.04 x86_64 hoston futó `qemu-aarch64`. Az Assemble-link-run munkafolyamat natív és cross környezetben lépéseire építünk; bare-metal indítással, GPIO-val és perifériacímekkel itt nem foglalkozunk.

3.3.2 A 64 bites környezet előkészítése

A processzor 64 bites képessége önmagában nem elegendő. A kernel, a felhasználói környezet és az elkészített végrehajtható állomány architektúráját külön kell ellenőrizni.

Az opcionális telepítés javasolt menete:

1. **Külön labor-adathordozót használj.** A rendszerkép kiírása törölheti a kiválasztott meghajtó tartalmát. Ellenőrizd az eszköz azonosítását és a szükséges mentést.
2. A Raspberry Pi hivatalos telepítőeszközével válassz az adott Pi 4/5 modellhez támogatott **Raspberry Pi OS, 64-bit** rendszerképet. Konzolos használathoz grafikus felület nem szükséges.
3. Állíts be egyedi gépnevet, például `lab-rpi`, valamint egyedi adminisztrációs fiókot, például `lab-admin`. Ne támaszkodj történeti alapértelmezett felhasználónévre vagy jelszóra.
4. Használj megfelelő tápegységet és hűtést. Teljesítménymérésnél az elégtelen tápellátás és a túlmelegedés miatti órajel-visszavétel különösen zavaró tényező.
5. Első indítás után végezd el a szükséges rendszerfrissítést. Rögzítsd a rendszerkép kiadását és a csomagverziókat: egy későbbi frissítés már más mérési környezetet jelenthet.

A telepítő menüpontjai és az egyes rendszerképek aktuális modelltámogatása változhat. Ezeket a tényleges telepítéskor a hivatalos dokumentáció alapján kell ellenőrizni; itt nincs aktuálisan ellenőrzött kiadásmegjelölés.

Célkörnyezet: natív Raspberry Pi, rendszerazonosítás.

```
uname -m
getconf LONG_BIT
dpkg --print-architecture
cat /etc/os-release
```

A kívánt, **várt** első három eredmény rendre `aarch64`, `64`, illetve `arm64`. Ezek külön nézetek:

Ellenőrzés	Mit jellemez?
`uname -m`	A futó kernel által jelentett géparchitektúrát
`getconf LONG_BIT`	A parancs futási környezetében a C `long` típus bitszélességét
`dpkg --print-architecture`	A csomagrendszer alapértelmezett architektúráját

A 64 bites kernel mellett létezhet 32 bites felhasználói környezet is. Ezért a `uname -m` önmagában nem igazolja a könyv natív laborprofilját. Eltérés esetén előbb a környezetet tisztázd, ne az assemblyforrást módosítsd.

3.3.3 SSH és elkülönített jogosultságok

A biztonságos távoli parancssor, a **Secure Shell (SSH)** kényelmessé teszi a fizikai gép használatát, de hálózati hozzáférési pontot is létrehoz.

Két szerepet válassz szét:

- A `lab-admin` fiók kizárólag telepítéshez és karbantartáshoz kapjon `sudo`-jogot.
- A `lab-user` fiók futtassa a laborprogramokat, adminisztrációs és közvetlen eszközhozzáférési jogosultság nélkül.

A `lab-user` ne kerüljön például `sudo`, `gpio` vagy `docker` csoportba. Az utóbbi tagság különösen nem tekinthető ártalmatlan kényelmi beállításnak.

Az adminisztrátor létrehozza a laborfiókot, majd telepíti annak nyilvános SSH-kulcsát. A privát kulcs marad a fejlesztői hoston. A célfiók `.ssh` könyvtárának tulajdonosa a célfiók legyen, jogosultsága 700; az `authorized_keys` fájl 600. A leggyakoribb út a hoston:

```
# Fejlesztői host: kulcs generálása, ha még nincs.
ssh-keygen -t ed25519 -C "lab-user@lab-rpi"

# A nyilvános kulcs átvitele a lab-user fiókba (egyszeri jelszavas belépés).
ssh-copy-id lab-user@lab-rpi.local
```

Ha az `ssh-copy-id` nem elérhető, a kézi változat a célgépen:

```
# Raspberry Pi, lab-user fiókban:
install -d -m 700 ~/.ssh
# A hoston: ssh lab-user@lab-rpi.local 'cat >> ~/.ssh/authorized_keys' < ~/.ssh/id_ed25519.pub
chmod 600 ~/.ssh/authorized_keys
```

Kulcsos belépés ellenőrzése után a jelszavas SSH-belépés kikapcsolható. A beállítást külön drop-in fájlba tedd, hogy a csomagfrissítés ne írja felül:

```
# Raspberry Pi, lab-admin fiókban:
echo 'PasswordAuthentication no' | sudo tee /etc/ssh/sshd_config.d/99-lab.conf
sudo systemctl reload ssh
```

A helyi konzolos helyreállítási lehetőséget őrizd meg, és a kikapcsolás előtt egy másik terminálban ellenőrizd, hogy a kulcsos belépés valóban működik.

Célkörnyezet: natív Raspberry Pi, egyszeri adminisztráció.

```
sudo adduser lab-user
sudo apt update
sudo apt install binutils file
```

Siker esetén a parancsok kilépési állapota `0`. A csomagtelepítéshez kell emelt jogosultság; a későbbi fordításhoz és futtatáshoz nem.

A gépazonosító kulcs ujjlenyomatát első kapcsolódáskor megbízható csatornán, például a Pi helyi konzolján ellenőrizd.

Célkörnyezet: natív Raspberry Pi, helyi konzol.

```
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

Ez a parancs a meglévő nyilvános hostkulcs ujjlenyomatát mutatja; nem generál új kulcsot.

Kapcsolódás: Ubuntu fejlesztői host → natív Raspberry Pi.

```
ssh lab-user@lab-rpi.local
```

A `lab-rpi.local` csak akkor használható, ha a helyi hálózaton működik az ehhez szükséges névfeloldás. Máskülönben a saját, ellenőrzött gépnevedet használd. Ne kapcsold ki automatikusan a hostkulcs-ellenőrzést, és a labor kedvéért ne állíts be internet felőli porttovábbítást.

3.3.4 Mit jelent ugyanaz a program QEMU-ban és a Pi-n?

A következő összevetés az **A64 ISA, Armv8-A alapprofil, little-endian Linux** környezetére vonatkozik.

Szempont	`qemu-aarch64`, user-mode	Natív Raspberry Pi
A64 utasítások jelentése	Az emulátor megvalósítása és CPU-beállítása szerint	A fizikai processzor megvalósítása szerint
Linux-rendszerhívások	A QEMU a host operációs rendszer szolgáltatásaihoz közvetít	Közvetlenül a Pi Linux kerneléhez kerülnek
Cache és elágazásbecslés	Nem a Pi mikroarchitektúrájának időzítéshű modellje	A tényleges processzor viselkedése érvényesül
Futási idő	Az emuláció és az x86_64 host terhelése is alakítja	A Pi terhelése, órajele, hőállapota is alakítja
Opcionális ISA-bővítések	A választott emulált CPU-tól függnek	A konkrét processzortól függnek

A **funkcionális helyesség (functional correctness)** azt jelenti, hogy a megadott bemenetre a szerződés szerinti eredmény és megengedett mellékhatás keletkezik. Ez nem azonos a teljesítménnyel.

A könyv alapprofiljába tartozó egész aritmetikai utasítások eredményének egyeznie kell. Ebből nem következik azonos futási idő. A QEMU user-mode ráadásul nem teljes Raspberry Pi gépet emulál: nem ad Pi-specifikus perifériakörnyezetet, és nem indít külön vendégkernelt.

3.3.5 Kidolgozott példa: ugyanaz az ELF két végrehajtási környezetben

A mérnöki feladat most egyszerű: igazoljuk, hogy ugyanaz a könyvtárfüggetlen AArch64 ELF mindkét környezetben ugyanazzal az eredménnyel fejeződik be.

A program `40 + 2` értékét kilépési állapotként adja vissza. A `42` tehát szándékos eredmény, nem önmagában hibajelzés. Nincs fájlművelet, hálózati hozzáférés, adatmemória-írás vagy ciklus.

Munkakönyvtár: `~/arm-labor/a64/a008/` — a buildkörnyezet Ubuntu x86_64 cross-toolchain, az első futtatási cél `qemu-user`.

```
mkdir -p ~/arm-labor/a64/a008
cd ~/arm-labor/a64/a008
```

Mentsd a következő forrást `lab-pi.s` néven.

A64 ISA, Armv8-A; GNU assembler-direktívák AArch64 ELF-célhoz.

```
    .text
    .p2align 2
    .global _start
    .type _start, %function

_start:
    movz    w0, #40
    add     w0, w0, #2
    movz    x8, #93
    svc     #0

    .size _start, . - _start
    .section .note.GNU-stack,"",%progbits
```

Az A64 `W0` a megfelelő általános célú regiszter 32 bites nézete. A beleírás az A64 `X0` felső 32 bitjét nullázza. Az összeadás eredménye itt előjel nélküli olvasatban `42`.

Az AArch64 Linux rendszerhívási megállapodása szerint az A64 `X8` tartalmazza a hívásszámot; a `93` az `exit`. Első argumentuma az A64 `X0`, itt a kilépési állapot. Ez Linux-szabály, nem az A64 ISA általános tulajdonsága; részletesen a Linux syscall-konvenció AArch64-en tárgyalja.

Az alkalmazott konstansok kódolhatók:

A64 utasításforma	Immediate-korlát
`MOVZ W0, #40`	16 bites mező; a 32 bites változatban az eltolás `0` vagy `16`
`ADD W0, W0, #2`	12 bites előjel nélküli mező, opcionális `12` bites balra tolással
`MOVZ X8, #93`	16 bites mező; a 64 bites változatban az eltolás `0`, `16`, `32` vagy `48`
`SVC #0`	16 bites immediate; itt nem ez hordozza a Linux-hívásszámot

Az elhagyott eltolás mindkét `MOVZ` és az `ADD` esetén `0`.

Build és futtatás: Ubuntu cross-toolchain → qemu-user, little-endian cél.

```
aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-pi.o lab-pi.s

aarch64-linux-gnu-ld -EL -static -e _start -z noexecstack
# A -m aarch64linux a cross-ld alapértelmezése; a -z noexecstack redundáns biztosíték a
↪  .note.GNU-stack mellett. \
    -o lab-pi-a64 lab-pi.o

aarch64-linux-gnu-readelf -h lab-pi-a64

qemu-aarch64 ./lab-pi-a64
lab_status=$?
printf 'qemu-user: exit=%s\n' "$lab_status"
test "$lab_status" -eq 42
```

A `readelf` fejlécében **várt** az `ELF64`, az `AArch64` géptípus és a little-endian adatjelölés. A futtatás után **várt** sor: `qemu-user: exit=42`. A végső `test` sikeres ellenőrzéskor `0` állapottal tér vissza.

A parancssort ne olyan `set -e` beállítású scriptből futtasd, amely a szándékos `42` miatt az ellenőrzés előtt megszakad.

Most ugyanazt az ELF-et és a forrást másoljuk át.

Átvitel: Ubuntu host → natív Raspberry Pi; munkakönyvtár mindkét gépen: `~/arm-labor/a64/a008`.

```
ssh lab-user@lab-rpi.local 'mkdir -p ~/arm-labor/a64/a008'
scp lab-pi.s lab-pi-a64 lab-user@lab-rpi.local:arm-labor/a64/a008/
```

A sikeres átvitel parancsainak várt kilépési állapota `0`.

Futtatás: natív Raspberry Pi, SSH-munkamenetben, `lab-user` fiókkal.

```
cd ~/arm-labor/a64/a008
chmod u+x lab-pi-a64

file lab-pi-a64
readelf -h lab-pi-a64

./lab-pi-a64
lab_status=$?
printf 'native: exit=%s\n' "$lab_status"
test "$lab_status" -eq 42
```

A várt eredmény `native: exit=42`, majd sikeres, `0` állapotú ellenőrzés. A célszámítógépen nincs szükség dinamikus betöltőre vagy C-könyvtárra ehhez a statikusan linkelt példához.

Opcionálisan natívan is újraépítheted.

Build és futtatás: natív Raspberry Pi, A64 little-endian cél.

```
as -EL -march=armv8-a -o lab-native.o lab-pi.s
ld -EL -static -e _start -z noexecstack \
    -o lab-native lab-native.o

./lab-native
lab_status=$?
printf 'native rebuild: exit=%s\n' "$lab_status"
test "$lab_status" -eq 42
```

Itt is `42` a program várt állapota. A két build fájljainak bájtonkénti azonosságát nem követeljük meg: eltérő toolchain-kiadások eltérő ELF-elrendezést készíthetnek.

A QEMU-s eredmények mérve (Ubuntu 24.04, GCC 13.3.0, QEMU 8.2.2); a natív Pi-futtatás nem mért, hardver hiányában.

3.3.6 Mikor ad valóban többet a fizikai hardver?

A fenti négyutasításos program nem megfelelő teljesítménymérésre. Teljes futási idejében a folyamatindítás és a rendszerhívás költsége is megjelenik; QEMU-ban ehhez emulációs többlet társul.

Fizikai gépen később releváns lehet:

- egy hosszabb számítás tényleges átbocsátóképessége;
- a cache-ben elférő és az azon túlnövő adathalmazok különbsége;
- az adatfüggőségek vagy elágazások hatása;
- skaláris és vektorizált megoldások összevetése.

Elméleti levezetés: állandó f magórajel mellett egy ciklus ideje $T = 1/f$, ahol f egysége Hz, T egysége másodperc. Ebből még nem következik egy utasítás végrehajtási ideje: az utasításkésleltetés, az átlapolt végrehajtás és a memóriavárakozás külön kérdés.

A tényleges frekvencia változhat. Egy architekturális időszámláló frekvenciája pedig nem feltétlenül azonos a magórajelével, ezért annak változását nem szabad automatikusan processzorciklusnak nevezni. A hardveres teljesítményszámlálók hozzáférését az operációs rendszer is korlátozhatja.

Méréskor rögzítsd a Pi modelljét, az OS- és kernelverziót, az eszközláncot, a hűtési és tápellátási körülményeket, valamint a háttérterhelést. A részletes módszert az Egyszerű instrumentáció: utasításszám és időmérés építi fel. QEMU-időt ne tüntess fel Pi-ciklusszámként.

3.3.7 Biztonsági döntés és hibakeresési ellenpróbák

A QEMU azért elsődleges, mert hardver nélkül reprodukálható, és kisebb a veszélye a fizikai eszközök téves kezelésének. **A QEMU user-mode azonban nem biztonsági homokozó (sandbox):** a vendégprogram rendszerhívásokon keresztül a futtató felhasználó fájljait és hálózati lehetőségeit is elérheti.

Gyakorlat	Fizikai hoston futtatható?
Átnézett, véges aritmetikai program, csak kilépéssel	Igen, normál felhasználóként
Saját tömb feldolgozása ismert méretkorláttal	Igen, helyességi ellenőrzés után
Ismeretlen eredetű bináris	Nem a szokásos laborhoston
Korlátlan allokáció, folyamatindítás vagy lemezírás	Nem változtatás nélkül
Nyers eszközhozzáférés, `/dev/mem`, GPIO, rootként futás	Nem ennek az alfejezetnek a keretében
Kontrollált biztonsági hibademonstráció	Külön izolált környezetet és eljárást igényel

Az assembly nyelv önmagában nem tesz egy programot veszélyessé vagy biztonságossá. Az elérhető jogosultságok, mellékhatások és erőforráskorlátok számítanak.

Hibakereséskor egyetlen hipotézist vizsgálj egyszerre:

- **`Exec format error`:** ellenőrizd a kernelarchitektúrát és az ELF fejlécét. Lehet, hogy rossz gépen vagy rossz architektúrájú fájlt indítottál.
- **`Permission denied`:** nézd meg a végrehajthatóságot és a fájlrendszer csatolási korlátozásait. Ne `sudo`-val kerüld meg a tünetet.
- **`Illegal instruction`:** hasonlítsd össze a fordítás célprofilját a CPU képességeivel. A példának nincs szüksége Armv8-A utáni bővítésre.
- **Váratlan kilépési állapot:** közvetlenül a program után mentsd a `$?` értékét. Egy közbeiktatott sikeres parancs már felülírja.
- **Eltérő SSH-hostkulcs:** újratelepítés is okozhatja, de előbb ellenőrizd helyi konzolon; ne töröld vakon a korábbi bejegyzést.

3.3.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért nem elegendő a `uname -m` eredménye a teljes 64 bites laborprofil igazolására? Mit nem igazol a QEMU-ban kapott helyes eredmény?

2. (implementációs, alapszint — cél: rutinépítés) Módosítsd a programot úgy, hogy előjel nélküli `70 + 5` eredményével lépjen ki. Futtasd QEMU-ban, majd opcionálisan a Pi-n. Add meg az A64 `W0` és `X0` várt tartalmát közvetlenül az `SVC` előtt.

3. (hibakeresési, középszint — cél: rutinépítés) A program után egy `echo kész` parancs fut, majd a hallgató `$?` értékeként `0`-t lát. Következtethet-e hibás összeadásra? Javítsd az ellenőrzés sorrendjét.

4. (biztonsági/mérési, középszint — cél: megértés-ellenőrzés) Egy ismeretlen program rootjogot kér „pontos ciklusméréshez”. Másik program saját, korlátozott méretű tömbön számol normál felhasználóként. Melyik vihető át a Pi-re, és milyen további ellenőrzés szükséges?

3.3.9 Megoldások

1. megoldás — Segítség. Nézd meg a fejezet táblázatát: a `uname -m`, a `getconf LONG_BIT` és a `dpkg --print-architecture` három KÜLÖNBÖZŐ réteget jellemez — létezhet olyan kombináció, ahol az egyik 64 bitet mutat, a másik mégsem?

1. megoldás — Teljes megoldás. A `uname -m` a kernel géparchitektúráját jelzi; mellette 32 bites felhasználói környezet is létezhet. A `getconf` és a csomagarchitektúra ezért további adat. A QEMU-eredmény a vizsgált funkcionális viselkedést támasztja alá, nem a Pi teljesítményét vagy valamennyi bővítés elérhetőségét.

2. megoldás — Segítség. `70+5=75`; nézd meg a kódolási táblázatot — belefér-e mindkét konstans a `MOVZ`/`ADD` immediate-mezőjébe eltolás nélkül?

2. megoldás — Teljes megoldás. Az A64 `MOVZ W0` immediate értéke legyen `70`, az A64 `ADD W0, W0` immediate értéke `5`. Mindkettő belefér a bemutatott kódolásba. Az eredmény `W0 = 0x0000004b`, illetve `X0 = 0x000000000000004b`; a felső bitek nullázását a 32 bites regiszterírás biztosítja. A várt kilépési állapot `75`, ezért a shell ellenőrzését is erre kell módosítani.

3. megoldás — Segítség. Melyik parancs fut el UTOLJÁRA közvetlenül a `$?` kiértékelése előtt — a laborprogram, vagy az `echo`?

3. megoldás — Teljes megoldás. Nem: a `$?` ekkor az `echo` állapotát jelenti. A helyes sorrend a mintában látható: programfuttatás, azonnali `lab_status=$?`, majd kiírás és összehasonlítás. Ez mérési hiba az ellenőrző környezetben, nem bizonyíték ISA-hibára.

4. megoldás — Segítség. Vesd össze mindkét programot a fejezet „Gyakorlat / Fizikai hoston futtatható?” táblázatával — melyik sorba illik egyik, melyikbe a másik?

4. megoldás — Teljes megoldás. Az ismeretlen programot nem futtatjuk rootként. A számlálóhozzáférés igénye nem indokol ellenőrizetlen jogosultságnövelést. A második program átnézés, méretkorlát-ellenőrzés és QEMU-helyességi próba után alkalmas lehet natív futásra. Időeredményét ettől még megfelelő ismétléssel és környezetleírással kell értelmezni.

3.3.10 Rövid összefoglaló

A Raspberry Pi 3/4/5 vagy Zero 2 W (64 bites OS-sel) opcionális natív ellenőrzési és később mérési környezet, nem a kötelező QEMU-labor helyettesítője. A processzor képessége mellett a kernelt, a felhasználói környezetet és az ELF architektúráját is ellenőrizni kell. Ugyanaz a statikus A64 program mindkét környezetben azonos eredményt adhat, miközben végrehajtási ideje teljesen eltér. A fizikai gép értéke a tényleges mikroarchitekturális viselkedés megfigyelése. A labor normál felhasználóként fusson; a QEMU user-mode használata sem szünteti meg a fájl- és hálózati mellékhatások kockázatát.

- **Natív végrehajtás:** a célutasításokat közvetlenül a fizikai processzor hajtja végre.
- **Funkcionális helyesség:** az eredmény és a mellékhatások megfelelnek a program szerződésének.

3.3.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Raspberry Pi dokumentáció — rendszertelepítés, távoli hozzáférés és modellfüggő üzemeltetés. A link feltehetően él, de tartalmilag itt nem lett ellenőrizve (nem tartozik a könyv visszatérő hivatkozási forráscsaládjai közé).
- QEMU dokumentáció — user-mode és system-mode szerepe. **Tartalmilag ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát).
- GNU assembler kézikönyv — AArch64-célopciók, utasításformák és direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU linker kézikönyv — belépési pont, statikus linkelés és veremjelölések. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a Raspberry Pi OS aktuális modelltámogatása, a telepítő kezelőfelülete és a csomagverziók helyben ellenőrizendők. Sem natív, sem QEMU-futtatás, sem teljesítménymérés nem történt; a megadott eredmények várt, levezetett ellenőrzési feltételek.

3.4 Szerkesztő és debug-eszközök: GDB, objdump, readelf első áttekintése

3.4.1 Miért kezdjük rögtön megfigyeléssel?

Egy assemblyprogram hibája gyakran nem a forrás olvasásakor válik nyilvánvalóvá. Lehet, hogy megfelelő utasításokat írtunk, de rossz architektúrára építettük a programot; lehet, hogy helyes a bináris, de másik fájlt indítottunk el; és lehet, hogy csak a végrehajtás közbeni regiszterérték hibás.

Az Assemble-link-run munkafolyamat natív és cross környezetben lépéseit ezért most megfigyelési pontokkal egészítjük ki. Három eszköz három különböző kérdésre válaszol:

Eszköz	Kérdés	Elindítja a vizsgált programot?
`readelf`	Milyen ELF-fájl készült, milyen szekciókkal és szimbólumokkal?	Nem
`objdump`	Milyen gépi utasítások találhatók benne?	Nem
`gdb-multiarch`	Hol tart a végrehajtás, és mi van a regiszterekben?	A jelen laborban a QEMU-ban futó programot vezérli

A **visszafejtés** (*disassembly*) a gépi utasítások assembly alakú megjelenítése. Nem állítja vissza az eredeti megjegyzéseket vagy minden forrásszintű nevet. A **hibakereső** (*debugger*) ezzel szemben megállíthatja és léptetheti a programot, majd megmutathatja annak pillanatnyi állapotát.

A teljes alfejezet célprofilja **A64 ISA, AArch64, Armv8-A, little-endian**. Az assemblyblokk összes utasítása és regisztere ebben a kontextusban értendő; direktívái GNU assembler-direktívák ugyanahhoz a célhoz. Cortex-M kód itt nem szerepel.

3.4.2 Szerkesztő és laborfájlok

Bármely egyszerű szövegszerkesztő használható: például `nano`, `vim`, Emacs vagy grafikus kódszerkesztő. A fontos tulajdonságok:

- egyszerű szöveges fájl, lehetőleg UTF-8 kódolással, BOM nélkül;
- Linuxos sorvégek;
- látható sorszámok;
- mentés a build előtt;
- a szerkesztőben kijelölt AArch64 assembly szintaxiskiemelés.

A kiemelés csak kényelmi funkció: nem bizonyítja, hogy egy utasítás vagy konstans kódolható. Erről az assembler dönt.

A laborban ugyanabból a munkakönyvtárból dolgozzunk, két terminállal. A fájlok neve:

- `lab-tools.s`: forrás;
- `lab-tools.o`: objektumfájl;
- `lab-tools`: futtatható ELF.

A hibakeresés alapvető előfeltétele, hogy **a QEMU és a GDB ugyanazt a binárist használja**. Újraépítés után indítsuk újra a futtatási és hibakeresési munkamenetet.

3.4.3 Kidolgozott példa: egy megfigyelhető összeadás

Munkakönyvtár: `~/arm-labor/a64/a009/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a009
cd ~/arm-labor/a64/a009
```

A program két értéket összead, majd az eredményt Linux-kilépési állapotként adja vissza. Nem ír szöveget a terminálra. Mentsük `lab-tools.s` néven:

```
// A64 ISA / AArch64, Armv8-A, little-endian.
// GNU assembler-direktívák, Linux felhasználói program.
.arch armv8-a

.text
.global _start
.type _start, %function

_start:
    mov x0, #0
    mov x1, #7
    add x0, x0, x1
    mov x8, #93
    svc #0

.size _start, . - _start

.section .note.GNU-stack,"",%progbits
```

Most nem a teljes regiszterkészletet tanuljuk meg; azt Az A64 regiszterkészlet áttekintése tárgyalja. A megfigyeléshez ennyi szükséges:

- az A64 `X0` és `X1` itt 64 bites egész értékeket tartalmaz;
- az A64 `ADD X0, X0, X1` az összeget `X0`-ba írja;
- a Linux AArch64 rendszerhívási megállapodása szerint `X8 = 93` az `exit` rendszerhívást választja, amelynek argumentuma `X0`;
- az A64 `SVC #0` átadja a vezérlést a rendszerhívás-kezelésnek.

Az utolsó két pont együtt Linux-interfészt használ: a `93` jelentése nem az A64 ISA önálló tulajdonsága.

Konstanskódolás: a példabeli A64 `MOV` alakok egyetlen `MOVZ` utasítással kódolhatók. Ennek konstansmezője 16 bites, és a 64 bites változatban 0, 16, 32 vagy 48 bittel eltolva helyezhető el; itt minden eltolás 0. Ez nem jelenti azt, hogy a `MOV` bármely 64 bites konstanst egy utasítással betölthet. Az A64 `SVC` immediate-mezője szintén 16 bites; itt értéke 0.

Cél: qemu-user, AArch64 Linux. Összeszerelés és linkelés:

```
aarch64-linux-gnu-as -EL -g -o lab-tools.o lab-tools.s
aarch64-linux-gnu-ld -EL -static -e _start -o lab-tools lab-tools.o
```

A `-EL` little-endian kimenetet kér. Az assembler `-g` kapcsolója hibakeresési információt (*debug information*) készít, amely többek között a gépi kód és a forrássorok összerendelését támogatja. A linker ebben a parancsban nem távolítja el ezt.

Cél: qemu-user. Első futtatás és ellenőrzés:

```
qemu-aarch64 ./lab-tools
lab_status=$?
printf 'Kilépési állapot: %s\n' "$lab_status"
```

Várt, levezetett eredmény: `Kilépési állapot: 7`. Ez szándékosan nem nulla; a program most adatot szemléltet a kilépési állapottal. A parancsokat ne olyan shell-munkamenetben futtassuk, amely `set -e` miatt minden nem nulla állapotnál azonnal kilép.

A példák itt nem kerültek futtatásra; a következő állapotok is vártak, nem mért laboreredmények.

3.4.4 A fájl vizsgálata: `readelf`

Először végrehajtás nélkül ellenőrizzük, mit építettünk.

Célbináris: qemu-user alatt futtatandó AArch64 ELF; hostoldali vizsgálat:

```
aarch64-linux-gnu-readelf -h lab-tools
aarch64-linux-gnu-readelf -SW lab-tools
aarch64-linux-gnu-readelf -sW lab-tools
```

A `-h` az ELF-fejlécet, a `-S` a szekciótáblát, a `-s` a szimbólumtáblákat mutatja. A `-W` széles kimenetet kér, csökkentve a sorok tördelését.

A fejlécben ezeket keressük:

Mező	Várt jelentés
`Class`	`ELF64`: 64 bites ELF-formátum
`Data`	Little-endian bájtsorrend
`Type`	`EXEC`: az itt használt linkelés futtatható állománya
`Machine`	`AArch64`: célarchitektúra, nem a host architektúrája
`Entry point address`	A `_start` belépési címének megfelelő érték

Nem rögzítünk számszerű betöltési címet: azt a linker alakítja ki, és a fájlból olvassuk ki. A hibakeresésben szimbolikus címeket használunk.

A **szekció** (*section*) az objektumfájl vagy ELF tartalmának névvel és tulajdonságokkal leírt egysége. A példában különösen ezek érdekesek:

Szekció	Szerep
`.text`	A64 gépi utasítások
`.symtab`	Szimbólumbejegyzések, például `_start`
`.strtab`	A hozzájuk tartozó nevek karakterláncai
`.debug_*`	Hibakeresési adatok
`.shstrtab`	Szekciónevek karakterláncai

A pontos szekciólista és sorrend eszközláncfüggő. Nem minden szekció kerül a futó program memóriájába. A szekciókat a betöltő által használt szegmensekkel se azonosítsuk; ezt Az ELF formátum alapjai és a program életciklusa részletezi.

A szimbólumtábla `_start` sorában a `Value` a végleges ELF-beli cím, az `Ndx` a hozzátartozó szekcióra utal. A példában a `.type` és `.size` direktívák miatt függvényszimbólumot és 20 bájtos méretet várunk.

Fontos különbség: **a szimbólumtábla nem ugyanaz, mint a forrássoros hibakeresési információ**. Egy név akkor is megmaradhat, ha a forrássorok leírását eltávolítottuk.

3.4.5 A gépi kód első olvasása: `objdump`

Célbináris: AArch64; hostoldali disassembly és szimbólumvizsgálat:

```
aarch64-linux-gnu-objdump -d lab-tools
aarch64-linux-gnu-objdump -t lab-tools
```

A `-d` a kódszekciók disassemblyjét, a `-t` a szimbólumtáblát jeleníti meg. Az architektúra-előtagos eszköz használata elkerüli, hogy a host alapértelmezett `objdump` programjának céltámogatására hagyatkozzunk.

A disassembly egy sorában jellemzően cím, gépi kód, mnemonic és operandusok találhatók. Az utasítások elvárt szerkezete:

`_start`-hoz viszonyított bájteltolás	A64 utasítás	Hatás
0x00	`mov x0, #0`	X0 = 0
0x04	`mov x1, #7`	X1 = 7
0x08	`add x0, x0, x1`	X0 = 7
0x0c	`mov x8, #93`	X8 = 93
0x10	`svc #0`	Linux-rendszerhívás

Levezetés: az A64 utasítások hossza 4 bájt, így öt utasítás $5 \cdot 4 = 20$ bájtot foglal. Ez kódméret, nem futási idő vagy ciklusszám.

Az `objdump` a konstansokat más számrendszerben, illetve bizonyos utasításokat más aliasnéven is megjelenítheti. Például `0x5d` és 93 ugyanaz az érték. Az ellenőrzés a jelentésre, nem a forrás karakterhű visszaadására irányul.

3.4.6 Élő megfigyelés: GDB kapcsolódása a QEMU-hoz

A `gdb-multiarch` több célarchitektúrát ismerő, hoston futó hibakereső. A QEMU **GDB-kiszolgálója** (*GDB stub*) fogadja a parancsait, és a vendégprogram állapotát teszi elérhetővé. A GDB által mutatott regiszterek tehát az emulált AArch64 programhoz tartoznak, nem az x86_64 hosthoz.

A terminál, cél: qemu-user, távoli hibakeresésre váró program:

```
qemu-aarch64 -g 1234 ./lab-tools
lab_status=$?
printf 'Kilépési állapot: %s\n' "$lab_status"
```

A `-g 1234` a megadott TCP-porton vár GDB-kapcsolatra. Ha nem akarsz TCP-portot nyitni, Unix-socket is használható: `qemu-aarch64 -g /tmp/lab.sock ./lab-tools`, majd GDB-ben `target remote /tmp/lab.sock` (QEMU 6 óta). Amíg nem kapcsolódunk és nem folytatjuk a programot, a látszólagos tétlenség helyes működés.

A GDB-kapcsolat nem hitelesített szolgáltatás. A labort elkülönített vagy megfelelően tűzfalazott hoston használjuk, a portot ne tegyük nyilvánosan elérhetővé. A kliensoldali `localhost` önmagában nem bizonyítja, hogy a QEMU csak helyi kapcsolatokra figyel.

B terminál, cél: ugyanaz a qemu-user folyamat:

```
gdb-multiarch -q ./lab-tools \
  -ex 'set architecture aarch64' \
  -ex 'set pagination off' \
  -ex 'target remote localhost:1234' \
  -ex 'break *_start+8' \
  -ex 'continue'
```

A parancsok jelentése:

- A `./lab-tools` a GDB számára is betölti a fájlt és annak szimbólumait.
- A `set architecture aarch64` explicit megadja a célarchitektúrát; az ELF alapján ez általában automatikusan is felismerhető.
- A `target remote` kapcsolódik a QEMU-hoz.
- A `break *_start+8` pontos utasításcímre helyez **töréspontot** (*breakpoint*).
- A `continue` elindítja a végrehajtást a töréspontig.

A `*` itt cím szerinti töréspontot jelent. A 8 bájtos eltolást az előbbi disassemblyből vezettük le: ott kezdődik az A64 `ADD`.

A megállás **az utasítás végrehajtása előtt** történik. Így ekkor az A64 `X0 = 0`, `X1 = 7`; az összeadás még hátravan.

A GDB parancssorába sorban írjuk:

GDB-parancs	Megfigyelés vagy hatás
`x/i $pc`	A következő A64 utasítás megjelenítése
`info registers x0 x1 x8 pc`	A kiválasztott AArch64 regiszterek állapota
`stepi`	Egy gépi utasítás végrehajtása
`info registers x0 x1 pc`	Az összeadás utáni állapot
`stepi`	Az A64 `MOV X8, #93` végrehajtása
`info registers x8`	Várt érték: 93, azaz `0x5d`
`continue`	A rendszerhívás végrehajtása, programbefejezés
`quit`	Kilépés a GDB-ből

Az első `stepi` után `x0 = 7`, a programszámláló pedig `_start+12`; az értékek mérve (GNU as/ld 2.42, qemu-aarch64 8.2.2, GDB 15.1). Az A terminálban a program befejezése után ismét 7-es kilépési állapotot várunk.

A `step` és a `stepi` nem felcserélhető:

- `step`: forrásszintű léptetés, a hibakeresési információ alapján;
- `stepi`: egyetlen gépi utasítás léptetése.

Ebben az egyutasításos forrássorokból álló programban hasonló eredményt adhatnak, de ez nem általános garancia. Assemblyállapot vizsgálatakor kezdetben a `stepi` legyen az alapértelmezett. Az `info registers` argumentumok nélkül több regisztert mutat; kezdőként célszerű a vizsgált adatfolyamra szűkíteni.

3.4.7 Tipikus hibák és ellenpróbák

A hasznos munkamódszer: **tünet → hipotézis → adatgyűjtés → egy változtatás → új ellenőrzés**.

Tünet	Hipotézis és adatgyűjtés	Egy változtatás
A GDB nem kapcsolódik	Fut-e a QEMU `-g 1234` kapcsolóval? Egyezik-e a port?	Indítsuk újra a QEMU-t a megadott paranccsal
`Address already in use`	Más folyamat használja a portot	Válasszunk másik portot mindkét oldalon
Váratlan regiszterérték	Nem a gondolt utasításnál állunk	Ellenőrizzük az `x/i $pc` kimenetét
Nem látszik a forrás	Hiányzik a `-g`, vagy nem érhető el a forrásfájl	Építsük újra `-g` mellett, majd nyissuk újra a binárist
Eltérő kód látszik a GDB-ben	Másik vagy régebbi ELF-et nyitottunk meg	Mindkét terminálban ugyanazt az újraépített fájlt használjuk

Az A64 `X8` értékét a program csak később állítja be. A töréspontnál látott korábbi értékére nem szabad következtetést vagy programlogikát alapozni. Hasonlóan: a 7-es kilépési állapot ebben a laborban nem önmagában hibajel. A GDB a kilépési kódot oktálisan írja ki (`exited with code 07` = 7 decimális); a 2.3-ban láttuk ugyanezt `052` = 42 formában.

3.4.8 Feladatok

1. **Magyarázó (alapszint — cél: megértés-ellenőrzés):** A host x86_64, de a `readelf -h AArch64` géptípust ír ki. Miért helyes ez? Miért vizsgálható a fájl a QEMU elindítása nélkül?
2. **Implementációs (alapszint — cél: rutinépítés):** Módosítsd a második A64 `MOV` konstansát 7-ről 11-re. Építsd újra a programot, majd ugyanazon töréspontnál és egy `stepi` után ellenőrizd `X0` és `X1` értékét. Mi lesz a kilépési állapot?
3. **Hibakeresési (középszint — cél: rutinépítés):** Készíts az eredeti ELF-ből hibakeresési információ nélküli másolatot. Vizsgáld meg, eltűnt-e a `_start` szimbólum és megváltozott-e a gépi kód. Indokold, miért maradhat használható az utasításszintű hibakeresés.

3.4.9 Megoldások

1. megoldás — Segítség. Gondold át, mit *értelmez* a `readelf`, és mit *hajt végre* a QEMU — a kettő nem ugyanaz a művelet.

1. megoldás — Teljes megoldás. Az ELF-fejléc a **célarchitektúrát** írja le. A `readelf` fájlstruktúrát értelmez, nem vendégutasításokat hajt végre. Ezért x86_64 hoston is megvizsgálhat AArch64 ELF-et.

2. megoldás — Segítség. A 16 bites immediate-mező elég tágas 11-hez is; nincs szükség kódolásváltásra, csak a konstans cseréjére és újrafordításra.

2. megoldás — Teljes megoldás. A módosítás `mov x1, #11`; a konstans továbbra is belefér a használt 16 bites mezőbe. Az eredeti assemble-link-run parancsokkal újraépítve a töréspontnál `X0`

`= 0`, `X1 = 11`. Egy `stepi` után mindkettő 11. A várt kilépési állapot 11. Ha továbbra is 7 látszik, elsőként a mentést és az újraépítést ellenőrizzük, ne a QEMU aritmetikáját gyanúsítsuk.

3. megoldás — Segítség. A `--strip-debug` célzottan a `.debug_*` szekciókat távolítja el, nem a szimbólumtáblát vagy a kódot — gondold át, melyik GDB-szolgáltatás épül kifejezetten a `.debug_*` adatra, és melyik nem.

3. megoldás — Teljes megoldás. Cél: qemu-user AArch64; a már összeszerelt és linkelt ELF kontrollált másolatának elkészítése, vizsgálata és futtatása:

```
aarch64-linux-gnu-objcopy --strip-debug lab-tools lab-nodebug
aarch64-linux-gnu-readelf -SW lab-nodebug
aarch64-linux-gnu-readelf -sW lab-nodebug
aarch64-linux-gnu-objdump -d lab-nodebug
qemu-aarch64 ./lab-nodebug
lab_status=$?
printf 'Kilépési állapot: %s\n' "$lab_status"
```

A várt különbség a `.debug_*` információk eltűnése. A globális `_start` szimbólum és a programutasítások megmaradnak; az eredeti változat kilépési állapota továbbra is 7. A `--strip-debug` nem azonos minden szimbólum eltávolításával. A GDB továbbra is használhat utasításcímeket, regisztereket és `stepi` léptetést, de a forrássoros szolgáltatások korlátozottak lesznek.

3.4.10 Rövid összefoglaló

A hibakeresés már az első assemblylaborban összekapcsolja a forrást, a binárist és a futás közbeni állapotot. A `readelf` a célarchitektúrát, szekciókat és szimbólumokat ellenőrzi; az `objdump` a tényleges gépi utasításokat mutatja. A `gdb-multiarch` a QEMU kiszolgálójához kapcsolódva megállítja és lépteti az AArch64 programot. A töréspont az érintett utasítás végrehajtása előtt állít meg, ezért a regiszterértékeket mindig a programszámlálóval együtt értelmezzük. A forrásszintű `step` és az utasításszintű `stepi` különbsége később optimalizált kódnál is fontos. Minden vizsgálat előfeltétele az azonos, frissen épített bináris használata.

- **Disassembly:** gépi kód assembly alakú megjelenítése.
- **Töréspont:** kijelölt végrehajtási helyhez kötött megállítás.
- **Hibakeresési információ:** többek között forráshelyeket és gépi kódot összerendelő leíró adat.

3.4.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GNU assembler kézikönyv: hibakeresési információ előállítása és AArch64 célbeállítások. **Ellenőrizve**: aktuális kiadás binutils 2.47.
- GNU Binutils kézikönyv: `readelf`, `objdump` és `objcopy`. **Ellenőrizve**: ugyanaz a 2.47-es kiadás.
- GNU GDB kézikönyv: távoli kapcsolat, töréspontok, léptetés és regisztervizsgálat. **Ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).
- QEMU dokumentáció: user-mode emuláció és GDB-kapcsolat. **Ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a parancsok tényleges futtatása, az Ubuntu 24.04 telepített csomagverzióinak rögzítése, a pontos szekciólista és diagnosztikai üzenetek ellenőrzése a hallgatói labor feladata. A QEMU GDB-portjának tényleges hálózati kötését szintén a helyi verzió és környezet alapján kell ellenőrizni. A programállapotokra megadott értékek levezetettek; teljesítménymérést ez a labor nem végez.

4 Az első program: regiszterek, ELF és a Linux syscall-felszín

4.1 Az A64 regiszterkészlet áttekintése

4.1.1 Miért kell külön megérteni a regisztereket?

Egy C-függvényben ugyanazt az értéket tárolhatod `uint64_t`, majd `uint32_t` típusú változóban. Assemblyben viszont nemcsak az érték szélességét kell megválasztanod: tudnod kell azt is, hogy egy 32 bites művelet mit tesz a hozzá tartozó 64 bites regiszter többi bitjével. Egy rosszul megválasztott regiszternév például elveszítheti egy 64 bites érték felső felét.

A **regiszter** (*register*) az utasítások által közvetlenül megnevezhető, rögzített szélességű processzorállapot. Nem C-változó: nincs hozzá automatikusan típus, élettartam vagy memóriacím rendelve. Egy általános célú regiszter bitmintája értelmezhető egész számként vagy címként; az értelmezést az utasítás és a program szerződése adja.

Ebben az alfejezetben minden utasítás és regiszternév az **A64 ISA, AArch64, Armv8-A alapprofil** része. A labor little-endian Linux-folyamatként, `qemu-aarch64` alatt fut. A korábbi debug-eszközök használatára építünk; memóriát még nem kell kezelned.

4.1.2 A 31 általános célú regiszter

Az A64 ISA általános célú regiszterei (*general-purpose registers*):

X0, X1, ..., X30, egyenként **64 bit** szélességgel.

Mindegyikhez tartozik egy 32 bites nézet: **W0, W1, ..., W30**. Az azonos sorszámú Xn és Wn nem két külön tároló. A Wn az Xn alsó 32 bitjének elérésére szolgál.

A regiszterek többségét az ISA hasonlóan kezeli, de a függvények együttműködéséhez szerepeket rendelünk hozzájuk. Ezeket az **eljáráshívási szabvány** (*Procedure Call Standard*), itt az AAPCS64 rögzíti.

A64 regiszter, 64/32 bites nézet	Előzetes, konvencionális szerep
X0–X7 / W0–W7	Egész és mutató jellegű argumentumok; az eredményátadásban is szerepet kapnak
X8 / W8	Közvetett eredmény helyének átadása bizonyos függvényhívásoknál
X9–X15 / W9–W15	Ideiglenes értékek; függvényhíváson át nem garantált a megőrzésük
X16–X17 / W16–W17	IP0/IP1 (intra-procedure-call): a hívás és a hívott függvény belépése között (linker-veneer, PLT) szabadon felülírható segédregiszterek

A64 regiszter, 64/32 bites nézet	Előzetes, konvencionális szerep
X18 / W18	Platformfüggő szerep; hordozható kódban ne foglald le önkényesen
X19–X28 / W19–W28	A hívott függvény által megőrzendő regiszterek
X29 / W29	Az X29 szokásos szerepe a keretmutató, FP; megőrzendő
X30 / W30	Az X30 a visszatérési címhez használt kapcsolatregiszter, LR

A „megőrzendő” azt jelenti: ha a hívott függvény módosítja a regisztert, visszatérés előtt helyre kell állítania az eredeti értékét. A táblázat ilyen kötelezettsége a teljes, 64 bites értékre vonatkozik.

A szerep és a hardverképesség nem ugyanaz. Az A64 X0 például nem kizárólag argumentumot tárolhat. Az A64 X30 azonban bizonyos utasításoknál különleges architekturális szerepet is kap: a `BL` ide írja a visszatérési címet. A részletes szerződést Az AAPCS64 regiszterszerződés tárgyalja.

4.1.3 A Wn/Xn nézetpár: olvasás és írás nem szimmetrikus

Az A64 Wn olvasása az Xn alsó 32 bitjét adja vissza, a felső biteket pedig figyelmen kívül hagyja. **Az olvasás önmagában nem módosítja az Xn értékét.**

Az A64 Wn célregiszterbe történő írás viszont:

1. beírja a 32 bites eredményt az Xn alsó felébe;
2. **nullázza az Xn felső 32 bitjét**.

Ez ISA-szabály, nem fordítói optimalizálás.

4.1.3.1 Bitszintű levezetés

Legyen x az A64 Xn előjel nélküli, 64 bites értéke. A Wn olvasásával kapott érték:

$$w = x \bmod 2^{32}.$$

Ha egy utasítás a Wn célregiszterbe a 32 bites r eredményt írja, ahol $0 \leq r < 2^{32}$, akkor utána:

$$Xn = r.$$

Az értékek mértékegység nélküli egészek; a szélességek bitben értendők. A második összefüggés azért igaz, mert a felső fél nulla lesz: ez **nullakiterjesztés** (*zero extension*), nem előjelkiterjesztés.

A64 művelet	Kezdőállapot	Következmény
W9 olvasása	X9 = `0xffffffffffffffff`	A kiolvasott 32 bites minta `0xffffffff`; X9 változatlan
W9-be `0xffffffff` írása	X9 tetszőleges	X9 = `0x00000000ffffffff`

A64 művelet	Kezdőállapot	Következmény
X9-be teljes 64 bites érték írása	X9 tetszőleges	Mind a 64 bit az új értékből származik

A `0xffffffff` bitminta 32 bites, előjeles kettes komplemens számként -1. Ugyanez a minta a nullázott felső féllel, 64 bites előjeles számként már 4294967295.

Az alábbi ábra ugyanezt a három sort mutatja egymás után, a táblázat sorrendjében: a kezdeti X9, majd a W9 olvasása (nincs hatás), majd a W9-be írás (a felső 32 bit nullázódik).

W9 olvasása vs. írása: a felső 32 bit sorsa



4.1. ábra: W9 olvasása vs. írása: a felső 32 bit sorsa - A fenti táblázat három sorának megfelelő X9/W9 állapotsor.

C-kapcsolatként: egy `uint32_t` érték `uint64_t` típusra alakítása hasonló nullakiterjesztést igényel. Egy negatív `int32_t` érték `int64_t` típusra alakításához viszont előjelkiterjesztés kell. **A Wn-be írás nem következtet C-típusra.**

4.1.4 Speciális állapot: SP, PC és a nulla-regiszter

4.1.4.1 A64 SP: veremmutató

Az **SP veremmutató** (*stack pointer*) 64 bites, speciális regiszter. A verem aktuális határát jelöli; nem az X0–X30 készlet egy újabb tagja.

Az A64 ISA bizonyos utasításformákban 32 bites WSP nézetet is biztosít, de 64 bites címek kezeléséhez ez nem helyettesíti az SP-t. Ebben a laborban egyiket sem módosítjuk. Az AAPCS64 nyilvános hívási határokon 16 bájtos veremigazítást követel meg; ennek indoklása A stack frame felépítése és a verem iránya témája.

4.1.4.2 A64 PC: programszámláló

A **programszámláló** (*program counter*), PC, az utasításvégrehajtás helyét határozza meg. A GDB megmutatja, de az A64 ISA-ban nem általános célú operandus: nem olvasható vagy írható egy közönséges `MOV` utasítással. PC-relatív címet ADR/ADRP adja, a visszatérési címet BL írja X30-ba.

A végrehajtási helyet elágazás, hívás, visszatérés vagy kivétel módosíthatja. A debugger PC-módosító képessége nem jelenti azt, hogy létezik ugyanilyen A64 regisztermásoló utasítás.

4.1.4.3 A64 XZR/WZR: nulla olvasáskor, eldobás íráskor

A **nulla-regiszter** (*zero register*) 64 bites neve XZR, 32 bites neve WZR.

- Forrásként olvasva mindig nulla.
- Célként írva eldobja a neki szánt eredményt.
- Nem használható értékek megőrzésére.

Az eredmény eldobása nem feltétlenül szünteti meg az utasítás más mellékhatásait.

Fontos kódolási részlet: az A64 utasítások számos regisztermezője 5 bites. A 31-es mezőérték **nem egy szabadon használható X31 regisztert jelöl**: az adott utasításformától és operandushelytől függően SP-t vagy nulla-regisztert jelenthet, illetve az adott forma korlátozhatja használatát. Az SP és az XZR ezért nem felcserélhető.

4.1.5 PSTATE és NZCV: a regisztereken túli állapot

Az A64 **processzorállapot** (*processor state*), PSTATE, többek között állapotjelzőket tartalmaz. A később használt négy jelző:

A64 állapotjelző	Előzetes jelentés
N	Az eredmény előjelbitjéhez kapcsolódik
Z	Nulla eredményt jelez
C	Átvitelhez, illetve kivonásnál kölcsönzéshez kapcsolódik
V	Előjeles túlcsordulást jelez

Együttes nevük **NZCV**. Nem minden aritmetikai vagy adatmozgató utasítás módosítja őket. Különösen: az A64 Wn-be írás felsőbites nullázása nem azonos a Z jelző beállításával.

A labor adatmozgató utasításai nem módosítják az NZCV jelzőket. A pontos szabályokat Az NZCV állapotjelzők és eredetük fejti ki.

4.1.6 Kidolgozott példa: ugyanaz a tároló, két nézet

Munkakönyvtár: `~/arm-labor/a64/a010/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a010
cd ~/arm-labor/a64/a010
```

A labor célja három külön tulajdonság megfigyelése:

1. A64 Wn olvasása nem törli az Xn felső felét.
2. A64 Wn írása törli az Xn felső felét.
3. Az A64 nulla-regiszter nem őrzi meg a beleírt értéket.

Mentsd a következő teljes programot `lab-a010.s` néven:

```
// A64 ISA, Armv8-A; GNU as szintaxis és direktívák.
// Cél: little-endian AArch64 Linux, qemu-user.

.arch armv8-a
.text
.balign 4
.global _start
.type _start, %function

_start:
    movn    x9, #0

lab_after_seed:
    mov     w10, w9

lab_after_read:
    mov     w9, w10

lab_after_write:
    mov     x11, xzr
    mov     xzr, x10
    mov     w12, wzr

lab_done:
    movz    x0, #0
    movz    x8, #93
    svc     #0

.size _start, . - _start
.section .note.GNU-stack,"",%progbits
```

Az itt használt GNU as direktívák az AArch64 forrás szekcióját, igazítását és szimbólumait írják le; nem processzorutasítások. Részletes szerepükre a Hello world tisztán assemblyben tér vissza.

Az A64 `MOVN X9, #0` a nullából képzett 64 bites bitminta bitenkénti negáltját írja be, tehát minden bitet egyre állít. A `MOVN` közvetlen konstansa (*immediate operand*) 16 bites; X célregiszternél a választható eltolás 0, 16, 32 vagy 48 bit. Itt az eltolás az alapértelmezett 0.

Az A64 regiszterek közötti `MOV` itt másoló utasításalias: nincs közvetlen konstansa. Az A64 `MOVZ` 16 bites konstanst helyez el, a többi bitet nullázva. X célregiszternél ugyanazok az eltolások választhatók; a labor 0 eltolást használ.

A programvégi A64 utasítások Linux-kilépési keretet alkotnak: X0 értéke az előjel nélküli 0 kilépési kód, X8 értéke a Linux AArch64 `exit` rendszerhívás 93-as száma. Az A64 `SVC` 16 bites immediate mezője itt 0; nem ez választja ki a rendszerhívást. Ez Linux-interfész, nem AAPCS64-függvényhívás: lásd Linux syscall-konvenció AArch64-en.

4.1.6.1 Összeszerelés, linkelés, futtatás

```
# Host: Ubuntu 24.04 x86_64.
# Target: qemu-user, little-endian AArch64 Linux.
aarch64-linux-gnu-as -EL -march=armv8-a -g \
    -o lab-a010.o lab-a010.s

aarch64-linux-gnu-ld -EL -e _start \
    -o lab-a010.elf lab-a010.o

qemu-aarch64 ./lab-a010.elf
printf 'exit=%d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): a program nem ír ki szöveget; a shell `exit=0` sort jelenít meg. Ez csak a szabályos befejezést igazolja, nem a köztes regiszterállapotokat.

4.1.6.2 Regisztervizsgálat GDB-vel

Az első terminálban:

```
# Target: qemu-user; a program debuggerkapcsolatra vár.
qemu-aarch64 -g 1234 ./lab-a010.elf
printf 'exit=%d\n' "$?"
```

A második terminálban:

```
# Target: az előző qemu-user folyamat A64 regiszterállapota.
gdb-multiarch -q ./lab-a010.elf \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_after_read' \
    -ex 'break lab_after_write' \
    -ex 'break lab_done' \
    -ex 'continue' \
    -ex 'info registers x9 x10 pc' \
    -ex 'continue' \
    -ex 'info registers x9 x10 pc' \
    -ex 'continue' \
    -ex 'info registers x9 x10 x11 x12 sp pc' \
    -ex 'continue' \
    -ex 'quit'
```

A töréspont a címkéhez tartozó következő utasítás **előtt** állít meg.

Megállási hely	Mért A64 regiszterállapot (GDB 15.1, QEMU 8.2.2)
`lab_after_read`	X9 = `0xffffffffffffffff`, X10 = `0x00000000ffffffff`
`lab_after_write`	X9 = X10 = `0x00000000ffffffff`
`lab_done`	X9 és X10 változatlan; X11 = 0, X12 = 0

A hexadecimális bejegyzések bitminták; a nulla itt előjel nélküli értékként szerepel. A GDB a vezető nullákat nem írja ki; a második oszlop az előjeles decimális érték. Teljes 64 bites hexához: `p/x $x10` vagy `info registers` + `x/gx`. Az SP-t a program nem módosítja. A PC konkrét száma helyett a megfelelő szimbólumhoz tartozást ellenőrizd.

A `lab_after_read` megállás mutatja az olvasás és írás különbségét: a W9 forrás olvasása megőrzi az X9 felső bitjeit, miközben a W10 cél írása már nullázza az X10 felső felét.

4.1.7 Tipikus hibák és ellenpróbák

„A W9 és az X9 két külön regiszter."
Tünet: egy 32 bites művelet után eltűnik a korábban X9-ben tárolt 64 bites érték felső fele. Hipotézis: az utasítás W9-et írta. Állj meg előtte és utána; mindig az egész X9-et vizsgáld.

„A W9-be írt −1 automatikusan 64 bites −1 lesz."
Nem: az A64 W9-be írás nullakiterjeszt. Az ellenpróba a labor `lab_after_write` állapota. A szükséges előjelkiterjesztés külön művelet, nem regisztertulajdonság.

„Az XZR ideiglenes tároló."
A beleírt érték elveszik. Egyetlen változtatásként cseréld a labor `mov w12, wzr` utasítását `mov x12, xzr` alakra: X12 továbbra is nulla, noha korábban X10-et „beleírtuk" az XZR-be.

„Az utasítás helyesnek látszik, de a debugger mást mutat."
Először gyűjts bizonyítékot a tényleges binárisból:

```
# Target: a qemu-userhez épített A64 ELF statikus vizsgálata.
aarch64-linux-gnu-objdump -d ./lab-a010.elf
```

Ellenőrizd a W/X operandusokat, majd egyetlen változtatás után építs újra és indíts új QEMU-folyamatot. A régi folyamat nem cseréli le automatikusan a már betöltött programját.

4.1.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért marad minden bit egy az A64 X9-ben a `lab_after_read` ponton, miközben az X10 felső fele nulla? Mennyi ekkor a W10 előjeles, illetve az X10 előjeles értéke?

2. (implementációs, alapszint — cél: rutinépítés) Módosítsd a programot úgy, hogy az eredeti, minden bitjén egyes 64 bites érték az A64 X13-ban a `lab_done` pontig megmaradjon. Az eredeti W9-be írást ne töröld. Építsd újra ugyanazokkal a parancsokkal, és egészítsd ki a GDB regiszterlistáját.

3. (hibakeresési, középszint — cél: megértés-ellenőrzés) Kontrollált hibaként cseréld az A64 `mov w10, w9` utasítást `mov x10, x9` alakra. Előre írd le az X9 és X10 várható értékét mindkét első töréspontón. Ellenőrizd disassemblyvel és GDB-vel, majd egyetlen módosítással állítsd vissza az eredeti viselkedést.

4.1.9 Megoldások

1. megoldás — Segítség. Különböztesd meg, melyik utasítás CSAK olvassa a forrásregisztert, és melyik ÍR egy célregiszterbe — a kettő között pont ez a nullázási szabály tesz különbséget.

1. megoldás — Teljes megoldás. A W9 csak forrás: olvasása nem módosítja az X9-et. A W10 célregiszter írása viszont nullázza az X10 felső felét. A W10 előjeles, 32 bites értéke -1; az X10 előjeles, 64 bites értéke 4294967295.

2. megoldás — Segítség. A mentést a W9-be írás ELŐTT kell elvégezni, különben már a nullázott érték másolódna. Gondold át, miért nem elég egy W-nézetű (32 bites) mentés.

2. megoldás — Teljes megoldás. A `movn x9, #0` után, de a W9-be írás előtt helyezd el az A64 `mov x13, x9` utasítást. A GDB-listába kerüljön `x13` is. A `lab_done` ponton X13 várt bitmintája `0xffffffffffffffff`, X9-é `0x00000000ffffffff`. A teljes érték megőrzéséhez 64 bites másolás kell; W13 használata már a másolat felső felét is nullázná.

3. megoldás — Segítség. A hiba az, hogy a második utasítás immár a TELJES 64 bitet másolja, nem csak az alsót — kövesd végig lépésenként, melyik regiszter melyik félbitje változik.

3. megoldás — Teljes megoldás. A hibás változat `lab_after_read` pontján X9 és X10 is `0xffffffffffffffff`. A következő, változatlan A64 `mov w9, w10` csak az alsó 32 bitet másolja: `lab_after_write` esetén X9 már `0x00000000ffffffff`, X10 továbbra is `0xffffffffffffffff`. Az eredeti 32 bites másolás visszaállítása helyreállítja a várt állapotokat. A kilépési kód mindkét változatban 0 lehet, ezért önmagában nem megfelelő ellenőrzés.

4.1.10 Rövid összefoglaló

Az A64 ISA 31 általános célú, 64 bites regisztert biztosít X0–X30 néven. W0–W30 ezek alsó 32 bites nézete, nem külön tároló. Wn olvasása nem módosítja Xn-t, Wn írása viszont nullázza Xn felső felét. Az SP speciális veremmutató; a PC nem közönséges adatmozgatási operandus. Az XZR/WZR mindig nullát szolgáltat, a beleírt eredményt eldobja. A hívási szerepek együttműködési szabályok, nem általános hardverkorlátozások. Az NZCV külön processzorállapot, amelyet nem minden utasítás módosít. A laborban a köztes állapotokat töréspontonként kell ellenőrizni, mert a sikeres kilépés önmagában nem igazolja őket.

- **Regiszternézet:** ugyanazon tároló meghatározott szélességű elérése.
- **Nullakiterjesztés:** szélesítés nulla felső bitekkel.
- **Nulla-regiszter:** nulla forrásérték és eldobható cél.

4.1.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile dokumentáció: A64 regisztermodell, PSTATE és utasításszemantika. *Ellenőrizve*: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- AAPCS64 – hivatalos Arm ABI-dokumentum: regiszterszerepek és megőrzési kötelezettségek. *Ellenőrizve*: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- GNU assembler kézikönyv: AArch64 szintaxis és direktívák. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.

- GNU GDB kézikönyv: távoli hibakeresés és regisztervizsgálat. *Ellenőrizve*: a „current” fejlesztői pillanatkép (közös verziókorlát).
- Linux syscall kézikönyv: architektúrafüggő rendszerhívási interfész. *Ellenőrizve*: a Linux man-pages projekt 6.19-es kiadása.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a közölt értékek a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 fő útján mért eredményei (`mérve`). Az Ubuntu 24.04-re telepített eszközök pontos verzióját és a GDB-megjelenítést a laborban kell ellenőrizni. A gyakorlat ISA-viselkedést vizsgál, nem hardverhű végrehajtási időt.

4.2 Hello world tisztán assemblyben: szekciók, `_start`, syscall hívás

4.2.1 Miért nem kell ehhez C futtatókörnyezet?

Egy C-ben megírt üdvözlőprogram rövid, de a végrehajtható állomány rendszerint jóval többet tartalmaz a saját függvényünknél: indulókódot, könyvtári kapcsolódásokat és a szabványos I/O kezelését. Most ezt a rétegzettséget bontjuk szét egy konkrét feladattal: írjunk ki egy üzenetet, majd fejezzük be a folyamatot **libc és C-indulókód nélkül**.

Az önállóság itt nem operációs rendszer nélküli működést jelent. A program Linux-szolgáltatásokat használ; a fájlleírók és a folyamat kezelése továbbra is az operációs rendszer feladata.

Profil: az alfejezet minden utasítása és regisztere az **A64 ISA, AArch64, I. rész** környezetéhez tartozik. Az assemblydirektívák GNU assembler-direktívák ebben az AArch64 ELF-profilban, nem processzorutasítások. A cél Armv8-A, little-endian Linux; az elsődleges futtatási mód `qemu-aarch64`, vagyis QEMU user-mode emuláció.

Az előfeltétel az A64 regiszterkészletének áttekintése. Veremkezelésre vagy függvényhívási keretre most nem lesz szükség.

4.2.2 A forrástól a belépési pontig

A **szekció** (*section*) az objektumfájl tartalmának névvel és tulajdonságokkal ellátott egysége. Ebben a példában két fontos szekciót használunk:

GNU assembler-direktíva, AArch64 ELF-profil	Szerep
`.text`	A végrehajtandó utasítások szekciója.
`.data`	Inicializált, írható adatok szekciója.

Az üzenet nem változik, ezért később természetesebb helye lehetne egy csak olvasható adatszekcióban. Most tudatosan `.data`-ba tesszük, hogy a kód és az inicializált adat elkülönítését lássuk.

A szekció nem azonos a futás közbeni memóriaszegmenssel. A linker a szekciókból alakítja ki a betölthető tartományokat; ezt az ELF formátum alapjai és a program életciklusa részletezi.

A **belépési pont** (*entry point*) az a cím, ahol a program végrehajtása megkezdődik. A `_start` elnevezés szokásos szimbólumnév, nem hardveresen fenntartott szó. A linkernek külön megmondjuk, hogy ezt válassza belépési pontnak.

Fontos különbség a C-hez képest:

- a `main` függvényt rendszerint indulókód hívja meg;
- a mi `_start` címkénknél nincs ilyen hívó;
- ezért a program végét nem A64 `RET` utasítással, hanem Linux-rendszerhívással jelezzük.

A `.global _start` láthatóvá teszi a szimbólumot a linker számára. Önmagában nem helyettesíti a belépési pont kijelölését.

4.2.3 A rendszerhívás mint regiszterekben leírt kérés

A **rendszerhívás** (*system call, syscall*) az alkalmazás és az operációs rendszer közötti szabályozott szolgáltatáskérés. A Linux AArch64 rendszerhívási bináris interfésze (*application binary interface, ABI*) meghatározza, mely regiszterekbe kerülnek az adatok.

Ez **Linux-konvenció**, nem az A64 ISA általános tulajdonsága.

Szerep	Linux AArch64 rendszerhívási ABI
Rendszerhívásszám	A64 `X8`
Első–hatodik argumentum	A64 `X0`–`X5`, sorrendben
Szolgáltatáskérés	A64 `SVC #0`
Visszatérési érték	A64 `X0`, ha a hívás visszatér

A64-en az `SVC` immediate operandusa 16 bites, tehát 0–65535 közötti értéket kódolhat. **A Linuxhoz itt `SVC #0` kell; a rendszerhívásszám nem ebbe az operandusba, hanem az A64 `X8` regiszterbe kerül.**

Fizikai AArch64 Linuxon az utasítás kivételt vált ki, amelyet az operációs rendszer kezel. QEMU user-mode környezetben a QEMU emulálja a vendégprogram rendszerhívását a host szolgáltatásaival. Itt nem külön AArch64 Linux-kernelt indítunk.

A két szükséges szolgáltatás:

Hívás	A64 `X8`	A64 `X0`	A64 `X1`	A64 `X2`
Linux `write`	64, azaz `0x40`	Fájlleíró: 1	Az üzenet címe	Hossz bájtban
Linux nyers `exit`	93, azaz `0x5d`	Kilépési állapot	Nem argumentum	Nem argumentum

A fájlleíró (*file descriptor*) 1 szokásosan a szabványos kimenetet jelöli. A shell azonban átirányíthatja vagy bezárhatja; ezért ez nem feltétlenül terminált jelent.

A nyers Linux `exit` nem a C könyvtár `exit()` függvénye: nem futtat könyvtári lezárási műveleteket, és nem ürít libc-puffereket. A hívó szálat szünteti meg; ebben az egyszálú programban ezzel a folyamat is véget ér.

4.2.4 Kidolgozott példa: teljes, libc nélküli program

Munkakönyvtár: `~/arm-labor/a64/a011/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a011
cd ~/arm-labor/a64/a011
```

Mentsük a következő forrást `lab-hello.s` néven:

```
// Profil: A64 ISA, AArch64, Armv8-A, little-endian Linux.
// Direktívák: GNU assembler, AArch64 ELF.
// Cél: qemu-user, qemu-aarch64.

.section .data
message:
    .ascii "Hello, ARM64!\n"
.equ message_len, . - message

.section .text
.balign 4
.global _start
.type _start, %function

_start:
    mov x0, #1
    ldr x1, =message
    mov x2, #message_len
    mov x8, #64

before_write:
    svc #0

after_write:
    mov x19, x0        // a nyers visszatérési érték megőrzése
lab_check:
    mov x0, #0
    mov x8, #93
    svc #0

.size _start, . - _start
.ltorg

.section .note.GNU-stack,"",%progbits
```

4.2.4.1 Az üzenet és a hossz kapcsolata

Az AArch64 GNU assembler `.ascii` direktívája a megadott bájtokat helyezi el, **lezáró nulla nélkül**. A `\n` ebben a példában egyetlen, `0x0a` értékű bájt.

A hossz levezetése:

Üzenetrész	Bájtok száma
`Hello`	5
Vessző	1
Szóköz	1
`ARM64`	5
Felkiáltójel	1
Újsor	1
Összesen	**14**

A `.equ message_len, . - message` assemblerdirektívában a `.` az aktuális helyszámláló. Közvetlenül az üzenet után állunk, ezért a különbség az üzenet bájthossza.

Ez **összeállításkor kiszámított konstans**, nem futás közben végzett kivonás és nem memóriában tárolt hosszmező.

A Linux `write` nem C-sztringet keres: a cím és a bájtszám alapján dolgozik. Ha az üzenetben nulla bájt volna, azt is kiírná, amennyiben beleesik a megadott hosszba. UTF-8 szövegnél ugyanezért a karakterek száma eltérhet a bájtok számától; a címkekülönbség akkor is a megfelelő bájthosszt adja.

4.2.4.2 A cím betöltése nem az adat beolvasása

Az A64-környezetben használt GNU assembler

```
ldr x1, =message
```

alakja **pszeudoutasítás** (*pseudo-instruction*). A célja itt a `message` szimbólum címének betöltése az A64 `X1` regiszterbe, nem az üzenet első nyolc bájtjának beolvasása.

A GNU assembler ehhez literálkészletet (*literal pool*) használ: a kód közelében elhelyezi a cím tárolására szolgáló bejegyzést, amelynek végleges értékét a linker rendezi. Az A64 `LDR` literálalakja innen tölti be a címet.

Ennek a valódi utasításnak az elérése korlátos: 19 bites előjeles eltolást kódol, néggyel skálázva. Az utasítás címéhez képesti tartomány ezért $-1\,048\,576$ és $+1\,048\,572$ bájt közötti. A `.ltorg` assemblerdirektíva itt, a rövid kód végén helyezi el a függő literálkészletet, bőven ezen a tartományon belül.

A literálkészlet adatait nem akarjuk utasításként végrehajtani. Előtte a Linux `exit` rendszerhívás megszünteti a programot.

4.2.4.3 Miért működnek ezek a konstansbetöltések?

A példában szereplő A64 `MOV` immediate alakok kis, nemnegatív konstansokat töltenek be. Ezek egyetlen A64 `MOVZ` utasítással kódolhatók.

Az A64 `MOVZ` egy 16 bites mezőt helyez el a célregiszterben; 64 bites cél esetén a megengedett eltolás 0, 16, 32 vagy 48 bit. Itt minden értékhez a 0 bites eltolás elég.

Ebből **nem következik**, hogy a `mov xn, #konstans` bármely 64 bites számot elfogad egyetlen utasításként. A részleteket a MOV-variánsok és immediate-kódolás tárgyalja.

A `.balign 4` négybájtos határra igazítja a kód kezdetét. A `.type` és `.size` szimbólummetaadatot ad az eszközöknek. A `.note.GNU-stack` szekcióbejegyzés pedig azt jelzi a GNU eszközláncnak, hogy ez az objektum nem igényel végrehajtható vermet; maga nem végrehajtott kód.

4.2.5 Összeállítás, linkelés és futtatás

A következő parancsok célja **qemu-user**, Ubuntu 24.04 x86_64 hoston, AArch64 kereszteszközlánccal. A szükséges telepítést a labor-alapkörnyezet ismerteti.

```
# Cél: qemu-user; AArch64 Armv8-A, little-endian Linux.

aarch64-linux-gnu-as -g -march=armv8-a -EL \
    -o lab-hello.o lab-hello.s

aarch64-linux-gnu-ld -static -e _start \
    -o lab-hello lab-hello.o

qemu-aarch64 ./lab-hello
```

```
lab_status=$?
printf 'exit=%d\n' "$lab_status"
```

Az első lépés objektumfájlt állít elő. A második feloldja a szimbólumhivatkozásokat, kijelöli a belépési pontot és létrehozza a végrehajtható ELF-fájlt. Nem adunk hozzá C-indulókódot vagy könyvtárat.

A `-static` itt nem „statikusan beépített libc”-t jelent: egyáltalán nem linkelünk libc-t. A létrejövő állomány futtatásához ezért nem kell AArch64 dinamikus betöltő vagy megosztott könyvtárkészlet.

Mért eredmény normál, írható szabványos kimenet mellett (mérve: GNU as 2.42, QEMU 8.2.2):

Megfigyelés	Mért érték
Program kimenete	`Hello, ARM64!`, majd újsor
A64 `X0` közvetlenül a `write` után	14
Shell által kiírt állapot	`exit=0`

A kilépési állapotot közvetlenül a QEMU futtatása után mentjük el. Ha közben más parancsot hajtanánk végre, a shell `$?` értéke már annak az állapotát jelentené.

A fenti értékek a lektor GNU as 2.42 + QEMU 8.2.2 fő útján mért eredményei; a laborban ugyanezeknek kell visszajönniük.

4.2.5.1 Bájtokra pontos kimenetellenőrzés

A terminálon látható szöveg önmagában nem elég: egy fölösleges nulla bájt például láthatatlan maradhat.

```
# Cél: qemu-user; kimenetrögzítés és ellenőrzés a hoston.

qemu-aarch64 ./lab-hello > lab-output.bin
lab_status=$?
printf 'exit=%d\n' "$lab_status"
wc -c < lab-output.bin
od -An -tx1 lab-output.bin
```

Mért állapot (mérve: GNU as 2.42, QEMU 8.2.2): a kilépési kód 0, a fájl hossza 14 bájt. A bájtsor hexadecimális jelöléssel:

```
0x48 0x65 0x6c 0x6c 0x6f 0x2c 0x20 0x41 0x52 0x4d 0x36 0x34 0x21 0x0a
```

A bájtok sorozatánál nincs több-bájtos egészértékből eredő bájtsorrend-kérdés.

4.2.6 A visszatérési érték tényleges megfigyelése

A Linux `write` visszatérésekor az A64 `X0` új szerepet kap: már nem fájlleíró, hanem eredmény.

- Nemnegatív, előjeles olvasatban az átadott bájtok száma.
- Negatív, előjeles olvasatban nyers Linux-hibakód.
- Pozitív eredmény is lehet kisebb a kért hossznál: ez részleges írás.

A libc `write()` burkolófüggvényével ellentétben itt nincs automatikus `errno`-beállítás és egységes `-1` visszatérés.

A mi programunk az `after_write` címkénél rögtön felülírja az A64 `X0` regisztert a kilépési állapottal. **A 0 kilépési kód ezért nem bizonyítja, hogy az írás sikerült.** Ez a kezdőprogram tudatos korlátozása: nincs hibakezelése és nincs részleges írást folytató ciklusa.

A visszatérési érték megfigyeléséhez indítsuk az első terminálban:

```
# Cél: qemu-user, GDB-kapcsolatra váró program.

qemu-aarch64 -g 1234 ./lab-hello
lab_status=$?
printf 'exit=%d\n' "$lab_status"
```

A második terminálban:

```
# Cél: a qemu-user AArch64 vendégprogramjának vizsgálata.
# A vizsgált regiszterek az A64 ISA regiszterei.

gdb-multiarch -q ./lab-hello \
    -ex 'target remote :1234' \
    -ex 'break before_write' \
    -ex 'continue' \
    -ex 'info registers x0 x1 x2 x8' \
    -ex 'x/14bx $x1' \
    -ex 'break lab_check' \
    -ex 'continue' \
    -ex 'p/d (long)$x19' \
    -ex 'continue' \
    -ex 'quit'
```

A töréspont a szolgáltatáskérés előtt állítja meg a programot; a második töréspont a `lab_check` címkén, közvetlenül az `SVC` után. A QEMU user-mode gdbstubja az `SVC`-n álló töréspontról adott `si`-nél két utasítást hajt végre (az `SVC`-t és az utána következőt), ezért a nyers visszatérési értéket az `SVC` utáni `mov x19, x0` menti el, és a `lab_check`-nél `$x19`-ból olvassuk ki.

Ellenőrzési pont	Mért állapot (GDB 15.1, QEMU 8.2.2)
A64 `SVC #0` előtt	`X0=1`, `X2=14`, `X8=64`
A64 `X1` által mutatott memória	A felsorolt 14 üzenetbájt
A `lab_check`-nél (SVC után)	`X19=14`, normál sikeres írásnál

A konkrét cím helyett a szimbólumot és a regiszterben lévő címet használjuk; nem építünk be feltételezett memóriacímet.

> **Figyelem: QEMU user-mode és `SVC`.** A QEMU 8.2.2 user-mode gdbstubja az `SVC` utasításon álló töréspontról adott `si`-nél nem egy, hanem két utasítást hajt végre (az `SVC`-t és az utána következőt), ezért a rendszerhívás visszatérési értéke csak akkor figyelhető meg, ha az `SVC` után egy megfigyelő utasítás (itt `mov x19, x0`) áll, és a töréspont az azt követő címkére kerül. A `catch syscall` ebben a QEMU-verzióban nem támogatott.

4.2.7 Tipikus hibák és ellenpróbák

A hibakeresés menete legyen: **tünet → hipotézis → adatgyűjtés → egy változtatás → új ellenőrzés**.

Tünet	Lehetséges ok	Ellenőrzés
`Exec format error` x86_64 hoston	Közvetlenül indítottuk az AArch64 fájlt	Indítás `qemu-aarch64` segítségével
Nincs üzenet, mégis 0 a kilépési kód	A `write` hibázott, az eredményt eldobtuk	A64 `X0` vizsgálata közvetlenül a hívás után
Hibás vagy túl hosszú kimenet	Helytelen bájtszám	A64 `X2`, majd a kimenet bájtjainak ellenőrzése
A hívás memóriahibát jelez	Az A64 `X1` nem a megfelelő adat címe	Memóriavizsgálat az A64 `X1` címen
A szöveg után láthatatlan többletbájt van	`.ascii` helyett `.asciz`, változatlan hosszszámítással	`wc` és `od`
Nem a kívánt szolgáltatás fut	Rossz rendszerhívásszám	A64 `X8` ellenőrzése, nem az `SVC` operandusának módosítása

Az utasítások és a szimbólumok kapcsolatát disassemblyvel is vizsgálhatjuk:

```
# Célállomány: qemu-user számára készült AArch64 ELF.
# Ez hostoldali vizsgálat, nem futtatás.

aarch64-linux-gnu-objdump -d lab-hello
aarch64-linux-gnu-readelf -h -S -l lab-hello
```

Az A64 `MOV` megjelenítése aliasfüggő lehet; a literálkészlet szintén megjelenhet a `.text` tartalmában. Nem minden ott tárolt bájt végrehajtandó utasítás. A részletes értelmezést a Disassembly objdump/readelf-fel tárgyalja.

4.2.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért működik a kiírás lezáró nulla nélkül? Mi változna, ha kizárólag az AArch64 GNU assembler `.ascii` direktíváját cserélnénk `.asciz` direktívára? Miért nem javítható egy rossz rendszerhívásszám az A64 `SVC` operandusának átírásával?

2. (implementációs, középszint — cél: rutinépítés) Készíts `lab-szia.s` nevű változatot, amely a `Szia, ARM64!` szöveget írja ki újsorral, majd 7 kilépési állapottal fejeződik be. A hosszt továbbra is címkekülönbség számítsa. Ellenőrizd a kimenetet és a kilépési állapotot.

3. (hibakeresési, középszint — cél: megértés-ellenőrzés) A GDB-munkamenetben, közvetlenül az A64 `SVC #0` előtt állítsd az A64 `X0` regisztert -1-re. A program forrását ne változtasd meg. Jegyezd fel a rendszerhívás előjeles visszatérési értékét és a folyamat kilépési állapotát. Magyarázd meg az eltérést.

4.2.9 Megoldások

1. megoldás — Segítség. Gondold végig, mit kap paraméterként a `write`: egy címet és egy hosszt, vagy egy sztringet? Ha `.asciz`-re cserélnéd a direktívát, hogyan változna a `. - message` különbség?

1. megoldás — Teljes megoldás. A Linux `write` cím–hossz párost kap, nem nullával lezárt sztringet. Az AArch64 GNU assembler `.asciz` direktívája hozzáadna egy `0x00` bájtot. Mivel a hosszszámítás az üzenet után áll, az új hossz 15 lenne, és a program a nullát is kiírná.

A szolgáltatást a Linux AArch64 ABI szerint az A64 `X8` választja ki. Az A64 `SVC` immediate mezője nem ennek alternatív tárolóhelye; a laborban maradjon `#0`.

2. megoldás — Segítség. Másold a teljes `lab-hello.s`-t, és csak három helyen változtass: az üzenetszöveg, a kilépési kód betöltése, és — mivel az üzenet hossza is más — semmit a hosszszámításon, hiszen az automatikusan követi a címkekülönbséget.

2. megoldás — Teljes megoldás. A teljes forrás másolatában az üzenet legyen `.ascii "Szia, ARM64!\n"`, az `after_write` címke utáni első utasítás pedig A64 `mov x0, #7`.

A várt hossz 13 bájt: négy betű, vessző, szóköz, öt üzenetbeli karakter, felkiáltójel és újsor.

```
# Cél: qemu-user; AArch64 Armv8-A, little-endian Linux.

aarch64-linux-gnu-as -g -march=armv8-a -EL \
    -o lab-szia.o lab-szia.s

aarch64-linux-gnu-ld -static -e _start \
    -o lab-szia lab-szia.o

qemu-aarch64 ./lab-szia
lab_status=$?
printf 'exit=%d\n' "$lab_status"
```

Várt kimenet: `Szia, ARM64!` és újsor, majd a shell ellenőrzésében `exit=7`. Ez szándékosan választott, nem véletlen hibajelzés.

3. megoldás — Segítség. A `-1` mint fájlleíró biztosan érvénytelen; gondold végig, a `write` szokásos Linux-hibajelzése hogyan jelenik meg A64 `X0`-ban (előjeles, negatív érték), és hogy a program forrása mikor írja felül ezt az értéket.

3. megoldás — Teljes megoldás. A GDB-indításban a `break lab_check` és a `continue` közé illesztendő argumentum: `-ex 'set $x0 = -1'`; a `lab_check`-nél `p/d (long)$x19` adja a nyers visszatérési értéket.

A -1 nem érvényes fájlleíró. A Linux AArch64 nyers visszatérés várt értéke -9, vagyis `-EBADF`; ugyanennek 64 bites bitmintája `0xfffffffffffffff7`.

A program ezután az A64 `X0` értékét 0-ra írja, és meghívja az `exit` szolgáltatást. A folyamat kilépési állapota tehát továbbra is 0. Az ellenpróba megmutatja: a rendszerhívás eredménye és a program által választott kilépési állapot két külön adat.

4.2.10 Rövid összefoglaló

A libc nélküli AArch64 program saját belépési pontból indul, de továbbra is Linux-szolgáltatásokra támaszkodik. A kódot és az inicializált adatot külön szekcióba helyeztük, az üzenet hosszát pedig összeállításkor számítottuk ki. A Linux AArch64 rendszerhívási szerződésében az A64 X8 választja ki a szolgáltatást, az A64 X0–X5 hordozza az argumentumokat, az A64 SVC #0 pedig elindítja a kérést. A visszatérő A64 X0 tartalmát felülírás előtt kell ellenőrizni. A sikeres folyamatkilépés önmagában nem igazolja a kiírás sikerét; ehhez a visszatérési érték és a kimeneti bájtok vizsgálata is szükséges.

- **Szekció:** névvel és tulajdonságokkal ellátott objektumfájl-egység.
- **Belépési pont:** a végrehajtás kezdőcíme.
- **Rendszerhívási ABI:** az operációsrendszer-kérések bináris szerződése.

4.2.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile dokumentáció: A64 utasítások, különösen `SVC`, `MOVZ` és a literálbetöltés. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv: szekciók, direktívák és AArch64 pszeudoutasítások. **Ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- Linux syscall(2): architektúrafüggő rendszerhívási konvenciók. **Ellenőrizve**: a Linux man-pages projekt 6.19-es kiadásából.
- Linux write(2): visszatérési érték, részleges írás és hibák. Ugyanazon man-pages kiadásból; tartalmilag itt nem lett külön lekérve.
- Linux _exit(2): a könyvtári kilépési felület és a nyers rendszerhívás különbsége. Ugyanazon man-pages kiadásból; tartalmilag itt nem lett külön lekérve.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenet, bájtsor és regiszterértékek a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 fő útján mért eredményei. A helyi Ubuntu 24.04 csomagverziókat és a labor tényleges eredményeit a hallgatónak kell ellenőriznie. Teljesítménymérés nem szerepel a példában.

4.3 Az ELF formátum alapjai és a program életciklusa

4.3.1 Miért kell az assembly program mellé az ELF-et is megérteni?

Egy fájl tartalmazhat helyes A64 gépi kódot, mégsem biztos, hogy elindítható. Az operációs rendszernek azt is tudnia kell, mely fájlrészek kerüljenek memóriába, milyen hozzáférési jogokkal, és honnan kezdődjön a végrehajtás. Ezeket az információkat nem maguk az utasítások hordozzák.

Az előző alfejezet Hello world programja már végrehajtható ELF-fájlként futott. Most ezt a fájlt vizsgáljuk meg kívülről: összekötjük az assembler szekcióit a linker kimenetével és a programbetöltő működésével.

A célkörnyezet végig **AArch64 Linux, A64 ISA, little-endian ELF64**, elsődlegesen `qemu-aarch64` felhasználói módú emulációval. A bemutatott kimenetek és ellenőrzési eredmények a

lektor GNU as/ld 2.42 + QEMU 8.2.2 mérésén alapulnak; a saját laborban ugyanezeket kell visszaellenőrizni.

4.3.2 Az ELF három különböző nézete

Az **ELF** (*Executable and Linkable Format*) objektumfájlok, végrehajtható programok és megosztott objektumok leírására szolgáló bináris fájlformátum. Nem kizárólag ARM-rendszereken használják.

Három szerkezetet kell megkülönböztetnünk:

1. **ELF-fejléc** (*ELF header*): az egész fájlt azonosítja, és megadja a további fejléc-táblák helyét.
2. **Programfejléc-tábla** (*program header table*): a futtatáshoz szükséges szegmenseket és egyéb betöltési információkat írja le.
3. **Szekciófejléc-tábla** (*section header table*): a fájl szekcióit írja le, elsősorban a linker és az elemzőeszközök számára.

A két utóbbi tábla nem ugyanannak az információnak kétféle elnevezése.

4.3.2.1 Az ELF-fejléc fontos mezői

Az alábbi táblázat az AArch64 Linux ELF64-környezetre vonatkozik.

Mező	Jelentés	A laborban várható érték
`e_ident`	Azonosító bájtok, osztály, bájtsorrend, formátumverzió	ELF, 64 bites, little-endian
`e_type`	Az objektum típusa	`ET_EXEC` a végleges programban
`e_machine`	Célarchitektúra	`EM_AARCH64`, numerikusan 183
`e_entry`	A program belépési virtuális címe	A linker által kiválasztott `_start` címe
`e_phoff`	A programfejléc-tábla fájlbeli eltolása	A linker állítja elő
`e_phentsize`, `e_phnum`	Programfejléc-bejegyzés mérete és darabszáma	A konkrét fájlból olvasható ki
`e_shoff`	A szekciófejléc-tábla fájlbeli eltolása	A konkrét fájlból olvasható ki
`e_shentsize`, `e_shnum`	Szekciófejléc-bejegyzés mérete és darabszáma	A konkrét fájlból olvasható ki

Az `e_type` legfontosabb értékei:

- `ET_REL`: áthelyezhető objektumfájl (*relocatable object*), például az assembler `.o` kimenete;
- `ET_EXEC`: végrehajtható objektum, a hagyományos, nem PIE programok típusa;
- `ET_DYN`: megosztott objektum típusa; a helyfüggetlen végrehajtható programok, azaz PIE-k is ezt használják.

Az `e_machine` az AArch64 architektúrát azonosítja, nem egy konkrét processzormodellt vagy az összes használt ISA-kiterjesztést.

Fontos különbség: az `e_phoff` **fájleltolás**, az `e_entry` viszont **virtuális cím**. Egyiket sem szabad automatikusan a másikként értelmezni.

4.3.3 Szekció és szegmens: szerkesztési egység és betöltési egység

A GNU assembler AArch64 ELF-célján a következő szekciók gyakoriak:

Szekció	Jellemző szekciótípus	Szerep
`.text`	`SHT_PROGBITS`	Gépi utasítások
`.rodata`	`SHT_PROGBITS`	Csak olvasható konstansok
`.data`	`SHT_PROGBITS`	Inicializált, írható adatok
`.bss`	`SHT_NOBITS`	Futáskor nullával inicializált tárterület
`.symtab`	`SHT_SYMTAB`	Szimbólumtábla, például címkék adatai

A szekciófejléc többek között a típust (`sh_type`), tulajdonságokat (`sh_flags`), méretet (`sh_size`), igazítást (`sh_addralign`) és fájlbeli helyet (`sh_offset`) írja le. A memóriába kerülő szekcióknál a `sh_addr` is jelentős.

A **szegmens** (*segment*) ezzel szemben a betöltési nézet egysége. Egy `PT_LOAD` programfejléc-bejegyzés betöltendő memóriatartományt ír le, amely több szekciót is tartalmazhat.

`PT_LOAD` mező	Jelentés
`p_offset`	A szegmens adatainak kezdete a fájlban
`p_vaddr`	A szegmens virtuális címe
`p_filesz`	A fájlból származó rész mérete, bájtban
`p_memsz`	A memóriabeli tartomány teljes mérete, bájtban
`p_flags`	Olvasási, írási, végrehajtási jogosultság
`p_align`	Igazítási követelmény

A `p_paddr` mező is létezik, de a szokásos Linux-felhasználói program nem ezzel választ fizikai memóriacímet.

A betöltő lényegében a következő leképezést hozza létre:

- a fájl `p_offset` helyétől `p_filesz` bájt tartalma megjelenik a megfelelő virtuális tartományban;
- ha `p_memsz > p_filesz`, a fennmaradó memóriarész nulla kezdőértéket kap.

`PT_LOAD` esetén ezért `p_filesz  p_memsz`. Ha `p_align > 1`, akkor kettőhatványnak kell lennie, és `p_vaddr  p_offset (mod p_align)` kell, hogy teljesüljön. A tényleges leképezésnél a rendszer lapigazítási követelményei is számítanak.

4.3.3.1 Miért nem foglal teljes méretének megfelelő helyet a `.bss` a fájlban?

Levezetés: legyen egy írható szegmens fájlbeli tartalma 4 bájt inicializált adat, memóriabeli tartalma pedig ehhez képest további 8 bájt nullázott tárterület. Igazítás nélkül ekkor:

- `p_filesz = 4` bájt;
- `p_memsz = 12` bájt;
- a különbség 8 bájt, amelyet nem kell nyolc nulla bájtként tárolni a fájlban.

Ez szemléltető számítás, nem a következő bináris pontos szegmensmérete: az igazítás és a linker elrendezése módosíthatja a különbséget.

A `.bss` tehát nem „hiányzó memória”. A helyét leírjuk, de a nulla kezdőértéket a betöltés biztosítja. Bare-metal környezetben ezt később saját startup kóddal kell majd megoldanunk.

4.3.4 Statikus és dinamikus program az ELF szemszögéből

A **statikus linkelésű** (*statically linked*) laborprogramhoz nem kell felhasználói térben futó dinamikus linker: minden szükséges saját kód benne van. Ettől még Linux-rendszerhívásokat használ; a „statikus” nem jelent operációs rendszer nélküli működést.

A szokásos **dinamikusan linkelt** (*dynamically linked*) Linux-program további megosztott objektumokra, például a libc-re támaszkodik.

ELF-jel	Jelentése
`PT_INTERP`	Megadja az indításhoz kért ELF-értelmező, tipikusan a dinamikus linker útvonalát
`PT_DYNAMIC`	Dinamikus linkelési információkat tartalmazó tartományt jelöl
`DT_NEEDED` bejegyzés	Egy szükséges megosztott objektum nevét adja meg

Az `ET_DYN` **önmagában nem bizonyítja**, hogy a fájl hagyományos értelemben dinamikusan linkelt program. Megosztott könyvtár, dinamikus PIE és statikus PIE is lehet ilyen típusú. Fordítva: egy `ET_EXEC` program is lehet dinamikusan linkelt.

A típust, a programfejléceket és szükség esetén a dinamikus táblát együtt kell vizsgálni. A következő saját belépési pontos programban nem várunk `PT_INTERP`, `PT_DYNAMIC` vagy `DT_NEEDED` bejegyzést.

4.3.5 A program életciklusa: az `execve` hívástól az első utasításig

Először a **natív AArch64 Linux** fogalmi menetét tekintsük át.

1. Egy folyamat meghívja az `execve` rendszerhívást. Siker esetén a meglévő folyamat programképe lecserélődik; maga az `execve` nem hoz létre új folyamatot. A shell előzetesen létrehozhatott külön gyermekfolyamatot.
2. A kernel ellenőrzi a fájl végrehajthatóságát, felismeri az ELF-formátumot, és megvizsgálja többek között az osztályt, architektúrát és programfejléceket.

3. Létrehozza az új címtér megfelelő tartományait. A `PT_LOAD` szegmenseket a megadott tartalommal és jogosultságokkal leképezi, a fájlban nem tárolt részek nulla kezdőértéket kapnak.
4. Előkészíti a kezdeti vermet az argumentumokkal, környezeti változókkal és a segédvektorral (*auxiliary vector*). Az A64 `SP` regiszter erre a veremre mutat.
5. Ha nincs `PT_INTERP`, a vezérlés a program belépési pontjára kerül. Ha van, először az ELF-értelmező belépési pontja következik.
6. A dinamikus linker szükség esetén betölti a függőségeket, elvégzi az induláshoz szükséges áthelyezéseket, majd átadja a vezérlést a főprogram belépési pontjának.

A leképezés nem feltétlenül jelent azonnali fizikai beolvasást: a lapok egy része igény szerinti lapozással (*demand paging*) válhat ténylegesen elérhetővé.

PIE esetén az ELF-ben tárolt címekhez betöltési eltolás (*load bias*) társulhat. A futásidejű belépési cím ilyenkor az eltolás és az `e_entry` összege.

4.3.5.1 Hol van a `_start` határa?

A **`_start` szimbólumnév**, nem az ELF-betöltő által kötelezően keresett különleges karakterlánc. A linker a választott szimbólum címét beírja az `e_entry` mezőbe. Futtatáskor már ez a cím számít.

A belépés nem szokásos AAPCS64 függvényhívás: nem egy A64 `BL` utasítás hívta meg a `_start` kódját. Az A64 `X30` ezért nem tekinthető használható visszatérési címnek. Saját belépési pontunkból rendszerhívással lépünk ki, nem A64 `RET` utasítással.

A betöltő nem hívja meg automatikusan a C `main` függvényét. A szokásos C-programban ezt a feladatot a futtatókörnyezet indulókódja készíti elő.

QEMU-különbség: x86_64 hoston a `qemu-aarch64` indításakor a host kernel magát a QEMU-programot indítja el. A vendég AArch64 ELF betöltését és az A64 utasítások végrehajtását a QEMU kezeli; a vendég rendszerhívásokat a host szolgáltatásaira közvetíti. Ez nem egy teljes AArch64 Linux-kernel emulációja.

4.3.6 Kidolgozott példa: a hello world ELF-szerkezetének vizsgálata

Munkakönyvtár: `~/arm-labor/a64/a012/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a012
cd ~/arm-labor/a64/a012
```

A következő, `lab-hello-elf.s` nevű fájl az előző program kismértékű bővítése. Az AArch64 ELF-szekciók közül a `.rodata` üzenetet, a `.data` kilépési állapotot, a `.bss` ellenőrizhető nullázott tárhelyet tartalmaz.

A teljes blokk kontextusa: A64 ISA, AArch64 Linux; a direktívák GNU assembler-direktívák AArch64 ELF-célhoz.

```
.section .text
.global _start
.type _start, %function

_start:
    ldr     x9, =lab_zero
```

```
    ldr     x10, [x9]
    cbnz    x10, lab_fail

    mov     x0, #1
    ldr     x1, =lab_message
    mov     x2, #lab_message_len
    mov     x8, #64
    svc     #0

    cmp     x0, #lab_message_len
    b.ne    lab_fail

    ldr     x9, =lab_exit_status
    ldr     w0, [x9]
    mov     x8, #93
    svc     #0

lab_fail:
    mov     x0, #1
    mov     x8, #93
    svc     #0

.size _start, . - _start
.ltorg

.section .rodata
lab_message:
    .ascii "Hello, ELF!\n"
.equ lab_message_len, . - lab_message

.section .data
.balign 4
lab_exit_status:
    .word 0

.section .bss
.balign 8
lab_zero:
    .skip 8

.section .note.GNU-stack,"",%progbits
```

A kód először ellenőrzi a `.bss` nyolc bájtjának nullázását. A kiírás után az A64 `CMP` és `B.NE` azt ellenőrzi, hogy a visszaadott bájtszám megegyezik-e az üzenet hosszával. Rövid írást hibának tekintünk; újrapróbálkozó I/O-ciklust itt nem építünk.

A használt konstansok kódolhatók egyetlen A64 `MOV` alias utasítással, 16 bites, eltolás nélküli `MOVZ` formában. Az A64 `CMP` immediate-változatának 12 bites, opcionálisan 12 bittel balra tolt konstansába a 12 bájtos hossz belefér; az A64 `SVC` immediate-mezője 16 bites. Az A64 `LDR =szimbólum` GNU pszeudoutasítás itt közeli literálkészletből tölt címet. A literálbetöltés, valamint az itt használt feltételes ágak előjeles, 19 bites, néggyel skálázott relatív eltolása elegendő ehhez a rövid kódhoz. A részletes konstanskezelést a MOV-variánsok tárgyalja.

4.3.6.1 Összeállítás, linkelés és futtatás

Cél: `qemu-user`, AArch64 Linux. A parancsok az Ubuntu x86_64 hoston futnak.

```
aarch64-linux-gnu-as -march=armv8-a \
    -o lab-hello-elf.o lab-hello-elf.s
```

```
aarch64-linux-gnu-ld -static -z separate-code -e _start \
    -o lab-hello-elf lab-hello-elf.o

qemu-aarch64 ./lab-hello-elf > lab-hello-elf.out
lab_status=$?

cat lab-hello-elf.out
printf 'exit=%s\n' "$lab_status"

printf 'Hello, ELF!\n' | cmp - lab-hello-elf.out
printf 'cmp=%s\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): egy `Hello, ELF!` sor, majd `exit=0` és `cmp=0`. A bájtpontos összehasonlítás a lezáró sortörést is ellenőrzi.

Az explicit `-e _start` választja ki a belépési szimbólumot. A `-z separate-code` a kód elkülönített betöltési elrendezését kéri; a mért kimenetben ez négy `LOAD` szegmenst ad.

4.3.6.2 A `readelf` kimenetének értelmezése

Vizsgálati cél: az előző `qemu-user` program; az eszközök a hoston futnak.

```
aarch64-linux-gnu-readelf -h lab-hello-elf.o
aarch64-linux-gnu-readelf -h lab-hello-elf
aarch64-linux-gnu-readelf -lW lab-hello-elf
aarch64-linux-gnu-readelf -SW lab-hello-elf
aarch64-linux-gnu-readelf -d lab-hello-elf
aarch64-linux-gnu-nm -n lab-hello-elf
aarch64-linux-gnu-objdump -d lab-hello-elf
```

A teljes kimenet helyett a következő ellenőrzési pontokra koncentráljunk:

Vizsgálat	Mért megfigyelés	Következtetés
Objektumfájl, `-h`	`REL`, `AArch64`	Még linkelendő AArch64 objektum
Végrehajtható fájl, `-h`	`ELF64`, little-endian, `EXEC`, `AArch64`	A választott profilnak megfelelő program
`Entry point address` és `nm`	A belépési cím egyezik `_start` címével	A linker a kívánt kezdőpontot választotta
`-lW`, `LOAD` sorok	Négy `LOAD` szegmens (R / R E / R / RW); az első csak fejléceket tartalmaz	Fájleltolások, virtuális címek, méretek, jogosultságok
`Section to Segment mapping`	Szekciók hozzárendelése szegmensekhez	Nem szükségszerű az egy-egy megfelelés
`-SW`, `.bss`	`NOBITS`, mérete 8 bájt	A tárhely tartalma nincs nyolc adatbájtként tárolva
Írható `LOAD`	`MemSiz > FileSiz`	Nullázandó memóriarész tartozik hozzá
`-lW`, `-d`	Nincs interpreter és dinamikus szakasz	Ehhez a programhoz nem kell dinamikus linker

A `Flg` oszlopban az `R` olvasást, a `W` írást, az `E` végrehajtást jelöl. A belépési címnek ebben a programban végrehajtható `PT_LOAD` tartományba kell esnie. A `-z separate-code` hatására a

kimenet négy `LOAD` szegmenst tartalmaz; az első csak az ELF- és programfejléceket képezi le (nem kódot), ezért a `Section to Segment mapping` megfelelő sora üres.

A címeket a saját fájlból olvassuk ki; nem használunk rögzített mintacímet. A linker alapértelmezett szkriptje és a toolchain változata az elrendezést befolyásolhatja.

4.3.7 Tipikus hibák és ellenpróbák

„Van `.text`, tehát futtatható.”
Hipotézis: az assembler `.o` kimenetét próbáltuk futtatni. Adatgyűjtés: `readelf -h` és `readelf -l`. Ha `ET_REL` fájlt látunk, a javítás a végső linkelési lépés, nem a fájlnév átírása.

„A `.bss` nem tárol adatbájtokat, ezért olvasni sem lehet.”
Ellenpróba: a laborprogram A64 `LDR` utasítása betölti a `lab_zero` értékét, és nem nulla esetén hibával kilép. Ez a nyolc bájt a memóriakép része, nem a fájl adatbájtjainak része.

„A `_start` név szükséges a betöltőnek.”
Adatgyűjtés: hasonlítsuk össze az `e_entry` értékét a szimbólumtábla címével. A betöltési szerződés a cím; a név a linkelésben segít. A futtatáshoz nem szükséges a normál `.symtab` szimbólumtábla megőrzése.

„Minden regiszter nulla a program elején.”
A nullázott `.bss` tartalmából nem következik az általános célú A64 regiszterek nullázottsága. A programnak minden felhasznált regisztert inicializálnia kell, vagy dokumentált indulási szerződésre kell támaszkodnia.

4.3.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért elegendő a betöltőnek a programfejléc-tábla a `.text`, `.data` és `.bss` szekciónevek ismerete nélkül? Miért nem ugyanaz az `e_entry` és az `e_phoff`?

2. (implementációs, középszint — cél: rutinépítés) Két külön kísérletben módosítsd a programot: - először csak a GNU assembler `.word 0` értékét változtasd `.word 7` értékre; - visszaállítás után csak a GNU assembler `.skip 8` méretét változtasd `.skip 4096` értékre.

Mindkét esetben építsd újra és futtasd a programot. Vizsgáld a kilépési állapotot, a szekcióméreteket és a `PT_LOAD` méreteit.

3. (hibakeresési, középszint — cél: rutinépítés) Készíts hibás belépésipont-választással külön binárist. **Cél: `qemu-user`; linkelés a hoston.**

```
aarch64-linux-gnu-ld -static -z separate-code -e lab_missing \
    -o lab-hello-elf-wrong lab-hello-elf.o

aarch64-linux-gnu-readelf -h lab-hello-elf-wrong
aarch64-linux-gnu-nm -n lab-hello-elf-wrong
aarch64-linux-gnu-objdump -d lab-hello-elf-wrong
```

Jegyezd fel a linker diagnosztikáját. Megbízható-e a bináris pusztán azért, mert esetleg ugyanazt a belépési címet kapta, mint a helyes változat? Egyetlen kapcsoló módosításával javítsd.

4.3.9 Megoldások

1. megoldás — Segítség. Nézd meg a `PT_LOAD` mezőtáblázatot — szerepel-e benne bárhol szekciónév? Az `e_entry` és az `e_phoff` közül melyik virtuális cím, melyik fájleltolás?

1. megoldás — Teljes megoldás. A `PT_LOAD` bejegyzések megadják a betöltendő fájltartományokat, virtuális címeket, memóriaméreteket és jogosultságokat. A nulla kitöltés a méretkülönbségből következik; ehhez nem kell felismerni a `.bss` nevet. Az `e_entry` végrehajtási cím, az `e_phoff` pedig egy fejléc-tábla fájlbeli eltolása.

2. megoldás — Segítség. A `.word 0`→`.word 7` csak a `.data` TARTALMÁT érinti, a méretét nem — mit vár ettől a kilépési állapot? A `.skip 8`→`.skip 4096` viszont a `.bss` MÉRETÉT növeli — ez `p_filesz`-t vagy `p_memsz`-t növeli inkább?

2. megoldás — Teljes megoldás. Az első változat mért kimenete változatlan, kilépési állapota 7. Az A64 `LDR W0` a 32 bites adatot tölti be, és az A64 `X0` felső 32 bitjét nullázza. A bájtösszehasonlítás továbbra is sikeres lehet: a helyes szöveg és a nulla kilépési állapot külön tulajdonság.

A második változatban a `.bss` mérete 4096 bájt, a kilépési állapot ismét 0. Az érintett szegmens memóriamérete nő, de nem kerül 4088 további nulla adatbájt a fájlba. A teljes fájlméret pontos változását nem jósoljuk meg pusztán ebből, mert egyéb elrendezési részletek is számítanak. A program csak az első nyolc bájtot ellenőrzi.

3. megoldás — Segítség. Ha a linker nem találja a kért szimbólumot, biztos, hogy hibával leáll, vagy választhat helyette valamilyen alapértelmezett címet? Mi történne, ha ez a cím véletlenül megegyezne a helyes `_start`-tal?

3. megoldás — Teljes megoldás. A GNU ld 2.42 a következő figyelmeztetést adja: `warning: cannot find entry symbol lab_missing; defaulting to 0000000000410000`. A linker **nem áll le hibával**: a kilépési állapota 0, csak figyelmeztet. A választott alapértelmezett cím ebben az elrendezésben egybeesik a `_start` címével, ezért a bináris lefut, és a kilépési állapota 0. A futási próba tehát önmagában elfedheti a hibát. A javítás `-e _start`; utána a diagnosztika megszűnik, és az `e_entry`–szimbólum egyezés is újra ellenőrizhető.

4.3.10 Rövid összefoglaló

Az ELF egyszerre szolgálja a linkelést és a programbetöltést, de ezek eltérő nézeteket használnak. A szekciók a kód és adat szervezését, a programfejlécek a futásidejű leképezést írják le. A `.bss` nulla kezdőértékű memóriát foglal anélkül, hogy tartalmát teljes egészében tárolnánk a fájlban. A linker választja ki a belépési címet; a betöltő nem a `_start` nevet keresi. Dinamikus programnál először az értelmező indulhat. A `readelf` és a szimbólumtábla összevetése ezért a futási próba fontos kiegészítője.

- **ELF-fejléc:** a fájlt és fő leíró tábláit azonosító szerkezet.
- **Szegmens:** a programfejléc-tábla által leírt futásidejű tartomány.
- **Betöltő:** a program memóriaképét és indulási környezetét létrehozó mechanizmus.

4.3.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Linux elf(5) — ELF-fejlécek, szekciók és szegmensek mezői. **Tartalmilag ellenőrizve**: a Linux man-pages projekt 6.19-es kiadása.
- Linux execve(2) — programkép-csere és interpreteres indítás. Ugyanaz a man-pages 6.19-es kiadás.
- GNU Binutils: readelf — az elemzéshez használt kapcsolók. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU ld: Setting the Entry Point — belépésipont-választás. Ugyanaz a binutils 2.47-es kiadás.
- QEMU User Mode Emulation — felhasználói módú emuláció. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát).

Nyitott ellenőrzések, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenet, a négy `LOAD` szegmens, az írható `LOAD` 4/12 bájtos mérete és a hibás belépési szimbólum diagnosztikája a lektor GNU as/ld 2.42 + QEMU 8.2.2 méréséből származik. Az Ubuntu 24.04 telepített binutils- és QEMU-verzióját, a linker-szkript helyi szegmenselrendezését és a laborprogram kimenetét a helyi laborban kell ellenőrizni.

4.4 Linux syscall-konvenció AArch64-en

4.4.1 Motiváció: a rendszerhívás nem közönséges függvényhívás

Egy fájl olvasásához a programnak olyan szolgáltatást kell kérnie, amelyhez önmagában néhány gépi utasítás nem elegendő: a Linux ellenőrzi az elérési utat, a jogosultságokat és a célpuffer hozzáférhetőségét. C-ben ezt többnyire könyvtári függvények mögé rejtjük. Assemblyben nekünk kell összeállítanunk a kérést, és értelmeznünk a választ.

A **rendszerhívási konvenció (system call convention)** meghatározza, hogyan adja át az alkalmazás a szolgáltatás azonosítóját és argumentumait, illetve hogyan kapja vissza az eredményt. Ez az operációs rendszer alkalmazásbináris interfészének (application binary interface, ABI) része, nem pusztán az utasításkészlet tulajdonsága.

Az első assembly program rendszerhívási mintáját most fájlmegnyitással, olvasással és hibakezeléssel egészítjük ki.

Hatókör: Linux/AArch64, A64 ISA, Armv8-A alapprofil, little-endian környezet. Az alfejezet valamennyi assembly utasítása és regisztere ebben a kontextusban értendő. A direktívák GNU `as`-direktívák, nem processzorutasítások. A labor egyszálú, libc nélküli programot használ.

4.4.2 A Linux/AArch64 regiszterszerződés

Szerep	Linux/AArch64, A64 regiszter vagy utasítás
Rendszerhívásszám	`X8`
Első argumentum	`X0`
Második argumentum	`X1`
Harmadik argumentum	`X2`

Szerep	Linux/AArch64, A64 regiszter vagy utasítás
Negyedik argumentum	`X3`
Ötödik argumentum	`X4`
Hatodik argumentum	`X5`
A kérés indítása	`SVC #0`
Nyers visszatérési érték	`X0`

Az argumentumok sorrendjét **az adott rendszerhívás kernelinterfésze** határozza meg. Nem kell minden alkalommal mind a hat argumentumregisztert feltölteni: például a `close` csak az elsőt használja.

Az A64 `SVC` utasítás 16 bites, előjel nélküli közvetlen operandust (*immediate*) kódol. A Linux/AArch64 szokásos hívássorozatában ez `0`; **nem ide kerül a rendszerhívásszám**, hanem az A64 `X8` regiszterbe. Az A64 `SVC` szinkron kivételt vált ki; natív Linuxon ennek nyomán a kernel kezeli a kérést. A `qemu-aarch64` user-mode emuláció ezt a felületet emulálja, nem teljes vendégkernelt futtat.

A mutatókat és a 64 bites méreteket az A64 `Xn` regiszterekben adjuk át. Egy cím A64 `Wn` nézetbe írása elveszítené a felső 32 bitet. Az egész argumentumok értelmezési szélességét ugyanakkor a rendszerhívás típusszerződése szabja meg: a fájlleíró például `int`, az olvasási méret `size_t`.

Fontos különbség a C-hez képest: az A64 `X0` először argumentumot, visszatéréskor már eredményt tartalmaz. Ha az eredeti argumentumra később is szükségünk lesz, előbb meg kell őriznünk.

4.4.3 Siker, negatív hibakód és az `errno`

A közvetlen Linux-rendszerhívás **nem állítja be a C-könyvtár `errno` változóját**. Hiba esetén az A64 `X0` a hibaszám negáltját, azaz `-errno` értéket tartalmazza. A szokásos libc-burkolófüggvény ebből állít elő `-1` visszatérési értéket és pozitív `errno` hibaszámot.

Művelet és állapot	A64 `X0`, előjeles olvasat	Jelentés
`read` sikeres	12	12 bájt beolvasva
`read` fájlvégen, pozitív kért mérettel	0	Nincs több adat
`openat` hibás, hiányzó útvonal	−2	`ENOENT`
`read` hibás fájlleíróval	−9	`EBADF`
`close` sikeres	0	A hívás sikerült

A fájlleíró (*file descriptor*) kis, nemnegatív egész azonosító. Sikeres megnyitáskor akár `0` is lehet; nem szabad feltételezni, hogy mindig `3`.

A nyers Linux-hibatartomány előjelesen `-4095` és `-1` közötti. Ugyanezek az értékek előjel nélkül a 64 bites tartomány legfelső 4095 értékét jelentik.

Az alábbi ellenőrzés ezt a pontos tartományt ismeri fel:

```
// A64 ISA, Linux/AArch64: x0 a nyers rendszerhívási eredmény.
cmn x0, #4095
b.cs lab_error
```

Az A64 `CMN` itt összeadást végez az állapotjelzők beállításához, de az összeget nem tárolja. Az A64 `B.CS` akkor ágazik el, ha az átviteljelző (*carry flag*) be van állítva.

Levezetés. Legyen a pozitív hibaszám E, ahol $1 \le E \le 4095$. A `-E` érték 64 bites kettes komplemens bitmintája előjel nélkül $2^{64} - E$. Ehhez 4095-öt adva:

$$(2^{64} - E) + 4095 \ge 2^{64}.$$

Ezért átvitel keletkezik a 64 bites eredményen túl. A változók itt mértékegység nélküli egész értékek. A hibatartományon kívüli bitmintáknál nincs ilyen átvitel.

Az A64 `CMN` ebben az immediate-változatban 12 bites, előjel nélküli mezőt használ, opcionális 12 bites balra tolással; a `4095` eltolás nélkül éppen kódolható.

A most használt hívásoknál egy előjeles „kisebb mint nulla” teszt is megfelelő lenne, mert sikeres eredményük nemnegatív. A pontos tartományteszt azonban általánosabb. A visszatérés utáni állapotjelzőkből önmagukban ne következtess hibára: **a Linux/AArch64 hibajelzése az A64 `X0` értéke**. Az állapotjelzőket mi képezzük belőle. Részletesebb hátterük az NZCV állapotjelzőkről szóló alfejezetben következik.

4.4.4 Gyakori hívások és a referencia helyes használata

Linux/AArch64 hívás	Szám az A64 `X8`-ban	Argumentumok az A64 regiszterekben	Sikeres eredmény
`openat`	56	`X0`: könyvtárleíró, `X1`: útvonal, `X2`: jelzők, `X3`: mód	Fájlleíró
`close`	57	`X0`: fájlleíró	0
`read`	63	`X0`: fájlleíró, `X1`: puffer, `X2`: méret	Beolvasott bájtok
`write`	64	`X0`: fájlleíró, `X1`: puffer, `X2`: méret	Kiírt bájtok
`exit`	93	`X0`: kilépési állapot	Nem tér vissza

A C-ben ismert `open()` nem jelenti azt, hogy minden architektúrán létezik azonos nevű nyers rendszerhívás. A Linux/AArch64 alapprofilban fájlt az `openat` hívással nyitunk meg; nincs külön hagyományos `open` rendszerhívásszám.

Az `openat` első argumentumaként az `AT_FDCWD`, vagyis `-100` azt jelenti, hogy a relatív útvonalat az aktuális munkakönyvtárhoz képest kell értelmezni. A laborban az `O_RDONLY` értéke `0`. Nem kérünk fájllétrehozást, ezért a módargumentum nem játszik szerepet; nullát adunk át.

A man7.org referenciáit három lépésben használd:

1. A syscall(2) architektúratáblázataiban keresd az `arm64` sort: ebből ellenőrizhető a belépési utasítás és a regiszterkiosztás.
2. Az adott hívás kézikönyvéből ellenőrizd az argumentumokat, a részleges teljesítés lehetőségét és a hibákat. Figyeld a libc és a kernel eltéréseit is.
3. A **számot a célarchitektúra Linux-fejléceiből** ellenőrizd, ne az x86_64 host számaiból.

A cross Linux-fejlécek szokásos Ubuntu-elrendezésén:

```
# Referenciaellenőrzés a qemu-user/AArch64 célhoz; nem build és nem futtatás.
grep -E '^#define __NR_(openat|close|read|write|exit)[[:space:]]' \
    /usr/aarch64-linux-gnu/include/asm-generic/unistd.h
```

A várt számok a fenti táblázat értékei. Ha a fájl hiányzik, előbb a célarchitektúra fejléceinek telepítését és helyét ellenőrizd; ne helyettesítsd automatikusan a host fejlécével.

4.4.5 Kidolgozott példa: egyszeri olvasás, ellenőrzött lezárás

Munkakönyvtár: `~/arm-labor/a64/a013/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a013
cd ~/arm-labor/a64/a013
```

A program megnyitja a `lab-input.txt` fájlt, legfeljebb 64 bájtot olvas, majd a kapott adatot a szabványos kimenetre írja. Ez **egyszeri olvasási demonstráció**, nem teljes fájlmásoló: sem a fájlvégig olvasást, sem a megszakított vagy részleges műveletek újrapróbálását nem valósítja meg.

A fájlleírót az A64 `X19`, a kilépési állapotot az A64 `W20`, az olvasott hosszt az A64 `X21` őrzi. Ezeket a Linux/AArch64 rendszerhívások megőrzik; itt nem közönséges függvényhívás történik.

Mentsd a következőt `lab-syscall.s` néven:

```
// A64 ISA, Armv8-A, Linux/AArch64, little-endian.
// GNU as direktívák; libc nélküli, egyszálú program.

.equ LAB_CAPACITY, 64

.section .text
.global _start
.type _start, %function
_start:
    // openat(AT_FDCWD, "lab-input.txt", O_RDONLY, 0)
    movn x0, #99                    // ~99 = -100, 64 biten
    ldr  x1, =lab_path
    movz x2, #0
    movz x3, #0
    movz x8, #56
    svc  #0

lab_after_open:
    cmn  x0, #4095
    b.cs .Lopen_error
    mov  x19, x0
    movz w20, #0

    // read(fd, buffer, 64)
    mov  x0, x19
    ldr  x1, =lab_buffer
    movz x2, #LAB_CAPACITY
    movz x8, #63
    svc  #0

lab_after_read:
    cmn  x0, #4095
    b.cs .Lread_error
    cbz  x0, .Lclose
    mov  x21, x0
```

```
    // write(1, buffer, ténylegesen_beolvasott_hossz)
    movz x0, #1
    ldr  x1, =lab_buffer
    mov  x2, x21
    movz x8, #64
    svc  #0

lab_after_write:
    cmn  x0, #4095
    b.cs .Lwrite_error
    cmp  x0, x21
    b.ne .Lwrite_error            // Részleges írás is sikertelen laborfutás.
    b    .Lclose

.Lread_error:
    movz w20, #2
    b    .Lclose

.Lwrite_error:
    movz w20, #3

.Lclose:
    mov  x0, x19
    movz x8, #57
    svc  #0
    cmn  x0, #4095
    b.cc .Lexit
    cbnz w20, .Lexit              // Az első hiba maradjon meg.
    movz w20, #4
    b    .Lexit

.Lopen_error:
    movz w20, #1                  // Nincs sikeresen megnyitott fd.

.Lexit:
    mov  w0, w20
    movz x8, #93
    svc  #0

.Lstop:
    b .Lstop                     // Az exit rendesen nem tér vissza.

.size _start, .-_start
.ltorg

.section .rodata
lab_path:
    .asciz "lab-input.txt"

.section .bss
.balign 8
lab_buffer:
    .skip LAB_CAPACITY

.section .note.GNU-stack,"",%progbits
```

Az A64 `MOVZ` és `MOVN` 16 bites immediate-mezőt használ. A64 `Xn` célregiszternél az engedélyezett eltolás `0`, `16`, `32` vagy `48`, A64 `Wn` esetén `0` vagy `16`; itt mindenütt nulla. Az A64 `MOVN` a kialakított értéket a célregiszter teljes szélességén invertálja, ezért kapunk `-100` értéket.

Az A64 `CBZ` és `CBNZ` közvetlenül nullára, illetve nemnullára vizsgálja a regisztert. Az A64 `CMP` után a `B.NE` eltérő értékeknél ágazik el. Az A64 `B.CC` az átvitel hiányát vizsgálja, tehát a használt hibateszt ellenkező ágát választja.

A címkés A64 `B` 26 bites, a használt feltételes elágazások 19 bites előjeles, néggyel skálázott relatív eltolást kódolnak. Ebben a rövid programban minden cél elérhető. Az A64 `LDR Xn, =cím` GNU assembler-pszeudoutasítás; a közeli, `.ltorg` által elhelyezett literálkészletből tölti be a címet.

4.4.5.1 Összeállítás, linkelés és futtatás

```
# Target: qemu-user, Linux/AArch64; host: Ubuntu 24.04 x86_64.
printf 'ARM syscall\n' > lab-input.txt

aarch64-linux-gnu-as -march=armv8-a -g \
    -o lab-syscall.o lab-syscall.s
aarch64-linux-gnu-ld -e _start \
    -o lab-syscall lab-syscall.o

qemu-aarch64 ./lab-syscall > lab-output.txt
lab_status=$?

printf 'Kilépési állapot: %d\n' "$lab_status"
cat lab-output.txt
cmp lab-input.txt lab-output.txt
printf 'Összehasonlítás állapota: %d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): a program kilépési állapota `0`, a kimenet `ARM syscall` és egy soremelés, az összehasonlítás állapota szintén `0`. A hiányzófájl-próba állapota `1`.

Ellenőrzési pont	Mért állapot (GDB 15.1, QEMU 8.2.2)
`lab_after_open`	`X0` nemnegatív fájlleíró
`lab_after_read`	`X0 = 12`; a puffer első 12 bájtja a fájl tartalma
`lab_after_write`	`X0 = 12`
Kilépés előtt	`W20 = 0`

A fájlolvasás nem fűz automatikusan nulla bájtot az adatok végére. A program ezért hosszal dolgozik, nem C-karakterláncként kezeli a puffert.

4.4.6 Hibák és ellenpróbák

A kilépési állapotok jelentése: `1` megnyitási, `2` olvasási, `3` írási vagy részleges írási, `4` lezárási hiba. Ezek **a program saját állapotkódjai**, nem Linux-hibaszámok.

Kontrollált hiányzófájl-próba:

```
# Target: qemu-user, Linux/AArch64; a már felépített programmal.
mv lab-input.txt lab-input.saved
qemu-aarch64 ./lab-syscall
lab_status=$?
mv lab-input.saved lab-input.txt
printf 'Várt állapot: 1; kapott: %d\n' "$lab_status"
```

A várt nyers eredmény a megnyitás után `-ENOENT`, vagyis `-2`. A program nem próbálja ezt fájlleíróként lezárni.

Hibakereséshez az egyik terminálban:

```
# Target: qemu-user, Linux/AArch64; GDB-kapcsolatra vár.
qemu-aarch64 -g 1234 ./lab-syscall
```

A másikban, helyreállított, 12 bájtos bemenettel:

```
# Debugger: AArch64 cél a qemu-user GDB-felületén.
gdb-multiarch -q ./lab-syscall \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_after_read' \
    -ex 'continue' \
    -ex 'p/d (long long)$x0' \
    -ex 'x/12bx &lab_buffer' \
    -ex 'continue' \
    -ex 'quit'
```

Mért érték: az A64 `X0` előjelesen `12`. A vizsgált memóriát szimbólum azonosítja, nem kézzel választott cím. A blokk záró `continue` parancsa a programot a kilépésig futtatja; a GDB `quit` parancsára a QEMU user-mode `QEMU: Terminated via GDBstub` üzenettel áll le, és a kilépési állapota `0` (mérve: QEMU 8.2.2, GDB 15.1).

> **Figyelem: töréspont és `SVC`.** Soha ne tegyél töréspontot magára az `SVC` utasításra, ha az utána következő állapotot akarod vizsgálni: a QEMU user-mode gdbstubja az `SVC`-n álló töréspontról adott `si`-nél két utasítást hajt végre. Itt a töréspont a `lab_after_read` címkén van, ezért az olvasási eredmény mérése megbízható.

Tünet	Hipotézis	Adatgyűjtés és egyetlen javítás
Óriási pozitív „bájtszám”	Negatív hiba előjel nélküli olvasata	A64 `X0` vizsgálata előjelesen; hibateszt a felhasználás elé
A kimenet a fájlon túli bájtokat is tartalmaz	A kért, nem a tényleges olvasási hossz került az íráshoz	A64 `X2` vizsgálata a `write` előtt; hossz átvétele az olvasási eredményből
Sikeres megnyitás után hibás olvasás	Elveszett vagy felülírt fájlleíró	A64 `X19` és az olvasás előtti `X0` összevetése
Eltűnik az olvasási hiba	A lezárás felülírta az eredményt	Hibautak követése; külön programállapot megőrzése

A `read` rövidebb eredménye önmagában nem hiba. A `write` is teljesíthet részlegesen; ezt a labor felismeri, de nem folytatja ciklussal. Linuxon a hibás `close` vak újrapróbálása sem helyes általános recept: a leíró rendszerint már felszabadult, ezért újrapróbálással más erőforrást is érinthetnénk.

4.4.7 Feladatok

1. **Magyarázó (alapszint — cél: megértés-ellenőrzés):** miért nem az A64 `SVC` operandusa a rendszerhívásszám? Mi a különbség a nyers `-9` eredmény és a libc `-1` eredménye között?
2. **Implementációs (alapszint — cél: rutinépítés):** módosítsd a `lab-syscall.s` pufferméretét és olvasási korlátját 8 bájtra. A fenti parancsokkal építsd újra és futtasd. Mi legyen a kimenet és a kilépési állapot?

3. **Hibakeresési (középszint — cél: rutinépítés):** az olvasás előtt az A64 `mov x0, x19` utasítást cseréld `movn x0, #0` utasításra. Újraépítés után állj meg a `lab_after_read` címkén. Jósolj nyers visszatérési értéket és programkilépési állapotot, majd ellenőrizd őket.

4.4.8 Megoldások

1. megoldás — Segítség. Gondold végig, melyik regiszter szerepe rögzített a kérés *kiválasztására*, és hogy a libc köztes rétege mit tesz a nyers kernel-hibakóddal, mielőtt a C programhoz visszaér.

1. megoldás — Teljes megoldás. A Linux/AArch64 szerződés az A64 `X8` regisztert rendeli a szolgáltatás azonosításához; az A64 `SVC #0` a kérésindítás mintája. A nyers `-9` közvetlenül az `EBADF` hibát jelenti. A libc ugyanebből általában `-1` visszatérési értéket és `errno = EBADF` állapotot készít. Közvetlen assembly hívásnál ez az átalakítás nem történik meg.

2. megoldás — Segítség. Keresd meg, hány helyen szerepel a pufferméret-konstans a forrásban — elég-e egyetlen definíciót módosítani, vagy több helyen van hardkódolva az érték?

2. megoldás — Teljes megoldás. Elég a `LAB_CAPACITY` értékét 8-ra változtatni, mert a foglalás és az olvasási kérés is ezt használja. A várt kimenet `ARM sysc`, soremelés nélkül; a kilépési állapot `0`. A program csak egy olvasást végez. A teljes eredeti fájllal végzett `cmp` ezért eltérést jelez, ami most nem programhiba.

3. megoldás — Segítség. Milyen 64 bites értéket ad a `movn x0, #0`, és fájlleíróként értelmezve ez érvényes-e? Ebből már következik, milyen hibát ad vissza a `read`.

3. megoldás — Teljes megoldás. Az A64 `movn x0, #0` minden bitet egyre állít, vagyis előjelesen `-1` értéket ad. Ez érvénytelen fájlleíró. A várt olvasási eredmény `-9`, azaz `-EBADF`; a hibateszt elágazik, és a program `2` állapottal lép ki. Az A64 `X19` továbbra is a ténylegesen megnyitott leírót tartalmazza, ezért a lezárási út azt tudja felszabadítani. Ellenőrzés után állítsd vissza az eredeti utasítást.

4.4.9 Rövid összefoglaló

A Linux/AArch64 rendszerhívás helyességét három együtt teljesülő feltétel adja: megfelelő szolgáltatásszám, megfelelő argumentumok és helyesen értelmezett visszatérés. Az A64 `X8` választ szolgáltatást, az A64 `X0-X5` argumentumot közvetít, az eredményt pedig az A64 `X0` adja. A nyers hibát még az eredmény felülírása előtt kell felismerni. A fájlkezelési példa külön kezeli a megnyitást, az olvasott hosszt és az erőforrás lezárását. A sikeres olvasás nem feltétlenül teljes olvasás, a sikeres írás pedig nem feltétlenül teljes írás. A labor kilépési kódjai nem azonosak a kernel hibaszámaival.

- **Nyers hibakód:** a rendszerhívás által visszaadott negatív hibaszám.
- **Fájlleíró:** folyamaton belüli, nemnegatív erőforrás-azonosító.
- **Részleges teljesítés:** a kértnél kevesebb bájt sikeres feldolgozása.

4.4.10 Források és ellenőrzési hiányok

Az alábbi hivatkozások dokumentációs kiindulópontok; közös állapotukat az Olvasói útmutató forrásállapot-táblája rögzíti. A fejezet mérései a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 fő útjáról származnak.

- Linux syscall(2): architektúrafüggő rendszerhívási konvenciók és a könyvtári felület.
- Linux open(2), openat(2): megnyitás, argumentumok, libc/kernel eltérések.
- Linux read(2) és write(2): bájtszám, részleges teljesítés és hibák.
- Linux close(2): lezárási hibák és az újrapróbálás veszélyei.
- GNU assembler kézikönyv: AArch64 utasításszintaxis és assembler-direktívák.

Nyitott ellenőrzési igény: a telepített Ubuntu-csomagok pontos verziója, a cross fejlécek helye, valamint a közölt mérési eredmények helyi laborvalidálása. A `qemu-aarch64` futtatás a host fájlrendszerét használja; nem elkülönített vendéggép és nem biztonsági homokozó.

5 Adatmozgatás, aritmetika és állapotjelzők

5.1 MOV-variánsok és immediate-kódolás

5.1.1 Miért nem egyetlen utasítás minden konstansbetöltés?

Egy C-programban a `uint64_t` változó inicializálása egy nagy konstanssal egyetlen forrássor. A fordító azonban több gépi utasítást is kibocsáthat hozzá. Ez akkor válik mérnöki kérdéssé, amikor disassemblyben keresünk egy értéket, kézzel írunk rövid inicializáló kódot, vagy megmagyarázzuk egy függvény kódméretét.

Az **utasításba kódolt közvetlen operandus** (*immediate operand*) olyan érték, amelyet maga az utasításkód tartalmaz. Nem memóriából olvassuk, és nem egy másik regiszter szolgáltatja.

Hatáskör: az alfejezet minden utasítása és regisztere az **A64 ISA, AArch64, I. rész** kontextusában értendő. Az assemblydirektívák a GNU assembler AArch64 céljához tartoznak; nem CPU-utasítások. A labor little-endian profilt használ. A konstansokat előjel nélküli bitmintaként kezeljük, kivéve, ahol külön előjeles értelmezést adunk.

Az előfeltétel a Linux syscall-konvenció AArch64-en: a labor végén az ott megismert kilépést használjuk.

5.1.2 A64 `MOV`: kényelmes név, többféle kódolás

Az A64 utasítások hossza rögzítetten 32 bit. Ebből az utasítás fajtáját, az operandusokat és az esetleges közvetlen értéket is ki kell olvasni.

Levezetés: egy tetszőleges 64 bites konstansnak 2^{64} lehetséges értéke van. Egy 32 bites utasításnak összesen csak 2^{32} különböző bitmintája lehet, ráadásul nem mind konstansbetöltés. Ezért egyetlen A64 utasítás nem kódolhatja közvetlenül az összes lehetséges 64 bites konstanst.

Az A64 `MOV` kényelmi, gyakran pszeudoutasításként emlegetett írásmód. Pontosabban az itt használt formái más utasítások **aliasnevei** (*instruction aliases*). Az assembler a megfelelő gépi kódolást választja ki.

A64 forrásalak	Mögöttes kódolás, illetve jelentés
`mov X4, X3`	`orr X4, XZR, X3`: regisztermásolás
`mov W4, W3`	`orr W4, WZR, W3`: 32 bites regisztermásolás
`mov X3, #0x1234`	Például `MOVZ`: közvetlen konstansbetöltés
`mov X3, #-1`	`MOVN` nulla közvetlen mezővel
`mov X3, #0x00ff00ff00ff00ff`	Logikai immediate-et használó `ORR`

Az A64 `XZR` és `WZR` olvasáskor nullát szolgáltató nullaregiszter (*zero register*). A nullával végzett bitenkénti VAGY változatlanul továbbadja a másik operandust, ezért alkalmas másolásra.

A veremmutatót tartalmazó A64 `MOV` alakok külön esetek: például a `mov X4, SP` mögött `ADD` áll nulla közvetlen operandussal. Ezekre nem szabad automatikusan ráhúzni az `ORR` magyarázatát.

Fontos: a GNU assembler AArch64 célján a `mov X3, #konstans` nem általános, több utasítást előállító konstansbetöltő makró. Ha nincs megfelelő egyutasításos kódolás, az assembler hibát jelez.

A logikai immediate kódolása sem tetszőleges. Röviden: a szó egy 2, 4, 8, 16, 32 vagy 64 bites blokk ismétléséből állhat; a blokk körbeforgatott, összefüggő egyesbitsorozatot és legalább egy nullát tartalmaz. A 32 bites alak legfeljebb 32 bites blokkot használ. Emiatt a fenti ismétlődő maszk kódolható, de egy tetszőleges hexadecimális érték általában nem. Részletesen a Logikai alapműveletek és bitmaszkolás tárgyalja ezt.

5.1.3 A64 `MOVZ`, `MOVK`, `MOVN`: építkezés 16 bites mezőkből

A széles közvetlenérték-mozgatások (*move wide immediate*) 16 bites közvetlen mezőt tartalmaznak. Ennek megengedett tartománya `0x0000…0xffff`.

A mező helyét az opcionális `lsl #s` módosító választja ki:

A64 célregiszter	Megengedett `s` értékek, bitben
`Wn`, 32 bites művelet	0, 16
`Xn`, 64 bites művelet	0, 16, 32, 48

Itt az `lsl` az operandus módosítója, **nem külön végrehajtott utasítás**. Elhagyása nulla eltolást jelent. A kódolás kétbites mezőválasztót használ; a 32 bites változatban a felső két választás nem megengedett.

Legyen `V` a közvetlen 16 bites mező `s` bittel balra tolva. A műveleti szélesség 32 vagy 64 bit.

A64 utasítás	Hatás a műveleti szélességen belül
`MOVZ`	Beírja `V` értékét; minden más bit nulla lesz.
`MOVK`	Csak a kiválasztott 16 bites mezőt cseréli ki; a többi bit megmarad.
`MOVN`	A teljes szélességen bitenként invertálja `V` értékét.

A nevekben a `Z` a nullázásra (*zero*), a `K` a megtartásra (*keep*), az `N` a bitenkénti negálásra (*NOT*) utal.

Két fontos következmény:

- Az A64 `MOVK` eredménye függ a célregiszter korábbi tartalmától. Ismeretlen kezdőértékre építve nem kapunk megbízható konstanst.
- Az A64 `MOVN` nem csak a kiválasztott 16 bitet invertálja. A többi, előzőleg nulla bitből is egyes lesz.

Például:

```
// A64 ISA, AArch64: önálló szemléltető utasítások.
movn x5, #0
movn w6, #0
```

Az első eredménye az A64 `X5 = 0xffffffffffffffff`. Ez 64 bites előjeles, kettes komplemens értelmezésben -1.

A második eredménye az A64 `W6 = 0xffffffff`, de a teljes `X6 = 0x00000000ffffffff`. Ennek oka az általános A64 szabály: egy `Wn` célregiszter írása nullázza a hozzá tartozó `Xn` felső 32 bitjét.

Ez az A64 `MOVK Wn, ...` esetében is igaz. A „megtartás” csak a 32 bites műveleti szélesség érintetlen mezőire vonatkozik.

Az itt tárgyalt A64 mozgatások nem módosítják az NZCV állapotjelzőket.

5.1.4 Levezetés: tetszőleges 64 bites érték felépítése

Legyen `C` egy előjel nélküli, 64 bites, mértékegység nélküli egész. Bontsuk négy 16 bites mezőre:

$$C = h_0 + h_1\,2^{16} + h_2\,2^{32} + h_3\,2^{48}, \qquad 0 \le h_j \le 65535.$$

A `j` mezőindex értéke 0–3; az egyes eltolások bitszámot jelentenek. A mezők kinyerése C-szemlélettel: jobbra tolás, majd maszkolás `0xffff` értékkel.

A `0x0123456789abcdef` konstans felépítése:

Lépés	A64 utasítás	Várt `X3` tartalom
1	`movz X3, #0xcdef`	`0x000000000000cdef`
2	`movk X3, #0x89ab, lsl #16`	`0x0000000089abcdef`
3	`movk X3, #0x4567, lsl #32`	`0x0000456789abcdef`
4	`movk X3, #0x0123, lsl #48`	`0x0123456789abcdef`

Az első utasítás ismert alapállapotot hoz létre, a továbbiak különálló mezőket írnak át. Így legfeljebb négy utasítás elegendő bármely 64 bites konstanshoz.

Ez nem állítás minden konstans minimális utasításszámáról. A nulla mezőket nem kell külön beírni a `MOVZ` után, sok egyesbit esetén pedig a `MOVN` adhat jobb kezdőállapotot. Például az A64 `movn X3, #0xedcb` önmagában előállítja a `0xffffffffffff1234` bitmintát.

5.1.5 Kidolgozott példa: konstansok ellenőrzése QEMU-ban

Munkakönyvtár: `~/arm-labor/a64/a014/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a014
cd ~/arm-labor/a64/a014
```

A következő teljes program nem használ C-futtatókörnyezetet. A konstansok adatok, nem memóriacímek. A `lab_check` szimbólumnál debuggerrel ellenőrizhető az eredmény.

A fájl neve legyen `lab-mov.s`:

```
// A64 ISA, AArch64, I. rész; GNU as; little-endian cél.
.section .text
.p2align 2
.global _start
.global lab_check
.type _start, %function

_start:
    movz x3, #0xcdef
    movk x3, #0x89ab, lsl #16
    movk x3, #0x4567, lsl #32
    movk x3, #0x0123, lsl #48

    mov  x4, x3
    movn x5, #0
    movn w6, #0
    mov  w7, w4

    mov  x9, #0x00ff00ff00ff00ff
    ldr  x10, =0x0123456789abcdef

lab_check:
    mov x0, #0
    mov x8, #93
    svc #0

.size _start, .-_start
.ltorg
.section .note.GNU-stack,"",%progbits
```

A GNU as AArch64 `.p2align 2` direktívája $2^2 = 4$ bájtos igazítást kér. A szekció- és szimbólumdirektívák a tárgyfájlt írják le; nem végrehajtandó műveletek.

Build és futtatás — cél: qemu-user, AArch64 little-endian; host: Ubuntu 24.04 x86_64.

```
aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-mov.o lab-mov.s &&
aarch64-linux-gnu-ld -EL -e _start \
    -o lab-mov lab-mov.o &&
qemu-aarch64 ./lab-mov
printf 'Kilépési állapot: %d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): a program nem ír a szabványos kimenetre; a shell `Kilépési állapot: 0` sort ír ki. A nulla kilépés önmagában még nem igazolja a konstansok helyességét: a program nem végez önellenőrzést.

A tényleges kódolások megtekintése:

```
# Cél: a qemu-user számára készült AArch64 ELF statikus vizsgálata.
aarch64-linux-gnu-objdump -d -M no-aliases lab-mov
```

A `no-aliases` kapcsolóval a regisztermásolás mögötti A64 `ORR`, illetve az egyszerű immediate-mozgatások mögötti kódolás látható. A megjelenítés pontos formája binutils-verziótól függhet. A literálkészlet előtt egy `00000000 udf #0` sor is megjelenhet: ez 8 bájtos igazítási kitöltés a 64 bites literál előtt, amely nem hajtódik végre, és nem a program ismeretlen utasítása.

A regiszterellenőrzéshez az első terminálban:

```
# Cél: qemu-user, AArch64; várakozás GDB-kapcsolatra.
qemu-aarch64 -g 1234 ./lab-mov
```

A második terminálban:

```
# Cél: a fenti qemu-user folyamat A64 regisztereinek ellenőrzése.
gdb-multiarch -q ./lab-mov \
    -ex 'set architecture aarch64' \
    -ex 'target remote :1234' \
    -ex 'break lab_check' \
    -ex 'continue' \
    -ex 'info registers x3 x4 x5 x6 x7 x9 x10' \
    -ex 'continue' \
    -ex 'quit'
```

Mért állapot a töréspontnál (mérve: GDB 15.1, QEMU 8.2.2):

A64 regiszter	Mért bitminta	Ellenőrzött tulajdonság
`X3`	`0x0123456789abcdef`	Négymezős felépítés
`X4`	`0x0123456789abcdef`	Teljes regisztermásolás
`X5`	`0xffffffffffffffff`	64 bites `MOVN`
`X6`	`0x00000000ffffffff`	32 bites írás felsőfél-nullázása
`X7`	`0x0000000089abcdef`	`W` nézetű másolás
`X9`	`0x00ff00ff00ff00ff`	Logikai immediate
`X10`	`0x0123456789abcdef`	Assembler-segített betöltés

5.1.6 Az A64 `LDR =konstans` előzetese

A labor A64 `ldr X10, =0x0123456789abcdef` sora valódi assembler-pszeudoutasítás (*pseudo-instruction*). Az egyenlőségjeles alak nem gépi operandusformátum.

A GNU assembler AArch64 célján ez a következő elvet használja:

1. A konstans bekerül egy **konstanstárolóba** (*literal pool*).
2. Az assembler egy PC-relatív A64 `LDR` utasítást állít elő.
3. A processzor végrehajtáskor memóriából tölti be az értéket.

A GNU as AArch64 `.ltorg` direktívája az aktuális helyre bocsátja ki a függőben lévő konstanstárolót. A példában ez a kilépés után található, így a normál vezérlés nem fut bele az adatba.

A valódi A64 `LDR` literal kódolás előjeles, 19 bites mezőt használ, négyszeres skálázással: a bájteltolás -1048576 és 1048572 közötti, négybájtos lépésekben. Ezért a tárolónak elérhető távolságban kell lennie.

Az egyetlen forrássor tehát itt egy betöltőutasítást **és külön tárolt adatot** jelent. Nem következik belőle, hogy gyorsabb a `MOVZ/MOVK` sorozatnál. Utóbbi nem végez adatmemória-olvasást; előbbi igen. Teljesítményt ebből mérés nélkül nem állítunk, QEMU pedig nem garantál hardverhű ciklusszámot.

5.1.7 Hivatkozható összefoglaló: A64 immediate-korlátok

Az A64 minden utasításformája saját, rögzített szélességű mezőben tárolja a közvetlen operandust vagy eltolást. A könyv későbbi alfejezetei nem sorolják fel újra ezeket a korlátokat, hanem erre a táblázatra hivatkoznak, és csak a példára jellemző eltérést emelik ki. A részletes levezetést az utolsó oszlopban megadott alfejezet adja.

Utasításcsalád	Mező	Megengedett érték	Részletesen
`MOVZ`, `MOVN`, `MOVK` (és a `MOV #konstans` alias)	16 bit, előjel nélküli + `lsl #s`	`0…0xffff`; `s` = 0, 16 (`Wn`), illetve 0, 16, 32, 48 (`Xn`)	ez az alfejezet
`ADD`, `SUB`, `ADDS`, `SUBS`; aliasok: `CMP`, `CMN`	12 bit, előjel nélküli + opcionális `lsl #12`	`0…4095`, vagy ennek 4096-szorosa (`4096…16773120`, 4096-os lépésben)	Egész aritmetika
`AND`, `ORR`, `EOR`, `ANDS`; alias: `TST`, bitmaszkos `MOV`	logikai (bitmaszk-) immediate	ismétlődő, forgatott, összefüggő egyesbit-blokk; a csupa 0 és a csupa 1 nem kódolható	Logikai alapműveletek
`LSL`, `LSR`, `ASR`, `ROR` immediate; eltolt regiszteroperandus	eltolásszám	0–31 (`Wn`), 0–63 (`Xn`)	Shift és rotate
`UBFX`, `SBFX`, `BFI`	pozíció és szélesség, 6–6 bit	a mező a műveleti szélességen belül marad (`Wn`: 0–31)	Bitfield-utasítások
`CCMP`, `CCMN` immediate	5 bit + 4 bites `#nzcv`	`0…31`	NZCV-összefoglaló
`MUL`, `MADD`, `SDIV`, `UDIV`, `CSEL`	nincs immediate-forma	a konstanst előbb regiszterbe kell tölteni	Egész aritmetika, CSET/CSEL
`LDR`/`STR` unsigned offset	12 bit, a hozzáférés méretével skálázva	`0…4095 × méret` bájt, a méret többszöröse (pl. `Wn`: 0–16380, `Xn`: 0–32760)	Load/store címzési módok
`LDUR`/`STUR`; pre- és post-index	9 bit, előjeles, skálázatlan	`-256…255` bájt	Load/store címzési módok
`LDP`/`STP`	7 bit, előjeles, skálázva	`Wn`: `-256…252`, `Xn`: `-512…504` bájt	Load/store címzési módok
`LDR` literal (az `LDR =konstans` mögött is)	19 bit, előjeles, ×4	`-1048576…1048572` bájt (±1 MiB)	ez az alfejezet
`ADR`	21 bit, előjeles, bájtos	`-1048576…1048575` bájt (±1 MiB)	Load/store címzési módok
`ADRP`	21 bit, előjeles, 4 KiB-os lapokban	±4 GiB; az alsó 12 bitet `ADD …, :lo12:` adja	Load/store címzési módok
`B`, `BL`	26 bit, előjeles, ×4	±128 MiB	Elágazás
`B.cond`, `CBZ`, `CBNZ`	19 bit, előjeles, ×4	±1 MiB	Elágazás
`TBZ`, `TBNZ`	14 bit, előjeles, ×4; bitszám 6 bit	±32 KiB; bitszám 0–63	Elágazás
`SVC`	16 bit, előjel nélküli	`0…65535`; Linuxon mindig `#0`, a hívásszám az `X8`-ban van	Linux syscall-konvenció

Ha egy konstans vagy eltolás nem fér bele a mezőbe, az assembler hibát jelez (vagy — `LDR`/`STR` esetén — más kódolást, például `LDUR`-t választhat). A megoldás ilyenkor a konstans regiszterbe töltése (`MOVZ`/`MOVK` sorozat vagy `LDR =konstans`), illetve a cím előzetes kiszámítása.

5.1.8 Tipikus hibák és ellenpróbák

„A `MOV` nem fogadja el a számot.” Először azt vizsgáljuk, belefér-e egyetlen támogatott kódolásba. Kontrollált negatív próba:

```
# Cél: AArch64/qemu-user labor; szándékos assemblerhiba, nincs futtatás.
printf '%s\n' '.text' 'mov X3, #0x0123456789abcdef' \
    > lab-mov-bad.s
aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-mov-bad.o lab-mov-bad.s
printf 'Assembler kilépési állapota: %d\n' "$?"
```

A GNU as 2.42 a következő üzenetet adja: `Error: immediate cannot be moved by a single instruction`, a kilépési állapota pedig `1` (mérve). Linkelésre és futtatásra ennél a negatív próbánál nem kerül sor.

Eltűnik egy korábban beírt mező. Hipotézis: `MOVK` helyett újabb `MOVZ` szerepel. GDB-ben utasításonként léptessünk, és keressük meg az első hibás regiszterállapotot. Csak ezt az utasítást javítsuk, majd ismételjük meg az ellenőrzést.

Nulla lesz a felső 32 bit. Keressünk `Wn` célregiszteres írást. A `MOVK Wn` sem őrzi meg az `Xn` felső felét.

Érvénytelen eltolás. A 16 bites mező nem helyezhető tetszőleges bitpozícióra: például az A64 `lsl #8` nem megengedett ezeknél az utasításoknál. A `Wn` alakhoz a `lsl #32` sem használható.

5.1.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért nem megfelelő magyarázat, hogy „a regiszter 64 bites, tehát a `MOV` bármely 64 bites számot elfogad”? Add meg az A64 `movn X2, #0x1234, lsl #16` eredményét is.

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a labort a `lab_check` előtt. Az A64 `X11` kapja a `0xfedcba9876543210` értéket `MOVZ`/`MOVK` segítségével, az `X12` pedig a `0xffffffffffff1234` értéket egyetlen utasítással. Ellenőrizd mindkettőt GDB-ben.

3. (hibakeresési, középszint — cél: rutinépítés) A labor második A64 utasításában cseréld a `MOVK` mnemonicot `MOVZ`-re, a többi operandust hagyd változatlanul. Jósolj végső `X3` értéket, majd keresd meg az első eltérést utasításonkénti léptetéssel. Egyetlen módosítással javítsd a hibát.

5.1.10 Megoldások

1. megoldás — Segítség. A regiszter szélessége és az UTASÍTÁSKÓDBAN tárolható mező mérete két külön korlát — melyik szabja meg, mit tud egyetlen `MOVN` ténylegesen beírni? A megadott `MOVN` előbb egy 16 bites mezőt helyez el eltolva, majd a TELJES 64 bitet invertálja.

1. megoldás — Teljes megoldás. A regiszter szélessége a tárolható értéket határozza meg, nem az utasítás közvetlen mezőjének méretét. A megadott A64 `MOVN` először a `0x0000000012340000` mintát képezi, majd 64 biten invertálja. Eredménye `0xffffffffedcbffff`.

2. megoldás — Segítség. Az `X11`-hez négy `MOVZ`/`MOVK` lépés kell (ugyanaz a minta, mint a fejezet 0x0123456789abcdef példájában). Az `X12`-nél nézd meg: a `0xffffffffffff1234` mintázat egyetlen `MOVN` bitenkénti negáltjaként kódolható-e.

2. megoldás — Teljes megoldás. A laborba illeszthető A64 kód:

```
// A64 ISA, AArch64: beszúrás a lab_check címke elé.
movz x11, #0x3210
movk x11, #0x7654, lsl #16
movk x11, #0xba98, lsl #32
movk x11, #0xfedc, lsl #48

movn x12, #0xedcb
```

Az első sorozat minden 16 bites mezőt kitölt. A második a `0xedcb` bitenkénti komplementereként `0x1234` alsó mezőt és csupa egyes felső biteket állít elő. Ugyanazzal a builddel futtatható; a GDB regiszterlistájához add hozzá az `x11 x12` neveket.

3. megoldás — Segítség. A `MOVZ` mindig NULLÁZZA a teljes műveleti szélességet a beírt mezőn kívül — mi történik emiatt az ELŐZŐ utasítás által már beírt mezővel, ha a másodikat is `MOVZ`-re cseréled?

3. megoldás — Teljes megoldás. A végső A64 `X3 = 0x0123456789ab0000`. A második utasítás lenullázza az első által beírt `0xcdef` mezőt. A GDB-ben `_start` töréspontól két `stepi` után már `0x0000000089ab0000` látható az elvárt `0x0000000089abcdef` helyett. A javítás kizárólag a második mnemonic visszaállítása `MOVK`-re; utána új build és azonos ellenőrzés szükséges.

5.1.11 Rövid összefoglaló

Az A64 konstansbetöltésnél külön kell választani a regiszter szélességét és az utasításban rendelkezésre álló kódolómezőket. A `MOV` kényelmes alias, de nem ígéret tetszőleges konstans egyutasításos előállítására. A `MOVZ` ismert, nullázott alapot ad, a `MOVK` kiválasztott mezőket cserél, a `MOVN` teljes műveleti szélességen invertál. Tetszőleges 64 bites érték legfeljebb négy ilyen utasítással felépíthető. A `Wn` célregiszteres írás mindig nullázza az `Xn` felső felét. Az `LDR =konstans` kényelme mögött külön tárolt adat és memóriaolvasás áll; működését disassemblyvel és regiszterellenőrzéssel igazoljuk. Az A64 immediate-mezők korlátait az alfejezet hivatkozható összefoglaló táblázata gyűjti egybe; a könyv későbbi alfejezetei erre utalnak vissza.

- **Immediate:** az utasításkódban tárolt közvetlen operandus.
- **Alias:** más utasításkódoláshoz rendelt kényelmi írásmód.
- **Literal pool:** betöltőutasítással elért, assembler által elhelyezett konstansadatok.

5.1.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — a DDI0487 referencia megfelelő `MOV`, `MOVZ`, `MOVK`, `MOVN`, `ORR` és `LDR` leírásaihoz. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- GNU assembler kézikönyv — AArch64 operandusformák, pszeudoutasítások és `.ltorg`. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU objdump kézikönyv — disassembly és architektúrafüggő megjelenítési kapcsolók. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt regiszterértékek és az assembler-üzenet a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). Az Ubuntu 24.04 telepített csomagverzióit és az objdump megjelenítését a hallgatói laborban kell ellenőrizni.

5.2 Egész aritmetika: ADD/SUB/MUL/DIV és a kettes komplemens viselkedése

5.2.1 Mérnöki feladat: egy C-kifejezés gépi műveletei

Egy adatfeldolgozó modulban a következő értéket kell kiszámítani:

$$(a + 7) \cdot b - c.$$

C-ben ez egyetlen kifejezés. Assemblyben nekünk kell kijelölni a részeredmények helyét, kiválasztani a műveleti szélességet, és eldönteni, hogyan értelmezzük a túlcsorduló eredményt. Ugyanilyen fontos kérdés: mi történik, ha később az eredményt negatív számmal vagy nullával osztjuk?

Hatókör: az alfejezet minden utasítása és regisztere az **A64 ISA, AArch64, Armv8-A alapprofil** környezetéhez tartozik. Az assemblyblokkok GNU assembler-szintaxist használnak; direktíváik ebben az AArch64-eszközláncban értendők. A labor little-endian Linux-program, célja `qemu-aarch64` user-mode emuláció. Cortex-M-utasítás itt nem szerepel.

Előfeltétel a MOV-variánsok és immediate-kódolás ismerete. Most nem az állapotjelzőket vizsgáljuk: az itt használt `ADD`, `SUB`, `MUL`, `UDIV` és `SDIV` nem módosítja az `NZCV` állapotjelzőket. Ezekkel Az NZCV állapotjelzők és eredetük foglalkozik.

5.2.2 Definíció: szélességhez kötött, moduláris aritmetika

Moduláris aritmetika (*modular arithmetic*) alatt itt azt értjük, hogy egy n bites eredményregiszter csak a matematikai eredmény alsó n bitjét őrzi meg. Az összeadás ezért:

$$r = (a + b) \bmod 2^n,$$

ahol n a műveleti szélesség bitben, a, b és r mértékegység nélküli egész számok. A64-en az itt tárgyalt egész műveleteknél $n = 32$ vagy $n = 64$.

A kettes komplemens (*two's complement*) ábrázolásban egy negatív x szám n bites kódja $2^n + x$. Ez ugyanabba a maradékosztályba tartozik, mint x. Innen következik, hogy az összeadás áramköri megvalósításának nem kell tudnia az előjelről.

5.2.2.1 Levezetés: miért elég ugyanaz az összeadás?

Legyen egy szemléltető, **8 bites** példában az egyik operandus -7, a másik 10. Ez nem külön 8 bites A64-utasítás, csak rövid bitszemléltetés.

Bitminta	Előjel nélküli olvasat	Előjeles olvasat
`0xf9`	249	-7
`0x0a`	10	10
Eredmény: `0x03`	3	3

A bitminták összege:

$$\texttt{0xf9} + \texttt{0x0a} = \texttt{0x103}.$$

Az alsó 8 bit `0x03`. Előjel nélkül ez $(249 + 10) \bmod 256 = 3$; előjelesen $-7 + 10 = 3$.

Általánosan egy negatív operandus kódjához hozzáadott 2^n a végső modulo műveletben eltűnik. **Ugyanaz az `ADD` utasításkód és ugyanazok az operandusbitminták ugyanazt az eredményt adják, függetlenül az előjeles vagy előjel nélküli értelmezéstől.**

A kivonás hasonlóan:

$$r = (a - b) \bmod 2^n.$$

Bitszinten ez megvalósítható $a + \sim b + 1$ összeadásként, ahol a bitenkénti invertálást n bitre értjük.

Ez azonban nem teszi azonossá az előjeles és előjel nélküli **tartományellenőrzést**. Nyolcbites szemléltetésben `0x7f + 1 = 0x80`: előjel nélkül 128, előjelesen -128, miközben a matematikai összeg 128.

Kapcsolat C-vel: a `uint64_t` aritmetika modulo 2^{64} működik. Az előjeles C-összeadás, -kivonás és -szorzás tartománytúllépése viszont nem definiált viselkedés (*undefined behavior*). Az assembly kiszámítható körbefordulása nem jogosít fel arra, hogy ugyanezt egy túlcsorduló előjeles C-kifejezéstől is elvárjuk.

5.2.3 Az A64 `ADD` és `SUB` operandusformái

A célregiszter az első operandus; a másik kettő a bemenet:

```
// A64 ISA, AArch64: egymástól független szintaktikai példák.
add x3, x0, x1       // x3 = x0 + x1, modulo 2^64
sub w3, w0, w1       // w3 = w0 - w1, modulo 2^32
add x4, x0, #7
sub x5, x4, #57
```

A `Wn` és `Xn` szélességeket egy ilyen utasításon belül nem keverjük. A `W3` írása az egész `X3` felső 32 bitjét nullázza; nem végez előjelkiterjesztést.

5.2.3.1 Közvetlen operandus: 12 bit és opcionális eltolás

Az `ADD`/`SUB` közvetlen operandusa (*immediate operand*) nem tetszőleges egész szám. Kódolása:

$$k = \mathrm{imm12} \cdot 2^s, \qquad 0 \le \mathrm{imm12} \le 4095, \qquad s \in \{0, 12\}.$$

A konstans és az eltolás mértékegység nélküli; az eltolás mértéke bit.

```
// A64 ISA, AArch64: minden sor külön példa.
add x3, x0, #4095
add x3, x0, #1, lsl #12       // hozzáadott érték: 4096
sub x3, x0, #4095, lsl #12    // kivont érték: 16773120
```

A GNU as az `add x3, x0, #4096` írásmódot is elfogadja, és automatikusan `#1, lsl #12` alakra kódolja; ez assembler-kényelmi átírás, nem új immediate-forma.

Az ábrázolható értékek tehát:

- 0–4095 között minden egész;
- ezenkívül a 4096 többszörösei, legfeljebb `0xfff000` értékig.

Nem egy összefüggő, 24 bites konstansmezőről van szó. Az 5000 például nem fér el ebben az immediate-formában. Betölthetjük külön regiszterbe, vagy felbonthatjuk 4096 és 904 összegére.

Negatív immediate helyett a szándékot egyértelműen az ellenkező művelettel írjuk: 7 hozzáadása helyett −7 hozzáadása legyen `SUB ..., #7`. Egyes assembler-átírások elfogadása nem változtatja meg az ISA kódolási korlátját.

5.2.3.2 Eltolt regiszteroperandus

A regiszteres forma második forrása eltolható:

```
// A64 ISA, AArch64
add x3, x0, x1, lsl #3       // x3 = x0 + (x1 << 3), modulo 2^64
```

Az itt tárgyalt *shifted register* forma `LSL`, `LSR` vagy `ASR` eltolást enged. Az eltolás 32 bites műveletnél 0–31, 64 bitesnél 0–63 bit. Ez **más kódolás**, mint az immediate-forma kizárólag 0 vagy 12 bites eltolása. Az eltolások részletes szemantikáját a Shift és rotate tárgyalja.

5.2.4 Szorzás és osztás: hol számít az előjel?

Az A64 alaputasítások:

A64-utasítás	Jelentés	Operanduskorlát
`MUL Xd, Xn, Xm`	A szorzat alsó 64 bitje	Csak regiszterek
`MUL Wd, Wn, Wm`	A szorzat alsó 32 bitje	Csak regiszterek
`UDIV Xd, Xn, Xm`	Előjel nélküli egész hányados	Csak regiszterek
`SDIV Xd, Xn, Xm`	Előjeles egész hányados	Csak regiszterek

Az osztásoknak is van `Wn` operandusú, 32 bites változatuk. Nincs ezekhez általános immediate-forma: a `MUL X3, X0, #9` nem érvényes utasítás. A 9-et előbb regiszterbe kell tölteni.

A `MUL` alsó n bites eredménye előjeles és előjel nélküli operandusokra is azonos. Ennek oka ismét a modulo 2^n aritmetika. Ez az állítás **nem** vonatkozik automatikusan a teljes, $2n$ bites szorzat felső felére. A `MUL` valójában a `MADD` nulla összeadandóval használt aliasa; használatához most elegendő a háromoperandusú alak.

Az osztás más: az előjeles értelmezés megváltoztatja a hányadost.

Csonkolás nulla felé (*truncation toward zero*): az `SDIV` a matematikai hányados törtrészét elhagyja, nem lefelé kerekít.

Művelet	A64-utasítás	Várt hányados
19 / 9, előjel nélkül	`UDIV`	2
−19 / 9, előjelesen	`SDIV`	−2
19 / −9, előjelesen	`SDIV`	−2
−19 / −9, előjelesen	`SDIV`	2

A hányados nem tartalmazza a maradékot. Ha szükséges, azt a $r = a - qb$ összefüggéssel, külön szorzással és kivonással képezhetjük.

Két fontos, **ISA által meghatározott** szélső eset:

- `UDIV` és `SDIV` nulla osztó mellett 0 eredményt ad; az utasítás nem vált ki egészosztási kivételt.
- Az `SDIV` legkisebb előjeles érték / -1 esetén a legkisebb előjeles érték bitmintáját adja vissza. Például 32 biten `0x80000000 / 0xffffffff` eredménye `0x80000000`.

C-ben a nullával osztás, illetve az adott előjeles eredménytípusban nem ábrázolható hányados nem definiált viselkedés. Ezeket ne használjuk C-referenciatesztként.

5.2.5 Kidolgozott példa: kézi fordítás és állapotellenőrzés

Munkakönyvtár: `~/arm-labor/a64/a015/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a015
cd ~/arm-labor/a64/a015
```

A számítás C-beli specifikációja legyen a következő, `lab-a015-ref.c` fájlban:

```
#include <stdint.h>

uint64_t lab_expr(uint64_t a, uint64_t b, uint64_t c)
{
    return (a + UINT64_C(7)) * b - c;
}
```

A bemeneteket a laborban közvetlenül regiszterekbe tesszük. Ez még nem C-ből hívható függvény, hanem önálló Linux-program.

Fájl: `lab-a015.s`.

```
// A64 ISA, Armv8-A; GNU as; Linux; little-endian.
.text
.global _start
.global lab_check
.type _start, %function

_start:
    MOVZ x0, #19
    MOVZ x1, #9
    MOVZ x2, #57

    add  x3, x0, #7
    mul  x4, x3, x1
    sub  x5, x4, x2

    udiv x6, x5, x1

    MOVN x7, #18          // ~18 = -19, 64 bites kettes komplemens
    sdiv x9, x7, x1

    MOVN w10, #6          // 0xfffffff9: előjelesen -7
    MOVZ w11, #10
    add  w12, w10, w11

    udiv x13, x5, XZR     // nulla osztó: eredmény 0
```

```
lab_check:
    mov  x0, x5           // Linux kilépési állapot: 177
    MOVZ x8, #93          // Linux AArch64 exit syscall
    svc  #0

.size _start, .-_start
.section .note.GNU-stack,"",%progbits
```

A használt `MOVZ`/`MOVN` és `SVC` immediate-ok mind kódolhatók (lásd az A64 immediate-korlátok összefoglalóját); a Linux-rendszerhívás száma az `X8` regiszterben van. Az `XZR` az A64 nullaregisztere.

A kézi leképezés függőségi lánca: `X3 = 26`, majd `X4 = 234`, végül `X5 = 177`. Nem kell és nem is szabad minden részeredményt memóriába kiírni csak azért, mert C-ben egy összetett kifejezésből indultunk.

5.2.5.1 Fordítás és futtatás

Az alábbi parancsok célja **AArch64 Linux, qemu-user**, Ubuntu 24.04 x86_64 hostról:

```
# A C-specifikáció külön fordításellenőrzése; nem része az executable-nek.
aarch64-linux-gnu-gcc -std=c11 -Wall -Wextra -O2 -march=armv8-a \
    -c lab-a015-ref.c -o lab-a015-ref.o

# Assembly -> objektum -> önálló ELF -> qemu-user.
aarch64-linux-gnu-as -g -EL -march=armv8-a \
    -o lab-a015.o lab-a015.s
aarch64-linux-gnu-ld -EL -e _start -o lab-a015 lab-a015.o
aarch64-linux-gnu-objdump -d lab-a015

qemu-aarch64 ./lab-a015
lab_status=$?
printf 'exit=%s\n' "$lab_status"
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): nincs programkimenet; a shell `exit=177` sort ír ki. Ez szándékos, nem hibát jelző laboreredmény. A shellben megfigyelhető kilépési kód csak az alsó 8 bit, ezért nagyobb eredmények ellenőrzésére debugger szükséges.

5.2.5.2 Regiszterek vizsgálata

Első terminál, **qemu-user cél**:

```
qemu-aarch64 -g 1234 ./lab-a015
```

Második terminál, ugyanennek az emulált AArch64-programnak a vizsgálata:

```
gdb-multiarch -q ./lab-a015 \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_check' \
    -ex 'continue' \
    -ex 'info registers x3 x4 x5 x6 x7 x9 x10 x12 x13' \
    -ex 'p/d $x9'
```

A `lab_check` címkénél, a kilépési kód beállítása előtt:

A64-regiszter	Mért bitminta (GDB 15.1, QEMU 8.2.2)	Értelmezés
`X3`	`0x1a`	26
`X4`	`0xea`	234
`X5`	`0xb1`	177
`X6`	`0x13`	Előjel nélküli hányados: 19
`X7`	`0xfffffffffffffed`	Előjelesen −19
`X9`	`0xfffffffffffffffe`	Előjelesen −2
`X10`	`0x00000000fffffff9`	A `W10` írása nullázta a felső felet
`X12`	`0x3`	32 bites összeadás eredménye: 3
`X13`	`0x0`	Nulla osztó eredménye

A vizsgálat után GDB-ben a `continue` futtatja le a kilépést; a QEMU-folyamat mért kilépési állapota ismét 177.

5.2.6 Tipikus hibák és ellenpróbák

- **„Az 5000 kis szám, mégsem fordul."** Hipotézis: az immediate nem kódolható. Ellenőrzés: hasonlítsuk össze a 12 bites mező szabályával; cseréljük két összeadásra, majd nézzük meg az `objdump` kimenetét.
- **Negatív osztandóból hatalmas pozitív hányados lesz.** Hipotézis: `SDIV` helyett `UDIV` szerepel. GDB-ben először hexadecimálisan vizsgáljuk meg a bemenetet, majd csak az osztási utasítást változtassuk.
- **A 32 bites −7 64 biten pozitívnak látszik.** A `W10` írása után `X10` nem −7. Ne keverjük a műveleti szélességeket; a labor szándékosan `W10`-zel számol.
- **Nullával osztáskor nem áll le a program.** Ez nem QEMU-hiba, hanem A64-szemantika. Ha a program szerződése tiltja a nulla osztót, külön ellenőrzés kell.
- **A végeredmény jó, de a részeredmény hibás.** A kilépési kód önmagában gyenge ellenőrzés. Álljunk meg a `lab_check` ponton, és hasonlítsuk össze a teljes regisztertáblát.

5.2.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) A 8 bites szemléltető modellben számítsd ki `0xff + 0x02` eredményét. Add meg mindkét operandus előjeles és előjel nélküli értelmezését, és magyarázd meg az azonos eredménybitmintát.

2. (implementációs, középszint — cél: rutinépítés) A labor `ADD X3, X0, #7` sorát helyettesítsd úgy, hogy `X3 = X0 + 5000` legyen. Adj meg egy új segédregisztert használó és egy segédregiszter nélküli változatot. Mennyi lesz `X5`, és mit mutat a shell kilépési kódja?

3. (hibakeresési, középszint — cél: rutinépítés) Cseréld a labor `SDIV` utasítását `UDIV`-re. A korábbi parancsokkal építsd újra és vizsgáld meg. Mely ellenőrzés jelzi a hibát, és miért nem jelzi a kilépési kód?

4. (szélsőérték-próba, középszint — cél: megértés-ellenőrzés) Egészítsd ki a labort a 32 bites legkisebb előjeles érték −1-gyel történő osztásával. A `lab_check` előtt ellenőrizd a hányadost. Miért nem megfelelő hozzá egy túlcsorduló `int32_t` C-kifejezés referenciaként?

5.2.8 Megoldások

1. megoldás — Segítség. Add össze a két bitmintát hexadecimálisan, majd nézd meg, mi marad a 8 biten belül — az eredmény bitmintája ugyanaz marad, csak az OLVASAT (előjeles/előjel nélküli) változik.

1. megoldás — Teljes megoldás. Az összeg `0x101`, amelyből 8 biten `0x01` marad. Előjel nélkül $(255 + 2) \bmod 256 = 1$; előjelesen $-1 + 2 = 1$. Az előjel az értelmezést változtatja meg, nem az összeadó bemeneti bitmintáit.

2. megoldás — Segítség. Nézd meg az immediate-kódolási szabályt: 5000 nem fér bele a 12 bites, 0/12 eltolású mezőbe egyben — bontsd fel a 4096 többszöröse és egy 0–4095 közötti maradék összegére.

2. megoldás — Teljes megoldás. Segédregiszterrel:

```
// A64 ISA, AArch64
MOVZ x14, #5000          // 16 bites immediate, eltolás nélkül
add  x3, x0, x14
```

Segédregiszter nélkül:

```
// A64 ISA, AArch64
add x3, x0, #1, lsl #12
add x3, x3, #904
```

Mindkettő 5019-et állít elő. Ezután `X5 = 5019*9-57 = 45114`, míg a shellben `45114 mod 256 = 58` jelenik meg. A két utasításos változatokból önmagában nem következik azonos hardveres futási idő.

3. megoldás — Segítség. Az `X7` bitmintáját `UDIV` nagy pozitív számként olvassa — mi történik egy ilyen nagyságú előjel nélküli osztással 9-cel, és ez hogyan jelenik meg `X9`-ben, de NEM jelenik meg a kilépési kódban (ami `X5`-ből jön)?

3. megoldás — Teljes megoldás. Az `X7` bitmintáját az `UDIV` nagy pozitív, előjel nélküli számként értelmezi. `X9` ezért nem `0xfffffffffffffffe` lesz. A GDB-s regiszterellenőrzés és az `objdump` utasításlistája kimutatja a hibát. A kilépési állapot változatlanul 177, mert azt `X5` adja, nem `X9`. Egyetlen javításként állítsuk vissza az `SDIV`-et, majd ismételjük meg az ellenőrzést.

4. megoldás — Segítség. A legkisebb előjeles 32 bites érték `0x80000000`; ennek `-1`-gyel osztott matematikai hányadosa nagyobb, mint a legnagyobb ábrázolható pozitív 32 bites előjeles szám — mire utal ez az ISA-szabály (lásd a fejezet “két fontos szélső eset” pontját)?

4. megoldás — Teljes megoldás. A `lab_check` címke elé illeszthető:

```
// A64 ISA, AArch64
MOVZ w14, #0x8000, lsl #16
MOVN w15, #0
sdiv w16, w14, w15
```

A `MOVZ` 16 bites mezője itt 16 bittel eltolt. A várt `W16` érték `0x80000000`; a teljes `X16` érték `0x0000000080000000`. Az előjeles matematikai hányados 2147483648, amely 32 bites előjeles eredményként nem ábrázolható. A megfelelő C-osztás emiatt nem definiált viselkedésű lenne, nem megbízható referencia.

5.2.9 Rövid összefoglaló

Az A64 egész aritmetikájában először a műveleti szélességet választjuk ki. Az összeadás, kivonás és az alsó szorzatbitet megtartó szorzás ugyanazzal az utasítással szolgálja az előjeles és előjel nélküli értelmezést. Osztáskor viszont külön `UDIV` és `SDIV` szükséges. Az `ADD`/`SUB` immediate-ja korlátozott: 12 bites mező, opcionálisan 12 bites balra tolással. A C-kifejezés kézi fordításánál a részeredményeket és az adatfüggőségeket követjük. A regiszterállapot erősebb ellenőrzés, mint a csonkolt kilépési kód. Az ISA meghatározott viselkedését mindig különválasztjuk a C nem definiált eseteitől.

- **Moduláris aritmetika:** eredményképzés modulo 2^n.
- **Kettes komplemens:** negatív x kódja n biten $2^n + x$.
- **Nulla felé csonkolás:** a hányados törtrészének elhagyása.

5.2.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487, az `ADD`, `SUB`, `MADD`/`MUL`, `UDIV` és `SDIV` leírásai. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 operandusformák és direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU objdump kézikönyv — az előállított utasítások vizsgálata. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenet és regiszterértékek a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A helyi Ubuntu 24.04 csomagverziókat, a parancsok elfogadását és a debugger megjelenítését a hallgatói laborban kell ellenőrizni; teljesítménymérést ez a gyakorlat nem tartalmaz.

5.3 Az NZCV állapotjelzők és eredetük

5.3.1 Miért kell az eredmény mellé négy további bit?

Egy előjel nélküli számláló növelésekor nemcsak az új érték érdekes, hanem az is, hogy a számláló átfordult-e nullára. Előjeles koordináták összeadásakor más kérdés merül fel: belefér-e a matematikai eredmény az ábrázolható tartományba? A két probléma ugyanahhoz az összeadóhoz kapcsolódik, de **nem ugyanazt a jelzést igényli**.

Az Egész aritmetika: ADD/SUB/MUL/DIV és a kettes komplemens viselkedése alfejezetben az eredmény alsó bitjeit vizsgáltuk. Most az eredmény mellett keletkező **állapotjelzőket (condition flags)** vezetjük le.

Architektúra és hatókör: az alfejezet valamennyi utasítása, regisztere és assembly példája **A64 ISA, AArch64, Armv8-A** környezetre vonatkozik. A direktívák GNU assemblerhez, AArch64 ELF-célhoz tartoznak; nem processzorutasítások. A labor little-endian Linux-program, `qemu-aarch64` user-mode céllal. Cortex-M-szabályokat itt nem használunk.

5.3.2 Az NZCV jelentése és a műveleti szélesség

Az A64 architekturális processzorállapotának, a **PSTATE**-nek négy jelzőjét együtt **NZCV** néven említjük. Egész összeadásnál és kivonásnál:

A64-jelző	Név	Jelentés
N	negatív, *negative*	A műveleti szélesség szerinti eredmény legfelső bitje
Z	nulla, *zero*	1, ha a teljes, adott szélességű eredmény nulla
C	átvitel, *carry*	Összeadásnál a legfelső biten túli átvitel; kivonásnál a kölcsönzés hiánya
V	előjeles túlcsordulás, *overflow*	1, ha a matematikai előjeles eredmény nem ábrázolható az adott szélességen

Az A64 `ADDS` és `SUBS` a számítás mellett **mind a négy jelzőt frissíti**. Az A64 `ADD` és `SUB` ezzel szemben változatlanul hagyja őket. Az `S` tehát itt nem „signed”, hanem a jelzőfrissítésre utal.

A műveleti szélességet az operandusok határozzák meg:

- A64 `Wn` operandusokkal 32 bites a művelet: az N jelző a 31. eredménybit.
- A64 `Xn` operandusokkal 64 bites a művelet: az N jelző a 63. eredménybit.

Az A64 `Wn` célregiszter írása továbbra is nullázza a hozzá tartozó `Xn` felső 32 bitjét. Ettől azonban az N még lehet 1: **a jelzők a műveleti szélességet követik, nem a nullakiterjesztett 64 bites tároló legfelső bitjét**.

A jelzők nem hordoznak C-típust. Ugyanaz az összeadás egyszerre képezi az előjel nélküli értelmezéshez használható C-t és az előjeles értelmezéshez használható V-t.

5.3.3 Levezetés: hogyan keletkezik a carry?

Legyen n a műveleti szélesség bitben, a és b pedig két n bites operandus előjel nélküli, mértékegység nélküli értéke. A matematikai összeg és a tárolt eredmény:

$$t = a + b, \qquad r = t \bmod 2^n.$$

Mivel $0 \le a, b \le 2^n - 1$, az összeg legfeljebb egy további bitet igényel:

$$C = \begin{cases} 1, & t \ge 2^n, \\ 0, & t < 2^n. \end{cases}$$

Ez **architekturális eredményszabály**, nem állítás arról, milyen kapuhálózatot használ egy konkrét processzor.

A bitszintű levezetéshez legyen a_i, b_i és r_i az operandusok, illetve az eredmény i-edik bitje. A c_i az adott bithelyre belépő átvitel; egyszerű összeadásnál $c_0 = 0$:

$$r_i = a_i \oplus b_i \oplus c_i,$$

$$c_{i+1} = (a_i \wedge b_i) \vee (a_i \wedge c_i) \vee (b_i \wedge c_i).$$

A kimenő átvitel akkor 1, ha a három bemeneti bit közül legalább kettő 1. A végső C jelző c_n.

Vizsgáljuk az alábbi **32 bites A64** műveletet:

- első operandus: `0xffffffff`, előjel nélkül 4294967295;
- második operandus: `0x00000001`, előjel nélkül 1.

Bithely	Első operandus bitje	Második operandus bitje	Belépő átvitel	Eredménybit	Kilépő átvitel
0	1	1	0	0	1
1–30, mindegyik külön	1	0	1	0	1
31	1	0	1	0	1

Az átvitel végighalad az összes bithelyen. A matematikai összeg `0x100000000`, de a 32 bites eredmény `0x00000000`. Ezért:

- N = 0;
- Z = 1;
- C = 1.

Előjeles kettes komplemens olvasatban viszont ugyanez $\mathbf{-1 + 1 = 0}$, ami ábrázolható. Így V = 0, a teljes NZCV pedig `0b0110`.

A carry nem azonos az előjeles túlcsordulással.

5.3.4 Levezetés: mikor keletkezik overflow?

n bites kettes komplemens ábrázolásban az előjeles tartomány:

$$-2^{n-1} \leq x \leq 2^{n-1} - 1.$$

Különböző előjelű számok összeadásakor az eredmény ebben a tartományban marad. Túlcsordulás akkor lehetséges, ha:

1. a két operandus előjele azonos;
2. a tárolt eredmény előjele ettől eltér.

Legyen s_a, s_b és s_r a két operandus, illetve az eredmény előjelbitje. Ekkor összeadásra:

$$V = \neg(s_a \oplus s_b) \wedge (s_a \oplus s_r).$$

Konkrét **32 bites A64** példa:

- `0x7fffffff`: előjelesen 2147483647;
- `0x00000001`: előjelesen 1;
- tárolt eredmény: `0x80000000`, előjelesen −2147483648.

A matematikai eredmény 2147483648 lenne, amely nem fér el előjeles 32 biten.

Bitszinten a 0. bithelyen átvitel keletkezik, amely végighalad az 1–30. bithelyeken. Az előjelbithez tehát $c_{31} = 1$ érkezik. Ott viszont:

$$0 + 0 + 1 = 1,$$

így az eredmény előjelbitje 1, a kilépő átvitel pedig $c_{32} = 0$. Következésképpen C = 0, miközben V = 1.

Az overflow másik, ezzel egyenértékű alakja:

$$V = c_{n-1} \oplus c_n.$$

Az indoklás az előjelbit összeadásából következik:

- Két 0 előjelbitnél túlcsordulás akkor van, ha 1 érkezik az előjelbithez; kifelé ekkor 0 távozik.
- Két 1 előjelbitnél túlcsordulás akkor van, ha 0 érkezik; kifelé ekkor 1 távozik.
- Eltérő előjelbiteknél a belépő és a kilépő átvitel azonos, ezért nincs overflow.

Az előző carry-példában $c_{31} = c_{32} = 1$, tehát V = 0. A mostani példában különböznek, ezért V = 1.

Az alábbi **elméleti, ISA-szemantikából levezetett**, 32 bites A64 eredmények mind a négy C/V-kombinációt megmutatják:

Operandusok bitmintája	Eredmény bitmintája	C	V	NZCV
`0x00000001 + 0x00000001`	`0x00000002`	0	0	`0b0000`
`0xffffffff + 0x00000001`	`0x00000000`	1	0	`0b0110`
`0x7fffffff + 0x00000001`	`0x80000000`	0	1	`0b1001`
`0x80000000 + 0x80000000`	`0x00000000`	1	1	`0b0111`

Az utolsó sor előjeles olvasata −2147483648 + −2147483648: a matematikai eredmény túl kicsi. Előjel nélkül ugyanakkor az összeg pontosan 2^{32}, ezért carry is keletkezik.

C-ben az előjel nélküli összeadás moduláris viselkedése meghatározott. A túlcsorduló előjeles összeadás viszont nem definiált viselkedés: egy C-programban nem szabad előbb túlcsorduló előjeles összeadást végezni, majd az eredmény előjeléből biztonságot ellenőrizni. Az ISA szabálya nem írja felül a C nyelv szabályát.

5.3.5 Kivonás és összehasonlítás: a C jelző itt nem „borrow”

Az A64 `SUBS` is frissíti az NZCV-t. Kivonáskor azonban C = 1 azt jelenti, hogy **nem kellett kölcsönözni (no borrow)**.

Ennek levezetéséhez az n bitre korlátozott bitenkénti komplementer értéke $2^n - 1 - b$. Az összeadóval végzett kivonás:

$$a + \mathrm{NOT}_n(b) + 1 = a + (2^n - 1 - b) + 1 = a + 2^n - b.$$

Ez akkor éri el 2^n értékét, ha $a \geq b$. Következésképpen, előjel nélküli olvasatban:

- C = 1, ha $a \geq b$;
- C = 0, ha $a < b$.

A 32 bites `0 - 1` eredménye `0xffffffff`, C = 0, V = 0. A `0 - 0` eredménye nulla, C = 1.

Kivonásnál az előjeles overflow feltétele:

$$V = (s_a \oplus s_b) \wedge (s_a \oplus s_r).$$

Itt eltérő előjelű operandusoknál lehet túlcsordulás. Az N önmagában ezért nem általános előjeles összehasonlítási eredmény; a feltételkódok ezt a V figyelembevételével kezelik majd a Feltételes végrehajtás: CSET/CSEL és feltételkódok alfejezetben.

Az A64 `CMP` **alias**, nem külön összehasonlító számítás:

```
// A64 ISA: a két sor azonos utasításkódolást jelent.
cmp  w1, w2
subs WZR, w1, w2
```

Az A64 `WZR` nullaregiszterbe írt eredmény elvész, de a jelzők frissülnek. A `CMP` nem módosítja az operandusait. Az A64 64 bites változatában `Xn` operandusok és `XZR` szerepelnek.

Immediate-korlát: az immediate-os `ADDS`/`SUBS` és a `CMP` ugyanazt a mezőt használja, mint az `ADD`/`SUB` (lásd az A64 immediate-korlátok összefoglalóját); tetszőleges 32 vagy 64 bites konstans közvetlenül nem írható beléjük.

5.3.6 Összefoglaló táblázat: mely utasítások írják az NZCV-t?

A könyv további alfejezetei erre a táblázatra hivatkoznak, ha az NZCV-t kell követni. Az Armv8-A alapprofilban a szabály egyszerű: **csak a kifejezetten jelzőállító utasítások írják az NZCV-t** — a legtöbb ilyen nevének `S` a végződése, illetve összehasonlító alias (`CMP`, `CMN`, `TST`). Minden más utasítás változatlanul hagyja, akkor is, ha az eredménye nulla vagy negatív.

Csoport	A64-utasítások	Hatás az NZCV-re
Aritmetika jelzőállítással	`ADDS`, `SUBS`, `ADCS`, `SBCS`, `NEGS`, `NGCS`; aliasok: `CMP` (= `SUBS` nullaregiszterbe), `CMN` (= `ADDS` nullaregiszterbe)	mind a négy jelzőt írja az eredmény szerint
Logika jelzőállítással	`ANDS`, `BICS`; alias: `TST` (= `ANDS` nullaregiszterbe)	N és Z az eredmény szerint; **C = 0, V = 0**

Csoport	A64-utasítások	Hatás az NZCV-re
Feltételes összehasonlítás	`CCMP`, `CCMN`	ha a feltétel teljesül, az összehasonlítás szerint; különben az utasításban megadott `#nzcv` értéket írja
Lebegőpontos összehasonlítás	`FCMP`, `FCMPE`	az összehasonlítás szerint (rendezetlen eredménynél `0011`); lásd a skaláris lebegőpontos utasításokat
Közvetlen írás	`MSR NZCV, Xn`	a regiszter 31–28. bitjét írja be
Nem írja	`ADD`, `SUB`, `ADC`, `SBC`, `NEG`, `MUL`, `MADD`, `MSUB`, `SDIV`, `UDIV`, `AND`, `ORR`, `EOR`, `BIC`, `MVN`, `LSL`/`LSR`/`ASR`/`ROR`, `MOV`/`MOVZ`/`MOVK`/`MOVN`, bitfield-utasítások, `LDR`/`STR` és társaik, `ADR`/`ADRP`, `B`, `BL`, `BR`, `BLR`, `RET`, `CBZ`/`CBNZ`, `TBZ`/`TBNZ`	változatlanul hagyja
Olvassa, de nem írja	`B.cond`, `CSEL`, `CSET`, `CSINC`, `CSINV`, `CSNEG`, `ADC`, `SBC` (a C jelzőt)	a jelzőkből dönt vagy számol, de nem módosítja őket

A táblázat az Armv8-A alapprofil utasításaira vonatkozik; a v8.4+ flag-manipuláló utasítások (`CFINV`, `RMIF`, `SETF8`/`SETF16`, `AXFLAG`/`XAFLAG`) nincsenek benne, mert azok szintén írják az NZCV-t.

Két következmény, amelyre a későbbi alfejezetek építenek:

- A `CBZ`/`CBNZ` és a `TBZ`/`TBNZ` jelzők **nélkül** teszteli a regisztert, ezért két NZCV-t használó utasítás közé is beilleszthető (Feltétel nélküli és feltételes elágazás).
- Egy `ADDS`/`SUBS` és a hozzá tartozó `B.cond` vagy `CSEL` közé tett nem jelzőállító utasítás (például `ADD`, `MOV`, `LDR`) nem rontja el a jelzőket; egy közbeiktatott `CMP` vagy `ANDS` viszont felülírja őket.

5.3.7 Kidolgozott labor: az NZCV pillanatfelvétele

Munkakönyvtár: `~/arm-labor/a64/a016/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a016
cd ~/arm-labor/a64/a016
```

A program minden vizsgált művelet után kiolvassa a jelzőket. Az A64 `MRS Xn, NZCV` az NZCV rendszerregiszteren keresztül másolja őket általános célú regiszterbe, és maga nem módosítja őket.

A64 `NZCV` kiolvasott bithelye	Tartalom
31	N
30	Z

A64 `NZCV` kiolvasott bithelye	Tartalom
29	C
28	V
Többi bit	0

Így például az NZCV = `0b0110` kiolvasott értéke `0x0000000060000000`, nem `0x6`. Ezek **regiszterértékek, nem memóriacímek**. GDB-ben ugyanez a `cpsr` pszeudoregiszter 31–28. bitjén látszik: az `info registers cpsr` például `0x60000000` értéket mutathat. A `cpsr` itt a PSTATE GDB-beli neve, nem külön A64 regiszter.

Mentsük a következő teljes forrást `lab-nzcv.s` néven:

```
// A64 ISA, Armv8-A; GNU as, AArch64 Linux ELF.
.text
.global _start
.global lab_inspect
.type _start, %function

_start:
    // A64 A: 0xffffffff + 1, 32 bit.
    MOVZ w0, #0xffff
    MOVK w0, #0xffff, lsl #16
    MOVZ w1, #1
    adds w2, w0, w1
    add  w9, w1, w1       // Nem modosítja az NZCV-t.
    mrs  x10, NZCV

    // A64 B: 0x7fffffff + 1, 32 bit.
    MOVZ w0, #0xffff
    MOVK w0, #0x7fff, lsl #16
    adds w3, w0, w1
    mrs  x11, NZCV

    // A64 C: 0x80000000 + 0x80000000, 32 bit.
    MOVZ w0, #0x8000, lsl #16
    adds w4, w0, w0
    mrs  x12, NZCV

    // A64 D: 1 + 1, 32 bit.
    adds w5, w1, w1
    mrs  x13, NZCV

    // A64 E: 0 - 1, 32 bit.
    subs w6, WZR, w1
    mrs  x14, NZCV

    // A64 F: osszehasonlitas; w1 erteke nem valtozik.
    cmp  w1, w1
    mrs  x15, NZCV

lab_inspect:
    // A64 Linux exit(0).
    MOVZ w0, #0
    MOVZ w8, #93
    svc  #0

.size _start, .-_start
.section .note.GNU-stack,"",%progbits
```

A konstansépítő `MOVZ/MOVK` és az `SVC #0` immediate-jai kódolhatók (lásd az A64 immediate-korlátok összefoglalóját).

Build és futtatás — target: AArch64 Linux, qemu-user; host: Ubuntu 24.04 x86_64.

```
aarch64-linux-gnu-as -g -march=armv8-a -EL \
  -o lab-nzcv.o lab-nzcv.s
aarch64-linux-gnu-ld -EL -e _start \
  -o lab-nzcv lab-nzcv.o

aarch64-linux-gnu-objdump -d -M no-aliases lab-nzcv

qemu-aarch64 ./lab-nzcv
printf 'exit=%d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): a program nem ír a szabványos kimenetre; a shell `exit=0` sort jelez. Ez csak a szabályos befejezést ellenőrzi, **nem a jelzők helyességét**.

Az aliasok nélküli disassemblyben az A64 `CMP w1, w1` helyén `subs wzr, w1, w1` alak látható (mérve: GNU objdump 2.42, `-M no-aliases`).

A regiszterellenőrzéshez az első terminálban indítsuk újra a qemu-user célprogramot:

```
qemu-aarch64 -g 1234 ./lab-nzcv
printf 'exit=%d\n' "$?"
```

Második terminálban csatlakozzunk:

```
gdb-multiarch -q ./lab-nzcv \
  -ex 'target remote localhost:1234' \
  -ex 'break lab_inspect' \
  -ex 'continue' \
  -ex 'info registers x2 x3 x4 x5 x6 x9' \
  -ex 'info registers x10 x11 x12 x13 x14 x15'
```

A töréspontban a következő **mért A64 állapot** (GDB 15.1, QEMU 8.2.2) ellenőrizendő:

Eset	Eredmény 32 bites nézetben	NZCV	Pillanatfelvétel
A	`W2 = 0x00000000`	`0b0110`	`X10 = 0x0000000060000000`
B	`W3 = 0x80000000`	`0b1001`	`X11 = 0x0000000090000000`
C	`W4 = 0x00000000`	`0b0111`	`X12 = 0x0000000070000000`
D	`W5 = 0x00000002`	`0b0000`	`X13 = 0x0000000000000000`
E	`W6 = 0xffffffff`	`0b1000`	`X14 = 0x0000000080000000`
F	Nincs tárolt kivonási eredmény	`0b0110`	`X15 = 0x0000000060000000`

Az A64 `X2`–`X6` felső 32 bitje nulla. Különösen: `X3 = 0x0000000080000000` mellett a B eset N jelzője mégis 1. Az A64 `W9 = 2`, de a közbeiktatott `ADD` nem írta felül az A eset jelzőit.

A GDB-ben kiadott `continue` után a program várt kilépési állapota 0.

5.3.8 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés és egyetlen változtatás
Az eredmény helyes, a jelző régi	A64 `ADD` szerepel `ADDS` helyett	`objdump` segítségével ellenőrizzük a kódot; csak a mnemonicot javítsuk
A várt 32 bites átfordulás elmarad	A64 `Xn` operandusokkal számoltunk	Ellenőrizzük az utasítás szélességét; csak a hibás összeadást módosítsuk
A kivonás C-je „fordítottnak” látszik	A kölcsönzést és annak hiányát összekevertük	Vessük össze a `0 - 1` és `0 - 0` esetet; a kód helyett először az elvárást javítsuk
Több eredményhez ugyanaz a jelző tartozik	A kiolvasás előtt egy későbbi művelet frissítette az NZCV-t	Helyezzük az A64 `MRS` utasítást közvetlenül a vizsgált művelet után

A jelzők **egyetlen közös, felülírható állapotot** alkotnak; nem kapcsolódik minden eredményregiszterhez külön NZCV. Hibakereséskor ezért egyszerre ellenőrizzük az operandusokat, a szélességet, az eredményt és a kiolvasás helyét.

5.3.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Határozd meg a 32 bites A64 `0x80000000 + 0xffffffff` eredményét és NZCV-jét. Külön indokold C és V értékét előjel nélküli, illetve előjeles értelmezéssel.

2. (implementációs, középszint — cél: megértés-ellenőrzés) A labor első összeadását módosítsd egyetlen sorban 64 bitesre, `ADDS X2, X0, X1` alakra. A konstansépítést hagyd változatlanul. A fenti qemu-user builddel és GDB-ellenőrzéssel vizsgáld meg `X2`, annak alsó 32 bites `W2` nézete, valamint `X10` értékét. Miért nem következik a nulla `W2` értékből Z = 1?

3. (hibakeresési, középszint — cél: rutinépítés) Az eredeti program B esetében szándékosan cseréld az A64 `ADDS W3, W0, W1` utasítást `ADD W3, W0, W1` alakra. Előre jelezd meg az eltérő regiszterértéket, majd `objdump` és GDB segítségével igazold a hipotézist. Egyetlen módosítással javítsd vissza.

5.3.10 Megoldások

1. megoldás — Segítség. Számítsd ki az előjel nélküli összeget binárisan (hány bitre nő), majd külön az előjeles összeget — fér-e a 32 bites előjeles tartományba?

1. megoldás — Teljes megoldás. Eredmény: `0x7fffffff`; NZCV: `0b0011`. Előjel nélkül az összeg `0x17fffffff`, ezért a 32. biten átvitel keletkezik: C = 1. Előjelesen $-2147483648 + -1$ adódik, ami kisebb a minimumértéknél: V = 1. A tárolt eredmény előjelbitje 0, és nem nulla, ezért N = 0, Z = 0.

2. megoldás — Segítség. Nézd meg, mit csinál a konstansépítő `MOVZ W0, .../MOVZ W1, ...` pár az `X0`/`X1` FELSŐ 32 bitjével, mielőtt egyáltalán az `ADDS`-hoz érnél.

2. megoldás — Teljes megoldás. `X2 = 0x0000000100000000`, `W2 = 0`, `X10 = 0`. A konstansépítés `W0` és `W1` írásával már nullázta az A64 `X0` és `X1` felső felét. A 64 bites összeadás ezért 4294967295 + 1 értéket számít, amely mindkét 64 bites értelmezésben ábrázolható. Nincs carry és overflow. Z a teljes 64 bites eredményt vizsgálja, amely nem nulla; az alsó 32 bites nézet önmagában nem döntő.

3. megoldás — Segítség. Ha az `ADD` nem frissíti az NZCV-t, melyik KORÁBBI eset jelzői maradnak érvényben az `MRS X11, NZCV` pillanatában?

3. megoldás — Teljes megoldás. Az A64 `W3` továbbra is `0x80000000`, de `X11 = 0x0000000060000000` lesz. A hibás `ADD` nem frissíti az NZCV-t, így a pillanatfelvétel az A eset `0b0110` jelzőit őrzi. A közbeeső konstansépítés sem változtatott rajtuk. Az `ADDS` visszaállítása után a várt `X11 = 0x0000000090000000`. A helyes eredmény tehát önmagában nem igazolja a helyes jelzőkezelést.

5.3.11 Rövid összefoglaló

Az A64 aritmetikában az eredmény és a jelzők együtt írják le, mi történt az adott műveleti szélességen. Az N az eredmény legfelső bitje, a Z a teljes eredmény nullaságát jelzi. Az összeadás C jelzője az előjel nélküli tartományból kilépést, V jelzője az előjeles tartomány megsértését mutatja; egyikből sem következik a másik. A kivonás C-je a kölcsönzés hiányát jelenti. Az A64 `CMP` eldobott eredményű `SUBS`, a jelzőket viszont megtartja. Az NZCV-t csak a jelzőállító utasítások írják (`…S` végződés, `CMP`, `CMN`, `TST`, `CCMP`, `FCMP`); a teljes áttekintést az alfejezet összefoglaló táblázata adja. A laborban közvetlen kiolvasással ellenőriztük a levezetett állapotokat.

- **Carry:** összeadáskor a műveleti szélességen túli átvitel.
- **Overflow:** előjelesen nem ábrázolható matematikai eredmény jelzése.
- **Jelzőfrissítés:** az NZCV felülírása az aktuális művelet alapján.

5.3.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; az `AddWithCarry` művelet, az `ADDS`, `SUBS`, `CMP`, valamint az `NZCV` leírása. *Ellenőrizve: a link él* (a `developer.arm.com` → `support.arm.com` átirányítás megerősített); a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 operandusok, immediate-formák és ELF-direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU objdump kézikönyv — disassembly és az aliasok megjelenítésének vezérlése. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt regiszterértékek és a disassembly-alak a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). Az Ubuntu 24.04 helyben telepített csomagverzióit, a disassembly konkrét megjelenését és a QEMU–GDB kapcsolat működését a laborban kell ellenőrizni. Teljesítményadatot nem közöltünk; a bitenkénti levezetés nem mikroarchitekturális időzítési modell.

5.4 Feltételes végrehajtás: CSET/CSEL és feltételkódok

5.4.1 Motiváció és hatókör

Egy minimumkeresésben vagy állapotérték előállításakor gyakori feladat: két érték összehasonlítása után vagy egy logikai eredményre, vagy a két érték egyikére van szükségünk. Ehhez nem feltétlenül kell megváltoztatni a program végrehajtási útvonalát. Az **elágazásmentes kiválasztás** (*branchless selection*) feltételtől függő adatot állít elő, miközben az utasítássor lineáris marad.

Az előfeltétel Az NZCV állapotjelzők és eredetük ismerete. Most azt vizsgáljuk, hogyan lesz az állapotjelzőkből C-szerű összehasonlítás, majd regiszterben tárolt eredmény.

Architektúra: az alfejezet valamennyi utasítása és regisztere **A64 ISA, AArch64, Armv8-A** környezethez tartozik. Az assemblydirektívák GNU assemblerhez, AArch64/ELF célhoz tartoznak. A labor little-endian profilt használ; Cortex-M/Thumb-2 utasítás nem szerepel benne.

5.4.2 Feltételkód: logikai állítás az NZCV-ről

A **feltételkód** (*condition code*) az aktuális NZCV állapotjelzőkből képzett logikai feltételt jelöl. Nem tárol összehasonlítási operandusokat, és nem emlékszik azok C-típusára.

Az A64 `CMP W1, W2` a 32 bites kivonás állapotjelzőit állítja elő. Ugyanezekből a jelzőkből választhatunk előjeles vagy előjel nélküli összehasonlítást. A választást a feltételkód végzi, nem a `CMP`.

Az alábbi táblázat összehasonlítási jelentése akkor érvényes, ha a jelzőket közvetlenül a megadott szélességű `CMP a, b` állította elő, és azóta nem íródtak felül.

A64-feltételkód	NZCV-feltétel	Jelentés `CMP a, b` után
`EQ`	$Z = 1$	`a == b`
`NE`	$Z = 0$	`a != b`
`CS`, más néven `HS`	$C = 1$	Előjel nélkül `a >= b`
`CC`, más néven `LO`	$C = 0$	Előjel nélkül `a < b`
`MI`	$N = 1$	A kivonási eredmény legfelső bitje 1
`PL`	$N = 0$	A kivonási eredmény legfelső bitje 0
`VS`	$V = 1$	Előjeles túlcsordulás történt
`VC`	$V = 0$	Nem történt előjeles túlcsordulás
`HI`	$C = 1$ és $Z = 0$	Előjel nélkül `a > b`
`LS`	$C = 0$ vagy $Z = 1$	Előjel nélkül `a <= b`
`GE`	$N = V$	Előjelesen `a >= b`
`LT`	$N \neq V$	Előjelesen `a < b`
`GT`	$Z = 0$ és $N = V$	Előjelesen `a > b`
`LE`	$Z = 1$ vagy $N \neq V$	Előjelesen `a <= b`
`AL`, `NV`	Mindig igaz	Az A64 feltételkiértékelésének különleges esetei

A `CS` és `HS`, illetve a `CC` és `LO` ugyanannak a kódnak két neve. Összehasonlításnál a `HS` és `LO` elnevezés gyakran olvashatóbb.

Az `NV` történeti neve ellenére az A64 itt használt feltételkiértékelésében **nem „soha”** jelentésű. Az A64 `CSEL` esetén az `AL` és `NV` egyaránt az első forrás kiválasztását eredményezi; az A64 `CSET` viszont egyiket sem fogadja el. Szokásos összehasonlításokhoz nincs rájuk szükség.

5.4.2.1 Levezetés: miért nem elég az N jelző?

Előjel nélküli kivonásnál C = 1 a kölcsönzés hiányát jelenti. Ez közvetlenül megadja az `a >= b` feltételt. A szigorú nagyobb relációhoz az egyenlőséget is ki kell zárni, ezért `HI` feltétele C = 1 és Z = 0.

Előjeles összehasonlításnál a kivonás túlcsordulhat. Vegyük a következő, **32 bites, levezetett** esetet:

- `a = 0x80000000`, előjelesen −2147483648;
- `b = 0x00000001`, előjelesen 1;
- a matematikai különbség −2147483649, amely nem ábrázolható 32 bites előjeles egészként;
- a tárolható eredmény bitmintája `0x7fffffff`.

Ezért N = 0, Z = 0, C = 1, V = 1. Az N önmagában pozitív eredményt sugallna, mégis `a < b`. A túlcsordulást figyelembe vevő feltétel N V, vagyis `LT`.

Ugyanezek a bitminták előjel nélkül 2147483648 és 1 értékűek, ezért `LO` hamis. Ez nem ellentmondás: más értelmezési tartományban hasonlítunk.

5.4.3 CSET: feltételből 0 vagy 1

Az A64 `CSET` egy feltétel igazságértékét írja regiszterbe:

```
// A64 ISA, AArch64: 32 bites összehasonlítás és logikai eredmény.
cmp     w1, w2
cset    w3, lt
```

A `W3` értéke 1, ha a feltétel igaz, különben 0. Ez megfelel egy előjeles C-összehasonlítás eredményének, ha az operandusok 32 bites előjeles értékeket jelentenek.

A `CSET`:

- az aktuális NZCV-t olvassa;
- nem módosítja az NZCV-t;
- nem végez új összehasonlítást;
- `W` célregiszter esetén a megfelelő `X` regiszter felső 32 bitjét nullázza.

A `CSET` **utasításálnév** (*instruction alias*). Például az A64 `CSET W3, LT` mögött `CSINC W3, WZR, WZR, GE` áll: az invertált feltétel igaz ágán nulla, hamis ágán nulla plusz egy keletkezik. Emiatt az aliasokat elrejtő disassembly eltérhet a forrásszövegtől.

5.4.4 CSEL: két regiszterérték közötti választás

Az A64 `CSEL` jelentése:

```
// A64 ISA, AArch64: ha LT igaz, w5 = w1; különben w5 = w2.
csel    w5, w1, w2, lt
```

Az első forrás az igaz, a második a hamisághoz tartozik. Mindhárom adatregiszter azonos szélességű: vagy `W`, vagy `X` nézetű. A kiválasztás magukat a bitmintákat másolja; nincs külön előjeles és előjel nélküli `CSEL`.

A `CSEL` nem módosítja az NZCV-t. Egyetlen összehasonlítás után ezért több `CSET` és `CSEL` is használhatja ugyanazt az állapotot.

Kódolási korlát: az A64 `CSEL` adatforrásai regiszterek, nem közvetlen konstansok (*immediate*). Nullához `WZR` vagy `XZR` használható; más konstanshoz előbb regiszterértéket kell előállítani.

A C feltételes operátorának (*conditional operator*) egyszerű leképezése:

C-kifejezés, megfelelő operandustípusokkal	A64-kiválasztás `CMP W1, W2` után
Előjeles `a < b ? a : b`	`CSEL W5, W1, W2, LT`
Előjel nélküli `a < b ? a : b`	`CSEL W5, W1, W2, LO`
Előjeles `a >= b ? a : b`	`CSEL W5, W1, W2, GE`
Előjel nélküli `a >= b ? a : b`	`CSEL W5, W1, W2, HS`

A C tényleges összehasonlítását a szokásos aritmetikai konverziók határozzák meg. Vegyes előjelű operandusoknál ne pusztán az egyik változó deklarációjából válassz feltételkódot.

5.4.4.1 Mit jelent itt az elágazásmentesség?

A `CSEL` nem választ következő végrehajtási címet, ezért ehhez a kiválasztáshoz nincs feltételes elágazás, amelyet az **elágazásbecslőnek** (*branch predictor*) meg kellene jósolnia.

Ettől azonban a megoldás nem automatikusan gyorsabb. Mindkét forrásérték előállítása munkát igényelhet, és az eredmény függ a jelzőktől, valamint a forrásregiszterektől. Az elágazásbecslés és az adatfüggőségek teljesítményhatását később tárgyaljuk.

Fontos szemantikai korlát: a C `?:` csak a kiválasztott operandusát értékeli ki. A `p != NULL ? *p : 0` ezért nem írható át általánosan feltétel nélküli betöltésre és utólagos `CSEL`-re. A hibás memóriaelérés addigra már megtörténne. Mellékhatásos függvényhívásoknál ugyanez az óvatosság szükséges.

Az elágazásmentesség önmagában állandó idejű, mellékcsatornákkal szemben védett végrehajtást sem garantál.

5.4.5 Kidolgozott példa: ugyanazok a bitek, kétféle sorrend

Munkakönyvtár: `~/arm-labor/a64/a017/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a017
cd ~/arm-labor/a64/a017
```

A labor előjeles és előjel nélküli minimumot, két logikai eredményt, valamint egy egyszerű `if/else` választását állítja elő.

Az alábbi teljes C-referencia legyen a `lab-csel-ref.c` fájl:

```
#include <stdint.h>

int main(void)
{
    int32_t a = INT32_MIN;
    int32_t b = 1;
    uint32_t ua = (uint32_t)a;
    uint32_t ub = (uint32_t)b;

    int signed_less = a < b;
    int unsigned_less = ua < ub;
    int32_t signed_min = a < b ? a : b;
    uint32_t unsigned_min = ua < ub ? ua : ub;

    int selected;
    if (a < b)
        selected = 100;
    else
        selected = 7;

    return !(signed_less == 1 &&
             unsigned_less == 0 &&
             signed_min == INT32_MIN &&
             unsigned_min == 1 &&
             selected == 100);
}
```

Az előjelesből előjel nélkülibe alakítás itt jól definiált C-művelet. A referencia szemantikai ellenőrzésre szolgál; nem állítjuk, hogy a fordító pontosan az alábbi kézi utasítássort állítja elő.

A teljes assemblyprogram legyen a `lab-csel.s` fájl:

```
// A64 ISA, Armv8-A, little-endian Linux.
// Direktívák: GNU as, AArch64/ELF cél.
.arch armv8-a
.text
.balign 4
.global _start
.type _start, %function

_start:
    movz    w1, #0x8000, lsl #16
    movz    w2, #1

    cmp     w1, w2
    cset    w3, lt
    cset    w4, lo
    csel    w5, w1, w2, lt
    csel    w6, w1, w2, lo
```

```
    movz    w9, #100
    movz    w10, #7
    csel    w7, w9, w10, lt

lab_selected:
    // Önellenőrzés: w0 végül 0 siker, 1 eltérés esetén.
    cmp     w3, #1
    cset    w0, ne
    orr     w0, w0, w4

    cmp     w5, w1
    cset    w11, ne
    orr     w0, w0, w11

    cmp     w6, w2
    cset    w11, ne
    orr     w0, w0, w11

    cmp     w7, w9
    cset    w11, ne
    orr     w0, w0, w11

    // AArch64 Linux exit: x8 = 93, x0 = kilépési állapot.
    movz    x8, #93
    svc     #0

.size _start, . - _start
.section .note.GNU-stack, "", %progbits
```

Az első utasítás a `MOVZ` 16. bitre tolt mezőjével pontosan `0x80000000` értéket állít elő (a korlátokat lásd az A64 immediate-korlátok összefoglalójában). A `MOVZ` nem módosítja az NZCV-t, ezért a két konstans betöltése nem rontja el a későbbi kiválasztást.

A Linux-híváshoz itt `SVC #0` szerepel; az utasítás és a Linux rendszerhívási szerződés nem ugyanaz a specifikációs réteg.

5.4.5.1 Összeállítás, linkelés és futtatás

Cél: AArch64 Linux, QEMU user-mode; host: Ubuntu 24.04 x86_64. A korábban telepített cross-eszközláncot használjuk.

```
# Target: qemu-user / AArch64, little-endian.
aarch64-linux-gnu-as -EL -march=armv8-a -g \
    -o lab-csel.o lab-csel.s
aarch64-linux-gnu-ld -EL -e _start \
    -o lab-csel lab-csel.o

qemu-aarch64 ./lab-csel
printf 'assembly kilépés=%d\n' "$?"

# Ugyanazon cél C-referenciája; statikus linkelés.
aarch64-linux-gnu-gcc -std=c11 -O0 -Wall -Wextra \
    -march=armv8-a -mlittle-endian -static \
    lab-csel-ref.c -o lab-csel-ref

qemu-aarch64 ./lab-csel-ref
printf 'C kilépés=%d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, GNU gcc 13, QEMU 8.2.2): mindkét kilépési állapot 0. Maguk a programok nem írnak a szabványos kimenetre.

A `lab_selected` címkénél a **mért** állapot (GDB 15.1, QEMU 8.2.2):

A64-regiszter vagy jelző	Mért érték	Értelmezés
`W3`	1	Előjeles kisebb: igaz
`W4`	0	Előjel nélküli kisebb: hamis
`W5`	`0x80000000`	Előjeles minimum: −2147483648
`W6`	1	Előjel nélküli minimum
`W7`	100	Az `if` igaz ágának értéke
NZCV	N=0, Z=0, C=1, V=1	Az első összehasonlítás jelzői

A `W5` írása miatt `X5 = 0x0000000080000000`. Ez **nem** a negatív szám 64 bites előjelkiterjesztett alakja.

5.4.5.2 Megfigyelés GDB-vel

Az első terminálban:

```
# Target: qemu-user / AArch64; induláskor debuggerre vár.
qemu-aarch64 -g 1234 ./lab-csel
```

A második terminálban:

```
# Target: a fenti QEMU-folyamat AArch64 regiszterállapota.
gdb-multiarch -q ./lab-csel \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_selected' \
    -ex 'continue' \
    -ex 'info registers x3 x4 x5 x6 x7 cpsr' \
    -ex 'p/x ($cpsr & 0xf0000000)'
```

A maszkolt jelzőérték mért értéke `0x30000000` (a `cpsr` pszeudoregisztert az 5.3. alfejezet vezette be). Ezután a GDB `continue` parancsa végigfuttatja az önellenőrzést; a mért kilépési állapot továbbra is 0.

Az utasításkódolás ellenőrzése:

```
# Célállomány: A64 ELF; ez elemzés, nem futtatás.
aarch64-linux-gnu-objdump -d -M no-aliases ./lab-csel
```

Az aliasok nélkül a `CSET` helyén `CSINC`, a `CMP` helyén nulla célregiszteres `SUBS` jelenhet meg. Ez nem fordítási hiba.

5.4.6 Tipikus hibák és ellenpróbák

- **`LT` helyett `LO`:** kis pozitív számokkal észrevétlen maradhat. Ellenpróba: legfelső bitjében 1-es operandus és kis pozitív érték összehasonlítása.
- **`MI` használata előjeles kisebbként:** túlcsorduló kivonásnál hibás. A laborban N = 0 mellett `LT` igaz.
- **Jelzők véletlen felülírása:** egy közbeékelt A64 `ADDS`, `SUBS` vagy új `CMP` után a kiválasztás már más művelet állapotát olvassa.
- **Források felcserélése:** a `CSEL` első forrása az igaz ág. Ellenőrizd külön az igaz és hamis esetet.

- **`W` és `X` összehasonlítás összekeverése:** ugyanaz a regiszterpár más szélességen más előjeles értéket jelenthet.

Hibakereséskor először az összehasonlítás szélességét és operandusait nézd meg, utána az NZCV-t, végül a feltételkódot. Egyszerre csak egy feltételezett hibát módosíts.

5.4.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) 32 bites A64 `CMP` után határozd meg az NZCV-t, valamint az `LT` és `LO` igazságértékét: - `0x00000000` és `0x00000000`; - `0xffffffff` és `0x00000001`; - `0x7fffffff` és `0xffffffff`.

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a labort az előjeles maximum `W12`-be és az előjel nélküli maximum `W13`-ba választásával. A két utasítás az első összehasonlítás jelzőit használja. Ellenőrizd GDB-vel a `lab_selected` címkénél, majd vizsgáld meg az egyenlő operandusok esetét is. Az eredeti önellenőrzés a rögzített laboradatokra készült; más bemenetnél ehhez is igazítani kell.

3. (hibakeresési, középszint — cél: rutinépítés) Szúrd be az A64 `adds w14, w2, w2` utasítást közvetlenül az első `CMP` után. Fordítsd újra, figyeld meg a kilépési állapotot és az NZCV-t, majd egyetlen utasítás módosításával javítsd a hibát.

5.4.8 Megoldások

1. megoldás — Segítség. Számítsd ki mindhárom esetben a 32 bites kivonást bitminta szinten (nem előjeles aritmetikával), és nézd meg, mikor kölcsönöz (C), mikor negatív a legfelső bit (N), és mikor csordul túl előjelesen (V) — az `LT` feltétele N V, az `LO` feltétele C=0.

1. megoldás — Teljes megoldás. A 32 bites kivonásokból levezetett eredmények:

Operandusok	Kivonási bitminta	NZCV	`LT`	`LO`
`0x00000000`, `0x00000000`	`0x00000000`	0, 1, 1, 0	Hamis	Hamis
`0xffffffff`, `0x00000001`	`0xfffffffe`	1, 0, 1, 0	Igaz	Hamis
`0x7fffffff`, `0xffffffff`	`0x80000000`	1, 0, 0, 1	Hamis	Igaz

A harmadik sorban előjelesen 2147483647 és -1 szerepel. A különbség túlcsordul, ezért N = 1 ellenére az első operandus nagyobb: N = V.

2. megoldás — Segítség. A maximum kiválasztása pontosan a minimum feltételének a FORDÍTOTTJA — ha a minimumhoz `LT`/`LO` kellett, a maximumhoz melyik feltételkód adja a "nagyobb vagy egyenlő" döntést?

2. megoldás — Teljes megoldás. A következő kiegészítés például közvetlenül a `lab_selected` címke elé helyezhető:

```
// A64 ISA, AArch64; az eredeti CMP jelzőit használja.
csel    w12, w1, w2, ge
csel    w13, w1, w2, hs
```

A laboradatokkal `W12 = 1`, `W13 = 0x80000000`. Egyenlőségnél mindkét feltétel igaz, ezért az első forrást választják; a két forrás számszerűen azonos.

3. megoldás — Segítség. Az `ADDS` is módosítja az NZCV-t — mi történik az eredeti `CMP` jelzőivel, mire a `CSET`/`CSEL` sorozat ténylegesen kiértékelné őket?

3. megoldás — Teljes megoldás. A beszúrt összeadás eredménye 2, jelzői N=0, Z=0, C=0, V=0. Emiatt `LT` hamis, `LO` igaz lesz: `W3 = 0`, `W4 = 1`, a minimumok felcserélődnek, `W7 = 7`. A mért hibás kilépési állapot 1.

GDB-ben a `lab_selected` ponton a maszkolt NZCV mért értéke nulla. Az `objdump` megmutatja a jelzőket felülíró `ADDS` utasítást. Ha az összeadás jelzőire nincs szükség, egyetlen javítás elegendő: `ADDS` helyett A64 `ADD`. Az eredmény továbbra is 2, de az eredeti összehasonlítás jelzői megmaradnak.

5.4.9 Rövid összefoglaló

Az A64 feltételes kiválasztása az NZCV állapotát alakítja adattá, nem új végrehajtási útvonallá. Ugyanaz a CMP előjeles és előjel nélküli döntést is megalapozhat, de a megfelelő feltételkódot nekünk kell kiválasztanunk. A CSET 0 vagy 1 értéket ad, a CSEL két regiszterérték egyikét másolja; egyik sem módosítja a jelzőket. Figyelni kell az összehasonlítás szélességére, a közbeékelt jelzőfrissítésre és a C operanduskiértékelési szabályaira. Az elágazás elhagyása nem automatikus gyorsítás: a forrásértékek előállításának költsége és az adatfüggőségek továbbra is számítanak. A laborban a regiszterek és az NZCV együttes ellenőrzése teszi láthatóvá a hibát.

- **Feltételkód:** az NZCV-ből képzett logikai feltétel jelölése.
- **Logikai materializálás:** feltétel átalakítása regiszterben tárolt 0 vagy 1 értékké.
- **Feltételes kiválasztás:** két előállított érték közötti döntés feltétel alapján.

5.4.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; `ConditionHolds`, `CSEL`, `CSINC`, valamint a `CSET` alias. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- GNU assembler kézikönyv — AArch64 szintaxis, célarchitektúra és ELF-direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU objdump kézikönyv — disassembly és az aliasmegjelenítés beállítása. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenetek és regisztерállapotok a lektor GNU as 2.42 + GNU gcc 13 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). Az Ubuntu 24.04-ben telepített konkrét eszközverziókkal a buildet, a GDB regisztermegjelenítését és az aliasok megjelenítését a hallgatói laborban kell ellenőrizni. Teljesítménymérést nem végeztünk; a QEMU-futtatás nem igazol hardverhű ciklusszámot.

6 Vezérlésátadás: elágazás és ciklusok C-szemmel

6.1 Feltétel nélküli és feltételes elágazás

6.1.1 Motiváció: az értékválasztástól a végrehajtási út kiválasztásáig

Egy bemenetellenőrző rutin hibás adat esetén azonnal kilép, érvényes adat esetén pedig folytatja a feldolgozást. Itt nem elég két érték közül választani: bizonyos utasításokat egyáltalán nem szabad végrehajtani. Ehhez **elágazásra** (*branch*) van szükség.

A Feltételes végrehajtás: CSET/CSEL és feltételkódok alfejezetben az A64 feltételkódjai az eredményként kiválasztott adatot befolyásolták. Most ugyanezekkel a feltételekkel a következő végrehajtandó utasítást választjuk ki.

Az alfejezet teljes egészében **A64 ISA, Armv8-A, little-endian Linux** környezetre vonatkozik. Előfeltétel az A64 `CMP` és az NZCV állapotjelzők értelmezése. A labor célja a vezérlés helyességének ellenőrzése, nem a teljesítménymérés.

6.1.2 Az elágazás működése

A **vezérlésátadás** (*control transfer*) megváltoztatja a következő végrehajtandó utasítás címét. Az A64 utasítások 4 bájtosak, ezért szokásos, szekvenciális végrehajtáskor a következő utasítás címe az aktuális cím plusz 4 bájt.

Az A64 közvetlen elágazásainál a cél az utasítás helyéhez viszonyított, előjeles eltolásból származik. A forráskódban ezt többnyire **címkével** (*label*) adjuk meg:

- az A64 `b lab_exit` feltétel nélkül átadja a vezérlést;
- az A64 `b.eq .Lzero` csak teljesülő `EQ` feltételnél ugrik;
- az A64 `cbz x0, .Lzero` az A64 `X0` regiszter nulla értékét vizsgálja.

Ha egy feltételes elágazás nem teljesül, a végrehajtás a következő utasítással folytatódik. Ez a **továbbhaladó ág** (*fall-through path*). A64-en nincs késleltetett elágazási rés (*branch delay slot*): teljesülő ugrás után a közvetlenül következő utasítást nem kell még végrehajtani.

A64 utasítás	Döntés alapja	Olvassa az NZCV-t?	Módosítja az NZCV-t?
`B`	Nincs feltétel	Nem	Nem
`B.cond`	A megadott feltételkód	Igen	Nem
`CBZ Wn/Xn`	A regiszter értéke nulla	Nem	Nem
`CBNZ Wn/Xn`	A regiszter értéke nem nulla	Nem	Nem

Az A64 `B` egyszerű ugrás: nem ment visszatérési címet. A függvényhívás ettől eltérő szerződését a Hívás és visszatérés: BL/RET bevezető mintája tárgyalja.

6.1.3 Feltételkód vagy közvetlen nullateszt?

Az A64 `B.cond` jelölésben a `cond` helyére konkrét feltételkód kerül, például `EQ`, `NE`, `LT` vagy `HI`. A GNU assemblerben kisbetűs írásmódot használunk: például A64 `b.lt`.

Az A64 `cmp x0, #10` után:

A64 elágazás	Jelentés
`b.eq`	Az A64 `X0` értéke egyenlő 10-zel
`b.ne`	Az A64 `X0` értéke nem egyenlő 10-zel
`b.lt`	Az A64 `X0` **előjelesen** kisebb 10-nél
`b.gt`	Az A64 `X0` **előjelesen** nagyobb 10-nél
`b.lo`	Az A64 `X0` **előjel nélkül** kisebb 10-nél
`b.hi`	Az A64 `X0` **előjel nélkül** nagyobb 10-nél

Az A64 `CMP` immediate-változata 12 bites, előjel nélküli konstanst fogad el, opcionális 12 bites balra tolással. A példában szereplő 10 közvetlenül kódolható.

Az összehasonlítás és az A64 `B.cond` között nem feltétlenül kell közvetlen szomszédságnak lennie. A követelmény az, hogy közben semmi ne írja felül a szükséges NZCV-állapotot. A közbeiktatott A64 `ADDS` például új jelzőket állítana elő, így az elágazás már nem az eredeti összehasonlításra válaszolna.

Nullatesztnél az A64 `CBZ` és `CBNZ` egyszerűbb mintázat:

- az A64 `cbz x0, cél` ugyanazt a nulla/nem-nulla döntést hozza, mint az A64 `cmp x0, #0` és `b.eq cél`;
- az A64 `cbnz x0, cél` a nem nulla értékre ugrik;
- egyik közvetlen nullateszt sem változtatja meg a korábban előállított NZCV-t.

Ez a megoldás külön összehasonlító utasítást takarít meg, de önmagában nem jelent bizonyított gyorsulást.

A regiszterszélesség a vizsgálat része. Az A64 `cbz w0, cél` csak az alsó 32 bitet vizsgálja; az A64 `cbz x0, cél` mind a 64 bitet. A `0x0000000100000000` bitmintánál az előbbi ugrik, az utóbbi nem. Ez olvasási művelet: az A64 `W0` operandus használata itt nem nullázza az A64 `X0` felső felét.

6.1.4 Relatív eltolás: a hatótávolság levezetése

Az A64 elágazás forrásbeli címkéje nem tetszőleges teljes címként kerül az utasításba. A kódolás egy előjeles mezőt tartalmaz, amelynek értékét 4 bájttal kell megszorozni.

Legyen:

- P az elágazási utasítás címe, bájtban;
- T a célutasítás címe, bájtban;
- k az előjeles kódolt mező értéke, mértékegység nélküli egész.

Ekkor:

$$T = P + 4k.$$

Fontos: az alap **maga az elágazási utasítás**, nem a következő utasítás címe. A cél és az elágazás különbségének 4 bájttal oszthatónak kell lennie.

Egy n bites kettes komplemens mezőnél:

$$-2^{n-1} \leq k \leq 2^{n-1} - 1.$$

Ebből az alábbi, **elméleti, levezetett** tartományok adódnak:

A64 utasítás	Eltolásmező	Legkisebb eltolás [bájt]	Legnagyobb eltolás [bájt]
`B`	26 bit	−134217728	+134217724
`B.cond`	19 bit	−1048576	+1048572
`CBZ`, `CBNZ`	19 bit	−1048576	+1048572

Az A64 `B` tehát megközelítőleg ±128 MiB, a többi felsorolt elágazás megközelítőleg ±1 MiB távolságot ér el. A pozitív szélsőérték mindkét esetben 4 bájttal kisebb a kerek határnál. Itt 1 MiB = 1048576 bájt.

Ha egy feltételes cél kívül esik a rövid tartományon, gyakori átalakítás:

1. a feltétel ellentettjével átugorjuk a következő A64 `B` utasítást;
2. az A64 `B` a távolabbi célra ugrik.

Például az A64 `b.gt lab_far` helyett az A64 `b.le .Lnear` következhet, utána az A64 `b lab_far`, majd a `.Lnear` címke. Ez csak akkor elég, ha az új A64 `B` saját helyétől mérve eléri a célt.

Még nagyobb távolsághoz hosszabb, célcímet előállító és közvetett ugrást végző utasítássor kell. Ilyen köztes ugrókódot a linker bizonyos esetekben létrehozhat; ezt gyakran *veneer* vagy *thunk* néven említik. **Nem szabad minden elágazásra automatikus javítást feltételezni.** A feltételes elágazás túlcsordulása hibaüzenethez vezethet. A linker konkrét viselkedését és az esetleges segédregiszter-használatot külön ellenőrizni kell.

6.1.5 Címkék, assembler és linker

A címke szimbolikus nevet rendel egy helyhez. A GNU assembler A64-forrásában a `.Lzero:` sor nem utasítás, és önmagában nem foglal kódbájtot. Az utána következő helyet nevezi el.

Ha az ugrás és célja ugyanabban a szekcióban található, az assembler rendszerint már ismeri a különbségüket, és közvetlenül kódolhatja azt. Másik objektumfájlban definiált cél esetén **relokációs bejegyzést** (*relocation entry*) hagy az objektumban.

A linker ekkor:

1. elrendezi a bemeneti szekciókat;
2. feloldja a szimbólumokat;
3. kiszámítja az elágazás eltolását;
4. ellenőrzi, hogy az eltolás kódolható-e.

A GNU AArch64 ELF-eszközláncban az A64 `B` külső céljához tipikusan `R_AARCH64_JUMP26` relokáció tartozik. A következő labor ezt külön objektumfájlokkal teszi megfigyelhetővé.

6.1.6 Kidolgozott példa: bemenetellenőrzés két objektumfájllal

Munkakönyvtár: `~/arm-labor/a64/a018/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a018
cd ~/arm-labor/a64/a018
```

A program szerződése:

- az A64 `X0` előjeles, 64 bites bemenet;
- az A64 `X2` előjel nélküli, 64 bites engedélyezési érték: nulla esetén tiltott, minden más értéknél engedélyezett;
- az elfogadott bemenet 1 és 10 közötti;
- az eredmény a Linux kilépési állapota.

A `lab-branch.s` fájl:

```
// A64 ISA, Armv8-A; GNU as direktívák AArch64 ELF-hez.
.ifndef LAB_X
.equ LAB_X, 7
.endif
.ifndef LAB_EN
.equ LAB_EN, 1
.endif

.text
.p2align 2
.global _start
.type _start, %function

_start:
    mov     x0, #LAB_X
    mov     x2, #LAB_EN

    cbz     x0, .Lzero

    cmp     x0, #0
    b.lt    .Lnegative

    cmp     x0, #10
    b.gt    .Llarge

    cbnz    x2, .Lenabled

    movz    w0, #3
    b       lab_exit

.Lenabled:
    movz    w0, #0
    b       lab_exit

.Lzero:
    movz    w0, #1
    b       lab_exit

.Lnegative:
```

```
    movz    w0, #2
    b       lab_exit

.Llarge:
    movz    w0, #4
    b       lab_exit

.size _start, .-_start
.section .note.GNU-stack, "", %progbits
```

A `lab-exit.s` fájl:

```
// A64 ISA, Armv8-A; GNU as direktívák AArch64 ELF-hez.
.text
.p2align 2
.global lab_exit
.type lab_exit, %function

lab_exit:
    movz    x8, #93
    svc     #0

.size lab_exit, .-lab_exit
.section .note.GNU-stack, "", %progbits
```

A példában a `mov` alias minden konstanshoz egyetlen kódolást választ: 7, 0 és 11 esetén `MOVZ`, -1 esetén `MOVN`; a 16 bites immediate-mező mindegyikhez elég. Az A64 `MOVZ` megengedett eltolásai `Wn` cél esetén 0 vagy 16, `Xn` cél esetén 0, 16, 32 vagy 48 bit. Az A64 `W0` írása nullázza az A64 `X0` felső 32 bitjét, így a kilépési kód teljes regiszterértéke is megfelelő.

Az A64 `SVC` 16 bites immediate-mezőjében a 0 kódolható. A 93-as rendszerhívásszám viszont nem ISA-tulajdonság, hanem a Linux AArch64 rendszerhívási felületének `exit` azonosítója.

Target: qemu-user, AArch64 little-endian Linux; host: Ubuntu 24.04 x86_64.

```
aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-exit.o lab-exit.s

for x in 7:1 0:1 -1:1 7:0 11:1; do
    xv=${x%%:*}
    en=${x##*:}
    aarch64-linux-gnu-as -EL -march=armv8-a \
        --defsym LAB_X="$xv" --defsym LAB_EN="$en" \
        -o lab-branch.o lab-branch.s
    aarch64-linux-gnu-ld -EL -e _start \
        -o lab-branch lab-branch.o lab-exit.o
    qemu-aarch64 ./lab-branch
    printf 'X=%s engedely=%s status=%s\n' "$xv" "$en" "$?"
done
```

Mért eredmény (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): a ciklus öt sora a táblázat `status` értékeit írja ki; az alapértelmezett 7/1 bemenetnél `status=0`. A program saját maga nem ír szöveget; a megjelenített sort a shell állítja elő.

A végrehajtási út (7/1 bemenetre):

1. Az A64 `X0` értéke 7, ezért az A64 `CBZ` nem ugrik.
2. A két A64 `B.cond` sem ugrik: 7 nem negatív és nem nagyobb 10-nél.
3. Az A64 `X2` értéke 1, ezért az A64 `CBNZ` a sikeres ágra ugrik.
4. Az A64 `W0` értéke 0 lesz.
5. Az A64 `B` a külön objektumban definiált `lab_exit` célra ugrik.

Bemenet: A64 `X0`, előjeles	Engedély: A64 `X2`, előjel nélküli	Mért kilépési állapot
7	1	0
0	1	1
−1	1	2
7	0	3
11	1	4

A táblázat öt bemenetét a fenti `--defsym`-es ciklus futtatja le, kézi forrásmódosítás nélkül. A negatív esetben (`X0 = -1`) a `mov` alias `MOVN` kódolást választ; az `LAB_EN` 0 értéke a tiltott ágra visz.

6.1.7 Az elágazások ellenőrzése

Target: a qemu-user labor AArch64 objektumai és ELF-fájlja; hostoldali vizsgálat:

```
aarch64-linux-gnu-objdump -dr lab-branch.o
aarch64-linux-gnu-readelf -rW lab-branch.o
aarch64-linux-gnu-objdump -d lab-branch
```

Az első két parancs kimenetében a `lab_exit` szimbólumra hivatkozó `R_AARCH64_JUMP26` bejegyzéseket látunk: a `readelf -rW lab-branch.o` öt ilyen relokációt mutat (mérve: GNU binutils 2.42). A végleges ELF disassemblyjében az A64 `B` utasításoknak már a feloldott `lab_exit` célra kell mutatniuk.

A konkrét címeket nem kell előre feltételezni: a saját linkerünk kimenetét vizsgáljuk. A címkülönbségből ellenőrizhető az eltolás. A kiírt utasításcímek nem jelentenek általánosan használható memóriacímeket.

6.1.8 Tipikus hibák és ellenpróbák

Hibás irányú feltétel. Ha a nulla bemenet nem a nullaágra jut, először ellenőrizzük, hogy nem A64 `CBNZ` szerepel-e A64 `CBZ` helyett. Adatgyűjtés: disassembly és kilépési állapot. Egyetlen mnemonic javítása után ismételjük meg a 0 és 7 bemenetes próbát.

Előjeles és előjel nélküli döntés összekeverése. A −1 bitmintája előjel nélkül nagy pozitív szám. Az A64 `B.LT` és `B.LO` ezért nem felcserélhető. A labor felső korlátjánál az A64 `B.GT` A64 `B.HI`-ra cserélése viszont nem feltétlenül látható: a korábbi negatívvizsgálat már kizárta az eltérő eseteket. A teszteket mindig az adott pontra ténylegesen eljutó értékek alapján válasszuk.

Helyes feltétel, rossz jelzők. Ha az összehasonlítás és az A64 `B.cond` közé jelzőfrissítő utasítás került, a disassemblyben keressük meg. Egyetlen változtatással távolítsuk el, majd futtassuk újra a határértékeket.

Túl távoli cél. A tartományhibát nem oldja meg a címke átnevezése. Vizsgáljuk meg a relokációt és a szekcióelrendezést; ezután válasszunk hosszabb ugrásmintát.

6.1.9 Feladatok

1. **Magyarázó (alapszint — cél: megértés-ellenőrzés):** Miért megfelelő az A64 `CBNZ` az engedélyezéshez, és miért kell a felső korlát vizsgálatához más mintázat? Mi történik, ha a bemenet és az engedélyezési érték egyaránt nulla?

2. **Implementációs (alapszint — cél: rutinépítés):** Módosítsd az elfogadott tartományt 1–20-ra, a hibakódok megtartásával. Ellenőrizd a 0, 1, 20 és 21 bemeneteket engedélyezett állapotban.

3. **Hibakeresési (középszint — cél: rutinépítés):** Állítsd az engedélyt az A64 `movz x2, #1, lsl #32` utasítással. Ezután szándékosan cseréld az A64 `cbnz x2, .Lenabled` utasítást A64 `cbnz w2, .Lenabled` alakra. Jósolj kilépési állapotot, majd a disassembly alapján magyarázd meg az eltérést.

4. **Kódolási (középszint — cél: megértés-ellenőrzés):** Eléri-e az A64 `B.cond` a saját címéhez képest +1048572, illetve +1048576 bájtra lévő célt? Hogyan alakítanál át egy túl távoli A64 `B.GT` elágazást, ha a cél az A64 `B` tartományán belül van?

6.1.10 Megoldások

1. megoldás — Segítség. Az engedélyezés egy bináris (nulla/nem-nulla) kérdés, a felső korlát viszont egy *reláció* (kisebb/nagyobb) — melyik A64-mintázat vizsgálja közvetlenül a regisztert (jelzők nélkül), és melyik igényel előzetes CMP-t és NZCV-kiértékelést?

1. megoldás — Teljes megoldás. Az engedélyezés kizárólag nulla/nem-nulla kérdés; az A64 `CBNZ` ezt NZCV nélkül megoldja. A felső korlát relációt igényel, ezért A64 `CMP` és megfelelő A64 `B.cond` kell. Két nulla bemenetnél az első A64 `CBZ` már a nullaágra ugrik, ezért a várt állapot 1; az engedélyezést nem vizsgáljuk meg.

2. megoldás — Segítség. Csak a felső határt összehasonlító immediate kell cserélni; ellenőrizd, hogy 20 belefér-e a `CMP` immediate-kódolási korlátjába.

2. megoldás — Teljes megoldás. Az A64 `cmp x0, #10` helyett A64 `cmp x0, #20` szükséges. A 20 belefér a 12 bites immediate-mezőbe. A várt állapotok rendre 1, 0, 0 és 4. A felső határ elfogadott, ezért marad az A64 `B.GT`, nem A64 `B.GE`.

3. megoldás — Segítség. Nézd meg, mi kerül `X2` alsó 32, illetve felső 32 bitjébe a `movz ... lsl #32` után, és hogy a `W2` nézet melyik felet látja.

3. megoldás — Teljes megoldás. Az A64 `X2` értéke `0x0000000100000000`, alsó 32 bitje nulla. Az A64 `CBNZ X2` változattal a mért állapot 0; az A64 `CBNZ W2` változattal 3. A hipotézist a disassembly operandusszélessége ellenőrzi. Az egyetlen javítás az A64 `X2` nézet visszaállítása.

4. megoldás — Segítség. Az A64 `B.cond` 19 bites, előjeles, 4 bájtra skálázott eltolást kódol — számítsd ki ennek a mezőnek a legnagyobb pozitív értékét bájtban, és vesd össze a két jelölt céllal.

4. megoldás — Teljes megoldás. Az első cél elérhető, mert az eltolás osztva 4-gyel 262143, a 19 bites előjeles mező legnagyobb értéke. A másodikhoz 262144 kellene, ami már nem fér bele. Az átalakítás: A64 `B.LE` egy közeli címkére, utána A64 `B` a távoli célra, végül a közeli címke. Az új A64 `B` hatótávolságát a saját címétől számítjuk.

6.1.11 Rövid összefoglaló

Az A64 elágazás a végrehajtás következő helyét választja ki. A64 `B` esetén nincs feltétel, A64 `B.cond` esetén az NZCV dönt, míg az A64 `CBZ/CBNZ` közvetlenül egy regiszter nulla vagy nem nulla értékét vizsgálja. A regiszterszélesség és az előjeles értelmezés a helyesség része. A relatív cél az ugrás saját címéhez képest számítandó, 4 bájtos lépésekben. A külön objektumban található cél feloldása a linker feladata; a túl nagy távolságot nem feltétlenül javítja automatikusan. A laborban kilépési állapotokkal és disassemblyvel ellenőriztük az útvonalakat.

- **Elágazás:** a következő végrehajtandó utasítás helyének kiválasztása.
- **Címke:** egy programhely szimbolikus neve.
- **Relokáció:** későbbi címfeloldáshoz megőrzött objektumfájl-bejegyzés.

6.1.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció: DDI0487; az A64 `B`, `B.cond`, `CBZ` és `CBNZ` kódolása és szemantikája. **Ellenőrizve**: a link él (a `support.arm.com`-ra irányított átirányítás megerősített), de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv: AArch64 szintaxis, címkék és objektumkészítés. **Ellenőrizve**: aktuális kiadás binutils 2.47.
- GNU linker kézikönyv: szimbólumfeloldás, relokációk és célspecifikus linkerfunkciók. **Ellenőrizve**: ugyanaz a 2.47-es kiadás.
- GNU objdump kézikönyv: disassembly és relokációk megjelenítése. **Ellenőrizve**: ugyanaz a 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a közölt kimenetek és a relokációszám a lektor GNU as/ld 2.42 + QEMU 8.2.2 méréséből származnak (`mérve`). Az Ubuntu 24.04-ben telepített konkrét binutils-verzió hibaüzeneteit és távoli ugrásokhoz generált esetleges köztes kódját a hallgatói laborban kell ellenőrizni.

6.2 C if/else és switch leképezése assemblyre

6.2.1 Motiváció és a vezérlési gráf fogalma

Egy protokollfeldolgozóban más művelet tartozhat az egyes üzenettípusokhoz; egy ellenőrző rutinban pedig több, egymást kizáró feltétel alapján választunk eredményt. C-ben mindkettő néhány sor. Assemblyben nekünk kell biztosítanunk, hogy **pontosan a megfelelő ág fusson le**, majd a végrehajtás a megfelelő helyen folytatódjon.

A **vezérlési gráf** (*control-flow graph*) a program végrehajtási útvonalainak modellje: csomópontjai elágazás nélküli utasítássorozatok, élei a lehetséges továbbhaladások. Az `if` és a `switch` fordításakor ezt a gráfot építjük fel összehasonlításokból, elágazásokból és közös folytatási pontokból.

Az előfeltétel a Feltétel nélküli és feltételes elágazás ismerete. **Az alfejezet minden assembly példája A64 ISA, AArch64, Armv8-A, little-endian környezetre vonatkozik; a direktívák GNU assembler-direktívák.** Cortex-M-kód itt nem szerepel. Az ugrótáblához

szükséges egyetlen memóriaolvasást helyben értelmezzük; a címzési módok rendszeres tárgyalása a Load/store címzési módok témája.

6.2.2 Többágú `if`: sorrend és közös folytatás

Tekintsük ezt a C-referenciafüggvényt:

```
#include <stdint.h>

uint32_t classify(int32_t x)
{
    if (x < 0)
        return 11;
    else if (x >= 10 && x <= 19)
        return 22;
    else if (x == 30 || x == 50)
        return 33;
    else
        return 44;
}
```

Az `x` előjeles, 32 bites érték. A feltételek sorrendje a program jelentésének része: az első igaz ág kiválasztása után a további feltételeket már nem értékeljük ki.

A kézi leképezés lépései:

1. Megvizsgáljuk az első feltételt; igaz esetben az első eredményblokkra ugrunk.
2. Hamis esetben a következő feltétel vizsgálata következik.
3. Az utolsó feltétel hamis kimenete a végső `else`ághoz vezet.
4. Minden eredményblokk a közös folytatásra ugrik, nem esik bele a következő eredményblokkba.

A **rövidzáras kiértékelés** (*short-circuit evaluation*) további vezérlési éleket jelent:

C-kifejezés	Első rész eredménye	Következmény
`A && B`	hamis	Az egész hamis; `B` kimarad
`A && B`	igaz	`B` kiértékelése szükséges
`A || B`	igaz	Az egész igaz; `B` kimarad
`A || B`	hamis	`B` kiértékelése szükséges

Ez nem egyszerűen optimalizálási lehetőség, hanem C-szemantika. Ha a második operandus mellékhatást tartalmaz, vagy csak az első feltétel teljesülésekor biztonságos, a feltétel nélküli kiértékelés megváltoztathatja a program működését. A C logikai operátorait ezért általában nem helyettesíthetjük a két operandus feltétel nélküli kiszámításával és bitenkénti összekapcsolásával.

6.2.3 Kidolgozott példa: `if`, `&&` és `||`

Munkakönyvtár: `~/arm-labor/a64/a019/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a019
cd ~/arm-labor/a64/a019
```

A következő teljes program a referenciafüggvény döntését valósítja meg, de nem függvényhívással: a kiválasztott eredmény Linux-kilépési állapot lesz. Mentsük `lab-if.s` néven.

```
// A64 ISA; GNU as; little-endian Linux.
// LAB_X: a laborban -1 vagy 0..65535.
.ifndef LAB_X
.equ LAB_X, 15
.endif

.section .text
.global _start
_start:
    mov w0, #LAB_X

    cmp w0, #0
    b.lt .Lnegative

    // x >= 10 && x <= 19
    cmp w0, #10
    b.lt .Lspecial
    cmp w0, #19
    b.le .Linterval

.Lspecial:
    // x == 30 || x == 50
    cmp w0, #30
    b.eq .Lselected
    cmp w0, #50
    b.eq .Lselected

    mov w0, #44
    b .Lexit

.Lnegative:
    mov w0, #11
    b .Lexit

.Linterval:
    mov w0, #22
    b .Lexit

.Lselected:
    mov w0, #33

.Lexit:
    mov w8, #93
    svc #0

.section .note.GNU-stack,"",%progbits
```

Az A64 `CMP W0, #10` után az A64 `B.LT` előjeles összehasonlítás alapján választ: ha `x < 10`, az `&&` első operandusa hamis, ezért a `19`-cel való összehasonlítás kimarad. Az `||` esetében a `30`-cal való egyezés rögtön az eredményblokkra vezet.

A közvetlen operandusok itt mind kódolhatók (lásd az A64 immediate-korlátok összefoglalóját). Egy példára jellemző részlet: a `-1` értéket az assembler `MOVN` kódolással állítja elő. A Linux-rendszerhívás számát az A64 `X8` tartalmazza; a `W8` írása annak felső 32 bitjét nullázza.

A konkrét összeállítás–linkelés–futtatás:

```
# Target: qemu-user, AArch64 Linux, little-endian.
aarch64-linux-gnu-as -EL -march=armv8-a --defsym LAB_X=15 \
    -o lab-if.o lab-if.s
```

```
aarch64-linux-gnu-ld -EL -e _start -o lab-if lab-if.o
qemu-aarch64 ./lab-if
lab_status=$?
printf 'status=%d\n' "$lab_status"
```

Mért kimenet (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): `status=22`. A nem nulla kilépési állapot ebben a gyakorlatban szándékos eredmény, nem automatikusan build- vagy futtatási hiba.

A `--defsym LAB_X=...` értékének módosítása és az újraépítés után:

`LAB_X`, előjeles	Mért állapot	Kiválasztott ág
−1	11	Negatív
9	44	Végső `else`
10	22	Intervallum alsó határa
19	22	Intervallum felső határa
20	44	Végső `else`
30	33	Az `||` első operandusa igaz
50	33	Az `||` második operandusa igaz

6.2.3.1 Rövidzár, amely védelmet ad: `p != NULL && *p != 0`

A fenti feltételek tiszta összehasonlítások: ha a gép mindkét operandust kiértékelné, csak lassabb lenne, az eredmény ugyanaz maradna. A C `&&` rövidzára azonban nem csak optimalizálás, hanem **szemantikai garancia**: a jobb oldali operandus csak akkor értékelődik ki, ha a bal oldali igaz. Ez akkor válik láthatóvá, amikor a jobb oldal érvényessége a bal oldaltól függ:

```c
/* C-referencia: igaz, ha p érvényes mutató, és nem üres karakterláncra mutat. */
int lab_nonempty(const char *p)
{
    if (p != NULL && *p != 0)
        return 1;
    return 0;
}
```

Itt a `*p` kiértékelése `NULL` mutatón definiálatlan viselkedés, a gépen jellemzően szegmentálási hiba. Az assemblyben ezért az ágak sorrendje maga a védelem: a betöltés csak a mutatóteszt *után*, annak igaz ágán következhet. Mentsük `lab-guard.s` néven:

```
// A64 ISA; GNU as; little-endian Linux.
// LAB_CASE: 0 = NULL mutató, 1 = üres karakterlánc, 2 = "A".
.ifndef LAB_CASE
.equ LAB_CASE, 2
.endif

.section .rodata
lab_empty: .byte 0
lab_text:  .asciz "A"

.section .text
.global _start
_start:
.if LAB_CASE == 0
    mov  x0, #0                     // p = NULL
.elseif LAB_CASE == 1
    adrp x0, lab_empty              // p -> ""
    add  x0, x0, :lo12:lab_empty
```

```
.else
    adrp x0, lab_text               // p -> "A"
    add  x0, x0, :lo12:lab_text
.endif

    // p != NULL && *p != 0
    cbz  x0, .Lfalse                // bal oldal hamis: *p NEM értékelődik ki
    ldrb w1, [x0]                   // csak érvényes mutatón olvasunk
    cbz  w1, .Lfalse                // jobb oldal hamis
    mov  w0, #1
    b    .Lexit

.Lfalse:
    mov  w0, #0

.Lexit:
    mov  w8, #93
    svc  #0

.section .note.GNU-stack,"",%progbits
```

Az `&&` bal operandusát itt nem `CMP`, hanem `CBZ` vizsgálja: a nullateszthez nem kell az NZCV. A lényeg a sorrend: az `LDRB` csak azon az úton érhető el, amelyen az `X0` már bizonyítottan nem nulla.

```
# Target: qemu-user, AArch64 Linux, little-endian.
for lab_case in 0 1 2; do
    aarch64-linux-gnu-as -EL -march=armv8-a --defsym LAB_CASE=$lab_case \
        -o lab-guard.o lab-guard.s &&
    aarch64-linux-gnu-ld -EL -e _start -o lab-guard lab-guard.o &&
    { qemu-aarch64 ./lab-guard; printf 'LAB_CASE=%d status=%d\n' "$lab_case" "$?"; }
done
```

Mért kimenet (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2):

`LAB_CASE`	Mutató	Mért állapot	Indoklás
0	`NULL`	0	a bal oldal hamis, a betöltés kimarad
1	`""`	0	a bal oldal igaz, a betöltött bájt 0
2	`"A"`	1	mindkét oldal igaz

Ellenpróba: felcserélt kiértékelési sorrend. Cseréljük fel a két első utasítást, mintha a gép „előbb betöltené az adatot, aztán döntene”:

```
    ldrb w1, [x0]                   // HIBÁS: a dereferálás megelőzi a NULL-tesztet
    cbz  x0, .Lfalse
    cbz  w1, .Lfalse
```

A `LAB_CASE=1` és `LAB_CASE=2` eset ugyanazt az eredményt adja, mint a helyes változat — a tiszta, érvényes bemenetekre írt tesztek tehát nem veszik észre a hibát. A `LAB_CASE=0` esetben viszont az `LDRB` a 0-s címet olvassa: a QEMU `qemu: uncaught target signal 11 (Segmentation fault) - core dumped` üzenettel áll le, a shell pedig `139` (128 + `SIGSEGV`) állapotot jelez (mérve: QEMU 8.2.2). Ezért kell a C rövidzáras sorrendjét az assemblyben is megőrizni, és ezért kell a tesztesetek közé érvénytelen bemenetet is felvenni.

6.2.4 A `switch` két tipikus leképezése

A C `switch` nem ír elő konkrét gépi megvalósítást. Két gyakori lehetőség:

- **Elágazáslánc:** a vezérlőértéket egymás után összehasonlítjuk az esetértékekkel.
- **Ugrótábla** (*jump table*): a vezérlőértékből indexet képezünk, majd egy táblából kiválasztjuk a végrehajtás célcímét.

Kevés vagy erősen szórt esethez az elágazáslánc egyszerű és helytakarékos lehet. Sűrű esetekhez az ugrótábla elkerüli a hosszú összehasonlítássorozatot.

Ez azonban nem merev szabály. A fordító figyelembe veheti az optimalizálási szintet, a kódméretet és az ágak gyakoriságát; használhat döntési fát vagy vegyes megoldást is. Ha az ágak csupán konstansokat adnak vissza, akár adatértékeket tartalmazó táblát is készíthet kódcímeket tartalmazó ugrótábla helyett.

6.2.5 Sűrű esetek: indexelt ugrótábla

A következő referenciafüggvényt most szándékosan ugrótáblával valósítjuk meg:

```
#include <stdint.h>

uint32_t dispatch(int32_t key)
{
    switch (key) {
    case 10: return 60;
    case 11: return 61;
    case 12: return 62;
    case 13: return 63;
    default: return 99;
    }
}
```

Az esetértékek a `[10, 13]` zárt intervallumot töltik ki. A táblához `0..3` index kell, ezért először levonjuk a `10` alsó határt.

Mentsük a teljes A64-programot `lab-switch.s` néven:

```
// A64 ISA; GNU as; little-endian Linux.
// LAB_KEY: a laborban -1 vagy 0..65535.
.ifndef LAB_KEY
.equ LAB_KEY, 12
.endif

.section .text
.global _start
_start:
    mov w0, #LAB_KEY

    sub w1, w0, #10
    cmp w1, #3
    b.hi .Ldefault

    adr x2, .Ltable
    ldr x3, [x2, w1, uxtw #3]
    br x3

.Lr60:
    mov w0, #60
    b .Lexit
```

```
.Lr61:
    mov w0, #61
    b .Lexit
.Lr62:
    mov w0, #62
    b .Lexit
.Lr63:
    mov w0, #63
    b .Lexit

.Ldefault:
    mov w0, #99

.Lexit:
    mov w8, #93
    svc #0

.section .rodata
.balign 8
.Ltable:
    .xword .Lr60, .Lr61, .Lr62, .Lr63

.section .note.GNU-stack,"",%progbits

# Target: qemu-user, AArch64 Linux, little-endian.
aarch64-linux-gnu-as -EL -march=armv8-a --defsym LAB_KEY=12 \
    -o lab-switch.o lab-switch.s
aarch64-linux-gnu-ld -EL -e _start -o lab-switch lab-switch.o
qemu-aarch64 ./lab-switch
lab_status=$?
printf 'status=%d\n' "$lab_status"
```

Mért kimenet (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): `status=62`.

6.2.5.1 Az indexellenőrzés levezetése

Az A64 `SUB W1, W0, #10` 32 bites, modulo 2^{32} eredményt képez. Az eredményt ezután **előjel nélküli indexként** vizsgáljuk.

`key`, előjeles	A64 `W1`, hexadecimális	`W1 > 3`, előjel nélkül	Útvonal
−1	`0xfffffff5`	Igaz	`default`
9	`0xffffffff`	Igaz	`default`
10	`0x00000000`	Hamis	Első bejegyzés
12	`0x00000002`	Hamis	Harmadik bejegyzés
13	`0x00000003`	Hamis	Negyedik bejegyzés
14	`0x00000004`	Igaz	`default`

A kivonás miatt az alsó határ alatti értékek nagy előjel nélküli indexszé válnak. Így az A64 `B.HI` egyetlen vizsgálattal kizárja mindkét oldali tartományhibát.

A megfelelő C-indexképzés:

```
uint32_t index = (uint32_t)key - 10u;
```

Ez nem azonos az általános, előjeles `key - 10` kifejezés kiértékelésével: utóbbi bizonyos bemeneteknél előjeles túlcsordulást okozhatna.

6.2.5.2 Hogyan lesz az indexből vezérlésátadás?

Az A64-példában:

1. Az A64 `ADR X2, .Ltable` kiszámítja a tábla címét.
2. Az A64 `LDR X3, [X2, W1, UXTW #3]` az indexet nullakiterjeszti 64 bitre, nyolccal szorozza, hozzáadja a táblacímhez, majd onnan 8 bájtot olvas.
3. Az A64 `BR X3` a beolvasott címre ugrik, visszatérési cím mentése nélkül.

Az effektív cím:

$$a = b + 8i$$

ahol a és b bájtcím, i mértékegység nélküli index, és az ellenőrzés után $0 \leq i \leq 3$.

A GNU `.xword` direktíva bejegyzésenként 8 bájtot hoz létre; a `.balign 8` 8 bájtos igazítást kér. A tábla tehát 32 bájtos. A linker helyettesíti a szimbolikus célokat tényleges kódcímekkel: nincs kézzel beírt memóriacím.

Kódolási korlátok:

- A `SUB` és az `ADR` korlátait az A64 immediate-korlátok összefoglalója adja; az ugrótábla az `ADR` ± 1 MiB-os hatótávolságán belül van.
- Ennél a 64 bites A64 `LDR` regisztereltolásos alaknál a skálázás `#0` vagy `#3` lehet; itt az utóbbi illeszkedik a bejegyzésmérethez.

Ez kisméretű, nem PIE laborprogram, abszolút kódcímeket tartalmazó táblával. Pozíciófüggetlen fordítói kódban gyakoriak a relatív eltolásokat tároló táblák is.

6.2.6 Szórt esetek: összehasonlítások lánca

Legyen a három esetérték `10`, `1000` és `60000`, eredményük rendre `60`, `61`, `62`. A teljes intervallumot lefedő, 8 bájtos bejegyzésű tábla 59 991 bejegyzést, azaz **479 928 bájtot** igényelne. Ehhez képest három összehasonlítás gazdaságos.

Másoljuk a `lab-switch.s` fájlt `lab-sparse.s` néven, majd az A64 `SUB` utasítástól az A64 `BR` utasításig terjedő döntési részt cseréljük erre:

```
// A64 ISA; a lab-sparse.s döntési részének helyettesítése.
    cmp w0, #10
    b.eq .Lr60
    cmp w0, #1000
    b.eq .Lr61
    mov w1, #60000
    cmp w0, w1
    b.eq .Lr62
    b .Ldefault
```

A megmaradó táblát és a `63`-as eredményblokkot ez a változat nem használja; eltávolíthatók.

A `60000` nem kódolható az A64 `CMP` immediate-alakjával: nem fér 12 bitbe, és nem ilyen érték 12 bites balra tolása. Egy 16 bites A64 `MOV` konstansmezőbe viszont belefér, ezért regiszteres összehasonlítást használunk.

```
# Target: qemu-user, AArch64 Linux, little-endian.
aarch64-linux-gnu-as -EL -march=armv8-a --defsym LAB_KEY=60000 \
    -o lab-sparse.o lab-sparse.s
aarch64-linux-gnu-ld -EL -e _start -o lab-sparse lab-sparse.o
qemu-aarch64 ./lab-sparse
lab_status=$?
printf 'status=%d\n' "$lab_status"
```

Mért kimenet (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): `status=62`; `LAB_KEY=11` esetén viszont `status=99`.

6.2.7 `default`, lefedettség és tipikus hibák

Az ugrótábla tartományellenőrzése csak az indexhatárokat ellenőrzi. Ha például a `12` eset hiányzik, a hozzá tartozó táblabejegyzésnek a `default` címére kell mutatnia. **A tartományon belüli lyuk is lefedendő bemenet.**

C-ben `default` nélkül egyetlen egyező eset hiányában a végrehajtás a `switch` után folytatódik. Ilyenkor az assembly megfelelő célja a közös folytatás, nem egy önkényesen kiválasztott eset. Szándékos C-átfolyásnál (*fall-through*) pedig nem szabad automatikusan minden eset végére kilépő elágazást tenni.

Hibakereséshez vizsgáljuk meg a tényleges gépi kódot:

```
# Target-objektumok: A64, qemu-user labor; hostoldali elemzés.
aarch64-linux-gnu-objdump -dr lab-if.o
aarch64-linux-gnu-objdump -dr lab-switch.o
```

Tünet	Hipotézis	Adatgyűjtés és egyetlen javítás
Mindig egy későbbi ág eredménye jelenik meg	Hiányzó ugrás miatt átfolyás történik	Disassemblyben követjük az eredményblokk végét; pótoljuk a közös kilépésre ugrást
Alsó határ alatti index átjut az ellenőrzésen	A64 `B.GT` került A64 `B.HI` helyére	Statikusan ellenőrizzük a feltételkódot; előjel nélküli feltételre javítjuk
A második táblabejegyzéstől hibás cél adódik	Hibás indexskálázás	Ellenőrizzük a 8 bájtos bejegyzést és az A64 `#3` skálázást; csak a skálát javítjuk
A hiányzó eset mégis eredményt kap	Rossz bejegyzés maradt a táblában	A megfelelő bejegyzést a `default` címére cseréljük

Ismerten hibás tartományellenőrzéssel ne futtassuk végig a programot: először statikus elemzéssel javítsuk, és csak utána végezzünk határértékteszteket.

6.2.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Kövesd végig az `if` programot `x=9`, illetve `x=30` esetén! Mely összehasonlítások maradnak ki a rövidzár miatt? **2. (implementációs, középszint — cél: rutinépítés)** A sűrű `switch` változatból töröld szemantikailag a `12` esetet, de tartsd meg az ugrótáblát! Ellenőrizd a `11`, `12`, `13` és `14` bemenetet. **3. (hibakeresési,**

középszint — cél: megértés-ellenőrzés) Egy módosított változatban az indexellenőrzés A64 `B.GT` utasítást használ. Mi történne `key=9` esetén? A hibás változat futtatása nélkül indokold meg a hibát, majd javítás után teszteld a `9`, `10`, `13`, `14` határértékeket!

6.2.9 Megoldások

1. megoldás — Segítség. Rajzold fel az `if`-láncot blokkonként, és minden `x`-re csak addig haladj, amíg az adott feltétel igaz vagy hamis volta ténylegesen eldönti a következő lépést.

1. megoldás — Teljes megoldás. `x=9` nem negatív, de kisebb `10`-nél. Ezért a `19`-cel való összehasonlítás kimarad. A `30` és `50` vizsgálata hamis, az eredmény `44`. `x=30` esetén az intervallum mindkét összehasonlítása megtörténik, de a második hamis. Ezután a `30`-cal való egyezés miatt az `50` vizsgálata marad ki; az eredmény `33`.

2. megoldás — Segítség. Az ugrótábla mérete és indexelése nem változik, csak az egyik bejegyzés célcíme — gondold át, hova kell mutatnia egy "törölt" esetnek.

2. megoldás — Teljes megoldás. A GNU `.xword` sor harmadik bejegyzése legyen `.Ldefault`. A tartományellenőrzés továbbra is `0..3` indexet enged. A várt állapotok sorrendben `61`, `99`, `63`, `99`: a `12` tartományon belüli lyuk, a `14` tartományon kívüli érték.

3. megoldás — Segítség. Számítsd ki, milyen bitmintát hagy maga után az indexszámítás `key=9`-re, és nézd meg, mit jelent ugyanaz a bitminta előjelesen, illetve előjel nélkül értelmezve.

3. megoldás — Teljes megoldás. `key=9` után az A64 `W1` értéke `0xffffffff`. Ez előjel nélkül 4 294 967 295, előjelesen -1. Az A64 `B.GT` ezért nem ugrana a `default` ágra, és érvénytelen táblán kívüli olvasás következhetne. A megfelelő utasítás az A64 `B.HI`. A hibás változat `key=9` futtatása a lektor mérésénél `139` állapotot adott (QEMU 8.2.2). Javítás után a mért állapotok: `99`, `60`, `63`, `99`.

6.2.10 Rövid összefoglaló

A C többágú döntéseit összehasonlításokkal, feltételes elágazásokkal és közös folytatási pontokkal képezhetjük le A64-re. Az ágak sorrendje és a rövidzár megőrzése a helyesség része: ha a jobb oldali operandus a bal oldal igaz voltától függ (például `p != NULL && *p != 0`), a sorrend felcserélése érvénytelen bemeneten hibás memóriaelérést okoz, amelyet a csak érvényes bemenetű tesztek nem mutatnak ki. A sűrű `switch` esetekhez ugrótábla, a szórtakhoz összehasonlításlánc lehet célszerű, de a fordító más szerkezetet is választhat. A normalizált index előjel nélküli ellenőrzése egyszerre kezelheti az alsó és felső határt. A táblán belüli lyukakat külön le kell fedni. Határértékekkel, kilépési állapottal és disassemblyvel ellenőrizzük a megvalósítást; utasításszámból önmagában nem állapítunk meg gyorsulást.

- **Vezérlési gráf:** a végrehajtási blokkok és lehetséges átmeneteik modellje.
- **Rövidzár:** a második logikai operandus feltételes kiértékelése.
- **Ugrótábla:** index alapján kiválasztott vezérlésátadási célok táblája.

6.2.11 Források és ellenőrzési hiányok

Az alábbiak dokumentációs kiindulópontok; közös állapotukat az Olvasói útmutató forrásállapot-táblája rögzíti:

- Arm A-profile architektúra-dokumentáció: DDI0487; A64 feltételkódok, `ADR`, `LDR` és `BR`.

- GNU assembler kézikönyv: AArch64 operandusalakok és adatdirektívák.
- GNU objdump kézikönyv: disassembly és relokációk vizsgálata.

Ellenőrzési hiányok: a közölt kilépési állapotok, a `lab-guard 139`-es szegmentálási hibája és a QEMU üzenete a lektor GNU as/ld 2.42 + QEMU 8.2.2 méréséből származnak (`mérve`). Az Ubuntu 24.04-ben telepített eszközverziókat és a tényleges disassemblyt a laborban kell ellenőrizni. Teljesítménymérést nem közöltünk; a QEMU user-mode futás nem szolgáltat garantáltan hardverhű ciklusszámot.

6.3 Ciklusszervezés: while/for/do-while mintázatok assemblyben

6.3.1 Miért számít a ciklus szerkezete?

Egy puffer feldolgozásakor nemcsak az fontos, mit végez a ciklusmag, hanem az is, hogy üres bemenetnél végrehajtódik-e, és egy kihagyott elem után biztosan továbblép-e az index. Assemblyben ezeket nem a C ciklusszintaxisa biztosítja: nekünk kell megfelelő helyre tenni az ellenőrzést, a léptetést és az ugrásokat.

Az if/else és switch leképezésénél megismert elágazásokból most ismétlődő vezérlési utakat építünk. **Minden alábbi assemblyutasítás és regiszter az A64 ISA, AArch64, I. rész környezetéhez tartozik.** A cél Armv8-A, little-endian Linux-program, elsődlegesen `qemu-aarch64` alatt.

Ciklusnak (*loop*) olyan vezérlési szerkezetet nevezünk, amely egy feltételtől függően ismételten végrehajt egy utasítássorozatot. A64-en nincs külön „C `for`” vagy „C `while`” utasítás: összehasonlítások, elágazások és visszafelé vezető vezérlési élek valósítják meg őket.

6.3.2 A három ciklusforma vezérlési mintája

Az alábbi C-részletekben `n` előjel nélküli, `uint64_t` típusú bemenet. A példákban az első `n` pozitív egész összegét számítjuk.

```
/* Szemantikai minták: n már rendelkezésre áll. */
uint64_t i = 1, sw = 0;
while (i <= n) {
    sw += i;
    ++i;
}

uint64_t sf = 0;
for (uint64_t j = 1; j <= n; ++j) {
    sf += j;
}

uint64_t k = 1, sd = 0;
do {
    sd += k;
    ++k;
} while (k <= n);
```

6.3.2.1 `while`: feltétel elöl

A feltétel-elöl tesztelő ciklus (*pre-test loop*) útvonala:

1. Inicializálás.
2. Feltételvizsgálat.
3. Hamis feltételnél kilépés.
4. Ciklusmag.
5. Visszaugrás a feltételvizsgálathoz.

Az A64-változatban az előjel nélküli `i <= n` feltétel tagadásával célszerű kilépni: `CMP Xi, Xn` után `B.HI` jelenti az előjel nélküli `i > n` esetet. Ha nincs elágazás, a végrehajtás a ciklusmaggal folytatódik.

`n = 0` esetén az első vizsgálat már hamis C-feltételt talál, ezért a ciklusmag egyszer sem fut le.

6.3.2.2 `for`: inicializálás, feltétel, lépés

A `for` három fejlécösszetevőjét külön vezérlési pontként kezeljük:

Összetevő	Végrehajtási hely
Inicializálás	Egyszer, az első feltételvizsgálat előtt
Feltétel	Minden lehetséges iteráció előtt
Lépés	A ciklusmag normál befejezése és a következő vizsgálat között

A fenti összegző `for` az alábbi `while` mintára vezethető vissza:

```
uint64_t sf = 0;
{
    uint64_t j = 1;
    while (j <= n) {
        sf += j;
        ++j;
    }
}
```

A külső blokk a C-változó hatókörét is megőrzi. A64-en a két összegző ciklus utasításszerkezete akár teljesen azonos lehet.

Fontos korlát: ez az egyszerű átírás csak akkor kezeli helyesen a `continue` működését, ha azt is átalakítjuk. A `for` ciklusban a `continue` a lépéshez vezet; a `while` ciklusban közvetlenül a feltételhez.

6.3.2.3 `do-while`: feltétel hátul

A feltétel-hátul tesztelő ciklus (*post-test loop*) útvonala:

1. Inicializálás.
2. Ciklusmag.
3. Feltételvizsgálat.
4. Igaz feltételnél visszaugrás a ciklusmaghoz.

Az első belépés előtt nincs vizsgálat, ezért a ciklusmag legalább egyszer lefut. A példában `n = 0` mellett is hozzáadjuk az első értéket: az eredmény `1`, a számláló pedig `2`.

A `do-while` tehát **nem általában ekvivalens** a `while` ciklussal. A példabeli összegzésnél `n >= 1` esetén azonos az eredményük; az üres tartományhoz előzetes őrfeltétel (*guard*) kellene.

6.3.3 `break` és `continue`: pontos ugráscélok

A64-en mindkettő megvalósítható feltétel nélküli `B` elágazással. Nem az utasítás különbözik, hanem a célpont.

C-utasítás	`while`	`for`	`do-while`
`break`	Ciklus utáni folytatás	Ciklus utáni folytatás	Ciklus utáni folytatás
`continue`	Feltételvizsgálat	Lépés, majd feltételvizsgálat	Ciklusvégi feltételvizsgálat

Egymásba ágyazott ciklusoknál a cél az adott C-utasítás által érintett legbelső szerkezethez tartozik. A C-ben `switch` is fogadhat `break` utasítást; ezért mindig a tényleges vezérlési célt kell kijelölni, nem pusztán a legközelebbi assemblycímkét.

6.3.4 Ciklusinvariáns: mit tudunk minden visszaérkezéskor?

A **ciklusinvariáns** (*loop invariant*) olyan állítás, amely egy kijelölt vezérlési pont minden elérésekor igaz. Nem feltétlenül igaz a ciklusmag minden köztes utasítása után.

Az összegző `while` feltételvizsgálati pontján legyen:

$$s = \frac{(i-1)i}{2}, \qquad 1 \le i \le n+1.$$

Itt `s`, `i` és `n` mértékegység nélküli egész értékek. A laborban feltételezzük, hogy $0 \le n \le 255$, így sem a számláló, sem az összeg nem fordul át a 64 bites tartomány határán.

Levezetés:

- **Inicializálás:** `i = 1`, `s = 0`, tehát az állítás teljesül.
- **Megőrzés:** a ciklusmag előbb hozzáadja `i` értékét: $s' = (i-1)i/2 + i = i(i+1)/2$. Ezután `i' = i + 1`, ezért ismét $s' = (i'-1)i'/2$.
- **Kilépés:** az első hamis feltételnél `i = n + 1`, ezért $s = n(n+1)/2$.

A helyes eredmény indoklása önmagában még nem mutatja a befejeződést. Ehhez megfigyeljük, hogy minden végrehajtott iteráció egyet csökkent a hátralévő elemek számán, azaz `n - i + 1` értékén.

A bemeneti korlát lényeges: teljes `uint64_t` tartományban például `n = UINT64_MAX` mellett a számláló átfordulása megakadályozná a várt kilépést.

6.3.5 Kidolgozott példa: három összegzés és egy megszakított ciklus

Munkakönyvtár: `~/arm-labor/a64/a020/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a020
cd ~/arm-labor/a64/a020
```

A teljes program `n = 5` mellett lefuttatja a három összegzést. Negyedikként ezt a C-mintát valósítja meg:

```
uint64_t filtered = 0;
for (uint64_t j = 1; j <= n; ++j) {
    if (j == 2)
        continue;
    if (j == 4)
        break;
    filtered += j;
}
```

A negyedik eredmény `1 + 3 = 4`: a `2` kimarad, a `4` feldolgozása előtt pedig kilépünk.

Mentsük a következőt `lab-loops.s` néven. Az assemblyblokk A64 ISA-környezetű; a ponttal kezdődő direktívák GNU `as`, AArch64 ELF-direktívák, nem végrehajtott utasítások.

```
// A64 ISA, Armv8-A, Linux, little-endian.
.ifndef LAB_N
.equ LAB_N, 5
.endif

.text
.global _start
.global lab_done
.type _start, %function

_start:
    movz x9, #LAB_N             // n, előjel nélküli 64 bit

    // while: x1 = i, x2 = összeg
    movz x1, #1
    movz x2, #0
.Lwhile_test:
    cmp  x1, x9
    b.hi .Lwhile_end
    add  x2, x2, x1
    add  x1, x1, #1
    b    .Lwhile_test
.Lwhile_end:

    // for: x3 = j, x4 = összeg
    movz x3, #1                 // inicializálás
    movz x4, #0
.Lfor_test:
    cmp  x3, x9                 // feltétel
    b.hi .Lfor_end
    add  x4, x4, x3             // ciklusmag
.Lfor_step:
    add  x3, x3, #1             // lépés
    b    .Lfor_test
.Lfor_end:

    // do-while: x5 = k, x6 = összeg
    movz x5, #1
```

```
    movz x6, #0
.Ldo_body:
    add  x6, x6, x5
    add  x5, x5, #1
    cmp  x5, x9
    b.ls .Ldo_body              // előjel nélküli <=

    // for break/continue használatával:
    // x7 = j, x10 = szűrt összeg
    movz x7, #1
    movz x10, #0
lab_filter_test:
    cmp  x7, x9
    b.hi .Lfilter_end

    cmp  x7, #2
    b.eq .Lfilter_step          // continue: nem a teszthez!
    cmp  x7, #4
    b.eq .Lfilter_end           // break

    add  x10, x10, x7
.Lfilter_step:
    add  x7, x7, #1
    b    lab_filter_test
.Lfilter_end:

lab_done:
    // Az n = 5 teszteset összegeredményeinek ellenőrzése.
    cmp  x2, #15
    b.ne .Lfail
    cmp  x4, x2
    b.ne .Lfail
    cmp  x6, x2
    b.ne .Lfail
    cmp  x10, #4
    b.ne .Lfail

    movz w0, #0                 // sikeres ellenőrzés
    b    .Lexit
.Lfail:
    movz w0, #1                 // eltérő eredmény
.Lexit:
    movz w8, #93                // Linux AArch64 exit
    svc  #0

.size _start, .-_start
.section .note.GNU-stack,"",%progbits
```

Az A64-példa kódolási korlátai:

- A `MOVZ` 16 bites közvetlen értéket (*immediate*) helyez el egy 16 bites sávban. X-regiszternél a választható eltolás `0`, `16`, `32` vagy `48` bit; W-regiszternél `0` vagy `16`. Itt minden eltolás `0`.
- Az `ADD` és a `CMP` használt immediate-alakja 12 bites előjel nélküli értéket fogad, opcionális `12` bites balra tolással. Itt minden konstans eltolás nélkül kódolható.
- A `B` címkére mutató eltolása 26 bites előjeles, a `B.cond` eltolása 19 bites előjeles mező, mindkettő 4 bájtos léptékkel. A rövid program minden célpontja elérhető.
- Az `SVC` immediate-mezője 16 bites; a Linux-hívás itt `SVC #0`.

Az A64 `CMP` állítja az NZCV jelzőket, a következő feltételes elágazás pedig ezeket olvassa. A ciklusfeltételeknél az előjel nélküli C-típushoz ezért `B.HI`, illetve `B.LS` tartozik, nem az előjeles `B.GT` és `B.LE`.

6.3.6 Összeállítás, futtatás és állapotellenőrzés

A következő parancsok Ubuntu 24.04 x86_64 hoston, a könyv cross-eszközláncával használhatók.

```
# Target: qemu-user, AArch64 Linux, Armv8-A, little-endian.
for n in 0 1 5; do
    aarch64-linux-gnu-as -g -march=armv8-a -EL \
        --defsym LAB_N="$n" -o lab-loops.o lab-loops.s
    aarch64-linux-gnu-ld -m aarch64linux -e _start \
        -o lab-loops lab-loops.o
    timeout 2s qemu-aarch64 ./lab-loops
    printf 'n=%s kilepesi_allapot=%d\n' "$n" "$?"
done

aarch64-linux-gnu-objdump -d lab-loops
```

A 3. feladat szándékos végtelen ciklust hoz létre; futtasd `timeout 2s qemu-aarch64 ./lab-loops`-szal. A `124`-es állapot időtúllépést jelent; GDB-ben a `Ctrl+C` állítja meg a futást.

Mért eredmény (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): nincs programkimenet, a shell `n=5 kilepesi_allapot=0` sort ír. Az `1` állapot azt jelzi, hogy valamelyik összeg eltér az `n = 5` tesztesethez rögzített értéktől; a `n = 0` és `n = 1` futásnál ezért a beépített ellenőrzés szándékosan eltérést jelez. Ez az önellenőrzés nem általános bemeneti tesztkeret.

Az A64-regiszterek mért állapota a `lab_done` címke első utasítása előtt (GDB 15.1, QEMU 8.2.2):

Regiszter	Szerep	Mért érték, előjel nélküli
X9	`n`	5
X1, X3, X5	A három összegzés számlálói	6
X2, X4, X6	A három összegzés eredményei	15
X7	A megszakított ciklus számlálója	4
X10	Szűrt összeg	4

Az első terminálban indítsuk a GDB-kapcsolatra váró emulációt:

```
# Target: qemu-user; a folyamat a debuggerre vár.
qemu-aarch64 -g 1234 ./lab-loops
```

A második terminálban:

```
# Target: a fenti qemu-user folyamat; mért állapot a táblázatban.
gdb-multiarch -q ./lab-loops \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_done' \
    -ex 'continue' \
    -ex 'info registers x9 x1 x2 x3 x4 x5 x6 x7 x10'
```

A GDB gyakran hexadecimálisan is megjeleníti az értékeket: `15 = 0xf`. A `continue` debugger-paranccsal az ellenőrzés és a kilépés is végrehajtható.

6.3.7 Tipikus hibák és ellenpróbák

Egy iterációval kevesebb. Ha az A64 `B.HI` helyett `B.HS` szerepel a felső tesztben, akkor már `i == n` esetén kilépünk. `n = 5` mellett az összeg `10`, nem `15`. Hipotézis: hibás határreláció. Az `objdump` kimenetében ellenőrizzük az elágazást, majd kizárólag a feltételkódot javítsuk.

Végtelen ciklus a `continue` miatt. Ha a szűrt ciklus `2` esetén közvetlenül `lab_filter_test` címkére ugrik, X7 változatlanul `2` marad. A következő kör ugyanoda jut. GDB-ben megszakítás után az ismétlődő utasításút és X7 értéke igazolja a hipotézist.

Az üres bemenetet nem teszteljük. `n = 1` mellett mindhárom összegzés `1` eredményt ad, tehát ez nem különbözteti meg a belépési szabályokat. `n = 0` mellett viszont a `while` és a `for` összege `0`, a `do-while` összege `1`.

Más összehasonlítás jelzőit használjuk. A feltételes A64-elágazás nem „emlékszik” a C-kifejezésre: az aktuális NZCV-t használja. Ha a megfelelő `CMP` után más jelzőmódosító utasítás kerül, a vizsgált feltétel megváltozhat. Az utasításszintű léptetés ezt láthatóvá teszi.

6.3.8 Feladatok

1. (magyarázó, középszint — cél: megértés-ellenőrzés) Adjuk meg a három összegzés ciklusmagjának végrehajtásszámát és eredményét `n = 0`, `1`, illetve `5` mellett. Miért nem elég a `for` szöveges átírása `while` alakra, ha a ciklusmag `continue` utasítást tartalmaz?

2. (implementációs, középszint — cél: rutinépítés) Cseréljük le a labor első, `while` szervezésű összegzését előzetes őrfeltétellel ellátott, feltétel-hátul tesztelő változatra. X1 és X2 végállapota minden `0 <= n <= 255` bemenetre maradjon azonos az eredetivel.

3. (hibakeresési, középszint — cél: megértés-ellenőrzés) A szűrt ciklus `continue` ágának célját szándékosan módosítsuk `lab_filter_test` címkére. Állítsuk össze újra a programot, és GDB-vel azonosítsuk a befejeződést megakadályozó állapotot. Egyetlen módosítással javítsuk, majd ellenőrizzük ismét a kilépési állapotot.

6.3.9 Megoldások

1. megoldás — Segítség. Kövesd külön-külön a három ciklus számlálóját `n=0`-ra — melyik fut le legalább egyszer akkor is, ha a feltétel már a belépéskor hamis? A `continue`/`for` kérdéshez nézd meg, hova ugrik a `for` fejlécének “lépés” része a `while`-átiratban.

1. megoldás — Teljes megoldás. A levezetett eredmények:

`n`	`while`/`for`: végrehajtások	`while`/`for`: összeg	`do-while`: végrehajtások	`do-while`: összeg
0	0	0	1	1
1	1	1	1	1
5	5	15	5	15

A `for` ciklusban a `continue` végrehajtja a fejléc lépését is. A naiv `while`-átiratban átugraná a ciklusmag végére helyezett léptetést. Az ekvivalenciához ezt az útvonalat külön meg kell őrizni.

2. megoldás — Segítség. Illessz egy `cmp`/`b.hi` párt a ciklusmag elé, amely `n=0` esetén közvetlenül a vég címkére ugrik, mielőtt a `do-while`-stílusú test lefutna.

2. megoldás — Teljes megoldás. Az első összegző blokk helyére az alábbi A64-részlet tehető:

```
// A64 ISA; a lab-loops.s első ciklusának helyettesítő blokkja.
    movz x1, #1
    movz x2, #0
    cmp  x1, x9
    b.hi .Lguarded_end

.Lguarded_body:
    add  x2, x2, x1
    add  x1, x1, #1
    cmp  x1, x9
    b.ls .Lguarded_body
.Lguarded_end:
```

Az őrfeltétel üres tartománynál megakadályozza az első végrehajtást. Egyébként ugyanazokat az értékeket adjuk össze, ugyanannyi léptetéssel. Az eredeti buildparancsokkal `n = 5` mellett továbbra is `0` a várt kilépési állapot.

Ha a bemenetet `0` értékre módosítjuk, a `lab_done` ponton X1 várt értéke `1`, X2 értéke `0`. A program változatlan, `n = 5` értékre rögzített végellenőrzése ekkor szándékosan hibát jelez.

3. megoldás — Segítség. Tegyél töréspontot `lab_filter_test`-re, és figyeld X7-et egymás utáni megállásokkor — ha ugyanaz az érték ismétlődik, a léptetés nem fut le a `continue` ágon.

3. megoldás — Teljes megoldás. A hibás programban X7 eléri a `2` értéket, majd a léptetést megkerülve visszaugrik a teszthez. GDB-ben a `lab_filter_test` töréspont ismételt elérésekor X7 tartósan `2`; X10 már `1`.

A javítás: a `B.EQ` célja ismét `.Lfilter_step` legyen. Így a kihagyott elem után X7 `3` lesz, később pedig `4` mellett a `break` kiléptet. Újraösszeállítás után a várt kilépési állapot `0`.

6.3.10 Rövid összefoglaló

A C ciklusformák A64-en összehasonlításokból, elágazásokból és gondosan kijelölt vezérlési pontokból állnak. A `while` belépés előtt ellenőriz, a `do-while` csak a ciklusmag után, ezért az utóbbi legalább egyszer végrehajtódik. A `for` inicializálásra, feltételre és lépésre bontható; `while` alakra visszavezetésekor a `continue` útvonalát is meg kell őrizni. A helyességet nemcsak a végső összeg, hanem a számlálók és a határesetek vizsgálata is támogatja. A ciklusinvariáns az állapot és az elvárt eredmény kapcsolatát teszi követhetővé. A befejeződést külön indokoljuk, figyelve a számláló átfordulására.

- **Ciklusinvariáns:** kijelölt ciklspontban mindig igaz állítás.
- **Őrfeltétel:** a ciklusmag első végrehajtását engedélyező vizsgálat.
- **Lépéspont:** a `for` normál és `continue` útvonalának közös léptetési helye.

6.3.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció: DDI0487; A64 összehasonlítások, feltételkódok és elágazáskódolások. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.

- GNU assembler kézikönyv: AArch64 szintaxis, címkék és ELF-direktívák. **Ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv: távoli hibakeresés, töréspontok és regisztervizsgálat. **Ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kilépési állapot és regiszterértékek a lektor GNU as/ld 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). Az Ubuntu 24.04-en telepített konkrét eszközverziókkal a laborban kell ellenőrizni a buildet és a debuggerkapcsolatot. Teljesítménymérést nem végeztünk.

6.4 Hívás és visszatérés: BL/RET bevezető mintája

6.4.1 Miért kell külön utasítás a híváshoz?

Az előző alfejezetben egy ciklus vezérlését elágazásokkal szerveztük meg. Most képzeljük el, hogy ugyanazt a rövid számítást több helyről szeretnénk elvégezni. A kód másolása helyett külön függvényt készítünk, amely a számítás után **mindig az aktuális hívási hely után folytatja a végrehajtást**.

Ehhez a célcím önmagában nem elég: a folytatás helyét is meg kell őrizni. Az **A64 ISA-ban, AArch64 környezetben** ezt a legegyszerűbb esetben a `BL` és a `RET` utasításpár biztosítja.

Ebben az alfejezetben kizárólag A64-kód szerepel, Armv8-A alapprofillal, little-endian céllal. Nem építünk veremkeretet, és nem mentünk regisztert a memóriába. Olyan függvényt írunk, amely nem hív további függvényt, és csak az eredményregisztert módosítja.

Az előfeltétel az A64 ciklusszervezés és a korábban használt Linux syscall-konvenció ismerete.

6.4.2 A hívás és a visszatérés állapotváltozása

A **visszatérési cím** (*return address*) a hívás után végrehajtandó utasítás címe. A **kapcsolóregiszter** (*link register*, LR) A64-en az `X30` regiszter szokásos szerepneve.

A64 utasítás	Vezérlési hatás	Az A64 `X30` változása
`B címke`	Ugrás a célhoz	Nem változik
`BL címke`	Ugrás a célhoz, a folytatás megjegyzésével	A hívást követő utasítás címét kapja
`RET`	Ugrás az `X30` által megadott címre	Nem változik

Az A64 `RET` operandus nélküli alakja a `RET X30` rövid írásmódja. Az utasításnak van más általános célú címregisztert megadó alakja is, de itt mindig az alapértelmezett `X30`-at használjuk.

Fontos különbség a magas szintű elképzeléshez képest:

> Az A64 `BL` nem készít automatikusan veremkeretet, és nem helyez visszatérési címet a veremre. Az A64 `RET` sem olvas automatikusan címet a veremből.

Mindkét utasítás vezérlésátadást végez. A függvényparaméterek elhelyezése, a regiszterek megőrzése és a verem kezelése külön szoftveres szerződés feladata.

6.4.2.1 A visszatérési cím levezetése

Legyen P az A64 `BL` utasítás címe, bájtban értelmezve. Az A64 utasítások hossza 4 bájt, ezért a közvetlenül következő utasítás címe:

$$L = P + 4.$$

A hívás egyszerű, kivételmentes végrehajtásának hatása:

1. az A64 `X30` megkapja L értékét;
2. a végrehajtás a hívás célcímén folytatódik;
3. a meghívott kód végén az A64 `RET` visszaugrik L-re.

Az A64 `BL` közvetlen célját előjeles, 26 bites, 4 bájttal skálázott relatív eltolás kódolja. A hívó utasításhoz képesti tartomány:

$$-2^{27} \le \Delta \le 2^{27} - 4 \qquad \text{bájtban, 4 bájtos lépésekben.}$$

Ez körülbelül ±128 MiB. A példákban a cél közeli, ugyanabban a kódszekcióban található címke. A nagy távolságú hívások linkeroldali megoldása nem része ennek a bevezetőnek.

6.4.3 Levélfüggvény: mikor elég egyetlen visszatérési cím?

A **levélfüggvény** (*leaf function*) nem hív másik függvényt. Emiatt a saját végrehajtása során nincs olyan beágyazott A64 `BL`, amely felülírná az érkezéskor kapott `X30` értékét.

Ez azonban nem jelenti azt, hogy minden levélfüggvény mentés nélkül megírható:

- az A64 `X30`-at maga a függvény is felülírhatná;
- megőrzendő regiszterek használata mentést igényelhet;
- lokális adatokhoz veremterületre lehet szükség.

A mi levélfüggvényünknek nincs mentési kényszere, mert csak az A64 `W0` eredményregisztert módosítja, és az `X30`-at érintetlenül hagyja.

A teljes hívási konvenció (*calling convention*) helyett most az AAPCS64 egy szűk részét használjuk: egy 32 bites egész argumentum az A64 `W0`-ban érkezik, és a 32 bites eredmény ugyanott távozik. Az AAPCS64 regiszterszerződés később pontosan meghatározza a további kötelezettségeket.

6.4.4 Kidolgozott példa: egy 32 bites érték átalakítása

Munkakönyvtár: `~/arm-labor/a64/a021/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a021
cd ~/arm-labor/a64/a021
```

A számítás C-ben a `uint32_t` típusú bemenetre értelmezett `2u * n + 1u` kifejezésnek felel meg. Az eredményt modulo 2^{32} értelmezzük, tehát előjel nélküli, 32 bites aritmetikát használunk.

Mentsük a következő teljes programot `lab-call.s` néven:

```
// A64 ISA / AArch64, Armv8-A, little-endian.
// A direktívák GNU as / AArch64 ELF kontextusúak.

.section .text
.p2align 2
.global _start
.global lab_after_call
.global lab_transform

.type _start, %function
_start:
    mov     w0, #21
    bl      lab_transform

lab_after_call:
    mov     x8, #93
    svc     #0
.size _start, . - _start

.type lab_transform, %function
lab_transform:
    add     w0, w0, w0
    add     w0, w0, #1
    ret
.size lab_transform, . - lab_transform

.section .note.GNU-stack,"",%progbits
```

Az A64 `W0` az `X0` alsó 32 bites nézete. A `W0` írása az `X0` felső 32 bitjét nullázza. A visszatérési címet viszont a teljes, 64 bites A64 `X30` hordozza: nem a `W30` nézetet használjuk címként.

A program A64 immediate-operandusai kódolhatók:

- a két `MOV` kis pozitív konstansa egyetlen, 16 bites immediate-ot tartalmazó `MOVZ` megfelelő aliasalakjával előállítható, nulla eltolással;
- az `ADD` immediate-változata 12 bites előjel nélküli értéket fogad, opcionális 12 bites balra tolással; itt az érték 1, az eltolás 0;
- a `SVC` immediate-mezője 16 bites, itt 0.

A 93 az AArch64 Linux `exit` rendszerhívásának száma, nem az A64 ISA tulajdonsága. A Linux syscall-azonosító az A64 `X8`-ba kerül; nem a `SVC` immediate-mezőjébe.

6.4.4.1 Összeállítás, linkelés, futtatás

Cél: **AArch64 Linux, QEMU user-mode**, Ubuntu 24.04 x86_64 hoston, a könyv cross-eszközláncával.

```
# Target: qemu-user / AArch64 Linux, little-endian
aarch64-linux-gnu-as -march=armv8-a -EL -g \
    -o lab-call.o lab-call.s

aarch64-linux-gnu-ld -m aarch64linux -e _start \
    -o lab-call.elf lab-call.o

qemu-aarch64 ./lab-call.elf
lab_status=$?
printf 'Kilépési állapot: %s\n' "$lab_status"
```

Mért eredmény (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): nincs szabványos kimenetre írás; a shell által megjelenített kilépési állapot 43. A 43 itt szándékosan számítási eredmény, nem laborhiba. A parancsokat ne olyan shellben futtassuk, amely nem nulla státusznál automatikusan megszakítja a parancssorozatot.

Végrehajtási pont	A64 `W0`, előjel nélküli érték	A64 `X30`
Közvetlenül a hívás előtt	21	A program nem támaszkodik rá
Belépés a `lab_transform` függvénybe	21	`lab_after_call` címe
Első A64 `ADD` után	42	Változatlan
Második A64 `ADD` után	43	Változatlan
A64 `RET` után	43	Változatlan

A `_start` nem szokásos függvényhívással kapta meg a vezérlést, ezért nem A64 `RET` utasítással fejezzük be. A folyamatból a Linux rendszerhívással lépünk ki.

6.4.4.2 Az állapot ellenőrzése GDB-vel

Először nézzük meg a tényleges utasításokat:

```
# Target: AArch64 ELF elemzése a hoston
aarch64-linux-gnu-objdump -d lab-call.elf
```

Keressük meg az A64 `BL` utáni utasítást és a `lab_after_call` szimbólumot: ugyanazt a folytatási pontot kell jelölniük.

A következő munkafolyamat a függvény belépésénél megáll, majd végrehajtja a két összeadást és a visszatérést:

```
# Target: qemu-user / AArch64 Linux; helyi GDB-kapcsolat
qemu-aarch64 -g 1234 ./lab-call.elf &
lab_qemu_pid=$!

gdb-multiarch -q -batch ./lab-call.elf \
    -ex 'target remote 127.0.0.1:1234' \
    -ex 'break lab_transform' \
    -ex 'continue' \
    -ex 'info registers x0 x30 pc' \
    -ex 'p/x (unsigned long)&lab_after_call' \
    -ex 'si' \
    -ex 'si' \
    -ex 'si' \
    -ex 'info registers x0 x30 pc' \
    -ex 'continue'

wait "$lab_qemu_pid"
lab_status=$?
printf 'QEMU kilépési állapot: %s\n' "$lab_status"
```

Mért ellenőrzések (GDB 15.1, QEMU 8.2.2):

- a belépésnél az A64 `X0` értéke `0x15`, azaz 21;
- az A64 `X30` megegyezik a `lab_after_call` címével;
- három utasításlépés után az A64 programszámláló, amelyet a GDB `pc` néven mutat, a `lab_after_call` címén áll;
- ekkor az A64 `X0` értéke `0x2b`, azaz 43.

A GDB a `pc`-t `<_start+8>` alakban is kiírhatja, mert a `.size _start` a teljes `_start` blokkot lefedi, és a GDB a függvényszimbólumot preferálja; a számértéket hasonlítsd a `p/x &lab_after_call` eredményével.

A konkrét betöltési címeket nem rögzítjük: a szimbólumokhoz való viszonyt ellenőrizzük. A GDB kimenetének formázása verziónként eltérhet.

6.4.5 Mi romlik el beágyazott híváskor?

Egyetlen A64 `X30` regiszter csak egy visszatérési címet tárol. Ha egy függvény újabb A64 `BL` utasítást hajt végre, a korábbi cím elvész, hacsak előtte nem őrizte meg.

A következő **szándékosan hibás**, teljes program legyen `lab-nested-bad.s`:

```
// A64 ISA / AArch64, Armv8-A, little-endian.
// Szándékos vezérlésihiba-demonstráció.
// Direktívák: GNU as / AArch64 ELF.

.section .text
.p2align 2
.global _start
.global lab_outer
.global lab_bad_resume

.type _start, %function
_start:
    mov     w0, #21
    bl      lab_outer

lab_process_exit:
    mov     x8, #93
    svc     #0
.size _start, . - _start

.type lab_outer, %function
lab_outer:
    bl      lab_inner
lab_bad_resume:
    ret
.size lab_outer, . - lab_outer

.type lab_inner, %function
lab_inner:
    add     w0, w0, w0
    add     w0, w0, #1
    ret
.size lab_inner, . - lab_inner

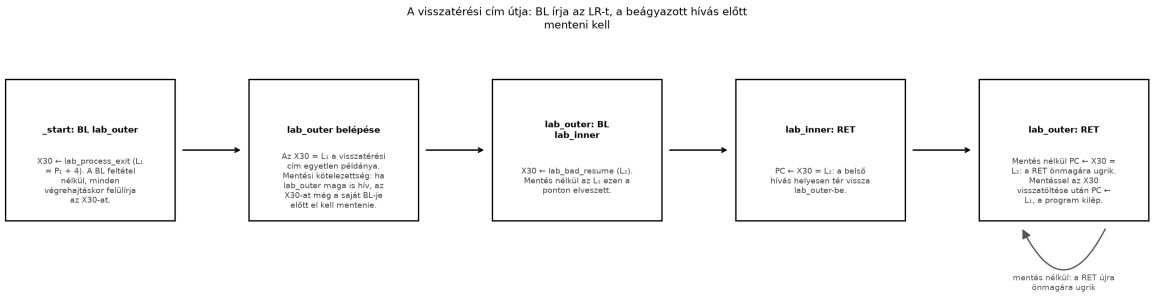
.section .note.GNU-stack,"",%progbits
```

A hiba levezetése:

Lépés	A64 vezérlésátadás	A64 `X30` utána
1	`_start` meghívja `lab_outer`-t	`lab_process_exit` címe
2	`lab_outer` meghívja `lab_inner`-t	`lab_bad_resume` címe
3	`lab_inner` visszatér	Továbbra is `lab_bad_resume` címe
4	A `lab_bad_resume` alatti A64 `RET` végrehajtódik	Változatlan

A negyedik lépésben az A64 `RET` **saját magára ugrik vissza**. A program végtelen ciklusba kerül, miközben az A64 `W0` már helyesen 43. Az aritmetikai eredmény helyessége tehát nem igazolja a vezérlés helyességét.

Az alábbi állapotábra egyetlen láncba köti a három tényt, amelyből a hiba következik: (1) a `BL` **feltétel nélkül** írja az `X30`-at, akkor is, ha abban még szükséges cím van; (2) a visszatérési címnek érkezéskor egyetlen példánya van, az `X30`; (3) ezért minden nem-levél függvénynek a saját `BL`-je **előtt** meg kell őriznie a kapott címet. Ez a harmadik pont a későbbi veremkezelés kiindulópontja: a mentés helyét és formáját (legkisebb alakjában `STP X29, X30, [SP, #-16]!` és `LDP X29, X30, [SP], #16`) a Prológus és epilógus alfejezet, a kötelezettség szerződéses alapját Az AAPCS64 regiszterszerződés adja.



6.1. ábra: A visszatérési cím útja: BL írja az LR-t, a beágyazott hívás előtt menteni kell - Az X30 (LR) állapota a lab-nested-bad.s futása során; a mentés megvalósítását a Prológus és epilógus alfejezet adja.

```
# Target: qemu-user / AArch64 Linux, little-endian
aarch64-linux-gnu-as -march=armv8-a -EL -g \
    -o lab-nested-bad.o lab-nested-bad.s

aarch64-linux-gnu-ld -m aarch64linux -e _start \
    -o lab-nested-bad.elf lab-nested-bad.o

timeout --foreground 1s qemu-aarch64 ./lab-nested-bad.elf
lab_status=$?
printf 'Timeout utáni állapot: %s\n' "$lab_status"

aarch64-linux-gnu-objdump -d lab-nested-bad.elf
```

Mért eredmény: a GNU `timeout` 124-es állapota. Az 1 másodperc működtetési korlát, nem teljesítménymérés. A timeout önmagában még nem bizonyítja a feltételezett hibát.

A gyökérok ellenőrzéséhez az előző GDB-lépéssorban használjuk ezt az ELF-fájlt, és a töréspont legyen `lab_bad_resume`. Az A64 `pc` és `X30` ekkor azonos címet mutat (a mért futásban `0x400014`, ami `&lab_bad_resume`); egyetlen utasításlépés után is ugyanaz marad mindkettő. A hibás programot ne engedjük korlátlanul továbbfutni; a GDB `kill` parancsával szándékosan megszakítható.

6.4.6 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés és célzott ellenpróba
A függvény számol, de nem tér vissza helyesen	A hívás A64 `B`, nem `BL`	Disassembly; az `X30` vizsgálata a belépésnél; csak a hívó utasítást javítsuk
Beágyazott hívás után végtelen ciklus	A belső A64 `BL` felülírta az `X30`-at	GDB-ben hasonlítsuk össze az `X30`-at a külső és belső folytatási címekkel
A függvényből visszatér, de a folyamat végén hibázik	A `_start` A64 `RET` utasítással zárul	Ellenőrizzük a kilépési útvonalat; használjuk a Linux `exit` syscallt
A visszatérési cím váratlanul megváltozik	A kód az A64 `X30`-at vagy `W30`-at munkaregiszterként használta	Keressük meg a regiszter összes írását, majd különítsük el a számítást a vezérlési állapottól

Nem általános javítás az A64 `X30` átmásolása egy tetszőleges másik regiszterbe. Tudnunk kellene, hogy a meghívott függvény azt megőrzi-e, illetve nekünk kell-e megőriznünk annak eredeti értékét.

A szabályos nem-levél függvény megoldásához a veremkeret, az AAPCS64 regiszterszerződés, valamint a prológus és epilógus együtt szükséges. Ezeket a könyv függvénykezelési fejezete tárgyalja; itt nem vezetünk be félkész mentési mintát.

6.4.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Egy A64 `BL` a P címen áll, és a `lab_transform` függvényt hívja. Mi kerül az A64 `X30`-ba? Miért nem helyettesíthető általában a hívás A64 `B` utasítással? Jelenti-e a levélfüggvény fogalma, hogy soha nincs szükség regisztermentésre?

2. (implementációs, alapszint — cél: rutinépítés) Készíts `lab-two-calls.s` fájlt a helyes példából. A belépési pont egymás után kétszer hívja ugyanazt a levélfüggvényt, a második hívás bemenete az első eredménye legyen. A `lab_after_call` címkét a második hívás után helyezd el. Vezesd le és ellenőrizd a kilépési állapotot.

3. (hibakeresési, középszint — cél: rutinépítés) A hibás programban GDB-vel igazold az A64 `pc` és `X30` egyenlőségét a beragadó visszatérésnél. Ezután készíts `lab-nested-fixed.s` változatot: a külső függvény saját maga végezze el a számítást, belső hívás nélkül. Egyetlen szerkezeti változtatást végezz: a beágyazott hívást váltsd ki a számítás törzsével. Ellenőrizd a befejeződést is.

6.4.8 Megoldások

1. megoldás — Segítség. Az A64 utasítások mind pontosan 4 bájt hosszúak — mi lesz a P című `BL` UTÁNI utasítás pontos címe? A `B` utasítás táblázatát nézve: ír-e valaha `X30`-ba?

1. megoldás — Teljes megoldás. Az A64 `X30` értéke $P + 4$, mert a hívó utasítás 4 bájtos. Az A64 `B` nem állítja be ezt a címet, ezért a későbbi `RET` nem kap megfelelő visszatérési információt. A levélfüggvény csak a további hívások hiányát jelenti; megőrzendő regiszter használata ettől még mentést igényelhet.

2. megoldás — Segítség. A második `bl lab_transform` ugyanazt a függvényt hívja, de már az ELSŐ hívás `W0` kimenetével, mint új bemenettel — mit ad `2*(2*21+1)+1`?

2. megoldás — Teljes megoldás. Az A64 hívó kód lényegi része:

```
// A64 ISA / AArch64: részlet a lab-two-calls.s fájlhoz.
_start:
    mov     w0, #21
    bl      lab_transform
    bl      lab_transform
lab_after_call:
    mov     x8, #93
    svc     #0
```

Az első eredmény 43, a második $2 \cdot 43 + 1 = 87$. Az egymás utáni hívások nem beágyazottak: az első már visszatért, mielőtt a második elkezdődik.

```
# Target: qemu-user / AArch64 Linux
aarch64-linux-gnu-as -march=armv8-a -EL -g \
    -o lab-two-calls.o lab-two-calls.s
aarch64-linux-gnu-ld -m aarch64linux -e _start \
    -o lab-two-calls.elf lab-two-calls.o
qemu-aarch64 ./lab-two-calls.elf
lab_status=$?
printf 'Várt állapot 87, kapott: %s\n' "$lab_status"
```

3. megoldás — Segítség. A `lab_outer`-ben lévő `bl lab_inner` felülírja `X30`-at — mi marad a `RET`-nek, amikor `lab_bad_resume`-nál újra `X30`-ra hivatkozik?

3. megoldás — Teljes megoldás. A hibás változatban a belső hívás az A64 `X30`-at a `lab_bad_resume` címére állítja. A visszatérés ezt nem állítja helyre. A javított külső függvény:

```
// A64 ISA / AArch64: helyettesítő függvénytörzs.
lab_outer:
    add     w0, w0, w0
    add     w0, w0, #1
lab_bad_resume:
    ret
```

A megmaradó, már nem hívott belső függvény nem befolyásolja a működést. Ez a javítás levélfüggvénnyé alakítja a külső függvényt; nem oldja meg általánosan a beágyazott hívások mentését.

```
# Target: qemu-user / AArch64 Linux
aarch64-linux-gnu-as -march=armv8-a -EL -g \
    -o lab-nested-fixed.o lab-nested-fixed.s
aarch64-linux-gnu-ld -m aarch64linux -e _start \
    -o lab-nested-fixed.elf lab-nested-fixed.o
timeout --foreground 1s qemu-aarch64 ./lab-nested-fixed.elf
lab_status=$?
printf 'Várt állapot 43, kapott: %s\n' "$lab_status"
```

6.4.9 Rövid összefoglaló

Az A64 `BL` egyszerre adja át a vezérlést és helyezi a következő utasítás címét az `X30` regiszterbe. Az A64 `RET` alapértelmezésben erre a címre ugrik; egyik utasítás sem kezel automatikusan veremkeretet. A bemutatott levélfüggvény mentés nélkül működik, mert nem hív további függvényt, nem írja felül az `X30`-at, és csak az eredményregisztert módosítja. Beágyazott híváskor az új visszatérési cím felülírja a régit. A helyes számítás ezért önmagában kevés: a visszatérést és a befejeződést is ellenőrizni kell.

- **Visszatérési cím:** a hívás utáni folytatás címe.
- **LR:** A64-en az `X30` kapcsolóregiszter-szerepének neve.
- **Levélfüggvény:** más függvényt nem hívó függvény.

6.4.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; az A64 `BL` és `RET` szemantikája és kódolása. Ellenőrizve: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- AAPCS64 – Arm hivatalos ABI-dokumentum — argumentumok, eredmények és regiszter-megőrzési kötelezettségek. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- GNU assembler kézikönyv — AArch64 szintaxis és ELF-direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv — távoli hibakeresés és utasításszintű léptetés. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kilépési állapotok és GDB-értékek a lektor GNU as/ld 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A telepített Ubuntu 24.04 csomagverziókat és a helyi debuggerkapcsolat működését a hallgató laborjában kell ellenőrizni.

7 Memóriacímzés és adatszerkezetek

7.1 Load/store címzési módok

7.1.1 Miért számít, mikor változik meg a mutató?

Egy C-ben írt pufferfeldolgozó rutin gyakran egyszerre olvas adatot és lépteti a mutatót. Assemblyben külön kell döntenünk arról, hogy **melyik címről történjen a hozzáférés**, illetve **változzon-e a címet tartalmazó regiszter**. Egy rosszul választott címzési mód érvényes memóriacímet is előállíthat, miközben a program következetesen a szomszédos elemet dolgozza fel.

Az alfejezetben az **A64 ISA, AArch64, Armv8-A alapprofilját** használjuk, little-endian bájtsorrenddel. Minden alábbi utasítás és regiszter ebben a kontextusban értendő; az assembly direktívái a GNU assembler AArch64-célú forrásához tartoznak. Cortex-M-kód itt nem szerepel.

Az előfeltétel a regiszterek, az alapvető aritmetika és a vezérlésátadás ismerete. A példaprogram szándékosan nem használ függvényhívást vagy vermet: kizárólag a memóriacímzésre összpontosít.

7.1.2 Effektív cím és bázisregiszter

A **betöltés** (*load*) memóriából regiszterbe, a **tárolás** (*store*) regiszterből memóriába mozgat adatot. Az A64 `LDR` és `STR` egészregiszteres alakjaiban a művelet szélességét az adatregiszter nézete határozza meg:

```
// A64 ISA; szintaktikai minták, nem önálló program.
ldr w0, [x1]       // 4 bájt betöltése
ldr x0, [x1]       // 8 bájt betöltése
str w0, [x1]       // 4 bájt tárolása
str x0, [x1]       // 8 bájt tárolása
```

Az **effektív cím** (*effective address*, EA) a memóriahozzáférés kezdőcíme. A **bázisregiszter** (*base register*) a címképzés kiinduló értékét tartalmazza. A példákban ez mindig 64 bites A64 `Xn` regiszter: a 32 bites adatbetöltés sem teszi 32 bitessé a címet.

Az A64 betöltések és tárolások önmagukban nem módosítják az NZCV állapotjelzőket. A cím számításához nem tartozik automatikus objektumhatár-ellenőrzés sem: a programozó feladata, hogy a teljes hozzáférés érvényes, megfelelő jogosultságú memóriába essen.

7.1.3 Három immediate címzési mód

Legyen B a bázisregiszter eredeti értéke, d pedig az eltolás bájtban. A következő táblázat sikeresen befejeződő A64 memóriahozzáférésre érvényes.

A64 címzési mód	Példa	Effektív cím	Bázisregiszter utána
Immediate offset	`LDR W0, [X1, #4]`	$B+4$	B
Pre-index	`LDR W0, [X1, #4]!`	$B+4$	$B+4$
Post-index	`LDR W0, [X1], #4`	B	$B+4$

A **visszaírás** (*writeback*) a kiszámított új báziscím regiszterbe írása. A táblázat első alakja nem végez visszaírást; a másik kettő igen.

Figyelmeztetés: a bázis- és a célregiszter nem fedhet át visszaírásnál. A pre- és post-index alakban ugyanabba a regiszterbe két érték kerülne: a betöltött adat és az új báziscím. Az architektúra ezt nem rendezi el, hanem *CONSTRAINED UNPREDICTABLE*-nek nyilvánítja: a megvalósítás választhat például az utasítás érvénytelenként kezelése, a visszaírás elhagyása vagy meghatározatlan érték írása között. Hordozható kód ezért nem építhet egyikre sem.

Alak	Tiltott átfedés (ha a bázis nem `SP`)
`LDR Xt, [Xn, #d]!`, `LDR Xt, [Xn], #d` (és a többi méretváltozat)	`t = n`
`STR Xt, [Xn, #d]!`, `STR Xt, [Xn], #d`	`t = n`
`LDP Xt1, Xt2, [Xn, #d]!`, `LDP Xt1, Xt2, [Xn], #d`	`t1 = n` vagy `t2 = n`
`STP Xt1, Xt2, [Xn, #d]!`, `STP Xt1, Xt2, [Xn], #d`	`t1 = n` vagy `t2 = n`
`LDP Xt1, Xt2, [...]` bármely címzéssel	`t1 = t2`

Visszaírás **nélkül** az átfedés szabályos és gyakori: a `LDR X0, [X0, #8]` (például láncolt lista bejárása, `p = p->next`) jól definiált. A GNU as 2.42 a tiltott alakokra `Warning: unpredictable transfer with writeback`, illetve `Warning: unpredictable load of register pair` üzenettel figyelmeztet (a kilépési állapota 0, az objektum elkészül); a Clang 18 integrált assemblere viszont `error: unpredictable LDR instruction, writeback base is also a source` hibát ad. A figyelmeztetés mellett az objektum elkészül, ezért a build-naplót ellenőrizni kell, vagy `-Wa,--fatal-warnings` kapcsolóval hibává kell tenni.

7.1.3.1 Immediate offset: hozzáférés a mutató megváltoztatása nélkül

Az A64 `LDR W0, [X1, #4]` a bázis után négy bájttal kezdődő adatot olvassa, de az A64 `X1` értéke változatlan marad. Ez alkalmas például ugyanazon objektum több mezőjének elérésére.

Az eltolás assemblyben **bájtban értendő**, nem elemszámban. Egy `uint32_t` sorozat következő eleméhez ezért négyet adunk a címhez, nem egyet.

7.1.3.2 Pre-index: előbb léptetett címet használunk

Az A64 `LDR W0, [X1, #4]!` a megnövelt címen olvas, és ezt az új címet írja vissza az A64 `X1` regiszterbe. A felkiáltójel nem díszítés: ez jelöli a visszaírást.

C-szemléletű megfeleltetés egy `uint32_t *p` esetén: előbb `p` léptetése, majd az új helyen levő érték olvasása.

7.1.3.3 Post-index: az eredeti címen férünk hozzá

Az A64 `LDR W0, [X1], #4` a régi címen olvas, majd néggyel növeli a bázisregisztert. Ez természetes választás előrefelé haladó adatfolyam feldolgozásához.

Az „előbb” és „utána” itt az utasítás **architekturális szemantikáját** írja le. Nem a processzor belső időzítésére vonatkozó állítás, és hibás memóriahozzáférésre nem szabad belőle általános visszaírási szabályt következtetni.

7.1.4 Az immediate mezők kódolási korlátai

Az A64 utasítás nem tartalmazhat tetszőleges méretű eltolást. A fő egészregiszteres alakok korlátai:

A64 forma	Kódolt mező	Assemblyben megadható bájteltolás
`LDR/STR Wt`, unsigned offset	12 bites előjel nélküli érték, négyszeres skálázással	0…16380, négy többszöröse
`LDR/STR Xt`, unsigned offset	12 bites előjel nélküli érték, nyolcszoros skálázással	0…32760, nyolc többszöröse
`LDRB/STRB`, unsigned offset	12 bites előjel nélküli érték	0…4095
`LDRH/STRH`, unsigned offset	12 bites előjel nélküli érték, kétszeres skálázással	0…8190, kettő többszöröse
Pre-index és post-index	9 bites előjeles, skálázatlan érték	−256…255, bájtban

Levezetés: a 12 bites előjel nélküli mező legnagyobb értéke 4095. Nyolcbájtos hozzáférésnél a legnagyobb eltolás ezért $4095 \cdot 8 = 32760$ bájt. A 9 bites előjeles mező tartománya $-2^8 \dots 2^8 - 1$, ezért a visszaírásos alakok eltolása −256…255 bájt.

Például az A64 `LDR X0, [X1, #24]` kódolt mezője 3, de az A64 `LDR X0, [X1, #24]!` mezője 24. A forrásban mindkettő 24 bájtot jelent.

Negatív vagy a hozzáférési mérettel nem osztható, **visszaírás nélküli** eltoláshoz az A64 `LDUR/STUR` család ad explicit skálázatlan alakot, ugyancsak −256…255 bájtos tartománnyal. Egyes assemblerírásmódok automatikusan ezt a kódolást választják. A tényleges utasítást ezért disassemblyben ellenőrizzük; ne kezeljük a megengedő szintaxist új ISA-kódolásként.

7.1.5 Regiszteres és skálázott eltolás

Ha az index futás közben változik, az eltolás másik regiszterből is származhat:

```
// A64 ISA; címzési minták.
ldr w0, [x1, x2]          // EA = x1 + x2
ldr w0, [x1, x2, lsl #2]  // EA = x1 + x2 * 4
ldr x0, [x1, x2, lsl #3]  // EA = x1 + x2 * 8
```

A példákban az A64 `X2` 64 bites eltolást vagy indexet tartalmaz. A címösszeg képzése nem módosítja sem az A64 `X1`, sem az A64 `X2` értékét.

Az itt használt A64 `LSL` indexmódosítás eltolásszáma nem tetszőleges. Megengedett a nulla, illetve a hozzáférési méret kettes alapú logaritmusa: négynél 2, nyolcnál 3. A skálázás tehát az elemszámot bájteltolássá alakítja.

Ez a regiszteres forma nem kombinálható pre-index vagy post-index visszaírással. A tömbbejárás részletes mintái a következő alfejezetben következnek.

7.1.6 Méretvariánsok és előjelkiterjesztés

Keskeny adat betöltésekor azt is meg kell határozni, hogyan töltődjenek ki az adatregiszter felső bitjei.

A64 betöltés	Olvasott méret	Kiterjesztés
`LDRB Wt, [...]`	1 bájt	Nullákkal 32 bitre
`LDRH Wt, [...]`	2 bájt	Nullákkal 32 bitre
`LDR Wt, [...]`	4 bájt	Nincs 32 biten belüli kiterjesztés
`LDR Xt, [...]`	8 bájt	Nincs kiterjesztés
`LDRSB Wt/Xt, [...]`	1 bájt	Előjellel 32/64 bitre
`LDRSH Wt/Xt, [...]`	2 bájt	Előjellel 32/64 bitre
`LDRSW Xt, [...]`	4 bájt	Előjellel 64 bitre

Az **előjelkiterjesztés** (*sign extension*) a forrás legfelső bitjét ismétli meg az új felső biteken. A `0x80` bájt előjel nélküli olvasata 128, nyolcbites előjeles olvasata −128.

Fontos A64-szabály: **bármely `Wt`-be írás nullázza a megfelelő `Xt` felső 32 bitjét**. Ezért:

- A64 `LDRSB W10, [...]`: `W10 = 0xFFFFFF80`, de `X10 = 0x00000000FFFFFF80`;
- A64 `LDRSB X11, [...]`: `X11 = 0xFFFFFFFFFFFFFF80`.

Az első eredmény 32 bites előjeles olvasatban −128, de a teljes 64 bites regiszter előjeles olvasata pozitív. A második 64 bites előjeles olvasatban is −128.

Tároláskor nincs előjelkiterjesztés: az A64 `STRB` az adatregiszter alsó 8, az A64 `STRH` az alsó 16 bitjét írja ki.

7.1.7 Kidolgozott példa: címzés és visszaírás ellenőrzése

Munkakönyvtár: `~/arm-labor/a64/a022/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a022
cd ~/arm-labor/a64/a022
```

A `lab-loadstore.s` önálló Linux/AArch64 program. A címeket saját ELF-szimbólumaiból képezi; nem használ rögzített memóriacímeket. Jelölje B a `lab_words`, D a `lab_output` címét.

```
// A64 ISA, Armv8-A; GNU as direktívák; little-endian.
.section .text
.global _start
.type _start, %function

_start:
    ADR     x1, lab_words

    ldr     w2, [x1, #8]          // w2 = 30; x1 = B
    ldr     w3, [x1, #4]!         // w3 = 20; x1 = B+4
    ldr     w4, [x1], #4          // w4 = 20; x1 = B+8
    MOVZ    x5, #1
```

```
    ldr     w6, [x1, x5, lsl #2]  // w6 = 40; x1 változatlan

    ADR     x7, lab_output
    str     w2, [x7, #8]          // D+8 címre 30
    str     w3, [x7], #4          // D címre 20; x7 = D+4
    str     w6, [x7, #8]!         // D+12 címre 40; x7 = D+12

    ADR     x8, lab_byte
    ldrb    w9, [x8]
    LDRSB   w10, [x8]
    LDRSB   x11, [x8]

    ADR     x12, lab_half
    ldrh    w13, [x12]
    LDRSH   x14, [x12]

    ADR     x15, lab_signed_word
    ldrsw   x16, [x15]

lab_observe:
    cmp     w2, #30
    b.ne    lab_fail
    cmp     w3, #20
    b.ne    lab_fail
    cmp     w4, #20
    b.ne    lab_fail
    cmp     w6, #40
    b.ne    lab_fail

    ADR     x17, lab_words
    add     x17, x17, #8
    cmp     x1, x17
    b.ne    lab_fail

    ADR     x17, lab_output
    ldr     w0, [x17]
    cmp     w0, #20
    b.ne    lab_fail
    ldr     w0, [x17, #4]
    cbnz    w0, lab_fail
    ldr     w0, [x17, #8]
    cmp     w0, #30
    b.ne    lab_fail
    ldr     w0, [x17, #12]
    cmp     w0, #40
    b.ne    lab_fail
    add     x17, x17, #12
    cmp     x7, x17
    b.ne    lab_fail

    MOVZ    x0, #0
    b       lab_exit

lab_fail:
    MOVZ    x0, #1
lab_exit:
    MOVZ    x8, #93               // Linux/AArch64: exit
    svc     #0
.size _start, .-_start

.section .data
.balign 8
lab_words:
```

```
    .word 10, 20, 30, 40
lab_output:
    .zero 16
lab_byte:
    .byte 0x80
.balign 2
lab_half:
    .hword 0x8001
.balign 4
lab_signed_word:
    .word 0x80000000

.section .note.GNU-stack,"",%progbits
```

A kiegészítő utasítások (`ADR`, `ADD`, `CMP`, `MOVZ`, `SVC`) konstansai mind kódolhatók (lásd az A64 immediate-korlátok összefoglalóját). A labor kis ELF-fájljának szimbólumai várhatóan az `ADR` ± 1 MiB-os hatótávolságán belül lesznek; az esetleges tartományhibát a linker jelzi. A példa egyetlen visszaíró utasítása sem fedi át a bázist és a célregisztert.

Build és futtatás — cél: qemu-user, Linux/AArch64, little-endian.

```
aarch64-linux-gnu-as -g -EL -march=armv8-a \
  -o lab-loadstore.o lab-loadstore.s
aarch64-linux-gnu-ld -EL -e _start \
  -o lab-loadstore lab-loadstore.o
aarch64-linux-gnu-objdump -d lab-loadstore
qemu-aarch64 ./lab-loadstore
printf 'kilepesi_allapot=%d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): nincs programkimenet, a shell `kilepesi_allapot=0` sort ír. Az 1 kilépési állapot valamely beépített címzési ellenőrzés hibáját jelzi. A méretvariánsok eredményeit külön, GDB-vel vizsgáljuk.

Állapot a `lab_observe` címkénél	Mért érték (GDB 15.1, QEMU 8.2.2)
A64 `X1`	$B + 8$
A64 `X7`	$D + 12$
`lab_output`, négy előjel nélküli 32 bites szóként	20, 0, 30, 40
A64 `X9`	`0x0000000000000080`
A64 `X10`	`0x00000000FFFFFF80`
A64 `X11`	`0xFFFFFFFFFFFFFF80`
A64 `X13`	`0x0000000000008001`
A64 `X14`	`0xFFFFFFFFFFFF8001`, előjelesen -32767
A64 `X16`	`0xFFFFFFFF80000000`, előjelesen -2147483648

7.1.8 Hibakeresés és ellenpróbák

Cél: ugyanaz a qemu-user program, GDB-vel. Az első terminálban:

```
qemu-aarch64 -g 1234 ./lab-loadstore
```

A másodikban:

```
gdb-multiarch -q ./lab-loadstore \
  -ex 'target remote localhost:1234' \
  -ex 'break lab_observe' \
  -ex 'continue' \
  -ex 'info registers x1 x7 x9 x10 x11 x13 x14 x16' \
  -ex 'x/4uw &lab_output'
```

A töréspontnál a táblázat szerinti állapot várható. Az ellenőrzés után a GDB `continue` parancsával a programnak 0 állapottal kell befejeződnie.

Tipikus hibakeresési útvonalak:

- **Egy elemmel elcsúszott adat:** hipotézis, hogy pre-index került post-index helyére. Disassemblyben ellenőrizd a zárójelet és a felkiáltójelet, majd utasításonként léptetve figyeld a bázisregisztert.
- **Helyes adat, hibás későbbi cím:** valószínűleg nem kívánt visszaírás történt. Csak a címzési módot változtasd meg, majd ismételd meg a tesztet.
- **Nagy pozitív szám várt negatív helyett:** ellenőrizd a nullakiterjesztés és előjelkiterjesztés választását, valamint a célregiszter szélességét.
- **Assemblerhiba vagy váratlan `LDUR`:** ellenőrizd az immediate tartományát és skálázhatóságát az objektumkódban.

Visszaírásos betöltésnél ne használd ugyanazt a regisztert adatcélként és bázisként, például A64 `LDR X1, [X1], #8` alakban. Ez architekturálisan korlátozottan meghatározatlan (*constrained unpredictable*) eset; nem hordozható mutatóléptetési technika.

Az igazítatlan hozzáférés sem feltétlenül okoz hibát. A labor szokásos Linux/QEMU normálmemóriájában gyakran végrehajtható, ezért egy hibás index inkább rossz adatként, mint összeomlásként jelentkezhet.

7.1.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Az A64 `X1` kezdetben $B + 8$. Külön-külön ebből az állapotból indulva mi az effektív cím és az új bázisérték az A64 `LDR W0, [X1, #-4]!`, illetve `LDR W0, [X1], #-4` esetén?

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a labort a négy bemeneti érték fordított sorrendű összegzésével. Indulj az objektum vége utáni címből, használj pre-index betöltést, és ellenőrizd a 100 eredményt. A kiegészítést a meglévő ellenőrzések után, a sikeres kilépési érték beállítása elé illeszd.

3. (hibakeresési, középszint — cél: rutinépítés) Változtasd a regiszteres betöltésben az A64 `LSL #2` módosítást `LSL #0` alakra. Jósolj eredményt little-endian bájtsorrend mellett, majd objdump és GDB segítségével keresd meg az eltérést. Egyetlen módosítással javítsd a hibát.

7.1.10 Megoldások

1. megoldás — Segítség. Nézd meg a fejezet első táblázatát: a pre-index és a post-index közül melyik írja vissza a MÓDOSÍTOTT, melyik az EREDETI bázist — és ez hogyan hat a negatív eltolásra?

1. megoldás — Teljes megoldás. A pre-index alak effektív címe $B + 4$, és az A64 `X1` új értéke is $B + 4$. A post-index alak effektív címe $B + 8$, az új bázisérték viszont szintén $B + 4$. A végső mutató tehát azonos, az olvasott adat nem.

2. megoldás — Segítség. Az objektum vége utáni cím $B + 16$; pre-index betöltéssel minden lépés ELŐBB csökkenti a bázist néggyel, utána olvas — milyen sorrendben éred el így a négy elemet?

2. megoldás — Teljes megoldás. A beilleszthető A64-részlet:

```
// A64 ISA; a lab-loadstore.s meglévő ellenőrzései után.
ADR     x1, lab_words
add     x1, x1, #16
MOVZ    w0, #0
MOVZ    w5, #4
lab_reverse:
    ldr     w2, [x1, #-4]!
    add     w0, w0, w2
    subs    w5, w5, #1
    b.ne    lab_reverse
cmp     w0, #100
b.ne    lab_fail
```

Az A64 `SUBS` immediate korlátja az `ADD` fent ismertetett korlátjával egyezik. Az effektív címek sorrendje $B+12$, $B+8$, $B+4$, B. A kezdeti, objektum utáni címet nem dereferáljuk. Az eredeti build–run lépéssorral a várt kilépési állapot továbbra is 0.

3. megoldás — Segítség. `LSL #0` azt jelenti, hogy az index regiszter értékét byte-egységben, skálázás NÉLKÜL adja a bázishoz — ha `X5 = 1`, ez hány bájttal tolja el a címet a szándékolt 4 helyett?

3. megoldás — Teljes megoldás. Az A64 `X1` értéke $B+8$, az A64 `X5` értéke 1. Skálázás nélkül ezért a betöltés $B+9$ címen kezdődik. A négy bájt `0x00`, `0x00`, `0x00`, `0x28`, így a várt A64 `W6` értéke `0x28000000`, nem 40. A hozzáférés az objektumon belül marad; a szokásos laborbeállítás mellett az ellenőrzés mért eredménye 1 kilépési állapot. Az A64 `LSL #2` visszaállítása helyreállítja a helyes bájteltolást.

7.1.11 Rövid összefoglaló

Az A64 memóriahozzáférésnél külön kérdés az elért cím, a mozgatott adatmennyiség és a bázisregiszter mellékhatása. Az immediate offset változatlanul hagyja a bázist; a pre-index a módosított, a post-index az eredeti címen fér hozzá. Visszaírásnál a bázisregiszter nem lehet egyben a betöltés vagy tárolás adatregisztere (páros alaknál egyik sem): az ilyen kód viselkedése architekturálisan nem meghatározott. Az eltolások a forrásban bájtban szerepelnek, de kódolásuk lehet méretskálázott vagy skálázatlan. Regiszteres indexnél a megfelelő skálázás elhagyása érvényes címen is hibás adatot eredményezhet. A betöltés szélessége és előjelkezelése egymástól külön döntés. Hibakereséskor ezért a regiszterérték mellett a disassemblyt és a memóriabájtokat is ellenőrizzük.

- **Effektív cím:** a memóriahozzáférés kezdőcíme.
- **Visszaírás:** az új báziscím regiszterbe helyezése.
- **Előjelkiterjesztés:** a forrás előjelbitjének ismétlése a hozzáadott felső biteken.

7.1.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; A64 betöltések, tárolások, címzés és visszaírás. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- GNU assembler kézikönyv — AArch64 szintaxis, direktívák és assembleropciók. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.

- GNU GDB kézikönyv — távoli hibakeresés, regiszter- és memóriavizsgálat. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt regiszterértékek és assembler-üzenetek a lektor GNU as/ld 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A konkrét Ubuntu 24.04 csomagverziókat, az assembler alternatív címzési írásmódjait és a GDB/QEMU együttműködését a hallgatói laborban kell ellenőrizni.

7.2 Tömbök és indexelt elérés assemblyben

7.2.1 Miért számít az elemméret?

Egy képfeldolgozó rutin, ellenőrzőösszeg-számítás vagy protokollfeldolgozó rendszer gyakran tömbökön dolgozik. C-ben az `a[i]` kifejezés elrejti az elemmérettel történő szorzást. Assemblyben ezt nekünk kell helyesen kifejeznünk: a processzor memóriacíme **bájtot jelöl, nem tömbelemet**.

Ebben az alfejezetben az egydimenziós indexeléstől eljutunk a sorfolytonos kétdimenziós tömbig és egy önellenőrző összegző programig. Előfeltétel a Load/store címzési módok ismerete.

Profil: minden alábbi utasítás és regiszter az **A64 ISA, AArch64, Armv8-A** környezetéhez tartozik. Az assemblydirektívák GNU `as` szintaxisúak, AArch64 ELF célhoz. A labor little-endian, Linux alatt, `qemu-aarch64` user-mode emulációval fut. Cortex-M-kódot itt nem használunk.

7.2.2 Az indexből bájteltolás lesz

A **skálázott indexelés** (*scaled indexing*) az elemindexet az elemmérettel megszorozva alakítja bájteltolássá.

Legyen:

- B a tömb első elemének bájtcíme;
- i az előjel nélküli elemindex, mértékegység nélküli egész;
- s az elemméret, bájt/elem egységben.

Ekkor:

$$\text{cím}(a[i]) = B + i \cdot s$$

A képlet használatának előfeltétele, hogy `i` a tömb határain belül legyen, és a címképzés ne csorduljon túl. A hardver nem ismeri a C-tömb hosszát: egy hozzáférhető cím még nem feltétlenül a megfelelő objektumhoz tartozik.

Levezetés: ha az elemméret $s = 2^k$, akkor az $i \cdot s$ szorzás az index k bites balra tolásával állítható elő. Az A64 regiszstereltolásos memóriautasítása ezt külön szorzóutasítás nélkül is kifejezheti.

Elemméret	A64 betöltés / tárolás	Elemindexhez szükséges skálázás	Megengedett `LSL` eltolás a címzésben
1 bájt	`LDRB Wt` / `STRB Wt`	0 bit	0

Elemméret	A64 betöltés / tárolás	Elemindexhez szükséges skálázás	Megengedett `LSL` eltolás a címzésben
2 bájt	`LDRH Wt` / `STRH Wt`	1 bit	0 vagy 1
4 bájt	`LDR Wt` / `STR Wt`	2 bit	0 vagy 2
8 bájt	`LDR Xt` / `STR Xt`	3 bit	0 vagy 3

Az A64 címzésbe épített eltolás tehát **nem tetszőleges shift**. Négybájtos hozzáférésnél például a `LSL #1` nem kódolható ebben a címzési formában.

Ha az A64 `X1` regiszter egy `uint32_t` tömb báziscíme, az `X2` pedig az előjel nélküli index:

```
// A64 ISA: x1 = báziscím, x2 = elemindex.
ldr w3, [x1, x2, lsl #2]   // w3 = a[i]
str w3, [x1, x2, lsl #2]   // a[i] = w3
```

A hozzáférés négybájtos, a bázisregiszter változatlan marad. Az A64 `LDR W3` a hozzá tartozó `X3` felső 32 bitjét nullázza.

Ha az index 32 bites előjel nélküli érték az A64 `W2` regiszterben, a megfelelő címzési alak például `LDR W3, [X1, W2, UXTW #2]`: előbb nullakiterjesztés történik 64 bitre, majd skálázás. Az index előjelkezelése és a betöltött adat előjelkezelése két külön kérdés.

7.2.3 Soros bejárás post-index címzéssel

Ha minden elemet sorrendben dolgozunk fel, nem szükséges minden iterációban újra előállítani a báziscím és az index összegét. Fenntarthatunk egy aktuális mutatót, amely minden feldolgozás után a következő elemre lép.

A következő C-függvény a labor összegző szakaszának szemantikai referenciája:

```
#include <stddef.h>
#include <stdint.h>

uint64_t sum_u32(const uint32_t *p, size_t n)
{
    uint64_t sum = 0;

    while (n != 0) {
        sum += *p++;
        --n;
    }
    return sum;
}
```

A megfelelő A64 post-index betöltés:

```
// A64 ISA: előbb olvasás az eredeti x1 címről, majd x1 += 4.
ldr w3, [x1], #4
```

A `#4` **bájteltolás**, nem négy elem. A post-index immediate előjeles, 9 bites, ezért -256 és 255 bájt közötti érték lehet; a négybájtos lépés belefér.

A ciklus állapotát három mennyiség írja le:

1. az aktuális elem címe;
2. a hátralévő elemszám;

3. az eddigi összeg.

A ciklusinvariáns szerint k feldolgozott elem után a mutató $B+4k$ bájtcímet tartalmaz, a számláló $n-k$, az akkumulátor pedig az első k elem összege. Ez egyszerre segíti a program megírását és a hibakeresést.

Az üres bemenetet **az első betöltés előtt** kell felismerni. Ha a nulla számlálót előbb csökkentjük, az A64 64 bites regiszterében `0xffffffffffffffff` keletkezik; ettől egy „nem nulla” feltételű ciklus nem fog helyesen leállni.

7.2.4 Kétdimenziós tömb: sorfolytonos elrendezés

A C valódi kétdimenziós tömbje, például a `uint32_t m[2][3]`, **sorfolytonos elrendezésű** (*row-major layout*): először az első sor elemei következnek, majd a második soré.

Ha K az oszlopszám, r a sorindex, c az oszlopindex, akkor:

$$\text{cím}(m[r][c]) = B + (rK + c) \cdot s$$

Itt r, c és K mértékegység nélküli egész, s az elemméret bájtban; feltétel a $0 \le r < R$ és $0 \le c < K$ tartomány, ahol R a sorok száma.

Háromoszlopos, négybájtos elemekből álló tömbnél:

Elem	Lineáris elemindex	Bájteltolás a bázishoz képest
`m[0][0]`	0	0
`m[0][2]`	2	8
`m[1][0]`	3	12
`m[1][2]`	5	20

Az A64 `MADD` előállíthatja az $rK + c$ kifejezést, majd a skálázott betöltés elvégzi a bájteltolássá alakítást. Soronkénti feldolgozáskor a belső ciklus négybájtos post-index lépésekkel járhatja be az adott sort; a következő sor kezdete 12 bájttal később található.

Ez **nem ugyanaz**, mint a `uint32_t **`: a mutatótömb sorai külön memóriaterületeken is lehetnek, elérésükhöz további mutatóbetöltés szükséges.

7.2.5 Kidolgozott példa: indexelés, tárolás és összegzés

Munkakönyvtár: `~/arm-labor/a64/a023/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a023
cd ~/arm-labor/a64/a023
```

Mentsük a következő teljes programot `lab-array.s` néven. Három műveletet ellenőriz:

- az egydimenziós tömb 3-as indexű elemének betöltését és másik tömbbe tárolását;
- a mátrix `m[1][2]` elemének elérését;
- az első `LAB_N` elem összegzését.

A `LAB_N` fordításkor megadható, értéke 0–5 lehet. Az A64 `X1` a két indexelt művelet során változatlan tömbbázis marad, majd az összegzés aktuális mutatója lesz.

```
// A64 ISA, GNU as, AArch64 Linux, little-endian.
.ifndef LAB_N
.equ LAB_N, 5
.endif

.if (LAB_N < 0) || (LAB_N > 5)
.error "LAB_N erteke 0..5 lehet"
.endif

.equ LAB_EXPECTED, 5 * LAB_N * (LAB_N + 1)

.section .text
.global _start
.type _start, %function

_start:
    adrp x1, lab_array
    add  x1, x1, :lo12:lab_array

    // lab_output[3] = lab_array[3]
    mov  x2, #3
    ldr  w3, [x1, x2, lsl #2]

    adrp x4, lab_output
    add  x4, x4, :lo12:lab_output
    str  w3, [x4, x2, lsl #2]
    ldr  w5, [x4, x2, lsl #2]
    cmp  w5, #40
    b.ne lab_fail

    // lab_matrix[1][2], három oszloppal.
    adrp x4, lab_matrix
    add  x4, x4, :lo12:lab_matrix
    mov  x5, #1
    mov  x6, #2
    mov  x7, #3
    madd x6, x5, x7, x6
    ldr  w3, [x4, x6, lsl #2]
    cmp  w3, #6
    b.ne lab_fail

    // Az első LAB_N elem összege.
    mov  x0, #0
    mov  x2, #LAB_N
    cbz  x2, lab_done

lab_loop:
    ldr  w3, [x1], #4
    add  x0, x0, x3
    subs x2, x2, #1
    cbnz x2, lab_loop

lab_done:
    cmp  x0, #LAB_EXPECTED
    b.ne lab_fail
    mov  x0, #0
    b    lab_exit

lab_fail:
    mov  x0, #1
```

```
lab_exit:
    mov  x8, #93
    svc  #0

.size _start, .-_start

.section .data
.balign 4
lab_array:
    .word 10, 20, 30, 40, 50
lab_array_end:
lab_guard:
    .word 99

lab_output:
    .word 0, 0, 0, 0, 0

lab_matrix:
    .word 1, 2, 3
    .word 4, 5, 6

.section .note.GNU-stack,"",%progbits
```

A GNU `as` `.word` direktívája itt 32 bites elemeket helyez el; a `.balign 4` négybájtos igazítást biztosít. A `lab_guard` külön őrszó, nem része az ötelemű logikai tömbnek.

A `MOV`, `ADD`, `SUBS` és `CMP` konstansai mind kódolhatók (lásd az A64 immediate-korlátok összefoglalóját). A címképző `ADRP` 4 KiB-os lapot választ; az utána következő `ADD` a cím alsó 12 bitjét adja hozzá. A szimbólumokra történő címképzést a linker oldja fel, nem kézzel választott memóriacímeket használunk.

Az A64 `LDR W3` nullakiterjesztése miatt az `ADD X0, X0, X3` a `uint32_t` elemet helyesen adja a 64 bites előjel nélküli összeghez. Általános bemenetnél ez az összeg modulo 2^{64} értendő; a labor kis számainál nincs túlcsordulás.

A kilépés a korábban tanult Linux syscall-konvenciót követi: a 93-as hívásszám Linux AArch64 ABI-előírás, nem az A64 ISA része.

7.2.6 Összeállítás és várt állapot

Cél: qemu-user, AArch64 little-endian Linux; host: Ubuntu 24.04 x86_64.

```
aarch64-linux-gnu-as -EL -march=armv8-a -g \
  --defsym LAB_N=5 -o lab-array.o lab-array.s
aarch64-linux-gnu-ld -EL -e _start -o lab-array lab-array.o
qemu-aarch64 ./lab-array
printf 'exit=%d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): `exit=0`. A program maga nem ír a terminálra; eltéréskor 1-es állapottal lép ki. A `--defsym LAB_N=6` build a `.error` direktíván áll le, `LAB_N erteke 0..5 lehet` üzenettel (mérve: GNU as 2.42).

A `lab_done` címkénél, még az ellenőrzés előtt:

Állapot, A64 környezet	`LAB_N=5`	`LAB_N=0`
`X0`, előjel nélküli összeg	150	0
`X1`, aktuális mutató	`lab_array + 20`	`lab_array`

Állapot, A64 környezet	`LAB_N=5`	`LAB_N=0`
`X2`, hátralévő elemszám	0	0
`lab_output[3]`	40	40

Az elvárt összeg levezetése: az elemek $10, 20, \ldots, 10n$, ezért összegük $10n(n+1)/2 = 5n(n+1)$. Ezt számolja ki összeállításkor a `LAB_EXPECTED` szimbólum.

Az üres eset külön buildje, ugyanezen **qemu-user célhoz**:

```
aarch64-linux-gnu-as -EL -march=armv8-a -g \
  --defsym LAB_N=0 -o lab-empty.o lab-array.s
aarch64-linux-gnu-ld -EL -e _start -o lab-empty lab-empty.o
qemu-aarch64 ./lab-empty
printf 'exit=%d\n' "$?"
```

Itt is `exit=0` a mért eredmény. Az **összegző szakasz** nem olvas memóriát; az előtte álló indexelési önellenőrzések továbbra is lefutnak.

7.2.7 Tipikus hibák és célzott ellenpróbák

Hiányzó skálázás. Ha az A64 `LDR W3, [X1, X2, LSL #2]` helyett eltolás nélküli alakot írunk, az index bájteltolássá válik. A 3-as index ekkor nem 12, hanem 3 bájttal lép előre. Ez nem feltétlenül okoz szegmentálási hibát: hozzáférhető, de hibás helyről olvashatunk.

Rossz hozzáférési szélesség. Az A64 `LDR X3, [X1], #4` nyolc bájtot olvasna, miközben csak négy bájttal léptet. A következő hozzáférés átfedne az előzővel. Előbb a betöltés szélességét, utána a lépésközt ellenőrizzük.

Előjeles adat nullakiterjesztése. `int32_t` elemek 64 bites előjeles összegzéséhez az A64 `LDRSW X3, [X1], #4` adja a szükséges előjelkiterjesztést. A ciklusszerkezet ettől még változatlan lehet.

Egy elemmel túlfutó ciklus (*off-by-one error*). A program a tömb utáni őrszót is elérheti, ezért a sikeres memóriahozzáférés nem igazolja a helyes ciklushatárt.

A hibakeresés menete: **tünet → hipotézis → adatgyűjtés → egyetlen módosítás → új ellenőrzés**. Például hibás összeg esetén előbb állapítsuk meg, hány bájttal mozdult el a mutató.

Cél: a qemu-user labor disassembly-vizsgálata és távoli hibakeresése. Az első terminálban:

```
aarch64-linux-gnu-objdump -d lab-array
qemu-aarch64 -g 1234 ./lab-array
```

A második terminálban:

```
gdb-multiarch -q lab-array \
  -ex 'target remote :1234' \
  -ex 'break lab_done' \
  -ex 'continue' \
  -ex 'info registers x0 x1 x2' \
  -ex 'p/d $x1 - (unsigned long)&lab_array' \
  -ex 'x/6uw &lab_array' \
  -ex 'x/5uw &lab_output'
```

Öt elemnél a mutató különbségének várt értéke 20 bájt. A hat kiolvasott szó rendre `10, 20, 30, 40, 50, 99`; az utolsó a szándékosan elhelyezett őrszó. A GDB `continue` parancsával a helyes program 0 állapottal befejeződik.

7.2.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Egy `uint64_t a[7]` tömbben mekkora az `a[5]` bájteltolása? Milyen skálázás kell az A64 regiszteltolásos betöltéshez? Miért nem használható hozzá `LSL #2`?

2. (implementációs, középszint — cél: rutinépítés) Készíts `lab-array-u64.s` változatot kizárólag a post-index összegzéshez, öt `uint64_t` elemmel: `10, 20, 30, 40, 50`. Módosítsd az adatelhelyezést, az igazítást és a betöltést. A fenti qemu-user munkafolyamattal, `lab-array-u64.o` és `lab-array-u64` nevekkel ellenőrizd a nulla-, egy- és ötelemű esetet.

3. (hibakeresési, középszint — cél: rutinépítés) Másold a programot `lab-offby.s` névre. Kizárólag az A64 `CBNZ X2, lab_loop` utasítást cseréld `B.PL lab_loop` utasításra. Öt elemre fordítsd `lab-offby.o` és `lab-offby` néven. A `lab_done` ponton vizsgáld meg az összeget, a mutatóeltolást és a számlálót. Miért nem szükségszerű a szegmentálási hiba?

7.2.9 Megoldások

1. megoldás — Segítség. Nézd meg a fejezet táblázatát: nyolcbájtos elemnél melyik `LSL` eltolások megengedettek a regiszteltolásos címzésben?

1. megoldás — Teljes megoldás. Az eltolás $5 \cdot 8 = 40$ bájt. A megfelelő A64 alak `LDR X3, [X1, X2, LSL #3]`. Nyolcbájtos hozzáférésnél a címzésben megengedett eltolás 0 vagy 3 bit; a `#2` nem egyszerűen hibás elemméretet jelent, hanem ebben az alakban nem kódolható.

2. megoldás — Segítség. Csak az adatdirektívát (nyolcbájtos elemhez), az igazítást és a betöltés szélességét kell nyolcbájtosra cserélni — a ciklusvezérlés logikája változatlan maradhat.

2. megoldás — Teljes megoldás. A GNU `as` adatdirektíva `.xword`, az igazítás `.balign 8`. A ciklus betöltése A64 `LDR X3, [X1], #8`. A 64 bites akkumulátor és a számláló kezelése változatlan. Nulla, egy, illetve öt elem után a várt összeg 0, 10 és 150, a mutatóeltolás pedig 0, 8 és 40 bájt. Mindhárom esetben 0 kilépési állapot várható. A nyolcbájtos lépés belefér a post-index előjeles 9 bites immediate mezőjébe.

3. megoldás — Segítség. A `B.PL` nem ugyanazt a feltételt vizsgálja, mint a `CBNZ` — mi történik, ha a számláló előjeles olvasatban még “nem negatív”, de ténylegesen már `0xffffffffffffffff`?

3. megoldás — Teljes megoldás. A `SUBS` után az A64 `B.PL` az `N=0` feltételt vizsgálja. Az ötödik elem után a számláló nulla, tehát `N=0`, és még egy iteráció következik. Ez beolvassa a 99 értékű őrszót. A mért összeg **249**, a mért mutatóeltolás **24 bájt**, a számláló `0xffffffffffffffff`. Az utolsó csökkentés már `N=1` állapotot eredményez, így a ciklus kilép. Az önellenőrzés mért kilépési állapota 1. Nem szükségszerű a memóriahiba, mert az őrszó a labor saját, hozzáférhető adatblokkjában található. A javítás a hátralévő elemszám nemnulla feltételének visszaállítása.

7.2.10 Rövid összefoglaló

A tömbindex elemben, a memóriacímzés bájtban értendő; a kettőt az elemméret kapcsolja össze. Az A64 skálázott regiszteretoltása a hozzáférés méretéhez igazodó címképzést egy memóriautasításba sűríti, de nem végez határellenőrzést. Sorrendi feldolgozásnál a post-index betöltés egyszerre olvassa az aktuális elemet és lépteti a mutatót. Az üres bemenetet az első olvasás előtt kell kezelni. Kétdimenziós C-tömbnél az oszlopszám határozza meg a sor lépésközét. A helyességet az eredmény mellett a mutató és a számláló végállapota is ellenőrzi.

- **Skálázott indexelés:** elemindex átalakítása bájteltolássá az elemmérettel.
- **Sorfolytonos elrendezés:** egy sor elemei egymás mellett, a sorok egymás után helyezkednek el.
- **Ciklusinvariáns:** az iterációk azonos pontján fennálló állítás a feldolgozás állapotáról.

7.2.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; A64 regiszteretoltásos és post-index memóriautasítások. Ellenőrizve: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 szintaxis, adatdirektívák, kifejezések és `--defsym`. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv — távoli kapcsolat, töréspontok, regiszter- és memóriavizsgálat. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kilépési állapotok és a 3. feladat regiszterértékei a lektor GNU as/ld 2.42 + QEMU 8.2.2 méréséből származnak (`mérve`). Az Ubuntu 24.04 alatt telepített konkrét binutils-, GDB- és QEMU-csomagverziót, valamint a teljes laborfutást a hallgatói környezetben kell ellenőrizni.

7.3 Struktúrák, igazítás és mezőelérés

7.3.1 Miért számít a struktúra tényleges elrendezése?

Egy C-ben létrehozott rekordot assemblyben szeretnénk módosítani. Ismerjük a mezők nevét és típusát, de az A64 memóriautasításai nem ismerik sem a struktúrát, sem a mezőneveket: báziscímet és bájteltolást kapnak. A mérnöki feladat ezért az, hogy a C-fordító által választott adatelrendezést pontos címzési szerződéssé alakítsuk.

A mezők méretének egyszerű összeadása nem elegendő. A fordító kitöltő bájtokat helyezhet közéjük, és a struktúra végét is kiegészítheti. Egy hibás offset ettől még gyakran érvényes memóriacímre mutat: a program nem feltétlenül omlik össze, csak rossz mezőt olvas.

Az alfejezet profilja **A64 ISA, AArch64 Linux, little-endian, LP64 adatmodell**. Normál C-struktúrákat vizsgálunk, bitmezők, csomagolási attribútumok és külön előírt igazítás nélkül. Az előfeltétel a Tömbök és indexelt elérés assemblyben című alfejezet: a struktúramező címét szintén bájtokban számoljuk, de az eltolást itt a típus elrendezése határozza meg.

7.3.2 Természetes igazítás és kitöltés

Az **igazítás** (*alignment*) azt írja elő, hogy egy objektum kezdőcíme milyen érték többszöröse legyen. Ha egy típus igazítása 8 bájt, akkor szabályosan elhelyezett objektumának címe 8-cal osztható.

A **természetes igazítás** (*natural alignment*) az adott típushoz az ABI által rendelt szokásos igazítás. A jelenlegi AArch64 LP64 profilban:

C-típus	Méret [bájt]	Természetes igazítás [bájt]
`char`	1	1
`int`	4	4
`long`	8	8
objektummutató	8	8

Ezek **nem általános C-nyelvi méretgaranciák**. Például a `long` más adatmodellben lehet 4 bájtos. A laborban fordítási idejű ellenőrzésekkel rögzítjük a feltevéseket.

A C megőrzi a nem bitmező tagok deklarációs sorrendjét, és az első tag elé nem tesz kitöltést. A további tagok között, valamint a struktúra végén azonban lehet **kitöltés** (*padding*).

A szokásos, itt használt ABI-elrendezésben:

1. Minden mező a saját igazításának megfelelő offsetre kerül.
2. A struktúra igazítása a tagok igazításának maximuma.
3. A struktúra mérete ennek az igazításnak egész számú többszöröse.

A harmadik szabály a struktúrákból álló tömbök miatt lényeges. Ha az első elem megfelelően igazított, akkor a következő, `sizeof` bájttal később kezdődő elemnek is annak kell lennie.

7.3.3 Levezetés: mezőoffsetek és teljes méret

Vizsgáljuk ezt a C-típust:

```
/* C-adatelrendezés: AArch64 Linux, LP64, little-endian. */
struct lab_record {
    char tag;
    int count;
    long total;
    char state;
};
```

Legyen p az előző mező utáni első szabad bájt offsetje, a a következő mező igazítása, s pedig a mérete. Mindhárom mennyiség bájtban értendő. A következő mező kezdete:

$$o = \mathrm{align_up}(p, a) = a \left\lceil \frac{p}{a} \right\rceil$$

A mező előtti kitöltés $o - p$, az új szabad pozíció pedig $o + s$. Ez a levezetés a fenti, normál ABI-elrendezést feltételezi.

Mező	Korábbi p [bájt]	Igazítás [bájt]	Offset [bájt]	Előtte padding [bájt]	Mező utáni pozíció [bájt]
`tag`	0	1	0	0	1
`count`	1	4	4	3	8
`total`	8	8	8	0	16
`state`	16	1	16	0	17

A struktúra igazítása 8 bájt. A 17 bájtos felhasznált tartományt ezért 24 bájtra kell felfelé kerekíteni:

$$\text{sizeof}(\texttt{struct lab_record}) = \text{align_up}(17, 8) = 24 \text{ bájt}.$$

Az utolsó 7 bájt **végi kitöltés** (*tail padding*).

Bájt-offsetek	Tartalom
0	`tag`
1–3	belső kitöltés
4–7	`count`
8–15	`total`
16	`state`
17–23	végi kitöltés

A hasznos mezők összmérete csak $1 + 4 + 8 + 1 = 14$ bájt. A fennmaradó 10 bájt kitöltés.

7.3.3.1 A mezősorrend hatása

Ha a nagyobb igazítású mezőket előrevesszük:

```
/* C-adatelrendezés: AArch64 Linux, LP64, little-endian. */
struct lab_record_compact {
    long total;
    int count;
    char tag;
    char state;
};
```

akkor a mezőoffsetek rendre 0, 8, 12 és 13. A mezők 14 bájtot foglalnak el, majd 2 bájt végi kitöltés következik: a teljes méret 16 bájt.

Ez rekordonként 8 bájt megtakarítás. A nagyobb igazítású mezők előrevétele jó kiindulópont, de egy már publikált interfészben a sorrend megváltoztatása **ABI-változás**. A régi assembly offsetjei, illetve a struktúra nyers memóriaképére támaszkodó fájlformátumok többé nem egyeznek az új elrendezéssel.

A padding nem alkalmazásadat. Ne használjuk rejtett mezőként, és ne feltételezzük, hogy minden kitöltő bájt értéke nulla.

7.3.4 `sizeof`, `offsetof` és az A64 mezőelérés

A C `sizeof` operátora a teljes objektumméretet adja vissza, **a kitöltéssel együtt**. Az `<stddef.h>` fejléc `offsetof` makrója egy tag bájteltolását adja meg a struktúra kezdetétől.

A példában:

- `sizeof(struct lab_record)` értéke 24;
- `offsetof(struct lab_record, count)` értéke 4;
- `offsetof(struct lab_record, total)` értéke 8;
- `_Alignof(struct lab_record)` értéke 8.

A `count` mező címe tehát:

$$\text{mezőcím} = \text{objektum báziscíme} + 4 \text{ bájt}.$$

A64 ISA, AArch64: ha az objektum címe az `X0` regiszterben van, a mezőelérés:

```
// A64 ISA, AArch64; x0 egy érvényes struct lab_record címe.
ldr w1, [x0, #4]       // A count 4 bájtos bitmintája.
ldr x2, [x0, #8]       // A total 8 bájtos bitmintája.
ldrb w3, [x0, #16]     // A state 1 bájtja, nullakiterjesztéssel.
```

Az utasítás szélességének egyeznie kell a mező szélességével. Az A64 `LDR W1` 4, az `LDR X2` 8, az `LDRB W3` pedig 1 bájtot olvas. Az A64 `LDR W1` az `X1` felső 32 bitjét nullázza: ez **nem** előjeles `int` → `long` konverzió. A negatív érték 32 bites bitmintája ettől még helyesen kerül a `W1` regiszterbe.

Az itt használt A64 unsigned-offset címzések korlátai (a teljes listát az A64 immediate-korlátok összefoglalója adja):

A64 utasításcsalád	Immediate kódolása	Leírható bájteltolás
`LDR/STR Wn`	12 bites előjel nélküli érték, 4-gyel skálázva	0–16380, 4-es lépéssel
`LDR/STR Xn`	12 bites előjel nélküli érték, 8-cal skálázva	0–32760, 8-as lépéssel
`LDRB/STRB Wn`	12 bites előjel nélküli érték	0–4095

Az assembly forrásban **bájteltolást** írunk: a `count` esetében `#4` szerepel, nem a kódolásban tárolt 1.

Az offset kódolhatósága és a tényleges cím igazítása külön kérdés. Egy 8-cal osztható offset nem javítja meg a hibásan igazított báziscímet.

7.3.5 Kidolgozott példa: C-objektum módosítása assemblyből

Munkakönyvtár: `~/arm-labor/a64/a024/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a024
cd ~/arm-labor/a64/a024
```

A C-program létrehozza az objektumot, ellenőrzi az elrendezést, majd meghív egy rövid assemblyfüggvényt.

A hívási konvencióból most csak a szükséges szerződést használjuk: **A64/AAPCS64** szerint az első mutatóargumentum az `X0` regiszterben érkezik; az `X1`–`X3` ideiglenesen felhasználható. A függvény nem hív más függvényt és nem módosítja a vermet. Az A64 `RET` az `X30` visszatérési címét használja. A teljes szabályrendszert Az AAPCS64 regiszterszerződés tárgyalja.

A `lab-layout.c` tartalma:

```c
/* C-célprofil: AArch64 Linux, LP64, little-endian. */
#include <limits.h>
#include <stddef.h>
#include <stdio.h>

struct lab_record {
    char tag;
    int count;
    long total;
    char state;
};

struct lab_record_compact {
    long total;
    int count;
    char tag;
    char state;
};

_Static_assert(CHAR_BIT == 8, "8 bites bajt szukseges");
_Static_assert(sizeof(int) == 4 && _Alignof(int) == 4,
               "eltero int profil");
_Static_assert(sizeof(long) == 8 && _Alignof(long) == 8,
               "LP64 long profil szukseges");
_Static_assert(offsetof(struct lab_record, tag) == 0, "tag offset");
_Static_assert(offsetof(struct lab_record, count) == 4, "count offset");
_Static_assert(offsetof(struct lab_record, total) == 8, "total offset");
_Static_assert(offsetof(struct lab_record, state) == 16, "state offset");
_Static_assert(sizeof(struct lab_record) == 24, "meret");
_Static_assert(_Alignof(struct lab_record) == 8, "igazitas");
_Static_assert(sizeof(struct lab_record_compact) == 16, "tomor meret");

extern void lab_update(struct lab_record *p);

int main(void)
{
    struct lab_record record = {
        .tag = 0x41,
        .count = -3,
        .total = 1000,
        .state = 1
    };

    printf("size=%zu align=%zu compact=%zu\n",
           sizeof record, _Alignof(struct lab_record),
           sizeof(struct lab_record_compact));
    printf("offsets=%zu,%zu,%zu,%zu\n",
           offsetof(struct lab_record, tag),
           offsetof(struct lab_record, count),
           offsetof(struct lab_record, total),
           offsetof(struct lab_record, state));

    lab_update(&record);
```

```
    printf("tag=0x%02x count=%d total=%ld state=%d\n",
           (unsigned int)(unsigned char)record.tag,
           record.count, record.total, (int)record.state);

    return !(record.tag == 0x41 && record.count == 2 &&
             record.total == 1024 && record.state == 2);
}
```

A `lab-layout.s` tartalma:

```
// A64 ISA, Armv8-A; GNU as direktívák AArch64 célhoz.
.text
.p2align 2
.global lab_update
.type lab_update, %function

lab_update:
    ldr w1, [x0, #4]
    add w1, w1, #5
    str w1, [x0, #4]

    ldr x2, [x0, #8]
    add x2, x2, #24
    str x2, [x0, #8]

    ldrb w3, [x0, #16]
    add w3, w3, #1
    strb w3, [x0, #16]

lab_after_update:
    ret

.size lab_update, .-lab_update
.section .note.GNU-stack,"",%progbits
```

Az `ADD` konstansai tolás nélkül kódolhatók (lásd az A64 immediate-korlátok összefoglalóját). A GNU as `.p2align 2` direktívája a **kódot** igazítja $2^2 = 4$ bájtra; nem a C-struktúra elrendezését módosítja.

7.3.5.1 A `lab_update` teljes szerződése

A `lab_update` akkor helyettesíthető a következő C-függvénnyel, ha a lenti előfeltételek teljesülnek:

```
/* C-referencia: a lab_update C-egyenértékű viselkedése a szerződés tartományán. */
void lab_update_ref(struct lab_record *p)
{
    p->count += 5;
    p->total += 24;
    p->state += 1;
}
```

	Feltétel
Előfeltétel: mutató	`p` nem `NULL`, élő, írható, `struct lab_record` típusú objektumra mutat (így a típus 8 bájtos igazítását is teljesíti); a hívás alatt más szál nem éri el az objektumot

	Feltétel
Előfeltétel: `count`	`p->count <= INT_MAX - 5`
Előfeltétel: `total`	`p->total <= LONG_MAX - 24`
`state`	nincs korlát: AArch64 Linuxon a sima `char` előjel nélküli, a `+= 1` modulo 256 definiált, és ugyanazt a bájtot adja, mint az `ADD` + `STRB`
Utófeltétel	`count`, `total`, `state` a régi érték + 5, + 24, + 1; a `tag` mező és a kitöltő bájtok nem változnak
Regiszterek és verem	csak `X1`–`X3`-at írja (caller-saved); a vermet és az NZCV-t nem módosítja (`ADD`, nem `ADDS`)

Miért kell a két számtani előfeltétel? Az assembly `ADD W1, W1, #5` minden bemeneten definiált: a 32 bites bitmintán modulo 2^{32} számol, így például `INT_MAX - 4`-ből pontosan `INT_MIN` lesz. A C `p->count += 5` ugyanezen a bemeneten **előjeles túlcsordulás, vagyis definiálatlan viselkedés**. A véges bitmintát módosító assembly és a signed C-összeadás ezért csak a C által definiált tartományon azonosítható; azon kívül az assemblynek van eredménye, a C-nek nincs. Aki a `lab_update`-et C-egyenértékűnek nevezi, annak az előfeltételt is ki kell mondania, és a hívónak kell betartania: az assembly nem ellenőrzi a C értéktartományait.

A tesztbemenet a tartomány belsejében van: a `count` −3-ról 2-re, a `total` 1000-ről 1024-re változik, előjeles túlcsordulás nélkül.

7.3.5.2 Fordítás, összeállítás, linkelés és futtatás

Target: AArch64 Linux ELF, QEMU user-mode. A statikus linkeléshez az AArch64 cross libc fejlesztői fájljai is szükségesek.

```
# Host: Ubuntu 24.04 x86_64.
# Target: AArch64 Linux, Armv8-A, alapértelmezett little-endian.
aarch64-linux-gnu-gcc -std=c17 -O0 -g -Wall -Wextra \
    -march=armv8-a -fno-pie \
    -c lab-layout.c -o lab-layout-c.o

aarch64-linux-gnu-as -g -march=armv8-a -EL \
    lab-layout.s -o lab-layout-asm.o

aarch64-linux-gnu-gcc -static -no-pie \
    lab-layout-c.o lab-layout-asm.o -o lab-layout

# Target: qemu-user; mért kilépési állapot: 0.
qemu-aarch64 ./lab-layout
printf 'exit=%d\n' "$?"
```

Mért kimenet (mérve: GNU gcc 13, GNU as 2.42, QEMU 8.2.2):

- `size=24 align=8 compact=16`
- `offsets=0,4,8,16`
- `tag=0x41 count=2 total=1024 state=2`
- `exit=0`

A fordítási idejű ellenőrzések az ABI-feltevéseket, a visszatérési érték a mezőmódosításokat vizsgálja. A kettő külön védelmi réteg: a hibás elrendezést jobb már fordításkor észrevenni.

7.3.6 Igazítatlan elérés: mikor működik?

Igazítatlan memóriaelérésről (*unaligned access*) beszélünk, ha a hozzáférés címe nem többszöröse a hozzáférési szélességhez tartozó természetes igazításnak.

Az A64 alapprofilban nem igaz sem az, hogy minden ilyen elérés tiltott, sem az, hogy minden megengedett.

Helyzet	A64/Armv8-A alapprofil szerinti lényeg
Szokásos `LDR/STR`, Normal memória, kikapcsolt általános igazításellenőrzés	Az igazítatlan elérés megengedett lehet.
Bekapcsolt általános igazításellenőrzés	A megfelelő igazítatlan hozzáférés igazítási hibát válthat ki.
Device memóriatípus	Az igazítatlan adatelérés nem megengedett.
Exkluzív és atomi hozzáférések	Külön, szigorúbb igazítási követelmények érvényesek; a sima `LDR/STR` szabálya nem vihető át automatikusan.
Hozzáférés nem leképezett lapba is átnyúlik	Az igazítás megengedettségétől függetlenül memóriahiba keletkezhet.

Az általános ellenőrzést többek között az AArch64 privilegizált `SCTLR_EL1.A` vezérlőbit befolyásolja. Ezt Linux-felhasználói programból nem állítjuk. QEMU user-mode alatt sem valódi vendégkernel-konfigurációt módosítunk.

Három fontos különbség:

- **C és ISA:** egy hibásan igazított `long *` dereferálása C-ben nem válik szabályossá attól, hogy egy A64 betöltés azon a címen működne. Külső bájtpufferből igazított helyi objektumba másoláshoz a `memcpy` megfelelő eszköz.
- **Kódolás és igazítás:** az A64 `LDUR` nevében az „unscaled” skálázatlan offsetet jelent, nem az igazításellenőrzés megkerülését. Immediate-je előjeles, 9 bites bájteltolás: −256–255.
- **Helyesség és teljesítmény:** a megengedett igazítatlan hozzáférés költsége implementációfüggő, különösen cache-vonal vagy lap határán. Itt nem rendelünk hozzá ciklusszámot; QEMU nem ad hardverhű abszolút időzítést.

A GCC csomagolt, `packed` elrendezése megváltoztathatja az offseteket és az igazítást. Ez külön szerződés, nem a normál struktúraelrendezés „helytakarékos kapcsolója”, amely következmények nélkül használható.

7.3.7 Tipikus hibák és ellenpróbák

Tünet: a mező értéke hibás, de nincs összeomlás. Első hipotézisünk legyen a rossz offset vagy hozzáférési szélesség, ne rögtön a bájtsorrend.

Target: AArch64 objektumkód vizsgálata; nem futtatás.

```
aarch64-linux-gnu-objdump -dr lab-layout-asm.o
```

Ellenőrizzük, hogy az A64 kódban valóban a kívánt `Wn`/`Xn` szélesség és bájteltolás szerepel-e.

A futás közbeni vizsgálathoz az első terminálban:

```
# Target: qemu-user, GDB-kapcsolatra vár.
# Folytatás után mért kilépési állapot: 0.
qemu-aarch64 -g 1234 ./lab-layout
```

A második terminálban:

```
# Target: a qemu-user alatt futó AArch64 program.
gdb-multiarch -q ./lab-layout \
    -ex 'set pagination off' \
    -ex 'target remote :1234' \
    -ex 'break lab_after_update' \
    -ex 'continue' \
    -ex 'info registers x0 x1 x2 x3' \
    -ex 'x/24bx $x0' \
    -ex 'x/wd $x0+4' \
    -ex 'x/gd $x0+8' \
    -ex 'x/ub $x0+16' \
    -ex 'continue'
```

A töréspontnál az A64 `X0` még az objektum báziscíme. A mért értékek (GDB 15.1, QEMU 8.2.2): `X1=2`, `X2=1024`, `X3=2`; a mezők vizsgálata rendre 2, 1024 és 2 értéket mutat. A padding bájtjaira nincs előírt érték.

Tünet: mezősorrend-változtatás után romlik el a program. Hasonlítsuk össze az új `offsetof` értékeket az assembly konstansaival. Egyetlen offsetet javítsunk, majd ismételjük meg az ellenőrzést.

Tünet: ugyanazon mezőértékek mellett eltér a teljes memóriakép. A padding eltérhet. A struktúrák logikai egyenlőségét ezért általában nem helyettesíti a teljes objektumon végzett `memcmp`.

7.3.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Vezesd le a `lab_record_compact` összes mezőoffsetjét és végi kitöltését! Mennyi tárhelyet foglal 1000 eredeti, illetve 1000 átrendezett rekord? Miért nem elég 14 bájtos lépésköz?

2. (implementációs, középszint — cél: rutinépítés) Az eredeti `lab_record` mezősorrendjét változtasd a kompakt sorrendre. Frissítsd a fordítási idejű ellenőrzéseket és az assembly mezőeléréseit úgy, hogy az értékmódosítások változatlanok maradjanak. Ugyanazokkal a laborparancsokkal építs és futtass.

3. (hibakeresési, középszint — cél: rutinépítés) Állítsd vissza az eredeti elrendezést. Kizárólag az első A64 betöltést módosítsd erre: `ldr w1, [x0, #8]`. A hozzá tartozó tárolás maradjon `#4` offsetű. Jósolj kimenetet, majd `objdump` és GDB segítségével különítsd el az offsethibát a bájtsorrendi problémától.

7.3.9 Megoldások

1. megoldás — Segítség. Rendezd sorba a `lab_record_compact` mezőit igazítás szerint csökkenő sorrendbe, és alkalmazd a fejezet `align_up` képletét egymás után mindegyikre.

1. megoldás — Teljes megoldás. A `total` offsetje 0, a `count` mezőé 8, a `tag` mezőé 12, a `state` mezőé 13. A végpozíció 14, amelyet 8 bájtos igazításhoz 16-ra kerekítünk. A két tömb mérete 24000, illetve 16000 bájt; a különbség 8000 bájt. A 14 bájtos lépésköz mellett a második elem `long` mezője már nem lenne 8 bájtra igazítva.

2. megoldás — Segítség. Az offsetek megváltoznak, de a MŰVELETEK (melyik mezőhöz mennyit adunk hozzá) nem — csak az assembly `#offset` konstansait és a `_Static_assert` értékeit kell a kompakt elrendezéshez igazítanod.

2. megoldás — Teljes megoldás. Az új ellenőrzött offsetek: `tag=12`, `count=8`, `total=0`, `state=13`; a méret 16, az igazítás továbbra is 8. Az A64 `count`-betöltés és -tárolás offsetje `#8`, a `total` műveleteié `#0`, a `state` műveleteié `#13`. A szélességeket nem változtatjuk meg. A kijelölt mezőneves C-inicializálás továbbra is helyes. A mezőértékeket kiíró sor változatlan, az offseteket és méreteket kiíró sor módosul; a várt kilépési állapot 0.

3. megoldás — Segítség. A `#8` offset a `total` mező ELSŐ 4 bájtjára mutat, nem a `count`-ra — little-endian sorrendben milyen 32 bites értéket adna ez vissza 1000-ből?

3. megoldás — Teljes megoldás. Little-endian profilban a `total` mező első négy bájtjának előjel nélküli 32 bites értéke 1000. A hibás A64 betöltés ezt adja a `W1` regiszternek, majd az összeadás után 1005 kerül a `count` mezőbe. A `total` végértéke továbbra is 1024. A mért kilépési állapot 1. Az `objdump` megmutatja az eltérő betöltési és tárolási offsetet; GDB-ben a `count` és `total` címén olvasott értékek alátámasztják a hipotézist. Egyetlen javítás szükséges: az első betöltés offsetjét visszaállítani `#4` értékre.

7.3.10 Rövid összefoglaló

A struktúra mezőcíme a báziscím és a mező bájteltolásának összege. Az eltolásokat nem pusztán a mezőméretek összeadásával kapjuk: minden mezőt a saját igazításához, a teljes méretet pedig a struktúra igazításához kell kerekíteni. A `sizeof` tartalmazza a kitöltést, az `offsetof` közvetlenül ellenőrizhetővé teszi az assembly konstansait. A mezősorrend csökkentheti a méretet, de megváltoztatja az interfész elrendezési szerződését. Az A64 hozzáférési szélességet a C-mező méretéhez választjuk. Az igazítatlan elérés ISA-szintű megengedettsége nem helyettesíti a C szabályait, és nem garantál kedvező teljesítményt.

- **Igazítás:** az objektum kezdőcímének oszthatósági követelménye.
- **Mezőoffset:** a tag kezdőpozíciója bájtban, a struktúra kezdetéhez képest.
- **Padding:** a mezők közötti vagy utáni, nem mezőértéket tároló kitöltés.

7.3.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm AAPCS64 — alapadattípusok, összetett típusok mérete és igazítása, regiszterszerződés. **Tartalmilag ellenőrizve**: aktuális kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-vonatkozású részek Beta).
- Arm A-profile architektúra-dokumentáció — DDI0487; memória-hozzáférések és igazítás-ellenőrzés. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- GNU assembler kézikönyv — AArch64 operandusformák és GNU as direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- C11 bizottsági tervezet, N1570 — struktúraelrendezés, `sizeof`, `_Alignof` és `offsetof`. **Ellenőrizve**: ez a WG14 utolsó nyilvános bizottsági tervezete a C11 szabványosítás előtt; a gyakorlatban mindenhol „C11 szabványként” hivatkozzák, mert a fizetős ISO-szöveg nem nyilvános, de technikailag tervezet, nem az eladott ISO/IEC 9899:2011 dokumentum (az eltérés túlnyomórészt szerkesztési jellegű).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenet és GDB-értékek a lektor GNU gcc 13 + GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A telepített Ubuntu-csomagverziók, a statikus cross libc rendelkezésre állása és a GDB–QEMU munkafolyamat tényleges eredménye a hallgatói laborban ellenőrizendő. Az igazítatlan hozzáférés konkrét költségéről nem áll rendelkezésre mérési adat.

7.4 Bájtsorrend: endianness és miért releváns a határon

7.4.1 Miért a rendszerhatáron derül ki a bájtsorrend?

Egy C-program helyesen kezeli a `uint32_t` változókat, mégis hibás fájlt készít: a fogadó program minden hosszmezőt értelmetlenül nagynak lát. Hasonló jelenség hálózati kommunikációnál is előfordulhat. A számítás helyes, de a többbájtos érték **memóriabeli bájtsorrendje** nem egyezik a külső formátum előírásával.

A bájtsorrend (*endianness*) akkor válik láthatóvá, amikor ugyanazt az adatot egyszer többbájtos értékként, máskor bájtsorozatként értelmezzük. Ehhez a Load/store címzési módok ismerete szükséges: különösen a hozzáférési szélesség és az effektív cím megkülönböztetése.

Profil: az alfejezet minden assembly példája **A64 ISA, AArch64, Armv8-A, I. rész**, little-endian Linux-felhasználói környezetben. A GNU assembler direktívái ehhez a célhoz tartozó eszközlánc-elemek, nem processzorutasítások. Az AArch64 architektúra adat-bájtsorrendjét nem szabad általánosan, minden rendszerre little-endianként rögzíteni; itt a könyv kiválasztott profilját használjuk.

7.4.2 Definíció: melyik bájt kerül a kisebb címre?

Egy többbájtos érték tárolásakor:

- **Little-endian:** a legkisebb helyi értékű bájt kerül a legkisebb memóriacímre.
- **Big-endian:** a legnagyobb helyi értékű bájt kerül a legkisebb memóriacímre.

Legyen az előjel nélküli, 32 bites érték `0x12345678`, és jelölje `p` az első bájt szimbolikus címét:

Cím	Little-endian bájt	Big-endian bájt
`p + 0`	`0x78`	`0x12`
`p + 1`	`0x56`	`0x34`
`p + 2`	`0x34`	`0x56`
`p + 3`	`0x12`	`0x78`

A táblázatban lefelé haladva nő a cím. A szám szokásos hexadecimális leírásában viszont balra vannak a nagyobb helyi értékű számjegyek. A kettő összekeverése gyakori hibaforrás.

Levezetés: ha a növekvő címeken található négy bájt rendre b_0, b_1, b_2, b_3, little-endian értelmezéskor

$$V = b_0 + 2^8 b_1 + 2^{16} b_2 + 2^{24} b_3.$$

Itt V mértékegység nélküli, előjel nélküli 32 bites szám; minden b_i mértékegység nélküli bájtérték a 0–255 tartományban, az index pedig bájtonként növekvő címeltolást jelöl.

Ezért a `0x12 0x34 0x56 0x78` bájtsorozat little-endian betöltése `0x78563412` értéket ad, nem `0x12345678`-at.

Három fontos következmény:

1. A bájtsorrend **nem az egy bájton belüli bitek megfordítását** jelenti.
2. A regiszterben lévő számértéknek nincs memóriacímek szerinti bájtsorrendje. A bájtsorrend a bájtsorozat és a többbájtos érték közötti leképezéshez tartozik.
3. Az előjelkezelés külön kérdés. Ugyanannak a bitsorozatnak előjeles és előjel nélküli olvasata lehet, de ettől a tárolt bájtok sorrendje nem változik.

Az A64 `LDR Wn` négy bájtból képez 32 bites értéket; az A64 `LDRB Wn` egyetlen bájtot tölt be és nullával kiterjeszt. Egyetlen bájton nincs mit bájtsorrend szerint felcserélni.

7.4.3 Három gyakorlati hibaforrás

7.4.3.1 Fájlformátum: a natív objektum nem feltétlenül kész fájlmező

A `fwrite(&value, sizeof value, 1, file)` az objektum memóriabeli bájtjait írja ki. Ha a fájlformátum big-endian 32 bites mezőt követel, egy little-endian gépen ez önmagában hibás.

A formátum szabálya az irányadó, nem a processzoré. Rögzíteni kell:

- a mező bájtban mért szélességét;
- a bájtsorrendet;
- az előjeles vagy előjel nélküli értelmezést.

Teljes C-struktúra közvetlen kiírásakor az igazítás és a kitöltő bájtok is problémát okozhatnak; ezt a Struktúrák, igazítás és mezőelérés tárgyalja.

7.4.3.2 Hálózati protokoll: a bájtfolyamnak saját szerződése van

Számos hálózati protokoll big-endian, úgynevezett hálózati bájtsorrendet (*network byte order*) használ. Ez azonban nem jelenti azt, hogy minden alkalmazási protokoll minden mezője ilyen.

A C `htonl` és `ntohl` függvényei 32 bites értékek gépi és hálózati bájtsorrendje között alakítanak. Az átalakítást a megfelelő határon egyszer kell elvégezni. Ha a beolvasó réteg már dekódolta a mezőt, a feldolgozó réteg újabb cseréje ismét elrontja.

7.4.3.3 Union-alapú trükk: helyi megfigyelésből nem lesz hordozható formátum

Tegyük fel, hogy egy C-union `uint32_t value` és `unsigned char byte[4]` tagot tartalmaz. A `value = 0x12345678` beállítás után a jelenlegi little-endian profilban `byte[0]` értéke `0x78`.

A téves feltételezés az, hogy az első tömbelem mindig a hexadecimális leírás bal szélső bájtját tartalmazza. Nem: a legkisebb című bájtot tartalmazza.

Az objektumábrázolás `unsigned char` bájtokon keresztül megvizsgálható; a hiba itt a platformfüggő sorrendre épített szerződés. A `memcpy` sem végez bájtsorrend-konverziót. Egy bájtpuffer `uint32_t *` mutatóvá alakítása pedig a bájtsorrenden túl igazítási és C-típuselérési problémákat is okozhat.

7.4.4 Bájtcserélő utasítások: a csoportméret számít

Az alábbi táblázat minden utasítása **A64 ISA**. A bemenetek előjel nélküli bitmintaként értendők.

Utasítás	Bemenet	Eredmény	Művelet
`REV Wd, Wn`	`0x12345678`	`0x78563412`	Mind a négy bájt sorrendjét megfordítja
`REV16 Wd, Wn`	`0x12345678`	`0x34127856`	Minden 16 bites csoporton belül cserél
`REV Xd, Xn`	`0x1122334455667788`	`0x8877665544332211`	Mind a nyolc bájt sorrendjét megfordítja
`REV16 Xd, Xn`	`0x1122334455667788`	`0x2211443366558877`	Minden 16 bites csoporton belül cserél
`REV32 Xd, Xn`	`0x1122334455667788`	`0x4433221188776655`	Minden 32 bites csoporton belül cserél

Az A64 `REV32 Xd, Xn` **nem cseréli fel a két 32 bites csoport helyét**. Nincs külön A64 `REV32 Wd, Wn` alak: egy 32 bites érték teljes bájtcseréjéhez az A64 `REV Wd, Wn` használható.

Ezek regiszterutasítások: nincs immediate operandusuk, nem olvasnak memóriát, és nem módosítják az NZCV állapotjelzőket. A W-regiszterbe író alakok a hozzá tartozó X-regiszter felső 32 bitjét nullázzák.

7.4.5 Kidolgozott példa: natív érték és külső bájtsorozat

Munkakönyvtár: `~/arm-labor/a64/a025/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a025
cd ~/arm-labor/a64/a025
```

A labor három dolgot ellenőriz:

1. a natív 32 bites érték little-endian tárolását;
2. egy big-endian bájtsorozat dekódolását;
3. a különböző A64 bájtcserék eredményét.

Mentsük a következőt `lab-endian.s` néven:

```
// A64 ISA, Armv8-A; GNU as; little-endian Linux, I. rész.
.section .text
.global _start
.type _start, %function

_start:
    adr     x0, lab_native
    ldr     w1, [x0]

    adr     x2, lab_wire
    ldr     w3, [x2]
    rev     w4, w3

    rev16   w5, w1
```

```
    adr     x6, lab_wide
    ldr     x7, [x6]
    rev     x10, x7
    rev16   x11, x7
    rev32   x12, x7

    adr     x13, lab_output
    str     w1, [x13]
    rev     w14, w1
    str     w14, [x13, #4]

lab_snapshot:
    cmp     w1, w4
    b.ne    .Lfail

    ldrb    w15, [x13]
    cmp     w15, #0x78
    b.ne    .Lfail

    ldrb    w15, [x13, #4]
    cmp     w15, #0x12
    b.ne    .Lfail

    movz    w0, #0
    b       .Lexit

.Lfail:
    movz    w0, #1

.Lexit:
    movz    x8, #93
    svc     #0

.size _start, . - _start

.section .rodata
.balign 4
lab_native:
    .word   0x12345678

lab_wire:
    .byte   0x12, 0x34, 0x56, 0x78

.balign 8
lab_wide:
    .quad   0x1122334455667788

.section .bss
.balign 4
lab_output:
    .skip   8

.section .note.GNU-stack,"",%progbits
```

Az AArch64 GNU assembler `.word` direktívája itt négybájtos értéket bocsát ki a kiválasztott bájtsorrendben. A `.byte` viszont a felsorolt bájtokat adja ki, sorrendben. Ezért ugyanannak a logikai számnak a natív és a külső ábrázolása különböző lesz.

Immediate-ok: a példa minden konstansa és eltolása kódolható (lásd az A64 immediate-korlátok összefoglalóját); a `STR Wn` eltolása négy többszöröse, és a kis laboron belüli címkék az `ADR` hatótávolságán belül vannak. Az elágazások címkés, korlátozott PC-relatív eléréséről lásd a Feltétel nélküli és feltételes elágazás alfejezetet.

7.4.5.1 Összeállítás, linkelés, futtatás

Ubuntu 24.04 x86_64 hoston, **qemu-user céllal**:

```
# Target: qemu-user, AArch64 little-endian.
aarch64-linux-gnu-as -EL -march=armv8-a -g \
    -o lab-endian.o lab-endian.s

aarch64-linux-gnu-ld -EL -static -e _start \
    -o lab-endian lab-endian.o

aarch64-linux-gnu-readelf -h lab-endian

qemu-aarch64 ./lab-endian
printf 'kilépés=%d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): a program nem ír a szabványos kimenetre, a shell pedig `kilépés=0` sort jelez. Az 1 kilépési állapot valamelyik beépített összehasonlítás hibáját jelenti. Az ELF-fejléc adatábrázolása little-endian, géptípusa AArch64.

A 0 állapot csak a beépített feltételeket igazolja; a 64 bites cserék részletes ellenőrzése a következő lépés.

7.4.6 Bájtsorrend ellenőrzése GDB-memóriadumpban

Első terminál:

```
# Target: qemu-user, AArch64; várakozás GDB-kapcsolatra.
qemu-aarch64 -g 1234 ./lab-endian
```

Második terminálban készítsünk név szerint megadott GDB-parancsfájlt, majd indítsuk a hibakeresőt:

```
# Target: az előző qemu-user folyamat AArch64 programja.
cat > lab-endian.gdb <<'EOF'
set pagination off
target remote :1234
break *lab_snapshot
continue
show endian
info registers x1 x3 x4 x5 x7 x10 x11 x12 x14
x/4bx $x0
x/4bx $x2
x/8bx $x13
x/wx $x2
x/2wx $x13
continue
quit
EOF

gdb-multiarch -q -x lab-endian.gdb ./lab-endian
```

A `lab_snapshot` címnél a tárolások már végrehajtódtak, a kilépési út még nem módosította az A64 X0 regisztert.

Ez az első labor, amely név szerint megadott GDB-parancsfájlt (`-x lab-endian.gdb`) használ a korábbi `-ex` láncok helyett. A szkript előnye, hogy a vizsgálat lépései egy újrafelhasználható, verziókövethető fájlban olvashatók, és ugyanaz a parancssor a labor ismétlésekor változtatás nélkül újrafuttatható; a könyv későbbi részeiben is ezt a formát preferáljuk.

Mért regiszterértékek — A64 (GDB 15.1, QEMU 8.2.2):

Regiszter	Előjel nélküli érték	Jelentés
W1	`0x12345678`	Natív betöltés
W3	`0x78563412`	Külső bájtok nyers betöltése
W4	`0x12345678`	Dekódolt külső érték
W5	`0x34127856`	16 bites csoportonkénti csere
X7	`0x1122334455667788`	64 bites bemenet
X10	`0x8877665544332211`	Teljes 64 bites bájtcsere
X11	`0x2211443366558877`	16 bites csoportonkénti csere
X12	`0x4433221188776655`	32 bites csoportonkénti csere
W14	`0x78563412`	Big-endian tároláshoz előkészített érték

A GDB-parancs X-nézetben listázza a regisztereket. A W-nézetben írt értékek felső 32 bitje nulla.

Mért memóriaállapot növekvő címek szerint:

Terület	Bájtsorozat
`lab_native`	`0x78 0x56 0x34 0x12`
`lab_wire`	`0x12 0x34 0x56 0x78`
`lab_output`, első négy bájt	`0x78 0x56 0x34 0x12`
`lab_output`, következő négy bájt	`0x12 0x34 0x56 0x78`

A GDB `x/4bx` parancsa négy egybájtos egységet jelenít meg hexadecimálisan. Az `x/wx` ezzel szemben egy négybájtos egységet **a hibakereső aktuális cél-bájtsorrendje szerint értelmez**.

Ezért az `x/wx $x2` mért eredménye `0x78563412`. A debugger nem tudja, hogy a puffer egy protokoll big-endian mezője: csak négy bájtot lát egy little-endian célon. A mért GDB-kimenetben a `show endian` parancs a „currently little endian” sort adja.

Ne állítsuk át a GDB bájtsorrendjét azért, hogy a protokollmező „szebben” jelenjen meg. A célgép és a külső formátum bájtsorrendje külön tulajdonság.

7.4.7 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés és egyetlen javítás
`0x12345678` helyett `0x78563412`	Hiányzó bájtcsere	GDB-bájtdump; egy A64 `REV Wd, Wn` beillesztése a dekódolási határon
A bájtpárok jók, a teljes érték hibás	A64 `REV16` került `REV` helyére	Disassembly; csak az utasítás cseréje
Egy rétegben jó, a következőben hibás	Kétszeres konverzió	Regiszterérték vizsgálata mindkét réteghatáron; egy fölösleges csere eltávolítása
Csak bizonyos puffercímeken jelentkezik hiba	Nem pusztán bájtsorrend-probléma	Cím, igazítás és hozzáférési szélesség ellenőrzése

A tényleges utasítások vizsgálata:

```
# Target: a qemu-user számára épített AArch64 ELF.
aarch64-linux-gnu-objdump -d lab-endian
```

Ellenpróbához ne kizárólag `0x00000000` vagy `0xffffffff` értéket használjunk: ezek bájtcserére változatlanok. A különböző bájtokat tartalmazó tesztminta sokkal informatívabb.

7.4.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Egy négybájtos puffer tartalma növekvő címeken `0x01 0x02 0x03 0x04`. Milyen előjel nélküli értéket ad az A64 `LDR Wn` a jelenlegi profilban? Miért ad mást a big-endian értelmezés?

2. (implementációs, középszint — cél: rutinépítés) Egészítsük ki a labort úgy, hogy az A64 X7 regiszterben lévő értéket big-endian bájtsorozatként is eltárolja. Használjunk új, nyolcbájtos puffert. Ellenőrizzük GDB-vel.

3. (hibakeresési, középszint — cél: rutinépítés) A labor dekódoló A64 `rev w4, w3` utasítását cseréljük `rev16 w4, w3` alakra. Jósoljuk meg W4 értékét és a kilépési állapotot, majd végezzünk bájtdumpot és disassembly-ellenőrzést.

7.4.9 Megoldások

1. megoldás — Segítség. Rendeld hozzá a négy bájtot a megfelelő helyi értékekhez a little-endian szabály szerint ($b_0 + 2^8 b_1 + ...$), majd ismételd meg fordított hozzárendeléssel.

1. megoldás — Teljes megoldás. Little-endian betöltéskor az eredmény `0x04030201`; big-endian értelmezéssel `0x01020304`. Az eltérés oka, hogy más helyi értéket rendelünk ugyanahhoz a cím szerinti bájtsorozathoz. Az egyes bájtok bitmintája nem változik.

2. megoldás — Segítség. Egyetlen `REV`-fel teljes 64 bites bájtcserét kapsz; ha ezt az eredményt little-endian tárolod, a memóriabeli bájtsorrend éppen a kívánt big-endian lesz. Hol kell ezt a két lépést (csere, tárolás) a meglévő kódba illeszteni?

2. megoldás — Teljes megoldás. A meglévő `.bss` részhez, az AArch64 GNU assembler környezetében:

```
// AArch64 GNU as direktívák; I. rész.
.balign 8
lab_output64:
    .skip 8
```

Az A64 kódban a `lab_snapshot` címke elé:

```
// A64 ISA, little-endian cél.
adr     x16, lab_output64
rev     x17, x7
str     x17, [x16]
```

Az előző assemble–link–run parancsokkal építsük újra. A GDB-parancsfájlban a második `continue` elé kerüljön az `x/8bx $x16` vizsgálat. A várt bájtok: `0x11 0x22 0x33 0x44 0x55 0x66 0x77 0x88`. A regiszterbeli bájtcsere és a little-endian tárolás együtt állítja elő a kívánt külső sorrendet.

3. megoldás — Segítség. A `REV16` 16 bites csoportokon belül cserél — nézd meg W3 két 16 bites felét külön-külön, és csak azokon belül fordítsd meg a bájtokat.

3. megoldás — Teljes megoldás. Az A64 W3 értéke `0x78563412`, ezért az A64 `REV16` eredménye W4-ben `0x56781234`. Ez nem egyezik W1 értékével; a várt kilépési állapot 1. A `lab_wire` bájtjai változatlanok, tehát a hiba a dekódoló műveletben van. Egyetlen javításként állítsuk vissza az A64 `REV` utasítást, majd ismételjük meg az ellenőrzést.

7.4.10 Rövid összefoglaló

A bájtsorrend a többbájtos érték és a növekvő memóriacímeken tárolt bájtok kapcsolatát határozza meg. A könyv AArch64 laborprofilja little-endian, de a fájlok és protokollok ettől független ábrázolást írhatnak elő. Az A64 `REV` teljes regiszterszélességen, a `REV16` és `REV32` meghatározott csoportokon belül cserél bájtokat. A helyes konverzióhoz ezért a mező szélességét is ismerni kell. Hibakereséskor a GDB bájtonkénti nézetét vessük össze a regiszterértékkel; egy többbájtos memóriamegjelenítés már értelmezést is végez. A konverzió helyét dokumentáljuk, és különböző bájtokat tartalmazó mintával teszteljünk.

- **Little-endian:** a legkisebb helyi értékű bájt van a legkisebb címen.
- **Big-endian:** a legnagyobb helyi értékű bájt van a legkisebb címen.

7.4.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; adat-bájtsorrend, A64 betöltések, `REV`, `REV16`, `REV32`. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 célopciók és adatdefiniáló direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv — memóriavizsgálat, cél-bájtsorrend és távoli hibakeresés. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).
- Linux man-pages: byteorder(3) — gépi és hálózati bájtsorrend közötti C-konverziók. **Tartalmilag ellenőrizve**: a Linux man-pages projekt 6.19-es kiadása.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenet, regiszter- és memóriaállapotok a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A konkrét Ubuntu 24.04 csomagverziókat és a labor tényleges működését a hallgatói környezetben kell rögzíteni és ellenőrizni.

7.5 Statikus és dinamikus memória: .bss, .data és egyszerű allokáció

7.5.1 Miért számít, honnan származik a memória?

Egy adatfeldolgozó programnak szüksége lehet egy kezdetben ismert számlálóra, egy nullaértékekkel induló munkapufferre és egy csak futás közben meghatározható méretű tárolóra. C-ben ezeket néhány deklaráció és egy allokátorhívás elfedi; assemblyben nekünk kell megkülönböztetnünk **az objektum helyét, kezdeti tartalmát és élettartamát**.

Ebben az alfejezetben Linux alatt futó, little-endian **AArch64/A64** programot készítünk GNU assemblerrel. Minden alábbi assemblyutasítás és regiszter ebben a profilban értendő;

a direktívák GNU assembler-direktívák, nem processzorutasítások. Cortex-M-en nincs Linux-betöltő vagy Linux-rendszerhívás: az ottani inicializálást a Startup kód: .data másolása, .bss törlése tárgyalja.

A cél nem egy teljes `malloc` megírása, hanem a statikus tárolás és a futásidejű memóriaterület-növelés pontos megkülönböztetése.

7.5.2 Statikus memória: `.data` és `.bss`

A **statikus tárolási időtartamú objektum** (*static storage duration object*) a program teljes futása alatt létezik. Ez nem azonos a C `static` kulcsszó valamennyi szerepével: például a fájlszinten definiált, külső kapcsolású változónak is statikus a tárolási időtartama.

A GNU assembler két fontos, írható adatszekciója:

- **`.data`:** rendszerint a nem nulla kezdőértékkel rendelkező statikus adatok helye. A kezdeti bájtok az ELF-fájlban is szerepelnek.
- **`.bss`:** rendszerint a nulla-inicializált statikus adatok helye. A fájl leírja a szükséges méretet és igazítást, de a nullabájtokat nem tárolja adatként.

C-kapcsolatként az alábbi objektumok szokásos ELF-elhelyezése rendre `.data`, illetve `.bss`:

```
/* C-példa; célprofil: AArch64 Linux, little-endian. */
#include <stdint.h>

uint32_t counter = 7;
unsigned char scratch[4096];
```

Ez fordítói és objektumformátum-konvenció, nem a C szabvány szekciónév-előírása. Az explicit nulla inicializáló, például `uint32_t counter = 0;`, szintén lehet `.bss`-beli.

A GNU assembler `.bss` direktívája **nem futás közben törli a memóriát**. A Linux ELF-betöltése biztosítja a megfelelő tartomány kezdeti nullaértékét. A betöltő elsősorban a program-szegmenseket használja, nem a `.bss` szekció nevét keresi; a részletes hátteret Az ELF formátum alapjai és a program életciklusa adja.

Az igazítás itt is szerződés. A későbbi példában a GNU assembler `.balign 4` direktívája négybájtos, `.balign 16` direktívája tizenhat bájtos határra igazítja a következő objektumot. Ez a Struktúrák, igazítás és mezőelérés szabályainak közvetlen alkalmazása.

7.5.3 Miért kisebb a fájl `.bss` használatával?

ELF-ben a `.data` jellemzően `PROGBITS`, a `.bss` pedig `NOBITS` típusú szekció. Utóbbinak van memóriabeli mérete, de nincs ugyanekkora fájlbeli adattartalma.

Levezetett példa: egy 4096 bájtos, nullával induló pufferhez:

Elhelyezés	Kezdeti adatbájtok a fájlban	Objektumméret a memóriában
`.data`, 4096 tárolt nullabájttal	4096 bájt	4096 bájt
`.bss`	0 bájt	4096 bájt

A teljes ELF-fájl méretkülönbsége nem szükségképpen pontosan 4096 bájt: szekciófejlécek, igazítási kitöltések és a linker elrendezése is számítanak.

A `.bss` tehát **fájlméretet takarít meg, nem az objektum virtuális címterületét tünteti el**. A fizikai memórialapok tényleges hozzárendelése már operációsrendszer-kérdés. A lusta lapkiosztás (*demand paging*) miatt a lefoglalt virtuális méretből önmagában nem következik azonnali, ugyanekkora fizikaimemória-fogyasztás.

7.5.4 Verem, heap és statikus tárolás

Tulajdonság	Statikus memória	Verem (*stack*)	Heap
Tipikus feladat	Globális állapot, állandó méretű munkaterület	Hívásokhoz kötött ideiglenes állapot	Futás közben változó méretű objektumok
Élettartam	A program teljes futása	Többnyire a hívási keret élettartama	Az allokációs szerződés szerint
Elhelyezés kezelője	Fordító, assembler, linker, betöltő	Hívó és hívott kód	Allokátor, alatta rendszerhívások
Kezdeti tartalom	`.data`: előírt bájtok; `.bss`: nulla	Nem általánosan nulla	Az API-tól függ; `malloc` nem nulláz
Felszabadítás	Nem objektumonként történik	Keret lebontásával	Allokátorhívással vagy terület-visszaadással

A verem részletes kezelése A stack frame felépítése és a verem iránya témája. Most elég látni: egy lokális tömb nem azért „verem”, mert tömb, és egy nagy tömb nem automatikusan „heap”. A tárolási módot a létrehozás és az élettartam szerződése határozza meg.

7.5.5 A `brk` és az `mmap` szerepe

A **programtöréspont** (*program break*) a hagyományos, `brk`-alapú adattartomány végcíme. A Linux `brk` rendszerhívás ezt a végcímet próbálja módosítani.

A Linux AArch64 rendszerhívási ABI szerint a `brk` száma 214, amelyet az A64 **X8** regiszter kap; a kért végcím az A64 **X0** regiszterbe kerül. Ezek Linux ABI-értékek, nem az A64 ISA részei.

Fontos a nyers rendszerhívás szerződése:

1. Nulla cím megadásával lekérdezhetjük az aktuális töréspontot.
2. Növeléskor új **végcímet** adunk meg, nem bájtszámot.
3. Siker esetén a visszatérési érték a beállított végcím.
4. Sikertelen növeléskor Linuxon az aktuális töréspont érkezik vissza, nem a szokásos negatív `-errno`.

Ez különbözik a C-könyvtári `brk()` burkolófüggvény visszatérési szerződésétől.

Ha a régi végcím B, a kért növelés pedig n bájt, akkor siker után az újonnan használható tartomány:

$$[B,\ B+n)$$

Itt B folyamatbeli virtuális bájtcím; $n > 0$, és a címösszegnek ábrázolhatónak kell lennie. **A lefoglalt rész kezdete a régi töréspont, nem a visszakapott új végcím.**

A `brk` önmagában nem tart nyilván objektumokat, nem kezel szabadlistát és nem oldja meg az általános igazítást. A tartomány közepéből nem lehet vele tetszőleges blokkot visszaadni. C-könyvtári `malloc` mellett ne módosítsuk önállóan a töréspontot: megsérthetjük az allokátor belső állapotát.

A Linux AArch64 `mmap` rendszerhívás száma 222. Ezzel különálló virtuális leképezés kérhető; anonim, privát, írható leképezést gyakran nagyobb memóriaterületekhez használnak. Nulla címjavaslat esetén a rendszer választ kezdőcímet, a leképezés lapigazított, kezdeti tartalma anonim leképezésnél nulla. A nyers visszatérési értéket itt negatív hibakódra kell ellenőrizni. A visszaadást a 215 számú `munmap` szolgálja.

A két hívás AArch64 Linux regiszterkiosztása (a `brk`-hoz hasonlóan X8 a hívásazonosító):

Hívás	X8	Bemenet	Visszatérés
`mmap`	222	X0 = címtipp (0 = a rendszer választ), X1 = hossz bájtban, X2 = védelem (`PROT_READ|PROT_WRITE` = 3), X3 = jelzők (`MAP_PRIVATE|MAP_ANONYMOUS` = 0x22), X4 = fájlleíró (anonim esetén −1), X5 = eltolás (0)	X0 = a leképezés kezdőcíme, hiba esetén negatív `-errno`
`munmap`	215	X0 = a leképezés kezdőcíme, X1 = hossz bájtban	X0 = 0 siker esetén, hiba esetén negatív `-errno`

A nyers rendszerhívás visszatérése nem a C-könyvtári `MAP_FAILED` (`(void *)-1`) értéket használja: a `[-4095, -1]` tartományba eső visszatérés hibát jelez, minden más érték érvényes leképezési cím. A `munmap` hosszát a rendszer felfelé kerekíti a lapméretre, és a leképezés kezdőcímének lapigazítottnak kell lennie.

A lapigazítás nem jelent tetszőleges, lapméretnél nagyobb igazítást; a lapméretet sem szabad általánosan 4096 bájtnak feltételezni.

7.5.6 Kidolgozott példa: statikus puffer és egyszerű heap-növelés

Munkakönyvtár: `~/arm-labor/a64/a026/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a026
cd ~/arm-labor/a64/a026
```

A `lab-memory.s` fájl önálló, C-könyvtár nélküli program. Ellenőrzi a statikus kezdőértékeket, 4096 bájttal növeli a töréspontot, használja a kapott területet, majd visszaállítja az eredeti töréspontot. A 4096 itt **kért bájtszám**, nem feltételezett rendszerszintű lapméret.

```
// AArch64 Linux, A64 ISA, Armv8-A, little-endian.
// GNU assembler-direktívák; nincs C runtime vagy malloc.

.arch armv8-a
```

```
.data
.balign 4
counter:
    .word 7

.bss
.balign 16
scratch:
    .skip 4096

.text
.global _start
.type _start, %function
_start:
    // Inicializált statikus adat: 7 -> 8.
    adrp x9, counter
    add  x9, x9, :lo12:counter
    ldr  w2, [x9]
    cmp  w2, #7
    b.ne static_error
    add  w2, w2, #1
    str  w2, [x9]

    // Két mintavételi pont a nulla-inicializált pufferben.
    adrp x10, scratch
    add  x10, x10, :lo12:scratch
    ldr  w3, [x10]
    cbnz w3, static_error
    ldr  w3, [x10, #4092]
    cbnz w3, static_error
    str  w2, [x10]

    // Linux AArch64: brk(0), aktuális töréspont lekérdezése.
    mov  x0, #0
    mov  x8, #214
    svc  #0
    mov  x19, x0

    // Új végcím = régi végcím + 4096 bájt.
    adds x20, x19, #1, lsl #12
    b.cs heap_error
    mov  x0, x20
    mov  x8, #214
    svc  #0
    cmp  x0, x20
    b.ne heap_error

    // Egyetlen bájt használata: nincs külön igazítási igény.
    mov  w2, #42
    strb w2, [x19]
    ldrb w3, [x19]
    cmp  w3, #42
    b.ne heap_error

lab_inspect:
    // A saját növelés visszavonása.
    mov  x0, x19
    mov  x8, #214
    svc  #0
    cmp  x0, x19
    b.ne heap_error
```

```
    mov  x0, #0
    b    finish

static_error:
    mov  x0, #1
    b    finish

heap_error:
    mov  x0, #2

finish:
    mov  x8, #93
    svc  #0

.size _start, .-_start
.section .note.GNU-stack,"",%progbits
```

Az A64 **X19** a régi, az **X20** a kért új végcímet őrzi. Nincs függvényhívás: a program közvetlenül a belépési ponttól a kilépési rendszerhívásig fut.

A korlátokat az A64 immediate-korlátok összefoglalója adja; ebből a példában két dolog számít:

- A 4096 bájtos növelés az `ADD` 12 bites mezőjébe csak eltolva fér bele, ezért `#1, lsl #12`.
- A 32 bites `LDR` skálázott eltolásaként a `#4092` még kódolható (4×1023). Az `ADRP` utáni `:lo12:` relokáció a szimbólumcím alsó 12 bitjét adja.

A szimbolikus címképzés miatt nincs szükség kézzel választott folyamatcímre.

7.5.6.1 Összeállítás, linkelés, futtatás

Az alábbi parancsok célja **qemu-user / AArch64**, Ubuntu 24.04 hoston:

```
# Target: qemu-user, AArch64 Linux, little-endian.
aarch64-linux-gnu-as -EL -g -o lab-memory.o lab-memory.s
aarch64-linux-gnu-ld -EL -e _start -o lab-memory lab-memory.o

aarch64-linux-gnu-readelf -SW lab-memory
aarch64-linux-gnu-readelf -lW lab-memory
aarch64-linux-gnu-objdump -d lab-memory

qemu-aarch64 ./lab-memory
printf 'kilepesi_allapot=%d\n' "$?"
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): `kilepesi_allapot=0`. A program saját szöveges kimenetet nem ad. Az 1 statikusadat-ellenőrzési, a 2 heapkezelési vagy heapadat-ellenőrzési hibát jelez.

A `readelf` mért kimenetében a `.bss` típusa `NOBITS`, mérete `0x1000`; a `.data` típusa `PROGBITS`, mérete 4 bájt. Az írható betölthető szegmensnél `FileSiz=0x4`, `MemSiz=0x1010`; a `MemSiz` és `FileSiz` különbsége mutatja a fájladattal nem teljesen lefedett memóriarészt.

A `lab_inspect` címkénél mért állapot (GDB 15.1, QEMU 8.2.2):

Objektum vagy A64 regiszter	Mért tartalom
`counter`	Előjel nélküli 32 bites 8
`scratch` első négy bájtja	Little-endian: `0x08 0x00 0x00 0x00`
`scratch` fennmaradó bájtjai	Nulla
X19	Eredeti töréspont

Objektum vagy A64 regiszter	Mért tartalom
X20	**X19** + 4096
X3	Előjel nélküli 64 bites 42
X19 címén egy bájt	`0x2A`

A program a `.bss` kezdeti nullázottságát csak két ponton mintavételezi; ez nem teljes tartományvizsgálat.

7.5.7 Hibák és ellenpróbák

Hibás elképzelés: a `.bss` inicializálatlan, tehát szemétértékű. Linux ELF-betöltés után a tartomány nullaértékű. Ha nem ezt látjuk, először ellenőrizzük, hogy a program nem írta-e már felül, illetve valóban a megfelelő szimbólum címét vizsgáljuk-e.

Hibás cím: az új töréspontot használjuk pufferkezdetként. Ez a foglalás utáni első cím. Az írás nem válik helyessé attól, hogy egy lapkerekítés miatt esetleg nem okoz azonnali hibát.

Hibás hibakezelés: a `brk` eredményében csak negatív értéket keresünk. A nyers Linux `brk` növelésénél a kért és a visszakapott végcímet kell összevetni.

Hibás élettartam: visszaállítás után tovább használjuk a területet. A régi mutató bitmintája megmaradhat, de már nem jogosít hozzáférésre. A frissen kért `brk`-tartományt se kezeljük általános nullázó allokátorként: korábban használt, visszaadott rész ismételt kiterjesztésekor nem alapozhatunk ilyen szerződésre.

A vizsgálathoz az első terminálban:

```
# Target: qemu-user; a fenti build, induláskor GDB-re vár.
qemu-aarch64 -g 1234 ./lab-memory
```

A második terminálban:

```
# Target: a qemu-user AArch64 folyamata.
gdb-multiarch -q ./lab-memory \
  -ex 'target remote :1234' \
  -ex 'break lab_inspect' \
  -ex continue \
  -ex 'info registers x19 x20 x3' \
  -ex 'x/wu $x9' \
  -ex 'x/wu $x10' \
  -ex 'x/ub $x19' \
  -ex continue \
  -ex quit
```

A mért eredmény a fenti táblázat, majd normál kilépés. Hibánál a munkamenet: **tünet → hipotézis → regiszterek és disassembly ellenőrzése → egyetlen módosítás → újrafuttatás**. Az emuláció itt funkcionális ellenőrzés, nem hardverhű időmérés.

7.5.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért nem jelent 4096 bájttal kisebb futásidejű objektumot, ha a puffert `.data` helyett `.bss` tárolja? Ki biztosítja a kezdeti nullákat?

2. (implementációs, középszint — cél: rutinépítés) Növeld a dinamikus területet 8192 bájtra. Írd a 42 értéket az első és az utolsó bájtjába, és ellenőrizd mindkettőt. A visszaállítás maradjon meg.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Készíts `lab-memory-filezeros.s` változatot, amelyben kizárólag a `.bss` direktívát cseréled `.data` direktívára. Hasonlítsd össze a szekciókat és a fájlméreteket. Miért nem feltétlenül pontosan 4096 bájt a különbség?

7.5.9 Megoldások

1. megoldás — Segítség. A `.bss` és a `.data` közül melyik befolyásolja a futásidejű VIRTUÁLIS CÍMTARTOMÁNY méretét, és melyik csak a FÁJLBAN tárolt bájtokat?

1. megoldás — Teljes megoldás. A puffer mindkét esetben 4096 bájt címtartományt igényel. A `.bss` csak a kezdő nullabájtok fájlbeli tárolását hagyja el; a Linux ELF-betöltése gondoskodik a nulla kezdeti tartalomról.

2. megoldás — Segítség. $2 \cdot 4096 = 8192$; az `ADD`/`ADDS` immediate 12 bites, opcionális 12 bites balra tolással kódolható — hogyan írható fel ezzel a 8192?

2. megoldás — Teljes megoldás. Az A64 növelés `adds X20, X19, #2, lsl #12` alakú: $2 \cdot 4096 = 8192$ bájt. Az utolsó bájt címe **X20** − 1; ezt például az A64 `sub X4, X20, #1` képezheti. Ezután `strb W2, [X4]`, majd `ldrb W3, [X4]` használható. Mindkét visszaolvasásnál 42 az elvárt előjel nélküli érték. A **X20** címére már tilos írni.

3. megoldás — Segítség. A `.data` ugyanazt a méretet jelenti a memóriában, mint a `.bss`, de a FÁJLBAN is tárolnia kell a nullabájtokat — mi minden más (fejlécek, igazítás) változhat még emellett a szekcióelrendezésben?

3. megoldás — Teljes megoldás. A változat létrehozása és vizsgálata:

```
# Target: qemu-user, AArch64 Linux, little-endian.
sed 's/^[[:space:]]*\.bss[[:space:]]*$/    .data/' \
  lab-memory.s > lab-memory-filezeros.s

aarch64-linux-gnu-as -EL -g \
  -o lab-memory-filezeros.o lab-memory-filezeros.s
aarch64-linux-gnu-ld -EL -e _start \
  -o lab-memory-filezeros lab-memory-filezeros.o

aarch64-linux-gnu-readelf -SW lab-memory-filezeros
wc -c lab-memory lab-memory-filezeros

qemu-aarch64 ./lab-memory-filezeros
printf 'kilepesi_allapot=%d\n' "$?"
```

A GNU assembler `.skip 4096` itt a `.data` részeként tárolt nullákat hoz létre. **Mért eredmény (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2):** továbbra is 0 kilépési állapot, de nagyobb fájl. A mért fájlméret 2392 bájtról 6408 bájtra nőtt; a különbség 4016 bájt, nem 4096. A `.data` a számlálót, az igazítási kitöltést és a puffert is tartalmazza; a szekcióelrendezés és a fejlécek változása miatt a teljes méretkülönbséget mérni kell, nem pontosan 4096-nak kijelenteni.

7.5.10 Rövid összefoglaló

A memória eredete meghatározza a kezdeti tartalom és az élettartam szerződését. Linuxon a `.data` kezdőértékei fájlból származnak, a `.bss` nulláit a betöltés biztosítja. Ettől a `.bss` kisebb fájlt eredményezhet, de a futó programban továbbra is helyet igényel. A nyers `brk` végcímet módosít: eredményét a kért címmel kell összevetni, a használható új rész pedig a régi végcímen kezdődik. Az `mmap` külön leképezést adhat. Egyik rendszerhívás sem helyettesíti önmagában az objektumokat és igazítást kezelő allokátort.

- **Statikus tárolás:** a program teljes futásáig élő objektumok tárolása.
- **Programtöréspont:** a `brk`-kel kezelt tartomány végcíme.
- **Allokátor:** memóriaterületek kiosztását és visszavételét kezelő komponens.

7.5.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GNU assembler kézikönyv — szekciók, adattároló és igazítási direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- Linux `brk(2)` — a nyers rendszerhívás és a C-könyvtári burkoló eltérése. **Tartalmilag ellenőrizve**: a Linux man-pages projekt 6.19-es kiadása.
- Linux `mmap(2)` — anonim leképezés, igazítás és hibakezelés. Ugyanaz a man-pages 6.19-es kiadás.
- Linux `elf(5)` — `SHT_NOBITS`, szekciók és betölthető szegmensek. Ugyanaz a man-pages 6.19-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenet, a szekciótáblázat, a GDB-állapot és a fájlméretek a lektor GNU as 2.42 + GNU ld 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A pontos Ubuntu-csomagverziókat, a QEMU adott kiadásának viselkedését és az ELF-elrendezést a laborban kell rögzíteni. Az `mmap`/`munmap` regisztertáblázata dokumentációból származó levezetés, nem ebben a laborban futtatott mérés.

8 Függvények és az AAPCS64 hívási konvenció

8.1 A stack frame felépítése és a verem iránya

8.1.1 Miért kell a függvénynek saját veremterület?

Egy C-függvény lokális változókat használ, közben másik függvényt hív, majd visszatér a hívójához. Assemblyben mindhárom művelethez konkrét tárolási és helyreállítási döntés tartozik. Hol marad meg az eredeti argumentum, ha a meghívott függvény felülírja az argumentumregisztert? Mi őrzi a visszatérési címet, ha egy újabb hívás megváltoztatja azt?

A megoldás egyik alapvető eszköze a **veremkeret (stack frame)**: a veremnek az aktuális függvényaktivációhoz rendelt területe. Rekurzív híváskor ugyanannak a függvénynek több, egymástól elkülönülő aktivációja és kerete lehet.

Az alfejezet teljes architekturális kontextusa: A64 ISA, AArch64 Linux, Armv8-A alapprofil, little-endian. Az alábbi regiszterek és utasítások ebben a környezetben értendők; az assemblydirektívák a GNU assembler AArch64-célú használatához tartoznak.

A Hívás és visszatérés: BL/RET bevezető mintája alapján már tudjuk: a `BL` az `X30` regiszterbe írja a visszatérési címet, de nem készít veremkeretet. Most ezt a hiányzó műveletsort építjük fel. A teljes regisztermegőrzési szerződést Az AAPCS64 regiszterszerződés tárgyalja.

8.1.2 A lefelé növekvő verem modellje

A **veremmutató (stack pointer)** A64-en az `SP`. Az AAPCS64 lefelé növekvő vermet ír le: új terület lefoglalásakor a veremmutató értéke csökken.

Legyen a függvény belépésekor:

$$SP_{\text{belépés}} = S.$$

Ha a függvény n bájtos keretet foglal, akkor:

$$SP_{\text{keret}} = S - n.$$

Itt S egy bájtcím, n pedig bájtban megadott, nemnegatív keretméret. A frissen lefoglalt tartomány:

$$[S - n,\ S).$$

Az alsó határ beletartozik, a felső nem. Ezért az új keret első bájtja az új `SP` által mutatott helyen van; az utolsó bájtja az eredeti `SP` előtti bájt.

Relatív hely	Jelentés
Az `S` címtől felfelé	A hívóhoz tartozó aktív veremtartomány
`[S-n, S)`	Az aktuálisan lefoglalt keret
Az `S-n` cím alatt	Még nem lefoglalt veremterület

Az A64 egyszerű keretfoglalási mintája:

```
// A64 ISA; szemléltető részlet.
sub sp, sp, #32       // 32 bájt lefoglalása
// A keret használata.
add sp, sp, #32       // Az eredeti SP visszaállítása
```

A „lefelé” itt **kisebb numerikus címeket** jelent, nem fizikai irányt. A memória bájtjai továbbra is növekvő címekkel követik egymást; egy mező bájtsorrendjét ettől függetlenül a little-endian profil határozza meg.

A keretfoglalás nem nullázza a memóriát, és nem kér új leképezést az operációs rendszertől. A `SUB` csak a regiszter értékét változtatja meg. A Linux veremkezelése, a rendelkezésre álló virtuális címtartomány és a veremméretkorlát külön réteg. Túl nagy keret esetén a későbbi memóriaelérés hibát okozhat.

Nincs általánosan használható, az aktuális `SP` alatti ideiglenes tárolóterület: az AAPCS64 szerint előbb le kell foglalni a szükséges részt, és csak utána szabad benne adatot elhelyezni.

8.1.3 Miért 16 bájtos az igazítás?

Az **igazítás (alignment)** azt jelenti, hogy egy cím egy megadott határ többszöröse. A 16 bájtosan igazított veremmutató feltétele:

$$SP \bmod 16 = 0.$$

Bitszinten ez azt jelenti, hogy az `SP` alsó négy bitje `0b0000`.

Két szabályréteget különböztessünk meg:

- **AAPCS64-szerződés:** a veremmutató nyilvános hívási határon 16 bájtosan igazított; az `SP`-n keresztüli memóriaeléréshez szintén teljesülnie kell az előírt igazításnak.
- **ISA és végrehajtási környezet:** az `SP`-alapú memóriaelérésekhez igazításellenőrzés kapcsolódhat. A szabálysértés konkrét hibajelenségét az engedélyezett ellenőrzések és az emulátor viselkedése is befolyásolja.

A labor egyszerűbb, erősebb szabályt követ: **az `SP` minden köztes állapotában is 16 bájtosan igazított marad**.

Ez nem azért szükséges, mert az A64 minden adatátvitele 16 bájtos lenne. Az itt használt `Xn` regiszterek 8 bájtos adatokat tárolnak. A közös 16 bájtos határ egységes ABI-alapot ad többek között a nagyobb igazításigényű adatok és a különböző fordítóprogramok által előállított keretek kezeléséhez. Egyedi utasítások igazítási lehetőségeiből nem következik, hogy az ABI-követelmény elhagyható.

8.1.3.1 Levezetés: a szükséges keretméret

Tegyük fel, hogy:

- az érkező `SP` 16 bájtosan igazított;
- a keretben két 8 bájtos mentett érték szerepel;
- a lokális tárolás további L bájt;
- nincs külön verembeli argumentumterület vagy más mentés;
- a lokális objektumok elrendezését már megfelelően igazítottuk.

Ekkor a minimális, 16 bájtra kerekített keretméret:

$$n = 16 \left\lceil \frac{16 + L}{16} \right\rceil .$$

Az L és n mennyiségek egysége bájt. Egyetlen 8 bájtos lokális értéknél:

$$n = 16 \left\lceil \frac{16 + 8}{16} \right\rceil = 32 \text{ bájt.}$$

A tényleges adattartalom 24 bájt; további 8 bájt **kitöltés (padding)** szükséges.

Az igazítás megmarad, mert:

$$(S - 32) \bmod 16 = 0.$$

Ezzel szemben 24 bájt levonása után:

$$(S - 24) \bmod 16 = 8.$$

A 24 bájtos keret tehát elég lenne az adatok mérete alapján, de nem tartaná meg az előírt veremigazítást. Maga a kivonás nem feltétlenül okoz kivételt; ettől a létrejött állapot még szabálytalan lehet.

8.1.4 A keret mint ablak a hívásláncban

A veremkeret a híváslánc egy szintjének saját „ablaka”. A függvény ismert eltolásokkal éri el benne a megőrzendő értékeket.

A példánk keretelrendezése:

Eltolás az új `SP`-hez képest	Méret	Tartalom
`+0`	8 bájt	A hívó mentett `X29` értéke
`+8`	8 bájt	A belépéskori `X30`, vagyis az LR
`+16`	8 bájt	Lokális tároló: az eredeti argumentum
`+24`	8 bájt	Kitöltés; tartalma nem meghatározott

A két első mező együtt **keretrekordot (frame record)** alkot. Az AAPCS64 szerinti rekord alacsonyabb című eleme az előző keretrekord mutatója, a magasabb című eleme a mentett

visszatérési cím. A rekord helye a kereten belül általánosan nem rögzített; itt tudatosan a keret aljára helyezzük.

A **keretmutató (frame pointer, FP)** szerepét az `X29` töltheti be. A példában:

```
// A64 ISA; a régi x29 mentése után.
mov x29, sp
```

Ettől kezdve az `X29` az új keretrekordra mutat. A régi `X29` mentése láncot hoz létre a korábbi keretek felé.

A keretmutató előnye a stabil viszonyítási pont. Ha egy összetettebb függvény később módosítja az `SP`-t, az `X29` továbbra is a kiválasztott keretbázist jelölheti.

A keretmutató használata nem minden függvényben kötelező. A platformszabályok és a fordítási beállítások függvényében optimalizált Linux-kód elhagyhatja. Fix méretű keret mezői közvetlenül az `SP`-hez képest is elérhetők. A keretmutató hiánya nem jelenti a veremkeret hiányát, és nem szünteti meg a regisztermegőrzési kötelezettségeket.

A példában a keretrekordot megmutatjuk, de automatikus veremvisszafejtési metaadatot nem készítünk; a debugger teljes visszahívásilánc-felismerését ezért nem tekintjük ellenőrzött tulajdonságnak.

8.1.5 Kidolgozott példa: argumentum megőrzése egy további híváson át

Munkakönyvtár: `~/arm-labor/a64/a027/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a027
cd ~/arm-labor/a64/a027
```

A program egy előjel nélküli, 64 bites bemeneten ezt számolja:

$$f(x) = (x + 5) + x.$$

Az összeadások A64-en modulo 2^{64} értendők. A választott $x = 7$ bemenetnél nincs túlcsordulás, az eredmény 19.

A `lab_frame` függvény:

1. lefoglalja a keretét;
2. elmenti a régi keretmutatót és visszatérési címet;
3. eltárolja az eredeti argumentumot;
4. meghívja a hozzáadó segédfüggvényt;
5. felhasználja a megőrzött argumentumot;
6. visszaállítja a mentett állapotot és az `SP`-t.

Mentsük a következő teljes forrást `lab-stack.s` néven. A program `_start`-tal induló, önálló szemléltetés, ezért **CFI nélküli**: a kerete csak a keretlánc alapján követhető. A C-ből hívható függvények CFI-szabályát a Prológus és epilógus alfejezet adja.

```
// A64 ISA, Armv8-A; GNU as; little-endian laborprofil.
.section .text
.p2align 2

.global _start
.type _start, %function
_start:
    mov x29, xzr              // A demonstráció keretláncának vége.
    mov x0, #7                // Előjel nélküli, 64 bites bemenet.

lab_call:
    bl lab_frame

lab_after_call:
    cmp x0, #19
    cset x0, ne               // 0: helyes eredmény; 1: eltérés.
    mov x8, #93               // Linux AArch64 exit rendszerhívás.
    svc #0
.size _start, . - _start

.p2align 2
.type lab_frame, %function
lab_frame:
    sub sp, sp, #32
    str x29, [sp, #0]
    str x30, [sp, #8]
    mov x29, sp
    str x0, [sp, #16]

lab_frame_ready:
    bl lab_bump

lab_after_child:
    ldr x1, [sp, #16]
    add x0, x0, x1

    ldr x30, [sp, #8]
    ldr x29, [sp, #0]
    add sp, sp, #32
    ret
.size lab_frame, . - lab_frame

.p2align 2
.type lab_bump, %function
lab_bump:
    add x0, x0, #5
    ret
.size lab_bump, . - lab_bump

.section .note.GNU-stack,"",%progbits
```

A `lab_bump` levélfüggvény: nem végez további hívást, és nincs szüksége saját keretre. A hívó `lab_frame` viszont nem hagyhatja kizárólag az `X30`-ban a visszatérési címét, mert a belső `BL` felülírja azt.

A kódbeli konstansok és eltolások mind kódolhatók (lásd az A64 immediate-korlátok összefoglalóját). A példára jellemző feltétel: az `[SP, #offset]` eltolásoknak 8 többszörösének kell lenniük, mert a 64 bites `LDR`/`STR` 8 bájttal skálázza a mezőt.

A prológusban a két külön `STR` helyett a Prológus és epilógus alfejezetben bemutatott `STP` ugyanezt egy utasítással teszi; a két forma között nincs szemantikai különbség, itt a mentés lépéseinek szemléltetése miatt szerepelnek külön tárolások.

A `MOV X29, SP` nem általános logikai adatmozgatás: assembleralias az `ADD X29, SP, #0` alakra.

8.1.5.1 Összeállítás, linkelés és futtatás

Cél: `qemu-user`, `qemu-aarch64`, little-endian AArch64 Linux. A parancsok az Ubuntu 24.04-es laborprofil telepített cross-eszközeit feltételezik.

```
# Target: qemu-user / AArch64 Linux, little-endian.
aarch64-linux-gnu-as -g -EL -march=armv8-a \
    -o lab-stack.o lab-stack.s

aarch64-linux-gnu-ld -EL -e _start \
    -o lab-stack lab-stack.o

qemu-aarch64 ./lab-stack
lab_status=$?
printf 'kilepesi_allapot=%d\n' "$lab_status"

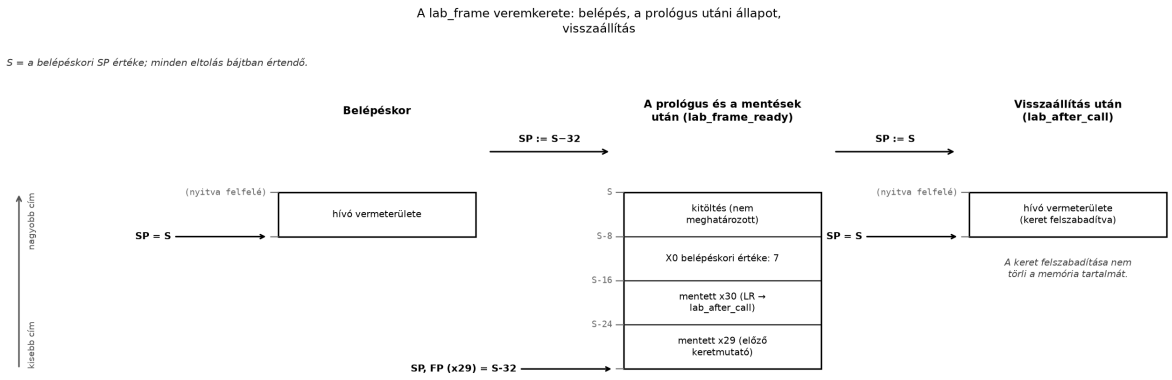
# Az előállított A64 gépi kód ellenőrzése.
aarch64-linux-gnu-objdump -d lab-stack
```

Mért eredmény (mérve: GNU as 2.42, QEMU 8.2.2): a program nem ír saját szöveget; a shell `kilepesi_allapot=0` sort jelenít meg. Az összeállítás és linkelés kilépési állapota szintén 0.

8.1.6 A keret állapotának ellenőrzése GDB-vel

A helyes számeredmény önmagában nem bizonyítja a helyes keretkezelést. Ellenőrizzük külön a veremmutatót és a mentett tartalmat.

Az alábbi ábra a `lab_frame` keretét mutatja három pillanatban — ugyanazokat az állapotokat, amelyeket a következő GDB-munkamenet számokkal is igazol: a belépéskori `SP = S`, a `lab_frame_ready`-nél rögzült, konkrét címekkel feliratozott, összefüggő 32 bájtos keret, majd a visszaállítás utáni, ismét `SP = S` állapot.



8.1. ábra: A lab_frame veremkerete: belépés, a prológus utáni állapot, visszaállítás - A mért futás állapotát illusztráló memóriaállapot a lab_frame példájához (x = 7).

Az első terminálban:

```
# Target: qemu-user; induláskor debuggerre vár.
qemu-aarch64 -g 1234 ./lab-stack
printf 'kilepesi_allapot=%d\n' "$?"
```

A második terminálban az alábbi parancs végigvezeti a vizsgálatot. A `$entry_sp` GDB-kényelmi változó, nem A64-regiszter.

```
# Host: gdb-multiarch; target: az előző qemu-aarch64 folyamat.
gdb-multiarch -q ./lab-stack \
    -ex 'target remote :1234' \
    -ex 'break lab_call' \
    -ex 'continue' \
    -ex 'set $entry_sp = $sp' \
    -ex 'break lab_frame_ready' \
    -ex 'continue' \
    -ex 'p/d $entry_sp - $sp' \
    -ex 'p/d ((unsigned long)$sp & 15)' \
    -ex 'x/4gx $sp' \
    -ex 'p/x &lab_after_call' \
    -ex 'break lab_after_child' \
    -ex 'continue' \
    -ex 'p/d $x0' \
    -ex 'p/d *(unsigned long *)($sp + 16)' \
    -ex 'break lab_after_call' \
    -ex 'continue' \
    -ex 'p/d $x0' \
    -ex 'p/d $sp == $entry_sp' \
    -ex 'continue' \
    -ex 'quit'
```

A vizsgálat nem feltételez rögzített numerikus veremcímet: a tényleges QEMU-folyamat regisztertérértékéből indul ki.

Megállási pont	Mért állapot (GDB 15.1, QEMU 8.2.2)
`lab_frame_ready`	`$entry_sp - $sp` értéke 32
`lab_frame_ready`	Az `SP` alsó négy bitjének értéke 0
`lab_frame_ready`	A négy 8 bájtos mező: 0; `lab_after_call` címe (a mért futásban `0x4000bc`); 7; 0 (a kitöltés tartalma nem meghatározott)
`lab_after_child`	`X0 = 12`; a lokális tároló értéke továbbra is 7
`lab_after_call`	`X0 = 19`; `$sp == $entry_sp` értéke 1

A mentett visszatérési cím az eredeti hívó folytatására, a `lab_after_call` címkére mutat. A belső hívás után az aktuális `X30` már a `lab_after_child` címet tartalmazza. Ezért szükséges a mentett érték visszatöltése a végső `RET` előtt.

8.1.7 Tipikus hibák és ellenpróbák

Helyes eredmény, hibás visszaállítás. Ha a foglalás 32 bájt, de a felszabadítás csak 16, a számítás még lehet helyes. A hívó azonban rossz `SP`-t kap vissza. Tünetből kiindulva ellenőrizzük a belépési és kilépési érték különbségét, majd az `objdump` kimenetében a két immediate mezőt.

Elveszett visszatérési cím. Ha kimarad az `X30` mentése vagy visszatöltése, a végső `RET` a belső hívás utáni helyre térhet vissza. Ez ismételt keretbontáshoz, hibás memóriaeléréshez vagy nem várt vezérléshez vezethet. Ellenpróba: megállás a `RET` előtt, majd az `X30` összevetése az elvárt hívói folytatással.

24 bájtos keret. Az adatméret helyes, az igazítás hibás. Egy QEMU-futtatás látszólagos sikere nem igazolja az ABI-megfelelést. Az alsó négy bit explicit ellenőrzése döntő.

A régi keretmutató túl korai felülírása. Ha a `MOV X29, SP` megelőzi a régi `X29` mentését, az előző keret címe elvész. A mentett mezőt a debuggerben kell megvizsgálni, nem csupán a visszatérési értéket.

A kitöltés nullának feltételezése. A keretfoglalás nem inicializál. A padding tartalma nem programeredmény, és összehasonlításban nem szabad meghatározott értéket várni tőle.

A javítási munkamenet minden esetben: **hipotézis → regiszter- vagy memóriaadat gyűjtése → egyetlen célzott módosítás → újbóli ellenőrzés**.

8.1.8 Feladatok

1. **Magyarázó feladat (alapszint — cél: megértés-ellenőrzés).** Miért kivonással foglalunk keretet? Miért nem elég a példában 24 bájt? Magyarázd el azt is, miért nem függ a verem növekedési iránya a little-endian bájtsorrendtől.
2. **Implementációs feladat (középszint — cél: rutinépítés).** Bővítsd a `lab-stack.s` keretét úgy, hogy a keretrekord mellett három 8 bájtos lokális értéket tároljon: 7-et, 11-et és 13-at. A segédfüggvény továbbra is az eredeti 7-hez adjon 5-öt. Ezután add hozzá mindhárom tárolt értéket. Határozd meg a keretméretet, az eltolásokat és az új elvárt eredményt. A fenti buildlépésekkel ellenőrizd.
3. **Hibakeresési feladat (középszint — cél: rutinépítés).** Az eredeti példában kizárólag a visszaállító `ADD SP, SP, #32` immediate értékét változtasd 16-ra. Miért adhat a program továbbra is 0 kilépési állapotot? Mutasd ki GDB-vel a hibát, majd állítsd vissza az eredeti utasítást.

8.1.9 Megoldások

1. megoldás — Segítség. Gondold át, hol kezdődik a verem szabad része az aktuális `SP`-hez képest, és mi történik, ha a 24 bájtos keretméretet kivonod `S`-ből — teljesül-e utána a 16 bájtos igazítás?

1. megoldás — Teljes megoldás. A lefelé növekvő verem szabad része az aktuális `SP` alatti címeken kezdődik, ezért a foglalás csökkenti a mutatót. A 24 bájtos adattartalomhoz 32 bájtos keret kell, mert a 24 levonása megsértené a 16 bájtos igazítást. A bájtsorrend egy több-bájtos érték bájtjainak elrendezését szabályozza, nem a keretfoglalás irányát.

2. megoldás — Segítség. Számold össze a keretrekord és a három új lokális érték bájtigényét, majd kerekítsd a legközelebbi 16 többszörösére — ez adja a foglalás/visszaállítás immediate-jét és az új eltolásokat.

2. megoldás — Teljes megoldás. A keretrekord 16 bájt, a lokális értékek összesen 24 bájtosak. A 40 bájtos igényt 48 bájtra kerekítjük. A lokális értékek eltolása `+16`, `+24`, `+32`; a `+40` helyen 8 bájt kitöltés marad. A foglaló és visszaállító immediate egyaránt 48. A várható eredmény:

$$(7 + 5) + 7 + 11 + 13 = 43.$$

A kimeneti ellenőrzést ezért 43-ra kell módosítani. A lokális értékek feltöltéséhez és visszaolvasásához az eredeti példával azonos, 8 bájtos skálázású tárolások használhatók.

3. megoldás — Segítség. A program kilépési kódja csak `X0`-t ellenőrzi — gondold át, mi lesz az `SP` és a belőle levezetett `$entry_sp - $sp` értéke, ha a visszaállítás fele akkora, mint a foglalás.

3. megoldás — Teljes megoldás. Visszatéréskor az `SP` értéke $S - 16$, nem S. A program csak az `X0` értékét ellenőrzi, ezért a veremhiba észrevétlen maradhat. A `lab_after_call` címkénél az `$entry_sp - $sp` kifejezés várt értéke a hibás változatban 16, az egyenlőségvizsgálaté 0. A visszaállítás 32-re javítása után a különbségnek 0-nak kell lennie.

8.1.10 Rövid összefoglaló

Az AArch64 Linux-verem lefelé növekszik: a függvény kisebb címre állítja az `SP`-t, majd a lefoglalt tartományban helyezi el megőrzendő adatait. A keret méretét nemcsak az adatok összege, hanem a 16 bájtos igazítás is meghatározza. A `BL` és a `RET` nem kezeli automatikusan ezt a területet. Az `X29` használható stabil keretbázisként, de nem minden függvényben szükséges. A helyes számítás mellett az eredeti veremmutató és visszatérési cím helyreállítását külön is ellenőrizni kell.

- **Veremkeret:** egy függvényaktivációhoz rendelt veremterület.
- **Keretmutató:** a keret kiválasztott, stabil viszonyítási pontját jelölő regiszter.
- **Keretrekord:** az előző keret mutatóját és a mentett visszatérési címet tartalmazó kételemű rekord.

8.1.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- AAPCS64 – Arm hivatalos ABI-dokumentum: veremkorlátok, igazítás, keretrekord és keretmutató. **Ellenőrizve**: aktuális kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-vonatkozású részek Beta).
- Arm A-profile architektúra-dokumentáció: az A64 utasítások és az `SP`-igazításellenőrzés architekturális háttere. **Ellenőrizve**: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv: AArch64-szintaxis, direktívák és összeállítási kapcsolók. **Ellenőrizve**: aktuális kiadás binutils 2.47.
- GNU GDB kézikönyv: távoli hibakeresés, kényelmi változók és memóriavizsgálat. **Ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a közölt kimenet és a GDB-állapotok a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A telepített Ubuntu-csomagok pontos verzióját és a QEMU adott kiadásának igazításihiba-kezelését itt nem ellenőriztük. A szabályosságot ezért nem pusztán a kivétel hiánya, hanem az explicit állapotvizsgálat alapján kell megítélni.

8.2 Az AAPCS64 regiszterszerződés

8.2.1 Miért kell a regiszterekre szerződés?

Egy assembly függvény önmagában helyesen számolhat, mégis hibássá teheti az egész C programot. Elég, ha felülír egy olyan regisztert, amelyben a hívó egy később felhasználandó mutatót

tartott. A fordító nem vizsgálja meg minden külső függvény belsejét: előre rögzített szabályokra támaszkodik.

Az **eljáráshívási szabvány** (*Procedure Call Standard*, PCS) meghatározza az argumentumok és eredmények átadását, valamint a hívásokon át megőrzendő gépállapotot. A könyv AArch64-részében ezt az **AAPCS64** rögzíti. A tágabb **bináris alkalmazásinterfész** (*Application Binary Interface*, ABI) ezen túl például objektumformátumokat és adatábrázolást is szabályoz.

Egy függvény helyességéhez tehát két eredmény szükséges: a megfelelő visszatérési érték és a hívási szerződés betartása.

Az alfejezet az A64 ISA, AArch64 Linux, little-endian, LP64 környezetére vonatkozik: a mutató és a `long` 64 bites. Az alap AAPCS64-et használjuk, SVE/SME-kiterjesztések nélkül. Előfeltétel a veremkeret működésének ismerete. Az összetett típusok részletes átadása későbbi téma.

8.2.2 A megőrzés két felelőse

A **hívó által megőrzendő regiszter** (*caller-saved register*) tartalmát a meghívott függvény szabadon felülírhatja. Ha a hívónak a régi érték a hívás után is kell, még a hívás előtt biztonságba kell helyeznie.

Ez nem jelenti minden ilyen regiszter feltétlen mentését. Csak a hívás után még szükséges, vagyis **élő értékeket** (*live values*) kell megóvni.

A **hívott által megőrzendő regiszter** (*callee-saved register*) belépéskori tartalmát a meghívott függvénynek visszatéréskor helyre kell állítania, ha módosította. A függvény közben használhatja munkaregiszterként; a szerződés a hívó számára látható végeredményt szabályozza.

Az alábbi táblázat minden regisztere **A64/AArch64, I. rész** kontextusú.

Regiszter	Szerep az alap AAPCS64-ben	Mire számíthat a hívó?
`X0`–`X7`, illetve `W0`–`W7`	Egész és mutató típusú argumentumok; eredményátadás	A régi tartalom elveszhet
`X8`	Közvetett eredményátadás helyének címe, amikor ez szükséges	Nem általános kilencedik argumentumregiszter; nem megőrzött
`X9`–`X15`	Általános ideiglenes regiszterek	Caller-saved
`X16`, `X17`	`IP0`, `IP1`: hívásközvetítő kód és linker által is használható ideiglenes regiszterek	Caller-saved; híváson át nem őriznek értéket
`X18`	Platformregiszter, ha a platform igényli; egyébként ideiglenes regiszter lehet	Hordozható kézi assemblyben kerüljük
`X19`–`X28`	Megőrzött általános célú regiszterek	Callee-saved
`X29`	Keretmutató, `FP`	Callee-saved
`X30`	Visszatérési cím, `LR`	Nem általános megőrzött munkaregiszter
`SP`	Veremmutató	Visszatéréskor a belépéskori értéket kell visszaállítani

Az A64 `X19`–`X29` regiszterek teljes 64 bites értékét kell megőrizni. Az A64 `W19` írása ezért nem „csak az alsó felét érinti” a szerződés szempontjából: az írás az `X19` felső 32 bitjét nullázza, tehát a teljes megőrzendő értéket megváltoztatja.

Az A64 `NZCV` állapotjelzők megőrzésére sem építhetünk híváson át. Egy hívás előtti összehasonlítás eredményét nem biztonságos a visszatérés után feltételes elágazással felhasználni.

8.2.2.1 Mit tesz a hardver, és mit ír elő az ABI?

Az A64 `BL` utasítás beírja a következő utasítás címét az `X30` regiszterbe, majd átadja a vezérlést. Nem menti automatikusan a munkaregisztereket.

A caller-saved besorolás nem azt jelenti, hogy maga az A64 `BL` minden ilyen regisztert felülír. Azt jelenti, hogy a meghívott kódnak nem kötelessége megőriznie őket. Egy adott függvény véletlenül érintetlenül hagyhatja például az A64 `X9`-et, de erre a hívó nem alapozhat.

Az újabb hívást végző függvénynek az eredeti visszatérési címét is meg kell óvnia: a következő A64 `BL` felülírja az `X30`-at. Az alapértelmezett A64 `RET` az `X30` címére tér vissza. A rendszeres mentési mintákat a prológus és epilógus tárgyalja.

Az A64 `X29` keretmutatóként való használatát a keretláncra vonatkozó platformszabályok is befolyásolják. Ha a függvény módosítja, akkor is meg kell őriznie a belépéskori értékét. Az A64 `SP`-re a visszaállításon túl a korábban tanult 16 bájtos igazítási követelmények is érvényesek.

8.2.3 Argumentumok és visszatérési értékek

Egyszerű egész és mutató típusú paramétereknél az első nyolc argumentum sorrendben az A64 `X0`–`X7` regisztercsaládba kerül. Az operandusszélességet a típus határozza meg.

C-függvényszignatúra	Argumentumok, A64/AArch64	Eredmény, A64/AArch64
`uint64_t f(uint64_t a, uint64_t b)`	`X0 = a, X1 = b`	`X0`
`uint32_t g(uint32_t a, uint32_t b)`	`W0 = a, W1 = b`	`W0`
`uint64_t sum(const uint32_t *p, uint64_t n)`	`X0 = p, X1 = n`	`X0`
`void *advance(void *p, uint32_t n)`	`X0 = p, W1 = n`	`X0`

Az A64 `W0` és `X0` nem két külön regiszter: ugyanannak a regiszternek 32, illetve 64 bites nézete. Egy `uint32_t` argumentumhoz a releváns 32 bitet használjuk. A szerződés által nem meghatározott felső bitekből nem következtethetünk az értékre.

Hasonlóképpen, egy `int32_t` argumentum 64 bites előjeles feldolgozásához tudatos előjelkiterjesztés szükséges; nem feltételezhetjük, hogy az A64 `X0` felső fele eleve megfelelő.

Az A64 `X0` egyszerre lehet argumentum- és eredményregiszter. A hívó tehát nem várhatja, hogy az első argumentum eredeti értéke a visszatérés után is ott legyen.

Ez a „paraméterenként egy regiszter” szabály nem általánosítható változtatás nélkül 128 bites egészekre vagy struktúrákra. Az A64 `X0`–`X1` többregiszteres eredményátadásban is részt vehet; az A64 `X8` közvetett eredményátadási szerepét a struktúra-visszaadás szerződése bontja ki.

8.2.3.1 Nyolcnál több egész argumentum

Kilenc, egyenként `uint64_t` típusú paraméternél:

- az első nyolc az A64 `X0`–`X7` regiszterekbe kerül;
- a kilencedik a hívott függvény **belépéskori** veremmutatójánál található;
- egy tizedik ugyanilyen argumentum ehhez képest 8 bájttal magasabb címen lenne.

A hívó foglalja le az argumentumterületet, a szükséges igazítással együtt. Egyetlen további 8 bájtos argumentumhoz a legegyszerűbb hívóoldali foglalás 16 bájt, mert az A64 `SP`-nek a hívási határon igazítottnak kell maradnia.

Ha a hívott ezután 32 bájtos saját keretet foglal, a kilencedik argumentum az új A64 `SP`-hez képest már 32 bájtos eltolásnál érhető el. Ezért fontos megkülönböztetni a belépéskori és az aktuális veremmutatót.

8.2.3.2 Előzetes: lebegőpontos és NEON-argumentumok

Az alap AAPCS64 külön regisztersorozatot használ a lebegőpontos és megfelelő SIMD-argumentumokhoz.

Regisztercsoport, A64/AArch64	Alapszabály
`V0`–`V7`	Lebegőpontos/SIMD argumentumok és eredmények; caller-saved
`V8`–`V15`	Csak az alsó 64 bit megőrzése kötelező
`V16`–`V31`	Caller-saved

Az A64 `Sn` az adott `Vn` alsó 32 bites, a `Dn` az alsó 64 bites nézete. Például a `double mix(uint32_t n, double x, const void *p, float y)` argumentumai:

- `n`: A64 `W0`;
- `x`: A64 `D0`;
- `p`: A64 `X1`;
- `y`: A64 `S1`;
- visszatérési érték: A64 `D0`.

A két argumentumregiszter-sorozat kiosztása itt egymástól független. Emiatt a kilencedik **összesített** paraméter még nem feltétlenül kerül verembe.

Fontos előzetes korlát: egy teljes, 128 bites A64 `V8` érték nem tekinthető megőrzöttnek. Részletes használatuk a lebegőpontos regiszterkészletnél következik.

8.2.4 Kidolgozott példa: mindkét megőrzési felelősség egy hívásban

Munkakönyvtár: `~/arm-labor/a64/a028/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a028
cd ~/arm-labor/a64/a028
```

A feladat a `uint64_t lab_score(uint64_t a, uint64_t b)` megvalósítása. A függvény meghívja a `uint64_t lab_twice(uint64_t a)` segédfüggvényt, majd kiszámítja:

$$\mathrm{score}(a, b) = (2a + a + b) \bmod 2^{64}.$$

Itt a, b és az eredmény mértékegység nélküli, előjel nélküli 64 bites egész érték. A moduloaritmetika a `uint64_t` C-szemantikájával egyezik.

Két megőrzési módszert szemléltetünk:

- `b` az A64 `X19`-be kerül. A segédfüggvény köteles ezt megőrizni, viszont a `lab_score` köteles helyreállítani saját hívójának eredeti `X19` értékét.
- `a` másolatát az A64 `X9` tartalmazza. Mivel ez caller-saved, a `lab_score` a segédhívás idejére külön veremhelyre menti.

Ez oktatási választás, nem minimális utasításszámú optimalizálás.

Mentsük a következő teljes programot `lab-aapcs.s` néven. A blokk minden utasítása és regisztere **A64 ISA**, minden direktívája **GNU assembler, AArch64 ELF** kontextusú. A kisbetűs `.s` kiterjesztés szándékos: a fájlt preprocesszor nélküli `as`-sel állítjuk elő, a nagybetűs `.S` a GCC előfordítóját igényelné (vö. Fejlesztői környezet és labor).

```
// A64 ISA, AArch64 Linux, little-endian.
// Direktívák: GNU assembler, AArch64 ELF.

.section .text
.p2align 2
.global _start
.type _start, %function
_start:
    mov     x19, #99
    mov     x20, SP
    mov     x29, XZR

    mov     x0, #7
    mov     x1, #11
    bl      lab_score

lab_return:
    cmp     x0, #32
    b.ne    .Lfail
    cmp     x19, #99
    b.ne    .Lfail
    mov     x10, SP
    cmp     x10, x20
    b.ne    .Lfail
    cbnz    x29, .Lfail

    mov     x0, #0
    b       .Lexit
.Lfail:
    mov     x0, #1
.Lexit:
    mov     x8, #93
    svc     #0
.size _start, .-_start

.p2align 2
.global lab_score
.type lab_score, %function
```

```
lab_score:
    .cfi_startproc
    stp     x29, x30, [SP, #-32]!
    .cfi_def_cfa_offset 32
    .cfi_offset 29, -32
    .cfi_offset 30, -24
    mov     x29, SP
    str     x19, [SP, #16]
    .cfi_offset 19, -16

    mov     x19, x1
    mov     x9, x0
    str     x9, [SP, #24]

    bl      lab_twice
lab_after_call:
    ldr     x9, [SP, #24]
    add     x0, x0, x9
    add     x0, x0, x19

    ldr     x19, [SP, #16]
    .cfi_restore 19
    ldp     x29, x30, [SP], #32
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.size lab_score, .-lab_score

.p2align 2
.global lab_twice
.type lab_twice, %function
lab_twice:
    .cfi_startproc
    mov     x9, #0x55
    add     x0, x0, x0
    ret
    .cfi_endproc
.size lab_twice, .-lab_twice

.section .note.GNU-stack,"",%progbits
```

A `.cfi_*` direktívák a hibakeresőnek írják le a keretet; jelentésüket és a könyv CFI-szabályát a Prológus és epilógus alfejezet tárgyalja. A `lab_twice` szándékosan felülírja az A64 `X9`-et. Ez teljesen szabályos: így a labor nem egy véletlenül együttműködő segédfüggvényre támaszkodik.

Az `_start` a Linux folyamatbelépési pontja, nem szokásos, visszatérő C-függvény. A végén az A64 `X8 = 93`, `X0 = státusz`, `SVC #0` a korábban tanult Linux syscall-konvenciót használja, nem az AAPCS64 függvényhívási mechanizmusát.

8.2.4.1 A keret és az eredmény levezetése

A `lab_score` belsejében, a foglalás utáni A64 `SP`-hez képest:

Eltolás	Méret	Tartalom
0 bájt	8 bájt	A hívó A64 `X29` értéke
8 bájt	8 bájt	Mentett A64 `X30`, a visszatérési cím
16 bájt	8 bájt	A hívó A64 `X19` értéke: 99

Eltolás	Méret	Tartalom
24 bájt	8 bájt	A segédhíváson át megőrzött `a`: 7

A teljes keret 32 bájt, tehát megtartja a 16 bájtos igazítást.

A levezetett állapotváltozások (a `lab_after_call` sort a lektor mérése is igazolta):

Megfigyelési pont	A64 `X0`	A64 `X9`	A64 `X19`
A segédhívás előtt	7	7	11
`lab_after_call` címkénél	14	85	11
Az A64 `X9` visszatöltése után	14	7	11
Az összeadások után	32	7	11
`lab_return` címkénél	32	7	99

Ezek **levezetett értékek**; a `lab_after_call` sor értékeit a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 mérése igazolta: `X0=14`, `X9=0x55`, `X19=11`.

8.2.4.2 A példában használt immediate-korlátok

A példa minden konstansa és eltolása belefér a megfelelő A64-mezőbe; a korlátokat az A64 immediate-korlátok összefoglalója sorolja fel. A keretkezelés szempontjából a `STP`/`LDP` számít: eltolása 8 bájttal skálázott, `-512…504` bájt.

A GNU assembler AArch64 ELF `.p2align 2` direktívája 4 bájtos kódigazítást kér; ez nem végrehajtott utasítás.

8.2.5 Összeállítás, futtatás és ellenőrzés

Target: qemu-user, AArch64 Linux. Ubuntu 24.04-es laborhoston, a telepített kereszt-eszközlánccal:

```
# Target: qemu-user, AArch64 Linux; nem igényel libc-t.
aarch64-linux-gnu-as -EL -march=armv8-a -g \
    -o lab-aapcs.o lab-aapcs.s
aarch64-linux-gnu-ld -e _start \
    -o lab-aapcs lab-aapcs.o

aarch64-linux-gnu-objdump -d lab-aapcs
qemu-aarch64 ./lab-aapcs
printf 'kilepesi_statusz=%d\n' "$?"
```

Mért kimenet (mérve: GNU as 2.42, GNU ld 2.42, QEMU 8.2.2): `kilepesi_statusz=0`. Maga a program nem ír szöveget. A vizsgált eredmény vagy megőrzési feltétel hibájánál a várt státusz 1.

A teszt ellenőrzi az eredményt, az A64 `X19`, `X29` és `SP` visszaállítását. Ez célzott ellenőrzés, nem minden lehetséges ABI-sértés teljes tesztje.

A köztes állapot vizsgálatához az első terminálban:

```
# Target: qemu-user, AArch64; várakozás a debuggerre.
qemu-aarch64 -g 1234 ./lab-aapcs
```

A második terminálban:

```
# Debugger-target: az előző qemu-user folyamat.
gdb-multiarch -q ./lab-aapcs \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_after_call' \
    -ex 'continue' \
    -ex 'info registers x0 x9 x19 x29 x30 sp' \
    -ex 'x/4gx $sp'
```

A GDB parancsában a kisbetűs nevek ugyanazokat az A64 regisztereket jelölik. A töréspont az A64 `X9` visszatöltése **előtt** áll meg. A négy veremszó mért tartalma sorrendben: 0, a `lab_return` szimbolikus címe, 99, 7. Rögzített abszolút címet nem feltételezünk. A CFI-nek köszönhetően a `bt` parancs a mért futásban helyesen mutatta a `_start` hívót. A futás GDB-ben a `continue` paranccsal fejezhető be.

8.2.6 Tipikus hibák és célzott ellenpróbák

Rossz eredmény, ép verem. Ha kimarad az A64 `X9` visszatöltése, az eredmény:

$$14 + 85 + 11 = 110.$$

Hipotézis: egy caller-saved értéket változatlannak tekintettünk. Adatgyűjtés: töréspont a `lab_after_call` címkénél, az A64 `X9` és a mentési hely összevetése. Egyetlen javítás: a hiányzó visszatöltés helyreállítása. Ellenőrzés: az eredmény újra 32, a kilépési státusz 0.

Jó eredmény, hibás hívóállapot. Az A64 `X19` visszatöltésének elhagyásakor az eredmény továbbra is 32 lehet, de a hívó 99 helyett 11-et kap vissza a regiszterben. Ezért nem elegendő kizárólag az eredményregisztert tesztelni.

Hibás visszatérési út. Ha egy újabb hívást végző függvény nem óvja meg az A64 `X30`-at, a `RET` rossz helyre ugorhat. Ilyenkor a visszatérés előtti regiszterállapotot és az objdumpban látható hívási helyeket kell összevetni.

32 bites írás megőrzött regiszterbe. Az A64 `W19` módosítása is megköveteli az eredeti teljes `X19` megőrzését. Ellenpróbához olyan hívóoldali értéket válasszunk, amelynek felső 32 bitje nem nulla.

8.2.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Adjuk meg a `double evaluate(uint64_t count, double scale, const void *base, float bias)` argumentum- és eredményregisztereit. Miért nem az A64 `X2` hordozza a `base` értékét?

2. (implementációs, középszint — cél: rutinépítés) Készítsünk `lab-aapcs-x20.s` változatot, amely az `a` értéket caller-saved regiszter és külön mentés helyett az A64 `X20`-ban tartja a segédhíváson át. A keret maradjon 32 bájtos.

3. (hibakeresési, középszint — cél: rutinépítés) Készítsünk `lab-aapcs-bug.s` másolatot, és csak az A64 `X9` hívás utáni visszatöltését hagyjuk el. Jósoljuk meg az eredményt, majd ellenőrizzük GDB-vel. Ezután külön kísérletben csak az A64 `X19` visszatöltését hagyjuk el.

8.2.8 Megoldások

1. megoldás — Segítség. Vedd sorra az argumentumokat típus szerint: az egész/mutató és a lebegőpontos paraméterek külön regisztercsaládot fogyasztanak, egymástól függetlenül sorszámozva.

1. megoldás — Teljes megoldás. A kiosztás: A64 `X0 = count`, `D0 = scale`, `X1 = base`, `S1 = bias`; az eredmény A64 `D0`. A lebegőpontos argumentumok nem fogyasztanak általános argumentumregisztert ebben az egyszerű esetben. A `scale` ezért nem foglalja el az A64 `X1`-et.

2. megoldás — Segítség. Az A64 `X20` callee-saved — gondold át, hol kell menteni és hol visszaállítani ahhoz, hogy a hívó ne vegyen észre változást, és hogy a segédhívás se írja felül idő előtt.

2. megoldás — Teljes megoldás. A keret 16 és 24 bájtos helyére az eredeti A64 `X19` és `X20` kerüljön. A segédhívás előtt `X19 = b`, `X20 = a`; utána az eredményhez ezeket adjuk hozzá. Mindkét regisztert vissza kell állítani. Az eredeti teszt ezt különösen jól ellenőrzi: az `_start` az A64 `X20`-ban őrzi az eredeti veremmutatót.

Target: qemu-user, AArch64 Linux; várt státusz: 0.

```
aarch64-linux-gnu-as -EL -march=armv8-a -g \
    -o lab-aapcs-x20.o lab-aapcs-x20.s
aarch64-linux-gnu-ld -e _start \
    -o lab-aapcs-x20 lab-aapcs-x20.o
qemu-aarch64 ./lab-aapcs-x20
printf 'kilepesi_statusz=%d\n' "$?"
```

3. megoldás — Segítség. Az A64 `X9` caller-saved (nincs megőrzési kötelezettség rá), az A64 `X19` callee-saved (van) — a két hiba más-más szerződést sért, és más tünetet ad.

3. megoldás — Teljes megoldás. Az első változat mért eredménye 110, kilépési státusza 1 (mérve: GNU as 2.42, QEMU 8.2.2). A másodiké levezetve 32, de az A64 `X19 = 11` miatt szintén 1. A két hibát külön kell vizsgálni, hogy az egyik ellenőrzés ne fedje el a másikat.

Target: qemu-user, AArch64 Linux; a szándékosan hibás változat mért státusza: 1.

```
aarch64-linux-gnu-as -EL -march=armv8-a -g \
    -o lab-aapcs-bug.o lab-aapcs-bug.s
aarch64-linux-gnu-ld -e _start \
    -o lab-aapcs-bug lab-aapcs-bug.o
qemu-aarch64 ./lab-aapcs-bug
printf 'kilepesi_statusz=%d\n' "$?"
```

8.2.9 Rövid összefoglaló

Az AAPCS64 a külön fordított függvények közös regiszterszerződése. A64-ben az egyszerű egész és mutató argumentumok az `X0`–`X7` regisztercsaládot használják; az egyszerű egész eredmény az `X0` megfelelő szélességű nézetében érkezik. A hívó a még szükséges ideiglenes értékeket védi, a hívott pedig a módosított megőrzött regisztereket állítja helyre. A visszatérési cím és a veremmutató kezelése ettől külön figyelmet igényel. A helyes eredmény önmagában nem igazolja az ABI-helyességet. A lebegőpontos argumentumok külön regisztersorozatot kapnak.

- **AAPCS64:** az AArch64 eljáráshívások alapvető bináris szerződése.
- **Caller-saved:** a hívó felelőssége megőrizni a hívás után is szükséges tartalmat.
- **Callee-saved:** módosítás esetén a hívott köteles visszaállítani a belépéskori tartalmat.

8.2.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- AAPCS64 – Arm hivatalos ABI-dokumentum: regiszterszerepek, megőrzési szabályok, argumentum- és eredményátadás. *Ellenőrizve*: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta) — ez az alfejezet szerződésének elsődleges, közvetlen forrása.
- Arm A-profile architektúra-dokumentáció: az A64 utasítások és regiszternézetek architekturális háttere. *Ellenőrizve*: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv: AArch64-szintaxis, direktívák és utasításkódolási korlátok. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv: távoli kapcsolat, töréspontok, regiszter- és memóriavizsgálat. *Ellenőrizve*: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: az alapváltozat kimenete, regiszterállapota és a CFI `bt`-viselkedése a lektor GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származik (`mérve`). Az Ubuntu 24.04-es laborban telepített eszközverziókat és a konkrét futást a hallgatónak kell rögzítenie; a 2. feladat `X20`-as és a 3. feladat `X19`-et elhagyó változata levezetett, nem mért eredmény. Az ABI-dokumentum mozgó `main` változata helyett auditálható projektben konkrét kiadást vagy commitot célszerű megjelölni (fentebb a 2025Q4-es kiadás a jelenlegi referencia).

8.3 Prológus és epilógus: regisztermentés és keretkezelés

8.3.1 Miért kell a függvénynek belépési és kilépési rend?

Egy assemblyben megírt függvény helyes eredményt számolhat, mégis elronthatja a hívóját: felülírhat egy megőrzendő regisztert, elveszítheti a visszatérési címet, vagy rossz helyen hagyhatja a veremmutatót. Ezek a hibák gyakran csak a következő függvényhíváskor jelentkeznek.

A **prológus** (*function prologue*) a függvény belépési műveletsora: létrehozza a szükséges veremkeretet, és elmenti a később visszaállítandó állapotot. Az **epilógus** (*function epilogue*) helyreállítja ezt az állapotot, felszabadítja a keretet, majd visszatér.

A **nem-levél függvény** (*non-leaf function*) másik függvényt is hív. A64-ben a `BL` felülírja az `X30` kapcsolatregisztert, vagyis az LR-t (*link register*). Ezért a hagyományos, hívás után tovább dolgozó nem-levél függvénynek meg kell őriznie saját visszatérési címét.

Az előfeltétel Az AAPCS64 regiszterszerződés ismerete. Itt nem ismételjük meg a teljes regisztertáblázatot: azt mutatjuk meg, hogyan lesz a szerződésből működő mentési és visszaállítási kód.

Architektúrajelölés: az alfejezet minden utasítása és regisztere az **A64 ISA, AArch64, I. rész** kontextusához tartozik. A direktívák GNU assembler-, AArch64 ELF-környezetben értendők. A labor little-endian profilt használ; Cortex-M-kód nem szerepel benne.

8.3.2 Páros mentés és visszaállítás: STP és LDP

Az A64 `STP` (*store pair*) két regisztert tárol egymást követő memóriahelyekre. Az A64 `LDP` (*load pair*) két regisztert tölt vissza. Két 64 bites regiszternél összesen 16 bájt adat mozog.

A következő részlet 32 bájtos keretet foglal:

```
// A64 ISA, AArch64; GNU as szintaxis.
// Szemléltető részlet, a teljes program lejjebb következik.
stp x29, x30, [SP, #-32]!
mov x29, SP
```

A `!` előindexelt, visszaíró címzést (*pre-indexed addressing with writeback*) jelöl. Ha a belépési veremmutató értékét `S` jelöli:

1. az új veremmutató `S - 32`;
2. az `X29` korábbi értéke az új veremmutatóhoz képest 0 bájtos eltolásra kerül;
3. az `X30` korábbi értéke 8 bájtos eltolásra kerül;
4. az `X29` felveszi az új keret alapcímét.

Az `S` itt szimbolikus cím, nem konkrét memóriatartomány.

A hozzá tartozó felszabadító részlet:

```
// A64 ISA, AArch64; GNU as szintaxis.
ldp x29, x30, [SP], #32
ret
```

Ez utóindexelt címzés (*post-indexed addressing*): előbb megtörténik a betöltés a régi `SP` alapján, majd a veremmutató 32 bájttal nő. Az operandus nélkül írt A64 `RET` az `X30` által megadott címre tér vissza.

Kódolási korlát: az itt használt, két 64 bites általános célú regisztert mozgató A64 `STP`/`LDP` eltolása előjeles, 7 bites, 8 bájttal skálázott mező. A tartomány −512…504 bájt, 8 bájtos lépéssel. A negatív és pozitív szélsőérték tehát nem szimmetrikus.

A `STP X29, X30, [SP, #-n]!` és `LDP X29, X30, [SP], #n` egylépéses keretkezelő mintában, pozitív és 16-tal osztható `n` mellett, a közös kódolható tartomány 16…496 bájt. Nagyobb kerethez külön veremmutató-módosítás és külön mentési művelet szükséges.

A páros utasítás csökkentheti a mentési kód utasításszámát. Ebből azonban **nem következik garantált ciklusszám vagy kétszeres sebesség**: a mikroarchitekturális végrehajtás külön kérdés.

8.3.3 Mit kell ténylegesen megőrizni?

Az AAPCS64 megőrzési követelménye a megőrzött, azaz *callee-saved* regiszterek **módosítására** vonatkozik.

- Ha csak olvasod az A64 `X19` regisztert, emiatt nem szükséges mentened.
- Ha munkaregiszterként felülírod, a belépési értékét vissza kell állítanod.
- Ha az A64 `W19` nézetbe írsz, az `X19` felső 32 bitje nullázódik. Ez is a teljes megőrzött regiszter módosítása.
- Ha az A64 `X29` regisztert keretmutatóként használod, a régi értékét meg kell őrizned.
- Az A64 `X30` mentésének oka itt a további `BL` hívás; nem egyszerűen az általános megőrzöttregiszter-lista alkalmazása.

Nem érdemes minden megőrzött regisztert automatikusan elmenteni. Ez felesleges memóriaforgalmat és nagyobb keretet okozna. A szükséges mentések a tényleges regiszterhasználatból következnek.

A labor a szokásos, két 64 bites elemből álló **keretrekordot** (*frame record*) használja: előző keretmutató, majd mentett visszatérési cím. Ez áttekinthető mintázat, de nem állítjuk, hogy minden optimalizált függvénynek pontosan ilyen kerete van. A platform keretlánc-szabályai és a fordító beállításai is számítanak.

A veremigazításról részletesebben A stack frame felépítése és a verem iránya szól. A példában a veremmutató végig 16 bájtra igazított; különösen minden hívási határon és minden veremmutatón keresztüli memóriaeléréskor.

8.3.4 Kidolgozott példa: számítás és ABI-ellenőrzés

Munkakönyvtár: `~/arm-labor/a64/a029/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a029
cd ~/arm-labor/a64/a029
```

A megvalósítandó számítás C-modellje:

```
/* Szemantikai modell; a laborban nem külön fordítási egység. */
#include <stdint.h>

static uint64_t lab_double_model(uint64_t a)
{
    return a + a;
}

uint64_t lab_eval_model(uint64_t a, uint64_t b)
{
    return lab_double_model(a) + a + b;
}
```

Az értékeket előjel nélküli, 64 bites számként kezeljük. Az összeadás modulo 2^{64} értendő; a tesztadatoknál nincs körbefordulás.

Az A64 assemblyváltozat:

- az `X19` regiszterben megőrzi az `a` argumentumot;
- az `X20` regiszterben megőrzi a `b` argumentumot;
- meghív egy segédfüggvényt;
- kiszámítja a `3*a + b` eredményt;
- visszaállítja a hívó regisztereit és veremmutatóját.

A segédfüggvény szándékosan felülírja az A64 `X1` regisztert. Ezt megteheti: a hívó nem támaszkodhat annak megőrzésére.

Mentsd a következő teljes programot `lab-frame.s` néven:

```
// A64 ISA, AArch64, Armv8-A; GNU as, little-endian ELF.
.section .text
.global _start
.global lab_eval

.type _start, %function
_start:
```

```
    // Magas biteket is tartalmazó megőrzési tesztminták.
    MOVZ x19, #0x1234, lsl #48
    MOVZ x20, #0x5678, lsl #48
    mov  x29, XZR
    mov  x21, SP

    mov  x0, #7
    mov  x1, #5
    bl   lab_eval

    cmp  x0, #26
    b.ne .Lbad_value

    mov  x9, SP
    cmp  x9, x21
    b.ne .Lbad_stack

    MOVZ x9, #0x1234, lsl #48
    cmp  x19, x9
    b.ne .Lbad_saved

    MOVZ x9, #0x5678, lsl #48
    cmp  x20, x9
    b.ne .Lbad_saved

    cbnz x29, .Lbad_saved

    mov  x0, #0
    b    .Lexit

.Lbad_value:
    mov  x0, #1
    b    .Lexit
.Lbad_stack:
    mov  x0, #2
    b    .Lexit
.Lbad_saved:
    mov  x0, #3
.Lexit:
    // AArch64 Linux exit rendszerhívás.
    mov  x8, #93
    svc  #0
.size _start, .-_start

.type lab_eval, %function
lab_eval:
    .cfi_startproc
    stp  x29, x30, [SP, #-32]!
    .cfi_def_cfa_offset 32          // CFA = SP + 32 (a belépési SP)
    .cfi_offset 29, -32             // x29 a CFA - 32 címen
    .cfi_offset 30, -24             // x30 a CFA - 24 címen
    mov  x29, SP
    stp  x19, x20, [SP, #16]
    .cfi_offset 19, -16
    .cfi_offset 20, -8

    mov  x19, x0
    mov  x20, x1
    bl   lab_double

    add  x0, x0, x19
    add  x0, x0, x20
```

```
lab_eval_epilogue:
    ldp  x19, x20, [SP, #16]
    .cfi_restore 19
    .cfi_restore 20
    ldp  x29, x30, [SP], #32
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.size lab_eval, .-lab_eval

.type lab_double, %function
lab_double:
    // levélfüggvény: a kezdeti szabály (CFA = SP) végig érvényes
    .cfi_startproc
    add  x0, x0, x0
    mov  x1, #0
    ret
    .cfi_endproc
.size lab_double, .-lab_double

.section .note.GNU-stack,"",%progbits
```

A GNU `.type` és `.size` direktívák függvényszimbólumokat írnak le, nem hajtanak végre mentést. A `.note.GNU-stack` szekció a GNU ELF-eszközláncnak jelzi, hogy a program nem igényel végrehajtható vermet.

A további immediate-ok: a `MOV`, `CMP` és `SVC` konstansai kódolhatók (lásd az A64 immediate-korlátok összefoglalóját); ez a kis pozitív konstansokra igaz, nem általános garancia tetszőleges 64 bites értékre.

8.3.4.1 A keret levezetése

Legyen `S` a `lab_eval` belépési veremmutatója. A prológus után az A64 `SP` és `X29` értéke egyaránt `S - 32`.

Eltolás az új A64 `SP`-hez képest	Méret	Tartalom
0 bájt	8 bájt	A hívó mentett `X29` értéke
8 bájt	8 bájt	A függvény belépési `X30` értéke
16 bájt	8 bájt	A hívó mentett `X19` értéke
24 bájt	8 bájt	A hívó mentett `X20` értéke

A középső `STP` nem módosítja a veremmutatót: nincs sem `!`, sem utóindexelt operandus. A már lefoglalt keret felső felét tölti ki.

A segédfüggvény visszatérésekor az A64 `X0` értéke 14, az `X19` értéke 7, az `X20` értéke 5. A két összeadás eredménye 26. Ezután a munkára használt megőrzött regiszterek visszakapják a **hívó** értékeit, nem a 7 és 5 argumentumot.

8.3.4.2 Assemble–link–run

Host: Ubuntu 24.04 LTS x86_64, a könyv cross-eszközláncával.
Target: AArch64 Linux, little-endian, `qemu-aarch64` user-mode.

```
# Target: AArch64 Linux, qemu-user.
aarch64-linux-gnu-as -EL -g -march=armv8-a \
  -o lab-frame.o lab-frame.s

aarch64-linux-gnu-ld -EL -e _start \
  -o lab-frame lab-frame.o

qemu-aarch64 ./lab-frame
lab_status=$?
printf 'kilepesi_allapot=%s\n' "$lab_status"

aarch64-linux-gnu-objdump -d ./lab-frame
```

Várt, levezetett kimenet: `kilepesi_allapot=0`. A program maga nem ír a terminálra.

Kilépési állapot	Jelentés
0	A beépített eredmény- és megőrzési ellenőrzések sikeresek
1	Hibás számítási eredmény
2	Elcsúszott veremmutató
3	Hibásan visszaállított `X19`, `X20` vagy `X29`

Az A64 `X21` a tesztelő kódban őrzi a belépési veremmutatót; a vizsgált függvény nem módosítja. A magas biteket tartalmazó tesztminták a véletlen 32 bites visszaállítást is kimutathatják.

Ez célzott laborellenőrzés, nem általános ABI-validátor.

8.3.5 CFI: a keret leírása a hibakereső számára

A `lab_eval` listájában a `.cfi_*` sorok nem processzorutasítások, hanem az assemblernek szóló direktívák. Belőlük készül az ELF `.eh_frame` szekciójának **hívási keretinformációja** (*Call Frame Information*, CFI): egy táblázat, amely a függvény minden utasításcímére megmondja, hol van a hívó keretének kezdete (*CFA*, *Canonical Frame Address* — itt a belépési `SP`), és hová mentettük a hívó regisztereit. A GDB `bt` parancsa, a C++-kivételkezelés és a profilozók ebből lépnek vissza a hívóhoz.

Direktíva	Jelentés
`.cfi_startproc` / `.cfi_endproc`	a függvény CFI-leírásának kezdete és vége
`.cfi_def_cfa_offset 32`	ettől az utasítástól CFA = `SP + 32`
`.cfi_offset 30, -24`	a 30-as DWARF-regiszter (`X30`) a CFA − 24 címen van mentve
`.cfi_restore 30`	az `X30` újra a saját regiszterében van

A DWARF-sorszámok AArch64-en: `X0`–`X30` = 0–30, `SP` = 31, `V0`–`V31` (így a `D8`–`D15` is) = 64–95.

CFI nélkül a hibakereső csak a keretláncból (`X29` → mentett `X29` → …) vagy heurisztikából tud visszalépni. Ez a prológus közepén, keretmutató nélküli függvényben vagy sérült veremben megbízhatatlan. **A könyv szabálya ezért a következő:**

- Minden **C-ből hívható, végleges** assembly függvény, amely keretet épít — vagyis módosítja az `SP`-t, vagy menti az `X29`/`X30`/megőrzött regisztereket —, teljes CFI-t kap.

- A keretet **nem** építő levélfüggvénynél a CFI elhagyható: ott a hibakereső alapértelmezése (CFA = `SP`, visszatérési cím az `X30`-ban) a függvény minden pontján helyes. A `.cfi_startproc`/`.cfi_endproc` pár ilyenkor is megengedett, a teljesség kedvéért.
- Ahol egy keretet építő példa szándékosan CFI nélküli (szemléltető részlet, `_start`-tal induló önálló program, biztonsági demonstráció), a szöveg ezt jelöli: az ilyen kód hívási lánca **csak a keretlánc alapján** követhető.

A CFI helyességét nem a futás, hanem a táblázat igazolja:

```
# Hostoldali vizsgálat; a fájl célja AArch64 Linux, qemu-user.
aarch64-linux-gnu-readelf --debug-dump=frames-interp lab-frame
```

A `lab_eval` FDE-jében öt sor várható (mérve: binutils 2.42): a belépéskor `CFA = sp+0`; az első `STP` után `sp+32`, `x29` és `x30` mentett helyével (`c-32`, `c-24`); a második `STP` után `x19`, `x20` is (`c-16`, `c-8`); az epilógus közben, az első `LDP` és a `.cfi_restore 19`/`.cfi_restore 20` után ismét `sp+32`, ahol `x19` és `x20` már `u` (undefined, azaz visszaálltak a saját regiszterükbe); az epilógus végén ismét `sp+0`. Ha a táblázat egy mentést nem mutat, a `bt` az adott ponton rossz hívót jelezhet, a program futása viszont ettől még hibátlannak látszik.

8.3.6 Közös ABI-teszthívó: `lab_abi_call`

Hogyan olvasd ezt a részt. A prológus–epilógus feladatokhoz elég a teszthívó *használatát* ismerned; a belső implementációját (a hosszú `lab-abi-call.S`) nem kell átolvasnod. A segéd verziózott támogatói fájl: a kísérőcsomag `tools/abi/` könyvtárában (`lab-abi-call.h`, `lab-abi-call.S`) is megtalálod, a könyvből is másolható. Szerződése: bemenet `fn` (a vizsgált függvény) és `a0..a5` (hat egész argumentum), kimenet a visszaadott **hibamaszk** (`0` = minden megőrzési szabály teljesült), a hívott `X0`-ja pedig a `*result` kimeneti paraméterben. **Észleli:** az `X19`–`X28`, a `D8`–`D15` megváltozását (a felső biteket is), az `SP` és az `X29` eltérő visszatérési értékét. **Nem észleli:** a hívott memóriaírásait a keretén kívülre, a hibás visszatérési értéket (az függvényspecifikus), az időzítést, és azt, ha a hibás függvény egy másik útvonalon rontana. A minimális, kézzel követhető ellenpróbát lásd lentebb (`lab_demo_good`/rossz példa); a „hogyan működik a teszthívó” részletek opcionálisak.

A `lab-frame.s` saját `_start`-ja csak a `lab_eval` két megőrzött regiszterét ellenőrzi. A könyv további C-ből hívható assembly függvényeihez **közös teszthívót** használunk, amely a teljes AAPCS64 megőrzési szerződést ellenőrzi:

- az `X19`–`X28` és a `D8`–`D15` regiszterbe hívás előtt **magas biteket is tartalmazó** mintát tölt (`0xa5a55a5a0000nnnn`), így egy véletlen `Wn`-írás — amely a felső 32 bitet nullázza — is lebukik;
- a hívás előtti keretcímet (`SP` = `X29`) egy **statikus horgonyba** menti, amelyhez a hívott nem fér hozzá;
- a visszatérés után összeveti a mintákat, valamint az `SP`-t és az `X29`-et **külön-külön** a horgonnyal, és saját keretét csak a horgonyból állítja vissza, a hívott által esetleg elrontott `SP`-ből vagy `X29`-ből soha;
- az eredményt hibamaszkként adja vissza, a hívott függvény `X0`-ját pedig külön kimeneti paraméterben.

A horgony miatt a teszthívó **nem reentráns és nem szálbiztos**: egyszerre csak egy `lab_abi_call` futhat. Ezt nem csak dokumentáljuk, hanem ki is mutatjuk: ha a hívott maga is `lab_abi_call`-t hív, a belső hívás nem indítja el a függvényt, hanem `LAB_ABI_BUSY` hibabitet ad vissza. Ugyanezt a megoldást használja a II. rész Thumb-2-es párja, a `lab_abi_call32` is.

A teszthívó maga is a fenti szabályt követi: teljes prológus–epilógus és CFI. A fájlok ebben a könyvtárban (`~/arm-labor/a64/a029/`) készülnek; a későbbi alfejezetek innen másolják át őket.

A `lab-abi-call.h` tartalma:

```
/* lab-abi-call.h - közös AAPCS64 ABI-teszthívó; cél: AArch64 Linux, LP64. */
#ifndef LAB_ABI_CALL_H
#define LAB_ABI_CALL_H

#include <stdint.h>

/* Tetszőleges, legfeljebb hat egész/mutató argumentumú függvény címe.
 * A hívás az assemblyben történik, ezért a C oldalon csak a címet adjuk át. */
typedef void (*lab_abi_fn)(void);

/* A hibamaszk bitjei; 0 = a hívott betartotta a megőrzési szerződést. */
#define LAB_ABI_X19      (1u << 0)   /* bit 0..9: X19..X28 */
#define LAB_ABI_D8       (1u << 10)  /* bit 10..17: D8..D15 */
#define LAB_ABI_SP       (1u << 18)  /* SP nem a hívás előtti */
#define LAB_ABI_FP       (1u << 19)  /* X29 nem a hívás előtti */
#define LAB_ABI_SP_FP    (LAB_ABI_SP | LAB_ABI_FP)
#define LAB_ABI_BUSY     (1u << 20)  /* újrabelépés: fn nem hívódott meg */

/* Meghívja fn(a0, ..., a5)-öt úgy, hogy előtte X19-X28 és D8-D15 ismert,
 * magas biteket is tartalmazó mintát kap; visszatérés után ellenőrzi
 * a mintákat, valamint SP és X29 értékét. *result = a hívott X0-ja.
 *
 * NEM reentráns és nem szálbiztos: a hívás előtti keretcímet egyetlen
 * statikus horgony őrzi. Egymásba ágyazott hívásnál a belső hívás nem
 * hívja meg fn-t, LAB_ABI_BUSY-t ad vissza, *result = 0. */
uint64_t lab_abi_call(lab_abi_fn fn,
                      uint64_t a0, uint64_t a1, uint64_t a2,
                      uint64_t a3, uint64_t a4, uint64_t a5,
                      uint64_t *result);

#endif
```

A `lab-abi-call.S` tartalma (nagybetűs `.S`: a GCC előbb a C-preprocesszort futtatja rajta):

```
// lab-abi-call.S - A64 ISA, AArch64 Linux; GNU as, AAPCS64.
// Közös ABI-teszthívó: uint64_t lab_abi_call(fn, a0..a5, *result)

// Minta: [63:48] = 0xa5a5, [47:32] = 0x5a5a, [15:0] = a regiszter sorszáma.
// A felső fél nem nulla, így egy W-regiszteres írás is lebukik.
.macro LAB_PATTERN reg, n
    movz    \reg, #\n
    movk    \reg, #0x5a5a, lsl #32
    movk    \reg, #0xa5a5, lsl #48
.endm

// Ha \reg nem egyezik a mintával, a \bit sorszámú bit 1 lesz x10-ben.
.macro LAB_CHECK reg, n, bit
    LAB_PATTERN x12, \n
    cmp     \reg, x12
    cset    x13, ne
    orr     x10, x10, x13, lsl #\bit
.endm

.macro LAB_CHECK_D dreg, n, bit
    fmov    x14, \dreg
    LAB_CHECK x14, \n, \bit
.endm
```

```
    .text
    .p2align 2
    .global lab_abi_call
    .type   lab_abi_call, %function
lab_abi_call:
    .cfi_startproc
    // Keret: x29/x30 (16) + x19-x28 (80) + D8-D15 (64) + result-mutató (8) -> 176
    stp     x29, x30, [sp, #-176]!
    .cfi_def_cfa_offset 176
    .cfi_offset 29, -176
    .cfi_offset 30, -168
    mov     x29, sp
    stp     x19, x20, [sp, #16]
    .cfi_offset 19, -160
    .cfi_offset 20, -152
    stp     x21, x22, [sp, #32]
    .cfi_offset 21, -144
    .cfi_offset 22, -136
    stp     x23, x24, [sp, #48]
    .cfi_offset 23, -128
    .cfi_offset 24, -120
    stp     x25, x26, [sp, #64]
    .cfi_offset 25, -112
    .cfi_offset 26, -104
    stp     x27, x28, [sp, #80]
    .cfi_offset 27, -96
    .cfi_offset 28, -88
    stp     d8,  d9,  [sp, #96]
    .cfi_offset 72, -80           // DWARF 72..79 = D8..D15
    .cfi_offset 73, -72
    stp     d10, d11, [sp, #112]
    .cfi_offset 74, -64
    .cfi_offset 75, -56
    stp     d12, d13, [sp, #128]
    .cfi_offset 76, -48
    .cfi_offset 77, -40
    stp     d14, d15, [sp, #144]
    .cfi_offset 78, -32
    .cfi_offset 79, -24
    str     x7, [sp, #160]        // result-mutató

    // Horgony: a hívás előtti SP (= x29) egy statikus szóba kerül, amelyhez
    // a hívott nem fér hozzá. 0 = szabad; nem nulla = egy hívás már fut.
    adrp    x11, lab_abi_anchor
    add     x11, x11, :lo12:lab_abi_anchor
    ldr     x12, [x11]
    cbnz    x12, 1f               // újrabelépés: fn nem hívható
    mov     x12, sp
    str     x12, [x11]

    // Argumentumok átrendezése: x0 = fn, x1..x6 = a0..a5.
    mov     x9, x0
    mov     x0, x1
    mov     x1, x2
    mov     x2, x3
    mov     x3, x4
    mov     x4, x5
    mov     x5, x6

    // Megőrzendő regiszterek feltöltése mintával.
    LAB_PATTERN x19, 19
    LAB_PATTERN x20, 20
```

```
LAB_PATTERN x21, 21
LAB_PATTERN x22, 22
LAB_PATTERN x23, 23
LAB_PATTERN x24, 24
LAB_PATTERN x25, 25
LAB_PATTERN x26, 26
LAB_PATTERN x27, 27
LAB_PATTERN x28, 28
LAB_PATTERN x10, 0x108
fmov    d8, x10
LAB_PATTERN x10, 0x109
fmov    d9, x10
LAB_PATTERN x10, 0x10a
fmov    d10, x10
LAB_PATTERN x10, 0x10b
fmov    d11, x10
LAB_PATTERN x10, 0x10c
fmov    d12, x10
LAB_PATTERN x10, 0x10d
fmov    d13, x10
LAB_PATTERN x10, 0x10e
fmov    d14, x10
LAB_PATTERN x10, 0x10f
fmov    d15, x10

blr     x9

// Ellenőrzés; x10 = hibamaszk. Az SP-t és az x29-et külön-külön a
// horgonyhoz hasonlítjuk, nem egymáshoz: azonos eltolásuk is lebukik.
mov     x10, xzr
adrp    x11, lab_abi_anchor
add     x11, x11, :lo12:lab_abi_anchor
ldr     x12, [x11]            // a hívás előtti keretcím
mov     x13, sp
cmp     x13, x12
cset    x13, ne
orr     x10, x10, x13, lsl #18
cmp     x29, x12
cset    x13, ne
orr     x10, x10, x13, lsl #19
mov     sp, x12               // helyreállítás csak a horgonyból:
mov     x29, x12              // sérült SP/x29 értékét nem használjuk
str     xzr, [x11]            // a horgony újra szabad

LAB_CHECK x19, 19, 0
LAB_CHECK x20, 20, 1
LAB_CHECK x21, 21, 2
LAB_CHECK x22, 22, 3
LAB_CHECK x23, 23, 4
LAB_CHECK x24, 24, 5
LAB_CHECK x25, 25, 6
LAB_CHECK x26, 26, 7
LAB_CHECK x27, 27, 8
LAB_CHECK x28, 28, 9
LAB_CHECK_D d8,  0x108, 10
LAB_CHECK_D d9,  0x109, 11
LAB_CHECK_D d10, 0x10a, 12
LAB_CHECK_D d11, 0x10b, 13
LAB_CHECK_D d12, 0x10c, 14
LAB_CHECK_D d13, 0x10d, 15
LAB_CHECK_D d14, 0x10e, 16
LAB_CHECK_D d15, 0x10f, 17
```

```
    b       2f

1:  mov     x10, #(1 << 20)       // LAB_ABI_BUSY; a horgonyt nem írjuk felül
    mov     x0, xzr

2:  ldr     x7, [sp, #160]
    str     x0, [x7]              // a hívott eredménye
    mov     x0, x10               // visszatérési érték: hibamaszk

    ldp     d14, d15, [sp, #144]
    ldp     d12, d13, [sp, #128]
    ldp     d10, d11, [sp, #112]
    ldp     d8,  d9,  [sp, #96]
    ldp     x27, x28, [sp, #80]
    ldp     x25, x26, [sp, #64]
    ldp     x23, x24, [sp, #48]
    ldp     x21, x22, [sp, #32]
    ldp     x19, x20, [sp, #16]
    .cfi_restore 72
    .cfi_restore 73
    .cfi_restore 74
    .cfi_restore 75
    .cfi_restore 76
    .cfi_restore 77
    .cfi_restore 78
    .cfi_restore 79
    .cfi_restore 19
    .cfi_restore 20
    .cfi_restore 21
    .cfi_restore 22
    .cfi_restore 23
    .cfi_restore 24
    .cfi_restore 25
    .cfi_restore 26
    .cfi_restore 27
    .cfi_restore 28
    ldp     x29, x30, [sp], #176
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
    .size   lab_abi_call, .-lab_abi_call

    .bss
    .p2align 3
lab_abi_anchor:                   // helyi szimbólum: csak a teszthívó látja
    .space  8

    .section .note.GNU-stack,"",%progbits
```

A teszthívó önellenőrzéséhez egy helyes és hat szándékosan hibás hívottat készítünk. Az SP/X29-szerződéshez három külön változat tartozik: csak az SP-t, csak az X29-et, illetve mindkettőt azonos mértékben eltoló hívott. A `lab-abi-demo.S` tartalma:

```
// lab-abi-demo.S - A64 ISA, AArch64; hívottak az ABI-teszthívó kipróbálásához.
    .text
    .p2align 2

// Helyes: x19-et használja, de menti és visszaállítja. Eredmény: a0 + a1 + a2.
    .global lab_demo_good
    .type   lab_demo_good, %function
lab_demo_good:
```

```
    .cfi_startproc
    stp     x29, x30, [sp, #-32]!
    .cfi_def_cfa_offset 32
    .cfi_offset 29, -32
    .cfi_offset 30, -24
    mov     x29, sp
    str     x19, [sp, #16]
    .cfi_offset 19, -16
    add     x19, x0, x1
    add     x0, x19, x2
    ldr     x19, [sp, #16]
    .cfi_restore 19
    ldp     x29, x30, [sp], #32
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
    .size   lab_demo_good, .-lab_demo_good

// HIBÁS: mentés nélkül felülírja x19-et.
    .global lab_demo_bad_x19
    .type   lab_demo_bad_x19, %function
lab_demo_bad_x19:
    .cfi_startproc
    mov     x19, x0
    add     x0, x19, x1
    ret
    .cfi_endproc
    .size   lab_demo_bad_x19, .-lab_demo_bad_x19

// HIBÁS: "ártalmatlannak" látszó W-írás; nullázza x20 felső 32 bitjét.
    .global lab_demo_bad_w20
    .type   lab_demo_bad_w20, %function
lab_demo_bad_w20:
    .cfi_startproc
    mov     w20, w20
    add     x0, x0, x1
    ret
    .cfi_endproc
    .size   lab_demo_bad_w20, .-lab_demo_bad_w20

// HIBÁS: D8 csak az alsó 64 bitjén megőrzendő, itt mégis felülíródik.
    .global lab_demo_bad_d8
    .type   lab_demo_bad_d8, %function
lab_demo_bad_d8:
    .cfi_startproc
    fmov    d8, xzr
    add     x0, x0, x1
    ret
    .cfi_endproc
    .size   lab_demo_bad_d8, .-lab_demo_bad_d8

// HIBÁS: 16 bájttal elállított SP-vel tér vissza.
    .global lab_demo_bad_sp
    .type   lab_demo_bad_sp, %function
lab_demo_bad_sp:
    .cfi_startproc
    sub     sp, sp, #16
    .cfi_def_cfa_offset 16
    add     x0, x0, x1
    ret
    .cfi_endproc
```

```
    .size   lab_demo_bad_sp, .-lab_demo_bad_sp

// HIBÁS: az SP rendben van, de az x29-et 16 bájttal elállítja.
    .global lab_demo_bad_fp
    .type   lab_demo_bad_fp, %function
lab_demo_bad_fp:
    .cfi_startproc
    sub     x29, x29, #16
    add     x0, x0, x1
    ret
    .cfi_endproc
    .size   lab_demo_bad_fp, .-lab_demo_bad_fp

// HIBÁS: az SP-t és az x29-et azonos mértékben tolja el. Az SP == x29
// összehasonlítás itt átmenne; csak a független horgony mutatja ki.
    .global lab_demo_bad_spfp
    .type   lab_demo_bad_spfp, %function
lab_demo_bad_spfp:
    .cfi_startproc
    sub     sp, sp, #16
    .cfi_def_cfa_offset 16
    sub     x29, x29, #16
    add     x0, x0, x1
    ret
    .cfi_endproc
    .size   lab_demo_bad_spfp, .-lab_demo_bad_spfp

    .section .note.GNU-stack,"",%progbits
```

A `lab-abi-test.c` tartalma:

```c
/* lab-abi-test.c - az ABI-teszthívó önellenőrzése; cél: AArch64 Linux. */
#include <inttypes.h>
#include <stdio.h>
#include "lab-abi-call.h"

extern void lab_demo_good(void);
extern void lab_demo_bad_x19(void);
extern void lab_demo_bad_w20(void);
extern void lab_demo_bad_d8(void);
extern void lab_demo_bad_sp(void);
extern void lab_demo_bad_fp(void);
extern void lab_demo_bad_spfp(void);

/* Szerződéstartó C-hívott, amely maga is meghívja a teszthívót: a belső
 * hívásnak LAB_ABI_BUSY-t kell adnia (a horgony foglalt). */
static uint64_t lab_demo_reenter(void)
{
    uint64_t inner = 0;
    return lab_abi_call(lab_demo_good, 1, 2, 3, 0, 0, 0, &inner);
}

static int lab_expect(const char *name, lab_abi_fn fn,
                      uint64_t want_mask, uint64_t want_result)
{
    uint64_t result = 0;
    uint64_t mask = lab_abi_call(fn, 1, 2, 3, 0, 0, 0, &result);
    int ok = (mask == want_mask) && (result == want_result);

    printf("%-18s maszk=0x%05" PRIx64 " eredmeny=%" PRIu64 " %s\n",
           name, mask, result, ok ? "OK" : "HIBA");
    return ok;
}
```

```
int main(void)
{
    int ok = 1;

    ok &= lab_expect("lab_demo_good",    lab_demo_good,    0,             6);
    ok &= lab_expect("lab_demo_bad_x19", lab_demo_bad_x19, LAB_ABI_X19,   3);
    ok &= lab_expect("lab_demo_bad_w20", lab_demo_bad_w20, LAB_ABI_X19 << 1, 3);
    ok &= lab_expect("lab_demo_bad_d8",  lab_demo_bad_d8,  LAB_ABI_D8,    3);
    ok &= lab_expect("lab_demo_bad_sp",  lab_demo_bad_sp,  LAB_ABI_SP,    3);
    ok &= lab_expect("lab_demo_bad_fp",  lab_demo_bad_fp,  LAB_ABI_FP,    3);
    ok &= lab_expect("lab_demo_bad_spfp", lab_demo_bad_spfp, LAB_ABI_SP_FP, 3);
    ok &= lab_expect("lab_demo_reenter", (lab_abi_fn)lab_demo_reenter, 0, LAB_ABI_BUSY);
    ok &= lab_expect("lab_demo_good",    lab_demo_good,    0,             6);
    return ok ? 0 : 1;
}
```

Build és futtatás — target: AArch64 Linux, qemu-user:

```
aarch64-linux-gnu-gcc -std=c11 -O2 -Wall -Wextra -march=armv8-a -static \
    lab-abi-test.c lab-abi-call.S lab-abi-demo.S -o lab-abi-test
qemu-aarch64 ./lab-abi-test
printf 'kilepesi_allapot=%d\n' "$?"
```

Várt kimenet:

```
lab_demo_good      maszk=0x00000 eredmeny=6 OK
lab_demo_bad_x19   maszk=0x00001 eredmeny=3 OK
lab_demo_bad_w20   maszk=0x00002 eredmeny=3 OK
lab_demo_bad_d8    maszk=0x00400 eredmeny=3 OK
lab_demo_bad_sp    maszk=0x40000 eredmeny=3 OK
lab_demo_bad_fp    maszk=0x80000 eredmeny=3 OK
lab_demo_bad_spfp  maszk=0xc0000 eredmeny=3 OK
lab_demo_reenter   maszk=0x00000 eredmeny=1048576 OK
lab_demo_good      maszk=0x00000 eredmeny=6 OK
kilepesi_allapot=0
```

A legtanulságosabb sor a `lab_demo_bad_w20`: a `mov w20, w20` „semmit sem csináló” utasításnak tűnik, és kis értékekkel tesztelve valóban nem okoz eltérést. A 64 bites minta felső fele azonban nullázódik, és a teszthívó ezt kimutatja. Az `SP`/`X29`-sorok azt mutatják, miért kell **független** horgony. Egy korábbi változat az `SP`-t az `X29`-hez hasonlította, és `mov sp, x29`-cel állt vissza. Ez csak addig működik, amíg a hívott az `X29`-et megőrzi. A `lab_demo_bad_spfp` mindkettőt 16 bájttal tolja el, ezért az `SP == X29` vizsgálat átmenne. A `lab_demo_bad_fp` esetén pedig a `mov sp, x29` egy 16 bájttal arrébb fekvő „keretből” töltené vissza a mentett regisztereket és a visszatérési címet. A horgonyos változat mindhárom hibát külön bittel jelzi (`LAB_ABI_SP`, `LAB_ABI_FP`, illetve mindkettő), és a horgonyból mindig ép kerettel tér vissza, így a hiba jelentés lesz, nem összeomlás. A `lab_demo_reenter` sor eredménye `0x100000` = `LAB_ABI_BUSY`. Ez a belső, egymásba ágyazott hívás maszkja, amelyet a C-hívott továbbad. Az utána következő `lab_demo_good` sor azt igazolja, hogy a horgony a külső hívás végén felszabadult.

Korlátok. A teszthívó legfeljebb hat egész vagy mutató argumentumot ad át, a hívott `X0`-ját olvassa vissza, és nem ellenőrzi a lebegőpontos argumentumokat, a veremben átadott argumentumokat, a `V8`–`V15` felső 64 bitjét (ezt az AAPCS64 nem is írja elő megőrzendőnek), sem a memóriabeli mellékhatásokat. Ezeket a hívott függvény saját tesztje vizsgálja. Az `SP`/`X29`-ellenőrzés csak a visszatéréskori **regiszterértékekre** vonatkozik. Ha a hívott a hívó keretét is felülírja (például a saját keretén túl ír, vagy előbb megemeli, majd használja az `SP`-t), a teszthívó a mentett regisztereket már sérült memóriából tölti vissza, és ezt nem ismeri fel. A rossz `X30`-cal visszatérő hívott pedig el sem jut az ellenőrzésig. Ezekre a debugger (`watch` a hívó keretére, illetve töréspont a visszatérési címen) a megfelelő eszköz. Egyszerre csak egy `lab_abi_call`

futhat (statikus horgony); többszálú vagy szignálkezelőből hívott tesztben ez a teszthívó nem használható.

8.3.7 Miért kell szimmetria?

A szimmetria **állapotbeli inverzt** jelent, nem egyszerűen azonos számú mentő és betöltő utasítást.

A keretméretre vonatkozó levezetés:

$$SP_{\text{kilépés}} = S - n_{\text{foglalás}} + n_{\text{felszabadítás}}.$$

Itt `S` bájtcím, a két `n` pedig bájtban megadott keretméret. A hívó veremmutatója akkor áll helyre, ha a foglalás és felszabadítás mérete azonos. Ez szükséges, de önmagában nem elég: a regisztereket is a megfelelő rekeszekből kell visszatölteni.

Különösen alattomos módosítás:

```
// A64 ISA; szándékosan hibás epilógusrészlet.
ldp x19, x20, [SP, #16]
ldp x29, x30, [SP], #16
ret
```

A visszatérési cím helyes helyről töltődik be, ezért a függvény visszatérhet. A veremmutató viszont `S - 16` marad. Még az igazítása is helyes, tehát az igazításellenőrzés nem feltétlenül leplezi le a hibát. A labor várt kilépési állapota ekkor 2.

8.3.8 Hibakeresés: tünettől az ellenőrzött hipotézisig

Tünet	Hipotézis	Ellenőrzés
Az eredmény helyes, később hibázik a program	Elmaradt megőrzöttregiszter-visszaállítás	Belépési és kilépési regiszterértékek összevetése
A visszatérés a függvény belsejébe vezet	A belépési LR elveszett	A64 `X30` és a mentett visszatérési cím vizsgálata
Minden visszatérés után változik a veremhelyzet	Eltérő foglalási és felszabadítási méret	A64 `SP` különbségének kiszámítása
Csak a felső 32 bit hibás	Téves `Wn` művelet	A teljes `Xn` érték és a disassembly ellenőrzése

A következő GDB-menettel közvetlenül az epilógus előtt állunk meg.

Első terminál, target: AArch64 Linux, qemu-user:

```
qemu-aarch64 -g 1234 ./lab-frame
```

A program a debugger kapcsolódására vár.

Második terminál, debugger: hoston futó `gdb-multiarch`, target: qemu-user:

```
gdb-multiarch -q ./lab-frame \
  -ex 'target remote localhost:1234' \
  -ex 'break *lab_eval' \
  -ex 'continue' \
  -ex 'set $lab_sp0 = $sp' \
  -ex 'break *lab_eval_epilogue' \
  -ex 'continue' \
  -ex 'p/d $lab_sp0 - $sp' \
  -ex 'x/4gx $sp' \
  -ex 'si' \
  -ex 'si' \
  -ex 'p/d $sp == $lab_sp0' \
  -ex 'info registers x19 x20 x29 x30 sp' \
  -ex 'continue' \
  -ex 'quit'
```

Várt állapot: az első különbség 32; a két A64 `LDP` után az egyenlőségvizsgálat eredménye 1. A memória négy nyolcbájtos rekesze a fenti táblázatot követi. A mentett visszatérési cím futásfüggő, ezért nem adunk hozzá számszerű címpéldát.

Hibakereséskor egyszerre egy dolgot módosíts: például csak a felszabadítás méretét. Ezután ismételd meg ugyanazt a regiszter- és veremellenőrzést. Ne következtess ABI-helyességre pusztán a számított eredményből.

8.3.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért kell a példában az A64 `X19`, `X20`, `X29` és `X30` értékét megőrizni? Miért nem mentjük az `X0` belépési értékét visszaállítás céljából?

2. (implementációs, középszint — cél: rutinépítés) Alakítsd át a `lab_eval` függvényt úgy, hogy két segédfüggvény-hívással számítsa ki a `2*a + 2*b` értéket. Az első hívás eredményét őrizd meg a második hívás idejére. Maradjon 32 bájtos keret, és frissítsd az eredményellenőrzést.

3. (hibakeresési, középszint — cél: megértés-ellenőrzés) Az eredeti példában változtasd az utolsó A64 `LDP` utóindexét 32-ről 16-ra. Fordítsd újra a korábbi assemble–link–run paranccsal. Jegyezd fel a kilépési állapotot, majd GDB-ben ellenőrizd a veremmutató eltérését. Végül állítsd vissza az eredeti kódot.

8.3.10 Megoldások

1. megoldás — Segítség. Két külön kérdést tegyél fel. (1) Melyik regiszterek **callee-saved**-ek az AAPCS64 szerint, és ezek közül melyiket írja felül a `lab_eval`? Ezeket minden hívó felé vissza kell állítania, akkor is, ha az éppen bemutatott `_start` némelyiküket nem olvassa: a függvény nem ismerheti a jövőbeli hívóit. (2) Mit veszélyeztet a `lab_eval` saját belső `BL`-je? Gondolj az `X30`-ra. A „csak azt, amit a hívó a hívás után még használ” szempont a **hívó** saját döntésére való: a caller-saved regisztereiből (`X0`–`X18`) azt kell a hívás előtt elmentenie, amire a hívás után is szüksége van.

1. megoldás — Teljes megoldás. Az A64 `X19` és `X20` megőrzött regiszter, és a függvény felülírja őket. Az `X29` szintén megőrzendő, és új keretmutatót kap. Az `X30` belépési értékét a belső `BL` felülírná, pedig arra kell majd visszatérni. Az `X0` eredményregiszter: kilépéskor az eredményt, nem a belépési argumentumot kell tartalmaznia.

2. megoldás — Segítség. A két hívás eredményét nem tarthatod egyszerre `X0`-ban — az egyiket a hívások között egy megőrzött regiszterbe kell átmenteni, mielőtt a második `BL` felülírja `X0`-t.

2. megoldás — Teljes megoldás. A teljes `lab_eval` függvény helyére:

```
// A64 ISA, AArch64; a lab-frame.s cserefüggvénye.
.type lab_eval, %function
lab_eval:
    .cfi_startproc
    stp x29, x30, [SP, #-32]!
    .cfi_def_cfa_offset 32
    .cfi_offset 29, -32
    .cfi_offset 30, -24
    mov x29, SP
    stp x19, x20, [SP, #16]
    .cfi_offset 19, -16
    .cfi_offset 20, -8

    mov x19, x1
    bl  lab_double
    mov x20, x0

    mov x0, x19
    bl  lab_double
    add x0, x0, x20

lab_eval_epilogue:
    ldp x19, x20, [SP, #16]
    .cfi_restore 19
    .cfi_restore 20
    ldp x29, x30, [SP], #32
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.size lab_eval, .-lab_eval
```

A cserefüggvény a könyv CFI-szabályát követi: keretet épít, ezért teljes `.cfi_*` leírást kap, ugyanúgy, mint az eredeti `lab_eval`. Az A64 `X19` a második argumentumot, az `X20` az első hívás eredményét őrzi. A tesztben a `CMP X0, #26` helyett `CMP X0, #24` szükséges. Az előző buildparancsokkal újrafordítva a várt kilépési állapot ismét 0.

3. megoldás — Segítség. A veremmutató-eltérés és a visszatérési cím helyessége két külön dolog — gondold végig, hogy a `LDP X29, X30, [SP], #16` melyiket állítja helyre, és melyiket nem.

3. megoldás — Teljes megoldás. A várt kilépési állapot 2. Az epilógus előtti különbség továbbra is 32 bájt; a hibás felszabadítás után 16 bájt marad. A GDB egyenlőségvizsgálata 0-t ad. A visszatérési cím helyreállítása sikerült, a veremkereté nem. Ezért nincs szükség azonnali összeomlásra ahhoz, hogy a kód hibás legyen.

8.3.11 Rövid összefoglaló

A nem-levél A64 függvénynek meg kell őriznie a további hívások által veszélyeztetett visszatérési címét, valamint minden általa módosított megőrzött regiszter belépési értékét. Az `STP` és `LDP` tömör eszközt ad a páros mentéshez, de címzési és immediate-korlátaikat figyelembe kell venni. A

helyes epilógus nemcsak a regisztereket, hanem a belépési veremmutatót is visszaállítja. A szimmetria ezért állapotbeli azonosságot jelent. Egy függvény helyes számítás mellett is megsértheti az ABI-t; ezt célzott regisztermintákkal és veremmutató-ellenőrzéssel vizsgáljuk. A könyv C-ből hívható assembly függvényeit a közös `lab_abi_call` teszthívó ellenőrzi; a végleges függvények CFI-t is kapnak, amelyet a `readelf --debug-dump=frames-interp` táblázata igazol.

- **CFI:** az `.eh_frame` táblázata arról, hol a hívó kerete és hová kerültek a mentett regiszterek.
- **Prológus:** keretfoglaló és állapotmentő belépési műveletsor.
- **Epilógus:** állapot-visszaállító, keretfelszabadító és visszatérő műveletsor.
- **Nem-levél függvény:** végrehajtása során másik függvényt is hívó függvény.

8.3.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- AAPCS64 – Arm hivatalos ABI-dokumentum: regisztermegőrzés, veremigazítás és keret-rekord. **Ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- Arm A-profile architektúra-dokumentáció: az A64 `STP`, `LDP`, `BL` és `RET` architekturális szemantikája. *Ellenőrizve: a link él*; a JavaScript-alapú megjelenítés miatt a pontos tartalom automatikus lekéréssel nem volt kinyerhető.
- GNU assembler kézikönyv: AArch64-szintaxis, célarchitektúra-opciók és ELF-direktívák. **Ellenőrizve**: a jelenlegi kiadás binutils 2.47.

Mérési állapot: a `lab-frame` és a `lab-abi-test` a GNU GCC 13.3.0, statikus glibc és QEMU 8.2.2 fő úton mérve: a `lab-frame` kilépési állapota `0`; a `lab-abi-test` kimenete betűre egyezik a fenti 9 sorral, kilépési állapota `0`; a GDB-menetben a különbség 32, a négy rekesz értéke 0 / visszatérési cím / `0x1234000000000000` / `0x5678000000000000`, két `si` után `$sp == $lab_sp0` = 1, az X19/X20 visszaállt, a `bt` helyes. A horgony szükségességét helyettesítő eszközlánccal (zig cc/Clang + musl, Unicorn-alapú emulátor) is ellenőriztük: a korábbi, `SP == X29` összehasonlítású és `mov sp, x29`-cel visszaálló teszthívó a `lab_demo_bad_fp` és a `lab_demo_bad_spfp` hívottnál a sérült keretből olvasott `result`-mutató miatt szegmentálási hibával (139) állt le, a horgonyos változat mindkettőnél a várt maszkot adta. A CFI a szinkron visszalépést (`bt`, kivételkezelés hívási pontokon) írja le; az epilógus utasításai közötti aszinkron visszafejtés pontosságát külön nem vizsgáltuk.

8.4 Rekurzív függvények assemblyben

8.4.1 Miért igényel külön figyelmet a rekurzió?

Egy fa bejárása vagy egy rekurzív feldolgozó assembly-átiratakor nem elég a matematikai algoritmust követni: minden aktív hívásnak meg kell őriznie a saját folytatásához szükséges állapotot. A hibás regisztermentés rossz eredményhez, a hiányzó megállási feltétel pedig a verem kimerüléséhez vezethet.

A **rekurzív függvény** (recursive function) közvetlenül vagy más függvényeken keresztül önmagát hívja. Egy **aktiváció** (activation) a függvény egy konkrét, még be nem fejezett végrehajtása. Az aktivációk ugyanazt az utasítássorozatot használják, de külön megőrzött állapotra van szükségük.

Ebben az alfejezetben az Armv8-A alapprofil **A64 ISA-ját**, little-endian Linux-környezetet és az AAPCS64 szerződését használjuk. Előfeltétel a Prológus és epilógus: regisztermentés és keretkezelés ismerete. A példában minden aktiváció azonos méretű keretet foglal; dinamikus méretű lokális objektumokat nem használunk.

8.4.2 Rekurzió mint egymásba ágyazott keretek láncolata

A nemnegatív egész argumentumú faktoriális definíciója:

$$f(n) = \begin{cases} 1, & n \leq 1, \\ n \cdot f(n-1), & n > 1. \end{cases}$$

Itt n mértékegység nélküli, előjel nélküli egész. A $n \leq 1$ feltétel a **báziseset** (base case): ez az ág további hívás nélkül állítja elő az eredményt.

Az A64 megvalósításban az argumentum és az eredmény az `X0` regiszterben halad. Az önhívás azonban felülírhatja az `X0` tartalmát, ezért az aktuális n értéket máshol kell megőrizni a visszatérés utáni szorzáshoz.

Az A64 `X19` erre alkalmas, mert az AAPCS64 szerint megőrzendő regiszter. Ez nem jelent automatikus hardveres védelmet: **minden aktivációnak el kell mentenie a belépési `X19` értéket, majd vissza kell állítania azt**.

Ugyanígy minden további hívást végrehajtó aktivációnak meg kell őriznie a saját visszatérési címét. Az A64 `BL` az `X30` regiszterbe írja a folytatás címét, így az újabb önhívás felülírná az előző címet.

A folyamat két szakasza:

1. **Lefelé haladás:** prológus, bázisvizsgálat, újabb hívás.
2. **Visszafejtés:** részeredmény felhasználása, epilógus, visszatérés.

A rekurzióhoz nincs külön „rekurzív utasítás”: közönséges függvényhívások láncolatáról van szó.

8.4.3 Kidolgozott példa: 64 bites faktoriális

Munkakönyvtár: `~/arm-labor/a64/a030/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a030
cd ~/arm-labor/a64/a030
```

Az interfész pontos matematikai eredményt a $0 \leq n \leq 20$ tartományban ígér. A `20!` még elfér egy előjel nélküli, 64 bites egészben; a `21!` már nem.

Az A64 `MUL` az eredmény alsó 64 bitjét adja vissza. Ezért a tartomány túllépése nem automatikus túlcsordulási hibajelzéshez, hanem modulo 2^{64} eredményhez vezet. Az alábbi függvény nem végez külön tartományellenőrzést; a tesztprogram csak megengedett bemeneteket ad át.

8.4.3.1 Assembly-megvalósítás

A fájl neve: `lab-a030-rec.s`. A blokk összes utasítása és regisztere A64-kontextusú. A ponttal kezdődő elemek GNU assembler-direktívák az AArch64 ELF-célhoz, nem processzorutasítások.

```
// A64 ISA, Armv8-A; AArch64 Linux, little-endian.
.arch armv8-a
.text
.p2align 2

.global lab_fact
.type lab_fact, %function
lab_fact:
    .cfi_startproc

    stp     x29, x30, [SP, #-32]!
    .cfi_def_cfa_offset 32
    .cfi_offset 29, -32
    .cfi_offset 30, -24

    mov     x29, SP
    str     x19, [SP, #16]
    .cfi_offset 19, -16

    mov     x19, x0
    cmp     x0, #1
    b.ls    lab_fact_base

    sub     x0, x0, #1
    bl      lab_fact
    mul     x0, x0, x19
    b       .Lfact_return

lab_fact_base:
    mov     x0, #1

.Lfact_return:
    ldr     x19, [SP, #16]
    .cfi_restore 19

    ldp     x29, x30, [SP], #32
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0

    ret
    .cfi_endproc
.size lab_fact, .-lab_fact

.section .note.GNU-stack,"",%progbits
```

Az A64 `CMP` utáni `B.LS` előjel nélküli „kisebb vagy egyenlő” feltételt vizsgál. Előjeles bemenetkezelést ebből nem szabad kikövetkeztetni: például a C-ben negatív értékből `uint64_t`-ra alakított argumentum igen nagy pozitív egész lehet.

Az aktiváció kerete:

Eltolás az aktuális A64 `SP`-hez képest	Méret	Tartalom
`+0`	8 bájt	A hívó mentett `X29` értéke
`+8`	8 bájt	Az aktiváció mentett visszatérési címe, `X30`
`+16`	8 bájt	A belépéskor kapott `X19` érték

Eltolás az aktuális A64 SP-hez képest	Méret	Tartalom
+24	8 bájt	Nem használt igazítási hely

A 32 bájtos keret megőrzi a 16 bájtos veremigazítást. Az aktuális n az A64 `X19` regiszterben él; a verembe mentett `X19` **nem az aktuális** n**, hanem a hívó megőrzendő értéke**.

A GNU `.cfi_*` direktívák hívási keretinformációt (*Call Frame Information*, CFI) írnak le a hibakereső számára. A kanonikus keretcím (*Canonical Frame Address*, CFA) itt a belépési veremmutató. A keretfoglalás után ez az aktuális A64 `SP + 32`; ehhez képest például a mentett `X30` eltolása `-24`. A CFI nem ment regisztert, csak leírja a valódi mentést.

Immediate-ok: a `STP`/`LDP` ±32, a `STR`/`LDR` `#16` eltolása, a `#1` konstansok és a helyi ugráscélok mind kódolhatók (lásd az A64 immediate-korlátok összefoglalóját).

8.4.3.2 C-tesztprogram

A fájl neve: `lab-a030-main.c`.

```c
/* AArch64 Linux célprogram; little-endian profil. */
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

extern uint64_t lab_fact(uint64_t n);

int main(void)
{
    uint64_t result = lab_fact(5);

    printf("5! = %" PRIu64 "\n", result);

    if (result != UINT64_C(120) ||
        lab_fact(0) != UINT64_C(1) ||
        lab_fact(1) != UINT64_C(1) ||
        lab_fact(20) != UINT64_C(2432902008176640000)) {
        fputs("teszthiba\n", stderr);
        return 1;
    }

    puts("tesztek: rendben");
    return 0;
}
```

8.4.3.3 Összeszerelés, linkelés és futtatás

Az Ubuntu 24.04 alapprofilon, **qemu-user célhoz**:

```
# Host: Ubuntu 24.04 x86_64; target: AArch64 little-endian, qemu-user.
aarch64-linux-gnu-as -EL -g -march=armv8-a \
    -o lab-a030-rec.o lab-a030-rec.s

aarch64-linux-gnu-gcc -std=c11 -O0 -g -mlittle-endian \
    -fno-omit-frame-pointer -Wall -Wextra \
    -c lab-a030-main.c -o lab-a030-main.o

aarch64-linux-gnu-gcc -static -no-pie -mlittle-endian \
    lab-a030-main.o lab-a030-rec.o -o lab-a030
```

```
qemu-aarch64 ./lab-a030
printf 'kilepesi allapot: %d\n' "$?"
```

Mért eredmény (mérve: GNU gcc 13, GNU as 2.42, QEMU 8.2.2): a program `5! = 120`, majd `tesztek: rendben` sort ír ki; kilépési állapota `0`. A `1` állapot eredményeltérést jelent. A statikus linkeléshez szükséges céloldali C-könyvtár rendelkezésre állását a laborban kell ellenőrizni.

8.4.4 A híváslánc megfigyelése GDB-vel

Az első `lab_fact(5)` számításban az argumentumok sorrendje `5, 4, 3, 2, 1`. Mivel a keretfoglalás a bázisvizsgálat előtt történik, a legmélyebb ponton öt, egyenként 32 bájtos keret él.

Első terminál, **qemu-user cél**, elkülönített laborban:

```
qemu-aarch64 -g 1234 ./lab-a030
```

Második terminál; a GDB az AArch64 célfolyamathoz kapcsolódik:

```
gdb-multiarch -q ./lab-a030 \
    -ex 'set pagination off' \
    -ex 'target remote localhost:1234' \
    -ex 'tbreak *lab_fact' \
    -ex 'continue' \
    -ex 'set $lab_entry_sp = (unsigned long)$sp' \
    -ex 'break *lab_fact_base' \
    -ex 'continue' \
    -ex 'p/d $lab_entry_sp - (unsigned long)$sp' \
    -ex 'info registers x0 x19 sp x29 x30' \
    -ex 'x/gx $sp+16' \
    -ex 'bt'
```

A `*lab_fact` az első utasításon állít meg, még a prológus előtt. Ez fontos: egy forrásszintű függvény-töréspontot a debugger a prológus utánra is helyezhetne.

Mért ellenőrzési feltételek a báziság elején (GDB 15.1, QEMU 8.2.2):

Megfigyelés	Mért érték
A64 `X0`	`1`
A64 `X19`	`1`
Belépési és aktuális A64 `SP` különbsége	160 bájt
Aktív faktoriális-aktivációk száma	5
A következő, nem rekurzív hívó	`main`
A mentett hívói A64 `X19` (`x/gx $sp+16`)	2

A visszahívási láncot (*backtrace*) a GDB `bt` parancsa jeleníti meg. A mért futásban a `bt` öt `lab_fact` keretet és a `main`-t mutatta. A pontos címek, sorszámok és a `lab_fact_base` címke megjelenítése eszközverzió-függő; öt faktoriális-aktiváció felismerése az ellenőrzési cél. Visszatérési címeknél a hibakereső a hívóhely környezetét mutathatja, nem feltétlenül a várt forrássort.

Vizsgálat után a GDB `disable breakpoints`, majd `continue` parancsával hagyható lefutni a teljes teszt. Mért kilépési állapot: `0`.

8.4.5 Levezetés: mekkora rekurziós mélység fér el?

Jelölje:

- V a vizsgált futtatáshoz rendelkezésre álló veremméretet, bájtban;
- R a már használt területre és biztonsági tartalékra félretett mennyiséget, bájtban;
- F egy rekurzív aktiváció többlet-veremigényét, bájt/aktiváció egységben;
- d az egyszerre aktív rekurzív aktivációk számát.

Állandó keretméret mellett szükséges:

$$R + dF \leq V.$$

Így a modell szerinti maximális mélység:

$$d_{\max} = \left\lfloor \frac{V - R}{F} \right\rfloor, \qquad V \geq R, \quad F > 0.$$

Elméleti, levezetett példa: ha $V = 65536$ bájt, $R = 4096$ bájt és $F = 32$ bájt/aktiváció, akkor:

$$d_{\max} = \frac{65536 - 4096}{32} = 1920.$$

Ez nem állítás a Linux alapértelmezett veremméretéről, és nem mért QEMU-határ. A ténylegesen használható tartományt befolyásolhatja a folyamat veremkorlátja, a kezdeti argumentum- és környezetterület, a hívók kerete, valamint például a jelkezelés helyigénye. A tartalékot a konkrét programra kell megválasztani.

A saját assembly-függvényünkben:

- `lab_fact(0)` és `lab_fact(1)` egy aktivációt használ;
- `lab_fact(n)` pozitív n mellett n aktivációt használ;
- a rekurzív rész csúcsidejű veremigénye ezért pozitív n esetén $32n$ bájt.

A pontos faktoriális megengedett legnagyobb argumentuma, `20`, csak 640 bájtot igényel ebben a függvényben. **Az aritmetikai tartományhatár és a veremkapacitás két külön korlát.**

Ha a báziság nem érhető el, a veremmutató minden aktivációval tovább csökken. A verem kimerülésének (*stack overflow*) tipikus Linux-tünete `SIGSEGV`, gyakran a következő keret mentésekor. Az ISA nem ír elő külön „rekurziós túlcsordulás” kivételt. A hibát célszerű néhány aktiváció megfigyelésével felismerni, nem szándékosan a teljes verem elfogyasztásáig futtatni.

8.4.6 Farokhívás: mit nem végez el az assembler?

A **farokhívás** (tail call) olyan hívás, amely után a hívóban már nem marad elvégzendő számítás; a hívott függvény eredménye közvetlenül továbbadható.

A példánk önhívása nem farokhívás: visszatérés után még meg kell szorozni az eredményt az aktuális n értékkel.

Akkumulátor bevezetésével viszont használható az

$$g(n, a) = g(n - 1, n \cdot a)$$

átmenet, $n \leq 1$ esetén a eredménnyel. Ez ciklussá alakítható.

A GNU assembler nem végez automatikus farokrekurzió-optimalizálást: utasításokat és relokációkat állít elő, nem alakítja át magas szinten az algoritmust. Ha A64 `BL` utasítást írunk, abból nem lesz magától keret nélküli ciklus. A puszta `BL` → `B` csere sem általános megoldás: a keretet és a visszatérési címet is helyesen kell kezelni.

8.4.7 Tipikus hibák és célzott ellenpróbák

Tünet	Hipotézis	Adatgyűjtés és egyetlen javítás
Az eredmény hibás, de a program visszatér	Az aktuális n hívás által felülírható regiszterben maradt	GDB-ben hasonlítsd össze hívás előtt és után; használj megfelelően mentett A64 `X19`-et
A visszatérés ugyanazon utasításokhoz vezet	Elveszett a külső aktiváció A64 `X30` értéke	Vizsgáld a mentett címet; javítsd a mentés–visszatöltés párt
Ugyanaz az argumentum ismétlődik, miközben fogy a verem	Nincs előrehaladás a báziseset felé	Állj meg két egymást követő belépésnél; javítsd a csökkentést
A számítás jó, a backtrace hibás	A CFI nem egyezik a kerettel	Hasonlítsd össze a leírt és tényleges eltolásokat; csak a hibás CFI-adatot módosítsd

A gépi kód és a keretleírás vizsgálata, **AArch64 qemu-user célállományon**:

```
aarch64-linux-gnu-objdump -d lab-a030-rec.o
aarch64-linux-gnu-readelf --debug-dump=frames lab-a030-rec.o
```

8.4.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Hány faktoriális-keret él a `0`, `1` és `5` bemenet legmélyebb pontján? A `5` bemenet legbelső aktivációjában milyen értéket őriz a mentett A64 `X19` helye? **2. (implementációs, középszint — cél: rutinépítés)** Írj `lab_fact_loop` nevű, akkumulátoros A64 változatot keretfoglalás nélkül. Ellenőrizd a rekurzív változattal való egyezését `0` és `20` között. **3. (becslési, alapszint — cél: megértés-ellenőrzés)** Egy 96 KiB-os veremkeretből 8 KiB tartalékot hagysz. Hány 32 bájtos aktiváció fér el a modell szerint? **4.**

(hibakeresési, középszint — cél: rutinépítés) A rekurzív változat csökkentő utasításában a `#1` értéket változtasd `#0`-ra. Újraépítés után állíts töréspontot a függvény első utasítására. Legfeljebb három belépésből diagnosztizáld a hibát; ne futtasd veremkimerülésig. Csak GDB alatt futtasd; ha mégis közvetlenül indítod: `timeout 5 qemu-aarch64 ./lab-a030`.

8.4.9 Megoldások

1. megoldás — Segítség. Számold meg, hány önhívás történik, mielőtt a báziseset ($n \leq 1$) teljesül — ez adja a keretek számát. A mentett `X19` a HÍVÓ aktiváció n-je, nem a jelenlegié.

1. megoldás — Teljes megoldás. A keretszám rendre `1`, `1`, `5`. A legbelső aktiváció aktuális A64 `X19` értéke `1`, de a mentett helyen `2` található: ezt használja majd a közvetlen hívó a saját szorzásához.

2. megoldás — Segítség. Egyetlen akkumulátorregiszter és egy ciklus elég; nincs szükség `STP`/`LDP`-re vagy `BL`-re, mert a függvény nem hív mást és nem nyúl callee-saved regiszterhez.

2. megoldás — Teljes megoldás. Az alábbi függvény a `lab-a030-rec.s` végére illeszthető. Minden utasítása és regisztere A64-kontextusú; a direktívák GNU assembler-direktívák.

```
// A64 ISA, Armv8-A; ugyanazon labor forrásfájljának kiegészítése.
.text
.p2align 2
.global lab_fact_loop
.type lab_fact_loop, %function
lab_fact_loop:
    .cfi_startproc
    mov     x1, #1

.Lloop:
    cmp     x0, #1
    b.ls    .Lloop_done
    mul     x1, x1, x0
    sub     x0, x0, #1
    b       .Lloop

.Lloop_done:
    mov     x0, x1
    ret
    .cfi_endproc
.size lab_fact_loop, .-lab_fact_loop
```

A függvény nem hív másik függvényt, nem módosít megőrzendő általános regisztert, ezért nincs szüksége keretre. Az A64 `X30` érintetlen marad. A C-fájlban deklaráld a `uint64_t lab_fact_loop(uint64_t n)` függvényt, és hasonlítsd össze mind a 21 bemenet eredményét a `lab_fact` eredményével. A korábbi assemble–compile–link–run parancssort ismételve a várt kilépési állapot továbbra is `0`.

3. megoldás — Segítség. Alakítsd mindkét mennyiséget bájtra (1 KiB = 1024 bájt), vond ki a tartalékot a teljes keretből, majd oszd el az egy aktivációra jutó mérettel.

3. megoldás — Teljes megoldás. Mivel 1 KiB = 1024 bájt:

$$d_{\max} = \frac{(96 - 8) \cdot 1024}{32} = 2816.$$

Ez a megadott modell korlátja, nem garantált üzemi határ.

4. megoldás — Segítség. Ha a csökkentés `#0`, az `X0` érték két egymást követő belépés között azonos marad — ez a báziseset felé vezető előrehaladás hiánya, nem a bázisfeltétel hibája.

4. megoldás — Teljes megoldás. A hibás programban minden belépésnél A64 `X0 = 5`, miközben az `SP` két egymást követő belépés között 32 bájttal csökken. Ez az előrehaladás hiányát mutatja. A `SUB` immediate-jét állítsd vissza `#1`-re, majd újraépítés után ellenőrizd a `5, 4, 3, 2, 1` sorozatot és a `120` eredményt.

8.4.10 Rövid összefoglaló

A rekurzív A64 függvény ugyanazt a kódot hajtja végre több, elkülönített állapotú aktivációban. Az önhívás felülírja a visszatérési címet, az eredményregiszter pedig nem őrzi meg automatikusan az eredeti argumentumot. Ezért a mentések és visszatöltések minden szinten szükségesek. A báziseset mellett a felé vezető előrehaladást is ellenőrizni kell. Állandó keretméretnél a rendelkezésre álló veremből egyszerű mélységbecslés adható, de az üzemi tartalék környezetfüggő. A GDB backtrace és a veremmutató változása együtt jól ellenőrizhetővé teszi a hívásláncot.

- **Aktiváció:** egy még be nem fejezett függvényvégrehajtás.
- **Rekurziós mélység:** az egyszerre aktív rekurzív aktivációk száma.
- **Farokhívás:** hívás, amely után nem marad további eredményfeldolgozás.

8.4.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- AAPCS64 – Arm hivatalos ABI-dokumentum — regisztermegőrzés, veremigazítás, keretrekord. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- Arm A-profile architektúra-dokumentáció — A64 utasításszemantika és kódolási korlátok. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64-szintaxis és CFI-direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GDB kézikönyv — távoli hibakeresés, keretek és backtrace. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a kimenet, a GDB-állapotok és a backtrace a lektor GNU gcc 13 + GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). A csomagolt eszközverziókat, a statikus linkelés feltételeit és a backtrace konkrét megjelenését a hallgatói laborban is ellenőrizni kell. A veremértékek levezetett adatok; tényleges veremkimerülési határt nem mértünk, és a `lab_fact_loop` változat eredménye levezetett, nem mért.

8.5 Több visszatérési érték és struktúra-visszaadás szerződése

8.5.1 Miért nem elég azt mondani, hogy „az eredmény az X0-ban van”?

Egy feldolgozófüggvény gyakran több összetartozó eredményt számít: például összeget és különbséget, minimumot és maximumot, vagy feldolgozott elemszámot és hibakódot. C-ben ezt

struktúra-visszaadással vagy kimenő mutatóparaméterekkel (*output parameters*) fejezhetjük ki. Assemblyben azonban pontosan meg kell állapodnunk arról, melyik eredmény hol jelenik meg.

Az AAPCS64 ezt a **visszaadott típus alapján**, a fordításkor rögzíti. A hívó és a hívott nem egyeztet futás közben a struktúra méretéről.

Ebben az alfejezetben kizárólag **A64 ISA, AArch64 Linux, little-endian, LP64** környezetben dolgozunk. A példák normál elrendezésű, egész mezőkből álló C-struktúrákat használnak. Minden assemblyutasítás és regiszter ebbe az A64-környezetbe tartozik; az assemblydirektívák a GNU assembler AArch64 céltárgyára vonatkoznak. Előfeltétel Az AAPCS64 regiszterszerződés ismerete.

8.5.2 Az eredmény helyét a típus határozza meg

A **közvetlen eredményátadás** (*direct result return*) során a függvény eredménye regiszterekben érkezik. A **közvetett eredményátadás** (*indirect result return*) esetén a hívó eredménytárolót biztosít, a hívott pedig ebbe ír.

A most vizsgált, egész mezőkből álló struktúrákra:

A visszaadott struktúra `sizeof` értéke	Eredmény helye, A64/AAPCS64
1–8 bájt	`X0` megfelelő bájtjai
9–16 bájt	`X0`, majd `X1` megfelelő bájtjai
16 bájtnál több	Memória; az eredménytároló címét a hívó `X8`-ban adja át

A határ tehát **16 bájt, beleértve az igazítás miatti kitöltést** (*padding*). Nem a mezők száma és nem pusztán a mezőméretek összege dönt.

Fontos hatókörkorlát: a „16 bájt felett memória” nem minden AAPCS64-típusra érvényes általános szabály. A homogén lebegőpontos és vektoraggregátumok (*HFA, HVA*) külön besorolást kapnak; például egy négy `double` mezőből álló homogén struktúra 32 bájtos, mégis lebegőpontos regiszterekben térhet vissza. Ezeket később tárgyaljuk. Itt egész típusú aggregátumokra alkalmazzuk a táblázatot.

8.5.2.1 A regiszterek nem feltétlenül mezőket jelentenek

A kis struktúra memóriaképe kerül az A64 eredményregiszterekbe, nem általánosan „egy mező egy regiszterbe”.

Little-endian profilban egy

```
/* AArch64 Linux, LP64, little-endian. */
struct lab_words {
    uint32_t first;
    uint32_t second;
};
```

típusú, 8 bájtos eredmény mindkét mezője az A64 `X0` regiszterben fér el: `first` az alsó, `second` a felső 32 biten. A definíció használatához természetesen `<stdint.h>` szükséges.

Ezzel szemben két `uint64_t` mező esetén a struktúra 16 bájtos: az első mező A64 `X0`-ba, a második A64 `X1`-be kerül.

A kitöltőbájtokból és a regiszter kihasználatlan részeiből ne következtessünk értékre. Az ellenőrzés a mezőket hasonlítsa össze, ne a teljes struktúrát `memcmp` segítségével.

8.5.2.2 Mit jelent a rejtett X8-paraméter?

Nagy struktúránál a hívó:

1. megfelelő méretű és igazítású, írható tárolót biztosít;
2. annak címét az A64 `X8` regiszterbe teszi;
3. a szokásos argumentumokat a szokásos helyükön adja át;
4. a visszatérés után az eredményt a tárolóból használja.

Az A64 `X8` különleges szerepe miatt egy nagy struktúrát visszaadó, két egész paraméteres függvény első két argumentuma továbbra is `X0` és `X1`. A rejtett eredménymutató **nem tolja el** őket.

A hívott nem köteles az eredménytároló címét A64 `X0`-ban visszaadni. Az A64 `X8` pedig nem megőrzött regiszter: ha a hívott további függvényt hív, és később még szüksége lesz a címre, saját magának kell megőriznie.

8.5.3 Kidolgozott példa: három interfész ugyanahhoz a számításhoz

Munkakönyvtár: `~/arm-labor/a64/a031/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a031
cd ~/arm-labor/a64/a031
```

Három függvényt készítünk:

- `lab_pair`: két eredmény visszaadása 16 bájtos struktúrában;
- `lab_triple`: három eredmény visszaadása 24 bájtos struktúrában;
- `lab_pair_out`: két eredmény kiírása explicit mutatóparaméteren keresztül.

Minden számítás előjel nélküli, 64 bites; az összeadás, kivonás és a későbbi szorzás eredménye modulo 2^{64} értendő.

Az interfészek szerződése:

Függvény	Bemenet, A64	Eredmény, A64
`lab_pair(a, b)`	`X0 = a`, `X1 = b`	`X0 = a+b`, `X1 = a-b`
`lab_triple(a, b)`	`X0 = a`, `X1 = b`, `X8 = eredménytároló`	24 bájt az eredménytárolóban
`lab_pair_out(a, b, out)`	`X0 = a`, `X1 = b`, `X2 = out`	16 bájt az `out` által jelölt tárolóban

Az explicit kimenő mutató közönséges harmadik argumentum, ezért A64 `X2`-ben érkezik, nem `X8`-ban. Mindkét memóriába író változat érvényes, kellően igazított tárolót igényel; ezt a C-teszt helyi objektumai biztosítják.

8.5.3.1 Assemblymegvalósítás

A `lab-return.S` fájl teljes tartalma:

```
// A64 ISA, Armv8-A; GNU as, AArch64 Linux, little-endian.
.arch armv8-a
.text
.p2align 2

.global lab_pair
.type lab_pair, %function
lab_pair:
    add x2, x0, x1
    sub x1, x0, x1
    mov x0, x2
    ret
.size lab_pair, .-lab_pair

.global lab_triple
.type lab_triple, %function
lab_triple:
    add x2, x0, x1
    sub x3, x0, x1
    eor x4, x0, x1
    stp x2, x3, [x8]
    str x4, [x8, #16]
    ret
.size lab_triple, .-lab_triple

.global lab_pair_out
.type lab_pair_out, %function
lab_pair_out:
    add x3, x0, x1
    sub x4, x0, x1
    stp x3, x4, [x2]
    ret
.size lab_pair_out, .-lab_pair_out

.section .note.GNU-stack,"",%progbits
```

Mindhárom levélfüggvény: nem hív további függvényt, nem módosít megőrzendő regisztert, ezért nincs szüksége veremkeretre.

Az első függvényben az A64 `X2` ideiglenesen megőrzi az összeget. Ez szükséges, mert a különbséghez még kell az eredeti `X0` bemenet. Ha előbb közvetlenül `X0`-ba írnánk az összeget, elveszítenénk ezt a bemenetet.

Az A64 `STP` két 64 bites regisztert tárol egymást követő 8 bájtos helyekre. A használt eltolásos alak előjeles, 7 bites, 8-cal skálázott eltolást kódolhat: -512 és 504 bájt között, 8 bájtos lépésekben. Itt az eltolás nulla.

Az A64 `STR X4, [X8, #16]` előjel nélküli, 12 bites, 8-cal skálázott eltolást használ: 0–32760 bájt között, 8 bájtos lépésekben. A 16 bájtos eltolás tehát kódolható.

A GNU AArch64 `.p2align 2` direktíva $2^2 = 4$ bájtos igazítást kér. A `.size` a függvény gépi kódjának méretét rögzíti, **nem a C-visszatérési típus méretét**.

8.5.3.2 C-interfész és ellenőrzőprogram

A `lab-return-main.c` fájl:

```
/* AArch64 Linux, LP64, little-endian. */
#include <stddef.h>
#include <stdint.h>
#include <inttypes.h>
#include <stdio.h>

struct lab_pair_value {
    uint64_t sum;
    uint64_t difference;
};

struct lab_triple_value {
    uint64_t sum;
    uint64_t difference;
    uint64_t xor_value;
};

extern struct lab_pair_value
lab_pair(uint64_t a, uint64_t b);

extern struct lab_triple_value
lab_triple(uint64_t a, uint64_t b);

extern void
lab_pair_out(uint64_t a, uint64_t b, struct lab_pair_value *out);

_Static_assert(sizeof(struct lab_pair_value) == 16, "pair size");
_Static_assert(offsetof(struct lab_pair_value, sum) == 0, "sum offset");
_Static_assert(offsetof(struct lab_pair_value, difference) == 8,
               "difference offset");

_Static_assert(sizeof(struct lab_triple_value) == 24, "triple size");
_Static_assert(offsetof(struct lab_triple_value, sum) == 0, "sum offset");
_Static_assert(offsetof(struct lab_triple_value, difference) == 8,
               "difference offset");
_Static_assert(offsetof(struct lab_triple_value, xor_value) == 16,
               "xor offset");

int main(void)
{
    const uint64_t inputs[][2] = {
        {20, 7},
        {0, 1},
        {UINT64_MAX, 1}
    };

    for (size_t i = 0; i < sizeof inputs / sizeof inputs[0]; ++i) {
        uint64_t a = inputs[i][0];
        uint64_t b = inputs[i][1];

        struct lab_pair_value p = lab_pair(a, b);
        struct lab_triple_value t = lab_triple(a, b);
        struct lab_pair_value out = {0, 0};
        lab_pair_out(a, b, &out);

        if (p.sum != a + b || p.difference != a - b ||
            t.sum != a + b || t.difference != a - b ||
            t.xor_value != (a ^ b) ||
            out.sum != a + b || out.difference != a - b) {
            fprintf(stderr, "hiba: eset %zu\n", i);
            return 1;
        }
```

```
        if (i == 0) {
            printf("sum=%" PRIu64 " difference=%" PRIu64
                   " xor=%" PRIu64 "\n",
                   t.sum, t.difference, t.xor_value);
        }
    }

    puts("ok: 3 eset, 3 visszaadasi mod");
    return 0;
}
```

A statikus állítások az assembly által feltételezett méreteket és mezőeltolásokat ellenőrzik. Nem helyettesítik a közös típusdeklarációt, de megakadályozzák, hogy egy észrevétlen struktúramódosítás hibás bináris szerződést hozzon létre.

8.5.3.3 Fordítás és futtatás

Cél: qemu-user, `qemu-aarch64`; host: Ubuntu 24.04 x86_64. A statikus linkeléshez az AArch64 cross libc fejlesztői állományai is szükségesek, az alapkörnyezet részeként.

```
# Assemble: A64 objektum.
aarch64-linux-gnu-gcc -march=armv8-a -g \
    -c lab-return.S -o lab-return.o

# Compile: AArch64 Linux C-objektum.
aarch64-linux-gnu-gcc -march=armv8-a -std=c11 -O0 -g \
    -Wall -Wextra -Werror -fno-pie \
    -c lab-return-main.c -o lab-return-main.o

# Link: statikus AArch64 Linux ELF.
aarch64-linux-gnu-gcc -static -no-pie \
    lab-return-main.o lab-return.o -o lab-return

# Run: qemu-user.
qemu-aarch64 ./lab-return
printf 'kilepesi allapot=%d\n' "$?"
```

Mért kimenet (mérve: GNU gcc 13, GNU as 2.42, QEMU 8.2.2):

- `sum=27 difference=13 xor=19`
- `ok: 3 eset, 3 visszaadasi mod`
- `kilepesi allapot=0`

Az első hívássorozat után:

Ellenőrzési pont	Állapot, A64
`lab_pair`, közvetlenül visszatéréskor	`X0 = 27`, `X1 = 13` (mért)
`lab_triple`, az eredménytároló 0 bájtos eltolásán	64 bites előjel nélküli 27
Ugyanott, 8 bájtos eltoláson	64 bites előjel nélküli 13
Ugyanott, 16 bájtos eltoláson	64 bites előjel nélküli 19

A `lab_pair` sort a lektor mérése igazolta; a `lab_triple` memóriatartalma levezetett.

A másik két teszt az előjel nélküli alulcsordulást és körbefordulást is ellenőrzi. Ezek C-ben definiált műveletek; nem szabad előjeles túlcsordulási példaként értelmezni őket.

8.5.4 Hibák és célzott ellenpróbák

8.5.4.1 A két fordítási egység más visszatérési típust feltételez

Ha a C-deklaráció 24 bájtos eredményt ír elő, de az assembly csak A64 `X0` és `X1` értékét állítja be, a hívó által kijelölt tároló nem kapja meg az eredményt.

Fordított esetben a hívott A64 `X8`-on keresztül próbálhat írni, miközben a hívó nem adott át érvényes eredménycímet. Ennek következménye lehet memóriahiba vagy memóriarongálás.

A linker rendszerint csak a szimbólumnevet egyezteti; ettől a C-típusok még nem lesznek kompatibilisek. **A sikeres linkelés nem ABI-ellenőrzés.**

8.5.4.2 A két mezőt tévesen két W-regiszterbe tesszük

Két `uint32_t` mező 8 bájtos struktúrája nem az A64 `W0` és `W1` regiszterben tér vissza. Mindkét mező az A64 `X0` regiszter része. Ez tipikus hiba, amikor a fejlesztő mezőszámból következtet regiszterszámra.

8.5.4.3 A kimenő mutatót összekeverjük a rejtett mutatóval

A példában `lab_pair_out` explicit harmadik paramétere A64 `X2`, míg `lab_triple` rejtett eredménycíme A64 `X8`. Ugyanaz a memóriaírási művelet eltérő interfészből származhat.

8.5.4.4 Adatgyűjtés objdump és GDB segítségével

Cél: a qemu-userhez épített AArch64 ELF vizsgálata. A következő parancsok nem futtatják a célprogramot:

```
aarch64-linux-gnu-objdump -d --disassemble=lab_pair ./lab-return
aarch64-linux-gnu-objdump -d --disassemble=lab_triple ./lab-return
```

A64 `lab_pair` bemeneteinek és eredményeinek követéséhez az első terminálban:

```
# Cél: qemu-user; várakozás a debugger kapcsolódására.
qemu-aarch64 -g 1234 ./lab-return
```

Második terminálban:

```
# Cél: az előző qemu-user folyamat A64 állapotának vizsgálata.
gdb-multiarch -q -batch ./lab-return \
    -ex 'target remote :1234' \
    -ex 'break *lab_pair' \
    -ex 'continue' \
    -ex 'info registers x0 x1' \
    -ex 'si 3' \
    -ex 'info registers x0 x1 x2' \
    -ex 'detach'
```

A belépéskor A64 `X0 = 20`, `X1 = 7`; három utasítás után, még az A64 `RET` előtt `X0 = 27`, `X1 = 13`, `X2 = 27` (mérve: GDB 15.1, QEMU 8.2.2). A leválasztás után a program befejeződik, és 0 állapottal lép ki.

A hibakeresési ciklus: tünet → feltételezett szerződés- vagy adatfüggőségi hiba → regiszteradatok → egyetlen javítás → teljes teszt újrafuttatása. Az emuláció itt szemantikai ellenőrzést szolgál; hardveres ciklusszámot nem állapítunk meg.

8.5.5 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Hol tér vissza a két `uint32_t` mezős, a két `uint64_t` mezős és a három `uint64_t` mezős struktúra? Az első esetben milyen hexadecimális érték lesz az A64 eredményregiszterben, ha a mezők értéke rendre 1 és 2?

2. (implementációs, középszint — cél: rutinépítés) Módosítsd `lab_triple` harmadik eredményét bitenkénti kizáró vagy helyett az `a * b` előjel nélküli szorzat alsó 64 bitjére. Ne változtasd meg a struktúra méretét vagy a visszaadási mechanizmust. Igazítsd hozzá a C-tesztet is.

3. (hibakeresési, középszint — cél: rutinépítés) A `lab_pair` számítását valaki az alábbi A64-részletre cserélte. Milyen eredményt kapsz az első tesztben? Ellenőrizd a hipotézist debuggerrel, majd javítsd az adatfüggőséget.

```
// Szándékosan hibás A64 függvénytörzs.
add x0, x0, x1
sub x1, x0, x1
ret
```

8.5.6 Megoldások

1. megoldás — Segítség. Nézd meg a fejezet első táblázatát (1–8 bájt / 9–16 bájt / 16 bájt felett), majd gondold át little-endian elrendezésben, hogy egy 8 bájtos regiszteren belül az ALACSONYABB című mező kerül-e az alsó vagy a felső félre.

1. megoldás — Teljes megoldás. A 8 bájtos eredmény A64 `X0`-ban, a 16 bájtos eredmény A64 `X0`/`X1`-ben, a 24 bájtos eredmény pedig az A64 `X8`-ban átadott címen tér vissza. Little-endian profilban az első struktúra eredménye `X0 = 0x0000000200000001`: az első mező foglalja el az alsó 32 bitet.

2. megoldás — Segítség. Az A64 `MUL`-nak ugyanaz a célregiszter/forrás-mintája van, mint az `ADD`/`SUB`/`EOR`-nak — csak a mnemonicot és a szemantikát kell cserélni, a struktúra-elrendezést nem.

2. megoldás — Teljes megoldás. A meglévő A64 `EOR` helyére ez kerül:

```
// A64: a 64 bites szorzat alsó 64 bitje.
mul x4, x0, x1
```

A C-forrásban a `xor_value` mezőnevet következetesen `product` névre cseréljük, az ellenőrzés pedig `t.product != a * b` legyen. A kiírás feliratát is módosítjuk. Az első eset harmadik eredménye 140. A struktúra továbbra is 24 bájtos, ezért az A64 `X8`-alapú szerződés változatlan. A korábbi buildlépéseket újra végre kell hajtani.

3. megoldás — Segítség. A hibás verzió az összeget ÚJRA `X0`-ba írja, mielőtt a különbséghez szüksége lenne az EREDETI `X0` bemenetre — melyik érték vész el emiatt a második sorban?

3. megoldás — Teljes megoldás. Az első A64 utasítás után `X0 = 27`; a második ezért már `27 - 7` értékét számítja. Az eredmény `X0 = 27`, `X1 = 20`, nem pedig 27 és 13. A C-teszt az első esetnél hibát jelez, és 1 állapottal lép ki. A debuggerben két utasítást léptessünk, hogy még a visszatérés előtt lássuk a hibát. A kidolgozott példa A64 `X2`-t használó változata megőrzi a szükséges bemenetet, ezért helyes.

8.5.7 Rövid összefoglaló

A visszatérési szerződést a függvény típusa határozza meg, nem a függvénytörzs választja futás közben. A vizsgált egész mezős struktúrák legfeljebb 16 bájtig A64 `X0` és szükség esetén `X1` segítségével térnek vissza; nagyobb eredménynél a hívó A64 `X8`-ban adja át a tároló címét. A kitöltés beleszámít a méretbe, a mezők pedig nem feltétlenül külön regisztereket jelentenek. Az explicit kimenő mutató közönséges paraméter. A helyes aritmetika mellett a típusdeklaráció, a mezőeltolás és az eredmény helye is ellenőrzendő.

- **Közvetlen eredményátadás:** az eredmény regiszterekben érkezik.
- **Közvetett eredményátadás:** az eredmény a hívó által biztosított tárolóba kerül.
- **Rejtett eredménymutató:** AAPCS64 alatt az A64 `X8`-ban átadott eredménycím.

8.5.8 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- AAPCS64 – Arm hivatalos ABI-dokumentum — eredményátadás, összetett típusok besorolása, az `X8` szerepe és HFA/HVA-kivételek. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (az SME-részek Beta).
- GNU assembler kézikönyv — AArch64-szintaxis, direktívák és címzési alakok. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- Arm A-profile architektúra-dokumentáció — az A64 utasítások architekturális jelentése. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a közölt kimenet, a `lab_pair` regiszterállapota és a GDB-munkamenet a lektor GNU gcc 13 + GNU as 2.42 + QEMU 8.2.2 + GDB 15.1 méréséből származnak (`mérve`). Az Ubuntu 24.04 környezet konkrét csomagverzióit, a statikus cross libc elérhetőségét és a debugger-munkamenetet a laborban is validálni kell. A `lab_triple` memóriatartalma és a feladatok módosított változatai levezetett elvárások, nem mért eredmények.

9 Bitmanipuláció és logikai műveletek

9.1 Logikai alapműveletek és bitmaszkolás

9.1.1 Miért fontos a megfelelő maszk?

Egy fájlformátum állapotmezőjében néhány bit jelölheti az érvényességet, mások a feldolgozási módot. A feladat gyakran nem a teljes érték lecserélése, hanem kijelölt bitek kiolvasása, beállítása vagy törlése, miközben minden más bit változatlan marad. Ugyanez jelenik meg jogosultságjelzők, tömörített adatok és protokollmezők kezelésénél.

A **bitmaszk (bit mask)** olyan bitminta, amely megadja, hogy egy bitenkénti művelet mely pozíciókat érintse. A maszk jelentése a művelettől függ: az egyik művelet megtartja, a másik beállítja, a harmadik törli a maszkkal kijelölt biteket.

Ebben az alfejezetben minden utasítás és regiszternév az **A64 ISA, AArch64, Armv8-A alapprofil** része. Az assemblyblokkok GNU `as` AArch64-szintaxist használnak; a direktívák assemblerutasítások, nem processzorutasítások. A labor little-endian Linux-program, célja `qemu-aarch64`.

Az előfeltétel a feltételkódok és feltételes kiválasztás ismerete. A példák előjel nélküli, 32 bites bitmintákkal dolgoznak. A bitenkénti műveletek maguk nem értelmeznek előjelet: ugyanazokon a biteken ugyanazt végzik.

9.1.2 Négy művelet, négy maszkolási szándék

Legyen az adat `x`, a maszk `m`, mindkettő `uint32_t` érték. Az A64 ISA regiszteres alakjai:

Szándék	C-kifejezés	A64 ISA-művelet	Az eredmény jelentése
Elkülönítés	`x & m`	`AND Wd, Wn, Wm`	Csak a maszk 1-es helyein maradhat 1
Beállítás	`x | m`	`ORR Wd, Wn, Wm`	A maszk 1-es helyein biztosan 1 lesz
Átváltás	`x ^ m`	`EOR Wd, Wn, Wm`	A maszk 1-es helyein megfordul a bit
Törlés	`x & ~m`	`BIC Wd, Wn, Wm`	A maszk 1-es helyein biztosan 0 lesz

A táblázatban az A64 `Wd` a cél, `Wn` az adatforrás, `Wm` a maszkforrás. A cél megegyezhet valamelyik forrással: az utasítás az eredeti forrásértékekből számítja ki az eredményt.

Egyetlen bitre az igazságtábla:

Adatbit	Maszkbit	AND	OR	XOR	AND NOT
0	0	0	0	0	0
0	1	0	1	1	0
1	0	0	1	1	1
1	1	1	1	0	0

Az A64 ISA itt használt `AND`, `ORR`, `EOR` és `BIC` alakjai **nem módosítják az NZCV állapotjelzőket**. Egy utánuk következő feltételes döntés ezért nem automatikusan a maszkolás eredményére vonatkozik. A labor külön összehasonlítással ellenőrzi az eredményeket.

9.1.2.1 Elkülönítés és törlés

Az A64 `AND` kétféleképpen is megfogalmazhat egy feladatot:

- **Elkülönítés (isolate):** a megtartandó bitek helyén 1 áll a maszkban.
- **Törlés (clear):** a törlendő bitek helyén 0 áll a maszkban.

Például az előjel nélküli `0x000000AC` érték alsó négy bitjének elkülönítése:

$$\texttt{0x000000AC} \;\&\; \texttt{0x0000000F} = \texttt{0x0000000C}.$$

A 0-tól számozott 3. bit törlése ugyanezen a 32 bites értéken:

$$\texttt{0x000000AC} \;\&\; \texttt{0xFFFFFFF7} = \texttt{0x000000A4}.$$

Az A64 `BIC` ugyanezt a törlést a **törlendő** biteket kijelölő `0x00000008` maszkkal fejezi ki. A komplementálás a műveleti szélességen belül történik: 32 bites műveletben a maszk 32 bitje fordul meg.

9.1.2.2 Beállítás és átváltás

Az A64 `ORR` beállító művelete idempotens: ugyanazt a maszkot ismét alkalmazva az eredmény nem változik tovább. Az A64 `EOR` ezzel szemben **átváltást (toggle)** végez: ugyanazzal a maszkkal kétszer alkalmazva visszaadja az eredeti értéket.

Ez mérnöki szempontból lényeges különbség. Az „érvényes bit legyen 1” feladatra beállítás kell, nem átváltás. Az utóbbi egy már érvényes értéket érvénytelenné tenne.

9.1.3 Miért nem írható minden maszk közvetlenül az utasításba?

Az A64 utasítások 32 bitesek. Az opcode, a regiszterazonosítók és más vezérlőmezők mellett nem fér el bennük tetszőleges 32 vagy 64 bites konstans.

A **közvetlen operandus (immediate operand)** ezért nem feltétlenül a teljes számot tárolja. Az A64 logikai immediate egy szabályos bitminta tömör leírása. Kódolása az `N`, `immr` és `imms` mezőket használja, rendre 1, 6 és 6 biten; nem minden mezőkombináció érvényes.

A megengedett minták előállításának gondolatmenete:

1. Válassz egy elemhosszt: 2, 4, 8, 16, 32 vagy 64 bitet.

2. Az elemen belül legyen egy összefüggő 1-es sorozat, a többi bit pedig 0.
3. Az 1-es sorozat hossza legalább 1, de kisebb az elemhossznál.
4. Az elemet körkörösen el lehet forgatni.
5. Az így kapott elemet ismételni kell a teljes műveleti szélesség kitöltéséig.

A64 32 bites műveletben legfeljebb 32 bites elem választható, és az `N` mező 0. A64 64 bites műveletben a 64 bites elem is használható.

Az elforgatás miatt az 1-es sorozat „átérhet” az elem végéről az elejére. A teljesen nulla és a teljesen egyes maszk viszont **nem kódolható logikai immediate-ként**.

Előjel nélküli, 32 bites maszk	Kódolható?	Indok
`0x0000000F`	Igen	32 bites elemben négy összefüggő 1
`0xFFFFFFF7`	Igen	Körkörösen nézve 31 összefüggő 1
`0x00FF00FF`	Igen	A `0x00FF` 16 bites elem ismétlődik
`0x55555555`	Igen	A `0b01` 2 bites elem ismétlődik
`0x80000001`	Igen	Két szomszédos 1 az elemhatáron át
`0x00000005`	Nem	Két különálló 1, megfelelő ismétlődés nélkül
`0x00000000`	Nem	Nincs 1-es sorozat
`0xFFFFFFFF`	Nem	Nincs 0 az elemben

A korlát tehát nem a szám nagyságáról szól: az 5 nem használható ilyen immediate-ként, miközben a sokkal nagyobb `0x55555555` igen.

A szélesség is számít. A `0x00FF00FF` 32 bites maszk kódolható, de a `0x00000000FF00FF` 64 bites maszk nem: a 16 bites elem nem ismétlődik végig, és a teljes 64 bites mintában sincs egyetlen összefüggő 1-es sorozat.

Az A64 `AND`, `ORR` és `EOR` rendelkezik ilyen immediate-alakkal. Önálló `BIC` immediate-utasításkódolás nincs: konstans bitek törléséhez használható a komplementer maszkkal végzett `AND`, vagy a maszk regiszterbe töltése és a regiszteres `BIC`. Az esetleges assembler-rövidítéseket nem szabad külön ISA-kódolással összetéveszteni.

9.1.4 Kidolgozott példa: önellenőrző maszkolási labor

Munkakönyvtár: `~/arm-labor/a64/a032/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a032
cd ~/arm-labor/a64/a032
```

A következő teljes program külön célregiszterekbe számolja az eredményeket, majd összehasonlítja őket a várt értékekkel. Nem ír ki szöveget: siker esetén 0, eltéréskor 1 a kilépési állapot.

Az A64 példában szereplő további konstanskorlátok:

- `MOVZ`: 16 bites immediate; 32 bites alakban opcionálisan 0 vagy 16 bites balra eltolással. Itt nincs eltolás.
- `CMP` immediate: 12 bites, előjel nélküli konstans, opcionálisan 12 bites balra eltolással. Itt minden érték közvetlenül elfér.
- `SVC`: 16 bites immediate; az AArch64 Linux rendszerhívási mintában itt 0.

A GNU AArch64 `.p2align 2` direktíva $2^2 = 4$ bájtos igazítást kér.

A fájl neve legyen `lab-mask.s`:

```
// A64 ISA, Armv8-A; GNU as AArch64; little-endian Linux.
.arch armv8-a
.text
.p2align 2
.global _start
.type _start, %function

_start:
    movz    w0, #0xAC

    and     w1, w0, #0x0F
    orr     w2, w0, #0x01
    eor     w3, w0, #0x08

    movz    w4, #0x08
    bic     w5, w0, w4

    // A 0x05 nem logikai immediate: regiszteres maszk kell.
    movz    w6, #0x05
    and     w7, w0, w6

    // XOR-alapú csere két különböző regiszteren.
    movz    w10, #0x12
    movz    w11, #0xA5
    eor     w10, w10, w11
    eor     w11, w10, w11
    eor     w10, w10, w11

lab_observe:
    cmp     w1, #0x0C
    b.ne    lab_fail
    cmp     w2, #0xAD
    b.ne    lab_fail
    cmp     w3, #0xA4
    b.ne    lab_fail
    cmp     w5, #0xA4
    b.ne    lab_fail
    cmp     w7, #0x04
    b.ne    lab_fail
    cmp     w10, #0xA5
    b.ne    lab_fail
    cmp     w11, #0x12
    b.ne    lab_fail

    movz    w0, #0
    b       lab_exit

lab_fail:
    movz    w0, #1

lab_exit:
    movz    x8, #93
    svc     #0

.size _start, .-_start
.section .note.GNU-stack, "", %progbits
```

A `0x05` előállítására használt A64 `MOVZ` más konstanskódolást használ, mint a logikai utasítások: ezért fogadja el azt, amit az `AND` immediate-alakja nem.

Az assemble–link–run lépéssor Ubuntu 24.04 x86_64 hoston:

```
# Target: qemu-user, AArch64 Linux, little-endian.
aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-mask.o lab-mask.s
aarch64-linux-gnu-ld -EL -e _start \
    -o lab-mask.elf lab-mask.o
qemu-aarch64 ./lab-mask.elf
printf 'kilepesi_allapot=%d\n' "$?"

# Target: ugyanennek az AArch64 ELF-nek az elemzése, nem futtatás.
aarch64-linux-gnu-objdump -d lab-mask.elf
```

Mérve (a lektor mérése, GNU binutils 2.42, QEMU 8.2.2): `kilepesi_allapot=0`.

A `lab_observe` címkénél várt állapot:

A64 regiszternézet	Várt, előjel nélküli érték	Jelentés
`W0`	`0x000000AC`	Eredeti adat
`W1`	`0x0000000C`	Alsó négy bit
`W2`	`0x000000AD`	0. bit beállítva
`W3`	`0x000000A4`	3. bit átváltva
`W5`	`0x000000A4`	3. bit törölve
`W7`	`0x00000004`	A 0. és 2. bit elkülönítve
`W10`	`0x000000A5`	Csere utáni első érték
`W11`	`0x00000012`	Csere utáni második érték

A64-en a `Wn` célregiszterbe írás nullázza a megfelelő `Xn` felső 32 bitjét. Ez nem a maszk különleges tulajdonsága, hanem a 32 bites regiszterírás általános szabálya.

9.1.5 Az XOR-alapú csere: levezetés és korlát

Legyen két különböző A64 regiszter kezdeti bitmintája a és b. Az XOR tulajdonságai: $x \oplus x = 0$, illetve $x \oplus 0 = x$.

A labor három lépése:

1. Az első regiszterbe $a \oplus b$ kerül.
2. A másodikba $(a \oplus b) \oplus b = a$ kerül.
3. Az elsőbe $(a \oplus b) \oplus a = b$ kerül.

A csere tehát ideiglenes regiszter nélkül működik, **ha a két regiszter különböző**. Ha ugyanazt a regisztert adjuk meg mindkét operandusként, az első XOR lenullázza az értéket. Két különböző regiszter azonos tartalma viszont nem hiba.

Ez oktatási azonosság, nem általános optimalizálási ajánlás. A három A64 `EOR` egymástól függő művelet; az ideiglenes regiszteres megoldás rendszerint világosabb. Pusztán az utasításszámból nem következik ciklusszám vagy gyorsulás, QEMU alatt pedig nem mérhető megbízhatóan egy fizikai processzor ilyen késleltetése.

9.1.6 Tipikus hibák és ellenpróbák

A hibakeresés menete: **tünet → hipotézis → adatgyűjtés → egy változtatás → ellenőrzés**.

Tünet	Hipotézis	Ellenpróba
A fordítás elutasít egy kicsi maszkot	Nem logikai immediate-minta	Töltsd regiszterbe, és csak az operandusalakot cseréld
Második alkalmazásra eltűnik egy jelző	Beállítás helyett átváltás történt	Ugyanazzal a maszkkal kétszer futtasd
Nem a kívánt bit törlődik	A megtartó és törlő maszk összekeveredett	Írd fel a maszkot binárisan
A feltételes döntés hibás	Régi NZCV alapján dönt	Vizsgáld meg az utolsó jelzőfrissítő utasítást
Eltűnik a 64 bites érték felső fele	32 bites célregisztert használtál	Ellenőrizd a teljes `Xn` nézetet

A laborállapot megfigyeléséhez az első terminálban:

```
# Target: qemu-user, AArch64; GDB-kapcsolatra vár.
qemu-aarch64 -g 1234 ./lab-mask.elf
```

A második terminálban:

```
# Target: a fenti qemu-user folyamat távoli hibakeresése.
gdb-multiarch -q ./lab-mask.elf \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_observe' \
    -ex 'continue' \
    -ex 'info registers x0 x1 x2 x3 x4 x5 x6 x7 x10 x11 cpsr'
```

A GDB itt a teljes A64 `Xn` nézeteket mutatja; a táblázat értékei ezek alsó 32 bitjében várhatók, nulla felső résszel. A `continue` debuggerparancs után a helyes program várhatóan normálisan, 0 állapottal lép ki.

Különösen hasznos ellenpróba a kezdőérték módosítása `0xA4`-re. Ekkor az átváltás eredménye `0xAC`, a törlésé továbbra is `0xA4`. A két művelet csak az eredeti adat miatt adott azonos eredményt.

9.1.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Melyik művelet kell a 6. bit biztos beállításához, biztos törléséhez és átváltásához? Miért kódolható a 32 bites `0x80000001`, de nem kódolható a `0x00000005` logikai immediate-ként?

2. (implementációs, középszint — cél: rutinépítés) Bővítsd a labort úgy, hogy az eredeti adat 0. és 2. bitjét beállítsa, a 7. bitjét törölje, majd a 3. bitjét átváltsa. Az eredmény legyen az A64 `W13` regiszterben, alsó négy bitje pedig `W14`-ben. Használj regiszteres maszkot ott, ahol szükséges, és bővítsd az önellenőrzést.

3. (hibakeresési, középszint — cél: rutinépítés) Készíts `lab-mask-bad.s` nevű másolatot. Cseréld az A64 `and w7, w0, w6` sort `and w7, w0, #0x05` alakra. Rögzítsd az assembler kilépési állapotát és diagnosztikáját. Miért nem futásidejű hiba ez? Javítsd egyetlen operandus visszacserélésével, majd ellenőrizd a programot.

9.1.8 Megoldások

1. megoldás — Segítség. A 6. bit maszkja 2^6; nézd meg az igazságtáblát, hogy a négy művelet közül melyik GARANTÁLJA az 1-et, melyik a 0-t, melyik csak megfordítja. A kódolhatósághoz: fogalmazd meg mindkét mintát mint "egy elem körkörös elforgatva, ismételve".

1. megoldás — Teljes megoldás. A64-en a 6. bit maszkja `0x40`: beállításhoz `ORR`, átváltáshoz `EOR` használható. Törléshez `BIC` alkalmazható regiszterben tartott `0x40` maszkkal, vagy `AND` a 32 bites `0xFFFFFFBF` immediate-maszkkal.

A `0x80000001` két 1-ese körkörösen szomszédos. A `0x00000005` 1-esei nem szomszédosak, és kisebb elem ismétlésével sem állítható elő a teljes bitminta.

2. megoldás — Segítség. Haladj lépésről lépésre a `0xAC` kezdőértéktől: először a beállítás (`ORR`) — a `0x05` nem logikai immediate, ezért regiszterbe kell tölteni; majd a 7. bit törlése (`AND` a `0xFFFFFF7F` komplementer immediate-tel, vagy `BIC` regiszteres maszkkal); végül az átváltás (`EOR #0x08`) — mindig az ELŐZŐ lépés eredményére építve.

2. megoldás — Teljes megoldás. A `lab_observe` címke elé beilleszthető:

```
// A64 ISA; kiegészítés a teljes lab-mask.s programhoz.
movz    w12, #0x05
orr     w13, w0, w12
and     w13, w13, #0xFFFFFF7F
eor     w13, w13, #0x08
and     w14, w13, #0x0F
```

A levezetett értéksor: `0xAC → 0xAD → 0x2D → 0x25`. Az alsó négy bit `0x05`. A sikeres kilépést előkészítő sor elé kerüljenek az ellenőrzések:

```
// A64 ISA; további ellenőrzések a meglévő lab_fail ággal.
cmp     w13, #0x25
b.ne    lab_fail
cmp     w14, #0x05
b.ne    lab_fail
```

Az eredeti assemble–link–run parancsokat újrafuttatva ismét 0 kilépési állapot várható.

3. megoldás — Segítség. A `0x05` ugyanaz az érték, amiről a fejezet elején már megállapítottuk, hogy nem logikai immediate — az assembler ezt mikor veszi észre: fordításkor vagy futáskor?

3. megoldás — Teljes megoldás. A hibás változat összeszerelési kísérlete:

```
# Target: qemu-userhez készülő AArch64 objektum.
# Szándékos fordítási hiba; nem követi linkelés vagy futtatás.
aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-mask-bad.o lab-mask-bad.s
printf 'assembler_allapot=%d\n' "$?"
```

Nem nulla assemblerállapot várható. GNU as 2.42-vel mérve a diagnosztika: `Error: immediate out of range at operand 3 -- `and w7,w0,#0x05'`, az állapot 1; a pontos szöveg verziófüggő. A hibás sorhoz nem készíthető megfelelő utasításkód, ezért ez összeszerelési, nem futásidejű hiba.

Az operandus visszacserélése után:

```
# Target: qemu-user, AArch64 Linux.
aarch64-linux-gnu-as -EL -march=armv8-a \
    -o lab-mask-bad.o lab-mask-bad.s
aarch64-linux-gnu-ld -EL -e _start \
    -o lab-mask-bad.elf lab-mask-bad.o
qemu-aarch64 ./lab-mask-bad.elf
printf 'kilepesi_allapot=%d\n' "$?"
```

A várt kilépési állapot 0. Az adat és a maszk nem változott; csak a maszk átadásának kódolható módját állítottuk vissza.

9.1.9 Rövid összefoglaló

A64-ben az AND megtartja a maszkkal kijelölt biteket, az ORR beállítja, az EOR átváltja, a BIC pedig törli őket. A helyes választást a kívánt állapotváltozás határozza meg, nem az, hogy egyetlen próbaadaton melyik ad megfelelő eredményt. A logikai immediate csak szabályos, elforgatott és szükség esetén ismételt bitmintát írhat le; egy kis szám sem feltétlenül kódolható. Ilyenkor regiszteres maszk használható. A műveleti szélességet mindig ellenőrizni kell, és az eredményvizsgálat nem támaszkodhat véletlenül megmaradt állapotjelzőkre. Az XOR-csere hasznos azonosság, de nem automatikus gyorsítás.

- **Bitmaszk:** bitpozíciókat kijelölő bitminta.
- **Logikai immediate:** korlátozott mintacsaládból kódolható közvetlen maszk.
- **Átváltás:** kijelölt bitek értékének megfordítása.

9.1.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; logikai utasítások, `DecodeBitMasks`, regiszterszélességek. Ellenőrizve: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 operandusszintaxis, célarchitektúra és direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU objdump kézikönyv — gépi kód visszafejtése és megjelenítése. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a példák a lektor mérési környezetében (GNU as/ld 2.42, QEMU 8.2.2) lefutottak, az itt közölt kimenetek onnan származnak; a hallgató saját Ubuntu 24.04 laborjában a helyi csomagverziókkal is ellenőrizendők. Teljesítménymérés vagy mért ciklusszám nem szerepel az alfejezetben.

9.2 Shift és rotate: LSL/LSR/ASR/ROR gyakorlati mintázatok

9.2.1 Motiváció: bájtokból adat, indexből eltolás

Egy bináris rekord feldolgozásakor gyakran kell biteket más helyi értékre mozgatni: egy bájtot kiemelni, két mezőt összecsomagolni, vagy egy tömbindexből bájteltolást számítani. Ezekhez nem feltétlenül szorzás vagy osztás szükséges. A **biteltolás** (*bit shift*) közvetlenül a bitpozíciókat módosítja; a **bitforgatás** (*bit rotation*) pedig a kieső biteket a másik oldalon visszahelyezi.

Az előfeltétel a Logikai alapműveletek és bitmaszkolás ismerete. Ebben az alfejezetben minden assembly utasítás és regiszter **A64 ISA, AArch64, I. rész** kontextusú. A labor Armv8-A alapprofilú, little-endian Linux-program, amely `qemu-aarch64` alatt fut. A GNU assembler direktívái az ehhez tartozó ELF-eszközlánc részei, nem processzorutasítások.

A fő kérdés nem az, hogy „hogyan kell jobbra tolni”, hanem az, hogy **mit jelentenek a bitek, mi kerül a megüresedő helyekre, és elveszhet-e információ**.

9.2.2 A négy művelet pontos jelentése

Legyen n a műveleti szélesség bitben: A64-ben `Wn` operandusokkal 32, `Xn` operandusokkal 64. Legyen k az eltolás bitszáma, egyelőre $0 \le k < n$.

A64 művelet	Mozgatás	Megüresedő helyek	Kieső bitek
`LSL` — logical shift left	balra	alul nullák	elvesznek
`LSR` — logical shift right	jobbra	felül nullák	elvesznek
`ASR` — arithmetic shift right	jobbra	felül az eredeti előjelbit másolatai	elvesznek
`ROR` — rotate right	körkörösen jobbra	alulról kieső bitek kerülnek felülre	megmaradnak

Az utasítás nem tudja, milyen C-típust szántunk az adatnak. A megfelelő művelet kiválasztása a programozó feladata.

9.2.2.1 Balra tolás: szorzás, rögzített szélességen

A64: `LSL X2, X1, #3` három bitpozícióval balra tolja az `X1` tartalmát. Az előjel nélküli eredmény levezetése:

$$r = (u \cdot 2^k) \bmod 2^n.$$

Itt u és r mértékegység nélküli egész értékek. Minden megmaradó bit helyi értéke 2^k-szorosára nő; az n bites tartományon túli bitek elvesznek.

Ezért 13 három bites balra tolása 104. A szorzás azonban csak akkor pontos, ha nincs eldobott, értékes felső bit. Például 32 bites előjel nélküli olvasatban a `0x80000000` egybites balra tolása nulla, nem pedig egy 33 bites eredmény.

Előjeles kettes komplemens értéknél ugyanaz a bitművelet használható szorzásra, **ha a matematikai eredmény reprezentálható**. Ez nem azonos a C `<<` teljes szabályrendszerével: a laborhoz viszonyításként használt C11/C17-ben negatív előjeles operandus balra tolása nem definiált viselkedés.

9.2.2.2 Logikai jobbra tolás: előjel nélküli osztás

A64: `LSR X2, X1, #3` felül nullákat léptet be. Előjel nélküli olvasatban:

$$r = \left\lfloor \frac{u}{2^k} \right\rfloor .$$

A kieső alsó k bit az osztási maradékot hordozta. Így 107 három bites logikai jobbra tolásának eredménye 13.

Negatív előjeles értékhez ez rendszerint rossz választás: a nullákkal feltöltött felső rész pozitív eredményt hoz létre, nem tartja meg az előjelet.

9.2.2.3 Aritmetikai jobbra tolás: előjelmegőrzés, eltérő kerekítés

A64: `ASR X2, X1, #1` a felső helyeket az eredeti 63. bit másolataival tölti ki. Kettes komplemens, előjeles olvasatban:

$$r = \left\lfloor \frac{s}{2^k} \right\rfloor .$$

Ez **negatív végtelen felé kerekít**, míg a C előjeles egész osztása nulla felé csonkol:

Művelet	Várt, előjeles eredmény
C: `-7 / 2`	-3
A64: -7 aritmetikai jobbra tolása 1 bittel	-4
A64: -8 aritmetikai jobbra tolása 1 bittel	-4

Negatív, pontosan osztható értéknél nincs különbség. Egyébként korrekció kell. $1 \leq k < n$ esetén:

$$q = \left\lfloor \frac{s + \begin{cases} 2^k - 1, & s < 0, \\ 0, & s \geq 0 \end{cases}}{2^k} \right\rfloor .$$

A hozzáadott korrekció (*bias*) a negatív értékek kerekítését nulla felé módosítja. Például $(-7 + 1)/2 = -3$.

A64-ben a 63 bites `ASR` 64 bites operanduson nulla vagy csupa egyes bitmintát állít elő: ez az **előjelmaszk** (*sign mask*). A labor ezt maszkolja egyre, majd hozzáadja az osztandóhoz.

C11/C17-ben negatív előjeles érték jobbra tolása implementációfüggő; ezért a C `/` helyettesítését ne pusztán egy C `>>` kifejezés látványa alapján igazold.

9.2.3 Forgatás és mezőkezelés

A64: a `ROR W7, W6, #8` utasítás 32 bites körön forgat:

Állapot	32 bites bitminta, hexadecimálisan
Bemenet	`0x11223344`
Jobbra forgatás 8 bittel	`0x44112233`
Az eredmény alsó bájtja	`0x33`

Ez bájtok körkörös átrendezése, **nem általános bájtsorrend-megfordítás**. A forgatás önmagában nem emel ki mezőt: a többi bitet is megtartja. A labor ezért utána A64 `AND` művelettel megtartja az alsó bájtot.

Ha kizárólag az eredeti 8–15. bit kell, az A64 `LSR` és maszkolás általában közvetlenebb megoldás. A forgatás akkor hasznos, ha a többi bitet is meg kell őrizni, vagy a mező a szóhatáron körbeforduló elrendezés része.

Két bájt csomagolására például az A64 `ORR W2, W0, W1, LSL #8` alkalmas, **ha mindkét bemeneti regiszterben csak az alsó nyolc bit lehet nem nulla**. Ellenkező esetben előzetes maszkolás szükséges. A speciális mezőutasításokat a Bitfield-utasítások: BFI/UBFX/SBFX és csomagolt adatszerkezetek tárgyalja.

A regiszteren belüli bitpozíciók jelentése nem függ a memória bájtsorrendjétől. A little-endian sorrend akkor válik fontossá, amikor ezeket a regiszterértékeket memóriába írjuk vagy onnan olvassuk.

9.2.4 Kódolási korlátok és beágyazott eltolás

A64 immediate-alakok: mind a négy műveletnél a közvetlen eltolás `Wn` operandusokkal 0–31, `Xn` operandusokkal 0–63. A teljes szélességgel való immediate-eltolás nem kódolható ezekben az alakokban.

A64 regiszteres alakok: az eltolásszám alsó öt, illetve hat bitje számít. A `LSL W2, W1, W3` tehát a `W3` értékét modulo 32 használja. Ha `W3` értéke 32, az effektív eltolás nulla, nem 32.

Ez lényegesen eltér a C-től: ott a negatív, illetve az egészérték-promóció utáni bal operandus szélességét elérő eltolásszám nem definiált viselkedést okoz.

A64 szélességi szabály: a `Wn` célregiszter írása a megfelelő `Xn` felső 32 bitjét nullázza. Egy 32 bites `ASR` tehát nem végez automatikus, 64 bitre történő előjel-kiterjesztést.

A beágyazott eltolás (*shifted operand*) egy másik utasítás bemenetét módosítja. Például A64-ben:

```
ADD X12, X10, X11, LSL #3
```

jelentése: `X12 = X10 + 8 × X11`, 64 bites moduláris aritmetikával. Nyolcbájtos tömbelemnél ez az indexből származó eltolás hozzáadása az alaphoz.

Az A64 `ADD` eltolt regiszteres alakja `LSL`, `LSR` vagy `ASR` módot fogad, `ROR` módot nem. A tartomány itt is 0–31 vagy 0–63. Az A64 logikai utasítások, például az `ORR`, eltolt regiszteres alakjában `ROR` is megengedett.

Kevesebb utasítás nem jelent garantáltan kisebb késleltetést: ez mikroarchitektúra- és függőségfüggő kérdés. QEMU alatt nem állapítható meg hardverhű abszolút ciklusszám.

Végül: az A64 `LSL`, `LSR`, `ASR` és `ROR` **nem módosítja az NZCV állapotjelzőket**, és nem teszi a kieső bitet a C jelzőbe.

9.2.5 Kidolgozott példa: önellenőrző shift-labor

Munkakönyvtár: `~/arm-labor/a64/a033/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a033
cd ~/arm-labor/a64/a033
```

A `lab-shift.s` hét ellenőrzési csoportot tartalmaz. Siker esetén nulla, hibánál az érintett csoport száma a kilépési állapot. Nincs szöveges programkimenet.

A64 ISA, Armv8-A; GNU assembler, AArch64 ELF:

```
.arch armv8-a
.text
.global _start
.type _start, %function

_start:
    movz    w0, #1
    movz    x1, #13
    lsl     x2, x1, #3
    cmp     x2, #104
    b.ne    lab_exit

    movz    w0, #2
    lsr     x6, x2, #3
    cmp     x6, x1
    b.ne    lab_exit

    movz    w0, #3
    movn    x3, #6              // x3 = ~6 = -7
    asr     x4, x3, #1
    cmn     x4, #4              // -4 + 4 == 0?
    b.ne    lab_exit

    movz    w0, #4
    asr     x5, x3, #63         // előjelmaszk
    and     x5, x5, #1          // negatívhoz 1, különben 0
    add     x5, x3, x5
    asr     x5, x5, #1
    cmn     x5, #3              // -3 + 3 == 0?
    b.ne    lab_exit

    movz    w0, #5
    movz    w6, #0x3344
    movk    w6, #0x1122, lsl #16
    ror     w7, w6, #8
    movz    w15, #0x2233
    movk    w15, #0x4411, lsl #16
    cmp     w7, w15
    b.ne    lab_exit
    and     w9, w7, #0xff
    cmp     w9, #0x33
    b.ne    lab_exit

    movz    w0, #6
    movz    x10, #100
    movz    x11, #7
    add     x12, x10, x11, lsl #3
    cmp     x12, #156
    b.ne    lab_exit
```

```
    movz    w0, #7
    movz    w13, #32
    lsl     w14, w1, w13
    cmp     w14, w1
    b.ne    lab_exit

lab_done:
    movz    x0, #0

lab_exit:
    movz    x8, #93             // Linux AArch64: exit
    svc     #0

.size _start, .-_start
.section .note.GNU-stack,"",%progbits
```

A 100 itt **egész tesztadat, nem memóriacím**; a program nem dereferálja.

A kiegészítő A64 immediate-ok is kódolhatók: a `MOVZ`/`MOVN`/`MOVK` 16 bites mezőt fogad; `Wn` esetén a fél szó eltolása 0 vagy 16, `Xn` esetén 0, 16, 32 vagy 48. A `CMP`/`CMN` itt használt konstansai belefférnek az eltolatlan 12 bites immediate-ba; ezek az alakok opcionális 12 bites balra tolást is engednek. Az `AND` maszkjai összefüggő egyesekből álló, kódolható logikai immediate-ok. Az `SVC` immediate-mezője 16 bites.

Build és futtatás — Ubuntu 24.04 x86_64 host, cél: AArch64 qemu-user, little-endian:

```
aarch64-linux-gnu-as -march=armv8-a -EL -g \
    -o lab-shift.o lab-shift.s
aarch64-linux-gnu-ld -EL -e _start \
    -o lab-shift lab-shift.o
qemu-aarch64 ./lab-shift
printf 'Kilépési állapot: %d\n' "$?"
```

Mérve (a lektor mérése, GNU binutils 2.42, QEMU 8.2.2): `Kilépési állapot: 0`.

A `lab_done` címkénél várható A64-regiszterállapot:

Regiszter	Várt tartalom	Olvasat
`X2`	104	előjel nélküli
`X3`	`0xfffffffffffffff9`	előjeles: -7
`X4`	`0xfffffffffffffffc`	előjeles: -4
`X5`	`0xfffffffffffffffd`	előjeles: -3
`W6`	`0x11223344`	bitminta
`W7`	`0x44112233`	bitminta
`W9`	`0x33`	előjel nélküli bájtérték
`X12`	156	előjel nélküli
`W14`	13	effektív eltolás: nulla

9.2.6 Hibák és ellenpróbák

A hibakeresés menete: **tünet → hipotézis → adatgyűjtés → egy változtatás → új ellenőrzés**.

- **Negatív osztásnál egyesével eltérő eredmény:** valószínűleg hiányzik a korrekció. Próbáld az előjeles -8, -7, 0 és 7 értékeket; a puszta `ASR` hibája csak bizonyos negatív bemeneteken látszik.

- **Szélességnyi eltolás után változatlan érték:** regiszteres eltolásnál a számláló maszkolása lehet az ok. Ellenőrizd a számlálót és a `Wn`/`Xn` szélességet.
- **Hibás feltételes ugrás:** a shift nem állította az NZCV-t. Tegyél közvetlen eredmény-összehasonlítást az elágazás elé.
- **Váratlan mnemonic a visszafejtésben:** az immediate-alakok álnevek (*aliases*): A64 `LSL`/`LSR` mögött `UBFM`, `ASR` mögött `SBFM`, `ROR` mögött `EXTR` állhat; a regiszteres alakok mögött pedig `LSLV`, `LSRV`, `ASRV` és `RORV`. Ez nem fordítási hiba.

A qemu-user célprogram visszafejtése; a parancsok várt kilépési állapota 0:

```
aarch64-linux-gnu-objdump -d lab-shift
aarch64-linux-gnu-objdump -d -M no-aliases lab-shift
```

Regiszterellenőrzés, cél: qemu-user. Első terminál:

```
qemu-aarch64 -g 1234 ./lab-shift
```

Második terminál; siker esetén a fenti táblázat állapota, majd normál kilépés várható:

```
gdb-multiarch -q ./lab-shift \
    -ex 'target remote localhost:1234' \
    -ex 'break lab_done' \
    -ex 'continue' \
    -ex 'info registers x2 x3 x4 x5 x6 x7 x9 x12 x14' \
    -ex 'continue' \
    -ex 'quit'
```

Hibás változatnál a `lab_done` nem feltétlenül érhető el: ilyenkor a töréspontot `lab_exit` címkére helyezd.

9.2.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Számítsd ki 32 bites A64 műveletekkel a `0x80000001` egybites `LSL`, `LSR`, `ASR` és `ROR` eredményét. Miért egyezik két eredmény, és miért nem általános ez?

2. (implementációs, középszint — cél: rutinépítés) Módosítsd a labor negyedik csoportját úgy, hogy az előjeles `X3` értéket nyolccal ossza, a C nulla felé csonkolásával. Tesztelj -9, -8, -7, 0, 7 és 8 bemenetet.

3. (hibakeresési, középszint — cél: rutinépítés) A hetedik csoportban kizárólag a számlálót változtasd 32-ről 31-re. Mi lesz a kilépési állapot és a `W14` tartalma? Ezután külön ellenpróbaként próbálj immediate `LSL W14, W1, #32` utasítást összeszerelni.

9.2.8 Megoldások

1. megoldás — Segítség. A `0x80000001` legfelső és legalsó bitje is 1 — melyik két művelet viselkedik ekkor azonosan, és milyen bemenetnél derülne ki a különbség?

1. megoldás — Teljes megoldás. A64, 32 bites eredmények:

Művelet	Eredmény
`LSL #1`	`0x00000002`
`LSR #1`	`0x40000000`

Művelet	Eredmény
`ASR #1`	`0xc0000000`
`ROR #1`	`0xc0000000`

Az utolsó kettő azért egyezik, mert az eredeti előjelbit és az alsó bit egyaránt egy. A `0x80000000` ellenpélda: aritmetikai jobbra tolása `0xc0000000`, forgatása `0x40000000`.

2. megoldás — Segítség. Az `ASR`-rel történő osztás lefelé kerekít, nem nulla felé — negatív bemenetnél szükség van egy korrekciós tagra, mielőtt az eltolás megtörténne. Gondold át, mekkora korrekció kell `-9`-nél és mekkora `0`-nál.

2. megoldás — Teljes megoldás. A labor negyedik csoportjának számítási részét cseréld erre. **A64 ISA; a meglévő teljes laborba illesztendő:**

```
asr     x5, x3, #63
and     x5, x5, #7
add     x5, x3, x5
asr     x5, x5, #3
```

A `#7` kódolható logikai immediate. A korrekció negatív bemenetnél hét, különben nulla. A várt előjeles eredmények rendre -1, -1, 0, 0, 0, 1. A csoport összehasonlítását is az adott várt értékhez kell igazítani; a build- és futtatási parancsok változatlanok.

3. megoldás — Segítség. Egy 32 bites regiszterben 31 bites balra tolás után hány bit marad az eredeti mintából? Az immediate eltolásmező hány bitet tud kódolni?

3. megoldás — Teljes megoldás. Mérve (a lektor mérése, GNU as 2.42, QEMU 8.2.2): kilépési állapot 7, `W14 = 0x80000000`. A 13 bitmintájából 31 bites balra toláskor csak a legalacsonyabb egyes bit marad meg a felső pozícióban. Az immediate `#32` viszont 32 bites alakban nem kódolható: GNU as 2.42-vel a diagnosztika `Error: immediate value out of range 0 to 31 at operand 3`, és az assemble-lépés sikertelen. A regiszteres számláló maszkolása nem vonatkozik az immediate szintaxisára.

9.2.9 Rövid összefoglaló

A bitmozgatás helyességét a műveleti szélesség, az előjeles vagy előjel nélküli értelmezés és a kieső bitek szerepe határozza meg. A balra tolás moduláris szorzás, a logikai jobbra tolás előjel nélküli osztás. Az aritmetikai jobbra tolás megőrzi az előjelet, de negatív értékeknél eltérhet a C osztásának kerekítésétől. A forgatás nem veszít biteket, ezért mezőkiemeléskor maszkolás is kellhet. A beágyazott eltolás egyszerűsítheti a címzési és csomagolási számításokat, gyorsulását azonban külön kell mérni. Mindig ellenőrizd a szélső eltolásszámokat.

- **Logikai eltolás:** nullákat beléptető bitmozgatás.
- **Aritmetikai jobbra tolás:** az eredeti előjelbitet másoló eltolás.
- **Forgatás:** a kieső biteket a másik oldalon visszahelyező bitmozgatás.

9.2.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487; A64 eltolások, forgatások és eltolt regiszteres operandusok. Ellenőrizve: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 operandusszintaxis és direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU objdump kézikönyv — visszafejtés és disassembler-beállítások. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a program a lektor mérési környezetében (GNU as/ld 2.42, QEMU 8.2.2) lefutott, az itt közölt kimenet és az assembler-üzenet onnan származik. A hallgató saját Ubuntu 24.04 laborjában a helyi csomagverziókkal és az álnevek tényleges megjelenítésével is ellenőrizze. Teljesítménymérés vagy mért ciklusszám nem szerepel az alfejezetben.

9.3 Bitfield-utasítások: BFI/UBFX/SBFX és csomagolt adatszerkezetek

9.3.1 Miért érdemes külön utasításokkal kezelni a bitmezőket?

Egy állapotszó gyakran több, egymástól független adatot hordoz: néhány bit üzemmódot, más bitek előjeles korrekciót, további bitek számlálót jelentenek. C-ben ezeket eltolásokkal és maszkolással vagy struktúrába szervezett bitmezőkkel kezelhetjük. Assemblyben a feladat két alapműveletre egyszerűsödik:

- egy mező kinyerése megfelelő előjelkezeléssel;
- egy mező felülírása a többi bit megőrzésével.

A **bitmező** (*bit field*) egy rögzített szélességű érték összefüggő bittartománya. A legkisebb helyi értékű bit sorszáma `0`. Egy mezőt kezdőpozíciója és bitszélessége határoz meg.

Az előfeltétel a Shift és rotate: LSL/LSR/ASR/ROR gyakorlati mintázatok ismerete. Itt már nem általános bitmozgatásokat építünk fel, hanem azok gyakori összetett mintáit fejezzük ki közvetlenül.

Architektúra és profil: az alfejezet minden assembly utasítása és regisztere az **A64 ISA-hoz, az AArch64 I. részhez** tartozik. A cél Armv8-A, Linux, little-endian. A példák nem Cortex-M-perifériákat érnek el.

9.3.2 A három alapművelet pontos szerződése

Az A64 utasítások szintaxisa:

A64 utasítás	Jelentés
`UBFX Wd, Wn, #lsb, #width`	Mezőkinyerés, nullakiterjesztéssel 32 bitre
`SBFX Wd, Wn, #lsb, #width`	Mezőkinyerés, előjelkiterjesztéssel 32 bitre
`BFI Wd, Wn, #lsb, #width`	A forrás alsó bitjeinek beillesztése a célmezőbe

Mindháromnak van 64 bites, `Xd` és `Xn` operandusokat használó változata. Egy utasításon belül az adatoperandusok szélessége azonos.

Legyen:

- N az operandus szélessége, bitben: `32` vagy `64`;
- p a mező kezdőpozíciója, bitsorszámként;
- k a mező szélessége, bitben.

Az immediate operandusok megengedett tartománya:

$$0 \le p < N, \qquad 1 \le k \le N - p.$$

A mező tehát nem lóghat túl a regiszter felső szélén. A pozíció és a szélesség **fordításkor ismert konstans**, nem regiszterből vett futásidejű paraméter. Például a 32 bites `UBFX W0, W1, #28, #5` érvénytelen: a mező utolsó bitje a nem létező `32` sorszámú bit lenne. A GNU as 2.42 ezt mérve `Error: immediate value out of range 1 to 4 at operand 4` üzenettel utasítja el.

9.3.2.1 Előjel nélküli kinyerés: UBFX

Az A64 `UBFX` a kiválasztott mezőt a cél alsó bitjeire helyezi, a cél többi bitjét nullázza.

Levezetés, előjel nélküli értelmezéssel: ha S a forrásregiszter értéke, akkor

$$u = (S \operatorname{LSR} p) \,\&\, (2^k - 1).$$

Az eredmény tartománya $0 \dots 2^k - 1$. Az eltolás eltávolítja a mező alatti biteket, a maszk pedig a mező feletti biteket.

Az A64 `UBFX W0, W0, #8, #8` tehát a forrás `15:8` bitjeit emeli ki. Az azonos forrás- és célregiszter megengedett.

9.3.2.2 Előjeles kinyerés: SBFX

Az A64 `SBFX` ugyanazt a bittartományt választja ki, de annak legfelső bitjét előjelbitként kezeli. Az **előjelkiterjesztés** (*sign extension*) ezt a bitet másolja a cél magasabb bitjeire.

Ha az előjel nélküli mezőérték u, akkor a kettes komplemens szerinti előjeles érték:

$$s = \begin{cases} u, & u < 2^{k-1}, \\ u - 2^k, & u \ge 2^{k-1}. \end{cases}$$

Az eredmény tartománya $-2^{k-1} \dots 2^{k-1} - 1$. Az előjelet **a mező legfelső bitje**, nem a teljes forrásregiszter legfelső bitje határozza meg.

Fontos A64-szélességi szabály: az `SBFX W0, ...` csak 32 bitre terjeszti ki az előjelet. A `W0` írása az `X0` felső 32 bitjét nullázza.

A64 művelet eredménye	`W0` bitmintája	`X0` bitmintája
32 bites előjeles `-3`	`0xFFFFFFFD`	`0x00000000FFFFFFFD`

Ha eleve 64 bites, előjeles eredmény kell, a megfelelő 64 bites forráson az `SBFX X0, X0, #3, #5` változatot használhatjuk. Ennek `-3` eredménye `0xFFFFFFFFFFFFFFFD`.

9.3.2.3 Beillesztés: BFI

Az A64 `BFI` a forrás **alsó** k **bitjét** másolja a cél `p+k-1:p` tartományába. A cél eredeti értéke tehát bemenet is.

Legyen a célmező maszkja:

$$M = (2^k - 1) \ll p.$$

Ekkor, N bites műveletként értelmezve:

$$D' = (D \mathbin{\&} \neg_N M) \mid ((S \mathbin{\&} (2^k - 1)) \ll p).$$

Itt $\neg_N$ pontosan N biten képzett komplementert jelent. Az első tag megőrzi a célmezőn kívüli biteket, a második a helyükre teszi az új mezőértéket.

Az A64 `BFI W0, W1, #8, #8`:

- a `W1` alsó 8 bitjét használja;
- a `W0 15:8` bitjeit felülírja;
- a `W0` többi bitjét megőrzi.

A túl nagy bemenetet nem utasítja el: **csonkolja**. A `0x123` forrásból 8 bites mezőbe `0x23` kerül.

A megőrzési ígéret itt csak a 32 bites célra vonatkozik: a `W0` írása továbbra is nullázza az `X0` felső felét.

9.3.2.4 Kódolási háttér és állapotjelzők

Az A64 `UBFX`, `SBFX` és `BFI` rendre az `UBFM`, `SBFM` és `BFM` utasítások alternatív nevei, azaz *aliasai*. Nem több utasításra kibontott makrók.

A kódolt immediate mezők:

A64 alias	`immr`	`imms`
`UBFX`, `SBFX`	p	$p + k - 1$
`BFI`	$(-p) \bmod N$	$k - 1$

A kódolás mindkét immediate mezője 6 bites; 32 bites alaknál csak a megfelelő 5 bites értéktartomány érvényes. Az assembler az olvasható pozíció–szélesség párból állítja elő ezeket.

E három művelet nem módosítja az A64 `NZCV` állapotjelzőket. A disassembler más aliast is megjeleníthet, különösen szélső paramétereknél; ez önmagában nem jelent eltérő működést.

9.3.3 Kidolgozott példa: csomagolt állapotszó

Munkakönyvtár: `~/arm-labor/a64/a034/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a034
cd ~/arm-labor/a64/a034
```

Egy programon belüli, 32 bites állapotszó formátuma legyen:

Bittartomány	Szélesség	Mező	Értelmezés
`2:0`	3 bit	`mode`	Előjel nélküli üzemmód
`7:3`	5 bit	`delta`	Előjeles korrekció
`15:8`	8 bit	`count`	Előjel nélküli számláló
`31:16`	16 bit	`tag`	Előjel nélküli azonosító

A bemenet `0xA55A3CED`.

Levezetett mezőértékek:

- `mode`: `0b101`, előjel nélkül `5`;
- `delta`: `0b11101`, előjel nélkül `29`, előjelesen $29 - 32 = -3$;
- `count`: `0x3C`, előjel nélkül `60`;
- `tag`: `0xA55A`, előjel nélkül értelmezett azonosító.

A `count` mezőbe `0x123` értéket illesztünk. A várt eredmény `0xA55A23ED`: csak a mező alsó 8 bitnyi új értéke kerül be.

9.3.3.1 Assembly modul

A `lab-bitfield.S` fájl teljes tartalma:

```
// A64 ISA, AArch64, Armv8-A.
// A direktívák GNU as / ELF környezethez tartoznak.
.arch armv8-a
.text
.p2align 2

.global lab_count
.type lab_count, %function
lab_count:
    UBFX w0, w0, #8, #8
    ret x30
.size lab_count, .-lab_count

.global lab_delta
.type lab_delta, %function
lab_delta:
    SBFX w0, w0, #3, #5
    ret x30
.size lab_delta, .-lab_delta

.global lab_put_count
.type lab_put_count, %function
lab_put_count:
    BFI w0, w1, #8, #8
    ret x30
```

```
.size lab_put_count, .-lab_put_count

.section .note.GNU-stack,"",%progbits
```

Az AAPCS64 szerint a 32 bites argumentumok itt a `W0`, illetve `W1` regiszterben érkeznek, az eredmény a `W0` regiszterben távozik. A függvények nem hívnak más függvényt, és nem használnak megőrzendő regisztert, ezért nincs szükség veremkeretre.

9.3.3.2 C bitmezők és reprezentáció

A C bitmezős struktúra nem hordozható fájl- vagy perifériaformátum-leírás. A mezők tárolóegységen belüli elhelyezése, a kitöltés és bizonyos típusválasztási részletek implementációfüggők; a konkrét ABI további szabályokat ad.

Különösen:

- előjeles mezőhöz írjunk explicit `signed int` típust;
- a puszta `int` bitmező előjelességére ne építsünk hordozható szerződést;
- bitmező címét nem lehet közönséges mutatóként elkérni;
- a `packed` attribútum önmagában nem teszi hordozhatóvá a bitkiosztást.

A labor az AArch64 GNU/Linux little-endian profil várható kiosztását **ellenőrzi**, nem általános C-szabályként feltételezi. A reprezentációt `memcpy` segítségével vizsgáljuk, nem inkompatibilis mutatótípuson keresztül.

A `lab-bitfield.c` fájl:

```c
/* Cel: AArch64 GNU/Linux, Armv8-A, little-endian. */
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

uint32_t lab_count(uint32_t word);
int32_t lab_delta(uint32_t word);
uint32_t lab_put_count(uint32_t word, uint32_t value);

struct lab_fields {
    unsigned int mode : 3;
    signed int delta : 5;
    unsigned int count : 8;
    unsigned int tag : 16;
};

_Static_assert(sizeof(struct lab_fields) == 4,
               "A labor 4 bajtos strukturakiosztast var");

uint32_t lab_c_count(const struct lab_fields *p)
{
    return p->count;
}

int32_t lab_c_delta(const struct lab_fields *p)
{
    return p->delta;
}

void lab_c_set_count(struct lab_fields *p, uint32_t value)
{
    p->count = value;
```

```
}

uint32_t lab_c_put_count_word(uint32_t word, uint32_t value)
{
    return (word & UINT32_C(0xFFFF00FF))
         | ((value & UINT32_C(0xFF)) << 8);
}

int main(void)
{
    const uint32_t word = UINT32_C(0xA55A3CED);
    const uint32_t value = UINT32_C(0x123);
    const uint32_t updated = lab_put_count(word, value);

    if (lab_count(word) != 60 || lab_delta(word) != -3)
        return 1;

    if (updated != UINT32_C(0xA55A23ED) ||
        ((updated ^ word) & UINT32_C(0xFFFF00FF)) != 0)
        return 2;

    struct lab_fields fields = {
        .mode = 5, .delta = -3, .count = 0x3C, .tag = 0xA55A
    };
    uint32_t raw;
    memcpy(&raw, &fields, sizeof raw);

    if (raw != word ||
        lab_c_count(&fields) != 60 || lab_c_delta(&fields) != -3)
        return 3;

    lab_c_set_count(&fields, value);
    memcpy(&raw, &fields, sizeof raw);

    if (raw != updated ||
        lab_c_put_count_word(word, value) != updated)
        return 4;

    printf("count=%" PRIu32 " delta=%" PRId32
           " updated=0x%08" PRIx32 "\n",
           lab_count(word), lab_delta(word), updated);
    return 0;
}
```

A C-szintű maszkolás itt biztonságos: előjel nélküli, 32 bites értékeken dolgozik, és az eltolásszám `8`. Általános segédfüggvényben az `(1u << width) - 1` kifejezés `width == 32` esetén már hibás lenne; a teljes szélességű mezőt külön kell kezelni.

9.3.3.3 Assemble–link–run

Az alábbi parancsok célja **qemu-user, AArch64**. A statikus linkeléshez a cross környezet céloldali C fejlesztői könyvtárai is szükségesek.

```
# Target: qemu-user / AArch64, Armv8-A, little-endian.
aarch64-linux-gnu-gcc -march=armv8-a -g \
    -c lab-bitfield.S -o lab-bitfield.o

aarch64-linux-gnu-gcc -std=c11 -O2 -g -Wall -Wextra \
    -march=armv8-a -mlittle-endian \
    -c lab-bitfield.c -o lab-bitfield-c.o
```

```
aarch64-linux-gnu-gcc -static -no-pie \
    lab-bitfield-c.o lab-bitfield.o -o lab-bitfield

qemu-aarch64 ./lab-bitfield
printf 'exit=%d\n' "$?"

# Az A64 kodolas es a C-bol generalt kod vizsgalata.
aarch64-linux-gnu-objdump -d -M no-aliases lab-bitfield.o
aarch64-linux-gnu-objdump -d lab-bitfield-c.o
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2):
`count=60 delta=-3 updated=0xa55a23ed`, majd `exit=0`.

A nem nulla kilépési állapot jelentése:

Állapot	Elsődleges vizsgálati irány
1	Kinyerési pozíció, szélesség vagy előjelkezelés
2	Beillesztés és a többi bit megőrzése
3	C-struktúra kezdeti kiosztása vagy kiolvasása
4	C-mezőírás és assembly eredményének eltérése

Little-endian memóriában a bemeneti szó bájtjai, növekvő címek sorrendjében: `0xED`, `0x3C`, `0x5A`, `0xA5`. Ez nem változtatja meg a regiszteren belüli bitsorszámokat.

9.3.4 Mit várhatunk a C-fordítótól?

A forráskód szerkezete nem ír elő egyetlen kötelező utasítássorozatot.

Az A64 célra fordított `lab_c_delta` esetében természetes minta egy betöltést követő `SBFX`. A `count` azonban bájthatárra esik, ezért a fordító közvetlen `LDRB` betöltést vagy `STRB` tárolást is választhat: ilyenkor nincs szükség külön regiszteres mezőműveletre.

A `lab_c_put_count_word` ezzel szemben már regiszterben érkező egész értéket módosít; kézenfekvő A64 megvalósítása a `BFI`. A konkrét GCC-verzió választását az objektumkód mutatja meg. GCC 13.3.0 `-O2` mellett (a lektor mérése) a minta pontosan ez: `lab_c_count` → `ldrb w0, [x0, #1]`, `lab_c_set_count` → `strb`, `lab_c_delta` → `sbfx x0, x0, #3, #5`, `lab_c_put_count_word` → `bfi w0, w1, #8, #8`.

A helyes ellenőrzési kérdés tehát nem az, hogy „megjelent-e a kívánt mnemonic”, hanem hogy **azonos-e a kinyert érték, az előjel és a megőrzött bittartomány**. Az egyutasításos alakból önmagában nem következik konkrét ciklusszám vagy gyorsulás; QEMU-val itt funkcionális helyességet vizsgálunk.

9.3.5 Kapcsolódás perifériaregiszterekhez

A példánk perifériastílusú állapotszó, de kizárólag normál programadat. A valódi perifériaregiszter kezelése további szerződést igényel, amelyet a Memory-mapped I/O alapelve tárgyal.

Az A64 `BFI` csak CPU-regisztert módosít. A memóriából betöltés, módosítás és visszaírás együtt **olvasás–módosítás–írás** (*read-modify-write*) műveletsor; nem válik atomivá attól, hogy középen egyetlen utasítás áll.

Perifériánál egy visszaírt `1` akár állapotbitet törölhet, az olvasásnak pedig mellékhatása lehet. A fenntartott biteket és a megengedett hozzáférési szélességet is a periféria dokumentációja

szabályozza. A C `volatile` minősítő önmagában sem az atomiságot, sem a megfelelő bitmező-hozzáférést nem garantálja.

9.3.6 Tipikus hibák és ellenpróbák

Tünet: `-3` helyett `29`.
Hipotézis: az előjeles mezőt A64 `UBFX` emeli ki. Az `objdump` kimenetében ellenőrizzük az utasítást és a `#3, #5` operanduspárt. Egyetlen változtatásként cseréljük `SBFX` alakra, majd futtassuk újra a tesztet.

Tünet: a számláló helyes, az azonosító mégis megváltozik.
Hipotézis: túl széles a célmező. A mező visszaolvasása önmagában elégtelen ellenpróba: a mezőn kívüli biteket is össze kell hasonlítani.

Tünet: egy korábbi 64 bites érték felső fele eltűnik.
Hipotézis: A64 `W` célregisztert használó művelet történt. A debuggerben az `X` nézetet is vizsgáljuk; a szélességválasztást javítsuk, ne a bitmaszkot.

Tünet: néhány negatív érték jó, mások hibásak.
Hipotézis: rossz a mezőszélesség. A `-3` véletlenül 4 és 5 bites előjeles mezőként is helyesen kijöhet. Ellenpróbának az 5 bites tartomány széleit, `-16` és `15` értéket használjuk.

9.3.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért ad a `0xA55A3CED` szó `7:3` mezőjére az A64 `UBFX 29`, az `SBFX` pedig `-3` értéket? Mi lesz az `X0` teljes bitmintája, ha az eredményt `SBFX W0, ...` állítja elő?

2. (implementációs, középszint — cél: rutinépítés) Készíts `lab_put_delta(uint32_t word, int32_t value)` függvényt, amely a `7:3` mezőt írja át. Teszteld az összes `-16` és `15` közötti értékkel; ellenőrizd a visszaolvasást és a többi bit megőrzését is.

3. (hibakeresési, középszint — cél: rutinépítés) A `lab_put_count` A64 `BFI` utasításában változtasd a szélességet `#8` helyett `#9` értékre. Jósolj eredményt, majd debuggerrel vizsgáld meg a beillesztés előtti és utáni regiszterállapotot. Miért fogadja el ezt az assembler?

9.3.8 Megoldások

1. megoldás — Segítség. Nézd meg a mező legfelső bitjét (a `7:3` mező 5 bites, tehát a 4. relatív bit) — ha ez `1`, az `SBFX` kettes komplemens előjelbitként kezeli.

1. megoldás — Teljes megoldás. A mező bitmintája `0b11101`, előjel nélkül `29`. Öt biten az előjelbit értéke `1`, ezért az előjeles érték $29 - 2^5 = -3$. Az A64 `SBFX W0, ...` eredménye `W0 = 0xFFFFFFFD`, de `X0 = 0x00000000FFFFFFFD`, mert a 32 bites célírás nullázza a felső 32 bitet.

2. megoldás — Segítség. A `lab_put_count` mintáját egyszerűen másolhatod: csak a pozíció és a szélesség immediate paramétere változik `#8,#8`-ról `#3,#5`-re.

2. megoldás — Teljes megoldás. Az assembly fájlhoz hozzáadható:

```
// A64 ISA; GNU as / ELF direktivak.
.text
.global lab_put_delta
.type lab_put_delta, %function
lab_put_delta:
    BFI w0, w1, #3, #5
    ret x30
.size lab_put_delta, .-lab_put_delta
```

A C-fájl deklarációihoz:

```
/* Cel: AArch64 GNU/Linux. */
uint32_t lab_put_delta(uint32_t word, int32_t value);
```

A `main` függvénybe, a kiírás elé:

```
/* Cel: AArch64 GNU/Linux; a main tovabbi ellenorzese. */
for (int32_t v = -16; v <= 15; ++v) {
    uint32_t changed = lab_put_delta(word, v);
    if (lab_delta(changed) != v)
        return 5;
    if (((changed ^ word) & UINT32_C(0xFFFFFF07)) != 0)
        return 6;
}
```

Az új mező maszkja `0x000000F8`, ennek 32 bites komplementere `0xFFFFFF07`. A negatív bemenet alsó öt bitje a kívánt kettes komplemens mezőkód. Az eredeti buildlépések megismétlésével a várt kilépési állapot továbbra is `0`. A tartományon kívüli bemeneteket ez az interfész csonkolná; az elutasításhoz külön ellenőrzés kellene.

3. megoldás — Segítség. Számold ki, melyik bitre esik a `#8, #9` mező teteje (`8+9-1`), és nézd meg, hogy a `0x123` forrás kilencedik bitje (index 8) nulla vagy egy.

3. megoldás — Teljes megoldás. A hibás A64 művelet a `16:8` biteket írja. A `0x123` kilencedik bitje `1`, ezért az eredmény **`0xA55B23ED`** lesz: az azonosító egyik bitje is megváltozik. A program várt kilépési állapota `2`. A mező kódolható, mert `8 + 9 <= 32`; a hiba szemantikai, nem szintaktikai.

Újraépítés után az első terminálban:

```
# Target: qemu-user / AArch64; varakozas GDB-kapcsolatra.
qemu-aarch64 -g 1234 ./lab-bitfield
```

A második terminálban:

```
# Target: a QEMU altal futtatott AArch64 program.
gdb-multiarch -q ./lab-bitfield \
    -ex 'target remote :1234' \
    -ex 'break lab_put_count' \
    -ex 'continue' \
    -ex 'x/2i $pc' \
    -ex 'info registers x0 x1' \
    -ex 'si' \
    -ex 'info registers x0 x1'
```

A léptetés után az A64 `X0` várt értéke `0x00000000A55B23ED`. Állítsuk vissza kizárólag a szélességet `#8` értékre, építsünk újra, és ellenőrizzük a `0` kilépési állapotot.

9.3.9 Rövid összefoglaló

Az A64 `UBFX` előjel nélküli, az `SBFX` előjeles mezőt emel ki, a `BFI` pedig a forrás alsó bitjeivel helyettesíti a cél kijelölt tartományát. A pozíció és a szélesség immediate operandus, és a mezőnek bele kell férnie a választott regiszterszélességbe. A 32 bites célírás a kapcsolódó 64 bites regiszter felső felét nullázza. A C bitmezős struktúrák kiosztását az implementáció és az ABI határozza meg; rögzített külső formátumhoz explicit szerződés szükséges. Teszteléskor a kinyert érték mellett a megőrzendő biteket és a szélső értékeket is ellenőrizzük.

- **Bitmező:** összefüggő bittartomány.
- **Előjelkiterjesztés:** a mező előjelbitjének másolása a magasabb bitekre.
- **Mezőbeillesztés:** kijelölt célbitek helyettesítése a többi célbit megőrzésével.

9.3.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — DDI0487, A64 bitmező-utasítások és aliasok. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64 szintaxis és assemblerdirektívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- AAPCS64 — argumentumátadás és bitmezők adatelrendezése. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta státuszúak).
- GCC: struktúrák, unionok, felsorolások és bitmezők implementációfüggő tulajdonságai — a C bitmezők implementációs döntései. **Tartalmilag ellenőrizve** (a gcc.gnu.org/onlinedocs/ főoldalon keresztül): a legújabb listázott kiadás GCC 16.2.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a program a lektor mérési környezetében (GCC 13.3.0, QEMU 8.2.2) lefutott; az itt közölt kimenet és a fordítói utasításválasztás onnan származik. A hallgató saját Ubuntu 24.04 laborjában a helyi csomagverziókkal és a struktúrakiosztási teszttel is ellenőrizze.

9.4 Gyakorlati bitmanipulációs feladatok: popcount, parity, bitfordítás

9.4.1 Három művelet, három különböző kérdés

Egy jogosultságokat tároló bitmaszkból szeretnénk megtudni, hány engedély aktív. Egy adatátviteli kerethez egyszerű hibajelző bitet számítunk. Egy algoritmus pedig a bemenet bitjeit fordított sorrendben várja. Mindhárom feladat bitmanipuláció, de eltérő eredményt és eltérő implementációt igényel.

Ebben az alfejezetben **32 bites, előjel nélküli értékeken** dolgozunk. A profil **A64 ISA, Armv8-A, little-endian Linux**; minden alábbi assemblyutasítás és regiszter ebben a környezetben értendő. Az előfeltétel a bitmezőkezelés, valamint a korábban megismert elágazás és függvényhívási szerződés.

Legyen x_i az előjel nélküli x érték i indexű bitje; a legkisebb helyi értékű bit indexe 0.

Művelet	Definíció	Eredménytartomány
Beállított bitek száma (*population count*, röviden *popcount*)	$\sum_{i=0}^{31} x_i$	0–32
Bitparitás (*bit parity*)	$x_0 \oplus x_1 \oplus \cdots \oplus x_{31}$	0 vagy 1
Bitfordítás (*bit reversal*)	Az eredmény $31 - i$ indexű bitje x_i	32 bites előjel nélküli érték

A paritás itt 1, ha páratlan számú bit van beállítva. Ez a bit egy adatmezőhöz hozzáfűzve páros összparitást ad.

9.4.2 Popcount: mindig egyetlen beállított bitet törlünk

A popcount hasznos bitkészletek elemszámának meghatározására. Két azonos szélességű bitsorozat eltérő pozícióinak száma, a **Hamming-távolság** (*Hamming distance*), az XOR-eredmény popcountja.

A legegyszerűbb algoritmus 32 alkalommal megvizsgálja az alsó bitet, majd jobbra tol. Ritka bitmaszknál azonban célszerűbb közvetlenül a beállított biteket eltávolítani:

```
x = x & (x - 1)
```

Levezetés. Ha $x \neq 0$, a legalacsonyabb beállított bit alatti bitek mind nullák. Az 1 kivonása ezt a beállított bitet nullára, az alatta álló nullákat egyesekre változtatja. A magasabb bitek változatlanok. Az eredeti értékkel végzett AND ezért pontosan a legalacsonyabb beállított bitet törli.

Például, előjel nélküli olvasattal:

Lépés	Bitminta
Eredeti érték	`0b10110000`
Eredeti érték mínusz 1	`0b10101111`
A két érték AND-ja	`0b10100000`

A ciklus tehát pontosan annyiszor fut, amennyi a popcount. A nulla bemenetet a ciklus előtt kezeljük. A módszer gyakran *Brian Kernighan algoritmusa* néven szerepel.

Fontos következmény: a végrehajtási útvonal adatfüggő. Titkos értékek feldolgozásakor ez önmagában elegendő ok arra, hogy ne tekintsük automatikusan állandó idejű megoldásnak.

9.4.3 Parity: XOR-összecsukás

Az XOR asszociatív és kommutatív, továbbá egy bit önmagával vett XOR-ja nulla. A paritás ezért fokozatos **XOR-összecsukással** (*XOR folding*) számítható.

Az előjel nélküli értéket először a 16 bittel jobbra tolt másolatával XOR-oljuk össze. Az alsó 16 pozíció ekkor már két-két eredeti bit XOR-ját tartalmazza. Ezután következik a 8, 4, 2 és 1 bites eltolás.

Feldolgozott eltolások	Egy hasznos eredménybithez hozzájáruló eredeti bitek száma
16	2
16, 8	4
16, 8, 4	8
16, 8, 4, 2	16
16, 8, 4, 2, 1	32

Az utolsó lépés után **csak a legalsó bitet használjuk**. A többi eredménybit nem feltétlenül nulla, ezért szükséges az 1 értékű maszkkal végzett AND.

Az A64 `EOR` eltolt regiszteroperandusa egyetlen utasításban fejezi ki az XOR-t és a forrás logikai jobbra tolását.

A paritás egyszerű hibajelzés: minden páratlan számú bitváltozást észlel, de páros számú bitváltozás észrevétlen maradhat. Nem helyettesít általános ellenőrző összeget vagy hibajavító kódot.

9.4.4 Bitfordítás: az alsó bitből építjük az eredményt

A bitfordításnál egy nulláról induló akkumulátort használunk. Minden lépés:

1. kiemeli a bemenet alsó bitjét;
2. balra tolja az akkumulátort egy bittel;
3. beilleszti a kiemelt bitet az akkumulátor alsó pozíciójába;
4. logikailag jobbra tolja a bemenetet.

Ciklusinvariáns: j lépés után az akkumulátor alsó j bitje az eredeti bemenet alsó j bitjének fordított sorrendű sorozata. A feldolgozatlan bemenetrész közben j bittel jobbra tolódott.

Pontosan 32 lépés szükséges. Az előjel nélküli `0x00000001` fordítottja `0x80000000`, nem `0x00000001`: a vezető nullák is a rögzített szélességű bitsorozat részei.

Ez **nem bájtsorrendcsere**. A regiszterérték bitfordítása nem függ attól, milyen bájtsorrenddel tároljuk az értéket a memóriában. Az A64 `REV` bájtsorrendet cserél, az A64 `RBIT` pedig biteket fordít.

9.4.5 Kidolgozott példa: három algoritmus és egy natív referencia

Munkakönyvtár: `~/arm-labor/a64/a035/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a035
cd ~/arm-labor/a64/a035
```

Az alábbi négy levélfüggvény (*leaf function*) az AAPCS64 szerződését követi: a bemenet és az eredmény az A64 `W0` regiszterben van. Csak hívó által mentendő munkaregisztereket módosítanak; nem hívnak más függvényt, ezért veremkeret sem szükséges.

Az A64 `Wn` célregiszter írása a hozzá tartozó `Xn` felső 32 bitjét nullázza. Itt ez összhangban áll a `uint32_t` interfésszel.

A `lab-bits.S` teljes tartalma:

```
// A64 ISA, Armv8-A, little-endian Linux.
// Minden utasítás és regiszter A64; a direktívák GNU as direktívák.
.arch armv8-a
.text
.p2align 2

.global lab_popcount32
.type lab_popcount32, %function
lab_popcount32:
    mov     w1, wzr
    cbz     w0, .Lpc_done
.Lpc_loop:
    sub     w2, w0, #1
    and     w0, w0, w2
    add     w1, w1, #1
    cbnz    w0, .Lpc_loop
.Lpc_done:
    mov     w0, w1
    ret
.size lab_popcount32, .-lab_popcount32

.global lab_parity32
.type lab_parity32, %function
lab_parity32:
    eor     w0, w0, w0, lsr #16
    eor     w0, w0, w0, lsr #8
    eor     w0, w0, w0, lsr #4
    eor     w0, w0, w0, lsr #2
    eor     w0, w0, w0, lsr #1
    and     w0, w0, #1
    ret
.size lab_parity32, .-lab_parity32

.global lab_reverse32
.type lab_reverse32, %function
lab_reverse32:
    mov     w1, wzr
    mov     w2, #32
.Lrev_loop:
    and     w3, w0, #1
    lsl     w1, w1, #1
    orr     w1, w1, w3
    lsr     w0, w0, #1
    subs    w2, w2, #1
    b.ne    .Lrev_loop
    mov     w0, w1
    ret
.size lab_reverse32, .-lab_reverse32

.global lab_reverse32_native
.type lab_reverse32_native, %function
lab_reverse32_native:
    rbit    w0, w0
    ret
.size lab_reverse32_native, .-lab_reverse32_native

.section .note.GNU-stack,"",%progbits
```

Immediate-ok: a példa konstansai (`#1`, `#32`, a 0–31 közötti eltolások) mind kódolhatók; a szabályokat az A64 immediate-korlátok összefoglalója adja. Az `AND #1` azért megengedett, mert az `1` érvényes logikai (bitmaszk-) immediate.

A tesztprogram, `lab-bits-test.c`, ismert eredmények mellett összefüggéseket is ellenőriz:

```c
/* C tesztharness; cél: AArch64 Linux, little-endian. */
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

extern uint32_t lab_popcount32(uint32_t);
extern uint32_t lab_parity32(uint32_t);
extern uint32_t lab_reverse32(uint32_t);
extern uint32_t lab_reverse32_native(uint32_t);

int main(void)
{
    static const struct {
        uint32_t x, count, parity, reversed;
    } cases[] = {
        {0x00000000u,  0, 0, 0x00000000u},
        {0x00000001u,  1, 1, 0x80000000u},
        {0x80000000u,  1, 1, 0x00000001u},
        {0x80000001u,  2, 0, 0x80000001u},
        {0xffffffffu, 32, 0, 0xffffffffu},
        {0xaaaaaaaau, 16, 0, 0x55555555u},
        {0x01234567u, 12, 0, 0xe6a2c480u},
        {0x00010000u,  1, 1, 0x00008000u}
    };

    for (size_t i = 0; i < sizeof cases / sizeof cases[0]; ++i) {
        uint32_t x = cases[i].x;
        uint32_t p = lab_popcount32(x);
        uint32_t q = lab_parity32(x);
        uint32_t r = lab_reverse32(x);

        if (p != cases[i].count ||
            q != cases[i].parity ||
            r != cases[i].reversed ||
            q != (p & 1u) ||
            r != lab_reverse32_native(x) ||
            lab_reverse32(r) != x) {
            printf("FAIL: x=0x%08" PRIx32
                   " count=%" PRIu32 " parity=%" PRIu32
                   " reversed=0x%08" PRIx32 "\n", x, p, q, r);
            return 1;
        }
    }

    puts("OK: 8 tesztvektor");
    return 0;
}
```

A kétszeri bitfordítás az eredeti értéket adja. Ez hasznos ellenőrzés, de önmagában kevés: az identitásfüggvény is teljesítené. Ezért használunk ismert eredményeket és külön A64 `RBIT`-referenciát is.

9.4.6 Assemble–link–run és ellenőrzés

Az alábbi munkafolyamat az Ubuntu 24.04 laborprofil cross-eszközláncát és a célrendszer statikus C-könyvtárát feltételezi.

```
# Target: qemu-user, AArch64 Linux, Armv8-A, little-endian.
# Assemble:
aarch64-linux-gnu-gcc -march=armv8-a -g \
    -c lab-bits.S -o lab-bits.o
```

```
# C-fordítás:
aarch64-linux-gnu-gcc -march=armv8-a -std=c11 -O2 -g \
    -Wall -Wextra -c lab-bits-test.c -o lab-bits-test.o

# Link:
aarch64-linux-gnu-gcc -static \
    lab-bits-test.o lab-bits.o -o lab-bits

# Run:
qemu-aarch64 ./lab-bits
printf 'kilepesi_allapot=%d\n' "$?"
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2): `OK: 8 tesztvektor`, majd `kilepesi_allapot=0`. Algoritmushiba esetén a tesztprogram `FAIL:` kezdetű sort ír és 1 állapottal lép ki. A fordítási vagy betöltési hiba ettől külön kezelendő.

A létrejött utasítások ellenőrzése:

```
# A qemu-user célprogram hostoldali, statikus vizsgálata; nincs futtatás.
aarch64-linux-gnu-objdump -d --disassemble=lab_popcount32 lab-bits
aarch64-linux-gnu-objdump -d --disassemble=lab_parity32 lab-bits
aarch64-linux-gnu-objdump -d --disassemble=lab_reverse32 lab-bits
```

Az objdump eltérő aliasnevet is kiírhat ugyanarra a kódolásra. Nem pusztán a szöveget, hanem az operandusszélességet és a művelet jelentését ellenőrizzük.

9.4.7 Teljesítmény-előzetes: utasításszám nem egyenlő ciklusszámmal

A konkrét függvények dinamikus utasításszáma egyszerűen levezethető. Az alábbi értékek a visszatérést tartalmazzák, a hívó utasítását nem.

Implementáció	Elméleti, levezetett utasításszám
Ciklusos popcount, k beállított bit	$4k + 4$
XOR-összecsukásos paritás	7
Ciklusos, 32 bites bitfordítás	$32 \cdot 6 + 4 = 196$
A64 `RBIT`-alapú bitfordítás	2

A popcountban négy ciklusutasítás jut minden beállított bitre, továbbá négy utasítás a cikluson kívüli részre. Így nulla bemenetre 4, minden bitet beállítva 132 utasítás adódik.

Ezek **nem ciklusszámok**. Az eredmény elkészülési késleltetése (*latency*), az időegységenként feldolgozható műveletek száma (*throughput*), az adatfüggőségek és az elágazásbecslés mikroarchitektúra-függők. Még a rögzített utasításszám sem bizonyít önmagában állandó futási időt.

Bitfordításnál az A64 `RBIT` az Armv8-A alapprofil része, ezért valódi implementációban általában ezt keressük először. A ciklus tanulási és referenciaértéke ettől megmarad.

Popcountnál az Armv8-A alapprofil általános A64 `Wn`/`Xn` regisztereire nincs egyetlen skaláris popcountutasítás. Az A64 Advanced SIMD/NEON `CNT` viszont **bájtonként** megszámolja a beállított biteket. Egy teljes szó popcountjához a bájtonkénti eredményeket még összegezni kell. Az Armv8.9/9.4 kiterjesztés (FEAT_CSSC) viszont már bevezetett skaláris `CNT`, `CTZ`, `ABS`, `SMAX` és hasonló utasításokat; a GCC `-march=armv9.4-a+cssc` mellett ezeket is generálhatja. Ez azonban túlmutat az itt használt Armv8-A alapprofilon, ezért a labor nem épít rá.

Egyetlen értéknél a regiszterek közötti adatmozgatás és az összegzés költsége is számít. Hosszú tömbnél ezek a költségek több elemre oszthatók el; különösen indokolt a natív lehetőségek vizsgálata, ha az adat már SIMD-regiszterben van. Ezt a NEON regiszterkészleteénél és a gyorsulás mérésénél bontjuk ki.

QEMU-ban a szemantika ellenőrizhető, de az ottani futásidő nem hardverhű abszolút ciklusszám. Fizikai AArch64 gépen külön mérés szükséges; itt nem közlünk mért gyorsulást.

9.4.8 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Célzott ellenpróba
A paritás néha 1-nél nagyobb	Hiányzik a végső maszkolás	Előjel nélküli `0xffffffff` bemenet
Csak magasabb biteknél hibás a paritás	Kimaradt egy összecsukási lépés	Előjel nélküli `0x00010000` bemenet
A bitfordítás rövid mintáknál hibás	A ciklus a bemenet nullává válásakor megáll	Előjel nélküli `0x00000001` bemenet
A 64 bitesre átírt változat hibás	Csak a regiszternevek változtak	Felső félben beállított bitek

A hibakeresési sorrend: konkrét hibás bemenet rögzítése, hipotézis megfogalmazása, disassembly gyűjtése, **egyetlen** változtatás, majd teljes újrafordítás és regressziós teszt.

A 64 bites változatnál nem elég az A64 `Wn` neveket `Xn` nevekre cserélni: a paritáshoz 32 bites első összecsukás, a bitfordításhoz 64 iteráció szükséges.

9.4.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Mi a popcount, a paritás és a 32 bites bitfordítás eredménye az előjel nélküli `0x00000010` értékre? Miért lenne hibás öt bit feldolgozása után leállítani a bitfordítást?

2. (implementációs, középszint — cél: rutinépítés) Rövidítsd a bitfordítás ciklustörzsét hatról öt A64 utasításra az eltolt regiszteroperandusú `ORR` használatával. Az interfész és a teszteredmények ne változzanak. Vezesd le az új utasításszámot!

3. (hibakeresési, középszint — cél: rutinépítés) Kontrollált hibaként töröld a paritás első, 16 bites eltolást használó A64 `EOR` utasítását. Futtasd újra a teljes buildet és tesztet. A teszt az első hibás vektornál áll le, ez a `0x80000000`; magyarázd meg ezt, majd mutasd meg külön a `0x00010000`-re is, hogy a hiba nem csak a legfelső bitre jellemző. Ellenőrizd a disassemblyt, majd állítsd vissza az utasítást.

9.4.10 Megoldások

1. megoldás — Segítség. `0x00000010` egyetlen beállított bitje a 4. indexen van — hová kerül ez a bitfordítás $31 - i$ szabálya szerint, és miért nem elég öt lépés, ha a cél index 27?

1. megoldás — Teljes megoldás. A popcount 1, a paritás 1. A beállított bit indexe 4, ezért a bitfordításban a $31 - 4 = 27$ indexre kerül: az eredmény `0x08000000`. Öt lépés után az akkumulátor még csak `0x00000001`; a további 27 lépés helyezi át a bitet a megfelelő pozícióba.

2. megoldás — Segítség. Az `ORR` eltolt regiszteroperandusa egy lépésben végzi el, amit most két külön utasítás (`LSL` majd `ORR`) csinál — a kiemelt bit melyik operandusba kerüljön, hogy ne kelljen előtte külön eltolni?

2. megoldás — Teljes megoldás. Az eredeti A64 `LSL` és `ORR` pár helyére ez az egyetlen utasítás kerül:

```
// A64 ISA, Armv8-A; a lab_reverse32 ciklustörzsében.
// w3 az előző AND által kiemelt 0 vagy 1.
orr     w1, w3, w1, lsl #1
```

Az eltolás immediate értéke a 32 bites alak 0–31 tartományába esik. Az eltolt akkumulátor alsó bitje nulla, ezért az OR pontosan beilleszti a kiemelt bitet. Az új számított utasításszám $32 \cdot 5 + 4 = 164$. Az eredeti assemble–link–run lépésekkel a nyolc teszt a lektor mérésében ezzel a változattal is sikerült (`OK: 8 tesztvektor`, kilépési állapot 0); gyorsulást ebből még nem mértünk.

3. megoldás — Segítség. A maradék négy összecsukás (8,4,2,1 eltolás) együtt legfeljebb mekkora távolságra tud egy bitet lefelé mozgatni — eléri-e ez a 16. indextől a 0. pozíciót?

3. megoldás — Teljes megoldás. A megmaradó 8, 4, 2 és 1 bites eltolások összesen legfeljebb 15 pozícióval továbbíthatják lefelé az eredeti bit hozzájárulását. A 16 indexű bit így nem jut el a nulladik pozícióba: a hibás eredmény 0, a helyes paritás 1. Az objdumpban az első összecsukás hiánya látható. A tesztprogram az első hibás vektornál leáll, ezért a kimenetben a `0x80000000` jelenik meg (`FAIL: x=0x80000000 count=1 parity=0 …`); a `0x00010000` ugyanígy hibás, de csak külön futtatva látszik. Visszaállítás és újraépítés után ismét 0 kilépési állapotot várunk.

9.4.11 Rövid összefoglaló

A három algoritmus ugyanazt a bitsorozatot más szempontból dolgozza fel. A ciklusos popcount minden lépésben egy beállított bitet töröl, ezért munkamennyisége adatfüggő. A paritás öt XOR-összecsukással számítható, de a végén csak az alsó bit érvényes eredmény. A bitfordításnál a rögzített szélesség miatt a vezető nullákat is fel kell dolgozni. A64-en az `RBIT` közvetlen alternatíva; a NEON `CNT` használatához további összegzés kell. Az utasításszám algoritmus-összehasonlításra alkalmas, hardveres futásidőt azonban nem helyettesít. A tesztelésben ismert eredmények és algebrai összefüggések egyaránt szükségesek.

- **Popcount:** a beállított bitek száma.
- **Paritás:** a popcount maradéka 2-vel osztva.
- **Bitfordítás:** a rögzített szélességű bitsorozat sorrendjének megfordítása.

9.4.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — A64 `RBIT`, eltolt logikai operandusok és Advanced SIMD `CNT`. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- GNU assembler kézikönyv — AArch64 szintaxis, aliasok és direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- AAPCS64 — argumentumátadás, visszatérési értékek és regisztermegőrzés. **Tartalmilag ellenőrizve**: aktuális kiadás 2025Q4 (2026-01-23), Release minőségű.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a kód a lektor mérési környezetében (GCC 13.3.0, QEMU 8.2.2) lefutott, az itt közölt kimenet onnan származik. A hallgató saját Ubuntu 24.04 laborjában a telepített csomagverziókkal, a statikus linkelés rendelkezésre állásával és a várt eredményekkel is ellenőrizze. Natív teljesítményadat és az alapprofilon túli utasításkiterjesztések (például FEAT_CSSC) tényleges támogatása nem lett vizsgálva.

10 C-assembly interoperabilitás és fordítóprogram-elemzés

10.1 Inline assembly GCC-ben

10.1.1 Motiváció: az utasítás mellett a szerződést is megírjuk

Tegyük fel, hogy egy C-ben írt többpontosságú aritmetikai modulhoz az összeg mellett az előjel nélküli átvitelre is szükségünk van. Az A64 `ADDS` kiszámítja az összeget és beállítja az állapotjelzőket; az A64 `CSET` ezekből közönséges egész értéket készíthet. Hogyan illesszük ezt a két utasítást C-kódba úgy, hogy optimalizálás mellett is helyesen működjön?

A GCC **bővített beágyazott assemblyje** (*extended inline assembly*) utasításablont és fordítói adatfüggőségi szerződést kapcsol össze. Megadjuk, mit olvas a blokk, mit állít elő, és milyen további állapotot módosít.

A helyes utasítássor önmagában nem elegendő. A fordító a deklarált operandusokból és mellékhatásokból dolgozik; az assemblyszövegből nem vezeti le automatikusan a teljes viselkedést.

Az alfejezet minden utasításablonja **A64 ISA, AArch64, Armv8-A alapprofil**, little-endian környezetben. Thumb-2 kód itt nem szerepel. Feltételezzük az egész aritmetika, az állapotjelzők és az AAPCS64 alapjainak ismeretét; a hívási szerződés előzménye Az AAPCS64 regiszterszerződés.

10.1.2 A bővített szintaxis és az operandusok

A GCC C-kiterjesztéseként használt forma:

```
/* Szintaxisvázlat, nem önállóan fordítható program.
   A sablon ebben az alfejezetben A64 utasításokat tartalmaz. */
__asm__ volatile (
    "utasításablon"
    : kimeneti_operandusok
    : bemeneti_operandusok
    : clobber_lista
);
```

A `__asm__` írásmód szigorúbb C-nyelvi móddal, például `-std=c11` mellett is használható GCC-kiterjesztésként. Nem A64 utasítás és nem assemblerdirektíva.

A négy rész szerepe:

1. **Utasításablon:** C-karakterlánc, amelybe a GCC behelyettesíti az operandusokat.
2. **Kimenetek:** a blokk által előállított vagy módosított C-értékek.
3. **Bemenetek:** a blokk által felhasznált C-értékek.
4. **Clobber-lista:** a kimenetként nem modellezett állapotmódosítások felsorolása.

Az operandus alakja például `[sum] "=r" (sum)`. A szögletes zárójelben szereplő név a sablonbeli hivatkozás neve, a karakterlánc az **operandus-megkötés** (*constraint*), a zárójelben pedig C-kifejezés áll. A kimenetnek írható C-objektumot kell jelölnie.

GCC-megkötés	Jelentés az itteni AArch64-példákban
`"r"`	A bemeneti értéket általános célú regiszterben kérjük.
`"=r"`	Csak írt regiszterkimenet; a korábbi érték nem bemenet.
`"+r"`	Ugyanaz az operandus bemenet és kimenet is.
`"=&r"`	Korán felülírt, csak írt regiszterkimenet.

Az `"r"` **nem konkrét regisztert választ**. A GCC dönt az elhelyezésről, ezért a kód különböző optimalizálásokkal más regisztereket használhat.

A sablonban `%x[name]` az operandus **A64 Xn, 64 bites** nézetét, `%w[name]` az **A64 Wn, 32 bites** nézetét írja ki. Ezek GCC-sablonmódosítók, nem utasítások. Nem végeznek C-típuskonverziót: a C-típus és a kiválasztott utasításszélesség összhangjáért mi felelünk.

10.1.2.1 Mikor osztozhat bemenet és kimenet egy regiszteren?

Az `"=r"` önmagában megengedi, hogy a kimenet valamelyik bemenet regiszterét kapja. Egyetlen A64 `ADD` esetén ez helyes: az utasítás a forrásokból számol, majd elhelyezi az eredményt.

Többutasításos blokknál viszont előfordulhat, hogy az első utasítás már felülírja a kimenetet, egy későbbi pedig még olvasna egy bemenetet. Ilyenkor az adott kimeneten `&`, azaz **korai felülírás** (*early clobber*) szükséges.

Ez nem stíluskérdés: nélküle a GCC olyan regiszterátfedést választhat, amely tönkreteszi a még szükséges bemenetet. Olvasott és írt operandusnál ugyanez a jelölés `"+&r"` lehet.

Teljes ellenpélda: két utasítás, egy elveszett bemenet. Számítsuk ki a `2*b + a` értéket két A64 utasítással. Az első utasítás már a kimenetet írja, a második még olvassa az `a` bemenetet. Mentsük `lab-early.c` néven a `~/arm-labor/a64/a036/` munkakönyvtárba (lásd a kidolgozott példa elején):

```
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

/* HIBÁS: az első utasítás már írja a kimenetet, a második még olvassa a-t. */
__attribute__((noinline, noipa))
static uint64_t lab_2b_plus_a_bad(uint64_t a, uint64_t b)
{
    uint64_t out;

    __asm__(
        "lsl %x[out], %x[b], #1\n\t"     /* out = 2*b   (kimenet írása)  */
        "add %x[out], %x[out], %x[a]"    /* out += a    (a még kell!)    */
        : [out] "=r" (out)
        : [a] "r" (a), [b] "r" (b)
    );
    return out;
}

/* JAVÍTOTT: "=&r" - a kimenet nem kaphatja egyik bemenet regiszterét sem. */
__attribute__((noinline))
static uint64_t lab_2b_plus_a_good(uint64_t a, uint64_t b)
```

```
{
    uint64_t out;

    __asm__(
        "lsl %x[out], %x[b], #1\n\t"
        "add %x[out], %x[out], %x[a]"
        : [out] "=&r" (out)
        : [a] "r" (a), [b] "r" (b)
    );
    return out;
}

int main(void)
{
    uint64_t bad = lab_2b_plus_a_bad(1, 10);
    uint64_t good = lab_2b_plus_a_good(1, 10);

    printf("bad=%" PRIu64 " good=%" PRIu64 "\n", bad, good);
    return good == 21 ? 0 : 1;
}

# Target: AArch64 Linux, qemu-user; a hibás változat szándékos.
for opt in 0 2; do
    aarch64-linux-gnu-gcc -std=c11 -Wall -Wextra -march=armv8-a \
        -O"$opt" -static lab-early.c -o "lab-early-O${opt}"
    qemu-aarch64 "./lab-early-O${opt}"
done
aarch64-linux-gnu-objdump -d --disassemble=lab_2b_plus_a_bad lab-early-O2
# Ha a függvény mégis .constprop utótagot kapna, a grep -A6 '2b_plus_a_bad' is működik.
```

A helyes eredmény `a = 1`, `b = 10` mellett 21. A hibás változatban a fordító `"=r"` alapján jogosan adhatja a kimenetnek az `a` regiszterét — `noinline` függvényben ez gyakran az `X0`, amely egyben a visszatérési regiszter is. Ekkor a `lsl x0, x1, #1` felülírja az `a`-t, és az `add x0, x0, x0` a `4*b = 40` értéket adja. Hogy ez bekövetkezik-e, a regiszterkiosztáson múlik: a GNU GCC 13.3.0 fő úton ugyanaz a forrás `-O0` és `-O2` mellett egyaránt `bad=40`, `good=21` eredményt ad. Clang `-O2` alatt más regiszterkiosztás miatt a hiba elrejtőzhet (`bad=21`) — ez épp a tanulság: a tünet fordítófüggő, a javított változat viszont minden szinten `good=21`. A javított változat `"=&r"` megkötése minden szinten kizárja az átfedést, ezért `good=21`. A disassemblyben a hibás függvény kimenetét és bemeneteit kell összevetni; ha a kimenet regisztere azonos egy olyan bemenetével, amelyet egy későbbi utasítás még olvas, az a hiba.

Mikor nem kell `&`? Ha minden bemenet elfogy, mielőtt a blokk bármelyik kimenetet írná. A kidolgozott példa `add64_carry` blokkjában az `ADDS` egyszerre olvassa a két bemenetet és írja a `sum`-ot, a `CSET` pedig már csak az NZCV-t olvassa. Ott a korai felülírás jelölése felesleges lenne, és csak a regiszterkiosztást szűkítené.

10.1.3 Mit jelent, és mit nem jelent a `volatile`?

Ha egy nem `volatile` blokk eredményét senki nem használja, a GCC eltávolíthatja. Azonos bemenetű számítást megfelelő feltételek mellett összevonhat vagy kiemelhet egy ciklusból.

Tisztán számoló blokk esetén ez kívánatos. Ha az eredmény szükséges, az adatfüggőségek biztosítják az előállítását; nem kell automatikusan `volatile`.

A `volatile` többek között azt jelzi, hogy a blokk nem dobható el pusztán azért, mert a kimeneteire nincs szükség. Kimenet nélküli bővített asm blokkot a GCC implicit módon is `volatile`-ként kezel.

A `volatile` azonban:

- nem deklarálja a módosított regisztereket vagy állapotjelzőket;
- nem jelenti automatikusan a memória olvasását vagy írását;
- nem teszi atomivá a műveletet;
- nem hardveres memóriakorlát;
- nem tilt meg minden átrendezést a környező C-kódhoz képest.

A `volatile` nem helyettesíti a pontos operandus- és clobber-listát.

10.1.4 A clobber-lista három fontos esete

Állapotjelzők. Az A64 `ADDS` módosítja az A64 NZCV állapotjelzőket, ezért az ilyen blokk clobber-listájában `"cc"` szerepeljen. Az A64 `ADD` nem módosítja ezeket. A `"cc"` nem eredmény-operandus: csak közli, hogy a korábbi feltételkód-állapot nem tekinthető megőrzöttnek.

Rögzített munkaregiszter. Ha a sablon közvetlenül felülírja az A64 X9 regisztert, ezt `"x9"` clobberrel kell jelezni, amennyiben nem operandusként modellezzük. A GCC ezután nem helyezi a blokk operandusait ebbe a regiszterbe, és nem feltételezi korábbi tartalmának megőrzését.

Az A64 X9 hívó által mentendő volta nem mentség: az inline blokk **nem függvényhívás**. A fordító nem alkalmaz rá automatikusan hívási regiszterrombolási szabályokat. Általában jobb a munkaregisztert operandusként kérni, mint rögzíteni.

Memória. A `"memory"` azt jelzi, hogy a blokk az operandusokkal pontosan le nem írt memóriát is olvashatja vagy módosíthatja. Emiatt a fordító nem tarthat fenn vele összeegyeztethetetlen memóriaérték-feltételezéseket.

Egy pointer `"r"` bemenetként átadása csak a **cím értékének használatát** jelenti. Nem mondja meg, hogy a címzett objektumot olvassuk vagy írjuk. Ezt pontos memóriaoperandussal vagy konzervatívabban `"memory"` clobberrel kell leírni. A `"memory"` fordítói korlát, nem A64 hardveres barrier, és önmagában nem oldja meg a szálak közötti adatversenyeket.

10.1.5 Kidolgozott példa: összeg és előjel nélküli átvitel

Munkakönyvtár: `~/arm-labor/a64/a036/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a036
cd ~/arm-labor/a64/a036
```

A következő teljes programban az első függvény egyszerű `"r"` bemeneteket és `"=r"` kimenetet használ. A második két eredményt állít elő, és deklarálja az állapotjelzők módosítását.

Mentsük `lab-inline.c` néven:

```
#include <stdint.h>
#include <stdio.h>

static inline uint64_t add64(uint64_t a, uint64_t b)
{
    uint64_t sum;

    /* A64 ISA: 64 bites ADD; nem módosítja az NZCV-t. */
    __asm__(
        "add %x[sum], %x[a], %x[b]"
        : [sum] "=r" (sum)
        : [a] "r" (a), [b] "r" (b)
    );
```

```
    return sum;
}

static inline uint64_t add64_carry(uint64_t a, uint64_t b,
                                   unsigned *carry_out)
{
    uint64_t sum;
    unsigned carry;

    /* A64 ISA: ADDS + CSET; a CS feltétel jelentése C == 1. */
    __asm__(
        "adds %x[sum], %x[a], %x[b]\n\t"
        "cset %w[carry], cs"
        : [sum] "=r" (sum), [carry] "=r" (carry)
        : [a] "r" (a), [b] "r" (b)
        : "cc"
    );

    *carry_out = carry;
    return sum;
}

int main(void)
{
    static const uint64_t cases[][2] = {
        { UINT64_C(0), UINT64_C(0) },
        { UINT64_C(7), UINT64_C(5) },
        { UINT64_MAX,  UINT64_C(1) },
        { UINT64_MAX,  UINT64_MAX }
    };

    for (unsigned i = 0; i < sizeof cases / sizeof cases[0]; ++i) {
        uint64_t a = cases[i][0];
        uint64_t b = cases[i][1];
        uint64_t expected = a + b;
        unsigned expected_carry = expected < a;
        unsigned carry;
        uint64_t sum = add64_carry(a, b, &carry);

        if (add64(a, b) != expected ||
            sum != expected || carry != expected_carry) {
            fprintf(stderr, "Hiba a(z) %u. tesztnel\n", i);
            return 1;
        }
    }

    puts("lab-inline: 4 teszt OK");
    return 0;
}
```

A C-referencia előjel nélküli aritmetikát használ, ezért a körbefordulás definiált. Ha a matematikai összeg eléri a 2^{64} értéket, az eltárolt összeg 2^{64}-gyel kisebb nála, és kisebb lesz az első összeadandónál. Ez indokolja az `expected < a` átvitelvizsgálatot.

A várható, levezetett eredmények:

Első bemenet, előjel nélküli	Második bemenet, előjel nélküli	64 bites összeg	Átvitel
0	0	0x0000000000000000	0
7	5	0x000000000000000c	0

Első bemenet, előjel nélküli	Második bemenet, előjel nélküli	64 bites összeg	Átvitel
`0xffffffffffffffff`	`1`	`0x0000000000000000`	1
`0xffffffffffffffff`	`0xffffffffffffffff`	`0xfffffffffffffffe`	1

A második blokkban nincs szükség korai felülírásra: az A64 `ADDS` az összes bemeneti regisztert felhasználja, mielőtt a későbbi A64 `CSET` lefutna. Utóbbi már csak az állapotjelzőktől függ. A két utasítás egy blokkban marad, így a fordító nem illeszt közéjük más, állapotjelzőket módosító utasítást.

A `carry_out` címére írást a C-kód végzi, a fordító számára láthatóan. Emiatt ehhez a blokkhoz nem szükséges `"memory"`.

10.1.5.1 Fordítás, linkelés, futtatás és vizsgálat

Az Ubuntu 24.04 LTS cross-környezetében:

```
# Target: AArch64 Linux, qemu-user; Armv8-A, little-endian.
# A GCC lépése C-fordítást, assemblálást és statikus linkelést végez.
for opt in 0 2 3; do
    aarch64-linux-gnu-gcc \
        -std=c11 -Wall -Wextra -Werror \
        -march=armv8-a -mlittle-endian -O"$opt" -static \
        lab-inline.c -o "lab-inline-O${opt}" || exit 1

    qemu-aarch64 "./lab-inline-O${opt}"
    status=$?
    printf 'O%s: kilepesi allapot=%s\n' "$opt" "$status"
    test "$status" -eq 0 || exit 1
done

# Target: ugyanazon qemu-user program A64 kódjának statikus vizsgálata.
aarch64-linux-gnu-gcc \
    -std=c11 -march=armv8-a -mlittle-endian -O2 \
    -S -fverbose-asm lab-inline.c -o lab-inline-O2.s

aarch64-linux-gnu-objdump -d lab-inline-O2 > lab-inline-O2.dump
```

A statikus linkeléshez a célarchitektúra libc-fejlesztői állományai is szükségesek. Hiányuk környezeti hiba, nem inline-asm probléma; az alapbeállítást a Labor-alapkörnyezet: eszközlánc telepítése és verifikáció tárgyalja.

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2): mindhárom változat kiírja a `lab-inline: 4 teszt OK` sort, és `0` állapottal lép ki; a `-S` kimenetben az `adds`–`cset …, cs` pár megtalálható.

A generált assemblyben keressük az A64 `ADDS`–`CSET` párt. A konkrét regiszterszám nem ellenőrzési feltétel: azt a GCC választja. A disassembler feltételkód-szinonimát is megjeleníthet.

10.1.6 Hibakeresési eset: a memória megváltozott, a C-érték mégsem

Az alábbi **szándékosan hibás** program kizárólag saját, érvényes C-objektumot módosít. Mentsük `lab-clobber.c` néven:

```c
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

static inline void store7(uint64_t *p)
{
    /* A64 ISA: STR. HIBA: a memóriaírás nincs deklarálva. */
    __asm__ volatile(
        "str %x[value], [%x[p]]"
        :
        : [value] "r" (UINT64_C(7)), [p] "r" (p)
        : /* Hiányzik a "memory". */
    );
}

int main(void)
{
    uint64_t value = 1;
    store7(&value);
    printf("value=%" PRIu64 "\n", value);
    return value == 7 ? 0 : 1;
}
```

```
# Target: AArch64 Linux, qemu-user; szándékosan hibás szerződés.
for opt in 0 2; do
    aarch64-linux-gnu-gcc \
        -std=c11 -Wall -Wextra -march=armv8-a -mlittle-endian \
        -O"$opt" -static lab-clobber.c -o "lab-clobber-O${opt}"

    qemu-aarch64 "./lab-clobber-O${opt}"
    printf 'O%s: kilepesi allapot=%s\n' "$opt" "$?"
done

# Target: a qemu-userhez készült hibás A64 bináris vizsgálata.
aarch64-linux-gnu-objdump -d lab-clobber-O2 > lab-clobber-O2.dump
```

GCC 13.3.0-val mérve (a lektor mérése): `-O0` mellett `value=7`, `0` kilépési állapot; `-O2` mellett `value=1`. A hibás változat kilépési állapota `-O2` mellett is `0` lehet: a kiírás és a visszatérési érték eltérően optimalizálódhat, ezért mindkettőt ellenőrizni kell. A tünet fordítóverziótól és a környező kódtól függ, ezért nem garantált, hanem jellemző viselkedés.

A fordító a hibás szerződés alapján nem látja a `value` módosítását. Az A64 `STR` megmaradhat, miközben a későbbi C-kifejezésekben a GCC továbbra is az `1` értékkel számol. A pontos tünet a fordítóverziótól és a környező kódtól függ.

A hibakeresés menete:

1. **Tünet:** optimalizálási szinttől függő eredmény.
2. **Hipotézis:** az asm valós mellékhatása hiányzik a szerződésből.
3. **Adatgyűjtés:** a disassemblyben ellenőrizzük a tárolást, majd a kiíráshoz és visszatéréshez előállított értéket.
4. **Egy változtatás:** az üres clobber-listát cseréljük `"memory"`-ra.
5. **Ellenpróba:** ugyanazzal az optimalizálással újrafordítjuk és futtatjuk.

A `volatile` már a hibás változatban is jelen van. Újbóli hozzáadása tehát nem megoldás.

10.1.6.1 Pontosabb javítás: célzott memóriaoperandus

A `"memory"` clobber helyes, de durva. Fordítói **memóriahatás-deklaráció**: azt közli, hogy a blokk az operandusokkal le nem írt memóriát is olvashat és írhat. Ezért a fordító érvényteleníti

azokat a feltételezéseit, amelyek a blokk által **potenciálisan elérhető** memóriára vonatkoznak: a blokk előtt kiírja az ilyen objektumok függő értékét, utána pedig újraolvassa őket. Nem jelenti viszont azt, hogy *minden* memóriabeli értéket újraolvas. Az alias- és escape-elemzésből származó tudása megmarad: egy olyan helyi változót, amelynek a címe soha nem szökik ki, a blokk nem érhet el, így az továbbra is regiszterben maradhat. Egy teljes, GCC-vel futtatható mini-labor ezt meg is mutatja. Mentsd `lab-memclobber.c` néven a munkakönyvtárba:

```
/* Cél: AArch64 Linux, qemu-user; GCC, -O2. */
#include <stdio.h>

static int g = 7;

__attribute__((noinline, noipa))
static int lab_memclobber(int x)
{
    int local = x * 3;
    int a = g;
    __asm__ volatile("" ::: "memory");
    return local + a + g;
}

int main(void)
{
    int r = lab_memclobber(5);
    printf("result=%d\n", r);
    return r == 22 ? 0 : 1;
}
```

```
# Target: AArch64 Linux, qemu-user; a clobber hatásának megfigyelése.
aarch64-linux-gnu-gcc -std=c11 -O2 -march=armv8-a \
    -static lab-memclobber.c -o lab-memclobber
qemu-aarch64 ./lab-memclobber
printf 'kilepesi allapot=%d\n' "$?"
aarch64-linux-gnu-objdump -d --disassemble=lab_memclobber lab-memclobber
```

A kimenet `result=22`, a kilépési állapot `0`; ezek C-szemantikából levezethetők. A disassemblyben a `local` értéke jellemzően végig regiszterben marad, mert a címe nem szökik ki, a blokk nem érheti el; a `g` globálist viszont a blokk előtt és után külön `ldr` olvassa. Az előbbi tehát nem sérti a clobbert, az utóbbi éppen a clobber hatása.

A `"memory"` ettől sem lesz hardveres memória-barrier: a processzor memóriaelérési sorrendjére nem hat. Ha a blokk csak egy ismert objektumot érint, pontosabb, ha ezt az objektumot memóriaoperandusként adjuk meg:

Megkötés	Jelentés
`"=m" (*p)`	a blokk a `*p` objektumot teljesen felülírja, korábbi tartalmát nem olvassa
`"+m" (*p)`	a blokk a `*p` objektumot olvassa és írja
`"m" (*p)`	a blokk a `*p` objektumot csak olvassa

A sablonban `%[obj]` a GCC által választott címzési alakot írja ki (például `[x0]`), ezért közvetlenül `LDR`/`STR` operandusa lehet. Mentsük `lab-memop.c` néven:

```
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>

/* Csak a *p objektumot írja: "=m" a teljes, felülírt objektumot írja le. */
```

```c
static inline void store7(uint64_t *p)
{
    __asm__(
        "str %x[value], %[obj]"
        : [obj] "=m" (*p)
        : [value] "r" (UINT64_C(7))
    );
}

/* Olvassa és írja *p-t: "+m"; a munkaregiszter "=&r", mert az első
   utasítás már írja, a harmadik pedig még a cím regiszterét használja. */
static inline void inc_in_place(uint64_t *p)
{
    uint64_t tmp;

    __asm__(
        "ldr %x[tmp], %[obj]\n\t"
        "add %x[tmp], %x[tmp], #1\n\t"
        "str %x[tmp], %[obj]"
        : [obj] "+m" (*p), [tmp] "=&r" (tmp)
    );
}

int main(void)
{
    uint64_t value = 1;
    uint64_t other = 100;

    store7(&value);
    inc_in_place(&value);
    printf("value=%" PRIu64 " other=%" PRIu64 "\n", value, other);
    return (value == 8 && other == 100) ? 0 : 1;
}
```

```
# Target: AArch64 Linux, qemu-user.
for opt in 0 2; do
    aarch64-linux-gnu-gcc -std=c11 -Wall -Wextra -march=armv8-a \
        -O"$opt" -static lab-memop.c -o "lab-memop-O${opt}"
    qemu-aarch64 "./lab-memop-O${opt}"
    printf 'O%s: kilepesi allapot=%s\n' "$opt" "$?"
done
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2): mindkét szinten `value=8 other=100`, kilépési állapot `0`. A fordító tudja, hogy a blokkok csak a `value`-t érintik, az `other`-re vonatkozó feltételezéseit megtarthatja. Az `inc_in_place` két tanulságot köt össze: a `"+m"` az olvasott és írt objektumot, a `tmp "=&r"` megkötése pedig azt írja le, hogy a munkaregiszter már az első utasításban íródik, miközben a harmadik utasítás még a `%[obj]` címregiszterét használja. Ha a `tmp` megkapná ezt a regisztert, a `STR` rossz címre írna.

A memóriaoperandus csak a ténylegesen megnevezett objektumot fedi le. Ha a blokk a mutatón túli memóriát is eléri (például egy egész tömböt), vagy a hozzáférés mérete eltér a C-objektum típusától, akkor vagy az egész érintett tartományt kell operandusként leírni, vagy marad a `"memory"` clobber. Hasonlóan alattomos lehet egy hiányzó munkaregiszter-clobber: a hiba csak akkor látszik, amikor a regiszterkiosztás éppen ütközést okoz.

10.1.7 Feladatok

1. **Magyarázó (alapszint — cél: megértés-ellenőrzés):** Miért nem `volatile` az `add64_carry` blokk? Miért kell mégis `"cc"`? Miért nincs szüksége `"memory"`-ra?

2. **Implementációs (alapszint — cél: rutinépítés):** Készíts `uint32_t add32(uint32_t a, uint32_t b)` függvényt A64 `ADD` utasítással, `"+r"` kimenettel és explicit 32 bites operandusnézettel. Egészítsd ki a `lab-inline.c` tesztjeit a `UINT32_MAX + 1` esettel.

3. **Hibakeresési (középszint — cél: rutinépítés):** Vizsgáld meg a `lab-clobber.c` két optimalizálási változatát. Javíts kizárólag a clobber-listán, majd ismételd meg a tesztet. Ha a hibás program sem mutat eltérést, magyarázd meg, miért nem igazolja ez a szerződés helyességét.

10.1.8 Megoldások

1. megoldás — Segítség. Vizsgáld meg külön-külön, mi számít „látható mellékhatásnak” a `volatile` szempontjából, és melyik A64 utasítás módosítja az NZCV jelzőket a blokkban.

1. megoldás — Teljes megoldás. Az `add64_carry` a bemenetekből eredményeket számol, amelyeket a C-kód felhasznál. Nem igényel külön, az eredményeitől független végrehajtási mellékhatást, ezért nem kell `volatile`. Az A64 `ADDS` módosítja az NZCV-t, ezért kell `"cc"`. Az asm nem ér el memóriát; a pointeren keresztüli írás látható C-művelet.

2. megoldás — Segítség. A `%w`-módosító a Wn nézetet választja; a `"+r"` pedig jelzi, hogy a bemenet eredeti értékére is szükség van a kimenet felülírásakor.

2. megoldás — Teljes megoldás. A `lab-inline.c` állományba illeszthető függvény:

```
static inline uint32_t add32(uint32_t a, uint32_t b)
{
    /* A64 ISA: 32 bites ADD, állapotjelző-módosítás nélkül. */
    __asm__(
        "add %w[a], %w[a], %w[b]"
        : [a] "+r" (a)
        : [b] "r" (b)
    );
    return a;
}
```

A `"+r"` közli, hogy `a` eredeti értéke is szükséges. A `%w` az A64 Wn nézetet választja. A várt eredmény `0`: az előjel nélküli 32 bites összeg modulo 2^{32} értendő. A korábbi build- és futtatási ciklussal ellenőrizhető.

3. megoldás — Segítség. A clobber-lista hiányzó eleme pontosan azt a fajta állapotváltozást takarja el a fordító elől, amit az asm blokk a pointeren keresztül, nem regiszterben végez.

3. megoldás — Teljes megoldás. A javított utolsó mező `: "memory"`. Ezután:

```
# Target: AArch64 Linux, qemu-user; javított lab-clobber.c.
aarch64-linux-gnu-gcc \
    -std=c11 -Wall -Wextra -march=armv8-a -mlittle-endian \
    -O2 -static lab-clobber.c -o lab-clobber-fixed

qemu-aarch64 ./lab-clobber-fixed
printf 'kilepesi allapot=%s\n' "$?"
```

Várt eredmény: `value=7`, majd `0` kilépési állapot. A hibás változat sikeres futása csak azt mutatná, hogy az adott fordítás nem használta ki a tévesen engedélyezett optimalizálást. Tesztelés mellett ezért a szerződés kézi ellenőrzése is szükséges.

10.1.9 Rövid összefoglaló

A GCC inline assembly két egymást kiegészítő leírásból áll: az A64 utasításokból és a fordítónak megadott szerződésből. Az operandusok írják le az értékek áramlását, a clobber-lista pedig a további állapotmódosításokat. A fordító maga osztja ki a regisztereket, ezért a sablonban következetesen kell használni a 32 vagy 64 bites nézetet. A `volatile` nem javítja ki a hiányzó memória- vagy regiszterdeklarációt. Optimalizálási szintek összehasonlítása és disassembly segíthet a hibakeresésben, de a szerződés helyességét nem helyettesíti.

- **Constraint:** az operandus elhelyezését és használatát meghatározó megkötés.
- **Clobber:** kimenetként nem modellezett állapotrombolás deklarációja.
- **Early clobber:** a bemenetek teljes felhasználása előtti kimenetírás jelölése.

10.1.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GCC: Extended Asm: operandusok, `volatile`, clobberek, az `m`, `=m`, `+m` memóriaoperandusok és AArch64 operandusmódosítók. **Ellenőrizve**: legújabb listázott kiadás GCC 16.2.
- GCC: Constraints for asm Operands és Constraints for asm Operands / Modifiers: regisztermegkötések, a `&` korai felülírás és a módosítók. **Ellenőrizve**: ugyanaz a 16.2-es kiadás.
- GNU assembler kézikönyv: az A64 utasítássablonok assembleroldali szintaxisa. **Ellenőrizve**: aktuális kiadás binutils 2.47.

Mérési állapot: a `lab-early`, a `lab-clobber`, a `lab-inline` és a `lab-memop` a GNU GCC 13.3.0 fő úton mérve (a lektor mérése): `lab-early` `-O0` és `-O2` mellett egyaránt `bad=40 good=21`; `lab-clobber` `-O0` mellett `value=7`, `-O2` mellett `value=1`, a hibás változat kilépési állapota mindkét esetben `0`; `lab-inline` `-O0`, `-O2` és `-O3` mellett `lab-inline: 4 teszt OK`, kilépési állapot `0`, a `-S` kimenetben az `adds/cset …, cs` pár megtalálható; `lab-memop` mindkét szinten `value=8 other=100`. Clang `-O2` alatt a hibás változat tünete elrejtőzhet (más regiszterkiosztás), ezért a laborban a saját fordítóval mért kimenetet kell rögzíteni; a javított változatok eredménye fordítófüggetlen. A hibás memóriaszerződés konkrét tünete szintén fordítófüggő.

10.2 Assembly függvény hívása C-ből és C hívása assemblyből

10.2.1 Miért kell külön szerződés a C és az assembly közé?

Egy C-program teljesítménykritikus részét külön assembly fájlban szeretnénk megírni, de a tesztelést, a kiírást és az általános programlogikát C-ben hagynánk. Fordított irányra is szükség lehet: az assembly rutin egy már meglévő C-függvényt használjon.

A két nyelv közötti határ nem automatikus típusellenőrzési pont. A C-fordító a deklarációból következtet a hívás módjára; az assembly szerzője pedig kézzel valósítja meg ugyanazt a szerződést. A linkelő elsősorban szimbólumokat és címhivatkozásokat kapcsol össze, nem C-függvényszignatúrákat ellenőriz.

Definíció — bináris interfész: az alkalmazás bináris interfésze (*Application Binary Interface*, ABI) meghatározza többek között az argumentumok, visszatérési értékek, regisztermegőrzések és

veremhasználat szabályait. A labor AArch64 Linux környezetében a függvényhívási szerződést az AAPCS64 adja.

Az Inline assembly GCC-ben esetében operandusokkal és clobber-listával tájékoztattuk a fordítót. Önálló assembly függvénynél ezt a szerepet a C-prototípus és az ABI tölti be. A keretkezeléshez a Prológus és epilógus: regisztermentés és keretkezelés ismeretére támaszkodunk.

A teljes alfejezet profilja: I. rész, A64 ISA, Armv8-A, AArch64 Linux, little-endian, LP64 adatmodell. Az assembly blokkok utasításai és regiszterei ebben a profilban értendők; direktíváik GNU assembler, AArch64 ELF környezethez tartoznak. Thumb-2 kód itt nem szerepel.

10.2.2 A forrásszintű és a gépi szerződés

A példában kizárólag `uint64_t` argumentumokat és visszatérési értékeket használunk. Ezek **előjel nélküli, 64 bites egészek**, így az összeadás és szorzás eredménye modulo 2^{64} értendő.

A labor C-interfészének eleme	AAPCS64-leképezés, A64
Első `uint64_t` argumentum	`X0`
Második `uint64_t` argumentum	`X1`
Harmadik `uint64_t` argumentum	`X2`
`uint64_t` visszatérési érték	`X0`
A hívásokon át megtartandó helyi érték	Például `X19`, belépési értékének mentésével
Visszatérési cím	`X30`; újabb `BL` felülírja
Veremmutató	`SP`; a példában végig 16 bájtra igazított

Ez nem általános szabály minden C-típusra. A lebegőpontos értékeknek és az összetett típusoknak külön besorolási szabályaik vannak; utóbbiakat a Több visszatérési érték és struktúra-visszaadás szerződése tárgyalja. A jelenlegi skaláris egész argumentumosztályban az első nyolc argumentum számára az A64 `X0`–`X7` használható, a további argumentumok veremre kerülnek.

A szimbólumkapcsolat két oldala:

- A C-beli `extern` deklaráció megadja a nevet és a típust, de nem szolgáltat függvénytörzset. Függvénydeklarációnál az `extern` elhagyható; itt szándékosan kiírjuk.
- Az AArch64 ELF/GNU as `.global` direktíva a szimbólumot más objektumfájlok számára láthatóvá teszi.
- Az ugyanitt használt `.type …, %function` és `.size` függvénymetaadatot ad az eszközöknek. **Ezek nem kódolják a teljes C-prototípust.**

Ezért egy sikeres linkelés még nem bizonyítja az interfész helyességét.

10.2.3 Kidolgozott példa: C → assembly → C

Munkakönyvtár: `~/arm-labor/a64/a037/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a037
cd ~/arm-labor/a64/a037
```

Négy forrásfájlt és egy Makefile-t készítünk:

- `lab-interop.h`: közös C-deklarációk;

- `lab-main.c`: tesztprogram;
- `lab-helper.c`: assemblyből hívott C-függvény;
- `lab-bridge.s`: C-ből hívott assembly függvények;
- `lab-Makefile`: reprodukálható build.

A számítás:

$$\mathrm{apply}(a, b, \mathrm{bonus}) = 3(a + b) + 1 + \mathrm{bonus} \pmod{2^{64}}.$$

Minden változó mértékegység nélküli, előjel nélküli 64 bites egész. A szorzást és az egy hozzáadását a C-segédfüggvény végzi.

10.2.3.1 Közös deklarációk és C-oldali teszt

A `lab-interop.h` tartalma:

```
#ifndef LAB_INTEROP_H
#define LAB_INTEROP_H

#include <stdint.h>

extern uint64_t lab_add(uint64_t a, uint64_t b);
extern uint64_t lab_apply(uint64_t a, uint64_t b, uint64_t bonus);
extern uint64_t lab_scale(uint64_t value);

#endif
```

A `lab-helper.c` tartalma:

```
#include "lab-interop.h"

uint64_t lab_scale(uint64_t value)
{
    return UINT64_C(3) * value + UINT64_C(1);
}
```

A `lab-main.c` tartalma:

```
#include <inttypes.h>
#include <stdio.h>
#include "lab-interop.h"

int main(void)
{
    const uint64_t sum = lab_add(7, 5);
    const uint64_t result = lab_apply(7, 5, 9);
    const uint64_t wrap = lab_add(UINT64_MAX, 1);

    printf("sum=%" PRIu64 " apply=%" PRIu64 " wrap=%" PRIu64 "\n",
           sum, result, wrap);

    return (sum == UINT64_C(12) &&
            result == UINT64_C(46) &&
            wrap == UINT64_C(0)) ? 0 : 1;
}
```

A `PRIu64` a `uint64_t` típushoz illeszkedő kiírást biztosítja. A közös fejléc a C-fájlok közötti eltéréseket fordításkor segít észlelni; az assembly implementációt továbbra is külön kell auditálni.

10.2.3.2 Az assembly implementáció

A `lab-bridge.s` tartalma:

```
// A64 ISA, Armv8-A; GNU as, AArch64 ELF, little-endian.
.arch armv8-a
.text

.p2align 2
.global lab_add
.type lab_add, %function
lab_add:
    .cfi_startproc
    add     x0, x0, x1
    ret
    .cfi_endproc
.size lab_add, .-lab_add

.p2align 2
.global lab_apply
.type lab_apply, %function
lab_apply:
    .cfi_startproc
    stp     x29, x30, [sp, #-32]!
    .cfi_def_cfa_offset 32
    .cfi_offset 29, -32
    .cfi_offset 30, -24
    mov     x29, sp
    str     x19, [sp, #16]
    .cfi_offset 19, -16

    mov     x19, x2
    add     x0, x0, x1
    bl      lab_scale
    add     x0, x0, x19

    ldr     x19, [sp, #16]
    .cfi_restore 19
    ldp     x29, x30, [sp], #32
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.size lab_apply, .-lab_apply

.section .note.GNU-stack,"",%progbits
```

A `lab_add` levélfüggvény: nem hív másik függvényt, és nem módosít megőrzendő regisztert. Nincs szüksége saját veremkeretre.

A `lab_apply` nem-levél függvény. A harmadik argumentumot nem hagyhatja egyszerűen az A64 `X2` regiszterben: a meghívott C-függvény ezt szabadon felhasználhatja. Ez akkor is igaz, ha a C-függvénynek csak egy argumentuma van.

Az A64 `X19` viszont megőrzendő regiszter. A `lab_apply` ezért:

1. elmenti saját hívójának A64 `X19` értékét;
2. saját köztes adatát az A64 `X19` regiszterbe helyezi;
3. meghívja a C-függvényt, amely köteles megőrizni ezt az értéket;
4. visszatérés előtt visszaállítja az eredeti A64 `X19` tartalmat.

Az A64 `X30` mentése ettől függetlenül szükséges: az A64 `BL` a C-segédfüggvényből való visszatérés címét írja bele.

A 32 bájtos keret kiosztása:

Eltolás az új A64 `SP`-hez képest	Tartalom	Méret
0	Belépési A64 `X29`	8 bájt
8	Belépési A64 `X30`	8 bájt
16	Belépési A64 `X19`	8 bájt
24	Nem használt kitöltés	8 bájt

A kitöltés biztosítja, hogy a keret mérete 16 bájt többszöröse legyen.

A két függvény C-ből hívható, végleges minta, ezért a Prológus és epilógus szabálya szerint CFI-t kap: a levélfüggvény `lab_add` csak a `.cfi_startproc`/`.cfi_endproc` párt, a keretet építő `lab_apply` a mentések helyét is. A `.cfi_offset` értékei a fenti kerettáblázatból következnek: a CFA a belépési `SP`, vagyis az új `SP` + 32.

Kódolási korlátok: a `STP`/`LDP` ±32, a `STR`/`LDR` 16 bájtos eltolása és a `BL` cél mind kódolható (lásd az A64 immediate-korlátok összefoglalóját); a `BL` végleges eltolása linkeléskor áll elő. Az AArch64 ELF/GNU as `.p2align 2` négybájtos igazítást jelent, nem két bájtot.

Az AArch64 ELF/GNU as `.note.GNU-stack` szekció nem végrehajtható veremigényt jelez a linkelőnek; nem futásidejű utasítás.

10.2.4 Vegyes build: GCC, assembler és linkelő

Az önálló fordítási egységek (*translation units*) külön objektumfájlokká alakulnak. A C-fájlokat GCC-vel fordítjuk; a kisbetűs `.s` fájlt közvetlenül az assemblerrel dolgozzuk fel.

A végső linkelést is a GCC meghajtóprogramra bízzuk. Ez elindítja a linkelőt, és hozzáadja a C-futási környezethez szükséges indítóobjektumokat és könyvtárakat. A puszta `ld` meghívás nem tenné meg mindezt automatikusan. A program belépési környezetét most a C-runtime szolgáltatja: nem írunk saját `_start` rutint.

A `lab-Makefile` tartalma; a receptsorok elején valódi tabulátor álljon:

```
# Target: AArch64 Linux, Armv8-A, little-endian, qemu-user.
CC := aarch64-linux-gnu-gcc
AS := aarch64-linux-gnu-as

CFLAGS := -std=c11 -O2 -g -Wall -Wextra -Wpedantic -march=armv8-a
ASFLAGS := -g -march=armv8-a
LDFLAGS := -static

.PHONY: lab-all lab-run lab-clean

lab-all: lab-interop

lab-interop: lab-main.o lab-helper.o lab-bridge.o
        $(CC) $(LDFLAGS) -o $@ $^

lab-%.o: lab-%.c lab-interop.h
        $(CC) $(CFLAGS) -c $< -o $@

lab-bridge.o: lab-bridge.s
        $(AS) $(ASFLAGS) -o $@ $<
```

```
lab-run: lab-interop
	qemu-aarch64 ./lab-interop

lab-clean:
	rm -f lab-main.o lab-helper.o lab-bridge.o lab-interop
```

A statikus linkelés miatt futtatáskor nincs szükség külön AArch64 dinamikus betöltő megadására. A buildhez viszont rendelkezésre kell állniuk az AArch64 C-könyvtár statikus állományainak és indítóobjektumainak. Az ezek hiányára utaló linkelési hiba környezeti probléma, nem ABI-hiba.

Target: qemu-user; host: Ubuntu 24.04 x86_64. Összeállítás, linkelés és futtatás:

```
make -f lab-Makefile lab-all
make -f lab-Makefile lab-run
printf 'make-status=%d\n' "$?"
```

Mérve (a lektor mérése, GCC 13.3.0, binutils 2.42, QEMU 8.2.2): a build kilépési állapota 0; a program kiírása `sum=12 apply=46 wrap=0`; a program és a `lab-run` cél kilépési állapota 0. A `make` ezenfelül a végrehajtott parancsokat is kiírhatja.

A számítás ellenőrzése:

Megfigyelési pont	Várt állapot, A64
`lab_apply` belépése	`X0=7`, `X1=5`, `X2=9`
Közvetlenül a C-hívás előtt	`X0=12`, `X19=9`
A C-függvény visszatérése után	`X0=37`, `X19=9`
`lab_apply` visszatérésekor	`X0=46`; `SP`, `X19`, `X29` belépési értéke visszaállt

10.2.4.1 ABI-ellenőrzés a közös teszthívóval

A fenti táblázat utolsó sora állítás, amelyet a `sum=12 apply=46` kimenet nem igazol: egy `X19`-et rontó `lab_apply` is kiírhatná a helyes összeget. Ezt a Prológus és epilógus közös teszthívójával ellenőrizzük, amely magas bites mintákkal a teljes megőrzött regiszterkészletet és az `SP`-t vizsgálja. A teszthívó fájljait átmásoljuk:

```
cp ~/arm-labor/a64/a029/lab-abi-call.h ~/arm-labor/a64/a029/lab-abi-call.S .
```

A `lab-abi-main.c` tartalma:

```
#include <inttypes.h>
#include <stdio.h>
#include "lab-abi-call.h"
#include "lab-interop.h"

int main(void)
{
    uint64_t r_add = 0, r_apply = 0;
    uint64_t m_add = lab_abi_call((lab_abi_fn)lab_add, 7, 5, 0, 0, 0, 0, &r_add);
    uint64_t m_apply = lab_abi_call((lab_abi_fn)lab_apply, 7, 5, 9, 0, 0, 0, &r_apply);

    printf("lab_add   maszk=0x%05" PRIx64 " eredmeny=%" PRIu64 "\n", m_add, r_add);
    printf("lab_apply maszk=0x%05" PRIx64 " eredmeny=%" PRIu64 "\n", m_apply, r_apply);

    return (m_add == 0 && r_add == 12 && m_apply == 0 && r_apply == 46) ? 0 : 1;
}
```

A `(lab_abi_fn)` típuskényszerítés C-ben megengedett: a `void (*)(void)` típusra minden függvénymutató átalakítható, és a `-Wextra` sem figyelmeztet rá. A függvényt nem C-ből hívjuk ezen a típuson át, hanem a teszthívó `BLR` utasítása.

```
# Target: AArch64 Linux, qemu-user.
aarch64-linux-gnu-gcc -std=c11 -O2 -Wall -Wextra -march=armv8-a -static \
    lab-abi-main.c lab-helper.c lab-bridge.s lab-abi-call.S -o lab-abi-interop
qemu-aarch64 ./lab-abi-interop
printf 'kilepesi_allapot=%d\n' "$?"
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2):

```
lab_add   maszk=0x00000 eredmeny=12
lab_apply maszk=0x00000 eredmeny=46
kilepesi_allapot=0
```

A nulla maszk a `lab_apply` esetén két szerződést igazol egyszerre: a saját `X19`-mentését és a közben meghívott, GCC által fordított `lab_scale` megőrzési fegyelmét. Ellenpróbaként töröld a `lab_apply` `ldr x19, [sp, #16]` sorát: a maszk `0x00001` lesz (a lektor mérése). Az eredeti `lab-interop` kimenete ekkor fordítófüggő: ha a `main` a `sum` értékét véletlenül éppen `X19`-ben tartja a `lab_apply` hívásán át, a kiírás `sum=9` lesz (a `bonus` értéke szivárog át), különben helyesnek látszik. A `sum=9` eset Clang-specifikus: a lektor mérése szerint a GCC 13.3.0-fordította `main` nem `X19`-ben tartja a `sum`-ot, ezért a GCC-vel fordított `lab-interop` a törölt `ldr x19` sorral is `sum=12`-t ír ki. A teszthívó maszkja viszont a fordítótól függetlenül jelzi a hibát.

10.2.5 Objektumvizsgálat és tipikus hibák

Az áthelyezési bejegyzés (*relocation*) azt jelöli, hogy egy hivatkozás végleges értékét később kell meghatározni. Az assembly objektum a C-függvény végleges címét még nem ismeri.

Target: qemu-userhez készült AArch64 ELF; hostoldali vizsgálat, futtatás nélkül:

```
aarch64-linux-gnu-readelf -Ws lab-bridge.o
aarch64-linux-gnu-objdump -dr lab-bridge.o
aarch64-linux-gnu-objdump -d --disassemble=lab_scale lab-interop
```

Mérve (a lektor mérése, binutils 2.42): a parancsok kilépési állapota 0, és a kimenet a leírásnak megfelel. A szimbólumtáblában `lab_add` és `lab_apply` globális függvénydefiníció, `lab_scale` pedig még nem definiált, `UND` jelölésű szimbólum. A tárgykód disassemblyjében a hívásnál `R_AARCH64_CALL26` áthelyezés várható. A végleges ELF-ben már a feloldott hívást vizsgáljuk.

Tünet	Hipotézis és adatgyűjtés	Egy változtatás, majd ellenőrzés
`undefined reference to lab_add`	A szimbólumtábla szerint nem globális, vagy neve eltér	Javítsuk az exportot vagy a nevet; linkeljünk újra
`lab_scale` nem található	Hiányzik a megfelelő objektum a linkparancsból	Vegyük fel a `lab-helper.o` fájlt; építsünk újra
Csak nagy bemenetnél hibás az eredmény	Disassemblyben A64 `W0/W1` szerepel a szükséges `X0/X1` helyett	Javítsuk az operandusszélességet; futtassunk határértéktesztet
A bónusz néha eltűnik	Hívó által mentendő regiszterben maradt a C-hívás alatt	Tároljuk mentett A64 `X19`-ben; ismételjük a teszteket

Tünet	Hipotézis és adatgyűjtés	Egy változtatás, majd ellenőrzés
A visszatérés hibás vagy a program elakad	Hiányzik az A64 `X30` mentése	Állítsuk helyre a prológus–epilógus párt; ellenőrizzük a vezérlési utat

Különösen veszélyes a prototípuseltérés. Ha a C-deklaráció két argumentumot ír le, az assembly pedig harmadikat is vár, az A64 `X2` tartalmára nincs argumentumként vállalt garancia. Ha a C-oldal `uint32_t`, az assembly pedig `uint64_t` értéket feltételez, már az argumentum-előkészítés és az eredmény értelmezése is eltérhet.

A 32 bites érték felső regiszterbitjeire nem szabad általános, 64 bites argumentumszerződésként támaszkodni. Ugyanígy egy előjeles 32 bites érték sem válik automatikusan helyesen előjelkiterjesztett 64 bites argumentummá. **A típus, a szélesség és az argumentumszám együtt az interfész része.**

10.2.6 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért nem garantálja a sikeres linkelés, hogy a `lab_apply` három `uint64_t` argumentumot kap? Miért kell megőriznie az A64 `X19` belépési értékét, ha a C-segédfüggvény is köteles megőrizni azt?

2. (implementációs, alapszint — cél: rutinépítés) Adj a projekthez `uint64_t lab_max(uint64_t a, uint64_t b)` assembly függvényt, amely előjel nélküli maximumot ad vissza. Ellenőrizd a 7 és 5, valamint a `0x8000000000000001` és 7 bemenetpárral.

3. (hibakeresési, középszint — cél: megértés-ellenőrzés) A `lab_add` A64 utasítását szándékosan módosítsd `add w0, w0, w1` alakra. Miért megy át továbbra is az eredeti teszt? Készíts olyan ellenpróbát, amely kimutatja a hibát, majd disassemblyvel igazold a javítást.

10.2.7 Megoldások

1. megoldás — Segítség. A linkelő és a C-fordító két különböző ellenőrzési szintet képvisel — gondold át, melyik szint mit lát a kézzel írt assembly esetén, és hogy az A64 `X19` megőrzése kinek a szerződését szolgálja (a hívóét vagy a hívottét).

1. megoldás — Teljes megoldás. A linkelő a szimbólumneveket és az objektumok kapcsolatait oldja fel; a kézzel írt assemblyhez nincs teljes C-típusellenőrzés. Az A64 `X19` megőrzése pedig egymásba ágyazott szerződés: a C-segédfüggvény a `lab_apply` által elhelyezett bónuszt őrzi meg, míg a `lab_apply` saját hívójának korábbi értékét köteles visszaadni.

2. megoldás — Segítség. Az előjel nélküli összehasonlításhoz a megfelelő feltételkódot válaszd (nem az előjeles változatot) — gondold át, miért számítana a magas helyiértékű bit előjelesen.

2. megoldás — Teljes megoldás. A fejlécbe kerüljön:

```
extern uint64_t lab_max(uint64_t a, uint64_t b);
```

A `lab-bridge.s` fájlban, a záró veremjelölő szekció előtt:

```
// A64 ISA; GNU as, AArch64 ELF; a .text szekcióban.
.p2align 2
.global lab_max
.type lab_max, %function
lab_max:
    cmp     x0, x1
    csel    x0, x0, x1, hs
    ret
.size lab_max, .-lab_max
```

Az A64 `HS` előjel nélküli „nagyobb vagy egyenlő” feltétel. Előjeles feltétellel a magas helyiértékű bitet tartalmazó második teszt hibás lehetne.

A `main` végső visszatérése elé illeszthető ellenőrzés:

```
if (lab_max(7, 5) != UINT64_C(7) ||
    lab_max(UINT64_C(0x8000000000000001), 7) !=
        UINT64_C(0x8000000000000001))
    return 1;
```

A korábbi, qemu-user célú build- és futtatási parancsok változatlanok; a várt kilépési állapot 0.

3. megoldás — Segítség. A kis teszt-bemenetek nem haladják meg a 32 bitet — gondold át, milyen bemenettel lehet olyan 64 bites összeget képezni, amelynek alsó 32 bitje önmagában nem árulkodik a hibáról.

3. megoldás — Teljes megoldás. A kis bemenetek elférnek 32 biten. A túlcsordulási tesztben pedig az alsó 32 bit összeadása is nullát ad, ezért az sem különbözteti meg a hibás implementációt.

A `main` végső visszatérése elé:

```
if (lab_add(UINT64_C(0x100000000), UINT64_C(5)) !=
    UINT64_C(0x100000005))
    return 1;
```

A várt helyes eredmény előjel nélkül 4294967301. A hibás A64 `W0`-írás lenullázza az A64 `X0` felső felét, így 5 lesz az eredmény. A program ekkor 1 állapottal lép ki; a `make` ezt hibaként jelzi, saját hibakódja nem feltétlenül 1.

A korábbi `objdump` paranccsal ellenőrizzük az operandusokat, állítsuk vissza az A64 `add x0, x0, x1` utasítást, majd ugyanazzal az ellenpróbával várjunk ismét 0 programkilépési állapotot.

10.2.8 Rövid összefoglaló

Az önálló assembly függvény és a C-program együttműködését a deklaráció, a szimbólumkapcsolat és az AAPCS64 hívási szabályai együttesen határozzák meg. A sikeres linkelés önmagában nem ellenőrzi az argumentumok típusát vagy számát. C-hívás előtt az assembly rutinnak meg kell őriznie visszatérési címét és a híváson át élő adatokat, visszatéréskor pedig helyre kell állítania a megőrzendő állapotot. A GCC meghajtóprogram leegyszerűsíti a C-runtime-mal történő linkelést. A szimbólumtábla, az áthelyezések és a határértéktesztek együtt adnak használható ellenőrzési módszert.

- **ABI:** a komponensek gépi együttműködésének szerződése.
- **Fordítási egység:** önállóan feldolgozott forrásegység.
- **Áthelyezési bejegyzés:** később feloldandó címhivatkozás leírása.

10.2.9 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- AAPCS64 – Arm hivatalos ABI-dokumentum: argumentumátadás, regisztermegőrzés és veremkövetelmények. *Ellenőrizve*: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- GCC: Link Options: a meghajtóprogram linkelési feladatai és a statikus linkelés. *Ellenőrizve*: a GCC-dokumentáció legújabb listázott kiadása 16.2; a helyben telepített verzió ettől eltérhet.
- GNU assembler kézikönyv: AArch64-szintaxis, szimbólum- és ELF-direktívák. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a könyv szerzői a kódot a referencia-eszközlánccal (GNU GCC 13.3.0, binutils 2.42, `qemu-aarch64` 8.2.2) itt nem fordították és nem futtatták; a lektori mérés viszont ezen a fő úton történt. A `make -f lab-Makefile lab-all` + `lab-run` kimenete `sum=12 apply=46 wrap=0` és 0-s kilépési állapot; a `lab-abi-interop` két `0x00000` maszkot ad `12`, illetve `46` eredménnyel; a readelf/objdump állítások (GLOBAL FUNC `lab_add`/`lab_apply`, UND `lab_scale`, `R_AARCH64_CALL26`) egyeznek; a törölt `ldr x19` sorral a maszk `0x00001`. A korábbi helyettesítő eszközláncos mérés (`zig cc`, Clang, musl, Unicorn-alapú emulátor) `sum=9`-et adott; ez a Clang-fordította `main` regiszterkiosztására jellemző, GCC-vel nem reprodukálódik (a lektor mérése szerint a GCC-vel fordított `lab-interop` a hiba injektálása után is `sum=12`-t ír ki). A laborban rögzíteni kell az Ubuntu 24.04 telepített GCC/binutils/QEMU csomagverzióját, és ellenőrizni kell a statikus AArch64 C-runtime elérhetőségét. Teljesítménymérést ez a példa nem tartalmaz.

10.3 Fordítói optimalizálási szintek hatása a generált kódra

10.3.1 Miért változik meg ugyanannak a C-függvénynek a gépi alakja?

Egy C-ből hívott számolófüggvény helyes eredményt ad, de a disassemblyben sok veremműveletet látsz. Érdemes ezeket kézzel assemblyre átírni? Először azt kell ellenőrizni, hogy optimalizált kódot vizsgálsz-e. Egy optimalizálás nélkül fordított függvény memóriaforgalmának jelentős része nem az algoritmusból, hanem a fordítóprogram választott kódgenerálási stratégiájából eredhet.

Ebben a laborban **ugyanazt a C-forrást** fordítjuk le `-O0`, `-O1`, `-O2` és `-O3` beállítással. A cél nem egy adott GCC-kiadás utasítássorozatának bemagolása, hanem három összefüggés felismerése:

- hogyan csökkenhet a lokális változókhoz kapcsolódó memóriaforgalom;
- hogyan tűnhetnek el belső függvényhívások;
- hogyan változhat meg a ciklus szerkezete.

Profil: I. rész, AArch64, A64 ISA, Armv8-A alapprofil, little-endian. Az alábbi assemblyregiszterek és utasítások kizárólag ebben a környezetben értendők. Előfeltétel az Assembly függvény hívása C-ből és C hívása assemblyből alfejezet hívási és build-ismerete.

10.3.2 Optimalizálás: a viselkedés marad, a megvalósítás változhat

A **fordítói optimalizálás** (*compiler optimization*) a program belső reprezentációjának olyan átalakítása, amely megőrzi a C által előírt megfigyelhető viselkedést, miközben például gyorsabb vagy kisebb gépi kódot eredményezhet. Nem követelmény, hogy minden C-változónak külön memóriacím, minden kifejezésnek külön utasítás feleljen meg.

Ez csak a C szerint értelmezett működésre vonatkozik. Az előjeles egész túlcsordulása például nem definiált viselkedés; az optimalizálás kikapcsolása sem teszi szabályossá. A labor ezért `uint32_t` típusú, **előjel nélküli** aritmetikát használ: az eredmények modulo 2^{32} értendők.

Az optimalizálási szint nem teljesítménygarancia:

Kapcsoló	Jellemző cél és hatás	Fontos korlát
`-O0`	Kevés optimalizálás; többnyire könnyebben követhető forrásszerkezet	Nem jelenti minden átalakítás teljes hiányát
`-O1`	Alapvető egyszerűsítések, holt számítások eltávolítása, kevesebb felesleges adatmozgatás	A pontos átalakításkészlet GCC- és célfüggő
`-O2`	Gyakori kiindulópont üzemi kódhoz; szélesebb optimalizálási készlet	Vektorizálás is megjelenhet, kiadástól és költségmodelltől függően
`-O3`	További, különösen ciklusokra és kódbeágyazásra irányuló átalakítások	Nagyobb kódot is adhat; nem szükségképpen gyorsabb

A kapcsolók nem szabványos C-fogalmak, hanem GCC-beállítások. A laborban használt kiadás ténylegesen engedélyezett optimalizálásait külön is le lehet kérdezni.

10.3.3 Kidolgozott példa: összegzés belső segédfüggvénnyel

Munkakönyvtár: `~/arm-labor/a64/a038/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a038
cd ~/arm-labor/a64/a038
```

Mentsd a következő fájlt `lab-kernel.c` néven:

```
/* AArch64/A64, Armv8-A, little-endian célprofil. */
#include <stddef.h>
#include <stdint.h>

static uint32_t lab_step(uint32_t x)
{
    return x * UINT32_C(3) + UINT32_C(1);
}

uint32_t lab_sum(const uint32_t *a, size_t n)
{
    uint32_t sum = 0;
```

```
    for (size_t i = 0; i < n; ++i) {
        uint32_t dead = a[i] ^ UINT32_C(0x5a5a5a5a);
        (void)dead;
        sum += lab_step(a[i]);
    }

    return sum;
}
```

A függvény szerződése:

- `n > 0` esetén `a` legalább `n` olvasható `uint32_t` elemet jelöl;
- az elemek a hívás alatt nem változnak;
- `n == 0` esetén nincs memóriaolvasás, ezért nullmutató is átadható;
- a visszatérési érték előjel nélküli, 32 bites összeg.

A `dead` változó szándékosan felesleges. A `(void)dead` kifejezés jelzi a szándékosan fel nem használt értéket, de **nem teszi megfigyelhetővé a számítást**, és nem akadályozza meg annak eltávolítását.

A tesztprogram legyen `lab-test.c`:

```
/* AArch64/A64, Armv8-A, little-endian célprofil. */
#include <inttypes.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

uint32_t lab_sum(const uint32_t *a, size_t n);

int main(void)
{
    static const uint32_t zero[] = {0};
    static const uint32_t small[] = {1, 2, 3, 4};
    static const uint32_t wrap[] = {UINT32_MAX, 0, 1};
    static const uint32_t tail[] = {0, 1, 2, 3, 4, 5, 6};

    const struct {
        const uint32_t *data;
        size_t n;
        uint32_t expected;
    } tests[] = {
        {NULL, 0, 0},
        {zero, 1, 1},
        {small, 4, 34},
        {wrap, 3, 3},
        {tail, 7, 70}
    };

    const size_t count = sizeof tests / sizeof tests[0];

    for (size_t i = 0; i < count; ++i) {
        uint32_t got = lab_sum(tests[i].data, tests[i].n);

        if (got != tests[i].expected) {
            fprintf(stderr,
                    "FAIL: teszt=%zu kapott=%" PRIu32
                    " vart=%" PRIu32 "\n",
                    i, got, tests[i].expected);
            return 1;
        }
    }
```

```
    printf("OK: %zu teszt\n", count);
    return 0;
}
```

A túlcsordulási teszt várt eredményének **levezetése**, előjel nélküli olvasatban:

- `lab_step(UINT32_MAX)` értéke modulo 2^{32} számolva `0xfffffffe`;
- `lab_step(0)` értéke `1`;
- `lab_step(1)` értéke `4`;
- összegük modulo 2^{32} értéke `3`.

Ez szemantikai ellenőrzés, nem teljesítménymérés.

10.3.4 Fordítás, assemblyelőállítás, linkelés és futtatás

A `gcc -S` megáll az assemblyforrás előállításánál: nem készít objektumfájlt és nem linkel. Az alábbi munkafolyamat ezért külön végzi el az assembly összeállítását és a linkelést.

A tesztprogram mindig `-O0` beállítással készül; kizárólag a vizsgált modul optimalizálási szintje változik. Nem használunk linkeléskori optimalizálást (*link-time optimization*, LTO), így a fordító nem látja együtt a tesztadatokat és a függvény implementációját. A következő blokk zárójelbe tett alhéjban fut, hogy a `set -eu` ne maradjon érvényben az interaktív terminálban; így egy későbbi, szándékosan nem nulla állapotú parancs nem zárja be a shellt.

```
# Host: Ubuntu 24.04 x86_64.
# Target: qemu-user, AArch64/A64, Armv8-A, little-endian.
# Előfeltétel: cross GCC/binutils, QEMU és statikus cross libc.
( set -eu

{
    aarch64-linux-gnu-gcc --version
    aarch64-linux-gnu-as --version
    aarch64-linux-gnu-ld --version
    qemu-aarch64 --version
} > lab-versions.txt

aarch64-linux-gnu-gcc \
    -std=c11 -Wall -Wextra -O0 -g \
    -march=armv8-a -mlittle-endian \
    -c lab-test.c -o lab-test.o

for level in 0 1 2 3; do
    aarch64-linux-gnu-gcc \
        -std=c11 -Wall -Wextra -O"$level" -g \
        -march=armv8-a -mlittle-endian \
        -fverbose-asm -S lab-kernel.c \
        -o "lab-kernel-O${level}.s"

    aarch64-linux-gnu-gcc \
        -march=armv8-a -mlittle-endian \
        -c "lab-kernel-O${level}.s" \
        -o "lab-kernel-O${level}.o"

    aarch64-linux-gnu-gcc \
        -march=armv8-a -mlittle-endian -static \
        lab-test.o "lab-kernel-O${level}.o" \
        -o "lab-O${level}"

    aarch64-linux-gnu-objdump -dr \
```

```
        "lab-kernel-O${level}.o" \
        > "lab-kernel-O${level}.dis"

    printf 'O%s: ' "$level"
    qemu-aarch64 "./lab-O${level}"
done
)
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2): mind a négy szintnél `O0: OK: 5 teszt`, illetve az ennek megfelelő `O1`, `O2`, `O3` kezdetű sor. Mindegyik program kilépési állapota `0`; eltérő eredménynél a tesztprogram `1` értékkel lép ki.

A statikus linkeléshez szükséges cross libc hiánya build-környezeti hiba, nem optimalizálási jelenség.

A generált forrás összevetéséhez:

```
# Target: qemu-user számára generált AArch64/A64 kód vizsgálata.
# A diff 1-es állapota eltérést jelent, nem hibás fordítást.
diff -u lab-kernel-O0.s lab-kernel-O2.s || test "$?" -eq 1

aarch64-linux-gnu-objdump \
    -d --disassemble=lab_sum lab-O2

aarch64-linux-gnu-gcc \
    -march=armv8-a -mlittle-endian -O2 \
    -Q --help=optimizers > lab-options-O2.txt
```

A GNU assembler AArch64-célú `.cfi_*`, `.loc` és `.size` direktívái metaadatokat írnak le, nem végrehajtandó A64 utasítások. A fájl sorainak száma ezért nem utasításszám. Az `objdump` segít elkülöníteni a tényleges gépi kódot.

10.3.5 Mit keress az eredményben?

10.3.5.1 Regiszterek a veremhelyek helyett

A **regiszterallokáció** (*register allocation*) azt rendeli hozzá, hogy a program élő értékei mikor mely fizikai regiszterekben legyenek. Ha egy értéket memóriába kell kitenni, majd később visszatölteni, az regiszter-kimentés (*spill*) és visszatöltés lehet. Nem minden veremművelet ilyen: argumentummentés, lokális objektum és ABI szerinti regisztermegőrzés is állhat mögötte.

`-O0` mellett gyakran külön veremhely tartozik az indexhez, az összeghez és az argumentumok másolatához. Egy módosítás ilyenkor A64 `LDR`, aritmetikai művelet és `STR` sorozatként jelenhet meg.

Optimalizálva az index és az összeg végig regiszterben maradhat. Ráadásul az index külön értékként meg is szűnhet: helyette egy mutató és egy hátralévő elemszám vezérelheti a ciklust.

Fontos árnyalat: regiszterallokáció `-O0` mellett is történik. A különbség nem pusztán „jobb regiszterválasztás”, hanem az allokáció elé kerülő programreprezentáció egyszerűsítése is.

10.3.5.2 A segédfüggvény beágyazása

A **függvénybeágyazás** (*function inlining*) a hívás helyére illeszti a hívott függvény műveleteit.

Az A64 kódban keresd a `lab_step` felé irányuló `BL` utasítást. Optimalizálva a szorzás és összeadás közvetlenül a ciklustörzsben jelenhet meg, a belső hívás pedig eltűnhet. A C `inline` kulcsszó ehhez nem szükséges; jelenléte sem jelent általános beágyazási garanciát.

Elméleti, feltételes levezetés: ha minden feldolgozott elemhez pontosan egy hagyományos segédfüggvény-hívás tartozik, akkor `n` elemhez `n` darab A64 `BL` és `n` darab A64 `RET` végrehajtás kapcsolódik. Beágyazáskor ez a belső hívási út eltűnik. Ebből nem következik meghatározott ciklusszám-megtakarítás.

A hatás továbbgyűrűzhet: a külső függvény levélfüggvénnyé válhat, ezért a belső hívás miatt szükséges A64 `X30`-mentés is feleslegessé válhat. A kívülről hívható `lab_sum` AAPCS64-szerződése ettől nem változik.

10.3.5.3 Holt számítás eltávolítása

A **holt kód eltávolítása** (*dead code elimination*, DCE) megszünteti azokat a számításokat, amelyek eredménye nem befolyásolja a megfigyelhető viselkedést.

A példában a `dead` előállítása eltávolítható: az értéket sem vissza nem adjuk, sem más számításban nem használjuk. A vizsgált érvényes bemeneteken ez nem változtatja meg a program eredményét.

Ne várd azonban, hogy `-O0` garantáltan megtartja. Egy egyszerű felesleges kifejezés már korábbi fordítási lépésekben is eltűnhet. Az optimalizálási szintek összehasonlításánál **megfigyelést rögzíts**, ne előre elvárt különbséget erőltess.

A `volatile` hozzáadása nem jó módszer az eredeti program vizsgálatára: az új, megfigyelendő hozzáférésekkel megváltoztatná a feladatot.

10.3.5.4 A ciklus alakja

GCC 13.3.0 `-O2` mellett mérve (a lektor mérése) a `lab_sum` 13 utasításból áll: a `lab_step` beágyazódik (`add w1, w1, w1, lsl #1`, majd `add w1, w1, #1`), és a ciklust végmutató-összehasonlítás vezérli. `-O0` mellett 45 utasítás és külön `bl lab_step` látható; `-O3` mellett 48 utasítás, NEON-vektorizált kód.

Az alábbi A64 részlet **illusztratív, normalizált skaláris ciklustörzs**, nem a laborban ellenőrzött GCC-kimenet betű szerinti alakja és nem önálló program. Belépéskor az A64 `X0` a következő elemre mutat, `X1` pozitív hátralévő elemszámot tartalmaz, `W2` az előjel nélküli részösszeg.

```
// A64 ISA, AArch64, Armv8-A, little-endian.
// Szemléltető részlet; a teljes programot a C-forrásból állítjuk elő.
.Lloop:
    ldr     w3, [x0], #4
    add     w3, w3, w3, lsl #1
    add     w3, w3, #1
    add     w2, w2, w3
    subs    x1, x1, #1
    b.ne    .Lloop
```

Az A64 `LDR` itt betölt egy 32 bites elemet, majd a mutatót négy bájttal növeli. A post-index `#4` eltolás belefér a mezőbe.

A következő A64 `ADD` az $x + (x \ll 1)$ azonossággal számítja ki a `3*x` értéket, modulo 2^{32}. Az itt használt eltolások, konstansok és a közeli `B.NE` cél mind kódolhatók (lásd az A64 immediate-korlátok összefoglalóját).

Más GCC-kimenet összehasonlíthatja a mutatót egy végcímmel, több iterációt egyesíthet, vagy SIMD-feldolgozást választhat. Utóbbit itt csak azonosítjuk; részletesen a Vektorizált összeadás/szorzás: egy skaláris ciklus NEON-átirata tárgyalja.

10.3.6 Hibák, ellenpróbák és értelmezési korlátok

Használd a **tünet → hipotézis → adatgyűjtés → egy változtatás → ellenőrzés** munkamenetet.

Tünet	Hipotézis és adatgyűjtés	Kontrollált ellenpróba
Sok veremművelet maradt	Más fájlt vagy a `-O0` változatot nézed	Vizsgáld a megfelelő `lab-kernel-O2.o` disassemblyjét
Nincs külön `lab_step` hívás	A segédfüggvény beágyazódott	Tiltsd meg csak ennek beágyazását, majd fordíts újra
`-O2` és `-O3` azonos	Ehhez a függvényhez ugyanazt a megoldást választotta a fordító	Ellenőrizd a kapcsolókat; az azonosság önmagában nem hiba
Csak optimalizálva hibás az eredmény	Nem definiált C-viselkedés vagy hibás C–assembly szerződés lehet jelen	Ellenőrizd a határokat, típusokat, megőrzendő regisztereket
GDB szerint egy változó „optimized out”	Nincs önálló, követhető tárolási helye	Ugyanazt a kódot fordítsd `-O0 -g` beállítással

A fordító döntéseihez kiegészítő jelentést kérhetsz:

```
# Target: qemu-user/AArch64; elemzési build, futtatás nélkül.
aarch64-linux-gnu-gcc \
    -std=c11 -O3 -g -march=armv8-a -mlittle-endian \
    -fopt-info-optimized=lab-opt-O3.txt \
    -S lab-kernel.c -o lab-report-O3.s
```

A jelentés tartalma és részletessége kiadásfüggő; hiányzó bejegyzés nem bizonyítja egy átalakítás hiányát.

`-O0 -g` tanuláshoz azért kényelmes, mert általában közvetlenebb a kapcsolat a C-sorok és a gépi műveletek között. A `-g` azonban optimalizálással együtt is használható. A GCC `-Og` külön hibakeresési kompromisszumot kínál, de most a négy kijelölt szintet hasonlítjuk össze.

Üzemi kiindulópontnak gyakran `-O2` célszerű; `-O3` akkor előnyös, ha a reprezentatív tesztek ezt igazolják. A kevesebb utasítás vagy kisebb veremkeret önmagában nem teljesítménymérés. QEMU user-mode alatt nincs garantált, hardverhű abszolút ciklusszám; az emulált futási időből nem vezethető le egy fizikai AArch64 processzor gyorsulása.

10.3.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Az optimalizált változatban nincs külön veremhely a `sum` változónak, és a `dead` számítása sem látható. Melyik érték maradt szemantikailag szükséges? Miért nem bizonyítja a kevesebb veremművelet önmagában a gyorsulást?

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a tesztet egy 17 elemű tömbbel, amelyre `a[i] = i`. Teszteld minden `n` hosszúsággal `0` és `17` között. A várt eredményt ne a vizsgált segédfüggvénnyel számítsd ki.

3. (hibakeresési, középszint — cél: megértés-ellenőrzés) A `-O3` változatból eltűnt a `lab_step` hívása. Tegyél a segédfüggvényre GCC `__attribute__((noinline))` attribútumot. Csak ezt módosítsd, fordítsd újra a négy változatot, és keresd meg a hívást. Változott-e a teszteredmény, a veremkezelés vagy a ciklus alakja?

10.3.8 Megoldások

1. megoldás — Segítség. Melyik változó értéke jelenik meg a függvény visszatérési értékében, és melyiké nem jelenik meg sehol a megfigyelhető kimenetben?

1. megoldás — Teljes megoldás. A részösszeg továbbra is szükséges, csak regiszterben vagy más, vele ekvivalens alakban él. A `dead` értéke szükségtelen, ezért a számítása megszüntethető. A veremműveletek számának csökkenése lehetséges előny, de a végrehajtási időt adatfüggőségek, elágazások, memóriahozzáférések és a mikroarchitektúra is befolyásolják.

2. megoldás — Segítség. Számítsd ki a várt összeget egy zárt képlettel (ne a `lab_step`-pel), és futtasd végig minden `n`-re `0`-tól `17`-ig egy külön ciklusban, mielőtt a meglévő tesztlistához csatolnád.

2. megoldás — Teljes megoldás. Az előjel nélküli tömbelemek összege $n(n-1)/2$, ezért ebben a kis tartományban a várt eredmény:

$$S(n) = 3\frac{n(n-1)}{2} + n.$$

Itt `n` mértékegység nélküli elemszám, `S(n)` mértékegység nélküli egész eredmény; `n` tartománya `0`–`17`, így nincs túlcsordulás. A következő részlet a meglévő `main` függvénybe, a sikerüzenet elé illeszthető:

```
/* AArch64/A64 célprofil; kiegészítés a lab-test.c main függvényéhez. */
uint32_t sequence[17];

for (size_t i = 0; i < 17; ++i)
    sequence[i] = (uint32_t)i;

for (size_t n = 0; n <= 17; ++n) {
    uint64_t k = n;
    uint32_t expected = k == 0
        ? 0
        : (uint32_t)(3 * k * (k - 1) / 2 + k);

    if (lab_sum(sequence, n) != expected)
        return 1;
}
```

A korábbi build-lépéssor változatlanul használható. Ellenőrző értékek: `S(0) = 0`, `S(4) = 22`, `S(17) = 425`. Minden változat várt kilépési állapota továbbra is `0`.

3. megoldás — Segítség. Az attribútum a `lab_step` deklarációjához tartozik, a `static` és a visszatérési típus közé — nézd meg a GCC attribútumszintaxisát. Gondold át, mit tilt meg a `noinline`: csak a beágyazást, a többi optimalizálást nem, és az algoritmust sem változtatja meg.

3. megoldás — Teljes megoldás. A deklaráció kezdete legyen `static __attribute__((noinline)) uint32_t lab_step(uint32_t x)`. A várt megfigyelés a belső hívás visszatérése; a konkrét A64 utasítássorozatot az `objdump` kimenete dönti el. A visszatérési cím és a híváson át élő értékek megőrzése ismét szükségessé válhat. A hívás a ciklus további optimalizálását is korlátozhatja. Az eredménynek változatlannak kell maradnia: az attribútum kódgenerálási ellenpróba, nem az algoritmus módosítása.

10.3.9 Rövid összefoglaló

Az optimalizálási szint a C-program megengedett megvalósításai közül segít választani; nem módosíthatja a definiált megfigyelhető viselkedést. Ugyanazt a forrást külön assemblyfájlokba fordítva felismerhető a veremforgalom csökkenése, a belső hívások eltűnése és a ciklus átszervezése. A pontos kód GCC- és célfüggő, ezért a megfigyelést tesztekkel, disassemblyvel és rögzített build-kapcsolókkal kell alátámasztani. A `-O0` tanulási és hibakeresési előnye nem üzemi teljesítményelőny; a `-O3` pedig nem automatikusan gyorsabb a `-O2` szintnél. A labor szemantikai összehasonlítás, nem hardveres sebességmérés.

- **Regiszterallokáció:** élő értékek fizikai regiszterekhez rendelése.
- **Beágyazás:** függvénytörzs beillesztése a hívás helyére.
- **Holt kód eltávolítása:** megfigyelhető hatás nélküli számítások megszüntetése.

10.3.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GCC: Options That Control Optimization: optimalizálási szintek, cél- és kiadásfüggő beállítások. *Ellenőrizve*: a GCC-dokumentáció legújabb listázott kiadása 16.2.
- GCC: An Inline Function is As Fast As a Macro: a beágyazás és az `inline` kapcsolatának korlátai. *Ellenőrizve*: ugyanaz a 16.2-es kiadás.
- GCC: Options for Debugging Your Program: hibakeresési információ optimalizált kódban. *Ellenőrizve*: ugyanaz a 16.2-es kiadás.
- GNU assembler kézikönyv: assemblyszintaxis és direktívák. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.

Ellenőrzési hiányok: a példák a lektor mérési környezetében (GCC 13.3.0, QEMU 8.2.2) lefutottak, az itt közölt kimenet és utasításszámok onnan származnak. A laborban rögzíteni kell az Ubuntu 24.04 telepített eszközverzióit, ellenőrizni a statikus cross libc rendelkezésre állását, továbbá megfigyelni a tényleges beágyazási és vektorizálási döntéseket. A normalizált ciklustörzs továbbra is illusztratív; tényleges teljesítményadat nem szerepel.

10.4 Tipikus fordítói mintázatok felismerése

10.4.1 Motiváció és felismerési stratégia

Egy C-függvény optimalizált assembly-listáját (*listing*) vizsgálva gyakran hiába keressük a forrás minden műveletét. A függvényhívás egyszerű elágazássá válhat, egy ciklustest többször szerepelhet, egy aritmetikai kifejezés pedig teljesen eltűnhet. Ez nem feltétlenül hibás fordítás: a fordító más utasítássorozattal valósíthatja meg ugyanazt a megfigyelhető viselkedést.

A mérnöki feladat ezért nem a C-forrás mechanikus „visszafordítása”, hanem három kérdés megválaszolása:

1. Milyen számítást végez a megmaradt kód?
2. Mely optimalizálási mintázatok magyarázzák a szerkezetét?
3. Milyen teszt vagy eszközkimenet támasztja alá az értelmezést?

Az előfeltétel a Fordítói optimalizálási szintek hatása a generált kódra ismerete. Itt nem teljesítményt mérünk, hanem mintázatokat azonosítunk.

Architektúraprofil: az alfejezet minden assembly-példája A64 ISA, Armv8-A, little-endian, Linux/AAPCS64 környezetre vonatkozik, az I. részben. A regiszterek és mnemonicok kizárólag ebben a kontextusban értendők.

10.4.2 Végpozíciós hívás: amikor nincs szükség visszatérésre a burkolóba

A **végpozíciós hívás** (*tail call*) olyan hívás, amely után a hívó függvénynek már nincs további érdemi teendője: a meghívott függvény eredményét közvetlenül továbbadja. A végpozíció lehetőséget ad optimalizálásra, de önmagában nem garantálja azt.

Például a későbbi labor `lab_tail` függvénye csak továbbítja az argumentumot a `lab_leaf` függvénynek, majd visszaadja annak eredményét.

A64 ISA: szokásos híváskor a `BL` a következő utasítás címét az `X30` linkregiszterbe írja. A minimális végpozíciós hívás ehelyett ilyen lehet:

```
// A64 ISA, I. rész - illusztratív függvénytörzs.
lab_tail:
    b lab_leaf
```

A működés lépései:

1. A külső hívó `BL` utasítása már beállította az `X30` visszatérési címét.
2. A `lab_tail` végén álló `B` nem módosítja az `X30` tartalmát.
3. A `lab_leaf` szokásos `RET` utasítása ezért közvetlenül a külső hívóhoz tér vissza.

Ez a minimális burkoló elkerülheti az `X30` mentését és visszaállítását, a saját veremkeretét, valamint a saját visszatérési utasítását. Az argumentum és az eredmény egyaránt a `W0` regisztert használja, így ebben a példában átrendezésük sem szükséges.

Fontos korlát: a végpozíciós hívás nem jelenti általánosan a teljes prológus és epilógus eltűnését. Ha a függvény korábban veremkeretet hozott létre vagy megőrzendő regisztert módosított, azt az átadás előtt rendeznie kell. A veremargumentumok, a hívási konvenció, illetve egy lokális objektum szükséges élettartama is korlátozhatja az optimalizálást.

A64 kódolási megjegyzés: a közvetlen `B` és `BL` előjeles, 26 bites, néggyel skálázott eltolást kódol. Az elérési tartomány az utasítás címéhez képest −128 MiB-tól +128 MiB−4 bájtig terjed. Távoli célhoz a linker köztes ugrókódot is beilleszthet.

Önmagában egy `B` látványa nem bizonyít végpozíciós hívást: lehet ciklusvisszaugrás vagy függvényen belüli elágazás is. A cél és a környező vezérlés szükséges az értelmezéshez.

10.4.3 Cikluskibontás: több elem egy vezérlési lépésre

A **cikluskibontás** (*loop unrolling*) a ciklustest több egymás utáni példányát helyezi egy új ciklusiterációba.

Ha az eredeti ciklus iterációnként egy elemet dolgoz fel, négyszeres kibontásnál a fő ciklustest négy elemmel foglalkozik. Ennek lehetséges jelei:

- ismétlődő betöltés–feldolgozás műveletcsoportok;
- nagyobb mutatóléptetés;
- a hátralévő elemszám csökkentése egynél nagyobb értékkel;
- külön kód a maradék elemekhez.

Az alábbi **A64 ISA** részlet szemléltető fő ciklustest, nem rögzített GCC-kimenet. Belépéskor az `X0` érvényes `uint32_t` elemekre mutat, az `X1` pozitív, néggyel osztható elemszám, a `W2` pedig a részösszeg:

```
// A64 ISA, I. rész - négyszeresen kibontott ciklusmag.
.Lfour:
    ldr  w3, [x0, #0]
    ldr  w4, [x0, #4]
    ldr  w5, [x0, #8]
    ldr  w6, [x0, #12]
    add  w2, w2, w3
    add  w2, w2, w4
    add  w2, w2, w5
    add  w2, w2, w6
    add  x0, x0, #16
    subs x1, x1, #4
    b.ne .Lfour
```

A64 kódolási korlátok: az itt használt `LDR Wt` nemnegatív eltolásos alakja 12 bites, néggyel skálázott eltolást fogad el. Az `ADD` és `SUBS` immediate alakjában 12 bites érték szerepelhet, opcionálisan 12 bites balra tolással. A `B.NE` előjeles, 19 bites, néggyel skálázott eltolásának tartománya −1 MiB és +1 MiB−4 bájt között van.

Levezetett vezérlési költség: pozitív, néggyel osztható n elemszámnál az egyenként feldolgozó ciklus n, a négyszeresen kibontott fő ciklus $n/4$ ciklusvégi feltételvizsgálatot végezhet. Például 16 elemnél ez 16 helyett 4 vizsgálat. Ez nem ciklusszámmérés és nem négyszeres gyorsulási ígéret: az elemenkénti munka megmarad, a kódméret pedig nőhet.

Általánosan $n = 4q + r$, ahol q a teljes csoportok száma, $0 \le r < 4$ pedig a maradék elemszám. A fenti ciklusmag csak a teljes csoportokat kezeli. A fordítónak a nulla elemszámot és a maradékot is helyesen kell kezelnie.

A tényleges GCC-kód használhat végmutató-összehasonlítást, páros betöltéseket vagy több részösszeget is. A kibontást ezért nem egyetlen konkrét utasítássorozathoz kötjük.

10.4.4 Konstans-összecsukás és műveleterősség-csökkentés

A **konstans-összecsukás** (*constant folding*) fordításkor kiszámítható kifejezések kiértékelése. A `(9 + 5) * 3` eredménye 42, ezért futáskor nem szükséges sem összeadást, sem szorzást végezni.

Egy lehetséges **A64 ISA** eredmény:

```
// A64 ISA, I. rész - lehetséges fordítói kimenet.
lab_fold:
    mov w0, #42
    ret
```

Az itt használt `MOV` egyetlen `MOVZ` utasítással megvalósítható alias: a 42 belefér a 16 bites immediate mezőbe. A 32 bites változatnál a mező elhelyezése 0 vagy 16 bites eltolással történhet. Nem minden 32 bites konstans tölthető be ezzel az egyutasításos formával.

A **műveleterősség-csökkentés** (*strength reduction*) más jelenség: egy futásidejű műveletet egyszerűbbnek ítélt művelettel helyettesít. Az előjel nélküli, 32 bites `x * 8` például lehet:

```
// A64 ISA, I. rész - lehetséges fordítói kimenet.
lab_scale:
    lsl w0, w0, #3
    ret
```

Az A64 `LSL Wd, Wn, #s` immediate eltolása 0–31 között lehet; 64 bites `Xn` nézetben 0–63 között.

A két optimalizálás különbsége:

- a konstans-összecsukásnál a teljes eredmény ismert fordításkor;
- a műveleterősség-csökkentésnél `x` továbbra is futásidejű adat.

A labor `uint32_t` típust használ, ezért a számolás modulo 2^{32} történik. Előjeles C-kifejezéseknél az overflow és a balra tolás szabályai miatt nem szabad ugyanezt mechanikusan alkalmazni. Egy betöltött 42-es értékből pedig önmagában nem állapítható meg, hogy eredetileg összetett kifejezés vagy egyszerű literál szerepelt-e a forrásban.

10.4.5 Kidolgozott labor: három fordítás, azonos szemantika

Munkakönyvtár: `~/arm-labor/a64/a039/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a039
cd ~/arm-labor/a64/a039
```

A labor célja **qemu-user, AArch64 Linux**. Az Ubuntu 24.04-es cross-eszközlánc mellett a statikus linkeléshez az AArch64 cross libc fejlesztői állományai is szükségesek.

A `lab-patterns.c` tartalma:

```
#include <stddef.h>
#include <stdint.h>

extern uint32_t lab_leaf(uint32_t x);

uint32_t lab_tail(uint32_t x)
{
    return lab_leaf(x);
}
```

```c
uint32_t lab_fold(void)
{
    return (UINT32_C(9) + UINT32_C(5)) * UINT32_C(3);
}

uint32_t lab_scale(uint32_t x)
{
    return x * UINT32_C(8);
}

uint32_t lab_sum(const uint32_t *p, size_t n)
{
    uint32_t sum = 0;
    while (n != 0) {
        sum += *p++;
        --n;
    }
    return sum;
}
```

A `lab-test.c` biztosítja a másik fordítási egységben levő hívási célt és az ellenőrzést:

```c
#include <inttypes.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

uint32_t lab_tail(uint32_t x);
uint32_t lab_fold(void);
uint32_t lab_scale(uint32_t x);
uint32_t lab_sum(const uint32_t *p, size_t n);

uint32_t lab_leaf(uint32_t x)
{
    return x + UINT32_C(7);
}

int main(void)
{
    const uint32_t a[] = {
        1, 2, 3, 4, 5, 6, 7, 8, 9,
        10, 11, 12, 13, 14, 15, 16, 17
    };
    const uint32_t wrap[] = { UINT32_MAX, 1 };

    uint32_t t = lab_tail(5);
    uint32_t f = lab_fold();
    uint32_t s = lab_scale(9);

    if (t != 12 || f != 42 || s != 72)
        return 1;
    if (lab_scale(UINT32_MAX) != UINT32_C(0xfffffff8))
        return 1;
    if (lab_sum(NULL, 0) != 0 || lab_sum(wrap, 2) != 0)
        return 1;

    for (size_t n = 0; n <= sizeof a / sizeof a[0]; ++n) {
        uint32_t expected = (uint32_t)(n * (n + 1) / 2);
        if (lab_sum(a, n) != expected)
            return 1;
    }
```

```
    printf("tail=%" PRIu32 " fold=%" PRIu32
           " scale=%" PRIu32 " sum10=%" PRIu32 "\n",
           t, f, s, lab_sum(a, 10));
    puts("PASS");
    return 0;
}
```

A külön fordítási egység és a kikapcsolt linkidejű optimalizálás (*link-time optimization*, LTO) megakadályozza, hogy a `lab_leaf` törzse egyszerűen beágyazódjon a `lab_tail` függvénybe.

A vektorizálást most kikapcsoljuk, hogy a skaláris cikluskibontást könnyebb legyen felismerni. Ez oktatási beállítás, nem általános teljesítményajánlás. A `lab_flags=(...)` Bash-tömb, amelynek elemeire a `"${lab_flags[@]}"` hivatkozik: így a közös kapcsolók egy helyen maradnak, és idézőjelben egyetlen argumentumlistaként adódnak át.

```
# Host: Ubuntu 24.04 x86_64.
# Target: qemu-user, AArch64 Linux, A64 Armv8-A, little-endian.
# Az alábbi parancsok Bash szintaxisúak.

aarch64-linux-gnu-gcc --version > lab-versions.txt
aarch64-linux-gnu-as --version >> lab-versions.txt
qemu-aarch64 --version >> lab-versions.txt

lab_flags=(
    -std=c11 -O2 -Wall -Wextra
    -march=armv8-a -mtune=generic
    -fno-lto -fno-pie
    -fno-tree-loop-vectorize -fno-tree-slp-vectorize
)

# C -> A64 assembly: alapváltozat.
aarch64-linux-gnu-gcc "${lab_flags[@]}" \
    -foptimize-sibling-calls -fno-unroll-loops \
    -S lab-patterns.c -o lab-base.s

# C -> A64 assembly: cikluskibontás ösztönzése.
aarch64-linux-gnu-gcc "${lab_flags[@]}" \
    -foptimize-sibling-calls -funroll-loops \
    -S lab-patterns.c -o lab-unroll.s

# C -> A64 assembly: végpozíciós hívás optimalizálása nélkül.
aarch64-linux-gnu-gcc "${lab_flags[@]}" \
    -fno-optimize-sibling-calls -fno-unroll-loops \
    -S lab-patterns.c -o lab-call.s

# Tesztkód fordítása AArch64 objektummá.
aarch64-linux-gnu-gcc "${lab_flags[@]}" \
    -c lab-test.c -o lab-test.o

# A64 assemble -> statikus Linux ELF -> qemu-user futtatás.
for lab_mode in base unroll call; do
    aarch64-linux-gnu-as -march=armv8-a \
        "lab-${lab_mode}.s" -o "lab-${lab_mode}.o"
    aarch64-linux-gnu-gcc -static -no-pie \
        "lab-${lab_mode}.o" lab-test.o -o "lab-${lab_mode}"
    aarch64-linux-gnu-objdump -dr \
        "lab-${lab_mode}.o" > "lab-${lab_mode}.dis"
    qemu-aarch64 "./lab-${lab_mode}"
    printf 'exit=%d\n' "$?"
done
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2): mindhárom program kiírja a

`tail=12 fold=42 scale=72 sum10=55`, majd a `PASS` sort; a jelzett kilépési állapot `exit=0`. Az 1-es kilépési állapot szemantikai teszthibát jelent.

Az assembly-mintázatok viszont kiadás- és döntésfüggők. A lektor mérésében (GCC 13.3.0) a táblázat minden sora igazolódott: a `lab-base.s`-ben a `lab_tail` `b lab_leaf` (`R_AARCH64_JUMP26`), a `lab-call.s`-ben `stp/bl/ldp/ret` (`R_AARCH64_CALL26`), a `lab_fold` `mov w0, #42`, a `lab_scale` `lsl w0, w0, #3`, a kibontott `lab_sum`-ban pedig 15 `ldr` áll szemben az alapváltozat egyetlen `ldr`-jével.

Vizsgálat	Mért vagy keresendő A64 mintázat	Értelmezés
`lab-base.s`: `lab_tail`	`B lab_leaf`	Végpozíciós átadás
`lab-call.s`: `lab_tail`	`BL`, visszatérési cím megőrzése, saját visszatérés	Szokásos hívás
`lab_fold`	42 betöltése, aritmetikai lánc nélkül	Konstans-összecsukással összeegyeztethető
`lab_scale`	`LSL` 3-mal	Szorzás helyett eltolás
`lab-unroll.s`: `lab_sum`	Több elem feldolgozása a fő ciklustestben	Cikluskibontás

A `-funroll-loops` nem ígér konkrét kibontási tényezőt minden ciklusra. Az objektum disassemblyjében a külső hívás mellett az AArch64 ELF `R_AARCH64_CALL26`, a közvetlen átadás mellett pedig `R_AARCH64_JUMP26` relokáció is segítheti az azonosítást.

10.4.6 Tipikus hibák és ellenpróbák

„Nincs hívásutasítás, tehát a hívás eltűnt.”
Hipotézis: beágyazás történt. Ellenpróba: vizsgáljuk meg a célszimbólumra mutató A64 `B` utasítást és a relokációt. Lehet, hogy a hívási határ megmaradt, csak végpozíciós átadássá alakult.

„Négy betöltés biztosan négyszeres kibontás.”
Négy külön C-kifejezés is okozhat négy betöltést. Ellenőrizzük együtt a visszaugrási élt, a mutatólépést és a ciklusfeltételt.

„A kibontott ciklus minden elemszámra ugyanúgy léphet négyet.”
Ha az egyetlen kilépési feltétel a számláló nullává válása, a nem négyszeres elemszám problémát okozhat. A labor 0–17 elemre vizsgálja a függvényt, így több csoporthatárt és maradékot is lefed.

„Kevesebb ugrás ugyanannyi arányú gyorsulást jelent.”
Nem: nőhet a kódméret és a regiszternyomás, miközben az összeadások függőségi lánca megmaradhat. A labor nem mér latencyt, throughputot vagy hardveres ciklusszámot. QEMU alatt az abszolút futási idő amúgy sem tekinthető hardverhű AArch64-időnek.

10.4.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Kövesd az A64 `X30` tartalmát a külső hívó → `lab_tail` → `lab_leaf` útvonalon! Miért elég a burkolóban egy `B`? Miért nem következik ebből, hogy minden végpozíciós hívásnak nincs veremkerete?

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a labort `uint32_t lab_mixed(uint32_t x)` függvénnyel, amely `x * 8 + (9 + 5) * 3` értékét adja vissza. Teszteld 9 és `UINT32_MAX` bemenettel! Mely optimalizálások működhetnek együtt?

3. (hibakeresési, középszint — cél: rutinépítés) A `lab-call` változatban a burkoló hívásutasítást és veremkezelést tartalmaz. Állíts fel hipotézist, vizsgáld meg az objektumot, majd egyetlen fordítási kapcsoló megváltoztatásával készíts ellenpróbát! Az eredmény mellett a `lab_sum` kibontását is értékeld a `lab-unroll.dis` alapján.

10.4.8 Megoldások

1. megoldás — Segítség. A `BL` és a `B` közül melyik írja át az `X30`-at? Ha a burkoló VÉGÉN álló utasítás nem nyúl hozzá, kinek a visszatérési címe marad érvényben?

1. megoldás — Teljes megoldás. A külső A64 `BL` beállítja az `X30` értékét. A burkoló `B` utasítása ezt változatlanul hagyja, így a célfüggvény a külső hívóhoz térhet vissza. Szokásos belső `BL` esetén a burkolónak meg kellene őriznie saját visszatérési címét. Egy összetettebb burkoló azonban korábbi munkája miatt rendelkezhet veremkerettel; ezt a végpozíciós átadás előtt bontja le.

2. megoldás — Segítség. $9 \cdot 8 + 42 = 114$; `UINT32_MAX` $\cdot 8$ modulo 2^{32} mennyi, és ehhez hozzáadva 42-t, megint modulo 2^{32}, mit kapsz?

2. megoldás — Teljes megoldás. A `lab-patterns.c` kiegészítése:

```
uint32_t lab_mixed(uint32_t x)
{
    return x * UINT32_C(8)
         + (UINT32_C(9) + UINT32_C(5)) * UINT32_C(3);
}
```

A `lab-test.c` deklarációihoz add hozzá a prototípust, a `main` ellenőrzéseihez pedig:

```
if (lab_mixed(9) != 114 || lab_mixed(UINT32_MAX) != 34)
    return 1;
```

Futtasd újra a teljes buildlépéssort. Az első eredmény $72 + 42 = 114$. A másodiknál az előjel nélküli szorzás eredménye `0xfffffff8`, ehhez 42-t adva, modulo 2^{32}, 34 keletkezik. A konstansrész összevonható, a szorzás eltolássá alakítható; az eltolás akár más A64 utasítás operandusába is beolvadhat.

3. megoldás — Segítség. A `lab-call` változatot explicit `-fno-optimize-sibling-calls`-lal fordítottuk — ez fordítói DÖNTÉS, nem C-nyelvi korlát. Mi történne, ha ezt a kapcsolót egyszerűen elhagynánk vagy megfordítanánk?

3. megoldás — Teljes megoldás. A hipotézis szerint nem a C-forrás tiltja az átalakítást, hanem a `-fno-optimize-sibling-calls` kapcsoló. Az A64 objektum vizsgálata:

```
# Target: qemu-userhez készített A64 objektum; hostoldali elemzés.
aarch64-linux-gnu-objdump -dr \
    --disassemble=lab_tail lab-call.o
```

Az ellenpróbához cseréld a tiltó kapcsolót `-foptimize-sibling-calls` értékre, és építsd újra ugyanazt a változatot. A tesztkimenetnek változatlannak kell maradnia, miközben megjelenhet a végpozíciós átadás. A cikluskibontás tényezőjét a visszaugrások között feldolgozott elemekből állapítsd meg, ne a teljes függvény betöltésszámából: a maradékkezelés külön műveleteket tartalmazhat.

10.4.9 Rövid összefoglaló

Az optimalizált assembly-lista értelmezésekor nem a C-forrás sorait, hanem az adatáramlást és a vezérlést követjük. A végpozíciós hívás megőrizheti a hívási határt, miközben elhagyja a burkoló saját visszatérési útját. A cikluskibontást az ismételt műveletcsoport, a nagyobb léptetés és a maradékkezelés együtt jelzi. A konstans-összecsukás eltünteti a fordításkor kiszámítható munkát; a változóval végzett szorzás eltolássá alakítása más optimalizálás. A pontos utasítássorrend fordítófüggő, ezért a felismerést disassemblyvel és határeseteket is tartalmazó tesztekkel ellenőrizzük. Az utasításszám változása önmagában nem teljesítménymérés.

- **Végpozíciós hívás:** közvetlen eredménytovábbítással záruló hívás.
- **Cikluskibontás:** több eredeti iteráció összevonása egy ciklustestbe.
- **Konstans-összecsukás:** ismert kifejezés fordításidejű kiértékelése.

10.4.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GCC: Options That Control Optimization — végpozíciós hívások, cikluskibontás és vektorizálási kapcsolók. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2.
- GNU Binutils: objdump — disassembly, szimbólumok és relokációk vizsgálata. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- Arm: Procedure Call Standard for the Arm 64-bit Architecture — regiszterszerepek és a hívási határon teljesítendő szerződés. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (az SME-részek Beta státuszúak).
- Arm Architecture Reference Manual for A-profile architecture — A64 utasításszemantika és kódolási korlátok. **Ellenőrizve**: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.

Ellenőrzési hiányok: a példák a lektor mérési környezetében (GCC 13.3.0, QEMU 8.2.2) lefutottak, az ott közölt kimenet és mintázattábla onnan származik. A telepített Ubuntu 24.04-es csomagok pontos verzióját, a statikus cross libc rendelkezésre állását és a GCC által választott kibontási tényezőt a hallgató laborjában kell rögzíteni. A bemutatott assembly-részletek továbbra is lehetséges vagy szemléltető mintázatok; a mért kimenet a lektor környezetére érvényes, a hallgatóé eltérhet.

11 Debugging, disassembly és instrumentáció

11.1 GDB assembly szinten

11.1.1 Motiváció: hol romlott el az érték?

Egy assemblyfüggvény helyesnek látszik, mégis hibás eredményt ad vissza a C-hívónak. A forrás átolvasása nem dönti el, hogy rossz adatot töltöttünk-e be, túl keskeny regiszternézetet használtunk-e, vagy a visszatéréskor sérült meg a veremállapot. Ehhez a ténylegesen végrehajtott utasítást, a regisztereket és a memóriát együtt kell megfigyelni.

Az **utasításszintű hibakeresés** (*instruction-level debugging*) során a programot gépi utasítások határán állítjuk meg, és a végrehajtási állapot változását vizsgáljuk. A GDB programszámlálója, a `PC` ilyenkor rendszerint a **következőként végrehajtandó utasításra** mutat. A töréspontnál látható állapot tehát még nem tartalmazza az ott álló utasítás hatását.

Az alfejezet a Szerkesztő és debug-eszközök: GDB, objdump, readelf első áttekintése, valamint a Prológus és epilógus: regisztermentés és keretkezelés ismereteire épül. Minden laborpélda **A64 ISA, Armv8-A, little-endian Linux ELF**, `qemu-aarch64` user-mode céllal. A GDB parancsai nem ISA-utasítások; a példákban az A64 célállapotot vizsgálják.

11.1.2 Lépés, regiszternézet és memórianézet

11.1.2.1 Forrássor vagy gépi utasítás?

A GDB két különböző léptetési szintet kínál:

GDB-parancs	Megállási egység	Függvényhívás kezelése
`step`	Következő forráshely, a hibakeresési információ alapján	Megpróbál belépni a hívott függvénybe
`next`	Következő forráshely ugyanabban a veremkeretben	Átlépi a hívást
`stepi`, röviden `si`	Egy gépi utasítás	Belép a hívásba
`nexti`, röviden `ni`	Egy gépi utasítás, hívásnál annak visszatérési helye	Átlépi a hívást

A64 célon, egy `BL` utasításon állva a `stepi` végrehajtja a hívást: beállítja az `X30` visszatérési címet, majd a hívott függvény elején áll meg. A `nexti` ehelyett normál visszatérés esetén a `BL` utáni utasításnál áll meg.

Az „átlépés” nem azt jelenti, hogy a függvény kimarad. Minden mellékhatása megtörténik. Egy közben elért töréspont vagy jelzés azonban korábban is megszakíthatja a futást.

A `stepi 5` öt gépi utasítást hajt végre. Ezzel szemben a `nexti 5` öt léptetési műveletet jelent: az átlépett függvények miatt ténylegesen jóval több utasítás is lefuthat.

Assemblyforrásban gyakran egy sorhoz egy utasítás tartozik, de ez nem teszi azonossá a két léptetési szintet. A direktívák nem végrehajtható utasítások, optimalizált C-kódnál pedig különösen összetett lehet a forrássorok és az utasítások kapcsolata.

11.1.2.2 Regiszterek lekérdezése

A következő GDB-parancsok A64 célon értendők:

Parancs	Jelentés
`info registers`	Az általános regiszterállapot megjelenítése
`info registers x0 x1 sp pc`	Kiválasztott regiszterek megjelenítése
`p/x $x0`	Az `X0` értéke hexadecimálisan
`p/u $x1`	Az `X1` értéke előjel nélküli decimális olvasatban
`x/i $pc`	A következő utasítás megjelenítése

Az `info registers` után a regiszternevek elé nem kell `$`; GDB-kifejezésben viszont kell. A `p` a `print` rövidítése.

A64-en az `X0` 64 bites, a `W0` ugyanannak a regiszternek a 32 bites nézete. A modern GDB (pl. 15.x) a `$w0`…`$w30` neveket is ismeri; régebbi verzióban helyette a `p/x ($x0 & 0xffffffff)` kifejezés használható. Ez csak olvasás; nem modellezi a `W0`-ba írás felső 32 bitet törlő hatását.

11.1.2.3 Az `x` parancs felépítése

A **memóriavizsgálat** (*memory examination*) alapformája:

```
x/Nfu cím
```

- `N`: a megjelenítendő egységek száma;
- `f`: a megjelenítési formátum, például `x` hexadecimális vagy `u` előjel nélküli decimális;
- `u`: az egység mérete, például `b` egy, `h` kettő, `w` négy, `g` nyolc bájt.

A GDB `w` jelölése tehát **négy bájt**, AArch64 célon is.

GDB-parancs, A64 cél	Megjelenített tartomány
`x/4xw $x0`	Négy, egyenként 4 bájtos érték hexadecimálisan
`x/2gx $x0`	Két, egyenként 8 bájtos érték hexadecimálisan
`x/16xb $x0`	Tizenhat különálló bájt
`x/2gx $sp`	Két 8 bájtos egység az aktuális verem tetejétől

A formátum- és méretbetű sorrendje felcserélhető: az `x/2gx` és az `x/2xg` ugyanazt jelenti. A GDB megjegyez korábbi megjelenítési beállításokat, ezért reprodukálható vizsgálatban célszerű mindent explicit megadni.

11.1.3 Kidolgozott példa: összegzés és veremellenőrzés

Munkakönyvtár: `~/arm-labor/a64/a040/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a040
cd ~/arm-labor/a64/a040
```

A laborfüggvény két 64 bites, előjel nélküli elemet és egy eltolási értéket ad össze. C-szemantikával a művelet `p[0] + p[1] + bias`, `uint64_t` operandusokkal. Az A64 bemenet az `X0` mutató és az `X1` eltolás; a visszatérési érték az `X0`.

Mentsük a következőt `lab-gdb.s` néven:

```
// A64 ISA, Armv8-A, little-endian Linux.
// A direktívák GNU as / AArch64 ELF környezethez tartoznak.

.arch armv8-a
.text
.p2align 2
.global _start, lab_call, lab_after, lab_stored
.type _start, %function

_start:
    adrp x0, lab_values
    add  x0, x0, :lo12:lab_values
    mov  w1, #7

lab_call:
    bl   lab_sum

lab_after:
    adrp x2, lab_result
    add  x2, x2, :lo12:lab_result
    str  x0, [x2]

lab_stored:
    mov  w0, #0
    mov  w8, #93
    svc  #0
.size _start, .-_start

.global lab_sum
.type lab_sum, %function
lab_sum:
    .cfi_startproc
    stp  x29, x30, [sp, #-16]!
    .cfi_def_cfa_offset 16
    .cfi_offset 29, -16
    .cfi_offset 30, -8
    mov  x29, sp
    ldr  x2, [x0]
    ldr  x3, [x0, #8]
    add  x0, x2, x3
    add  x0, x0, x1
    ldp  x29, x30, [sp], #16
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc
.size lab_sum, .-lab_sum
```

```
.data
.balign 8
lab_values:
    .xword 0x0000000200000001
    .xword 0x0000000400000003
lab_result:
    .xword 0

.section .note.GNU-stack,"",%progbits
```

A függvény levélfüggvényként nem igényelné ezt a veremkeretet; itt szándékosan tartjuk meg, hogy vizsgálható legyen a mentés és visszaállítás. A kilépés Linux/AArch64 rendszerhívás-konvenció: az `X8` értéke 93, az `X0` a kilépési állapot. Ez nem önmagában az A64 ISA szolgáltatása.

Immediate-ok: a példa minden konstansa és eltolása kódolható (lásd az A64 immediate-korlátok összefoglalóját). Az `ADRP + ADD …, :lo12:` pár együtt állítja elő a szimbólum teljes címét.

11.1.3.1 Összeállítás és indítás

Ubuntu 24.04 x86_64 hoston, **A64 qemu-user célra**:

```
# Target: qemu-user / AArch64 Linux, little-endian.
aarch64-linux-gnu-as -g -EL -march=armv8-a \
    -o lab-gdb.o lab-gdb.s
aarch64-linux-gnu-ld -EL -e _start \
    -o lab-gdb.elf lab-gdb.o

# Első terminál: várakozás a GDB kapcsolódására.
qemu-aarch64 -g 12345 ./lab-gdb.elf
printf 'QEMU kilepesi allapot: %d\n' "$?"
```

A `-g` után megadott szám TCP-port, nem memóriaérték. A várakozás alatt a program látszólag nem csinál semmit: ez most helyes működés. A hibakereső portot csak megbízható, elkülönített laborhálózatban használjuk.

A második terminálban készítsünk GDB-parancsfájlt. A következő `bash` blokk csak létrehozza azt és elindítja a hibakeresőt; a `GDB` határolók közötti sorok **GDB-parancsok**.

```
# Target: a fenti qemu-user / AArch64 folyamat.
cat > lab-step.gdb <<'GDB'
set pagination off
target remote localhost:12345
break *lab_call
continue

x/i $pc
info registers x0 x1 sp pc
x/4xw $x0
x/2gx $x0
set $entry_sp = $sp
set $entry_fp = $x29

stepi
info registers x30 sp pc
x/i $pc

stepi
x/2gx $sp
p $sp == $entry_sp - 16
```

```
stepi 5
p/x $x0
x/i $pc

stepi
p $sp == $entry_sp
p $x29 == $entry_fp

stepi
x/i $pc

break *lab_stored
continue
x/1gx &lab_result
continue
quit
GDB
```

```
gdb-multiarch -q ./lab-gdb.elf -x lab-step.gdb
```

Kézi gyakorláshoz a GDB-t a `-x lab-step.gdb` rész nélkül indítsuk, majd a fájl parancsait egyenként adjuk ki.

11.1.4 A megállások értelmezése

A következő állapotok **mérve (GDB 15.1 + qemu-aarch64 8.2.2)**: a lektor a bemutatott szkriptet lefuttatta, az `x/4xw` nézet az 1, 2, 3, 4 értéket, az `x/2gx` a várt párokat mutatta, az összeg `0x000000060000000b` lett, az `LDP` visszaállította a mentett regisztereket, a `lab_result` helyesnek adódott, a `nexti` pedig a leírt módon lépett.

A `break *lab_call` a `BL` végrehajtása előtt állít meg. Ekkor az A64 `X0` a tömb címét, az `X1` a 7 értéket tartalmazza. Nem kell előre ismernünk a numerikus címet: a GDB az ELF szimbólumából határozza meg.

A két memórianézet ugyanazt a 16 bájtot mutatja:

Nézet	Várt értékek, növekvő címsorrendben
`x/4xw $x0`	`0x00000001`, `0x00000002`, `0x00000003`, `0x00000004`
`x/2gx $x0`	`0x0000000200000001`, `0x0000000400000003`

Little-endian elrendezésben egy 64 bites érték alsó 32 bitje található az alacsonyabb címen. A két megjelenítés tehát nem eltérő adatot, hanem eltérő csoportosítást jelent.

Az első `stepi` végrehajtja az A64 `BL` utasítást. A `PC` a `lab_sum` elejére, az `X30` a `lab_after` címére kerül. A következő `stepi` végrehajtja az A64 `STP` utasítást:

Hely vagy feltétel	Várt tartalom/eredmény
`SP`	Belépési `SP` − 16
`[SP]`, 8 bájt	A belépési `X29` értéke
`[SP+8]`, 8 bájt	A `lab_after` visszatérési címe
`p $sp == $entry_sp - 16`	Igaz, jellemzően `1` alakban

A `$entry_sp` és `$entry_fp` GDB-segédváltozók (*convenience variables*), nem célprocesszorbeli regiszterek. Létrehozásuk nem módosítja a programot.

Az öt további lépés a keretmutató beállítását, két betöltést és két összeadást hajt végre. A várt eredmény:

`0x0000000200000001 + 0x0000000400000003 + 7 = 0x000000060000000b`

Ekkor a következő A64 utasítás még az `LDP`. Annak végrehajtása után mindkét állapot-összehasonlítás igaz. Az A64 `RET` után a `PC` a `lab_after` helyre mutat.

A `lab_stored` töréspontnál az eredménytárolás már megtörtént, ezért a `lab_result` várt tartalma `0x000000060000000b`. A program nem ír szöveget; normál, 0 állapotú kilépést várunk.

11.1.4.1 Ugyanez `nexti` használatával

Indítsuk újra az első terminál QEMU-parancsát, majd kapcsolódjunk új GDB-munkamenettel:

```
# Target: újraindított qemu-user / AArch64 folyamat.
gdb-multiarch -q ./lab-gdb.elf \
    -ex 'target remote localhost:12345' \
    -ex 'break *lab_call' \
    -ex 'continue' \
    -ex 'nexti' \
    -ex 'info registers x0 pc' \
    -ex 'continue' \
    -ex 'quit'
```

A `nexti` után közvetlenül a `lab_after` helyen kell állnunk, az A64 `X0` regiszterben ugyanazzal az összeggel. A függvény belső állapotai ezúttal nem láthatók.

11.1.5 Töréspont szimbólumra vagy pontos címre

A `break lab_sum` szimbólumhoz kapcsolódó töréspontot kér. Függvényszimbólumnál a GDB a rendelkezésre álló információ alapján a prológus utáni helyet is választhatja. A `break *lab_sum` ezzel szemben kifejezetten a függvény kezdőcímét jelöli ki.

A `*` azt jelzi, hogy a megadott kifejezést utasításcímként értelmezzük. Így címre törhetünk beégetett numerikus cím nélkül is, például `break *$pc` használatával. A töréspontokat az `info breakpoints` listázza; a `delete N` egy adott sorszámút töröl.

Mindig a futó programhoz tartozó ELF-et nyissuk meg. Újrafordítás után a korábban feljegyzett numerikus címek és a régi GDB-munkamenet szimbólumai elavulhatnak.

11.1.6 QEMU user-mode és system-mode: előzetes különbség

A jelenlegi **user-mode emulációban** egy Linux-folyamatot vizsgálunk. A `qemu-aarch64 -g 12345` a program indulása előtt várja a kapcsolatot. A megnyitott ELF helyi példánya szolgáltatja a szimbólumokat; a `target remote` létrehozza a kapcsolatot, a `continue` elindítja a már betöltött célt. Ebben a munkafolyamatban nem kell GDB `load`, és nem a `run` indítja a célt.

A Cortex-M futási modelljében, **Thumb-2/Armv7-M kontextusban**, a `qemu-system-arm` teljes gépet emulál. A későbbi `mps2-an385` laborban a `-S` az induló processzort állítja meg, a `-gdb tcp::12345` pedig a hibakereső kiszolgálót engedélyezi. A kiszolgáló engedélyezése önmagában nem helyettesíti a `-S` kapcsolót.

Ott a firmware, a resetállapot és a gép memóriatérképe is a vizsgálat része lesz. Ez most csak működésbeli előzetes, nem Cortex-M firmware-indítási recept.

11.1.7 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés és egyetlen következő változtatás
Nem jön létre a kapcsolat	Eltérő port vagy nem várakozó QEMU	Ellenőrizzük a két portszámot; indítsuk újra csak a QEMU-folyamatot
Váratlan utasításnál állunk	Régi ELF vagy forrásszintű léptetés	Nézzük meg az `x/i $pc` eredményét; váltsunk `stepi` használatára
A 64 bites adat „szétesett”	Négybájtos nézetet használunk	Ugyanazon címen váltsunk `x/4xw` alakról `x/2gx` alakra
A számítás jó, de a visszatérés hibás	Sérült mentés vagy epilógus	Vizsgáljuk a mentett A64 `X30` értéket és az `SP` visszaállását
„Cannot access memory”	Nem mutatót vagy már megváltozott regisztert vizsgálunk	Ellenőrizzük a `PC`-t és a regiszter szerepét; ne próbálgassunk véletlen címeket

Különösen fontos: a függvény visszatérése után az A64 `X0` már **eredmény**, nem a bemeneti mutató. Ekkor a tömb vizsgálatához az `x/2gx &lab_values` parancsot használjuk.

11.1.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért különbözik az `x/4xw` és az `x/2gx` kimenete, miközben mindkettő 16 bájtot olvas? Mit látunk, ha az A64 `STR` utasítás címére teszünk töréspontot: a tárolás előtti vagy utáni állapotot?

2. (implementációs, alapszint — cél: rutinépítés) Módosítsuk az eltolást 7-ről 8-ra. Építsük újra a programot, indítsunk új QEMU/GDB-munkamenetet, és `nexti` segítségével ellenőrizzük a visszatérési értéket, majd a tárolt eredményt.

3. (hibakeresési, középszint — cél: rutinépítés) Cseréljük az A64 `ldr x3, [x0, #8]` utasítást `ldr w3, [x0, #8]` alakra, az eltolást pedig állítsuk vissza 7-re. Hol jelenik meg először az eltérés? Törjünk a `lab_sum` első utasítására, lépjünk négyet, és hasonlítsuk össze az `X3` értékét a memória 8 bájtos nézetével.

11.1.9 Megoldások

1. megoldás — Segítség. Ugyanaz a 16 bájt két KÜLÖNBÖZŐ csoportosításban jelenik meg a két parancsnál — nem az adat változik. A töréspont-szemantikára gondolj: a GDB mindig a KÖVETKEZŐ, még végre nem hajtott utasításnál áll meg.

1. megoldás — Teljes megoldás. Az `x` parancs eltérő méretű egységekre bontja ugyanazt a bájtsorozatot. Little-endian esetben az alacsonyabb címen a 64 bites szám alsó része található. Az A64 `STR` címére helyezett töréspont **a tárolás előtt** állít meg; egy `stepi` után vizsgálható az új memóriaérték.

2. megoldás — Segítség. Az A64 `mov w1, #8` ugyanabba a 16 bites immediate-mezőbe fér, mint a `#7`; csak a végeredmény változik — számold ki fejben az új összeget az eredeti példa alapján.

2. megoldás — Teljes megoldás. A módosítás A64 alakja `mov w1, #8`. A várt eredmény `0x000000060000000c`. A `nexti` után ezt az `X0`, a `lab_stored` helyen pedig a `lab_result` tartalmazza. Az új munkamenet biztosítja, hogy a betöltött program és a szimbólumok ugyanahhoz az új buildhez tartozzanak.

3. megoldás — Segítség. A `W3`-ba írás csak 4 bájtot olvas, és nullázza `X3` felső felét — a memória második 64 bites eleme ettől még a helyén marad, csak `X3` nem fogja tükrözni.

3. megoldás — Teljes megoldás. A hibás betöltés csak négy bájtot olvas. Az A64 `W3`-ba írás nullázza az `X3` felső felét, ezért az `X3` értéke `0x0000000000000003`, miközben a memória második 64 bites eleme változatlan. A hibás végeredmény `0x000000020000000b`. Négy lépés után már látszik az eltérés; a következő utasítás az első összeadás. A javítás kizárólag a 64 bites betöltés visszaállítása.

11.1.10 Rövid összefoglaló

A GDB-ben a megfigyelt értéket mindig a következő utasítással együtt értelmezzük. A `stepi` belép a függvényhívásba, a `nexti` rendszerint annak visszatérése után áll meg, de mindkettő végrehajtja a program megfelelő részét. A regisztervizsgálat megmutatja a számítás állapotát, az `x` parancs pedig explicit méretben és formátumban jeleníti meg a memóriát. Ugyanaz a bájtsorozat többféleképpen csoportosítható; ettől az adat nem változik. A reprodukálható vizsgálathoz azonos ELF, szimbolikusan meghatározott helyek és előre rögzített ellenőrzési feltételek szükségesek.

- **Utasításszintű léptetés:** megállítás gépi utasítások határán.
- **Memóriavizsgálat:** bájtok megjelenítése választott egységmérettel és formátummal.
- **GDB-segédváltozó:** a hibakeresőben tárolt, célállapotot nem módosító érték.

11.1.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GNU GDB kézikönyv — léptetés, regiszterek, memóriavizsgálat, töréspontok és távoli kapcsolat. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).
- GNU assembler kézikönyv — hibakeresési információ és AArch64 assemblerkörnyezet. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- QEMU dokumentáció — user-mode és system-mode hibakeresési kapcsolatok. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok: a program és a GDB-parancsfájl a lektor mérési környezetében (GDB 15.1, qemu-aarch64 8.2.2) lefutott, az itt közölt állapotok onnan származnak. Az Ubuntu 24.04-ben telepített konkrét eszközverziókkal ellenőrizendő a távoli kapcsolat, a GDB regiszterneveinek elérhetősége és a megállási üzenetek pontos formája. A labor nem tartalmaz időmérést vagy hardverhű ciklusszámra vonatkozó állítást.

11.2 Disassembly objdump/readelf-fel

11.2.1 Motiváció: mit tartalmaz valójában a futtatható fájl?

Egy C-program hibájának vizsgálatakor gyakori kérdés: **valóban azt a függvényt linkeltük be, amelyet javítottunk?** Máskor azt kell eldönteni, hogy egy hívás saját kódra, könyvtári függvényre vagy köztes ugrókódra mutat-e. Ehhez nem feltétlenül kell elindítani a programot.

A **visszafejtett utasításlista (disassembly)** a gépi kód utasításként értelmezett, szöveges megjelenítése. Nem állítja vissza az eredeti C-forrást: az utasításokat, a hozzájuk tartozó címeket és a rendelkezésre álló szimbólumneveket mutatja meg.

A GDB assembly szinten alfejezetben futás közbeni állapotot vizsgáltunk. Most a fájlban tárolt kódot és metaadatokat olvassuk:

- az `objdump` elsősorban az utasítások és kapcsolódó információk vizsgálatára szolgál;
- a `readelf` az ELF szerkezetét, többek között szimbólumait és relokációit jeleníti meg.

Hatókör: I. rész, AArch64, A64 ISA, Armv8-A alapprofil, little-endian Linux ELF. Az alábbi assembly utasítások és regiszterek kizárólag ebben a környezetben értendők. A GNU assembler direktívái az AArch64-célú objektum előállítását vezérlik; önmagukban nem processzorutasítások.

11.2.2 Három külön információréteg

Egy `objdump`-kimenet olvasásakor különítsük el a következőket.

Réteg	Mit tárol vagy jelenít meg?	Tipikus eszközkapcsoló
Gépi kód	A processzor által végrehajtható utasítások kódolását	`objdump -d`
Szimbólumtábla	Neveket, értékeket, méreteket, típusokat és kötési tulajdonságokat	`objdump -t`, `readelf -s`
Relokáció	Utasítást a linkernek vagy betöltőnek egy helyfüggő érték javítására	`objdump -r`, `readelf -r`

A **szimbólumtábla (symbol table)** nem maga a programkód. Egy függvénynév eltávolítható úgy, hogy a függvény utasításai változatlanul megmaradjanak.

A **relokációs bejegyzés (relocation entry)** azt írja le, hogy melyik helyen, milyen szabály szerint kell egy szimbólumhoz kapcsolódó értéket kialakítani. Ez különösen a még nem linkelt objektumfájlokban fontos: ott egy külső függvény végleges címe általában még ismeretlen.

Az `objdump -d` megjelenítésében rendszerint szerepel az utasítás címe, gépi kódolása, mnemonicja és operandusai. A mellettük látható szimbólumnév az eszköz annotációja: **a processzor nem függvényneveket hajt végre**.

11.2.3 Kidolgozott példa: ugyanaz a függvény kétféle linkeléssel

Munkakönyvtár: `~/arm-labor/a64/a041/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a041
cd ~/arm-labor/a64/a041
```

A mérnöki feladat egy kis assembly függvény ellenőrzése C-hívóprogramból. A szerződés:

- a bemenet előjeles, 32 bites egész, ebben a laborban -1000 és 1000 között;
- negatív bemenetre az eredmény 0;
- egyébként az eredmény $3x + 5$.

A tartománykorlát miatt a matematikai eredmény előjeles 32 biten ábrázolható. Nem kell összemosnunk az assembly moduloaritmetikáját a C előjeles túlcsordulásának szabályaival.

11.2.3.1 A forrásfájlok

A `lab-adjust.S` teljes tartalma:

```
// A64 ISA, AArch64, Armv8-A; GNU as szintaxis.
.arch armv8-a
.text
.p2align 2
.global lab_adjust
.type lab_adjust, %function

lab_adjust:
    add     w1, w0, w0, lsl #1
    add     w1, w1, #5
    cmp     w0, #0
    csel    w0, w1, WZR, ge
    ret

.size lab_adjust, .-lab_adjust
.section .note.GNU-stack,"",%progbits
```

Az A64 `ADD` első előfordulása eltolt regiszteres alak: a 32 bites változat `LSL` eltolása 0–31 lehet. A második `ADD` 12 bites, előjel nélküli immediate-et fogad, opcionális 12 bites balra tolással; az 5 közvetlenül kódolható. A `CMP` itt a `SUBS WZR, W0, #0` aliasa, ugyanilyen immediate-korláttal.

Az A64 `CSEL` a `GE`, vagyis előjeles „nagyobb vagy egyenlő” feltételt használja: az NZCV állapotból $N = V$ esetén választja az első forrását. A `CMP` az eredeti bemenetet vizsgálja, mert az első két utasítás csak `W1` értékét módosította.

A `lab-driver.c`:

```
/* C-hívóprogram; cél: AArch64 Linux, little-endian. */
#include <stdio.h>

extern int lab_adjust(int x);

int main(void)
{
    int positive = lab_adjust(7);
    int negative = lab_adjust(-7);

    printf("pos=%d neg=%d\n", positive, negative);
```

```
    return (positive == 26 && negative == 0) ? 0 : 1;
}
```

11.2.3.2 Összeállítás, linkelés és futtatás

A következő parancsok az Ubuntu 24.04 x86_64 host cross-eszközláncát használják. A szükséges csomagokat a Labor-alapkörnyezet: eszközlánc telepítése és verifikáció szerint telepítettnek tekintjük.

```
# Target: AArch64 Linux, qemu-user; little-endian.
# 1. Assembly és C forrásból külön objektumok.
aarch64-linux-gnu-gcc -march=armv8-a -g \
    -c lab-adjust.S -o lab-adjust.o

aarch64-linux-gnu-gcc -march=armv8-a -O0 -g \
    -fno-pie -fno-builtin -fplt \
    -c lab-driver.c -o lab-driver.o

# 2. Ugyanazon objektumok dinamikus és statikus linkelése.
aarch64-linux-gnu-gcc -no-pie \
    lab-driver.o lab-adjust.o -o lab-dynamic.elf

aarch64-linux-gnu-gcc -static -no-pie \
    lab-driver.o lab-adjust.o -o lab-static.elf

# 3. Futtatás qemu-user környezetben.
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab-dynamic.elf
printf 'dynamic exit=%d\n' "$?"

qemu-aarch64 ./lab-static.elf
printf 'static exit=%d\n' "$?"
```

A GCC-meghajtó azért végzi a linkelést, mert a C futtatókörnyezet indulókódjára és a `printf` könyvtári megvalósítására is szükségünk van. A `-no-pie` mindkét változatnál kikapcsolja a pozíciófüggetlen végrehajtható fájl készítését, így az összehasonlítás nem keveredik a PIE különbségeivel.

Mérve (a lektor mérése, GCC 13.3.0, binutils 2.42, QEMU 8.2.2): mindkét program `pos=26 neg=0` sort ír ki; a shell által kiírt kilépési állapot mindkét esetben 0.

A dinamikus változatnál a `-L` az AArch64 betöltő és könyvtárak kereséséhez ad prefixet. Ha a program a betöltő hiányára panaszkodik, először a cross-könyvtárak telepítését és ezt az útvonalat ellenőrizzük, ne az assembly aritmetikát.

11.2.4 Függvény keresése és a szimbólumtábla olvasása

Először ellenőrizzük az objektum és a végrehajtható fájl típusát, majd keressük meg a függvényt:

```
# Target: qemu-userhez készült AArch64 ELF-ek; offline vizsgálat.
aarch64-linux-gnu-readelf -h lab-adjust.o
aarch64-linux-gnu-readelf -h lab-dynamic.elf

aarch64-linux-gnu-objdump -t lab-dynamic.elf
aarch64-linux-gnu-readelf -sW lab-dynamic.elf

aarch64-linux-gnu-objdump -d \
    --disassemble=lab_adjust lab-dynamic.elf
```

```
aarch64-linux-gnu-objdump -d \
    --disassemble=lab_adjust lab-static.elf
```

Mért ellenőrzés (a lektor mérése, binutils 2.42): az ELF-fejléc `ELF64`, little-endian és `AArch64` tulajdonságokat jelez. Az objektum típusa `REL`, a két, itt nem PIE-ként linkelt programé `EXEC`. A dinamikus linkelés tehát önmagában nem jelenti azt, hogy a fájl típusa `DYN`. A `lab_adjust` bejegyzése 20 bájtos `FUNC`, `GLOBAL` kötéssel, a 13. szekcióban.

A `readelf -sW` kapcsolóiban a `-s` szimbólumtáblát kér, a `-W` széles kimenetet engedélyez. A `lab_adjust` bejegyzésében ezeket keressük:

Mező	Értelmezés	A példában várt tartalom
`Value`	A szimbólum értéke	Objektumban szekción belüli eltolás; itt a linkelt ELF-ben virtuális cím
`Size`	A szimbólum mérete bájtban	20
`Type`	Szimbólumtípus	`FUNC`
`Bind`	Kötés	`GLOBAL`
`Vis`	Láthatóság	`DEFAULT`
`Ndx`	Szekcióindex vagy különleges jelölés	A kódot tartalmazó szekció indexe
`Name`	Név	`lab_adjust`

A 20 bájt a GNU assembler `.size` direktívájából származik: öt A64 utasítás, utasításonként 4 bájttal. Az előtte álló igazítási kitöltés nem része ennek a méretnek. A szekcióindexet a `readelf -SW` kimenetéhez lehet hozzárendelni.

Az `objdump -t` ugyanezt más oszloprendben, tömör jelölésekkel mutatja; nem kell a két eszköz formátumát azonosnak várni.

Fontos a táblák megkülönböztetése:

- a `.symtab` a linkeléshez és elemzéshez hasznos teljesebb szimbólumtábla;
- a `.dynsym` a dinamikus linkeléshez szükséges szimbólumokat tartalmazza.

Attól, hogy a `lab_adjust` globális kötésű, még nem feltétlenül kerül be a végrehajtható fájl dinamikus szimbólumtáblájába. A `GLOBAL` nem azonos azzal, hogy „mindenképpen exportált dinamikus API”.

11.2.5 A függvény disassembly-jének annotálása

A konkrét linkelt cím helyett most a függvény elejétől mért eltolást használjuk. Ezek nem önálló memóriacímek.

Eltolás [bájt]	A64 utasítás	Annotáció
`0x00`	`add W1, W0, W0, lsl #1`	`W1 = x + 2x`
`0x04`	`add W1, W1, #5`	`W1 = 3x + 5`
`0x08`	`cmp W0, #0`	NZCV beállítása az eredeti bemenet és nulla összehasonlításával

Eltolás [bájt]	A64 utasítás	Annotáció
0x0c	`csel W0, W1, WZR, ge`	Nemnegatív bemenetre a számított érték, egyébként nulla
0x10	`ret`	Visszatérés az A64 `X30` regiszterben lévő címre

A disassembler megjelenítheti a regiszterneveket kisbetűvel; ez nem jelent eltérő regisztert. Az aliasok megjelenítése is változhat: a `CMP` mögötti alaputasítás megmutatásához használható az AArch64-specifikus `-M no-aliases` kapcsoló.

Levezetett állapotok:

- 7 bemenetnél `W1` előbb 21, majd 26; a feltétel igaz, ezért `W0 = 26`.
- -7 bemenetnél `W1` előjeles olvasata előbb -21, majd -16; a feltétel hamis, ezért `W0 = 0`.

Az annotáció célja nem a mnemonic magyarra fordítása, hanem az adatfolyam és a szerződés összekapcsolása. Különösen ellenőrizzük, **melyik utasítás hozza létre a felhasznált állapotjelzőket**.

Az `objdump -d` rendesen a kódot tartalmazó szekciókat disassemblálja. A `-D` ennél szélesebben próbál utasításként értelmezni tartalmat; adatbájtokból is születhet hihető mnemonic. Ez önmagában nem bizonyítja, hogy az adott bájtok végrehajtandó kódot alkotnak.

11.2.6 Relokáció: a még kitöltendő kapcsolat

Vizsgáljuk meg a C-hívó objektumát még linkelés előtt:

```
# Target: AArch64/qemu-user objektum és ELF; offline vizsgálat.
aarch64-linux-gnu-objdump -r -j .text lab-driver.o
aarch64-linux-gnu-objdump -dr lab-driver.o

aarch64-linux-gnu-objdump -R lab-dynamic.elf
aarch64-linux-gnu-readelf -rW lab-dynamic.elf
```

Az `objdump -r` a normál relokációkat listázza. A `-j .text` szűkítés azért hasznos, mert a `-g`-vel fordított objektumban a kimenet nagy része a `.debug_*` szekciók relokációja; a hívásokhoz a `.text` mindössze három bejegyzése tartozik. A `-dr` együtt különösen hasznos: a disassembly megfelelő helyei mellé kerülnek a relokációs adatok.

A `lab-driver.o` fájlban a lektor mérésében (binutils 2.42) három `R_AARCH64_CALL26` bejegyzés szerepel: kettő a `lab_adjust`, egy a `printf` hívásához kapcsolódik. Egy bejegyzés lényeges elemei:

1. a javítandó hely szekción belüli eltolása;
2. a relokáció típusa;
3. a hivatkozott szimbólum és az esetleges addend, vagyis hozzáadandó korrekció.

Az A64 `BL` közvetlen hívás 26 bites, előjeles, négyszeres skálázású eltolást kódol. Elérési tartománya -128 MiB és $+128$ MiB-4 bájt között van. A linker számítja ki a megfelelő eltolást; szükség esetén köztes ugrókódot is alkalmazhat.

Objektumfájlban ne tekintsük a hívás mellett látható ideiglenes célcímet véglegesnek. A relokáció mondja meg, milyen kapcsolatot kell a linkernek létrehoznia.

A kisbetűs `-r` és a nagybetűs `-R` nem felcserélhető: utóbbi a dinamikus relokációkat jeleníti meg. A végleges fájlban üres normál relokációs lista nem jelenti azt, hogy nincsenek függvényhívások.

11.2.7 Statikus és dinamikus linkelés a disassembly-ben

A saját `lab_adjust` függvényünk mindkét fájlba bekerül. Mérete és utasításai a lektor mérésében azonosak, elhelyezése azonban eltérhet.

A könyvtári `printf` kapcsolata más:

Szempont	Statikus változat	Dinamikus változat
Könyvtári megvalósítás	A szükséges könyvtári objektumok bekerülnek az ELF-be	A megvalósítás jellemzően megosztott könyvtárban található
Hívás megjelenítése	A beépített megvalósításra vagy kapcsolódó belső szimbólumra mutathat	Gyakran `printf@plt` cél látható
Teljes disassembly	Rendszerint jóval nagyobb	Nem tartalmazza a teljes libc kódját
Függőségek	Nincs szükség dinamikusan betöltött libc-re	Dinamikus betöltő és megfelelő könyvtárak szükségesek

A statikus glibc-ben a `printf _IO_printf` néven (aliasként) jelenik meg, ezért a statikus változat disassemblyjében ne a `printf` szót keressük: a lektor mérésében a hívás célpontja `bl …<_IO_printf>` volt, a dinamikus változatban pedig `bl …<printf@plt>`.

Az **eljáráskapcsoló tábla (Procedure Linkage Table, PLT)** köztes hívókódot tartalmazhat. Ehhez a **globális eltolástábla (Global Offset Table, GOT)** címet tároló bejegyzései kapcsolódhatnak. A `printf@plt` tehát nem feltétlenül a formázást végző függvény törzse.

```
# Target: AArch64/qemu-user ELF-ek; offline összehasonlítás.
aarch64-linux-gnu-objdump -d --disassemble=main lab-dynamic.elf
aarch64-linux-gnu-objdump -d --disassemble=main lab-static.elf
aarch64-linux-gnu-objdump -d -j .plt lab-dynamic.elf

aarch64-linux-gnu-readelf -dW lab-dynamic.elf
aarch64-linux-gnu-readelf -lW lab-dynamic.elf
```

A dinamikus táblában a `NEEDED` bejegyzéseket, a programszegmenseknél az `INTERP` bejegyzést keressük. A pontos PLT-kódot, szimbólumaliasokat és könyvtári belső hívásokat a toolchain befolyásolja. Statikus ELF-ben is előfordulhatnak speciális, induláskor feldolgozott relokációk; a „statikus” nem minden relokáció teljes hiányát jelenti.

11.2.8 Tipikus hibák és ellenpróbák

A reprodukálható sorrend: **tünet → hipotézis → adatgyűjtés → egy változtatás → ellenőrzés**.

Tünet	Hipotézis	Adatgyűjtés és ellenpróba
Az eszköz nem tud disassemblálni	Nem megfelelő architektúrájú `objdump`	`readelf -h`, majd az AArch64 cross-`objdump` használata
Név szerint nincs függvénykimenet	Hiányzó szimbólum, elírás vagy eltávolított függvény	Szimbólumtábla vizsgálata; az eredeti, nem csupaszított fájl összehasonlítása
Objektumban furcsa híváscél látszik	Még fel nem oldott relokáció	Ugyanazon objektum megnyitása `-dr` kapcsolóval
A könyvtári függvény helyén rövid ugrókód van	PLT-bejegyzést nézünk	`.plt`, dinamikus relokációk és `NEEDED` vizsgálata
Assembly javítása után sem változik a kód	Régi objektumot linkeltünk	Assembly újraösszeállítása, újralinkelés, majd célzott disassembly

11.2.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért lehet a `lab_adjust` definiált szimbólum a `lab-adjust.o`, de `UND` jelölésű a `lab-driver.o` fájlban? Mi történik linkeléskor?

2. (implementációs, középszint — cél: rutinépítés) Módosítsd a függvényt úgy, hogy nemnegatív bemenetre $5x + 9$ legyen az eredmény. Maradjon öt utasításos. Frissítsd a C-ellenőrzést, építsd újra mindkét változatot, és annotáld a módosult utasításokat.

3. (hibakeresési, középszint — cél: rutinépítés) Távolítsd el a nem szükséges szimbóluminformációt egy külön laborpéldányból, majd vizsgáld meg, hogy a név szerinti keresés sikertelensége együtt jár-e működésképtelenséggel:

```
# Target: AArch64 Linux, qemu-user.
aarch64-linux-gnu-objcopy --strip-all \
    lab-dynamic.elf lab-stripped.elf

aarch64-linux-gnu-objdump -t lab-stripped.elf
aarch64-linux-gnu-readelf --dyn-syms -W lab-stripped.elf
aarch64-linux-gnu-objdump -d \
    --disassemble=lab_adjust lab-stripped.elf

qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab-stripped.elf
printf 'stripped exit=%d\n' "$?"
```

Jegyezd fel, melyik szimbólumtábla maradt meg, és miért lehet szükséges.

11.2.10 Megoldások

1. megoldás — Segítség. Az `UND` jelölés azt jelzi, hogy az adott OBJEKTUMFÁJL maga nem tartalmazza a szimbólum definícióját — melyik eszköz az, amelyik később a két objektumot egymás mellé olvasva a hivatkozást a definícióhoz köti?

1. megoldás — Teljes megoldás. A `lab-adjust.o` tartalmazza a függvény definícióját és kódját. A `lab-driver.o` csak hivatkozik rá; az `UND` feloldatlan, nem definiált szimbólumot jelez az adott objektumban. A linker összekapcsolja a hivatkozásokat a definícióval, és javítja a hívási helyeket.

2. megoldás — Segítség. $5x = x + 4x$; az A64 `ADD`-nak van olyan alakja, amely egy regisztert egy másik, `LSL`-lel skálázott regiszterhez ad hozzá egyetlen utasításban — ez pont a négyszerezést végzi el.

2. megoldás — Teljes megoldás. Az első két sor helyére az alábbi A64 utasítások kerülnek:

```
// A64 ISA: a lab_adjust első két utasításának cseréje.
add     w1, w0, w0, lsl #2
add     w1, w1, #9
```

Az eltolás négyszerez, ezért az első eredmény $x + 4x$. A 9 kódolható az immediate mezőben. A pozitív teszt várt eredménye 44, a negatívé továbbra is 0; a C-ellenőrzésben 26 helyett 44 szükséges. A korábbi buildlépések megismétlése után a méret továbbra is 20 bájt.

3. megoldás — Segítség. A `--strip-all` a NEVEKET (szimbólumtáblát) távolítja el, nem a gépi kódot — mi a különbség aközött, hogy egy függvényt "név szerint" nem találsz, és aközött, hogy a függvény ténylegesen hiányzik a bináris kódból?

3. megoldás — Teljes megoldás. A hagyományos `.symtab` eltűnik, ezért a `lab_adjust` név szerinti disassembly-je nem adja a korábbi függvénylistát. A dinamikus linkeléshez szükséges `.dynsym` megmarad. A lektor mérésében a stripelt fájlon az `nm no symbols`-t írt, a `.dynsym` 7 bejegyzést tartalmazott, a program pedig továbbra is `pos=26 neg=0` kimenetet adott, 0 kilépési állapottal. A teljes `objdump -d` továbbra is mutat kódot: a név elvesztése nem azonos az utasítások eltávolításával. A `--strip-debug` más művelet: önmagában nem távolítja el az összes hagyományos szimbólumot.

11.2.11 Rövid összefoglaló

A bináris olvasását az ELF architektúrájának és típusának ellenőrzésével kezdjük, majd szimbólumnév alapján szűkítjük a disassembly-t. Az utasítások mellett mindig vizsgáljuk az adatfolyamot, az állapotjelzők eredetét és a függvény szerződését. Objektumfájlban a relokáció nélkül félrevezető lehet egy hívás megjelenített célja. Statikus és dinamikus linkelésnél ugyanaz a saját függvény hasonló maradhat, miközben a könyvtári hívások útvonala eltér. A hiányzó név önmagában nem bizonyít hiányzó kódot; ezt szimbólumtáblával, teljes disassembly-vel és futtatási ellenpróbával különítjük el.

- **Disassembly:** gépi kód utasításként értelmezett szöveges megjelenítése.
- **Szimbólumtábla:** nevekhez rendelt értékek és tulajdonságok táblája.
- **Relokáció:** helyfüggő érték kialakítását vagy javítását előíró bejegyzés.

11.2.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GNU Binutils: objdump — disassembly, szimbólumok, normál és dinamikus relokációk. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU Binutils: readelf — ELF-fejlécek, szimbólumtáblák és dinamikus információk. Ugyanaz a binutils 2.47-es kiadás.
- GNU assembler kézikönyv — AArch64 szintaxis és szimbólumleíró direktívák. Ugyanaz a binutils 2.47-es kiadás.

Ellenőrzési hiányok: a példák a lektor mérési környezetében (GCC 13.3.0, binutils 2.42, QEMU 8.2.2) lefutottak, az itt közölt eredmények onnan származnak. Az Ubuntu 24.04 tényleges csomagverzióit, a cross-könyvtárak elérhetőségét, a PLT pontos alakját és a megjelenített szimbólumaliasokat a hallgató laborjában kell ellenőrizni. A vizsgálat nem tartalmaz teljesítménymérést.

11.3 Hibakeresési esettanulmány: szegmentálási hiba gyökérokának megtalálása

11.3.1 Motiváció: a hibás utasítás nem feltétlenül a hiba oka

Egy C-ből hívott assembly függvény tömbelemet olvas. Az első elemmel működik, a másodikkal szegmentálási hibát kapunk. A hibakereső egy betöltőutasításnál áll meg, de ebből még nem következik, hogy maga a betöltés kódolása rossz: hibás lehet a cím előállítása, egy korábban felülírt regiszter vagy a hívási szerződés megsértése is.

A **szegmentálási hiba** (*segmentation fault*, Linux alatt jellemzően `SIGSEGV`) azt jelzi, hogy a folyamat olyan memória-hozzáférést kísérelt meg, amelyet a virtuálismemória-leképezés vagy annak jogosultságai nem engednek meg. Ez operációsrendszer-szintű esemény, nem egyetlen assembly programozási hibafajta neve.

Két címet különböztessünk meg:

- **Hibázó utasítás címe:** hol található a végrehajtani próbált utasítás; ezt mutatja a programszámláló.
- **Hozzáférési hibacím** (*fault address*): melyik memóriacímet próbálta elérni az utasítás.

A labor a Disassembly objdump/readelf-fel munkafolyamatára épít. Minden assembly utasítás és regiszternév **A64 ISA, AArch64, Armv8-A** környezetre vonatkozik. A direktívák GNU `as` direktívák ehhez a célhoz. A célprofil little-endian Linux, az elsődleges futtató `qemu-aarch64`.

Az alábbi eredmények **mérve (GNU + glibc + QEMU 8.2.2 + GDB 15.1)**: a lektor az esettanulmány lépéseit végig lefuttatta, és az itt közölt kimenetek onnan származnak.

11.3.2 Kontrollált hibafeltétel: tömb egy védőlap előtt

Egy hibás indexelés nem szükségképpen okoz azonnali összeomlást. Ha a rossz cím továbbra is olvasható lapon van, a program egyszerűen más adatot olvas. Ezért a laborban egy **védőlapot** (*guard page*) hozunk létre: egy leképezett, de a program számára nem olvasható és nem írható memórialapot.

A címeket az `mmap` szolgáltatja. Nem használunk rögzített vagy találomra választott címet. A lapméretet futás közben kérdezzük le.

Legyen:

- B az allokált terület kezdőcíme, bájtcímként értelmezve;
- P a lapméret, bájtban;
- n a tömb elemszáma;
- i az előjel nélküli elemindex, ahol $0 \leq i < n$.

Az első lap olvasható és írható; a következő lap hozzáférhetetlen. A lap végére helyezett `uint32_t` tömb kezdete:

$$A = B + P - 4n.$$

A helyes elemcím $A + 4i$. A hibás függvény ehelyett $A + 8i$ címet képez: összekeveri a 64 bites mutató méretét a tömbelem 4 bájtos méretével.

Levezetés az alapteszthez: $n = 2$, $i = 1$ esetén:

Mennyiség	Levezetett érték
Tömb kezdete	$B + P - 8$
Helyes második elem címe	$B + P - 4$
Hibásan képzett cím	$B + P$
Hibás olvasás célterülete	A védőlap első 4 bájtja

Így a hiba nem a környező memória véletlen tartalmától függ.

11.3.3 Kidolgozott példa: hibás elemskálázás

Munkakönyvtár: `~/arm-labor/a64/a042/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a042
cd ~/arm-labor/a64/a042
```

A függvény C-interfésze:

```
uint32_t lab_get(const uint32_t *p, size_t i);
```

Előfeltétele, hogy `p` megfelelően igazított, legalább `i+1` olvasható elemet tartalmazó tömbre mutasson. A függvény nem végez határellenőrzést.

Az AAPCS64 szerint a mutató az A64 `X0`, az index az A64 `X1` regiszterben érkezik; a 32 bites eredmény az A64 `W0` regiszterben tér vissza.

Mentsük a hibás függvényt `lab-offset.s` néven:

```
// A64 ISA, Armv8-A; GNU as; little-endian laborprofil.
.arch armv8-a
.text
.p2align 2
.global lab_get
.type lab_get, %function
lab_get:
    .cfi_startproc
    add x2, x0, x1, lsl #3     // HIBA: az indexet 8-cal szorozza.
    ldr w0, [x2]              // 4 bájtos elem betöltése.
    ret
    .cfi_endproc
.size lab_get, .-lab_get

.section .note.GNU-stack,"",%progbits
```

Az A64 `ADD` itt eltolt regiszteres operandust használ. A 64 bites változat `LSL` eltolási értéke `0…63` lehet; a `#3` tehát szabályosan kódolható. Ez **szemantikai hiba**, nem assemblerhiba. A helyes eltolás `#2`, mert $2^2 = 4$.

Az A64 `X2` hívó által mentendő ideiglenes regiszter, ezért használata nem sérti a hívási szerződést. A függvény nem hív másik függvényt, és nem módosítja a vermet. A `.cfi_startproc` és `.cfi_endproc` a veremvisszafejtéshez használható metaadatot jelöli; nem hajt végre regiszter-mentést.

A teljes tesztprogram legyen `lab-offset.c`:

```
#define _GNU_SOURCE
#include <stdint.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

extern uint32_t lab_get(const uint32_t *p, size_t i);

static void fail(const char *what)
{
    perror(what);
    exit(2);
}

__attribute__((noinline))
static void run_case(size_t n, size_t i, int tail)
{
    long raw_page = sysconf(_SC_PAGESIZE);
    if (raw_page <= 0) {
        fputs("Nem kerdezheto le a lapmeret.\n", stderr);
        exit(2);
    }

    size_t page = (size_t)raw_page;
    if (page > SIZE_MAX / 2 || n == 0 ||
        n > page / sizeof(uint32_t) || i >= n) {
        fputs("Ervenytelen teszteset.\n", stderr);
        exit(2);
    }

    unsigned char *map = mmap(NULL, 2 * page, PROT_NONE,
                              MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (map == MAP_FAILED)
        fail("mmap");

    if (mprotect(map, page, PROT_READ | PROT_WRITE) != 0)
        fail("mprotect");

    size_t offset = tail ? page - n * sizeof(uint32_t) : 0;
    uint32_t *p = (uint32_t *)(map + offset);

    for (size_t j = 0; j < n; ++j)
        p[j] = 1000u + 17u * (uint32_t)j;

    uint32_t expected = p[i];
    uint32_t got = lab_get(p, i);

    if (got != expected) {
        fprintf(stderr,
```

```
                "FAIL: n=%zu i=%zu tail=%d got=%u expected=%u\n",
                n, i, tail, (unsigned)got, (unsigned)expected);
        exit(1);
    }

    if (munmap(map, 2 * page) != 0)
        fail("munmap");
}

int main(int argc, char **argv)
{
    unsigned cases = 0;

    if (argc == 1) {
        run_case(2, 1, 1);
        cases = 1;
    } else if (argc == 2 && strcmp(argv[1], "--all") == 0) {
        const size_t lengths[] = {1, 2, 3, 16};

        for (size_t k = 0; k < sizeof lengths / sizeof lengths[0]; ++k) {
            for (int tail = 0; tail <= 1; ++tail) {
                for (size_t i = 0; i < lengths[k]; ++i) {
                    run_case(lengths[k], i, tail);
                    ++cases;
                }
            }
        }
    } else {
        fputs("Hasznalat: lab-offset [--all]\n", stderr);
        return 2;
    }

    printf("OK: %u eset\n", cases);
    return 0;
}
```

A `tail=0` esetek a lap elejére, a `tail=1` esetek közvetlenül a védőlap elé helyezik a tömböt. Így egyszerre teszteljük a helyes értéket és a határközeli címzést.

11.3.3.1 Összeszerelés, linkelés, reprodukció

Ubuntu 24.04 cross-toolchainnel, **qemu-user célra**:

```
# Target: qemu-user / AArch64 Linux, Armv8-A, little-endian.
aarch64-linux-gnu-as -g -EL -march=armv8-a \
    -o lab-offset.o lab-offset.s

aarch64-linux-gnu-gcc -std=c11 -g3 -O0 -Wall -Wextra \
    -march=armv8-a -mlittle-endian \
    -fno-omit-frame-pointer -fno-optimize-sibling-calls \
    -c lab-offset.c -o lab-driver.o

aarch64-linux-gnu-gcc -static -no-pie \
    -o lab-offset lab-driver.o lab-offset.o

aarch64-linux-gnu-objdump -d --disassemble=lab_get lab-offset

# Szándékos összeomlás: ne keletkezzen core fájl.
ulimit -c 0
lab_status=0
qemu-aarch64 ./lab-offset || lab_status=$?
printf 'status=%s\n' "$lab_status"
```

A statikus linkeléshez a célarchitektúra statikus C-könyvtára is szükséges. Ha a linker például a céloldali `libc.a` hiányát jelzi, az környezeti probléma, nem a vizsgált címzési hiba.

Mérve (a lektor mérése, QEMU 8.2.2): `SIGSEGV`; a QEMU üzenete `qemu: uncaught target signal 11 (Segmentation fault) - core dumped`, a Bash által kiírt állapot `status=139`, azaz $128 + 11$. A QEMU és a shell diagnosztikai szövege verziófüggő. Az `OK` sor nem jelenhet meg.

11.3.4 A hibapont és a hívási lánc felderítése

Első terminálban indítsuk el a célt a QEMU távoli hibakereső-felületével:

```
# Target: qemu-user / AArch64; várakozás a GDB kapcsolatára.
ulimit -c 0
qemu-aarch64 -g 12345 ./lab-offset
```

Második terminálban:

```
# Debugger: x86_64 host; target: qemu-user / AArch64.
gdb-multiarch -q ./lab-offset \
    -ex 'set pagination off' \
    -ex 'target remote 127.0.0.1:12345' \
    -ex 'break lab_get' \
    -ex 'continue' \
    -ex 'info registers x0 x1 x2 sp x29 x30 pc' \
    -ex 'set $base = $x0' \
    -ex 'x/2uw $x0' \
    -ex 'si' \
    -ex 'p/d $x2 - $base' \
    -ex 'continue' \
    -ex 'x/i $pc' \
    -ex 'info registers x0 x1 x2 sp x29 x30 pc' \
    -ex 'bt'
```

A lépések jelentése:

1. A töréspontnál ellenőrizzük a bemeneti regisztereket.
2. A `x/2uw $x0` két 32 bites, előjel nélküli decimális értéket olvas: várhatóan `1000` és `1017`.
3. A `si` egyetlen gépi utasítást léptet: végrehajtja a hibás címképzést.
4. A címkülönbség a mérésben `8`, holott az index `1`, az elemméret pedig 4 bájt.
5. A folytatás végrehajtja a betöltési kísérletet; a GDB a mért esetben `SIGSEGV` miatt állt meg az `ldr w0, [x2]` utasításon.

A hiba pillanatában:

A64 állapot	Mért értelmezés
`PC`	Az A64 `LDR W0, [X2]` utasítás címe
`X0`	A tömb kezdőcíme; a sikertelen betöltés még nem adott eredményt
`X1`	`1`, előjel nélküli index
`X2`	A védőlap kezdőcíme
`SP`	A függvénybelépéshez képest változatlan

A GDB-ben a `p $_siginfo` parancs a jel részletes adatait kérdezné le. **qemu-user alatt azonban a `$_siginfo` nem érhető el**: a lektor mérésében (QEMU 8.2.2, GDB 15.1) a válasz

`Unable to read siginfo` volt. A hibás címet ezért a `x/i $pc` és az érintett bázisregiszter (`p/x $x2`) alapján határozzuk meg; ennél az A64 betöltésnél az effektív cím pontosan az `X2` tartalma.

A `bt` által megjelenített **hívási lánc** (*backtrace*) a lektor mérésében a következő szerkezetet adta:

Keret	Függvény
#0	`lab_get`
#1	`run_case`, `n=2`, `i=1`, `tail=1`
#2	`main`

A `frame 1`, majd az `info args` és `info locals` parancsokkal ellenőrizhetjük a C-oldali előfeltételeket és a lapméretet. Ezután a `frame 0` visszavisz a hibázó függvényhez.

A hívási lánc nem önmagában bizonyíték: sérült verem vagy hibás visszafejtési metaadat esetén téves vagy hiányos lehet. Ugyanígy a debugger esetleges sikeres memóriaolvasása sem bizonyítja, hogy a program ugyanazt a címet jogosultan olvashatja: a hibakereső memóriaelérése eltérhet a célszálétól.

11.3.5 Hipotézis, egy változtatás, ellenőrzés

A hipotézisünk konkrét:

> A tömbelem címe azért kerül a védőlapra, mert a függvény az indexet 4 helyett 8 bájttal skálázza.

Ez ellenőrizhető előrejelzést ad: kizárólag az eltolás javítása után a címkülönbség `4`, a visszatérési érték `1017`, a kilépési állapot `0` lesz.

Ne módosítsuk egyszerre a teszttömböt, a lapvédelmet és az assembly kódot. Hozzunk létre külön javított változatot:

```
# Target: qemu-user / AArch64; egyetlen forrásváltoztatás.
sed 's/lsl #3/lsl #2/' lab-offset.s > lab-offset-fixed.s

aarch64-linux-gnu-as -g -EL -march=armv8-a \
    -o lab-offset-fixed.o lab-offset-fixed.s

aarch64-linux-gnu-gcc -static -no-pie \
    -o lab-offset-fixed lab-driver.o lab-offset-fixed.o

aarch64-linux-gnu-objdump -d --disassemble=lab_get lab-offset-fixed

qemu-aarch64 ./lab-offset-fixed
printf 'status=%s\n' "$?"

qemu-aarch64 ./lab-offset-fixed --all
printf 'status=%s\n' "$?"
```

Mérve (a lektor mérése): először `OK: 1 eset`, majd `OK: 44 eset`; mindkét futás kilépési állapota `0`. A teljes tesztszám:

$$2(1 + 2 + 3 + 16) = 44,$$

ahol a kettes szorzó a két elhelyezést jelenti.

A javított binárist a korábbi GDB-lépéssorral is vizsgáljuk meg: a címképzés után `4` legyen az eltérés, a betöltés után az A64 `W0` értéke `1017`. Ez összekapcsolja a forrásváltozást, a tényleges gépi kódot és a megfigyelt eredményt.

11.3.6 Más hibaforrások elkülönítése

Tünet	Hipotézis	Célzott adatgyűjtés és ellenpróba
Már az első betöltés címe rossz	Hibás offset vagy elemskálázás	Bemeneti mutató, index, elemméret és effektív cím összevetése
Egy belső hívás után romlik el a mutató	Élő érték maradt egy felülírható regiszterben	Regiszterérték rögzítése a hívás előtt és után; kizárólag a megőrzés javítása
A hívó állapota változik meg váratlanul	A függvény nem állít helyre megőrzendő regisztert	Az A64 `X19-X29` érintett regisztereinek összevetése belépéskor és kilépéskor
Visszatéréskor vagy később történik a hiba	Aszimmetrikus prológus/epilógus	Veremmutató és mentési helyek követése; a foglalás és felszabadítás párosítása

Az A64 `X30` visszatérési címét egy újabb A64 `BL` felülírja. Egy további függvényt hívó rutin ezért nem támaszkodhat feltétel nélkül az eredeti regisztertartalomra. A jelenlegi levélfüggvény viszont nem végez ilyen hívást.

A veremhiba sem mindig egyszerű igazítási hiba. Ha a prológus 16 bájtot foglal, az epilógus pedig 32 bájtot szabadít fel, az A64 `SP` továbbra is 16 bájtra igazított lehet, mégsem áll helyre. Az igazítás ellenőrzése szükséges, de nem helyettesíti a belépési és kilépési érték összevetését.

11.3.7 Mit jelent a hiba tényleges megszűnése?

A „most lefutott” gyenge bizonyíték. Ebben a laborban négy külön ellenőrzés kell:

- **Eredeti reprodukció:** ugyanaz a határhelyzet már nem hibázik.
- **Értékellenőrzés:** nem pusztán olvasható címre jutunk, hanem a helyes elemet kapjuk.
- **Regressziós teszt:** több hossz, minden érvényes index és két elhelyezés működik.
- **Negatív ellenpróba:** az eredeti bináris továbbra is elbukik; a teszt valóban érzékeli a hibát.

Az érvénytelen, például `i == n` index nem része a függvény szerződésének. Annak tesztelése nem helyettesíti az érvényes bemenetek ellenőrzését.

A hibajegyzőkönyvet például `lab-report.md` néven rögzíthetjük:

Mező	Tartalom
Tünet	`SIGSEGV` a második elem olvasásakor
Gyökérok (*root cause*)	Hibás, nyolcbájtos elemskálázás
Közreható tényező (*contributing factor*)	A mutatóméret és az elemméret összekeverése
Felismerést késleltető tényező	A nulladik index tesztje mindkét skálázással helyes címet ad

Mező	Tartalom
Bizonyíték	`X1=1`, címkülönbség `8`, betöltési cím a védőlapon
Javítás	Kizárólag az eltolás módosítása
Elfogadási feltétel	Helyes címkülönbség, helyes eredmény, 44 sikeres eset

A védőlap nem a programhiba gyökéroka: a meglévő hibát tette determinisztikusan láthatóvá.

11.3.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért nem különbözteti meg a helyes és hibás függvényt az `i=0` teszt? Lapvégi, 16 elemű tömbnél mi a helyes utolsó elem címe a védőlap kezdetéhez viszonyítva?

2. (implementációs, alapszint — cél: rutinépítés) Egészítsd ki a hosszlistát a `8` értékkel. Számítsd ki az új esetszámot, majd építsd újra a javított programot azonos fordítási kapcsolókkal.

3. (hibakeresési, középszint — cél: rutinépítés) Futtasd az eredeti hibás binárist `--all` kapcsolóval. Miért lehetséges, hogy nem `SIGSEGV`, hanem `FAIL` az első eredmény? GDB-ben ellenőrizd az első értékeltérésnél a képzett címet és az olvasott adatot.

11.3.9 Megoldások

1. megoldás — Segítség. Helyettesítsd be $i = 0$-t mindkét képletbe ($4i$ és $8i$) — különböznek-e az eredmények? A lapvégi elhelyezésnél induljon ki abból, hogy a tömb kezdete $B + P - 4n$.

1. megoldás — Teljes megoldás. Nulla index esetén $4i = 8i = 0$, ezért mindkét függvény a tömb elejét olvassa. A 16 elemű lapvégi tömb kezdete $B + P - 64$; az utolsó elem helyes címe $B + P - 64 + 4 \cdot 15 = B + P - 4$. A teljes négysávos bájttartomány még az olvasható lapon van.

2. megoldás — Segítség. Az esetszám-képlet $2 \sum n$ minden hosszra; csak egy taggal kell bővíteni az összeget.

2. megoldás — Teljes megoldás. A hosszlista `{1, 2, 3, 8, 16}` lesz. Az esetszám $2(1 + 2 + 3 + 8 + 16) = 60$. A javított változat várt kimenete `OK: 60 eset`, kilépési állapota `0`. Az új esetek az indexelési tartományt bővítik, nem módosítják a függvény szerződését.

3. megoldás — Segítség. Melyik `tail` elhelyezésnél (0 vagy 1) esik a hibásan képzett cím MÉG a leképezett, csak épp rossz tartalmú területre, nem a védőlapra?

3. megoldás — Teljes megoldás. A ciklussorrendben az első eltérés a lektor mérésében `n=2`, `i=1`, `tail=0` esetén jelentkezett. A hibás cím a tömb kezdete után 8 bájttal van: a tömbön kívül, de még az olvasható lapon. Az anonim leképezés érintetlen része kezdetben nulla, ezért a mért diagnózis `FAIL: n=2 i=1 tail=0 got=0 expected=1017`, a kilépési állapot `1`. Ez bizonyítja, hogy a szegmentálási hiba hiánya nem jelent helyes címzést.

11.3.10 Rövid összefoglaló

A szegmentálási hiba vizsgálatakor külön kezeljük a hibázó utasítás címét és az elérni próbált memóriacímet. A regiszterekből rekonstruált címképzést a függvény szerződésével és a C-oldali adatelrendezéssel vetjük össze. A védőlap reprodukálhatóvá teszi a határátlépést, az értékellenőrzés pedig az olvasható memórián belüli téves hozzáférést is felismeri. Egyetlen célzott változtatást tesztelünk, majd regressziós és negatív ellenpróbával ellenőrizzük a javítást. A hívási láncot mindig a verem és a visszafejtési információ megbízhatóságával együtt értelmezzük.

- **Hozzáférési hibacím:** a sikertelen memória-hozzáférés célcíme.
- **Gyökérok:** a hibás működést közvetlenül előidéző alapvető hiba.
- **Közreható tényező:** a hibát elősegítő vagy felismerését késleltető körülmény.

11.3.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- GNU GDB kézikönyv — regisztervizsgálat, utasításszintű léptetés, hívási lánc és jelkezelés. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).
- GNU assembler kézikönyv — AArch64 szintaxis és CFI-direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- AAPCS64 — paraméterátadás és regisztermegőrzési kötelezettségek. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- QEMU User Mode Emulation — user-mode futtatás és hibakeresési kapcsolódás. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok: az esettanulmány a lektor mérési környezetében (GNU + glibc + QEMU 8.2.2 + GDB 15.1) végig lefutott, az itt közölt kimenetek onnan származnak. A laborban rögzíteni kell a telepített eszközverziókat és ellenőrizni kell a statikus célkönyvtár elérhetőségét. A dokumentációs linkek nem rögzítik automatikusan az Ubuntu 24.04 csomagverzióit.

11.4 Egyszerű instrumentáció: ciklusszámlálás és időmérés

11.4.1 Motiváció: a helyes eredmény mellé mérési szerződés kell

Egy assembly rutin már helyesen számol, de szeretnénk tudni, érdemes-e tovább optimalizálni. A teljes program futásideje önmagában kevés: tartalmazhat fájlkezelést, memóriakezelést, formázott kiírást és inicializálást is. Ezek könnyen elfedik a vizsgált rutin költségét.

Az **instrumentáció (instrumentation)** olyan kiegészítő kód beillesztése, amely megfigyelhetővé teszi a program működését. Ebben az alfejezetben két időbélyeget veszünk egy C-ből hívott assembly rutin ismételt végrehajtása körül. A módszer előfeltétele a helyes eredmény és a szabályos memóriaelérés; ezek vizsgálatát az előző hibakeresési esettanulmány tárgyalta.

Profil: I. rész, A64 ISA, Armv8-A, AArch64 Linux, little-endian, LP64 adatmodell. Az assembly példában minden utasítás és regiszter ehhez a profilhoz tartozik; a direktívák az AArch64 célú GNU assembler direktívái, nem processzorutasítások.

A bemutatott program a lektor mérési környezetében (GCC 13.3.0, QEMU 8.2.2) lefutott; a számszerű ellenőrzőösszeg mérve 16781312000. A futási idő konkrét értéke hostfüggő, ezért azt nem közöljük: a hallgató a saját laborjában méri.

11.4.2 Mit mér a két időbélyeg?

A libc `clock_gettime` függvénye a kiválasztott óra állapotát egy `struct timespec` objektumba írja. Ennek két mezője:

- `tv_sec`: egész másodperc;
- `tv_nsec`: a másodpercen belüli nanomásodperc, értéke sikeres hívás után 0 és 999999999 közötti.

A libc-felület nem feltétlenül jelent minden híváskor kernelbe lépést. Natív Linuxon például vDSO-alapú gyors út is lehetséges; a konkrét végrehajtási út libc-, kernel- és emulációfüggő. A mérési szerződésünket ezért az API-ra, nem egy feltételezett belső implementációra építjük.

Időtartamhoz `CLOCK_MONOTONIC` órát használunk. Ez nem a naptári időt adja, és a rendszeróra ugrásszerű átállítása nem okoz benne megfelelő ugrást. Nem azonos azonban a folyamat által ténylegesen felhasznált CPU-idővel: az ütemezés miatti várakozás is növelheti a mért időt.

Levezetés. Ha a kezdő időbélyeg (s_0, n_0), a záró pedig (s_1, n_1), akkor:

$$T_{\text{ns}} = (s_1 - s_0) \cdot 10^9 + (n_1 - n_0).$$

Itt s_0, s_1 másodpercben, n_0, n_1 nanomásodpercben értendő, az eredmény pedig nanomásodperc. A kivonást előjeles, megfelelő szélességű típussal végezzük: másodperchatár átlépésekor $n_1 - n_0$ negatív lehet, miközben a teljes időtartam pozitív.

A nanomásodperces reprezentáció **nem garantál nanomásodperces felbontást vagy pontosságot**.

11.4.2.1 Mi maradjon a mérési blokkon kívül?

Az adat-előkészítés, allokáció, kiírás és eredményellenőrzés ne kerüljön a két időbélyeg közé. A mérési intervallum itt szándékosan tartalmazza:

- az assembly rutin hívását és visszatérését;
- a rutin munkáját;
- a C-oldali ismétlőciklust és ellenőrzőösszeg-képzést;
- az időbélyegzés intervallumba eső járulékos költségét.

Ez tehát **hívási költséggel együtt értett rutinmérés**, nem egyetlen utasítás késleltetésének mérése.

Egy rövid rutint sokszor hívunk két időbélyeg között. Ha az ismétlésszám R, akkor a számított átlagos költség T_{ns}/R, mértékegysége ns/hívás. Az ismétlés csökkenti az időbélyegzés egy hívásra jutó hatását, de nem távolítja el a ciklus és a függvényhívás költségét.

11.4.3 Időmérés és ciklusszámlálás nem ugyanaz

Három különböző mennyiséget kell elválasztani:

1. **Algoritmikus iterációszám:** például hány tömbelemet dolgozunk fel.
2. **Dinamikus utasításszám:** hány ISA-utasítás hajtódik végre.
3. **Processzorciklusok száma:** egy meghatározott hardveres számláló által számolt ciklusok.

Ezek között nincs általános, állandó szorzótényező. Egy utasítás több ciklust igényelhet, miközben egy processzor bizonyos feltételek mellett több utasítás munkáját is átfedheti.

Az A64 környezetben elérhető `CNTVCT_EL0` időzítőszámláló sem általános CPU-ciklusszámláló. A teljesítményfigyelő egység ciklusszámlálójának, például az A64 `PMCCNTR_EL0` regiszterének alkalmazásszintű olvashatósága pedig rendszerkonfigurációtól függ. Ezekre itt nem építünk.

QEMU alatt különösen hibás lenne a mért másodperceket egy elképzelt target-órajel frekvenciájával megszorozni, majd az eredményt AArch64 ciklusszámnak nevezni.

11.4.4 Kidolgozott példa: tömbösszegzés időméréssel

Munkakönyvtár: `~/arm-labor/a64/a043/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a043
cd ~/arm-labor/a64/a043
```

Az assembly függvény előjel nélküli, 32 bites elemek összegét adja vissza előjel nélküli, 64 bites értékként. Általánosan az összeadás modulo 2^{64} értendő; a labor adatai mellett nincs túlcsordulás.

Az AAPCS64 szerint az A64 `X0` kapja a tömbmutatót, az A64 `X1` az elemszámot, a visszatérési érték pedig az A64 `X0` regiszterbe kerül.

A `lab-sum.S` fájl:

```
// A64 ISA, Armv8-A, little-endian; GNU as direktívák.
.arch armv8-a
.text
.p2align 2
.global lab_sum_u32
.type lab_sum_u32, %function

lab_sum_u32:
    mov     x2, xzr
    cbz     x1, .Llab_done

.Llab_loop:
    ldr     w3, [x0], #4
    add     x2, x2, x3
    subs    x1, x1, #1
    b.ne    .Llab_loop

.Llab_done:
    mov     x0, x2
    ret

.size lab_sum_u32, .-lab_sum_u32
.section .note.GNU-stack,"",%progbits
```

Az A64 `W3` regiszterbe írás nullázza az A64 `X3` felső 32 bitjét, ezért az előjel nélküli elem helyesen kerül a 64 bites összegbe. A függvény csak hívó által mentendő munkaregisztereket módosít, és nem hív másik függvényt.

A kódolási korlátok ebben a mintában:

- Az A64 `ldr` utóindexelt alakjában az eltolás előjeles, 9 bites bájtérték: $-256 \ldots 255$. A `#4` belefér.
- Az A64 `subs` immediate alakja 12 bites előjel nélküli konstanst fogad, opcionális 12 bites balra tolással. A `#1` közvetlenül kódolható.
- Az A64 `cbz` és `b.ne` elágazása előjeles, 19 bites, 4 bájttal skálázott PC-relatív eltolást használ. A közeli címkék elérhetők.

Pozitív n elemszámra a függvény dinamikus utasításszáma levezethető: két kezdőutasítás, elemenként négy ciklusutasítás, végül két záróutasítás, vagyis $4n + 4$. Ez **utasításszám, nem processzorciklusszám**.

A `lab-time.c` fájl:

```
#define _POSIX_C_SOURCE 200809L
#include <inttypes.h>
#include <math.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

/* C-oldal: AArch64 Linux, LP64, little-endian. */
extern uint64_t lab_sum_u32(const uint32_t *p, size_t n);

enum { N = 4096, SAMPLES = 9 };

#ifndef REPEATS
#define REPEATS 2000
#endif

static void lab_stamp(struct timespec *t)
{
    if (clock_gettime(CLOCK_MONOTONIC, t) != 0) {
        perror("clock_gettime");
        exit(2);
    }
}

static int64_t lab_elapsed_ns(const struct timespec *a,
                              const struct timespec *b)
{
    return ((int64_t)b->tv_sec - (int64_t)a->tv_sec)
               * INT64_C(1000000000)
           + (int64_t)b->tv_nsec - (int64_t)a->tv_nsec;
}

int main(void)
{
    static uint32_t data[N];
    int64_t elapsed[SAMPLES];
    uint64_t totals[SAMPLES];

    for (size_t i = 0; i < N; ++i)
        data[i] = (uint32_t)(i + 1);
```

```
    const uint64_t expected = (uint64_t)N * (N + 1) / 2;

    /* Bemelegítés és első helyességellenőrzés: mérésen kívül. */
    if (lab_sum_u32(data, N) != expected) {
        fputs("warmup checksum error\n", stderr);
        return 1;
    }

    for (unsigned s = 0; s < SAMPLES; ++s) {
        struct timespec start, end;
        uint64_t total = 0;

        lab_stamp(&start);
        for (unsigned r = 0; r < REPEATS; ++r)
            total += lab_sum_u32(data, N);
        lab_stamp(&end);

        elapsed[s] = lab_elapsed_ns(&start, &end);
        totals[s] = total;
    }

    /* Ellenőrzés, statisztika és minden normál kiírás: mérésen kívül. */
    int64_t minimum = elapsed[0];
    double mean = 0.0;

    for (unsigned s = 0; s < SAMPLES; ++s) {
        if (totals[s] != expected * REPEATS || elapsed[s] <= 0) {
            fputs("invalid sample or checksum\n", stderr);
            return 1;
        }
        if (elapsed[s] < minimum)
            minimum = elapsed[s];
        mean += (double)elapsed[s];
    }
    mean /= SAMPLES;

    double squared_deviations = 0.0;
    for (unsigned s = 0; s < SAMPLES; ++s) {
        double d = (double)elapsed[s] - mean;
        squared_deviations += d * d;
        printf("sample[%u]_ns=%" PRId64 "\n", s, elapsed[s]);
    }

    double sd = sqrt(squared_deviations / (SAMPLES - 1));
    printf("checksum=%" PRIu64 "\n", totals[SAMPLES - 1]);
    printf("min_ns=%" PRId64 " mean_ns=%.1f sd_ns=%.1f\n",
           minimum, mean, sd);
    printf("ns_per_call=%.3f\n", mean / REPEATS);
    return 0;
}
```

A külön fordított assembly függvényhez nem adunk `pure` vagy `const` attribútumot, és nem használunk LTO-t. A C-fordító nem ismeri a függvény törzsét; a visszatérési értékeket felhasználjuk és ellenőrizzük. Így nincs szükség arra, hogy az egész adattömböt `volatile` minősítésűvé tegyük — az megváltoztatná a vizsgált programot.

A bemelegítés csökkentheti az első kódfordítás és adatérintés hatását, de **nem garantál meghatározott cache-állapotot**.

11.4.4.1 Fordítás és futtatás

Az alábbi lépések targetje **qemu-user, AArch64 Linux**; a parancsokat az Ubuntu x86_64 hoston adjuk ki. A statikus linkeléshez szükséges a cross libc és libm fejlesztői állománya, amelyet a megfelelő cross fejlesztői csomagok, például a `libc6-dev-arm64-cross` biztosítanak.

```
# Target: qemu-user / AArch64 Linux. Assembly -> objektum.
aarch64-linux-gnu-gcc -march=armv8-a -g \
    -c lab-sum.S -o lab-sum.o

# Target: qemu-user / AArch64 Linux. C -> objektum.
aarch64-linux-gnu-gcc -std=c11 -O2 -g -Wall -Wextra \
    -march=armv8-a -fno-lto \
    -c lab-time.c -o lab-time.o

# Target: qemu-user / AArch64 Linux. Statikus linkelés.
aarch64-linux-gnu-gcc -static -fno-lto \
    lab-time.o lab-sum.o -lm -o lab-time

# Target: qemu-user / AArch64 Linux. Futtatás.
qemu-aarch64 ./lab-time
printf 'exit=%d\n' "$?"
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2):

Megfigyelés	Mért érték vagy feltétel
Egy hívás eredménye	$4096 \cdot 4097/2 = 8390656$
A kiírt `checksum`	16781312000
Időminták száma	9
Minden `sample[...]_ns`	Pozitív időtartam
Sikeres kilépési állapot	0
Hibás összeg vagy érvénytelen minta	1
Sikertelen időbélyegzés	2

A kimenet alakja a következő; a csúcsos zárójelű mezők helyén a hosttól függő időértékek állnak, ezért azokat szándékosan nem közöljük:

```
sample[0]_ns=<mért időtartam>
...
sample[8]_ns=<mért időtartam>
checksum=16781312000
min_ns=<mért minimum> mean_ns=<mért átlag> sd_ns=<mért szórás>
ns_per_call=<számított érték>
```

A lektor futásában mind a kilenc `sample[i]_ns` sor pozitív értéket adott, a `checksum` pontosan 16781312000 volt, és a program `0` állapottal lépett ki. Konkrét futási időt nem közlünk. A `mean_ns` egy teljes, 2000 hívásból álló mérési blokk átlaga, az `ns_per_call` pedig az ebből számított hívásonkénti költség.

11.4.5 Ismétlés, szóródás és QEMU-korlát

A kilenc időminta átlaga mellett a program minimumot és **mintaszórást (sample standard deviation)** is közöl. A variancia (variance) és a szórás nem ugyanaz:

$$s^2 = \frac{\sum_{i=1}^{m} (T_i - \overline{T})^2}{m-1}, \qquad s = \sqrt{s^2}.$$

Itt $m > 1$ a mintaszám, T_i és $\overline{T}$ nanomásodpercben értendő; s^2 egysége ns², s egysége ns. A nagy szórás instabil mérési környezetre vagy változó végrehajtási költségre utalhat. A minták nem feltétlenül függetlenek, ezért ez egyszerű leíró statisztika, nem automatikus megbízhatósági bizonyíték. A minimum sem tekintendő a rutin „igazi” hardveridejének.

Érdemes a teljes folyamatot is többször elindítani:

```
# Target: qemu-user / AArch64 Linux.
# Három külön folyamat; mindegyik várt kilépési állapota 0.
for run in 1 2 3; do
    qemu-aarch64 ./lab-time || break
done
```

QEMU user-mode alatt az A64 utasításokat a gazdagépen futó emulátor hajtja végre, jellemzően dinamikus bináris fordítással. A mért időre hat a host processzora, terhelése, ütemezése és a QEMU fordítási működése. A target memóriaelérések nem egy konkrét AArch64 processzor cache-hierarchiájának hardverhű modelljén futnak.

Ezért a QEMU-eredmény:

- alkalmas a mérési program és az eredményellenőrzés gyakorlására;
- azonos környezetben adhat relatív, illusztratív összehasonlítást;
- nem szolgáltat garantált, hardverhű abszolút késleltetést vagy ciklusszámot;
- még az implementációk fizikai hardveren várható sebességi sorrendjét sem garantálja.

11.4.6 Tipikus hibák és ellenpróbák

A második futás gyorsabb. Hipotézis: első adatérintés, QEMU-fordítás vagy környezeti terhelés befolyásolta az elsőt. Gyűjtsük össze az összes mintát, majd egyetlen változtatással növeljük a mérésen kívüli bemelegítést. Ezután ismételjük meg a teljes futtatást.

A mérés kiírással együtt szokatlanul lassú. Ellenőrizzük, nem került-e formázott I/O a két időbélyeg közé. Helyezzük át a kiírást, mást ne változtassunk, majd mérjünk újra.

A futás túl rövid vagy az eredmény gyanús. Vizsgáljuk meg a generált kódot:

```
# Target: qemu-userre készült AArch64 objektumok statikus vizsgálata.
aarch64-linux-gnu-objdump -dr lab-time.o
aarch64-linux-gnu-objdump -dr lab-sum.o
```

Ellenőrizzük a rutin hívását, a körülötte lévő ismétlést és az időbélyegzés helyét. Az objektumban a külső hívások célját relokációs bejegyzések is azonosíthatják. GDB-vel ellenőrizhető a paraméter és az eredmény, de a megállításokat tartalmazó futás időadatait el kell dobni.

Nagy, megmagyarázhatatlan ingadozás. Gyűjtsünk több teljes futást, ellenőrizzük a host terhelését, majd növeljük csak az ismétlésszámot. Ha a relatív ingadozás nem csökken, ne fedjük el pusztán több tizedesjegy kiírásával.

11.4.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Miért hibás csak az időbélyegek `tv_nsec` mezőit kivonni? Miért nem target-ciklusszám a QEMU alatt kapott idő és egy feltételezett 2 GHz-es frekvencia szorzata?

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a programot ns/elem mutatóval. Add meg a nevezőt, és fogalmazd meg pontosan, milyen járulékos költségeket tartalmaz az eredmény.

3. (mérési/hibakeresési, középszint — cél: rutinépítés) A `REPEATS` értékét most a `-DREPEATS=...` fordítói kapcsoló adja, ezért nem kell szerkeszteni a forrást. Állítsd 1000-re, 2000-re, majd 4000-re; minden változathoz végezz három teljes futtatást. Őrizd meg az időmintákat és az ellenőrzőösszeget. Mit vizsgálnál, ha a teljes mérési idő nem közelítőleg arányosan változna?

```
# Target: qemu-user / AArch64 Linux; három ismétlésszám, változatként három futás.
for r in 1000 2000 4000; do
    aarch64-linux-gnu-gcc -std=c11 -O2 -g -Wall -Wextra \
        -march=armv8-a -fno-lto -DREPEATS="$r" \
        -c lab-time.c -o "lab-time-$r.o"
    aarch64-linux-gnu-gcc -static -fno-lto \
        "lab-time-$r.o" lab-sum.o -lm -o "lab-time-$r"
    for run in 1 2 3; do
        qemu-aarch64 "./lab-time-$r"
        printf 'REPEATS=%s run=%s exit=%d\n' "$r" "$run" "$?"
    done
done
```

11.4.8 Megoldások

1. megoldás — Segítség. Gondolj arra, mi történik `tv_nsec`-kel, ha a mérés átlép egy másodperchatárt — lehet-e a vég `tv_nsec`-je kisebb, mint a kezdeté, miközben az eltelt idő pozitív?

1. megoldás — Teljes megoldás. Másodperchatár átlépésekor a nanomásodpercmező kisebb értékre vált. A teljes különbséghez a másodpercmező különbsége is kell. A QEMU-idő a hoston végrehajtott emuláció eltelt ideje; nincs hozzá igazolt target-ciklus/futási másodperc megfeleltetés.

2. megoldás — Segítség. Egy mérési blokk `REPEATS` hívást végez, mindegyik `N` elemet dolgoz fel — hány elemfeldolgozás történt összesen egy blokkban?

2. megoldás — Teljes megoldás. A nevező `REPEATS * N`, de a számítást célszerű lebegőpontos típussal végezni: `mean / ((double)REPEATS * N)`. Az eredmény tartalmazza a hívások, a C-ciklus és az időbélyegzés járulékos költségét is. Nem egy izolált betöltés késleltetése.

3. megoldás — Segítség. Ha a fix (REPEATS-független) mérési rezsiköltség elhanyagolható, a teljes idő és a `REPEATS` között közel lineáris kapcsolatot vársz — mikor térne el ettől a minta?

3. megoldás — Teljes megoldás. A lektor mérésében az ellenőrzőösszegek rendre 8390656000, 16781312000 és 33562624000 voltak. Közel arányos időnövekedés és nagyjából állandó ns/hívás azt jelzi, hogy a fix mérési költség nem dominál. Eltéréskor először a szóródást és a host terhelését vizsgáljuk, majd az `objdump` segítségével ellenőrizzük a ciklust. A linearitás önmagában nem igazolja a hardverhű időzítést.

11.4.9 Rövid összefoglaló

A használható időméréshez pontosan meg kell határozni, mi kerül a két időbélyeg közé. A példában az adat-előkészítés, ellenőrzés és kiírás kívül marad, a függvényhívás és az ismétlőciklus költsége viszont beleszámít. A `CLOCK_MONOTONIC` eltelt időt szolgáltat; a nanomásodperces ábrázolás nem jelent ugyanekkora pontosságot. Több mintát és szóródási mutatót közlünk, nem egyetlen kedvező eredményt. QEMU alatt ezek emulációs időadatok, nem target-ciklusszámok. Erre a munkafolyamatra épít a lebegőpontos és NEON-fejezet, majd a teljesítmény-tudatos mérés.

- **Instrumentáció:** megfigyelést lehetővé tevő kódkiegészítés.
- **Mérési intervallum:** a két időbélyeg közötti végrehajtási szakasz.
- **Mintaszórás:** az időminták szóródását az eredeti mértékegységben jellemző mutató.

11.4.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Linux man-pages: clock_gettime — libc-felület, óratípusok, hibajelzés és időábrázolás. **Tartalmilag ellenőrizve**: a Linux man-pages projekt 6.19-es kiadása.
- QEMU User Mode Emulation — user-mode futtatási modell és használat. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát); a hivatalos szöveg nem állít semmit a cache-hierarchia vagy a memóriasávszélesség modellezéséről, ezért a fejezet QEMU-korlátra vonatkozó állításai a TCG ismert, funkcionális (nem cikluspontos) tervezéséből következnek, nem egy explicit QEMU-dokumentált korlátból.
- GNU assembler kézikönyv — AArch64 szintaxis és assembler-direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv — a mérési kód működésének hibakeresési ellenőrzése. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok: a labor a lektor mérési környezetében (GCC 13.3.0, QEMU 8.2.2) lefutott, az itt közölt ellenőrzőösszegek és kimeneti forma onnan származnak; a konkrét időértékek hostfüggők, ezért nem szerepelnek. A helyi Ubuntu-csomagverziókat, a statikus cross könyvtárak rendelkezésre állását és a `clock_gettime` konkrét QEMU/libc végrehajtási útját a laborban kell ellenőrizni.

12 Lebegőpontos aritmetika és NEON/SIMD vektorizáció

12.1 A lebegőpontos regiszterkészlet és az IEEE 754 ábrázolás

12.1.1 Motiváció és hatókör

A Linux syscall-konvenció tárgyalásakor a rendszerhívás argumentumai egész regiszterekben utaztak, és a visszatérési érték is egész szám volt. Amint azonban a C program `float` vagy `double` változóval számol, olyan gépi szintű reprezentációhoz érkezünk, amely nem azonos a kettes komplemens egészekkel, és nem is az `Xn` regisztereken keresztül közlekedik. Ha egy mérnöki feladatban a hallgató egy szenzor kalibrációs értékét `0.1f`-ként tárolja, majd azt tapasztalja, hogy tíz egymás utáni összeadás után az eredmény nem pontosan `1.0`, akkor a hibát nem a kód logikájában, hanem a lebegőpontos **ábrázolás** korlátjában kell keresni. Ez az alfejezet pontosan ezt a réteget nyitja fel: az IEEE 754 bitmezőket, az AArch64 lebegőpontos regiszterkészletét és a kettő kapcsolatát.

A hatókör a következő: bitmező-szintű levezetés egy konkrét számon, a `V0`–`V31` regiszterek `Sn`/`Dn` skaláris nézete, valamint a kerekítés gyakorlati következménye. Az A64 lebegőpontos **utasítások** (összeadás, szorzás, konverzió) a Skaláris lebegőpontos utasítások alfejezetben következnek; a vektoros adatelrendezés pedig a NEON regiszterkészlet anyagához tartozik. Itt kizárólag az adatreprezentációt és a regiszterhozzáférést építjük fel.

12.1.2 Definíció: az IEEE 754 bináris ábrázolás

Az IEEE 754-2019 szabvány bináris lebegőpontos formátuma egy számot három bitmezőre bont: **előjel** (sign), **exponens** (exponent) és **mantissza** (mantissa, más néven significand tört része). Az érték normált esetben:

$$x = (-1)^s \times 1.m_2 \times 2^{e-b}$$

ahol s az előjelbit, m_2 a mantissza tört bitjei kettes számrendszerben, e a tárolt exponensmező egész értéke, b pedig az **eltolás** (bias). A tárolt exponensbe azért kerül eltolás, hogy ne kelljen külön előjelbitet fenntartani az exponensnek: a negatív és pozitív kitevőket egy eltolt, előjel nélküli egészként tároljuk.

Az egyszeres pontosság (single precision, `float`) és a kétszeres pontosság (double precision, `double`) bitmezői:

Formátum	Előjel	Exponens	Mantissza	Összesen	Bias
single	1 bit	8 bit	23 bit	32 bit	127
double	1 bit	11 bit	52 bit	64 bit	1023

Két konvenciót érdemes rögzíteni. Először: a mantissza a **normalizált** alakban mindig `1.xxx` formájú, ezért a vezető `1` bitet nem tároljuk — ez az úgynevezett *implicit* vagy *rejtett* bit. Másodszor: az eltolás miatt a tárolt exponensmező `e = kitevő + bias`.

12.1.3 Levezetés: a -12.375 kódolása single precision szerint

Válasszunk egy olyan számot, amely pontosan ábrázolható, így a bitmezők tisztán látszanak. A gondolatmenet lépései, feltételekkel:

1. Előjel. A szám negatív, ezért $s = 1$.

2. Abszolút érték bináris alakja. Az egész rész: $12 = 8 + 4 = 1100_2$. A tört rész 0.375 kettővel szorozva, a tört részt visszatartva:

$$0.375 \times 2 = 0.75 \Rightarrow 0, \quad 0.75 \times 2 = 1.5 \Rightarrow 1, \quad 0.5 \times 2 = 1.0 \Rightarrow 1$$

A maradék nulla, tehát $0.375 = 0.011_2$. Együtt: $12.375 = 1100.011_2$.

3. Normalizálás. A kettes pontot három hellyel balra visszük, hogy a vezető bit `1` legyen:

$$1100.011_2 = 1.100011_2 \times 2^3$$

A kitevő tehát 3. Ez az a pont, ahol a legtöbb hiba keletkezik: a kitevőt nem a *bináris pont pozíciójából*, hanem a *normalizált* alakból kell leolvasni.

4. Eltolt exponensmező. $e = 3 + 127 = 130$, ami binárisan $1000\,0010_2$.

5. Mantissza. Az implicit vezető `1`-et elhagyjuk, a maradék `100011`-et balra igazítjuk és 23 bitre nullákkal feltöltjük:

$$m = 1000110\,0000\,0000\,0000\,0000_2$$

6. Összeállítás. A 32 bit: `1 | 10000010 | 10001100000000000000000`.

Nibblesként csoportosítva:

Bitek	1100	0001	0100	0110	0000	0000	0000	0000
Hexa	C	1	4	6	0	0	0	0

Tehát a `-12.375f` bitmintája `0xC1460000`. Ellenőrzésképpen a pozitív `12.375f` ennek előjelbit nélküli változata, `0x41460000`.

12.1.4 Az AArch64 lebegőpontos regiszterkészlet

Az A64 ISA harminckét 128 bites regisztert biztosít a lebegőpontos és SIMD számításokhoz: `V0-V31`. Ezek **architekturális regiszterek**, amelyekhez többféle szélességű **nézet** (view) tartozik; ugyanannak az architekturális regiszternek a nézetei (`Bn`, `Hn`, `Sn`, `Dn`, `Qn`, `Vn`) átfedik egymást. Az, hogy ezek mögött hány és milyen fizikai tároló van, mikroarchitektúra-függő (átnevezés, regiszterfájl), és az ISA nem rögzíti:

Nézet	Szélesség	Tipikus használat
`Vn`	128 bit	teljes vektor, NEON
`Qn`	128 bit	NEON 128 bites nézet
`Dn`	64 bit	`double` skalár, NEON 64 bites
`Sn`	32 bit	`float` skalár
`Hn`	16 bit	half precision
`Bn`	8 bit	bájt

A skaláris nézetek **átfedésben** vannak: $S_n = V_n[31{:}0]$, $D_n = V_n[63{:}0]$. Ha egy `float`-ot az `S0`-ba írunk, az pontosan `V0` alsó 32 bitjén jelenik meg. Két fontos szerződés:

- Az A64 skaláris írás (`Sn`, `Dn`, `Hn`, `Bn` célregiszterként) **nullázza** a regiszter íráson felüli felső bitjeit. Az `S0` írása tehát `V0` felső 96 bitjét nullázza; `D0` írása a felső 64 bitet. Ez determinisztikus, nem „megmaradó szemét”.
- A lebegőpontos regiszterek és az általános célú `Xn`/`Wn` regiszterek **külön regisztertömböt** alkotnak. Nincs olyan aritmetikai utasítás, amely `W0`-t és `S0`-t közvetlenül összeadná; a **bitmintát** `fmov` utasítás viszi át a két tér között. Értékkonverzióhoz (`int` → `float`) viszont nem `fmov`, hanem `scvtf`/`ucvtf` kell (lásd 12.2).

A lebegőpontos viselkedést két rendszerregiszter szabályozza. Az `FPCR` (Floating-point Control Register) tartalmazza a kerekítési módot; az alapértelmezés a *round to nearest, ties to even* (a legközelebbi értékre, holtversenynél a páros mantisszára). Az `FPSR` (Floating-point Status Register) pedig a kumulatív kivételjelzőket (invalid, division by zero, overflow, underflow, inexact) gyűjti. A `V0`–`V31` regiszterkészlet általános áttekintése a regiszterkészlet alfejezetben szerepelt.

A Linux alatt futó alkalmazásszintű kód ezeket a regisztereket közvetlenül használhatja, mert a kernel az `FPEN`/`CPACR` beállítással engedélyezi a lebegőpontos végrehajtást a felhasználói folyamat számára. QEMU user-mode emulációban (`qemu-aarch64`) ez a viselkedés alapértelmezett.

12.1.5 Kidolgozott példa: bitminta előállítása és visszaolvasása

Munkakönyvtár: `~/arm-labor/a64/a044/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a044
cd ~/arm-labor/a64/a044
```

Az alábbi assembly program a `0xC1460000` bitmintát tölti `W0`-ba, átviszi `S0`-ba, majd visszaolvassa és kiírja hexadecimálisan. Célja a regiszter-átjárás és a reprezentáció azonosságának igazolása.

```
// lab-fp-bits.s -- target: qemu-aarch64 (user-mode), A64 ISA
        .section .rodata
hexfmt: .asciz "%08x\n"

        .text
        .global main
main:
        .cfi_startproc
        stp x29, x30, [sp, #-16]!
        .cfi_def_cfa_offset 16
        .cfi_offset 29, -16
        .cfi_offset 30, -8
        mov x29, sp
        // 1) bitminta betöltése: a felső 16 bit = 0xC146, az alsó 16 bit 0, ezért elég egy MOVZ
```

```
    movz    w0, #0xC146, lsl #16    // w0 = 0xC1460000

    // 2) ugyanaz a 32 bit átkerül az FP regiszterterbe
    fmov    s0, w0                  // s0 = -12.375 (single)

    // 3) visszaolvasás: ugyanazt a bitmintát kell latnunk
    fmov    w1, s0

    // 4) kiírás: printf(hexfmt, w1)
    adrp    x0, hexfmt
    add     x0, x0, :lo12:hexfmt
    bl      printf

    mov     w0, #0                  // visszatérési ertek
    ldp x29, x30, [sp], #16
    .cfi_restore 29
    .cfi_restore 30
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc

    .section .note.GNU-stack,"",%progbits
```

A `.cfi_*` sorok a keretet írják le a hibakereső számára (lásd a Prológus és epilógus CFI-szabályát); a `main` a C-futtatókörnyezetből hívott függvény, ezért ugyanúgy kap CFI-t, mint bármely más C-ből hívható assembly függvény.

Fordítás és futtatás:

```
aarch64-linux-gnu-gcc -static -o lab-fp-bits lab-fp-bits.s
# elvárt: GNU_STACK RW
readelf -lW lab-fp-bits | awk '$1=="GNU_STACK"{f=$7} END{print "GNU_STACK", f; exit !(f=="RW")}'
qemu-aarch64 ./lab-fp-bits
```

A forrás végi `.note.GNU-stack` szekció jelzi a linkernek, hogy az objektum nem igényel végrehajtható vermet (Az első assembly program). Ennek hatását nem a forrás szövegében, hanem a **kész ELF**-ben ellenőrizzük. A `readelf` sor a `GNU_STACK` programfejléc jelzőit írja ki, és nem nulla állapottal tér vissza, ha azok nem pontosan `RW` (például ha `RWE`, vagy ha a fejléc hiányzik). A könyv C-ből hívható vagy önálló Linux/ELF assembly forrásai mind ezzel a direktívával zárulnak, és a FP/NEON-fejezetek buildjei ugyanezzel a sorral ellenőrzik az eredményt. A linker `missing .note.GNU-stack section implies executable stack` figyelmeztetése ezért nem kozmetikai: a build ellenőrzendő eredményére hívja fel a figyelmet. A viselkedés azonban cél- és verziófüggő: a lektor mérése szerint AArch64-en, binutils 2.42-vel jelzés nélküli objektumnál a linker nem figyelmeztet, és a kimenetben egyáltalán nincs `GNU_STACK` programfejléc — ezt a fenti `awk`-ellenőrzés üres értékkel, hibaként jelzi. `RWE` vermet csak a `-z execstack` állít elő; a figyelmeztetés jellemzően x86-64-en és újabb binutils-verziókkal jelenik meg.

Mérve (a lektor mérése, GCC 13.3.0, binutils 2.42, QEMU 8.2.2): `c1460000`. Az azonosság azt mutatja, hogy az `S0` pontosan ugyanazt a 32 bitet tartalmazza, amit a `W0`-ba betöltöttünk — a lebegőpontos regiszter nem „értelmezi át” az adatot, csak tárolja. Az értelmezést az adja, hogy melyik utasítás olvassa.

Ugyanez C-ből, unióval, ami a bájt-szintű reprezentációt teszi láthatóvá:

```
// lab-fp-dump.c -- target: qemu-aarch64 (user-mode)
#include <stdio.h>
#include <stdint.h>

int main(void) {
    union { float f; uint32_t u; } v;
```

```
    v.f = -12.375f;
    printf("%08x\n", v.u);      /* mért: c1460000 */

    v.f = 0.1f;
    printf("%08x\n", v.u);      /* mért: 3dcccccd */
    return 0;
}
```

Fordítás és futtatás:

```
# Target: qemu-user / AArch64 Linux, little-endian.
aarch64-linux-gnu-gcc -std=c17 -O2 -static \
    -o lab-fp-dump lab-fp-dump.c
qemu-aarch64 ./lab-fp-dump
```

A lektor mérésében a program a `c1460000`, majd a `3dcccccd` sort írta ki, és `0` állapottal lépett ki.

12.1.6 Speciális értékek

Az exponensmező szélső értékei nem számokat, hanem kijelölt állapotokat jelölnek (single precision; double esetén a normált exponensmező 1..2046, a denormálisok skálája 2^{-1022}, a legkisebb pozitív denormális pedig 2^{-1074}):

Exponensmező	Mantissza	Jelentés
`0`	`0`	± 0 (előjeles nulla) — **szám** (érték)
`0`	$\neq 0$	denormális (subnormal) — **szám** (érték), legkisebb pozitív: 2^{-149} (`0x00000001`)
`1..254`	bármely	normált szám — **szám**
`255`	`0`	$\pm\infty$ — nem véges érték, de a rendezett számegyenes része (a `0 < ∞`-összehasonlítás értelmes)
`255`	$\neq 0$	NaN — **nem-szám** állapot (rendezetlen összehasonlítás)

A denormális tehát *szám*, nem nem-szám. A végtelen viselkedése a feltételezett `FPCR`-környezetre vonatkozik: alapértelmezett kerekítés (RNE), `FZ = 0` (a denormálisok nem lapulnak nullára), `DN = 0`, a kivételek elnyomva (nincs trap).

A **denormális** számoknál az implicit vezető bit `0`-ra vált, és az érték $x = (-1)^s \times 0.m_2 \times 2^{-126}$. Ez a fokozatos alulcsordulást (gradual underflow) biztosítja: a legkisebb denormális single érték 2^{-149}. A **NaN** (Not a Number) akkor keletkezik, ha például $\sqrt{-1}$-et vagy $0/0$-t számolunk; a mantissza legfelső bitje különbözteti meg a *quiet* és *signaling* NaN-t. Az ∞ jön létre túlcsordulásnál vagy nullával osztásnál.

Gyakorlati szempontból a legfontosabb: a `0` és a `-0` **egyenlőnek** számít összehasonlításkor ($+0 = -0$), a NaN viszont **nem egyenlő önmagával sem**, ami a `==` operátorral írt feltételeknél meglepetést okozhat. Az egyenlőség ellenére a két nulla nem felcserélhető: a C `signbit(-0.0f)` igaz, a `copysign(1.0f, -0.0f)` eredménye `-1.0f`, az `1.0f / -0.0f` pedig `-Inf`. Az `FCMP`/`FCMEQ` utasítások viselkedését a Feltételes végrehajtás mintázataihoz hasonlóan, de külön szabályok szerint kell olvasni (Skaláris lebegőpontos utasítások).

12.1.7 Kerekítés és pontosságvesztés

A `0.1f` nem ábrázolható pontosan. A bináris tört végtelen szakaszos: $0.1 = 1.1001100110011\ldots_2 \times 2^{-4}$, ahol a „1001” végtelenségig ismétlődik. A tárolt exponensmező: $e = -4 + 127 = 123$, azaz `01111011`. A 23 mantisszabiten a végtelen sorozatot le kell vágni; a kerekítési szabály (round to nearest, ties to even) a 24. bit alapján felfelé kerekít.

12.1.7.1 A kerekítési szabály részletesen

A megtartott 24 bit utáni maradékot *guard* bit, *round* és *sticky* bit — az összes további bit VAGY-ja — írja le: `guard = 1` és a `sticky/round` valamelyike 1 → félút fölött, felfelé; `guard = 1`, minden más 0 → pontos félút, ekkor a páros utolsó bitre kerekítünk; `guard = 0` → lefelé). Három pontos példa egész számokkal (2^{24} fölött a binary32 lépése 2): 16 777 217 pontos félút, a megtartott utolsó bit páros → lefelé, 16 777 216; 16 777 219 pontos félút, a megtartott utolsó bit páratlan → felfelé, 16 777 220; 33 554 435 (lépés 4, félút fölött) → felfelé, 33 554 436.

Visszatérve a `0.1`-hez: a 24. bit alapján a kerekítés felfelé történik, így a mantissza `10011001100110011001101`. Az eredmény:

`0 | 01111011 | 10011001100110011001101` = `0x3DCCCCCD`.

Ez a tárolt érték nem `0.1`, hanem `0.100000001490116119384765625`. Az **abszolút hiba** $\approx 1.49 \times 10^{-9}$, a **relatív hiba** $\approx 1.49 \times 10^{-8}$, ami 0,1 környékén kb. 0,2 **ULP** (egy ULP = $2^{-27} \approx 7{,}45 \times 10^{-9}$, mert $0{,}1 \in [2^{-4}, 2^{-3})$). Az abszolút hibát nem szabad a gépi epsilonhoz ($2^{-23} \approx 1{,}19 \times 10^{-7}$, az 1,0 körüli ULP) mint univerzális küszöbhöz mérni: az ULP a kitevővel skálázódik. Ugyanaz a fél-ULP-os hiba 0,1 környékén $\approx 3{,}7 \times 10^{-9}$ abszolút, 10^6 környékén (ULP = $2^{-4} = 0{,}0625$) viszont $\approx 0{,}031$ abszolút hibát jelent. A későbbi toleranciákat ezért a skálához (relatív hiba vagy ULP) kell kötni. A hibák a műveletek során **halmozódhatnak**, de nem szükségszerűen. Tanulságos példa: single precisionben a `0.1f + 0.2f` eredménye bitre azonos a `0.3f`-fel (mindkettő `0x3E99999A`), mert a két tárolási hiba és az összeadás kerekítése éppen ugyanarra a legközelebbi `float`-ra vezet. Double pontosságban viszont a `0.1 + 0.2` eredménye `0x3FD3333333333334`, a `0.3` pedig `0x3FD3333333333333`: a kettő egy egységnyi utolsó helyiértékkel (ULP) eltér. A „lebegőpontos számítás mindig pontatlan” leegyszerűsítés tehát éppúgy hibás, mint a „mindig pontos”: hogy két érték egyezik-e, azt a konkrét bitminták döntik el. Ezért `==` helyett általában toleranciás összehasonlítás kell, de a toleranciát a feladathoz kell választani (lásd a 3. feladat megoldását).

A **double** pontosság esetén a 52 mantisszabit miatt a hiba sok nagyságrenddel kisebb, de eltűnni nem fog. A `0.1` double alakja `0x3FB999999999999A`, hibatartalma $\approx 5.55 \times 10^{-18}$. A pontosságvesztés mértéke a művelet jellegétől is függ: az azonos nagyságrendű számok kivonásánál (**katasztrofális kancelláció**) a relatív hiba hirtelen megnő, mert a vezető értékes bitek kiesnek. Ez az oka annak, hogy numerikus algoritmusokban a kivonásos lépéseket igyekeznek átalakítani stabil formára.

12.1.8 Tipikus hibák és ellenpróbák

Hiba 1: az exponens eltolásáról megfeledkezés. A `-12.375` exponensmezőjébe `3` kerül a helyes `130` helyett. *Tünet:* a bitminta `0x81C60000`, az érték nagyságrendekkel téves. *Hipotézis:* az eltolás kimaradt. *Adatgyűjtés:* másoljuk ki a 32 bitet, és maszkolással/eltolással fejtsük vissza a mezőket. *Egy változtatás:* adjunk hozzá 127-et, ellenőrizzük újra.

Kódolás	Előjel	Tárolt exponens	Fraction (23 bit)	Hex	Visszafejtett érték
helytelen (bias nélkül)	1	00000011 (3)	10001100000000000000000	0x81800000	$-1{,}546875 \times 2^{3-127}$ (nagyon kicsi, normál szám)
helyes	1	10000010 (130)	10001100000000000000000	0xC1460000	$-1{,}546875 \times 2^3 = -12{,}375$

A mezők kinyerése: `(x >> 31)`, `(x >> 23) & 0xFF`, `x & 0x7FFFFF`. Ellenőrzés Pythonban: `struct.unpack('>I', struct.pack('>f', -12.375))[0] == 0xC1460000` (pontos bitminta-összevetés, nem tizedes közelítés).

Hiba 2: 32 bites bitminta betöltése egyetlen MOV-val. A `mov w0, #0x12345678` nem assemblálható le, mert a `MOV` immediate-kódolása korlátozott (egyetlen 16 bites mező, eltolással, vagy `MOVN`/logikai bitminta-immediate). *Megjegyzés:* a `0xC1460000` éppen kódolható egyetlen `MOVZ`-zal (`mov w0, #0xC1460000` = `movz w0, #0xC146, lsl #16`), ezért a példában ezt használjuk. Valóban nem egy utasítással képezhető például a `0x12345678`. *Tünet:* `mov w0, #0x12345678` esetén a GNU as 2.42 üzenete mérve `Error: immediate cannot be moved by a single instruction`. *Megoldás:* `movz w0, #0x5678` + `movk w0, #0x1234, lsl #16`. Ezt az immediate-korlátot a MOV-variánsok tárgyalja részletesen.

Hiba 3: a `Sn` és `Dn` nézet téves használata. Ha egy double értéket `Sn`-ként olvasunk vissza, csak az alsó 32 bitet kapjuk meg (a mantissza egy részét), az eredmény szemét. *Ellenpróba:* mindig a C típusához illeszkedő skaláris nézetet használjuk (`float` → `Sn`, `double` → `Dn`), és a `printf` formátumát is egyeztessük (`%f` a `float`/`double` esetén: a `printf` változó számú argumentumában a `float` a C nyelv alapértelmezett argumentum-előléptetési szabálya szerint `double`-ként adódik át — ez nyelvi szabály, nem platformkonvenció).

12.1.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Bontsd fel kézzel a `0xC1200000` bitmintát! Add meg az előjelet, a nyers exponensmezőt, a kitevőt, a mantisszát és a decimális értéket!

2. (implementációs, középszint — cél: rutinépítés) Készíts assembly rutint, amely egy `S0`-ban tárolt `float` előjel-, exponens- és mantisszamezőjét `W0`-ba, `W1`-be, `W2`-be bontja! Az előjelet `LSR`-rel a 31. bitről, az exponenst `LSR` és maszkolás útján, a mantisszát 23 bites maszkkal nyerd ki.

3. (hibakeresési/mérési, középszint — cél: megértés-ellenőrzés) Írj C programot, amely kiírja `0.1f`, `0.2f`, `0.3f` és `0.1f + 0.2f` bitmintáját, majd ugyanezt `double` típussal is (`0.1`, `0.2`, `0.3`, `0.1 + 0.2`)! Melyik pontosságban egyezik az összeg a `0.3` tárolt értékével, és miért? Írj toleranciás összehasonlítást, és add meg, milyen bemenetekre érvényes!

12.1.10 Megoldások

1. megoldás — Segítség. Csoportosítsd a 32 bitet 1+8+23 szerint, az exponensmezőből vond ki a biast (127), majd a mantisszát implicit vezető `1`-gyel olvasd.

1. megoldás — Teljes megoldás. A `0xC1200000` binárisan `1100 0001 0010 0000 ....` Előjel: `1` → negatív. Exponensmező: `10000010` = 130, tehát kitevő = 130 − 127 = 3. Mantissza: `0100000...` → implicit `1`-gyel $1.01_2 = 1{,}25$. Érték: $-1{,}25 \times 2^3 = -10$. Ellenőrzés: a `-10.0f` kanonikus bitmintája valóban `0xC1200000`.

2. megoldás — Segítség. Előbb vidd át a bitmintát `fmov`-val egy általános célú regiszterbe — onnantól a feladat ugyanaz a bitmező-kinyerés, mint a fejezet IEEE 754 levezetésében.

2. megoldás — Teljes megoldás. A teljes megoldás két fájlból áll. A `lab-fp-fields.s` a C-ből hívható függvény; a három mezőt három kimeneti mutatón keresztül adja vissza, mert a `W0`–`W2` egyszerre nem lehet egy C-függvény visszatérési értéke:

```
// lab-fp-fields.s -- target: qemu-aarch64 (user-mode), A64 ISA
// void lab_fp_fields(float x, unsigned *sign, unsigned *exp, unsigned *frac)
// x: S0; sign: X0; exp: X1; frac: X2
        .text
        .p2align 2
        .global lab_fp_fields
        .type lab_fp_fields, %function
lab_fp_fields:
        .cfi_startproc
        fmov    w3, s0
        lsr     w4, w3, #31             // elojel: 0 vagy 1
        str     w4, [x0]
        ubfx    w4, w3, #23, #8         // exponensmezo (8 bit a 23. bitrol)
        str     w4, [x1]
        and     w4, w3, #0x007FFFFF     // mantissza (also 23 bit)
        str     w4, [x2]
        ret
        .cfi_endproc
        .size lab_fp_fields, .-lab_fp_fields

        .section .note.GNU-stack,"",%progbits
```

A `lab-fp-fields.c` C-driver:

```
#include <stdio.h>

extern void lab_fp_fields(float x, unsigned *sign,
                          unsigned *exp, unsigned *frac);

int main(void)
{
    unsigned sign = 0, exp = 0, frac = 0;

    lab_fp_fields(-12.375f, &sign, &exp, &frac);
    printf("sign=%u exp=%u frac=0x%06x\n", sign, exp, frac);
    return (sign == 1 && exp == 130 && frac == 0x460000) ? 0 : 1;
}
```

Fordítás és futtatás:

```
aarch64-linux-gnu-gcc -static \
    -o lab-fp-fields lab-fp-fields.s lab-fp-fields.c
qemu-aarch64 ./lab-fp-fields
```

A `-12.375f` bemenetre a bitmező-levezetésből adódó kimenet `sign=1 exp=130 frac=0x460000`, 0 kilépési állapottal.

A `UBFX` (Unsigned Bit Field Extract) utasítás pontosan két pozíció és szélesség közötti bitmezőt emel ki — ez a bitfield-utasítások körébe tartozik. A `#0x007FFFFF` maszk az alsó 23 bitet tartja meg.

3. megoldás — Segítség. Ne feltételezd előre, hogy az összeg eltér: nézd meg a bitmintákat. A `0.1f` és a `0.2f` tároláskor kerekített érték, de a hibák előjele és nagysága dönti el, hová kerekít az összeadás.

3. megoldás — Teljes megoldás. Single precisionben a `0.1f`, `0.2f`, `0.3f` bitmintái rendre `0x3DCCCCCD`, `0x3E4CCCCD`, `0x3E99999A`, és a `0.1f + 0.2f` eredménye (alapértelmezett, round-to-nearest-ties-to-even kerekítéssel) szintén `0x3E99999A`: az összeg **egyezik** a `0.3f`-fel. Double pontosságban a `0.1 + 0.2` eredménye `0x3FD3333333333334`, a `0.3` értéke `0x3FD3333333333333`: itt egy ULP az eltérés, ezért a `==` hamis. A tanulság: a pontos egyezés vagy eltérés a konkrét bitmintákon múlik, nem általános szabályon.

Toleranciás összehasonlítás **véges** értékekre, **pozitív** abszolút toleranciával:

```c
#include <math.h>

/* Előfeltétel: a és b véges (nem NaN, nem ±Inf), eps > 0. */
int feq_abs(float a, float b, float eps) { return fabsf(a - b) <= eps; }
```

Ez csak akkor helyes, ha az értékek nagyságrendje ismert, és ehhez választjuk az `eps`-et. Külön esetek:

- **NaN:** a `fabsf(a - b) <= eps` NaN-ra mindig hamis, ezért két NaN-t ez a függvény sosem tekint egyenlőnek. Ha ez nem a kívánt viselkedés, a NaN-t előbb `isnan`-nal külön kell kezelni.
- **±Inf:** `Inf - Inf` NaN, ezért két egyező végtelen sem „egyenlő” a fenti függvény szerint; az `isinf` vizsgálata és az előjel összevetése külön ág.
- **Relatív tolerancia:** nagyon különböző nagyságrendű értékeknél az abszolút `eps` vagy túl szigorú, vagy túl laza. Ilyenkor a különbséget a nagyobb abszolút értékhez viszonyítjuk: `fabsf(a - b) <= rel * fmaxf(fabsf(a), fabsf(b))`, ahol `rel > 0`, és a nullához közeli értékekre ezt egy abszolút alsó küszöbbel egészítjük ki.

12.1.11 Rövid összefoglaló

Az IEEE 754 single precision formátum egy előjelbitből, egy 8 bites, 127-tel eltolt exponensből és egy 23 bites, implicit vezető bites mantisszából áll; a double megfelelője 11 és 52 bites. A `-12.375` levezetése megmutatta, hogy a bitmezőket a **normalizált** alakból kell számolni, és az eredmény `0xC1460000`. Az AArch64 `V0-V31` regiszterei 128 bitesek, skaláris nézeteik (`Sn`, `Dn`) ezek alsó szeletei, és a skaláris írás nullázza a felső biteket. A két regisztertér között `fmov` viszi át a bitmintát; értékkonverzióhoz (`int` → `float`) `scvtf`/`ucvtf` kell.

- **Eltolás (bias):** az exponensmezőbe írt érték és a valódi kitevő különbsége (single: 127).
- **Implicit vezető bit:** a normalizált mantissza nem tárolt `1`-ese.
- **Denormális:** nulla exponensmező melletti nem nulla mantissza; kis számok fokozatos ábrázolása.

12.1.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- IEEE 754-2019, Standard for Floating-Point Arithmetic — a bitmezők, a kerekítési módok és a speciális értékek normatív meghatározása. A link feltehetően él, de tartalmilag itt nem lett ellenőrizve (a szabvány fizetős, csak a tartalomjegyzék érhető el nyilvánosan).

- Arm Architecture Reference Manual for A-profile architecture (Arm DDI0487) — a `V0-V31`, `Sn`/`Dn` regiszterek és a skaláris írás nullázó viselkedésének specifikációja. Ellenőrizve: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU Compiler Collection – AArch64 Options — a `float`/`double` ABI-kezelése és a lebegőpontos modell kapcsolói. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2.
- QEMU User Mode Emulation — a `qemu-aarch64` futtatási modellje és korlátai. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok: a `qemu-aarch64` user-mode lebegőpontos viselkedése hardveresen nem garantált, ciklusszintű következtetésre nem alkalmas. Az IEEE 754 speciális értékeinek pontos `FPSR`-jelzőkezelése (`invalid`, `inexact`) utasítás-specifikus, ezt az utasítások alfejezet tárgyalja. A `printf` változó argumentumaiban a `float` → `double` előléptetés C-nyelvi szabály (alapértelmezett argumentum-előléptetés), nem az ISA része és nem pusztán platformkonvenció.

A fejezet számszerű értékeit a lektor a fő úton (GCC 13.3.0, binutils 2.42, QEMU 8.2.2) mérte: a `lab-fp-bits` a `GNU_STACK RW` fejlécet és `c1460000` kimenetet adta, a `lab-fp-dump` kimenete `c1460000`, majd `3dcccccd` volt. Ugyanígy mérve `0.1f + 0.2f` = `0x3E99999A` = `0.3f`; double-ben `0.1 + 0.2` = `0x3FD3333333333334`, a `0.3` = `0x3FD3333333333333`; `-10.0f` = `0xC1200000`; a kerekítési példák pedig 16777217 → 16777216, 16777219 → 16777220 és 33554435 → 33554436.

12.2 Skaláris lebegőpontos utasítások

12.2.1 Motiváció: ahol az egész aritmetika szerződése megszűnik

A lebegőpontos regiszterkészlet és az IEEE 754 ábrázolás alfejezetben egy konkrét szám bitmezőin keresztül láttuk, hogy a `float` és a `double` nem kettes komplemens egész, és nem az `Xn` regisztereken utazik. Az ott felépített reprezentáció azonban önmagában passzív: ahhoz, hogy egy mérnöki feladatot megoldjunk, **utasítások** kellenek, amelyek ezeken a bitmintákon aritmetikát végeznek.

Vegyük a tipikus kalibrációs láncot: az ADC egy nyers egész értéket ad, abból `(x - offset) * scale` alakú lineáris transzformációval lesz fizikai mennyiség, majd statisztikához (átlag, szórás) lebegőpontoson számolunk, végül a megjelenítéshez vagy egy protokoll-mezőhöz egészre kell visszakonvertálni. Ez a lánc **két határátlépést** tartalmaz az egész és a lebegőpontos világ között, és ezek a határátlépések az assembly szintjén nem automatikusak: külön utasítás kell rájuk. Ez az alfejezet pontosan ezt a réteget tanítja: a skaláris lebegőpontos aritmetikát, a bit szintű mozgatást és a konverziókat.

A hatókör szűk: `FADD`, `FSUB`, `FMUL`, `FDIV`, `FMOV`, `SCVTF`/`UCVTF`, `FCVTZS`/`FCVTZU`, valamint az `FCMP` és az abból származó feltételkódok. A vektoros (NEON) párhuzamosítás a NEON regiszterkészlet és adatelrendezés és a Vektorizált összeadás/szorzás: egy skaláris ciklus NEON-átirata témája; a feltételkódok általános mechanikáját pedig a Feltételes végrehajtás: CSET/CSEL és feltételkódok alfejezetben már felépítettük — itt csak azt nézzük meg, hogyan viselkednek lebegőpontos összehasonlítás után.

12.2.2 Definíció: mitől „skaláris”, és hogyan kódolja a mnemonic a pontosságot

Skaláris lebegőpontos utasítás (scalar floating-point instruction): olyan A64 ISA utasítás, amely egyetlen lebegőpontos értéken (egy `Sn` vagy `Dn` regiszteren) hajt végre műveletet, szemben a vektorossal, amely egyszerre több elemet dolgoz fel egy `Vn` regiszterben.

Az AArch64 sajátossága, hogy a **mnemonic nem tartalmaz pontosság-suffixet**. Nincs `fadd.f32`; a pontosságot a regiszternév kódolja:

Regiszternév	Nézet	Pontosság
`Sn` (pl. `S0`)	`Vn` alsó 32 bitje	binary32 (`float`)
`Dn` (pl. `D0`)	`Vn` alsó 64 bitje	binary64 (`double`)

Ebből két kemény szabály következik:

1. **Precízióegyezés.** Az operandusoknak és az eredménynek azonos pontosságúnak kell lenniük: `FADD Sd, Sn, Sm` és `FADD Dd, Dn, Dm` érvényes, a `FADD S0, S1, D2` viszont assemble-idejű hiba. Nincs implicit pontosságnövelés (widening).
2. **Nincs immediate operandus.** Az `FADD/FSUB/FMUL/FDIV` egyik formája sem fogad el azonnali konstanst. A konstans háromféleképpen kerülhet be: az `FMOV` korlátozott immediate-készletével, egy egész regiszterből `SCVTF/UCVTF`-fal, vagy memóriából (`LDR Sn, [Xn]`). Az `FMOV` immediate-kódolása 8 bites: a kódolható értékek $n \in [-3, 4]$ mellett $\pm(1 + m/16) \times 2^n$ alakúak ($m = 0..15$), így a nagyságok 0,125 és 31,0 közöttiek. A 0,0 **nem** kódolható FMOV-immediate-ként; nullázáshoz `fmov s0, wzr` vagy `movi d0, #0` kell. Ez az MOV-variánsok és immediate-kódolás korlátjának lebegőpontos megfelelője.

Az utasításcsoportok áttekintése:

Utasítás	Mit csinál	Regisztertípus
`FADD`, `FSUB`, `FMUL`, `FDIV`	aritmetika	`Sn/Dn`
`FMOV`	bit szintű másolás (nem konverzió!)	FP egész / FP FP
`SCVTF`, `UCVTF`	egész → lebegőpontos	`Wn/Xn` → `Sn/Dn`
`FCVTZS`, `FCVTZU`	lebegőpontos → egész, csonkítva	`Sn/Dn` → `Wn/Xn`
`FCVT`	`float` `double`	`Sn` `Dn`
`FCMP`, `FCMPE`	összehasonlítás, NZCV beállítás	`Sn/Dn`

12.2.3 Aritmetika: FADD, FSUB, FMUL, FDIV

Az alapműveletek szemantikája az IEEE 754-2019 binary32/binary64 szabvány szerint **helyesen kerekített** (correctly rounded): az eredmény a pontos matematikai eredmény kerekítése **az aktuális kerekítési mód szerint**. A módot az `FPCR` (Floating-point Control Register) `RMode` mezője választja ki: round-to-nearest-ties-to-even (RN, ez az alapérték), $+\infty$ felé (RP), $-\infty$ felé (RM) vagy nulla felé (RZ). „A legközelebbi ábrázolható szomszéd” csak RN módban igaz; a többi módban az eredmény a pontos érték megfelelő irányú szomszédja. Az `FPCR` további bitjei (például a flush-to-zero `FZ` és az alapértelmezett NaN `DN`) szintén eltéríthetik az eredményt az IEEE-alapviselkedéstől; Linux alatt a felhasználói programok alapértelmezésben IEEE-módban indulnak. A program `MRS Xt, FPCR` / `MSR FPCR, Xt` utasításokkal érheti el a regisztert.

Néhány, a C-fejlesztőt ténylegesen érintő tulajdonság:

- `FADD/FSUB/FMUL/FDIV` **nem tesz különbséget előjeles és előjel nélküli** között — a lebegőpontos ábrázolás előjel-nagyság (sign-magnitude) alakú, nincs kettes komplemens viselkedés. Ez éles ellentét az Egész aritmetika `MUL/DIV` világával.

- **Véges** `x` esetén `x + (-x)` eredménye RN, RP és RZ módban `+0.0`, RM módban viszont `-0.0`. Nem véges bemenetre a szabály nem érvényes: `Inf + (-Inf)` NaN (és IOC-t állít), NaN bemenetre NaN az eredmény. A nulla előjele nem csak osztásnál számít (`1.0 / -0.0 = -Inf`): a C `signbit` és `copysign` függvényei is megkülönböztetik a két nullát, és a `printf` is `-0`-t ír ki.
- A nem ábrázolható eredmény nem állítja meg a programot. A kivételek az `FPSR` (Floating-point Status Register) **halmozódó** bitjeibe íródnak (IOC – invalid operation, DZC – divide by zero, OFC – overflow, UFC – underflow, IXC – inexact). Alapértelmezésben nincs trap. Fontos: ezek a flagek **nem** az `NZCV`-ben vannak, tehát feltételes elágazással nem olvashatók ki; `MRS Xt, FPSR` kell hozzájuk.
- `FDIV` költsége lényegesen nagyobb, mint a többi műveleté. Az alábbi nagyságrendek irodalmi/illusztratív értékek, amelyeket **nem mértünk ebben a könyvben**, és mikroarchitektúránként változnak; a pontos latenciát az adott CPU technikai leírásából kell ellenőrizni.

Művelet	Illusztratív latencia-nagyságrend (ellenőrizendő)	Jelleg
`FADD`/`FSUB`	néhány ciklus	teljesen futószalagosítható
`FMUL`	néhány ciklus	teljesen futószalagosítható
`FDIV`	egy nagyságrenddel több	részben vagy nem futószalagosítható

Ez az eltérés a Teljesítmény-tudatos kód mérési szakaszában lesz fontos. A reciprokos helyettesítés (`x / d` → `x * (1/d)`) **nem azonos eredményű**: két kerekítést végez egy helyett, ezért a bitminta eltérhet (pl. nem 2-hatvány `d` esetén), és `d = 0`, `inf`, `NaN` esetén a kivételek is mások. Ezért a fordító csak `-ffast-math`/`-freciprocal-math` mellett végzi el automatikusan; kézzel csak akkor szabad, ha a feladat tűri az eltérést (toleranciás tesztelés), és a nyereség a célmagon mért, nem általános: egy reciprok előre számítása ciklusban sok osztásnál térül meg, egy osztásnál nem.

12.2.4 FMOV: bit szintű mozgatás, nem konverzió

Az `FMOV` a leggyakrabban félreértett utasítás ebben a csoportban. Az `FMOV` **nem konvertál**, hanem a biteket másolja, és a célregiszter ezeket úgy értelmezi, ahogyan az ő típusa előírja.

Forma	Hatás
`FMOV Wd, Sn`	`Sn` alsó 32 bitje → `Wd` (a bitminta egészként olvasva)
`FMOV Sd, Wn`	`Wn` → `Sd` alsó 32 bitje (a bitminta lebegőpontosként értelmezve)
`FMOV Xd, Dn`	`Dn` → `Xd`
`FMOV Dd, Xn`	`Xn` → `Dd`
`FMOV Sd, Sn` / `FMOV Dd, Dn`	FP → FP, bitazonos
`FMOV Sd, #imm`	korlátozott, 8 bites kódolású konstanskészlet

A gyakorlati haszna az irányított olvasás: `FMOV Wd, Sn`-nel hozzáférünk az IEEE 754 bitmezőkhöz (előjel, exponens, mantissza) egész műveletekkel — ez a Logikai alapműveletek és bitmaszkolás

eszköztárával kombinálva adja a bitmanipulációs trükköket. Fordított irányban `FMOV Sd, Wn` akkor hasznos, ha egy memóriából olvasott nyers 32 bites szót akarunk `float`-ként értelmezni (pl. protokoll-deszerializálás).

A tipikus hiba pontosan itt keletkezik: **`FMOV`-ot használni konverzióra.** Ha `W0 = 10`, akkor az `FMOV S0, W0` nem `10.0f`-et ad, hanem a `0x0000000A` bitmintát, ami egy rendkívül kicsi denormális szám (`1.4e-44`). Egész → lebegőpontos átalakításra az `SCVTF`/`UCVTF` való.

12.2.5 Konverzió: SCVTF/UCVTF és FCVTZS/FCVTZU

Utasítás	Irány	Forrás előjelkezelés
`SCVTF Sd, Wn` / `SCVTF Dd, Wn` / `SCVTF Sd, Xn` / `SCVTF Dd, Xn`	egész → FP	előjeles
`UCVTF Sd, Wn` / …	egész → FP	előjel nélküli
`FCVTZS Wd, Sn` / `FCVTZS Xd, Dn` / …	FP → egész	előjeles, **Z** = toward zero
`FCVTZU Wd, Sn` / …	FP → egész	előjel nélküli, toward zero

Az `SCVTF` a forrásregiszter méretéből és az `S`/`D` célból egyértelműen azonosítja a konverziót: `SCVTF Dd, Xn` egy 64 bites előjeles egészt alakít `double`-ré. Az egész → FP irányban a kerekítés az `FPCR` módja szerint történik, és 64 bites forrásból `Sn`-be konvertálva pontosságvesztés lehetséges.

Az FP → egész irányban a `Z` az `FCVTZS` **kerekítési módját** jelöli: csonkítás nullához (round toward zero). Ez azért kritikus, mert a C `(int)y` kasztja is csonkít nullához. Ha más kerekítés kell, külön utasítás létezik: `FCVTNS` (nearest), `FCVTPS` (plus infinity), `FCVTMS` (minus infinity), illetve ezek `…U` előjel nélküli párjai, valamint a `FRINT*` család, ha nem egészbe, hanem lebegőpontosba kerekítünk.

Két fontos részlet:

- **Telítés (saturation).** Ha az érték nem fér el a cél egésztípusban, az eredmény a legnagyobb/legkisebb ábrázolható egészre telítődik, és az `FPSR` IOC bitje beáll — nem lesz belőle „körbefordulás”, mint fixpontos környezetben. A specifikáció szerint NaN bemenetre az eredmény 0. Ez a viselkedés eltér az x86 `CVTTSS2SI` „integer indefinite” (`0x80000000`) mintájától, ezért portoláskor gyakori hibaforrás. **Ez az ISA viselkedése, nem a C-é:** C-ben a tartományon kívüli (vagy NaN) `float` → egész konverzió ettől még **definiálatlan viselkedés** (C17 6.3.1.4). A fordító nem köteles `FCVTZS`-t használni, és optimalizáláskor feltételezheti, hogy ilyen eset nem fordul elő. Aki a telítésre épít, annak a tartományt a konverzió előtt C-ben kell ellenőriznie, vagy az utasítást assemblyben (illetve intrinsic-kel) kell kiadnia.
- **Nincs `NZCV`-hatás.** A konverzió nem állítja be az `NZCV` flageket; aki a konverzió sikerességét elágazással akarja ellenőrizni, annak előbb `MRS`-sel ki kell olvasnia az `FPSR`-t.

Lebegőpontos lebegőpontos átalakításra az `FCVT Dd, Sn` és `FCVT Sd, Dn` szolgál — ez az egyetlen módja a `float` és `double` közötti váltásnak, mert az aritmetikai utasítások nem keverik a pontosságot.

12.2.6 Összehasonlítás: FCMP és a feltételkódok

Az `FCMP Sn, Sm`, `FCMP Dn, Dm` és `FCMP Sn, #0.0` formák lebegőpontos összehasonlítást végeznek, és az eredményt az `NZCV`-be írják. Innen a Feltételes végrehajtás feltételkódjai használhatók — de **nem ugyanúgy, mint egész összehasonlítás után**.

A `NaN` jelenléte miatt az összehasonlításnak négy kimenetele van: egyenlő, nagyobb, kisebb és **rendezetlen (unordered)**. Az ARM az alábbi flag-kiosztást használja:

Összehasonlítás	N	Z	C	V
egyenlő	0	1	1	0
nagyobb	0	0	1	0
kisebb	1	0	0	0
rendezetlen (NaN)	0	0	1	1

Ebből a feltételkódok lebegőpontos jelentése:

Feltételkód	NZCV-feltétel	Lebegőpontos jelentés	C-beli megfelelő
`EQ`	`Z==1`	egyenlő	`a == b`
`NE`	`Z==0`	nem egyenlő **vagy rendezetlen**	`a != b`
`MI`	`N==1`	kisebb (rendezett)	`a < b`
`CC/LO`	`C==0`	kisebb (rendezett)	`a < b`
`GT`	`Z==0 && N==V`	nagyobb (rendezett)	`a > b`
`GE`	`N==V`	nagyobb vagy egyenlő (rendezett)	`a >= b`
`LS`	`C==0 || Z==1`	kisebb vagy egyenlő (rendezett)	`a <= b`
`LT`	`N!=V`	kisebb **vagy rendezetlen**	—
`LE`	`Z==1 || N!=V`	kisebb vagy egyenlő **vagy rendezetlen**	—
`HI`	`C==1 && Z==0`	nagyobb **vagy rendezetlen**	—
`CS/HS`	`C==1`	nagyobb vagy egyenlő **vagy rendezetlen**	—
`VS`	`V==1`	rendezetlen (NaN)	`isnan(a) || isnan(b)`
`VC`	`V==0`	rendezett	egyik sem NaN

A tanulság: a C `>` operátora **rendezett »nagyobb, mint« összehasonlítás**, ezért a helyes feltételkód a `GT`, nem a `HI` — `HI` ugyanis `NaN` esetén is igaz lenne. A `!=` viszont szándékosan `NE`, mert az IEEE 754 szerint `NaN != x` igaz. A `FCMPE` a jelző (signaling) változat: csendes NaN esetén is kiváltja az invalid operation kivételt, `FCMP` nem. A C relációs operátoraihoz (`<`, `>`, `<=`, `>=`) a fordító jellemzően `FCMPE`-t, az egyenlőséghez (`==`, `!=`) `FCMP`-ot használ; a feltételkódok ugyanazok.

12.2.7 Kidolgozott példa: átlag számítása és csonkítása

Munkakönyvtár: `~/arm-labor/a64/a045/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a045
cd ~/arm-labor/a64/a045
```

A példa három egész átlagát számolja `float`-ként, majd csonkítva visszaalakítja egésszé, és a kilépési kódban adja vissza.

```
// lab-a045-avg.s - skaláris lebegőpontos átlag és csonkítás
// target: qemu-aarch64 (user-mode), A64 ISA, little-endian
    .text
    .global _start

_start:
    mov     w0, #10
    mov     w1, #20
    mov     w2, #13

    scvtf   s0, w0              // s0 = 10.0f
    scvtf   s1, w1              // s1 = 20.0f
    scvtf   s2, w2              // s2 = 13.0f

    fadd    s3, s0, s1          // s3 = 30.0f
    fadd    s3, s3, s2          // s3 = 43.0f

    mov     w3, #3
    scvtf   s4, w3              // s4 = 3.0f  (a 3.0 FMOV-immediate-tel is képezhető: fmov s4,
↪   #3.0)
    fdiv    s5, s3, s4          // s5 = 14.333333f

    fcvtzu  w0, s5              // w0 = 14 (csonkítás nullához)
    mov     x8, #93             // __NR_exit
    svc     #0
    .section .note.GNU-stack,"",%progbits
```

Assemble, link és futtatás:

```
aarch64-linux-gnu-as -o lab-a045-avg.o lab-a045-avg.s
aarch64-linux-gnu-ld -o lab-a045-avg lab-a045-avg.o
# elvárt: GNU_STACK RW
readelf -lW lab-a045-avg \
  | awk '$1=="GNU_STACK"{f=$7} END{print "GNU_STACK", f; exit !(f=="RW")}'
qemu-aarch64 ./lab-a045-avg ; echo $?
```

Mérve (a lektor mérése, GNU binutils 2.42, QEMU 8.2.2): a `GNU_STACK` fejléc `RW`, a kilépési állapot `14`. A `43/3 = 14.333…` pontos értéke `binary32`-ban nem ábrázolható, ezért az `FDIV` itt IXC-t állít az `FPSR`-ben; a csonkítás után mégis `14` lesz. A kilépési státusz a `w0` alsó 8 bitje, ezért a `14` közvetlenül olvasható.

Kiegészítő részlet (nem önálló program), amely a többi műveletet és az összehasonlítást mutatja:

```
    // feltételezés: s0 = a, s1 = b  (mindkettő float)
    fsub    s2, s0, s1          // s2 = a - b
    fmul    s3, s0, s1          // s3 = a * b
    fdiv    s4, s0, s1          // s4 = a / b
    fcmp    s0, s1              // a ? b
    cset    w5, gt              // w5 = 1, ha a > b (rendezett)
    // s6 = (a > b) ? a : b; rendezetlen összehasonlításnál b, nem FMAX
    fcsel   s6, s0, s1, gt
    fcvt    d7, s0              // d7 = (double)a, pontosságnövelés
```

12.2.8 Tipikus hibák és ellenpróbák

1. `FMOV` konverzióra használata. Tünet: az `SCVTF` helyett `FMOV S0, W0` után az `FCVTZU` eredménye `0` lesz, holott a bemenet `10` volt. Hipotézis: a bitminta denormálisként értelmeződött, és a csonkítás nullát ad. Adatgyűjtés: `aarch64-linux-gnu-objdump -d lab-a045-avg | grep -A3 -B3 fmov` — ha `fmov` szerepel ott, ahol `scvtf`-nek kellene, megvan a hiba. Egy változtatás: cseréld `scvtf`-re, majd `echo $?` ellenőrzés.

2. Precízió-keverés. Az `fadd s0, s0, d1` assemble-idejű hibát ad. A GNU as 2.42 üzenete mérve: `Error: operand mismatch -- `fadd s0,s0,d1'`, majd `Info: did you mean this? fadd s0, s0, s1` és `other valid variant(s): fadd d0, d0, d1`. Ez nem hiba, hanem a szerződés működése: előbb `fcvt s1, d1`, utána az összeadás. A pontos szöveg verziófüggő.

3. Rossz feltételkód a rendezetlen (NaN) esetre. Hibás: `HI` az `a > b` helyett (a `HI` NaN-ra igaz), illetve `LE`/`LT` az `a <= b` helyett (azok NaN-ra szintén igazak). Helyes: `GT` az `a > b`-hez és `LS` (rendezett `<=`) az `a <= b`-hez — az `LS` tehát *nem* hibás választás. Tünet: `NaN` bemenetre az elágazás a „nagyobb” (vagy „kisebb”) ágra fut. Ellenpróba: futtasd a kódot `a = NaN`, `b = 1.0f` bemenettel `HI`-vel, majd `GT`-vel; ugyanígy `LE` és `LS` között.

4. Negatív érték csonkítása előjel nélküli célba. `FCVTZU -1.5f`-re telít a `0`-ra (és IOC-t állít), míg `FCVTZS -1`-et ad. Tünet: a negatív mérési értékek `0`-ként jelennek meg. Ellenpróba: futtasd mindkét változatot `-1.5f`-en, és hasonlítsd össze.

5. Pontosságra épülő egyenlőségvizsgálat. `0.1f` tízszeri összeadása mérve `0x3F800001` (1.00000012), nem pontosan `1.0f`, ezért az `fcmp s0, s1` `NE`-t adhat. Ez nem assembly-hiba, hanem IEEE 754 következmény; a megoldás epsilon-összehasonlítás (`fsub` + `fabs` + `fcmp`).

6. `FMOV` immediate nulla értékkel. A `fmov s0, #0.0` nem assemblálható, mert a 0,0 nem része az FMOV-immediate készletének; a GNU as 2.42 üzenete mérve: `Error: invalid floating-point constant at operand 2 -- `fmov s0,#0.0'`. *Megoldás:* nullázáshoz `fmov s0, wzr` vagy `movi d0, #0`. Ez gyakori kezdő hiba akkumulátor nullázásakor.

12.2.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd el, miért nincs `FADD`-nak előjeles és előjel nélküli változata, miközben `SCVTF`-nek és `UCVTF`-nek van! Legyen `X1 = 3`. Add meg a `D0` bitmintáját és numerikus értékét `FMOV D0, X1`, illetve `SCVTF D0, X1` után. Melyik forma ad `3.0`-t `double`-ként, és melyik ad egy `3` értékű egész bitmintát lebegőpontosnak értelmezve?

2. (implementációs, középszint — cél: rutinépítés) Írj olyan assembly-sorozatot, amely az `S0`-ban lévő `x float` bemenetből kiszámítja `y = (x - 2.5f) * 0.5f`-et, az eredményt `S0`-ban hagyja, majd egy `W1`-be csonkítva konvertálja! Használj `FMOV` immediate-ot, ha az assembler elfogadja a `2.5`-öt és a `0.5`-öt; ha nem, töltsd a konstansokat `.word` direktívával a `.rodata` szekcióból. Ellenőrizd `x = 5.0f`-re, hogy az eredmény `1.25f`, `W1 = 1` legyen, és indokold, mi történik `x = 1.0f` esetén.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Adott egy assembly program, amely állítólag `3.0f`-et konvertál `int`-té, de `0`-t ad. Írj fel egy háromlépéses hibakeresési tervet: (a) milyen `objdump`-kimenetet keresel, (b) mit vizsgálsz GDB-ben, (c) milyen egyszeri változtatással ellenőrzöd a hipotézist! Milyen bitmintát vársz `3.0f` esetén `W` regiszterben, ha `FMOV Wd, Sn`-nel olvasod ki? Ugyanezt add meg `1.0f`-re is.

12.2.10 Megoldások

1. megoldás — Segítség. Gondold át, mit jelent az, hogy az IEEE 754 formátum előjel-nagyság alakú, és hogy az `FMOV` egy *bitminta-másolás*, nem egy *érték-konverzió* — melyik utasítás végez ténylegesen konverziót?

1. megoldás — Teljes megoldás. Az IEEE 754 lebegőpontos szám előjel-nagyság (sign-magnitude) alakú: nincs kettes komplemens, ezért az „előjeles/előjel nélküli” megkülönböztetésnek nincs értelme az aritmetikában — a negatív szám ábrázolása önmagában tartalmazza az előjelet. Az egész ↔ lebegőpontos átalakításnál viszont a forrás *egész* értelmezése kétértelmű: a `0xFFFFFFFF` jelenthet `-1`-et vagy `4294967295`-öt, ezért itt explicit `SCVTF`/`UCVTF` döntés kell. `X1 = 3` esetén `FMOV D0, X1` a `3` egész bitmintáját másolja: `D0 = 0x0000000000000003`, ami lebegőpontosként egy denormált szám ($1{,}5 \cdot 10^{-323}$), nem `3.0`. `SCVTF D0, X1` konvertál: `D0 = 0x4008000000000000`, azaz `3.0`. Fordítva, `D1 = 3.0` esetén `FMOV X0, D1` a `0x4008000000000000` bitmintát adja `X0`-ba, `FCVTZS X0, D1` pedig az egész `3`-at. Tehát a `3.0` értéket az `SCVTF` adja; az `FMOV` bitmintát másol.

2. megoldás — Segítség. Építsd fel a kifejezést két lépésben (`FSUB`, majd `FMUL`), és gondold át `x = 1.0f`-re, hogy a `(x - 2.5f) * 0.5f` eredménye negatív lesz-e, és mit tesz ilyenkor a nullához csonkító konverzió.

2. megoldás — Teljes megoldás. A feladat elsődleges kérése az FMOV-immediate használata; ez a `2.5`-re és a `0.5`-re is működik. Bemenete `S0 = x`, kimenete `S0 = y` és `W1 = (int)y`; a belépési pont és a hívó nem része a részletnek:

```
    // Kódrészlet: a hívó előzőleg s0-ba helyezte az x bemenetet.
    fmov    s1, #2.5              // s1 = 2.5f
    fmov    s2, #0.5              // s2 = 0.5f
    fsub    s0, s0, s1            // s0 = x - 2.5f
    fmul    s0, s0, s2            // s0 = (x - 2.5f) * 0.5f
    fcvtzs  w1, s0                // w1 = (int)y, csonkítva
```

Az `fmov s1, #2.5` és `fmov s2, #0.5` assemble-elhető: az A64 `FMOV` skaláris immediate 8 bites kódolású, ezért csak korlátozott konstanshalmazt ($\pm(1 + m/16) \times 2^n$, $n \in [-3, 4]$, $m = 0..15$) ábrázol — a `0.5` (`0x3F000000`), a `2.5` (`0x40200000`) és a `3.0` (`0x40400000`) beletartozik, a `0.1` nem. Az aritmetikai utasítások (`fadd`, `fmul`…) viszont nem fogadnak immediate-et, csak regiszter-operandust.

Tetszőleges konstansra a `.word` + `LDR` változat működik:

```
    .section .rodata
    .align 3
lab_c25:    .word 0x40200000      // 2.5f
lab_c05:    .word 0x3F000000      // 0.5f

    .text
    // Kódrészlet: a hívó előzőleg s0-ba helyezte az x bemenetet.
    ldr     s1, lab_c25           // s1 = 2.5f
    ldr     s2, lab_c05           // s2 = 0.5f
    fsub    s0, s0, s1            // s0 = x - 2.5f
    fmul    s0, s0, s2            // s0 = (x - 2.5f) * 0.5f
    fcvtzs  w1, s0                // w1 = (int)y, csonkítva
```

A `ldr s1, lab_c25` PC-relatív literálbetöltés: az eltolás előjeles, 19 bites, néggyel skálázott, ezért a konstansnak a betöltő utasítástól számított ±1 MiB tartományban kell lennie; ezt a Load/store címzési módok tárgyalja.

`x = 5.0f` esetén `5.0 - 2.5 = 2.5`, `2.5 * 0.5 = 1.25`, csonkítva `1` — a várt eredmény teljesül. `x = 1.0f` esetén `1.0 - 2.5 = -1.5`, `-1.5 * 0.5 = -0.75`, amit a nullához csonkítás `0`-ra visz.

Itt az FCVTZS és az FCVTZU egyaránt 0-t ad, mert -0.75 a [-1,0] sávban van. A választás x = 0.0f-nél számít: ekkor y = -1.25, FCVTZS → -1, FCVTZU → 0 (telítés, IOC). A negatív értékek helyes kezeléséhez tehát FCVTZS kell, de a különbség csak |y| 1 esetén látszik.

3. megoldás — Segítség. Az objdump-ban keress olyan mnemonicot, amely bitmintát másol ott, ahol a forrás konverziót ígér — a kettő bitmintája ugyanarra a kiinduló egészre egészen más float-ot ad.

3. megoldás — Teljes megoldás. (a) Az objdump-kimenetben fmov mnemonicot keresünk ott, ahol scvtf-nek kellene lennie: aarch64-linux-gnu-objdump -d lab-a045-x | grep -E 'fmov|scvtf|fcvtzu'. (b) GDB-ben töréspontot teszünk az SCVTF után, és megnézzük $s0 lebegőpontos nézetét (p $s0.f, várt: 3; a p $s0 a teljes uniót mutatja — f, u és s mezők —, ezért ott nem 3-at, hanem a {...} szerkezetet látjuk), majd egy fmov w9, s0 után p/x $w9 — a várt bitminta 3.0f-re 0x40400000. Ha itt 0x00000000-t látunk, akkor a mozgatás nem történt meg vagy hibás forrásból indult. (c) Egyetlen változtatás: ha az objdump fmov-ot mutat, cseréljük scvtf-re, újraépítünk, és echo $?-lel ellenőrizzük a 3-at. 1.0f bitmintája 0x3F800000 — érdemes ezt is fejben tartani, mert a lebegőpontos konstansok azonosítása disassembly-ben leggyakrabban ezzel a két értékkel kezdődik.

12.2.11 Rövid összefoglaló

A skaláris lebegőpontos utasítások pontosságát a regiszternév (Sn/Dn) kódolja, nem a mnemonic, és a két pontosság nem keverhető. Az FADD/FSUB/FMUL/FDIV helyesen kerekít az IEEE 754 szerint, kivételeiket az FPSR halmozza, nem az NZCV. Az FMOV bitmintát másol, nem konvertál; a határátlépésre az SCVTF/UCVTF és az FCVTZS/FCVTZU szolgál, utóbbi nullához csonkít és telít. Az FCMP négy kimenetele közül a rendezetlen (NaN) eset miatt a >-hez GT, a <=-hez LS kell, nem HI/LE.

Továbbhasznált definíciók: **skaláris lebegőpontos utasítás**; **FMOV (bit szintű másolás)**; **FCVTZS/FCVTZU (csonkító, telítő konverzió)**.

12.2.12 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Architecture Reference Manual for A-profile architecture (DDI0487) — az FADD/FMOV/SCVTF/FCVTZS/FCMP pontos pszeudokódja és az NZCV FP-kiosztása. **Ellenőrizve**: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Az FPCR (RMode, FZ, DN) és az FPSR (IOC, DZC, OFC, UFC, IXC) mezőinek szerződése ugyanebben a dokumentumban (DDI0487), az utasítások pszeudokódjával együtt: ez határozza meg, hogy a „helyesen kerekített” eredmény melyik kerekítési módra értendő. A C-nyelvi oldalon a konverziók (C17 6.3.1.4) és az alapértelmezett argumentum-előléptetés (C17 6.5.2.2) szabálya a mérvadó.
- Arm A-profile architecture dokumentáció — utasításkészlet-referencia. **Ellenőrizve**: a link él; ugyanaz a JS-renderelési korlát érvényes.
- Arm Neon intrinsics referencia — a következő alfejezetek skaláris/vektoros párhuzamához. **Ellenőrizve**: a link él; ugyanaz a JS-renderelési korlát érvényes.

- QEMU dokumentáció — a `qemu-aarch64` user-mode viselkedéséhez. **Ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát); a hivatalos user-mode fejezet (`docs/master/user/main.html`) nem állít semmit a cache- vagy ciklusszám-modellezésről, ezért az alábbi 3. pont levezetés, nem QEMU-dokumentált tény.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak, és amelyeket a hallgatónak a saját laborjában kell lezárnia:

1. Az `FCVTZS`/`FCVTZU` NaN- és túlcsordulás-viselkedésének (NaN → 0, telítés a szélsőértékre, IOC beállítás) pontos ARM ARM megfogalmazása — itt specifikáció szerinti viselkedésként áll, de a konkrét szöveghely a fenti JS-renderelési korlát miatt nem volt automatikusan kinyerhető.
2. Az `FMOV Sd, #imm` által ténylegesen elfogadott konstanskészlet az adott `as` verzióban — az assembler hibája a mérvadó.
3. Az `FADD`/`FDIV` latenciák táblázata **illusztratív, nem mért**; a QEMU TCG egy funkcionális, utasításszintű emulátor, nem ciluspontos szimulátor, ezért abszolút latenciát csak fizikai AArch64 hoston vagy a gyártó technikai leírásából szabad idézni.
4. A `qemu-aarch64` és a fizikai AArch64 CPU közötti esetleges eltérés az `FPSR` flagjeinek jelentésében és a NaN-propagációban jelölendő, ha a laborban eltérést tapasztalsz.

12.3 A NEON regiszterkészlet és adatelrendezés

12.3.1 Motiváció

Vegyünk egy tipikus mérnöki feladatot: négy `float` értéket kell egy `for` ciklusban `s` skalárral szorozni. A C fordító erre a ciklusra – kapcsolókkal – néhány vektoros utasítást generálhat, de ahhoz, hogy ezt kézzel is meg tudjuk írni, először azt kell tisztázni, **hol van a négy érték**. A skaláris világban minden érték egy külön regiszterben (`S0`, `S1`, …) lakik, és az `FADD`/`FMUL` egyetlen elemet dolgoz fel. A NEON (Arm Advanced SIMD) világában viszont **egy regiszter több elemet tárol párhuzamosan**, ezért az első kérdés nem az aritmetika, hanem a **regiszterkészlet és az adatelrendezés**: hány elem fér egy regiszterbe, milyen széles egy elem, és hogyan kerülnek az értékek a memóriából a regiszter megfelelő „rekeszeibe”. Ha ezt nem rögzítjük, a vektoros utasítások szemantikája értelmezhetetlen.

Ez az alfejezet a skaláris lebegőpontos utasítások (A lebegőpontos regiszterkészlet és az IEEE 754 ábrázolás, Skaláris lebegőpontos utasítások) után a NEON *keretét* adja meg: a `V0`–`V31` regiszterek nézeteit, a lane-jelölést, valamint az `LD1`/`ST1` betöltő/tároló utasításokat. A tényleges vektoros aritmetikát a Vektorizált összeadás/szorzás: egy skaláris ciklus NEON-átirata tárgyalja; itt csak annyit használunk belőle, amennyi a nézetek bemutatásához kell.

12.3.2 Definíció: a NEON regiszterkészlet és nézetei

Az A64 ISA **32 darab 128 bites vektorregisztert** definiál: `V0`–`V31`. Minden `Vn` regiszter egy 128 bites tár, amelyet az utasítás *arrangement*-je (elrendezés-jelölése) bont fel azonos méretű **lane**-ekre (sávokra). Egy utasítás a `Vn` regisztert a 128/64/32/16/8 bites nézetek közül pontosan egyben éri el.

A jelölés: `Vn.<darabszám><típus>`, ahol a típusbetű a lane méretét adja meg:

Típusbetű	Lane méret	Lehetséges nézetek (128 bit)	64 bites nézet
`B`	8 bit (byte)	`Vn.16B` (16 sáv)	`Vn.8B` (8 sáv)
`H`	16 bit (halfword)	`Vn.8H` (8 sáv)	`Vn.4H` (4 sáv)
`S`	32 bit (single)	`Vn.4S` (4 sáv)	`Vn.2S` (2 sáv)
`D`	64 bit (double)	`Vn.2D` (2 sáv)	`Vn.1D` (1 sáv)

A szabály kötelező: **a darabszám × lane szélesség = 128 vagy 64 bit**. Ezért `V0.4S`, `V0.8H` és `V0.16B` ugyanannak a 128 bites tárnak a három különböző felbontása; bármelyik használható, de az utasításnak és az elrendezés-jelölésnek összhangban kell lenniük.

A lane-ek **indexelhetők** is: `Vn.<T>[i]`, ahol `i` a 0 és a darabszám-1 közötti sávszám. A **0. lane a legkisebb helyértékű** biteket foglalja el. Ez azért fontos, mert little-endian tárolásnál az `LD1` betöltés során a **memória legkisebb címén lévő elem kerül a 0. lane-be**, és így tovább növekvő címekkel növekvő lane-indexbe. Ha az értelmezés mégis fordítottnak tűnik, az szinte mindig endianness- vagy lane-számozási félreértés (Bájtsorrend: endianness és miért releváns a határon).

> **Definíció.** *NEON-lane*: egy `Vn` vektorregiszter egyetlen, `B`/`H`/`S`/`D` méretű, a megadott nézet szerinti sávja; az utasítások lane-enként (SIMD-módon) párhuzamosan dolgoznak fel egy `Vn.<darabszám><T>` elrendezésben.

Megjegyzés: a `Qn` a 128 bites egész (`Vn.16B` megfelelője), a `Dn` a 64 bites alsó rész (`Vn.8B`), az `Sn`/`Hn`/`Bn` pedig a legalacsonyabb 32/16/8 bit skaláris nézete. NEON-utasításoknál a `Vn.<T>` alakot írjuk.

12.3.3 Működés: betöltés és tárolás, lane-ek közötti mozgatás

12.3.3.1 LD1/ST1 – vektoros betöltés és tárolás

A `LD1` (load single structure) egy vagy több `Vn` regiszterbe tölt **összefüggő** memóriából, a `ST1` (store single structure) ugyanígy tárol. Alapszintaxis (A64):

```
ld1   {v0.4s}, [x1]          // 16 bájtot tölt v0-ba, 4 db 32 bites lane
ld1   {v0.16b}, [x1]         // ugyanaz, byte-nézetben
ld1   {v0.2d}, [x1]          // 16 bájt, 2 db 64 bites lane
ld1   {v0.2s}, [x1]          // CSAK 8 bájtot tölt (2 db 32 bites lane)
st1   {v0.4s}, [x0]          // 16 bájtot tárol
ld1   {v0.4s}, [x1], #16     // post-index: x1 += 16 a betöltés után
ld1   {v0.4s, v1.4s}, [x1]   // 32 bájtot tölt (v0-ba az első 16, v1-be a második 16)
```

A kulcsészrevétel: az `LD1` által átvitt **bájtszám a nézetből következik**, nem külön paraméter. A `{v0.4s}` és a `{v0.16b}` ugyanazt a 16 bájtot tölti be, csak az értelmezést adja meg. A `{v0.2s}` ezzel szemben csak 8 bájtot mozgat (a `v0` felső 64 bitje az `LD1` 64 bites formájánál nullázódik) – az assembler nem kifogásolja. Ez **túl rövid betöltés**: a program a szándékolt 16 bájt helyett 8-at kap, de a memóriaelérés érvényes, tehát nem várunk faultot. A **tényleges túlolvasás** ezzel szemben az objektum határán túli hozzáférés (pl. `ld1 {v0.4s}` egy 12 bájtos objektum végén): ez tér el attól, amit a program lefoglalt, és őrlapos memóriateszttel (az objektum után `PROT_NONE` lap, vagy `mmap` + `mprotect`) fault-ként kimutatható, míg a túl rövid olvasás nem.

Az `LD1` „single structure” változat; a rokon `LD2`/`LD3`/`LD4` ugyanabból a memóriából *összefésült* (interleaved) adatot bont szét több regiszterbe. Az `LD2`/`LD3`/`LD4` (AoS→SoA szétválasztás) és a `TBL` (táblázatos bájtpermutáció) **nem része a könyv hatókörének**; jelen alfejezet és a

következők az egyregiszteres `LD1` formánál maradnak. Továbbolvasás: Arm Architecture Reference Manual (DDI0487), „Advanced SIMD load/store structure” és `TBL/TBX` utasításleírások, valamint az Arm Neon intrinsics referencia (`vld2q_f32`, `vqtbl1q_u8`).

12.3.3.2 Lane-ek közötti mozgatás: DUP, INS, MOV

Három mnemonic (**DUP**, **INS**, **MOV**), öt forma arra, hogy egy lane-t értékkel lássunk el:

- `dup Vd.<T>, Rn` – egy általános célú regiszter bitmintáját **mindegyik** lane-be másolja (`dup v1.4s, w2`).
- `dup Vd.<T>, Vn.<T>[i]` – egy meglévő lane értékét terjeszti minden lane-be (broadcast).
- `ins Vd.<T>[i], Rn` – egy általános célú regisztert ír a megadott lane-be.
- `ins Vd.<T>[i], Vn.<T>[j]` – egy lane-t másol egy másik regiszter megadott lane-jébe.
- `mov Vd.16b, Vn.16b` – teljes 128 bites **regisztermásolás** (nem lane-másolás!).

Visszafelé, egy lane-t általános célú regiszterbe a `umov`/`smov` visz (`umov w3, v0.s[2]`).

Az `objdump` az `INS` utasítást a preferált `mov Vd.T[i], …` alias alakjában írja ki (a lektor mérése, binutils 2.42): az `ins v1.s[0], v0.s[3]` forrássor a disassemblyben `mov v1.s[0], v0.s[3]`-ként jelenik meg. A disassembly ellenőrzésekor tehát ne az `ins` mnemonicot keressük.

Az `ins` csak a megadott lane-t írja; a **többi lane megtartja a korábbi értékét** (az utasítás szemantikája ez, nem „definiálatlan”). Definiálatlan csak az az érték, amit a program még nem inicializált: ha a célregisztert előtte nem töltöttük fel, a megmaradó lane-ek tartalma nem a program által ismert. Ezért lane-enkénti összeállítás előtt rendszerint inicializálunk (például `movi v1.4s, #0` vagy `mov v1.16b, v0.16b`). Ellenpróba: négy ismert 32 bites lane-nel (`v1.4s = {1,2,3,4}`) az `ins v1.s[2], w0` után csak a 2. lane változik, a másik három megmarad; a teljes `mov v2.16b, v1.16b` után mind a négy lane egyezik a forrással.

12.3.3.3 Miért számít az igazítás?

Három különböző igazításfogalmat kell szétválasztani: a **16 bájtos SIMD-teljesítményigazítást**, a **C-típus igazítását** (`float` = 4 bájt) és az **ABI stackigazítást** (SP 16 bájtos). Az alábbi állítás Normal memory-ra, AArch64 Linuxra és *érvényes C-objektumra* vonatkozik (a `float*` legyen 4 bájtra igazított; tetszőleges bájtcímű `float*` dereferálása C-ben nem érvényes). A támogatott eset: a cím 4 bájtra (float) igazított, de nem 16-ra. A64-en az általános betöltők (köztük a NEON `LD1/ST1`) **alapértelmezés szerint megengedik a nem igazított címeket**, amennyiben a rendszer `SCTLR.A = 0` (ez a szokásos user-mode Linux és a QEMU user-mode konfigurációja). Ez azt jelenti, hogy az *igazítás nem korrektségi, hanem teljesítménykérdés*:

- Egy **16 bájtos határra eső** `{v0.4s}` betöltés jellemzően egyetlen cache-sor-hozzáféréssel kiszolgálható.
- Egy **cache-sor határán átnyúló** hozzáférés két cache-sort érint, ami extra ciklusokat és busz-tranzakciót hozhat.

Ezért a vektoros adatszerkezeteket érdemes `_Alignas(16)` (C) vagy `.balign 16` (assembly) direktívával 16 bájtos határra igazítani. Azt, hogy egy konkrét mikroarchitektúrán mennyi a többletköltség, **az adott mag dokumentációjából vagy mérésből kell igazolni**, nem az ISA garantálja.

12.3.3.4 AAPCS64 és a V-regiszterek

A C-interoperabilitás miatt jegyezzük meg: az AAPCS64 szerint a `V0`–`V7` és a `V16`–`V31` **caller-saved**; a `V8`–`V15`-nek csak az **alsó 64 bitje** callee-saved. Ez azt jelenti, hogy egy híváson át egyetlen `V`-regiszter teljes 128 bitjében sem bízhatunk. A `V0`–`V7` és `V16`–`V31` egészét, a `V8`–`V15` felső 64 bitjét a hívott szabadon felülírhatja. Ha a hívónak egy teljes 128 bites értékre a hívás után is szüksége van, azt a hívás előtt 16 bájtra igazított veremhelyre kell mentenie, vagy a hívás után újra kell számolnia. Ezt gyakran elfelejtik. Részletek: Az AAPCS64 regiszterszerződés alfejezet.

12.3.4 Kidolgozott példa

Munkakönyvtár: `~/arm-labor/a64/a046/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a046
cd ~/arm-labor/a64/a046
```

Írjunk egy assembly függvényt, amely betölt négy `float`-ot `LD1`-gyel, `MOV+INS` segítségével megcseréli a két szélső lane-t, majd `ST1`-gyel visszaírja. Ez tisztán a regiszter/lane kezelést demonstrálja, aritmetika nélkül.

```
// lab-swapends.S
// void lab_swapends(float *dst, const float *src);
//   dst[0..3] = { src[3], src[1], src[2], src[0] }
        .text
        .global lab_swapends
        .type   lab_swapends, %function
lab_swapends:
        ld1     {v0.4s}, [x1]           // v0 = {s0, s1, s2, s3}  (lane 0..3)
        mov     v1.16b, v0.16b          // v1 = teljes másolat
        ins     v1.s[0], v0.s[3]        // v1.s[0] = s3
        ins     v1.s[3], v0.s[0]        // v1.s[3] = s0
        st1     {v1.4s}, [x0]           // dst[0..3] = {s3, s1, s2, s0}
        ret
        .size   lab_swapends, .-lab_swapends

        .section .note.GNU-stack,"",%progbits
```

A C-oldali driver és a build (cross-toolchain + QEMU user-mode):

```
/* lab-swapends-main.c */
#include <stdio.h>

extern void lab_swapends(float *dst, const float *src);

int main(void)
{
    _Alignas(16) float src[4] = {1.0f, 2.0f, 3.0f, 4.0f};
    _Alignas(16) float dst[4] = {0};
    lab_swapends(dst, src);
    for (int i = 0; i < 4; ++i)
        printf("%g\n", (double)dst[i]);
    return 0;
}
```

```
aarch64-linux-gnu-gcc -static -O0 -g -o lab-swapends lab-swapends.S lab-swapends-main.c
# mért: GNU_STACK RW
readelf -lW lab-swapends \
  | awk '$1=="GNU_STACK"{f=$7} END{print "GNU_STACK", f; exit !(f=="RW")}'
qemu-aarch64 ./lab-swapends
```

Mérve (a lektor mérése, GCC 13.3.0, binutils 2.42, QEMU 8.2.2): a `GNU_STACK` jelző `RW`, a kimenet:

```
4
2
3
1
```

Regiszterállapot ellenőrzése GDB-vel (a lényeg a lane-szemlélet):

```
qemu-aarch64 -g 1234 ./lab-swapends   # az egyik terminálban
# A másik terminálban:
gdb-multiarch ./lab-swapends
(gdb) target remote localhost:1234
(gdb) break lab_swapends
(gdb) continue
(gdb) si                  # ld1 után: v0.4s = {1,2,3,4}
(gdb) p $v0.s.f           # a négy lane lebegőpontos nézete: {1, 2, 3, 4}
(gdb) p/x $v0.s.u         # ugyanaz unsigned egészként
(gdb) info registers v0 v1
```

Mérve (a lektor mérése, GDB 15.1, QEMU 8.2.2): a `break lab_swapends` a függvény első utasítására állítja a töréspontot, és `si` után a `p $v0.s.f` kimenete `{1, 2, 3, 4}`; a 0. lane bitmintája `0x3f800000` (1.0f), a 3. lane-é `0x40800000` (4.0f), ami igazolja, hogy a memória kis címe a 0. lane-be került (little-endian). A `p/x $v0` a teljes uniót írja ki (`d/s/h/b/q` mezők `f/u/s` alnézettel, több száz karakter), ezért a lane-eket célzottan a `p $v0.s.f`, illetve `p/x $v0.s.u` alakkal érdemes megjeleníteni. A GDB vektorregiszter-nézetének mezőnevei verziónként eltérhetnek.

12.3.5 Tipikus hibák és ellenpróbák

- **Nézet-darabszám tévesztés.** `V0.4S` és `V0.8H` ugyanaz a 128 bit, de `FADD v0.4S, …` helyett `FADD v0.8H, …` már értelmezhetetlen vagy hibás eredményt ad. *Tünet:* assembler-hiba vagy furcsa numerikus érték. *Ellenpróba:* `objdump -d` és a mnemonic visszaellenőrzése.
- **`LD1` méret tévesztés.** `ld1 {v0.2s}, [x1]` csak 8 bájtot olvas; a felső két lane nullázódik, így négy `float` helyett csak kettőt töltöttünk be. *Adatgyűjtés:* `gdb`-ben `x/4fw $x1` összevetése `$v0.s`-sel.
- **Post-index elhagyása.** Az `ld1 {v0.4s}, [x1]` nem lépteti `x1`-et, ciklusban a mutató nem halad. Az `ld1` nem ismeri a `[x1, #16]` normál offsetes formát (az assembler elutasítja; negatív teszt). Három érvényes megoldás a következő 16 bájt olvasására: `ldr q0, [x1, #16]` (x1 változatlan), `add x2, x1, #16` majd `ld1 {v0.4s}, [x2]`, illetve a post-indexes `ld1 {v0.4s}, [x1], #16` (x1 += 16). *Ellenpróba:* fordítsd le mindhármat, és vesd össze a betöltött bájtokat és `$x1` végértékét.
- **Lane-indexelés iránya.** Little-endian környezetben a 0. lane a legalacsonyabb című elem. Big-endian (`aarch64_be`) célon a lane-sorrend és a memóriasorrend kapcsolata más; ez a könyv hatókörén kívül esik (Bájtsorrend: endianness és miért releváns a határon).

- **`MOV` vs `INS`.** `mov v1.16b, v0.16b` teljes regisztert másol, nem egy lane-t; aki lane-másolást vár, az egész regiszter felülíródik, és a cél korábbi lane-jei elvesznek; a lane-másolást `ins`-szel kell végezni, és előtte a célregiszter lane-jeit inicializálni.
- **Teljes vektor egy híváson át.** A `V8`–`V15` felső 64 bitje a híváson át elveszhet, ha csak a callee-saved szerződés alsó felére támaszkodunk. A `V0`–`V7` (és a `V16`–`V31`) ebben nem segít, mert ezek teljes 128 bitje caller-saved: a hívások **között** szabadon használható ideiglenes regiszterek, de a hívás **után** egyikük tartalmában sem bízhatunk. *Javítás:* a híváson át szükséges teljes vektort a hívás előtt 16 bájtra igazított veremhelyre mentjük, vagy a hívás után újraszámoljuk:

```
str     q16, [sp, #-16]!        // a teljes 128 bit a verembe; az SP 16-ra igazított marad
bl      lab_helper
ldr     q16, [sp], #16          // a hívás után pontosan ugyanaz a 128 bit
```

  Ellenpróba: egy tesztsegéd szándékosan felülír mindent, amit az AAPCS64 megenged neki: a `V0`–`V7` és a `V16`–`V31` teljes tartalmát, valamint a `V8`–`V15` felső 64 bitjét. A laborban ellenőrizhető, hogy a `V0`-ban és a `V8`-ban tartott 128 bites érték a segéd hívása után elveszik, a fenti verembe mentéssel viszont bájtra pontosan megmarad.

12.3.6 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Adott egy `V0` nézet `V0.4S`, ahol `V0 = 0x40490FDB_3F800000_00000000_C0490FDB` (hex). Add meg `V0.S[0]`, `V0.S[1]`, `V0.S[3]` IEEE 754 szerinti közelítő értékét, és indokold, hogy a bájt-nézetben `V0.B[0]` és `V0.B[4]` miért különböző lane-ekhez tartozik.

2. (implementációs, alapszint — cél: rutinépítés) Írj egy `lab_bcast4(float *dst, float s)` assembly függvényt, amely az `s` skalárt `DUP`-pal mind a négy lane-be terjeszti (`fmov w1, s0` → `dup v0.4s, w1`), majd `ST1`-gyel `dst[0..3]`-ba írja. Készíts hozzá C drivert és futtasd `qemu-aarch64`-gyel.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Az alábbi szándékosan hibás kódrészlet csak két forráselemet tölt be, és a várt megfordítás helyett a `dst[2]` és `dst[3]` nulla marad, a `dst[0..1]` pedig csak a két első elem cseréje. Add meg a hiba okát, és javítsd úgy, hogy `dst = {src[3], src[2], src[1], src[0]}` legyen. Használj `gdb`-t a regisztertartalom ellenőrzéséhez.

```
lab_rev4:
    ld1     {v0.2s}, [x1]       // FELTÉTELEZETTEN hibás
    mov     v1.16b, v0.16b
    ins     v1.s[0], v0.s[1]
    ins     v1.s[1], v0.s[0]
    st1     {v1.4s}, [x0]
    ret
```

12.3.7 Megoldások

1. megoldás — Segítség. Bontsd a 128 bitet négy darab 32 bites lane-re a hexadecimális mintából, és minden lane bitmintáját olvasd IEEE 754 egyszeres pontosságú kódolásként — a bájt-indexelés little-endian, ez dönti el, melyik bájt melyik lane-hez tartozik.

1. megoldás — Teljes megoldás. Bontsuk 32 bites lane-ekre: `S0 = 0xC0490FDB`, `S1 = 0x00000000`, `S2 = 0x3F800000`, `S3 = 0x40490FDB`. IEEE 754 szerint: `S0` -3.1415927

(0xC0490FDB a - bitmintája), S1 = +0.0, S3 +3.1415927. A V0.B[0] a S0 legalacsonyabb bájtját jelenti (little-endian: a legalacsonyabb helyértékű bájt), V0.B[4] pedig a S1 legalacsonyabb bájtját; ugyanabban a 128 bites tárban más lane-hez tartoznak, csak a bontás más.

2. megoldás — Segítség. A dup a skalár bitmintáját egy általános célú regiszteren keresztül terjeszti szét — a lépéssorrend: skalár → GP-regiszter → DUP → ST1.

2. megoldás — Teljes megoldás.

```
// void lab_bcast4(float *dst, float s)
        .text
        .global lab_bcast4
        .type   lab_bcast4, %function
lab_bcast4:
        fmov    w1, s0              // s bitmintája GP regiszterbe
        dup     v0.4s, w1           // minden lane = s
        st1     {v0.4s}, [x0]       // 16 bájt kiírása
        ret
        .size   lab_bcast4, .-lab_bcast4

        .section .note.GNU-stack,"",%progbits
```

Driver és build:

```
/* lab-bcast4-main.c */
#include <stdio.h>

extern void lab_bcast4(float *dst, float s);

int main(void)
{
    _Alignas(16) float dst[4] = {0};
    lab_bcast4(dst, 2.5f);
    for (int i = 0; i < 4; ++i)
        printf("%g\n", (double)dst[i]);
    return 0;
}
```

```
aarch64-linux-gnu-gcc -static -O0 -g -o lab-bcast4 lab-bcast4.S lab-bcast4-main.c
qemu-aarch64 ./lab-bcast4
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2): 2.5 négyszer:

```
2.5
2.5
2.5
2.5
```

A dup v0.4s, w1 a bitmintát négyszer másolja, ami a lebegőpontos s-re pontosan s-t ad minden lane-ben, mert a forrás bitminta már helyes IEEE 754 kódolás.

3. megoldás — Segítség. Nézd meg pontosan, hány bájtot tölt be az ld1 {v0.2s} forma (egy regiszter, két 32 bites lane, 8 bájt) a v0.4s nézethez (egy regiszter, négy 32 bites lane, 16 bájt) képest — a hiányzó lane-ek innen erednek. A teljes megfordításhoz két utasítás kell: a REV64 a két 64 bites félen belül cserél, az EXT pedig a két 64 bites fél felcserélését végzi #8 bájt eltolással: {1,2,3,4} → REV64 → {2,1,4,3} → EXT #8 → {4,3,2,1}.

3. megoldás — Teljes megoldás. Az ld1 {v0.2s} csak **8 bájtot** tölt be; az utasítás a felső 64 bitet nullázza, nem a másik két forráselemet tölti oda. Emellett a két INS csak az első két elemet cseréli fel. A teljes megfordításhoz mind a négy forráselem kell. A REV64 a két 64 bites félen belül cserél, az EXT #8 pedig felcseréli a feleket:

```
lab_rev4:
    ld1     {v0.4s}, [x1]
    rev64   v0.4s, v0.4s
    ext     v0.16b, v0.16b, v0.16b, #8
    st1     {v0.4s}, [x0]
    ret
```

Mérve (a lektor mérése, GCC 13.3.0, QEMU 8.2.2): `{1,2,3,4}` bemenetre a megoldás `{4,3,2,1}`-et ad, a szándékosan hibás kód pedig `{2,1,0,0}`-t. A feladat szándékosan hibás kódja fent megmarad.

12.3.8 Rövid összefoglaló

A NEON 32 db 128 bites `Vn` regisztere ugyanazt a tárat több nézetben mutatja: `Vn.16B`, `Vn.8H`, `Vn.4S`, `Vn.2D` (és feleakkora 64 bites változataik). A darabszám × lane szélesség = 128 vagy 64 bit, a 0. lane a legalacsonyabb helyértékű – little-endian memóriában ez a legalacsonyabb cím. Az `LD1/ST1` a nézetből vezeti le az átvitt bájtszámot; a `DUP`, `INS` és `MOV` a lane-ek közötti mozgatás eszközei. Az igazítás nem korrektségi, de teljesítményt érintő tényező. AAPCS64 szerint `V0`–`V7` és `V16`–`V31` caller-saved, `V8`–`V15` csak alsó 64 bitjében callee-saved.

Továbbhasznált definíciók: **NEON-lane**; **arrangement (elrendezés-jelölés: `Vn.<darabszám><T>`)**; **LD1/ST1 (egyregiszteres vektoros betöltés/tárolás)**.

12.3.9 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Architecture Reference Manual for A-profile architecture (DDI0487) — az Advanced SIMD register file, az `LD1/ST1` és a lane-jelölés (`Vn.<T>[i]`) pontos specifikációja. *Ellenőrizve*: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Arm A-profile architecture dokumentáció — utasításkészlet-referencia. *Ellenőrizve*: a link él, ugyanaz a JS-renderelési korlát érvényes.
- Arm Neon intrinsics referencia — a C-intrinsicek és a native utasítások megfeleltetése. *Ellenőrizve*: a link él, ugyanaz a JS-renderelési korlát érvényes.
- Procedure Call Standard for the Arm 64-bit Architecture (AAPCS64) — a `V0`–`V31` megőrzési szabályai. *Ellenőrizve*: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- GNU Binutils kézikönyv — a GNU assembler A64 szintaxisa. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.
- QEMU dokumentáció — a `qemu-aarch64` user-mode és a `-static` linkelés viselkedése. *Ellenőrizve*: a `master`ághoz tartozó dokumentáció, nem egy konkrét kiadott verzióhoz kötött; a ténylegesen telepített QEMU-verzió dokumentációja ettől eltérhet.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a könyv szerzői a kódot a GNU eszközlánccal és `qemu-aarch64`-gyel itt nem futtatták; a lektori mérés viszont a fő úton (GCC 13.3.0, binutils 2.42, QEMU 8.2.2) megtörtént: a `lab-swapends` kimenete `4 2 3 1`, a kész ELF `GNU_STACK` jelzője `RW`, a GDB-menet a leírtak szerint működik. A teljes vektor verembe mentésének ellenpróbáját (lásd Tipikus hibák) a laborban kell elvégezni. Az Ubuntu 24.04-re telepített GCC/binutils/QEMU pontos verzióját a laborban kell rögzíteni.

Nyitott ellenőrzési igény: az `SCTLR.A = 0` alapértelmezés és az igazítatlan `LD1` viselkedése a konkrét QEMU-verzióban és a fizikai AArch64 hoston; a cache-sor határán átnyúló hozzáférés ciklusköltsége az adott mikroarchitektúrán. A példák kimenetét a lektor mérte (GCC 13.3.0, binutils 2.42, QEMU 8.2.2); a hallgató laborjában a saját eszközverzióival érdemes megismételni. Az `LD2`/`LD3`/`LD4` és a `TBL` nem része a könyvnek; továbbolvasás az Arm Architecture Reference Manual és a Neon intrinsics referencia.

12.4 Vektorizált összeadás/szorzás: egy skaláris ciklus NEON-átirata

12.4.1 Motiváció

Képfeldolgozó vagy jelfeldolgozó kódban gyakran ilyen ciklus áll: két `int32_t` tömböt elemenként összeadunk egy harmadikba. A skaláris változat minden elemre külön `ldr`/`ldr`/`add`/`str` sorozatot hajt végre. Egy modern AArch64 mag egy órajel alatt akár 128 bitet is be tud tölteni, ha a kód erre alkalmas alakban kéri – a skaláris ciklus viszont elemekre bontja azt a szélességet, és a betöltési sáv szélességét nem használja ki.

A feladat: ugyanazt a szemantikát („`c[i] = a[i] + b[i]` minden `i`-re”) úgy megfogalmazni, hogy egyetlen utasítás egyszerre négy `int32_t`-n dolgozzon, miközben a tömb hossza tetszőleges – nem kell 4 többszörösének lennie, és 0 is lehet. Ez a NEON-átirat (NEON rewrite) magja, és innentől a skaláris ciklus csak referencia-implementációként szolgál a korrektség ellenőrzéséhez.

12.4.2 Definíció és működés

Vektorizálás (vectorization): egy skaláris művelet újrafogalmazása úgy, hogy egy utasítás egyszerre $L = 128/w$ darab w bites adatelemen hajtsa végre ugyanazt a műveletet, ahol L a **lane-ek száma**. `int32_t` esetén $w = 32$, tehát $L = 4$. A `Vn.4S` elrendezés pontosan ezt a négy 32 bites sávot jelenti (lásd A NEON regiszterkészlet és adatelrendezés).

A vektorizálhatóság feltétele az **elemenkénti függetlenség (element-wise independence)**: az eredmény i-edik eleme kizárólag a bemenetek i-edik elemétől függ. Az `add` ilyen úgynevezett *map* minta; a prefix-összeg vagy a futó maximum viszont nem, mert ott a szomszédos elemekre is szükség van. Ez a megkülönböztetés később, a szorzás-összeadás (multiply-accumulate) mintánál is fontos marad.

A fő ciklus / maradékciklus szerkezet. Jelölje n az elemek számát. Ha n nem osztható L-lel, a ciklust ketté kell bontani:

$$n_{\text{fő}} = L \cdot \left\lfloor \frac{n}{L} \right\rfloor, \qquad n_{\text{maradék}} = n - n_{\text{fő}} = n \bmod L$$

A fő ciklus $n_{\text{fő}}$ elemet dolgoz fel, L elemes lépésekkel, vektoros utasításokkal; a **maradékciklus (remainder loop)** a maradék $n_{\text{maradék}} \in \{0, \dots, L-1\}$ elemet skalárisan. Pszeudokód:

```
i = 0
while i < n_fő:      # vektoros ág
    c[i..i+3] = a[i..i+3] + b[i..i+3]
    i += 4
while i < n:         # maradék ág
    c[i] = a[i] + b[i]
    i += 1
```

Fontos megfigyelés: $n_{\text{fő}} = n \,\&\, \sim(S-1)$, azaz `int32_t`-re $n \,\&\, \sim 3$, ami kettes komplemensben $n \,\&\, -4$ – bitmanipulációval `(n >> 2) << 2` alakban is megkapható (lásd Shift és rotate: LSL/LSR/ASR/ROR gyakorlati mintázatok). Az alábbi táblázat néhány n-re mutatja a felosztást; az értékek számítottak, nem mérték:

n	$\lfloor n/4 \rfloor$	$n_{\text{fő}}$ (elem)	$n_{\text{maradék}}$
0	0	0	0
7	1	4	3
8	2	8	0
17	4	16	1

Két tervezési döntés van, amelyet érdemes tudatosan meghozni. Az egyik az **indexelés egysége**: számolhatunk elemben (és a címzéshez `lsl #2` skálázást használunk), vagy bájtban (ekkor a lépés `#16` a fő és `#4` a maradék ciklusban). Elemszám-alapú indexelésnél a fő ciklus határa közvetlenül $n_{\text{fő}}$, bájtalapúnál $4n_{\text{fő}}$. A másik, hogy a határt előre kiszámítjuk-e, vagy minden iterációban újraszámoljuk – az előbbi rövidebb és gyorsabb ciklusmagot ad.

12.4.3 Kidolgozott példa

Munkakönyvtár: `~/arm-labor/a64/a047/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a047
cd ~/arm-labor/a64/a047
```

Az alábbi kód A64 ISA (AArch64, Armv8-A alapprofil), az AAPCS64 szerint `x0 = a`, `x1 = b`, `x2 = c`, `x3 = n`. A rutinok `void` visszatérésűek, csak a `c` tömböt írják.

Nevek és interfész. A függvénynevekben az `i32` az elemtípust jelöli. A következő alfejezet `float` tömbökkel és más paramétersorrenddel (`dst, a, b, n`) dolgozik, és ott a függvények `add_f32_*` nevet kapnak. Azonos név esetén egy régebbi, más interfészű objektum észrevétlenül belinkelődhetne. A deklarációk közös fejlécbe kerülnek, `lab-a047-add-i32.h` néven:

```
/* lab-a047-add-i32.h */
#ifndef LAB_A047_ADD_I32_H
#define LAB_A047_ADD_I32_H
#include <stddef.h>
#include <stdint.h>

/* c[i] = a[i] + b[i], 0 <= i < n.
 * Előfeltétel: a, b, c legalább n elemű, érvényes tömbök (n = 0 esetén
 * nem dereferáljuk őket); c nem fedi át részlegesen a-t vagy b-t; minden
 * i-re INT32_MIN <= a[i] + b[i] <= INT32_MAX (a C-referencia csak így definiált). */
void add_i32_scalar(const int32_t *a, const int32_t *b, int32_t *c, size_t n);
void add_i32_neon  (const int32_t *a, const int32_t *b, int32_t *c, size_t n);

#endif
```

A túlcsordulási tartomány. Az assembly `ADD W5, W5, W6` és a vektoros `ADD V0.4S, …` minden bemeneten modulo 2^{32} számol, a két változat tehát a teljes tartományon bitre azonos eredményt ad. A C-referencia `a[i] + b[i]` kifejezése viszont `int32_t` operandusokkal (amelyek `int`-té léptetődnek elő) előjeles túlcsordulásnál definiálatlan viselkedés. A „C-vel egyenértékű” állítás ezért csak a fejlécben rögzített tartományon igaz; a teszt bemenetei ezen belül maradnak.

Az elvárt viselkedést a fejléc C-szerződése rögzíti: `c[i] = a[i] + b[i]`. A teszt ezt a C-kifejezést elemenként maga számolja ki, és **orákulumként** használja. Mindkét assembly rutint külön-külön ehhez hasonlítja, nem egymáshoz. Először a skaláris változat assemblyben:

```
// lab-a047-scalar.s
    .text
    .global add_i32_scalar
    .type   add_i32_scalar, %function
add_i32_scalar:
    cbz     x3, .Lsc_done
    mov     x4, #0
.Lsc_loop:
    ldr     w5, [x0, x4, lsl #2]
    ldr     w6, [x1, x4, lsl #2]
    add     w5, w5, w6
    str     w5, [x2, x4, lsl #2]
    add     x4, x4, #1
    cmp     x4, x3
    b.lo    .Lsc_loop
.Lsc_done:
    ret
    .size   add_i32_scalar, . - add_i32_scalar

    .section .note.GNU-stack,"",%progbits
```

A NEON-változat ugyanezt a szerkezetet követi, csak a fő ciklusban vektorosan:

```
// lab-a047-neon.s
    .text
    .global add_i32_neon
    .type   add_i32_neon, %function
add_i32_neon:
    mov     x4, #0              // i = 0 (elemben)
    lsr     x5, x3, #2          // n / 4        (teljes csoportok)
    lsl     x5, x5, #2          // n_fő = 4 * (n/4) elemben
    cbz     x5, .Lnv_rem        // n < 4 -> nincs fő ciklus
.Lnv_main:
    lsl     x9, x4, #2              // bájteltolás = i*4; q-címzéshez külön képezve
    ldr     q0, [x0, x9]            // a[i..i+3]
    ldr     q1, [x1, x9]            // b[i..i+3]
    add     v0.4s, v0.4s, v1.4s     // 4 párhuzamos 32 bites összeadás
    str     q0, [x2, x9]            // c[i..i+3]
    add     x4, x4, #4              // 4 elemmel lépünk
    cmp     x4, x5
    b.lo    .Lnv_main
.Lnv_rem:
    cmp     x4, x3
    b.hs    .Lnv_done           // nincs maradék
.Lnv_rloop:
    ldr     w6, [x0, x4, lsl #2]
    ldr     w7, [x1, x4, lsl #2]
    add     w6, w6, w7
    str     w6, [x2, x4, lsl #2]
    add     x4, x4, #1
    cmp     x4, x3
    b.lo    .Lnv_rloop
.Lnv_done:
    ret
    .size   add_i32_neon, . - add_i32_neon

    .section .note.GNU-stack,"",%progbits
```

A `ldr q0, [x0, x4, lsl #2]` alak **nem assemblálható**: 128 bites (`q`) regiszteres átvitelnél a regiszter-offset `lsl` mezője a kódolásban csak `#0` vagy `#4` (log2(16)) lehet, mert a hardver csak a tényleges, 16 bájtos transzferméretnek megfelelő vagy nulla skálázást enged meg — `#2` (4 bájtos skálázás) ennél az utasításformánál nem létező kódolás. A lektor mérése szerint (GNU as 2.42) az assembler a sort `Error: invalid shift amount at operand 2` üzenettel utasítja el. Mivel `x4` itt elemindex (`int32_t`, 4 bájt/elem), a bájteltolást (`i*4`) külön `lsl x9, x4, #2` utasítással kell kiszámítani, és azt shift nélkül, `[Xn, X9]` alakban kell a `q`-címzésben felhasználni. A skaláris (`w`, 4 bájtos) hozzáférésnél ugyanez a `lsl #2` közvetlenül a címzésben érvényes, mert ott a transzferméret pontosan a shift által jelzett 4 bájt.

Regiszterhasználat: `x0`–`x2` a paraméterek, `x3` a hossz, `x4` a futó index, `x5` a fő ciklus elemszám-határa, `x6`/`x7` skaláris temporálisok, `x9` a `q`-címzéshez külön képezett bájteltolás, `q0`/`q1` a vektoros operandusok. Mind `x4`–`x9`, mind `v0`/`v1` caller-saved, ezért nem kell menteni őket (lásd Az AAPCS64 regiszterszerződés).

A korrektségi teszt minden $n \in [0, 17]$-re lefut, így mind a négy lehetséges maradékosztály (0, 1, 2, 3) lefedésre kerül. A két rutint **külön-külön** egy független C-orákulumhoz méri. A két rutin egymáshoz hasonlítása csak az eltérésüket mutatná meg: egy mindkettőben azonos paramétersorrend-, művelet- vagy indexhiba így észrevétlen maradna. A kimeneti tömb előtt és után négy-négy őrszó áll, és a `c` minden eleme előre ugyanezzel az őrértékkel töltődik. Így a meg nem írt elem és a tömbön kívüli írás is lebukik. `n = 0`-ra a teszt `NULL` mutatókat ad át, tehát bármilyen memória-hozzáférés laphibát okozna.

```c
/* lab-a047-main.c - mindkét rutint külön-külön egy független C-orákulumhoz méri */
#include <stdint.h>
#include <stdio.h>
#include "lab-a047-add-i32.h"

#define NMAX  17
#define GUARD 4                         /* őrszavak c előtt és után */
#define GVAL  ((int32_t)0x5A5A5A5A)     /* őrérték, egyben a c előtöltése */

typedef void (*add_fn)(const int32_t *, const int32_t *, int32_t *, size_t);

/* 0, ha f(a, b, c, n) pontosan c[i] = a[i] + b[i]-t írt minden i < n-re, és
   mást nem; a c-n kívüli őrszavak változatlanok. n = 0-ra NULL mutatókat kap:
   bármilyen hozzáférés laphiba. */
static int check(const char *name, add_fn f, size_t n)
{
    int32_t a[NMAX], b[NMAX], buf[GUARD + NMAX + GUARD];
    int32_t *c = buf + GUARD;
    for (size_t i = 0; i < n; i++) {
        a[i] = (int32_t)(i * 3 + 1);        /* minden elem különböző */
        b[i] = (int32_t)i * 5 - 2;          /* |a[i] + b[i]| < 200: nincs túlcsordulás */
    }
    for (size_t k = 0; k < GUARD + NMAX + GUARD; k++)
        buf[k] = GVAL;
    if (n == 0)
        f(NULL, NULL, NULL, 0);
    else
        f(a, b, c, n);
    for (size_t k = 0; k < GUARD + NMAX + GUARD; k++) {
        int in_c = k >= GUARD && k - GUARD < n;
        int32_t want = in_c ? a[k - GUARD] + b[k - GUARD] : GVAL;   /* C-orákulum */
        if (buf[k] != want) {
            printf("%s n=%zu: %s, c-index %ld: %d (elvárt %d)\n", name, n,
                   in_c ? "hibás elem" : "írás a c-n kívül", (long)k - GUARD, buf[k], want);
            return 1;
        }
    }
```

```
    return 0;
}

int main(void)
{
    int fails = 0;
    for (size_t n = 0; n <= NMAX; n++) {
        int bad = check("add_i32_scalar", add_i32_scalar, n)
                | check("add_i32_neon",   add_i32_neon,   n);
        if (!bad)
            printf("n=%2zu OK\n", n);
        fails += bad;
    }
    puts(fails ? "TEST: FAIL" : "TEST: PASS");
    return fails != 0;
}
```

Build és futtatás (target: `qemu-aarch64`, user-mode emuláció; a statikus link azért kell, hogy a guest dinamikus linkere ne hiányozzon):

```
aarch64-linux-gnu-as -o lab-a047-scalar.o lab-a047-scalar.s
aarch64-linux-gnu-as -o lab-a047-neon.o   lab-a047-neon.s
aarch64-linux-gnu-gcc -O2 -c -o lab-a047-main.o lab-a047-main.c
aarch64-linux-gnu-gcc -static -o lab-a047-test \
    lab-a047-main.o lab-a047-scalar.o lab-a047-neon.o
# elvárt: GNU_STACK RW
readelf -lW lab-a047-test \
  | awk '$1=="GNU_STACK"{f=$7} END{print "GNU_STACK", f; exit !(f=="RW")}'
qemu-aarch64 ./lab-a047-test; echo "exit=$?"
```

Mérve (a lektor mérése, GCC 13.3.0, glibc, QEMU 8.2.2): 18 sor `n= 0 OK` … `n=17 OK`, majd `TEST: PASS` és `exit=0`. Bármely eltérésnél a program megnevezi a rutint, az `n`-t és az első hibás `c`-indexet, megkülönbözteti a hibás elemet a `c`-n kívüli írástól, és 1-es kilépési kóddal áll le.

Ellenpróba: a teszt valóban elkapja a hibát. Mérve (a lektor mérése, GCC 13.3.0, glibc, QEMU 8.2.2):

Beültetett hiba	Megfigyelt eredmény
nincs (helyes rutinok)	`n= 0 OK` … `n=17 OK`, `TEST: PASS`, `exit=0`
`add` → `sub` **mindkét** rutinban, ugyanúgy	`add_i32_scalar n=1: hibás elem, c-index 0: 3 (elvárt -1)`, ugyanígy az `add_i32_neon`; `TEST: FAIL`
a NEON-maradékciklus feltétele `b.lo` helyett `b.ls` (egy elemmel túlír)	`add_i32_neon n=1: írás a c-n kívül, c-index 1: <szemét> (elvárt 1515870810)`; a 4-gyel osztható `n`-ek átmennek; `TEST: FAIL`
a `lsl x5, x5, #2` törlése	`TEST: PASS` (a hiba csak teljesítménybeli)

Az első sor azt mutatja, miért kell független orákulum: egy naiv változat a két rutint egymáshoz hasonlította, és ugyanezzel a kétoldali `add` → `sub` hibával hibátlanul lefutott.

Ellenőrzési lépés: `aarch64-linux-gnu-objdump -d lab-a047-neon.o` kimenetében a `q0`-ás betöltés, az `add v0.4s, ...` és a `str q0, ...` sorok mellett a ciklusmag hossza vizuálisan is látható; ha nem ezek a sorok látszanak, elavult vagy rossz objektum került a linkbe (lásd 12.5, 3. feladat).

12.4.4 Tipikus hibák és ellenpróbák

A maradékciklus hiánya. A leggyakoribb tünet: $n = 4k$ hosszon minden helyes, $n = 17$-nél viszont a 16. elem (index 16) változatlan marad. Hipotézis: a fő ciklus után nincs `while i < n` második ciklus. Adatgyűjtés: `gdb-multiarch` alatt töréspont a `.Lnv_done` cím előtti utasításon (`break *add_i32_neon+<offset>`; a `.L` helyi címke nem szerepel a szimbólumtáblában, ezért a `break .Lnv_done` nem működik), majd `p $x4`, `p $x3`; ha `$x4 != $x3`, maradék elem maradt. Ellenpróba: tegyük be a hiányzó maradék ágat, és futtassuk újra a 0–17-es tesztet.

Mértékegység-keveredés a ciklushatárban. Ha a `lsl x5, x5, #2` kimarad, akkor `x5 = n/4` *csoport*, miközben `x4` *elemben* nő `#4`-gyel. $n = 17$-re `x5 = 4` lesz, és a fő ciklus az első iteráció után kilép. A teljes vezérlési út azonban itt nem ér véget: a maradékciklus az aktuális `x4 = 4`-től `n`-ig fut, és a 4–16. elemet skalárisan elkészíti. Az eredmény tehát **helyes**, a korrektségi teszt átmegy; 4 elem készül vektorosan és 13 skalárisan. A hiba teljesítménybeli: $n = 17$-re a forrásból megszámolt dinamikus utasításszám 46 helyett 105. Ez az egyik legtanulságosabb hiba, mert a korrektségi teszt nem látja, csak a vezérlési út vizsgálata vagy az utasításszám. Ha egy mértékegység-hiba (elem vs. csoport) korrektségi hibát is okoz, az csak akkor jön elő, ha a maradékág sem a futó indexből indul. Ezt is a teljes vezérlési úton kell ellenőrizni, nem a fő ciklusban magában.

Skálázási hiba. Ha a `lsl x9, x4, #2` sor kimarad, és a `q`-betöltés/tárolás közvetlenül `x4`-et (az elemindexet) használja bájteltolásként (`[x0, x4]`), akkor a tényleges cím iterációnként csak 4 bájttal (egy elemmel) mozdul a helyes 16 bájt (négy elem) helyett — a négy lane nagyrészt ugyanazt, egymást átfedő bemenetet dolgozza fel többször, és az eredmény az elemek döntő részénél hibás, miközben a program nem áll le hibával. Ellenpróba: olyan mini-teszttel érdemes kezdeni, ahol az `a` és `b` tömb elemei mind különbözőek (ahogy a fenti harnessben) — ekkor az átfedő olvasás azonnal `hibás elem`-ként látszik.

`b.lo` vs. `b.lt`. `x4`, `x3`, `x5` mind unsigned összehasonlításban értelmezendő (indexek, nem előjeles mennyiségek), ezért a helyes feltételkód `lo`/`hs`. Ha véletlenül `lt`-t írunk, akkor n 2^{63} elem fölötti értékeinél az elágazás rossz irányba megy; degenerált tesztadatokkal ez nem derül ki.

Előjeles túlcsordulás a referenciában. A C skaláris változat `int32_t` összeadása előjeles túlcsordulásnál undefined behavior (UB), a NEON `add v0.4s` viszont definiáltan modulo 2^{32} működik. A korrektségi teszt ezért olyan értékeket használjon, ahol nincs túlcsordulás, vagy a C-referenciában `uint32_t` aritmetikát; a NEON `add v0.4s` mindkét esetben ugyanazt a bitmintát adja – a szemantikai különbséget dokumentálni kell.

`n = 0` eset. A `cbz x5` nélkül a fő ciklus magja minden $n < 4$ esetén egyszer lefut, és 16 bájtot olvas és ír: $n = 0$-ra (a teszt NULL mutatóival) SIGSEGV, $n = 1..3$-ra a `c` végén túli írás – mindkettőt a 0..17-es teszt elkapja. A nulla hossz explicit kezelése nem stílus kérdése: a `do-while` szerkezetű assembly-ciklusok tipikus hibája.

12.4.5 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Az `add_i32_neon` fő ciklusa `do-while` szerkezetű: az index növelése a `cmp` *előtt* történik. Fogalmazd meg szavakkal a ciklusinvariánst (mi igaz `x4`-re minden iteráció elején), és mutasd meg, miért `b.lo` a helyes feltétel, ha `x4` és `x5` csoportszám helyett elemszámot jelöl. Mennyiben változna az invariancia, ha az indexelést bájtban végeznénk?

2. (implementációs, középszint — cél: rutinépítés) Írj `mul_acc_neon` rutint, amely `c[i] += a[i] * b[i]`-t számol a NEON `MLA` (multiply-accumulate) utasítással a fő ciklusban és skaláris `MADD`-dal a maradékban. Készíts skaláris `mul_acc_scalar` párt is, és mindkettőt külön a

C-orákulumhoz (`c0[i] + a[i]*b[i]`, ahol `c0` a `c` előtöltött másolata) hasonlítsd. Tartsd meg a fő/maradék szerkezetet, és bővítsd a korrektségi tesztet úgy, hogy minden $n \in [0, 17]$-re mindkét implementációt összevesd. Figyelj rá, hogy `c` most *bemenet is*.

3. (hibakeresés és mérés, középszint — cél: rutinépítés) Adott egy `add_neon_buggy` változat, amelyben a fő ciklus határa `lsr x5, x3, #2` (a `lsl` kimaradt). Mutasd meg `objdump` és `gdb-multiarch` segítségével, hogy $n = 17$ esetén hány elemet dolgoz fel a fő ciklus, és hányat a maradékciklus. Indokold a teljes vezérlési út alapján, miért megy át mégis a korrektségi teszt, és mivel lehet a hibát kimutatni. Készíts ezen felül tételes utasításszámlálást: `objdump -d` alapján számold meg, hány utasítás fut le $n = 16$ elemre a skaláris és a helyes NEON-változatban. Számold a címképzést, az indexnövelést, az összehasonlítást és az elágazást is, és add meg külön a ciklusmagra és a teljes útvonalra (belépés, ciklus, maradékág, `ret`) kapott értéket. Az eredményt *dinamikus utasításszám-arányként* nevezd meg; a futásidejű gyorsulás külön mérés tárgya (NEON-gyorsulás mérése és korlátai), és QEMU alatt csak illusztratív.

12.4.6 Megoldások

1. megoldás — Segítség. Az index `x4` minden iteráció ELEJÉN mit tud már biztosan a feldolgozott tartományról — és mivel az `add #4` a `cmp` ELŐTT fut, mi a kapcsolat `x4` és `x5` közt a feltétel kiértékelésekor?

1. megoldás — Teljes megoldás. Az invariancia: „az `[x0, x0+4*x4)` intervallum és a hozzá tartozó `b`, `c` szakasz már feldolgozott, `x4` mindig néggyel osztható, és `x4 <= x5`". Az index növelése a feltételvizsgálat előtt történik, ezért a ciklusmag egyszer már lefutott, amikor a `cmp` kiértékelődik – ez a `do-while` minta. A `b.lo` azért helyes, mert `x4` és `x5` elemszám, azaz nem negatív mennyiség; `b.lt` előjeles olvasatot kérne, és 2^{63} elem fölötti, irreális hosszakat leszámítva ugyanazt adná, de a szándékot nem fejezi ki. Bájtalapú indexelésnél az invariancia `x4` értékére „16 többszöröse, `x4 <= 4*x5`" lenne, a fő ciklus lépése `#16`, a maradéké `#4`, és a címzésből eltűnne az `lsl #2` skálázás.

2. megoldás — Segítség. A szorzás-összeadás pontosan ugyanazt a fő/maradék vázat követi, mint az `add_i32_neon` — csak egy harmadik betöltés (`c` jelenlegi értéke) és egy `MLA`/`MADD` kerül az `ADD` helyére. Melyik regiszter szerepel KÉTSZER is (forrásként és célként)?

2. megoldás — Teljes megoldás. A fő ciklus lényege a három betöltés és az egyetlen `MLA`:

```
// x0 = a, x1 = b, x2 = c, x3 = n
    mov     x4, #0
    lsr     x5, x3, #2
    lsl     x5, x5, #2
    cbz     x5, .Lma_rem
.Lma_main:
    lsl     x9, x4, #2              // bájteltolás = i*4; q-címzéshez külön képezve
    ldr     q0, [x0, x9]
    ldr     q1, [x1, x9]
    ldr     q2, [x2, x9]
    mla     v2.4s, v0.4s, v1.4s     // v2 = v2 + v0*v1, lane-enként
    str     q2, [x2, x9]
    add     x4, x4, #4
    cmp     x4, x5
    b.lo    .Lma_main
.Lma_rem:
    cmp     x4, x3
    b.hs    .Lma_done
.Lma_rloop:
    ldr     w6, [x0, x4, lsl #2]
    ldr     w7, [x1, x4, lsl #2]
```

```
    ldr     w8, [x2, x4, lsl #2]
    madd    w8, w6, w7, w8          // w8 = w8 + w6*w7
    str     w8, [x2, x4, lsl #2]
    add     x4, x4, #1
    cmp     x4, x3
    b.lo    .Lma_rloop
.Lma_done:
    ret
```

Indoklás: az `MLA` (Advanced SIMD, `Vd.4S`) és a skaláris `MADD` ugyanazt a szemantikát adja lane-enként, ezért a maradék ág eredménye bitre azonos a vektoros ágéval. Megjegyzés: ez a 32 bites akkumulátor túlcsordulhat; nagy szorzók esetén szélesítő (widening) változat kell, ami az NEON-gyorsulás mérése és korlátai tárgya. A korrektségi tesztben `c`-t előre fel kell tölteni (pl. `c[i] = 7`), hogy a `+=` tényleg érvényesüljön.

3. megoldás — Segítség. Ha a `lsl x5, x5, #2` kimarad, `x5` CSOPORTSZÁM marad (`n/4`), de a ciklus `x4`-et ELEMSZÁMKÉNT, néggyel lépteti — hány iteráció után lesz `x4 >= x5`, ha `x5` csak 4 (nem 16)? És honnan indul utána a maradékciklus?

3. megoldás — Teljes megoldás. A hibás változatban $n = 17$-re `x5 = n/4 = 4`, miközben `x4` elemenként `#4`-gyel nő. `x4` értékei 0, majd 4; ekkor `4 < 4` hamis, és a fő ciklus kilép. A fő ciklus tehát pontosan 4 elemet (index 0–3) dolgoz fel. A vezérlés ezután a `.Lnv_rem` ágra kerül, amely az aktuális `x4 = 4`-től `x3 = 17`-ig skalárisan futtatja a maradékot: ez 13 elem (index 4–16). Az eredmény minden elemre **helyes**, ezért a korrektségi teszt átmegy, és első eltérés sincs. A felosztás $4 + 13$ a helyes $16 + 1$ helyett. Általában `x4` a fő ciklus után sosem lépi túl `n`-t, így a hiba minden n-re csak teljesítménybeli. GDB-vel a `.Lnv_rem` cím előtti utasításra tett töréspontnál (`break *add_i32_neon+<offset>`) `p $x4`, `p $x5`, `p $x3` (4, 4, 17) azonnal megmutatja a mértékegység-keveredést (elem vs. csoport). A hibát egy utasításszám-mérés is kimutatja: $n = 17$-re a helyes változat 46, a hibás 105 utasítást hajt végre.

Tételes utasításszámlálás $n = 16$-ra (a forrásból levezetve; a lektor a 115/39 és 46/105 értékeket a forrásból újraszámolva helyesnek találta):

Változat	Belépés	Ciklusmag × iteráció	Maradékág	Kilépés	Összesen
skaláris	`cbz`, `mov` = 2	(`ldr`, `ldr`, `add`, `str`, `add`, `cmp`, `b.lo`) = 7 × 16 = 112	–	`ret` = 1	**115**
NEON	`mov`, `lsr`, `lsl`, `cbz` = 4	(`lsl`, `ldr q`, `ldr q`, `add v`, `str q`, `add`, `cmp`, `b.lo`) = 8 × 4 = 32	`cmp`, `b.hs` = 2	`ret` = 1	**39**

A ciklusmagok dinamikus utasításszám-aránya 112/32 = 3,5, a teljes útvonalé (belépéssel és kilépéssel) 115/39 ≈ 2,95. Ez nem gyorsulási plafon. A négy sávból vagy ebből az arányból nem vezethető le ISA-szintű felső korlát, mert az utasítások ideje nem azonos: egy `ldr q` vagy `str q` más költségű, mint egy skaláris `ldr w`, és a futásidőt a memóriasávszélesség, a késleltetések és a mikroarchitektúra párhuzamossága is alakítja. A futásidő egy megnevezett végrehajtási modell vagy mérés tárgya (NEON-gyorsulás mérése és korlátai).

12.4.7 Rövid összefoglaló

A vektorizált tömb-összeadás két fázisra bomlik: **fő ciklusra**, amely L elemes lépésekkel, `ldr q` / `add Vd.4S, Vn.4S, Vm.4S` / `str q` mintával dolgozik, és **maradékciklusra**, amely a $n \bmod L$ elemet skalárisan fejezi be. A fő ciklus határa $L\lfloor n/L \rfloor$, elemszám- és bájtalapú indexelés közül tudatosan kell választani; a kettő keverése a legjellemzőbb hiba. A korrektséget független C-orákulummal, minden maradékosztályt lefedő hosszúsorozaton és a tömbhatárok körüli őrszavakkal kell igazolni, mindkét változatra külön. A szorzás-összeadás mintázat ugyanezt a vázat használja `MLA`/`MADD` utasításokkal.

Továbbhasznált definíciók: **fő ciklus (main loop)**; **maradékciklus (remainder loop)**; **lane-párhuzamos feldolgozás (element-wise map minta)**.

12.4.8 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Architecture Reference Manual for A-profile architecture (DDI0487) — az `ADD (vector)`, `MLA (vector)`, `LDR (register)` vektoros formáinak pontos kódolása, a `q`-regiszteres betöltés igazítási követelményei és a regiszter-offset skálázási korlátai. **Ellenőrizve**: a link él (a `developer.arm.com` → `support.arm.com` átirányítás megerősített); a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Arm A-profile architecture dokumentáció — utasításkészlet-referencia. Ugyanaz a JS-renderelési korlát érvényes, mint fent.
- Arm Neon intrinsics referencia — a `vaddq_s32`, `vmlaq_s32` intrinsic-ek szemantikája, ha a hallgató C-intrinsic felől közelít. Ugyanaz a korlát.
- QEMU dokumentáció, user-mode fejezet — user-mode emuláció, `qemu-aarch64` futtatási opciók. **Tartalmilag ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát). A hivatalos dokumentáció nem állít semmit a cache-hierarchia vagy a memóriasávszélesség modellezéséről; ez a TCG funkcionális, nem cikluspontos emulátor-tervezéséből következik, nem egy explicit QEMU-dokumentált korlát.

A lektor a fő úton (GCC 13.3.0, glibc, QEMU 8.2.2) futtatta a teszthajtót: a helyes rutinokkal `n= 0 OK` … `n=17 OK`, `TEST: PASS`, `exit=0`; a beültetett hibák a táblázat szerint viselkednek; a `lsl x5` törlésével a teszt `TEST: PASS` (a hiba csak teljesítménybeli); a `ldr q0, [x0, x4, lsl #2]` alakot az assembler `Error: invalid shift amount at operand 2` üzenettel utasítja el. A build blokk `readelf`-ellenőrzése a kész ELF `GNU_STACK` jelzőjét `RW`-nek várja; `-z execstack`-kel linkelve `RWE`-t jelez és nem nulla állapottal bukik.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: (1) A `qemu-aarch64` verziója és a `-static` linkelés szükségessége Ubuntu 24.04 LTS csomagolt toolchain mellett nem ellenőrzött itt; a pontos verziót és a `-L` sysroot-opció szükségességét a hallgató laborjában kell validálni. (2) A dinamikus utasításszámok a forrásból levezetve a lektor ellenőrzése szerint helyesek; QEMU user-mode nem ad hardverhű ciklusszámot, ezért futásidőre vonatkozó következtetés nem vezethető le belőlük. (3) Az `int32_t` előjeles túlcsordulás és a NEON modulo-2^{32} viselkedés eltérésének gyakorlati kezelése (fordítóflag-ek, sanitizerek) nincs ebben az alfejezetben specifikálva – ez az NEON-gyorsulás mérése és korlátai felé mutat.

12.5 NEON-gyorsulás mérése és korlátai

12.5.1 Motiváció: a vektorizáció nem önmagában gyorsulás, hanem mérhető állítás

Az előző alfejezetben megírtuk a NEON-átiratot: a fő ciklus L elemes lépésekben dolgozik (a 12.4-ben bevezetett lane-szám jelölése), a maradékot skalárisan fejezzük be. Ettől a kód *elvileg* gyorsabb. A mérnöki kérdés azonban nem az, hogy „gyorsabb-e”, hanem az, hogy **mennyivel**, milyen feltételek mellett, és hol van az a pont, ahol a többletkomplexitás nem térül meg. Egy teljesítménykritikus modulban ez a döntés közvetlenül mérnöki: ha egy kernel a futásidő 8%-át adja, és a vektorizációval annak felét megnyerjük, a teljes programra eső nyereség néhány százalék — miközben a kód karbantarthatósága és hordozhatósága romlik.

Ehhez két dolog kell. Egyrészt egy **reprodukálható mérési eljárás**, amely azonos bemeneten, ismételt futással, szóródást is közölve hasonlítja össze a két változatot. Másrészt egy **egyszerű modell**, amelyhez a mért értéket viszonyítani lehet — különben nem tudjuk megmondani, hogy egy 2,3-szeres gyorsulás jó eredmény vagy figyelmeztetés. Ezt az Amdahl-törvény adja, a feltevéseivel együtt. A mérést a debugging és instrumentáció fejezet `CLOCK_MONOTONIC`-alapú mintavételi vázára építjük; a részletes ciklusszám-elemzést a teljesítmény-tudatos fejezet folytatja.

12.5.2 Definíció: gyorsulás, vektorizálható hányad, Amdahl-modell

Legyen $T_{\text{skalár}}$ a skaláris változat futásideje és T_{neon} a NEON-változaté, azonos bemeneten, azonos mérési intervallummal. A **gyorsulás (speedup)**:

$$S = \frac{T_{\text{skalár}}}{T_{\text{neon}}}.$$

A **vektorizálható hányad (vectorizable fraction)** $p \in [0, 1]$: a skaláris változat **futásidejének** az a része, amelyre a vektorizálás hat. Nem utasításszám-hányad: az eltérő késleltetésű, egymással átfedve végrehajtott utasítások darabszámából nem következik, mennyi időt visz el a vektorizálható rész. A maradék $1 - p$ időhányad olyan munka, amelyre a vektorizálás nem hat.

Az **Amdahl-modell** adott p és a vektorizált részre vonatkozó s gyorsítási tényező mellett:

$$S_{\text{Amdahl}}(p, s) = \frac{1}{(1 - p) + \dfrac{p}{s}}.$$

A képlet akkor érvényes, ha a két részidő rögzített, és a vektorizálás csak a p részt gyorsítja, azon belül egyenletesen s-szeresen. **Az $s = 4$ ebben az alfejezetben egyszerű modellfeltevés**: azt mondja ki, hogy a vektoros rész ideje pontosan a sávok számával osztódik (128 bites `float32x4_t` esetén 4 sáv). Ez nem az ISA-ból következő felső korlát. Abból, hogy négy `float` fér egy vektorregiszterbe, nem következik a mért $S \leq 4$ általános érvénye: a vektoros ciklusban megváltozik a ciklusvezérlés aránya, az utasításkibocsátás és a függőségi láncok hatása is. A tényleges s ezért lehet 4 alatt (például ha a memóriarendszer korlátoz) és elvben 4 felett is (például ha a skaláris változatot elemenként a ciklusvezérlés vagy egy függőségi lánc fogta vissza).

12.5.3 A mérési módszer

Munkakönyvtár: `~/arm-labor/a64/a048/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a048
cd ~/arm-labor/a64/a048
```

A mérés elve: azonos bemenet, a méréstől **külön** korrektségi ellenőrzés kernelenként, melegítés, ismételt futás váltakozó sorrendben, valamint medián és szóródás (interkvartilis terjedelem, IQR) közlése. A váz négy lépése:

1. **Korrektség.** Mindkét kernel **saját kimeneti pufferbe** ír, amelyet előtte NaN őrértékkel töltünk fel, és az eredményt kernelenként, külön ellenőrizzük. Ha a két kernel ugyanabba a pufferbe írna, és csak a második mérés után ellenőriznénk, egy hibás skaláris kernel eredményét a helyes NEON-kernel felülírná, és a program sikert jelentene.
2. **Melegítés.** Néhány nem mért futás, hogy az első futás egyszeri költségei (laphibák, gyorsítótár-feltöltés, QEMU alatt a kódfordítás) ne torzítsák a mintát.
3. **Mérés.** Körönként váltakozó sorrend (páros körben skalár–NEON, páratlanban NEON–skalár), hogy a sorrendből adódó hatás ne terhelje mindig ugyanazt a kernelt.
4. **Szóródás.** A medián mellett az IQR-t is kiírjuk; egy medián szóródás nélkül nem mutatja meg, mennyire megbízható az összehasonlítás.

A deklarációk saját fejlécbe kerülnek. A függvénynevekben az `f32` megkülönbözteti őket az előző alfejezet `add_i32_*` függvényeitől, amelyek más elemtípussal és más paramétersorrenddel dolgoznak:

```
/* lab-a048-add-f32.h */
#ifndef LAB_A048_ADD_F32_H
#define LAB_A048_ADD_F32_H
#include <stddef.h>

/* dst[i] = a[i] + b[i] (float), 0 <= i < n.
 * Paramétersorrend: dst, a, b, n - eltér az a047 add_i32_* függvényeitől.
 * Előfeltétel: a tömbök legalább n eleműek; dst nem fedi át részlegesen a-t vagy b-t. */
void add_f32_scalar(float *dst, const float *a, const float *b, size_t n);
void add_f32_neon  (float *dst, const float *a, const float *b, size_t n);

#endif

/* lab-a048-bench.c */
#define _GNU_SOURCE
#include <math.h>
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <time.h>
#include "lab-a048-add-f32.h"

#ifndef N
#define N      (1u << 20)   /* 1 048 576 float = 4 MiB tömbönként */
#endif
#define WARMUP 3            /* kernelenként, nem mért futások */
#define REPS   31           /* páratlan, hogy a medián egyértelmű legyen */

typedef void lab_kernel(float *dst, const float *a, const float *b, size_t n);

/* aligned_alloc: a méret a 64 többszöröse legyen (C11); a logikai elemszám n marad.
 * n == 0 esetén is foglal 64 bájtot (dokumentált minimális memóriahasználat). */
static float *alloc_f32(size_t n) {
```

```
    if (n > (SIZE_MAX - 63) / sizeof(float)) return NULL;      /* size_t túlcsordulás */
    size_t bytes = (n * sizeof(float) + 63) & ~(size_t)63;
    return aligned_alloc(64, bytes ? bytes : 64);
}

static double now_ms(void) {
    struct timespec ts;
    /* a043 mintája */
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) { perror("clock_gettime"); exit(2); }
    return ts.tv_sec * 1000.0 + ts.tv_nsec / 1e6;
}

static int cmp_double(const void *p, const void *q) {
    double a = *(const double *)p, b = *(const double *)q;
    return (a > b) - (a < b);
}

/* Őrérték: NaN-nal töltött pufferben egy ki nem írt elem sem mehet át. */
static void fill_nan(float *d, size_t n) {
    for (size_t i = 0; i < n; ++i) d[i] = NAN;
}

/* Indexenként eltérő, vegyes előjelű, pontosan reprezentálható bemenet:
 * a lane-csere, a[i+1] olvasás, konstans kimenet és az elhagyott tail eltérést ad. */
static float va(size_t i) { return (float)((int)(i % 13) - 6) * 0.5f; }
static float vb(size_t i) { return (float)((int)(i % 7) - 3) * 0.25f + (float)(i % 5); }
static void fill_in(float *a, float *b, size_t n) {
    for (size_t i = 0; i < n; ++i) { a[i] = va(i); b[i] = vb(i); }
}

/* Külön referencia: d[i] == va(i) + vb(i) (az összeg pontos, így == összehasonlítható). */
static int check(const char *name, const float *d, size_t n) {
    for (size_t i = 0; i < n; ++i) {
        float want = va(i) + vb(i);
        if (d[i] != want) {
            printf("FAIL %s: d[%zu] = %g (vart: %g)\n", name, i, (double)d[i], (double)want);
            return 0;
        }
    }
    return 1;
}

/* Hosszsöprés őrértékkel: n = 0..17 és a 4-gyel nem osztható / határ menti hosszak.
 * A kimenet utáni 4 elem NaN őrérték marad, a bemenet utáni elemek 1e30f csapdák. */
static int sweep(const char *name, lab_kernel *fn) {
    static const size_t extra[] = { 31, 33, 63, 65, 1023, 1025, 4099 };
    size_t cnt = 18 + sizeof extra / sizeof extra[0];
    for (size_t k = 0; k < cnt; ++k) {
        size_t n = k < 18 ? k : extra[k - 18];
        float *a = malloc((n + 4) * sizeof(float)), *b = malloc((n + 4) * sizeof(float));
        float *d = malloc((n + 4) * sizeof(float));
        if (!a || !b || !d) { free(a); free(b); free(d); return 0; }
        fill_in(a, b, n);
        for (size_t i = n; i < n + 4; ++i) { a[i] = b[i] = 1e30f; d[i] = NAN; }
        fill_nan(d, n);
        fn(d, a, b, n);
        int ok = check(name, d, n);
        for (size_t i = n; ok && i < n + 4; ++i)
            if (!isnan(d[i])) { printf("FAIL %s: n=%zu, a határon túli írás d[%zu]\n", name, n,
            ↪  i); ok = 0; }
        if (!ok) printf("  (hibás hossz: n = %zu)\n", n);
        free(a); free(b); free(d);
        if (!ok) return 0;
```

```
    }
    return 1;
}

/* Medián és interkvartilis terjedelem (IQR), egyszerű rangalapú kvartilisekkel. */
static void stats(double *t, int n, double *med, double *iqr) {
    qsort(t, (size_t)n, sizeof t[0], cmp_double);
    *med = t[n / 2];
    *iqr = t[(3 * n) / 4] - t[n / 4];
}

int main(void) {
    lab_kernel *fn[2] = { add_f32_scalar, add_f32_neon };
    const char *name[2] = { "add_f32_scalar", "add_f32_neon" };
    float *a = alloc_f32(N);
    float *b = alloc_f32(N);
    float *d[2] = { alloc_f32(N),    /* külön kimeneti */
                    alloc_f32(N) };  /* puffer kernelenként */
    static double t[2][REPS];
    int rc = 1;

    if (!a || !b || !d[0] || !d[1]) goto out;
    fill_in(a, b, N);

    /* 1. Korrektség a méréstől külön: mindkét kernel saját, NaN-nal töltött pufferbe ír. */
    for (int k = 0; k < 2; ++k) {
        if (!sweep(name[k], fn[k])) goto out;
        fill_nan(d[k], N);
        fn[k](d[k], a, b, N);
        if (!check(name[k], d[k], N)) goto out;
    }
    printf("ellenorzes: %s OK, %s OK\n", name[0], name[1]);

    /* 2. Melegítés: gyorsítótárak, laptábla, QEMU-kódfordítás. */
    for (int w = 0; w < WARMUP; ++w)
        for (int k = 0; k < 2; ++k) fn[k](d[k], a, b, N);

    /* 3. Mérés váltakozó sorrendben: páros körben skalár-NEON, páratlanban NEON-skalár. */
    for (int k = 0; k < 2; ++k) fill_nan(d[k], N);
    for (int r = 0; r < REPS; ++r)
        for (int j = 0; j < 2; ++j) {
            int k = j ^ (r & 1);
            double t0 = now_ms();
            fn[k](d[k], a, b, N);                  /* csak a kernel van a mérésen belül */
            t[k][r] = now_ms() - t0;
        }

    /* 4. A mért futások eredménye is helyes-e? */
    for (int k = 0; k < 2; ++k)
        if (!check(name[k], d[k], N)) goto out;

    double med[2], iqr[2];
    for (int k = 0; k < 2; ++k) stats(t[k], REPS, &med[k], &iqr[k]);
    if (med[0] <= 0 || med[1] <= 0) { rc = 2; goto out; }

    for (int k = 0; k < 2; ++k)
        printf("%-15s median %8.3f ms  IQR %6.3f ms\n", name[k], med[k], iqr[k]);
    printf("gyorsulas (median-arany): %.2fx\n", med[0] / med[1]);
    rc = 0;
out:
    free(a); free(b); free(d[0]); free(d[1]);
    return rc;
}
```

A két kernel az 12.4 vázát követi; itt a teljes, önállóan összeszerelhető alak. Az `.ifdef LAB_BREAK_SCALAR` ág az ellenpróbát szolgálja, normál buildben nem fordul be:

```
// lab-a048-kernels.s  (A64 ISA, AArch64)
    .text

    .globl add_f32_scalar
    .type  add_f32_scalar, %function
add_f32_scalar:                 // x0=dst, x1=a, x2=b, x3=n
    cbz  x3, 2f
    mov  x4, #0
1:  ldr  s0, [x1, x4, lsl #2]
    ldr  s1, [x2, x4, lsl #2]
.ifdef LAB_BREAK_SCALAR
    fsub s0, s0, s1             // ELLENPRÓBA: szándékosan hibás eredmény
.else
    fadd s0, s0, s1
.endif
    str  s0, [x0, x4, lsl #2]
    add  x4, x4, #1
    cmp  x4, x3
    b.lo 1b
2:  ret
    .size add_f32_scalar, .-add_f32_scalar

    .globl add_f32_neon
    .type  add_f32_neon, %function
add_f32_neon:                   // x0=dst, x1=a, x2=b, x3=n
    lsr  x5, x3, #2             // n/4
    lsl  x5, x5, #2             // n & ~3 : a 4-tel osztható határ
    mov  x4, #0
    cbz  x5, 2f
1:  lsl  x6, x4, #2             // j*4 bájt, külön címeltolás Q-hozzáféréshez
    ldr  q0, [x1, x6]           // 4 float betöltése a-ból
    ldr  q1, [x2, x6]           // 4 float betöltése b-ből
    fadd v0.4s, v0.4s, v1.4s    // sáv-párhuzamos összeadás
    str  q0, [x0, x6]
    add  x4, x4, #4
    cmp  x4, x5
    b.lo 1b
2:  cmp  x4, x3
    b.hs 4f
3:  ldr  s0, [x1, x4, lsl #2]   // maradékciklus (n mod 4 elem)
    ldr  s1, [x2, x4, lsl #2]
    fadd s0, s0, s1
    str  s0, [x0, x4, lsl #2]
    add  x4, x4, #1
    cmp  x4, x3
    b.lo 3b
4:  ret
    .size add_f32_neon, .-add_f32_neon

    .section .note.GNU-stack,"",%progbits
```

Build és futtatás (target: qemu-user, cross-toolchain):

```
aarch64-linux-gnu-gcc -c lab-a048-kernels.s -o lab-a048-kernels.o
aarch64-linux-gnu-gcc -O2 -std=c11 -Wall -Wextra -static \
    lab-a048-bench.c lab-a048-kernels.o -o lab-a048-bench
qemu-aarch64 ./lab-a048-bench ; echo "exit=$?"
```

Hipotetikus számpélda a kimenet formájára (nem mért adat; a tényleges értékek host-, QEMU-verzió- és futásfüggők):

```
ellenorzes: add_f32_scalar OK, add_f32_neon OK
add_f32_scalar  median   12.004 ms  IQR  0.150 ms
add_f32_neon    median    5.198 ms  IQR  0.090 ms
gyorsulas (median-arany): 2.31x
exit=0
```

Mért eredmény a QEMU fő úton (`qemu-aarch64` 8.2.2, x86-64 host, $N = 2^{20}$; a jelölés `mérve (GNU + QEMU 8.2.2)`):

```
ellenorzes: add_f32_scalar OK, add_f32_neon OK
add_f32_scalar  median    5.577 ms  IQR  0.079 ms
add_f32_neon    median    3.672 ms  IQR  0.059 ms
gyorsulas (median-arany): 1.52x
exit=0
```

A fenti hipotetikus 2.31× helyett a QEMU-ban jellemzően 1.3–2× körüli arány várható; a 2.31× nem mérés, csak a formátum és az Amdahl-példa szemléltetése. Az arány emulációs, natív AArch64 hardveren külön mérendő.

Ellenpróba: hibás skaláris kernel. A teszt akkor ér valamit, ha egy hibás kernelre megbukik. Fordítsuk újra a kerneleket úgy, hogy a skaláris változat szándékosan `FSUB`-ot végezzen:

```
aarch64-linux-gnu-gcc -Wa,--defsym,LAB_BREAK_SCALAR=1 \
    -c lab-a048-kernels.s -o lab-a048-kernels-bad.o
aarch64-linux-gnu-gcc -O2 -std=c11 -Wall -Wextra -static \
    lab-a048-bench.c lab-a048-kernels-bad.o -o lab-a048-bench-bad
qemu-aarch64 ./lab-a048-bench-bad ; echo "exit=$?"
```

Várt kimenet: `FAIL add_f32_scalar: d[0] = -2.25 (vart: -3.75)`, majd `(hibás hossz: n = 1)`, végül `exit=1`; mérés nem indul. A hiba az `n = 1`-es sweep-esetből jön, ezért jelenik meg a hosszra vonatkozó második sor is. A régi, közös pufferes elrendezés ugyanerre a hibára sikert jelentett volna.

Kritikus megjegyzés a QEMU-korlátról. A `qemu-aarch64` user-mode emuláció a guest-kódot host-utasításokra fordítja, és **nem modellezi hűen a valós cache-hierarchiát, a memóriasávszélességet és a betöltési késleltetéseket**. Az itt kapott idő ezért emulációs idő, nem target-ciklusszám. A QEMU-ban kapott arányból a fizikai memóriasávszélességre nem lehet következtetni, és az sem mondható általánosan, hogy a QEMU-arány a valós hardverhez képest jobb vagy rosszabb lenne — az eltérés iránya sem garantált. Natív Raspberry Pi 4/5 alatt ugyanez a bináris valódi hardveres időt ad; a kettőt mindig külön kell jelölni.

12.5.4 Az Amdahl-törvény levezetése

Levezetés. Jelölje T_0 a skaláris változat teljes futásidejét. Bontsuk fel két részre: $p \cdot T_0$ a vektorizálható és $(1-p) \cdot T_0$ a nem vektorizálható rész. A modell szerint a vektorizálás csak az első tagra hat, és ott s-szeres gyorsulást ér el (azaz a vektorizált rész ideje pT_0/s-re csökken). Így

$$T_{\text{neon}} = (1-p)\,T_0 + \frac{p\,T_0}{s}.$$

A gyorsulás a definíció szerint:

$$S(p,s) = \frac{T_0}{T_{\text{neon}}} = \frac{T_0}{(1-p)T_0 + pT_0/s} = \frac{1}{(1-p) + p/s}.$$

Két határeset érdekes:

- **A modell határértéke**: ha $s \to \infty$ (a vektorizált rész ideje nullára csökken), akkor $S \to \dfrac{1}{1-p}$. Ez csak p-től függ: ha a futásidő 10%-ára a vektorizálás nem hat, a modell szerint a gyorsulás nem haladhatja meg a 10×-et.
- **Egyszerű sávszám-modell**: az $s = 4$ feltevéssel $S_4(p) = \dfrac{1}{(1-p) + p/4}$. Ez a feltevésből következő modellérték, nem az ISA által garantált plafon (lásd a definíciót).

Mért S és ismert p mellett a modellből visszaszámítható az az s_{eff}, amely a mérést reprodukálná:

$$s_{\text{eff}} = \frac{p}{\dfrac{1}{S} - (1-p)}.$$

Az s_{eff} **a modellbe illesztett szám**, nem mért mennyiség: értéke a feltételezett p-n múlik, és az $s_{\text{eff}}/4$ hányados nem mért sávkihasználtság. Abból, hogy $s_{\text{eff}} < 4$, önmagában sem cache-, sem sávkihasználási diagnózis nem következik.

Hogyan becsülhető p? Mivel p időhányad, utasításszámlálással (például `objdump`-listából) nem határozható meg. Lehetséges módszerek: natív hardveren profilozó (például `perf record` / `perf report`), amely megmutatja, a futásidő mekkora része esik a kernel vektorizálható ciklusára; vagy kísérleti felbontás, amelyben a nem vektorizálható részt (például a maradékciklust vagy a környező kódot) külön mérjük, és $p = 1 - T_{\text{nem vekt.}}/T_0$ — ennek is van torzítása, mert a részek átfedve futhatnak. Ha egyik módszer sem áll rendelkezésre, p feltételezett érték, és az erre épülő számítás matematikai szemléltetés.

12.5.5 Kidolgozott példa: 1M elemes tömb-összeadás

Az alábbi számítás **hipotetikus számpélda**: a fenti formátumú kimenet $T_0 = 12{,}00$ ms és $T_{\text{neon}} = 5{,}20$ ms értékét, valamint **feltételezett** $p = 0{,}95$-öt használ. Egyik szám sem mérés eredménye; a cél az Amdahl-modell kezelésének bemutatása.

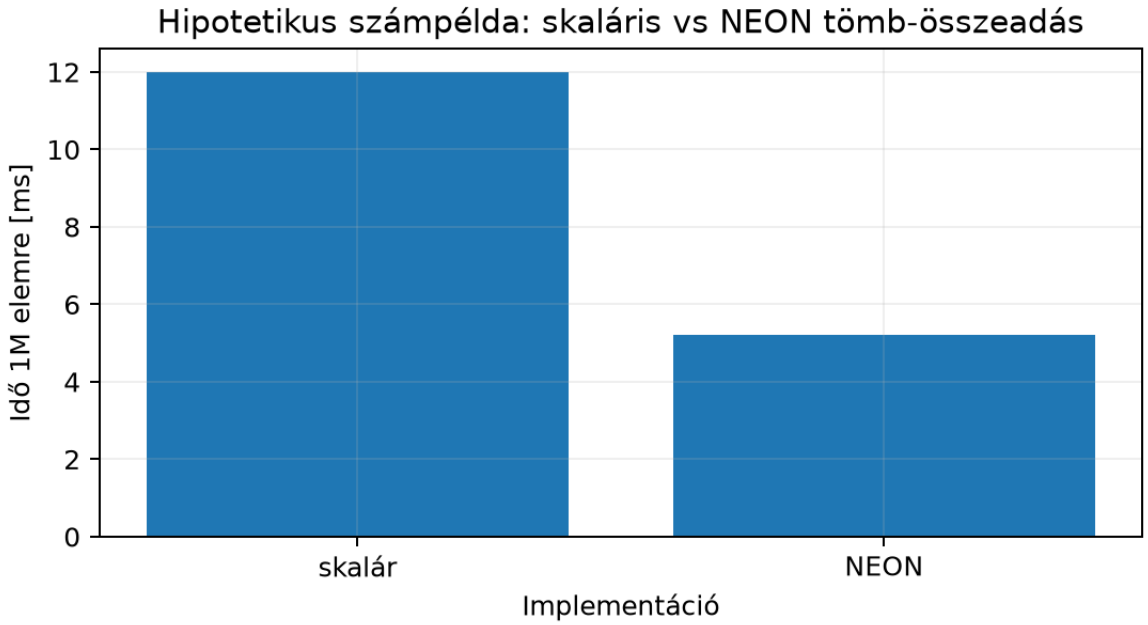
Változat	Idő [ms]	Gyorsulás	Megjegyzés
Skaláris (`add_f32_scalar`)	12,00	1,00×	hipotetikus számpélda, referencia
NEON (`add_f32_neon`)	5,20	2,31×	hipotetikus számpélda
Amdahl-modell, $p = 0{,}95$, $s = 4$	3,45	3,48×	modellérték az $s = 4$ feltevéssel
Amdahl-modell, $p = 0{,}95$, $s \to \infty$	0,60	20,0×	a modell határértéke

Az utolsó sor: $T = (1-p)\,T_0 = 0{,}05 \cdot 12{,}00 = 0{,}60$ ms, és $S = 1/0{,}05 = 20$.

A hipotetikus 2,31× a modell 3,48×-os $s = 4$ értéke alatt van. A modellbe illesztett s_{eff}:

$$s_{\text{eff}} = \frac{0{,}95}{\frac{1}{2{,}31} - 0{,}05} = \frac{0{,}95}{0{,}4329 - 0{,}05} = \frac{0{,}95}{0{,}3829} \approx 2{,}48.$$

Ez a szám csak annyit mond: **ha** $p = 0{,}95$ lenne, a vektorizált résznek kb. 2,5-szeresen kellene gyorsulnia ahhoz, hogy a megadott időket kapjuk. Más p-vel más s_{eff} adódik (lásd az 1. feladatot), ezért belőle egyik lenti ok sem tekinthető igazoltnak.



12.1. ábra: Hipotetikus számpélda: skaláris vs NEON tömb-összeadás - Hipotetikus számpélda az Amdahl-modell bemutatásához; nem mért adat, és nem hardverhű abszolút idő.

12.5.6 Az elméleti és a mért gyorsulás eltérésének tipikus okai

Ha egy *valódi* mérés elmarad az $s = 4$ modellértéktől, a következő okok jönnek szóba. Mindegyik hipotézis, amelyet natív hardveren végzett célzott méréssel kell igazolni (például n-sweep a gyorsítótár-méretek körül, teljesítményszámlálók); a fenti hipotetikus számpéldából egyik sem következik.

1. **Memóriasávszélesség (memory bandwidth).** A vektoros mag gyorsabb, de a `ldr q` / `str q` ugyanannyi bájtot mozgat, mint a skaláris változat összesen. Ha a kernel memória-kötött, a vektorizáció csak a számítási részt gyorsítja, a betöltés/kiírás idejét nem. Nagyméretű tömbökön ez gyakori ok. Fontos: **ezt a hatást a QEMU nem modellezi megbízhatóan**, ezért QEMU-mérésből nem lehet rá következtetni.
2. **Ciklusoverhead.** A NEON-változatnak is van számlálója, összehasonlítása és elágazása — igaz, negyed annyi iterációra elosztva. Kis n-nél ez dominál.
3. **Setup-költség.** Az `lsr`/`lsl` határszámítás és a maradékciklus egyszeri, fix költség. Nagy n-nél elhanyagolható, kis n-nél meghatározó.
4. **Nem vektorizálható munka.** Ha a ciklus feltételes ágakat, táblaindexelést, nem összefüggő hozzáférést vagy sávonként eltérő vezérlést tartalmaz, a p értéke jelentősen csökken, és vele együtt az elérhető S is.

Mielőtt bármelyik okot vizsgálnánk, ellenőrizzük, hogy a mért bináris valóban a kívánt kernelt futtatja: az `aarch64-linux-gnu-objdump -d --disassemble=add_f32_neon lab-a048-bench` kimenetében `ldr q`, `fadd v0.4s` és `str q` utasításoknak kell lenniük. Ha nem ezek látszanak, a mérés nem a NEON-kernelt méri (elavult objektum, rossz szimbólum, hibás linkelés — lásd a 3. feladatot).

12.5.7 Mikor NEM térül meg a vektorizáció

Nem minden vektorizálás éri meg. Négy tipikus eset:

- **Kis tömbök.** Ha n néhány tucat alatt van, a setup- és maradékciklus-költség meghaladhatja a nyereséget. A küszöb hardver- és kernel-függő; méréssel kell megállapítani, nem tippelni. Egyetlen hívás ideje ilyenkor az óra felbontásánál is kisebb lehet, ezért **kötegelt mérés** kell: K egymás utáni hívást mérünk egy időablakban (adaptívan választott K, pl. addig duplázva, amíg az ablak 1 ms), majd az időt K-val osztjuk. Az optimalizálás elleni védelmet (`volatile` nyelő, asm-akadály) és az eredményellenőrzést meg kell tartani, a ciklusvezérlés overheadjét (üres kötegciklus) külön kell mérni és közölni. Elfogadható a becslés, ha az ns/hívás a K ésszerű növelésére stabilizálódik; a jelentés a K-t és a szóródást (IQR) is megadja, nem csak egy kis különbségből számolt gyorsulási arányt.
- **Sok elágazás.** Ha a ciklusmag `if`-eket tartalmaz, a döntés a feltételek alakjától függ. Egyszerű feltételeknél a fordító gyakran ág nélküli, maszkos kiválasztássá alakítja őket (CMxx + BSL), és ilyenkor a vektoros változat gyorsabb lehet. Ha a ciklusmag elágazásai nem alakíthatók maszkos kiválasztássá (CMxx + BSL), a vektorizálás nehéz vagy nem éri meg; ilyenkor a skaláris változat predikcióbarát ága előnyben lehet.
- **Nem összefüggő (gather/scatter) hozzáférés.** A NEON szekvenciális betöltést szeret; ha az indexek futásidőben számítottak és szétszórtak, a csomagolás (packing) költsége meghaladja a nyereséget.
- **Egyszeri végrehajtás — költségmodell, nem szabály.** A „csak egyszer fut” önmagában nem dönt. A döntés mérhető feltételekből áll: a teljes munka (W elem és elemenkénti nyereség), a setupköltség (adatátrendezés, igazítás, belépési/maradék kód), az újrafelhasználás (hányszor fut ugyanaz a kernel), a memóriaforgalom (ha a kernel sávszél-korlátos, a SIMD kevesebbet nyer) és a fejlesztési/audit cél. Egyszeri, de nagy munkánál (például egy 100 MiB-os puffer egyszeri feldolgozása) a SIMD jelentős nyereséget adhat; kis egyszeri munkánál a setup dominál. A döntést a fenti »A mérési módszer« szakasz szerint, a fenti költségmodellből kell meghozni, nem a „csak egyszer fut” szabályból.

Ezekben az esetekben a helyes döntés az, hogy **mérünk, és a mérés alapján nem vektorizálunk**.

12.5.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) A hipotetikus számpélda $S = 2{,}31$ gyorsulásához vezesd le a modellbe illesztett s_{eff} értékét $p = 0{,}90$, $p = 0{,}95$ és $p = 0{,}98$ feltevéssel. Mit mond az eredmény arról, mennyire függ az s_{eff} a feltételezett p-től, és következik-e belőle bármilyen diagnózis?

2. (implementációs, középszint — cél: rutinépítés) Módosítsd a mérési vázat úgy, hogy n-t lépcsőzetesen növelje 16-tól 1 048 576-ig, és minden n-re írja ki a skaláris és NEON-időt. Készíts táblázatot, és állapítsd meg, melyik n a vektorizáció megtérülési küszöbe ezen a hoston. A `lab-a048-sweep.c` és `lab-a048-sweep` build-célnevet használd.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Egy újrafordítás után a NEON-változat mért ideje gyanúsan közel van a skalárishoz. Kolléga szerint „a GCC a `-O2` szinten átírta a NEON-kódot”. Mutasd meg, hogy ez a kézzel írt `.s` kernelre nem lehet igaz, majd keresd meg a valódi okot `aarch64-linux-gnu-nm`, `aarch64-linux-gnu-objdump -d --disassemble=add_f32_neon` és a build-időbélyegek segítségével. Milyen hibák vezethetnek ahhoz, hogy a mért bináris nem a várt kernelt futtatja?

4. **(integráló, elmélyült — cél: fejezetek összekapcsolása: Az AAPCS64 regiszterszerződés + korábbi n=0 határeset-fegyelem)** Ellenőrizd `objdump`-pal, hogy az `add_f32_scalar` és az `add_f32_neon` egyike sem nyúl az AAPCS64 szerint megőrzendő (callee-saved) regiszterekhez — `X19`–`X28`, illetve `V8`–`V15` alsó 64 bitje — anélkül, hogy `STP`/`LDP`-vel visszaállítaná őket. Ezután kövesd nyomon GDB-vel mindkét kernelt $n = 0$ bemenetre: igazold, hogy a `RET` előtt egyetlen `LDR`/`STR` sem fut le. Mi történne, ha valamelyik kernel elfelejtené ellenőrizni az $n = 0$ esetet, és már az első iterációban olvasna a (nem létező) tömbelemből?

12.5.9 Megoldások

1. megoldás — Segítség. Az $s_{\text{eff}} = p/(1/S - (1-p))$ képletbe helyettesíts be sorban három p-értéket, és figyeld, mennyit változik az eredmény.

1. megoldás — Teljes megoldás. $S = 2{,}31$ mellett $1/S = 0{,}4329$.

- $p = 0{,}90$: $s_{\text{eff}} = 0{,}90/(0{,}4329 - 0{,}10) = 0{,}90/0{,}3329 \approx 2{,}70$.
- $p = 0{,}95$: $s_{\text{eff}} = 0{,}95/(0{,}4329 - 0{,}05) = 0{,}95/0{,}3829 \approx 2{,}48$.
- $p = 0{,}98$: $s_{\text{eff}} = 0{,}98/(0{,}4329 - 0{,}02) = 0{,}98/0{,}4129 \approx 2{,}37$.

Ugyanabból az S-ből a feltételezett p-től függően más-más s_{eff} adódik; az s_{eff} tehát a modell paramétere, nem mérési eredmény. Egyik érték sem igazol memóriasávszélesség-, cache- vagy sávkihasználási problémát: ehhez p-t időalapú módszerrel kellene megmérni (lásd „Hogyan becsülhető p?”), és a feltételezett okot célzott, natív hardveres méréssel kellene ellenőrizni. Az $s_{\text{eff}} < 4$ az $s = 4$ modellfeltevéssel sem áll ellentmondásban, mert az csak modell, nem korlát.

2. megoldás — Segítség. A mérés a `main`-ben inline áll, a méretet a fordítási idejű `N` makró adja; ezért a sweephez a programot különböző `-DN=…` értékekkel kell újrafordítani. A küszöböt ott keresd, ahol a NEON-idő először lesz kisebb a skalárisnál.

2. megoldás — Teljes megoldás. A mérés a `main`-ben inline áll, ezért a sweepet a `-DN=` makróval, külön binárisokkal kell futtatni:

```
for n in 16 64 256 1024 4096 16384; do
    aarch64-linux-gnu-gcc -O2 -std=c11 -Wall -Wextra -static -DN="$n" \
        lab-a048-bench.c lab-a048-kernels.o -o "lab-a048-bench-$n"
    echo "n=$n"
    qemu-aarch64 "./lab-a048-bench-$n"
done
```

A küszöb jellemzően ott van, ahol a NEON-idő először lesz kisebb a skalárisnál; tipikusan néhány száz és néhány ezer elem közé esik, de konkrét értéke host- és QEMU-verziófüggő, ezért **méréssel kell megállapítani**. A helyes jelentés: „ezen a hoston, ezzel a kernellel, ezen a bemeneteloszláson”. Nem általános érvényű szám.

3. megoldás — Segítség. A `.s` fájlt a GCC csak továbbadja az assemblernek — melyik lépésben lenne alkalma bármit „átírni”? Utána nézd meg, honnan jön a binárisban az `add_f32_neon` szimbólum.

3. megoldás — Teljes megoldás. A kézzel írt `lab-a048-kernels.s` a GCC-től csak az assemblerhez jut: a `-O2` a C-forrásokra vonatkozik, a `.s` utasításait sem nem vektorizálja, sem nem alakítja vissza skaláris `FADD`-dá. Ha az `add_f32_neon` alatt a binárisban nem `fadd v0.4s` áll, a valódi ok a buildben keresendő:

- **elavult objektum:** a `lab-a048-kernels.o` régebbi, mint a `.s` (a `make` függőségi sora hiányzik, vagy kézi build után nem fordítottuk újra) — `ls -l --time-style=full-iso` a két fájlra;
- **rossz objektum vagy másik függvény:** egy másik `.o` (például az 12.4 könyvtárából átmásolt fájl) definiál ugyanilyen nevű függvényt — ezt előzi meg a különböző `add_i32_*`/`add_f32_*` név; `aarch64-linux-gnu-nm -A *.o | grep add_` megmutatja, melyik objektum definiálja;
- **hibás linkelés:** a linkparancs nem a várt objektumot kapja, vagy egy könyvtár elsőbbséget élvez — a `-Wl,-Map=lab-a048.map` linkertérkép megmutatja, honnan jött a szimbólum.

Javítás: tiszta újrafordítás (`rm -f *.o lab-a048-bench`, majd a build), és az `objdump -d --disassemble=add_f32_neon` ellenőrzése a mérés előtt. Az autovektorizáció vizsgálatához külön, C-ben írt kernelek kellenek (például `-O2` és `-O3 -fopt-info-vec` összevetése); kézzel írt assemblyn ez a kérdés nem értelmezhető.

4. megoldás — Segítség. Nézd meg, mely regisztereket érinti ténylegesen a két kernel (a forrás alapján `X0`–`X6` és `V0`–`V1`), és vesd össze az AAPCS64 regiszterszerződés callee-saved listájával — esik-e bármelyik abba a tartományba? Az $n = 0$ nyomkövetéséhez tegyél töréspontot a kernel elejére és a `RET` elé.

4. megoldás — Teljes megoldás. Az `add_f32_scalar` és az `add_f32_neon` kizárólag `X0`–`X6` (az `X6` a NEON-ág bájteltolása) és `V0`–`V1` regisztereket használ — ezek egyike sem szerepel az AAPCS64 callee-saved listáján (`X19`–`X28`, `V8`–`V15`), ezért egyik kernelnek sem kell `STP`/`LDP`-vel megőriznie semmit; az `objdump -d` kimenetében emiatt nem is várunk ilyen utasítást. Az $n = 0$ esetben: az `add_f32_scalar` `cbz x3, 2f`-je azonnal a `ret`-hez ugrik; az `add_f32_neon`-ban a `lsr`/`lsl` 0-t számol ki `x5`-nek, a `cbz x5, 2f` idevisz, majd a `cmp x4, x3` / `b.hs 4f` (0 és 0 egyenlő, a „higher or same” ilyenkor is teljesül) szintén a `ret`-hez vezet — egyik ágban sem fut le `ldr`/`str`. Ha ez a határellenőrzés hiányozna, az első iteráció egy 0 méretű tömb nemlétező elemét olvasná: ez kisebb léptékben ugyanaz a hibaosztály, amit a Záróprojekt I `n*n*sizeof(float)`-túlcsordulás- és `n==0`-ellenőrzése tárgyal (15.1–15.4) — ott a hiányzó korai kilépés egy egész mátrixszorzó modult tenne memóriabiztonsági szempontból megbízhatatlanná, itt egy két-utasításos kernelt.

12.5.10 Rövid összefoglaló

A NEON-gyorsulás mérése azonos bemeneten, kernelenként külön kimeneti pufferrel és a méréstől elválasztott korrektségi ellenőrzéssel, melegítés után, váltakozó sorrendű ismételt futással, medián és IQR közlésével történik; a teszt értékét egy szándékosan hibás kernel ellenpróbája igazolja. Az Amdahl-modell $S = 1/((1-p) + p/s)$ alakban írja le a gyorsulást, ahol p **időhányad**, nem utasításszám-hányad, és csak időalapú módszerrel becsülhető. A modell határértéke $1/(1-p)$; az $s = 4$ egyszerű modellfeltevés, nem az ISA-ból következő felső korlát. A mért S-ből visszaszámolt s_{eff} a feltételezett p-n múlik, ezért nem mért sávkihasználtság, és diagnózist sem igazol. QEMU-ban a kapott idő emulációs idő; belőle fizikai memóriasávszélességre nem lehet következtetni. Kis tömböknél, sok elágazásnál és szétszórt hozzáférésnél a vektorizáció jellemzően nem térül meg — ezt is mérés dönti el.

Továbbhasznált definíciók: **gyorsulás (speedup)**; **vektorizálható hányad (vectorizable fraction)**, mint időhányad; **Amdahl-modell**.

12.5.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Architecture Reference Manual for A-profile architecture (DDI0487) — az `FADD (vector)`, `LDR (immediate, SIMD&FP)` és `STR (immediate, SIMD&FP)` kódolásának, valamint a `q`-regiszteres hozzáférés igazítási követelményeinek elsődleges forrása. *Ellenőrizve: a link él* (a `developer.arm.com` → `support.arm.com` átirányítás megerősített); a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött — a konkrét utasításkódolást a laborban kézzel kell visszakeresni.
- Arm Neon intrinsics referencia — a NEON belső műveletek listája és szemantikája. *Ellenőrizve: a link él*; ugyanaz a JS-renderelési korlát érvényes, mint fent.
- QEMU dokumentáció, user-mode fejezet — **tartalmilag is ellenőrizve**: a `master` ághoz tartozó hivatalos dokumentáció nem állít semmit a cache-hierarchia, a memóriasávszélesség vagy a betöltési késleltetés modellezéséről — a fejezet lényegében a rendszerhívás-fordítást és a szál-/memóriamodell host–guest eltéréseit tárgyalja. A fenti, „QEMU nem modellezi hűen a cache-hierarchiát” állítás ezért nem egy explicit QEMU-dokumentált korlátból, hanem a TCG (Tiny Code Generator) ismert tervezési elvéből következik: funkcionális, utasításszintű emulátor, nem cikluspontos szimulátor. A hivatkozott dokumentáció emellett a `master`, nem egy konkrét kiadott verzió állapotát tükrözi; a ténylegesen telepített (pl. Ubuntu 24.04 csomagolt) QEMU-verzió dokumentációja ettől eltérhet.
- GNU Binutils kézikönyv — `objdump -d` és a többi binutils-eszköz referenciája. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv — a `disassemble` és a távoli hibakeresés referenciája. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).
- GCC kézikönyv — a `-O2` és az AArch64-specifikus kapcsolók referenciája. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2. (Korábban ez a hivatkozás tévesen a Binutils-kézikönyvvel egy közös link alá volt sorolva — a GCC a Binutilstól és a GDB-től független projekt, külön dokumentációval.)

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak:

1. A QEMU pontos időzítésmodellje — mennyire tér el egy adott natív host mért idejétől — sem ebben, sem korábban nincs ellenőrizve; ehhez tényleges mérés kellene fizikai hoston és QEMU alatt egyaránt, amire ennek a könyvnek a készítése során nem került sor.
2. A `p = 0,95` vektorizálható hányad feltételezett érték, az erre épülő számítás matematikai szemléltetés. A tényleges p időhányad, amelyet csak időalapú módszerrel (profilozás natív hardveren, a részek külön mérése) lehet becsülni; utasításszámlálással nem.
3. A kimenetminta, a táblázat és az ábra idő- és gyorsulásértékei (12,004 ms, 5,198 ms, 2,31×) **hipotetikus számpéldák** — nem fizikai hoston vagy QEMU-ban lefuttatott mérés eredményei. A benchmark logikáját (külön ellenőrzés, a hibás skaláris kernel `FAIL` ellenpróbája) helyettesítő eszközlánccal — `zig cc` (Clang, musl), Unicorn-alapú AArch64 user-mode emulátorban, kisebb n-nel — futtattuk; az ott kapott időket szándékosan nem közöljük, mert az emulátor ideje semmilyen hardverre nem jellemző.
4. A telepített Ubuntu 24.04 LTS csomagolt `gcc-aarch64-linux-gnu`, `qemu-user` és `binutils` pontos verziószáma itt nincs lekérdezve; a fenti dokumentációs linkek ezért nem feltétlenül a laborban ténylegesen használt verzióra mutatnak.
5. A „mikor nem térül meg” küszöböket (kis tömbök, sok elágazás) méréssel kell megerősíteni; az itt szereplő tartományok tájékoztató jellegűek.

13 Teljesítmény-tudatos kód: cache, pipeline és mérés

13.1 Cache-hierarchia és a memóriahozzáférési minták hatása

13.1.1 Motiváció: ugyanaz a ciklus, többszörös idő

Vegyünk két olyan assembly rutint, amelyek utasításszámban, elágazásban és regiszterhasználatban szinte azonosak, mégis drasztikusan eltérő futási időt mutatnak. Mindkettő egy nagy `int` tömböt jár be: az egyik minden elemet sorban érint, a másik minden lépésben több ezer bájtot ugrik. A különbség oka nem az utasításokban, hanem a memóriahozzáférési mintában rejlik. Ez az alfejezet megmutatja, hogyan épül fel a cache-hierarchia, miért számít a sorrend, és hogyan mérhető ez assembly szinten — a korlátok őszinte jelölésével.

Az 7.2 alfejezetben már láttuk, hogyan indexelünk egy tömböt skálázott regiszterelтolással. Most azt vizsgáljuk, mitől lesz egy ilyen bejárás gyors vagy lassú.

13.1.2 Definíció: cache, cache-sor, lokalitás

A **cache** (gyorsítótár) egy kicsi, gyors memória a processzor és a főmemória között. Célja, hogy a gyakran használt adatokat közel tartsa a maghoz, mert a főmemória (DRAM) elérése nagyságrendekkel lassabb.

A cache nem bájtokként, hanem **cache-sorként** (cache line) mozgat adatot. Egy cache-sor tipikusan 64 bájt. Amikor a processzor egy bájtot kér, a hardver a teljes sort betölti a cache-be. Ez a **térbeli lokalitás** (spatial locality) kihasználása: ha egy címre hivatkozunk, valószínűleg a szomszédos címekre is hamarosan szükség lesz.

A **temporális lokalitás** (temporal locality) ezzel rokon fogalom: ha egy adatot egyszer elértünk, rövid időn belül újra elérhetjük, ezért érdemes a cache-ben tartani. A mi vizsgálataink elsősorban a térbeli lokalitásra épülnek.

13.1.3 Cache-hierarchia

A modern processzorok több cache-szintet használnak:

Szint	Jellemző méret	Jellemző elérési idő	Megosztás
L1	16–128 KB	1–4 ciklus	magonként, I- és D-cache külön
L2	128 KB – 8 MB	10–20 ciklus	magonként
L3	4–64 MB	30–50 ciklus	magok között / csomagon
DRAM	GB nagyságrend	100–300 ciklus	rendszer

A táblázat értékei **jellemző tartományok**, nem konkrét hardverre vonatkozó mérések. A pontos méret és latencia processzorfüggő, ezért minden konkrét állítást hardverhez kell kötni.

A hierarchia lényege: minél közelebb van az adat a maghoz, annál gyorsabb az elérés, de annál kisebb a kapacitás. Az L1 tipikusan két részre oszlik: utasítás-cache (I-cache) és adat-cache (D-cache). Az itteni vizsgálatok az adat-cache-re vonatkoznak.

13.1.4 Cache-sor és térbeli lokalitás

Tegyük fel, hogy egy 64 bájtos cache-sorba 16 darab 32 bites `int` fér bele. Sorrendi olvasásnál:

```
elem 0, 1, 2, ..., 15   -> egy cache-sor betöltése, mind a 16 elem elérhető
elem 16, 17, ..., 31    -> következő cache-sor betöltése
```

Minden betöltött cache-sor **16 elemet szolgálhat ki**. Ez a térbeli lokalitás lehetősége: a vizsgált cache-szinten a sorbetöltések száma legfeljebb az elemszám 1/16-a. Hogy ebből ténylegesen hány DRAM-tranzakció lesz, azt a sorméret egyedül nem határozza meg. Számít, melyik cache-szintről beszélünk, mi volt korábban a cache-ben, mit tölt be előre a hardveres előbetöltő (prefetcher), és mekkora az asszociativitás.

Nagy stride-ú ugrálásnál viszont:

```
elem 0, 1024, 2048, ...  -> minden hozzáférés új cache-sor betöltését igényli
```

Egyetlen ilyen végigfutásban minden hozzáférés másik cache-sort érint, és abból csak egy elemet használunk: a betöltött 64 bájtból 60 bájt kihasználatlan, hacsak a sor nem marad a cache-ben addig, amíg a bejárás egy későbbi menete a szomszédos elemekért visszatér.

13.1.5 Kidolgozott példa: sorrendi vs. stride-os bejárás

Munkakönyvtár: `~/arm-labor/a64/a049/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a049
cd ~/arm-labor/a64/a049
```

Írjunk egy assembly programot, amely egy nagy tömböt sorrendben, majd nagy stride-pal jár be. A mérés QEMU-emulációban fut, ahol a cache-viselkedés nem hardverhű — ezt a példa után külön kiemeljük.

13.1.6 Sorrendi bejárás (A64 ISA)

```
// lab-seq-a049.s - sorrendi bejárás
// AArch64 (A64 ISA), Linux, qemu-aarch64 user-mode

    .section .data
    .balign 64              // cache-sor határára igazítunk
arr:
    .fill 1048576, 4, 1     // 4 MB, 1 048 576 darab 32 bites int, mind 1
    .set ARR_SIZE, 1048576

    .section .text
    .global _start
_start:
    mov x0, #0              // összeg
    ldr x1, =arr            // tömb alapcíme
```

```
    mov x2, #ARR_SIZE       // elemek száma
    mov x3, #0              // index

seq_loop:
    cmp x3, x2
    b.hs seq_done
    ldr w4, [x1, x3, lsl #2] // w4 = arr[x3]
    add x0, x0, x4           // összegzés
    add x3, x3, #1
    b seq_loop

seq_done:
    cmp x0, x2
    cset w0, ne
    mov x8, #93             // exit syscall
    svc #0

    .section .note.GNU-stack,"",%progbits
```

13.1.7 Stride-os bejárás (A64 ISA)

```
// lab-stride-a049.s - stride-os bejárás
// AArch64 (A64 ISA), Linux, qemu-aarch64 user-mode

    .section .data
    .balign 64
arr:
    .fill 1048576, 4, 1
    .set ARR_SIZE, 1048576
    .set STRIDE, 1024       // 1024 * 4 bájt = 4096 bájt ugrás

    .section .text
    .global _start
_start:
    mov x0, #0
    ldr x1, =arr
    mov x2, #ARR_SIZE
    mov x5, #0              // oszlop a stride szerinti bejarashoz
stride_column:
    mov x3, x5

stride_loop:
    cmp x3, x2
    b.hs stride_next_column
    ldr w4, [x1, x3, lsl #2]
    add x0, x0, x4
    add x3, x3, #STRIDE
    b stride_loop

stride_next_column:
    add x5, x5, #1
    cmp x5, #STRIDE
    b.lt stride_column
stride_done:
    cmp x0, x2
    cset w0, ne
    mov x8, #93
    svc #0

    .section .note.GNU-stack,"",%progbits
```

13.1.8 Build és futtatás

```
# Assemble és link
aarch64-linux-gnu-as -o lab-seq-a049.o lab-seq-a049.s
aarch64-linux-gnu-ld -o lab-seq-a049 lab-seq-a049.o
aarch64-linux-gnu-as -o lab-stride-a049.o lab-stride-a049.s
aarch64-linux-gnu-ld -o lab-stride-a049 lab-stride-a049.o
# elvárt: GNU_STACK RW
readelf -lW lab-seq-a049 \
  | awk '$1=="GNU_STACK"{f=$7} END{print "GNU_STACK", f; exit !(f=="RW")}'
readelf -lW lab-stride-a049 \
  | awk '$1=="GNU_STACK"{f=$7} END{print "GNU_STACK", f; exit !(f=="RW")}'

# Futtatás QEMU user-mode-ban
qemu-aarch64 ./lab-seq-a049
echo $?   # várt: 0
qemu-aarch64 ./lab-stride-a049
echo $?   # várt: 0
```

13.1.9 Mérés

Az időméréshez tisztán assemblyben a `clock_gettime` syscall használható, de bonyolult. Egyszerűbb a shell `time` parancsa vagy a `perf` eszköz. A két program futásidejének összehasonlításához például:

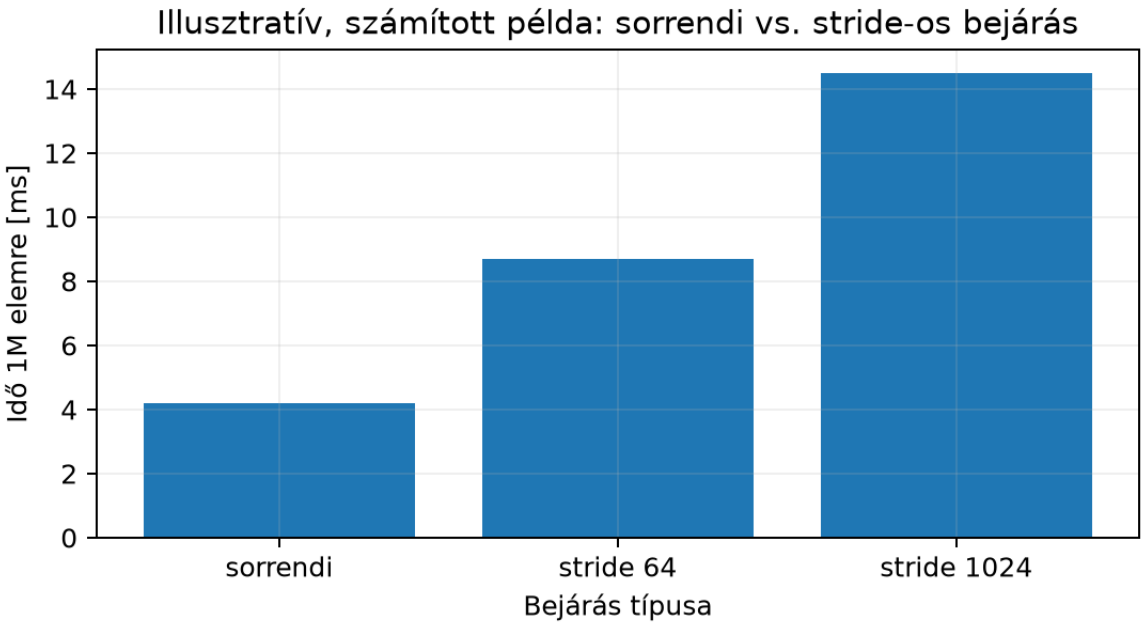
```
time qemu-aarch64 ./lab-seq-a049
time qemu-aarch64 ./lab-stride-a049
```

Az alábbi táblázat **hipotetikus/illusztratív** értékeket mutat, mert QEMU-emulációban a cache-viselkedés nem hardverhű.

Bejárás	Feldolgozott elemek	Idő (ms)	Megjegyzés
Sorrendi	1 048 576	4,2	illusztratív
Stride 1024	1 048 576	14,5	minden elem egyszer, oszloponként
Stride 64	1 048 576	8,7	minden elem egyszer, oszloponként

Fontos: mindkét program azonos számú elemet dolgoz fel. A stride-os változat külső ciklusa az összes maradékosztályt bejárja (`0, STRIDE, ...`, majd `1, 1+STRIDE, ...`), ezért minden elem pontosan egyszer szerepel; ennek levezetését és ellenőrzését lásd lent. Az idők szemléltető számok; a sorrend és a különbség tényleges mérést igényel.

Mérve (QEMU 8.2.2, a lektor mérése): a teljes folyamat ideje indítással együtt sorrendi bejárásnál 8–10 ms, stride 1024-nél 15–20 ms. A különbség QEMU alatt is látható, de ez **host-hatás**: a QEMU TCG-je és a host cache-e alakítja ki, nem az Arm-cél cache-viselkedése.



13.1. ábra: Illusztratív, számított példa: sorrendi vs. stride-os bejárás - Hipotetikus szemléltetés, nem QEMU- vagy hardvermérés. Azonos elemszám mellett becsült relatív viszony.

13.1.10 Fair összehasonlítás: azonos elemszám

A fenti stride-os verzió a kezdőindexet változtatja (`0, 1024, 2048, ...`, majd `1, 1025, 2049, ...`). Hogy így minden elem pontosan egyszer szerepel, azt nem a futás eredménye, hanem a ciklusok szerkezete igazolja.

- **Sorrendi bejárás.** Ciklusinvariáns: a `seq_loop` elején `x3` a következő olvasandó index, és a `0 … x3-1` indexeket a ciklus már pontosan egyszer olvasta. `x3 = 0`-ról indul, lépésenként 1-gyel nő, és `x3 = N`-nél kilép; így a `0 … N-1` mindegyike pontosan egyszer kerül sorra, összesen `N` olvasás.
- **Stride-os bejárás.** A külső ciklus `c = x5` értékei `0 … d-1` (d = `STRIDE`), a belső ciklus a `k = c + d·j` indexeket olvassa, amíg `k < N`. A $(c, j) \mapsto k = c + dj$ leképezés a bejárt párokon **bijektív** a `0 … N-1` tartományra: minden `k`-hoz a maradékos osztás egyetlen párt ad, `c = k mod d` és `j = k div d`, és ez a pár valóban sorra kerül, mert `c < d`, és `k < N`. Ez bármely $d \geq 1$ mellett igaz, mert a belső ciklus a `k < N` feltételnél áll meg. Az olvasások száma tehát pontosan `N`, mint a sorrendi bejárásnál.

Az oszloponkénti sorrend tehát ugyanazt a munkát, ugyanannyi olvasással végzi el, csak más sorrendben; a cache-sorok újratöltése és a TLB-viselkedés miatt a sorrendi bejárásnál lassabb lehet.

13.1.11 Várható kimenet és ellenőrzés

- A sorrendi bejárás összege: `1 048 576` (minden elem 1).
- A stride-os bejárás összege is `1 048 576`.
- Az exit kód helyes összegnél `0`, eltérésnél `1`.

A `0` tehát **csak annyit** igazol, hogy az összeg az elvárt `N`. Azt nem, hogy minden elemet pontosan egyszer olvastunk. Csupa 1-es adaton egy olyan hibás rutin is `N`-et ad, amely `N`-szer ugyanazt az `arr[0]`-t olvassa, és ugyanígy láthatatlan marad egy kihagyott elem, ha helyette egy másikat kétszer olvasunk. A helyettesítő eszközlánccal kipróbálva a `lab-seq-a049.s` olyan mutánsa, amelyben a `ldr w4, [x1, x3, lsl #2]` helyére `ldr w4, [x1]` került, `0`-s állapottal lépett ki: ez **hamis PASS**. Az összeg egyezéséből a dinamikus munka mennyisége sem következik, hiszen fölösleges vagy elhagyott, majd algebrailag pótolt olvasásokat nem különít el.

A lefedettséget a fenti levezetés igazolja. Oktatási ellenpróbaként egy külön C-program ugyanazokat az indexsorozatokat járja be, **heterogén** adattal (`a[i] = i + 1`, 64 bites összeg) és **explicit látogatásszámlálóval**. A számláló minden olvasásnál növeli az adott index `visits[]` értékét, a végén pedig külön jelzi a hiányzó és a többször olvasott indexeket, valamint az `N`-től eltérő olvasásszámot. A mért assembly programokba nem teszünk ilyen nyomkövetést, hogy a mért ciklus változatlan maradjon:

```c
/* lab-cover-a049.c - a két bejárás lefedettségi próbája (oktatási ellenpróba, host C).
 * Az indexsorozatok pontosan a lab-seq-a049.s és a lab-stride-a049.s ciklusait
 * követik; a mutánsok szándékosan hibásak. Minden olvasás a visits[] számlálót is
 * növeli: ebből derül ki a kihagyás, a duplikáció és az eltérő olvasásszám. */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#define N      1048576u
#define STRIDE 1024u

static uint32_t a[N];
static uint8_t visits[N];
static uint64_t reads, sum;

static void visit(uint32_t k)            /* egy olvasás: adat + látogatási nyom */
{
    sum += a[k];
    reads++;
    if (visits[k] < 255u)
        visits[k]++;
}

static void seq(void)       { for (uint32_t k = 0; k < N; k++) visit(k); }
static void stride(void)                 /* k = c + STRIDE·j, oszloponként */
{
    for (uint32_t c = 0; c < STRIDE; c++)
        for (uint32_t k = c; k < N; k += STRIDE)
            visit(k);
}
/* szándékosan hibás változatok */
static void mut_a0(void)    { for (uint32_t k = 0; k < N; k++) visit(0); }   /* mindig a[0] */
static void mut_skipdup(void)                    /* a[5] helyett a[4]: kihagyás + duplikáció */
{
    for (uint32_t k = 0; k < N; k++) visit(k == 5u ? 4u : k);
}
static void mut_extra(void) { seq(); visit(7); }                      /* egy többletolvasás */

static int run(const char *name, void (*f)(void))
{
    for (uint32_t k = 0; k < N; k++) visits[k] = 0;
    reads = sum = 0;
    f();
    uint64_t missing = 0, dup = 0;
    for (uint32_t k = 0; k < N; k++) {
        missing += visits[k] == 0;
```

```
        dup += visits[k] > 1;
    }
    const uint64_t want = (uint64_t)N * (N + 1u) / 2u;    /* a[i] = i + 1 összege */
    int ok = reads == N && missing == 0 && dup == 0;
    printf("%-14s olvasás=%-8llu hiányzó=%-8llu többször=%-3llu összeg %s  %s\n", name,
           (unsigned long long)reads, (unsigned long long)missing,
           (unsigned long long)dup, sum == want ? "egyezik" : "ELTÉR", ok ? "OK" : "HIBA");
    return ok;
}

int main(void)
{
    for (uint32_t k = 0; k < N; k++)
        a[k] = k + 1u;                    /* heterogén adat: minden elem különböző */
    int ok = run("sorrendi", seq) & run("stride", stride);
    puts("-- szándékosan hibás változatok (HIBA az elvárt):");
    int bad = !run("a[0] N-szer", mut_a0) & !run("kihagy+dupláz", mut_skipdup)
            & !run("többletolvasás", mut_extra);
    puts(ok && bad ? "COVER: PASS" : "COVER: FAIL");
    return !(ok && bad);
}
```

```
gcc -std=c11 -O2 -Wall -Wextra -o lab-cover-a049 lab-cover-a049.c
./lab-cover-a049; echo "kilépési állapot: $?"
```

Kimenet (host `gcc` 13.3, mért):

```
sorrendi       olvasás=1048576  hiányzó=0        többször=0   összeg egyezik  OK
stride         olvasás=1048576  hiányzó=0        többször=0   összeg egyezik  OK
-- szándékosan hibás változatok (HIBA az elvárt):
a[0] N-szer    olvasás=1048576  hiányzó=1048575  többször=1   összeg ELTÉR  HIBA
kihagy+dupláz olvasás=1048576  hiányzó=1        többször=1   összeg ELTÉR  HIBA
többletolvasás olvasás=1048577  hiányzó=0        többször=1   összeg ELTÉR  HIBA
COVER: PASS
kilépési állapot: 0
```

A két helyes bejárás minden indexet pontosan egyszer érint. A mutánsok közül az `a[0]`-t `N`-szer olvasó változatnál a látogatásszámláló `N - 1` hiányzó indexet mutat; a kihagyás és duplikáció egy-egy hiányzó és többször olvasott indexet ad; a többletolvasásnál az olvasásszám `N + 1`. A heterogén összeg itt mindhárom esetben eltér, de önmagában csak egy kijelölt hibacsaládot tesztel: egy összegtartó csere (például két elem helyett két másik, azonos összegű elem olvasása) mellett egyezhetne. Ezért a döntő ellenőrzés a látogatásszámláló, a teljes bizonyíték pedig a levezetés.

Mit mér a `perf` QEMU alatt? A `qemu-aarch64` egy x86_64 hostfolyamat, amely a vendég A64-kódját host-gépi kódra fordítja (TCG), és azt futtatja. Ezért az alábbi parancs **kizárólag hostoldali emulátormérés**:

```
# HOST-mérés: az x86_64 host CPU PMU-jának cache-references / cache-misses eseményei,
# a qemu-aarch64 folyamatra (fordító + generált hostkód + emulált memória) - NEM ARM-mérés.
perf stat -e cache-references,cache-misses qemu-aarch64 ./lab-seq-a049
```

A kapott számok a host processzor cache-eseményei: a QEMU saját kódjának, a generált hostkódnak és a vendégmemóriát tároló hosttömböknek a hozzáférései. Ezek nem az ARM-cél cache-missei, nem is azok közelítései vagy helyettesítői, és a relatív arányukból sem következik az ARM-cél cache-viselkedése. QEMU-n a labor **funkcionális** ellenőrzése marad: a kilépési állapot (`0`) az elvárt összeget igazolja; a teljes, egyszeri bejárást a fenti levezetés és a lefedettségi próba.

A cél hardverének cache-vizsgálata natív AArch64 Linuxon (például Raspberry Pi 4/5-ön) történik, az ott elérhető PMU-eseményekkel:

```
# NATÍV AArch64-mérés: az adott Arm mag PMU-jának L1D_CACHE (hozzáférés) és
# L1D_CACHE_REFILL (újratöltés) eseménye; a perf ezeket általában
# L1-dcache-loads / L1-dcache-load-misses néven képezi le - perf list-tel ellenőrizendő.
perf stat -e L1-dcache-loads,L1-dcache-load-misses ./lab-seq-a049
perf stat -e L1-dcache-loads,L1-dcache-load-misses ./lab-stride-a049
```

Minden számot azzal együtt kell rögzíteni, hogy melyik processzor (mag típusa) melyik eseményét mérte.

13.1.12 Tipikus hibák és ellenpróbák

1. **Nem 64 bájtos igazítás.** A tömböt `.balign 64`-gyel igazítjuk, hogy a cache-sorok határain ne csússzunk el. Igazítás nélkül a tömb eleje egy sor közepére esik, így egy-két további sort érintünk – ez itt elhanyagolható, de kis, sorhatárra érzékeny objektumoknál (vektoros betöltés, megosztott számlálók) számít.
2. **Azonos setre leképeződő sorok.** Két külön jelenséget kell szétválasztani. Ha a lépés kisebb a sorméretnél, az egymás utáni hozzáférések **ugyanabban a sorban** maradnak: ez soron belüli lokalitás, nem ütközés. Ha a lépés több sort ugrik át, a hozzáférések **különböző sorokat** érintenek, és az a kérdés, hány különböző setre képeződnek le. Egyszerű, közvetlen indexelésű modellben a set indexe $\lfloor \text{cím}/L \rfloor \bmod S$, ahol L a sorméret és S a setek száma. Egy k soros lépés ($k \geq 1$) $S/\gcd(k, S)$ különböző setet jár be. Ha ez kevés, és egy setre több különböző sor jut, mint az asszociativitás, akkor a sorok kiszorítják egymást (**conflict miss**, cache-thrashing).

   Konkrét modell: $L = 64$ bájt, $S = 64$ set, 8 utas asszociativitás (32 KiB, egy tipikus L1D mérete).

Lépés	Sorban mért lépés k	Az első hozzáférések (sor, set)	Bejárt setek	Jelenség
4 B	1/16	(0,0), (0,0), (0,0), … 16-szor, majd (1,1)	mind a 64	soron belüli lokalitás
64 B	1	(0,0), (1,1), (2,2), (3,3), …	64	minden hozzáférés új sor, a setek egyenletesen telnek
128 B	2	(0,0), (2,2), (4,4), …	$64/\gcd(2, 64) = 32$	minden második sor; a setek fele
4096 B	64	(0,0), (64,0), (128,0), (192,0), …	$64/\gcd(64, 64) = 1$	minden hozzáférés ugyanarra az egy setre esik; 8 sor után kiszorítás

   A 4 bájtos és a 4096 bájtos lépés tehát nem ugyanaz az eset, pedig mindkettő kettő hatványa. Az egyik soron belül marad, a másik azonos setre leképeződő különböző sorokat érint. Pusztán abból, hogy a lépés kettő hatványa, nem következik ütközés: a k és az S közös osztói döntenek. A valós hardver gyakran hash-elt setindexet, fizikai címzést és többszintű,

eltérő geometriájú cache-eket használ, ezért ez a levezetés csak a megadott modellen belül érvényes, és natív méréssel ellenőrizendő.

3. **A tömb mérete kisebb, mint a cache.** Ha a tömb elfér az L1 cache-ben, a sorrendi és a stride-os bejárás között kicsi lesz a különbség. A méréshez **nagy** tömb kell (legalább az L2-nél nagyobb).
4. **QEMU-ban mért értékek összekeverése valós hardverrel.** A QEMU TCG-je funkcionálisan emulál, cache-hierarchiát nem modellez. A QEMU alatt mért idők és a `perf`-számlálók a **host** gépre vonatkoznak, ezért a relatív sorrendjük sem jelzi az ARM-cél viselkedését. QEMU-n csak a funkcionális helyesség és az azonos munkamennyiség ellenőrizhető. Erről bővebben a Mikrobenchmark készítése és eredmények értelmezése alfejezetben lesz szó.

13.1.13 Feladatok

A feladatcímkék és a kétlépcsős megoldások használatát az Olvasói útmutató írja le.

13.1.14 1. feladat (magyarázó, alapszint — cél: megértés-ellenőrzés)

Magyarázd el, miért hatékonyabb a sorrendi bejárás, mint a nagy stride-ú bejárás, a cache-sor fogalmát használva. Térj ki a térbeli lokalitásra is.

13.1.15 2. feladat (implementációs, középszint — cél: rutinépítés)

Módosítsd a stride-os assembly programot úgy, hogy a stride értéke 32 legyen (128 bájt). Hasonlítsd össze az eredményt a stride 1024-es verzióval. Mit tapasztalsz?

13.1.16 3. feladat (hibakeresési/mérési, középszint — cél: fejezetek összekapcsolása: Mikrobenchmark készítése és eredmények értelmezése)

Írj egy C programot, amely sorrendben és stride-pal is bejár egy nagy tömböt, és méri az időt `clock_gettime`-nal, az 13.4 mikrobenchmark-vázának mintáját követve (ismételt futás, medián, szóródás). Futtasd QEMU-ban, és értelmezd az eredményt. Miért nem hardverhű a QEMU-ban mért érték?

13.1.17 Megoldások

1. megoldás — Segítség. Egy cache-sor 64 bájt, egy `int` 4 bájt — hány elemet szolgál ki EGYETLEN cache-sor-betöltés sorrendi bejárásnál, és hányat nagy stride-nál?

1. megoldás — Teljes megoldás. A cache-sor 64 bájt, ami 16 darab 32 bites `int`-et tartalmaz. Sorrendi bejárásnál egy cache-sor betöltése után a következő 15 elem is elérhető a cache-ből, így a vizsgált cache-szinten a sorbetöltések száma legfeljebb az elemszám 1/16-a. Ez a térbeli lokalitás lehetősége; a tényleges DRAM-forgalom a cache-szinttől, az előbetöltéstől és a korábbi állapottól is függ. Nagy stride-ú bejárásnál minden hozzáférés új cache-sor betöltését igényli, de abból csak egy elemet használunk, így a térbeli lokalitás nem érvényesül. A **térbeli lokalitás** azt jelenti, hogy a szomszédos címekre nagy valószínűséggel hamarosan szükség lesz — ezt a cache-sor tölti ki.

2. megoldás — Segítség. 128 bájt két 64 bájtos cache-sor — hány cache-sort HAGYUNK KI minden lépésnél stride 32-nél, és hányat stride 1024-nél?

2. megoldás — Teljes megoldás. A stride 32 (128 bájt) esetén egy oszlopbejárás pontosan **minden második** cache-sort érinti, és mindegyikből egy elemet használ. A teljes külső ciklust is figyelembe kell venni: a következő oszlop (`x5 + 1`) ugyanezeket a sorokat érinti újra, a szomszédos 4 bájtért. Hogy ilyenkor a sor még a cache-ben van-e, az attól függ, mi szorította ki közben. Stride 32-nél egy oszlop 32 768 különböző sort érint (2 MiB), stride 1024-nél 1024 sort (64 KiB), de azokat egyetlen L1-setre képezi le.

A fenti modell-L1-ben ($L = 64$, $S = 64$, 8 utas, LRU) egy egyszerű szimuláció szerint a sorrendi bejárás 65 536 L1-hibát ad, a stride 32 és a stride 1024 pedig **egyaránt** minden hozzáférésnél hibázik (1 048 576). Az L1-modell tehát a kettőt nem rangsorolja. A különbséget az L2 mérete és geometriája, a hardveres előbetöltő és a TLB dönti el; ezek modellenként más sorrendet adhatnak. Ezért a sebességrangsor csak **feltételes modelljóslat**: például ha az L2 elég nagy és asszociatív ahhoz, hogy egy stride 1024-es oszlop 64 KiB-os sorkészlete a következő oszlopig benne maradjon, a stride 1024 akár gyorsabb is lehet a stride 32-nél. A tényleges sorrendet natív AArch64-en kell mérni, azonos elemszámmal és ismételt futtatással (lásd az 13.4 mintáját). A mért rangsor akkor is helyesen értelmezhető, ha eltér a jóslattól, feltéve hogy a mag típusa, a cache-paraméterek és a mért PMU-esemény dokumentálva van.

3. megoldás — Segítség. Gondold át, mit szimulál ténylegesen a QEMU TCG-je (Tiny Code Generator): utasításonkénti funkcionális fordítást, vagy a cache/DRAM-késleltetés cikluspontos modelljét is?

3. megoldás — Teljes megoldás. A `clock_gettime`-nal mért idő QEMU-ban **nem hardverhű**, mert a QEMU egy szoftveres emulátor, amely nem modellezi a valódi cache-hierarchiát, a DRAM-késleltetést és a processzor pipeline-ját. A mért idő a host gépen futó emulátor ideje. Sem az abszolút értéke, sem a sorrendi és a stride-os bejárás közötti különbség iránya nem következtet az ARM-cél viselkedésére, mert a különbséget a host cache-e és a QEMU belső adatszerkezetei alakítják. QEMU-n a program funkcionális helyessége ellenőrizhető (azonos elemszám, helyes összeg). Hardverhű méréshez fizikai Raspberry Pi vagy más AArch64 hardver szükséges, a PMU-eseményekkel együtt dokumentálva.

13.1.18 Rövid összefoglaló

A cache-hierarchia (L1/L2/L3/DRAM) a processzor és a főmemória közötti sebességkülönbséget hidalja át. A cache-sor (jellemzően 64 bájt) betöltése a térbeli lokalitást használja ki: a szomszédos címek együtt kerülnek a cache-be. A sorrendi bejárás ezért hatékony, míg a nagy stride-ú bejárás pazarló. QEMU-n csak a funkcionális helyesség ellenőrizhető; a QEMU alatt mért idő és `perf`-számláló a hostra vonatkozik. Az ARM-cél cache-viselkedését natív AArch64-en, PMU-eseményekkel kell mérni.

Továbbhasznált definíciók:

- **Cache-sor:** a cache és a főmemória közötti adatmozgatás alapegysége, jellemzően 64 bájt.
- **Térbeli lokalitás:** a szomszédos memóriacímek együttes elérésének valószínűsége.
- **Stride:** a bejárás során az egymást követő hozzáférések közötti ugrás mérete.

13.1.19 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — A64 memória-hozzáférések, cache-kezelés. **Ellenőrizve**: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- QEMU dokumentáció — user-mode emuláció, korlátok. **Tartalmilag ellenőrizve**: a `master` ág hivatalos user-mode dokumentációja (`user/main.html`) nem állít semmit a cache-hierarchia vagy a memóriasávszélesség modellezéséről — ez összhangban van a fejezet „nem hardverhű” állításával, de az indoklás pontosabban a TCG (Tiny Code Generator) funkcionális, nem ciluspontos tervezéséből következik, nem egy explicit QEMU-dokumentált korlátból.
- GNU assembler kézikönyv — AArch64 szintaxis, `.fill`, `.balign` direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak:

- A konkrét cache-méretek és elérési idők processzorfüggők; az itt megadott értékek jellemző tartományok, nem konkrét hardverre vonatkoznak.
- A QEMU cache-emulációjának pontossága verziófüggő; a mért értékek illusztratívak.
- A `perf` eszköz QEMU user-mode alatti viselkedése nem ellenőrzött itt.
- A telepített Ubuntu 24.04 csomagolt QEMU-verzió és a benne elérhető (hiányzó) cache-modellezés pontos viselkedése nincs lekérdezve; a fejezetben szereplő értékek hipotetikusak.

13.2 Pipeline alapfogalmak és adatfüggőségi akadályok

13.2.1 Motiváció

Tegyük fel, hogy egy numerikus rutin forró ciklusát elemezzük. Az `a` tömb mérete néhány ezer elem, így az előző alfejezet szerint a cache-találati arány magas, a DRAM-elérés ritka. A mért futási idő mégis többszöröse annak, amit az utasításszám és az órajel alapján naivan várnánk. A szűk keresztmetszet — egy több utasítást egyszerre kibocsátó valós magon — gyakran nem a memória, hanem egy szerkezeti tény: a ciklusmag minden iterációja ugyanarra a regiszterre épül, és a pipeline nem tudja átlapolni az egymásra váró műveleteket. Az alfejezet egyszerű modelljén azt is látni fogjuk, mikor *nem* ez a szűk keresztmetszet.

Ez az alfejezet azt a mechanizmust bontja fel, amely az ilyen lassulást okozza. Célunk, hogy az olvasó a `s = s + a[i]` típusú kódot ne csak helyesnek, hanem **ütemezhetőnek** is lássa, és meg tudja becsülni, mennyi ciklust veszít egy adott függőségi minta. A modell, amit használunk, szándékosan egyszerű: egy sorrendi (in-order), egyszeres kibocsátású, négyfokozatú pipeline. Nem ciklusra pontos leírás egy konkrét ARM magról, hanem **jelölt, didaktikus modell**, amelyre pontos ciklusszám-levezetés adható, és amelynek korlátait külön jelöljük.

13.2.2 Definíció és működés

Pipeline (futószalag): a processzor azon szervezési elve, hogy az utasítás végrehajtását egymást követő, egyszerű lépésekre bontja, és a különböző utasítások különböző lépéseit egyidejűleg, átlapolva végzi.

A modellünkben négy fokozat van:

Fokozat	Név	Mit tesz
F	fetch	a következő utasítás lehívása az utasítás-memóriából
D	decode	dekódolás, regiszterolvasás (operandusok betöltése)
E	execute	ALU-művelet, címgenerálás, elágazás-kiértékelés
W	writeback	az eredmény visszaírása a regiszterfájlba

Miért gyorsít az átlapolás? Különbséget kell tennünk **latency (késleltetés)** és **throughput (áteresztőképesség)** között. Egyetlen utasítás négy ciklus alatt fut végig (latency = 4), de ha a fokozatok átlapolhatók, akkor amikor az első utasítás a W fokozatban van, a második már az E-ben, a harmadik a D-ben, a negyedik az F-ben. Stabil állapotban minden ciklusban befejeződik egy utasítás: throughput = 1 utasítás/ciklus. N egymástól független utasításra az összes ciklusszám $N + 3$ (a feltöltési és kiürítési ciklusokkal együtt), szemben a nem pipeline-olt $4N$-nel.

Adatfüggőségi akadály (data hazard): olyan helyzet, amikor egy utasítás operandusa olyan eredmény, amelyet egy **korábbi, még a pipeline-ban lévő utasítás** állít elő, és nincs olyan útvonal (pl. forwarding), amely az értéket időben eljuttatná a fogyasztóhoz. Ilyenkor a fogyasztó **várakozási ciklust (stall)** kap, és a pipeline-ba egy **buborék (bubble)** kerül.

A klasszikus besorolás szerint read-after-write (RAW, olvasás írás után) függőségről van szó: a fogyasztó olvasni akar, a termelő még nem írt. Sorrendi pipeline-ban write-after-write (WAW) és write-after-read (WAR) veszély nem lép fel, mert a kibocsátási sorrend megegyezik a program sorrendjével.

Modellfeltevések (jelölt, egyszerűsített):

1. Sorrendi kibocsátás, egy utasítás/ciklus, rögzített négy fokozat.
2. A regiszterfájlt a D fokozat elején olvassuk, a W fokozat végén írjuk.
3. Az adat-memória elérés (load/store) az E fokozatban fejeződik be.
4. Először **nincs forwarding**; később bevezetjük.
5. Az elágazásnak nincs büntetése (branch penalty), és nincs cache-miss a modellben.

Ez a modell **nem ciklusra pontos leírás** egy AArch64 magról: a valós Cortex-A magok hosszabb, többszörös kibocsátású, sorrenden kívüli (out-of-order) pipeline-nal és bonyolult forwarding-hálózattal rendelkeznek. Ciklusszám-levezetésre azonban pontosan használható.

13.2.3 Levezetés: két utasítás, egy RAW-függőség

Tekintsük ezt a két utasítást (A64 ISA):

```
add x1, x2, x3      // termelő: x1 előállítása
sub x4, x1, x5      // fogyasztó: x1 azonnali felhasználása
```

Kiindulás: nincs forwarding.

A termelő `add` ciklusai:

Ciklus	1	2	3	4
add	F	D	E	W

Az `x1` tényleges értéke a regiszterfájlban a **4. ciklus végén** jelenik meg, mert a W fokozat ekkor fejeződik be.

A fogyasztó `sub` ideálisan a 2. ciklusban kerülne F-be, és a D fokozata a 3. ciklusban olvasná az `x1`-et. Ekkor viszont a regiszterfájlban még a régi érték van. **A fogyasztó D fokozata csak azután olvashat, hogy a termelő W fokozata befejeződött**, azaz az 5. ciklusnál korábban nem.

Ezért a `sub` D fokozata a 3. ciklus helyett az 5. ciklusban indul. A köztes 3. és 4. ciklus a `sub` számára **várakozási buborék**. A `sub` teljes életciklusa:

Ciklus	3	4	5	6	7
sub	stall	stall	D	E	W

A két utasítás összes ciklusszáma 7 (a buborékokkal), szemben a független eset 5 ciklusával: **2 stall ciklus**.

Forwarding bevezetése. A forwarding (továbbítás, bypass) lényege: az eredmény nem várja meg a W fokozatot, hanem a termelő E fokozatának kimenetéről közvetlenül a fogyasztó E fokozatának bemenetére kerül. Esetünkben a termelő E fokozata a 3. ciklusban fejeződik be; a `sub` E fokozata a 4. ciklusban kezdődik, így az érték épp időben érkezik. **A stall 2-ről 0-ra csökken** — feltéve, hogy a fogyasztó E fokozata legalább egy ciklussal a termelő E után kezdődik. Ez pontosan teljesül a szomszédos ALU-utasításokra.

Load-use eset. A `ldr x1, [x2]` utasítás adata a modell 3. feltevése szerint az E fokozat végén áll elő, vagyis a modellben ugyanúgy továbbítható, mint egy ALU-eredmény: a szomszédos fogyasztó **nem kap várakozást**. Ez a modell egyszerűsítése. A valós magokban a betöltés eredménye általában később érhető el, mint egy egyszerű ALU-művelet eredménye; a *load-to-use* késleltetés azonban mikroarchitektúra-jellemző, nem ISA-szabály, és magonként eltér. A modellből számolt ciklusszám ezért egy konkrét gépre sem alsó korlát, csak a modell szerinti érték.

Ciklusvesztés általános képlete (jelölt modell). Időkonvenció: minden ciklusszám a megnevezett fokozat **kezdő ciklusát** jelöli, és egy fokozat a ciklusa végén fejeződik be. Legyen

- $c_{\text{available}}$ az a legkorábbi ciklus, amelynek **elején** a fogyasztó érintett fokozata már hozzáfér a termelő eredményéhez;
- c_{required} az a ciklus, amelyben a fogyasztó érintett fokozata várakozás nélkül **kezdődne**.

Ekkor

$$\text{stall} = \max\left(0,\ c_{\text{available}} - c_{\text{required}}\right).$$

Ha az eredmény az 5. ciklus elején érhető el, a fogyasztónak pedig a 4. ciklusban lenne rá szüksége, $\max(0, 5-4) = 1$ ciklust kell várni; ha az eredmény korábban kész, mint ahogy kell, a várakozás 0.

Ellenőrzés a fenti két esettel.

- *Forwarding nélkül:* a fogyasztó D fokozata olvassa a regiszterfájlt. Az `add` W fokozata a 4. ciklus végén zárul, így az `x1` az 5. ciklus elején olvasható: $c_{\text{available}} = 5$. A `sub` D fokozata várakozás nélkül a 3. ciklusban kezdődne: $c_{\text{required}} = 3$. A képlet szerint $\max(0, 5-3) = 2$ stall — egyezik a táblázattal.

- *Forwarding mellett:* a fogyasztó E fokozata kapja az értéket. Az `add` E fokozata a 3. ciklus végén zárul, az eredmény a 4. ciklus elején továbbítható: $c_{\text{available}} = 4$. A `sub` E fokozata várakozás nélkül a 4. ciklusban kezdődne: $c_{\text{required}} = 4$. A képlet $\max(0, 4-4) = 0$ stallt ad.

13.2.4 Kidolgozott példa: cikluson átívelő függőség

Munkakönyvtár: `~/arm-labor/a64/a050/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a050
cd ~/arm-labor/a64/a050
```

A kiindulópont a következő C-függvény:

```
long sum_seq(const long *a, long n) {
    long s = 0;
    for (long i = 0; i < n; i++) s = s + a[i];
    return s;
}
```

Assemblyben két változatot írunk ugyanerre a számításra: `lab_sum1` egy akkumulátorral (az `x0` lánc), `lab_sum2` kettővel (`x0` és `x5`). Mindkettő teljes, C-ből hívható függvény az AAPCS64 szerint (`x0 = a`, `x1 = n`, eredmény `x0`-ban). A `lab-pipe-sum.s` tartalma:

```
// lab-pipe-sum.s - A64 ISA, AArch64 Linux; GNU as.
// long lab_sum1(const long *a, long n);  egy akkumulátor (x0)
// long lab_sum2(const long *a, long n);  két akkumulátor (x0, x5)
// Levélfüggvények, keretet nem építenek, ezért CFI nélküliek (lásd 8.3).
    .text
    .p2align 2

    .global lab_sum1
    .type   lab_sum1, %function
lab_sum1:                           // belépéskor: x0 = a, x1 = n
    mov  x4, x1                     // x4 = n
    mov  x1, x0                     // x1 = a
    mov  x0, #0                     // x0 = s
    mov  x2, #0                     // x2 = i
    cmp  x2, x4
    b.ge 2f
1:  ldr  x3, [x1, x2, lsl #3]       // x3 = a[i]
    add  x0, x0, x3                 // s = s + x3
    add  x2, x2, #1                 // i++
    cmp  x2, x4
    b.lt 1b
2:  ret
    .size   lab_sum1, .-lab_sum1

    .global lab_sum2
    .type   lab_sum2, %function
lab_sum2:                           // belépéskor: x0 = a, x1 = n
    mov  x4, x1
    mov  x1, x0
    mov  x0, #0                     // 1. akkumulátor
    mov  x5, #0                     // 2. akkumulátor
    mov  x2, #0
    cmp  x4, #0
    b.le 4f
1:  add  x7, x2, #1
```

```
    cmp  x7, x4
    b.ge 2f
    ldr  x3, [x1, x2, lsl #3]
    ldr  x6, [x1, x7, lsl #3]
    add  x0, x0, x3
    add  x5, x5, x6
    add  x2, x2, #2
    b    1b
2:  cmp  x2, x4                     // páratlan n: egy elem maradt
    b.ge 3f
    ldr  x3, [x1, x2, lsl #3]
    add  x0, x0, x3
3:  add  x0, x0, x5
4:  ret
    .size   lab_sum2, .-lab_sum2

    .section .note.GNU-stack,"",%progbits
```

A `lab_sum1` ciklusában az `add x0, x0, x3` **minden iterációban ugyanarra az x0-ra épül**: az előző iteráció eredményét használja. Ez egy **cikluson átívelő függőség (loop-carried dependency)**. A `lab_sum2` két független láncot vezet: az `add x0` és az `add x5` egyike sem várja a másik eredményét.

Ciklusonkénti levezetés a jelölt modellen. A modellben az utasítások sorrendben, ciklusonként egyesével lépnek be, és — a feltevések szerint — az elágazásnak nincs külön költsége. Várakozás nélkül egy N végrehajtott utasításból álló futás $N+3$ ciklus (a $+3$ a pipeline feltöltése). Meg kell tehát nézni, hol keletkezik várakozás:

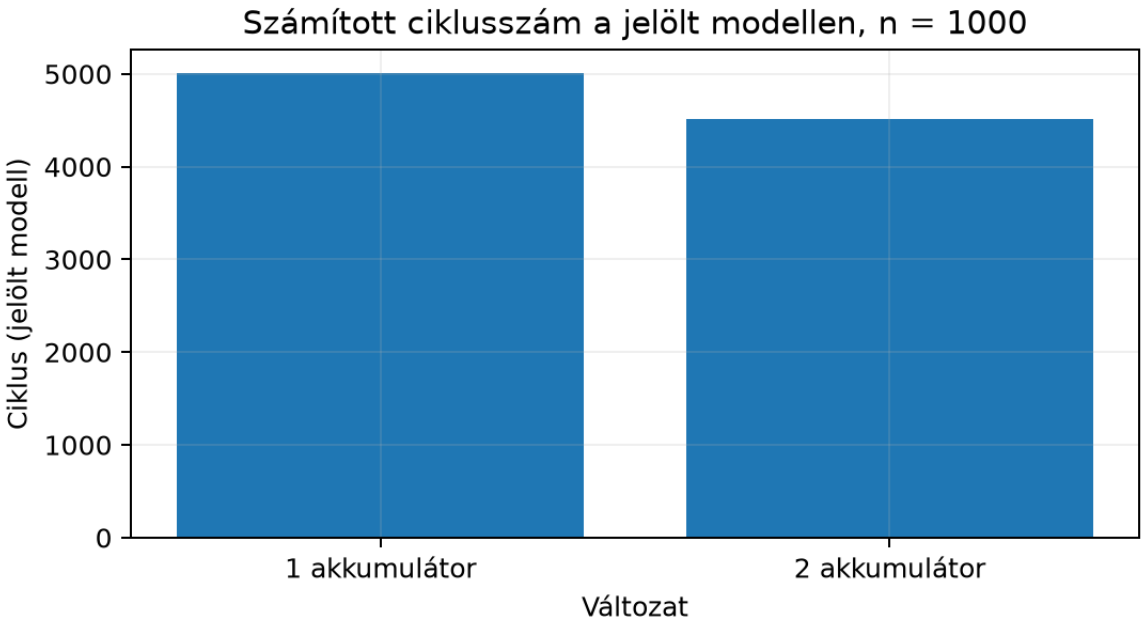
Függőség a `lab_sum1` ciklusában	Távolság (utasítás)	$c_{\text{available}} - c_{\text{required}}$	Stall
`ldr x3` → `add x0, x0, x3`	1	0	0
`add x2` → `cmp x2, x4`	1	0	0
`cmp` → `b.lt` (NZCV)	1	0	0
`add x0` → következő iteráció `add x0`	5	negatív	0

Forwarding mellett a szomszédos függőség sem okoz várakozást, a lánc két tagja között pedig öt utasítás van. A `lab_sum2`-ben ugyanez a helyzet (`add x7` → `cmp x7`: 0; `ldr x6` → `add x5`: 0). **Ebben a modellben tehát egyik változatban sincs stall**, és a ciklusszám a végrehajtott utasítások számából adódik. $n = 1000$ esetén:

Változat	Végrehajtott utasítás	Ciklus a modellben	Ciklus / elem
`lab_sum1`: 6 belépő + 1000 × 5 + `ret`	5007	5010	5,0

Változat	Végrehajtott utasítás	Ciklus a modellben	Ciklus / elem
`lab_sum2`: 7 belépő + 500 × 9 + 3 kilépő + 2 farok + 1 összevonás + `ret`	4514	4517	4,5

A különbség (kb. 10%) **nem a függőségi lánc megtöréséből** származik — a modellben a lánc nem okozott várakozást —, hanem abból, hogy a `lab_sum2` két elemre 9, a `lab_sum1` két elemre 10 utasítást hajt végre: a ciklusvezérlés aránya kisebb. Az utasításszámokat a laborban az emulátorral is megszámoltuk (lásd a Források szakaszt).



13.2. ábra: Számított ciklusszám a jelölt modellen, n = 1000 - Sorrendi, egyszeres kibocsátású, négyfokozatú, forwarding-os modell; a végrehajtott utasításszámból számítva. Nem QEMU- és nem hardvermérés.

Mikor számít a függőségi lánc? A lánc akkor válik szűk keresztmetszetté, ha a lánc egy körének késleltetése nagyobb, mint amennyi idő alatt a gép a ciklus többi utasítását amúgy is végrehajtaná. Egyszeres kibocsátású gépen, k utasításos ciklusmaggal és L ciklus késleltetésű láncművelettel a modell szerint:

$$\text{ciklus/iteráció} = \max(k,\ L)\,.$$

A fenti modellben $L = 1$ (továbbított `add`) és $k = 5$, ezért a lánc nem számít. Valós, **többszörös kibocsátású** (és gyakran sorrenden kívüli) magon a k utasítás egy ciklusnál jóval rövidebb idő alatt is végrehajtódhat, a hosszabb késleltetésű láncműveletek (például lebegőpontos összeadás) pedig több ciklust igényelnek — ilyenkor a lánc dönt, és több akkumulátor valóban gyorsít. Hogy egy adott magon mennyit, azt ez a modell nem mondja meg: valós ciklusszámhoz egy megnevezett, fizikai AArch64 processzoron végzett mérés kell (lásd Mikrobenchmark készítése és eredmények értelmezése).

13.2.4.1 Két külön lépés: objektum és teljes labor

1. Csak objektummá fordítás — a ciklusmag statikus vizsgálatához elég az assembly-fájl:

```
# Target: AArch64 Linux; csak összeállítás és visszafejtés, futtatás nélkül.
aarch64-linux-gnu-as -o lab-pipe-sum.o lab-pipe-sum.s
aarch64-linux-gnu-objdump -d lab-pipe-sum.o
```

2. A teljes labor — a futtatáshoz teszthajtó is kell, amely a két változatot egy C-referenciával veti össze, a farokágat is lefedő n értékekre. A `lab-pipe-main.c` tartalma:

```
/* lab-pipe-main.c - teszthajtó a lab-pipe-sum.s két függvényéhez. */
#include <stdio.h>

long lab_sum1(const long *a, long n);
long lab_sum2(const long *a, long n);

/* C-referencia; az elemek kicsik, előjeles túlcsordulás nem fordul elő. */
static long lab_sum_ref(const long *a, long n)
{
    long s = 0;
    for (long i = 0; i < n; i++) s = s + a[i];
    return s;
}

int main(void)
{
    static long a[1001];
    static const long ns[] = { 0, 1, 2, 3, 999, 1000, 1001 };
    int ok = 1;

    for (long i = 0; i < 1001; i++) a[i] = (i % 7) - 3;   /* -3..3 */

    for (unsigned k = 0; k < sizeof ns / sizeof ns[0]; k++) {
        long n = ns[k];
        long r = lab_sum_ref(a, n), s1 = lab_sum1(a, n), s2 = lab_sum2(a, n);
        int good = (s1 == r) && (s2 == r);
        printf("n=%4ld ref=%4ld sum1=%4ld sum2=%4ld %s\n",
               n, r, s1, s2, good ? "OK" : "HIBA");
        ok &= good;
    }
    return ok ? 0 : 1;
}
```

```
# Target: AArch64 Linux, qemu-user. Tiszta könyvtárban, a két fenti fájlból.
aarch64-linux-gnu-gcc -std=c11 -O2 -Wall -Wextra -static \
    lab-pipe-main.c lab-pipe-sum.s -o lab-pipe
qemu-aarch64 ./lab-pipe ; echo "exit=$?"
```

Kimenet (mérve: GNU + glibc + QEMU 8.2.2, a lektor mérése):

```
n=   0 ref=   0 sum1=   0 sum2=   0 OK
n=   1 ref=  -3 sum1=  -3 sum2=  -3 OK
n=   2 ref=  -5 sum1=  -5 sum2=  -5 OK
n=   3 ref=  -6 sum1=  -6 sum2=  -6 OK
n= 999 ref=  -5 sum1=  -5 sum2=  -5 OK
n=1000 ref=  -3 sum1=  -3 sum2=  -3 OK
n=1001 ref=   0 sum1=   0 sum2=   0 OK
exit=0
```

A program a helyességet ellenőrzi; ciklusszámot nem mér, és a QEMU futási idejéből sem olvasható ki ciklusszám.

QEMU-korlát. A `qemu-aarch64` user-mode emulátor (és a `qemu-system-aarch64` is) **nem ciklusra pontos**: utasítás-szintű dinamikus fordítást (TCG) végez, és nem modellezi a valós pipeline hosszát, a forwarding-késleltetést vagy a kibocsátási korlátokat. Az itt megadott ciklusszámok **a jelölt modellből számított értékek**; a QEMU-ból csak az utasítás-viselkedés és a helyesség ellenőrizhető, relatív időmérésre is csak nagyon fenntartással. Valós ciklusszámhoz fizikai AArch64 host és pl. hardveres teljesítményszámláló szükséges (lásd Mikrobenchmark készítése és eredmények értelmezése).

13.2.5 Hibák és ellenpróbák

Hiba 1: a QEMU futási időből ciklusszámra következtetni. Tünet: két implementáció `time`-különbsége nem korrelál a levezetett ciklusszámmal. Hipotézis: QEMU TCG nem pipeline-modellez. Adatgyűjtés: `qemu-aarch64 -d in_asm` és `objdump -d` összehasonlítás. Egy változtatás: hardver nélkül csak az utasításszám és a statikus elemzés marad; mérés fizikai hostra tolása.

Hiba 2: a modell várakozásmentességét valós gépre átvinni. A jelölt modellben a továbbított load-use és ALU-függőség 0 stallt ad. Valós magon a load-to-use késleltetés általában nagyobb, és mikroarchitektúrától függ — nem ISA-szabály, ezért egy konkrét számot sem lehet „kötelezőnek” tekinteni. Ellenpróba: fizikai gépen mérjük ugyanazt a ciklust úgy is, hogy az `ldr` és a fogyasztó közé független utasítást teszünk; ha az idő csökken, a betöltés késleltetése számított.

Hiba 3: az utasításszámot a ciklusszámmal azonosítani. Ha egy ciklusmag 8 utasítás, az nem jelenti azt, hogy 8 ciklus/iteráció: ha van benne 1 cikluson átívelő lánc, a mag hosszabb lehet, mint az utasításszám.

Hiba 4: a fordító optimalizálására hagyatkozni vakon. A `-O2`/`-O3` sokszor megtöri a függőségeket, de nem garantált; az `objdump -d` megmutatja a tényleges ütemezést. Ellenpróba: a kódot `-O0`-val és `-O3`-mal is disassemblálni és a ciklusmag utasításlistáját összevetni.

Hiba 5: az utasítás-átrendezést (instruction scheduling) összekeverni a programszemantika megváltoztatásával. Az ütemezés csak olyan utasításokat cserélhet fel, amelyek között nincs adat- vagy vezérlési függőség. Az átrendezés ötlete fogalmi szinten: a termelő és a fogyasztó közé független utasításokat ékelünk, hogy a stall „hasznos” munkával teljen. Ellenpróba: a cserélt utasítások operandushalmazainak metszetét ellenőrizzük.

13.2.6 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Rajzold fel az alábbi utasítássor pipeline-diagramját a jelölt modellben (nincs forwarding), és add meg az összes ciklusszámot:

```
mul  x1, x2, x3
add  x4, x1, x5
add  x6, x4, x5
```

Hány stall ciklus keletkezik, ha forwarding elérhető?

2. (implementációs, középszint — cél: rutinépítés) Írj A64 assembly-t egy `int64_t dot_product(const int32_t *a, const int32_t *b, int n)` függvényre, amely 64 bites szorzatok összegét adja vissza. Feltétel: az összeg elfér `int64_t`-ban; `n <= 0` esetén az eredmény nulla. Azonosítsd a ciklusmag függőségi láncait, majd alakítsd át két párhuzamos akkumulátorra. Add meg a módosítás előtti és utáni várható ciklus/iterációt a jelölt modellen, és indokold a különbséget.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Fordítsd le a kidolgozott példa `sum_seq`-ét `-O0` és `-O2` szinten. `objdump -d`-vel hasonlítsd össze a generált ciklusmag-utasításokat. Mutasd meg, hol töri meg a fordító a függőségi láncot, és hol nem. Magyarázd meg, miért **nem** elfogadható a QEMU futási idejéből ciklusszámra következtetni, és nevezd meg, milyen mérési módszer adna hiteles adatot.

13.2.7 Megoldások

1. megoldás — Segítség. Rajzold fel mindhárom utasítás F/D/E/W négyzetrácsát ciklusonként; a `D` fokozat csak akkor indulhat, ha a termelő `W` fokozata már lezárult (forwarding nélkül) vagy `E` fokozata lezárult (forwarding mellett).

1. megoldás — Teljes megoldás. Nincs forwarding: `mul` W-je a 4. ciklus végén zárul, `add x4` D-je csak az 5. ciklusban indulhat → 2 stall; a fogyasztó `add x6` ugyanígy 2 stallt kap az előző `add x4`-hez képest. Teljes ciklusszám: $3+3+2+2=10$ ciklus. Forwarding mellett az ALU→ALU láncokon a stall 0-ra csökken, így az összes ciklusszám $3+3=6$ (független esettel azonos). A modellben a `mul` is egyciklusos E-fokozatú; valós magon 3–5 ciklus is lehet, ekkor forwarding mellett is marad stall.

2. megoldás — Segítség. A legbelső `ldr→mul→add` lánc egyetlen akkumulátorral (`x0`) cikluson átívelő láncot alkot — hogyan lehet két független `x0`/`x8`-szerű akkumulátort bevezetni, hasonlóan a fejezet `sum_seq` kétakkumulátoros átalakításához?

2. megoldás — Teljes megoldás. AAPCS64-konform változat: az `a` az x0-ban, a `b` az x1-ben, az `n` a w2-ben jön, ezért a 64 bites ciklushatárhoz `sxtw` kell.

```
.global dot_product
.type dot_product, %function
dot_product:
    sxtw  x2, w2
    cmp   x2, #0
    b.le  .Ldot_zero
    mov   x3, #0
    mov   x4, #0
.Ldot_loop:
    ldrsw x5, [x0, x3, lsl #2]
    ldrsw x6, [x1, x3, lsl #2]
    mul   x5, x5, x6
    add   x4, x4, x5
    add   x3, x3, #1
    cmp   x3, x2
    b.lt  .Ldot_loop
    mov   x0, x4
    ret
.Ldot_zero:
    mov   x0, #0
    ret
.size dot_product, .-dot_product
```

Egy akkumulátorral a ciklusmag 7 utasítás/elem (2×`ldrsw`, `mul`, `add`, `add`, `cmp`, `b.lt`), a jelölt modellben 7 ciklus/elem. Kétakkumulátoros, kettesével kibontott változat:

```
.global dot_product2
.type dot_product2, %function
dot_product2:
    sxtw  x2, w2
    cmp   x2, #0
    b.le  .Ldot2_zero
    mov   x3, #0
```

```
    mov   x4, #0
    mov   x5, #0
    lsr   x6, x2, #1
    cbz   x6, .Ldot2_tail
.Ldot2_loop:
    ldrsw x7, [x0, x3, lsl #2]
    ldrsw x8, [x1, x3, lsl #2]
    mul   x7, x7, x8
    add   x4, x4, x7
    add   x3, x3, #1
    ldrsw x9, [x0, x3, lsl #2]
    ldrsw x10, [x1, x3, lsl #2]
    mul   x9, x9, x10
    add   x5, x5, x9
    add   x3, x3, #1
    subs  x6, x6, #1
    b.ne  .Ldot2_loop
.Ldot2_tail:
    cmp   x3, x2
    b.ge  .Ldot2_done
    ldrsw x7, [x0, x3, lsl #2]
    ldrsw x8, [x1, x3, lsl #2]
    mul   x7, x7, x8
    add   x4, x4, x7
.Ldot2_done:
    add   x0, x4, x5
    ret
.Ldot2_zero:
    mov   x0, #0
    ret
.size dot_product2, .-dot_product2
```

A kétakkumulátoros mag 12 utasítás / 2 elem, a ciklusvezérléssel együtt $13/2 = 6{,}5$ utasítás/elem; `madd`-dal $11/2 = 5{,}5$. A különbség itt is a ciklusvezérlés arányából jön: a modellben a két `add x4/add x5` lánc között elég utasítás van ahhoz, hogy ne keletkezzen várakozás. A láncmegtörés akkor számít, ha a láncművelet késleltetése meghaladja a többi munka idejét (többszörös kibocsátású vagy hosszú szorzású magon); ezt csak mérés dönti el. A teljes kód C-ből hívható: `.global`, `.type`, `ret`, és a visszatérési érték az x0-ban.

3. megoldás — Segítség. Gondold át, mit kell garantálnia az `-O0`-nak (minden változó bármikor megfigyelhető legyen) — ez miért kényszerít ki extra load/store láncot az akkumulátor körül, amit `-O2` elkerülhet?

3. megoldás — Teljes megoldás. `-O0` mellett a fordító gyakran minden iterációban kiírja/beolvassa az `s`-t a verembe, ami extra load/store láncot ad és rontja az ütemezést. `-O2` mellett `s` regiszterben marad, és a fordító néha maga is unroll-ol és több akkumulátort vezet be. A lánc **nem mindig** törik meg: a `-O2` is hagyhat szigorúan sorrendi `add`-sorozatot. A QEMU futási ideje TCG-alapú, nem pipeline-modellező, ezért relatív ciklusszámra alkalmatlan; hiteles adathoz fizikai AArch64 host és hardveres PMU-számláló (pl. `perf stat cycles` eseménnyel) szükséges, és ott is jelölni kell a QEMU-tól eltérő futási feltételeket.

A lektor mérése (GCC 13.3, `aarch64-linux-gnu-gcc`): `-O0` 24 utasítás (az `s` a veremben); `-O2` 13 utasítás, **egy** akkumulátor, mutató–végcím-összehasonlítás (`ldr x1, [x2], #8; add x0, x0, x1; cmp x2, x3; b.ne`), sem unroll, sem több akkumulátor; `-O3` 33 utasítás, NEON-vektorizált (2 sávos) változat. A `-O2` négyutasításos ciklusmagja jó ellenpélda a kézi öt utasításos változatra.

13.2.8 Rövid összefoglaló

A pipeline átlapolással növeli az áteresztőképességet: egy négyfokozatú modellben a latency 4 ciklus marad, de a throughput elérheti az 1 utasítás/ciklust. Az adatfüggőségi akadály (RAW) akkor keletkezik, ha a fogyasztó a termelő eredményét a rendelkezésre állás előtt akarja olvasni: a jelölt modellben forwarding nélkül 2 stall, forwarding mellett ALU→ALU esetben 0 stall adódik, és a modell a load-use esetet is így kezeli (a valós load-to-use késleltetés mikroarchitektúra-függő). A ciklusvesztés azonos időkonvenció mellett $\max(0, c_{\text{available}} - c_{\text{required}})$. A modellben a kétakkumulátoros összegzés csak a ciklusvezérlés arányát csökkenti; a függőségi lánc többszörös kibocsátású vagy hosszú késleltetésű esetben dönt, amit mérni kell. A QEMU nem ciklusra pontos, ezért az itt szereplő ciklusszámok a modellből számított értékek.

Továbbhasznált definíciók:

- **Adatfüggőségi akadály (data hazard):** a fogyasztó operandusa olyan érték, amelyet egy korábbi, még a pipeline-ban lévő utasítás állít elő, és nincs időben elérhető útvonal.
- **Forwarding:** az eredmény visszaírás előtti továbbítása a termelő kimenetéről a fogyasztó bemenetére.
- **Cikluson átívelő függőség (loop-carried dependency):** egy iteráció eredményére a következő iteráció számítása épül.

13.2.9 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — A64 utasítások, memóriahozzáférés. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- QEMU dokumentáció — user-mode és system-mode emuláció, TCG-korlátok. **Tartalmilag ellenőrizve**: a hivatalos user-mode dokumentáció (qemu.org/docs/master/user/main.html) semmit nem állít a cache-hierarchia, a pipeline vagy a ciklus-pontosság modellezéséről; a „nem ciklusra pontos” állítás a TCG ismert tervezési elvéből következik (funkcionális, utasításszintű emulátor), nem egy explicit QEMU-dokumentált korlátból. A hivatkozott dokumentáció emellett a `master`, nem egy konkrét kiadott verzió állapotát tükrözi.
- GNU assembler kézikönyv — AArch64 szintaxis, szekciók. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak:

- A négyfokozatú, sorrendi modell és a belőle származó ciklusszámok didaktikusak; egy konkrét ARM mag pipeline-mélysége, forwarding-útvonalai és kibocsátási korlátai eltérnek, ezért a tényleges ciklus/iteráció gépenként és modellenként más.
- A load-to-use késleltetés valós magokon mikroarchitektúra-függő, nem ISA-szabály; a négyfokozatú modell szándékosan 0 extra várakozással számol.
- A `lab-pipe` teszthajtó kimenete és a két függvény végrehajtott utasításszáma ($n = 1000$: 5007 és 4514) mérve (GNU + glibc + QEMU 8.2.2, a lektor mérése); a kimenet betűre egyezik a fenti blokkal, a kilépési állapot 0, és az utasításszámok a forrásból újraszámolva helyesek. A ciklusszámok a modell szerint számítottak, nem mértek.
- A QEMU TCG ciklus-pontosságának mértéke verziófüggő; a dokumentáció ezt nem garantálja cikluspontosként, ezért minden relatív mérés jelölt és illusztratív.

- A fordítói optimalizálás utasítás-átrendezése (instruction scheduling) GCC-verzió- és szintfüggő; a konkrét viselkedés az adott Ubuntu 24.04-es `gcc-aarch64-linux-gnu` buildjében ellenőrizendő.

13.3 Branch prediction és elágazás-szervezés hatása

13.3.1 Motiváció

Egy jelfeldolgozó algoritmusban egy ciklus feltételesen ad hozzá egy mintát egy gyűjtőváltozóhoz. A feltétel a bemeneti jel értékétől függ. Ha a jel rendezett (például monoton növekvő), a feltétel hosszú sorozatokban igaz vagy hamis. Ha a jel véletlenszerű, a feltétel kiszámíthatatlanul váltakozik. Ugyanaz az utasításszám, ugyanaz a memóriahozzáférési minta, mégis drasztikusan eltérő futási időt mérhetünk. Ennek oka a processzor elágazás-előrejelző (branch predictor) egysége. Ez az alfejezet megmutatja, hogyan mérhető ez a hatás, és hogyan kerülhető el az elágazás feltételes utasítással.

13.3.2 A branch prediction fogalmi szintje

Definíció (branch prediction): Az elágazás-előrejelzés a processzor azon képessége, hogy egy feltételes elágazás (például `b.cond`) végrehajtása előtt megjósolja, hogy az elágazás feltétele teljesülni fog-e, és ennek megfelelően spekulatívan folytatja a végrehajtást a célcímről vagy a következő utasítással. Ha a jóslat helyes, az elágazás gyakorlatilag nulla plusz késleltetéssel hajtódik végre. Ha téves (misprediction), a spekulatívan betöltött és részben végrehajtott utasításokat el kell dobni, és a helyes útvonalat be kell tölteni: ez a **branch misprediction penalty**, amely nagyságrendileg 10–20 ciklus (magfüggő) a modern nagy teljesítményű ARM magokon.

A prediktor működése fogalmi szinten: a processzor tárolja az elágazás korábbi kimeneteleit, és mintázatot keres. Egy egyszerű kétbites prediktor négy állapotot tart nyilván (erősen nem ugrik, gyengén nem ugrik, gyengén ugrik, erősen ugrik), és a hiszterézis segít elkerülni a zajos váltakozást. A fejlettebb prediktorok (például TAGE, correlating) az elágazások közötti korrelációt is figyelembe veszik. A lényeg: minél szabályosabb a minta, annál nagyobb a találati arány.

Kiszámítható vs. kiszámíthatatlan minta. Tekintsünk egy `if (a[i] >= threshold)` feltételt, és jelölje q annak valószínűségét, hogy egy elemre igaz. Ha a tömb rendezett, a feltétel először sokáig hamis, majd egy ponton igaz lesz, és utána végig igaz marad: a prediktor ezt megtanulja, a tévesztések száma a váltás körüli néhány esetre korlátozódik, a találati arány közel 100%.

Ha a tömb véletlen sorrendű, a találati arány a **bemenet eloszlásától** függ, nem önmagában a sorrendtől. A legegyszerűbb modell: a kimenetelek egymástól függetlenek, és mindegyik q valószínűséggel igaz. Ilyenkor a múltból nincs mit tanulni, és a legjobb, amit egy prediktor tehet, hogy mindig a gyakoribb kimenetelt jósolja:

$$\text{találati arány} \le \max(q,\, 1-q), \qquad \text{tévesztési arány} \ge \min(q,\, 1-q).$$

Az 50%-os találati arány tehát a **független, kiegyensúlyozott** ($q = 0{,}5$) esetre érvényes. Ha a feltétel az elemek 90%-ára igaz, a véletlen sorrend mellett is 90% találatot ér el az állandó „igaz” jóslat. A kísérletben ezért a két bemenet **ugyanazt az értékkészletet** és **ugyanannyi igaz**

esetet tartalmazza, és a küszöböt úgy választjuk, hogy $q = 0{,}5$ legyen; így a két futás csak a sorrendben tér el.

Fontos megjegyezni: a prediktor tanulási ideje miatt a rövid ciklusokban a hatás kevésbé érvényesül. A méréshez elég hosszú ciklus kell (például $N = 10^6$).

13.3.3 Elágazás elkerülése CSEL-lel

A korábban megismert feltételes kiválasztás (`CSEL`, `CSINC`, `CSINV`, `CSNEG`) lehetővé teszi, hogy adatfüggő elágazás helyett feltételes mozgatást használjunk. A Feltételes végrehajtás: CSET/CSEL és feltételkódok alfejezetben részletesen tárgyaltuk a működést. Itt most arra használjuk, hogy egy elágazást kiváltsunk: a feltétel kiértékelése után mindkét ág eredményét kiszámítjuk (vagy egyiket konstanssal helyettesítjük), és a `CSEL` választja ki a feltételnek megfelelőt. Ezáltal a vezérlési folyam mindig ugyanaz marad, nincs adatfüggő elágazás, a prediktornak nincs mit elvétenie. Az ára: mindkét ág munkáját el kell végezni (több utasítás), és a feltétel kiszámítása is plusz utasítás. Kis ágak esetén ez nyereséges lehet, mert elkerüli a misprediction penalty-t.

13.3.4 Kidolgozott példa: azonos munka, eltérő elágazásminta

Munkakönyvtár: `~/arm-labor/a64/a051/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a051
cd ~/arm-labor/a64/a051
```

A mérés alapelve: ugyanaz a kódrészlet, ugyanaz az adatmennyiség, csak az elágazás mintázata más. Két tömböt használunk: az egyik rendezett, a másik ugyanazokat az értékeket tartalmazza véletlen sorrendben. A feltétel ugyanaz.

Bemenet. Az elemek a $0, 1, \ldots, n-1$ értékek, a küszöb $n/2$, így pontosan $q = 0{,}5$. A *rendezett* tömb ezeket növekvő sorrendben, a *permutált* tömb ugyanezeket egy determinisztikus álvéletlen-generátorral (xorshift64) végzett Fisher–Yates-keverés után tartalmazza. Mivel a permutált tömb ugyanazokat az értékeket tartalmazza, az igaz esetek száma és az összeg is azonos.

A kiinduló C-függvény (a teszthajtóban referenciaként szerepel, a mérés nem ezt méri — lásd az 1. tipikus hibát):

```
long sum_if(const long *a, long n, long threshold) {
    long s = 0;
    for (long i = 0; i < n; i++) {
        if (a[i] >= threshold) {
            s += a[i];
        }
    }
    return s;
}
```

A két mért változat teljes, C-ből hívható függvény. A `lab-a051_asm.S` tartalma:

```
// lab-a051_asm.S - A64 ISA, AArch64 Linux; GNU as.
// long sum_if_asm (const long *a, long n, long threshold);  adatfüggő elágazással
// long sum_if_csel(const long *a, long n, long threshold);  CSEL-lel, elágazás nélkül
// Levélfüggvények, keretet nem építenek, ezért CFI nélküliek (lásd 8.3).
    .text
    .p2align 2

    .global sum_if_asm
```

```
    .type   sum_if_asm, %function
sum_if_asm:                         // x0 = a, x1 = n, x2 = threshold
    mov     x3, #0                  // s = 0
    mov     x4, #0                  // i = 0
    b       .Lcheck
.Lloop:
    ldr     x5, [x0, x4, lsl #3]    // x5 = a[i]
    cmp     x5, x2
    b.lt    .Lskip                  // adatfüggő elágazás
    add     x3, x3, x5
.Lskip:
    add     x4, x4, #1
.Lcheck:
    cmp     x4, x1
    b.lt    .Lloop                  // ciklusvezérlő elágazás
    mov     x0, x3
    ret
    .size   sum_if_asm, .-sum_if_asm

    .global sum_if_csel
    .type   sum_if_csel, %function
sum_if_csel:                        // x0 = a, x1 = n, x2 = threshold
    mov     x3, #0
    mov     x4, #0
    b       .Lcheck2
.Lloop2:
    ldr     x5, [x0, x4, lsl #3]
    cmp     x5, x2
    csel    x5, x5, xzr, ge         // ha a[i] >= threshold: a[i], különben 0
    add     x3, x3, x5
    add     x4, x4, #1
.Lcheck2:
    cmp     x4, x1
    b.lt    .Lloop2
    mov     x0, x3
    ret
    .size   sum_if_csel, .-sum_if_csel

    .section .note.GNU-stack,"",%progbits
```

A `sum_if_asm` ciklusában két feltételes elágazás van: a `b.lt .Lskip` (adatfüggő) és a `b.lt .Lloop` (ciklusvezérlő, kiszámítható). A `sum_if_csel` csak a ciklusvezérlő elágazást tartalmazza; a `CSEL` a nullaregisztert (`XZR`) választja, ha a feltétel hamis.

A teszthajtó előbb ellenőrzi, hogy a két bemenet igaz-darabszáma egyezik, és mind a négy (függvény, bemenet) pár eredménye megegyezik a C-referenciával. Csak ezután mér: melegítés, 11 ismétlés, medián és IQR, elemenkénti nanoszekundumban. A `lab-a051.c` tartalma:

```
/* lab-a051.c - elágazásminta-kísérlet: rendezett vs. permutált bemenet. */
#define _GNU_SOURCE
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#ifndef LAB_N
#define LAB_N  (1L << 20)       /* elemszám */
#endif
#define WARMUP 2
#define REPS   11               /* páratlan: egyértelmű medián */

long sum_if_asm (const long *a, long n, long threshold);
```

```c
long sum_if_csel(const long *a, long n, long threshold);
typedef long lab_fn(const long *, long, long);

/* C-referencia (a kísérlet nem ezt méri; csak a helyességet vetjük össze vele). */
static long sum_if_ref(const long *a, long n, long threshold)
{
    long s = 0;
    for (long i = 0; i < n; i++)
        if (a[i] >= threshold) s += a[i];
    return s;
}

/* Determinisztikus álvéletlen-generátor (xorshift64), hogy a permutáció reprodukálható
 * legyen. */
static uint64_t lab_rng = 0x9E3779B97F4A7C15u;
static uint64_t lab_next(void)
{
    lab_rng ^= lab_rng << 13; lab_rng ^= lab_rng >> 7; lab_rng ^= lab_rng << 17;
    return lab_rng;
}

static double now_ns(void)
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) { perror("clock_gettime"); exit(2); }
    return ts.tv_sec * 1e9 + ts.tv_nsec;
}

static int cmp_double(const void *p, const void *q)
{
    double a = *(const double *)p, b = *(const double *)q;
    return (a > b) - (a < b);
}

int main(void)
{
    const long n = LAB_N, threshold = LAB_N / 2;  /* q = P(a[i] >= threshold) = 0,5 pontosan */
    long *sorted = malloc(n * sizeof *sorted), *perm = malloc(n * sizeof *perm);
    lab_fn *fn[2] = { sum_if_asm, sum_if_csel };
    const char *fname[2] = { "sum_if_asm", "sum_if_csel" };
    const long *in[2] = { NULL, NULL };
    const char *iname[2] = { "rendezett", "permutalt" };
    int rc = 1;

    if (!sorted || !perm) goto out;

    /* Azonos értékkészlet: 0..n-1, egyszer rendezve, egyszer Fisher-Yates-permutációval. */
    for (long i = 0; i < n; i++) sorted[i] = i;
    memcpy(perm, sorted, n * sizeof *perm);
    for (long i = n - 1; i > 0; i--) {
        long j = (long)(lab_next() % (uint64_t)(i + 1));
        long t = perm[i]; perm[i] = perm[j]; perm[j] = t;
    }
    in[0] = sorted; in[1] = perm;

    /* Ellenőrzés a mérés előtt: azonos igaz-darabszám és mind a négy eredmény = referencia. */
    long ref = sum_if_ref(sorted, n, threshold), trues[2] = { 0, 0 };
    for (int d = 0; d < 2; d++)
        for (long i = 0; i < n; i++) trues[d] += in[d][i] >= threshold;
    printf("n=%ld threshold=%ld igaz: rendezett=%ld permutalt=%ld\n",
           n, threshold, trues[0], trues[1]);
    if (trues[0] != trues[1] || sum_if_ref(perm, n, threshold) != ref) goto out;
    for (int f = 0; f < 2; f++)
```

```
        for (int d = 0; d < 2; d++)
            if (fn[f](in[d], n, threshold) != ref) {
                printf("FAIL %s / %s\n", fname[f], iname[d]);
                goto out;
            }
    printf("ellenorzes: mind a negy eredmeny = %ld\n", ref);

    /* Mérés: melegítés, majd REPS ismétlés, medián és IQR, elemenként ns-ban. */
    for (int f = 0; f < 2; f++)
        for (int d = 0; d < 2; d++) {
            double t[REPS];
            volatile long sink = 0;
            for (int w = 0; w < WARMUP; w++) sink += fn[f](in[d], n, threshold);
            for (int r = 0; r < REPS; r++) {
                double t0 = now_ns();
                sink += fn[f](in[d], n, threshold);
                t[r] = (now_ns() - t0) / (double)n;
            }
            qsort(t, REPS, sizeof t[0], cmp_double);
            printf("%-11s %-9s median %7.3f ns/elem  IQR %6.3f\n",
                   fname[f], iname[d], t[REPS / 2], t[(3 * REPS) / 4] - t[REPS / 4]);
            (void)sink;
        }
    rc = 0;
out:
    free(sorted); free(perm);
    return rc;
}
```

Build és futtatás. A fenti két fájlból, tiszta könyvtárban, kiegészítés nélkül:

```
# Csak objektummá fordítás (statikus vizsgálathoz):
aarch64-linux-gnu-gcc -c lab-a051_asm.S -o lab-a051_asm.o
aarch64-linux-gnu-objdump -d lab-a051_asm.o

# A teljes labor (target: AArch64 Linux, qemu-user):
aarch64-linux-gnu-gcc -std=c11 -O2 -Wall -Wextra -static \
    -o lab-a051 lab-a051.c lab-a051_asm.S
qemu-aarch64 ./lab-a051 ; echo "exit=$?"
```

A kimenet első két sora determinisztikus (mérve: GNU + glibc + QEMU 8.2.2, a lektor mérése): `n=1048576 threshold=524288 igaz: rendezett=524288 permutalt=524288`, majd `ellenorzes: mind a negy eredmeny = 412316598272`; a kilépési állapot 0. A négy időzítési sor értékei futásfüggők; QEMU alatt a vendég feltételes elágazásai a host processzoron futnak, ezért a **host** prediktora hat. A mért különbség valós, de a host gépre jellemző, nem az Arm-célra.

A QEMU 8.2.2 alatt, x86-64 hoston, $n = 2^{20}$ elemre mért medián értékek (ns/elem): `sum_if_asm` rendezett 3.515, permutált 10.080 (2.87×); `sum_if_csel` rendezett 3.730, permutált 4.026 (1.08×). A hatás tehát QEMU alatt sem tűnik el, de a mért arány a host prediktoráé; natív AArch64-en külön mérendő.

A `-static` elkerüli a dinamikus linker QEMU-beli bonyodalmát. A QEMU TCG dinamikus bináris fordítója nem az Arm prediktorát modellezi, hanem a vendég feltételes elágazását host-elágazássá fordítja, ezért a host prediktora a mérésben megjelenik. A mért különbség valós, de a host mikroarchitektúrájára jellemző; a célhardverre vonatkozó következtetéshez natív AArch64 mérés kell (Raspberry Pi 4/5).

Hipotetikus modellszámítás. Az alábbi táblázat nem mérés, hanem egy egyszerű modell kiértékelése a következő, **feltételezett** paraméterekkel: elágazásos változatnál 4 ciklus/elem a tévesztés nélküli törzsidő, `CSEL`-es változatnál 5 ciklus/elem, a tévesztési büntetés 20 ciklus,

$n = 10^6$, $q = 0{,}5$. A tévesztési arány a rendezett bemenetnél elhanyagolható (néhány tévesztés a váltásnál), a permutáltnál a fenti modell alsó korlátja, $\min(q, 1-q) = 0{,}5$; a `CSEL`-es változatban nincs adatfüggő elágazás.

Változat	Bemenet	Adatfüggő tévesztési arány (modell)	Hipotetikus ciklusszám (10^6 elem)
feltételes elágazás	rendezett	0	$10^6 \cdot 4 \approx 4{,}0 \cdot 10^6$
feltételes elágazás	permutált	0,5	$10^6 \cdot 4 + 10^6 \cdot 0{,}5 \cdot 20 = 14{,}0 \cdot 10^6$
`CSEL`	bármelyik	nincs adatfüggő elágazás	$10^6 \cdot 5 = 5{,}0 \cdot 10^6$

Ha a bemenetben a feltétel 90%-ban igaz ($q = 0{,}9$), ugyanez a modell permutált sorrendre is csak 0,1 tévesztési arányt ad: $4 \cdot 10^6 + 10^6 \cdot 0{,}1 \cdot 20 = 6{,}0 \cdot 10^6$ ciklus — ekkor a `CSEL` előnye kicsi, vagy el is tűnik.

A QEMU-ban mért időket mindig a fenti korlát figyelembevételével kell értelmezni. A Mikrobenchmark készítése és eredmények értelmezése alfejezet részletesen tárgyalja a mérési módszertant.

13.3.5 Tipikus hibák és ellenpróbák

1. **A fordító elágazás nélkülire alakítja a kódot.** `-O2` szinten a GCC gyakran `CSEL`-t vagy `CSINC`-et generál az `if` helyett (if-conversion). Ezért a C változat mérése félrevezető lehet. A lektor mérése szerint a GCC 13.3 `-O2` a fejezet `sum_if` függvényét valóban `csel`-lel fordítja (if-conversion). Ellenpróba: `objdump -d` segítségével ellenőrizzük a generált assemblyt. Ha a cél a feltételes elágazás mérése, kézzel írt assemblyt vagy `-O0`-t használjunk.
2. **A rendezett és véletlen tömb nem ugyanazokat az elemeket tartalmazza.** Ellenpróba: a két tömböt ugyanabból az adathalmazból készítsük, csak az egyiket permutáljuk. Az igaz esetek számának és az összegnek meg kell egyeznie — a teszthajtó ezt a mérés előtt ellenőrzi. Ha az igaz-arány eltér, a két mérés nem csak a sorrendben különbözik.
3. **A ciklus túl rövid.** A prediktor tanulási ideje és a mérési zaj miatt kis N esetén nem látszik a különbség. Ellenpróba: $N \geq 10^6$ elemet használjunk, és több futás mediánját és szóródását (IQR) közöljük.
4. **A QEMU a host prediktorát használja.** A TCG nem az Arm prediktorát szimulálja, de a vendég elágazását host-elágazássá fordítja, ezért a host prediktora megjelenhet a mérésben. Ellenpróba: a mérést fizikai AArch64 hardveren is megismételni, és az eredményeket külön jelölni.

13.3.6 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd el, miért lehet lassabb egy véletlen elágazásminta ugyanannyi utasítás mellett, mint egy rendezett. Térj ki a prediktor működésére és a misprediction penalty fogalmára. Hogyan változik a válasz, ha a ciklus törzse 1 utasításból áll, és a misprediction penalty 15 ciklus?

2. (implementációs, középszint — cél: rutinépítés) Írj A64 assembly rutint, amely egy `long` tömb elemeit összegzi úgy, hogy a negatív értékeket az abszolút értékükkel helyettesíti. Készíts két

változatot: (a) feltételes elágazással (`b.ge`/`b.lt`), (b) `CSEL`-lel elágazás nélkül. Fordítsd le `gcc-aarch64-linux-gnu`-val, futtasd `qemu-aarch64`-val, és mérd az időt `clock_gettime`-nal. Készíts rendezett és véletlen bemenetet. Hasonlítsd össze az eredményeket, és indokold a különbséget.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Adott az alábbi C program, amely `-O2`-vel fordítva feltételes elágazást tartalmaz, mert a feltételes ág két függő frissítést végez:

```
long sum_if(const long *a, long n, long t) {
    long s = 0, cnt = 0;
    for (long i = 0; i < n; i++)
        if (a[i] >= t) { s += a[i]; cnt++; }
    return s + cnt;
}

long sum_if_ternary(const long *a, long n, long t) {
    long s = 0, cnt = 0;
    for (long i = 0; i < n; i++) {
        long hit = (a[i] >= t);
        s   += hit ? a[i] : 0;
        cnt += hit;
    }
    return s + cnt;
}
```

`objdump -d` segítségével keresd meg az elágazást. Ezután módosítsd a kódot úgy, hogy a fordító `CSEL`-t generáljon (például `cond ? x : y` kifejezéssel). Mérd meg mindkét változat idejét rendezett és véletlen bemeneten. Magyarázd el, miért tér el a két változat viselkedése.

13.3.7 Megoldások

1. megoldás — Segítség. Számítsd ki az átlagos ciklusköltséget mindkét esetre külön: rendezettnél a prediktor szinte mindig helyesen jósol, véletlennél csak az esetek feléről — mi az elvárt érték egy súlyozott átlagban?

1. megoldás — Teljes megoldás. A prediktor a korábbi kimenetelek alapján mintázatot tanul. Rendezett bemenet esetén a feltétel hosszú sorozatokban azonos, így a prediktor közel 100%-os találati arányt ér el. Véletlen sorrendű bemenetnél a találati arány a kimenetelek eloszlásától függ: független, q valószínűséggel igaz kimeneteleknél legfeljebb $\max(q, 1-q)$; a kiegyensúlyozott, $q = 0{,}5$ esetben ez 50%. Minden téves jóslatnál a processzor eldobja a spekulatívan végrehajtott utasításokat, és újratölti a pipeline-t: ez a misprediction penalty, nagyságrendileg 10–20 ciklus (magfüggő). A feladatban megadott 15 ciklusos értékkel: ha a ciklus törzse 1 utasítás, a helyes végrehajtás 1 ciklus, a tévesztés 15 ciklus plusz. A véletlen esetben a futási idő jelentősen megnő. Számszerűen: rendezett esetben 1 ciklus/iteráció, véletlen esetben $0{,}5 \cdot 1 + 0{,}5 \cdot (1 + 15) = 8{,}5$ ciklus/iteráció. Ez a levezetés feltételezi, hogy a prediktor a rendezett mintát tökéletesen jósolja, a véletlen bemenet pedig független és kiegyensúlyozott ($q = 0{,}5$). Ha például $q = 0{,}9$, az állandó „igaz” jóslat 90%-ot ér el, és a véletlen eset várható ideje $0{,}9 \cdot 1 + 0{,}1 \cdot 16 = 2{,}5$ ciklus/iteráció.

2. megoldás — Segítség. Az `if`-ágas változat szinte szóról szóra lefordítja a `cmp`/`b.ge`/`neg`/összegzés sorrendet; a `CSEL`-es változat előre kiszámítja MINDKÉT lehetséges eredményt (`a[i]` és `-a[i]`), majd a feltétel alapján választ köztük elágazás nélkül.

2. megoldás — Teljes megoldás. Az assembly rutinok:

```
// long sum_abs_if(const long *a, long n)
// x0 = a, x1 = n
    mov     x2, #0              // s = 0
```

```
    mov     x3, #0              // i = 0
    b       .Lcheck
.Lloop:
    ldr     x4, [x0, x3, lsl #3]
    cmp     x4, #0
    b.ge    .Lpos
    neg     x4, x4
.Lpos:
    add     x2, x2, x4
    add     x3, x3, #1
.Lcheck:
    cmp     x3, x1
    b.lt    .Lloop
    mov     x0, x2
    ret
```

CSEL-es változat:

```
// long sum_abs_csel(const long *a, long n)
// x0 = a, x1 = n
    mov     x2, #0              // s = 0
    mov     x3, #0              // i = 0
    b       .Lcheck2
.Lloop2:
    ldr     x4, [x0, x3, lsl #3]
    neg     x5, x4              // x5 = -a[i]
    cmp     x4, #0
    csel    x4, x4, x5, ge      // ha a[i] >= 0, x4 marad, különben x5
    add     x2, x2, x4
    add     x3, x3, #1
.Lcheck2:
    cmp     x3, x1
    b.lt    .Lloop2
    mov     x0, x2
    ret
```

A méréshez a 13.3 `lab-a051.c` teszthajtójának átalakított változata használható. A cserélendő részek: a `lab_fn *fn[2]` tömb elemei `sum_abs_if` és `sum_abs_csel`, a `fname` tömb pedig ezek nevei; a `sum_if_ref` helyére `sum_abs_ref` kerül; a két bemenet ugyanazt az értékkészletet tartalmazza (például $-n/2 \ldots n/2$), az egyikben a negatív értékek elöl, a másikban xorshift64 + Fisher–Yates után. A referencia:

```
static long sum_abs_ref(const long *a, long n)
{
    long s = 0;
    for (long i = 0; i < n; i++) {
        long v = a[i];
        s += v < 0 ? -v : v;
    }
    return s;
}
```

A mérési törzs (melegítés, 11 ismétlés, medián, IQR) változatlanul használható.

Az A64-nek van a `neg`+`cmp`+`csel` hármasnál idiomatikusabb, egyutasításos megoldása is: `cmp x4, #0` után `cneg x4, x4, lt` a negálást és a kiválasztást együtt végzi. Megjegyzés: `INT64_MIN` abszolút értéke nem ábrázolható `int64_t`-ben, ezért a bemenet ne tartalmazzon `INT64_MIN`-t; a fenti értékkészlet ezt kizárja. A mért idő QEMU-ban illusztratív; a feltételes elágazás változatánál a véletlen minta lassabb lehet, a CSEL változatnál a minta nem befolyásolja az időt.

3. megoldás — Segítség. `objdump -d`-ben keress `b.`-kezdetű feltételes elágazás mnemonicot az eredeti programban; az `-O2`-vel generált `CSEL`-nél viszont nincs ilyen, csak egy `CMP` és egy `CSEL`.

3. megoldás — Teljes megoldás. `-O2` mellett a GCC gyakran `CSEL`-t generál. Például:

```
long f(long x, long y) {
    return x > y ? x : y;
}
```

A generált kód (GCC 13.3, `-O2`, a lektor mérése; az operandussorrend és a feltételkód fordítófüggő):

```
    cmp     x1, x0
    csel    x0, x1, x0, ge
    ret
```

Ha feltételes elágazást akarunk, `-O0`-t vagy `volatile`-t használhatunk, vagy kézzel írhatunk assemblyt. A mérésnél a `CSEL` változat időben független a bemeneti sorrendtől, míg a feltételes elágazás változat a véletlen mintánál lassabb. Az eltérés oka a misprediction penalty. A disassembly ellenőrzése `objdump -d`-vel történik.

13.3.8 Rövid összefoglaló

Az elágazás-előrejelző (branch predictor) a korábbi kimenetelekből tanuló mintázatfelismerő. Kiszámítható (rendezett) elágazásminta esetén a találati arány közel 100%. Független, véletlen kimeneteleknél a találati arány legfeljebb $\max(q, 1-q)$ — kiegyensúlyozott esetben 50%, de egy 90%-ban igaz feltételnél 90% —, és minden téves jóslat (misprediction) nagyságrendileg 10–20 ciklus (magfüggő) veszteséget okoz. Ugyanaz az utasításszám mellett a véletlen elágazásminta ezért lényegesen lassabb lehet. A `CSEL` feltételes kiválasztás elkerüli az adatfüggő elágazást, cserébe mindkét ág kiszámításáért. QEMU-ban a mért elágazáshatás a host prediktorát tükrözi, ezért nem hardverhű; a mérést fizikai AArch64 hardveren kell validálni.

Továbbhasznált definíciók:

- **Elágazás-előrejelzés (branch prediction):** a processzor azon képessége, hogy egy feltételes elágazás kimenetelét a végrehajtás előtt megjósolja.
- **Misprediction penalty:** a téves jóslat miatt eldobott spekulatív munka és a helyes útvonal betöltésének ciklusvesztesége.
- **If-conversion:** feltételes elágazás átalakítása feltételes utasítássá (például `CSEL`), így elkerülve az elágazást.

13.3.9 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — A64 utasítások, memóriahozzáférés, `CSEL`. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- QEMU dokumentáció — user-mode emuláció, TCG-korlátok. **Tartalmilag ellenőrizve**: a `master` ág dokumentációja (nem konkrét kiadott verzió); a hivatalos user-mode fejezet nem állít semmit a branch prediction modellezéséről — ez a TCG funkcionális (nem ciluspontos) jellegéből következik, nem egy explicit QEMU-dokumentált korlát.

- GNU assembler kézikönyv — `asm` szintaxis, direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak:

- A `lab-a051` helyességi része és a négy időzítési sor mérve (GNU + glibc + QEMU 8.2.2, x86-64 host, a lektor mérése); a determinisztikus kimeneti sorok ezzel egyeznek, a kilépési állapot 0. A QEMU-s időzítés a host prediktorát tükrözi, ezért nem hardverhű.
- A QEMU TCG branch prediction viselkedése QEMU user-mode alatt nem ellenőrzött itt; a mérésekhez fizikai AArch64 hardver javasolt.
- A konkrét ARM magok misprediction penalty értékei (például Cortex-A76, Apple M1) a mikroarchitektúra-specifikus dokumentációból ellenőrizendők; a 10–20 ciklus itt illusztratív nagyságrend.

13.4 Mikrobenchmark készítése és eredmények értelmezése

13.4.1 Motiváció: a mérés mint mérnöki műszer

Egy assembly-optimalizálás akkor válik mérnöki döntéssé, ha a „gyorsabbnak tűnik” helyett mérhető, ismételhető és értelmezhető adatunk van. A C-fejlesztő gyakran egyetlen `time`-futásból következtet, pedig a modern AArch64 rendszerben az eredményt cache-állapot, frekvencia-skálázás, ütemező, megszakítások és a QEMU TCG emulációs sajátosságai is torzítják. Ez az alfejezet egy újrafelhasználható mikrobenchmark-sablont ad, és megmutatja, hogyan kell az eredményt táblázatban, ábrán és mérnöki korlátokkal együtt közölni. A fejezet korábbi mérései — cache-viselkedés 13.1, pipeline-akadályok 13.2, elágazás-előrejelzés 13.3 — csak közös statisztikai keretben hasonlíthatók össze. A NEON-mérés tanulsága 12.5 szintén ide fut ki: gyorsulást csak azonos bemeneten, ismételt futással, mediánnal és szóródással szabad közölni.

13.4.2 Definíció és hatókör

Mikrobenchmark: kis, izolált kódrészlet (kernel) ismételt, kontrollált futtatása, amelynek célja egy konkrét gépi szintű hatás — például memóriahozzáférési minta, adatfüggőség vagy elágazás-kiszámíthatóság — relatív költségének becslése. Nem azonos egy teljes alkalmazás benchmarkjával: itt nem a felhasználói élményt, hanem egy mechanizmust mérünk.

A sablon A64 ISA-szemléletű, Linux user-mode környezetben fut: `qemu-aarch64` alatt vagy natív AArch64 hoston. Előfeltétel, hogy a kernel önmagában is összeálljon, a mérési kör pedig képes legyen a hőmérséklet, frekvencia és rendszerterhelés legalább kvalitatív kontrolljára. QEMU-ban a kapott idő emulációs idő, nem target-ciklusszám; a QEMU TCG nem modellezi hardverhűen a cache-t, a pipeline-t és a branch predictort. Ezért a mikrobenchmark QEMU-ban elsősorban algoritmikus és utasításszámbeli különbségeket mutat, mikroarchitektúra-hatást csak natív AArch64 hardveren lehet validálni.

A mérési lánc négy részből áll: kernel, harness, statisztika, értelmezés. A kernel a mérendő assembly vagy C függvény. A harness méri az időt és ismétel. A statisztika mediánt, átlagot, interkvartilis terjedelmet (IQR) és kiugró értékeket számol. Az értelmezés a kontrollkísérletekkel együtt mondja meg, mit jelent az adat.

13.4.3 A mikrobenchmark-sablon

A sablon kötelező lépései:

1. **Bemelegítés (warm-up).** Az első futások feltöltik a cache-t, a TLB-t, a branch predictorokat, és QEMU-ban a TCG translation cache-t. A bemelegítő futásokat nem mérjük, csak a rendszer állapotát visszük közel egy stabil pontra. Általában 3–10 bemelegítés elég, de a pontos szám a kernel hosszától függ.
2. **Ismétlés.** Egyetlen mérés helyett sok mintát veszünk. Egy mintán belül a kernel többször fut (belső iteráció), hogy az idő nagyobb legyen, mint az óra felbontása. A mintákból statisztikát számolunk. A cél nem a legjobb vagy legrosszabb érték, hanem a jellemző érték és a szóródás.
3. **Medián és átlag.** A medián robusztus a szélsőséges értékekre; az átlag érzékeny rájuk. Mindkettőt érdemes közölni. Ha a medián és az átlag lényegesen eltér, valószínűleg kiugró értékek vagy aszimmetrikus eloszlás van.
4. **Kiugró érték (outlier) kezelése.** Egyetlen mérés nem elég, mert egy megszakítás, laphiba vagy frekvenciaváltás könnyen 10–100× torzít. Az IQR-alapú szabály: $IQR = Q_3 - Q_1$, kiugró az az érték, amely kívül esik a $[Q_1 - 1{,}5 \cdot IQR,\ Q_3 + 1{,}5 \cdot IQR]$ intervallumon. A kiugrókat nem kell feltétlenül törölni; számoljuk meg, és közöljük. A hallgatói jegyzőkönyvben a nyers minták is maradjanak meg.
5. **Kontrollmérés.** Minden kísérlethez tartozzon baseline: például üres ciklus, azonos hosszú de független lánc, vagy szekvenciális hozzáférés. Az arány csak azonos feltételek mellett értelmezhető.

13.4.4 Kidolgozott példa: cache-érzékeny összegzés

Munkakönyvtár: `~/arm-labor/a64/a052/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a052
cd ~/arm-labor/a64/a052
```

Az alábbi A64 assembly kernel egy `long` tömböt összegez, megadott bájt-lépéssel. `stride=8` szekvenciális, `stride=64` cache-line léptetésű hozzáférést jelent (64 bájtos cache-sort feltételezve). A kernel önmagában nem tartalmaz cache-optimalizálást; éppen a memóriahozzáférési minta mérése a cél. A `lab-a052-sum.S` tartalma:

```
// lab-a052-sum.S - A64 ISA, AArch64 Linux; GNU as.
// long lab_sum_stride(const long *p, long n, long stride)
// x0 = p, x1 = n, x2 = stride bájtban. Levélfüggvény, CFI nélkül (lásd 8.3).
    .text
    .p2align 2
    .global lab_sum_stride
    .type lab_sum_stride, %function
lab_sum_stride:
    mov x3, #0              // sum = 0
    cmp x1, #0
    b.le 2f
1:
    ldr x4, [x0]            // A64: 64 bites betöltés
    add x3, x3, x4
    add x0, x0, x2          // következő cím
    subs x1, x1, #1
```

```
    b.ne 1b
2:
    mov x0, x3
    ret
    .size lab_sum_stride, .-lab_sum_stride

    .section .note.GNU-stack,"",%progbits
```

Két kísérlet, két különböző kérdés. Ha az elemszám rögzített, a nagyobb lépés nemcsak a cache-sorok kihasználását változtatja meg, hanem a **bejárt memóriatartományt** is: $n = 65\,536$ elemnél `stride=8` mellett 512 KiB-ot, `stride=64` mellett közel 4 MiB-ot jár be a kernel, más lap- és TLB-igénnyel. Az eltérés ezért nem rendelhető kizárólag egyetlen cache-mechanizmushoz. A harness ezért két üzemmódot ismer:

Üzemmód	Rögzített	Változik a stride-dal	Mit izolál jobban
`n` (alapértelmezés)	elemszám, $n = 2^{16}$	bejárt tartomány (`bytes`), érintett lapok száma	a lépés teljes hatását egy adott elemszámra
`span`	tartomány, 4 MiB	elemszám, $n = 4\,\text{MiB}/\text{stride}$	a laponkénti és soronkénti kihasználtságot azonos *címtartomány* mellett

Mikor azonos a lapkészlet? A `span` mód azonos címtartományt jár be, de az *érintett lapok halmaza* csak akkor azonos, ha a stride nem nagyobb a lapméretnél (4 KiB lapokon pl. `stride` = 8 és 64: mindkettő mind az 1024 lapot érinti). `stride = 8192` esetén $n = 512$ elem csak minden második lapot érinti (512 lap a 1024-ből), tehát a TLB- és lapkészlet **más**; az „azonos TLB-terhelés” állítás ezért csak a `stride  4096` tartományra érvényes. A 8192-es esetet érdemes külön lefuttatni; ott a lapkészlet eltér (512 lap a 1024-ből), ezért az „azonos TLB-terhelés” állítás arra nem érvényes.

Egyik kísérlet sem választ le teljesen egyetlen mechanizmust (például a hardveres előbetöltés mindkettőben szerepet játszhat); a kettő együtt mutatja meg, mennyi múlik a tartományon és mennyi a lépésen. A napló ezért minden sorban kiírja az üzemmódot, `n`-t, `stride`-ot és `bytes`-t.

A C harness bemelegít, ismétel, majd mediánt, átlagot és IQR-t számol. A mérési időt az ismétlésszámmal és az elemszámmal is osztjuk, így az egység ns/elem. A `lab-a052-bench.c` tartalma:

```
/* lab-a052-bench.c - stride-mikrobenchmark két kísérleti üzemmóddal.
 * Használat: lab-a052 STRIDE [n|span]
 *   n    (alapértelmezés): rögzített elemszám, n = LAB_N; a bejárt tartomány a stride-dal nő
 *   span : rögzített tartomány, bytes = LAB_SPAN; az elemszám n = bytes / stride */
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

extern long lab_sum_stride(const long *p, long n, long stride);

#ifndef LAB_N
```

```
#define LAB_N    (1L << 16)          /* 65 536 elem */
#endif
#ifndef LAB_SPAN
#define LAB_SPAN ((size_t)4 << 20)   /* 4 MiB */
#endif
#ifndef INNER
#define INNER 1000
#endif
#define WARMUP 3
#define SAMPLES 31

static int cmp_double(const void *a, const void *b) {
    double da = *(const double *)a, db = *(const double *)b;
    return (da > db) - (da < db);
}

static double now_ns(void) {
    struct timespec t;
    if (clock_gettime(CLOCK_MONOTONIC, &t) != 0) { perror("clock_gettime"); exit(2); }
    return (double)t.tv_sec * 1e9 + (double)t.tv_nsec;
}

static double measure_once(const long *p, long n, long stride) {
    volatile long sink = 0;
    double t0 = now_ns();
    for (int i = 0; i < INNER; i++)
        sink += lab_sum_stride(p, n, stride);
    double t1 = now_ns();
    __asm__ volatile("" : : "r"(sink) : "memory");
    return (t1 - t0) / INNER / n;
}

static int usage(void) {
    fputs("Hasznalat: lab-a052 STRIDE [n|span]\n"
          "  STRIDE: pozitiv, sizeof(long)-gal oszthato bajtlepes\n", stderr);
    return 2;
}

int main(int argc, char **argv) {
    char *end = NULL;
    errno = 0;
    long stride = (argc > 1) ? strtol(argv[1], &end, 0) : (long)sizeof(long);
    if (argc > 3 || errno || (argc > 1 && (!*argv[1] || *end)) ||
        stride <= 0 || (size_t)stride % sizeof(long))
        return usage();
    int fixed_span = 0;
    if (argc > 2) {
        if (strcmp(argv[2], "span") == 0) fixed_span = 1;
        else if (strcmp(argv[2], "n") != 0) return usage();
    }

    long n;
    size_t bytes;
    if (fixed_span) {
        bytes = LAB_SPAN;
        n = (long)(bytes / (size_t)stride);          /* (n-1)*stride + 8 <= bytes */
        if (n < 2) return usage();
    } else {
        n = LAB_N;
        if ((size_t)stride > (SIZE_MAX - sizeof(long)) / (size_t)(n - 1)) return 2;
        bytes = (size_t)(n - 1) * (size_t)stride + sizeof(long);
    }
```

```
    long *p = malloc(bytes);
    if (!p) return 1;
    for (long i = 0; i < n; i++)
        *(long *)((unsigned char *)p + (size_t)i * (size_t)stride) = i;
    /* Helyességi ellenőrzés a mérés előtt: a kernel tényleg az n elemet összegzi. */
    if (lab_sum_stride(p, n, stride) != n * (n - 1) / 2) { free(p); return 1; }

    volatile long sink = 0;
    for (int w = 0; w < WARMUP; w++)
        sink += lab_sum_stride(p, n, stride);

    double t[SAMPLES];
    for (int s = 0; s < SAMPLES; s++)
        t[s] = measure_once(p, n, stride);

    /* Nyers minták: a rendezés ELŐTT, konfigurációval, sorszámmal, egységgel (ns/elem). */
    for (int s = 0; s < SAMPLES; s++)
        printf("raw mod=%s n=%ld stride=%ld idx=%d t=%.3f ns/elem\n",
               fixed_span ? "span" : "n", n, stride, s, t[s]);
    double sorted[SAMPLES];                          /* a statisztika másolaton számol */
    memcpy(sorted, t, sizeof sorted);
    qsort(sorted, SAMPLES, sizeof sorted[0], cmp_double);
    double sum = 0.0;
    for (int s = 0; s < SAMPLES; s++) sum += sorted[s];
    double mean = sum / SAMPLES;
    double median = sorted[SAMPLES / 2];
    double q1 = sorted[SAMPLES / 4];
    double q3 = sorted[(3 * SAMPLES) / 4];
    int outliers = 0;                                /* kimondott szabály: Tukey-kerítés, 1,5·IQR
    ↪  */
    for (int s = 0; s < SAMPLES; s++)
        if (sorted[s] < q1 - 1.5 * (q3 - q1) || sorted[s] > q3 + 1.5 * (q3 - q1)) outliers++;

    printf("mod=%s n=%ld stride=%ld bytes=%zu median=%.3f ns/elem mean=%.3f ns/elem IQR=%.3f
↪   outliers=%d\n",
           fixed_span ? "span" : "n", n, stride, bytes, median, mean, q3 - q1, outliers);
    free(p);
    return 0;
}
```

Az eredmény felhasználása nem helyességi garancia. A `volatile sink` és az üres `asm volatile` akadály azt akadályozza meg, hogy a fordító a hívásokat mint felesleges munkát elhagyja. Arról azonban nem szól, hogy a kernel azt számolja-e, amit kell, vagy hogy a mért ciklus azonos-e a vizsgálttal. Ezt három külön lépés adja: a mérés előtti összegellenőrzés (`n * (n - 1) / 2`), a `aarch64-linux-gnu-objdump -d --disassemble=lab_sum_stride` ellenőrzése, és az, hogy a kernel külön assembly-fájlban van, így a C-fordító nem írhatja át.

Build és futtatás a fenti két fájlból, qemu-user módban, illetve natív AArch64 hoston:

```
aarch64-linux-gnu-gcc -std=c11 -O2 -Wall -Wextra -static \
    -o lab-a052 lab-a052-bench.c lab-a052-sum.S
# 1. kísérlet: rögzített elemszám
qemu-aarch64 ./lab-a052 8
qemu-aarch64 ./lab-a052 64
# 2. kísérlet: rögzített memóriatartomány
qemu-aarch64 ./lab-a052 8 span
qemu-aarch64 ./lab-a052 64 span
qemu-aarch64 ./lab-a052 8192 span   # eltérő lapkészlet: 512 lap a 1024-ből
# natív AArch64 hoston ugyanez qemu-aarch64 nélkül, pl.: ./lab-a052 64 span
# A konfigurációk futási sorrendje ne legyen rögzített: váltogasd vagy fix seeddel keverd,
# őrizd meg a futási sorrendet a nyers adatok mellett, azonos bemelegítéssel, pl.:
```

```
#   for cfg in 8 64 64 8 8 64; do qemu-aarch64 ./lab-a052 $cfg; done   # fordított kontrollmérés
↪  is
# Ha a fordított sorrend lényegesen változtatja a következtetést, közöld mérési
↪  bizonytalanságként.
```

A mért összegző sorok (QEMU 8.2.2, **a lektor mérése**; futásonként 31 `raw` minta és egy összegző sor; a `bytes` értékei a fenti definíciókból számítottak, és a méréssel egyeznek):

```
mod=n    stride=8    bytes=524288   median=2.488 ns/elem IQR=0.797
mod=n    stride=64   bytes=4194248  median=3.640 ns/elem IQR=0.464
mod=span stride=8    n=524288       median=2.867 ns/elem IQR=0.292
mod=span stride=64   n=65536        median=3.520 ns/elem IQR=0.566
mod=span stride=8192 n=512          median=6.469 ns/elem
```

A program ezen felül átlagot és kiugrószámot is kiír; érvénytelen stride (például 12) esetén a használati üzenet jelenik meg, 2-es kilépési állapottal.

QEMU-ban a *vendég* cache-t a TCG nem modellezi, de ez nem azt jelenti, hogy cache-hatás nincs: a mért idő a host és az emuláció együttes költsége, amelyben a host cache-e és az emuláció memóriakezelése is szerepet játszik. A fenti mérésben a stride 64-re váltás +46%-ot ad (2,488 → 3,640 ns/elem); QEMU-ban a host cache-e miatt is látható különbség (pl. +30–50%); ennek oka a host, nem az Arm-cél. Ezért nem jósolunk univerzálisan kis (vagy nagy) különbséget; a hallgató bármelyik QEMU-eredményt elfogadhatóan értelmezi, ha a következtetést az emulációra korlátozza. A célhardver cache-következtetéséhez natív mérés kell. Natív AArch64-en a cache-line léptetés lassulása nagyobb lehet; a mértékét és azt, hogy mennyi jut belőle a tartományra (1. kísérlet) és mennyi a lépésre (2. kísérlet), mérni kell. A példa tanulsága: a sablon képes adatot adni, de az adat értelmezése mindig tartalmazza a platform korlátját és a kísérlet felépítését.

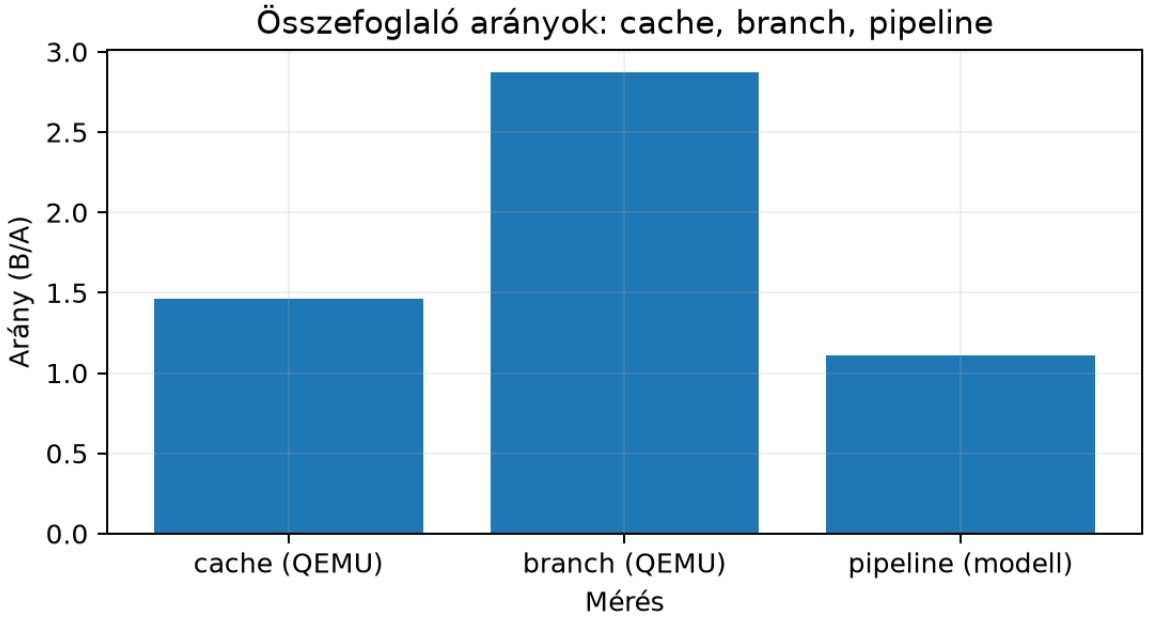
13.4.5 Eredmény-összefoglaló táblázat és numerikus ábra

A fejezet három mérése — cache, pipeline, branch — közös táblázatban foglalható össze. A platform oszlop jelöli, melyik érték mérés és melyik jelölt modell számítása; egyik sem mért hardveradat.

Mérés	Változat A	Változat B	A érték	B érték	Arány	Platform / megjegyzés
Cache (13.1)	szekvenciális	64 bájtos stride	2,49 ns/elem	3,64 ns/elem	1,46×	QEMU 8.2.2, mért (13.4-01)
Pipeline (13.2)	független lánc	függő lánc	5010 ciklus	4517 ciklus	1,11×	jelölt modell, nem mérés
Branch (13.3)	rendezett	permutált	3,515 ns/elem	10,080 ns/elem	2,87×	QEMU 8.2.2, host prediktor (13.3-01)

Mérés	Változat A	Változat B	A érték	B érték	Arány	Platform / megjegyzés
Branch	rendezett	véletlen	28	73	2,61	branch predictor találati arány, misprediction penalty

Ugyanez numerikus ábrán:



13.3. ábra: Összefoglaló arányok: cache, branch, pipeline - A cache- és branch-arány QEMU 8.2.2 mérés (a lektor mérése), a pipeline-arány a jelölt modell számított értéke; eltérő kontrollelemek miatt az arányok nem rangsorolhatók.

Az ábra és a táblázat ugyanazt az értelmezési keretet kéri. Először normalizáljunk a baseline-hoz: minden arány az A változat mediánjához viszonyít. Másodszor nézzük az IQR-t: ha az IQR nagyobb, mint a két medián különbsége, a következtetés bizonytalan. Harmadszor ellenőrizzük a kontrollelemeket: cache-nél a munkahalmaz méretét, pipeline-nál az utasításszámot, branch-nél az elágazások számát. Végül a platformot: QEMU-ban a három közül egyik mikroarchitektúra-hatás sem mérhető hardverhűen, ezért a QEMU-eredmény csak relatív, emulációs jelzés.

13.4.6 Mire NEM használható ez a sablon

A mikrobenchmark-sablon nem ad **hardverhű ciklusszám-garanciát**. Nem alkalmas valós idejű határidők igazolására, mert a legrosszabb végrehajtási idő (WCET) nem mediánból származik. Nem használható különböző mikroarchitektúrák összehasonlítására önmagában: egy Cortex-A76 és egy QEMU TCG által mért idő nem ugyanaz a mérőszám. Nem helyettesíti az alkalmazásszintű

benchmarkot, mert a teljes program viselkedését nem írja le. Nem ad biztonsági tanúsításhoz elegendő bizonyítékot. Végül nem megbízható, ha a kernel túl rövid az óra felbontásához, ha a fordító kioptimalizálja, vagy ha a mérést frekvencia-skálázás, termikus throttling vagy zajos szomszédos folyamatok terhelik.

13.4.7 Tipikus hibák és ellenpróbák

- **Nincs bemelegítés.** Tünet: az első minta kiugróan lassabb. Ellenpróba: dobjuk el az első N mintát, és nézzük a medián stabilitását.
- **Egyetlen mérés.** Tünet: másodpercenként változó eredmény. Ellenpróba: 31 minta, medián és IQR közlése.
- **A fordító eltünteti a kernelt.** Tünet: 0 ns vagy azonos idő minden változatra. Ellenpróba: `volatile` sink, `asm volatile` memória-akadály, vagy a függvény külön assembly fájlban.
- **QEMU-t hardverként kezeljük.** Tünet: cache/pipeline/branch hatás eltűnik. Ellenpróba: natív AArch64 futtatás, vagy a következtetés korlátozása emulációs jelzésre.
- **Kiugrók elrejtése.** Tünet: túl szép medián. Ellenpróba: nyers hisztogram vagy legalább az outlierek számának közlése.

13.4.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Fejtsd ki, miért robusztusabb a medián, mint az átlag, amikor a mérési sorozatban megszakítások és laphibák vannak. Adj példát olyan számsorra, ahol az átlag félrevezető, a medián nem.

2. (implementációs, középszint — cél: rutinépítés) Bővítsd a C harnesst a medián bootstrap-konfidenciaintervallumával: 2000 újramintavétel a nyers mintákból, majd a 2,5 és 97,5 percentilis kiírása (`median_ci=lo..hi`). Futtasd `stride=8` és `stride=64` értékkel, és hasonlítsd össze a CI-ket.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) A hallgató azt tapasztalja, hogy `stride=8` és `stride=64` esetén azonos mediánt kap, QEMU-ban. Adj legalább három lehetséges magyarázatot, és mindegyikhez egy ellenőrző lépést.

4. (értelmezési, alapszint — cél: megértés-ellenőrzés) A táblázatban a cache-arány 1,46, a branch-arány 2,87. Miért nem szabad azt mondani, hogy „a cache nagyobb hatású, mint a branch”, ha a mérések különböző platformon és különböző kontrollelemekkel készültek?

13.4.9 Megoldások

1. megoldás — Segítség. Képzelj el egy mérési sorozatot, ahol a legtöbb érték közel azonos, de EGYETLEN minta sokszorosan nagyobb (pl. egy megszakítás miatt) — mit tesz ez az átlaggal, és mit a rendezett sorozat középső elemével?

1. megoldás — Teljes megoldás. Ha a sorozat például $[10, 11, 10, 12, 500]$ ns, az átlag 108,6 ns, a medián 11 ns. A 500 ns egy megszakítás vagy laphiba miatti kiugró érték. A medián azt mondja, hol van a tipikus futás; az átlag a szélsőség miatt eltolódik. Ezért kell mediánt és IQR-t közölni.

2. megoldás — Segítség. A bootstrap lényege: a mért mintából visszatevéssel húzz ugyanannyi elemet, számíts mediánt, és ismételd meg sokszor. A 2000 medián rendezett sorozatából a 2,5%

és 97,5% index adja a CI-t; ügyelj a `rand()` helyett minőségi PRNG-re és a reprodukálható seedre.

2. megoldás — Teljes megoldás. A bootstrap-váz a meglévő `samples[]` tömbre épül:

```
static int cmp_double(const void *p, const void *q);
static double median_of(double *v, size_t n);   /* a harness meglévő mediánja */

static void bootstrap_ci(const double *samples, size_t n, int reps,
                         double *lo, double *hi) {
    double *res = malloc((size_t)reps * sizeof *res);
    double *tmp = malloc(n * sizeof *tmp);
    if (!res || !tmp) { perror("malloc"); exit(2); }
    for (int r = 0; r < reps; ++r) {
        for (size_t i = 0; i < n; ++i)
            tmp[i] = samples[rand() % n];      /* visszatevéses minta */
        res[r] = median_of(tmp, n);
    }
    qsort(res, (size_t)reps, sizeof *res, cmp_double);
    *lo = res[(int)(0.025 * reps)];
    *hi = res[(int)(0.975 * reps) - 1];
    free(res); free(tmp);
}
```

A kimenetbe `median_ci=lo..hi` kerül. A CI szélessége a minta szórását és az `n`-et tükrözi; QEMU-ban a stride növelése kicsi vagy nulla mediánkülönbséget adhat, de a CI átfedése alapján eldönthető, hogy a különbség a zajon belül van-e. Natív hardveren a cache-hatás nagyobb mediánt és esetleg szélesebb CI-t ad.

3. megoldás — Segítség. Gondold át külön-külön: mit tud(hat) a QEMU TCG a cache-ről; mekkora a ténylegesen bejárt munkahalmaz a választott n-nél; biztos, hogy a fordító valóban lefuttatja a ciklust minden iterációban?

3. megoldás — Teljes megoldás. Lehetséges okok: (a) a QEMU TCG nem modellezi a cache-t, ezért a stride hatása eltűnik; ellenőrzés: natív AArch64 futtatás. (b) a munkahalmaz kisebb, mint a cache, és a stride nem elég nagy; ellenőrzés: növeljük n-t és a stride-ot. (c) a fordító kioptimalizálta a tömb bejárást; ellenőrzés: `objdump -d` a kernel disassemblyjén, és `volatile`/`asm` akadályok ellenőrzése. (d) az óra felbontása vagy a kernel túl rövid; ellenőrzés: növeljük az `INNER` értéket.

4. megoldás — Segítség. Nézd meg a fejezet “Mire NEM használható” szakaszát — mi hiányzik ahhoz, hogy két KÜLÖNBÖZŐ mérési eset arányait közvetlenül összevethesd?

4. megoldás — Teljes megoldás. Az arány önmagában nem összehasonlítható, mert más a mérési platform, más a kontrollelem, és a QEMU nem modellezi egyik mechanizmust sem hardverhűen. A cache-arányt befolyásolja a munkahalmaz mérete, a branch-arányt az elágazásminta hossza és a prediktor állapota. Csak azonos platformon, azonos kernelhosszon és azonos statisztikai protokollal szabad összevetni.

13.4.10 Rövid összefoglaló

A mikrobenchmark izolált kernel ismételt, statisztikailag kezelt mérése. A sablon bemelegítést, sok mintát, mediánt, átlagot és IQR-t használ; a kiugrókat meg kell számolni, nem eltüntetni. A cache-, pipeline- és branch-mérések közös kerete a baseline-hoz normalizálás, a szóródás ellenőrzése és a kontrollelemek azonosítása. QEMU-ban az idő emulációs idő, mikroarchitektúra-hatás csak natív AArch64-en validálható. A sablon nem ad hardverhű ciklusszám-garanciát, és nem helyettesíti a valós idejű vagy alkalmazásszintű mérést.

Továbbhasznált definíciók:

- **Mikrobenchmark:** izolált kódrészlet ismételt mérése statisztikai kiértékeléssel.
- **Medián:** a rendezett minta középső értéke, robusztus a kiugrókra.
- **IQR:** az interkvartilis terjedelem, $Q_3 - Q_1$, a szóródás robusztus mérőszáma.

13.4.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile architektúra-dokumentáció — A64 utasítások, memóriahozzáférés, architektúra-profilos szemlélet. Ellenőrizve: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- QEMU dokumentáció — user-mode emuláció, TCG-korlátok, emulációs idő értelmezése. **Tartalmilag ellenőrizve**: a `master` ág hivatalos user-mode dokumentációja (`user/main.html`) nem állít semmit a cache-, pipeline- vagy branch-predictor-modellezésről — a fejezet szövegében szereplő "a QEMU TCG nem modellezi hardverhűen" állítás ezért nem egy explicit QEMU-dokumentált korlátból, hanem a TCG ismert tervezési elvéből (funkcionális, utasításszintű emulátor, nem cikluspontos szimulátor) következik. A dokumentáció emellett a `master`, nem egy konkrét kiadott verzió állapotát tükrözi.
- Arm Architecture Reference Manual for A-profile architecture (DDI0487) — A64 utasításkészlet hivatalos referenciája. Ellenőrizve: a link él (a `developer.arm.com` → `support.arm.com` átirányítás megerősített); JS-renderelt oldal, tartalmilag nem ellenőrizhető automatikus lekéréssel.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a `lab-a052` mindkét üzemmódját és argumentum-ellenőrzését QEMU 8.2.2 alatt, x86-64 hoston mértük (31 minta, medián és IQR); az itt közölt időértékek ebből a mérésből származnak. Natív AArch64 hardveren nem történt mérés: a QEMU TCG cache/pipeline/branch modellezésének aktuális részletei, a `clock_gettime(CLOCK_MONOTONIC)` tényleges felbontása a célplatformon, a frekvencia-skálázás és termikus throttling hatása, valamint a konkrét natív AArch64 hardver viselkedése külön validálást igényel, ezért a natív hardverre vonatkozó következtetés ebben az alfejezetben **nem mérve – csak hardveren** ellenőrizhető.

14 Biztonsági szemlélet: verem, védelem és ROP

14.1 Verem-túlcsordulás mechanizmusa assembly szinten

14.1.1 Motiváció: a C-ben elkövetett hiba assembly szinten válik láthatóvá

A határellenőrzés nélküli másolás a C-programok egyik legrégebbi és legveszélyesebb hibaosztálya. A `strcpy`, a `gets` vagy a `sprintf` korlátlanul ír, ha a célpuffer mérete nem elegendő. A C-fejlesztő számára ez „csak” egy memóriahiba; assembly szinten viszont pontosan látszik, hogy a másolás a verem szomszédos tartalmát írja felül. Ebben az alfejezetben kontrollált demonstrációval megmutatjuk, hogyan íródik felül a mentett visszatérési cím (return address), és miért lesz ebből vezérlésátvétel. A cél nem egy működő exploit elkészítése, hanem a mechanizmus és a védelem megértése. A példa a saját, jelölt labor-binárisunkra épül, fix bemenettel, és a túlcsordulást még a `RET` előtt detektálja.

14.1.2 Definíció: verem-túlcsordulás

Verem-túlcsordulás (stack buffer overflow): olyan írási művelet, amely egy verembeli puffer határán túlra ír, és ezzel a szomszédos veremtartalmat — lokális változókat, mentett keretmutatót (frame pointer, FP) vagy mentett visszatérési címet (link register, LR) — módosítja. Nem azonos a **veremkimerüléssel** (stack exhaustion), amikor végtelen rekurzió miatt elfogy a veremterület. Itt a verem mérete elegendő, de egy másolás túlnyúlik a kijelölt pufferen.

Az AAPCS64 szerinti verem lefelé növekszik: a függvény az `SP`-t kisebb címre állítja. Ebben a példában a keret alján (alacsonyabb címen) helyezkedik el a lokális puffer, felette a mentett `X29`/`X30` pár. Egy felfelé, növekvő címre író másolás először a puffert, majd a mentett FP-t, végül a mentett LR-t éri el. Pontosan ez a sebezhető elrendezés.

Veremterület	Offset az `SP`-hez képest	Tartalom
Lokális puffer	0–31	32 bájtos tömb
Mentett `X29` (FP)	32–39	hívó keretmutatója
Mentett `X30` (LR)	40–47	visszatérési cím
Kihasználatlan tartalék	48–63	kihasználatlan (a példa kedvéért kerekített keret; 48 bájt is elég lenne az igazításhoz)

A táblázat a példabeli 64 bájtos keretre vonatkozik. A konkrét elrendezés fordító- és optimalizálási szintfüggő; a lényeg, hogy a puffer és a mentett LR között nincs hardveres határ.

14.1.3 Hogyan lesz a túlírásból felülírás?

A másoló ciklus minden bájtot egymás után ír a puffer kezdetétől. A 0-tól számozott indexek közül a 32. már a mentett `X29`, a 40. a mentett `X30` első bájtját éri el: ehhez rendre 33, illetve 41 bájtos másolás kell. A szokásos epilógus az `LDP` utasítással tölti vissza a mentett `X30`-at; a `RET` ezután a regiszterben lévő címre ugrik. Ha a felülírt mentett értéket ellenőrzés nélkül töltenénk vissza, a `RET` nem a hívóba térne vissza, hanem az így kapott címre. Ez a **vezérlésátvétel** (control-flow hijack) fogalmi szintje: a támadó által befolyásolt érték a program számlálójába (PC) kerül. A példában ezt nem érjük el, mert a `RET` előtt ellenőrizzük a mentett LR épségét, és szándékosan leállítjuk a programot.

A mentett FP felülírása önmagában nem ad vezérlést, de a láncolt keretlistát és a hibakeresést megzavarja. A lokális változók sérülése pedig akár a másolás méretét vagy egy mutatót is módosíthat, ami további hibákhoz vezet.

14.1.4 Kidolgozott példa: kontrollált túlcsordulás A64 assembly-ben

Munkakönyvtár: `~/arm-labor/a64/a053/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a053
cd ~/arm-labor/a64/a053
```

Az alábbi `lab-overflow.S` egyetlen fájl, amely `_start` belépési ponttal fut. **CFI nélküli demonstráció:** a `lab_vuln_copy` keretet épít, de szándékosan nem kap `.cfi_*` direktívákat — a vizsgálat tárgya éppen a veremben lévő keretrekord, ezért a hívási lánc itt csak a keretlánc (`X29`) alapján, és a felülírás után már úgy sem követhető (a CFI-szabályt lásd a Prológus és epilógus alfejezetben). A `lab_vuln_copy` függvény 32 bájtos puffert feltételez, de a hosszt nem ellenőrzi. A forrás 48 bájt: 32 bájt minta, 8 bájt a mentett FP helyére, 8 bájt hamis LR érték. A hamis LR (`0xdeadbeefcafebabe`) szándékosan nem érvényes cím, és soha nem hajtódik végre, mert a detektálás megelőzi a `RET`-et.

```
// lab-overflow.S - A64 ISA, AArch64, little-endian
    .section .data
src:
    .ascii "AAAAAAAAAAAAAAAA"   // 16 bájt
    .ascii "BBBBBBBBBBBBBBBB"   // 16 bájt -> puffer 32 bájt
    .quad  0x1122334455667788   // mentett x29 felülírása
    .quad  0xdeadbeefcafebabe   // mentett x30 felülírása
src_end:
    .equ SRC_LEN, src_end - src // 48

    .section .text
    .global _start
_start:
    adrp x0, src
    add  x0, x0, :lo12:src
    mov x1, #SRC_LEN
    bl  lab_vuln_copy
    // Ide normál esetben vissza kellene térni.
    mov x0, #0
    mov x8, #93          // exit
    svc #0

// x0 = forrás, x1 = hossz
lab_vuln_copy:
    sub sp, sp, #64
```

```
    stp x29, x30, [sp, #32]   // mentett FP/LR a puffer felett
    add x29, sp, #32
    mov x2, #0
1:
    cmp x2, x1
    b.ge 2f
    ldrb w3, [x0, x2]
    strb w3, [sp, x2]         // nincs határellenőrzés
    add x2, x2, #1
    b   1b
2:
    // Defenzív ellenőrzés: a mentett LR érintetlen?
    ldr x3, [x29, #8]         // mentett x30
    cmp x3, x30               // x30 = eredeti visszatérési cím
    b.ne lab_overflow_detected
    ldp x29, x30, [sp, #32]
    add sp, sp, #64
    ret

lab_overflow_detected:
    mov x0, #1                // jelzett kilépési kód
    mov x8, #93
    svc #0

    .section .note.GNU-stack,"",%progbits   // nem kér végrehajtható vermet
```

Build és futtatás (qemu-user, AArch64):

```
aarch64-linux-gnu-gcc -c -o lab-overflow.o lab-overflow.S
aarch64-linux-gnu-ld -o lab-overflow lab-overflow.o
readelf -lW lab-overflow | grep GNU_STACK   # elvárt jelző: RW
qemu-aarch64 ./lab-overflow
echo $?
```

A `GNU_STACK` ellenőrzése a kész ELF-re vonatkozik. A forrásbeli direktíva a szándékot rögzíti, de egy hiányzó direktívából önmagában nem következik végrehajtható verem: hogy a linker jelzés nélküli objektumból mit állít elő, az eszközlánc- és verziófüggő.

Várható kimenet: a program az `lab_overflow_detected` ágra kerül, ezért a kilépési kód `1`. A `0xdeadbeefcafebabe` hamis LR nem hajtódik végre. A `qemu-aarch64` itt statikus binárist futtat, külső `-L` rendszerkönyvtár nélkül.

Ellenőrzés GDB-vel (gdb-multiarch):

```
qemu-aarch64 -g 1234 ./lab-overflow  # az egyik terminálban
# A másik terminálban:
gdb-multiarch -q ./lab-overflow
(gdb) set architecture aarch64
(gdb) break lab_overflow_detected
(gdb) target remote localhost:1234
(gdb) continue
(gdb) info registers x29 x30 sp
(gdb) x/8gx $sp
```

A töréspontnál a `sp` a keret aljára mutat. A `[sp+32]` és `[sp+40]` tartományban a felülírt mentett FP és LR látható. A `$x30` ekkor még az eredeti visszatérési cím, a `[x29, #8]` pedig a hamis érték; a `cmp` mutatja az eltérést.

14.1.5 Kapcsolat a C-s buffer overflow hibaosztállyal

C-ben a tipikus minta:

```
void f(const char *input) {
    char buf[32];
    strcpy(buf, input);   // nincs méretellenőrzés
}
```

A `strcpy` addig másol, amíg nullát nem lát. Ha az `input` hosszabb 31 bájtnál, ugyanaz a veremfelülírás történik, mint a példában. A `-fstack-protector-strong` kapcsolóval a fordító a védelemre kijelölt, ténylegesen verembeli objektumot tartalmazó függvényekben **canary** (stack canary) értéket helyez el; a függvény epilógusa ellenőrzi annak épségét. A tényleges generált kódot kell vizsgálni: az optimalizálás a fel nem használt lokális puffert el is tüntetheti. A `_FORTIFY_SOURCE` megfelelő fordítási feltételek és ismert célobjektumméret mellett ellenőrzött változatot, például `__strcpy_chk`-t rendelhet a `strcpy` hívásához. Ezek a védelmek a Védelmi mechanizmusok: canary, ASLR, NX/DEP, PAC alfejezetben részletesen szerepelnek. A mostani példa canary nélküli, ezért a mentett LR közvetlenül elérhető.

14.1.6 Tipikus hibák és ellenpróbák

- **Az elrendezés nem garantált.** A fordító a puffert a mentett LR alá vagy fölé is teheti. A példa szándékosan a sebezhető sorrendet mutatja. Más elrendezés vizsgálatakor a mentési címeket, az `X29` képzését, a puffer kezdőcímét és a visszatöltést együtt kell módosítani; egyetlen prológusutasítás cseréje nem elég.
- **Az `X30` változhat.** A `cmp x3, x30` csak akkor érvényes, ha a függvény nem hív más függvényt. Ha `bl`-t használunk, az `X30` felülíródik, és a detektálás hibás. Ilyenkor a mentett LR-t kell referenciaként elmenteni.
- **Igazítás.** Az `SP` 16 bájtos igazítása miatt a keretméret 16 többszöröse; a 48 bájt is megfelelne, a példa 64 bájtos kerete egyszerűen kerekített, a 48–63 közötti rész kihasználatlan tartalék.
- **A másolás iránya.** A bemutatott bájtonkénti ciklus növekvő címre ír. A C-könyvtári másolófüggvények belső hozzáférési sorrendjét és műveleti szélességét ne feltételezzük az itteni ciklus alapján; azok implementációfüggők.
- **A `RET` mást is használhat.** A `RET xN` alakban a megadott regiszterből tér vissza; a példa a `RET` alapértelmezett `X30` viselkedését mutatja.

Ellenpróbák a példán (a `_start mov x1, #SRC_LEN` sorát `mov x1, #32`-re, illetve `#40`-re cserélve): 32 bájtos bemenet → nincs túlcsordulás, a program `0` kóddal lép ki. 40 bájtos bemenet → a mentett FP felülíródik, de a mentett LR nem; a detektálás nem jelez, a `ret` a helyes címre tér vissza, de a `X29` hamis. 48 bájtos bemenet → a mentett LR is felülíródik; a detektálás `1` kóddal leáll.

14.1.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Az alapdemonstráció 64 bájtos keretében add meg a puffer, a mentett `X29` és a mentett `X30` offsetjét az `SP`-hez képest. Melyik másolási hossznál íródik felül a mentett LR első bájtja? **2. (implementációs, középszint — cél: rutinépítés)** Készítsd el az alapdemonstráció canary-val kiegészített változatát `lab-overflow-canary.S` néven. A `lab_vuln_copy` a puffer utáni 32–39. offsetre írja ki a `0x0123456789abcdef` értéket **a másolás előtt**, majd a másolás után hasonlítsa össze az eredeti értékkel. Eltéréskor a `lab_overflow_detected` út állítsa le a programot. Add meg a teljes keretelrendezést, az `X29`

képzését, a mentett FP/LR helyét és a sikeres epilógust is. Az eredeti 48 bájtos forrás 0, 32, 33, 40 és 48 bájtos prefixével végezzen ellenpróbát: az első két eset térjen vissza, a többi jelezzen canary-sérülést. A hívó a sikeres visszatérés után külön ellenőrizze az `SP` és az `X29` eredeti értékét. **3. (hibakeresési/mérési, középszint — cél: rutinépítés)** Az eredeti, canary nélküli `lab-overflow.S` programban GDB-vel állíts töréspontot a `strb` utasításra. Figyeld az `x2` regisztert és a `[sp, x2]` címet. Melyik iterációnál íródik felül a mentett LR? Add meg az `x2` értékét és a `strb` által írt bájt címét.

14.1.8 Megoldások

1. megoldás — Segítség. Nézd meg a fejezet eleji offset-táblázatot, és gondold át: a másolás a 0. indextől indul — hány bájtot kell írni ahhoz, hogy az index elérje a `SP+40`-et?

1. megoldás — Teljes megoldás. Puffer: `SP+0`–`SP+31`. Mentett `X29`: `SP+32`–`SP+39`. Mentett `X30`: `SP+40`–`SP+47`. A mentett LR első bájtja a `SP+40` címen van. Mivel a másolás a 0. indextől indul, a 40. indexű bájt írásához 41 bájtos másolás szükséges. A 40 bájtos másolás a 0–39. indexeket fedi, így a `SP+40`-et még nem éri el.

2. megoldás — Segítség. A canary a puffer és a mentett FP/LR közé kerüljön. Előbb rajzold fel a teljes keretet, majd egyeztesd vele a mentést, az `X29` értékét és a visszatöltést. Mikor kell a jelzőértéket a verembe írni? Mely regiszter őrizheti az összehasonlítás referenciáját, ha a másoló ciklus nem hív más függvényt?

2. megoldás — Teljes megoldás. Az alábbi **önálló, teljes programot** mentsd `lab-overflow-canary.S` néven. Az alapdemonstráció forrásadatait használja, és a canary alapján dönt; az eredeti `lab-overflow.S` változatlanul megtartható összehasonlításra.

CFI nélküli demonstráció: az alapváltozathoz hasonlóan a `lab_vuln_copy` itt is keretet épít `.cfi_*` direktívák nélkül, mert a vizsgálat tárgya maga a verembeli keretrekord (a CFI-szabályt lásd a Prológus és epilógus alfejezetben). A választott keret 80 bájtos, így az `SP` 16 bájtos igazítása megmarad. A függvény törzsében az offsetek a keret lefoglalása utáni `SP`-hez képest értendők:

Offset	Tartalom
0–31	32 bájtos puffer
32–39	Canary: `0x0123456789abcdef`
40–47	Kitöltés
48–55	Mentett `X29`
56–63	Mentett `X30`
64–79	Kitöltés

Az `X29 = SP + 48` a mentett FP/LR párra mutat. A canary első bájtját 33, az új helyen lévő mentett FP első bájtját 49, a mentett LR első bájtját 57 bájtos másolás érné el. A mostani 48 bájtos forrás tehát a canary-t és az utána álló kitöltést írja felül, a feljebb helyezett FP/LR párt még nem éri el.

```
// lab-overflow-canary.S - A64 ISA, AArch64 Linux, little-endian
    .section .data
src:
    .ascii "AAAAAAAAAAAAAAAA"
    .ascii "BBBBBBBBBBBBBBBB"
    .quad  0x1122334455667788
    .quad  0xdeadbeefcafebabe
```

```
src_end:
    .equ SRC_LEN, src_end - src       // 48 bájt, mint az alapdemóban

    // A --defsym COPY_LEN=N az N bájtos prefixet választja ki.
    .ifndef COPY_LEN
    .equ COPY_LEN, SRC_LEN
    .endif
    .if COPY_LEN < 0
    .error "COPY_LEN nem lehet negativ"
    .endif
    .if COPY_LEN > SRC_LEN
    .error "COPY_LEN nagyobb a forras meretenel"
    .endif

    .section .text
    .global _start
_start:
    // Hívói keret: érvényes, nullával lezárt keretrekord.
    sub sp, sp, #16
    stp xzr, xzr, [sp]
    mov x29, sp
    mov x19, sp                       // elvárt SP és x29; a hívott nem írja
    adrp x0, src
    add  x0, x0, :lo12:src
    mov x1, #COPY_LEN
    bl  lab_vuln_copy

    // Csak a sikeres visszatérési út jut ide.
    mov x4, sp
    cmp x4, x19
    b.ne lab_frame_error
    cmp x29, x19
    b.ne lab_frame_error
    add sp, sp, #16                   // a hívó saját keretének felszabadítása
    mov x0, #0
    b   lab_exit

// x0 = forrás, x1 = hossz; a laborban 0 <= x1 <= SRC_LEN.
// A másolás szándékosan nem vizsgálja a 32 bájtos pufferhatárt.
lab_vuln_copy:
    sub sp, sp, #80
    stp x29, x30, [sp, #48]
    add x29, sp, #48

    movz x6, #0xcdef
    movk x6, #0x89ab, lsl #16
    movk x6, #0x4567, lsl #32
    movk x6, #0x0123, lsl #48
    str x6, [sp, #32]                 // inicializálás a másolás ELŐTT

    mov x2, #0
1:
    cmp x2, x1
    b.hs 2f                           // hossz és index: előjel nélküli
    ldrb w3, [x0, x2]
    strb w3, [sp, x2]
    add x2, x2, #1
    b   1b
2:
    ldr x5, [sp, #32]
    cmp x5, x6                        // x6-ot a ciklus nem módosította
    b.ne lab_overflow_detected
    add x7, sp, #48
```

```
    cmp x29, x7                     // az x29 a saját keretrekordra mutat?
    b.ne lab_frame_error
    ldp x29, x30, [sp, #48]
    add sp, sp, #80
    ret

lab_overflow_detected:
    mov x0, #1                      // a canary megváltozott
    b   lab_exit
lab_frame_error:
    mov x0, #2                      // hibás x29-képzés, vagy SP/x29 nem állt helyre
lab_exit:
    mov x8, #93                     // Linux SYS_exit; nem tér vissza
    svc #0

    .section .note.GNU-stack,"",%progbits
```

Az `X6` a `0x0123456789abcdef` referenciaértéket tartja, a `STR` ugyanezt írja a guard helyére. Little-endian memóriában a bájtsor `ef cd ab 89 67 45 23 01`. A ciklus csak az `X2` és `W3` regisztert módosítja, nem hív más függvényt, ezért az `X6`-ban lévő referencia megmarad. Ha a rutin később hívásokat kap, az `X6` caller-saved szerepe miatt a referencia megőrzését újra kell tervezni.

A sikeres út előbb a hívott oldalán ellenőrzi, hogy az `X29` valóban a saját keretrekordjára (`SP + 48`) mutat; ez a **képzést** vizsgálja. Ezután az epilógus a párt a **48-as offsetről** tölti vissza, és **80 bájtot** ad az `SP`-hez. A hívó az `X19`-ben megőrzött címhez hasonlítja az `SP`-t és az `X29`-et; ez a **visszaállítást** vizsgálja. Bármelyik eltérésnél `2` a program kilépési állapota. Canary-sérüléskor `1` az állapot: ez az ág befejezi az egész demonstrációt, ezért nem hajt végre epilógust és `RET`-et.

Build és egy futás (GNU eszközlánc + QEMU):

```
aarch64-linux-gnu-gcc -c -o lab-overflow-canary.o lab-overflow-canary.S
aarch64-linux-gnu-ld -o lab-overflow-canary lab-overflow-canary.o
readelf -lW lab-overflow-canary | grep GNU_STACK   # elvárt: RW
# Az alapértelmezett COPY_LEN = 48 esetben 1 az elvárt állapot.
if qemu-aarch64 ./lab-overflow-canary; then rc=0; else rc=$?; fi
printf 'exit=%s\n' "$rc"
```

Önellenőrzés az öt bemenetre (bash): a `--defsym` csak a másolási hosszt változtatja; a forrásadat és a függvény ugyanaz marad. Az `if` az elvárt `1` állapotot is kezeli akkor, ha a shellben a `set -e` be van kapcsolva. A ciklus alhéjban fut, így az `exit 1` hibánál csak az alhéjat állítja le, interaktív terminálban sem zárja be a shellt.

```
(
for n in 0 32 33 40 48; do
    aarch64-linux-gnu-gcc -c -Wa,--defsym,COPY_LEN=$n \
        -o lab-overflow-canary.o lab-overflow-canary.S || exit 1
    aarch64-linux-gnu-ld -o lab-overflow-canary \
        lab-overflow-canary.o || exit 1
    if qemu-aarch64 ./lab-overflow-canary; then rc=0; else rc=$?; fi
    if [ "$n" -le 32 ]; then want=0; else want=1; fi
    printf 'n=%s exit=%s elvart=%s\n' "$n" "$rc" "$want"
    [ "$rc" -eq "$want" ] || exit 1
done
)
```

Elvárt eredmény:

```
n=0 exit=0 elvart=0
n=32 exit=0 elvart=0
n=33 exit=1 elvart=1
```

```
n=40 exit=1 elvart=1
n=48 exit=1 elvart=1
```

A 33 bájtos prefix utolsó bájtja a forrásban `0x88`, amely az `SP+32` címen lévő `0xef` canary-bájtot változtatja meg. A sérülés így nem a verem korábbi tartalmától függ. A 0 és 32 bájtos futás csak akkor ad `0`-t, ha a jelző sértetlen, és az `SP`/`X29` is helyreállt.

Ez a fordítói stack protector alapelvét szemlélteti: a mentett vezérlési adatok felhasználása előtt ellenőrizzük a közéjük és a puffer közé tett jelzőt. **A canary nem akadályozza meg a túlírást, és az egyezése nem bizonyítja a másolás szabályosságát.** A fix, nyilvános érték **determinisztikus oktatási jelző**: azért választottuk, hogy a próba ismételhető legyen, és változatlan bájtokkal felül is lehet írni. A valódi védelemhez megfelelő fenyegetési modell, rendszerint kiszámíthatatlan guard és a biztonságos bemenethossz-kezelés is szükséges; a mostani ellenpróbák a guardot biztosan megváltoztató mintát használják.

3. megoldás — Segítség. A ciklus `x2`-t 0-tól indexeli; ugyanazzal a logikával, mint az 1. feladatban, keresd meg, melyik `x2` érték esik egybe a mentett LR első bájtjának offsetjével.

3. megoldás — Teljes megoldás. A `strb` már az első iterációban (`x2 = 0`) ír, és minden iterációban pontosan egy bájtot. A mentett LR a `[sp+40]` címen kezdődik. Az `x2 = 40` értéknél a `strb w3, [sp, x2]` a `[sp+40]` címre ír, ez az első bájt, amely a mentett `X30`-at érinti. Mivel az index 0-tól indul, ez a **41.** végrehajtott írás: a GDB-ben a 41. iteráció után változik meg a `[sp+40]` tartalma. A detektálás a másolás után, a `2:` címkénél fut, és a `cmp` jelzi az eltérést.

GDB-ben a `strb` címét a `disassemble lab_vuln_copy` listája adja; onnan `break *<cím> if $x2 == 40`, majd `continue`. A `[sp+40]` tartalmát a `stepi` előtt és után is `x/gx $sp+40` mutatja, így nem kell 41 iteráción egyenként végiglépni.

14.1.9 Rövid összefoglaló

A verem-túlcsordulás akkor jön létre, ha egy határellenőrzés nélküli másolás túlnyúlik a pufferen, és felülírja a szomszédos veremtartalmat. A mentett visszatérési cím a `RET` miatt kitüntetett célpont: felülírása vezérlésátvételhez vezethet. A példa ezt kontrolláltan, detektálással állítja meg a `RET` előtt. A C-s `strcpy`-hez hasonló hiba ugyanezt a mechanizmust idézi elő; a canary, a `_FORTIFY_SOURCE` és a fordítói védelmek ezt hivatottak kivédeni.

- **Verem-túlcsordulás:** pufferen túlnyúló írás, amely szomszédos veremtartalmat módosít.
- **Mentett visszatérési cím:** a `BL` által az `X30`-ba írt visszatérési cím verembeli másolata.
- **Canary:** a puffer és a mentett LR közé szúrt ellenőrző érték, amelynek sérülése túlcsordulást jelez.

14.1.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- AAPCS64 – Arm hivatalos ABI-dokumentum — veremkeret, igazítás, keretrekord. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta státuszúak).
- Arm A-profile architektúra-dokumentáció — A64 `STP`, `LDP`, `RET`, `SP`-igazítás. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU assembler kézikönyv — AArch64-szintaxis, direktívák. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.

- GNU GDB kézikönyv — töréspontok, regiszter- és memóriavizsgálat. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).
- GCC kézikönyv: Instrumentation Options — `-fstack-protector`, `-fstack-protector-strong`: a guard belépéskori beállítása és kilépéskori ellenőrzése.
- GNU C Library: Source Fortification — `_FORTIFY_SOURCE`, ismert célobjektumméret és ellenőrzött könyvtári hívások.

A canary-megoldás ellenőrzési állapota: mérve (GNU + QEMU 8.2.2 + GDB 15.1, a lektor mérése). A fenti teljes `lab-overflow-canary.S` programot a fő laborúton, GNU eszközlánccal fordítottuk, `qemu-aarch64` alatt futtattuk, és a GDB-lépéseket is végrehajtottuk. A 0/32/33/40/48 bájtos esetek állapota rendre 0/0/1/1/1, betűre egyezik a fenti elvárt kimenettel. A sikeres utakon a program saját SP/X29-vizsgálata is teljesült. A kísérőcsomag tesztkészletében (`tools/a053/`) a guard inicializáló tárolásának elhagyása a sértetlen bemenetet is hibára vitte; a sikeres epilógus hibás, 64 bájtos SP-felszabadítása és a hibás, `SP + 32`-ből képzett `X29` egyaránt `2` állapotot adott. Ezek a hibás kontrollok a teszt érzékenységét ellenőrzik.

Fennmaradó ellenőrzések: a bemutatott offsetek a kézzel írt demonstrációra érvényesek; egy C-fordító által létrehozott keret elrendezése ettől függetlenül fordító- és optimalizálásfüggő. A fordítói stack protector és a glibc fortifikációja külön mechanizmus; konkrét támogatásuk és generált kódjuk a labor eszközverzióival ellenőrizendő.

14.2 Védelmi mechanizmusok: canary, ASLR, NX/DEP, PAC

14.2.1 Motiváció: a védelem pontos garanciája auditálható állítás

Egy biztonsági audit során ritkán elég annyit mondani, hogy „a bináris védelemmel fordított”. A kérdés mindig az: **mit** véd, **melyik** támadási lépést nehezíti, és **mi az, amit kifejezetten nem** garantál. A gyakorló mérnök számára ez nem elméleti finomság: ha egy kernelmodul vagy beágyazott szolgáltatás fordítói kapcsolóit kell dokumentálni, a „van canary” állítás önmagában megtévesztő lehet, ha a folyamat például nem indít új címtérrel, vagy a hardver nem támogatja a mutatóhitelesítést.

Ebben az alfejezetben négy mechanizmust rendszerezünk: a stack canary-t, az ASLR-t, az NX/DEP-et és az AArch64-specifikus PAC-ot. Mindegyiknél megadjuk a **pontos garanciát**, és külön kiemeljük a **garancia határát**. Az előző alfejezet megmutatta, hogyan lesz a mentett visszatérési címből (`X30` verembeli másolata) a vezérlésátvétel kitüntetett pontja; itt azt vizsgáljuk, milyen rétegek állnak közte és a `RET` utasítás között. A keret végig **defenzív**: a cél a mechanizmusok felismerése és helyes értékelése, nem a megkerülésük receptje. A támadási lépéseket csak annyiban nevezzük meg, amennyiben a védelem hatókörének megértéséhez szükséges.

14.2.2 Definíció: mit jelent egy védelmi mechanizmus garantálni

Védelmi mechanizmus (mitigation): a program végrehajtásának olyan tulajdonsága vagy a fordító/linker/kernel által bevezetett szerkezeti elem, amely egy adott támadási lépés sikerességének valószínűségét csökkenti, vagy annak következményét észlelhetővé teszi.

Három fogalmat érdemes élesen szétválasztani, mert a mindennapi szóhasználat összemossa őket:

Tulajdonság	Jelentés	Példa
Megelőzés (prevention)	a hibás művelet létre sem jön	NX: az adatlap nem végrehajtható
Detektálás (detection)	a hibás művelet megtörténik, de később kiderül	canary: a túlírás észlelhető a visszatérés előtt
Nehezítés (hardening)	a siker valószínűsége csökken	ASLR: a cím nem ismert előre

Ez a hármas tagolás adja a fejezet gerincét. Fontos következmény: **a detektálás nem előzi meg a memóriamódosítást.** A canary csak azt jelzi, hogy a puffer túlírása megtörtént; a támadó által beírt adat a memóriában van, legfeljebb a kihasználás nem folytatódik tovább. Hasonlóképp az ASLR nem *tilt* semmit, csak valószínűséget módosít. A **mélységi védelem (defense in depth)** elve épp abból indul ki, hogy egyik mechanizmus sem ad abszolút garanciát, ezért egymásra rétegzik őket: a támadónak minden réteget külön kell legyőznie, és ha bármelyik réteg kiesik (például egy nem PASLR-alapú rendszerben), a többi még fog.

14.2.3 A négy mechanizmus és garanciájuk

14.2.3.1 Stack canary: detektálás a visszatérés előtt

A stack canary (veremőr, stack guard) egy véletlen érték, amelyet a fordító a helyi pufferek mellé helyez; a konkrét sorrend a veremkeret elrendezésétől függ. A függvény prológusában beíródik, az epilógusban pedig a `RET` előtt összehasonlítódik az eredetivel.

Pontos garancia: ha a puffer túlírása **folytonosan, egy irányban** terjed, és **eléri** a canary által elfoglalt bájtokat úgy, hogy azokat nem pontosan az eredeti értékkel írja felül, akkor a függvény visszatérése előtt a `__stack_chk_fail` hibakezelő lefut, és a folyamat hibával leáll.

Amit nem garantál:

- **Nem észleli, ha a túlírás nem éri el a canary-t.** Egy pufferen belüli, szomszédos lokális változót felülíró írás (például egy mutatót vagy egy hossz-mezőt) érintetlenül hagyja a canary-t, és a függvény hibátlanul tér vissza.
- **Nem észleli a nem folytonos írást.** Ha a támadó egy indexszámítási hibát kihasználva közvetlenül a mentett visszatérési címre ír, a canary érintetlen marad.
- **Nem észlel, ha a canary értéke kiszivárog.** Ha egy külön hiba (például formázott kimenet vagy nem inicializált memória kiolvasása) kiadja a canary bájtjait, az változatlanul visszaírható, és az ellenőrzés sikeres lesz. Ezért a canary hatékonysága **erősen függ a rendszerben jelen lévő információ-szivárgási hibáktól**.
- **Késleltetett:** az észlelés a függvény visszatérésekor történik, nem az írás pillanatában.

A canary **nem** akadályozza meg a memóriamódosítást; csak a vezérlésátvétel lépését vágja el.

14.2.3.2 ASLR: cím-véletlenítés

Az **ASLR (Address Space Layout Randomization)** a folyamat egyes memóriaterületeinek (verem, heap, `mmap`-elt szakaszok, megosztott könyvtárak, PIE esetén maga a programkép) kezdőcímét véletleníti el minden új folyamatindításnál.

Pontos garancia: a támadó számára a célszerkezet címe **nem ismert előre**; a sikerhez vagy cím-információra van szüksége, vagy nagyszámú találgatásra.

Amit nem garantál:

- **Nem véd információ-szivárgás ellen.** Ha egyetlen cím kiszivárog, a layout **relatív eltolásai** (offsetjei) változatlanok maradnak a modulon belül: egy kiszivárgott visszatérési címből a teljes bináris belső szerkezete kiszámítható. Ez az ASLR leggyakoribb gyakorlati kudarca.
- **Nem ad teljes címtér-entrópiát.** A véletlenítés lapkeretben (4 KiB) történik, és a kernel elrendezése korlátozza a lehetséges pozíciókat. A gyakorlati entrópia **jóval kisebb**, mint a címtér mérete sugallná (lásd a levezetést alább).
- **Nem állít meg nem-címfüggő vezérlést.** Ha a támadás nem igényel abszolút címet (például egy adott regiszterre mutató, meglévő kódban rejlő sorozat), az ASLR nem releváns.
- **Nem véd hosszan futó, sok próbát engedő folyamatokat**, ha a hibás kísérlet nem omlasztja össze a szolgáltatást (például `fork`-alapú szerver esetén a címek utódokra öröklődnek).

Az ASLR tehát **nehezítés**, nem megelőzés; és a védelme **részleges**, mert egyetlen kiszivárgott cím deaktiválhatja.

14.2.3.3 NX/DEP: nem-végrehajtható adatterület

Az **NX (No-eXecute)**, más néven **DEP (Data Execution Prevention)** a lapok lapasztal-bejegyzésében jelöli, hogy egy adott memórialapon lévő bájtok **adatként** értelmezendők, nem utasításként. AArch64-en a könyv Linux-céljában ezt a lapleírók végrehajtási jogosultságbitjei adják. A program az **EL0** kivételszinten (végrehajtási szinten) fut; az EL0 és az EL1 címei egyaránt az **EL1&0 címfordítási rezsimhez** tartoznak, amelynek lapasztalait a kernel kezeli, és az MMU érvényesíti. A lapleíróban két külön bit van: az EL0-beli végrehajtásra az `UXN` (Unprivileged eXecute Never) vonatkozik, ez a felhasználói program NX-védelme; a `PXN` (Privileged eXecute Never) a privilegizált, EL1-beli végrehajtást tiltja, például hogy a kernel ne futtasson felhasználói lapon lévő kódot. Fogalmi megjegyzés: az **EL0...EL3** kivételszintek, vagyis a jogosultsági szintek, és a táblabejárás **L0...L3 szintjei** két külön dolog. Az utóbbiak egyetlen címfordítás egymásba ágyazott laptábláinak mélységét jelölik, és nincs „EL0-s laptábla". A Linux általában `W^X` (write XOR execute) szabályt követ: egy lap vagy írható, vagy végrehajtható, de nem mindkettő.

Pontos garancia: a veremre, heapre vagy `.data` szakaszba írt bájtok **nem hajtódnak végre**. A klasszikus kód-injektálás (beírni a gépi kódot a pufferbe, majd a visszatérési címet a pufferre állítani) ettől **meghiúsul**: a vezérlés átvevődhet, de az első utasításnál hardveres laphiba keletkezik (`SIGSEGV`).

Amit nem garantál:

- **Nem véd a meglévő kód újrafelhasználása ellen.** A **ROP (Return-Oriented Programming)** és a JOP épp azt használja ki, hogy a folyamat kód-szakaszában már vannak végrehajtható, rövid utasítássorozatok; az NX ezekre nem hat. Ezért az NX önmagában nem elég; az ASLR és a PAC épp a ROP-lánc összeállítását nehezíti.
- **Nem véd JIT-fordítókat és értelmezőket**, amelyek szándékosan írható és végrehajtható lapokat használnak.
- **Nem jelzi a hibát**, csak megakadályozza a végrehajtást; a `SIGSEGV` a támadónak is visszajelzés.

14.2.3.4 PAC: visszatérési cím kriptográfiai jelölése (AArch64)

A **PAC (Pointer Authentication)** az Armv8.3-A architektúra-kiterjesztés része, tehát **AArch64-specifikus**. Lényege: a mutató felső, a virtuális címtérben nem használt bitjeibe

egy **rövid kriptográfiai aláírás (PAC-mező)** kerül, amelyet a processzor egy, az operációs rendszer által beállított kulccsal (jellemzően `APIA`/`APIB` az utasítás-mutatókhoz, `APDA`/`APDB` az adatmutatókhoz, valamint `APGA` generikus kulcs) számít ki.

A tipikus `pac-ret` minta a veremkeretben:

- a prológusban az `X30` (LR) **aláíródik**: `PACIASP` — az aláírás az `SP`-t is bevonja a módosítóba (context), így egy másik verempozícióra átvitt érték is érvénytelenné válik;
- a verembe az **aláírt** érték mentődik;
- visszatérés előtt `AUTIASP` ellenőrzi és eltávolítja az aláírást, vagy a `RETAA`/`RETAB` utasítás az ellenőrzést és a visszatérést egy lépésben végzi.

Fenyegetési modell. A garancia egy pontosan körülhatárolt helyzetre szól: a támadó a veremben tárolt, aláírt visszatérési címet **módosítja**; a program a függvény **hitelesítő epilógusán** át tér vissza; és a megfelelő kulcs a futtatókörnyezetben **aktív** (lásd a példa 4. lépését).

Melyik utasítás hitelesít? A disassemblyben három alakot kell megkülönböztetni:

Alak	Mit csinál	Hitelesít?
`AUTIASP` (`hint #29`), utána sima `RET`	az `AUTIASP` ellenőrzi az `X30` aláírását, és sikeres esetben eltávolítja; a `RET` csak ugrik	igen, az `AUTIASP`
`RETAA` / `RETAB`	egyetlen utasítás: hitelesít **és** visszatér	igen, maga a `RETAA`
sima `RET`, előtte `AUTIASP` nélkül	csak `PC ← X30`	**nem**

Pontos garancia: ebben a modellben, ha a támadó a mentett visszatérési címet érvényes aláírás előállítása nélkül megváltoztatja, a hitelesítő utasítás (`AUTIASP` vagy `RETAA`) meghiúsul. A processzor ilyenkor a címet jellemzően „mérgezett” értékké alakítja, ami a vezérlésátadásnál azonnali laphibát okoz (`FEAT_FPAC` esetén már maga a hitelesítés kivételt vált ki). Az ellenőrzést azonban **csak az az útvonal** végzi el, amely a hitelesítő utasítást ténylegesen végrehajtja. Egy `AUTIASP` nélküli `RET`-útvonal nem hitelesít, és egy olyan vezérlésátadás sem, amely nem a hitelesítő epilóguson át megy (például egy `BR Xn` egy nem aláírt mutatóval). A `pac-ret` tehát a **visszatérési címet** védi a szabályos epilógusban, nem az összes közvetett vezérlésátadás célját, és nem általános kontrollfolyam-integritás (CFI).

Amit nem garantál:

- **Nem véd minden visszajátszás (replay) ellen.** Ha a támadó nem módosítja, csak **újra felhasználja** egy korábban aláírt értéket, a hitelesítés akkor sikerül, ha három dolog egyezik: ugyanaz a kulcs, ugyanaz a mutatóérték, és ugyanaz a módosító (a `PACIASP`/`AUTIASP` esetén az aláíráskori, illetve a hitelesítéskori `SP`). Egy más `SP`-jű keretbe átvitt aláírt LR ezért nem megy át, egy azonos `SP`-vel futó másik visszatérési ponton viszont igen. Ezért fontos az `SP`-függő módosító, és ezért van jelentősége a veremkeret pontos elrendezésének.
- **Csak az aláírt mutatókat védi.** A program által nem aláírt (például JIT-elt vagy régi objektumfájlból származó), vagy másutt tárolt mutatók védtelenek.
- **Az aláírás erőssége a PAC-mező méretétől függ.** A mező a virtuális címtér szabad felső bitjeibe kerül; 39 bites címtérnél lényegesen több bit áll rendelkezésre, mint 48 vagy 52 bitesnél. A gyakorlati entrópia tehát **konfigurációfüggő**.
- **Hardver- és utasításalak-függő.** A PAuth (`FEAT_PAuth`) az Armv8.3-A része; régebbi magokon nincs. Hogy ilyenkor mi történik, az **az utasítás kódolásától** függ, nem általában „a PAC-utasításoktól”:

  - a `PACIASP`, `AUTIASP` (és a `PACIBSP`/`AUTIBSP`, `XPACLRI`) a **HINT-térbe** kódolt alakok (`HINT #25`, `HINT #29` stb.). PAuth nélküli magon NOP-ként futnak le, a program működik, de **védelem nincs**. Ugyanez a helyzet, ha a mag ismeri a PAuth-ot, de az operációs rendszer nem engedélyezte a kulcsot;
  - a `RETAA`/`RETAB`, a `BRAA`/`BLRAA`, a `LDRAA`, valamint a regiszteroperandusos `PACIA Xd, Xn` és `AUTIA Xd, Xn` **nem** HINT-kódolású. PAuth nélküli magon definiálatlan utasításként `SIGILL`-lel leállítják a programot.

  Ezért a GCC és a Clang alapértelmezett (`armv8-a`) célon a `-mbranch-protection=standard` mellett csak a HINT-alakot használja. `RETAA`-t csak akkor ad ki, ha a cél legalább Armv8.3-A (például `-march=armv8.3-a`), és az ilyen bináris régebbi magon nem fut. Az, hogy a védelem érvényes-e, futásidőben ellenőrizendő (lásd a példát).
- **Nem helyettesíti a canary-t vagy az ASLR-t**: a memóriabeli adatstruktúrák felülírása ellen nem véd.

Kiegészítő, kapcsolódó hardverelem a **BTI (Branch Target Identification)**, saját feltételekkel. A BTI a `BR`/`BLR` közvetett elágazások célpontjait korlátozza a `BTI` landing pad utasításokra (a `paciasp` implicit landing padként is szolgál), de a `RET` célját nem érinti. Csak akkor hat, ha a lap **védett lapként** (guarded page) van leképezve. Ehhez a `FEAT_BTI`-t támogató hardver, az ELF BTI-jegyzete minden linkelt objektumban (lásd a ROP-alfejezet 3. megoldását) és az operációs rendszer támogatása kell. A GCC `-mbranch-protection=standard` kapcsolója a `pac-ret` és a `bti` kombinációját kéri. A kettő együtt sem teljes CFI: a közvetett hívások *érvényes* célpontjai között nem tesz különbséget.

14.2.4 Levezetés: entrópia, és mit jelent a „nehezítés” számszerűen

Feltételek. Tegyük fel, hogy a védelmi mechanizmus N bitnyi független, egyenletes eloszlású véletlent használ, és a támadó egyetlen „vak” próbálkozást tesz, amelynek sikere pontosan akkor következik be, ha a helyes értéket találja el. (Ez erős egyszerűsítés: a valóságban részleges kiszivárgás, ismételt próbák és a hibás próba következményei is számítanak.)

Ekkor egyetlen próba siker-valószínűsége:

$$p = 2^{-N}$$

ASLR-entrópia levezetése. Ha egy S méretű, lapkeretre igazított régiót egy R méretű tartományon belül véletlenítünk, a lehetséges pozíciók száma legfeljebb R/S, így

$$N \approx \log_2 \frac{R}{S} \quad \text{[bit]}$$

Példa: 32 bites címtér ($R = 2^{32}$ B $= 4$ GiB), 8 MiB-os ($S = 2^{23}$ B) veremrégió esetén $N \approx \log_2 2^9 = 9$ bit. Ez **lényegesen kevesebb**, mint amennyit a „32 bites ASLR” kifejezés sugall — a valós rendszerekben a kernel elrendezése és további korlátok miatt a tényleges érték ettől eltérhet, ezért **a helyi gépen kell ellenőrizni**.

Az alábbi táblázat illusztratív, levezetett/irodalmi nagyságrendeket mutat; **egyik érték sem tekintendő itt mért adatnak**, és a tényleges érték kernelkonfiguráció-, illetve hardverfüggő.

Mechanizmus	Illusztratív entrópia N	Egy vak próba siker-valószínűsége 2^{-N}	Jelleg
Stack canary (8 bájt, alsó bájt kötött `0x00`)	56 bit	$1{,}4 \cdot 10^{-17}$	detektálás
Verem-ASLR, 32 bites címtér (levezetve: $R = 2^{32}$, $S = 2^{23}$)	9 bit	$2{,}0 \cdot 10^{-3}$	nehezítés
ASLR, 64 bites PIE + `mmap` (tipikus konfiguráció)	~28 bit	$3{,}7 \cdot 10^{-9}$	nehezítés
PAC-mező, Linux, 48 bites VA (utasításmutatók / adatmutatók, TBI miatt)	~15 / ~7 bit	$3{,}1 \cdot 10^{-5}$ / $7{,}8 \cdot 10^{-3}$	megelőzés/detektálás

A levezetés tanulsága. A négy mechanizmus nem ugyanazt a támadási lépést zárja el. Az ASLR alacsony entrópiája önmagában nem elég: 9 bit azt jelenti, hogy néhány ezer próbálkozás matematikailag elég lehet — ezért van szükség arra, hogy a hibás próba **összeomoljon** (ne lehessen folytatni), és ezért kritikus, hogy egyetlen kiszivárgott cím ne adja meg a teljes elrendezést. A canary magas entrópiája viszont csak a **folytonos, felfelé terjedő** írás ellen hatásos, és csak akkor, ha a canary értéke nem szivárog ki.

14.2.5 Kidolgozott példa: a canary felismerése a generált kódban és a mechanizmusok állapotának ellenőrzése

Munkakönyvtár: `~/arm-labor/a64/a054/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a054
cd ~/arm-labor/a64/a054
```

Cél. Egy adott binárison **auditálható módon** állapítsuk meg, melyik mechanizmus van jelen, és milyen bizonyíték támasztja alá. Target: `aarch64-linux-gnu-gcc` + `qemu-aarch64` (user-mode), illetve opcionálisan natív AArch64 host.

1. lépés — forrás és build. A vizsgált függvény egy külön fájlban van; a hívó `main` és a `sink` definíciója a második fájlba kerül, így a 3–4. lépés futtatható állománya csak a két, itt látható forrásból épül.

```
/* lab-a054-vuln.c */
#include <string.h>

extern void sink(const char *);

void lab_vuln(const char *in) {
    char buf[32];           /* helyi puffer */
    strcpy(buf, in);        /* határellenőrzés nélküli másolás */
    sink(buf);
}
```

```c
/* lab-a054-main.c - hívó és sink a lab_vuln vizsgálatához */
#include <stdio.h>

void lab_vuln(const char *in);

void sink(const char *s)            /* a lab-a054-vuln.c extern sink-je */
{
    printf("sink: \"%s\"\n", s);
}

int main(void)
{
    int local = 0;
    lab_vuln("labor a054");          /* 10 bájt + '\0': belefér a 32 bájtba */
    /* betöltési címek az ismételt futtatásos ASLR-vizsgálathoz */
    printf("main:  %p\n", (void *)main);
    printf("verem: %p\n", (void *)&local);
    return 0;
}
```

```
# target: cross-build x86_64 hoston, futtatás qemu-aarch64 user-mode-ban
aarch64-linux-gnu-gcc -O1 -fPIE -fstack-protector-strong -c -o lab-a054-vuln.o lab-a054-vuln.c
aarch64-linux-gnu-gcc -O1 -fPIE -c -o lab-a054-main.o lab-a054-main.c
aarch64-linux-gnu-gcc -pie -Wl,-z,relro,-z,now \
    -o lab-a054-vuln lab-a054-vuln.o lab-a054-main.o
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab-a054-vuln
```

A program a `sink: "labor a054"` sort, majd a `main` és egy veremváltozó címét írja ki. Az Ubuntu GCC `-O1` fölött alapértelmezésben `_FORTIFY_SOURCE=3`-at definiál, ezért a `strcpy` helyén `bl __strcpy_chk` áll, `mov x2, #32` mérettel; a canary önálló vizsgálatához `-U_FORTIFY_SOURCE` (vagy `-D_FORTIFY_SOURCE=0`) kapcsolóval fordíts, és hasonlítsd össze a két kimenetet. A bemenet szándékosan rövid: ez a labor a védelmek *felismeréséről* szól, nem a túlírás kiváltásáról. A 2. lépés a `lab-a054-vuln.o` objektumfájlt, a 3–4. lépés a linkelt `lab-a054-vuln` állományt vizsgálja.

Megjegyzés a kapcsolókhoz. Az Ubuntu GCC alapértelmezésben `-fstack-protector-strong`-ot, `_FORTIFY_SOURCE=3`-at és PIE-t használ; a „védelem nélküli” összehasonlításhoz ezért `-fno-stack-protector -U_FORTIFY_SOURCE` (és `-no-pie`) kell. Kapcsolók nélkül, `-O1`-gyel a `lab_vuln`-ban négy `__stack_chk_*/__strcpy_chk` hivatkozás van, tehát az alapértelmezett és a „védett” változat kódja azonos lehet.

2. lépés — a canary-minta felismerése a disassemblyben.

```
aarch64-linux-gnu-objdump -d lab-a054-vuln.o | sed -n '/<lab_vuln>:/,/^$/p'
```

AArch64 glibc mellett a globális `__stack_chk_guard` betöltését keressük; a `TPIDR_EL0 + 40` nem hordozható canary-cím. Az alábbi felismerési vázlat kihagyja a függvénytörzset; a fordító konkrét regiszter- és keretkiosztása eltérhet:

```
    stp  x29, x30, [sp, #-48]!      // keretrekord: sp+48
    mov  x29, sp
    // buf: sp+8..sp+39; a canary a buf fölött, sp+40-en
    adrp x9, :got:__stack_chk_guard
    ldr  x9, [x9, :got_lo12:__stack_chk_guard]
    ldr  x10, [x9]
    str  x10, [sp, #40]             // guard mentése
    // ... a függvény törzse ...
    mov  x2, #32                    // _FORTIFY_SOURCE=3: a strcpy helyett
    bl   __strcpy_chk               // __strcpy_chk, ismert mérettel
    // ... sink hívása ...
    ldr  x10, [sp, #40]
```

```
    subs x10, x10, x11
    b.ne .Lfail
    ldp  x29, x30, [sp], #48
    ret
.Lfail:
    bl   __stack_chk_fail          // detektálás: a folyamat leáll
```

A **felismerési minta** a guard betöltése és verembe mentése, majd a visszatérés előtti összehasonlítás és a `__stack_chk_fail` hibája. A fenti GOT-címzés a globális `__stack_chk_guard` szimbólumhoz vezet. A pontos utasítássor libc-, fordító- és kapcsolófüggő; egyetlen `MRS` vagy adott TLS-eltolás hiányából nem következik a védelem hiánya.

3. lépés — ASLR, NX, RELRO, PIE állapotának ellenőrzése.

Az alábbi parancsok egyetlen konkrét állományra, az 1. lépésben linkelt `lab-a054-vuln` ELF-re vonatkoznak, nem az objektumfájlra. Egyik sor sem bizonyít önmagában semmit; az állítás a fejléc, a szegmensek, a dinamikus bejegyzések és a futásidejű címek **együttes** értékeléséből születik. A parancsokat két csoportra bontjuk: az első a **fájl** tulajdonságait olvassa (bárhol futtatható, a host `readelf`-je is jó), a második a **futó vendégprogramot** vizsgálja.

```
# (A) a fájl statikus tulajdonságai - host-eszközzel olvasható
readelf -hW lab-a054-vuln | grep Type                          # EXEC vagy DYN
readelf -lW lab-a054-vuln | grep -E 'INTERP|LOAD|GNU_RELRO|GNU_STACK'
readelf -dW lab-a054-vuln | grep -E 'FLAGS'                    # BIND_NOW, FLAGS_1

# (B) a vendég futásidejű viselkedése - ugyanabban a környezetben kétszer futtatva
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab-a054-vuln
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab-a054-vuln

# (C) a HOST beállításai - csak a host folyamataira vonatkoznak
cat /proc/sys/kernel/randomize_va_space                        # host: 2 = teljes ASLR
cat /proc/sys/vm/mmap_rnd_bits                                 # host: mmap-entrópia
```

A fő úton, GNU eszközlánccal (GNU ld 2.42) linkelt állományon az (A) csoport kimenete (a lektor mérése):

```
  Type:  DYN (Position-Independent Executable file)
  INTERP     ...  R    [Requesting program interpreter: /lib/ld-linux-aarch64.so.1]
  LOAD       ...  R E
  LOAD       ...  RW
  GNU_STACK  ...  RW
  GNU_RELRO  ...  R
 (FLAGS)    BIND_NOW
 (FLAGS_1)  Flags: NOW PIE
```

A (B) csoport mért eredménye QEMU 8.2.2 alatt (a lektor mérése): a `main:` cím futásonként változik (0x7fd353540908, 0x7f96059d0908, 0x7fb33b420908), a `verem:` cím viszont mindháromszor 0x4000007fb5f4. A QEMU user-mode tehát a PIE-bázist véletleníti, a vendégvermet nem — épp ez a host- és vendégmegfigyelés megkülönböztetése.

Az értékelés tulajdonságonként, a szükséges bizonyítékok együttesével:

Állítás	Szükséges bizonyítékok együtt	Mit **nem** jelent önmagában
PIE futtatható állomány	`Type: DYN` **és** `INTERP` szegmens (vagy `FLAGS_1`-ben `PIE`)	a `DYN` önmagában megosztott könyvtár (`.so`) is lehet

Állítás	Szükséges bizonyítékok együtt	Mit **nem** jelent önmagában
teljes RELRO	`GNU_RELRO` szegmens **és** azonnali kötés (`FLAGS: BIND_NOW` vagy `FLAGS_1: NOW`)	`GNU_RELRO` `BIND_NOW` nélkül csak részleges RELRO: a lusta kötésű GOT-bejegyzések írhatók maradnak
a verem nem végrehajtható	`GNU_STACK` `RW` (nem `RWE`)	csak a **verem** végrehajthatóságára vonatkozó kérés; a betöltő és a kernel teljesíti, nem a fájl
egyetlen betöltött szegmens sem írható és végrehajtható egyszerre	minden `LOAD` sor jelzője: `R`, `R E` vagy `RW`, egyik sem `RWE`	a futás közbeni `mmap`/`mprotect` hívásokat (például JIT) a fájl nem mutatja
a programkép címe futásonként változik	két futás `main:` sora eltér (B csoport)	a PIE csak *lehetővé teszi* a véletlenítést; hogy megtörténik-e, a betöltőn múlik

A (B) és a (C) csoportot nem szabad összemosni. Natív AArch64 gépen a kettő ugyanaz a rendszer: ott a `randomize_va_space` is a vizsgált program környezetét írja le. Cross-hoston viszont a `/proc` a **host** kernel beállításait mutatja, és csak a host folyamataira (köztük magára a `qemu-aarch64` folyamatra) érvényes. A vendégprogram betöltési címét a QEMU user-mode betöltője választja ki, ezért a host-beállításból nem következik, hogy a vendég programképe és verme futásonként máshová kerül. A vendégre vonatkozó állítás forrása csak a (B) csoport két futásának összevetése lehet, és ez is csak az adott QEMU-verzióra és kapcsolókra érvényes. A jegyzőkönyvben ezért minden sor mellé oda kell írni, hogy host- vagy vendégmegfigyelés. Ha statikus, fix címre linkelt (`Type: EXEC`) binárist vizsgálunk, az ASLR a programképre biztosan nem hat, legfeljebb a veremre és az `mmap`-régiókra.

4. lépés — PAC jelenlétének, hardvertámogatásának és tényleges alkalmazásának ellenőrzése.

Ez a lépés három **külön** kérdést tesz fel, és mindhárom válaszát külön kell rögzíteni:

Kérdés	Bizonyíték	Mit nem dönt el
1. A **kód** kéri-e a PAC-ot?	`objdump`: `paciasp`/`autiasp` (régebbi `objdump` alatt `hint #25`/`hint #29`), illetve `retaa`/`retab`; `readelf -n`: `GNU_PROPERTY_AARCH64_FEATURE_1_PAC`	hogy a futtató mag ismeri-e az utasítást
2. A **hardver** támogatja-e?	`getauxval(AT_HWCAP)`: `HWCAP_PACA`	hogy a kernel bekapcsolta-e a kulcsot ennél a folyamatnál
3. **Érvényesül-e** a futó folyamatban?	`prctl(PR_PAC_GET_ENABLED_KEYS)`: az `APIA` kulcs engedélyezett	hogy a program *minden* függvénye aláírt-e (ezt az 1. pont dönti el, objektumonként)

A `HWCAP_PACA` a **címhitelesítést** jelzi (az `APIA`/`APIB`/`APDA`/`APDB` kulcsok és a `PACIA*`/`AUT*` utasításcsalád); a `pac-ret` ehhez kötődik. A `HWCAP_PACG` a **generikus hitelesítést** jelzi (`PACGA`

utasítás, `APGA` kulcs), amely tetszőleges adatból számít aláírást. A visszatérési cím védelméhez nem kell, és nem is helyettesíti a `HWCAP_PACA`-t; a kettő külön képesség, külön bitben.

```
aarch64-linux-gnu-gcc -O1 -fPIE -fstack-protector-strong -mbranch-protection=standard \
    -c -o lab-a054-vuln-pac.o lab-a054-vuln.c
aarch64-linux-gnu-gcc -O1 -fPIE -mbranch-protection=standard \
    -c -o lab-a054-main-pac.o lab-a054-main.c
aarch64-linux-gnu-gcc -pie -Wl,-z,relro,-z,now \
    -o lab-a054-pac lab-a054-vuln-pac.o lab-a054-main-pac.o
aarch64-linux-gnu-objdump -d lab-a054-pac \
    | grep -E 'paci|auti|retaa|retab|hint[[:space:]]+#(25|29)'
readelf -n lab-a054-vuln-pac.o | grep -i 'AArch64 feature'
```

A GCC 13.3 `-mbranch-protection=standard` mellett a `lab_vuln` prológusának első utasítása `paciasp`, az epilógusban `autiasp` + `ret` áll; `bti c` nincs, mert a `paciasp` landing padként szolgál (ez megerősíti a 14.2 BTI-megjegyzését). A két saját objektum `.note.gnu.property` bejegyzése „AArch64 feature: BTI, PAC”, a `gcc -static`-kal linkelt ELF-ben viszont nincs ilyen jegyzet: az Ubuntu 24.04 cross-glibc startupfájljai (crt1.o, crti.o) nem hordozzák, és a linker ÉS-sel vonja össze. Ezért a jegyzetet az objektumokon kell ellenőrizni (`readelf -n lab-a054-vuln-pac.o`), vagy `ld -r`-rel összefűzve; a végső ELF csak akkor mutatja, ha minden bemenet hordozza.

A 2. és 3. kérdést a vizsgált környezetben futó próba válaszolja meg:

```
/* lab-a054-pac-probe.c - PAuth: hardvertámogatás és engedélyezett kulcsok */
#include <stdio.h>
#include <sys/auxv.h>
#include <sys/prctl.h>

#ifndef HWCAP_PACA                  /* Linux arm64 uapi: asm/hwcap.h */
#define HWCAP_PACA (1UL << 30)
#endif
#ifndef HWCAP_PACG
#define HWCAP_PACG (1UL << 31)
#endif
#ifndef PR_PAC_GET_ENABLED_KEYS     /* Linux 5.13+ */
#define PR_PAC_GET_ENABLED_KEYS 61
#endif
#ifndef PR_PAC_APIAKEY
#define PR_PAC_APIAKEY (1UL << 0)
#endif

int main(void)
{
    unsigned long hw = getauxval(AT_HWCAP);
    printf("AT_HWCAP = 0x%lx\n", hw);
    printf("HWCAP_PACA (címhitelesítés, pac-ret ehhez kell): %s\n",
           (hw & HWCAP_PACA) ? "igen" : "nem");
    printf("HWCAP_PACG (generikus hitelesítés, PACGA):      %s\n",
           (hw & HWCAP_PACG) ? "igen" : "nem");

    long keys = prctl(PR_PAC_GET_ENABLED_KEYS, 0, 0, 0, 0);
    if (keys < 0)
        printf("engedélyezett kulcsok: nem lekérdezhető (régi kernel, nincs PAuth, vagy az
↪   emulátor nem támogatja ezt a prctl-t)\n");
    else
        printf("engedélyezett kulcsok: 0x%lx, APIA %s\n", keys,
               (keys & PR_PAC_APIAKEY) ? "be" : "ki");
    return 0;
}

aarch64-linux-gnu-gcc -O1 -o lab-a054-pac-probe lab-a054-pac-probe.c
# vendég, PAuth nélküli modell
```

```
qemu-aarch64 -cpu cortex-a53 -L /usr/aarch64-linux-gnu ./lab-a054-pac-probe
# vendég, PAuth-ot is tartalmazó modell
qemu-aarch64 -cpu max -L /usr/aarch64-linux-gnu ./lab-a054-pac-probe
```

QEMU alatt a próba az **emulált** CPU-modell és a QEMU által szimulált kernelfelület képességeit mutatja, a hostét nem; a QEMU user-mode a `PR_PAC_GET_ENABLED_KEYS` prctl-t nem emulálja, ezért a 3. kérdés prctl-lel nem dönthető el, és a `-cpu cortex-a53`/`-cpu max` összevetése mutatja a PAuth-modell különbségét; az eredményt ezért a használt `-cpu` értékkel és QEMU-verzióval együtt kell rögzíteni. A PAC-védelemről akkor állíthatjuk, hogy **érvényesül**, ha mindhárom kérdésre igen a válasz: a kód aláír, a `HWCAP_PACA` be van állítva, és az `APIA` kulcs engedélyezett. Ha a kód csak a HINT-alakot használja, a hardver vagy a kulcs hiányában a program hiba nélkül fut, csak védelem nélkül. Ez a leggyakoribb félreértés ezen a területen.

Ellenőrzési feltétel. A példa sikeres, ha (a) a `lab_vuln` disassemblyjében azonosítható a canary-minta, (b) a `lab-a054-vuln` állományról a PIE, a teljes RELRO és a nem végrehajtható verem állítása a fenti táblázat **összes** szükséges bizonyítékával alá van támasztva, (c) a két futás betöltési címe rögzítve van, és minden ASLR-megfigyelés host- vagy vendégcímkét kapott, (d) a PAC három kérdésére (kódkérés, `HWCAP_PACA`, engedélyezett kulcs) **külön-külön** van válasz, a futtatási környezet megnevezésével.

14.2.6 Tipikus hibák és ellenpróbák

Hiba 1 — „A canary megvédi a puffert." *Tünet:* a canary jelen van, mégis egy másik lokális változó sérül. *Hipotézis:* a túlírás nem érte el a canary-t, vagy nem folytonosan írt. *Adatgyűjtés:* `objdump -d` a veremkeret elrendezéséhez, `gdb`-vel a `sp`-hez viszonyított címek vizsgálata. *Egy változtatás:* a puffer méretének és a szomszédos lokálisok sorrendjének megváltoztatása a forrásban, újrafordítás. *Ellenőrzés:* a canary-minta pozíciója elmozdul-e, és a hibás írás most már detektálódik-e. **Tanulság:** a canary a *keret végére* őrködik, nem a keret minden mezőjére.

Hiba 2 — „Van PAC, tehát védett." *Tünet:* a binárisban `paciasp` található, a folyamat mégsem hibázik hamis visszatérési címen. *Hipotézis:* a hardver nem támogatja a PAuth-ot, vagy a kernel nem engedélyezte a kulcsot, így a HINT-kódolású `paciasp`/`autiasp` NOP-ként fut; vagy QEMU-modell, amely nem emulálja. (Ha a kódban `retaa` is van, PAuth nélkül nem csendes futást, hanem `SIGILL`-t látnánk.) *Adatgyűjtés:* a `lab-a054-pac-probe` kimenete a célrendszeren (`HWCAP_PACA`, engedélyezett kulcsok), QEMU `-cpu` beállítás és a modell dokumentációja. *Egy változtatás:* a QEMU CPU-modell cseréje PAuth-támogató modellre, vagy natív AArch64 hardveren futtatás. *Ellenőrzés:* a `HWCAP_PACA` bit és az `APIA` kulcs megjelenik-e, és a hamis cím most hibát okoz-e. **Megjegyzés:** a QEMU user-mode és a `-cpu` modellek PAuth-támogatása **verziófüggő, ellenőrizendő**.

Hiba 3 — „Az ASLR miatt a cím kitalálhatatlan." *Tünet:* a program fix címre hivatkozik, vagy a címtér töredéke véletlenített. *Hipotézis:* nem PIE bináris, vagy `randomize_va_space=0` a konténerben. *Adatgyűjtés:* `readelf -hW` (`DYN` vs. `EXEC`), `/proc/sys/kernel/randomize_va_space`, `/proc/self/maps` ismételt kiolvasása. *Egy változtatás:* `-fPIE -pie` hozzáadása a linkeléshez. *Ellenőrzés:* a betöltési bázis két egymást követő futás között eltér-e. **Tanulság:** az ASLR egy *rendszer- és build-szintű* tulajdonság, nem a forráskódé.

14.2.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Az alfejezet négy mechanizmusához rendeld hozzá, hogy a verem-túlcsordulás melyik támadási lépését nehezítik meg: (a) a puffer

felülírása, (b) a visszatérési cím eltalálása, (c) a beírt bájtok végrehajtása, (d) a visszatérési cím megváltoztatása. Minden mechanizmusnál írj egy mondatot arról, **mit nem** garantál.

2. (implementációs, középszint — cél: rutinépítés) Készíts egy `lab-a054-probe.c` programot, amely futásidőben kiírja: a `getauxval(AT_HWCAP)` értékét hexadecimálisan, és a `readelf`-ből kiolvasott `GNU_STACK` és `Type` mezőt. Fordítsd háromféleképp: (i) alapértelmezett, (ii) `-fstack-protector-strong`, (iii) `-mbranch-protection=standard`. Mindhárom binárison dokumentáld a canary-minta, a PIE és az NX állapotát, valamint a PAC-utasítások jelenlétét.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Állítsd elő a hibás állapotot: linkeld a binárist `-z execstack`-kel, és ellenőrizd, hogy a `GNU_STACK` sor `RWE`. Fogalmazz hipotézist arról, melyik build-lépés okozhat máskor is végrehajtható vermet (például hiányzó `.note.GNU-stack` egy assembly-objektumban), és egyetlen változtatással igazítsd ki. Igazold `readelf -lW`-vel, és írd le, miért **nem** elég ehhez a forráskód vizsgálata.

14.2.8 Megoldások

1. megoldás — Segítség. Nézd meg az alfejezet “Pontos garancia” / “Amit nem garantál” bontását minden mechanizmusnál — melyik lépést éri el közvetlenül, és melyiket csak közvetve?

1. megoldás — Teljes megoldás. (a) A puffer felülírását egyik mechanizmus sem akadályozza meg közvetlenül; a canary csak **utólag detektálja**, ha a túlírás eléri. (b) A visszatérési cím eltalálását az **ASLR** nehezíti (a cím nem ismert előre), és a **PAC** védi (a módosított cím nem hitelesíthető). (c) A beírt bájtok végrehajtását az **NX/DEP** akadályozza meg; ez az egyetlen valódi *megelőzés* a négy közül arra a lépésre. (d) A visszatérési cím megváltoztatását a **canary** detektálja (ha a túlírás folytonos és eléri), a **PAC** pedig megelőzi az érvényesítést. **Amit nem garantálnak:** a canary nem észlel nem folytonos írást és nem véd kiszivárgott érték ellen; az ASLR nem véd kiszivárgott cím ellen és entrópiája konfigurációfüggő; az NX nem véd meglévő kód újrafelhasználása (ROP) ellen; a PAC nem véd a nem aláírt mutatókra, a hardver-támogatás hiányában hatástalan, és csak az aláírt mutatók integritását garantálja.

2. megoldás — Segítség. Mindhárom fordítási változatnál külön kell rögzítened, hogy a bináris mit *kér* (a kód jelenléte) és a futó rendszer mit *tud ténylegesen biztosítani* (HWCAP) — ez a két dolog a (iii) esetben válik külön élesen.

2. megoldás — Teljes megoldás. A mérési jegyzőkönyv három blokkból áll. (i) Alapértelmezett: jellemzően nincs canary-minta, `Type: DYN` (a disztribúció PIE-t linkel alapból), `GNU_STACK: RW`. (ii) `-fstack-protector-strong`: a helyi puffert használó függvényekben megjelenik a guard betöltése (glibc alatt a `__stack_chk_guard` GOT-on keresztül), a verembe mentése, majd a `LDR/CMP/B.NE` és a `__stack_chk_fail` hívás; a konkrét címzés libc-függő. (iii) `-mbranch-protection=standard`: megjelennek a `paciasp/autiasp` (régebbi `objdump`-nál `hint #25/hint #29`) és `bti` utasítások. **Kulcs:** a (iii) esetben a védelem csak akkor tényleges, ha a futásidejű `HWCAP_PACA` jelzi a címhitelesítés támogatását **és** az `APIA` kulcs engedélyezett; a `HWCAP_PACG` erről nem dönt. Ezt minden futtatási környezetben külön kell rögzíteni, és a QEMU-modell támogatása ellenőrizendő. Az `objdump` és a `readelf` adatai a **bináris képességét** írják le, nem a futó rendszer védelmi állapotát.

3. megoldás — Segítség. A `GNU_STACK` jelző a linkelés eredménye, és az ÖSSZES bemeneti objektum jelzéseinek kombinációjából áll elő — melyik objektumtípusnál (kézzel írt vagy fordítóból származó) szokott hiányozni az erre vonatkozó direktíva?

3. megoldás — Teljes megoldás. A `GNU_STACK: RWE` gyakori oka, hogy valamelyik linkelt objektumfájl nem tartalmaz `.note.GNU-stack` szakaszt (tipikusan kézzel írt assembly), és a linker ezt – célarchitektúrától és verziótól függően – végrehajtható veremigényként értelmezi;

a másik ok egy explicit `-z execstack`. Fordítva viszont nem következtethetünk: a direktíva hiánya nem jelent automatikusan `RWE`-t. Binutils 2.42 mellett jegyzet nélküli objektumból `ld`-vel linkelve **nincs** `GNU_STACK` fejléc (sem `RW`, sem `RWE`), `gcc`-vel linkelve `RW` lesz, és `RWE` csak `-z execstack`-kel áll elő; ezért mindig a kész ELF-et kell ellenőrizni. **Hipotézis → adatgyűjtés:** `readelf -SW` minden objektumfájlon, a `.note.GNU-stack` szakasz keresése; a hiányzó elem azonosítja a felelős objektumot. **Egy változtatás:** a kérdéses assembly-forrásba `.section .note.GNU-stack,"",%progbits` direktíva felvétele (ez üres, nem allokált szakasz, csak jelzi a szándékot). **Ellenőrzés:** `readelf -lW` a végleges binárison a `-z execstack` esetben `RWE`-t, a javított linkelésnél `RW`-t mutat. **Miért nem elég a forráskód:** a `GNU_STACK` jelző a linkelési lépésben, az **összes** bemeneti objektum jelzéseinek kombinálásából keletkezik, ezért egyetlen forrásfájl olvasásából nem következtethetünk rá.

14.2.9 Rövid összefoglaló

Négy mechanizmust rendszereztünk. A **stack canary** a puffer és a mentett visszatérési cím közé szúrt véletlen érték: csak **detektál**, a visszatérés előtt, és csak folytonos, a canary-t elérő túlírás esetén. Az **ASLR** a memóriaterületek kezdőcímét véletleníti: nehezíti a cím eltalálását, de entrópiája konfigurációfüggő, és egyetlen kiszivárgott cím deaktiválhatja. Az **NX/DEP** az adatlapokat nem-végrehajthatóvá teszi: ez az egyetlen valódi megelőzés a kódinjektálás ellen, de a ROP-ra nem hat. A **PAC** az AArch64-en a visszatérési címet hardveresen, kriptográfiailag hitelesíti, de csak azon az úton, amely a hitelesítő utasítást (`AUTIASP` vagy `RETAA`) ténylegesen végrehajtja; a sima `RET` nem hitelesít, és a `pac-ret` nem általános CFI; hatékonysága a hardvertámogatástól (`HWCAP_PACA`), a kulcs engedélyezésétől és a PAC-mező méretétől függ; a HINT-kódolású `paciasp`/`autiasp` támogatás nélkül csendben NOP, a `retaa` viszont `SIGILL`. Egyik sem abszolút: a **mélységi védelem** lényege, hogy a támadónak minden réteget külön kell legyőznie.

- **Védelmi mechanizmus:** megelőző, detektáló vagy nehezítő szerkezeti elem; pontos hatóköre mindig megadandó.
- **Entrópia (N bit):** a vak próba siker-valószínűsége 2^{-N}, feltéve független, egyenletes eloszlású véletlent.
- **Mélységi védelem:** egymásra rétegzett, egyenként hiányos garanciájú mechanizmusok együttese.

14.2.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek. A gyorsan változó tények (QEMU CPU-modellek PAuth-támogatása, glibc TLS-eltolás, kernel ASLR-entrópia, HWCAP-bitek) továbbra sem ellenőrzöttek — ezeket a dokumentumellenőrzés nem tudja pótolni, mert rendszer- és verziófüggők.

- Arm A-profile architektúra-dokumentáció — Pointer Authentication (Armv8.3-A), `PACIASP`/`AUTIASP`/`RETAA`, `UXN`/`PXN` lapbitek. Ellenőrizve: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- AAPCS64 – Arm hivatalos ABI-dokumentum — veremkeret, igazítás, keretrekord. **Tartalmilag ellenőrizve**: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (a SME-vonatkozású részek Beta státuszúak).
- GCC kézikönyv — `-fstack-protector-strong`, `-mbranch-protection`, PIE-kapcsolók pontos szemantikája. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2.

- GNU Binutils kézikönyv — `objdump`, `readelf`, `.note.GNU-stack` szakasz jelentése. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU GDB kézikönyv — veremkeret és regiszterállapot vizsgálata a detektálás megfigyeléséhez. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a „current” fejlesztői pillanatkép (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak (külön-külön kezelendők):

1. A `readelf`/`objdump` kimenetek és a glibc TLS-eltolás (`tpidr_el0 + 40`) **verziófüggő**; minden konkrét rendszeren ellenőrizendő.
2. A `qemu-aarch64` user-mode és a `qemu-system-arm` CPU-modellek **PAuth- és BTI-támogatása nem ellenőrzött** itt; a PAC-példa QEMU-ban csak a támogató modell kiválasztásával érvényes.
3. Az ASLR tényleges entrópiája (`mmap_rnd_bits`, `randomize_va_space`) **kernelkonfiguráció-függő**; a fejezetben szereplő 9/28 bit **illusztratív, levezetett**, nem mért érték.
4. A PAC-mező mérete a virtuális címtér méretétől függ; Linuxon 48 bites VA mellett az utasításmutatók PAC-ja ~15, az adatmutatóké ~7 bit (TBI miatt), a pontos érték a TCR-konfigurációtól függ.
5. A stack canary alsó bájtjának `0x00` kötöttsége és a `__stack_chk_fail` viselkedése **glibc-implementációfüggő**.
6. A fő úton, GNU eszközlánccal és QEMU 8.2.2-vel mérve (a lektor mérése): a `readelf -lW` a linkelt `lab-a054-vuln` állományon `Type: DYN`, INTERP, két LOAD (`R E`, `RW`), `GNU_STACK RW`, `GNU_RELRO R`, `(FLAGS) BIND_NOW` és `(FLAGS_1) NOW PIE` sorokat ad; a `qemu-aarch64` alatt a `main:` cím futásonként változik, a `verem:` cím nem. A `lab-a054-pac-probe` QEMU 8.2.2 alatt lefut; a `prctl(PR_PAC_GET_ENABLED_KEYS)` a QEMU user-mode alatt nem emulált, ezért a 3. kérdés QEMU-ban prctl-lel nem dönthető el. Natív AArch64 hardveren nem történt mérés; az ottani `HWCAP`-biteket és betöltési címeket ott kell rögzíteni.

14.3 ROP-lánc felismerése és miértje

14.3.1 Motiváció

Egy biztonsági audit első kérdése nem az, hogy „hogyan törjük fel a rendszert”, hanem az, hogy **a támadónak milyen képessége marad, miután a védelmi rétegeink átengedték**. A Védelmi mechanizmusok szakaszban láttuk, hogy az NX/DEP az egyetlen valódi megelőzés az újonnan injektált gépkód futtatása ellen, és hogy ez önmagában nem elegendő: a támadó a program saját, már jelen lévő, végrehajtható kódját használhatja fel. Az a technika, amely ezt rendszerezetté teszi, a **return-oriented programming (ROP)**; a jelen alfejezet a felismerés és a kódreview eszközkészletét adja: mi a gadget, mi a lánc, és pontosan melyik védelmi réteg melyik lépést nehezíti meg. Ez az audit-készség a Verem-túlcsordulás mechanizmusa assembly szinten mechanizmusára épül, és a Biztonságos assembly- és C-interop gyakorlat szakaszban folytatódik.

A ROP-ismeret nem exploit-építési tudás, hanem **defenzív elemzési készség**: annak eldöntése, hogy egy adott kódbázisban van-e olyan primitív, amely a láncot elindíthatja, és hogy a védelmi rétegek mely része zárja le melyik lépcsőt.

14.3.2 Definíció és működés

ROP-gadget: olyan rövid, általában 2–6 utasításból álló gépi kódrészlet egy létező, végrehajtható memóriaterületen (a program saját szövegszegmensében vagy betöltött megosztott könyvtárban), amely egy meghatározott minimális műveletet végez, és **`RET`-tel zárul**. A „hasznos" rész jellemzően: regiszterbe töltés, regiszter-regiszter másolás, egyszerű aritmetika, vagy a veremmutató (SP) átállítása.

ROP-lánc: gadget-címek és a hozzájuk tartozó adatkonstansok sorozata a veremben, úgy elrendezve, hogy minden gadget a záró `RET` **előtt** a veremből az X30-ba tölti a következő gadget címét (és közben lépteti az SP-t), a `RET` pedig erre a regiszterben lévő címre ugrik. A verem lesz tehát a „program", a gadget-címek pedig a „programutasítások". A gépi kód, amit a processzor végrehajt, **változatlanul a program saját, hiteles részeiből származik** – ez az egész technika lényege.

JOP és COP: ha a gadget nem `RET`-tel, hanem regiszteres ugrással (`BR Xn`) zárul, a technika neve **jump-oriented programming (JOP)**; a `BLR Xn`-nel záródó változaté **call-oriented programming (COP)**. Ezeknél a következő cél nem a visszatérési útvonalon (X30) érkezik, hanem egy tetszőleges regiszterben, a láncot ezért jellemzően egy külön *dispatcher gadget* lépteti. A ROP, a JOP és a COP együtt a **kód-újrafelhasználó (code-reuse) támadások** családja; ez az alfejezet a ROP-ra összpontosít, a JOP/COP-ot csak ott említi, ahol a védelmek hatóköre eltér.

Miért működik mindez? Az A64 operandus nélküli `RET` (azaz `RET X30`) **regiszteres visszatérés**: a PC új értéke az X30 (LR) tartalma. A `RET` önmagában nem olvas memóriát, és az SP-t sem változtatja. Ez lényeges eltérés az x86 `ret`-jétől, amely a verem tetejéről veszi a címet, és lépteti a veremmutatót. A64-en a láncolást ezért nem a `RET`, hanem egy **előtte álló betöltés** valósítja meg: az az utasítás, amely a veremből az X30-ba tölti a következő gadget címét. A tipikus ilyen *chain driver* a szokásos epilógus:

```
// Chain driver - jellemző epilógus, amely a lánc egyik "lépcsője"
ldp x29, x30, [sp], #16   // (1) veremolvasás: x29 <- [SP], x30 <- [SP+8]
                          // (2) léptetés: SP <- SP + 16 (post-index visszaírás)
ret                       // (3) ugrás: PC <- x30; az SP és a memória változatlan
```

Lépés	Utasítás	Memória	SP	Regiszter / PC
(1) veremolvasás	`ldp`	olvassa `[SP]`-t és `[SP+8]`-at	–	X29 ← `[SP]`, X30 ← `[SP+8]` (a következő gadget címe)
(2) léptetés	`ldp` post-index `#16`	–	SP ← SP + 16	–
(3) ugrás	`ret`	nem olvas	**változatlan**	PC ← X30

Az SP-t tehát egyedül az `ldp` post-index alakja változtatja meg; a `ret` előtti és utáni SP azonos. Az új célcím a veremből származik, de az `ldp` hozza át az X30-ba, és a `ret` már csak a regiszterből olvassa. Ha a lánc egyes lépcsői így néznek ki, a verem 16 bájtos `[kísérő_adat][next_LR]` párok ismételt sorozata lesz: az alacsonyabb címen lévő szó az X29-be, a magasabb címen lévő az X30-ba kerül.

Ebből egy A64-specifikus következmény is adódik: az a gadget, amely **nem tölti újra** az X30-at (például `add x0, x0, #1; ret`), nem lépteti a láncot. A `ret` ugyanarra az X30-ra ugrik, ahonnan a gadget indult, vagyis önmagára tér vissza. Ugyanezt a hibát mutatja be a Hívás és

visszatérés alfejezet mentés nélküli `RET`-je. A64-en tehát a lánc csak akkor halad, ha a gadget hasznos részét X30-at betöltő epilógus követi. A támadó **nem futtat új kódot**, csak meglévő gépi sorozatokat használ fel – ezért lett a ROP az NX/DEP utáni domináns kontrolláram-támadási osztály.

Defenzív szempontból a lánc mindig **négy fázisra bontható**. Az auditnak ezeket a fázisokat egyenként kell lezárnia:

1. **Kezdeti hijack.** Kell lennie egy első pontnak, ahonnan a vezérlés eltéríthető. Az 14.1 szakaszban láttuk: a mentett visszatérési cím felülírása a leggyakoribb primitív, de függvénymutató vagy hívóvisszatérési slot is ugyanide vezethet.
2. **Stack pivot (opcionális).** Ha a lánc hosszabb, mint a hijack-puffer, egy *pivot gadget* átállítja SP-t a támadó által ellenőrzött területre (pl. `mov sp, x0; ret` vagy hasonló minta). Ettől kezdve a lánc ott fekszik, ahol a támadó szeretné.
3. **Gadget-feltérképezés.** A binárisban minden `RET`-et megelőző néhány utasítás jelölt gadget; az ipari eszközök jellemzően több ezer kandidátust találnak egy tipikus dinamikusan linkelt Linux-binárisban.
4. **Lánc-összeállítás.** A gadget-címek és adatok végrehajtási sorrendbe rendezése egy összefüggő veremképben.

A fázisok **Turing-teljes** számítást engednek meg: elég gadgetből tetszőleges műveletsor kirakható. Audit szempontból ez azt jelenti, hogy **egyetlen kontrolláram-eltérítési primitív már elegendő** egy súlyos következményhez; nem kell közvetlen injektálás.

Az alábbi táblázat egy tipikus dinamikusan linkelt AArch64 bináris gadget-kategóriáinak **illusztratív, hipotetikus** megoszlását mutatja. Az értékek nem mérések, csak azt érzékeltetik, hogy melyik kategóriából szokott sok kandidátus lenni.

#	Kategória	Tipikus alak	Szerep az elemzésben
1	LDP/STP epilógus	`ldp x29, x30, [sp], #16; ret`	chain driver, a lánc „motorja”
2	Regiszter-töltés	`ldr x0, [x0]; ret`	argumentum-előkészítés
3	Aritmetika/logika	`add x0, x0, #1; ret`	értékmanipuláció
4	SP-átállítás	`mov sp, x0; ret`	pivot a támadó bufferébe
5	Branch-to-register (JOP/COP, nem ROP)	`ldp x16, x17, [sp], #16; br x16`	a `RET`-hitelesítést (PAC-RET) nem érintő útvonal, ha elérhető

14.3.3 A védelmi rétegek hatása a ROP-lépésekre

Fontos tisztázni, melyik mechanizmus melyik fázist zárja. A teljes rendszerezést lásd 14.2-ben.

Mechanizmus	Melyik ROP-lépést érinti	Hatásmechanizmus
NX/DEP	a *kód-injektálás* útját lezárja	nem-végrehajtható adatoldal; pontosan emiatt kényszerül a támadó ROP-ra

Mechanizmus	Melyik ROP-lépést érinti	Hatásmechanizmus
Stack canary	a *kezdeti hijack*-et detektálja	mentett LR előtt elhelyezett őrérték; csak folytonos, őrértéket elérő túlírás esetén szólal meg
ASLR	a *gadget-cím feloldását* nehezíti	a szövegszegmens báziscíme véletlen; a láncépítéshez cím-szivárgás kell
PAC	a *visszatérési cím manipulálását* nehezíti	a mentett LR kriptográfiai címkét tartalmaz, a `RET`-hez tartozó változat ellenőrzi

Két dolgot érdemes kiemelni.

- **NX nem érinti a ROP-ot.** Az NX a kód-injektálás ellen véd, nem a kód-újrafelhasználás ellen. Aki azt hiszi, hogy `-z noexecstack` mellett „ROP nem lehetséges”, pontosan azt a támadási osztályt hagyja nyitva, amit az NX kikényszerített.
- **PAC az LR-hitelesítésre fókuszál.** Minden, ami nem a `RET`-útvonalra épül (például a `BR Xn`-nel záródó JOP-láncok, vagy a pusztán adatmanipuláló támadások), kimarad a hatóköréből.

14.3.4 PAC hatása konkrétan

AArch64-en PAuth-támogatással a függvény-prológus és -epilógus így nézhet ki (a szemantikát lásd a 14.2 PAC-szakaszában és az Arm ARM-ben):

```
// PAC-kompatibilis prológus (Armv8.3-A PAuth)
paciasp                       // x30 aláírása SP-vel mint modifierrel
stp x29, x30, [sp, #-16]!     // mentés
// ... függvénytest ...
ldp x29, x30, [sp], #16       // visszatöltés
autiasp                       // az aláírás ellenőrzése; hiba esetén "mérgezett" x30
ret
```

A `paciasp`/`autiasp` (illetve a kombinált `retaa`/`retab`) gondoskodnak róla, hogy a veremben tárolt LR-hez tartozó PAC-mező érvényes legyen. Ha a támadó a mentett LR-t gadget-címre cseréli, az `autiasp` érvénytelen címkét kap. Az architektúra szabályai szerint a hibás autentikáció **mérgezett (érvénytelenné tett, poisoned) mutatót** ad vissza, és az azt követő `ret` nem a gadgetre, hanem *fault*-ra vezet. A ROP-lánc első lépcsője (mentett LR → gadget-cím) itt elakad. (`FEAT_FPAC` jelenlétében már maga az `autiasp` kivételt vált ki, nem csak a rákövetkező `ret`.)

Mindez csak akkor igaz, ha a PAuth **ténylegesen működik**, és hogy mi történik, ha nem, az az utasítás alakjától függ. A fenti `paciasp`/`autiasp` a HINT-térbe kódolt alak (`HINT #25`/`HINT #29`): Armv8.3-A előtti, `FEAT_PAuth` nélküli magon, illetve ha a kernel nem engedélyezte az `APIA` kulcsot, NOP-ként fut, a függvény hibátlanul visszatér, és a láncot **semmi nem állítja meg**. A `retaa`/`retab` (és a `braa`, `blraa`, `ldraa`, valamint a regiszteroperandusos `pacia`/`autia`) nem HINT-kódolású: PAuth nélküli magon definiálatlan utasításként `SIGILL`-t okoz. Ezért az alapértelmezett célra fordított kód a HINT-alakot használja, `retaa`-t csak legalább Armv8.3-A célra fordítva kapunk. A védelem meglétéhez három külön ellenőrzés kell (lásd a Védelmi mechanizmusok alfejezet kidolgozott példájának 4. lépését): a kód aláír-e (`objdump`), a hardver támogatja-e a címhitelesítést (`HWCAP_PACA`; a `HWCAP_PACG` a generikus `PACGA`-t jelzi, a visszatérési címhez nem elég), és a kernel engedélyezte-e a kulcsot (`PR_PAC_GET_ENABLED_KEYS`).

A védelem korlátai is audit-tények:

- **A PAC-mező mérete.** A PAC-hez rendelkezésre álló bitek száma processzor- és konfigurációfüggő. Vakon próbálkozva $p = 2^{-N}$ a siker valószínűsége, feltéve független, egyenletes eloszlású kulcsot és ismeretlen címkét; N pontos értéke ellenőrzést igényel, és erősen implementációfüggő.
- **A `RET`-alapú lánc helyett más útvonal.** Ha egy támadó olyan kódhoz jut, amely aláírt mutatót ad ki neki (egyfajta *signing orákulum*), a PAC védelme megkerülhető. Ez az oka annak, hogy a PAC **nem helyettesíti** sem a canaryt, sem az ASLR-t, sem a W^X-et.
- **Data-only támadások.** Ha a támadó nem a vezérlést, hanem az adatot manipulálja (pl. egy jogosultsági zászlót ír át), a PAC teljesen irreleváns, mert a támadás nem érinti a `RET`-et.

A tanulság: a PAC **megdrágítja, de nem akadályozza meg** a ROP-ot. Konkrétan a lánc első lépcsőjét nehezíti meg; a többi fázisra nem hat.

14.3.5 Kidolgozott példa: lánc és miért törik el PAC-cal

Munkakönyvtár: `~/arm-labor/a64/a055/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a055
cd ~/arm-labor/a64/a055
```

Vegyünk egy szándékosan sebezhető laborprogramot (a laborban csak a saját, `lab-` előtagú objektumainkon dolgozunk). A vizsgált függvény külön fájlba kerül, és a pufferét egy **másik fordítási egységben** definiált `lab_sink` olvassa. Erre két okból van szükség. Ha a puffer tartalmát senki nem olvassa, az optimalizáló a tárolásokat, sőt az egész törzset elhagyhatja: a `lab_sink` nélküli változatból már `-O1`-en is puszta `ret` lesz. Másodszor, a `lab_sink` hívása teszi a függvényt **nem-levél** függvénnyé, amely menti az LR-t. A `-mbranch-protection=standard` visszatérésicím-aláírása (`pac-ret`) alapértelmezésben csak ilyen függvényekre vonatkozik. Levélfüggvényre csak a `pac-ret+leaf` beállítással terjedne ki, ezért a `lab_sink` nélküli változatban nem keletkezne `paciasp`.

```
/* lab-a055-vuln.c - a vizsgált függvény; a puffert egy másik fordítási egység olvassa */
extern void lab_sink(const char *buf, unsigned long n);

void lab_vulnerable(const char *input, unsigned long len)
{
    char buf[16];
    unsigned long i;
    for (i = 0; i < len && input[i] != '\0'; i++)
        buf[i] = input[i];      /* hosszellenőrzés nincs: len > 16 esetén túlír */
    lab_sink(buf, i);           /* a puffer tartalma megfigyelhető: a tárolások maradnak */
}
```

A ciklus a lezáró nullánál is kilép. Emiatt a fordító nem alakítja `memcpy`-hívássá, így a bájtonkénti tárolás közvetlenül látszik a disassemblyben.

```
/* lab-a055-main.c - hívó és lab_sink; külön fordítási egység, ezért a
 * lab-a055-vuln.c fordításakor a lab_sink nem ágyazható be (LTO nélkül) */
#include <stdio.h>

void lab_vulnerable(const char *input, unsigned long len);

void lab_sink(const char *buf, unsigned long n)
{
```

```
    printf("lab_sink: %lu bajt: \"%.*s\"\n", n, (int)n, buf);
}

int main(void)
{
    lab_vulnerable("labor a055", 16);   /* 10 bájt + '\0': belefér a 16 bájtos pufferbe */
    return 0;
}
```

A bemenet szándékosan rövid: a labor a keret és a védelem **felismeréséről** szól, nem a túlírás kiváltásáról.

```
# target: cross-build x86_64 hoston, futtatás qemu-aarch64 user-mode-ban
# -fno-stack-protector: a canaryt a Védelmi mechanizmusok alfejezet tárgyalja; itt csak a PAC-
# hatást nézzük
# a -mbranch-protection mindig explicit, mert egyes disztribúciók GCC-je alapértelmezésben
# bekapcsolja
CF="-O2 -fno-stack-protector"                 # -O1-gyel is ugyanez a minta várható
aarch64-linux-gnu-gcc $CF -mbranch-protection=none \
    -c -o lab-a055-vuln-none.o lab-a055-vuln.c
aarch64-linux-gnu-gcc $CF -mbranch-protection=standard \
    -c -o lab-a055-vuln-pac.o lab-a055-vuln.c
aarch64-linux-gnu-gcc $CF -c -o lab-a055-main.o lab-a055-main.c
aarch64-linux-gnu-gcc -static -o lab-a055-none lab-a055-vuln-none.o lab-a055-main.o
aarch64-linux-gnu-gcc -static -o lab-a055-pac  lab-a055-vuln-pac.o  lab-a055-main.o
qemu-aarch64 ./lab-a055-none                  # várt kimenet: lab_sink: 10 bajt: "labor a055"
qemu-aarch64 ./lab-a055-pac                   # ugyanez
aarch64-linux-gnu-objdump -d lab-a055-vuln-none.o
aarch64-linux-gnu-objdump -d lab-a055-vuln-pac.o
```

A disassemblyben négy dolgot keresünk: a bájtonkénti tárolást (`strb`) a veremben lévő pufferbe; a keretrekord mentését és visszatöltését (`stp`/`ldp x29, x30`); a `bl lab_sink` hívást; valamint a PAC-os buildben a `paciasp`/`autiasp` párt (régebbi `objdump` alatt `hint #25`/`hint #29`). Ha ezek közül bármelyik hiányzik, a fordító mást optimalizált, mint amit a labor feltételez, és a további elemzés nem erre a forrásra vonatkozik.

A GCC 13.3 `-O2` kimenete (fő út). A GCC a keretrekordot a keret aljára teszi: `stp x29, x30, [sp, #-32]!`; a puffer a `sp+16..sp+31`, a mentett X29 a `sp+0`, a mentett LR a `sp+8`, a hívó kerete `sp+32`-től indul. A 17. bájt túlírása ezért nem a saját, hanem a **hívó** mentett LR-jét éri el; a lánc a hívó `ret`-jénél indul. PAC-os buildben a prológus első utasítása `paciasp`, az epilógus `autiasp` + `ret`; `bti c` nincs, mert a `paciasp` landing padként szolgál.

Offset (GCC)	Tartalom
+0	mentett X29
+8	mentett X30 (LR)
+16..+31	`buf[0..15]`
+32	a hívó kerete (a túlírás a hívó mentett LR-jét éri el)

Az alábbi két lista **megfigyelés**, nem előírás, és a Clang/musl összehasonlító kimenetét mutatja (zig cc, `aarch64-linux-musl`, `-O2`; `-O1`-en betűre ugyanez), a címeket elhagyva és megjegyzésekkel kiegészítve; a GCC fő út elrendezését az előző bekezdés és táblázat adja. A GNU GCC kimenete ettől eltérhet, különösen a keretrekord helyében (lásd lent). Kiemelt utasítássorozatról van szó, ezért nincs benne `.cfi_*` direktíva; a fordító a forrásszintű kimenetben (`-S`) a CFI-t maga generálja. Az alfejezet többi, kézzel írt gadget-részlete szemléltetés, nem C-ből hívható függvény.

Build A – PAC nélkül (`-mbranch-protection=none`), megfigyelt kimenet:

```
lab_vulnerable:
    sub   sp, sp, #32
    stp   x29, x30, [sp, #16]      // keretrekord: x29 -> [SP+16], x30 (LR) -> [SP+24]
    add   x29, sp, #16
    cbz   x1, 2f
    mov   x8, x1
    mov   x1, xzr                  // i = 0
    mov   x9, sp                   // buf = SP+0 .. SP+15
1:  ldrb  w10, [x0, x1]
    cbz   w10, 2f                  // lezáró nulla: kilépés
    strb  w10, [x9, x1]            // buf[i] = input[i]; i >= 16 esetén a keretrekordot írja
    add   x1, x1, #1
    cmp   x8, x1
    b.ne  1b
    mov   x1, x8
2:  mov   x0, sp
    bl    lab_sink                 // nem-levél függvény: ezért mentette az LR-t
    ldp   x29, x30, [sp, #16]
    add   sp, sp, #32
    ret                            // nincs autentikáció
```

Itt a mentett LR egyszerűen egy memóriaszó a `[SP+24]` címen, és a puffer (`SP+0`…`SP+15`) alatta fekszik. Az 14.1 szerinti, növekvő címek felé haladó túlírás ezért a 17. bájttól a mentett X29-et, a 25.-től a mentett LR-t éri el. Ez a **megfigyelt** keret. A GCC a keretrekordot gyakran a keret aljára teszi (`stp x29, x30, [sp, #-N]!`, a puffer fölötte; vö. a Védelmi mechanizmusok vázlatával), és akkor a túlírás nem a saját, hanem a **hívó** mentett LR-jét éri el. Hogy a saját buildben melyik eset áll fenn, azt mindig a saját `objdump`-kimenetből kell kiolvasni.

A verem képe a `ret` előtt (Build A, a fenti megfigyelt keret alapján):

Offszet (SP-hez képest)	Szerep	Kitöltés
+0	`buf[0..15]`	a bemenet bájtjai (`strb`)
+16	mentett X29	a hívó keretmutatója
+24	mentett X30 (saved LR)	*(érintetlen esetben visszatérési cím; túlírás esetén a lánc első bejegyzése)*
+32	hívó stack frame	érintetlen

Build A esetén az elemző azt látja, hogy a mentett LR egy sima, **hitelesítés nélküli** memóriaszó. A lánc innen koncepcionálisan indítható: a `+24` slotba egy pivot vagy chain-driver gadget címe kerülne, és a folytatás a `+32`-től induló pufferekben kapna helyet.

Ez a szerkezet **fogalmi/illusztratív**. Egy konkrét, működő payload összeállítása nem célja ennek a szakasznak, és a könyv más szakaszaiban sem szerepel receptként.

Build B – PAC-kal (`-mbranch-protection=standard`), megfigyelt kimenet; a Build A-tól eltérő sorok:

```
lab_vulnerable:
    paciasp                        // x30 aláírása, modifier: a belépéskori SP
    sub   sp, sp, #32
    stp   x29, x30, [sp, #16]      // már az aláírt x30 kerül a [SP+24] slotba
    // ... ugyanaz a ciklus és bl lab_sink, mint a Build A-ban ...
    ldp   x29, x30, [sp, #16]
    add   sp, sp, #32
    autiasp                        // ellenőrzés ugyanazzal az SP-vel, mint az aláíráskor
    ret
```

Ugyanaz a +24 slot itt **aláírt** érték. Ha a támadó ezt gadget-címre cseréli, az autiasp a címhez tartozó címkét ellenőrzi – és mivel a támadó nem ismeri a megfelelő PAC-kulcsot és az SP-alapú modifier kontextust – az ellenőrzés elbukik, és a ret nem a gadgetre ugrik, hanem *fault*-ot vált ki. **A lánc első lépcsője tehát itt elakad**, hacsak a támadó nem talál olyan primitívet (pl. BR Xn egy már hitelesített mutatóból), amely megkerüli a RET-hitelesítést.

Ez a különbség jól mutatja, hogy a mentett LR egyetlen memóriaszó pozíciót vesz igénybe, és hogy az NX + PAC **együtt** már komoly akadályt állít – de nem oldja meg a problémát önmagában.

14.3.6 Hibák és ellenpróbák

Hibás feltevés 1: „NX miatt nincs ROP." Ellenpróba: az NX pontosan az injektálást zárja le; a ROP a meglévő kódot használja. Építs egy lab-binárist -z noexecstack-kel, listázd a szövegszegmensben a ret-et megelőző rövid sorozatokat (objdump -d | grep -B2 ret), és számold meg őket. Ha nem nulla, akkor van támadási felület, csak a támadás „hordozója" más.

Hibás feltevés 2: „PAC miatt nincs ROP." Ellenpróba: a PAC csak a hitelesítő epilógust (autiasp + ret, illetve retaa) végrehajtó visszatérést hitelesíti, a sima ret nem; a BR Xn-nel záródó JOP-láncok és a data-only manipulációk hatókörén kívül esnek. A logikailag korrekt állítás az, hogy a PAC **megnehezíti** a mentett LR manipulálását, nem hogy kizárja a ROP-ot.

Hibás feltevés 3: „a canary kivédi a ROP-ot." Ellenpróba: a canary csak *detektál*, és csak folytonos, az őrértéket elérő túlírás esetén. Nem akadályozza meg a mentett LR olvasását, sem a lánc más ponton (pl. függvénymutató cseréjével) való indítását.

Hibás feltevés 4: „az ASLR megoldja a problémát." Ellenpróba: az ASLR entrópiája konfigurációfüggő, és egyetlen kiszivárgott cím (pl. egy formázott printf, egy részleges visszaírás, egy hibás hibakezelési útvonal) az egész réteget deaktiválhatja.

14.3.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Fogalmazd meg 4–6 mondatban, miért volt az NX/DEP bevezetése után **elkerülhetetlen** technikai irányváltás a kód-újrafelhasználás (ROP) felé. Térj ki arra, mi az a primitív, amit az NX nem érint, és miért elég ez egy lánc elindításához.

2. (implementációs/elemző, középszint — cél: rutinépítés) Készíts egy rövid lab-C programot (lab-rop-scan.c), amelyben néhány függvény és egy lokális tömb szerepel. Fordítsd -O1 és -O2 opciókkal, majd objdump -d-vel keresd meg azokat az utasítássorozatokat, amelyek ret-tel zárulnak és legfeljebb 4 utasításból állnak. Készíts táblázatot: hány ilyen mintázat van -O1-ben és -O2-ben? Indokold a különbséget.

3. (mérési/debug, középszint — cél: rutinépítés) A kidolgozott példa lab-a055-vuln.c-jét (nem-levél függvény: a lab_sink hívása miatt menti az LR-t) és egy egysoros levélfüggvényt (lab-a055-leaf.c: unsigned long lab_leaf(unsigned long x) { return 3 * x + 1; }) fordítsd le -mbranch-protection=none, illetve -mbranch-protection=standard opcióval. Az ellenőrzést öt, egymástól független rétegre bontsd: (1) mit kér a fordító **policyje**; (2) mi áll ténylegesen a két **függvény** prológusában és epilógusában (objdump -d); (3) mit jelez az egyes **objektumok** .note.gnu.property jegyzete (readelf -n); (4) mit jelez a **linkelt ELF** jegyzete, ha a saját objektumok mellé egy jegyzet nélküli, kézzel írt kiegészítő objektumot is linkelsz (például egyetlen ret-tel záródó lab-a055-nonote.s-t); (5) támogatja-e a **futtatókörnyezet** a címhitelesítést. Melyik réteg eredményéből következik a többi?

14.3.8 Megoldások

1. megoldás — Segítség. Gondold át, mit zár ki pontosan az NX (adatterület végrehajtásának tiltása), és mi az a primitív (kontrolláram-eltérítés), ami egy memóriahibából önmagában még az NX ellenére is adódik.

1. megoldás — Teljes megoldás. Az NX/DEP a nem-végrehajtható adatlapokkal kizárja, hogy a program egy támadó-ellenőrzött puffert gépkódként futtasson. A támadónak azonban továbbra is szüksége van **vezérlésátadásra**: ha egyetlen memóriahiba (mentett LR felülírása, függvénymutató-felülírás) megvan, a vezérlés eltéríthető. Mivel új kódot nem tud befecskendezni, a program saját szövegszegmensében keres rövid, hasznos utasítássorozatokat. A64-en a `RET` regiszteres visszatérés: a PC-t az X30-ból tölti, a veremhez nem nyúl. Ha azonban a gadget egy olyan epilógussal zárul, mint az `ldp x29, x30, [sp], #16; ret`, akkor az `ldp` a támadó által írt veremből tölti az X30-ba a következő címet, és lépteti az SP-t, a `ret` pedig erre a címre ugrik. Így a támadó a verem tartalmával választja ki a következő végrehajtandó címet, és így láncolhatók a gadgetek. Az NX tehát nem érintette azt a primitívet (vezérlés-eltérítés), amelyre a ROP épül; csak a támadás *hordozóját* változtatta meg.

2. megoldás — Segítség. Gondold át, hogy a magasabb optimalizálási szint inline-olása és közös-kódrész-kiemelése tipikusan növeli vagy csökkenti a különböző, `ret`-tel végződő rövid utasítássorozatok számát egy adott bináris méretéhez képest.

2. megoldás — Teljes megoldás. Tipikus megfigyelés: `-O1`-en a függvények egyszerűbbek, kevesebb a beágyazás, ezért a `ret`-tel záródó 4 utasításos minták száma alacsonyabb, de a minták tisztábbak, könnyebben azonosíthatók. `-O2`-n a beágyazás, a ciklus-átalakítás és a közös kód-rész kiemelése növelheti a `ret`-ek számát és az epilógusok szórását, így a jelölt gadgetek száma is nőhet. A pontos szám erősen függ a GCC verziójától, a forrásmérettől és a hívási gráftól; a laboreredményt jelölten kell rögzíteni, nem általánosítani. **Ellenőrzési igény:** a konkrét darabszám itt nem ellenőrzött.

3. megoldás — Segítség. A `standard` policy alapértelmezett hatóköre nem minden függvény: gondold át, mi különbözteti meg a `lab_vulnerable`-t a `lab_leaf`-től az LR mentése szempontjából. A linkelt ELF jegyzetéről pedig gondold át: ha a linker a bemeneti objektumok feature-bitjeit ÉS-sel vonja össze, mi lesz az eredmény egyetlen jegyzet nélküli objektum mellett?

3. megoldás — Teljes megoldás. Az öt réteg külön kérdés, és egyik sem helyettesíti a másikat.

1. **Policy.** A `-mbranch-protection=standard` a visszatérésicím-aláírást (`pac-ret`) és a BTI-t kéri. A `pac-ret` alapértelmezett hatóköre a **nem-levél** függvények köre, vagyis azoké, amelyek az LR-t elmentik; levélfüggvényekre csak a `pac-ret+leaf` terjeszti ki.
2. **Függvények.** A `standard` buildben a `lab_vulnerable` prológusa `paciasp`-pal kezdődik (régebbi `objdump` alatt `hint #25`), az epilógusa `autiasp` + `ret` (`hint #29`). A `lab_leaf`-ben **nincs** `paciasp`: csak egy `bti c` landing pad, majd a számítás és a `ret`. `pac-ret+leaf`-fel fordítva már ott is megjelenik a `paciasp`. A záró `ret` helyett `retaa`/`retab` csak legalább Armv8.3-A célra fordítva (`-march=armv8.3-a`) jelenik meg, és az ilyen bináris PAuth nélküli magon `SIGILL`-lel áll le. A `none` buildben egyik függvényben sincs PAC-utasítás.
3. **Objektumjegyzet.** A `standard`-dal fordított objektumok `.note.gnu.property` jegyzete (`GNU_PROPERTY_AARCH64_FEATURE_1_AND`) a `PAC` és a `BTI` bitet jelzi, **a `lab-a055-leaf.o`-é is**. A jegyzet tehát azt állítja, hogy az objektum a policy szerint készült, nem azt, hogy minden függvénye aláír. A `none` objektumban nincs PAC-bit, a kézzel írt `lab-a055-nonote.o`-ban pedig egyáltalán nincs jegyzet.

4. **Linkelt ELF.** A linker a bemeneti objektumok feature-bitjeit **ÉS-sel** vonja össze: a végső ELF csak akkor kapja meg a PAC-bitet, ha **minden** linkelt objektum hordozza, a futtatókörnyezet startupfájljait és a statikusan linkelt könyvtárakat is beleértve. A saját, `standard` objektumokból linkelt ELF jegyzete PAC-ot és BTI-t mutat. Ha mellé linkeljük a jegyzet nélküli `lab-a055-nonote.o`-t, a végső jegyzet **eltűnik**, pedig a `lab_vulnerable` utasításai változatlanul aláírnak. A hiányzó végső jegyzet tehát nem fordítóhiba: azt jelenti, hogy legalább egy bemenet nem deklarálta a tulajdonságot. Ilyenkor a bemeneteket egyenként kell `readelf -n`-nel végignézni, a kézzel írt assembly fájlokat a megfelelő `.note.gnu.property` szekcióval kell ellátni, és a startupfájlok, valamint a könyvtárak jegyzetét is ellenőrizni kell.
5. **Futtatókörnyezet.** Az előző négy réteg csak azt mutatja, mit tartalmaz a bináris. Hogy a `paciasp`/`autiasp` ténylegesen véd-e, vagy NOP-ként fut, az a hardver címhitelesítési képességétől (`HWCAP_PACA`) és attól függ, hogy a kernel engedélyezte-e a kulcsot. Ezt a Védelmi mechanizmusok kidolgozott példájának 4. lépése szerint kell ellenőrizni. A QEMU oldali PAuth-támogatás verziófüggő.

A következtetés iránya tehát egyirányú: a végső ELF PAC-bitje a függvények szintjén semmit sem bizonyít (a levélfüggvény bit mellett sem ír alá), a hiánya pedig nem bizonyítja, hogy a függvények nem írnak alá. A védelemről csak a policy, a kiválasztott nem-levél függvény tényleges utasításai, a bemenetek jegyzetei és a futtatókörnyezet képessége **együtt** mondanak valamit.

14.3.9 Rövid összefoglaló

A **ROP-gadget** egy rövid, `RET`-tel záródó, meglévő kódrészlet (a `BR Xn`/`BLR Xn`-nel záródó változat a JOP, illetve a COP). A **ROP-lánc** gadget-címek és adatok verembe rendezett sorozata. A64-en a láncot nem a `RET` lépteti: a `RET` csak az X30-ra ugrik, és az SP-t nem változtatja. A következő címet a `RET` előtti betöltés hozza a veremből az X30-ba (tipikusan `ldp x29, x30, [sp], #16`, amely egyben 16 bájttal lépteti az SP-t). A ROP az NX/DEP bevezetésének **következménye**: a kód-injektálás ellehetetlenítése után a támadó kód-újrafelhasználásra vált. A PAC a visszatérési cím kriptográfiai hitelesítésével a lánc **első lépcsőjét** nehezíti meg, de a `BR`-rel záródó JOP-láncokra és a data-only támadásokra nem hat. Audit szempontból a ROP-ismeret **felismerési** készség: egyetlen kontrolláram-eltérítési primitív már elég a súlyos következményhez.

- **ROP-gadget:** meglévő kódrészlet, `RET`-tel záródik, minimális műveletet végez; a `BR Xn`/`BLR Xn`-nel záródó változat a JOP/COP.
- **ROP-lánc:** verembe írt gadget-címek és adatok sorozata. Egy A64-es lánclépés: az `ldp … x30, [sp], #16` olvassa a vermet, és lépteti az SP-t, a `ret` pedig a PC-t az X30-ból tölti, az SP-t változatlanul hagyva.
- **Visszatérési cím hitelesítés (PAC):** a mentett LR-hez tartozó kriptográfiai címke; `autiasp`/`retaa` ellenőrzi.

14.3.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile Architecture Reference Manual – a `PAC*`, `AUT*`, `RETAA`/`RETAB`, valamint a `RET` viselkedésének elsődleges forrása. **Ellenőrizve**: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.

- Arm Procedure Call Standard (AAPCS64, IHI0055) – a prológus/epilógus és a veremkeret szabályai; a PAC-utasítások szemantikájának forrása az Arm ARM. **Ellenőrizve**: aktuális kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-vonatkozású részek Beta).
- GNU Binutils dokumentáció – `objdump`, `readelf` és a `.note.gnu.property` értelmezéséhez. **Ellenőrizve**: aktuális kiadás binutils 2.47.
- GCC AArch64 Options – Branch Protection – a `-mbranch-protection` opciócsalád. **Ellenőrizve**: legújabb listázott kiadás GCC 16.2.
- GCC Branch Protection dokumentáció – a jegyzet-bit és a kapcsolódó konvenciók. **Ellenőrizve**: ugyanaz a 16.2-es kiadás.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak:

- A PAC-mező pontos bitszáma adott Armv8.3-A implementáción és konfiguráción; nem validált.
- A QEMU `qemu-aarch64` PAuth-támogatása – különösen az `autiasp` hibához vezető „merging/poisoning" szemantikája – build- és CPU-modell-függő.
- A 3. feladat rétegeit a helyettesítő eszközlánccal (zig cc/Clang, `aarch64-linux-musl`, `ld.lld`) ellenőriztük. A `standard` buildben a `lab_vulnerable` `paciasp`-pal kezdődött, a `lab_leaf` csak `bti c`-vel (`pac-ret+leaf`-fel `paciasp`-pal). Mindhárom `standard` objektum jegyzete `BTI, PAC`-ot mutatott, a levélfüggvényé is; `ld -r`-rel összefűzve is megmaradt. A `gcc -static`-kal linkelt ELF-ben viszont nincs jegyzet, mert az Ubuntu 24.04 cross-glibc startupfájljai (crt1.o, crti.o) nem hordozzák, és a linker ÉS-sel vonja össze; a jegyzetet ezért az objektumokon vagy `ld -r`-rel összefűzve kell ellenőrizni. A program ettől függetlenül helyesen fut.
- A `-mbranch-protection=standard` pontos hatása az Ubuntu 24.04-beli GCC verziójában ellenőrzést igényel. A helyettesítő eszközlánccal (Clang) ellenőrizve: alapértelmezett célon `hint #25`/`hint #29` + `ret`, Armv8.3-A célon `paciasp` + `retaa` keletkezik.
- A `lab-a055-vuln.c` fordítói kimenetét a helyettesítő eszközlánccal (zig cc/Clang, `aarch64-linux-musl`, `-fno-stack-protector`) `-O1` és `-O2` mellett, `-mbranch-protection=none` és `standard` beállítással is megnéztük. Mind a négy esetben megjelent a `strb`-ciklus, az `stp`/`ldp x29, x30` és a `bl lab_sink`, a `standard` buildben pedig a `paciasp`/`autiasp`. A két statikus build az Unicorn-alapú emulátorban a `lab_sink: 10 bajt: "labor a055"` sort írta ki. A GNU GCC kimenete (keretrekord helye, ciklusforma) a hallgatói laborban ellenőrizendő.
- A lánclépés (`ldp x29, x30, [sp], #16; ret`) regiszterhatását a helyettesítő eszközlánccal (Unicorn AArch64-emuláció, utasításonkénti nyomkövetés) mértük, nem GNU/qemu alatt: az `ldp` az X30-at a `[SP+8]`-ról tölti, és SP-t 16-tal növeli; a `ret` előtt és után az SP azonos, a PC az X30 értékét kapja; az X30-at újra nem töltő `add x0, x0, #1; ret` gadget önmagára ugrik vissza.
- A gadget-szám tipikus binárisokban erősen függ a toolchaintől, az optimalizálási szinttől és a linkelt könyvtáraktól; a fenti táblázatban szereplő értékek **illusztratívak, nem mért adatok**.

14.4 Biztonságos assembly- és C-interop gyakorlat

14.4.1 Motiváció: a határ a leggyengébb láncszem

Az Assembly függvény hívása C-ből és C hívása assemblyből alfejezet egyik fő megállapítása az volt, hogy a sikeres linkelés **semmit nem ellenőriz**. A C fordító a `lab_kernel(p, n)` híváskor legfeljebb annyit tud, amit a fejlécbeli prototípusból kiolvas: a pointer és a hossz futásidejű értékéről, a fogadó fél tényleges kapacitásáról, a pufferek átfedéséről nem tud semmit. Amint

a vezérlés átlép a saját kezűleg írt assembly modulba, megszűnik a típusrendszer, a Fortify, az automatikus konverzió, sőt a `-Wall` is.

A verem-túlcsordulás mechanizmusa assembly szinten alfejezet pontosan ezt a rést használta ki demonstrációs célból: egy `strb` ciklus, amely egy fix méretű keretbeli puffert írt, miközben a hossz a hívótól, ellenőrizetlenül érkezett. A ROP-lánc felismerése és miértje pedig megmutatta, mi lesz ebből a következmény: a felülírt mentett visszatérési cím kontrolláram-eltérítési primitív.

A mérnöki feladat ebben az alfejezetben tehát nem az, hogy *felismerjük* a hibát (azt már megtettük), hanem hogy **megtervezzük a határt**, ahol a hibát meg lehet fogni: a hívó és a hívott közötti szerződést rögzítjük, és a hívott fél a saját kapacitásához köti a kapott hosszt, mielőtt egyetlen bájtot is írna.

14.4.2 Definíció: az interop-határ biztonsági szerződése

Definíció. A C–assembly interop-határ *szerződése* négy dimenzió együttes rögzítése, amelyet a két oldalnak azonos módon kell értelmeznie:

1. **értéktartomány** — minden argumentum megengedett értékkészlete (méret, index, kapacitás, kapcsoló bit);
2. **pointer-érvényesség** — mit jelent az, hogy egy pointer használható (nem-NULL, olvasható, elég hosszú, nem fed át a céllal);
3. **tulajdonjog és élettartam** — ki írja és ki olvassa a memóriát, meddig él a puffer;
4. **hibajelzési konvenció** — hogyan jelzi a hívott fél, hogy a szerződés megsértése miatt *nem* hajtotta végre a műveletet.

A négy dimenzióból az első kettő az, amit a legtöbbször elhanyagolunk. Fontos mérnöki belátás: a **pointer érvényessége általánosan nem eldönthető** C-ben. A `p != NULL` csak a legdurvább esetet fogja meg; egy `0x1000`-szerű, nem leképezett cím éppúgy átcsúszik, mint egy olyan, amely a szomszéd tömbbe mutat. Ezért a *működő* stratégia nem az, hogy „megvizsgáljuk, érvényes-e a pointer”, hanem hogy **a kapott hosszt a fogadó fél saját, fordítási időben ismert kapacitásához kötjük**. Ez az az ellenőrzés, amely olcsó, hordozható és determinisztikus.

Ellenőrzés	Hol végezhető	Költség	Mit nem fog meg
NULL-ellenőrzés	határon, 1 regisztertesztt	~1 utasítás	érvénytelen, de nem-NULL pointert
hossz felső korlát	határon, 1 összehasonlítás	~2 utasítás	hibás, de a korláton belüli hosszt
pointer-tartomány (linker szimbólumok, pl. `__bss_start`, `_end`)	futásidőben, platformfüggő	több tíz ciklus	nem hordozható, nem teljes
ASan / hardveres címcímkézés	teszt- vagy speciális build	2–10× lassulás	éles buildben nincs jelen

14.4.3 Kidolgozott példa: határellenőrzött másoló

Munkakönyvtár: `~/arm-labor/a64/a056/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/a64/a056
cd ~/arm-labor/a64/a056
```

A cél egy olyan modul, amely egy **64 bájtos belső puffert** tart karban, és egy `(src, n)` párt fogad el. A szerződés teljes alakja:

Szerződési elem	Kinek a kötelezettsége	Ellenőrzi-e a modul?
`src != NULL`	hívó	igen (`cbz`)
`n  63` (a lezáró `'\0'` helye is kell)	hívó	igen (`cmp/b.hi`)
`src[0 … n-1]` olvasható, azaz legalább `n` érvényes forrásbájt	hívó	**nem** – általánosan nem is ellenőrizhető
a forrás nem fed át a belső puffer írt tartományával (`[0 … n]`); megengedett átfedés nincs, így a `lab_buffer()` mutatója sem adható át forrásként	hívó	nem
a belső puffer egyetlen, közös, statikus példány; a modul **nem reentráns** és nem szálbiztos, párhuzamos (vagy jelkezelőből érkező) hívásokat a hívónak kell sorosítania	hívó	nem
siker: visszatérés `n`, a pufferben `n` bájt és egy `'\0'`	modul	–
elutasítás (`src == NULL` vagy `n > 63`): visszatérés `-1`, **a puffer változatlan**	modul	–

A `lab_buffer()` által visszaadott mutató a modul belső tárára mutat, **nem önálló élettartamú eredményobjektum**: a mutató a program teljes futása alatt érvényes, a tartalom viszont csak a következő sikeres `lab_copy_bounded` hívásig. Akinek tartós eredmény kell, a hívás után átmásolja. A hosszellenőrzés ezen semmit nem változtat: egy jól ellenőrzött hossz sem teszi a közös puffert használó függvényt reentránssá.

A „mindent vagy semmit” garancia tehát a **szerződés alapján elutasított** argumentumokra vonatkozik (NULL, túl nagy `n`). Ha a hívó megszegi a nem ellenőrzött feltételeket (például `n` megengedett, de a forrásból csak 10 bájt olvasható), a másolás közben memóriahiba keletkezhet, és ekkor a puffer félig felülírt állapotban maradhat. Erre a modul nem ad és nem is adhat garanciát.

A teszthajtó minden esetnél a hívás előtt és után is lemásolja a puffert. Elutasításkor azt ellenőrzi, hogy a tartalom bájtra azonos maradt, sikernél pedig a másolt bájtokat és a lezáró nullát. Az esetek a határ két oldalát (`62`, `63`, `64`) és a konverziós szélsőértéket (`SIZE_MAX`) is lefedik:

```
/* lab_a056.c - biztonságos interop-határ demonstráció (A64 ISA) */
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>

#define LAB_CAP 64u

/* Szerződés (a teljes szöveget lásd a fejezetben):
 *  - src != NULL, és src[0..n-1] olvasható (legalább n bájt);
 *  - n <= LAB_CAP-1 (63);
```

```
 *  - a forrás nem fedhet át a belső pufferrel (src nem lehet lab_buffer()-ből);
 *  - a belső puffer egyetlen, közös példány: a függvény nem reentráns,
 *    párhuzamos hívásokat a hívónak kell sorosítania.
 * Visszatérés: n siker esetén (a puffer: n bájt + '\0');
 *              -1, ha src == NULL vagy n > 63 - ekkor a puffer érintetlen. */
long           lab_copy_bounded(const uint8_t *src, size_t n);
const uint8_t *lab_buffer(void);   /* csak megfigyeléshez; a tartalom a
                                      következő sikeres hívásig érvényes */

static uint8_t src[4096];

static int check(const char *name, const uint8_t *p, size_t n, long want)
{
    uint8_t before[LAB_CAP], after[LAB_CAP];
    memcpy(before, lab_buffer(), LAB_CAP);
    long got = lab_copy_bounded(p, n);
    memcpy(after, lab_buffer(), LAB_CAP);

    int ok = (got == want);
    if (want < 0)                       /* elutasítás: nincs mellékhatás */
        ok = ok && memcmp(before, after, LAB_CAP) == 0;
    else                                /* siker: n bájt + lezáró nulla  */
        ok = ok && memcmp(after, p, n) == 0 && after[n] == 0;
    printf("%-16s -> %3ld (várt %3ld) %s\n",
           name, got, want, ok ? "OK" : "HIBA");
    return ok;
}

int main(void)
{
    int ok = 1;
    memset(src, 'A', sizeof(src));

    ok &= check("n=5",        src, 5,          5);
    ok &= check("n=62",       src, 62,        62);
    ok &= check("n=63",       src, 63,        63);
    ok &= check("n=64",       src, 64,        -1);
    ok &= check("NULL, n=5",  NULL, 5,        -1);
    ok &= check("n=SIZE_MAX", src, SIZE_MAX, -1);
    printf("puffer hossza: %zu\n", strlen((const char *)lab_buffer()));
    return ok ? 0 : 1;
}
```

Az assembly oldal (A64 ISA, little-endian):

```
/* lab_bound.S - határellenőrzött másoló (A64 ISA) */
    .arch armv8-a
    .equ    LAB_CAP,  64
    .equ    LAB_MAXN, LAB_CAP - 1       /* 63: a lezáró '\0' indexe is beleférr */

    .bss
    .align  4
lab_buf:
    .skip   LAB_CAP
    .size   lab_buf, LAB_CAP

    .text
    .global lab_copy_bounded
    .type   lab_copy_bounded, %function
lab_copy_bounded:
    /* x0 = src, x1 = n  (A64 ISA) */
    cbz     x0, .Lreject                /* pointer-érvényesség: nem-NULL  */
    mov     x2, #LAB_MAXN
```

```
    cmp     x1, x2
    b.hi    .Lreject                    /* n > 63 -> elutasítás, MÉG NEM ÍRTUNK */

    adrp    x3, lab_buf                 /* lapcím + 12 bites eltolás      */
    add     x3, x3, :lo12:lab_buf

    mov     x4, #0                      /* i = 0                          */
.Lloop:
    cmp     x4, x1
    b.hs    .Ldone
    ldrb    w5, [x0, x4]
    strb    w5, [x3, x4]
    add     x4, x4, #1
    b       .Lloop
.Ldone:
    strb    wzr, [x3, x1]               /* '\0' az n. indexre (n <= 63)   */
    mov     x0, x1                      /* siker: a másolt bájtok száma   */
    ret
.Lreject:
    mov     x0, #-1                     /* MOVN -> x0 = 0xFFFF_FFFF_FFFF_FFFF */
    ret
    .size   lab_copy_bounded, . - lab_copy_bounded

    .global lab_buffer
    .type   lab_buffer, %function
lab_buffer:
    adrp    x0, lab_buf
    add     x0, x0, :lo12:lab_buf
    ret
    .size   lab_buffer, . - lab_buffer

    .section .note.GNU-stack,"",%progbits   /* nem kér végrehajtható vermet */
```

Fordítás és futtatás (`qemu-user` target). A teszthajtó kilépési állapota 0, ha minden eset `OK`, egyébként 1:

```
aarch64-linux-gnu-gcc -O2 -Wall -Wextra -fstack-protector-strong -fPIE -pie \
    -o lab-a056 lab_a056.c lab_bound.S
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab-a056; echo "kilépési állapot: $?"
```

Kimenet a fő úton mérve (GNU + QEMU 8.2.2, a lektor mérése); a kimenet csak a programlogikától függ:

```
n=5              ->   5 (várt   5) OK
n=62             ->  62 (várt  62) OK
n=63             ->  63 (várt  63) OK
n=64             ->  -1 (várt  -1) OK
NULL, n=5        ->  -1 (várt  -1) OK
n=SIZE_MAX       ->  -1 (várt  -1) OK
puffer hossza: 63
kilépési állapot: 0
```

A határ körüli három bemenet a helyes kódra és a két jellegzetes elrontott változatra (mindegyik ugyanabból a forrásból, egyetlen sor módosításával készült és lefutott):

Változat	`n = 62`	`n = 63`	`n = 64`	A hiba jellege
helyes: `cmp x1, #63` + `b.hi`	62	63	−1	–

Változat	`n = 62`	`n = 63`	`n = 64`	A hiba jellege
`b.hs` a `b.hi` helyett (63-mal hasonlítva)	62	**−1**	−1	túl szigorú: az érvényes 63-at is elutasítja, memóriahibát nem okoz
`cmp x1, #64` + `b.hi` (a kapacitással, nem a maximális hosszal hasonlítva)	62	63	**64**	túl engedékeny: a `'\0'` a 64. indexre, a puffer mögé kerül

Levezetés az indexhatárra. Tegyük fel, hogy az elutasítás után az `n  63` feltétel fennáll. A ciklusváltozóra `i  {0, …, n-1}`, így a cikluson belüli legnagyobb írási index `n-1  62`. A lezáró bájt indexe `n  63`. A puffer érvényes indexei `0 … 63`, tehát **minden írás a pufferen belül marad**. A feltétel tehát elégséges. A `cmp x1, #63` után a `b.hi` a 63-nál nagyobb, előjel nélküli hosszt utasítja el: `n = 63` még elfogadott, `n = 64` már elutasított. `b.hs`-re cserélve a 63-as értéket is elutasítanánk. A lezáró nulla a belső 64 bájtos puffer `n` indexű bájtjába kerül. Elfogadási táblázat: `n = 62` → siker (62); `n = 63` → siker (63); `n = 64` és `n = SIZE_MAX` → `-1`, a puffer bájtra változatlan. Mind a négy sort a fenti teszthajtó ellenőrzi.

Miért fontos a sorrend? Az elutasítási ág a *másolás előtt* fut le, ezért a szerződés alapján elutasított argumentumokra a modul „mindent vagy semmit” módon viselkedik: nem marad félig felülírt puffer. Ez a tulajdonság nem esztétikai kérdés — egy részleges írás a hívó oldalon konzisztensnek látszó, de tartalmilag hibás állapotot hagyhat maga után.

14.4.4 Tipikus hibák és ellenpróbák

1. `mov w0, #-1` a `mov x0, #-1` helyett. A W-regiszterbe írás a felső 32 bitet nullázza, így `x0 = 0x0000_0000_FFFF_FFFF`: a hívó `long` visszatérési értéke `4294967295` lesz, nem `-1`. A hívóoldali `if (ret < 0)` ellenőrzés csendben elmarad. Ellenpróba: `aarch64-linux-gnu-objdump -d --disassemble=lab_copy_bounded lab-a056 | grep -E 'mov[nz]?\s+[wx]0, #'` (a `movn` keresése), illetve a C oldalon `printf("%ld")`, nem `%d`.

2. A határellenőrzés a konverzió után történik-e. Ha a C hívó `int`-et ad át és a fogadó `size_t`-ként kapja, a negatív érték előjeles→előjel nélküli konverzióval hatalmas számmá válik. Ez *nem* hiba, ha az ellenőrzés az átvétel **utáni** értéken fut (nálunk így van: a `cmp x1, x2` a nyers bitmintát vizsgálja). Ellenpróba: `lab_copy_bounded(src, (size_t)-1)` → várt `-1`.

3. Off-by-one a feltételkódban vagy a konstansban. A `cmp x1, #63` után a `b.hi` az `n > 63`, a `b.hs` az `n  63` esetet utasítja el. A `b.hs` tehát **nem** engedi át a 64-et, hanem túl szigorú: az érvényes `n = 63`-at is visszautasítja (a tesztben az `n=63` sor `-1`-et ad, és a puffer hossza 62 marad). Ez funkcionális hiba, memóriát nem ír felül. A veszélyes off-by-one a másik irány: ha a kódot a kapacitással (`cmp x1, #64`) hasonlítjuk a maximális hossz helyett, a `b.hi` átengedi a 64-et, és a `strb wzr, [x3, x1]` a 64. indexre ír, egy bájttal a puffer mögé. Ez az 7.1 óta ismert `[Xn, Xm]` címzési mód klasszikus csapdája. Ellenpróba: a fenti táblázat három sora; az `n=63` sornak `63`-at, az `n=64` sornak `-1`-et kell adnia.

4. Hiányos clobber-lista inline assemblynél. Ha ugyanezt a logikát C `asm volatile` blokkban írnánk meg (lásd Inline assembly GCC-ben), az `x0-x5` regisztereket vagy kimeneti/bemeneti

megszorításként kell deklarálni, vagy szerepelniük kell a clobber-listában. Hiányzó bejegyzés esetén a fordító egy élő értéket tart nyilván ugyanabban a regiszterben, és a keletkező hibát semmi nem jelzi — a program „néha” rossz eredményt ad, optimalizálási szinttől függően.

5. Mentési aszimmetria. A fenti rutin levélfüggvény: nem hív mást, és csak az `x0-x5` caller-saved (hívó által mentendő) regisztereket használja, veremhasználat nélkül. Amint a rutin `bl`-t kap vagy `x19-x28`-at használ, minden `stp`-hez tartoznia kell egy `ldp`-nek, és a veremmutatót 16 bájtos határra kell igazítani (AAPCS64 regiszterszerződés, Prológus és epilógus). Ellenpróba: a `ret` előtt `mov sp, x29`-szerű visszaállítás hiánya GDB-ben „nem oda visszatér” tünettel jelenik meg.

14.4.5 Fordítói védelmi kapcsolók (rövid említés)

A fenti ellenőrzéseket érdemes kiegészíteni, nem helyettesíteni a fordítói védelmi kapcsolókkal:

- `-fstack-protector-strong` — canary-t tesz a helyi tömböt vagy címképzett lokális változót tartalmazó függvények keretébe;
- `-D_FORTIFY_SOURCE=2 -O2` — a `memcpy`/`strcpy` családot futásidejű méretellenőrzéssel ellátott változatra cseréli, ahol a méret fordítási időben ismert;
- `-fPIE -pie` — a kód pozíciófüggetlen, így az ASLR a teljes binárist áthelyezi;
- `-Wl,-z,relro,-z,now` — a dinamikus szerkezetek írásvédetté válnak a betöltés után.

Ubuntu alatt ezek alapértelmezettek; más disztribúción ellenőrizd (`gcc -Q --help=common | grep stack-protector`, `-dM -E` a `_FORTIFY_SOURCE` makróra).

Fontos: ezek a **következményt** detektálják vagy nehezítik, nem az okot szüntetik meg. A canary a felülírás után szól, a határellenőrzés viszont az írás előtt. Ez a két mechanizmus nem versenytárs, hanem réteg. A konkrét kapcsolók elérhetősége és viselkedése az adott GCC-verzióhoz kötött; a fenti parancsokkal a saját toolchainen ellenőrizhető (lásd az utolsó szakaszt).

14.4.6 Kódreview-checklista assembly-modult tartalmazó C projekthez

#	Ellenőrzés	Kérdés a review-n	Bizonyíték
1	Szerződés rögzítve	Hol van írva az argumentumok értéktartománya, az olvasható forráshossz, a megengedett átfedés, a közös állapot (reentrancia) és a hibajelzés?	fejléc-komment, `.h` prototípus
2	Határellenőrzés a határon	A hossz a fogadó kapacitásához van kötve?	`cmp`/`b.hi` az első írás előtt
3	Ellenőrzés a konverzió után	Az `int`→`size_t` konverzió előtt vagy után vizsgálunk?	disassembly, `objdump -d`
4	Nulla mellékhatás elutasításkor	Elutasítás után érintetlen-e az állapot?	határérték-teszt kimenete

#	Ellenőrzés	Kérdés a review-n	Bizonyíték
5	Mentési szimmetria	Minden `stp`-nek van `ldp` párja minden kimeneti úton?	`objdump`, minden `ret` áttekintése
6	Veremigazítás	`sp` 16 bájtos többszöröse minden `bl` előtt?	pro-/epilógus hossza
7	Callee-saved szabály	Az `x19`–`x28`, `x29`, `sp` értéke helyreáll?	AAPCS64-tábla összevetése
8	Clobber-lista / megszorítások	Minden érintett regiszter deklarált inline asm-nál?	GCC output, `-S`
9	Visszatérési érték konvenciója	A `-1` `x0`-ban, nem `w0`-ban képződik?	disassembly
10	Fordítói védelmi kapcsolók	Bekapcsoltak a fenti flagek az éles buildben?	`makefile`, `readelf -h` (PIE), `readelf -d` (BIND_NOW)
11	Fuzz / határérték-tesztek	Van teszt `n = 0`, `n = max`, `n = max+1`, `NULL` esetre?	CI-kimenet

Ez a lista egyben a visszacsatolás és auditmódszer előképe: a Záróprojekt I auditlépése ugyanezeket a pontokat kéri számon, kiegészítve a mérési terv és a reprodukálhatóság ellenőrzésével.

14.4.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Egy kollégád azt állítja: „elég, ha a határon ellenőrizzük, hogy a pointer nem `NULL`, mert a hosszt úgyis a hívó számolja”. Fogalmazz három ellenérvet, és add meg mindegyikhez azt a konkrét tesztesetet, amely megkülönbözteti a két álláspontot!

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a modult egy `long lab_copy_n(const uint8_t *src, size_t n, size_t cap)` rutinnal, amely a pufferkapacitást (a lezáró nulla helyét is beleértve) **paraméterként** kapja, és ott is explicit határellenőrzést végez! Gondoskodj arról, hogy a C oldali `int` átadás esetén is helyes legyen a viselkedés, és hogy a rutin ne írjon a pufferen kívülre. Írj hozzá legalább négy határérték-tesztet!

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Adott egy variáns, amelyben a `cmp x1, x2` utasítás a ciklus **után** szerepel, és a `cbz` a ciklus **előtt**. GDB-vel (a `qemu-aarch64 -g` és `gdb-multiarch` párossal) keresd meg, melyik konkrét tároló utasítás ír a puffer mögé az `n = 64` esetben, majd egyszeri módosítással javítsd és igazold a javítást!

14.4.8 Megoldások

1. megoldás — Segítség. Gondold át külön: mit szűr ki a `NULL`-ellenőrzés, és mit NEM — egy pointer lehet nem-`NULL` és mégis érvénytelen, a hívó által adott hossz pedig maga is hibás lehet.

1. megoldás — Teljes megoldás. (a) A `NULL`-ellenőrzés csak egyetlen érvénytelen értéket szűr ki; egy `0x1000`-re mutató vagy a szomszéd objektumba mutató pointer átmegy rajta, és az

írás a puffer mögé kerül. Teszt: `lab_copy_bounded((const uint8_t *)0x1000, 8)` — a hívás nem `NULL`, mégis érvénytelen. (b) A hívó által számolt hossz maga is lehet hibás (túlcsorduló `size_t` szorzás, off-by-one), és a fogadó fél az egyetlen, aki ismeri a saját kapacitását. Teszt: `n = LAB_CAP` → várt `-1`. (c) A hívó és a hívott közötti szerződés nem kényszeríthető ki a hívó oldalon, mert a hívó lehet harmadik féltől származó kód. Teszt: az `n=64` sor kimenete `-1` kell legyen, függetlenül attól, ki hívja a rutint.

2. megoldás — Segítség. A sorrend számít: előbb a `cap` saját felső korlátját ellenőrizd (nehogy a hívó maga is túlígérjen), utána a kért `n`-t a `cap`-hez képest, és csak ezután induljon a másoló ciklus.

2. megoldás — Teljes megoldás. A kulcs a sorrend: először `cap` felső korlátja (a modul statikus puffere), majd a hívó által kért `n` összevetése a `cap`-pel, és csak ezután a ciklus. Vázlat:

```
lab_copy_n:                     /* x0=src, x1=n, x2=cap (A64 ISA) */
    cbz     x0, .Lrej
    mov     x6, #LAB_CAP
    cmp     x2, x6              /* cap <= LAB_CAP (a pufferkapacitás) */
    b.hi    .Lrej
    cmp     x1, x2              /* n < cap ?  -- a lezáró nulla is belefér */
    b.hi    .Lrej
    /* ... ugyanaz a másoló ciklus, n helyett x1 ... */
```

A tesztek: `n = 0, cap = 64` → 0; `n = 63, cap = 64` → 63; `n = 64, cap = 64` → −1; `n = 5, cap = 65` → −1 (a `cap` maga is korlátos); `src = NULL` → −1. A `cap` jelentése végig **kapacitás**, nem maximális hossz: a hasznos hossz legfeljebb `cap - 1`. Az `int`→`size_t` eset azért biztonságos, mert a negatív érték hatalmas előjel nélküli számmá alakul, és a `b.hi` úgyis elutasítja.

3. megoldás — Segítség. Számítsd ki pontosan, mely index a ciklus utolsó, még engedélyezett írása, és melyik index az, amelyre a lezáró tárolóutasítás ír — a kettő eltérése mutatja meg, hol csúszik túl a határon.

3. megoldás — Teljes megoldás. A vizsgált hibás változat teljes kódja:

```
.global lab_copy_bounded_bad
.type lab_copy_bounded_bad, %function
lab_copy_bounded_bad:
    stp     x29, x30, [sp, #-16]!
    mov     x29, sp
    cbz     x0, .Lbad_rej
    mov     x3, x0
    ldr     x4, =lab_buf
    mov     x5, #0
.Lbad_loop:
    cmp     x5, x1
    b.hs    .Lbad_check
    ldrb    w6, [x3, x5]
    strb    w6, [x4, x5]
    add     x5, x5, #1
    b       .Lbad_loop
.Lbad_check:
    strb    wzr, [x4, x1]       /* a lezáró nulla ELŐBB íródik */
    mov     x6, #LAB_MAXN
    cmp     x1, x6
    b.hi    .Lbad_rej
    mov     x0, x1
    ldp     x29, x30, [sp], #16
    ret
.Lbad_rej:
    mov     x0, #-1
    ldp     x29, x30, [sp], #16
```

```
    ret
.size lab_copy_bounded_bad, .-lab_copy_bounded_bad
```

A hibás variánsban a ciklus legfeljebb a `strb w5, [x3, x4]` utasítással ír az `x4 = 63` indexre (ez még a pufferen belül van), majd a lezáró `strb wzr, [x3, x1]` az `x1 = 64` indexre ír — ez már a `lab_buf` utáni bájtra mutat. Mivel a lezáró tárolás az ellenőrzés **előtt** fut, az `n = 64` esetben a puffer tartalma is megváltozik, nem csak egy bájttal túlíródik. A GDB-ben `break *lab_copy_bounded+...` töréspontot érdemes a lezáró tárolóra tenni, majd `x/8bx &lab_buf+60` paranccsal ellenőrizni a puffer környékét. A javítás: a `cmp/b.hi` párt a ciklus elé mozgatni, és a `strb wzr` indexét a már ellenőrzött `n`-re hagyni. Igazolás: az `n=64` sor `-1`-et ad `OK`-val (a teszthajtó a puffer változatlanságát is ellenőrzi), az `n=63` sor továbbra is `63`, a kilépési állapot 0.

Mérési megjegyzés: a határellenőrzés költsége itt négy utasítás (`cbz`, `mov`, `cmp`, `b.hi`), ami elhanyagolható a másolási ciklus mellett; érdemi változás akkor várható, ha a rutin néhány bájtos bemenetre hívódik, és az ellenőrzés dominálja a futásidőt. **QEMU-emulációban ez relatív/illusztratív adat, nem hardverhű abszolút ciklusszám.**

14.4.9 Rövid összefoglaló

A C–assembly határ biztonsági szerződése négy dimenzióból áll: értéktartomány, pointer-érvényesség, tulajdonjog és hibajelzési konvenció. A linkelés egyiket sem ellenőrzi. Mivel a pointer érvényessége általánosan nem eldönthető, a legértékesebb ellenőrzés az, amely a kapott hosszt a fogadó fél saját kapacitásához köti, a konverzió **után** és az első írás **előtt**. A fordítói védelmi kapcsolók a következményt detektálják, nem az okot. A review-checklista tizenegy pontja (szerződés, határellenőrzés, mentési szimmetria, clobber-lista, hibajelzés, tesztek) közvetlenül a záróprojekt auditlépésére készít fel.

Továbbhasznált definíciók:

- **interop-szerződés:** az argumentumok értéktartománya, a pointer-érvényesség értelmezése, a tulajdonjog és a hibajelzési konvenció együttese.
- **kapacitáskötött határellenőrzés:** a fogadott hossz összevetése a hívott fél saját, fordítási időben ismert kapacitásával.
- **mentési szimmetria:** minden regisztermentő utasításhoz tartozó, minden kimeneti úton végrehajtott helyreállítás.

14.4.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile Architecture Reference Manual — az A64 utasítások, a `MOVN` és a feltételkódok elsődleges forrása. *Ellenőrizve*: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- AAPCS64 – Arm hivatalos ABI-dokumentum — regiszterhasználat, mentési kötelezettségek, veremigazítás. *Ellenőrizve*: a jelenlegi kiadás 2025Q4 (2026-01-23), Release minőségű (csak az SME-részek Beta).
- GCC kézikönyv — `-fstack-protector-strong`, `-fPIE`, `_FORTIFY_SOURCE`; az inline assembly megszorításai és clobber-listája. *Ellenőrizve*: a legújabb listázott kiadás GCC 16.2; a helyben telepített verzió ettől eltérhet.

- GNU assembler kézikönyv — AArch64-szintaxis, `.equ`, `.size`, `.bss` direktívák. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.
- QEMU User Mode Emulation — a `qemu-aarch64 -L` loader-útvonal és a `-g` GDB-stub használata. *Ellenőrizve*: a `master`ághoz tartozó dokumentáció, nem egy konkrét kiadott verzióhoz kötött; nem állít semmit a cache/sávszélesség-modellezésről sem.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: a `lab_a056.c` + `lab_bound.S` páros kimenete, a 62/63/64-es táblázat mindhárom sora és a két mutáns viselkedése a fő úton, GNU + QEMU 8.2.2 alatt mérve (a lektor mérése); a `-D_FORTIFY_SOURCE=2` figyelmeztetés nélkül fordul. A laborban, a saját környezetben ellenőrizendő: (1) az Ubuntu 24.04 LTS-ben csomagolt `gcc-aarch64-linux-gnu`, `binutils-aarch64-linux-gnu` és `qemu-user` konkrét verziója és a `/usr/aarch64-linux-gnu` sysroot-útvonal; (2) a `-fstack-protector-strong` tényleges viselkedése AArch64-en az adott GCC-verzióval; (3) a `readelf -h` által mutatott PIE-típus és a `readelf -d BIND_NOW` bejegyzése; (4) a `qemu-aarch64` user-mode GDB-stubjának (`-g`) működése `gdb-multiarch`-gyal.

15 Záróprojekt I: teljesítménykritikus assembly modul C programban

15.1 Projektspecifikáció: feladat, interfész és mérési terv

15.1.1 Motiváció: a specifikáció mint mérnöki szerződés

A záróprojekt első lépése nem a kódírás, hanem a feladat rögzítése. Ez nem bürokrácia: a teljesítménykritikus assembly modulnál a specifikáció az a pont, ahol a *szándék* és a *mérhető követelmény* szétválik a *megvalósítástól*. Ha a feladat, az interfész és az elfogadási feltétel nincs rögzítve az első sor assembly megírása előtt, akkor később nem lehet eldönteni, hogy a modul jó-e. A „gyorsnak tűnik” nem elfogadási kritérium; a „medián idő a referencia medián idejének fele $n \geq 64$ esetén, és minden korrektségi teszt zöld” már az.

Ez az alfejezet a biztonsági interop-szerződés fogalmait ülteti át egy konkrét projekt tervébe. Az ott bevezetett négy dimenzió – értéktartomány, pointer-érvényesség, tulajdonjog és hibajelzési konvenció – itt kap konkrét értékeket. A NEON-mérési módszertan adja a statisztikai keretet, a fordítói optimalizálási tapasztalat pedig figyelmeztet arra, hogy a baseline nem lehet „szándékosan lassú C”: a referencia ugyanolyan fordítói kapcsolókkal kell fusson, mint amilyeneket egy valódi projekt használna.

A fejezet célja nem az, hogy egy konkrét modult implementáljunk – az a megvalósítás alfejezet feladata –, hanem hogy olyan specifikációs dokumentumot állítsunk össze, amelyből egy másik fejlesztő (vagy a kurzus auditálója) egyértelműen meg tudja ítélni a kész modult. A specifikáció önmagában is tesztelhető: ha a korrektségi terv nem fedi le a határeseteket, az a specifikáció hibája, nem az implementációé.

15.1.2 Definíció és scope: mi tartozik a projekt-specifikációba

Projekt-specifikáció (project specification): olyan rögzített dokumentum, amely egy szoftvermodul *feladatát*, *külső interfészét*, *korrektségi követelményeit*, *teljesítmény-követelményeit* és *biztonsági szerződését* írja le annyira pontosan, hogy a modul implementációjától függetlenül eldönthető legyen annak elfogadása vagy elutasítása.

A specifikációnak öt része van ebben a projektben:

1. **Feladat és indoklás.** Mit számol a modul, és miért assembly-optimalizálásra érdemes.
2. **C-interfész.** A függvényszignatúra, a visszatérési érték, a hibajelzési konvenció, az argumentumok értékhatárai.
3. **Korrektségi terv.** Referenciaimplementáció, összehasonlítási metrika, határesetek listája.
4. **Teljesítmény-mérési terv.** Bemenetméretek, ismétlésszám, statisztika, elfogadási küszöb.
5. **Biztonsági önellenőrzési terv.** Argumentum-ellenőrzés, túlcsordulás, memóriabiztonság, mentési szimmetria.

A scope-ból kizárt: a C alapszintaxis újratanítása, a konkrét assembly utasítássor megírása (a 15.2 alfejezet), a mérés tényleges végrehajtása (a 15.3 alfejezet), valamint az auditmódszer részletes kidolgozása (a 15.4 alfejezet). Ez az alfejezet a *terv*, nem a *végrehajtás*.

15.1.3 Feladatválasztás és indoklás

Három jelölt merül fel a könyv korábbi fejezetei alapján. Mindegyik szerepelt már valamilyen formában, és mindegyik rendelkezik az assembly-optimalizáláshoz szükséges tulajdonságokkal – de nem egyformán.

1. jelölt: CRC32 bitek fölött. A bitmanipulációs fejezet mintái közvetlenül alkalmazhatók: táblás megoldás, táblamentes (bitenkénti) megoldás, illetve az Armv8.1 CRC extension utasításai (Armv8.0-ban opcionális, Armv8.1-től kötelező). Erőssége, hogy az interfész triviális (`uint32_t crc32(const uint8_t *data, size_t len)`), és a korrekció bitre pontosan ellenőrizhető. Gyengesége, hogy a bázis A64 ISA-ban nincs dedikált CRC-utasítás (az csak az Armv8.1-től kötelező kiterjesztés része), ezért a kézi optimalizálás gyorsulása behatárolt; a táblás változat memóriafüggő, a bitenkénti változat pedig nem vektorizálható jól: a vektoros gyorsítás PMULL-alapú hajtogatást (folding) igényel, ami más algoritmus, így a gyorsulás mértéke erősen függ a bemenet hosszától.

2. jelölt: memcpy-szerű másolás. Az interfész szintén egyszerű (`void lab_memcpy(void *dst, const void *src, size_t n)`), és a NEON vektorizáció természetes. Problémája, hogy nagy n esetén a memóriasávszélesség a szűk keresztmetszet, nem a számítási kapacitás; az Amdahl-korlát itt nem a nem vektorizálható hányadból, hanem a memóriából ered. Kis és közepes méreteknél viszont látványos a gyorsulás, és a cache-viselkedés (cache-hierarchia) jól demonstrálható.

3. jelölt: vektorizált (i-k-j) mátrixszorzás. A feladat: $C = A \cdot B$, ahol A, B, C mind $n \times n$-es, sor-fő (row-major) elrendezésű `float` tömbök. A számítási igény $O(n^3)$, a memóriaigény $O(n^2)$ – vagyis nagy n-nél a számítás dominál, és a memóriasávszélesség nem feltétlenül szűk keresztmetszet. A NEON vektorizáció mindhárom ciklusban alkalmazható, a cache-tudatos i-k-j sorrend (cache-tudatos kód) pedig a cache-találati arányt javítja. A teljesítmény iparági standard metrikával, GFLOP/s-ban mérhető. A referenciaimplementáció (naiv három egymásba ágyazott ciklus) triviálisan megírható, és a lebegőpontos összehasonlítás módszertana is tanítható rajta.

A választás a **3. jelölt: vektorizált (i-k-j) mátrixszorzás**. Indoklás táblázatban:

Szempont	CRC32	memcpy	vektorizált (i-k-j) mátrixszorzás
Számítási dominancia	közepes	alacsony (memóriakötött)	magas ($O(n^3)$)
NEON-vektorizálhatóság	részleges	magas	magas (mindhárom ciklus)
Cache-tudatosság szerepe	alacsony	magas	magas (i-k-j sorrend)
Referenciaimplementáció egyszerűsége	magas	magas	magas
Iparági mérőszám	átviteli sebesség	sávszélesség	GFLOP/s
Gyorsulás a modell szerint	korlátozott	memóriakötött	Amdahl-modell $s = 4$ feltevéssel: legfeljebb 4× (modellérték, nem hardvergarancia)

Szempont	CRC32	memcpy	vektorizált (i-k-j) mátrixszorzás
Korrektség ellenőrzése	bitre pontos	bitre pontos	toleranciás

A mátrixszorzás választását az is indokolja, hogy a vektorizálható hányad fogalma itt kézzelfogható: a belső szorzat-összegzés szinte teljesen vektorizálható, a maradék (setup, ciklusvezérlés, vektorhatárok) pedig mérhetően elválik a vektorizált magtól.

15.1.4 A C-interfész rögzítése

Munkakönyvtár: `~/arm-labor/a64/zaro1/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez az I. záróprojekt közös könyvtára: mind a négy alfejezete (15.1–15.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/a64/zaro1
cd ~/arm-labor/a64/zaro1
```

Az interfész a C hívó és az assembly implementáció közötti interop-szerződés. A szignatúra:

```
/* lab_sgemm.h */
#ifndef LAB_SGEMM_H
#define LAB_SGEMM_H

#include <stddef.h>

#define LAB_SGEMM_OK      0
#define LAB_SGEMM_EINVAL -1

/*
 * C = A * B, mindharom matrix n x n-es, row-major, float32.
 * A, B, C teruletei nem fedhetik at egymast (restrict-szerzodes).
 *
 * Visszateres:
 *   LAB_SGEMM_OK    (0)  sikeres vegrehajtas
 *   LAB_SGEMM_EINVAL(-1) ervenytelen argumentum (NULL pointer vagy
 *                        n * n * sizeof(float) tulcsordulas)
 *
 * n == 0 eseten a fuggveny nem ir semmit, es LAB_SGEMM_OK-t ad.
 */
int lab_sgemm(size_t n, const float *restrict A,
              const float *restrict B, float *restrict C);

#endif
```

Az `int` visszatérési érték tudatos döntés. A biztonsági interop-szerződés szerint a hívó és a hívott közötti hibajelzés konvencióját rögzíteni kell. A `void` visszatérés ezt a csatornát megszüntetné, és a hibát a hívóra terhelné, aki viszont nem feltétlenül tudja ellenőrizni a pointerek érvényességét. Az `int` visszatérés egyetlen regiszterben (`W0`) elfér, az AAPCS64 szerint ez a standard visszatérési regiszter, és az assembly implementációban egyetlen feltételes ág elég hozzá.

Az argumentumok az AAPCS64 szerint az `X0`–`X3` regiszterekben érkeznek: `n` az `X0`-ban, `A` az `X1`-ben, `B` az `X2`-ben, `C` az `X3`-ban. A `restrict` minősítő nem változtat az ABI-n, de szerződéses ígéret: az `A`, `B`, `C` által mutatott területek nem fednek át. Ez az assembly implementációt mentesíti az alias-analízis alól, és a hívót terheli a felelősség a megsértéséért.

Az értéktartomány szerződése:

Argumentum	Érvényes tartomány	Viselkedés érvénytelen esetén
`n`	$0 \le n \le \lfloor\sqrt{SIZE_MAX/sizeof(float)}\rfloor$	`LAB_SGEMM_EINVAL`
`A`, `B`, `C`	nem NULL (ellenőrzött); n^2 elemre érvényes (nem ellenőrizhető, a hívó felelőssége)	NULL esetén `LAB_SGEMM_EINVAL`, egyébként meghatározatlan
`A`, `B`, `C` átfedés	nincs	meghatározatlan (szerződésszegés)

A `NULL` pointer és a túlcsordulás ellenőrzése $O(1)$ költségű, szemben a mátrixszorzás $O(n^3)$ költségével. Ezért a biztonsági ellenőrzések beépítése nem rontja érdemben a teljesítményt – ezt az alábbi biztonsági önellenőrzési terv számszerűsíti.

15.1.5 Korrektségi teszt-terv

A korrektségi terv célja: az assembly implementáció eredménye minden releváns bemeneten egy előre rögzített toleranción belül egyezzen a referenciaeredménnyel, és a visszatérési érték minden esetben a szerződésnek megfelelő legyen.

Referenciaimplementáció. A referencia nem a naiv `float`-os három ciklus, mert az maga is kerekítési hibát halmoz fel, és nem tekinthető „igazságnak”. A referencia `double` pontossággal számol, `float` bemenetből, és `double` kimenetet ad:

```c
/* lab_sgemm_ref.c */
#include "lab_sgemm.h"

void lab_sgemm_ref(size_t n, const float *A, const float *B, double *C)
{
    for (size_t i = 0; i < n; ++i) {
        for (size_t j = 0; j < n; ++j) {
            double acc = 0.0;
            for (size_t k = 0; k < n; ++k) {
                acc += (double)A[i * n + k] * (double)B[k * n + j];
            }
            C[i * n + j] = acc;
        }
    }
}
```

Numerikus szerződés. A projekt egyetlen numerikus szerződést használ; a tesztkészlet, a mérés előtti ellenőrzés és az audit is erre hivatkozik.

1. **Lebegőpontos környezet.** `FPCR`: `RMode = 00` (legközelebbi, páros felé kerekítés), `FZ = 0` (nincs flush-to-zero), `DN = 0` (nincs alapértelmezett NaN), és az `AH/FIZ/NEP` bitek is 0. Ez a Linux alapállapota; a teszthajtó induláskor kiolvassa az `FPCR`-t, és eltérés esetén nem fut le.
2. **Bemeneti tartomány.** A szerződés **véges, túlcsordulásmentes** bemenetekre szól: minden `A`- és `B`-elem véges, és minden kimeneti elemre $s_{ij} = \sum_k |a_{ik}|\,|b_{kj}| \le \mathrm{FLT_MAX}/2$. A felezés a kerekítési tartalék, indoklását lásd lent. Ezen kívül (NaN, Inf vagy túlcsorduló bemenet) a modul viselkedése nem része az elfogadásnak; a túlcsordulást a tesztkészlet külön, dokumentált elvárással vizsgálja.

3. **A modul számítási modellje.** Minden c_{ij} a `0`-ról indul, és $k = 0, 1, \ldots, n-1$ sorrendben `float` pontosságú **fúzionált** szorzás-összeadással frissül ($c \leftarrow \mathrm{fma}(a_{ik}, b_{kj}, c)$, minden lépés után egy kerekítés). A vektoros `FMLA` sávonként és a skalár farok `FMADD`-je ugyanezt a műveletet végzi, ezért a sáv és a farok között nincs numerikus különbség.
4. **Referencia.** A `lab_sgemm_ref` ugyanebben a k-sorrendben, `double` pontossággal összegez. Két `float` szorzata `double`-ban pontosan ábrázolható (24 + 24 bit 53 bit), ezért a referenciában a fordítói kontrakció (`-ffp-contract`) sem változtat az eredményen.
5. **Elfogadás.** Minden elemre: c_{ij} véges, és

$$|c_{ij}^{\mathrm{asm}} - c_{ij}^{\mathrm{ref}}| \le \mathrm{atol} + \mathrm{rtol} \cdot s_{ij}, \qquad \mathrm{rtol} = 10^{-4},\ \mathrm{atol} = 10^{-30}.$$

A nem véges kimeneti elem véges referencia mellett mindig FAIL, és az összehasonlítás `!(eltérés <= korlát)` alakú, így a NaN nem csúszhat át rajta.

Miért nem bitre pontos, és miért ez a korlát? A k-sorrend az i-k-j átírásban nem változik: egy adott c_{ij}-hez a modul és a referencia ugyanabban a sorrendben adja hozzá a tagokat. A különbség forrása a **pontosság**: a modul minden lépés után 24 bites mantisszára kerekít, a referencia 53 bitesre. A fúzionált művelet ezen belül csak annyit számít, hogy lépésenként egy kerekítés van, nem kettő. Az ilyen összegzés ismert hibakorlátja $|c - r| \le \gamma_n \sum_k |a_{ik} b_{kj}|$, ahol $\gamma_n \approx n \cdot 2^{-24}$; ez $n \le 256$ esetén legfeljebb $1{,}6 \cdot 10^{-5}$, tehát az $\mathrm{rtol} = 10^{-4}$ biztonsági tartalékkal fedi. A korlátot ezért s_{ij}-hez és nem $|c_{ij}^{\mathrm{ref}}|$-hez mérjük: kioltásnál (pozitív és negatív tagok összegénél) a referencia lehet közel nulla, miközben a kerekítési hiba a tagok nagyságával arányos. Az atol csak az alulcsordulási tartományt nyeli el; $n > 256$ esetén az rtol értékét újra kell számolni.

15.1.5.1 Függelék: a FLT_MAX/2 tartalék indoklása

A felezés indoklása a következő. A szerződés véges kimenetet vállal, de a `float` FMA-lánc **részösszegei** a kerekítés miatt nagyobbak lehetnek a pontos abszolút összegnél. Ellenpélda $n = 4$-re: az `A` első sora {`0x1.fffffap127`, `0x1.08p103`, `0x1.08p103`, `0x1.08p103`}, a többi sora nulla, `B` minden eleme 1. A pontos összeg $s_{00} =$ `0x1.fffffd18p127` $<$ FLT_MAX $=$ `0x1.fffffep127`. A k-sorrendű `float` FMA részösszegei mégis `0x1.fffffcp127`, `0x1.fffffep127`, majd $+\infty$: minden 2^{103} nagyságú tag egy teljes `ulp`-ot (2^{104}) ad hozzá, mert a pontos érték a két szomszédos `float` közötti felezőpont fölé esik. A tartalék levezetése. Legyen $u = 2^{-24}$ az egységkerekítés, $\eta = 2^{-149}$ a legkisebb pozitív szubnormális `float`, és x_k a k-adik FMA pontos, kerekítés előtti értéke. Az RNE-kerekítésre a normális tartományban a relatív korlát $|\mathrm{fl}(z)| \le (1+u)|z|$ érvényes, de `FZ = 0` mellett a szerződés a szubnormális eredményeket is megengedi, ahol a hiba abszolút is lehet, legfeljebb $\eta/2$. Ezért a mindkét tartományban érvényes modell:

$$|\mathrm{fl}(z)| \le (1+u)\,|z| + \eta/2.$$

Egy tisztán relatív korlát itt nem elég. Ellenpélda: az első szorzat $a =$ `0x1.000002p-126`, $b =$ `0x1p-24`, a második szorzat 0. Ekkor $s_{ij} =$ `0x1.000002p-150`, az első FMA a pontos értéket `0x1p-149`-re kerekíti, így a második lépés pontos értéke $x_2 =$ `0x1p-149`. Ez nagyobb a relatív korlát $(1+u)\,s_{ij} =$ `0x1.000003p-150` értékénél, de belefér az abszolút taggal bővített $(1+u)\,s_{ij} + \eta/2$ korlátba (natív C `fmaf`-fal ellenőrizve). A modellt lépésenként alkalmazva $|x_{k+1}| \le |\hat{c}_k| + |a_{i,k} b_{k,j}| \le (1+u)\,|x_k| + \eta/2 + |a_{i,k} b_{k,j}|$, amiből indukcióval

$$|x_k| \le (1+u)^{k-1} s_{ij} + \frac{\eta}{2} \sum_{t=0}^{k-2} (1+u)^t \le (1+\gamma_n)\left(s_{ij} + \frac{n\,\eta}{2}\right).$$

$s_{ij} \le$ FLT_MAX/2 és $\gamma_n \le 1$ (azaz $n \lesssim 2^{23}$) mellett ez FLT_MAX alatt marad: az abszolút tag $n\,\eta/2 \le 2^{23} \cdot 2^{-150}$ elhanyagolható a FLT_MAX/2 nagyságrendű tartalékhoz képest. Egyetlen lépés sem kerekíthet tehát $+\infty$-re. A tankönyvi $n \le 256$ körben $\gamma_n \le 1{,}6 \cdot 10^{-5}$, tehát a felezés bőkezű. Egy szorosabb határ FLT_MAX/$(1+\gamma_n) - n\,\eta/2$ lenne (a második tag gyakorlatilag nulla), de a kerek, könnyen ellenőrizhető FLT_MAX/2 a szerződés része; a `near-max` és az `fma-edge` teszteset besorolása ettől nem változik. A tagságot a tesztkészlet `lab_sgemm_in_contract` függvénye egy bizonyítottan **elégséges**, konzervatív szűrővel vizsgálja: a `double`-ben számított összeget a kerekítési hibájával felfelé becsli, így 1-et csak biztosan belső esetre ad, a határ közvetlen közelében pedig egy belső esetet is elutasíthat.

Határesetek. A teszterv minden alábbi bemenetre lefut, és a referenciaeredményhez hasonlít:

Eset	`n` érték	Cél
nulla méret	0	`LAB_SGEMM_OK`, nincs írás
egységelem	1	egyetlen szorzat
vektorhossz (4 float) alatt	2, 3	vektorizációs határ alatt
vektorhossz felett	5, 7	maradékkezelés
vektorhossz többszöröse	8, 16	teljes vektor
vektorhatár átlépése	17, 63, 65	részleges vektor
közepes	64, 128	tipikus mérési méret
nagy	256	stressz, cache-hatás
további farokesetek	4, 6, 12	4-gyel osztható és nem osztható kis méretek

A fenti méretek mindegyike determinisztikus, mintázott tartalommal fut.

Speciális mátrixtartalmak. A tényleges tesztmátrix (a tesztkészlet pontosan ezt hajtja végre):

Tartalom	`n` értékek	Cél
csupa nulla, csupa egyes, csupa mínusz egyes, identitás ($B = I$)	32, 64	strukturált hibák (sávkitöltés, előjel, indexelés)
véletlen, egyenletes $[-1, 1)$, rögzített maggal	32, 64, 128, 256	kioltás, általános eset, nagy méret
nagyérték-teszt ($\pm 10^6$)	32, 64	nagy, de a szerződésen belüli értékek: $64 \cdot 10^{12} \approx 6{,}4 \cdot 10^{13}$, messze a `float` határa ($\approx 3{,}4 \cdot 10^{38}$) alatt
határközeli (`near-max`): minden `A`-elem `0x1.fffffcp126`/n, `B` csupa 1	32, 64	szerződésen belül, egynemű előjellel: $s_{ij} =$ `0x1.fffffcp126` $\le$ FLT_MAX/2; a kimenetnek végesnek és tűrésen belülinek kell lennie

Tartalom	`n` értékek	Cél
túlcsordulás (10^{20}, szerződésen kívül)	8	a szerződésnek el kell utasítania; dokumentált elvárás a k-sorrendű `float` FMA-modell eredménye: minden szorzat $10^{40} >$ `FLT_MAX`, ezért minden kimeneti elem $+\infty$
kerekítési határeset (`half-edge`, szerződésen kívül): `A` első sora $\{\text{FLT_MAX}/2, 1\}$, `B` első oszlopa $\{1, 1\}$	2	a pontos $s_{00} = \text{FLT_MAX}/2 + 1$ a határ fölött van, `double`-ben mégis egyenlő vele; a szűrőnek a kerekítési hibát is figyelembe véve el kell utasítania
FMA-határ-ellenpélda (`fma-edge`, a fenti 4×4-es eset, szerződésen kívül)	4	a régi $s_{ij} < \text{FLT_MAX}$ feltétel átengedné, az új elutasítja; dokumentált elvárás az FMA-modell: a 0. sor $+\infty$, a többi 0

A nagyérték-teszt tehát **nem** túlcsordulás-teszt; a szerződés felső határát a `near-max` eset terheli, a valódi túlcsordulást pedig a két külön, szerződésen kívüli eset vizsgálja, saját elvárással. A tesztkészlet minden esetnél azt is ellenőrzi, hogy a `lab_sgemm_in_contract` a várt osztályba sorolja-e a bemenetet. NaN- és Inf-bemenetet a szerződés kizár. A nem véges **kimenetet** viszont az ellenőrző minden esetben FAIL-nek minősíti, és ezt egy önellenőrzés igazolja: a teszthajtó egy helyes eredménybe szándékosan NaN-t, $\pm\infty$-t és tűrésen kívüli eltérést ír, és mindegyiknek le kell buknia.

Memóriaszerződés. A kimeneten csak a `C` n^2 eleme változhat. A teszthajtó a bemeneti `A` és `B` másolatát a hívás után bájtra összeveti, a kimeneti puffer előtt őrbájtokat tart, utána pedig védőlap (`PROT_NONE`) áll. Ez a túlindexelt olvasást és írást is laphibává teszi. A C oldali AddressSanitizer erre nem elég, mert a kézzel írt assembly memóriaeléréseit nem instrumentálja.

15.1.6 Teljesítmény-mérési terv

A teljesítmény-mérés célja: eldönteni, hogy az assembly implementáció a rögzített elfogadási küszöböt teljesíti-e, és dokumentálni a mért értékeket. A mérés nem az assembly utasítások számát méri, hanem a falióra-időt egy rögzített munkamennyiségre.

Bemenetméretek. $n \in \{16, 32, 64, 128, 256\}$ (a mérőprogram alapértelmezése). Az alsó határ a vektorizációs overhead dominanciáját mutatja, a felső a cache-hatást. Kis n-nél az assembly setup-költsége látszik, nagy n-nél a memória és a cache.

Melegítés és ismétlésszám. Minden mérethez $W = 3$ melegítő futás (a cache és az elágazás-előrejelző felmelegítése), majd $R = 11$ mért futás. A páratlan ismétlésszám lehetővé teszi a medián egyszerű kiszámítását. A melegítő futások eredménye nem kerül a statisztikába.

Statisztika. Az elsődleges metrika a medián idő. Mellette a szóródás mérésére a medián abszolút eltérés (MAD, median absolute deviation) szolgál:

$$\text{MAD} = \text{median}\left(|t_i - \text{median}(t)|\right)$$

A mediánt azért használjuk, mert a falióra-idő eloszlása gyakran ferde (egy-egy lassabb futás az ütemező vagy a cache miatt), és a medián robusztusabb az átlagnál. Az átlag és a szórás is közölhető, de nem ez az elsődleges metrika.

Származtatott metrika. A mátrixszorzás műveletigénye $2n^3$ lebegőpontos művelet (szorzás és összeadás páronként), ezért a teljesítmény:

$$\text{GFLOP/s} = \frac{2n^3}{t \cdot 10^9}$$

ahol t a medián idő másodpercben. Ez a metrika függetleníti a mérést a bemenetmérettől, és lehetővé teszi a különböző n-ek összehasonlítását.

Összehasonlítás. A kézzel írt `.S` fájlt a `-O2` nem optimalizálja: az assembler azt fordítja le, amit leírtunk. A C-oldali kapcsolók tehát csak a C-változatokat befolyásolják, és a „ugyanazzal a szinttel fordítva” önmagában nem tesz igazságossá egy összehasonlítást. Az is fontos, hogy a `-march=armv8-a` már alapból tartalmazza az FP- és SIMD-támogatást, ezért a `+simd` utótag nem kapcsol be egy addig tiltott autovektorizálást. Az, hogy a fordító vektorizál-e, az optimalizálási kapcsolókon (`-ftree-vectorize`, `-O3`) és a ciklus alakján múlik.

Az assembly modul a naiv változathoz képest **három dolgot változtat egyszerre**: a ciklussorrendet (i-j-k helyett i-k-j, így `B` oszlopai helyett folytonos `B`- és `C`-sorokat jár be), a vektorizálást (4 sávos `FMLA`) és a fúzionált szorzás-összeadást. A teljes nyereség ezért nem nevezhető „assembly/NEON-gyorsulásnak”. A mérés négy változatot hasonlít össze, hogy a hatások szétváljanak:

Változat	Forrás	Kapcsolók (GCC)	Mit választ le
1. naiv C, i-j-k, skalár	`lab_sgemm_ijk_scalar`	`-O2 -fno-tree-vectorize`	kiindulópont
2. átrendezett C, i-k-j, skalár	`lab_sgemm_ikj_scalar`	`-O2 -fno-tree-vectorize`	a ciklussorrend hatása ($1 \to 2$)
3. átrendezett C, i-k-j, vektorizált	`lab_sgemm_ikj_vec`	`-O3 -fopt-info-vec-optimized`	a fordítói vektorizálás hatása ($2 \to 3$)
4. assembly	`lab_sgemm`	– (a `.S` fájlra nem hat)	a kézi kód többlete a vektorizált C-hez képest ($3 \to 4$)

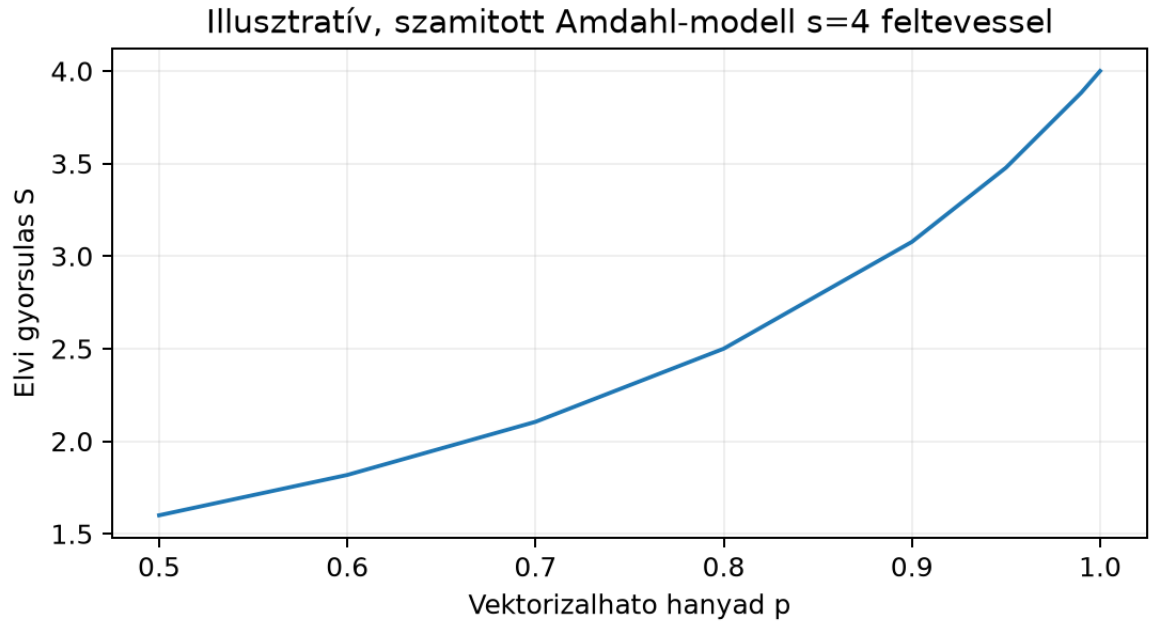
Minden C-változat azonos célra és FP-modellre fordul: `-std=gnu17 -march=armv8-a -ffp-contract=fast`, `-ffast-math` nélkül. A kontrakció engedélyezése azt jelenti, hogy a fordító is használhat `FMADD`/`FMLA`-t, mint a modul. A `-ffast-math` hiánya miatt viszont a fordító nem rendezheti át az összeadásokat, ezért az i-j-k redukciós ciklust nem tudja vektorizálni, az i-k-j belső ciklusát igen. A jegyzőkönyvbe kerül a GCC pontos verziója (`aarch64-linux-gnu-gcc --version`) és a vektorizációs riport. A végső binárison `objdump -d`-vel ellenőrizzük, hogy az 1–2. változatban nincs vektoros `fmla`, a 3.-ban van.

Elfogadási küszöb. Az assembly implementáció elfogadott, ha

1. minden korrektségi teszt zöld;
2. a **jegyzőkönyvben rögzített környezetben** (host vagy QEMU-verzió és CPU-modell, fordítóverzió, kapcsolók) $n \geq 64$ esetén az assembly medián ideje legfeljebb a naiv i-j-k skalár változat medián idejének a fele;
3. a jegyzőkönyv tartalmazza mind a négy változat mediánját és MAD-ját, a három részgyorsulást (sorrend, C-vektorizálás, kézi assembly) és az n, W, R, atol, rtol értékeket.

A 2. pont kétszeres küszöbe **natív AArch64 hardverre** vonatkozik. QEMU user-mode alatt a TCG nem modellezi a cache-t, a pipeline-t és a prefetchert, ezért ott az elfogadás feltétele a korrektség és a négy változat dokumentált mérése; a natív 2× küszöb QEMU-eredményre nem alkalmazható. Natív hardveren a küszöböt újra kell mérni. Nem állítjuk, hogy minden Arm-processzoron vagy QEMU-verzión teljesül, sem azt, hogy ott megengedő volna: egy másik mag vagy emulátor más arányt adhat, ezért minden új környezetben újra kell mérni. A küszöb teljesülése önmagában azt sem igazolja, hogy a kézi assembly megérte. Erről a $3 \to 4$ részgyorsulás dönt. Ha a modul nem gyorsabb a vektorizált C-nél, az is érvényes, dokumentálandó eredmény. Abszolút GFLOP/s-csúcs előírása hardver- és QEMU-függő lenne, és sértené a „ne találj ki benchmarkot” elvet. A NEON-gyorsulás mérése szerint QEMU-emulációban a mért érték emulációs idő, nem a célprocesszor ciklusszáma, és ezt a jegyzőkönyvben jelölni kell.

Illusztratív modell, nem garancia. Ha a skaláris referencia futásidejének p hányadát s-szeresére gyorsítjuk, és a többi rész ideje változatlan, akkor $S = 1/((1-p) + p/s)$ (Amdahl-modell). A négysávos NEON egyszerre négy `float`-elemet dolgozhat fel, de ebből önmagában nem következik, hogy $s = 4$, sem az, hogy 4× általános plafon volna: a memóriaelérés, a fúzionált műveletek, a kibontás és a skalár farok mind eltolhatja a tényleges s-t. A p értékét sem a forráskódból olvassuk ki, hanem a skaláris referencia **időhányadaként mérjük** (profilozással vagy a vektorizálható szakasz külön időmérésével). Az alábbi ábra egy $s = 4$ feltevésű modell számított görbéje p függvényében, nem a processzor garantált teljesítménye. Például $p = 0{,}95$ és $s = 4$ mellett $S \approx 3{,}48$.



15.1. ábra: Illusztratív, szamitott Amdahl-modell s=4 feltevessel - Szamitott ertek az S=1/((1-p)+p/s) kepletbol s=4 feltevessel; nem meresi adat es nem hardvergarancia, csak tervezesi modell.

A mérési terv ezzel szemben a tényleges medián időt rögzíti; a modell és a mérés viszonya az 15.3 validációs lépés tárgya.

15.1.7 Biztonsági önellenőrzési terv

A biztonsági terv a biztonsági interop-szerződés közvetlen alkalmazása. Négy ellenőrzési pont:

1. **Argumentum-ellenőrzés.** A függvény belépéskor ellenőrzi, hogy `A`, `B`, `C` nem `NULL`, és hogy `n` nem nulla. Ha `n == 0`, a függvény azonnal visszatér `LAB_SGEMM_OK`-val, anélkül hogy bármit írna a `C` pointerre. Az argumentumok érvényességét és a méret aritmetikai érvényességét (túlcsordulás) az első írás előtt ellenőrizzük.
2. **Túlcsordulás-ellenőrzés.** Az n^2 elem mérete `size_t`-ben túlcsordulhat. Az ellenőrzés: ha $n > \lfloor\sqrt{SIZE_MAX / sizeof(float)}\rfloor$, akkor `LAB_SGEMM_EINVAL`. Ez megakadályozza, hogy egy hibás `n` érték miatt a függvény a memórián kívülre írjon.
3. **Mentési szimmetria.** Az assembly implementációban minden `STP` (store pair) utasításhoz tartozó `LDP` (load pair) a visszatérési úton, minden kimeneti ágon (sikeres és hibás visszatérésnél is) végrehajtódik. Ez a mentési szimmetria követelménye; megsértése a hívó regisztereinek csendes módosulását okozza.
4. **Memóriaátfedés.** A `restrict` szerződés szerint az `A`, `B`, `C` területek nem fednek át. Ezt a függvény nem ellenőrzi (nem is tudná általánosan eldönteni), de a szerződés a hívó felelőssége. A betartását a hívói oldali kódreview-ban / statikus elemzéssel kell ellenőrizni.

A biztonsági ellenőrzések költsége $O(1)$, a műveleté $O(n^3)$, ezért $n \geq 16$ esetén az ellenőrzés futásideje elhanyagolható. Ez indokolja, hogy az ellenőrzéseket nem érdemes feltételes fordítási kapcsoló mögé rejteni – a záróprojektben mindig aktívak.

15.1.8 Kidolgozott példa: a specifikációs dokumentum minta

Az alábbi példa egy teljes specifikációs vázlat, amely a fenti döntéseket egy dokumentumba rendezi. A kódblokkok szemléltető példák, nem végrehajtandó utasítások.

```
/* lab_sgemm_spec.c - a mérési terv vaza (szemlelteto) */
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
#include "lab_sgemm.h"

#define LAB_W 3
#define LAB_R 11

static double now_seconds(void)
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts) != 0) { perror("clock_gettime"); exit(2); }
    return (double)ts.tv_sec + (double)ts.tv_nsec * 1e-9;
}

static double median(double *v, size_t n)
{
    /* n paratlan; egyszeru buborek-rendezes a vazlatban */
    for (size_t i = 0; i < n; ++i)
        for (size_t j = i + 1; j < n; ++j)
            if (v[j] < v[i]) { double t = v[i]; v[i] = v[j]; v[j] = t; }
    return v[n / 2];
}

int main(void)
{
    const size_t sizes[] = { 16, 32, 64, 128, 256 };

    for (size_t s = 0; s < sizeof(sizes) / sizeof(sizes[0]); ++s) {
        size_t n = sizes[s];
        size_t nelem = n * n;
        float *A = malloc(nelem * sizeof(float));
```

```
        float *B = malloc(nelem * sizeof(float));
        float *C = malloc(nelem * sizeof(float));
        if (!A || !B || !C) { free(A); free(B); free(C); return 1; }

        /* bemenet feltoltese: illusztrativ, determinisztikus */
        for (size_t i = 0; i < nelem; ++i) {
            A[i] = (float)((int)(i % 17) - 8) / 8.0f;
            B[i] = (float)((int)(i % 13) - 6) / 6.0f;
        }

        /* melegites */
        for (size_t w = 0; w < LAB_W; ++w)
            if (lab_sgemm(n, A, B, C)) { fputs("kernel error\n", stderr); return 1; }

        /* meres */
        double t[LAB_R];
        for (size_t r = 0; r < LAB_R; ++r) {
            double t0 = now_seconds();
            if (lab_sgemm(n, A, B, C)) { fputs("kernel error\n", stderr); return 1; }
            t[r] = now_seconds() - t0;
        }
        double med = median(t, LAB_R);
        if (med <= 0) { fputs("timer resolution\n", stderr); return 2; }
        double gflops = (2.0 * (double)n * (double)n * (double)n)
                        / (med * 1e9);
        printf("n=%zu median=%.6f s GFLOP/s=%.3f\n", n, med, gflops);

        free(A); free(B); free(C);
    }
    return 0;
}
```

A példa három dolgot mutat: a melegítés és a mért ismétlések szétválasztását, a medián mint elsődleges statisztikát, és a GFLOP/s származtatott metrikát. A tényleges korrektségi összehasonlítás (`lab_sgemm` kontra `lab_sgemm_ref`) ettől külön függvényben történik, a toleranciás összehasonlítással.

Előretekintés: a projekt fájllistája. Ebben az alfejezetben még nincs futtatható build. A specifikáció a szerződést rögzíti, a forrásokat a következő alfejezetek készítik el. Az alábbi táblázat a projekt **egyetlen hiteles fájllistája**: minden fájlnév, a hiteles változatát közlő alfejezet és az, hogy melyik buildbe kerül. A későbbi alfejezetek buildparancsai pontosan ezeket a fájlokat linkelik, a `~/arm-labor/a64/zaro1/` közös könyvtárban.

Fájl	Tartalom	Hiteles változat	Build
`lab_sgemm.h`	a `lab_sgemm` C-interfésze, `LAB_SGEMM_OK`/`LAB_SGEMM_EINVAL`	15.2 (az itt közölt változattal bitenkénti azonos)	mindegyik
`lab_sgemm.S`	az assembly modul	15.2	mindegyik
`lab_sgemm_main.c`, `Makefile`	egyszerű hívó program	15.2	`lab_sgemm`
`lab_sgemm_abi.c` (+ `lab-abi-call.h`, `lab-abi-call.S` az 8.3-ből)	ABI-ellenőrzés a közös teszthívóval	15.2	`lab_sgemm_abi`
`lab_sgemm_ref.c`	`double` referencia	15.3 (az itt közölt változattal kódra azonos)	`lab_sgemm_test`, `lab_sgemm_bench`

Fájl	Tartalom	Hiteles változat	Build
`lab_sgemm_check.h`, `lab_sgemm_check.c`	a numerikus szerződés ellenőrzője (`lab_fpcr_ok`, `lab_sgemm_in_contract`, `lab_sgemm_check`)	15.3	`lab_sgemm_test`, `lab_sgemm_bench`
`lab_sgemm_test.c`	korrektségi teszthajtó	15.3	`lab_sgemm_test`
`lab_sgemm_c.c`	C összehasonlító változatok	15.3	`lab_sgemm_bench`
`lab_sgemm_bench.c`	mérőprogram	15.3	`lab_sgemm_bench`
`lab_sgemm_spec.c`	a fenti mérési vázlat	ez az alfejezet; csak szemléltetés	nincs

A korrektségi teszt tényleges buildje és futtatása tehát A korrektségi tesztkészlet kidolgozott példájában van. Ott a teszthajtó mellé az ellenőrző modul (`lab_sgemm_check.c`) is linkelődik, és a hívott `lab_sgemm` az assembly modul. A projektben nincs C nyelvű `lab_sgemm` implementáció. A C összehasonlító változatok a mérési baseline szerepét töltik be, saját, eltérő nevekkel, és csak a mérőprogramba kerülnek. A teszthajtó ezért hoston nem fordítható natív `gcc`-vel, mert a hívott függvény AArch64 assembly. A futtatás `qemu-aarch64` alatt vagy natív AArch64-en történik.

15.1.9 Tipikus hibák és ellenpróbák

Tünet: a korrektségi teszt $n = 17$ esetén elbukik, de $n = 16$-nál zöld. **Hipotézis:** a vektorhossz-határ (4 float) kezelése hibás; a maradék vektor túlcsordul a következő sorra. **Adatgyűjtés:** a teszt kiírja az első eltérő (i, j) indexet; ez megmutatja, melyik vektorhatáron csúszik el. **Egy változtatás:** a maradékkezelő ág feltételének módosítása. **Ellenőrzés:** a teszt újrafuttatása mind a kilenc határesetre.

Tünet: a mért gyorsulás $n = 16$-nál 1 alatti (az assembly lassabb a C-nél). **Hipotézis:** a setup-költség (regisztermentés, ciklusbejáratás) nagyobb, mint a nyereség. **Adatgyűjtés:** az assembly disassemblyjében a függvény prologusának hossza; a `perf`-szerű ciklusszámlálás hiányában a medián idők összehasonlítása. **Egy változtatás:** a belső ciklus kibontásának (unrolling) csökkentése vagy a setup egyszerűsítése. **Ellenőrzés:** $n = 16$ és $n = 64$ újramérése.

Tünet: a mérés szórása nagy (MAD > a medián 10%-a). **Hipotézis:** a host más folyamatai zavarnak, vagy a QEMU nem determinisztikus. **Adatgyűjtés:** a 11 mért idő kiírása; ha egy kiugró érték van, az külső zavar. **Egy változtatás:** az ismétlésszám növelése vagy a mérés más időpontban ismétlése. **Ellenőrzés:** a MAD újraszámítása.

Tünet: a korrektségi teszt minden n-re „zöld”, de a modul a valós bemeneten hibás. **Hipotézis:** a teszt bemenete nem fedi le a valós értéktartományt (pl. csak kis értékek). **Adatgyűjtés:** a teszt bemeneteinek statisztikája. **Egy változtatás:** nagy értékek és előjeles értékek bevonása. **Ellenőrzés:** a toleranciás összehasonlítás újrafuttatása.

15.1.10 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Hasonlítsd össze a három jelölt feladatot (CRC32, memcpy, vektorizált (i-k-j) mátrixszorzás) az alábbi szempontok szerint: számítási dominancia, vektorizálhatóság, cache-érzékenység, referenciaimplementáció egyszerűsége.

Melyik esetben lenne az Amdahl-korlát a legszűkebb, és miért? Készíts táblázatot, és indokold, melyik feladatot választanád záróprojektnek, ha a cél a legnagyobb mért gyorsulás elérése.

2. (implementációs, középszint — cél: rutinépítés) Írd meg a `lab_sgemm_ref` függvényt `double` pontossággal, valamint a korrektségi teszt keretrendszer vázát, amely a határesettáblázatot és a speciális tartalmak tesztmátrixát lefedi. A keretrendszer a numerikus szerződés szerinti összehasonlítást (`atol + rtol * s_ij`, nem véges elem = FAIL) alkalmazza, és minden eltérésnél kiírja az (i, j) indexet és a három értéket (asm, ref, különbség).

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Tervezd meg a mérési jegyzőkönyv sablonját: milyen mezőket tartalmaz (host, QEMU-verzió, fordítói verzió, optimalizálási szint, n, W, R, atol, rtol, mért medián, MAD, GFLOP/s, jelölés hogy emulációs idő). Egészítsd ki egy olyan eljárással, amely a mérési zajt csökkenti (melegítés, medián, ismétlés), és indokold, miért nem használnál `clock()`-ot `CLOCK_MONOTONIC` helyett.

15.1.11 Megoldások

1. megoldás — Segítség. Melyik feladatnál dominál a memóriasávszélesség a számítás helyett, és ez hogyan hat az Amdahl-modell érvényességére (amely eredetileg számítási párhuzamosításra vonatkozik)?

1. megoldás — Teljes megoldás. A CRC32 esetén a vektorizálható hányad alacsony (a bitenkénti feldolgozás nehezen párhuzamosítható, a táblás változat memóriafüggő), ezért az Amdahl-korlát szűk. A memcpy esetén a memóriasávszélesség a szűk keresztmetszet, nem a nem vektorizálható hányad; az Amdahl-korlát formálisan kevésbé releváns, mert a modell nem a számítási kapacitást írja le. A mátrixszorzásnál a belső ciklus szinte teljesen vektorizálható, a memóriaigény $O(n^2)$, a számítási igény $O(n^3)$, ezért az Amdahl-korlát a legkevésbé szűk. Egy $s = 4$ feltevésű modell szerint a gyorsulás legfeljebb $4\times$ lehet; ez a feltevés következménye, nem a hardver garanciája, és a tényleges s-t mérni kell. A választás indoklása: legnagyobb mért gyorsulás és legjobb mérési standard (GFLOP/s) a mátrixszorzásnál várható.

2. megoldás — Segítség. A referencia `double` akkumulátorral dolgozik, a bemenet `float` marad — nézd meg a fejezet korábbi `lab_sgemm_ref`-re vonatkozó definícióját, és a kilenc határeset + hét tartalomtípus listáját a specifikáció szerint.

2. megoldás — Teljes megoldás. A `lab_sgemm_ref` a fenti `double`-os verzió. A teszt keretrendszer vázlata: minden `n`-re és minden tartalomra generálja a bemenetet, lefuttatja a referenciát és az assemblyt, majd minden elemre ellenőrzi a toleranciát. A határesetek listája a táblázat `n` értékei; a speciális tartalmak a tesztmátrix szerinti `n`-eken futnak. Az eltérés kiírása segít lokalizálni a hibát: ha az első eltérés a $(0, 0)$ indexnél van, a hiba a teljes mátrixra hat; ha egy vektorhatárnál, a maradékkezelés a hibás. Az összehasonlítás `!(eltérés <= korlát)` alakú, és a nem véges elemet külön is FAIL-nek minősíti, így a NaN nem adhat „0 maximális hibát”. Hogy ez valóban működik, azt a szándékosan beírt NaN/Inf hibák lebukása igazolja. A szerződésen kívüli túlcsordulási eset saját, dokumentált elvárást kap (minden elem $+\infty$).

3. megoldás — Segítség. Gondold át, mit mér a `clock()` valójában (CPU-idő vs. falióra-idő), és miért probléma ez egy olyan mérésnél, ahol más folyamatok vagy I/O is futhatnak a háttérben.

3. megoldás — Teljes megoldás. A jegyzőkönyv sablonja táblázatos formában rögzíti a környezetet és a paramétereket. A `clock()` a CPU-időt méri, amely több szálon vagy blokkoló I/O esetén félrevezető; a `CLOCK_MONOTONIC` a falióra-időt méri, amely a valós felhasználói élményt tükrözi. A melegítés a cache és az elágazás-előrejelző feltöltését szolgálja; a medián a kiugró

értékeket kiszűri; az ismétlés a véletlen zaj hatását csökkenti. A jegyzőkönyvben jelölni kell, hogy QEMU-emulációban a mérés emulációs idő, nem target-ciklusszám.

15.1.12 Rövid összefoglaló

A projekt-specifikáció öt részből áll: feladat és indoklás, C-interfész, korrektségi terv, teljesítménymérési terv és biztonsági önellenőrzési terv. A választott feladat a vektorizált (i-k-j) mátrixszorzás, mert számításigényes, jól vektorizálható, cache-tudatos és iparági standard metrikával mérhető. Az interfész `int lab_sgemm(size_t n, const float *restrict A, const float *restrict B, float *restrict C)`, hibajelzése `LAB_SGEMM_OK`/`LAB_SGEMM_EINVAL`. A korrektségi terv egyetlen numerikus szerződést használ (rögzített `FPCR`, véges bemenet, `double` referencia, s_{ij}-hez mért tűrés); a mérési terv négy változatot (naiv, átrendezett, vektorizált C és assembly) hasonlít össze mediánnal, MAD-dal és GFLOP/s-sal; a biztonsági terv argumentum-ellenőrzést, túlcsordulás-védelmet és mentési szimmetriát ír elő.

Továbbhasznált definíciók:

- **projekt-specifikáció:** a feladat, interfész, korrektségi, teljesítmény- és biztonsági követelmények rögzített dokumentuma.
- **elfogadási küszöb:** a mérési tervben rögzített, számszerű feltétel, amelynek teljesülése esetén a modul elfogadható.
- **numerikus szerződés:** rögzített `FPCR`, véges bemeneti tartomány ($s_{ij} \leq$ FLT_MAX/2, kerekítési tartalékkal), a modul és a referencia számítási modellje, valamint az `atol + rtol * s_ij` elfogadási feltétel, ahol `s_ij = sum_k |a_ik||b_kj|`.

15.1.13 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile Architecture Reference Manual — az AAPCS64 argumentum-átadás és a visszatérési regiszter elsődleges forrása. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- AAPCS64 – Procedure Call Standard for the Arm 64-bit Architecture — az `X0`–`X3` argumentumregiszterek és a `W0` visszatérési regiszter szerződése. **Tartalmilag ellenőrizve**: aktuális kiadás 2025Q4 (2026-01-23), Release minőségű.
- GCC: Options That Control Optimization — az optimalizálási szintek és a baseline összehasonlítás követelménye. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2.
- GNU assembler kézikönyv — az assembly forrás és a linker szintaxis. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- QEMU User Mode Emulation — a `qemu-aarch64` futtatási modell és a mérés emulációs jellege. **Tartalmilag ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát); a hivatalos dokumentáció nem állít semmit a cache- vagy sávszélesség-modellezésről, ami a TCG funkcionális, nem cikluspontos tervezéséből következik.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a `CLOCK_MONOTONIC` felbontása és a QEMU emulációs idő viselkedése az adott hoston és QEMU-verzióban nem ellenőrzött itt; a konkrét `atol`/`rtol` értékek az első mérési ciklusban validálandók. A specifikáció ebben az alfejezetben nem tartalmaz mérést: a négy mérési változat és a tesztkészlet eredménye a 15.3 alfejezetben, a fő úton mérve (GNU + glibc + QEMU 8.2.2) szerepel. A GCC

vektorizációs riportját és a célhardveres időket a labor mérési ciklusa rögzíti; ezekre itt nem teszünk állítást.

15.2 Megvalósítás: az assembly modul implementációja

15.2.1 Motiváció

Az előző alfejezetben rögzítettük a `lab_sgemm` projekt specifikációját: mit kell a modulnak tudnia, milyen szerződés szerint illeszkedik a C hívó programhoz, hogyan mérjük és auditáljuk. Most eljön a pillanat, amikor a papíron rögzített követelményeket **A64 assembly utasításokká** kell alakítani. Ez az a munkafázis, ahol a könyv addigi technikái egyetlen, összefüggő modulban találkoznak: regiszterkiosztás (Az A64 regiszterkészlet áttekintése), címzési módok (Load/store címzési módok), eltolás (Shift és rotate: LSL/LSR/ASR/ROR gyakorlati mintázatok), AAPCS64 (Az AAPCS64 regiszterszerződés), keretkezelés (Prológus és epilógus: regisztermentés és keret-kezelés), NEON (A NEON regiszterkészlet és adatelrendezés) és C-együttműködés (Assembly függvény hívása C-ből és C hívása assemblyből).

A gyakorlati kihívás háromrétegű. Egyrészt **helyesnek** kell lennie a bit szintjén: a floating-point szorzás-összegzés sorrendje, a mutatóaritmetika bájt- és elemszám-konverziója, a határellenőrzés nem téveszthet. Másrészt **illeszkednie** kell az AAPCS64-hez, különben a C hívó kódja néma, nehezen lokalizálható hibát szenved. Harmadrészt **elég gyorsnak** kell lennie ahhoz, hogy a mérési terv elfogadási küszöbét teljesítse. Mindezt úgy, hogy a kód auditálható legyen: minden regiszterhez tartozzon indoklás, minden határellenőrzés mögött legyen levezetett feltétel.

15.2.2 Definíció és szerződés

Ebben az alfejezetben három fogalmat használunk pontosan.

assembly modul (assembly module). Egy `.S` fordítási egység, amely egy vagy több `.global` szimbólumot exportál, és amelyet a linker a C fordító objektumfájljával illeszt össze egyetlen futtatható ELF-be. Az assembly modul **nem** külön program; nincs saját `_start` belépési pontja, nincs saját `main`-je. Ő egy szolgáltató, akit a C oldal függvényhívás formájában ér el.

modul-szerződés (module contract). Az a pontos, regiszterszintű megállapodás, amelyet a modul a hívó felé garantál. A modul-szerződést öt dimenzióban rögzítjük; az első négy az AAPCS64-ből következik, az ötödik a projekt saját szerződése:

1. **argumentum-átvétel** — az első nyolc egész/pointer argumentum rendre `X0`–`X7`-ben, a lebegőpontosak `V0`–`V7`-ben;
2. **visszatérési érték** — egész `W0`-ban (32 bites) vagy `X0`-ban (64 bites), lebegőpontos `S0`/`D0`-ban;
3. **callee-saved regiszterek megőrzése** — ha a modul használ `X19`–`X28`-at, `X29`-et vagy `X30`-at, azok eredeti értékét vissza kell állítania `RET` előtt; ugyanez igaz `V8`–`V15` **alsó 64 bitjére** (azaz `D8`–`D15`-re);
4. **verem-igazítás** — a publikus interfész belépésekor és elhagyásakor `SP` 16 bájt többszöröse;
5. **mellékhatás** — csak azok a memóriaterületek módosulhatnak, amelyeket a szerződés megenged (jelen esetben a `C` tömb elemei).

határellenőrzés (boundary check). Az a kód, amely a mag (kernel) futása előtt eldönti, hogy a bemenet egyáltalán értelmezhető-e: null mutatók, nulla méret, méret-túlcsordulás. A

határellenőrzés nem „védekezés a hívó ellen” — szerződésszintű precondíció-ellenőrzés, amelynek hiányában a `C` tömb írása meg nem engedett memóriát érintene.

A specifikáció (Projektspecifikáció: feladat, interfész és mérési terv) szerinti interfész:

```
int lab_sgemm(size_t n,
              const float *restrict A,
              const float *restrict B,
              float *restrict C);
```

`size_t` az AArch64 LP64 ABI-n 64 bites, ezért az `n` `X0`-ban érkezik. Az `A`, `B`, `C` mutatók rendre `X1`, `X2`, `X3`-ban. A visszatérési érték `W0` (32 bites `int`), értéke `LAB_SGEMM_OK` (0) vagy `LAB_SGEMM_EINVAL` (`-1`, az 15.1 specifikáció szerint). A `restrict` kulcsszó C99 óta része a nyelvnek; **fordítói** szinten jelzi, hogy a három tömb nem fed át, ami a vektorizálhatóságot erősíti, de az assembly oldalon nem keletkezik belőle kód — a szerződést mi tartjuk be.

15.2.3 AAPCS64-illesztés a modulban

Az AAPCS64 regiszterszerződés fejből tudása kulcs a helyes modulhoz. Foglaljuk táblázatba, mit használunk és miért.

regiszter	tartomány	AAPCS64 szerep	a modulban
`X0`–`X3`	0–3	argumentum, caller-saved	bemenet (`n`, `A`, `B`, `C`), prologusban azonnal továbbmásoljuk
`X4`–`X17`	4–17	caller-saved temp	lokális ciklusváltozók, indexek (a modul `X2`–`X6`-ot használja)
`X18`	18	**platformregiszter**: az AAPCS64 a platformra bízza. Linuxon a felhasználói kód ideiglenes regiszterként használhatja, más platformokon (például Windows, macOS, vagy Androidon a shadow call stack) foglalt	**nem használjuk**, így a modul platformtól függetlenül helyes
`X19`–`X28`	19–28	**callee-saved**	a hosszú életű állapot (`n`, `A`, `B`, `C`, ciklusváltozók)
`X29`	29	frame pointer	**nem használjuk** (nincs szükség visszafejtésre)
`X30`	30	link register	nem módosítjuk (nincs `BL` a modulban)

regiszter	tartomány	AAPCS64 szerep	a modulban
V0–V7	0–7	caller-saved	V0: skalár a broadcast; V1/V2: C/B vektorok; V5: nulla-konstans a C_row nullázásához (külön regiszter, nem V0)
V8–V15	8–15	**callee-saved (alsó 64 bit)**	nem használjuk → nincs mentési teher
V16–V31	16–31	caller-saved	nem használjuk

Két tervezési döntés következik. Egyrészt az **összes** hosszú életű értéket X19–X28-ba tesszük, hogy a BL-hívásokon (bár a modulban nincs ilyen) és a NEON utasításokon túl is túléljék a ciklusokat; cserébe a prologusnak öt STP-t és az epilógusnak öt LDP-t kell tartalmaznia. Másrészt szándékosan **nem** használunk V8–V15-öt: ezek alsó 64 bitje (D8–D15) callee-saved, így használatukhoz további mentés és visszaállítás kellene. Egy S8-ba írás például a V8 felső bitjeit is nullázza, tehát a teljes D8 megváltozik. A V0–V7 közül választott regiszterek (V0, V1, V2, V5) caller-saved-ek, ezért ingyenesek. Levélfüggvényben a caller-saved X9–X15 is elég lenne; azért használunk callee-saved regisztereket, hogy a mentési szimmetria auditja (15.4) bemutatható legyen.

A prologus–epilógus pár:

```
stp     x19, x20, [sp, #-80]!
stp     x21, x22, [sp, #16]
stp     x23, x24, [sp, #32]
stp     x25, x26, [sp, #48]
stp     x27, x28, [sp, #64]
```

Az első STP pre-index címzésmóddal dolgozik: SP -= 80, és az X19, X20 párt az **új** SP-re írja. A többi STP a frissen kialakított frame belső offsetjeire ment. A 80 bájt 16 bájt többszöröse, ezért a publikus interfész verem-igazítása megmarad. Az epilógus ugyanazokról a helyekről tölt vissza; az utolsó pár a +64 eltoláson van. Csak ezután szabad a 80 bájtos keretet felszabadítani:

```
ldp     x19, x20, [sp]
ldp     x21, x22, [sp, #16]
ldp     x23, x24, [sp, #32]
ldp     x25, x26, [sp, #48]
ldp     x27, x28, [sp, #64]
add     sp, sp, #80
ret
```

X29-et (frame pointer) szándékosan nem állítjuk: az egyszerű, nem rekurzív moduloknál a frame pointer csak felesleges utasításokat és mentést hozna. A RET az X30-ban lévő visszatérési címet használja, amelyet a hívó BL-je automatikusan beírt; mivel a modulban nincs további BL, az X30 érintetlen marad, és RET a C hívóba tér vissza.

15.2.4 Bemenet-validálás és határellenőrzés

A határellenőrzést a prologus után, de a fő hurkok előtt végezzük, egyszer. Sorrend:

1. n == 0 → üres szorzat, nincs teendő, LAB_SGEMM_OK. Ez azért nem EINVAL, mert a nulla méret C-szemantikailag jól definiált; a null mutatókat viszont ebben az esetben nem büntetjük.

2. **valamelyik mutató null** → `LAB_SGEMM_EINVAL`. A `CBZ` (Compare and Branch on Zero) A64 utasítás 32/64 bites regiszterre: `cbz x20, .Linvalid` egyszerre hasonlít és elágaztat.
3. **túlcsordulás** — nem az `n*n` elemszámnak, hanem a tényleges, címzéshez használt **bájtméretnek** (`n*n*4`) kell 64 biten elférnie. A `umulh x5, x19, x19` az `n*n` **magas 64 bitjét** adja vissza (a szorzat formálisan 128 bites); ha ez nem nulla, `n*n` maga sem fér el 64 biten, és ez már elegendő a `LAB_SGEMM_EINVAL`-hoz. De ha `x5 == 0`, ez **önmagában nem garantálja**, hogy `n*n*4` is elfér: `n*n` lehet 64 biten belüli, mégis `2^62`-nél nagyobb érték, és az ilyen `n*n` már `*4`-gyel túlcsordulna. Ezért külön is ellenőrizni kell az alsó 64 bites szorzat (`mul x6, x19, x19`) felső két bitjét: ha ezek közül bármelyik 1, `n*n >= 2^62`, azaz `n*n*4 >= 2^64` — túlcsordulás.

Ez a `umulh`-alapú ellenőrzés az Adatmozgatás, aritmetika és állapotjelzők óta a bitszintű gondolkodás egyenes alkalmazása: nem elég a `MUL` alsó 64 bitjét nézni, mert ott az `n*n`-re vonatkozó túlcsordulás elveszne — de a `*4` szorzóra vonatkozó, *másik* túlcsordulás éppen ennek az alsó 64 bitnek a felső két bitjéből olvasható ki.

```
    cbz     x19, .Lempty_ok
    cbz     x20, .Linvalid
    cbz     x21, .Linvalid
    cbz     x22, .Linvalid
    umulh   x5, x19, x19          // n*n magas 64 bitje
    cbnz    x5, .Linvalid         // n*n > 2^64-1 -> tulcsordulas
    // n*n alacsony 64 bitje (itt mar tudjuk, hogy ez a teljes ertek)
    mul     x6, x19, x19
    lsr     x6, x6, #62           // a felso 2 bitet hozza le a 0/1. bitre
    cbnz    x6, .Linvalid         // n*n >= 2^62 -> n*n*4 tulcsordulna
```

A `CBZ`/`CBNZ` **közvetlen elágazás** ±1 MiB hatótávolságon belülre mutat, fix immediate-kódolással; a modul méreténél ez bőven elég. A `#LAB_SGEMM_OK` az `.equ` által definiált szimbolikus konstans, amelyet az assembler literal értékkel helyettesít.

15.2.5 Kidolgozott példa — a modul teljes implementációja

Munkakönyvtár: `~/arm-labor/a64/zaro1/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez az I. záróprojekt közös könyvtára: mind a négy alfejezete (15.1–15.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/a64/zaro1
cd ~/arm-labor/a64/zaro1
```

Az alábbi teljes, önálló `.S` fájl (`lab_sgemm.S`), a hozzá tartozó fejléc, C hívó program és `Makefile`. A futtatás a hallgató laborjában, `qemu-aarch64` alatt vagy natív AArch64-en történik.

Először a fejléc, közös `restrict` szerződéssel:

```
/* lab_sgemm.h */
#ifndef LAB_SGEMM_H
#define LAB_SGEMM_H
#include <stddef.h>
#define LAB_SGEMM_OK     0
#define LAB_SGEMM_EINVAL -1
int lab_sgemm(size_t n,
              const float *restrict A,
              const float *restrict B,
              float *restrict C);
#endif
```

A modul forrása:

```
/* lab_sgemm.S  -  A64 ISA, AArch64.  C := A * B  (minden nx n float). */
    .section .text
    .global  lab_sgemm
    .type    lab_sgemm, %function

    .equ     LAB_SGEMM_OK,     0
    .equ     LAB_SGEMM_EINVAL, -1

lab_sgemm:
    .cfi_startproc
    /* Prologus: 80 bájt frame, x19-x28 mentése. CFA = belépési SP = SP + 80. */
    stp      x19, x20, [sp, #-80]!
    .cfi_def_cfa_offset 80
    .cfi_offset 19, -80
    .cfi_offset 20, -72
    stp      x21, x22, [sp, #16]
    .cfi_offset 21, -64
    .cfi_offset 22, -56
    stp      x23, x24, [sp, #32]
    .cfi_offset 23, -48
    .cfi_offset 24, -40
    stp      x25, x26, [sp, #48]
    .cfi_offset 25, -32
    .cfi_offset 26, -24
    stp      x27, x28, [sp, #64]
    .cfi_offset 27, -16
    .cfi_offset 28, -8

    /* Paraméterek átmentése callee-saved regiszterekbe. */
    mov      x19, x0                 // n
    mov      x20, x1                 // A
    mov      x21, x2                 // B
    mov      x22, x3                 // C

    /* Határellenőrzés. */
    cbz      x19, .Lempty_ok
    cbz      x20, .Linvalid
    cbz      x21, .Linvalid
    cbz      x22, .Linvalid
    umulh    x5, x19, x19            // n*n magas 64 bitje
    cbnz     x5, .Linvalid           // n*n > 2^64-1 -> tulcsordulas
    mul      x6, x19, x19            // n*n alacsony 64 bitje
    lsr      x6, x6, #62             // felso 2 bit a 0/1. bitre
    cbnz     x6, .Linvalid           // n*n >= 2^62 -> n*n*4 tulcsordulna

    /* Nulla-konstans a C_row nullázásához. Szándékosan NEM v0: a k-ciklus
       minden iterációban felülírja v0-t az "a" broadcast értékével, ezért
       egy közös regiszteren a nullázás és a broadcast ütközne (a második
       sortól a nullázás a broadcast maradék értékét írná ki nulla helyett). */
    movi     v5.4s, #0

    /* i ciklus: i = 0..n-1 */
    mov      x23, #0
.Li_loop:
    cmp      x23, x19
    b.hs     .Ldone

    mul      x24, x23, x19           // i*n  (elemek)
    lsl      x24, x24, #2            // i*n*4 (bájtok)
    add      x25, x20, x24           // A_row = A + i*n*4
    add      x26, x22, x24           // C_row = C + i*n*4
```

```
    /* C_row[i, :] = 0, NEON-nal 4-esével. */
    mov     x2, #0
.Lzero_loop:
    add     x3, x2, #4
    cmp     x3, x19
    b.hi    .Lzero_tail
    lsl     x4, x2, #2              // bájteltolás = j*4; q-címzéshez külön képezve
    str     q5, [x26, x4]
    add     x2, x2, #4
    b       .Lzero_loop
.Lzero_tail:
    cmp     x2, x19
    b.hs    .Lzero_done
    str     wzr, [x26, x2, lsl #2]
    add     x2, x2, #1
    b       .Lzero_tail
.Lzero_done:

    /* k ciklus: k = 0..n-1 */
    mov     x27, #0
.Lk_loop:
    cmp     x27, x19
    b.hs    .Lnext_i

    ldr     s0, [x25, x27, lsl #2]  // a = A_row[k]
    dup     v0.4s, v0.s[0]          // a broadcast 4 sávra

    mul     x28, x27, x19           // k*n
    lsl     x28, x28, #2            // k*n*4
    add     x28, x21, x28           // B_row = B + k*n*4

    /* j ciklus NEON-nal, 4 elemenként. */
    mov     x2, #0
.Lj_loop:
    add     x3, x2, #4
    cmp     x3, x19
    b.hi    .Lj_tail
    lsl     x4, x2, #2              // bájteltolás = j*4; q-címzéshez külön képezve
    ldr     q1, [x26, x4]           // C_row[j..j+3]
    ldr     q2, [x28, x4]           // B_row[j..j+3]
    fmla    v1.4s, v0.4s, v2.4s     // C += a * B
    str     q1, [x26, x4]
    add     x2, x2, #4
    b       .Lj_loop

.Lj_tail:
    cmp     x2, x19
    b.hs    .Lnext_k
    ldr     s1, [x26, x2, lsl #2]
    ldr     s2, [x28, x2, lsl #2]
    fmadd   s1, s0, s2, s1          // skaláris maradék
    str     s1, [x26, x2, lsl #2]
    add     x2, x2, #1
    b       .Lj_tail

.Lnext_k:
    add     x27, x27, #1
    b       .Lk_loop

.Lnext_i:
    add     x23, x23, #1
    b       .Li_loop
```

```
.Ldone:
    mov     w0, #LAB_SGEMM_OK
    b       .Lepi

.Linvalid:
    mov     w0, #LAB_SGEMM_EINVAL
    b       .Lepi

.Lempty_ok:
    mov     w0, #LAB_SGEMM_OK

.Lepi:
    ldp     x19, x20, [sp]
    ldp     x21, x22, [sp, #16]
    ldp     x23, x24, [sp, #32]
    ldp     x25, x26, [sp, #48]
    ldp     x27, x28, [sp, #64]
    .cfi_restore 19
    .cfi_restore 20
    .cfi_restore 21
    .cfi_restore 22
    .cfi_restore 23
    .cfi_restore 24
    .cfi_restore 25
    .cfi_restore 26
    .cfi_restore 27
    .cfi_restore 28
    add     sp, sp, #80
    .cfi_def_cfa_offset 0
    ret
    .cfi_endproc

    .size   lab_sgemm, . - lab_sgemm

    .section .note.GNU-stack,"",%progbits   // nem kér végrehajtható vermet
```

Nézzük lépésről lépésre, mit csinálunk, és melyik korábbi alfejezet technikáját alkalmazzuk.

Prologus és paramétermentés. A `STP ... [sp, #-80]!` pre-index írás és a többi `STP [sp, #imm]` címzésmódja a Load/store címzési módok és a Prológus és epilógus: regisztermentés és keretkezelés közvetlen alkalmazása. A `MOV Xn, Xm` alak MOV-variánsok és immediate-kódolás — nincs immediate, csak regiszter-regiszter másolás.

Címzés és skálázás. Az `[X25, X27, lsl #2]` címzési mód regiszter + eltolt regiszter, 2-vel shiftelve, azaz `X25 + (X27 << 2)`; ez az Tömbök és indexelt elérés assemblyben mintapéldája: az `lsl #2` konvertálja az **elemszámot** bájtra, mert `sizeof(float) == 4`. A `mul` + `lsl #2` páros ugyanezt teszi a soronkénti stride-nál. Ez a forma `S`/`W` (32 bites, 4 bájtos) hozzáférésnél — mint a fenti `ldr s0, [x25, x27, lsl #2]` — helyes, mert a shift (`#2`) pontosan a transzferméretnek (4 bájt) megfelelő skálázás. A `Q` (128 bites, 16 bájtos) regiszteres betöltésnél/tárolásnál viszont a hardver a regiszter-offset `lsl` mezőjében csak `#0` vagy `#4` (log2(16)) kódolást enged meg; `lsl #2` ott **nem létező kódolás**, az assembler visszadobja. Ezért a `q`-s hozzáféréseknél (`C_row`/`B_row` négyesei) a bájteltolást (`j*4`) külön `lsl x4, x2, #2` utasítással képezzük előre, és shift nélkül, `[Xn, X4]` alakban hivatkozunk rá.

NEON vektorizálás. A `movi v5.4s, #0` és `dup v0.4s, v0.s[0]` a NEON regiszterkészlet és adatelrendezés tárgyalt lane-kezelése: a `dup` a forrás regiszter 0. sávját mind a négy sávba másolja. A `fmla v1.4s, v0.4s, v2.4s` az `fused multiply-accumulate` vektoros változata, a `C += a * B` műveletet végzi el egyetlen utasításban négy elemre. Ez a Vektorizált összeadás/szorzás: egy skaláris ciklus NEON-átirata közvetlen továbbvitele. A skalár farok (`fmadd s1, s0, s2, s1`) az `n` nem-néggyel-osztható hátralékát dolgozza fel, elkerülve a túlindexelést. Fontos regiszterkezelési

szabály: a V0 minden k-iterációban felülíródik az a broadcast értékével (a ldr s0, ... a V0 alsó 32 bitjét írja, és a hardver az írással a felső biteket nullázza, mielőtt a dup szétmásolná), ezért a C_row nullázásához **nem** szabad V0-t újrafelhasználni egy másik, hosszabb életű konstanshoz — ez az oka, hogy a nulla-konstans külön V5-ben él, amelyet a k-ciklus sosem ír.

Egész-aritmetika. A mul, add, cmp, valamint a cbz/cbnz Feltétel nélküli és feltételes elágazás szerinti feltételes vezérlést ad. Az umulh bitszintű viselkedése az Egész aritmetika: ADD/SUB/MUL/DIV és a kettes komplemens viselkedése hatókörébe tartozik, de konkrét használata (túlcsordulás-detektálás) új itt. Az b.hs feltétel (unsigned higher or same) — >= jelentéssel — a CMP által beállított C flagre épül, és fontos, hogy itt **unsigned** összehasonlításról van szó, mert X19 (n) size_t.

Caller-saved vs callee-saved. Az X2–X6 ideiglenes ciklus- és címváltozókat bátran felülírjuk, mert caller-saved (az X2/X3 eredeti argumentumait a modul előtte X21/X22-be menti); az X19–X28 callee-saved állapotot a prologus/epilógus védi. A .cfi_* direktívák ezt a keretet írják le a hibakeresőnek: a függvény nem hív mást, ezért nem épít X29/X30 keretrekordot, a CFA-t az SP-hez képest adjuk meg (Prológus és epilógus). Az X30 végig a saját regiszterében marad, ezért nincs hozzá .cfi_offset. Ez levélfüggvénynél szabályos: az LR-t csak akkor kell menteni, ha a függvény felülírja (például BL-lel).

Veremjelző. A fájl végi .note.GNU-stack szakasz azt jelzi a linkernek, hogy a modul nem kér végrehajtható vermet. A korábbi alfejezetek mintáiban is szerepel, ezért a projektmagban is ott a helye. Ez **következetességi** javítás, nem egy igazolt hiba kijavítása: a direktíva hiányából nem következik automatikusan végrehajtható verem. Hogy a linker mit csinál jelzés nélküli objektummal, az eszközlánc- és verziófüggő (egy AArch64 Binutils 2.40-es próbában a direktíva nélkül is RW lett a verem). A döntő bizonyíték ezért nem a forrás, hanem a kész ELF GNU_STACK szegmense, amit a build ellenőriz (lásd lent). Az S0–S2 és V5 (a V0–V2, V5 alsó 32, illetve teljes 128 bitje) caller-saved, így nem igényel mentést.

A C hívó program, amely referenciát és aktuális modult futtat, összevetve őket:

```c
/* lab_sgemm_main.c */
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include "lab_sgemm.h"

static void ref_sgemm(size_t n, const float *A, const float *B, float *C)
{
    for (size_t i = 0; i < n; i++)
        for (size_t j = 0; j < n; j++) {
            double s = 0.0;
            for (size_t k = 0; k < n; k++)
                s += (double)A[i*n + k] * (double)B[k*n + j];
            C[i*n + j] = (float)s;
        }
}

int main(void)
{
    const size_t n = 8;
    float *A = malloc(n*n*sizeof A[0]);
    float *B = malloc(n*n*sizeof B[0]);
    float *C = malloc(n*n*sizeof C[0]);
    float *R = malloc(n*n*sizeof R[0]);
    if (!A || !B || !C || !R) { free(A); free(B); free(C); free(R); return 1; }
    for (size_t i = 0; i < n*n; i++) {
        A[i] = (float)((i * 7) % 5) * 0.25f;
        B[i] = (float)((i * 3) % 7) * 0.125f;
```

```
    }
    ref_sgemm(n, A, B, R);
    int rc = lab_sgemm(n, A, B, C);
    if (rc != LAB_SGEMM_OK) {
        printf("rc=%d\n", rc);
        free(A); free(B); free(C); free(R);
        return 1;
    }
    double max_err = 0.0;
    size_t bad = 0;
    for (size_t i = 0; i < n*n; i++) {
        double d = fabs((double)C[i] - (double)R[i]);
        /* NaN-ra a d > max_err és a d <= 1e-4 is hamis: a nem véges elemet
           külön, a NaN-t is elutasító !(d <= tűrés) alakkal számoljuk hibának. */
        if (!isfinite(C[i]) || !(d <= 1e-4)) bad++;
        else if (d > max_err) max_err = d;
    }
    printf("max abs err = %.3e, hibás elemek: %zu\n", max_err, bad);
    free(A); free(B); free(C); free(R);
    return bad == 0 ? 0 : 2;
}
```

Build és futtatás:

```
aarch64-linux-gnu-gcc -O2 -c lab_sgemm.S -o lab_sgemm.o
aarch64-linux-gnu-gcc -O2 -c lab_sgemm_main.c -o lab_sgemm_main.o
aarch64-linux-gnu-gcc -static lab_sgemm.o lab_sgemm_main.o -lm -o lab_sgemm
readelf -lW lab_sgemm | awk '$1=="GNU_STACK" {print "GNU_STACK", $7}'   # elvárt: RW
qemu-aarch64 ./lab_sgemm
echo "exit=$?"
```

vagy `make`-kel:

```
# Makefile
CC      = aarch64-linux-gnu-gcc
CFLAGS  = -O2 -Wall -Wextra
LDFLAGS = -static
LDLIBS  = -lm
lab_sgemm: lab_sgemm.o lab_sgemm_main.o
        $(CC) $(LDFLAGS) $^ $(LDLIBS) -o $@
        readelf -lW $@ | awk '$$1=="GNU_STACK" {f=$$7} END {exit f!="RW"}' \
            || { echo "HIBA: a GNU_STACK nem RW"; rm -f $@; exit 1; }
%.o: %.c
        $(CC) $(CFLAGS) -c $< -o $@
%.o: %.S
        $(CC) $(CFLAGS) -c $< -o $@
run: lab_sgemm
        qemu-aarch64 ./$<
```

Kimenet a fő úton mérve (GNU + glibc + QEMU 8.2.2, a lektor mérése): `GNU_STACK RW`, `max abs err = 0.000e+00, hibás elemek: 0`, `exit=0`. A nulla eltérés ennél a bemenetnél nem véletlen: az elemek kis kettedes törtek (a 0,25 és a 0,125 többszörösei), így minden szorzat és részösszeg pontosan ábrázolható `float`-ban is. Általános bemenetnél nem nulla eltérés várható, és ezt a projektspecifikáció numerikus szerződése kezeli. Ez a gyors próba egyszerűsített abszolút küszöböt (`1e-4`) használ, de már NaN-biztos: a `!(d <= 1e-4)` alak és az `isfinite` miatt egy NaN-os kimenet nem adhat „0 maximális hibát”. (A régi `if (d > max_err)` minta NaN-ra hamis ágat ad, így a NaN-os elem észrevétlen maradt volna.) A teljes ellenőrzés az 15.3 tesztkészletének dolga.

15.2.5.1 ABI-ellenőrzés a közös teszthívóval

A numerikus egyezés nem igazolja, hogy a modul visszaadta a hívónak az `X19`–`X28` regisztereket és a belépési `SP`-t. Ezt a Prológus és epilógus közös teszthívójával ellenőrizzük, mindhárom kilépési úton (normál, üres bemenet, érvénytelen bemenet), mert mindegyik ugyanazon az epiloguson halad át:

```
cp ~/arm-labor/a64/a029/lab-abi-call.h ~/arm-labor/a64/a029/lab-abi-call.S .
```

```
/* lab_sgemm_abi.c - a lab_sgemm ABI-ellenőrzése a közös teszthívóval. */
#include <inttypes.h>
#include <stdint.h>
#include <stdio.h>
#include "lab-abi-call.h"
#include "lab_sgemm.h"

static int lab_case(const char *name, size_t n,
                    const float *A, const float *B, float *C, int want_rc)
{
    uint64_t x0 = 0;
    uint64_t mask = lab_abi_call((lab_abi_fn)lab_sgemm, n,
                                 (uintptr_t)A, (uintptr_t)B, (uintptr_t)C,
                                 0, 0, &x0);
    /* int visszatérési érték: az AAPCS64 szerint csak a W0 (alsó 32 bit)
       értelmes, az X0 felső fele tetszőleges lehet. */
    int rc = (int32_t)(uint32_t)x0;
    int ok = mask == 0 && rc == want_rc;

    printf("%-8s maszk=0x%05" PRIx64 " rc=%d %s\n",
           name, mask, rc, ok ? "OK" : "HIBA");
    return ok;
}

int main(void)
{
    static float A[25], B[25], C[25];
    int ok = 1;

    for (int i = 0; i < 25; i++) {
        A[i] = (float)i;
        B[i] = 1.0f;
    }
    ok &= lab_case("n=5", 5, A, B, C, LAB_SGEMM_OK);       /* NEON + skalár farok */
    ok &= lab_case("n=0", 0, A, B, C, LAB_SGEMM_OK);       /* üres bemenet */
    ok &= lab_case("A=NULL", 5, NULL, B, C, LAB_SGEMM_EINVAL);
    ok &= C[0] == 10.0f && C[24] == 110.0f;                /* sor-összegek */
    return ok ? 0 : 1;
}
```

```
# Target: AArch64 Linux, qemu-user.
aarch64-linux-gnu-gcc -std=c11 -O2 -Wall -Wextra -static \
    lab_sgemm_abi.c lab_sgemm.S lab-abi-call.S -o lab_sgemm_abi
qemu-aarch64 ./lab_sgemm_abi
echo "exit=$?"
```

Kimenet a fő úton mérve (GNU + glibc + QEMU 8.2.2, a lektor mérése):

```
n=5      maszk=0x00000 rc=0 OK
n=0      maszk=0x00000 rc=0 OK
A=NULL   maszk=0x00000 rc=-1 OK
exit=0
```

Két részlet érdemel figyelmet. A teszthívó a `lab_sgemm` `X0`-ját 64 bitesen adja vissza, a függvény viszont `int`-et ad: az AAPCS64 szerint ilyenkor csak a `W0` értelmes, ezért a teszt az alsó 32 bitet veszi. Az `n = 5` eset a NEON-ciklust és a skalár farkat is bejárja, így mindkét kódút a mintákkal feltöltött regiszterek mellett fut.

15.2.6 Hibák és ellenpróbák

Néhány tipikus hiba a megvalósítás során, tünettel, diagnózissal és ellenpróbával.

Tünet 1: a C hívó egyik változója a `lab_sgemm` hívás után megváltozik, látszólag véletlenszerűen. **Hipotézis:** hiányos callee-saved mentés — valamelyik `X19`–`X28` regisztert felülírtuk, és nem mentettük. **Adatgyűjtés:** `objdump -d lab_sgemm.o | grep -E 'stp|ldp'` mutatja a prologust; a `gdb-multiarch` `break lab_sgemm` után `info registers x19 x20 ...` hívás előtt és után. **Egy változtatás:** adjuk hozzá a hiányzó `STP/LDP` párt. **Ellenőrzés:** ismételt futtatás; a bug megszűnik.

Tünet 2: `EINVAL` olyankor, amikor a tesztek null méretet adtak `n==0`-val. **Hipotézis:** a `cbz x19, .Lempty_ok` előtt a null pointer ellenőrzés fut. **Ellenőrzés:** sorrend a forrásban. A megoldás: a `n==0` eset legyen **első**.

Tünet 3: a modul néha néma, hibás eredményt ad, de más inputon jó. **Hipotézis:** `n` nem osztható 4-gyel, és a NEON farok hiányzik vagy hibás. **Ellenőrzés:** futtassuk `n = 3`, `n = 5`, `n = 7` értékkel; ha csak ezek hibásak, a `.Lj_tail` (skalár farok) a hunyó. Ellenpróba: ideiglenesen tiltsuk le a vektoros ágat (minden `j` a farkon fusson); ha így helyes, a vektoros ág a hibás, ha nem, a farok.

Tünet 4: a `qemu-aarch64 ./lab_sgemm` „cannot execute binary file”-t ír. **Hipotézis:** a linkelés nem cross-toolchainnel történt (natív `gcc` külső x86 ELF-et ad). **Ellenőrzés:** `file lab_sgemm` mutatja az architektúrát; helyes esetben `ELF 64-bit LSB executable, ARM aarch64`. **Egy változtatás:** `aarch64-linux-gnu-gcc` használata.

Tünet 5: a mért idő `n`-től függően ingadozik, a vártnál rosszabb. **Hipotézis:** a 16 bájtos igazítás megsérült — a hívó `float*` mutató nem 16-bájt-igazított. **Adatgyűjtés:** `printf("%p\n", (void*)C)` a C oldalon; ha `p mod 16 != 0`, a `STR Q1` 4-bájt-igazításon is működik A64-en (nem ad `SIGBUS`-t), de a teljesítmény zuhan. **Ellenőrzés:** `posix_memalign`-nel 16-bájt-igazított puffert adjunk, és mérjük újra.

15.2.7 Feladatok

1. (magyarázó, középszint — cél: megértés-ellenőrzés) Magyarázd el, miért nem elég önmagában a `umulh`-alapú ellenőrzés a túlcsordulás kizárásához, és miért kell mellé a `mul` eredményének felső két bitjét is megnézni. Add meg azt a legnagyobb `size_t n` értéket, amelyre a modul ellenőrzése még átmegy (`LAB_SGEMM_OK`-t adna a határellenőrzésen), és indokold, miért pontosan ez a határ.

2. (implementációs, elmélyült — cél: fejezetek összekapcsolása: Mikrobenchmark készítése és eredmények értelmezése) Bővítsd a `.Lj_loop`-ot úgy, hogy egyszerre **8** `float`-ot dolgozzon fel egy iterációban: két `q` regisztert használva (`q1`, `q3` bemenet, `q0` a broadcast `a`). Az `n` páratlan maradékára a skalár farok maradjon. Mérd meg a gyorsulást a medián-alapú mérési tervvel (vö. Mikrobenchmark készítése és eredmények értelmezése).

3. (hibakeresési/mérési, középszint — cél: rutinépítés) A `Makefile`-ban fordítsd `-O0`-val a `lab_sgemm.S`-t (ez az `as` szintjén nem változtat), és hasonlítsd össze a `lab_sgemm_main.c`-t `-O0`, `-O2`, `-O3` szinteken. Az assembly modul időzítését `clock_gettime(CLOCK_MONOTONIC, …)`

köré csomagolva mérd. Melyik szinten és miért tapasztalsz eltérést? Add meg a gyökér-okot (a C oldal vektorizálása vs a modul fix kódja).

15.2.8 Megoldások

1. megoldás — Segítség. A `umulh` csak azt dönti el, hogy `n*n` fér-e 64 biten — de a modul nem `n*n` ELEMET, hanem `n*n*4` BÁJTOT címez. Van-e olyan `n` tartomány, ahol `n*n` még belefér, de a néggyel szorzott érték már nem?

1. megoldás — Teljes megoldás. A `umulh x5, x19, x19` önmagában csak azt dönti el, hogy `n*n` elfér-e 64 biten (`x5 == 0` pontosan akkor, ha `n  0xFFFFFFFF`, azaz $2^{32}-1$). Ez **nem elég**: a modul nem `n*n`-et, hanem `n*n*4` **bájtot** címez. Ha `n` a 2^{31} és $2^{32}-1$ közötti tartományban van, `n*n` még befér 64 biten (`x5 == 0`), de `n*n*4` már nem — ez pontosan az a sáv, ahol a `umulh`-only ellenőrzés hibásan átengedné a bemenetet, és a modul a `C` tömbön kívülre írna. Ezért kell a `mul x6, x19, x19` eredményének (az `n*n` alsó 64 bitjének) felső két bitjét is megnézni: ha ezek közül bármelyik 1, `n*n  2`^{2}, azaz `n*n*4  2` — túlcsordulás, `LAB_SGEMM_EINVAL`. A legnagyobb elfogadott `n` az, amelyre `n*n < 2`^{2}, azaz `n <` 2^{31} — a legnagyobb egész érték `n =` $2^{31}-1$ `= 2 147 483 647`. Ennél az `n`-nél `n*n =` $2^{2}-2^{32}+1$, ami `*4`-gyel $2^{3}-4$ bájt: éppen 64 biten belül marad, tehát ez a tényleges, pontos felső korlát — nem a $(2^{32}-1)^{2} \cdot 4$ `2` bájt, amit a puszta `umulh`-ellenőrzés hibásan megengedne.

2. megoldás — Segítség. A négyes változat egyetlen `fmla` lánca függőséget hoz létre az iterációk között; mi történik, ha KÉT független `fmla`-láncot futtatsz párhuzamosan két `q`-regiszterpáron?

2. megoldás — Teljes megoldás. A 8-széles belső ciklus lényege:

```
.Lj_loop8:
    add     x3, x2, #8
    cmp     x3, x19
    b.hi    .Lj_loop4
    lsl     x4, x2, #2              // bájteltolás = j*4
    ldr     q1, [x26, x4]           // C_row[j..j+3]
    ldr     q2, [x28, x4]           // B_row[j..j+3]
    add     x6, x4, #16             // bájteltolás = (j+4)*4 = j*4+16
    ldr     q3, [x26, x6]           // C_row[j+4..j+7]
    ldr     q4, [x28, x6]           // B_row[j+4..j+7]
    fmla    v1.4s, v0.4s, v2.4s
    fmla    v3.4s, v0.4s, v4.4s
    str     q1, [x26, x4]
    str     q3, [x26, x6]
    add     x2, x2, #8
    b       .Lj_loop8
```

A két `fmla` független regiszterláncon fut, ami jobb utasításszintű párhuzamosságot (ILP) enged, mint a négyes változat. Mérési terv: medián és MAD rögzítése mindkét változatra, `n=256`-ra, cache-meleg futtatásokkal. Az iterációk között nincs regiszterfüggőség (minden iteráció más C-szeletet tölt be); a 8-as kibontás a ciklusvezérlés és a címszámítás arányát csökkenti. A várható nyereséget mérni kell; ha a szűk keresztmetszet a memória, a gyorsulás elmarad.

3. megoldás — Segítség. A referencia (`ref_sgemm`) és a modul közül melyiket fordítja a GCC változó szinten, és melyik marad bitre azonos minden `-O` szint mellett, mert már kézzel vektorizált assembly?

3. megoldás — Teljes megoldás. A mért értékek (GCC 13.3, QEMU 8.2.2, n = 128, a `ref_sgemm` 5 futásának átlaga): `-O0` 54,6 ms, `-O2` 34,3 ms, `-O3` 33,7 ms, azaz a különbség ~1,6×, nem ~10×. A `-O2` nem vektorizál; `-O3` mellett a GCC a j-ciklust vektorizálja (16 bájtos

vektorokkal), ami itt alig gyorsít. A `-O0` → `-O2` nyereség fő forrása a regiszterallokáció: `-O0` mellett minden változó a veremben van. A modul assembly kódja bitre azonos minden szinten. Az összehasonlítás ezért a **referencia** és a **modul** közti gyorsulást adja: `-O0`-n a látszólagos gyorsulás nagy, de ez a referencia gyengesége, nem a modul ereje. A tanulság: a teljesítménymérést mindig az optimalizált referenciához kell hasonlítani, és rögzíteni kell a fordítói szintet. A mérés `CLOCK_MONOTONIC`-kal mediánt és MAD-ot rögzítsen, legalább tíz futtatásból.

15.2.9 Rövid összefoglaló

A `lab_sgemm` modul az I. rész technikáit egyetlen AAPCS64-konform assembly függvényben egyesíti: `X0`–`X3` argumentum, `W0` visszatérés, `X19`–`X28` mentése `STP/LDP` párokkal, `SP` 16 bájtos igazítása. A határellenőrzés `CBZ`-vel végzi a null-tesztet, `umulh`+`mul`-lal a túlcsordulás-tesztet (a tényleges bájtméretre, nem csak az elemszámra). A kernel ikj sorrendben halad, `MOVI/DUP/FMLA` NEON-utasításokkal vektorizál, skalár farokkal kezeli az `n` nem-többszörös esetét. A build `aarch64-linux-gnu-gcc`-vel, `qemu-aarch64`-gyel reprodukálható.

Továbbhasznált definíciók:

- **modul-szerződés:** AAPCS64 argumentum-, visszatérési, callee-saved és stack-igazítási megállapodás.
- **határellenőrzés:** `CBZ` a null-pointerekhez, `umulh`+`mul` a bájtméret-túlcsordulás teljes (nem csak elemszám-) ellenőrzéséhez, a mag előtt.
- **fused multiply-accumulate (FMLA/FMADD):** egy utasításban `d = a*b + c`, NEON sávonként vagy skalárisan.

15.2.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile Architecture Reference Manual — `UMULH`, `FMLA`, `DUP`, `MOVI`, `STP/LDP` pre/post-index szemantika, NZCV és `b.hs` elsődleges forrása. **Ellenőrizve**: a link él; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU Compiler Collection kézikönyv — `restrict`, `-O2`/`-O3` viselkedés, cross-build lépések. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2.
- GNU Binutils `as` kézikönyv — `.section`, `.global`, `.type`, `.size`, `.equ`, `.S` előfeldolgozás. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.

Ellenőrzési állapot: a `lab_sgemm_main` és a `lab_sgemm_abi` kimenetét, a kész ELF `GNU_STACK RW` jelzőjét, valamint a `lab_sgemm` CFI-táblázatát a fő úton, GNU + glibc + QEMU 8.2.2 alatt mértük (a lektor mérése). **Fennmaradó ellenőrzési hiányok:** (1) a `umulh`+`mul`-alapú védelem pontos felső korlátját `n = 2`^{31}`-1`-re (`n*n*4 = 2 -2`^{3} `+4` bájt) vezettük le, de a gyakorlati `malloc` felső korlát ettől jóval kisebb — a valós host memóriahatárok ellenőrzése nem része e szakasznak. (2) A `qemu-aarch64` NEON-teljesítménye és a natív AArch64 hardver viselkedése eltérhet; a szakaszban közölt gyorsulási számok **levezetett, illusztratív** értékek, nem mért hardveradatok. (3) A `restrict` tényleges hatása a hívó C-fordító vektorizálására az adott GCC-verziótól függ; az Ubuntu 24.04 csomagolt GCC-verziója ellenőrzést igényel a hallgató laborjában.

15.3 Mérés és validáció

15.3.1 Motiváció

Az 15.2 alfejezetben elkészült `lab_sgemm` modul addig csak *feltételezett* tulajdonságokkal rendelkezik: az AAPCS64-szerződést betartja, `FMLA`-val vektorizál, és a QEMU-ban lefut. Ez azonban önmagában nem elég. Egy teljesítménykritikus modul akkor számít késznek, ha **számokkal** bizonyítjuk, hogy (a) ugyanazt a kimenetet adja, mint a C referencia, és (b) az átirat tényleges gyorsulást hoz a mért hardveren/emulátoron. Ez az alfejezet a *mérési és validációs szerződés* végrehajtását mutatja be: a tesztkészlet lefuttatását, a referenciaegyezés dokumentálását, a benchmark-futtatást az 13.4 sablonjával, az eredmény vizuális összefoglalását, és az elfogadási küszöb értékelését. A tanulság átvihető bármely későbbi assembly-modulra: mérni nem lehet, ha a korrektségről nincs tényszerű állítás, és validálni nem lehet, ha a mért értékeket nem jelöljük megfelelően (emulált vs. valós).

15.3.2 Definíció: validáció és mérés szétválasztása

A projekt lezárásában két különböző tevékenységet kell szigorúan szétválasztani:

Validáció (validation) — annak eldöntése, hogy a modul *funkcionálisan* megfelel-e a specifikációnak. Eszköze egy determinisztikus, reprodukálható tesztkészlet, amely a `lab_sgemm` kimenetét az 15.1 **numerikus szerződése** szerint veti össze a `double` pontosságú C referenciával (rögzített `FPCR`, véges bemenet, s_{ij}-hez mért tűrés), és közben a memóriaszerződést (csak a `C` írható) is ellenőrzi. Ugyanez a szerződés érvényes a mérés előtti ellenőrzésre és az auditra is; bitre pontos egyezést sehol nem követelünk.

Mérés (measurement) — annak számszerűsítése, hogy a modul *mennyivel* gyorsabb a referenciánál, adott bemenetméreteken, adott build- és futtatási profilban. A mérés soha nem igazol korrektséget; a validáció soha nem ad gyorsulási állítást.

A két tevékenység együtt adja a **kiadási döntést**: a modul akkor fogadható el, ha egyszerre teljesül a funkcionális egyezés és a teljesítmény-küszöb. A küszöb konkrét értékét az 15.1 projektspecifikáció rögzíti; a jelen alfejezet a küszöb **kiértékelésének módszerét** tanítja, nem annak számszerű megválasztását.

Továbbhasznált fogalmak:

- **korrektségi tesztvektor:** adott `(n, A, B)` bemenet-kombináció, amelyre a kimenet elvárt értéke *a referencia implementációból* számolható.
- **referenciaegyezés:** minden kimeneti elem véges, és az 15.1 szerinti $|c_{ij}^{\text{asm}} - c_{ij}^{\text{ref}}| \le \text{atol} + \text{rtol} \cdot s_{ij}$ feltétel teljesül, ahol $s_{ij} = \sum_k |a_{ik}||b_{kj}|$. Bitre pontos (`memcmp`) egyezés a `double` referenciával nem várható, de nem az összegzési sorrend miatt: a k-sorrend a modulban és a referenciában ugyanaz. Az ok a pontosság: a modul minden fúzionált lépés után `float`-ra kerekít, a referencia `double`-ban összegez.
- **elfogadási küszöb:** az 15.1 által rögzített feltétel: a jegyzőkönyvben megnevezett környezetben `n  64` mellett a modul medián ideje legfeljebb a naiv i-j-k C-változat medián idejének a fele.

15.3.3 A korrektségi tesztkészlet

Munkakönyvtár: `~/arm-labor/a64/zaro1/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez az I. záróprojekt közös könyvtára: mind a négy alfejezete (15.1–15.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/a64/zaro1
cd ~/arm-labor/a64/zaro1
```

A tesztkészlet az 15.1 projektspecifikáció által rögzített határesetek és speciális mátrixtartalmak **végrehajtása**, nem egy új terv kitalálása. A végrehajtott tesztmátrix pontosan a specifikáció táblázatait követi:

1. **határesetek és nagy méretek:** `n ∈ {0, 1, 2, 3, 5, 7, 8, 16, 17, 63, 64, 65, 128, 256}`, továbbá `n ∈ {4, 6, 12}`, mintázott tartalommal. Ezekhez jön egy méret nélküli csoport: `A`, `B` és `C` **egyenként** `NULL`-pointerrel (három különálló hívás), és egy `n = 2^31`, amelyre az 15.2 szerint `n*n*sizeof(float)` 64 biten túlcsordul.
2. **speciális tartalmak:** csupa nulla, csupa egyes, csupa mínusz egyes, identitás, nagyérték-teszt (`±1e6`) és a szerződés felső határát terhelő, egynemű előjelű `near-max` ($s_{ij} =$ `0x1.fffffcp126` $\le$ FLT_MAX/2) `n ∈ {32, 64}` mellett; rögzített magú véletlen tartalom `n ∈ {32, 64, 128, 256}` mellett. Az identitásmátrix ($B = I$, tehát $C = A$) a `DUP`-os sávkitöltés és az indexelés hibáit teszi láthatóvá.
3. **szerződésen kívüli esetek:** `n = 8`, minden elem `1e20` (`overflow`), az 15.1 4×4-es FMA-határ-ellenpéldája (`fma-edge`), amelyet a régi $s_{ij} <$ FLT_MAX feltétel még átengedett, valamint egy `n = 2`-es határeset (`half-edge`: `A` első sora {FLT_MAX/2, 1}, `B` első oszlopa {1, 1}, a többi elem 0). Az utóbbinál a pontos $s_{00} =$ FLT_MAX/2 + 1 a határ fölött van, de `double`-ben FLT_MAX/2 + 1 = FLT_MAX/2; egy egyszerű `abs_sum <= LAB_SMAX` összevetés ezért elfogadná. Mindhárom esetnél előbb azt ellenőrizzük, hogy a `lab_sgemm_in_contract` elutasítja-e. Utána a kimenetet nem a numerikus ellenőrző értékeli: a dokumentált elvárás a modul számítási modellje, vagyis a k-sorrendű `fmaf`-lánc bitre pontos eredménye (`overflow`: minden elem $+\infty$; `fma-edge`: a 0. sor $+\infty$, a többi 0; `half-edge`: véges eredmény).

A `lab_sgemm_in_contract` **konzervatív (elégséges) szűrő**, nem „akkor és csak akkor” predikátum. A `float·float` szorzatok `double`-ben pontosak (24 + 24 ≤ 53 bit), `n` nemnegatív tag `double`-összegének relatív hibája legfeljebb γ_{n-1}, a $(1+g)$-vel való szorzás saját kerekítésével együtt γ_{n+2} fedi le. A függvény ezért akkor ad 1-et, ha $\hat{s} \cdot (1+\gamma_{n+2}) \le$ FLT_MAX/2; ilyenkor a pontos s_{ij} **biztosan** a határon belül van. Egy pontosan belső, de a határtól relatíve $(n+2)\cdot 2^{-53}$-nál közelebbi esetet viszont elutasíthat: ez a dokumentált konzervatív irány. A `near-max` eset (relatíve kb. 2^{-23}-nal a határ alatt) ettől szerződésen belül marad.

Minden érvényes esetnél a teszthajtó a bemenet **osztályozását** is ellenőrzi: a 2. pont esetei szerződésen belüliek, a 3. pontéi azon kívül esnek. Ha a `lab_sgemm_in_contract` ettől eltér, az eset `osztályozás` okkal FAIL.

4. **az ellenőrző önellenőrzése:** a referencia `float`-ra kerekített eredményébe szándékosan NaN-t, `+Inf`-et, `-Inf`-et és tűrésen kívüli eltérést írunk; mindegyiknek le kell buknia. Ez a vizsgált modultól független, tehát egy hibás modul nem tudja elrontani.

A memóriaszerződést minden eset ellenőrzi. A teszthajtó a mátrixokat **őrzött pufferbe** teszi: az adat előtt őrbájtok állnak, az adat vége pedig pontosan egy `PROT_NONE` védőlapnál ér véget, így a sor- vagy mátrixvégen túlnyúló `ldr q`/`str q` laphibát okoz. A `C` puffer a hívás előtt `0xFF` bájtokkal (NaN-bitmintával) töltődik, ezért egy meg nem írt elem is FAIL. A hívás után a

teszthajtó az `A`-t és a `B`-t bájtra összeveti a másolatával. Érvénytelen bemenetnél a kimeneti puffer speciális bitmintákat tartalmaz (`-0.0`, NaN-payloadok, `+Inf`, denormális szám). A teljes tartomány bájtjait az őrbájtokkal együtt `memcmp`-pel vetjük össze, nem `float`-egyenlőséggel, mert a NaN önmagával sem egyenlő, a `-0.0 == +0.0` pedig igaz. A C oldali AddressSanitizer ezt nem helyettesíti, mert a kézzel írt assembly memóriaeléréseit nem instrumentálja.

A referencia ugyanaz a `double` pontosságú, `float` bemenetű hármas ciklus, amelyet a specifikáció rögzített. A numerikus szerződés ellenőrzője külön fordítási egység, mert a mérőprogram is használja:

```
/* lab_sgemm_ref.c - double pontosságú korrektségi referencia (a057). */
#include "lab_sgemm.h"

void lab_sgemm_ref(size_t n, const float *A, const float *B, double *C)
{
    for (size_t i = 0; i < n; ++i) {
        for (size_t j = 0; j < n; ++j) {
            double acc = 0.0;
            for (size_t k = 0; k < n; ++k)   /* ugyanaz a k-sorrend, mint a modulban */
                acc += (double)A[i * n + k] * (double)B[k * n + j];
            C[i * n + j] = acc;
        }
    }
}
```

```
/* lab_sgemm_check.h - az a057 numerikus szerződésének ellenőrzője. */
#ifndef LAB_SGEMM_CHECK_H
#define LAB_SGEMM_CHECK_H
#include <float.h>
#include <stddef.h>

#define LAB_RTOL     1e-4   /* n <= LAB_TOL_NMAX esetén: n * 2^-24 <= 1.6e-5 < LAB_RTOL */
#define LAB_ATOL     1e-30  /* csak az alulcsordulási tartomány elnyelésére */
#define LAB_TOL_NMAX 256
/* s_ij felső korlátja: kerekítési tartalék a float-FMA lánchoz */
#define LAB_SMAX     (FLT_MAX / 2)

void lab_sgemm_ref(size_t n, const float *A, const float *B, double *C);

/* 1, ha FPCR: RMode = RNE, FZ = 0, DN = 0 (és AH/FIZ/NEP = 0). */
int lab_fpcr_ok(void);

/* Konzervatív (elégséges) szűrő: 1 csak akkor, ha minden bemeneti elem véges, és
 * minden PONTOS s_ij = sum_k |a_ik||b_kj| bizonyítottan <= LAB_SMAX. A határ körüli
 * relatív (n+2)*2^-53 sávban egy pontosan belső esetet is elutasíthat (0-t ad). */
int lab_sgemm_in_contract(size_t n, const float *A, const float *B);

/* 0, ha minden elem véges és |c_ij - r_ij| <= LAB_ATOL + LAB_RTOL * s_ij;
   különben 1, és *bad az első hibás elem indexe. */
int lab_sgemm_check(size_t n, const float *A, const float *B,
                    const float *C, const double *R, size_t *bad);
#endif
```

```
/* lab_sgemm_check.c - az a057 numerikus szerződésének ellenőrzője. */
#include <float.h>
#include <math.h>
#include <stdint.h>
#include "lab_sgemm_check.h"

int lab_fpcr_ok(void)
{
    uint64_t fpcr;
    __asm__ volatile("mrs %0, fpcr" : "=r"(fpcr));
```

```
    const uint64_t mask = (3u << 22)    /* RMode: 00 = RNE        */
                        | (1u << 24)    /* FZ: nincs flush-to-zero */
                        | (1u << 25)    /* DN: nincs default NaN   */
                        | 0x7u;         /* NEP, AH, FIZ            */
    return (fpcr & mask) == 0;
}

static double abs_sum(size_t n, const float *A, const float *B, size_t i, size_t j)
{
    double s = 0.0;
    for (size_t k = 0; k < n; ++k)
        s += fabs((double)A[i * n + k]) * fabs((double)B[k * n + j]);
    return s;
}

int lab_sgemm_in_contract(size_t n, const float *A, const float *B)
{
    for (size_t e = 0; e < n * n; ++e)
        if (!isfinite(A[e]) || !isfinite(B[e]))
            return 0;
    /* A float*float szorzatok double-ben pontosak (24 + 24 <= 53 bit); n nemnegatív
     * tag összegének hibája <= gamma_{n-1} * pontos összeg. A (1 + g) szorzás saját
     * kerekítésével együtt gamma_{n+2} fedi: számított * (1 + g) <= LAB_SMAX esetén a
     * pontos összeg biztosan <= LAB_SMAX. */
    const double nu = (double)(n + 2) * 0x1p-53;
    const double g = nu / (1.0 - nu);
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < n; ++j)
            if (!(abs_sum(n, A, B, i, j) * (1.0 + g) <= LAB_SMAX))
                return 0;
    return 1;
}

int lab_sgemm_check(size_t n, const float *A, const float *B,
                    const float *C, const double *R, size_t *bad)
{
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < n; ++j) {
            size_t e = i * n + j;
            double c = (double)C[e];
            double bound = LAB_ATOL + LAB_RTOL * abs_sum(n, A, B, i, j);
            /* A !(x <= y) alak NaN-ra is igaz: a NaN-os elem FAIL. */
            if (!isfinite(c) || !(fabs(c - R[e]) <= bound)) {
                *bad = e;
                return 1;
            }
        }
    return 0;
}
```

A teszthajtó:

```
/* lab_sgemm_test.c - korrektségi teszthajtó az a057 numerikus szerződésével. */
#define _DEFAULT_SOURCE
#include <math.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>
#include "lab_sgemm.h"
#include "lab_sgemm_check.h"
```

```c
/* Őrzött puffer: [őrbájtok | adat | PROT_NONE védőlap]. Az adat vége pontosan
   a védőlapnál ér véget, így a túlindexelt olvasás vagy írás laphibát okoz;
   az adat előtti írást az őrbájtok megváltozása mutatja. */
#define LAB_GUARD_BYTE 0xA5
typedef struct { unsigned char *map, *data; size_t maplen, front; } guarded_t;

static float *guarded_alloc(guarded_t *g, size_t bytes)
{
    size_t pg = (size_t)sysconf(_SC_PAGESIZE);
    size_t body = (64 + bytes + pg - 1) / pg * pg;   /* legalább 64 őrbájt */
    g->maplen = body + pg;
    g->map = mmap(NULL, g->maplen, PROT_READ | PROT_WRITE,
                  MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    if (g->map == MAP_FAILED) { perror("mmap"); exit(2); }
    if (mprotect(g->map + body, pg, PROT_NONE)) { perror("mprotect"); exit(2); }
    g->front = body - bytes;
    g->data = g->map + g->front;
    memset(g->map, LAB_GUARD_BYTE, g->front);
    return (float *)(void *)g->data;
}

static int guard_intact(const guarded_t *g)
{
    for (size_t i = 0; i < g->front; ++i)
        if (g->map[i] != LAB_GUARD_BYTE) return 0;
    return 1;
}

static void guarded_free(guarded_t *g) { munmap(g->map, g->maplen); }

typedef enum {
    CONTENT_PATTERN, CONTENT_ZERO, CONTENT_ONES, CONTENT_MINUS_ONES,
    CONTENT_IDENTITY, CONTENT_RANDOM, CONTENT_LARGE, CONTENT_NEAR_MAX,
    CONTENT_OVERFLOW, CONTENT_FMA_EDGE, CONTENT_HALF_EDGE
} content_t;

static void fill_content(float *A, float *B, size_t n, content_t kind)
{
    size_t nelem = n * n;
    uint32_t seed = 0x2545F491u;
    for (size_t j = 0; j < nelem; ++j) {
        switch (kind) {
        case CONTENT_ZERO:       A[j] = 0.0f;  B[j] = 0.0f;  break;
        case CONTENT_ONES:       A[j] = 1.0f;  B[j] = 1.0f;  break;
        case CONTENT_MINUS_ONES: A[j] = -1.0f; B[j] = -1.0f; break;
        case CONTENT_IDENTITY:   /* B = I, tehát C = A */
            A[j] = (float)((int)(j % 17) - 8) / 8.0f;
            B[j] = (j / n == j % n) ? 1.0f : 0.0f;
            break;
        case CONTENT_RANDOM:     /* LCG, [-1, 1) egyenletes, rögzített mag */
            seed = seed * 1664525u + 1013904223u;
            A[j] = ((float)(seed >> 8) / (float)(1u << 24)) * 2.0f - 1.0f;
            seed = seed * 1664525u + 1013904223u;
            B[j] = ((float)(seed >> 8) / (float)(1u << 24)) * 2.0f - 1.0f;
            break;
        case CONTENT_LARGE:      /* nagyérték-teszt: n*1e12 << FLT_MAX */
            A[j] = 1.0e6f; B[j] = (j % 2) ? -1.0e6f : 1.0e6f;
            break;
        case CONTENT_NEAR_MAX:   /* szerződésen belül, a határ közelében: egynemű előjel,
                                    s_ij = n * A = 0x1.fffffcp126 <= FLT_MAX/2 */
            A[j] = (float)(0x1.fffffcp126 / (double)n); B[j] = 1.0f;
            break;
```

```
        case CONTENT_OVERFLOW:   /* szándékosan a szerződésen kívül */
            A[j] = 1.0e20f; B[j] = 1.0e20f;
            break;
        case CONTENT_FMA_EDGE:   /* n = 4: s_00 = 0x1.fffffd18p127 < FLT_MAX, mégis +Inf
                                    (V8 határ-ellenpélda); a szerződés elutasítja */
            A[j] = (j == 0) ? 0x1.fffffap127f : (j < n) ? 0x1.08p103f : 0.0f;
            B[j] = 1.0f;
            break;
        case CONTENT_HALF_EDGE:  /* n = 2: s_00 = FLT_MAX/2 + 1 > LAB_SMAX, de double-ben
                                    FLT_MAX/2 + 1 == FLT_MAX/2; a szűrő elutasítja */
            A[j] = (j == 0) ? FLT_MAX / 2 : (j == 1) ? 1.0f : 0.0f;
            B[j] = (j % n == 0) ? 1.0f : 0.0f;
            break;
        case CONTENT_PATTERN:
        default:                 /* int-kasztolás a kivonás előtt (size_t!) */
            A[j] = (float)((int)(j % 17) - 8) / 8.0f;
            B[j] = (float)((int)(j % 13) - 6) / 6.0f;
            break;
        }
    }
}

static const char *content_name[] = {
    "pattern", "zero", "ones", "minus-ones", "identity", "random", "large", "near-max",
    "overflow", "fma-edge", "half-edge"
};

/* A modul számítási modellje (a057, 3. pont): c_ij = 0-ról, k-sorrendben
   float FMA. A szerződésen kívüli esetek dokumentált elvárása ez a modell. */
static void sgemm_fma_model(size_t n, const float *A, const float *B, float *M)
{
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < n; ++j) {
            float c = 0.0f;
            for (size_t k = 0; k < n; ++k)
                c = fmaf(A[i * n + k], B[k * n + j], c);
            M[i * n + j] = c;
        }
}

/* Érvényes eset: őrzött A, B, C; C előre NaN-bájtokkal (0xFF) töltve, így a
   meg nem írt elem is FAIL. Ellenőrzés: rc, A és B változatlansága, őrbájtok,
   a bemenet osztályozása (szerződésen belül/kívül), majd szerződésen belül a
   numerikus szerződés, kívül a float-FMA modellel való bitre egyezés. */
static int run_case(size_t n, content_t kind)
{
    size_t bytes = n * n * sizeof(float);
    guarded_t gA, gB, gC;
    float *A = guarded_alloc(&gA, bytes), *B = guarded_alloc(&gB, bytes);
    float *C = guarded_alloc(&gC, bytes);
    float  *A0 = malloc(bytes ? bytes : 1), *B0 = malloc(bytes ? bytes : 1);
    double *R = malloc(n ? n * n * sizeof(double) : 1);
    float  *M = malloc(bytes ? bytes : 1);
    if (!A0 || !B0 || !R || !M) { perror("malloc"); exit(2); }
    int want_in = kind != CONTENT_OVERFLOW && kind != CONTENT_FMA_EDGE
                  && kind != CONTENT_HALF_EDGE;

    fill_content(A, B, n, kind);
    memcpy(A0, A, bytes); memcpy(B0, B, bytes);
    memset(C, 0xFF, bytes);
    int in_contract = lab_sgemm_in_contract(n, A, B);

    int rc = lab_sgemm(n, A, B, C);
```

```
    lab_sgemm_ref(n, A, B, R);

    size_t bad = 0, ninf = 0;
    const char *why = NULL;
    if (in_contract != want_in) why = "osztályozás";
    else if (rc != LAB_SGEMM_OK) why = "rc";
    else if (memcmp(A, A0, bytes) || memcmp(B, B0, bytes)) why = "bemenet módosult";
    else if (!guard_intact(&gA) || !guard_intact(&gB) || !guard_intact(&gC)) why = "őrbájt";
    else if (in_contract) {
        if (lab_sgemm_check(n, A, B, C, R, &bad)) why = "numerikus";
    } else {
        /* Szerződésen kívül nincs numerikus elfogadás; dokumentált elvárás a
           float-FMA modell bitre pontos eredménye (overflow: minden elem +Inf,
           fma-edge: a 0. sor +Inf, a többi 0). */
        sgemm_fma_model(n, A, B, M);
        for (size_t e = 0; e < n * n && !why; ++e)
            if (memcmp(&C[e], &M[e], sizeof(float))) {
                bad = e;
                why = "eltér a float-FMA modelltől";
            }
        for (size_t e = 0; e < n * n; ++e)
            ninf += isinf(C[e]) != 0;
    }
    printf("[%s] n=%4zu  rc=%2d  %s", content_name[kind], n, rc, why ? "FAIL" : "OK");
    if (!want_in && !why) printf(" (szerződésen kívül, +Inf elemek: %zu)", ninf);
    if (why && n) printf(" (%s, első hibás elem: %zu, asm=%g ref=%g)", why, bad,
                         (double)C[bad], R[bad]);
    putchar('\n');
    guarded_free(&gA); guarded_free(&gB); guarded_free(&gC);
    free(A0); free(B0); free(R); free(M);
    return why != NULL;
}

/* Érvénytelen bemenet: a C puffert (ha van) speciális bitmintákkal töltjük
   (-0.0, NaN-payload, Inf), és a teljes kimeneti tartomány bájtjait az
   őrbájtokkal együtt memcmp-pel vetjük össze a hívás előtti másolattal. */
static int run_invalid_case(size_t n, int a_null, int b_null, int c_null,
                            const char *label)
{
    static const uint32_t pat[] = { 0x80000000u, 0x7fc12345u, 0xffa00001u,
                                    0x7f800000u, 0x449a5000u, 0x00000001u };
    guarded_t g;
    float *buf = guarded_alloc(&g, 16 * sizeof(float));
    for (size_t e = 0; e < 16; ++e)
        memcpy(&buf[e], &pat[e % 6], sizeof(uint32_t));
    size_t len = g.front + 16 * sizeof(float);
    unsigned char *before = malloc(len);
    if (!before) { perror("malloc"); exit(2); }
    memcpy(before, g.map, len);

    float dummy[4] = { 1, 2, 3, 4 };
    int rc = lab_sgemm(n, a_null ? NULL : dummy, b_null ? NULL : dummy,
                       c_null ? NULL : buf);
    int unchanged = memcmp(before, g.map, len) == 0;
    int ok = rc == LAB_SGEMM_EINVAL && unchanged;
    printf("invalid[%s] n=%zu  rc=%2d  %s\n", label, n, rc, ok ? "OK" : "FAIL");
    free(before); guarded_free(&g);
    return !ok;
}

/* Az ellenőrző önellenőrzése, a vizsgált modultól függetlenül: a referencia
   float-ra kerekített eredményének át kell mennie, a szándékosan elrontott
   példányoknak pedig le kell bukniuk. */
```

```
static int run_checker_selftest(void)
{
    enum { N = 17 };
    const size_t n = N;
    float A[N * N], B[N * N], C[N * N], D[N * N];
    double R[N * N];
    size_t bad;
    fill_content(A, B, n, CONTENT_RANDOM);
    lab_sgemm_ref(n, A, B, R);
    for (size_t e = 0; e < n * n; ++e) C[e] = (float)R[e];
    int fail = lab_sgemm_check(n, A, B, C, R, &bad) != 0;
    const float inject[] = { NAN, INFINITY, -INFINITY, 0.5f };
    const char *name[] = { "NaN", "+Inf", "-Inf", "+0.5" };
    for (int t = 0; t < 4; ++t) {
        memcpy(D, C, sizeof C);
        D[5 * n + 11] = (t == 3) ? D[5 * n + 11] + inject[t] : inject[t];
        int caught = lab_sgemm_check(n, A, B, D, R, &bad) && bad == 5 * n + 11;
        printf("selftest[%s beinjektálva] %s\n", name[t],
               caught ? "lebukott: OK" : "NEM bukott le: FAIL");
        fail |= !caught;
    }
    return fail;
}

int main(void)
{
    int fail = 0;
    setvbuf(stdout, NULL, _IOLBF, 0);   /* összeomláskor is látszik az utolsó sor */
    if (!lab_fpcr_ok()) { puts("FPCR nem a szerződés szerinti (RNE, FZ=0, DN=0)"); return 2; }

    fail |= run_checker_selftest();

    /* 1. réteg: határesetek és nagyobb méretek, mintázott tartalommal. */
    const size_t sizes[] = { 0, 1, 2, 3, 5, 7, 8, 16, 17, 63, 64, 65, 128, 256 };
    for (size_t i = 0; i < sizeof sizes / sizeof sizes[0]; ++i)
        fail |= run_case(sizes[i], CONTENT_PATTERN);

    /* 2. réteg: kis méretek, további farok-/blokkesetek. */
    const size_t small[] = { 4, 6, 12 };
    for (size_t i = 0; i < sizeof small / sizeof small[0]; ++i)
        fail |= run_case(small[i], CONTENT_PATTERN);

    /* 3. réteg: hét speciális tartalom n = 32 és 64 mellett (köztük a
       FLT_MAX/2-közeli near-max), véletlen tartalom n = 128 és 256 mellett is. */
    const size_t special[] = { 32, 64 };
    for (size_t i = 0; i < 2; ++i)
        for (content_t k = CONTENT_ZERO; k <= CONTENT_NEAR_MAX; ++k)
            fail |= run_case(special[i], k);
    fail |= run_case(128, CONTENT_RANDOM);
    fail |= run_case(256, CONTENT_RANDOM);

    /* Szerződésen kívüli esetek: az osztályozásnak el kell utasítania őket,
       a kimenet dokumentált elvárása a float-FMA modell. */
    fail |= run_case(8, CONTENT_OVERFLOW);
    fail |= run_case(4, CONTENT_FMA_EDGE);
    fail |= run_case(2, CONTENT_HALF_EDGE);

    /* Érvénytelen bemenetek. n = 2^31: n*n*4 már nem fér el 64 biten. */
    fail |= run_invalid_case(1, 1, 0, 0, "A=NULL");
    fail |= run_invalid_case(1, 0, 1, 0, "B=NULL");
    fail |= run_invalid_case(1, 0, 0, 1, "C=NULL");
    fail |= run_invalid_case((size_t)1 << 31, 0, 0, 0, "n-overflow");
```

```
    puts(fail ? "TESTS: FAIL" : "TESTS: PASS");
    return fail;
}
```

15.3.4 Kidolgozott példa: a tesztkészlet futtatása

A build és futtatás az 15.2 buildláncot követi. A `qemu-aarch64` user-mode emuláció `-L` kapcsolójával adhatjuk meg a sysrootot (a `-L` értéke disztró-függő):

```
# Build: assembly-modul + referencia + ellenőrző + teszthajtó.
aarch64-linux-gnu-gcc -O2 -c lab_sgemm.S -o lab_sgemm.o
aarch64-linux-gnu-gcc -std=gnu17 -O2 -Wall -Wextra -static \
    lab_sgemm_test.c lab_sgemm_check.c lab_sgemm_ref.c lab_sgemm.o -lm -o lab_sgemm_test
readelf -lW lab_sgemm_test | awk '$1=="GNU_STACK" {print "GNU_STACK", $7}'   # elvárt: RW

# Futtatás qemu-user módban.
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab_sgemm_test; echo "exit=$?"
```

Kimenet a fő úton mérve (GNU + glibc + QEMU 8.2.2, a lektor mérése):

```
selftest[NaN beinjektálva] lebukott: OK
selftest[+Inf beinjektálva] lebukott: OK
selftest[-Inf beinjektálva] lebukott: OK
selftest[+0.5 beinjektálva] lebukott: OK
[pattern] n=   0  rc= 0  OK
[pattern] n=   1  rc= 0  OK
[pattern] n=   2  rc= 0  OK
[pattern] n=   3  rc= 0  OK
[pattern] n=   5  rc= 0  OK
[pattern] n=   7  rc= 0  OK
[pattern] n=   8  rc= 0  OK
[pattern] n=  16  rc= 0  OK
[pattern] n=  17  rc= 0  OK
[pattern] n=  63  rc= 0  OK
[pattern] n=  64  rc= 0  OK
[pattern] n=  65  rc= 0  OK
[pattern] n= 128  rc= 0  OK
[pattern] n= 256  rc= 0  OK
[pattern] n=   4  rc= 0  OK
[pattern] n=   6  rc= 0  OK
[pattern] n=  12  rc= 0  OK
[zero] n=  32  rc= 0  OK
[ones] n=  32  rc= 0  OK
[minus-ones] n=  32  rc= 0  OK
[identity] n=  32  rc= 0  OK
[random] n=  32  rc= 0  OK
[large] n=  32  rc= 0  OK
[near-max] n=  32  rc= 0  OK
[zero] n=  64  rc= 0  OK
[ones] n=  64  rc= 0  OK
[minus-ones] n=  64  rc= 0  OK
[identity] n=  64  rc= 0  OK
[random] n=  64  rc= 0  OK
[large] n=  64  rc= 0  OK
[near-max] n=  64  rc= 0  OK
[random] n= 128  rc= 0  OK
[random] n= 256  rc= 0  OK
[overflow] n=   8  rc= 0  OK (szerződésen kívül, +Inf elemek: 64)
[fma-edge] n=   4  rc= 0  OK (szerződésen kívül, +Inf elemek: 4)
[half-edge] n=   2  rc= 0  OK (szerződésen kívül, +Inf elemek: 0)
invalid[A=NULL] n=1  rc=-1  OK
invalid[B=NULL] n=1  rc=-1  OK
```

```
invalid[C=NULL] n=1  rc=-1  OK
invalid[n-overflow] n=2147483648  rc=-1  OK
TESTS: PASS
exit=0
```

A `TESTS: PASS` sor a kiértékelés egyetlen szerződéses állítása: a modul a felsorolt tesztmátrixon az 15.1 numerikus szerződése szerint egyezik a referenciával, nem ír a `C`-n kívülre, nem módosítja a bemenetet, érvényes bemenetnél `W0 = LAB_SGEMM_OK` (0, `n == 0`-ra is), érvénytelennél `W0 = LAB_SGEMM_EINVAL` (-1), és a puffer bájtra változatlan marad.

Ellenpróba: a teszt valóban elkapja a hibát. A modulon két, az ellenőrzőn egy szándékos hibával újrafuttattuk a tesztet (ugyanazzal az eszközlánccal):

Beültetett hiba	Megfigyelt eredmény
a `C`-sor nullázásának skalár farka (`str wzr, ...`) kihagyva	`[pattern] n=1 ... FAIL (numerikus, első hibás elem: 0, asm=-nan ref=1)`, majd minden nem 4-gyel osztható `n` FAIL; a régi `d > max_err` minta ezt a NaN-t nem jelezte volna
a `lab_sgemm_in_contract` egyszerű `abs_sum <= LAB_SMAX` összevetéssel, $(1+\gamma)$-tartalék nélkül	`[half-edge] n=2 ... FAIL (osztályozás, …)`: a pontos határ fölötti esetet elfogadta volna
a `lab_sgemm_in_contract` határa a régi `< FLT_MAX`	`[fma-edge] n=4 ... FAIL (osztályozás, első hibás elem: 0, asm=inf ref=3.40282e+38)`: a régi tartomány a végtelen kimenetű bemenetet szerződésen belülinek fogadta volna el
a `j`-ciklus feltétele `j + 4 <= n` helyett `j < n` (a vektoros ág a sor végén túlnyúlik)	az első érvényes esetnél írás a védőlapra: laphiba, a folyamat `SIGSEGV`-vel áll le

15.3.5 Teljesítménymérés az 13.4 sablonnal

Az 13.4 alfejezet mikrobenchmark-sablonja négy komponensből áll: *melegítő futás*, *ismételt futás*, *időmérés* (monoton forrás), és *statisztikai összegzés* (medián, átlag, interkvartilis terjedelem és kiugrószám). A sablon itt kibővül a *beviteli méretsorozat* ciklusával, hogy minden tervezett `n` értékre megkapjuk a `t_asm(n)` és `t_ref(n)` értékeket. A szóródás jellemzésére itt az IQR helyett a medián abszolút eltérést (MAD) használjuk, mert a mediánhoz ugyanazon a rendezett mintán, ugyanazzal az eljárással számolható, és kis R mellett nem igényel külön kvartilisbecslést.

A mérendő bemenetek az 15.1 szerint: `n  {16, 32, 64, 128, 256}`. Az időmérés forrása `clock_gettime(CLOCK_MONOTONIC, ...)`, mivel a `CLOCK_PROCESS_CPUTIME_ID` qemu-user módban gyakran nem ad valós processzoridőt.

A mérés **nem** a korrektségi referenciát (`lab_sgemm_ref`, `double` kimenetű) méri időben, mert az nem tisztességes `float`–`float` összehasonlítás. Az 15.1 szerint négy változatot mérünk, hogy a ciklussorrend, a fordítói vektorizálás és a kézi assembly hatása szétváljon. A két C-ciklus egy forrásban van, és két objektum készül belőle eltérő kapcsolókkal:

```
/* lab_sgemm_c.c - C összehasonlító változatok (csak mérési baseline).
 * Ugyanebből a forrásból két objektum készül, eltérő névutótaggal:
 *   -DLAB_SUFFIX=_scalar  vektorizálás tiltva (skalár kontroll)
 *   -DLAB_SUFFIX=_vec     vektorizálás engedélyezve (optimalizált C)       */
#include <stddef.h>
```

```c
#ifndef LAB_SUFFIX
#define LAB_SUFFIX _scalar
#endif
#define LAB_CAT2(a, b) a##b
#define LAB_CAT(a, b)  LAB_CAT2(a, b)

/* i-j-k: a belső ciklus B egy oszlopán lépked (n*4 bájtos ugrásokkal). */
void LAB_CAT(lab_sgemm_ijk, LAB_SUFFIX)(size_t n, const float *restrict A,
                                        const float *restrict B, float *restrict C)
{
    for (size_t i = 0; i < n; ++i)
        for (size_t j = 0; j < n; ++j) {
            float acc = 0.0f;
            for (size_t k = 0; k < n; ++k)
                acc += A[i * n + k] * B[k * n + j];
            C[i * n + j] = acc;
        }
}

/* i-k-j: ugyanaz a ciklussorrend és k-sorrend, mint a lab_sgemm.S-ben;
   a belső ciklus B és C egy-egy sorát folytonosan járja. */
void LAB_CAT(lab_sgemm_ikj, LAB_SUFFIX)(size_t n, const float *restrict A,
                                        const float *restrict B, float *restrict C)
{
    for (size_t i = 0; i < n; ++i) {
        float *c = C + i * n;
        for (size_t j = 0; j < n; ++j)
            c[j] = 0.0f;
        for (size_t k = 0; k < n; ++k) {
            const float a = A[i * n + k];
            const float *b = B + k * n;
            for (size_t j = 0; j < n; ++j)
                c[j] += a * b[j];
        }
    }
}
```

A mérőprogram az 11.4 mintáját követi: az ellenőrzés a mért blokkon **kívül**, előtte fut. Minden változat eredményét ugyanaz a numerikus szerződés ellenőrzi, és ha bármelyik hibás, a program nem mér. Ezután változatonként melegítés következik, majd a mért futások váltakozó sorrendben, hogy a sorrendhatás ne egy változatot terheljen:

```c
/* lab_sgemm_bench.c - négy változat mérése: naiv i-j-k C, átrendezett i-k-j C,
   vektorizált i-k-j C és a lab_sgemm assembly modul. Használat:
   ./lab_sgemm_bench [n ...]   (alapértelmezés: 16 32 64 128 256; 1 <= n <= 256) */
#define _POSIX_C_SOURCE 200809L
#include <errno.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include "lab_sgemm.h"
#include "lab_sgemm_check.h"

#define LAB_W 3    /* melegítő futás változatonként */
#define LAB_R 11   /* mért futás változatonként (páratlan: egyértelmű medián) */

typedef void (*cfun_t)(size_t, const float *restrict, const float *restrict, float *restrict);
void lab_sgemm_ijk_scalar(size_t, const float *restrict, const float *restrict,
                          float *restrict);
void lab_sgemm_ikj_scalar(size_t, const float *restrict, const float *restrict,
                          float *restrict);
```

```c
void lab_sgemm_ikj_vec(size_t, const float *restrict, const float *restrict, float *restrict);

static int asm_wrapper(size_t n, const float *A, const float *B, float *C)
{
    return lab_sgemm(n, A, B, C);
}

enum { V_IJK, V_IKJ, V_IKJ_VEC, V_ASM, V_COUNT };
static const char *vname[V_COUNT] = { "C ijk skalár", "C ikj skalár", "C ikj vektor", "asm" };
static const cfun_t cfun[V_COUNT] = { lab_sgemm_ijk_scalar, lab_sgemm_ikj_scalar,
                                      lab_sgemm_ikj_vec, NULL };

static void run(int v, size_t n, const float *A, const float *B, float *C)
{
    if (v == V_ASM) {
        if (asm_wrapper(n, A, B, C) != LAB_SGEMM_OK) {
            fputs("lab_sgemm hiba\n", stderr);
            exit(1);
        }
    } else {
        cfun[v](n, A, B, C);
    }
}

static double now_ms(void)
{
    struct timespec ts;
    if (clock_gettime(CLOCK_MONOTONIC, &ts)) { perror("clock_gettime"); exit(2); }
    return (double)ts.tv_sec * 1e3 + (double)ts.tv_nsec * 1e-6;
}

static int cmp_double(const void *a, const void *b)
{
    double x = *(const double *)a, y = *(const double *)b;
    return (x > y) - (x < y);
}

static double median(double *v, size_t m)       /* rendezi v-t; m páratlan */
{
    qsort(v, m, sizeof v[0], cmp_double);
    return v[m / 2];
}

/* A mérés a numerikus szerződés tartományában fut: 1 <= n <= LAB_TOL_NMAX.
   Nagyobb n csak új tűrésszerződéssel (LAB_TOL_NMAX és LAB_RTOL együtt) választható.
   E határ mellett a lenti size_t szorzások egyike sem csordulhat túl: */
_Static_assert((size_t)LAB_TOL_NMAX * LAB_TOL_NMAX <= SIZE_MAX / sizeof(double),
               "n*n*sizeof(double) túlcsordulna");

/* Egy parancssori méret: csak decimális számjegyek, teljesen elfogyasztva,
   1 <= n <= LAB_TOL_NMAX. Hibánál 0. (A strtoul magában elfogadná a vezető
   szóközt és a '-' előjelet, ezért az első karaktert külön vizsgáljuk.) */
static size_t parse_n(const char *s)
{
    char *end;
    if (*s < '0' || *s > '9')
        return 0;                                   /* üres, negatív, nem szám */
    errno = 0;
    unsigned long v = strtoul(s, &end, 10);
    if (errno != 0 || *end != '\0' || v < 1 || v > LAB_TOL_NMAX)
        return 0;                                   /* túl nagy, maradék karakter, 0 */
    return (size_t)v;
}
```

```
int main(int argc, char **argv)
{
    size_t def[] = { 16, 32, 64, 128, 256 }, sizes[16];
    size_t ns = 0;
    if (argc - 1 > 16) { fputs("legfeljebb 16 méret adható meg\n", stderr); return 64; }
    /* minden ellenőrzés az első allokáció előtt */
    for (int a = 1; a < argc; ++a) {
        size_t n = parse_n(argv[a]);
        if (n == 0) {
            fprintf(stderr, "használat: %s [n ...], ahol 1 <= n <= %d; hibás méret: \"%s\"\n",
                    argv[0], LAB_TOL_NMAX, argv[a]);
            return 64;                              /* használati hiba (EX_USAGE) */
        }
        sizes[ns++] = n;
    }
    if (!ns) { memcpy(sizes, def, sizeof def); ns = sizeof def / sizeof def[0]; }
    if (!lab_fpcr_ok()) { puts("FPCR nem a szerződés szerinti"); return 2; }

    for (size_t s = 0; s < ns; ++s) {
        size_t n = sizes[s], ne = n * n;
        float *A = malloc(ne * sizeof *A), *B = malloc(ne * sizeof *B);
        float *C = malloc(ne * sizeof *C);
        double *R = malloc(ne * sizeof *R);
        if (!A || !B || !C || !R) { perror("malloc"); return 1; }
        for (size_t j = 0; j < ne; ++j) {           /* int-kasztolás: size_t! */
            A[j] = (float)((int)(j % 41) - 20) / 20.0f;
            B[j] = (float)((int)(j % 53) - 26) / 26.0f;
        }

        /* 1. Ellenőrzés a mért blokk ELŐTT: minden változat a szerződés szerint helyes. */
        lab_sgemm_ref(n, A, B, R);
        for (int v = 0; v < V_COUNT; ++v) {
            size_t bad;
            memset(C, 0xFF, ne * sizeof *C);        /* NaN-bájtok: a kihagyott elem is FAIL */
            run(v, n, A, B, C);
            if (lab_sgemm_check(n, A, B, C, R, &bad)) {
                printf("n=%zu %s: HIBÁS eredmény a(z) %zu. elemnél, nem mérünk\n",
                       n, vname[v], bad);
                return 1;
            }
        }

        /* 2. Melegítés, majd mérés váltakozó sorrendben (a sorrendhatás ellen). */
        double t[V_COUNT][LAB_R];
        for (int w = 0; w < LAB_W; ++w)
            for (int v = 0; v < V_COUNT; ++v) run(v, n, A, B, C);
        for (int r = 0; r < LAB_R; ++r)
            for (int q = 0; q < V_COUNT; ++q) {
                int v = (r % 2) ? V_COUNT - 1 - q : q;
                double t0 = now_ms();
                run(v, n, A, B, C);
                t[v][r] = now_ms() - t0;
            }

        double med[V_COUNT];
        for (int v = 0; v < V_COUNT; ++v) {
            double dev[LAB_R];
            med[v] = median(t[v], LAB_R);
            for (int r = 0; r < LAB_R; ++r)
                dev[r] = t[v][r] > med[v] ? t[v][r] - med[v] : med[v] - t[v][r];
            double mad = median(dev, LAB_R);
            double gflops = med[v] > 0 ? 2.0 * n * n * n / (med[v] * 1e6) : 0.0;
```

```
            printf("n=%4zu %-13s medián=%10.4f ms  MAD=%8.4f ms  %8.3f GFLOP/s\n",
                   n, vname[v], med[v], mad, gflops);
        }
        if (med[V_IKJ] > 0 && med[V_IKJ_VEC] > 0 && med[V_ASM] > 0)
            printf("n=%4zu gyorsulás: sorrend(ijk/ikj)=%.2f  C-vektor(ikj/ikj_vec)=%.2f\n"
                   "       asm(ikj_vec/asm)=%.2f  összesen(ijk/asm)=%.2f\n", n,
                   med[V_IJK] / med[V_IKJ], med[V_IKJ] / med[V_IKJ_VEC],
                   med[V_IKJ_VEC] / med[V_ASM], med[V_IJK] / med[V_ASM]);
        free(A); free(B); free(C); free(R);
    }
    return 0;
}

# A négy változat: azonos cél (-march=armv8-a: FP és SIMD alapból benne) és FP-modell.
CF="-std=gnu17 -march=armv8-a -ffp-contract=fast -Wall -Wextra"
aarch64-linux-gnu-gcc --version | head -1                    # a jegyzőkönyvbe
aarch64-linux-gnu-gcc $CF -O2 -fno-tree-vectorize -DLAB_SUFFIX=_scalar \
    -c lab_sgemm_c.c -o lab_sgemm_c_scalar.o
aarch64-linux-gnu-gcc $CF -O3 -fopt-info-vec-optimized -DLAB_SUFFIX=_vec \
    -c lab_sgemm_c.c -o lab_sgemm_c_vec.o 2> vec-report.txt     # vektorizációs riport
aarch64-linux-gnu-gcc $CF -O2 -static -o lab_sgemm_bench \
    lab_sgemm_bench.c lab_sgemm_check.c lab_sgemm_ref.c \
    lab_sgemm_c_scalar.o lab_sgemm_c_vec.o lab_sgemm.o -lm

# A végső bináris ellenőrzése: hol van vektoros fmla?
for f in lab_sgemm_ijk_scalar lab_sgemm_ikj_scalar lab_sgemm_ikj_vec lab_sgemm; do
    printf '%-22s fmla: ' "$f"
    aarch64-linux-gnu-objdump -d --disassemble="$f" lab_sgemm_bench | grep -c 'fmla'
done
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab_sgemm_bench
```

A mérőprogram a parancssori méreteket a `parse_n`-nel ellenőrzi, minden allokáció és referenciahívás előtt. Csak `1 n LAB_TOL_NMAX` (256) fogadható el, mert a numerikus tűrés (15.1) csak eddig érvényes; e határ mellett a `size_t` szorzások sem csordulhatnak túl (`_Static_assert`). Az üres, a negatív, a nem szám, a maradék karakterrel végződő, a 256-nál nagyobb és a 2^{32}-es argumentum, valamint a 16-nál több megadott méret 64-es használati hibával áll le. A helyettesítő eszközlánccal mindegyiket kipróbáltuk. Az alapértelmezett méretsor változatlan. Az `n = 0` funkcionális próbája a korrektségi tesztkészletben marad, a mérésben nincs értelme.

Az `objdump`-ellenőrzés elvárt eredménye: a két `_scalar` függvényben 0 vektoros `fmla`, az `ikj_vec`-ben és a `lab_sgemm`-ben legalább 1. Ha az `ikj_scalar`-ban mégis van, a vektorizálás tiltása nem hatott, és a 2 → 3 lépés hatása nem választható le. A fő úton (GCC + QEMU 8.2.2) ellenőrizve: az i-k-j belső ciklus 16 bájtos vektorokkal vektorizálódott, az i-j-k redukciós ciklus `-O3` mellett sem (átrendezés `-ffast-math` nélkül nem megengedett), a skalár változatokban csak `fmadd` van.

A mérés konvenciói (rögzítendők a jegyzőkönyvben):

- **méretegység:** az idő milliszekundumban; a gyorsulás mértékegység nélküli arány.
- **statisztika:** medián és MAD változatonként, $W = 3$ melegítő és $R = 11$ mért futásból, az 15.1 szerint.
- **környezet:** host vagy QEMU-verzió és `-cpu` modell, GCC-verzió, kapcsolók, a vektorizációs riport és az `objdump`-ellenőrzés eredménye.
- **bevitel:** fix, szorosan csomagolt (nincs padding/stride) `n × n float` mátrixok.

Az illusztratív kimenet (hipotetikus, nem mért érték; a formátumot mutatja `n = 256`-ra):

```
n= 256 C ijk skalár medián=   91.1230 ms  MAD=  0.4100 ms     0.368 GFLOP/s
n= 256 C ikj skalár medián=   47.6000 ms  MAD=  0.2200 ms     0.705 GFLOP/s
n= 256 C ikj vektor  medián=   18.9500 ms  MAD=  0.1100 ms     1.771 GFLOP/s
```

```
n= 256 asm           medián=   20.8750 ms  MAD=  0.1300 ms     1.607 GFLOP/s
n= 256 gyorsulás: sorrend(ijk/ikj)=1.91  C-vektor(ikj/ikj_vec)=2.51
       asm(ikj_vec/asm)=0.91  összesen(ijk/asm)=4.37
```

A számítási komplexitás `O(n³)`, ezért `n` kétszerezése nyolcszoros időt várna; ezt az alábbi táblázat naiv oszlopa illusztrálja.

15.3.6 Számított gyorsulás és az elfogadási feltétel

Az illusztratív (hipotetikus) adatokon a teljes, naiv C-hez mért gyorsulás `n = 256`-nál 4,37. Ez azonban **három hatás szorzata**, és a felbontás mutatja meg, mi minek köszönhető:

$$\frac{t_{ijk}}{t_{asm}} = \underbrace{\frac{t_{ijk}}{t_{ikj}}}_{\text{sorrend: } 1{,}91} \cdot \underbrace{\frac{t_{ikj}}{t_{ikj,vec}}}_{\text{C-vektorizálás: } 2{,}51} \cdot \underbrace{\frac{t_{ikj,vec}}{t_{asm}}}_{\text{kézi asm: } 0{,}91} \approx 4{,}37$$

Az 15.1 natív AArch64-re szóló elfogadási küszöbe (`n  64` mellett `t_asm  0,5 · t_ijk`) a hipotetikus, natív nagyságrendű adatokon teljesül; a QEMU-mérés ezt nem igazolja (lásd a mért táblázatot). A felbontás viszont azt is mutatja, hogy ebben a hipotetikus környezetben a nyereség a ciklusok átrendezéséből és a vektorizálásból jön, és mindkettőt a C-fordító is elérte. A kézi modul a vektorizált C-nél kissé **lassabb** (0,91×), ami egyetlen akkumulátorlánccal és kibontás nélküli belső ciklussal hihető is (lásd az 15.2 2. feladatát). Ez nem a mérés hibája, hanem a projekt egyik legfontosabb, jegyzőkönyvbe kerülő eredménye: a modul korrektségi szempontból elfogadható, de a natív teljesítmény-küszöb igazolása nyitott, és a „kézi assembly a gyorsabb” állítást a mért adat nem támasztja alá. Az ilyen megállapítás csak a mért környezetre szól; más magon vagy QEMU-verzión mindhárom tényező más lehet.

Az elfogadási feltétel **két további megkötéssel** egészül ki:

1. **stabilitás:** minden változat MAD-ja legfeljebb a mediánja néhány százaléka legyen. Ha a `t_asm` szórása relatíve nagyobb, mint a C-változatoké, az arra utalhat, hogy a modul valamely `n`-nél cache-viselkedés vagy igazítás miatt instabillá válik.
2. **skálázhatóság:** a részgyorsulások `n`-nel közel állandók, vagy változásuk magyarázható. Ha például a `sorrend` tényező `n`-nel nő, az a `B` oszlopok bejárásának cache-hatása (lásd az 13.1 alfejezetet), nem az assemblyé.

Ha a küszöb nem teljesül, az 15.4 visszacsatolási módszere szerint egyszerre egy változtatással kell kísérletezni (például két független `FMLA`-lánc, 8 elemes belső ciklus), majd a mérést és a teljes tesztkészletet megismételni, hogy a korrektség visszaesése kizárható legyen.

15.3.7 Numerikus ábra: assembly-modul vs. C-referencia

Az ábra mellé a numerikus táblázat, amely a forrás Markdownból is olvasható (minden érték hipotetikus):

n	t_ijk [ms]	t_ikj [ms]	t_ikj,vec [ms]	t_asm [ms]	sorrend	C-vektor	kézi asm	összesen	t_ijk köbös skálázás
32	0.1781	0.0950	0.0372	0.0412	1.87	2.55	0.90	4.32	—
64	1.4218	0.7420	0.2890	0.3256	1.92	2.57	0.89	4.37	7.98×

n	t_ijk [ms]	t_ikj [ms]	t_ikj,vec [ms]	t_asm [ms]	sorrend	C-vektor	kézi asm	össze-sen	t_ijk köbös skálá-zás
128	11.3840	5.9300	2.3400	2.6120	1.92	2.53	0.90	4.36	8.01×
256	91.1230	47.6000	18.9500	20.8750	1.91	2.51	0.91	4.37	8.00×

Az értékek **illusztratívak, kitaláltak**, nem tényleges QEMU- vagy natív mérés eredményei; a natív AArch64 hardverre várt nagyságrendet mutatják. A hallgató laborja ezeket a saját futtatása során kapott értékekkel tölti fel. Ha az adatok fizikai AArch64 hoston (Raspberry Pi 4/5) készülnek, az 3.3 jelölése szerint ezt a táblázat **fejlécében** külön fel kell tüntetni (pl. „natív, RPi 4”).

15.3.7.1 Mért eredmény a QEMU fő úton

A következő táblázat a könyv saját parancsaival, GNU eszközlánccal és `qemu-aarch64` 8.2.2-vel, x86-64 hoston mért értékeket mutat (két független futás mediánja, `W = 3`, `R = 11`; a jelölés `mérve (GNU + QEMU 8.2.2)`):

n	ijk [ms]	ikj [ms]	ikj_vec [ms]	asm [ms]	sorrend	C-vektor	kézi asm
64	1.81	1.98	1.44	1.61	0.91	1.38	0.89
128	16.4	17.0	12.2	13.4	0.96	1.39	0.91
256	122	130	96.7	131	0.94	1.34	0.74

A `sorrend` (ijk/ikj) 0.84–0.98, a C-vektorizálás 1.33–1.40, a kézi assembly 0.74–0.93 közé esik. Az `objdump` a könyv szerinti `fmla`-számot adja (0 / 0 / 5 / 1), a GCC riportja „loop vectorized using 16 byte vectors”. A natív AArch64-re várt 2× küszöböt a QEMU-mérés nem teljesíti: a ciklus-átrendezés QEMU alatt nem hoz nyereséget, mert a host cache a 256×256-os mátrixot is tartja, és a TCG nem modellezi a cél hardver cache- és pipeline-viselkedését. A táblázat ezért a QEMU-környezetre érvényes; natív hardveren a mérés megismételendő.

15.3.8 QEMU-korlát és fizikai host

Az 13.4 és a verzióprofil szabályai szerint qemu-user módban nincs garantált, hardverhű abszolút ciklusszám. Ennek három következménye van a mérési eredményre:

- **abszolút idő:** a qemu-user JIT-tel emulált `t_asm`, `t_ref` értékek nagyságrendileg lassabbak a fizikai AArch64 processzoron mérteknél; önmagukban nem hordoznak információt a valódi ciklusszámról.
- **relatív arány:** a `speedup(n)` arány *irányában* robusztusnak tekinthető, mert mindkét oldalt ugyanaz az emulációs réteg terheli. Az arány azonban **nem azonos** a fizikai hoston mérhető aránnyal — a NEON-utasítások JIT-fordítása és a memóriahozzáférés-emuláció más súlyozást adhat.
- **hiányzó kemény tényezők:** qemu-user módban a vendégkód a host cache-ével és prediktorával fut, ezért a mért arány a hostra jellemző; a célhardverre vonatkozó következtetéshez natív AArch64 mérés kell. Az ezekre érzékeny méretek (`n = 128, 256`) gyorsulási értéke fizikai hoston szisztematikusan eltérhet.

Ezért a jegyzőkönyvben kötelező megadni a futtató környezetet:

Mező	qemu-user eset	natív AArch64 eset
Emulátor / host	`qemu-aarch64 -L …`	Raspberry Pi 4/5 64-bit OS
Abszolút idő jelentése	relatív, illusztratív	relatív és abszolút (ciklus közeli)
Cache-viselkedés	nem modellezett	valós
Branch predictor	nem modellezett	valós
Eredmény használhatósága	tendencia, teszt korrektség	kiadási döntés alapja
Jelölés a táblázatban	„emulált, qemu-user”	„mért, natív host”

Ha sem qemu, sem fizikai host nem áll rendelkezésre a hallgató laborjában, az alfejezet tanulsága akkor is érvényes: a **korrektségi tesztkészlet** és a **mérési keretrendszer** forráskód szinten reprodukálható, és bármely későbbi hardveren azonnal futtatható.

15.3.9 Tipikus hibák és ellenpróbák

- **Tünet:** a tesztkészlet `FAIL (numerikus, …)`-et jelez `n = 4` vagy `n = 8` esetén, de `n = 2`, `n = 16` esetén OK.
  - *Hipotézis:* a NEON blokk belsejében a sávokon túlnyúló `FMLA` írja felül a `C` következő sorát, vagy a `DUP` konstans nem a helyes lane-t tölti fel a `k` index fordulójakor.
  - *Adatgyűjtés:* `gdb-multiarch` egy `watch` breakpointtal a `C` érintett bájtjára (az 11.1 szerint), majd `objdump -d lab_sgemm.o` az `FMLA` operandus-sorrend és a `j`-ciklus vektoros határának (`j + 4  n`) ellenőrzésére.
  - *Egy változtatás:* a blokk `k` iterációjának felső határát `n & ~3`-ra szorítani, és a fennmaradó `n % 4` lépést a skalár farokra bízni.
- **Tünet:** a mérés eredménye azonos `t_asm` és `t_ref` értéket ad.
  - *Hipotézis:* egy régebbi `lab_sgemm.o` került a linkbe (lásd 12.5, 3. feladat). Ellenőrzés: `nm lab_sgemm_test | grep lab_sgemm`.
  - *Egy változtatás:* a `make` újrafuttatása tiszta könyvtárból, majd újramérés.
- **Tünet:** a skalárnak szánt i-k-j változat ugyanolyan gyors, mint a vektorizált, ezért a „C-vektorizálás” tényező 1 körüli.
  - *Hipotézis:* a vektorizálás tiltása nem hatott: hiányzik a `-fno-tree-vectorize`, vagy egy későbbi kapcsoló (például `-O3`) visszakapcsolta. Az `.S` fájlra az `-O` szint nincs hatással, ezért ott nem kell keresni.
  - *Adatgyűjtés:* `aarch64-linux-gnu-objdump -d --disassemble=lab_sgemm_ikj_scalar lab_sgemm_bench | grep -c fmla` a **végső** binárison.
  - *Egy változtatás:* a skalár objektum fordítási sorának javítása, újramérés.
- **Tünet:** a kézi assembly nem gyorsabb a vektorizált C-nél.
  - *Hipotézis:* ez nem feltétlenül hiba. A fordító ugyanazt a ciklussorrendet és vektorizálást érte el, és kibontással akár több független láncot is futtat.
  - *Adatgyűjtés:* a `-fopt-info-vec-optimized` riport és az `ikj_vec` disassemblyje (vektorszélesség, kibontás).
  - *Egy változtatás:* a modulban két független `FMLA`-lánc (15.2 2. feladat), majd újramérés; ha így sem gyorsabb, a jegyzőkönyv ezt rögzíti.

15.3.10 Feladatok

1. feladat (magyarázó, középszint — cél: fejezetek összekapcsolása: 15.2 CBZ/umulh-határellenőrzés). A fenti tesztkészlet `A`, `B` és `C` egyenkénti `NULL`-esetét és az `n = 2^31` túlcsordulási határt már lefedi. Terjeszd ki a fedettséget **összetett** érvénytelen esetekre is — amelyekben egyszerre több feltétel sérül, pl. `A` és `C` egyszerre `NULL`, vagy `B == NULL` és `n = SIZE_MAX` egyszerre —, és indokold, hogy ezekben az esetekben miért elegendő egyetlen `EINVAL`-visszatérés annak eldöntésére, hogy a határellenőrzés helyesen, a mag végrehajtása *előtt* lezárult.

2. (implementációs, középszint — cél: rutinépítés) Írj olyan `bench_scaling` függvényt, amely `n = 16, 32, 64, 128, 256, 512` értékekre megméri a `t_ref / n³` arányt (idő köbös egységre jutó nagyságát), és jelezze, ha ez az arány egy `n`-küszöb fölött 20%-ot változik. Ez a normalizált metrika jobban láthatóvá teszi az `O(n³)` skálázást, mint a nyers időérték. Ehhez külön, ellenőrzés nélküli időmérő függvényt írj, mert a numerikus tűrészerződés `n 256`-ig érvényes, és a mérőprogram a 256-nál nagyobb méreteket 64-es használati hibával elutasítja.

3. feladat (hibakeresési/mérési, elmélyült — cél: fejezetek összekapcsolása: 13.2 pipeline- és adatfüggőségi modell). A mérést futtasd le kétszer: `qemu-aarch64` alatt és — ha elérhető — natív AArch64 hoston. Mindkét futtatásból készíts táblázatot, és adj rövid elemzést arról, hogy a `speedup` milyen mértékben tér el a két környezet között. Külön nevezd meg a qemu-korlátozásból fakadó eltéréseket (cache, prefetcher, JIT-overhead), és kösd össze az `FMLA`-lánc esetleges adatfüggőségi akadályaival.

15.3.11 Megoldások

1. megoldás — Segítség. Nézd meg az 15.2 határellenőrzési ágát: a négy feltételt (NULL×3, túlcsordulás) SOROS `CBZ`/`CBNZ` vizsgálja. Ha bármelyik sérül, hova ugrik a vezérlés, és számít-e, hogy közben MÁS feltétel is sérül-e?

1. megoldás — Teljes megoldás. Az 15.2 `CBZ`/`umulh`-alapú határellenőrzése a négy feltételt (`A`/`B`/`C` `NULL`, `n*n*4` túlcsordulás) egymás után, soros `CBZ`/`CBNZ` ágakkal vizsgálja, és **bármelyik** sérülésnél azonnal `.Linvalid`-ra ugrik, mielőtt a mag egyetlen ciklusa is lefutna. Ebből következik, hogy összetett esetekben (pl. `A == NULL` **és** `C == NULL` egyszerre, vagy `B == NULL` **és** `n == SIZE_MAX` egyszerre) a visszatérési érték ugyanaz az `EINVAL`, függetlenül attól, melyik feltétel „sül el" elsőként a soros ellenőrzésben — a híváskor **csak egy** feltétel sérülésének ellenőrzése is elég lenne a korrektség bizonyításához, az összetett eset tehát nem egy *új* végrehajtási ágat gyakorol be, hanem azt igazolja, hogy a sorrendben *korábbi* ellenőrzés nem csúszik át hibásan a *későbbi*, akár érvénytelen bemeneten. Teszt: `run_invalid_case`-hez hasonló hívás, ahol egyszerre két paraméter hibás; az elvárt eredmény ugyanaz `W0 == LAB_SGEMM_EINVAL`, és — ahol `C` nem `NULL` — a tartalma változatlan marad, pontosan úgy, mint az egyszerű esetekben.

2. megoldás — Segítség. A köbös tag (n^3) dominanciáját úgy láthatod meg, ha a mért időt ELOSZTOD n^3-nel — ha a hányados nagy n-ekre konstanshoz tart, az `O(n³)` skálázást igazolja.

2. megoldás — Teljes megoldás. A normalizált metrika lényege:

```
double norm = t_ref / ((double)n * n * n);
```

A várt viselkedés: `norm(n)` konstans (`n 32` esetén), mert a köbös tag dominál. Az implementáció a futásidőt egy, a mérőprogram mintájára írt medián-mérő függvénnyel (`t_median_ms`: melegítés, R mért futás, medián) kapja, és a `norm` értéket a szomszédos `n`-ek között relatív eltérésként hasonlítja össze:

```
double prev = -1.0;
for (int i = 0; i < N; ++i) {
    double t = t_median_ms(lab_sgemm_ijk_scalar, sizes[i], A, B, C);
    double norm = t / ((double)sizes[i]*sizes[i]*sizes[i]);
    if (prev > 0 && fabs(norm - prev) / prev > 0.20)
        printf("FIGYELEM: n=%zu, elteres=%.1f%%\n", sizes[i],
               100.0 * fabs(norm - prev) / prev);
    prev = norm;
}
```

Az indoklás: kis `n`-nél a konstans költség (hívás, határellenőrzés, alloc) dominálhat, ezért a `norm` csökken; a nagy `n`-ek felé a `norm` stabilizálódik. Ha nem stabilizálódik, az vagy a memóriahozzáférés-minta, vagy a határellenőrzés hibáját mutatja.

3. megoldás — Segítség. Gondold át 13.2 alapján: ha az `FMLA`-lánc cikluson átívelő függőséget alkot (egyetlen akkumulátor), ez hogyan korlátozza a csúcs-sávkihasználtságot QEMU alatt és natív hoston egyaránt — és melyik oldalon várnál nagyobb hatást?

3. megoldás — Teljes megoldás. Az elvárt tendencia: `qemu-user` alatt az abszolút idők nagyobbak (a JIT és a memóriaszimuláció miatt), de a relatív `speedup` esetleg *kisebb*, mert a JIT a NEON-vektorizációt nem feltétlenül olyan hatékonyan fordítja, mint egy valódi hardveres pipeline. A fizikai hoston a négy sáv nem jelent automatikusan négyszeres részgyorsulást: a tényleges s a memóriaeléréstől, az `FMLA`-lánc függőségétől (lásd 13.2) és a kibontástól függ, ezért mérni kell, nem a sávszámból levezetni. Az elemzésben külön kell szerepelnie: (a) abszolút idő eltérése — magyarázható JIT-overhead-del; (b) arány eltérése — magyarázható a cache és prefetcher hiányával qemu alatt; (c) a QEMU a vendég elágazásait host-elágazássá fordítja, ezért a host prediktora valós hatást okoz; a mért arány a hostra jellemző, nem a cél Arm-magra.

15.3.12 Rövid összefoglaló

A mérés és validáció az 15.2 modult tesztelhetővé és elfogadhatóvá teszi. A korrektségi tesztkészlet az 15.1 numerikus szerződése szerinti referenciaegyezést, a memóriaszerződést és a NaN-biztos ellenőrző működését is igazolja. A mikrobenchmark négy változatot mér, és a teljes gyorsulást sorrendi, C-vektorizálási és kézi assembly-tényezőre bontja. Az elfogadási küszöb kiértékelése csak a jelölt futtatási környezetben érvényes. A qemu-user eredményt relatív tendenciaként, a natív AArch64 host eredményt kiadási döntés alapjaként kell kezelni.

Továbbhasznált definíciók:

- **referenciaegyezés:** minden elem véges, és `|c - ref| <= atol + rtol * s_ij` a `double` pontosságú C referenciával, az `n × n` tartomány minden elemére.
- **gyorsulásfelbontás:** `t_ijk / t_asm = (t_ijk / t_ikj) · (t_ikj / t_ikj_vec) · (t_ikj_vec / t_asm)`; a három tényező a ciklussorrend, a fordítói vektorizálás és a kézi assembly hatása.
- **elfogadási küszöb:** az 15.1 által rögzített feltétel (rögzített környezetben, `n  64` mellett `t_asm  0,5 · t_ijk`).
- **normalizált idő:** `t / n³`, amely megmutatja, hogy a mért idő a köbös komplexitású várható görbén halad-e.

15.3.13 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm A-profile Architecture Reference Manual — `FMLA`/`FMADD` szemantika, NEON sávműveletek, `UMULH`, `CBZ` elsődleges forrása. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU Compiler Collection kézikönyv — `-O2`, `-fno-tree-vectorize`, statikus linkelés, cross-toolchain konvenciók. **Tartalmilag ellenőrizve**: a legújabb listázott kiadás GCC 16.2.
- GNU Binutils `as` kézikönyv — szimbólumnevek, `.type`, `.global` direktívák, szekciókezelés. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak:

- A qemu-user `-L` sysroot elérési útja disztró-verziótól függ; a konkrét hely kitöltése az adott Ubuntu 24.04 LTS csomagolástól függ, és itt nem ellenőrzött.
- A `clock_gettime(CLOCK_MONOTONIC, ...)` felbontása qemu-user módban rendszerint mikroszekundum körüli; az itt bemutatott `ms`-alapú mérés ezzel konzisztens, de a nanoszekundumos felbontást nem garantálja.
- Az `n = 256` mérethez tartozó példaidők illusztratívak; a valós értékek qemu-verziótól, gazda CPU-tól, cache-állapottól és a kernel scheduler-viselkedésétől függően széles sávban mozoghatnak. Aktuális benchmark-adat nem ellenőrzött.
- A fizikai Raspberry Pi 4/5 hoston mérhető `speedup` nagyságrendje és stabilitása az adott processzor- és memória-konfigurációtól függ; ezt a könyv nem állítja számszerűen, és a hallgató laborjában mérendő.
- A `-fno-tree-vectorize` és a `-fopt-info-vec-optimized` GCC-verziónként eltérően viselkedhet; ezért a vektorizáció jelenlétét és hiányát a végső binárison, `objdump`-dal kell ellenőrizni.
- A tesztkészletet a fő úton (GNU + glibc + QEMU 8.2.2) futtattuk; a közölt kimenet ebből származik, a beültetett hibák ellenpróbájával együtt (a `fma-edge` bemenet $+\infty$ részösszegét natív C17 `fmaf`-fal is reprodukáltuk: `0x1.fffffap127`, `0x1.fffffcp127`, `0x1.fffffep127`, majd `inf`). A mérőprogram futását szintén a fő úton ellenőriztük; a hipotetikus táblázat értékei továbbra is szemléltető példák, nem mért adatok. Natív AArch64 hardveren nem futtattuk.

15.4 Visszacsatolás és auditmódszer

15.4.1 Motiváció: a mérés önmagában nem zárja le a projektet

Az 15.3 alfejezetben a mátrixszorzó assembly modul korrektségi szempontból teljesítette az elfogadási feltételt, a kimenet az 15.1 numerikus szerződése szerint egyezett a C referenciával, és a gyorsulás tényezőire bontva is dokumentáltuk. A natív AArch64-re előírt 2× teljesítménykküszöb QEMU alatt nem igazolható; ott a mért összesített arány 0,88–1,23×. A mérnöki munka azonban ezzel nem fejeződik be. Egy assembly modul ugyanis **csendes hibákat** hordozhat: a regiszter-ütközés, a hiányos farokkezelés vagy a stack-igazítás megsértése nem feltétlenül okoz azonnali összeomlást. A hiba csak bizonyos `n` értékeknél, bizonyos regiszterkiosztásnál vagy egy másik fordítógenerálta hívói környezetben jelentkezik. A projekt lezárásához ezért nem elég „lefutott, és jó lett a szám” — strukturált, ismételhető **auditra** van szükség.

A motiváció kettős. Egyrészt **technikai**: az assembly modul helyességét és hordozhatóságát csak rendszerszintű ellenőrzéssel lehet igazolni. Másrészt **módszertani**: a hallgatónak olyan önellenőrzési folyamatot kell elsajátítania, amelyet később csapatban, kódreview-ban és a II. rész bare-metal firmware-fejlesztésében is alkalmazni tud. Az audit nem azonos a hibakereséssel

(debugging): míg a hibakeresés egy ismert tünet okát keresi, az audit **megelőző** jellegű — a rendszer állapotát hasonlítja össze egy előre rögzített követelménylistával.

Ebben az alfejezetben egy négy pillérű auditchecklistet építünk fel, végigvisszük egy konkrét mátrixszorzó modulon, kitérünk a mérési eredmény újraellenőrzésére, a biztonsági önellenőrzésre, a csapat-kódreview reakcióira, és lezárásként megmutatjuk, mi marad ebből a fegyelemből a Cortex-M bare-metal világában.

15.4.2 Definíció: auditchecklist és önaudit

Auditchecklist (audit checklist): előre rögzített, pontokba szedett követelmény- és ellenőrzéslista, amely egy szoftvermodul vagy firmware auditalapját adja. Minden pont tartalmazza az ellenőrzési célt, a vizsgálati módszert, az elvárt eredményt és a lehetséges hiba tünetét. A checklist célja, hogy az audit **reprodukálható** legyen: ugyanaz a lista ugyanarra a modulra ugyanazt az eredményt adja, függetlenül attól, ki végzi.

Önaudit (self-audit): a fejlesztő saját maga által végzett strukturált audit, amelynek célja a projekt lezárása előtti hiányosságok feltárása. Az önaudit nem helyettesíti a független kódreview-t, de csökkenti a review során feltárt hibák számát, és fejleszti a fejlesztő saját kritikai szemléletét.

Az audit itt négy pillérre épül:

Pillér	Kérdés	Elsődleges módszer
AAPCS64-illesztés	Betartja-e a modul a hívási konvenciót?	`objdump -d`, pro/epilógus-elemzés
Határellenőrzés	Minden memóriahozzáférés a megengedett tartományban van?	Tesztkészlet szélső `n` értékekkel, őrbájtokkal és védőlappal; a C oldali `AddressSanitizer` a kézi assembly elérését nem instrumentálja
Mentési szimmetria	Minden callee-saved regiszter visszaállításra kerül?	Regisztermentési napló és `ldp`/`stp` párok összevetése
Build-reprodukálhatóság	Ugyanaz a forrás ugyanazt a binárist adja?	Két izolált build, `sha256sum` összehasonlítás

A checklist kiegészül **mérési reprodukálhatósággal** és **biztonsági önellenőrzéssel**; ezeket külön szakaszokban tárgyaljuk.

15.4.3 Kidolgozott példa: a mátrixszorzó modul auditja

Munkakönyvtár: `~/arm-labor/a64/zaro1/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez az I. záróprojekt közös könyvtára: mind a négy alfejezete (15.1–15.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/a64/zaro1
cd ~/arm-labor/a64/zaro1
```

Az auditált modul ugyanaz, amelyet az 15.1 specifikált és az 15.2 implementált — nem egy új, hasonló modul, hanem a tényleges `lab_sgemm`:

```c
/* lab_sgemm.h */
#ifndef LAB_SGEMM_H
#define LAB_SGEMM_H
#include <stddef.h>
#define LAB_SGEMM_OK     0
#define LAB_SGEMM_EINVAL -1
int lab_sgemm(size_t n,
              const float *restrict A,
              const float *restrict B,
              float *restrict C);
#endif
```

Az `A`, `B`, `C` mutatók `n × n` méretű, soronként folytonos `float` tömbökre mutatnak; a visszatérési érték `LAB_SGEMM_OK`/`LAB_SGEMM_EINVAL`. A modul NEON-nal négy `float`-ot dolgoz fel egyszerre, a maradékot skaláris farokkal. Az auditot az alábbi parancsokkal kezdjük.

15.4.3.1 1. AAPCS64-illesztés ellenőrzése

```
aarch64-linux-gnu-gcc -O2 -c lab_sgemm.S -o lab_sgemm.o
aarch64-linux-gnu-objdump -d lab_sgemm.o | sed -n '/<lab_sgemm>:/,/^$/p'
```

Az audit a **tényleges** implementáció regiszter- és memóriaelérési szerződését követi, nem egy általános prológusmintát. A `lab_sgemm` levélfüggvény: nem hív mást, és keretrekordot (`X29`/`X30`) sem épít. A keretet egyetlen pre-indexelt `stp x19, x20, [sp, #-80]!` nyitja (nincs külön `sub sp, sp, #N`), a mentéseket négy további `stp` végzi. Ellenőrizzük:

- **veremigazítás:** a keretet nyitó `stp ..., [sp, #-80]!` és a záró `add sp, sp, #80` ugyanazt a 16 többszörösét adja, és minden kimeneti út (`.Ldone`, `.Linvalid`, `.Lempty_ok`) a közös `.Lepi` epilógusra fut;
- **callee-saved regiszterek:** a törzsben írt összes callee-saved regiszter (itt `x19`–`x28`) szerepel egy prológusbeli `stp`-ben és ugyanazon a helyen egy epilógusbeli `ldp`-ben. A `v8`–`v15` regiszterekhez nincs mentés, és nem is kell, mert a modul csak `v0`, `v1`, `v2` és `v5` regisztert ír. Az auditnak ezt is ellenőriznie kell: egyetlen `v8`–`v15` regiszterre (vagy azok `s`/`d` nevére) író utasítás sem lehet a törzsben;
- **`x29`/`x30`:** a modul egyiket sem írja, és nincs benne `bl`/`blr`. Ezért szabályos, hogy az `x30` nincs mentve: levélfüggvény a változatlan LR-rel tér vissza. Mentés akkor kellene, ha a függvény hívást tartalmazna vagy az `x30`-at másra használná;
- **`x18`:** a modul nem írja. A platformregiszter szerepe platformfüggő (lásd az 15.2 regiszter-táblázatát), ezért a nem használat hordozható választás;
- **argumentumok:** az 15.2 szerződése szerint `x0` = `n`, `x1` = `A`, `x2` = `B`, `x3` = `C`, és a modul ezeket a határellenőrzés előtt az `x19`–`x22`-be másolja.

Egy tipikus hiba ebben a szerkezetben: a törzs használ egy callee-saved regisztert, amely egyik `stp` párban sem szerepel. A nyom tehát nem egy konkrét `str x19` hiánya, hanem az, hogy egy írt callee-saved regiszter hiányzik a mentett regiszterek halmazából. Ezt a 2. feladat szkriptje gépiesen ellenőrzi. A hiba csak akkor jelentkezik, ha a hívó (például a `main`) abban a regiszterben élő értéket tart, ezért a statikus ellenőrzés mellé az 15.2 `lab_abi_call` alapú ABI-tesztje is kell.

15.4.3.2 2. Határellenőrzés

Az `n × n` mátrixszorzás indexei `0 ≤ i, j, k < n`. A mag memóriaelérési szerződése az i-k-j sorrendből következik:

Elérés	Utasítás	Tartomány
`A` egy eleme, 4 sávra szétmásolva	`ldr s0, [x25, x27, lsl #2]` + `dup v0.4s, v0.s[0]`	`A[i·n + k]`, `k < n`
`B` egy sorának 4 eleme	`ldr q2, [x28, x4]`	`B[k·n + j … k·n + j + 3]`, csak ha `j + 4  n`
`C` egy sorának 4 eleme (olvasás és írás)	`ldr q1` / `str q1, [x26, x4]`	`C[i·n + j … i·n + j + 3]`, csak ha `j + 4  n`
skalár farok	`ldr s1/ldr s2/str s1`	a `j + 4 > n` maradék, elemenként
sor nullázása	`str q5` / `str wzr`	`C[i·n … i·n + n − 1]`

A mag tehát **nem** olvas `B`-oszlopot, és `A`-ból egyszerre csak egy elemet tölt. A határhiba jellegzetes helye a `j + 4  n` feltétel: ha a vektoros ág `j < n`-ig futna, a `B`- és `C`-sor végén legfeljebb 3 elemmel túlnyúlna. Ez belső soroknál a következő sor elejét érinti, az utolsó sornál a mátrixon kívülre ér. Az 15.3 `lab_sgemm_test` programja a határeseteket (`n = 0, 1, 2, 3, 5, 7, 8, 16, 17, 63, 64, 65, 128, 256`) argumentum nélkül, magában végigsöpri, a mátrixokat pedig védőlap elé teszi. Az audit során ezért elég egyszer, változatlan bemenettel újrafuttatni:

```
qemu-aarch64 -L /usr/aarch64-linux-gnu ./lab_sgemm_test || echo "FAIL"
```

Az elvárt eredmény az 15.1 numerikus szerződése szerinti egyezés minden elemre (véges érték és `|c_asm − c_ref|  atol + rtol · s_ij`), nem bitre pontos egyezés. A k-sorrend a modulban és a referenciában ugyanaz; az eltérés a `float`, illetve a `double` pontosságú összegzésből jön. Ha `n = 3`-ra numerikus FAIL jön, a skalár farok hibás. Ha `n = 17`-re laphiba jön a védőlapon, a vektoros ág `j + 4  n` feltétele csúszik el a `B`/`C` sorvégen. Ha egy NaN-os elem miatt jön FAIL, valamelyik `C`-elem kimaradt a nullázásból vagy az írásból (a teszt a `C`-t előre NaN-bitmintával tölti). `n = 0`-ra `LAB_SGEMM_OK`-t várunk (nem `EINVAL`-t), írás nélkül a `C`-re.

15.4.3.3 3. Mentési szimmetria

Készítsünk egyszerű szimmetria-ellenőrzést a `objdump` kimenetéből:

```
aarch64-linux-gnu-objdump -d lab_sgemm.o \
  | awk '/<lab_sgemm>:/,/ret/ {print}' \
  | grep -E '(stp|ldp|str|ldr)\s.*\[sp' > lab-sgemm-save.log
```

A logban minden `stp xN, xM, [sp, ...]` párnak meg kell jelennie `ldp xN, xM, [sp, ...]` formában, ugyanazzal a regiszterpárral és ugyanazzal az offsettel. Ha egy `str x20, [sp, #16]` mellett nincs `ldr x20, [sp, #16]`, a függvény nem állítja vissza a hívó `x20`-át. Ez a hiba a C oldalon gyakran úgy jelentkezik, hogy egy hívó ciklusváltozó „elromlik” a `lab_sgemm` hívása után — látszólag összefüggéstelenül a mátrixszorzástól.

15.4.3.4 4. Build-reprodukálhatóság

Két izolált könyvtárban, ugyanazzal a toolchain-verzióval és ugyanazokkal a flag-ekkel lefuttatjuk a **teljes** buildet (a teszthajtót linkelve), `-g`-vel, hogy a debug-információ is az összehasonlítás része legyen. Mivel az 15.1–15.3 nem vezet be `BUILD_DIR`-paraméteres Makefile-t a `lab_sgemm` modulhoz (csak közvetlen fordítóhívásokat mutat), az izolációt itt is közvetlenül, két célkönyvtárral érjük el:

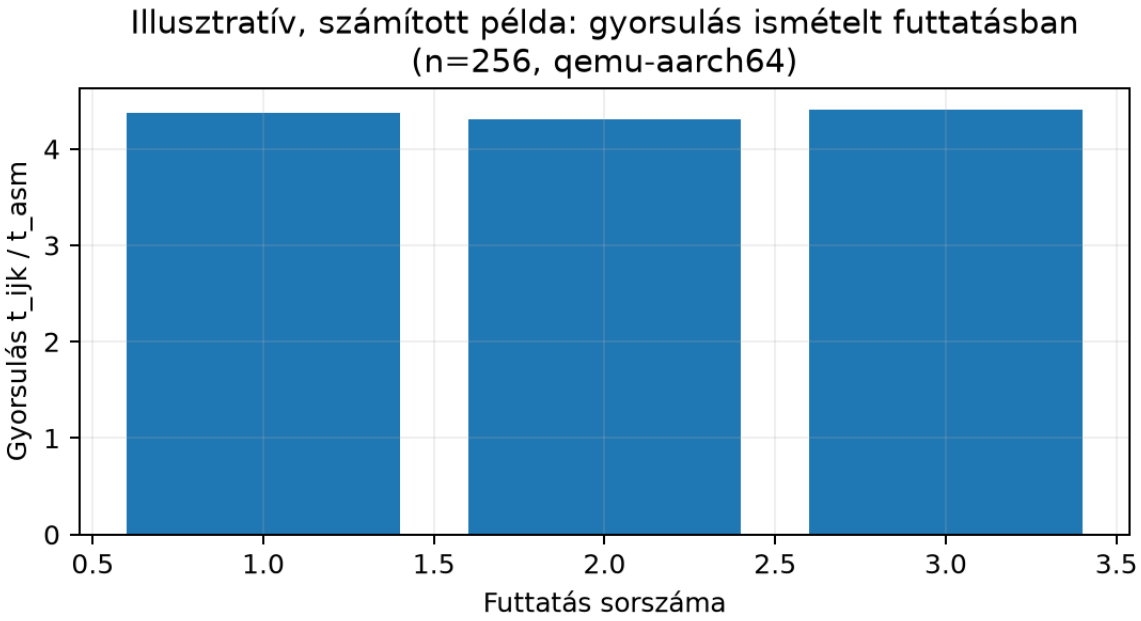
```
mkdir -p /tmp/lab-build-a /tmp/lab-build-b
for d in /tmp/lab-build-a /tmp/lab-build-b; do
  cp lab_sgemm.S lab_sgemm_ref.c lab_sgemm_check.c lab_sgemm_check.h \
     lab_sgemm_test.c "$d/"
  ( cd "$d" && aarch64-linux-gnu-gcc -O2 -c lab_sgemm.S -o lab_sgemm.o \
      && aarch64-linux-gnu-gcc -g -std=gnu17 -O2 -static \
           lab_sgemm_test.c lab_sgemm_check.c lab_sgemm_ref.c lab_sgemm.S \
           -lm -o lab_sgemm_test )
done
sha256sum /tmp/lab-build-a/lab_sgemm.o /tmp/lab-build-b/lab_sgemm.o
sha256sum /tmp/lab-build-a/lab_sgemm_test /tmp/lab-build-b/lab_sgemm_test
```

Az elvárt eredmény: az `as`-kimenet (`lab_sgemm.o`) két hash-e azonos, mert az assembler kimenete nem tartalmaz útvonalat. A `-g`-vel linkelt **teljes** bináris hash-e viszont eltér, mert a DWARF a build könyvtárának abszolút útvonalát (`DW_AT_comp_dir`) tartalmazza. Az útvonal semlegesíthető a `-ffile-prefix-map` kapcsolóval (A-ban `-ffile-prefix-map=/tmp/lab-build-a=.`, B-ben `-ffile-prefix-map=/tmp/lab-build-b=.`), és ekkor a két teljes bináris hash-e ismét azonos lesz. Máskor a beágyazott időbélyeg (`__DATE__`, `__TIME__`) vagy a nem determinisztikus szekciósorrend okozhat eltérést. A reprodukálhatóság nem öncél: biztosítja, hogy a mért eredmény egy adott binárisra vonatkozik, és később is azonos körülmények között megismételhető.

15.4.4 A mérési eredmény újraellenőrzése

Az 15.3 mérése egy futási menetből származott (W = 3 melegítő, R = 11 mért futás mediánja). Az audit kötelező lépése a **második, független futtatás** és a szórásvizsgálat. A mátrixszorzás `n = 256` esetén, `qemu-aarch64` user-mode emulációban, három egymást követő futtatás eredménye a 15.3 hipotetikus adatkészletével (a gyorsulás a naiv i-j-k C-hez mért teljes arány, ugyanazon környezetben):

Futtatás	`t_asm` [ms]	`t_ijk` [ms] (naiv C)	Gyorsulás (`t_ijk` / `t_asm`)
1	20.875	91.123	4.37
2	21.200	91.400	4.31
3	20.600	90.900	4.41



15.2. ábra: Illusztratív, számított példa: gyorsulás ismételt futtatásban (n=256, qemu-aarch64) - A 15.3 hipotetikus adatkészletével; QEMU-emulációban a ciklusszám nem hardverhű, a szórás tendenciája szemléltethető.

A relatív szórás a szórás és az átlag hányadosa — **nem** a (maximum − minimum) és az átlag hányadosa, ami a *relatív terjedelmet* adná, egy durvább és a mintaszámmal rosszul skálázó mérőszámot. A három gyorsulás-érték átlaga $\bar{x} = (4.37 + 4.31 + 4.41)/3 = 4.363$. A (mintás) szórás:

$$s = \sqrt{\frac{1}{N-1}\sum_{i=1}^{N}(x_i - \bar{x})^2} = \sqrt{\frac{(0.007)^2 + (-0.053)^2 + (0.047)^2}{3-1}} = \sqrt{\frac{0.0051}{2}} \approx 0.0505$$

és ebből a relatív szórás:

$$s_{\text{rel}} = \frac{s}{\bar{x}} \approx \frac{0.0505}{4.363} \approx 1.16\%$$

(A relatív terjedelem, $(\max - \min)/\bar{x} = (4.41 - 4.31)/4.363 \approx 2.3\%$, egy másik, durvább szóródási mérőszám lenne — csak két futtatást (a legkisebbet és legnagyobbat) vesz figyelembe, a szórás viszont mindhármat.)

Ha a második futtatás gyorsulása több mint 10%-kal eltér az elsőtől, az alábbi hipotéziseket kell sorra venni:

1. **Háttérfolyamat** fut a hoston (másik build, frissítés, víruskereső). Ellenőrzés: `top`, `uptime`.
2. **CPU-frekvencia-skálázás** (cpufreq governor) vagy thermal throttling. Ellenőrzés: `cpupower frequency-info`, hőmérséklet.
3. **QEMU JIT-fordítás** hideg indulása: az első futtatás lassabb lehet, mert a QEMU dinamikusan fordítja a blokkokat. Ellenőrzés: egy „bemelegítő” futtatás a mérés előtt.
4. **Nem determinisztikus allokáció**: a `malloc`-olt pufferek eltérő laphatárokra esnek, ami QEMU-ban is befolyásolhatja a memóriahozzáférést.

A mérési audit akkor zárul le, ha a második futtatás **ugyanabban a sávban** van (±5%), vagy ha az eltérésre dokumentált, elfogadott magyarázat született.

15.4.5 Biztonsági önellenőrzés a 14. fejezet (Biztonsági szemlélet) szempontjai szerint

A vizsgált szempontok forrása a Biztonsági szemlélet: verem, védelem és ROP (14.) fejezet.

Az 14.1–14.4 alfejezetek a verem-túlcsordulást, a canary/ASLR/NX/PAC védelmet és a ROP-lánc felismerését tárgyalták. Az audit során ezeket a szempontokat **defenzív** módon kell érvényesíteni:

- **Stack-keret fegyelem:** a modul soha nem ír a saját stack-keretén kívülre. Ellenőrzés: a keretet nyitó utasítás (a `lab_sgemm`-ben `stp x19, x20, [sp, #-80]!`, más modulban akár `sub sp, sp, #N`) és a záró `add sp, sp, #N` között minden veremre író utasítás célcíme a `[sp, #offset]` tartományban van, ahol `0 ≤ offset < N`.
- **Veremjelző:** a kész ELF `GNU_STACK` szegmense `RW` (`readelf -lW`). A modul forrása tartalmazza a `.note.GNU-stack` direktívát, de a bizonyíték a végső binárisról szól, nem a forrásról; a direktíva hiánya önmagában nem igazol végrehajtható vermet.
- **Return address integritás:** ha a modul az `x30`-et a stackre menti, azt a visszatérés előtt kötelező visszaállítani. PAC (Pointer Authentication) használatakor a visszatérési cím aláírása és ellenőrzése a hardver feladata; az assembly modulnak a `paciasp`/`autiasp` párt kell használnia, ha a rendszer PAC-ot vár. A `retaa` csak Armv8.3-A-t megkövetelő binárisban használható, mert régebbi magon `SIGILL`-t okoz, míg a HINT-kódolású `paciasp`/`autiasp` ott NOP.
- **Canary:** ha a C fordító `-fstack-protector-strong` kapcsolóval fordítja a hívó kódot, az assembly modulnak nem kell canary-t kezelnie, de **nem szabad** a canary által használt stack-helyet felülírnia. Az assembly modul saját stack-keretét ezért úgy kell kialakítani, hogy a hívó canary-je érintetlen maradjon.
- **Visszatérési út (A64):** A64-en a visszatérés két lépés. Előbb egy betöltés adja az `X30` értékét, majd a `ret` erre ugrik (`PC ← X30`; lásd a ROP-lánc felismerése levezetését). Az `X30` forrása levélfüggvényben a belépéskori, változatlan érték: ez a helyzet a `lab_sgemm`-nél, amely az `X30`-at nem írja és nem menti. Nem-levél függvényben a prológusban mentett, és az epilógusban a **saját keretéből** visszatöltött érték, tipikusan `ldp x29, x30, [sp], #16` vagy `ldr x30, [sp, #off]`. Az audit három dolgot vizsgál: honnan kap értéket az `X30` a `ret` előtt; ez a betöltés a modul saját, kiegyensúlyozott keretéből történik-e; és minden visszatérési út ugyanezen a visszatöltésen megy-e át. A szabályos epilógus maga nem hiba. Összehasonlításként, ISA-váltással: a II. rész Thumb-2 kódjában a `pop {…, pc}` **önálló visszatérési művelet**. Egyetlen utasítás tölti vissza a PC-t a veremből, külön `ret` nincs, és A64-en nincs ilyen alak (lásd 16.1).

Az önellenőrzés nem azt jelenti, hogy a modulnak minden támadást ki kell védenie — hanem azt, hogy **nem növeli** a rendszer támadási felületét a C referencia implementációhoz képest.

15.4.6 Csapat-kódreview: mit tegyünk hiányosság esetén

Ha egy csapat-kódreview hiányosságot talál — például egy review-társ észreveszi, hogy az `x21` nincs visszaállítva, vagy hogy a farokkezelés `n = 0` esetén osztással próbálkozik —, az alábbi folyamatot érdemes követni:

1. **Reprodukálás:** a hiba leírása mellé készüljön minimális reprodukciós lépés (konkrét `n`, konkrét bemenet, konkrét parancs). Ha a hiba nem reprodukálható, először a reprodukciós körülményeket kell tisztázni.
2. **Hatókör:** meg kell határozni, hogy a hiba csak a modult érinti-e, vagy a teljes projektet. Egy regisztermentési hiba például a hívó kódot is érintheti.
3. **Javítás:** egy változtatás egyszerre. Ha több gyanús pont van, azokat külön commitban kell javítani, hogy a javítás hatása izolálható legyen.
4. **Regressziós teszt:** a javításhoz tartozzon olyan teszt, amely a hibát **előtte** elkapja. Ez a teszt a jövőben véd a visszaesés ellen.
5. **Újramérés:** a javítás után a teljesítménymérést meg kell ismételni, mert a javítás (pl. extra `stp`/`ldp` pár) ronthatja a gyorsulást. Az 15.3 mérési eredményét a javított binárisra újra kell igazolni.
6. **Dokumentálás:** a review megállapítását, a javítást és az újramérést a projekt `CHANGELOG`-jában vagy a commit üzenetében rögzíteni kell. Ez az auditnyom.

A kódreview nem verseny, hanem **közös hibamegelőzés**. A review-társ által talált hiba nem a szerző kudarca, hanem a folyamat működésének bizonyítéka.

15.4.7 Lezárás: mi marad releváns a II. részben?

A II. rész (16. fejezet) felé fordulva érdemes összegezni, mi az, ami az AArch64 alkalmazásszintű auditból **átvihető** a Cortex-M bare-metal világába:

- **Hívási konvenció fegyelme:** az AAPCS64 helyett az AAPCS32 (Armv7-M) lesz érvényes, de a callee-saved regiszterek mentésének és a stack-igazításnak a fegyelme ugyanúgy követelmény. A `PUSH`/`POP` és a `STMFD`/`LDMFD` párok szimmetriája ugyanaz az auditpont.
- **Határellenőrzés:** bare-metal környezetben nincs `valgrind` vagy `AddressSanitizer`; a memóriahibák gyakran a perifériaregiszterek véletlen felülírásában jelentkeznek. Az auditnak itt is ki kell térnie a tömb- és pufferhatárokra, különösen az ISR-ek és a DMA-pufferek esetén.
- **Mérési fegyelem:** a QEMU `mps2-an385`-ön mért idő (SysTick, semihosting-óra) szintén emulációs idő, nem ciklusszám; a relatív összehasonlítás és a szórásvizsgálat módszertana változatlan.
- **Biztonsági szemlélet:** a Cortex-M-en nincs ASLR és NX a klasszikus értelemben, de a stack overflow elleni védelem (MPU, stack canary) és a visszatérési cím integritása (PAC-ra emlékeztető mechanizmusok) itt is releváns.
- **Ami nem vihető át:** a Linux syscall-felszín, az ELF dinamikus linkelés és a NEON regiszterkészlet. A Cortex-M3 nem rendelkezik NEON-nal; a DSP-kiterjesztés (Cortex-M4/M7) más utasításkészletet ad.

Az auditmódszer tehát nem az AArch64-hez kötött, hanem **architektúra-független mérnöki fegyelem**, amelynek konkrét ellenőrzési pontjai igazodnak az adott ISA-hoz és futási modellhez.

15.4.8 Feladatok

1. feladat (magyarázó, alapszint — cél: megértés-ellenőrzés): Mit jelent a mentési szimmetria egy assembly függvényben? Adj példát egy tipikus aszimmetriára, és magyarázd el, milyen tünetet okoz a hívó C kódban. Hogyan lehet `objdump` kimenetéből automatikusan ellenőrizni?

2. feladat (implementációs, középszint — cél: rutinépítés): Készíts `lab-audit.sh` shell-scriptet, amely a `lab_sgemm.o` objektumfájlon végrehajtja a négy pillér legalább egy-egy

ellenőrzését: (a) stack-igazítás és egyensúly, (b) a `stp`/`ldp` párok regisztereinek és tényleges veremhelyeinek összevetése, (b2) annak ellenőrzése, hogy a törzsben **írt** minden callee-saved regiszter (`x19`–`x29`, `d8`–`d15`, a `w`/`s` nevekkel együtt) mentve és visszaállítva van, (c) az LR megőrzése, ha a függvény további hívást végez, (d) két explicit buildtermék hash-ének összevetése. A script adjon `PASS`/`FAIL` jelzést pontonként, és hiba esetén nem nulla kilépési állapotot.

3. feladat (hibakeresési/mérési, elmélyült — cél: rutinépítés): Az alábbi, szándékosan hibás farokkezelésű, teljes A64 függvény a `lab_sgemm` belső ciklusához hasonló szerkezetű: vektoros főág négyesével, utána skaláris farok az `n mod 4` maradékra. Feladat: `c[i] += a[i]`, `0 i < n`. A függvény levélfüggvény, keretet nem épít, és csak caller-saved regisztereket (`x0`–`x6`, `x9`, `v0`, `v1`) használ, ezért nincs megőrzendő állapota; a hiba tehát nem ABI-hiba. Azonosítsd a hibát, vezesd le, mely `n` értékekre ad hibás kimenetet és melyekre helyeset, majd javítsd ki, és igazold a mellékelt teszttel.

```
// lab-a060-tail.s - A64 ISA, AArch64 Linux; SZÁNDÉKOSAN HIBÁS farokkezelés (3. feladat)
// void lab_add_f32(float *c, const float *a, size_t n): c[i] += a[i], 0 <= i < n
// Előfeltétel: c és a legalább n elemű, nem fedik át egymást; n = 0 esetén nincs hozzáférés.
// Levélfüggvény: nem hív mást, keretet nem épít, és csak caller-saved regisztereket
// (x0-x6, x9, v0, v1) használ, ezért nincs megőrzendő állapota.
    .text
    .global lab_add_f32
    .type   lab_add_f32, %function
lab_add_f32:
    bic     x4, x2, #3              // x4 = 4 * floor(n/4): a vektoros rész vége (elemben)
    mov     x6, #0                  // i = 0
    cbz     x4, .Ltail              // n < 4: nincs vektoros rész
.Lvec:
    lsl     x9, x6, #2              // bájteltolás = 4 * i
    ldr     q0, [x0, x9]
    ldr     q1, [x1, x9]
    fadd    v0.4s, v0.4s, v1.4s     // c[i..i+3] += a[i..i+3]
    str     q0, [x0, x9]
    add     x6, x6, #4
    cmp     x6, x4
    b.lo    .Lvec
.Ltail:
    and     x5, x2, #3              // x5 = n mod 4: a farok hossza
    cbz     x5, .Ldone
.Ltail_loop:
    ldr     s0, [x0, x6, lsl #2]
    ldr     s1, [x1, x6, lsl #2]
    fadd    s0, s0, s1              // c[i] += a[i]
    str     s0, [x0, x6, lsl #2]
    sub     x5, x5, #1
    cbnz    x5, .Ltail_loop
.Ldone:
    ret
    .size   lab_add_f32, . - lab_add_f32

    .section .note.GNU-stack,"",%progbits
```

A teszt `n = 0 … 12`-re független C-referenciával veti össze az eredményt, `n = 0`-ra `NULL` mutatókkal:

```
/* lab-a060-tail-test.c - a lab_add_f32 tesztje n = 0..12-re, független C-orákulummal */
#include <stdio.h>
#include <stddef.h>

void lab_add_f32(float *c, const float *a, size_t n);

int main(void)
```

```
{
    int fails = 0;
    for (size_t n = 0; n <= 12; n++) {
        float c[12], a[12], want[12];
        for (size_t i = 0; i < n; i++) {
            c[i] = (float)(100 * i);          /* kis egészek: a float-összeg pontos */
            a[i] = (float)(i + 1);
            want[i] = c[i] + a[i];
        }
        lab_add_f32(n ? c : NULL, n ? a : NULL, n);
        size_t bad = n;
        for (size_t i = 0; i < n && bad == n; i++)
            if (c[i] != want[i]) bad = i;
        if (bad == n)
            printf("n=%2zu (n mod 4 = %zu) OK\n", n, n % 4);
        else
            printf("n=%2zu (n mod 4 = %zu) HIBA: c[%zu] = %g, elvárt %g\n",
                   n, n % 4, bad, (double)c[bad], (double)want[bad]);
        fails += bad != n;
    }
    puts(fails ? "TAIL: FAIL" : "TAIL: PASS");
    return fails != 0;
}
```

```
aarch64-linux-gnu-gcc -O2 -static -o lab-a060-tail lab-a060-tail-test.c lab-a060-tail.s
qemu-aarch64 ./lab-a060-tail; echo "kilépési állapot: $?"
```

4. feladat (biztonsági önellenőrzés, középszint — cél: megértés-ellenőrzés): Válaszd ki a saját modulod egy olyan pontját, ahol stack-re írsz. Ellenőrizd, hogy az írás a `sub sp, sp, #N` által kijelölt kereten belül marad-e. Dokumentáld, hogyan változna a modul, ha a hívó `-fstack-protector-strong` kapcsolóval fordulna.

15.4.9 Megoldások

1. megoldás — Segítség. Gondolj arra, hogy a prológus TÖBB regisztert is menthet — elég-e csak a lementett párokat ellenőrizni, vagy minden ténylegesen módosított regisztert?

1. megoldás — Teljes megoldás. A mentési szimmetria azt jelenti, hogy minden callee-saved regiszter, amelyet a függvény módosít, a visszatérés előtt az eredeti értékére áll vissza. Tipikus aszimmetria: a prológus `stp x19, x20, [sp, #16]`-tal ment, de az epilógus csak `ldp x19, x20, [sp, #16]`-tal állít vissza — látszólag rendben. A valódi hiba az, ha a függvény az `x21`-et is módosítja, de azt egyáltalán nem menti. A hívó C kódban ez úgy jelentkezhet, hogy egy `for` ciklus számlálója vagy egy pointer „elromlik” a hívás után. Objump-pal: gyűjtsük ki a függvény összes `str`/`stp` és `ldr`/`ldp` utasítását, és párosítsuk őket regiszter és offset szerint; a páratlanul maradó bejegyzések jelzik a problémát.

2. megoldás — Segítség. Szűkítsd a disassemblyt a `lab_sgemm` szimbólumra, és kövesd az SP változását. Az azonos darabszám önmagában kevés: ugyanaz a regiszter ugyanarról a belépési SP-hez viszonyított helyről töltődjön vissza. Levélfüggvényben a változatlan LR nem igényel veremmentést.

2. megoldás — Teljes megoldás (a modul ismert, egy közös epilógusú keretére):

A shell-fájlba ágyazott Python-ellenőrző nem igényel külön segédfájlt. Használat: `bash lab-audit.sh lab_sgemm.o`, illetve két elkészült build összevetéséhez `bash lab-audit.sh /tmp/lab-build-a/lab_sgemm.o --compare /tmp/lab-build-b/lab_sgemm.o`. Az elágazó vezérlési utak teljes ABI-bizonyítását ez az egyszerű statikus minta nem helyettesíti.

Hatókör és támogatott alakok. A script egyetlen, lineárisan végigolvasott függvényt vizsgál, egy közös epilógussal. Mentésként az `sp`-hez viszonyított `stp`/`ldp` (`x` és `d` regiszterpár, pre- és post-index) és az egyes `str`/`ldr` alakot ismeri fel. A regisztermegőrzési próbához a törzs **minden** utasításának célregiszterét összegyűjti. A neveket a 64 bites callee-saved névre képezi (`w21` → `x21`; `s8`, `q8`, `v8.4s` → `d8`, mert a V8–V15-ből csak az alsó 64 bit megőrzendő). Az `ldp` és az `ld1 {…}` teljes célregiszter-listáját, valamint a címzési visszaírás (`[xN], #…`, `[xN, #…]!`) bázisregiszterét is írásnak veszi. Az írt callee-saved halmaznak a mentett **és** visszaállított halmazban kell lennie. A script csak az előre felsorolt utasításokat ismeri; ismeretlen utasítás esetén a regisztermegőrzési próba eredménye `SKIP`, megnevezve az utasítást, és nem hamis `PASS`.

```
#!/bin/bash
# lab-audit.sh - a lab_sgemm ismert keretének statikus ellenőrzése
set -eu
python3 - "$@" <<'PY_AUDIT'
import argparse
import hashlib
import re
import subprocess
from pathlib import Path

# A callee-saved állapot (AAPCS64): X19-X29 és a V8-V15 alsó 64 bitje (D8-D15).
CALLEE = {f'x{i}' for i in range(19, 30)} | {f'd{i}' for i in range(8, 16)}
# Támogatott utasítások: amelyik nem ír általános vagy SIMD/FP regisztert ...
NOWRITE = {'str', 'strb', 'strh', 'stur', 'stp', 'stnp', 'st1', 'cmp', 'cmn', 'tst',
           'fcmp', 'fcmpe', 'ccmp', 'b', 'bl', 'blr', 'br', 'ret', 'cbz', 'cbnz', 'tbz',
           'tbnz', 'nop', 'dsb', 'dmb', 'isb', 'prfm', 'bti', 'paciasp', 'autiasp'}
# ... és amelyik az első operandusát (ldp/ld1 esetén a teljes listát) írja.
WRITE = {'add', 'adds', 'sub', 'subs', 'mul', 'madd', 'msub', 'umulh', 'smulh', 'udiv',
         'sdiv', 'lsl', 'lsr', 'asr', 'ror', 'and', 'ands', 'orr', 'eor', 'bic', 'mvn',
         'neg', 'mov', 'movz', 'movk', 'movn', 'adr', 'adrp', 'csel', 'cset', 'csinc',
         'ubfx', 'sbfx', 'bfi', 'uxtw', 'sxtw', 'ldr', 'ldrb', 'ldrh', 'ldrsw', 'ldur',
         'ldp', 'ld1', 'fmov', 'fadd', 'fsub', 'fmul', 'fdiv', 'fmadd', 'fmsub', 'fneg',
         'fabs', 'scvtf', 'ucvtf', 'fcvtzs', 'fcvtzu', 'movi', 'dup', 'ins', 'umov',
         'fmla', 'mla', 'addv', 'ext', 'rev64'}

def canon(reg):
    """W/X -> x<n>; B/H/S/D/Q/V (lane- és alakjelöléssel is) -> d<n>."""
    reg = reg.strip().lower()
    if (m := re.fullmatch(r'[wx](\d+)', reg)):
        return 'x' + m[1]
    if (m := re.fullmatch(r'[bhsdqv](\d+)(?:\.\w+)?(?:\[\d+\])?', reg)):
        return 'd' + m[1]
    return reg

parser = argparse.ArgumentParser(description=__doc__)
parser.add_argument('image', type=Path, nargs='?', default=Path('lab_sgemm.o'))
parser.add_argument('--objdump', default='aarch64-linux-gnu-objdump')
parser.add_argument('--compare', type=Path)
args = parser.parse_args()
try:
    dump = subprocess.check_output(
        [args.objdump, '-d', '--disassemble=lab_sgemm', '--no-show-raw-insn',
         str(args.image)], text=True)
    if '<lab_sgemm>:' not in dump: raise ValueError('Hianyzik a lab_sgemm szimbolum')
    sp = 0; slots = {}; restored = set(); calls = False; returns = False
    written = set(); unsupported = set(); lowest = 0
    number = r'(-?(?:0x[0-9a-f]+|[0-9]+))'
    pair = re.compile(
        r'(stp|ldp)\s+([xd][0-9]+),\s*([xd][0-9]+),\s*'   # művelet és regiszterpár
        r'\[sp(?:,\s*#' + number + r')?\]'                 # [sp] vagy [sp, #off]
        r'(!)?'                                            # pre-index visszaírás
        r'(?:,\s*#' + number + r')?')                      # post-index lépés
```

```
single = re.compile(r'(str|ldr)\s+([xd][0-9]+),\s*\[sp(?:,\s*#' + number + r')?\]')
for line in dump.splitlines():
    match = re.match(r'\s*[0-9a-f]+:\s+(.*)', line)
    if not match: continue
    instruction = match[1].split('//')[0].strip()
    mnemonic, operands = (instruction.split(None, 1) + ['',])[:2]
    calls |= mnemonic in ('bl', 'blr')
    returns |= instruction == 'ret'
    if (match := pair.fullmatch(instruction)) or (match := single.fullmatch(instruction)):
        op, regs = match[1], [r for r in match.groups()[1:3] if r and r[0] in 'xd']
        if op in ('stp', 'ldp'):
            offset, pre, post = int(match[4] or '0', 0), match[5], match[6]
        else:
            offset, pre, post = int(match[3] or '0', 0), None, None
        if pre: sp += offset
        lowest = min(lowest, sp)
        if sp % 16: raise ValueError('Nem 16 bajtos SP a memoriaeleresnel')
        address = sp + (0 if pre else offset)
        width = 8 * len(regs)
        if op in ('stp', 'str') and not (lowest <= address and address + width <= 0):
            raise ValueError(f'iras a keret hatarain kivul: SP0{address:+d}, {width} bajt')
        for register, position in zip(regs, (address, address + 8)):
            if op in ('stp', 'str'): slots[position] = register
            else:
                if slots.get(position) != register:
                    raise ValueError(f'{register}: rossz visszaallitas '
                                     f'a(z) SP0{position:+d} helyrol')
                restored.add(register)
        if post: sp += int(post, 0)
        lowest = min(lowest, sp)
        if sp % 16: raise ValueError('Nem 16 bajtos SP az utasitas utan')
        continue                                    # mentés/visszatöltés: nem törzsírás
    if (match := re.fullmatch(r'(add|sub)\s+sp,\s*sp,\s*#'+number, instruction)):
        amount = int(match[2],0); sp += amount if match[1]=='add' else -amount
        lowest = min(lowest, sp)
    elif mnemonic.startswith('b.') or mnemonic in NOWRITE:
        pass
    elif mnemonic in WRITE:
        if operands.startswith('{'):                # ld1 {v0.4s, v1.4s}, [...]
            dests = operands[1:operands.index('}')].split(',')
        elif mnemonic == 'ldp':
            dests = operands.split(',')[:2]
        else:
            dests = operands.split(',')[:1]
        for r in (canon(r) for r in dests):
            if r in restored:
                raise ValueError(f'{r}: iras a visszaallitas utan')
            written.add(r)
    else:
        unsupported.add(mnemonic)
    if mnemonic in ('str', 'stp', 'st1', 'stur') and '[sp' in operands and not (
            pair.fullmatch(instruction) or single.fullmatch(instruction)):
        unsupported.add(mnemonic + ' [sp] (nem tamogatott alak)')
    if (wb := re.search(r'\[(\w+)(?:,[^\]]*)?\]!|\[(\w+)\],\s*#', instruction)):
        base = wb[1] or wb[2]                       # címzési visszaírás: a bázis is íródik
        if base != 'sp':
            if canon(base) in restored:
                raise ValueError(f'{canon(base)}: iras a visszaallitas utan')
            written.add(canon(base))
    if sp % 16: raise ValueError('Nem 16 bajtos SP-igazitas')
if sp or not returns: raise ValueError('Kiegyensulyozatlan verem vagy hianyzo RET')
saved = set(slots.values())
if not saved.issubset(restored): raise ValueError('Hianyzo regiszter-visszaallitas')
```

```
    if calls and 'x30' not in restored: raise ValueError('Beagyazott hivas LR-mentes nelkul')
    print('PASS: SP-egyensuly es 16 bajtos igazitas')
    print('PASS: mentett regiszterek es a visszatoltesi helyek egyeznek')
    print('PASS: LR-szerzodes ('+('mentett' if calls else 'level fuggveny')+')')
    if unsupported:
        print('SKIP: callee-saved megorzes (nem tamogatott utasitas: '
              + ', '.join(sorted(unsupported)) + ')')
    else:
        unsaved = sorted((written & CALLEE) - (saved & restored))
        if unsaved:
            raise ValueError('irt, de nem mentett/visszaallitott callee-saved regiszter: '
                             + ', '.join(unsaved))
        print('PASS: minden irt callee-saved regiszter mentve es visszaallitva ('
              + ', '.join(sorted(written & CALLEE)) + ')')
    if args.compare:
        digest_a = hashlib.sha256(args.image.read_bytes()).digest()
        digest_b = hashlib.sha256(args.compare.read_bytes()).digest()
        if digest_a != digest_b:
            raise ValueError('A ket explicit buildtermek elter')
        print('PASS: buildtermekek byte-azonosak')
    else:
        print('SKIP: build-reprodukálhatóság (nincs --compare fájl)')
except (OSError, ValueError, subprocess.CalledProcessError) as error:
    print('FAIL: '+str(error))
    raise SystemExit(1)
PY_AUDIT
```

Ellenpróba. A fő úton, GNU binutils 2.42-vel (`aarch64-linux-gnu-objdump`) futtatva (a lektor mérése):

Vizsgált függvény	Eredmény
a `lab_sgemm` modul	minden `PASS` (`x19`–`x28` mentve és visszaállítva)
`x19`/`x20`-at mentő keret, a törzsben `mov x21, x0`	`FAIL: … nem mentett/visszaallitott callee-saved regiszter: x21`
ugyanez `fmov d8, x0`-val	`FAIL: … d8`
ugyanez `add w21, w0, #1`-gyel (W-név)	`FAIL: … x21`
ugyanez `ldr x0, [x22], #8`-cal (bázis-visszaírás)	`FAIL: … x22`
a `lab_sgemm`, törölt `x27`/`x28` mentéssel, változatlan használattal	`FAIL: … x27, x28`
`stp x21, x22` mentés, hiányzó `ldp`	`FAIL: Hianyzo regiszter-visszaallitas`
`ldp` visszaállítás után `mov x19, #0`	`FAIL: x19: iras a visszaallitas utan`
`stp x19, x20, [sp, #-24]!` / `ldp …, [sp], #24`	`FAIL: Nem 16 bajtos SP a memoriaeleresnel`
`sub sp, sp, #32` + `stp x19, x20, [sp, #16]` (pontosan a keret végén)	`PASS`
ugyanez `[sp, #24]`-gyel (8 bájttal túlnyúl)	`FAIL: iras a keret hatarain kivul: SP0-8, 16 bajt`
`str q8, [sp, #16]` (a szkript nem modellezi)	`SKIP: … str [sp] (nem tamogatott alak)`
a lista nem ismert utasítás (`rbit`)	`SKIP: callee-saved megorzes (nem tamogatott utasitas: rbit)`

A `lab_sgemm.o`-n a szkript öt sort ad (négy `PASS` és a build-összevetés `SKIP`), `--compare`-rel öt `PASS`; az `x27`/`x28` mentését törölve a fenti `FAIL` és 1-es kilépési állapot jön. Egy naiv változat, amely csak a mentett párok szimmetriáját nézte, az első két hibás esetre három `PASS`-t és 0-s

kilépési állapotot adott. A statikus ellenőrzés mellett az 15.2 `lab_abi_call` alapú futtatásos ABI-tesztje továbbra is kell: az a mintákat a hívó szemszögéből ellenőrzi, és olyan útvonalat is lefed, amelyet egy lineáris parser nem lát.

3. megoldás — Segítség. A vektoros főág után `x6` a farok első indexe, `x5` a farok hossza. Kövesd a farokciklust `n = 6`-ra: melyik elemet olvassa és írja az első és a második iteráció? Mi változik egy iterációban, és mi nem?

3. megoldás — Teljes megoldás. A farokciklus csak a számlálót (`x5`) csökkenti, az indexet (`x6`) nem lépteti: mindegyik iteráció ugyanazt a `c[n - r]` elemet frissíti, ahol `r = n mod 4`. A hiba tehát a maradékosztálytól függ:

- `n mod 4 = 0`: nincs farok, az eredmény **helyes**;
- `n mod 4 = 1`: a farok egyetlen iterációja a helyes `c[n - 1]`-et frissíti, az eredmény **helyes**;
- `n mod 4 = 2` vagy `3`: a `c[n - r]` elem `r`-szer kapja meg az `a[n - r]`-et, a farok többi eleme érintetlen, az eredmény **hibás**.

A fő úton (GNU + QEMU 8.2.2) futtatva a teszt kimenete (a lektor mérése):

```
n= 0 (n mod 4 = 0) OK
n= 1 (n mod 4 = 1) OK
n= 2 (n mod 4 = 2) HIBA: c[0] = 2, elvárt 1
n= 3 (n mod 4 = 3) HIBA: c[0] = 3, elvárt 1
n= 4 (n mod 4 = 0) OK
n= 5 (n mod 4 = 1) OK
n= 6 (n mod 4 = 2) HIBA: c[4] = 410, elvárt 405
n= 7 (n mod 4 = 3) HIBA: c[4] = 415, elvárt 405
n= 8 (n mod 4 = 0) OK
n= 9 (n mod 4 = 1) OK
n=10 (n mod 4 = 2) HIBA: c[8] = 818, elvárt 809
n=11 (n mod 4 = 3) HIBA: c[8] = 827, elvárt 809
n=12 (n mod 4 = 0) OK
TAIL: FAIL
kilépési állapot: 1
```

Például `n = 6`-ra `c[4] = 400 + 2 · 5 = 410` a várt 405 helyett, míg `n = 5`-re minden elem helyes. A javítás egyetlen utasítás a farokciklusban, a számláló csökkentése előtt:

```
    add     x6, x6, #1              // a farok indexe is lép
```

A javított változat a teszten `n = 0 … 12` mindegyikére `OK`, a vége `TAIL: PASS`, kilépési állapot 0. Mivel a függvény csak caller-saved regisztereket ír, az 15.2 `lab_abi_call` ABI-teszthívója mindkét változatra 0-s maszkot ad (`n = 6`-tal, a helyettesítő eszközlánccal kipróbálva): ezt a hibát a numerikus referencia, nem az ABI-teszt különbözteti meg.

4. megoldás — Segítség. Egy canary a HÍVÓ keretében, a visszatérési cím közelében ül — mi történne, ha a te moduld `sp`-csökkentése nem elég nagy ahhoz, hogy a saját mentéseid ne nyúljanak bele ebbe a tartományba?

4. megoldás — Teljes megoldás. A stack-keret ellenőrzése: a `sub sp, sp, #N` után minden `str`/`stp` célcíme `[sp, #offset]`, ahol `0  offset < N`. Ha a hívó `-fstack-protector-strong` kapcsolóval fordul, a hívó saját stack-keretében a canary a visszatérési cím előtt helyezkedik el. Az assembly modulnak **nem kell** a canary-t kezelnie, de a saját `sub sp, sp, #N`-jével nem nyúlhat a hívó keretéhez. Ha a modul a `sp`-t a kelleténél kevesebbel csökkenti, és a mentés a hívó canary-jének helyére kerül, a canary sérül, és a hívó `__stack_chk_fail`-t hív. Az auditnak ezt a határtételt is vizsgálnia kell. A fenti szkript ezt bájtalapú szerződéssel teszi: a keret `N` bájt, egy `w` bájtos írás akkor érvényes, ha `lowest  cím` és `cím + w  0` (az `SP0`, a belépéskori SP-hez képest), tehát a pontosan a végén befejeződő hozzáférés megfelel, az egy bájttal túlnyúló nem;

az `stp`/`ldp` szélessége két regiszter (16 bájt), a `q` regiszteres és `st1` alakot a szkript SKIP-pel jelzi, nem bizonyítja. A korábbi, csak a kezdőoffsetet vizsgáló változat az `[sp, #24]` esetet átengedte.

15.4.10 Rövid összefoglaló

Az auditmódszer a projekt lezárásának önálló, reprodukálható szakasza. Négy pillére az AAPCS64-illesztés, a határellenőrzés, a mentési szimmetria és a build-reprodukálhatóság; kiegészül a mérési eredmény ismételt ellenőrzésével és a defenzív biztonsági önellenőrzéssel. Az audit feltárja a csendes hibákat, amelyek a mérési fázisban rejtve maradnak. Csapat-kódreview esetén a reprodukálás–hatókör–javítás–regresszió–újramérés–dokumentálás sorrend a követendő. A módszertan architektúra-független: a Cortex-M bare-metal fejlesztésben a konkrét ellenőrzési pontok változnak, de a fegyelem és a reprodukálhatóság igénye változatlan.

Továbbhasznált definíciók:

- **auditchecklist:** előre rögzített ellenőrzési pontok listája, amely az auditot reprodukálhatóvá teszi.
- **önaudit:** a fejlesztő saját maga által végzett strukturált audit a projekt lezárása előtt.
- **mentési szimmetria:** minden callee-saved regiszter, amelyet a függvény módosít, a visszatérés előtt visszaáll az eredeti értékére.

15.4.11 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblázatát):

- Arm A-profile Architecture Reference Manual — az AAPCS64, a `stp`/`ldp` utasítások és a PAC-mechanizmusok elsődleges forrása.
- Procedure Call Standard for the Arm 64-bit Architecture (AAPCS64) — **elsődleges forrás** a hívási szerződéshez (callee-saved regiszterek: „Parameter Passing and Register Use”, a `r19`–`r29` (`x19`–`x28`, FP) és `v8`–`v15` alsó 64 bitje callee-saved; az SP 16 bájtos igazítása: „The Stack”, `aapcs64` dokumentum, a csomagolt kiadás: 2025Q4, lásd az Olvasói útmutató forrásállapot-táblázatát). Az Arm Architecture Reference Manual (fenti pont) csak az utasításszemantika forrása (`stp`/`ldp`, PAC), a hívási szabályok nem.
- GNU Compiler Collection kézikönyv — `-fstack-protector-strong`, `-O2`, `-fno-tree-vectorize` és a build-reprodukálhatóságot befolyásoló kapcsolók.
- GNU Binutils `as` és `objdump` kézikönyv — a disassembly és a szimbólumtábla-elemzés parancssori háttere.

Ellenőrzési hiányok: az 1–2. auditpontban leírt regiszter- és memóriaelérési szerződést az 15.2 forrásával vetettük össze; a 2. feladat audit-szkriptjének kimenetét GNU binutils 2.42-vel, a 3. feladat faroktesztjét GNU + QEMU 8.2.2-vel mértük (a lektor mérése); a határesetek és a védőlapos teszt kimenete ezekből származik. A bemutatott gyorsulási számok illusztratív, levezetett értékek, nem tényleges mérések. A PAC-viselkedés és a `-fstack-protector-strong` assembly modulra gyakorolt hatása a jelölt toolchain-verziótól függ, ezért a laborban a tényleges eredményt kell rögzíteni; natív AArch64 hardveren nem mértük.

II. RÉSZ – Cortex-M: bare-metal mikrokontroller-programozás

16 Thumb-2, a Cortex-M futási modell és a minimal runtime

16.1 A-profile és M-profile különbségei: miért más a Cortex-M

Előfeltétel a II. részhez: a Cortex-M eszközlánc (`gcc-arm-none-eabi`, `binutils-arm-none-eabi`, `qemu-system-arm`) telepítése és smoke tesztje a Labor-alapkörnyezet alfejezet „A II. rész eszközlánca” szakaszában. Az I. rész `aarch64-linux-gnu-` eszközlánca nem a Cortex-M eszközlánca, ezért a II. rész példái előtt a `~/arm-labor/cm/a006/` könyvtárban a smoke tesztnek sikeresen le kell futnia (`Cortex-M smoke: OK`, kilépési állapot `0`). A II. rész minden laborja a `~/arm-labor/cm/` ágon, alfejezetenként külön könyvtárban dolgozik.

16.1.1 Motiváció: ugyanaz a ciklus, két teljesen más világ

Az I. részben megszoktuk, hogy a program `_start`-ból indul, a kimenetet `write` rendszerhívással kérjük, a címeink virtuálisak, és a kernel gondoskodik a lapozásról, a pufferelésről, a hibajelzésről. A Cortex-M-en ugyanaz a számlálós ciklus fut le, de a `write` helyett egyetlen `STRB` utasítás ír egy periféria regiszterébe, és nincs kernel, amely elkapná a hibát. Ez az alfejezet rendszerezi, mi tűnik el az AArch64 Linux-userspace modellből, mi marad meg, és miért nem lehet a két oldalt mechanikusan egymásra másolni. A cél nem a teljes Cortex-M firmware megírása, hanem annak megértése, hogy a „gép” fogalma profilonként változik: a regiszterek és a cikluslogika átvihető, a futtatási szerződés nem.

16.1.2 Definíció: A-profile, M-profile és a bare-metal futási modell

A-profile (Application profile): olyan architektúraprofil, amelyet összetett operációs rendszerek alatti alkalmazásokhoz terveztek. Az Armv8-A AArch64 végrehajtási állapota ide tartozik; jellemzője az MMU, a többszintű kivételkezelés (exception level, EL0–EL3), a virtuális memória és a gazdag rendszerhívási felület.

M-profile (Microcontroller profile): mikrokontrolleres beágyazott feladatokra szabott profil. A könyv II. része az Armv7-M architektúrát és a Cortex-M3 referenciamagot használja. Thumb-2 utasításkészlet, NVIC (Nested Vectored Interrupt Controller), opcionális MPU, nincs MMU. Az architektúra **nem** zárja ki az operációs rendszert: Cortex-M-en gyakori RTOS-ek (FreeRTOS, Zephyr, RTX stb.) is futnak, saját ütemezővel és `SVC`-alapú szolgáltatáshívással. Az, hogy a könyv II. részének laborjaiban nincs operációs rendszer, a választott **bare-metal tanulási modell** sajátossága, nem M-profile architektúra-követelmény.

Bare-metal: operációs rendszer nélküli programfuttatás, amelyben a program saját maga biztosítja az indulási környezetet: a verem beállítását, az inicializált adatok másolását, a `.bss` törlését, majd a fő rutin meghívását. Reset után a hardver a vektortábla **első** bejegyzéséből (a kezdeti veremmutató értékéből) tölti be az `SP`-t, majd a **második** bejegyzésben található resetkezelő címére ugrik; ez utóbbi a tényleges belépési pont (lásd Reset és boot-folyamat: vektortábla).

Thread mód és Handler mód: az M-profile egyszerűsített privilege-modellje. A Thread módban fut a normál program, amely lehet privileged vagy unprivileged. A Handler módban futnak a kivételkezelők és megszakításkezelők; ez mindig privileged. Nincs EL0–EL3 hierarchia: a hardver csak azt különbözteti meg, hogy éppen kivételkezelés történik-e.

Memory-mapped I/O: olyan perifériakezelési modell, amelyben a periféria regiszterei a normál memóriacím-térben jelennek meg. A program `STR`/`LDR` utasításokkal éri el őket; nincs külön `IN`/`OUT` utasítás, mint egyes más architektúrákon. A Cortex-M-en a perifériák fix fizikai címeken vannak, és a programozott cím közvetlenül a buszra kerül.

16.1.3 Mi tűnik el az AArch64 modellhez képest?

A profilváltás négy dolgot szüntet meg vagy változtat meg alapvetően. Az első három architektúraszintű különbség — az M-profile maga zárja ki vagy alakítja át ezeket; a negyedik, mint alább kiderül, nem architektúra-korlát, hanem a könyv laborjának választása.

1. Nincs MMU. Az AArch64 Linux alatt a program virtuális címeket használ, amelyeket az MMU lapozási táblákon keresztül képez le fizikai címekre. A Cortex-M-en nincs lapozás: a programban szereplő cím az a fizikai cím, amely a memóriabuszra kerül. Ez egyszerűbbé teszi a címszámítást, de elveszik a virtuális memória nyújtotta izoláció, a laphiba-alapú memóriakezelés és a klasszikus ASLR. Az M-profile opcionális MPU-ja (Memory Protection Unit) tud védelmet adni régiókra, de ez nem lapozás, és nem hoz létre virtuális címtérképet. Ez architektúraszintű tény: az M-profile sosem ad MMU-t.

2. Nincs operációs rendszer-szintű syscall-szerződés. Az AArch64-en a `write` rendszerhívás az `SVC #0` utasítással kerül a kernelbe, amely ellenőrzi az argumentumokat, a fájlleírót, a puffert, majd a megfelelő illesztőprogramot hívja. Az M-profile architektúrájában az `SVC` utasítás és a hozzá tartozó SVCall kivétel (kivételszám 11, a tábla 0x2C eltolása; lásd 20.2) **létezik** — ezt használják például az RTOS-ek privilégium-váltáshoz —, de **ebben a bare-metal laborban** nincs operációs rendszer, amely kiszolgálná: nincs `X8`-hoz hasonló egységes syscall-szám-konvenció, nincs kernel-oldali pufferelés vagy blokkolás. A kimenet ezért egy perifériaregiszter közvetlen írása, például a UART adatregiszterébe. A hiba detektálása sem automatikus: ha a periféria nem tudja fogadni az adatot, azt a programnak kell ellenőriznie egy állapotregiszterből.

3. Nincs exception level-hierarchia. Az AArch64-en az alkalmazás EL0-ban, a kernel EL1-ben fut; a kivételkezelés szintjei szigorúan elválnak. Az M-profile-on csak Thread mód és Handler mód van. A Handler mód mindig privileged, a Thread mód lehet unprivileged, de nincs négyfokozatú skála. Ez egyszerűbb, de kevesebb izolációs lehetőséget ad — és ez is architektúraszintű tény, nem labor-választás.

4. Nincs operációs rendszer — ebben a laborban. Ez, az előző hárommal szemben, **nem** M-profile architektúra-korlát: Cortex-M-en kereskedelmi és nyílt forráskódú RTOS-ek (FreeRTOS, Zephyr, RTX stb.) is futnak, saját ütemezővel és szál- vagy folyamatmodellel. A jelen laborban nincs ütemező, nincs folyamat, nincs fájlrendszer, nincs dinamikus linker — ez a könyv II. részének pedagógiai döntése, a bare-metal modell egyszerűsége miatt. A program a reset vektorból indul, és jellemzően soha nem „tér vissza”: vagy végtelen ciklusban fut, vagy alacsony fogyasztású módba kerül. A megszakítások jelentik a valódi párhuzamosságot, nem a folyamatok vagy szálak.

16.1.4 Miért tipikus a Cortex-M mikrokontroller és a beágyazott felhasználás?

A Cortex-M-et alacsony fogyasztásra, kis chipterületre és determinisztikus válaszidőre tervezték. Egy tipikus beágyazott rendszer egyetlen vezérlési feladatot lát el: motorvezérlés, szenzorolvasás,

kommunikációs protokoll kezelése. Az MMU hiánya nem hátrány, hanem költségcsökkentő egyszerűsítés: nincs szükség lapozási táblákra, TLB-re, és a címfordítás elmarad. A megszakítás-latencia kiszámíthatóbb, mert nincs kernel, amely előbb ütemezne. A perifériák közvetlenül címezhetők, így a válaszidő néhány tíz ciklus nagyságrendű lehet. Ezért a Cortex-M a valós idejű, erőforrás-korlátos rendszerek tipikus választása.

16.1.5 Mi marad meg az I. rész tudásából?

A jó hír, hogy az I. rész jelentős része átvihető. A **load/store architektúra** elve ugyanaz: a memóriát `LDR`/`STR` típusú utasítások érik el, a műveletek regisztereken folynak. A **ciklusszervezés**, a számláló dekrementálása és a feltételkód-alapú elágazás szemantikailag azonos. A **bitmanipuláció** (maszkolás, shift, logikai műveletek) nagy része megmarad, még ha a mnemonicok és az immediate-kódolási korlátok el is térnek. A **stack használat** és a hívási konvenció is megmarad, de az M-profile saját AAPCS-változattal rendelkezik, és két veremmutatót használ (MSP, PSP). Az **utasítások szemantikája** gyakran átvihető, a **kódolás és a regiszternevek** azonban nem.

Ez az utolsó mondat a II. rész kulcsa: a C-driverkódok mögött mindig egy **assembly-szintű hívási szerződés** áll, és ez Cortex-M-en nem az AAPCS64, hanem az AAPCS32. Mielőtt a vektortáblával, a startup kóddal és a perifériákkal foglalkoznánk, ezt a szerződést és a Thumb hívási mechanikáját rendezzük el, majd ugyanazt a C-függvényt mindkét ISA-n megírjuk.

16.1.5.1 Átvezetés: az AAPCS32 hívási szerződése

Az alábbi táblázat az I. részben megismert AAPCS64-et (Az AAPCS64 regiszterszerződés) állítja párba a Cortex-M-en érvényes AAPCS32-vel (Armv7-M, Thumb-2):

Szerep	AAPCS64 (AArch64)	AAPCS32 (Cortex-M, Thumb-2)
egész/mutató argumentumok	`X0`–`X7`, a többi a veremben	`R0`–`R3`, a többi a veremben
64 bites argumentum	egy `Xn`	páros–páratlan regiszterpár (`R0:R1` vagy `R2:R3`), a veremben 8 bájtra igazítva
visszatérési érték	`X0` (128 bites: `X0:X1`)	`R0`; 64 bites: `R0:R1`
hívó által mentett (scratch)	`X0`–`X17`	`R0`–`R3`, `R12` (IP: a linker által beszúrt veneer is használhatja)
hívott által mentett	`X19`–`X29`, valamint `V8`–`V15` alsó 64 bitje (`D8`–`D15`)	`R4`–`R11` (platformfüggő részletekkel, lásd lent)
link register	`X30`	`R14` (`LR`)
veremmutató	`SP`, 16 bájtra igazítva	`SP` (Cortex-M-en `MSP` vagy `PSP`); nyilvános interfészen **8 bájtra** igazítva, közben legalább 4-re
keretmutató	`X29`	nem kötelező; a GCC Thumb-kódban `R7`-et használja keretmutatónak, ARM-állapotban `R11`-et
platformregiszter	`X18`	`R9`

Az `R4`–`R11` platformfüggő részletei. Az AAPCS32 szerint mind a nyolc regiszter callee-saved, de kettőnek a platform külön szerepet adhat. Az `R9` a **platformregiszter**: egy platform kijelölheti statikus bázisnak (SB, pozíciófüggetlen adat) vagy szálhelyi tárolónak. Ilyenkor a függvények nem használhatják szabadon. Ha a platform semmihez nem rendeli, közönséges callee-saved regiszter. Az `R7` a GCC Thumb-kódjában keretmutató, ha a keretmutató be van kapcsolva (`-fno-omit-frame-pointer`); egy assembly-függvény ilyenkor is használhatja, ha visszaállítja, de a hibakereső veremvisszakövetése rá épül. Bare-metal Cortex-M3 laborjainkban egyik sincs lefoglalva, de egy RTOS vagy más fordító ezt másként döntheti el: a szerződés részleteit mindig a platform dokumentációja rögzíti.

A kivételbelépés és a hívási szerződés. A Cortex-M kivételbelépéskor a hardver pontosan a hívó által mentett halmazt menti a verembe (`R0`–`R3`, `R12`, `LR`, plusz `PC` és `xPSR`). Ezért írható a megszakításkezelő közönséges C-függvényként (ISR írása): az `R4`–`R11` megőrzése a fordító dolga, ugyanúgy, mint bármely más függvényben.

A 8 bájtos igazítás. Nyilvános interfészen, vagyis minden olyan hívási ponton, ahol a hívott fél más fordítási egységből vagy külső kódból származhat, az `SP` 8-cal osztható kell legyen. Ez a 64 bites adatok (`LDRD`/`STRD`, `double`, `uint64_t`) miatt kell. Egy függvényen belül az `SP` átmenetileg csak 4-gyel osztható is lehet. A gyakorlati következmény: a prológus **páros számú** regisztert mentsen (például `push {r4, lr}`; öt regiszter helyett, mint a `push {r4-r7, lr}`, inkább hat: `push {r3-r7, lr}`), különben a következő `bl` már igazítatlan veremmel hív.

16.1.5.2 Hívás és visszatérés: BL, BX és a Thumb függvényjelölés

- `BL címke`: a hívás az `LR`-be a visszatérési címet írja, **bit0 = 1**-gyel, mert a hívó Thumb-állapotban van; a visszatérés erre a címre ugrik.
- `BLX Rm`: regiszteres (közvetett) hívás, például függvénymutatón keresztül. Az `Rm` bit0-jának 1-nek kell lennie; a Cortex-M nem ismer ARM-állapotot, ezért egy páros cím `INVSTATE` UsageFaultot okoz.
- **Visszatérés**: `BX LR`. Ha a függvény más függvényt is hív, és ezért az `LR`-t mentenie kell, a prológus `push {..., lr}`, az epilógus pedig `pop {..., pc}`: a mentett cím közvetlenül a `PC`-be töltődik. Levélfüggvény az `LR`-t nem menti.
- **Thumb függvényjelölés**: az assemblyben a `.thumb_func` (a `.type név, %function` mellett) jelzi, hogy a szimbólum Thumb-függvény. Ettől lesz a szimbólum értéke páratlan, és ezt a páratlan értéket kapja minden, ami a címét veszi: függvénymutató, `BLX`-cél, vektortábla-bejegyzés (vektortábla). A jelölés hiányában a `BL` még működik (a linker közvetlen ágat fordít), de a cím alapú hívás páros címet kap, és hibát okoz.

Az AArch64-hez képest a mechanika tehát ugyanaz (a `BL` az LR-be ír, a visszatérés az LR-en át megy), két különbséggel: a visszatérési cím bit0-ja állapotjelző, és a nem levélfüggvény `pop {pc}`-vel is visszatérhet.

16.1.5.3 Páros példa: ugyanaz a C-függvény A64-ben és Thumb-2-ben

Munkakönyvtár: `~/arm-labor/cm/a061/`. A példa egyetlen C-szerződést valósít meg mindkét ISA-n, és **ugyanaz a teszthajtó** ellenőrzi mindkettőt. A teszthajtó a függvény eredménye mellett a hívási szerződést is méri: A64-en az 8.3 `lab_abi_call` teszthívójával, Thumb-2-n annak itt bevezetett párjával, a `lab_abi_call32`-vel. A Thumb-futtatáshoz a Linker script alapjai és a Startup kód kanonikus párja kell. Ha azok még nincsenek meg, a Thumb-oldali futtatás az alfejezetek után végezhető el; a fordítás és a disassembly addig is ellenőrizhető.

```
mkdir -p ~/arm-labor/cm/a061 && cd ~/arm-labor/cm/a061
cp ~/arm-labor/a64/a029/lab-abi-call.h ~/arm-labor/a64/a029/lab-abi-call.S .
cp ~/arm-labor/cm/a065/lab-mps2.ld ~/arm-labor/cm/a066/lab-startup.s .   # a Thumb-futtatáshoz
```

A közös szerződés:

```
/* lab-sum.h - a közös C-szerződés mindkét ISA-ra */
#ifndef LAB_SUM_H
#define LAB_SUM_H
#include <stdint.h>
/* p[0..n-1] összege modulo 2^32; n == 0 esetén 0. */
uint32_t lab_sum_u32(const uint32_t *p, uint32_t n);
#endif
```

A64 (AAPCS64: `x0` = `p`, `w1` = `n`, eredmény `w0`):

```
// lab-sum-a64.s - A64, AAPCS64: x0 = p, w1 = n; visszatérés w0-ban
    .text
    .global lab_sum_u32
    .type   lab_sum_u32, %function
lab_sum_u32:
    mov     w2, #0                  // összeg: caller-saved w2, nincs mentés
    cbz     w1, 2f
1:  ldr     w3, [x0], #4            // post-index: p++
    add     w2, w2, w3
    subs    w1, w1, #1
    b.ne    1b
2:  mov     w0, w2
    ret                             // visszatérés az x30-ban kapott címre
    .size   lab_sum_u32, . - lab_sum_u32
    .section .note.GNU-stack,"",%progbits
```

Thumb-2 (AAPCS32: `r0` = `p`, `r1` = `n`, eredmény `r0`). A változat szándékosan callee-saved regisztereket (`r4`, `r5`) használ, hogy a mentés és a páros `push` látható legyen:

```
@ lab-sum-t32.s - Thumb-2, AAPCS32: r0 = p, r1 = n; visszatérés r0-ban
    .syntax unified
    .thumb
    .text
    .global lab_sum_u32
    .thumb_func                     @ a szimbólum Thumb-függvény: címének bit0-ja 1
    .type   lab_sum_u32, %function
lab_sum_u32:
    @ callee-saved regiszterek; páros darabszám: SP 8-ra igazított marad
    push    {r4, r5}
    movs    r4, #0
    cbz     r1, 2f
1:  ldr     r5, [r0], #4            @ post-index: p++
    adds    r4, r4, r5
    subs    r1, r1, #1
    bne     1b
2:  mov     r0, r4
    pop     {r4, r5}
    bx      lr                      @ visszatérés: LR bit0 = 1, Thumb marad
    .size   lab_sum_u32, . - lab_sum_u32
```

A Thumb-2 ABI-teszthívó az `R4`–`R11`-be ismert mintát tölt, a hívás előtti `SP`-t egy statikus horgonyba (`lab_abi32_sp`) menti, `BLX`-szel meghívja a függvényt, majd ellenőrzi a mintákat és az `SP`-t. Az A64-es párhoz hasonlóan (8.3) saját keretét csak a horgonyból állítja vissza (`mov sp, r3`), így a hívott által elrontott `SP`-ről sem a `&result`-ot, sem a mentett regisztereket és az `LR`-t nem olvassa. A horgony miatt **nem reentráns**: egymásba ágyazott hívásnál a belső hívás

nem indítja el a függvényt, hanem `LAB_ABI32_BUSY` hibabitet ad. Ő maga 10 regisztert ment (40 bájt), így a hívás pontján az `SP` 8-ra igazított marad:

```
/* lab-abi-call32.h - AAPCS32 ABI-teszthívó Thumb-2-re (a lab_abi_call párja). */
#ifndef LAB_ABI_CALL32_H
#define LAB_ABI_CALL32_H
#include <stdint.h>
typedef void (*lab_abi_fn32)(void);
/* hibamaszk: bit 0..7 = R4..R11 sérült, bit 8 = SP nem a hívás előtti */
#define LAB_ABI32_R4   (1u << 0)
#define LAB_ABI32_SP   (1u << 8)
#define LAB_ABI32_BUSY (1u << 9)  /* újrabelépés: fn nem hívódott meg */
/* fn(a0, a1) hívása úgy, hogy előtte R4-R11 ismert mintát kap; visszatérés
   után ellenőrzi a mintákat és az SP-t. *result = a hívott R0-ja.
   NEM reentráns: a hívás előtti SP-t egyetlen statikus horgony őrzi;
   egymásba ágyazott hívásnál a belső LAB_ABI32_BUSY-t ad, *result = 0. */
uint32_t lab_abi_call32(lab_abi_fn32 fn, uint32_t a0, uint32_t a1, uint32_t *result);
#endif
```

```
@ lab-abi-call32.s - AAPCS32 megőrzési teszthívó (Thumb-2, Armv7-M)
@ r0 = fn, r1 = a0, r2 = a1, r3 = &result; visszatérés r0 = hibamaszk
    .syntax unified
    .thumb
    .text
    .global lab_abi_call32
    .thumb_func
    .type   lab_abi_call32, %function
lab_abi_call32:
    @ 10 regiszter = 40 bájt: SP 8-ra igazított marad; r3 = &result
    push    {r3-r11, lr}
    mov     r12, r0               @ fn (r12: scratch, a hívó szabadon használja)
    ldr     r4, =lab_abi32_sp     @ horgony: 0 = szabad, különben egy hívás már fut
    ldr     r5, [r4]
    cmp     r5, #0
    bne     3f                    @ újrabelépés: fn nem hívható
    mov     r5, sp
    str     r5, [r4]              @ a hívás előtti SP
    ldr     r4,  =0x44444444      @ minták a callee-saved regiszterekbe
    ldr     r5,  =0x55555555
    ldr     r6,  =0x66666666
    ldr     r7,  =0x77777777
    ldr     r8,  =0x88888888
    ldr     r9,  =0x99999999
    ldr     r10, =0xAAAAAAAA
    ldr     r11, =0xBBBBBBBB
    mov     r0, r1                @ a0
    mov     r1, r2                @ a1
    blx     r12                   @ BLX: LR = visszatérési cím | 1
    movs    r2, #0                @ hibamaszk
    ldr     r1, =0x44444444
    cmp     r4, r1
    it      ne
    orrne   r2, r2, #(1 << 0)
    ldr     r1, =0x55555555
    cmp     r5, r1
    it      ne
    orrne   r2, r2, #(1 << 1)
    ldr     r1, =0x66666666
    cmp     r6, r1
    it      ne
    orrne   r2, r2, #(1 << 2)
    ldr     r1, =0x77777777
    cmp     r7, r1
```

```
    it      ne
    orrne   r2, r2, #(1 << 3)
    ldr     r1, =0x88888888
    cmp     r8, r1
    it      ne
    orrne   r2, r2, #(1 << 4)
    ldr     r1, =0x99999999
    cmp     r9, r1
    it      ne
    orrne   r2, r2, #(1 << 5)
    ldr     r1, =0xAAAAAAAA
    cmp     r10, r1
    it      ne
    orrne   r2, r2, #(1 << 6)
    ldr     r1, =0xBBBBBBBB
    cmp     r11, r1
    it      ne
    orrne   r2, r2, #(1 << 7)
    ldr     r1, =lab_abi32_sp
    ldr     r3, [r1]              @ a hívás előtti SP a horgonyból
    mov     r12, sp
    cmp     r12, r3
    it      ne
    orrne   r2, r2, #(1 << 8)
    @ helyreállítás csak a horgonyból: a sérült SP-t nem használjuk
    mov     sp, r3
    movs    r3, #0
    str     r3, [r1]              @ a horgony újra szabad
    b       4f
3:  mov     r2, #(1 << 9)         @ LAB_ABI32_BUSY; a horgonyt nem írjuk felül
    movs    r0, #0
4:  ldr     r3, [sp]              @ a mentett &result
    str     r0, [r3]              @ *result = a hívott r0-ja
    mov     r0, r2
    pop     {r3-r11, pc}          @ visszatérés: a mentett LR közvetlenül a PC-be
    .size   lab_abi_call32, . - lab_abi_call32

    .bss
    .align  2
lab_abi32_sp:
    .space  4
```

A közös teszthajtó. Csak a platformréteg (kiírás, ABI-teszthívó, kilépés) különbözik, a tesztesetek és az ellenőrzés logikája azonos:

```
/* lab-sum-test.c - közös teszthajtó: ugyanaz a logika A64-en és Thumb-2-n.
 * Csak a platformréteg (kiírás, ABI-teszthívó, kilépés) különbözik. */
#include <stdint.h>
#include "lab-sum.h"

/* ---------- platformréteg ---------- */
#if defined(__aarch64__)
#include <stdio.h>
#include "lab-abi-call.h"
static void out(const char *s) { fputs(s, stdout); }
static uint32_t call_checked(const uint32_t *p, uint32_t n, uint32_t *mask)
{
    uint64_t x0 = 0;
    *mask = (uint32_t)lab_abi_call((lab_abi_fn)lab_sum_u32, (uintptr_t)p, n, 0, 0, 0, 0, &x0);
    return (uint32_t)x0;                    /* uint32_t visszatérés: csak W0 értelmes */
}
static int finish(int fails) { return fails != 0; }
#else
```

```
#include "lab-abi-call32.h"
static void sh_call(uint32_t op, const void *arg)
{
    register uint32_t r0 asm("r0") = op;
    register const void *r1 asm("r1") = arg;
    asm volatile ("bkpt 0xab" : "+r"(r0) : "r"(r1) : "memory");
}
static void out(const char *s) { sh_call(0x04u, s); }          /* SYS_WRITE0 */
static uint32_t call_checked(const uint32_t *p, uint32_t n, uint32_t *mask)
{
    uint32_t r0 = 0;
    *mask = lab_abi_call32((lab_abi_fn32)lab_sum_u32, (uint32_t)(uintptr_t)p, n, &r0);
    return r0;
}
static int finish(int fails)
{
    sh_call(0x18u, (const void *)(fails ? 0x20023u : 0x20026u)); /* SYS_EXIT */
    return fails != 0;
}
#endif

/* ---------- közös tesztlogika ---------- */
static void out_hex(uint32_t v)
{
    char b[11] = "0x";
    for (int i = 0; i < 8; i++) b[2 + i] = "0123456789abcdef"[(v >> (28 - 4 * i)) & 0xFu];
    b[10] = '\0';
    out(b);
}

static const uint32_t data[8] = { 1, 2, 3, 0xFFFFFFFFu,
                                  0x80000000u, 0x80000000u, 7, 0x12345678u };

int main(void)
{
    static const uint32_t ns[] = { 0, 1, 3, 4, 8 };
    int fails = 0;
    for (unsigned c = 0; c < sizeof ns / sizeof ns[0]; c++) {
        uint32_t n = ns[c], ref = 0, mask;
        for (uint32_t i = 0; i < n; i++) ref += data[i];       /* C-referencia, mod 2^32 */
        uint32_t got = call_checked(data, n, &mask);
        int ok = got == ref && mask == 0;
        fails += !ok;
        out("n="); out_hex(n); out(" sum="); out_hex(got);
        out(" abi_mask="); out_hex(mask); out(ok ? " OK\n" : " HIBA\n");
    }
    out(fails ? "SUMTEST: FAIL\n" : "SUMTEST: PASS\n");
    return finish(fails);
}

# A64 (qemu-user)
aarch64-linux-gnu-gcc -O1 -static -o lab-sum-a64 lab-sum-test.c lab-sum-a64.s lab-abi-call.S
qemu-aarch64 ./lab-sum-a64
# Thumb-2 (Cortex-M3, qemu-system-arm, semihosting)
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 -T lab-mps2.ld \
    -o lab-sum-t32.elf lab-startup.s lab-sum-t32.s lab-abi-call32.s lab-sum-test.c
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -nographic -semihosting \
    -kernel lab-sum-t32.elf </dev/null
```

A `-nographic` mellett a QEMU a terminál stdin-jét is olvassa; a `</dev/null` lezárja a bemenetet, így szkriptből vagy egymás után másolt parancsokból futtatva a QEMU nem nyeli el a következő sorokat. Kimenet mindkét platformon, a fő úton mérve (GNU + QEMU 8.2.2, a lektor mérése); a két kimenet karakterre azonos:

```
n=0x00000000 sum=0x00000000 abi_mask=0x00000000 OK
n=0x00000001 sum=0x00000001 abi_mask=0x00000000 OK
n=0x00000003 sum=0x00000006 abi_mask=0x00000000 OK
n=0x00000004 sum=0x00000005 abi_mask=0x00000000 OK
n=0x00000008 sum=0x12345684 abi_mask=0x00000000 OK
SUMTEST: PASS
```

A 8 elemes eset a modulo 2^{32} összeadást is próbálja: $1+2+3+(2^{32}-1)+2\cdot 2^{31}+7+\texttt{0x12345678} \equiv \texttt{0x12345684}$.

Ellenpróba. Ha a Thumb-változatból kihagyjuk a `push {r4, r5}` / `pop {r4, r5}` párt, az összegek ugyanúgy helyesek, de az `abi_mask` `0x00000003` lesz (sérült az `R4` és az `R5`; az `n = 0` esetben csak az `R4`, ezért ott `0x00000001`), és a teszt `SUMTEST: FAIL`-lel zárul. Az eredmény helyessége tehát nem igazolja a hívási szerződést; azt külön kell mérni, mindkét ISA-n.

A teszthívó saját negatív tesztje. Magát a `lab_abi_call32`-t is ellenpróbázni kell, mert a `LAB_ABI32_SP` bit csak akkor ér valamit, ha a teszthívó a hibát **túléli**. Két hívott szabályosan, `BX LR`-rel tér vissza, de az `SP`-t 4, illetve 8 bájttal eltolva hagyja. A teszthívót egy assembly hívó (`lab_abi32_outer`) hívja meg, amely előtte saját mintát tölt az `R4`–`R11`-be, és utána ellenőrzi, hogy a mintái és az `SP`-je megmaradt-e. Az eredményszó két oldalán őrszó áll, így az is kiderül, ha a teszthívó idegen címre írna. Elvárás: mindkét esetben pontosan `LAB_ABI32_SP` a maszk, a 31. bit (a hívó állapota sérült) nem áll be, az eredmény 3, és az őrszavak épek.

```
@ lab-abi32-neg.s - a lab_abi_call32 negatív tesztjei (Thumb-2, Armv7-M)
    .syntax unified
    .thumb
    .text

@ HIBÁS hívottak: szabályosan, BX LR-rel térnek vissza, de eltolt SP-vel.
    .global lab_t32_bad_sp4
    .thumb_func
    .type   lab_t32_bad_sp4, %function
lab_t32_bad_sp4:
    sub     sp, sp, #4
    adds    r0, r0, r1
    bx      lr
    .size   lab_t32_bad_sp4, . - lab_t32_bad_sp4

    .global lab_t32_bad_sp8
    .thumb_func
    .type   lab_t32_bad_sp8, %function
lab_t32_bad_sp8:
    sub     sp, sp, #8
    adds    r0, r0, r1
    bx      lr
    .size   lab_t32_bad_sp8, . - lab_t32_bad_sp8

@ uint32_t lab_abi32_outer(lab_abi_fn32 fn, uint32_t *result)
@ A teszthívó saját hívója: r4-r11-be mintát tölt, meghívja a
@ lab_abi_call32(fn, 1, 2, result)-ot, majd ellenőrzi, hogy a mintái és az
@ SP-je megmaradt. Visszatérés: a teszthívó maszkja, bit 31 = a hívó állapota sérült.
    .global lab_abi32_outer
    .thumb_func
    .type   lab_abi32_outer, %function
lab_abi32_outer:
    push    {r4-r11, lr}          @ 9 regiszter
    sub     sp, sp, #4            @ + 4 bájt: SP 8-ra igazított
    mov     r3, r1                @ result
    mov     r12, r0               @ fn
    ldr     r4,  =0x14141414
    ldr     r5,  =0x15151515
```

```
    ldr     r6,  =0x16161616
    ldr     r7,  =0x17171717
    ldr     r8,  =0x18181818
    ldr     r9,  =0x19191919
    ldr     r10, =0x1A1A1A1A
    ldr     r11, =0x1B1B1B1B
    mov     r1, sp
    str     r1, [sp]              @ a saját SP-nk, a keretünk aljára
    mov     r0, r12
    movs    r1, #1
    movs    r2, #2
    bl      lab_abi_call32
    ldr     r1, [sp]
    mov     r2, sp
    cmp     r1, r2                @ a mi SP-nk változatlan?
    bne     9f
    ldr     r1, =0x14141414
    cmp     r4, r1
    bne     9f
    ldr     r1, =0x15151515
    cmp     r5, r1
    bne     9f
    ldr     r1, =0x16161616
    cmp     r6, r1
    bne     9f
    ldr     r1, =0x17171717
    cmp     r7, r1
    bne     9f
    ldr     r1, =0x18181818
    cmp     r8, r1
    bne     9f
    ldr     r1, =0x19191919
    cmp     r9, r1
    bne     9f
    ldr     r1, =0x1A1A1A1A
    cmp     r10, r1
    bne     9f
    ldr     r1, =0x1B1B1B1B
    cmp     r11, r1
    beq     1f
9:  orr     r0, r0, #0x80000000   @ a hívó állapota sérült
1:  add     sp, sp, #4
    pop     {r4-r11, pc}
    .size   lab_abi32_outer, . - lab_abi32_outer

/* lab-abi32-neg-test.c - a lab_abi_call32 negatív tesztje: eltolt SP-vel visszatérő
 * hívottak. */
#include <stdint.h>
#include "lab-abi-call32.h"

extern void lab_t32_bad_sp4(void), lab_t32_bad_sp8(void);
uint32_t lab_abi32_outer(lab_abi_fn32 fn, uint32_t *result);

static void sh_call(uint32_t op, const void *arg)
{
    register uint32_t r0 asm("r0") = op;
    register const void *r1 asm("r1") = arg;
    asm volatile ("bkpt 0xab" : "+r"(r0) : "r"(r1) : "memory");
}
static void out(const char *s) { sh_call(0x04u, s); }          /* SYS_WRITE0 */
static void out_hex(uint32_t v)
{
    char b[11] = "0x";
    for (int i = 0; i < 8; i++) b[2 + i] = "0123456789abcdef"[(v >> (28 - 4 * i)) & 0xFu];
```

```
    b[10] = '\0';
    out(b);
}

/* result körül őrszavak: a teszthívó csak a középső szót írhatja. */
static int expect(const char *name, lab_abi_fn32 fn)
{
    volatile uint32_t box[3] = { 0xC0DEC0DEu, 0, 0xC0DEC0DEu };
    uint32_t mask = lab_abi32_outer(fn, (uint32_t *)&box[1]);
    int ok = mask == LAB_ABI32_SP && box[1] == 3
          && box[0] == 0xC0DEC0DEu && box[2] == 0xC0DEC0DEu;
    out(name); out(" mask="); out_hex(mask); out(" result="); out_hex(box[1]);
    out(ok ? " OK\n" : " HIBA\n");
    return !ok;
}

int main(void)
{
    int fails = expect("bad_sp4", lab_t32_bad_sp4);
    fails += expect("bad_sp8", lab_t32_bad_sp8);
    out(fails ? "ABI32-NEG: FAIL\n" : "ABI32-NEG: PASS\n");
    sh_call(0x18u, (const void *)(fails ? 0x20023u : 0x20026u)); /* SYS_EXIT */
    return fails;
}
```

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 -T lab-mps2.ld \
    -o lab-abi32-neg.elf lab-startup.s lab-abi32-neg-test.c lab-abi32-neg.s lab-abi-call32.s
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -nographic -semihosting \
    -kernel lab-abi32-neg.elf </dev/null
```

Kimenet a fő úton mérve (GNU + QEMU 8.2.2, a lektor mérése):

```
bad_sp4 mask=0x00000100 result=0x00000003 OK
bad_sp8 mask=0x00000100 result=0x00000003 OK
ABI32-NEG: PASS
```

Ez azért működik, mert a teszthívó az összehasonlítás után **először** a horgonyból tölti vissza az `SP`-t (`mov sp, r3`), és csak utána olvassa a vermet (`ldr r3, [sp]`, `pop {r3-r11, pc}`). A helyreállítás tehát sem a hívott által elrontott `SP`-től, sem egy esetleg sérült callee-saved regisztertől nem függ. Egy korábbi változat az ellenőrzés után is a sérült `SP`-ről hajtotta végre ezt a két olvasást. Ugyanezen a teszten a `pop` a hívó `R11`-mintáját töltötte a `PC`-be (`0x1b1b1b1a`), és a futás fault-tal leállt: éppen azt a hibát nem élte túl, amelyre külön bitje van.

A két megvalósítás összevetése:

Szempont	A64	Thumb-2
argumentumok	`x0`, `w1`	`r0`, `r1`
munkaregiszterek	`w2`, `w3` (caller-saved, nincs mentés)	`r4`, `r5` (callee-saved, ezért `push`/`pop`)
post-index betöltés	`ldr w3, [x0], #4`	`ldr r5, [r0], #4`
visszatérés	`ret` (X30)	`bx lr` (LR bit0 = 1)
függvényjelölés	`.type …, %function`	`.thumb_func` + `.type …, %function`

16.1.6 Kidolgozott példa: karakter kiírása N alkalommal

Munkakönyvtár: `~/arm-labor/cm/a061/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a061
cd ~/arm-labor/cm/a061
```

A feladat: írjuk ki az `'X'` karaktert tízszer egymás után. Az AArch64-oldalon a Hello world tisztán assemblyben `write`-syscall mintáját és a Ciklusszervezés számlálós ciklusát használjuk újra. A Cortex-M-oldalon ugyanazt a ciklus- és számlálólogikát tartjuk meg, csak a kimeneti lépést cseréljük le.

AArch64 változat (A64 ISA, A-profile):

```
    .section .rodata
ch:     .ascii "X"
    .text
    .global _start
_start:
    mov     x19, #10            // N = 10
    adr     x20, ch             // buffer címe
loop:
    mov     x0, #1              // fd = stdout
    mov     x1, x20             // buffer
    mov     x2, #1              // hossz = 1 bájt
    mov     x8, #64             // write syscall száma
    svc     #0                  // rendszerhívás
    subs    x19, x19, #1        // számláló csökkentése
    b.ne    loop                // ha nem nulla, ismétlés
    mov     x0, #0
    mov     x8, #93             // exit
    svc     #0
```

Build és futtatás:

```
aarch64-linux-gnu-as -o lab-a061-a64.o lab-a061-a64.s
aarch64-linux-gnu-ld -o lab-a061-a64 lab-a061-a64.o
qemu-aarch64 ./lab-a061-a64
```

Várt kimenet: `XXXXXXXXXX`, kilépési kód 0. A `write` visszatérési értékét itt nem ellenőrizzük; ez a példa egyszerűsítése.

Cortex-M3 változat (Thumb-2, Armv7-M ISA, M-profile):

```
    .syntax unified
    .thumb
    .equ UART0_DATA, 0x40004000    @ QEMU mps2-an385 CMSDK UART0 adatregisztere
    .text
    .global main
main:
    ldr     r1, =UART0_DATA        @ r1 = UART0 adatregiszter címe
    movs    r2, #0x58              @ 'X' ASCII kódja
    movs    r0, #10                @ N = 10
loop:
    strb    r2, [r1]               @ egy bájt kiírása a UART adatregiszterbe
    subs    r0, r0, #1             @ számláló csökkentése
    bne     loop                   @ ha nem nulla, ismétlés
done:
    b       done                   @ nincs OS, amely visszatérjen; végtelen ciklus
```

Ez a töredék **csak előkészített UART mellett** értelmezhető, és nem lesz belőle önálló firmware pusztán attól, hogy startup kódot adunk mellé. A CMSDK APB UART az adatregiszterbe írt bájtot csak akkor küldi ki, ha három előfeltétel teljesül, és a töredék egyiket sem teljesíti:

1. **baud-osztó:** a `BAUDDIV` regiszter (`+0x10`) értéke legalább 16;
2. **engedélyezett adó:** a `CTRL` regiszter (`+0x08`) `TXEN` bitje (bit0) 1;

3. **szabad adópuffer:** minden írás előtt a `STATE` regiszter (`+0x04`) `TXFULL` bitje (bit0) 0 legyen; tele pufferbe írt bájt elveszik. A töredék ezt nem teszi; a `lab-a061-uart.s` igen (3. lépés).

Letiltott adóval a valódi UART semmit nem küld ki. A QEMU modelljétől sem várható, hogy ilyenkor kiírja a karaktert; a pontos viselkedés a használt QEMU-verzióban ellenőrizendő, de a programnak nem szabad rá építenie. A töredék tehát egy olyan állapotot feltételez, amelyet valaki (bootloader, korábbi kód) már beállított. Az önállóan induló változat ugyanaz a ciklus, előtte a két beállítással, és minden írás előtt a `TXFULL` figyelésével:

```
/* lab-a061-uart.s - 'X' tízszer, előkészített UART0-val (CMSDK APB UART) */
    .syntax unified
    .cpu cortex-m3
    .thumb
    .equ UART0_BASE,   0x40004000      @ QEMU mps2-an385: CMSDK UART0
    .equ UART_DATA,    0x00            @ adatregiszter
    .equ UART_STATE,   0x04            @ bit0: TXFULL (az adópuffer tele)
    .equ UART_CTRL,    0x08            @ bit0: TXEN (adó engedélyezése)
    .equ UART_BAUDDIV, 0x10            @ baud-osztó, legalább 16

    .text
    .thumb_func
    .global main
    .type   main, %function
main:
    ldr     r1, =UART0_BASE
    movs    r3, #16                    @ 1. baud-osztó (a CMSDK minimuma)
    str     r3, [r1, #UART_BAUDDIV]
    movs    r3, #1                     @ 2. adó engedélyezése
    str     r3, [r1, #UART_CTRL]
    movs    r2, #0x58                  @ 'X' ASCII kódja
    movs    r0, #10                    @ N = 10
loop:
1:  ldr     r3, [r1, #UART_STATE]      @ 3. várakozás, amíg az adópuffer tele
    lsls    r3, r3, #31                @ bit0 -> N jelző
    bmi     1b
    str     r2, [r1, #UART_DATA]       @ egy karakter kiírása
    subs    r0, r0, #1                 @ számláló csökkentése
    bne     loop                       @ ha nem nulla, ismétlés
done:
    b       done                       @ nincs OS, amelyhez visszatérhetne
    .size   main, . - main
```

A vektortáblát, a linker scriptet és a `main`-t hívó startup kódot a Reset és boot-folyamat: vektortábla, a Linker script alapjai és a Startup kód alfejezetek építik fel. Ha azok elkészültek, a két kanonikus fájllal ez a változat már önálló firmware:

```
cp ~/arm-labor/cm/a065/lab-mps2.ld ~/arm-labor/cm/a066/lab-startup.s .
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -nostdlib -T lab-mps2.ld \
    -o lab-a061.elf lab-startup.s lab-a061-uart.s
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -serial stdio \
    -monitor none -display none -kernel lab-a061.elf </dev/null   # Ctrl-C-vel leállítható
```

A `-serial stdio -monitor none -display none` a UART0 kimenetét a terminálra irányítja, a `</dev/null` pedig lezárja a bemenetet, így a QEMU nem olvassa és nem nyeli el a következő sorokat. A fő úton (GNU + QEMU 8.2.2, a lektor mérése) a `lab-a061.elf` UART0-n a `XXXXXXXXXX` kimenetet adta; a tíz karakter azt is igazolja, hogy a várakozó ág minden írásnál átengedett. A `UART0_DATA` címe és a regiszterek eltolása a QEMU `mps2-an385` dokumentált memóriatérképéből, illetve a CMSDK UART leírásából származik. A bitmezők részletes kezelése,

a vételi oldal és a hibakezelés a Memory-mapped perifériák és GPIO, illetve az Egyszerű UART driver témája; itt a kimeneti store-utasítás és a minimális előkészítés szemléltetése a cél.

Összehasonlítás: mi azonos és mi új?

Elem	AArch64 (A64 ISA)	Cortex-M3 (Thumb-2)	Átvihető?
Számláló dekrementálása	`SUBS x19, x19, #1`	`SUBS r0, r0, #1`	Igen, a szemantika azonos
Feltételes elágazás	`B.NE loop`	`BNE loop`	Igen, a feltételkód-logika azonos
Ciklusinvariáns	„a hátralévő iterációk száma”	ugyanaz	Igen
Kimeneti mechanizmus	`SVC #0`, `X8` = 64	`STRB` fix fizikai címre	Nem, teljesen új
Címkezelés	virtuális cím, MMU	fizikai cím, nincs MMU	Nem
Kernel szerepe	pufferelés, hibakezelés, blokkolás	nincs kernel	Nem
Privilegizáltság	EL0/EL1	Thread/Handler mód	Nem

A táblázat két réteget mutat. Az **első réteg** a vezérlési logika: a számláló, a dekrementálás, a feltételkód és az elágazás szó szerint azonos elven működik. A **második réteg** a kimeneti mechanizmus: az AArch64-en a `write` egy rendszerhívás, amelynek van syscall-száma, argumentumszerződése és visszatérési értéke; a Cortex-M-en a `STRB` egyetlen memóriahozzáférés, amelynek nincs visszatérési értéke, nincs pufferelése, és a cím fix. A „kimenet” fogalma rendszerhívásból hardverregiszter-írássá alakul.

16.1.7 Tipikus hibák és ellenpróbák

1. `SUB` helyett `SUBS`. Ha a Cortex-M kódban `subs` helyett `sub`-ot írunk, a feltételkódok nem frissülnek, és a `BNE` a ciklusban **utoljára lefutott flagíró utasítás** Z értéke alapján dönt. A végén tesztelő ciklus törzse ettől függetlenül legalább egyszer lefut: ha Z = 1, pontosan egyszer, ha Z = 0, végtelen ciklus lesz belőle. Hogy melyik eset áll fenn, azt nem a „véletlen”, hanem a ciklus utasításai döntik el (lásd a 3. megoldást). Ellenpróba: `objdump -d`-vel ellenőrizzük, hogy a `SUBS` kódolása szerepel-e.

2. Immediate-kódolási korlát. A 16 bites `MOVS` csak 8 bites immediátot fogad el; a 32 bites `0x40004000` ugyan `MOV.W`-vel is kódolható (ismétlődő bájtminta: `0xXY00XY00`, itt `f04f 2140`), de egy tetszőleges 32 bites címhez `LDR =` vagy `MOVW`/`MOVT` kell. A példa ezért használ `LDR r1, =UART0_DATA` pszeudoutasítást, amely literálpoolból tölt. Ellenpróba: `objdump -d` mutatja a literálbetöltést és a pool tartalmát.

3. `STR` és `STRB` a UART adatregiszterére. A CMSDK APB UART regiszterei szóhozzáférésűek (32 bit); az adatregiszter alsó 8 bitje értelmes. A könyv példája ezért `str`-t használ; a `strb` csak akkor helyes, ha a periféria dokumentációja kifejezetten megengedi a bájtírást. QEMU-ban mindkettő működik, de a szóírás a hordozhatóbb.

4. A UART FIFO tele lehet. A valódi drivernek a state regiszterből kell ellenőriznie, hogy a TX puffer fogad-e adatot. Itt ezt nem tesszük; a teljes kezelés a Egyszerű UART driver témája.

5. A QEMU kimenetének irányítása. A `qemu-system-arm -machine mps2-an385` a UART kimenetét a host soros konzoljára irányítja; ennek beállítása a Firmware build és QEMU-betöltés munkafolyamat alfejezetben következik.

6. Az AArch64 `write` visszatérési értéke. A példa nem ellenőrzi az `X0`-t; részleges írás előfordulhat. A hibakezelés az Egyszerű instrumentáció és a GDB assembly szinten eszközeivel vizsgálható.

16.1.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Sorold fel, melyik négy jellemző tűnik el az AArch64 Linux-userspace modellből a Cortex-M3-ra lépve. Mindegyikhez írj egy mondatot arról, hogy ez mit jelent a gyakorlatban a program számára.

2. (implementációs, alapszint — cél: rutinépítés) Módosítsd a Cortex-M3 kódot úgy, hogy a kiírt karakter `'A'` legyen, és a ciklus 20-szor fusson le. Nevezd meg, melyik utasítás változik, és miért.

3. (hibakeresési, középszint — cél: rutinépítés) A `lab-a061-uart.s`-ben cseréld a `subs r0, r0, #1` utasítást `sub r0, r0, #1`-re. Utasításonként keresd meg, melyik az utolsó NZCV-író utasítás a `bne` előtt, és ebből vezesd le, hány karakter jelenik meg. Hogyan igazolhatod `objdump`-pal, hogy valóban `SUB` szerepel a kódban, és milyen kódolást választott az assembler?

4. (átviteli, középszint — cél: fejezetek összekapcsolása) Válaszd ki az alábbiak közül, mi vihető át szó szerint az I. részből a Cortex-M programozásba: (a) a számlálós ciklus szerkezete, (b) a `write` syscall szerződése, (c) a bitmanipulációs műveletek elve, (d) az AAPCS64 regiszterszerződés. Indokolj.

16.1.9 Megoldások

1. megoldás — Segítség. Nézd át a fejezet "Mi tűnik el az AArch64 modellhez képest?" négy pontját — melyik architektúraszintű tény, és melyik a labor saját választása?

1. megoldás — Teljes megoldás. A négy jellemző: (i) MMU — nincs virtuális cím, a programozott cím fizikai, ez architektúraszintű M-profile tény; (ii) operációs rendszer-szintű syscall-szerződés — az `SVC` utasítás és az SVCall kivétel architekturálisan létezik M-profilon, de ebben a laborban nincs kernel, amely kiszolgálná, ezért nincs `X8`-hoz hasonló syscall-szám-konvenció, a kimenet közvetlen regiszterírás; (iii) exception level-hierarchia — csak Thread és Handler mód van, nincs EL0–EL3, ez is architektúraszintű tény; (iv) operációs rendszer — ez a labor választása, nem architektúra-korlát: Cortex-M-en RTOS is futhatna, de a jelen laborban nincs ütemező, folyamat, fájlrendszer. A gyakorlatban ez azt jelenti, hogy a programnak magának kell kezelnie az indulást, a perifériák állapotát és a hibákat.

2. megoldás — Segítség. Két konstans vár cserére: az ASCII karakterkód és az ismétlésszám — a ciklusogikát (léptetés, feltétel) kell-e egyáltalán hozzányúlni?

2. megoldás — Teljes megoldás. A `movs r2, #0x58` helyére `movs r2, #0x41` kerül (`'A'` ASCII kódja), a `movs r0, #10` helyére pedig `movs r0, #20`. A `strb`, a `subs` és a `bne` változatlan marad, mert a ciklusogika független a konkrét karaktertől és a számláló kezdőértékétől. Ez pontosan azt mutatja, hogy a vezérlési szerkezet átvihető.

3. megoldás — Segítség. A `BNE` a Z flag állapotától függ. Ha a `SUB` nem frissíti, akkor melyik utasítás írta utoljára a flageket, mielőtt a vezérlés a `bne`-hez ért? Nézd meg, milyen feltétellel jut el a vezérlés a várakozó ciklusból a `str`-ig.

3. megoldás — Teljes megoldás. A `SUB` nem frissíti a feltételkódokat. A `bne` tehát a ciklusban utoljára lefutott flagíró utasítás Z értékét látja. A `lab-a061-uart.s` egy iterációjában a `bne` előtt ezek futnak le:

Utasítás	Ír NZCV-t?	Hatás
`ldr r3, [r1, #UART_STATE]`	nem	
`lsls r3, r3, #31`	**igen** (N, Z, C)	a `STATE` bit0-ja (`TXFULL`) kerül a 31. bitre; Z = 1, ha az eredmény 0
`bmi 1b`	nem	csak akkor esik át, ha N = 0, azaz `TXFULL = 0`
`str r2, [r1, #UART_DATA]`	nem	
`sub r0, r0, #1`	**nem**	
`bne loop`	olvassa	Z a `lsls`-től

A `str`-ig csak `TXFULL = 0` mellett jut el a vezérlés. Ekkor a `lsls` eredménye 0, tehát Z = 1. A `bne` ezért **soha nem ugrik**, a ciklustörzs pontosan **egyszer** fut le: egyetlen `X` jelenik meg, `r0 = 9` marad, és a program a `done` ciklusba kerül. A viselkedés determinisztikus, nem a ciklus előtti „véletlen” Z-értéktől függ.

Az `objdump -d` a `sub r0, r0, #1`-et 32 bites `sub.w r0, r0, #1` alakban mutatja (Thumb-2 T3 kódolás, S = 0, `f1a0 0001`). A `subs r0, r0, #1` IT-blokkon kívül 16 bites, flagbeállító utasítás (`1e40`). A kódolás hossza tehát már a listában is elárulja a különbséget.

A flagelőzmény hatása egy periféria nélküli, minimális ciklusban tisztán levezethető. Itt a ciklus előtti `cmp` az utolsó flagíró, és a törzs `add`-ja (S nélkül) sem ír flaget:

```
    movs    r0, #3          @ számláló
    movs    r1, #0          @ lefutott törzsek száma
    cmp     r0, r0          @ Z = 1   (változat: cmp r0, #0 -> Z = 0)
1:  add     r1, r1, #1      @ add.w: flaget nem ír
    sub     r0, r0, #1      @ sub.w: flaget nem ír
    bne     1b              @ Z a cmp-től
```

Előre beállított Z = 1 mellett a `bne` nem ugrik: a törzs egyszer fut le (`r1 = 1`, `r0 = 2`). Z = 0 mellett a `bne` mindig ugrik, a számláló túlcsordul, és a ciklus végtelen. A fő úton (GNU + QEMU 8.2.2, `qemu-system-arm`, a lektor mérése) mindhárom esetet lefuttattuk: a minimális ciklus Z = 1-nél egyszer futott le, Z = 0-nál 1000 utasítás után is futott, a `sub`-os `lab-a061-uart.s` pedig pontosan egy `X`-et írt. Ez a hiba jól mutatja, hogy a feltételkódok explicit állapotot hordoznak, és a ciklus helyes működése azon múlik, melyik utasítás írta őket utoljára.

4. megoldás — Segítség. Melyik kettő tisztán algoritmikus-logikai mintázat, amely nem függ az operációs rendszertől vagy az ABI-tól — és melyik kettő kifejezetten az AArch64 Linux-userspace konkrét szerződése?

4. megoldás — Teljes megoldás. Átvihető: (a) a számlálós ciklus szerkezete és (c) a bitmanipulációs műveletek elve. Nem vihető át: (b) a `write` syscall szerződése, mert M-profile-on nincs syscall, és (d) az AAPCS64 regiszterszerződés, mert az M-profile saját, 32 bites AAPCS-változatot használ. Az indoklás: az (a) és (c) tisztán algoritmikus-logikai mintázat, amely nem függ a privilegizáltsági szinttől vagy a memóriamodelltől; a (b) és (d) viszont az operációs rendszer és az AArch64 ABI konkrét szerződései.

16.1.10 Rövid összefoglaló

Az A-profile és M-profile közötti váltás nem utasításkészlet-csere, hanem futtatási modellváltás. Az MMU és az exception level-hierarchia architektúraszinten eltűnik; az operációs rendszer és a hozzá kötött syscall-szerződés hiánya ezzel szemben a könyv választott bare-metal laborjának, nem az M-profile architektúrának a következménye — Cortex-M-en RTOS is futtatható, `SVC`-alapú szolgáltatáshívással. Helyette fizikai címek, Thread/Handler mód, közvetlen perifériaírás és reset vektoros indulás jelenik meg. A vezérlési logika – számláló, dekrementálás, feltételkód, elágazás – átvihető; a kimeneti mechanizmus és a címkezelés nem. A Cortex-M azért tipikus mikrokontroller-választás, mert kiszámítható, kis erőforrású és közvetlen hardverelérést ad.

Továbbhasznált definíciók:

- **M-profile:** mikrokontrolleres architektúraprofil, Armv7-M, Thumb-2, NVIC, opcionális MPU, MMU nélkül.
- **Bare-metal:** operációs rendszer nélküli futtatás, saját indulási környezettel.
- **Memory-mapped I/O:** perifériaregiszterek a normál memóriacím-térben, `LDR`/`STR` hozzáféréssel.

16.1.11 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Arm M-profile architektúra-dokumentáció — az M-profile, a Thread/Handler mód és az NVIC elsődleges forrása.
- Armv7-M Architecture Reference Manual, DDI0403E — az Armv7-M programozási és végrehajtási modellje; a könyv referenciarevíziója DDI0403E.
- GNU assembler kézikönyv — Thumb-2 szintaxis, `LDR =` pszeudoutasítás, szekciók és direktívák.
- QEMU dokumentáció — a `mps2-an385` machine és a CMSDK perifériák dokumentált memóriatérképe.

Ellenőrzési hiányok: a `lab_abi_call32` negatív tesztjét (a fenti, 4 és 8 bájttal elállított `SP`-vel visszatérő két hívott) a fő úton, `arm-none-eabi-gcc` + `qemu-system-arm` alatt mértük (a lektor mérése); a `LAB_ABI32_BUSY` ágat (egymásba ágyazott hívás, majd újra helyes hívott a horgony felszabadulásának igazolására) a helyettesítő `tools/cm-run.py` emulátorban futtattuk. A `0x40004000` UART0-cím és a CMSDK UART adatregiszter offsetje a QEMU `mps2-an385` 8.2.2 verziójában a fenti `XXXXXXXXXX` kimenetet adja; más QEMU-verzióval a laborban ellenőrizendő. A UART FIFO állapotkezelése, a TX-megszakítás és a teljes driver a Memory-mapped perifériák és GPIO és az Egyszerű UART driver alfejezetekben következik. A QEMU `mps2-an385` UART-kimenetének host konzolra irányítása a Firmware build és QEMU-betöltés munkafolyamat témája; ennek pontos paraméterezése QEMU-verziófüggő lehet.

Kísérőcsomag, helyettesítő futtató és közös ellenőrzési táblázat. A könyv „helyettesítő eszközlánccal mérve” jelölt eredményei, laborforrásai és próbái egy verziózott kísérőcsomagban érhetők el: `dsk-arm-assembly-tools-1.0.tar.gz`, SHA-256: `33651bab4c62eb93f6f668d1409dc844a4c50ab6976df1c8acbe20f0de9487bb`, letöltés: github.com/gellistvan/dsk-arm-assembly-tools, v1.0 kiadás, vagy ugyanez a fájl a könyv HTML-kiadása mellől: `tar.gz`, `.sha256`; ellenőrzés: `sha256sum -c dsk-arm-assembly-tools-1.0.tar.gz.sha256`. A csomag tartalma:

- a futtatók: `lab-run.py` (AArch64 Linux user-mode) és `cm-run.py` (Cortex-M3), mindkettő `--version` kapcsolóval;
- a függőségek rögzített verziói: Python 3.12, unicorn 2.1.4, capstone 5.0.7, pyelftools 0.33, `zig cc` 0.17.0;
- a `cm-run.py` támogatási mátrixa (`README.md`): UART0 RX/TX, Timer0 megszakítással, Timer1 időalapként, NVIC, SysTick szoftveres pendingként, `WFI`, semihosting; GPIO és DWT nélkül; nem cikluspontos;
- a könyv kódblokkjaival bájtra azonos laborforrások;
- a mutációs és negatív próbák (az a060 auditmutánsai, a SysTick-prioritásmutáns, a hibás protokoll-DUT-ok);
- a `MANIFEST.sha256`, és a csomag kiadásakor futtatott próbák naplói (`logs/`).

Egy üres munkakönyvtárban a csomag további fájlkeresés nélkül futtatható: `sha256sum -c MANIFEST.sha256`, majd `bash tests/run.sh substitute` (helyettesítő út), illetve `bash tests/run.sh gnu-qemu` (a könyv fő útja: `aarch64-linux-gnu-*`, `arm-none-eabi-gcc`, `qemu-aarch64`, `qemu-system-arm -machine mps2-an385`). Mindkét mód ugyanazokból a fájlokból épít, próbánként `PASS`/`FAIL` sort ír, és naplót készít, amelynek elején a csomagverzió, a manifest-ellenőrzés és az eszközverziók állnak. A helyettesítő eredmények **kiegészítő** bizonyítékok a saját hatókörükben. Nem helyettesítik a fő utat, és nem igazolják automatikusan a QEMU-parancsokat, a QEMU NVIC-jét vagy timerét. A II. rész ellenőrzési állapota egy helyen, a v1.0 csomaghoz kötve:

Labor	Helyettesítő úton (`logs/substitute-1.0-20261028.log`)	Fő úton (GNU + QEMU `mps2-an385`)	Ciklusmérés (DWT)
16.1 páros példa, `lab_abi_call32` negatív teszt	`PASS` (`SUMTEST`, `ABI32-NEG`)	`PASS` (`SUMTEST`, `ABI32-NEG`; a lektor mérése)	—
16.4–16.7 vektortábla, linker script, startup, firmware build	`PASS`: minden labor-ELF név szerinti vektorellenőrzése	ugyanígy	—
17.2 NVIC-regisztertesztt	futott (`--prio-bits 3`), a csomag tesztkészletén kívül	a QEMU 8 prioritásbitje miatt ott `0xff` a visszaolvasás	—
17.3, 17.4, 19.2, 19.3 IRQ-laborok	`PASS`: Timer0 IRQ8 boot, `LATENCY: SKIP`, SysTick-preempció és mutánsa, `UART ready` + echo, `tick` sorok	ugyanígy	`SKIP` (a QEMU 8.2.2-n is)
20.2–20.3 záróprojekt, `lab-m19-systest.py`	`PASS` (`--no-timing`): boot, echo, 50 bájtos burst, tick, reset 0 és 3 s késéssel; a hat hibás protokoll-DUT `FAIL`; T1 és K5 `SKIP`	ugyanígy	`SKIP`

A fő út kiadási elfogadásához a csomag `tests/run.sh gnu-qemu` futtatása kell, a kijelölt QEMU-verzióval és a GNU eszközlánccal; a `logs/gnu-qemu-*.log` napló kerül a csomagba. A nem

támogatott időmérés ott is `SKIP` marad. A későbbi alfejezetek „Ellenőrzési hiányok” pontjai erre a táblázatra épülnek.

16.2 Thumb-2 ISA áttekintés

16.2.1 Motiváció: miért kell ismerni a Thumb-2 kódolást?

Képzeljünk el egy Cortex-M3 alapú motorvezérlőt, ahol a megszakításkezelő rutin (ISR) minden egyes ciklusa számít. A rendszer egy 72 MHz-es órajelen fut, és a szabályozási hurok 10 µs-onként lefut. Ha az ISR kódja néhány tíz bájttal hosszabb a szükségesnél, a hosszabb kód több flash-helyet foglal, és több utasításlehívást igényel, ami a végrehajtási időt is növelheti. A Cortex-M3 utasításkészlete, a Thumb-2, változó hosszúságú kódolást használ: a gyakori műveletek 16 biten, a bonyolultabbak 32 biten érhetők el. A kódméret és a sebesség közötti egyensúly megértéséhez elengedhetetlen, hogy tisztában legyünk azzal, melyik utasítás melyik kódolási formában érhető el, milyen regisztereket ér el, és hogyan viselkedik az állapotregiszter. Ez az alfejezet az I. részben megismert A64 ISA után áttér a Thumb-2-re, és megalapozza a Cortex-M3 assembly-programozást.

16.2.2 Definíció: a Thumb-2 ISA és a Cortex-M3 regiszterkészlete

A **Thumb-2** egy olyan utasításkészlet-architektúra (ISA), amely 16 bites és 32 bites utasításkódolást egyaránt tartalmaz. Ez az Armv7-M architektúra kötelező utasításkészlete; a Cortex-M3 processzor kizárólag Thumb-2 módban működik, ARM (A32) mód nem érhető el rajta. A 16 bites utasítások az eredetileg ARMv4T-ben bevezetett Thumb utasításkészlet továbbfejlesztett változatai, amelyek korlátozottabbak: jellemzően csak az R0–R7 regisztereket érik el, és szűkebb immediate-tartományt kódolnak. A 32 bites utasítások – amelyeket gyakran „Thumb-2” néven emlegetünk szűkebb értelemben – már a teljes R0–R15 regiszterkészletet elérik, nagyobb immediate-eket és összetettebb címzési módokat támogatnak.

A **Cortex-M3 regiszterkészlete** 16 darab 32 bites regiszterből áll. Ezek szerepe a következő:

Regiszter	Név	Szerep
R0–R12	általános célú	szabadon használható adattárolásra; R0–R7 „low”, R8–R12 „high” regiszter
R13	SP (Stack Pointer)	veremmutató; a verem tetejére mutat
R14	LR (Link Register)	visszatérési cím tárolása `BL` utasítás esetén
R15	PC (Program Counter)	a következő végrehajtandó utasítás címe

Három különböző „szélességet” érdemes szétválasztani, mert a mindennapi szóhasználat összemossa őket:

Fogalom	Cortex-M3 (Armv7-M)	Megjegyzés
regiszterszélesség	minden általános célú regiszter 32 bites	nincs Xn/Wn-szerű nézetpár

Fogalom	Cortex-M3 (Armv7-M)	Megjegyzés
eredmény- és adatszélesség	jellemzően 32 bit, de van **64 bites eredmény** is: az `UMULL`/`SMULL` ($32 \times 32 \rightarrow 64$ bit) és az `UMLAL`/`SMLAL` két regiszterbe írja az eredményt, az `LDRD`/`STRD` egy duplaszót (8 bájt) mozgat két regiszter között	64 bites összeadás `ADDS` + `ADC` lánccal készül
utasításhossz	16 vagy 32 bit (2 vagy 4 bájt)	független a regiszter- és az adatszélességtől

A regiszterek tehát **egységesen 32 bitesek**, és ez alapvető eltérés az AArch64-hez képest, ahol az Xn regiszter 64 bites, a Wn pedig ugyanazon regiszter 32 bites nézete. A 32 bites regiszterekből viszont nem következik, hogy a hardver semmilyen 64 bites műveletet nem ismer: a hosszú szorzás és a duplaszavas memóriaelérés két regiszter együttes használatával, de egyetlen utasítással történik.

A **PC** (R15) olvasásakor Thumb állapotban a kapott érték **az aktuális utasítás címe + 4**, függetlenül attól, hogy az aktuális utasítás 16 vagy 32 bites. (Ez tehát nem általánosan a „következő utasítás + 4”: 16 bites utasításnál a következő utasítás címe +2, 32 bitesnél +4.) Néhány használatnál további igazítás is számít. A PC-relatív betöltés (`LDR Rt, [PC, #imm]`, illetve a `LDR Rt, =érték` literáljai) és az `ADR` az `Align(PC, 4)` értékből indul, vagyis az aktuális utasítás címe + 4 lefelé 4 többszörösére kerekítve. Ezt a részletet a későbbi, elágazásokat és literálokat tárgyaló alfejezetekben hasznos lesz szem előtt tartani. A **Thumb bit** (az EPSR egyik bitje) mindig 1, és a `BX`/`BLX` utasítások kezelik a váltást. Mivel ARM mód nincs, ha egy `BX`/`BLX`-cél vagy vektorbejegyzés bit0-ja 0, a mag `INVSTATE` UsageFaultot ad (lásd 16.4).

16.2.3 A 16 és 32 bites kódolás mechanizmusa

A Thumb-2 utasítások változó hosszúságúak. A processzor a memóriából mindig 16 bites félszavakat (halfword) olvas. Az első félszó felső 5 bitje dönti el, hogy az utasítás 16 vagy 32 bites-e. Ha a felső 5 bit értéke `11101`, `11110` vagy `11111`, akkor az utasítás 32 bites, és a következő félszó is az utasításhoz tartozik. Minden más esetben az utasítás 16 bites. Ez a séma teszi lehetővé a tömör kódolást: a gyakori, egyszerű műveletek 16 biten, a ritkább, összetettebbek 32 biten érhetők el. Az alábbi táblázat néhány tipikus utasítást mutat be mindkét csoportból. Két írásmódot következetesen megkülönböztetünk:

- **bájtsor (memóriasorrend):** ahogy a bájtok a memóriában egymás után következnek, növekvő címen (little-endian);
- **félszavak (objdump-alak):** ahogy az `objdump -d` kiírja. Minden 16 bites félszó egy hexa szám (a két bájtja little-endian olvasatban), 32 bites utasításnál az **első** félszó áll elöl. A `f245 6378` tehát a `45 f2 78 63` bájtsort jelenti.

A táblázat minden sora tényleges assembler-kimenetből származik (a forrás és az ellenőrzés módja a Források szakaszban):

Utasítás	Hossz	Bájtsor (memória)	Félszavak (objdump)	Megjegyzés
`MOVS r0, #1`	16 bit	`01 20`	`2001`	R0–R7, 8 bites immediate
`ADDS r2, r0, r1`	16 bit	`42 18`	`1842`	R0–R7, flagfrissítés
`CMP r2, #30`	16 bit	`1E 2A`	`2a1e`	R0–R7, 8 bites immediate
`ADD.W r0, r0, r1, LSL #2`	32 bit	`00 EB 81 00`	`eb00 0081`	eltolt regiszteroperandus
`MOVW r3, #0x5678`	32 bit	`45 F2 78 63`	`f245 6378`	16 bites immediate az alsó félbe, a felső fél 0
`MOVT r3, #0x1234`	32 bit	`C1 F2 34 23`	`f2c1 2334`	16 bites immediate a felső félbe, az alsó marad

A táblázat kódolásait a fő úton, GNU binutils 2.42-vel (`arm-none-eabi-as`, `arm-none-eabi-objdump`) ellenőriztük (a lektor mérése); minden félszó egyezik. A `0x12345678` konstans **egyetlen** utasításba nem kódolható. A `MOV.W` csak „módosított immediate”-et fogad el: egy 8 bites értéket eltolva, vagy bizonyos ismétlődő bájtmintákat (`0x00XY00XY`, `0xXYXYXYXY` stb.). Az assembler a `MOV.W r3, #0x12345678` sort ezért elutasítja (GNU as 2.42: `Error: invalid constant (12345678) after fixup`; a lektor mérése). A teljes 32 bites konstanst a `MOVW` + `MOVT` pár tölti be. Egy gyakori hibás idézet itt a `4F F0 78 56` bájtsor; ez valójában a `MOV.W r6, #0x3e000000` utasítás.

A 16 bites utasítások előnye a kisebb kódméret, hátrányuk a korlátozott regiszter- és immediate-elérés. A 32 bites utasítások nagyobb rugalmasságot adnak, de négy bájtot foglalnak. A fordítóprogram és az assembly-programozó feladata eldönteni, mikor melyiket használja. A `MOVS r0, #1` 16 biten elfér, de a `MOV r0, #0x1234` már 32 bites kódolást igényel (az assembler `MOVW`-t választ).

16.2.4 Az APSR állapotregiszter és az NZCV-jelzők

Az **APSR** (Application Program Status Register) egy 32 bites regiszter, amelynek néhány bitje van definiálva. A legfontosabbak az **NZCV-jelzők**: N (negative), Z (zero), C (carry) és V (overflow). Ezek megegyeznek az AArch64-ben megismert szemantikával, de elérésük módja eltér. Az APSR-t az `MRS` és `MSR` utasításokkal lehet olvasni, illetve írni, közvetlenül nem címezhető.

Bit	Név	Jelentés
31	N	negatív eredmény
30	Z	nulla eredmény
29	C	átvitel (carry) vagy kölcsön (borrow)
28	V	előjeles túlcsordulás

A Thumb-2 utasítások többsége csak akkor frissíti a jelzőket, ha az utasítás neve **S** utótagot kap. Például az `ADDS` frissíti az NZCV-t, az `ADD` nem. A `CMP` utasítás mindig frissíti a jelzőket, mert az összehasonlítás lényege a flagbeállítás. A 16 bites Thumb utasításoknál gyakran az S utótag implicit: az `ADDS r2, r0, r1` 16 bites formája mindig frissít, míg a 32 bites `ADD.W` alapértelmezésben nem frissít, csak `ADDS.W` esetén. Ez a látszólagos következetlenség a történelmi fejlődés eredménye, és a programozónak tudatosan kell kezelnie.

16.2.5 Kidolgozott példa: 16 és 32 bites utasítások keverése

Munkakönyvtár: `~/arm-labor/cm/a062/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a062
cd ~/arm-labor/cm/a062
```

Az alábbi assembly rutin bemutatja a Thumb-2 kódolás kevert használatát. A példa összeszerelhető az `arm-none-eabi-as` assemblerrel, és az `objdump` segítségével ellenőrizhető a kódolás hossza.

```
    .syntax unified
    .thumb
    .global lab_thumb2_mix
    .text
lab_thumb2_mix:
    movs    r0, #10          @ 16 bites: r0 = 10
    movs    r1, #20          @ 16 bites: r1 = 20
    adds    r2, r0, r1       @ 16 bites: r2 = 30, NZCV frissül
    cmp     r2, #30          @ 16 bites: Z = 1
    BEQ     lab_ok           @ 16 bites: elágazás, ha Z = 1
    movw    r3, #0x5678      @ 32 bites: az alsó fél, a felső fél nulla
    movt    r3, #0x1234      @ 32 bites: a felső fél beállítása
lab_ok:
    bx      lr               @ 16 bites: visszatérés
```

Assemble és disassemble:

```
arm-none-eabi-as -mcpu=cortex-m3 -mthumb -o lab-thumb2-mix.o lab-thumb2-mix.s
arm-none-eabi-objdump -d lab-thumb2-mix.o
```

Kimenet a könyv `tools/thumb-dump.py` disassemblerével; a bájtsorokat a fő úton, GNU binutils 2.42-vel (`arm-none-eabi-as`, `arm-none-eabi-objdump`) ellenőriztük (a lektor mérése). A második oszlop az objdump-féle félszóalak, a szögletes zárójelben a memóriabeli bájtsor áll. A GNU `objdump` ugyanezeket a kódokat adja, csak az immediate-eket tízes számrendszerben írja (`#10`, `#22136 @ 0x5678`), a szűk elágazást pedig `beq.n` alakban:

```
00000000 <lab_thumb2_mix>:
       0:        200a             [0a 20      ]        movs        r0, #0xa
       2:        2114             [14 21      ]        movs        r1, #0x14
       4:        1842             [42 18      ]        adds        r2, r0, r1
       6:        2a1e             [1e 2a      ]        cmp        r2, #0x1e
       8:        d003             [03 d0      ]        beq        #0x12
       a:        f245 6378        [45 f2 78 63]        movw        r3, #0x5678
       e:        f2c1 2334        [c1 f2 34 23]        movt        r3, #0x1234

00000012 <lab_ok>:
      12:        4770             [70 47      ]        bx        lr
```

A disassembly lista jól mutatja a 2 bájtos (16 bites) és 4 bájtos (32 bites) utasításokat. A `0x12345678` konstans egyetlen `MOV.W` utasításban nem kódolható; a `MOVW` az alsó, a `MOVT` a felső 16 bitet állítja be. Mindkettő 4 bájtos. A `beq.n` utasítás 16 bites, a `.n` utótag a szűk kódolásra utal. A `BX lr` szintén 16 bites.

Ellenőrzési feltétel: Az `objdump` kimenetében a `movw` és `movt` 4–4 bájtot foglaljon, a többi utasítás 2 bájtot. A `beq.n` célja a `0x12` eltolású `lab_ok`. A megadott bemeneti konstansokkal a branch átugorja a két konstansbetöltést: visszatéréskor R0=10, R1=20, R2=30, Z=1; R3 értékét ez az útvonal nem változtatja meg. (A rutint emulátorban lefuttatva, R3 = `0xdead`

kezdőértékkel, pontosan ez az állapot adódott, `C = 1`-gyel együtt, mert a `CMP 30, 30` kivonás nem kér kölcsönt.)

16.2.6 Tipikus hibák és ellenpróbák

Hiba 1: High regiszter használata 16 bites utasításban.
Ha a programozó `MOVS r8, #1` utasítást ír, a GNU as `.syntax unified` mellett **nem jelez hibát**, hanem csendben a 32 bites `MOVS.W r8, #1` alakot választja (mérve: `f05f 0801`). A kód hosszabb lesz, és ezt csak a disassembly mutatja. Ha 16 bites kódolást akarunk kikényszeríteni, `.n` utótagot kell írni (`MOVS.N r8, #1`), ami viszont hibát ad, mert a 16 bites alak csak R0–R7-et ér el.
Tünet → hipotézis → adatgyűjtés: a disassemblyben 4 bájtos `movs.w` → a high regiszter 16 bites alakja nem elérhető → `.n` utótaggal ellenőrizzük a hibát.

Hiba 2: Immediate túlcsordulás.
A `MOVS r0, #0x100` utasítás 16 biten nem kódolható, mert a 16 bites `MOVS` immediate csak 8 bites. A GNU as `.syntax unified` mellett **nem jelez hibát**, hanem csendben a 32 bites `MOVS.W r0, #256` alakot választja (mérve: `f45f 7080`); a kód hosszabb lesz. Ha 16 bites alakot akarunk, `MOVS.N r0, #0x100` hibát ad. Ellenpróba: `MOV.W r0, #0x100` és a disassembly összevetése.
Tünet → hipotézis → adatgyűjtés: 4 bájtos `movs.w` → a 16 bites immediate-korlát → `.n` utótaggal ellenőrizzük a hibát.

Hiba 3: PC-relatív számítás figyelmen kívül hagyása. Az `ADR r0, cimke` és a PC-relatív `LDR` az `Align(PC, 4)` értékből számol, ahol a PC az `ADR` saját címe + 4. Ha a `ADR` egy 4-gyel nem osztható (csak 2-vel osztható) címen áll, a kiinduló érték az utasítás címe + 2. A kézi eltolásszámításnál mindkét lépést figyelembe kell venni. Ellenpróba: disassemble, és ellenőrizzük a betöltött címet.

16.2.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Ismertesd a Cortex-M3 regisztereit (R0–R15), és térj ki az SP, LR, PC konvencionális szerepére! Miért nincs a Cortex-M3-nak 64 bites regiszterpárja, mint az AArch64 Xn/Wn nézete?

2. (implementációs, középszint — cél: rutinépítés) Írj Thumb-2 assembly rutint, amely két 32 bites számot összead, és az eredményt az R0 regiszterbe teszi! A carry flaget (C) egy külön regiszterbe (R1) mentsd el 0 vagy 1 értékkel! Használj `ADDS` és `ADC` utasításokat! A rutin végén `BX lr`-rel térj vissza.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Adott az alábbi objdump kimenet. Azonosítsd, mely utasítások 16 és melyek 32 bitesek! Számítsd ki a teljes kódméretet bájtban! Melyik utasítás töltene be nagyobb immediate-et, és miért?

```
  0:   2305            movs    r3, #5
  2:   240a            movs    r4, #10
  4:   191b            adds    r3, r3, r4
  6:   f04f 5568       mov.w   r5, #0x3a000000
  a:   429d            cmp     r5, r3
  c:   d101            bne.n   12
  e:   2000            movs    r0, #0
 10:   e000            b.n     14
 12:   2001            movs    r0, #1
 14:   4770            bx      lr
```

16.2.8 Megoldások

1. megoldás — Segítség. Nézd meg a fejezet eleji regisztertáblázatot, és gondold át: egy nézetpárhoz (mint Xn/Wn) két ELTÉRŐ szélességű hardveres tárolóra lenne szükség ugyanahhoz a fizikai regiszterhez — van-e ilyen a Cortex-M3-ban?

1. megoldás — Teljes megoldás. - R0–R12: általános célú, szabadon használható. R0–R7 „low”, R8–R12 „high”.
- R13 (SP): veremmutató, a verem tetejére mutat.
- R14 (LR): visszatérési cím tárolása `BL` esetén.
- R15 (PC): programszámláló. Thumb utasításból olvasva az aktuális utasítás címe + 4; a PC-relatív betöltés és az `ADR` ennek 4-re lefelé igazított értékét használja. - Nincs 64 bites regiszter, mert az Armv7-M csak 32 bites általános célú regisztereket definiál. Az AArch64-ben az Xn a 64 bites, Wn a 32 bites nézet ugyanazon regiszternek, de itt minden regiszter fixen 32 bites. - Indoklás: nézetpárra azért nincs szükség, mert nincs 64 bites regiszter, amelynek egy része lehetne. Ez nem jelenti, hogy a 64 bites műveletek teljesen hiányoznának: az `UMULL`/`SMULL` 64 bites szorzatot ír két regiszterbe, az `LDRD`/`STRD` duplaszót mozgat, csak az eredmény itt mindig regiszterpárban keletkezik.

2. megoldás — Segítség. Az `ADDS` beállítja a C jelzőt, az `ADC` pedig egy harmadik operandusként hozzáadja azt egy másik összeadáshoz — hogyan használható ez arra, hogy a C értékét (0 vagy 1) egy sima regiszterbe “csapold ki”?

2. megoldás — Teljes megoldás.

```
    .syntax unified
    .thumb
    .global lab_add_carry
    .text
lab_add_carry:
    adds    r0, r0, r1      @ összeadás, flagfrissítés
    movs    r1, #0          @ r1 = 0
    ADC     r1, r1, #0      @ r1 = r1 + 0 + C
    bx      lr
```

- Az `ADDS` végzi az összeadást, beállítja a C flaget, ha átvitel keletkezett.
- A `MOVS r1, #0` az N és Z jelzőt frissíti, a C-t **nem** módosítja (immediate forrású `MOVS` esetén nincs eltolás, amely C-t adhatna), ezért a carry megmarad a következő utasításig.
- Az `ADC` a carry figyelembevételével ad hozzá 0-t, így R1 a C flag értékét veszi fel.
- Az `ADC r1, r1, #0` **32 bites** utasítás: az immediate-es `ADC`-nek nincs 16 bites kódolása, a disassembly `f141 0100`-t mutat (bájtsor: `41 f1 00 01`). A rutin így 2 + 2 + 4 + 2 = 10 bájt. Ha 16 bites alakot szeretnénk, a regiszteres `ADCS r1, r1, r1` megfelel (R1 = 0 + 0 + C), de az a jelzőket is írja.
- Ellenőrzés (emulátorban lefuttatva): R0 = 0xFFFFFFFF, R1 = 1 bemenetnél ADDS után R0 = 0, C = 1, a rutin végén R1 = 1; R0 = 5, R1 = 7 bemenetnél R0 = 12, R1 = 0.

3. megoldás — Segítség. Az objdump bal oszlopában a bájt-cím ugrása (pl. 0→2, 2→4 vs. 4→a) közvetlenül elárulja, melyik utasítás hány bájtos — nem kell a mnemonicokat fejből tudni hozzá.

3. megoldás — Teljes megoldás.
- 16 bites utasítások: `movs r3, #5`, `movs r4, #10`, `adds r3, r3, r4`, `cmp r5, r3`, `bne.n`, `movs r0, #0`, `b.n`, `movs r0, #1`, `bx lr`. - 32 bites utasítás: `mov.w r5, #0x3a000000`, kódja `f04f 5568` (bájtsor: `4f f0 68 55`). - Teljes kódméret: 9 db 16 bites (18 bájt) + 1 db 32 bites (4 bájt) = 22 bájt; ezt a címoszlop is mutatja (`0x14 + 2 = 0x16 = 22`). - A `mov.w` azért 32 bites, mert a 0x3a000000 immediate nem fér el 16 biten; a 16 bites `MOVS` csak 8 bites immediate-et

kódol. A 32 bites alak a `0x3a000000`-t módosított immediate-ként ábrázolja. A második félszó `5568` mezői: `imm3 = 101`, `Rd = 0101` (R5), `imm8 = 0x68`; az első félszó `i` bitje 0. Az `imm12 = 0:101:01101000 = 0x568` felső bitjei nem nullák, ezért elforgatott alakról van szó: az alapérték `1:1101000 = 0xE8`, a forgatás `imm12[11:7] = 01010 = 10` bit, és `0xE8 ROR 10 = 0x3A000000`. Ugyanez a dekódolás a gyakran idézett `f04f 4561` sorra `0xE1 ROR 8 = 0xE1000000`-t adna, ezért az a sor nem a `0x3a000000`-t töltötte volna be.
- A `bne.n` és `b.n` szűk elágazások, 16 bitesek.

16.2.9 Rövid összefoglaló

A Thumb-2 ISA 16 és 32 bites utasításokat kever, lehetővé téve a tömör kódméretet és a rugalmas regiszterelérést. A Cortex-M3 16 darab 32 bites regiszterrel rendelkezik: R0–R12 általános, R13 SP, R14 LR, R15 PC. Nincs Xn/Wn nézetpár, mert nincsenek 64 bites regiszterek; 64 bites eredményt adó utasítások (`UMULL`, `SMULL`, `LDRD`/`STRD`) viszont vannak, regiszterpárral. A regiszterszélesség, az eredményszélesség és az utasításhossz három külön fogalom. A PC olvasáskor az aktuális utasítás címe + 4. Az APSR tartalmazza az NZCV-jelzőket, amelyeket az S utótagú utasítások és a `CMP` frissítenek. A 16 bites utasítások jellemzően R0–R7-et érnek el, a 32 bitesek a teljes regiszterkészletet. A kódolás hosszát az első félszó felső 5 bitje dönti el.

Továbbhasznált definíciók:

- **Thumb-2:** 16 és 32 bites utasításokat egyaránt tartalmazó ISA, az Armv7-M kötelező utasításkészlete.
- **APSR:** alkalmazásprogram állapotregisztere, amely az NZCV-jelzőket tárolja.
- **low/high regiszter:** R0–R7 (low) és R8–R12 (high); a 16 bites utasítások jellemzően csak a low regisztereket érik el.

16.2.10 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm M-profile architecture documentation — az M-profile architektúra áttekintése; a Thumb-2 és a regiszterkészlet elsődleges forrása. *Ellenőrizve: a link él*; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Armv7-M Architecture Reference Manual, DDI0403E — a Thumb-2 utasításkódolás és az APSR pontos specifikációja. *Ellenőrizve: a link él* (a `developer.arm.com` → `support.arm.com` átirányítás megerősített); ugyanaz a JS-renderelési korlát érvényes, mint fent.
- GNU Binutils as dokumentáció — az assembler direktívák és a Thumb-2 szintaxis. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- QEMU dokumentáció — a `qemu-system-arm -machine mps2-an385` használata. **Tartalmilag ellenőrizve**, de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verzió-korlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a QEMU `mps2-an385` machine pontos Thumb-2 utasítástámogatása és az `arm-none-eabi-as` verziófüggő viselkedése itt nem került ellenőrzésre. A kódolási táblázatot, a kidolgozott példa listáját, a 3. feladat listáját és a 2. megoldás kódját a fő úton, GNU binutils 2.42-vel (`arm-none-eabi-as`, `arm-none-eabi-objdump`) ellenőriztük (a lektor mérése); minden közölt kódolás egyezik, a `MOV.W r3, #0x12345678` sort az assembler `Error: invalid constant (12345678)`

`after fixup` üzenettel utasítja el. A GNU `objdump` kiírási formája (decimális immediate-ek, `.n`/`.w` utótagok) a fenti listáétól eltér. A `lab_thumb2_mix` és a `lab_add_carry` rutin futási eredményét (regiszter- és jelzőállapot) Unicorn-emulátorban ellenőriztük; ezt a részt a GNU eszközlánc nem futtatta.

16.3 Memóriatérkép és a bare-metal futási modell

16.3.1 Motiváció

Tételezzük fel, hogy egy kollégánk átad egy `.bin` firmware-t, és azt kérdezi: „Ez most miért nem indul el a QEMU-ban, amikor a `main()`-ben már csak egy LED-villogtatás van?” A hibakeresés kulcsa ilyenkor nem a C-forrásban, hanem a **futási modell** ismeretében van: hol keresi a processzor az első utasítást, honnan tölti be a veremmutatót, milyen címen éri el az első periféria-regisztert, és melyik címtartomány mit jelent egy `linker script`-ben. A Cortex-M világában ezek nem operációsrendszer-konfiguráció kérdései: **rögzített, architektúra-szintű címek**, amelyeket a linker script és a startup kód együtt „töltenek fel” tartalommal.

Ez az alfejezet a Cortex-M3-as futási modell alapját adja meg: megrajzoljuk a memóriatérképet, levezetjük a régiók méretét, majd nyomon követjük, mi történik **közvetlenül a reset után, mielőtt bármilyen `main()` futna**. Az AArch64 Linuxos világából érkező olvasó számára a legnagyobb váltás: nincs MMU, nincs virtuális cím, nincs `_start` libc-ről; a memória fizikai, fix, és a linker script dönti el, mi hova kerül.

16.3.2 Definíció: a Cortex-M3 fix memóriatérképe

Az **Armv7-M architektúra** (amelybe a Cortex-M3 tartozik) egy **fix, 4 GiB-os fizikai címtér**-et definiál, nyolc darab 512 MiB-os régióra osztva. Nem opcionális: ezek a címtartományok a specifikáció részei, és a mag viselkedése (pl. milyen bitet lehet gyorsítótárazni, milyen utasítás engedélyezett) ettől függ.

Régió	Kezdőcím	Végcím	Méret	Tipikus tartalom
Code	`0x00000000`	`0x1FFFFFFF`	512 MiB	Flash / ROM, vektorok, konstans adat
SRAM	`0x20000000`	`0x3FFFFFFF`	512 MiB	statikus változók, `.data`, `.bss`, verem, halom
Peripheral	`0x40000000`	`0x5FFFFFFF`	512 MiB	memory-mapped I/O (UART, GPIO, timer)
External RAM	`0x60000000`	`0x9FFFFFFF`	1 GiB	külső memória (opcionális)
External device	`0xA0000000`	`0xDFFFFFFF`	1 GiB	külső periféria (opcionális)
System: **Private Peripheral Bus (PPB)**	`0xE0000000`	`0xE00FFFFF`	1 MiB	SCS (SCB, SysTick, NVIC, MPU), ITM, DWT, FPB

Régió	Kezdőcím	Végcím	Méret	Tipikus tartalom
System: vendor system	`0xE0100000`	`0xFFFFFFFF`	511 MiB	gyártóspecifikus

(Jelölés: a méretek bináris egységek, **1 KiB = 2^1 bájt, 1 MiB = 2^2, 1 GiB = 2^3**; a könyvben a memóriaméreteket következetesen így jelöljük. A 511 MiB = 4 GiB − 3,5 GiB − 1 MiB, ellenőrizve a végcím − kezdőcím + 1 számítással minden sorra.)

A sorok hézag és átfedés nélkül fedik a teljes 4 GiB-ot: minden kezdőcím az előző sor végcíme + 1, az utolsó végcím `0xFFFFFFFF`, és a méret mindig végcím − kezdőcím + 1. A két külső tartomány egyenként 1 GiB, vagyis két-két 512 MiB-os egység; a `0xE0000000`-tól kezdődő utolsó 512 MiB-os egységet (System) az architektúra tovább bontja PPB-re és gyártói részre.

A táblázat fogalmi jelentése: **a 32 bites cím felső 3 bitje dönti el, melyik 512 MiB-os egységbe esik** (a két 1 GiB-os külső tartomány két-két egységből áll, a `0xE0000000`-s System egységet pedig az architektúra tovább bontja). A régiók *méretét* a mag nem korlátozza — egy adott chipen belül csak egy töredékük van „kitöltve” memóriával; a lyukakra irányuló hozzáférés *fault-ot okozhat* (BusFault, vagy ha az nincs engedélyezve, HardFault), de ez nem általános garancia. Három réteget kell szétválasztani: (1) a **mag memóriatérképe** csak a régiókat és attribútumaikat adja; (2) a **SoC címdekódolása** dönti el, hogy egy cím mögött van-e eszköz, és a busz hibát jelez-e, vagy ismeretlen címre nullát olvas / eldobja az írást; (3) a **QEMU-modell** a géptípus saját leképezését valósítja meg (az `mps2-an385` hézagain más a viselkedés, mint egy valódi STM32-n). Egy adott címen a fault tehát a célrendszer dokumentációjából vagy célzott teszttel igazolandó: a tesztben engedélyezni kell a BusFault-ot (`SHCSR.BUSFAULTENA`), és a handlerben ki kell olvasni a `CFSR/BFAR` regisztert; egy igazoltan faultoló és egy eldobó/nulla-olvasó címre nem szabad ugyanazt az elvárást írni.

Definíció (memóriatérkép). A *memory map* az a hivatalos hozzárendelés, amely a 32 bites címtér szeleteit funkciókhoz (kód, RAM, periféria, mag-belső regiszterek) rendeli, és amelyet a mag és a linker script egyaránt ismer.

Definíció (bare-metal futási modell). *Bare-metal* (csupasz fém) futás alatt azt értjük, hogy a mag közvetlenül a reset-vektorról indul, nincs operációs rendszer, nincs dinamikus betöltő, minden címet (kód, verem, periféria) a fejlesztő által adott **linker script** és **startup kód** rendez el.

16.3.3 A régiók méretének levezetése

A 32 bites címtér 2^{32} bájt = 4 GiB. Ha nyolc egyenlő részre osztjuk:

$$\frac{2^{32}}{2^3} = 2^{29} \text{ bájt} = 512 \text{ MiB}.$$

Ezért a régiók **felső 3 bit** alapján szelektálhatók. Az Address bitjei `A[31:29]` a régióindex; az alsó 29 bit a régión belüli eltolás. Például:

- `0x00000000` → `A[31:29] = 0b000` → Code.
- `0x20000000` → `A[31:29] = 0b001` → SRAM.
- `0x40000000` → `A[31:29] = 0b010` → Peripheral.
- `0xE0000000` → `A[31:29] = 0b111` → System (ezt az egységet az architektúra tovább bontja).

A cím értelmezése tehát **két külön lépés**: előbb a régió azonosítása, aztán a régión belüli dekódolás.

1. **Régióazonosítás.** `A[31:29]` kiválasztja az 512 MiB-os egységet. A System egységen belül `A[31:20] = 0xE00` jelöli az 1 MiB-os PPB-t (`0xE0000000`–`0xE00FFFFF`), minden más (`0xE0100000`–`0xFFFFFFFF`) gyártói terület.
2. **Dekódolás a PPB-n belül.** A PPB minden címe ugyanazt a magas mintát hordozza (`A[31:20] = 0xE00`), ezért ezek a bitek az egyes eszközöket **nem** különítik el. Az eszközt az alacsonyabb bitek választják ki: `A[19:12]` a 4 KiB-os blokkot (például `0x0E` a *System Control Space*, SCS: `0xE000E000`–`0xE000EFFF`), azon belül `A[11:0]` a regisztert.

Kidolgozott dekódolás: `0xE000E100`.

Lépés	Bitek	Érték	Jelentés
régió	`A[31:29]`	`0b111`	System egység
PPB-e?	`A[31:20]`	`0xE00`	igen, PPB
blokk a PPB-n belül	`A[19:12]`	`0x0E`	SCS (`0xE000E000`–`0xE000EFFF`)
regiszter az SCS-ben	`A[11:0]`	`0x100`	NVIC `ISER0` (az NVIC `0xE000E100`-tól kezdődik)

Ugyanígy `0xE000E010` az SCS-en belül a `0x010` eltolás, a SysTick vezérlőregisztere (`SYST_CSR`, a SysTick blokk `0xE000E010`–`0xE000E01F`). A `0xE000E010` és a `0xE000E100` tehát `A[31:12]`-ben megegyezik, és csak az SCS-en belüli eltolásban különbözik.

16.3.4 Mit tartalmaz az egyes régió?

Code régió (`0x00000000`–). Ezen a címen található a **vektortábla** első szava (kezdeti SP), majd a második szava (reset-vektor), utána a többi kivétel belépési címe. Ezt követi a tényleges programkód (`.text`) és a konstans inicializált adatok (`.rodata`). Fizikailag ez lehet beágyazott Flash, vagy — QEMU-ban — egyszerű RAM, amit „ROM-ként” használunk. Fontos: a vektortáblának **a régió kezdetén kell lennie** (alapértelmezésben `0x00000000`), mert a mag reset után ezt a címet használja; később a VTOR (Vector Table Offset Register) átírásával elmozdítható, ezzel a következő alfejezetekben még foglalkozunk.

SRAM régió (`0x20000000`–). Itt él a statikus adat (`.data`), a nulla-kezdőértékű (`.bss`), a verem (stack) és a halom (heap). A verem **a legmagasabb címről lefelé nő** (a `PUSH` a `SP`-t csökkenti ARM-en). A `.data` szakasz tartalma a Flash-ből másolódik át a startup kódban (lásd Startup kód: .data másolása, .bss törlése), mert a módosítható C-adatoknak normál írható adattár kell, a flashből betöltött inicializáló kép pedig nem ilyen. (Ez nem tiltja, hogy egy mikrokontroller a saját flashét a vezérlőjén keresztül programozza, és a QEMU is ROM-szerű modellel tölti be a képet; az állítás a *közönséges C-változók* tárolására vonatkozik, nem minden Cortex-M-re általános tiltás.)

Periféria régió (`0x40000000`–). Itt helyezkednek el a `Memory-mapped I/O` regiszterek. A mag ugyanazokkal a `LDR`/`STR` utasításokkal éri el őket, mint a RAM-ot; a különbség az, hogy az írás/olvasás itt hardveres mellékhatást vált ki (UART-ba küld, GPIO-t billent, időzítőt indít). A pontos címek és bitmezők gyártó- és táblaspecifikusak; a részletes kezelés a Memory-mapped perifériák és GPIO fejezetben következik. Most csak annyi a fontos: a `0x40000000`-s régióban **nem szabad** „véletlen” programadatot elhelyezni.

PPB (`0xE0000000`–`0xE00FFFFF`). A mag belső perifériái: NVIC, SysTick, SCB, MPU, hibaregiszterek. Ide írva konfiguráljuk a megszakításokat (Kivételek és megszakítások: NVIC) és a rendszerszintű viselkedést.

16.3.5 A reset mechanizmusa

Amikor a mag feszültség alá kerül (vagy a `qemu-system-arm` elindul), a hardver a reset-szekvenciát hajtja végre. A Reset a vektortáblában az 1-es **vektorszámot** kapja (a második szó), de a reset **nem** szokásos kivételbelépés: nincs veremre mentett keret, nincs `EXC_RETURN`, és a mag nem Handler módba lép. A szekvencia az architektúrában rögzített, operációs rendszertől függetlenül:

1. **SP betöltése.** A mag beolvassa a `0x00000000` címen lévő 32 bites szót, és beírja a fő veremmutatóba (`MSP`, az R13 reset utáni nézete). Ez a **kezdeti veremmutató**, tipikusan az SRAM legfelső címe.
2. **PC és T-bit a reset-vektorból.** A `0x00000004` címen lévő szó a **reset-vektor**. A bit0-ja nem a cím része: az `EPSR.T` bitbe kerül, és 1-nek kell lennie, mert a Cortex-M csak Thumb állapotot ismer. A `PC`-be a bit0 nélküli érték kerül, ez a reset-handler első utasításának (páros) címe. A vektor tehát például `0x00000041`, a végrehajtás pedig a `0x00000040`-en kezdődik. Ha a vektor bit0-ja 0, az első utasítás `INVSTATE` UsageFaultot okoz, ami HardFaulttá eszkalálódik.
3. **LR beállítása.** `LR` (R14) `0xFFFFFFFF`-re áll; ez azt jelzi, hogy nincs visszatérési cím (a `BX LR` itt fault-ot okoz, ami néha hasznos a „nem szabadna ide visszatérni” jelzésre).
4. **Mód és IPSR.** A mag **privilegizált Thread módban** indul a fő veremmel (`CONTROL.nPRIV = 0`, `CONTROL.SPSEL = 0`), és az `IPSR` (Interrupt Program Status Register) értéke **0**. Az IPSR az *aktív* kivétel számát mutatja Handler módban, Thread módban pedig 0-t. A „vektorszám 1” és az „aktív kivételszám 0” tehát nem ellentmondás: a reset-handler Thread módban fut, nem egy aktív Reset-kivétel kezelőjeként.

Ezek után a CPU már a saját reset-handlerünk első utasításánál tart, **a `main()` fogalmilag még nem létezik** — a reset-handler az, ami majd meghívja.

A mechanizmus kulcsa: a mag **nem tudja**, hogy az `[SP]` és `[PC]` érték honnan „származik”; csak azt tudja, hogy a 0-s címen lévő szavak tartalmát tölti be. Ezért a linkelési folyamatnak gondoskodnia kell arról, hogy a `.vectors` szekció **pontosan a 0-s címre** kerüljön, benne a helyes kezdeti SP-vel és reset-vektorral.

A reset utáni állapot a reset-handler **első utasítása előtt** GDB-vel ellenőrizhető. A `-S` kapcsolóval indított QEMU a resetet már elvégezte, de még egyetlen utasítást sem hajtott végre:

```
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -kernel lab-a063.elf \
    -S -gdb tcp::3333 -display none -monitor none -serial null   # egyik terminál
gdb-multiarch lab-a063.elf -ex "target remote :3333"   # másik terminál

(gdb) x/2xw 0                 # vektor[0] = kezdeti SP, vektor[1] = reset-vektor (bit0 = 1)
(gdb) p/x $sp                 # = vektor[0]
(gdb) p/x $pc                 # = vektor[1] & ~1 (páros cím), a Reset_Handler első utasítása
(gdb) p/x $xpsr & 0x1ff       # IPSR = 0: Thread mód
(gdb) p/x ($xpsr >> 24) & 1   # EPSR.T = 1
(gdb) p/x $lr                 # 0xffffffff
(gdb) p/x $control            # 0: privilegizált, MSP (ha a QEMU/GDB kiteszi ezt a regisztert)
```

Az ellenőrzés lényege a vektor és a végrehajtási cím különbsége: a `x/2xw 0` második szava páratlan, a `$pc` ugyanez bit0 nélkül.

A fő úton (QEMU 8.2.2 `-S -gdb`, GDB 15.1) az `x/2xw 0` a `0x20010000 0x00000009` szavakat adta, `$sp = 0x20010000`, `$pc = 0x8`, IPSR = 0, `EPSR.T = 1`, `$lr = 0xffffffff`, `$control = 0`, `$msp = 0x20010000` (a lektor mérése); minden érték egyezik a lentebbi táblázattal.

16.3.6 A QEMU `mps2-an385` memóriatérképe

A GDB-blokkban használt `lab-a063.elf` a fejezet vektortábla-példájából így épül:

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -nostdlib -T lab-a063.ld \
    lab-a063-vectors.s -o lab-a063.elf
```

A könyv II. részének QEMU-baseline-ja az `ARM MPS2+ AN385` tábla, Cortex-M3 maggal. A dokumentáció szerinti fő tartományok (a pontos címeket és bitmezőket a QEMU hivatalos machine-definíciója és a `CMSDK` periféria-dokumentáció alapján **ellenőrizni szükséges** a használt QEMU-verzióban (`info mtree`); az alábbi értékek a QEMU `hw/arm/mps2.c` forrásában és a hivatalos MPS2+ dokumentációban szereplő baseline-t tükrözik):

Címtartomány	Méret	Szerep
`0x00000000`–`0x003FFFFF`	4 MiB	ZBT SSRAM1 — *Code* (vektortábla, kód, `.rodata`); a linker scriptben `FLASH` néven
`0x20000000`–`0x203FFFFF`	4 MiB	ZBT SSRAM 2 & 3 **együtt** — *SRAM* (`.data`, `.bss`, stack); ez az egyetlen kanonikus RAM-tartomány
`0x20400000`–`0x207FFFFF`	(4 MiB)	az előző sor **tükre** (alias): ugyanaz a fizikai tár, nem második blokk
`0x40000000`–	—	perifériatartomány kezdete (CMSDK/MPS2 profil: UART, timer, dual-timer, `FPGAIO`, `SCC` APB-n; a GPIO AHB-n csatlakozik)
`0xE0000000`–`0xE00FFFFF`	1 MiB	PPB: SCS (SCB, SysTick, NVIC)

A QEMU 8.2.2 `hw/arm/mps2.c` forrása szerint az AN385-ön a `0x20000000`–`0x203fffff` tartomány az SSRAM 2 és 3 együttese, a `0x20400000`–`0x207fffff` pedig ennek tükre. Aki a tükröt külön tárolóként számolja (például oda teszi a vermet, a `.data`-t pedig az alsó tartományba), a program saját adatait írja felül egy másik címen keresztül: a `0x20400000 + x` és a `0x20000000 + x` ugyanaz a bájt. A könyv linker scriptjei ezért csak a `0x20000000`-tól induló 4 MiB-ot használják.

A linker scriptek `FLASH` nevű régiója itt egy **betöltött SRAM-ra adott név**, nem fizikai flash-tulajdonság: a QEMU az ELF-et ide tölti, és a tartalom futás közben akár írható is. A név a valódi mikrokontrollerek szokását követi, ahol ugyanez a szerep valódi flashre esik; a „flash nem írható” jellegű állítások a QEMU-gépre nem érvényesek.

A konkrét `0x40000000`-ás periféria-offsetek (melyik UART hol van, hol a timer) a Memory-mapped perifériák és GPIO fejezetben és a 18. fejezet laborjaiban lesznek tételesen bevezetve. Itt csak annyit szögezzünk le: **amit a linker script `MEMORY` blokkja megad, annak egyeznie**

kell a QEMU machine tényleges térképével — különben a firmware betöltődik ugyan, de a reset handler inicializáló másolása/nullázása vagy a verem az első `PUSH`-nál hibás címre hivatkozik, és fault-ot (BusFault/HardFault) okozhat; hogy melyiket, az a célrendszertől függ.

> **Megjegyzés.** A fenti 4 MiB-os ZBT SSRAM értékek a *szimulált* QEMU-memóriamodellek méretei. A fizikai MPS2+ táblán a tényleges SSRAM ennél nagyobb is lehet; natív hardveren a linker script által kijelölt régiók a tábla CMSIS-fájljából (device header) származnak, nem a QEMU-ból.

16.3.7 Kidolgozott példa: minimális vektortábla és SP

Munkakönyvtár: `~/arm-labor/cm/a063/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a063
cd ~/arm-labor/cm/a063
```

Az alábbi GNU assembler forrás egy **minimális, de teljes** vektortáblát mutat, amely pontosan azt kódolja, amiről az előbb szó volt. A feltételezett címtér: 4 MiB Code `0x00000000`-tól, 4 MiB SRAM `0x20000000`-tól.

```
/* lab-a063-vectors.s: minimális vektortábla Cortex-M3-hoz */
    .syntax unified
    .cpu cortex-m3
    .thumb

    .section .vectors, "a", %progbits
    .word   0x20010000          /* [0x00] kezdeti SP: a verem teteje az SRAM-ban */
    .word   Reset_Handler       /* [0x04] reset-vektor (a .thumb_func miatt bit0 = 1) */
    /* [0x08] NMI, [0x0C] HardFault, ... itt kihagyva a rövidség kedvéért */

    .section .text
    .thumb_func
    .global Reset_Handler
Reset_Handler:
    /* Itt még nincs C-runtime. Legalább egy végtelen ciklus kell. */
    ldr     r0, =0x20000000
1:  b       1b                  /* végtelen ciklus, önmagára ugrik */
```

A hozzá tartozó linker script-részlet a `MEMORY` régiókat a fenti térképhez köti:

```
/* lab-a063.ld: memóriatérkép MPS2-AN385-höz */
MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    SRAM  (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

SECTIONS
{
    .vectors : { KEEP(*(.vectors)) } > FLASH
    .text         : { *(.text*) }             > FLASH
}
```

A vektortábla és a reset-handler itt a II. rész kanonikus neveit használja (`.vectors`, `Reset_Handler`); a teljes, közös startup/linker párt a Linker script alapjai és a Startup kód alfejezet adja.

Regiszterállapot a `Reset_Handler` első utasítása előtt (a fő úton mérve: QEMU 8.2.2 `-S -gdb`, GDB 15.1; a lektor mérése):

Regiszter	Érték	Miért
`SP` (R13, `MSP`)	`0x20010000`	a `[0x00000000]` szó betöltve
`[0x00000004]` (reset-vektor)	`Reset_Handler` címe, bit0 = 1	a `.thumb_func` miatt páratlan
`PC` (R15)	a reset-vektor bit0 nélkül (páros)	a végrehajtás címe
`LR` (R14)	`0xFFFFFFFF`	reset szekvencia
`EPSR.T`	`1`	a reset-vektor bit0-jából
`IPSR`	`0`	Thread mód, nincs aktív kivétel
`CONTROL`	`0`	privilegizált, `MSP`

A futtatás QEMU-ban (`qemu-system-arm`) úgy indul, hogy a QEMU az ELF szegmenseit a betöltési címükre tölti, majd a reset-szekvencia a `0x00000000`-n lévő vektortáblából olvassa az `SP`-t és a `PC`-t; az ELF belépési pontja itt nem számít. A fenti regiszterértékeket a laborban is ellenőrizheted GDB-vel (`info registers`). Ezt a Firmware build és QEMU-betöltés munkafolyamat alfejezet részletesen tárgyalja.

16.3.8 Tipikus hibák és ellenpróbák

1. **„A firmware elindul, de azonnal HardFault-ba esik."** Gyakori ok: a vektortábla nincs a 0-s címen, vagy a linker script másik szekciót tett oda előbb. Ellenőrzés: `arm-none-eabi-objdump -h lab-a063.elf` — a `.vectors` szekció `VMA`-ja legyen `0x00000000`. Ha nem, a `SECTIONS` sorrendet vagy a `KEEP`-et kell módosítani.
2. **„A `PUSH` utasítás BusFault-ot okoz."** A kezdeti SP rossz helyre mutat — tipikusan az SRAM-on kívülre, például egy nem inicializált vagy rossz méretű régióba. Ellenőrzés: a `[0x00000000]` szó (`objdump -s -j .vectors`), illetve a linker script `LENGTH` értéke.
3. **„A QEMU elfogadja a binárist, de natív MPS2+-on nem indul."** A QEMU-memóriamodell és a fizikai tábla memóriatérképe eltérhet (pl. SSRAM méret). Az ilyen eltéréseket **külön kell jelölni**; a QEMU-eredmény nem helyettesíti a fizikai validálást.

Az ellenpróba minden esetben ugyanaz: gyanú → adatgyűjtés (`objdump -h`, `objdump -s`, GDB `info registers`) → egyetlen változtatás → újrafuttatás.

16.3.9 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Egy kolléga ezt a `MEMORY` definíciót adta: `FLASH (rx) : ORIGIN = 0x20000000, LENGTH = 1M`. Magyarázd el, mi a hiba, és mi fog történni a reset után.

2. (implementációs, alapszint — cél: rutinépítés) Módosítsd a fenti `lab-a063-vectors.s` vektortáblát úgy, hogy a verem a 4 MiB-os kanonikus SRAM-tartomány (`0x20000000`–`0x203FFFFF`) tetejéről induljon, 8 bájtra igazítva. Írd le, melyik szót kell megváltoztatni, miért marad érvényben a vektorok sorrendje, és miért **nem** szabad a `0x20400000`-tól kezdődő területet második veremblokkként használni.

3. (hibakeresési, középszint — cél: rutinépítés) A firmware-ed `qemu-system-arm -machine mps2-an385` alatt elindul, de azonnal `HardFault`-ba esik. Írj egy lépéslistát, amivel `gdb-multiarch` + `objdump -h` segítségével eldöntöd, hogy a vektortábla rossz helyen van-e, vagy a kezdeti SP értéke rossz.

16.3.10 Megoldások

1. megoldás — Segítség. Melyik régió kezdődik a `0x20000000`-n, és hol olvassa a mag reset után az első két szót?

1. megoldás — Teljes megoldás. A `0x20000000` az **SRAM** régió kezdete, nem a Code régió. A reset szekvencia a `0x00000000`-s címről olvassa az SP-t és a PC-t, nem a `0x20000000`-ról. Következmény: a mag a 0-s címen lévő (esetleg törölt vagy véletlen) szavakat tölti be; tipikusan `HardFault` vagy `UsageFault` az első utasításnál, mielőtt bármit is kiírna. Javítás: `ORIGIN = 0x00000000`.

2. megoldás — Segítség. A verem „full descending”: a tartomány utáni első cím a kiindulópont; számold ki a `0x20000000 + 4 MiB` értéket, és ellenőrizd a 8 bájtos igazítást.

2. megoldás — Teljes megoldás. Csak az első `.word` sor értékét kell módosítani: `0x20000000 + 4 MiB = 0x20400000`. Ez a tartomány utáni első cím (kizárólagos felső határ), és ez a helyes kezdeti SP. A Cortex-M verme „full descending”: a `PUSH` előbb csökkenti az SP-t, és csak utána ír, ezért az első mentett szó a `0x203FFFFC`-re kerül, ami még a tartományon belül van. A `0x20400000` 8-cal (sőt 16-tal) osztható, így teljesül az AAPCS 8 bájtos veremigazítási szabálya a nyilvános interfészeken. A vektorok sorrendje változatlan marad, csak az SP szó értéke változik. A `0x20400000`–`0x207FFFFF` terület nem második blokk, hanem az alsó 4 MiB tükre: egy oda tett verem a `.data`/`.bss` bájtjait írná felül egy másik címen keresztül.

3. megoldás — Segítség. Három helyen lehet a hiba: a vektortábla VMA-ja, a benne lévő szavak értéke, illetve a QEMU betöltési módja. Melyik ellenőrzés melyiket zárja ki?

3. megoldás — Teljes megoldás. Lépések: (a) `arm-none-eabi-objdump -h lab-a063.elf` — nézd meg a `.vectors` VMA-ját; ha nem `0x00000000`, a linker script a hibás. (b) `objdump -s -j .vectors` — az első szó legyen érvényes SRAM-cím (`0x20xxxxxx`), a második a `Reset_Handler` címe, páratlanul. (c) `gdb-multiarch` + `target remote :3333` (a QEMU-t `-S -gdb tcp::3333`-mal indítva) — a reset utáni, első utasítás előtti állapot a fenti GDB-lépésekkel: ha `SP` és `PC` a vektortáblából várt értéktől eltér, akkor vagy a betöltés rossz (QEMU `-kernel` vs. `-device loader`), vagy a szekció rossz helyen van. Ha `PC` jó, de máshol faultol, akkor az SRAM mérete/igazítása a gyanús.

16.3.11 Rövid összefoglaló

A Cortex-M3 fix, architektúra-szintű memóriatérképet használ: `0x00000000`-tól a Code régió, `0x20000000`-tól az SRAM, `0x40000000`-tól a periféria, `0xE0000000`-tól a mag belső perifériái (PPB). A régiókat a cím felső 3 bitje választja el, egyenként 512 MiB-os tartományban. Reset után a mag az első szót az MSP-be tölti, a második szó bit0-ját az `EPSR.T`-be, a bit0 nélküli értékét a PC-be, LR-t `0xFFFFFFFF`-re állítja, és privilegizált Thread módban, `IPSR = 0` mellett indul; a `main()` csak ezután, a reset-handler által hívva jelenik meg. QEMU `mps2-an385` alatt a 4 MiB Code (`FLASH` nevű, de betöltött SSRAM) / 4 MiB kanonikus SRAM (`0x20400000`-tól tükör) / `0x40000000`-ás periféria-baseline az irányadó, de a konkrét periféria-offseteket és a QEMU-machine verzióját a generálás idején ellenőrizni szükséges.

Továbbhasznált definíciók:

- **memóriatérkép (memory map):** a 32 bites címtér hivatalos felosztása funkciók (Code/SRAM/Peripheral/PPB) szerint.
- **reset-vektor:** a `0x00000004` címen lévő szó, amely a reset-handler belépési címét tartalmazza.
- **bare-metal futási modell:** nincs operációs rendszer; a linker script és a startup kód rendezi el a címtér használatát.

16.3.12 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Armv7-M Architecture Reference Manual, DDI0403E — a fix memóriatérkép és a reset szekvencia elsődleges forrása.
- Arm M-profile architecture documentation — a régiók fogalmi áttekintése.
- QEMU System Emulation dokumentáció — az `mps2-an385` machine-definíció; a konkrét periféria-címek forrása.
- GNU Binutils `as` dokumentáció — a `.section`, `.word`, `.thumb_func` direktívák szemantikája.
- QEMU forrás, `hw/arm/mps2.c` (8.2.2) — az AN385 memóriaelrendezése, köztük az SSRAM 2&3 és tükre.

Ellenőrzési hiányok (jelölve):

- Az `mps2-an385` konkrét periféria-offsetjei (melyik UART hol, `FPGAIO`/`SCC`/GPIO pontos címe) **a laborban ellenőrizendők** a QEMU `hw/arm/mps2.c` forrása és a hivatalos MPS2+ dokumentáció alapján. A QEMU-machine definíció verzióról verzióra változhat.
- Az SRAM-tükör (`0x20400000`–`0x207FFFFF` = a `0x20000000`–`0x203FFFFF` aliasa) a QEMU 8.2.2 `hw/arm/mps2.c` forrásában ellenőrzött állapot; más QEMU-verzióban a monitor `info mtree` parancsával erősítendő meg.
- A reset szekvencia lépései (MSP, PC és T-bit a reset-vektorból, LR, privilegizált Thread mód, `IPSR = 0`) az Armv7-M ARM-ben rögzítettek (a Cortex-M3 útmutató is Thread módot ír). A közölt GDB-lépéseket itt nem futtattuk QEMU alatt; a `$control` regiszter elérhetősége QEMU- és GDB-verziófüggő. A vektortábla-tulajdonságokat (bit0 = 1, páros végrehajtási cím) a helyettesítő eszközlánccal épített ELF-en ellenőriztük.

16.4 Reset és boot-folyamat: vektortábla

16.4.1 Motiváció: miért nem indul el a firmware?

Egy új Cortex-M3 alapú panel bring-upja során a fejlesztő lefordítja a `main()`-t, letölti a lapra, és semmi nem történik: nincs soros kimenet, a debugger nem áll meg a `main` első során, néha a mag HardFault-ba kerül. A leggyakoribb gyökér nem a C-kódban, hanem a címtér legelső nyolc bájtjában van. A Cortex-M mag reset után **nem** egy fix, architektúrába égetett címről kezd utasítást végrehajtani, hanem a `0x00000000`-tól induló **vektortáblából** olvassa ki a kezdeti veremmutatót és a reset-handler címét. Ha ez a nyolc bájt hibás, a processzor vagy rossz veremmel indul, vagy a rossz címre ugrik, és minden további kód irreleváns.

Ez az alfejezet a boot-folyamat első, hardveresen definiált szerződését építi fel: hogyan néz ki a minimális vektortábla, hogyan kerül a flash elejére linker scripttel, mit tesz a reset-handler a

`.data`/`.bss` kezelése **előtt**, és hogyan ellenőrizhető mindez `objdump`-pal. A `.data` másolása és a `.bss` törlése a startup kód témája; itt csak a belépési pontig jutunk el.

16.4.2 Definíció: vektortábla és reset-szekvencia

Vektortábla (vector table): a címtér elején elhelyezkedő, 4 bájtos szavak rendezett listája, amely a processzor kivétel- és megszakításkezelőinek belépési címeit tartalmazza. Armv7-M esetén az első szó nem ugrási cím, hanem a **kezdeti veremmutató (initial stack pointer)** értéke. A tábla első 16 bejegyzése fix sorrendű rendszerkivétel, ezután következnek az NVIC által kezelt külső megszakítások (IRQ0, IRQ1, …).

A reset-szekvencia hardveresen, utasítás végrehajtása nélkül történik:

Lépés	Művelet	Forrás
1	`SP ← mem[0x00000000]`	vektortábla 0. szava
2	`PC ← mem[0x00000004]` (bit0 = Thumb jelző)	vektortábla 1. szava
3	`LR ← 0xFFFFFFFF`	visszatérési cím hiányát jelzi

A `0xFFFFFFFF` azért fontos, mert a reset-handlerből **nincs értelmes visszatérés**: nincs hívó, aki visszavárna. Ha a handler `BX LR`-rel próbál visszatérni, a mag `0xFFFFFFFE`-re ugrik, és HardFault lesz belőle. Ezért a reset-handler vagy egy startup rutinra adja tovább a vezérlést, vagy végtelen ciklusba kerül.

A 16 rendszerkivétel-bejegyzés sorrendje Armv7-M-en kötelező:

Offset	Bejegyzés	Megjegyzés
0x00	Initial SP	nem kód, hanem adat
0x04	Reset	ide ugrik a mag reset után
0x08	NMI	nem maszkolható megszakítás
0x0C	HardFault	minden nem kezelt fault ide fut
0x10	MemManage	MPU-sértés
0x14	BusFault	buszhiba
0x18	UsageFault	érvénytelen utasítás / állapot
0x1C–0x28	Reserved	négy szó, 0-val kitöltve
0x2C	SVCall	supervisor call
0x30	DebugMonitor	debug kivétel
0x34	Reserved	egy szó
0x38	PendSV	függőbe tehető szolgáltatáskérés (kontextusváltáshoz)
0x3C	SysTick	rendszer-tick

Két különböző eset van, és külön kezeljük őket. A **fenntartott (reserved)** bejegyzések (a 7–10. és a 13. index) nem kivételek: a tábla elcsúszásának elkerülésére helyet foglalnak, tartalmuk `0`, és az ellenőrző ezt várja. A **megvalósított, de nem használt** kivételek és IRQ-k (NMI, fault-ok, SVCall, DebugMonitor, PendSV, SysTick, IRQn) viszont létező kivételek: ha egy ilyen bejegyzés `0`, a kivétel bekövetkeztekor a mag a `0x00000000`-ra próbál ugrani, és másodlagos HardFaultba

fut. A biztonságos minta: minden megvalósított bejegyzés vagy valódi handlerre mutat, vagy egy közös `Default_Handler` végtelen ciklusára. Az ellenőrzőnek külön tesztesete van a fenntartott slotra (nulla kell) és a nem használt IRQ-ra (`Default_Handler` cím kell). Az alábbi példában a MemManage/BusFault/UsageFault/DebugMonitor bejegyzés mégis `0`: ezek resetkor tiltottak, és ha mégis bekövetkeznek, HardFaultra eszkalálódnak; a rövid példa ezt a minimális esetet mutatja. Éles firmware-ben ezekre is `Default_Handler` ajánlott.

16.4.3 Hogyan kerül a tábla a flash elejére?

A linker script feladata, hogy a `.vectors` szekciót a `FLASH` régió `ORIGIN`-jára, azaz `0x00000000`-ra helyezze, és **semmilyen optimalizálás ne dobja el**. A `KEEP(*(.vectors))` direktíva azért kötelező, mert a `--gc-sections` linkelés a nem hivatkozott szekciókat kigyomlálná — a vektortáblára assembly szinten nem hivatkozik `bl` utasítás, csak a hardver olvassa.

```
/* ~/arm-labor/cm/a064/link.ld - minimális linker script a vektortáblához
 * Target: qemu-system-arm -machine mps2-an385 (Cortex-M3)
 * FIGYELEM: a FLASH/RAM ORIGIN és LENGTH a QEMU machine verziójától
 * és a valódi MCU-tól függ; ellenőrizni szükséges.
 */
ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

_estack = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS
{
    .vectors : {
        KEEP(*(.vectors))
    } > FLASH

    .text : {
        *(.text*)
        *(.rodata*)
    } > FLASH
}
```

Az `ENTRY(Reset_Handler)` csak az ELF fejléc `e_entry` mezőjét állítja be — debugger és bootloader szempontjából hasznos, de **nem** helyettesíti a vektortáblát. A hardver resetkor kizárólag a `0x00000004`-en lévő szót olvassa.

16.4.4 A reset-handler minimális formája

A reset-handler assembly szintű belépési pont. A mag már érvényes SP-vel érkezik, ezért a handler első dolga nem a verem beállítása, hanem a továbbadás a startup kódnak. A `.thumb_func` direktíva biztosítja, hogy a szimbólumra hivatkozva a linker a cím bit0-ját 1-re állítsa — enélkül a mag páros címet lát, és `INVSTATE` HardFaultot vált.

```
/* ~/arm-labor/cm/a064/startup.s - minimális vektortábla és reset-handler
 * ISA: Thumb-2 / Armv7-M
 * Target: qemu-system-arm -machine mps2-an385
 */
    .syntax unified
```

```
    .cpu cortex-m3
    .thumb

    .section .vectors, "a", %progbits
    .align 4
    .global __vectors
__vectors:
    .word _estack                /* 0x00: kezdeti SP       */
    .word Reset_Handler          /* 0x04: Reset            */
    .word NMI_Handler            /* 0x08: NMI              */
    .word HardFault_Handler      /* 0x0C: HardFault        */
    .word 0                      /* 0x10: MemManage        */
    .word 0                      /* 0x14: BusFault         */
    .word 0                      /* 0x18: UsageFault       */
    .word 0                      /* 0x1C: Reserved         */
    .word 0                      /* 0x20: Reserved         */
    .word 0                      /* 0x24: Reserved         */
    .word 0                      /* 0x28: Reserved         */
    .word SVC_Handler            /* 0x2C: SVCall           */
    .word 0                      /* 0x30: DebugMonitor     */
    .word 0                      /* 0x34: Reserved         */
    .word PendSV_Handler         /* 0x38: PendSV           */
    .word SysTick_Handler        /* 0x3C: SysTick          */

    .text
    .thumb_func
    .global Reset_Handler
    .type Reset_Handler, %function
Reset_Handler:
    /* 1) A mag már betöltötte SP-t a vektortábla 0. szavából.   */
    /* 2) Itt még NEM történt meg a .data másolás / .bss törlés. */
    /* 3) Tovább a startup kódra, amely ezt elvégzi.             */
    b    _startup

    .thumb_func
    .global NMI_Handler
    .type NMI_Handler, %function
NMI_Handler:
    b    .


    .thumb_func
    .global HardFault_Handler
    .type HardFault_Handler, %function
HardFault_Handler:
    b    .

    .thumb_func
    .global SVC_Handler
    .type SVC_Handler, %function
SVC_Handler:
    b    .

    .thumb_func
    .global PendSV_Handler
    .type PendSV_Handler, %function
PendSV_Handler:
    b    .

    .thumb_func
    .global SysTick_Handler
    .type SysTick_Handler, %function
SysTick_Handler:
```

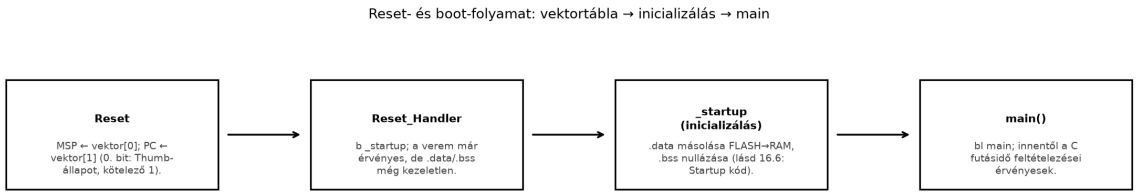
```
    b     .

    .thumb_func
    .global _startup
    .type _startup, %function
_startup:
    b     .     /* helykitöltő; a .data/.bss kezelés a következő alfejezet */
```

Az `_startup` itt még egy helykitöltő végtelen ciklus. A startup kód alfejezetben ez a rutin kapja meg a `.data` másolását és a `.bss` törlését, majd onnan `bl main` következik.

Az, hogy a `Reset_Handler` nem maga végzi az inicializálást, hanem azonnal továbbadja a vezérlést az `_startup`-nak, a jelen példaprogram tudatos megoldása: a vektortábla által hivatkozott belépési pont így triviális és stabil marad, miközben a tényleges `.data`/`.bss` logika külön, önállóan tesztelhető és bővíthető rutinba kerül.

Az alábbi ábra a teljes boot-folyamatot foglalja össze, a vektortáblától a `main()`-ig:



16.1. ábra: Reset- és boot-folyamat: vektortábla → inicializálás → main - A hardveres reset-szekvencia és a Reset_Handler/__startup láncolat összefoglalása.

16.4.5 Kidolgozott példa: build és objektumellenőrzés

Munkakönyvtár: `~/arm-labor/cm/a064/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a064
cd ~/arm-labor/cm/a064
```

A példa célja, hogy a vektortábla **tényleges bájttartalmát** ellenőrizzük, mielőtt a QEMU-ra bíznánk a futtatást.

```
# ~/arm-labor/cm/a064 munkakönyvtárban
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb \
    -nostdlib -nostartfiles \
    -T link.ld -Wl,-Map=lab-a064.map \
    -o lab-a064.elf startup.s

# szekciók VMA szerinti ellenőrzése
arm-none-eabi-objdump -h lab-a064.elf

# a vektortábla nyers tartalma
arm-none-eabi-objdump -s -j .vectors lab-a064.elf

# a reset-handler címe és Thumb bitje
arm-none-eabi-nm -n lab-a064.elf | grep -E 'Reset_Handler|_estack'
```

Kimenet a fő úton mérve (GNU binutils 2.42, `arm-none-eabi-as`, `arm-none-eabi-ld`, `arm-none-eabi-objdump`; a lektor mérése); a `.vectors`-dump betűre egyezik a közölttel:

```
Contents of section .vectors:
 0000 00004020 41000000 45000000 47000000  ..@ A...E...G...
 0010 00000000 00000000 00000000 00000000  ................
 0020 00000000 00000000 00000000 49000000  ............I...
 0030 00000000 00000000 4b000000 4d000000  ........K...M...
```

Az `objdump -s` bájtokat ír ki memóriasorrendben, négyesével csoportosítva, ezért minden szót little-endian módon kell visszaolvasni. A `0000` sor első szava `00 00 40 20` → `0x20400000`: ez az `_estack`, a 4 MiB-os SRAM-tartomány kizárólagos felső határa. A második szó `41 00 00 00` → `0x00000041`: a `Reset_Handler` címe a Thumb bittel együtt. Maga a kód a `0x00000040`-en kezdődik, közvetlenül a 64 bájtos (16 szavas) vektortábla után, a `.text` szekció elején. A további handlerek (`0x45`, `0x47`, `0x49`, `0x4b`, `0x4d`) ugyanígy páratlanok, és mind a `.text`-be mutatnak. A GNU `nm` a Thumb-bitet levágja, ezért `00000040 T Reset_Handler`-t ír; a páratlan értéket a `readelf -s` mutatja (`st_value = 0x41`, `FUNC GLOBAL`). Az ELF az ARM-konvenció szerint a Thumb-bitet a függvényszimbólum értékében tárolja.

A **konkrét címeket nem kell kézzel megjósolni**: egy másik assembler vagy kapcsoló más utasításhosszt választhat, és ettől a későbbi handlerek címe eltolódik. Ami fontos, az a **tulajdonság**. Ezt nem szemmel, hanem programmal ellenőrizzük:

```
#!/usr/bin/env python3
"""lab-check-vectors.py - a vektortábla tulajdonságainak ellenőrzése egy ELF-en.

Használat: python3 lab-check-vectors.py lab-a064.elf [--relocated] [INDEX=SZIMBÓLUM ...]
Ellenőrzi: (1) van .vectors szekció, mérete szóhatáron van, és legalább a kezdeti
SP-t és a Reset-bejegyzést tartalmazza; (2) a .vectors a 0x00000000 címen kezdődik
(--relocated mellett ez nem kötelező); (3) a tábla bázisa VTOR-kompatibilis:
a 128 bájt és a táblaméretnél nem kisebb legkisebb kettőhatvány közül a nagyobbik
többszöröse; (4) a 0. szó (kezdeti SP) a RAM-régióba esik, vagy annak felső
határa, és 8 bájtra igazított; (5) az 1. szó (Reset) KÖTELEZŐ: nem nulla, páratlan
(Thumb-bit = 1), és a bit0 nélküli címe egy végrehajtható szekcióba esik;
(6) a fenntartott bejegyzések (a 7-10. és a 13. index) nullák; (7) minden további nem
nulla kezelőbejegyzés páratlan, és kódszekcióba mutat (a nulla itt megengedett:
nincs kezelő); (8) minden INDEX=SZIMBÓLUM argumentumra a tábla INDEX-edik szava
létezik, és pontosan a SZIMBÓLUM értéke (pl. 1=Reset_Handler, 24=TIMER0_IRQHandler;
IRQn indexe 16 + n). Függőség: python3-pyelftools.
"""
import struct
import sys
from elftools.elf.elffile import ELFFile

RAM_LO, RAM_HI = 0x20000000, 0x20400000     # a kanonikus 4 MiB-os SRAM
RESERVED = (7, 8, 9, 10, 13)                # Armv7-M: fenntartott kivételszámok

args = [a for a in sys.argv[2:] if a != '--relocated']
relocated = '--relocated' in sys.argv[2:]
fails = 0
def check(ok, msg):
    global fails
    print(('OK   ' if ok else 'HIBA ') + msg)
    fails += not ok

with open(sys.argv[1], 'rb') as f:
    elf = ELFFile(f)
    vec = elf.get_section_by_name('.vectors')
    code = [(s['sh_addr'], s['sh_addr'] + s['sh_size'], s.name)
            for s in elf.iter_sections() if s['sh_flags'] & 0x4]      # SHF_EXECINSTR
    if vec is None:
        check(False, 'nincs .vectors szekció')
        sys.exit(1)
    size, base = vec['sh_size'], vec['sh_addr']
```

```
    if size % 4 or size < 8:
        check(False, f'.vectors mérete {size} bájt: nem szóhatáron végződik, '
                     'vagy kevesebb két szónál (SP + Reset)')
        sys.exit(1)
    check(True, f'.vectors mérete {size} bájt ({size // 4} szó)')
    check(relocated or base == 0, '.vectors a 0x00000000 címen' if not relocated
          else f'.vectors áthelyezve, bázis 0x{base:08x}')
    align = 128
    while align < size:
        align *= 2
    # VTOR: a bázis max(128, a táblaméretnél nem kisebb kettőhatvány) többszöröse
    check(base % align == 0, f'táblabázis 0x{base:08x}: {align} bájtra igazított (VTOR)')
    words = struct.unpack(f'<{size // 4}I', vec.data())
    sp = words[0]
    check(RAM_LO < sp <= RAM_HI and sp % 8 == 0,
          f'[0x00] kezdeti SP = 0x{sp:08x} (RAM, 8 bájtra igazított)')
    def in_code(w):
        target = w & ~1
        return target, next((n for lo, hi, n in code if lo <= target < hi), None)
    reset = words[1]
    target, sec = in_code(reset)
    check(reset != 0 and reset & 1 == 1 and sec is not None,
          f'[0x04] Reset (kötelező) 0x{reset:08x}: Thumb-bit={reset & 1}, cél 0x{target:08x} '
          + (f'a(z) {sec} szekcióban' if sec else 'NEM végrehajtható szekcióban'))
    nonzero_reserved = [i for i in RESERVED if i < len(words) and words[i] != 0]
    check(not nonzero_reserved, 'fenntartott bejegyzések (7-10., 13. index): '
          + ('mind 0' if not nonzero_reserved
             else 'NEM nulla: ' + ', '.join(map(str, nonzero_reserved))))
    for i, w in enumerate(words[2:], start=2):
        if w == 0 or i in RESERVED:
            continue
        target, sec = in_code(w)
        check(w & 1 == 1 and sec is not None,
              f'[0x{4 * i:02x}] 0x{w:08x}: Thumb-bit={w & 1}, cél 0x{target:08x} '
              + (f'a(z) {sec} szekcióban' if sec else 'NEM végrehajtható szekcióban'))
    symtab = elf.get_section_by_name('.symtab')
    for arg in args:
        idx, name = arg.split('=', 1)
        idx = int(idx, 0)
        syms = symtab.get_symbol_by_name(name) if symtab else None
        want = syms[0]['st_value'] if syms else None
        if idx >= len(words):
            check(False, f'[{idx}] = {name}: a tábla csak {len(words)} szó, '
                         f'a {idx}. slot hiányzik')
            continue
        got = words[idx]
        check(want is not None and got == want,
              f'[{idx}] = {name}: tábla 0x{got:08x}, szimbólum '
              + (f'0x{want:08x}' if want is not None else 'NINCS'))
    sys.exit(1 if fails else 0)

sudo apt install python3-pyelftools        # egyszer
python3 lab-check-vectors.py lab-a064.elf; echo "kilépési állapot: $?"
```

Negatív próbák. Az ellenőrzőnek a hibás táblákat rendezett `HIBA` sorral és nem nulla kilépési állapottal kell elutasítania. A helyettesítő eszközlánccal (Clang + `ld.lld`) mind kipróbáltuk:

Hibás vagy határeset	Eredmény	Kilépési állapot
a Reset helyén `.word 0`	`HIBA [0x04] Reset (kötelező) 0x00000000 …`	1

Hibás vagy határeset	Eredmény	Kilépési állapot
a kanonikus 16 szavas tábla, de a labor `24=TIMER0_IRQHandler`-t vár	`HIBA [24] = TIMER0_IRQHandler: a tábla csak 16 szó, a 24. slot hiányzik`	1
csonka, 6 bájtos `.vectors`	`HIBA .vectors mérete 6 bájt: …`	1
áthelyezett tábla `0x20000104`-en (`--relocated`): szóra igazított, de 128-ra nem	`HIBA táblabázis 0x20000104: 128 bájtra igazított …`	1
áthelyezett tábla `0x20000200`-on (`--relocated`)	minden sor `OK`	0

A szóigazítás és a táblabázis igazítása két külön követelmény. Minden bejegyzés 32 bites szó, ezért a táblán belül a szavak 4 bájtos határon állnak; ezt adja a `.align 2`. A tábla **bázisára** ennél erősebb szabály vonatkozik, ha a táblát a `VTOR` (Vector Table Offset Register, `0xE000ED08`) segítségével áthelyezzük. Armv7-M-en a bázisnak a tábla méreténél nem kisebb kettőhatványra kell igazítva lennie, de legalább 128 bájtra. A táblaméretet a magban ténylegesen implementált kivételek száma adja: 16 rendszerslot és az implementált IRQ-k; ez a linkelt táblánál nagyobb is lehet. Az ellenőrző a linkelt tábla méretéből számol, ez tehát alsó becslés, a konkrét mag dokumentációjával kell egyeztetni. A könyv kanonikus elhelyezése, a `0x00000000`, minden ilyen követelményt teljesít. Áthelyezett táblánál a linker scriptben egy `ASSERT(ADDR(.vectors) % 128 == 0, …)` jellegű feltétel (a ténylegesen szükséges határral) rögzíti ugyanezt.

A későbbi laborok a bővített vektortáblát ugyanezzel az eszközzel **név szerint** is ellenőrzik: az `INDEX=SZIMBÓLUM` argumentum azt igazolja, hogy a tábla adott szava pontosan a megnevezett kezelő címe (az `IRQn` indexe $16 + n$). Például: `python3 lab-check-vectors.py lab-a070.elf 1=Reset_Handler 24=TIMER0_IRQHandler`. Argumentum nélkül a kimenet a fenti.

A fenti ELF-re a kimenet (a fő úton, GNU binutils 2.42-vel mérve; a lektor mérése):

```
OK   .vectors mérete 64 bájt (16 szó)
OK   .vectors a 0x00000000 címen
OK   táblabázis 0x00000000: 128 bájtra igazított (VTOR)
OK   [0x00] kezdeti SP = 0x20400000 (RAM, 8 bájtra igazított)
OK   [0x04] Reset (kötelező) 0x00000041: Thumb-bit=1, cél 0x00000040 a(z) .text szekcióban
OK   fenntartott bejegyzések (7-10., 13. index): mind 0
OK   [0x08] 0x00000045: Thumb-bit=1, cél 0x00000044 a(z) .text szekcióban
OK   [0x0c] 0x00000047: Thumb-bit=1, cél 0x00000046 a(z) .text szekcióban
OK   [0x2c] 0x00000049: Thumb-bit=1, cél 0x00000048 a(z) .text szekcióban
OK   [0x38] 0x0000004b: Thumb-bit=1, cél 0x0000004a a(z) .text szekcióban
OK   [0x3c] 0x0000004d: Thumb-bit=1, cél 0x0000004c a(z) .text szekcióban
kilépési állapot: 0
```

Az ellenőrzési feltétel tehát: **a `.vectors` létezik, mérete szóhatáron van, és legalább két szó; a bázisa `0x00000000` (áthelyezett táblánál VTOR-kompatibilis); az első szó érvényes, 8 bájtra igazított SRAM-határ; a Reset-bejegyzés kötelezően nem nulla, páratlan, és kódra mutat; a fenntartott bejegyzések nullák; és minden további nem nulla kezelőbejegyzés páratlan, bit0 nélkül pedig a kódszekcióba esik.** A nulla tehát kétféle bejegyzésben megengedett: a fenntartott slotokban, illetve a nem kötelező, kezelő nélküli kivételeknél. A Reset helyén viszont hiba, mert a firmware így nem indul. A labor által ténylegesen használt IRQ-slotokat az `INDEX=SZIMBÓLUM` argumentum kéri számon; ha a tábla

rövidebb, a hiányzó slot is hiba. Ha egy bejegyzés páros, a mag `INVSTATE` UsageFaultot vált ki, amely HardFaulttá eszkalálódik. Ha a bit0 nélküli cím a vektortábla belsejébe vagy adatterületre mutat, a mag adatot próbálna utasításként végrehajtani.

Ahhoz, hogy a vektortábla tartalma ne csak a linker térképén, hanem a tényleges betöltött képen is stimmeljen, QEMU-ban is ellenőrizhetjük (a fő úton, QEMU 8.2.2-vel mérve; a lektor mérése):

```
# A program végtelen ciklusban áll; timeout-tal futtatjuk.
timeout 2 qemu-system-arm -machine mps2-an385 \
    -kernel lab-a064.elf -display none -serial null -monitor none
# Mért: nincs kimenet, a QEMU a timeout után 124-es kóddal kilép (a lektor mérése).
# FIGYELEM: a néma futás és a timeout önmagában NEM bizonyítja, hogy a Reset_Handler fut:
# egy rossz vektortábla vagy lockup is csendes. Ezért célzott PC-ellenőrzés kell:
#   qemu-system-arm -machine mps2-an385 -kernel lab-a064.elf -S -gdb tcp::3333 \
#       -display none -monitor none -serial null &
#   timeout 10 gdb-multiarch -batch -ex 'target remote :3333' -ex 'p/x $pc' \
#       -ex 'break _startup' -ex 'continue' -ex 'p/x $pc' lab-a064.elf
# Várt: a target remote után $pc = 0x40 (vektor[1] & ~1), majd a _startup elején áll meg.
# Negatív kontroll: ha a vektortábla 2. szava páros (vagy rossz), a $pc nem 0x40, és a
# _startup töréspont nem teljesül; szándékos HardFault esetén a HardFault_Handler
# töréspontja (felismerhető diagnosztika) teljesül.
# Ha a vektortábla rossz, rövid időn belül HardFaultba és/vagy
# hibás PC-értékbe futhat; ezt a QEMU verziója szerint ellenőrizni kell.
```

16.4.6 Hogyan tölti be a QEMU és a valódi mikrokontroller?

QEMU (`qemu-system-arm -machine mps2-an385`): az ELF szekcióit a VMA-jukra tölti, majd a reset-szekvenciát szimulálja: a `0x00000000`-ról olvassa SP-t és PC-t. Az `mps2-an385` géptípus Code régiója a `0x00000000`-tól indul; a pontos régióméretek és a QEMU machine-definíció verziófüggők.

Valódi MCU (pl. STM32 Cortex-M3): a boot-konfiguráció (BOOT0/BOOT1 pinek) dönti el, hogy a `0x00000000`-ra a belső flash vagy a rendszer-memória aliasingje látszik-e. A mag ugyanúgy az első két szót olvassa; a flash tartalmát a gyártói bootloader vagy a letöltő eszköz írja oda. A viselkedés eltérhet a QEMU-tól a hibás vektortábla kezelésében (pl. valódi hardveren a lockup állapot részletei), ezt fizikai hoston külön kell validálni.

Közös mindkét esetben: a vektortábla **nem opcionális**. A reset-handler első utasítása csak akkor fut le, ha a `0x00000004`-en lévő szó érvényes, páratlan címet tartalmaz.

16.4.7 Tipikus hibák és ellenpróbák

1. tünet: a QEMU azonnal leáll vagy HardFaultot jelez; a debugger nem áll meg `Reset_Handler`-ben. **Hipotézis:** a `.vectors` szekció nem a `0x00000000`-ra került, vagy a `KEEP` hiányában kiesett a linkelésből. **Adatgyűjtés:** `arm-none-eabi-objdump -h lab-a064.elf` — nézzük a `.vectors` VMA-ját; `objdump -s -j .vectors` — üres-e. **Egy változtatás:** `KEEP(*(.vectors))` hozzáadása, `MEMORY ORIGIN` ellenőrzése. **Ellenőrzés:** újra `objdump -h`, a `.vectors` VMA `0x00000000`.

2. tünet: a mag `INVSTATE` HardFaultot vált, bár a vektortábla cím szerint jó helyen van. **Hipotézis:** a `Reset_Handler` címe páros a táblában; hiányzik a `.thumb_func` vagy a `.type ..., %function`. **Adatgyűjtés:** `objdump -s -j .vectors`, a második szó bit0-jának ellenőrzése. **Egy változtatás:** `.thumb_func` a handler elé. **Ellenőrzés:** a második szó páratlan.

3. tünet: a program elindul, de az első `push` után szemét értékek, vagy a kód felülíródik. **Hipotézis:** az `_estack` nem az SRAM teteje (pl. `0` vagy flash-cím), így a verem a kódra nő. **Adatgyűjtés:** `objdump -s -j .vectors` első szava; `nm _estack`; a linker script `RAM LENGTH` értéke. **Egy változtatás:** `_estack = ORIGIN(RAM) + LENGTH(RAM);` **Ellenőrzés:** az első szó az SRAM felső határára mutat.

Ellenpróba: szándékosan állítsuk a vektortábla első szavát `0`-ra, és figyeljük meg QEMU-ban a viselkedést. Ez kontrollált, jelölt demonstráció; a tanulság az, hogy a hardver nem figyelmeztet, egyszerűen rossz veremmel indul.

16.4.8 Feladatok

A feladatcímkék és a kétlépcsős megoldások használatát az Olvasói útmutató írja le.

16.4.9 1. feladat (magyarázó, alapszint — cél: megértés-ellenőrzés)

Magyarázd el, miért a **kezdeti veremmutató** áll a vektortábla első szavában, és miért nem egy ugrási cím. Hasonlítsd össze az AArch64 reset-viselkedésével, ahol nincs ilyen módon a címtér elején elhelyezett SP-bejegyzés. Milyen hardveres egyszerűsítést jelent a Cortex-M megoldása, és mi ennek az ára?

16.4.10 2. feladat (implementációs, középszint — cél: rutinépítés)

Egészítsd ki a vektortáblát úgy, hogy a `HardFault_Handler` a verem tetején lévő **stacked frame**-ből kiolvassa a mentett PC-t (a folytatási címet) egy regiszterbe, majd végtelen ciklusba kerüljön. A stacked frame a kivétel belépésekor a következő sorrendben kerül a veremre: `R0, R1, R2, R3, R12, LR, PC, xPSR`. Ellenőrizd `objdump -s -j .vectors`-sel, hogy a `HardFault_Handler` bejegyzés a `0x0C` offseten van, és páratlan címet tartalmaz.

16.4.11 3. feladat (hibakeresési/mérési, elmélyült — cél: rutinépítés)

Két hibás linker script-változatot kapsz. (a) A `.vectors` szekcióból hiányzik a `KEEP`. (b) A `.vectors` szekció a `.text` **után** van a scriptben, és a `FLASH ORIGIN` változatlan. Mindkét esetben írd le a várható tünetet QEMU-ban, add meg az `objdump`-parancsot, amellyel a hiba igazolható, és a javítást. Mérési rész: becsüld meg, hány órajel telik el a reset-vektor beolvasása és a `Reset_Handler` első utasítása között, ha a flash-hozzáférés 1 várakozási ciklust igényel. Jelöld, hogy ez **levezetett** érték, nem QEMU-ban mért ciklusszám.

16.4.12 Megoldások

A feladatcímkék és a kétlépcsős megoldások használatát az Olvasói útmutató írja le.

16.4.13 1. megoldás — Segítség

Gondold át, mi történne, ha a mag a reset-handlerbe úgy lépne be, hogy még nincs érvényes SP-je — milyen hardveres lépés előzi meg logikailag az első `push`-t.

16.4.14 1. megoldás — Teljes megoldás

A Cortex-M hardveresen, utasítás nélkül tölti be SP-t az első szóból. Ezzel a reset-handlernek nem kell külön verembeállítást végeznie, és a C-kód előtti első `push` már érvényes veremre kerül. Az AArch64-en a reset az implementáció által rögzített (RVBAR) címen kezd, és a veremkezelés szoftveres feladat. A Cortex-M megoldás előnye a hardveres egyszerűség és a gyors indulás; ára, hogy a vektortáblát kötelezően a címtér elejére kell helyezni, és a tábla első szava elveszik a kód elől. Ha a boot-konfiguráció miatt a `0x00000000`-ra nem a flash látszik, a tábla rossz helyről olvasódik — ez a bring-up egyik klasszikus csapdája.

16.4.15 2. megoldás — Segítség

A stacked frame nyolc szava fix sorrendben kerül a veremre; számold ki, hányadik szó (és milyen bájt-offszet) felel meg a PC-nek, és melyik veremmutatóból (MSP vagy PSP) kell olvasni.

16.4.16 2. megoldás — Teljes megoldás

```
    .thumb_func
    .global HardFault_Handler
    .type HardFault_Handler, %function
HardFault_Handler:
    mov  r0, lr            /* EXC_RETURN: melyik verem volt használatban */
    tst  r0, #4            /* bit2 = 0: MSP, bit2 = 1: PSP */
    ite  eq
    mrseq r0, msp
    mrsne r0, psp
    ldr  r1, [r0, #24]     /* stacked PC a frame 24. bájtján */
    b    .                 /* r1-ben a mentett PC (folytatási cím) */
```

Az első `mov`/`tst` azt dönti el, hogy a kivétel a fő veremből (MSP) vagy a folyamat veremből (PSP) érkezett-e. A `ldr r1, [r0, #24]` a stacked frame 7. szavát, a PC-t olvassa. Ez a folytatási cím: precíz szinkron hibánál (például UsageFault) jellemzően a hibát okozó utasítás címe, de pontatlan BusFault esetén egy később végrehajtott utasításra mutathat, ezért a kiértékelésnél a fault-státuszregisztereket is meg kell nézni (kivételmodell). A `b .` szándékos: a handler nem tér vissza, mert a fault oka nem megszűnt. Az `objdump -s -j .vectors` kimenetén a `0x0C` offseten a `HardFault_Handler` címe látható, páratlan bit0-val.

16.4.17 3. megoldás — Segítség

Mindkét hibánál abból indulj ki, hogy a linker mit ír ténylegesen a `.vectors` VMA-jára — a `--gc-sections` és a szekciósorrend is a “mi kerül a 0x00000000-ra” kérdést dönti el, csak más-más okból.

16.4.18 3. megoldás — Teljes megoldás

(a) Hiányzó `KEEP`: a `--gc-sections` linkelés a `.vectors` szekciót eltávolíthatja, mert assembly szinten nincs rá hivatkozás. Tünet: a `.vectors` mérete 0, a `0x00000000`-n szemét vagy üres flash; a QEMU nem indul, vagy HardFaultba fut. Igazolás: `objdump -h` — a `.vectors` hiányzik vagy 0 méretű. Javítás: `KEEP(*(.vectors))`. A hiba csak `-Wl,--gc-sections` mellett jelentkezik; add hozzá a buildhez.

(b) Rossz sorrend: a `.vectors` a `.text` után kerül, így a VMA nem `0x00000000`, hanem a `.text` vége utáni cím. Tünet: a mag a `0x00000000`-ról olvas, ahol vagy üres flash, vagy a `.text` első szava van; SP és PC értelmezhetetlen. Igazolás: `objdump -h`, a `.vectors` VMA nem `0x00000000`. Javítás: a `.vectors` szekciót a `SECTIONS` első helyére tenni.

Mérési rész (levezetés): reset után a mag két szót olvas a `0x00000000`-ról és `0x00000004`-ről. Ha a flash 1 várakozási ciklust igényel szavanként, a két olvasás elméleti alsó korlátja 2 órajel a betöltéshez, plusz a belső szekvenálás. Ez **levezetett** érték; QEMU-emulációban nincs hardverhű abszolút ciklusszám, ezért mért értékként nem szerepelhet.

16.4.19 Rövid összefoglaló

A Cortex-M reset-szekvenciája a címtér első nyolc bájtjára támaszkodik: a `0x00000000`-n a kezdeti SP, a `0x00000004`-en a reset-handler címe áll. A vektortábla első 16 bejegyzése fix sorrendű rendszerkivétel, ezután következnek az IRQ-k. A linkernek a `KEEP` direktívával a flash elejére kell helyeznie a `.vectors` szekciót. A reset-handler érvényes SP-vel érkezik, és a `.data`/`.bss` kezelés előtt csak a startup kódra adja tovább a vezérlést. Az `objdump -s -j .vectors` a tábla tényleges tartalmát mutatja: az első szó SRAM-cím, a második páratlan Thumb-cím.

Továbbhasznált definíciók:

- **vektortábla (vector table):** a címtér elején elhelyezkedő, 4 bájtos bejegyzések listája; első szava a kezdeti SP, második a reset-handler címe.
- **kezdeti veremmutató (initial stack pointer):** a vektortábla 0. szava, amelyet a mag reset után hardveresen SP-be tölt.
- **reset-vektor:** a `0x00000004` címen lévő szó, a reset-handler Thumb-jelölt belépési címe.

16.4.20 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Armv7-M Architecture Reference Manual, DDI0403E — a vektortábla fix szerkezetének, a reset-szekvenciának és a Thumb-bit követelményének elsődleges forrása. **Ellenőrizve**: a link él (a `support.arm.com`-ra irányított átirányítás megerősített), de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- GNU Binutils as dokumentáció — `.section`, `.word`, `.thumb_func`, `.type` direktívák szemantikája. **Ellenőrizve**: aktuális kiadás binutils 2.47.
- QEMU System Emulation dokumentáció — `mps2-an385` machine memóriatérképe, betöltési és reset-viselkedése. **Ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak:

- Az `mps2-an385` pontos Code/SRAM régióméretei és a `0x00000000` aliasing viselkedése a QEMU verziójától függ; a laborban ellenőrizendő.
- A közölt `.vectors`-dump és az ellenőrző program kimenete a fő úton, GNU binutils 2.42-vel (`arm-none-eabi-as`, `arm-none-eabi-ld`, `arm-none-eabi-objdump`) épített ELF-en ellenőrizve (a lektor mérése): a dump betűre egyezik, az ellenőrző minden sora egyezik, kilépési állapot 0; a kanonikus firmware-en (16.7) is minden sor OK. Egy kézzel írt dump gyakori hibája, hogy a `0x0000000d` a vektortábla belsejébe mutat; ez érvénytelen vektor.
- A reset és a handler első utasítása közötti ciklusszám **levezetett**, QEMU-ban nem hardverhű mérés.

16.5 Linker script alapjai: szekciók memóriába helyezése

16.5.1 Motiváció: hol dől el, hogy egy bájt a flashbe vagy a RAM-ba kerül

A Reset és boot-folyamat: vektortábla alfejezet megmutatta, hogy a Cortex-M3 reset után a címtér első nyolc bájtjából olvassa a kezdeti SP-t és a reset-handler címét. A vektortábla tehát **fizikailag a címtér elején** kell legyen — de honnan tudja a linker, hogy az indító vektortáblának a flash `0x00000000` címére kell kerülnie, a `.data` inicializáló képe pedig szintén a flashbe, míg a `.data` futásidejű példánya a RAM-ba? Az `arm-none-eabi-gcc` alapértelmezett (built-in) linker scriptje egy általános, egyetlen összefüggő memóriát feltételező elrendezést ad, amely nem ismeri a flash/RAM megkülönböztetést, nem tud a Thumb reset-vektorról, és nem hozza létre azokat a szimbólumokat, amelyekből a startup kód a másolási és nullázási tartományokat kiolvassa. Bare-metal firmware-hez ezért **saját linker script** kell.

Ez az alfejezet egy minimális, de teljes linker scriptet épít fel: deklaráljuk a memóriarégiókat, elhelyezzük bennük a szekciókat, exportáljuk a startup kód számára szükséges szimbólumokat, és bekötjük a scriptet a buildbe az `ld --script` kapcsolóján keresztül. A startup kód belső logikáját (másólociklus, `bss`-nullázás) a Startup kód: `.data` másolása, `.bss` törlése tárgyalja; itt az a szerződés, amit a startup kód **feltételez**.

16.5.2 Definíció: linker script, MEMORY és SECTIONS

A **linker script** (linker script, összekötő leírás) egy szöveges leírás, amellyel a linkernek megadjuk, hogy a bemeneti objektumfájlok szekcióit milyen sorrendben és milyen abszolút címekre fűzze össze egy kimeneti ELF-fájlban. A GNU `ld` saját, kis nyelvet értelmez; ez a nyelv nem assembly és nem C, bár a `/* ... */` komment, a szimbólumnevek és az aritmetikai operátorok ismerősek benne.

Két kulcsblokk alkotja a csontvázat:

- **MEMORY blokk** — puszta *deklaráció*. Felsorolja a célprocesszor címtartományában rendelkezésre álló, egymást nem átfedő régiókat, mindegyikhez megadva egy nevet, egy kezdőcímet és egy hosszt, opcionális attribútumokkal. A MEMORY blokk önmagában **semmit nem helyez el**: csak nevet ad a címtartományoknak, hogy a SECTIONS blokk hivatkozhasson rájuk.
- **SECTIONS blokk** — a *helykiosztás*. Itt mondjuk meg, hogy mely bemeneti szekciók (`.text`, `.rodata`, `.data`, `.bss`, `.vectors`, …) melyik kimeneti szekcióba kerüljenek, milyen sorrendben, és a `>` operátorral azt, hogy melyik memóriarégióba.

A MEMORY blokk szintaxisa:

```
MEMORY
{
  FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}
```

- `ORIGIN` (röviden `org` vagy `o`) a régió első címe, `LENGTH` (röviden `len` vagy `l`) a hossza. A hossz `K`/`M` utótagot fogad el, decimális vagy `0x` előtagú hexadecimális értéket egyaránt.
- Az attribútumlista betűi: `r` (olvasható), `w` (írható), `x` (végrehajtható), `a` (allokálható), `i`/`l` (inicializált). Az attribútumok a hozzárendelés nélküli (árva) szekciók elhelyezését irányítják; a GNU ld nem figyelmeztet, ha egy írható szekciót kifejezetten `rx` régióba teszünk, és az átfedő MEMORY-régiókat sem jelzi. Ezért kellenek a scriptben az `ASSERT`-ek.

- A régiók **átfedése tilos**: ha két régió címtartománya metszi egymást, a linker hibát jelez.

A labor QEMU-profiljában (ARM MPS2+ AN385, Cortex-M3) az alábbi elrendezést használjuk, a memóriatérkép alfejezet QEMU-táblázata szerint. A tényleges címtartományt a helyi QEMU-verzióban a monitor `info mtree` parancsával érdemes visszaigazolni.

Régió	Kezdőcím	Méret	Attribútum	Tipikus tartalom
FLASH	`0x00000000`	4 MiB	`rx`	`.vectors`, `.text`, `.rodata`, a `.data` betöltési képe (LMA)
RAM	`0x20000000`	4 MiB	`rwx`	`.data` futásidejű mása (VMA), `.bss`, verem

Két megszorítás. A `FLASH` itt csak **régiónév**: QEMU-ban egy betöltött SSRAM-ra adott név, nem fizikai flash. A RAM pedig **egyetlen kanonikus tartomány**: a `0x20400000`-tól kezdődő terület ennek tükre, ezért egyik linker script sem használja második blokként.

16.5.2.1 A SECTIONS blokk és a helykiosztás

A SECTIONS blokkban minden kimeneti szekcióhoz megadjuk a tartalmát (bemeneti szekció-mintákkal, `*` helyettesítő karakterrel), az elhelyezés régióját, és a szükséges igazítást:

```
SECTIONS
{
  .text : {
    KEEP(*(.vectors))
    *(.text*)
    *(.rodata*)
    . = ALIGN(4);
  } > FLASH
}
```

- A **`.` (dot) a helyszámláló** (location counter): mindig az aktuális kimeneti cím. A `. = ALIGN(4);` azt jelenti, hogy a következő bájt 4 bájtos határra kerüljön. Ez a startup kód szavas (4 bájtos) másoló- és törlőciklusa miatt fontos: a ciklus csak akkor áll meg pontosan a végcímen, ha a kezdő- **és** a végcím is 4 bájtra igazított.
- A **`KEEP(...)`** direktíva megakadályozza, hogy a linker a `--gc-sections` (nem hivatkozott szekciók eldobása) hatására kidobja a vektortáblát. A vektortáblára C-kódból senki nem hivatkozik szimbolikusan, ezért a garbage collector „feleslegesnek” látná — a `KEEP` nélkül a firmware reset után azonnal hibás címre ugrana. Ez a Reset és boot-folyamat: vektortábla alfejezet `KEEP` követelményének a linkeroldali párja.
- A **`ENTRY(Reset_Handler)`** (a script elején vagy végén) megadja az ELF belépési pontját; a szimulátor és a debugger innen tájékozódik. Ez nem azonos a hardveres reset-vektorral, de bare-metal esetben ugyanazt a szimbólumot jelöli.

16.5.2.2 VMA és LMA: a `.data` kettős élete

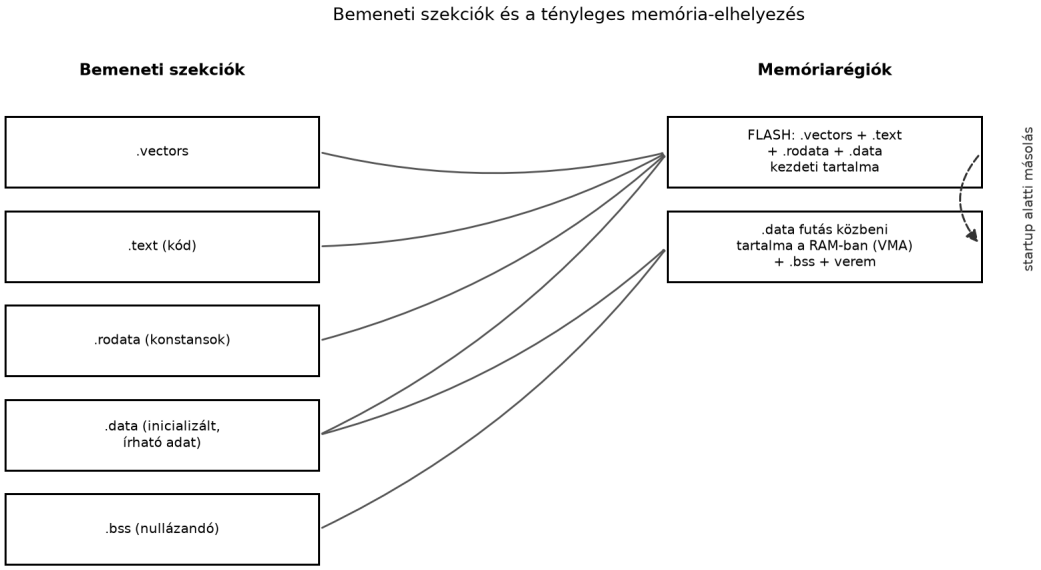
A linker kétféle címet tart nyilván minden szekcióhoz:

- **VMA (Virtual Memory Address)** — a futásidejű cím, ahol a program a szekciót *használja*. A `.data` VMA-ja a RAM-ban van.
- **LMA (Load Memory Address)** — a betöltési cím, ahonnan a szekció tartalma *származik*. A `.data` LMA-ja a flashben van, mert az inicializált változók értékeit a nem felejtő tároló tartalmazza.

A `> RAM AT> FLASH` szintaxis pontosan ezt fejezi ki: a szekció a RAM-ba kerül (VMA), de a tartalma a flashből töltődik (LMA). Alapértelmezésben LMA = VMA; a felülírás csak ott szükséges, ahol a kettő eltér — tipikusan a `.data`-nál. A `.bss`-nél az LMA-nak nincs gyakorlati szerepe, mert nincs betöltendő tartalma: csupa nulla, amit a startup kód állít elő. Ezért a `.bss (NOLOAD)` típusú, és nem növeli a firmware bináris méretét.

A VMA és az LMA a **linker** fogalma. Az ELF-fájlban nem ugyanott jelenik meg a kettő. A szekciófejléc (`Elf32_Shdr`, `Elf64_Shdr`) egyetlen címmezőt tartalmaz, az `sh_addr`-t: ez a szekció futásidejű címe, vagyis a VMA. Külön LMA-mező nincs. A betöltési cím a **programfejlécben** (`Elf32_Phdr`) látszik: a `LOAD` szegmens `p_vaddr` (`VirtAddr`) mezője a futásidejű, `p_paddr` (`PhysAddr`) mezője a fizikai, betöltési cím. A `readelf -SW` az előbbit, a `readelf -lW` az utóbbit mutatja. Az `objdump -h` LMA-oszlopa **származtatott** érték: az objdump a szekciót tartalmazó `LOAD` szegmensből számolja ki, `p_paddr + (sh_addr - p_vaddr)` alakban.

Az alábbi ábra a `lab-mps2.ld` bemeneti szekcióit (balra) és a tényleges kimeneti elhelyezést (jobbra) kapcsolja össze. A `.data` két nyíllal szerepel: egy a FLASH-be (LMA, a betöltési kép), egy a RAM-ba (VMA, a futásidejű másolat) — ez pontosan a fenti kettősség.



16.2. ábra: Bemeneti szekciók és a tényleges memória-elhelyezés - A lab-mps2.ld SECTIONS blokkja szerinti hozzárendelés; a KEEP és a NOLOAD jelentését a szövegtörzs tárgyalja.

16.5.2.3 Szimbólumok exportálása a linker scriptből

A linker scriptből **szimbólumokat** exportálhatunk a C/assembly oldal felé. A `_sdata = .;` forma azt jelenti: „a `_sdata` szimbólum értéke legyen a helyszámláló aktuális értéke”. Ez **cím**, nem változó: a C oldalon `extern char _sdata[];` alakban kell deklarálni, hogy a `_sdata` név

önmagában pointerré alakuljon (a gyakori hiba: `extern uint32_t _sdata;` majd `&_sdata` — ez a címzés szempontjából ugyanoda vezet, de a típus félrevezető).

Szimbólum	Érték	Jelentés
`_sidata`	`LOADADDR(.data)`	a `.data` inicializáló képének kezdete a flashben
`_sdata`	`.data` VMA kezdete	a másolás célja a RAM-ban
`_edata`	`.data` VMA vége	a másolás vége (kizárólagos)
`_sbss`	`.bss` kezdete	a nullázás kezdete
`_ebss`	`.bss` vége	a nullázás vége (kizárólagos)
`_estack`	`ORIGIN(RAM) + LENGTH(RAM)`	a kezdeti SP értéke — egyeznie kell a vektortábla 0. szavával

A `_sidata` értékét a `LOADADDR(.data)` adja vissza: a `.data` kimeneti szekció **betöltési címét** (LMA), amely a függvény szerződése szerint mindig **abszolút** cím. A kanonikus script a hozzárendelést a `.data` blokk után írja, közvetlenül a szekció mellé. Ez **olvashatósági döntés**, nem nyelvi követelmény. A linker kétféle kifejezést különböztet meg. Az **elhelyezési** kifejezéseket (például a helyszámlálót, `ALIGN`-t vagy egy szekció kezdőcímét meghatározó értéket) ott és akkor kell kiértékelnie, ahol a helykiosztás tart. Egy egyszerű **szimbólum-hozzárendelést** viszont, amelytől semmilyen elhelyezés nem függ, a helykiosztás végén is kiértékelhet. A `_sidata = LOADADDR(.data);` ilyen, ezért a `.data` előtt vagy a `.data` blokkon belül leírva is ugyanazt az abszolút címet adja. A GNU ld 2.42 és az `ld.lld` egy minimális scripttel mindhárom helyen ugyanazt az `A` típusú értéket adta.

16.5.3 Kidolgozott példa: `lab-mps2.ld` és a hozzá tartozó build

Munkakönyvtár: `~/arm-labor/cm/a065/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a065
cd ~/arm-labor/cm/a065
```

A példa linkeléséhez a `lab-startup.o` és a `lab-main.o` is kell; ezek forrása (`lab-startup.s`, `lab-main.c`) csak a Startup kód és a Firmware build és QEMU-betöltés munkafolyamat alfejezetben készül el. Addig a linker script önmagában vizsgálható; utána a két forrást másold át ide (`cp ~/arm-labor/cm/a066/lab-startup.s ~/arm-labor/cm/a067/lab-main.c .`), és fordítsd le őket a saját könyvtárukban megadott kapcsolókkal.

16.5.3.1 A script

Ez a II. rész **kanonikus linker scriptje**. A startup kód, a firmware build és a későbbi laborok (köztük a II. záróprojekt) ezt használják változatlanul, vagy ennek a másolatát bővítik. A párja a Startup kód alfejezet `lab-startup.s` fájlja: a vektortábla szekcióneve mindkettőben `.vectors`, a belépési pont `Reset_Handler`.

```
/* lab-mps2.ld - a II. rész kanonikus linker scriptje
 * Target: qemu-system-arm -machine mps2-an385 (Cortex-M3)
 * A "FLASH" itt csak régiónév: QEMU-ban a 0x00000000-tól induló, betöltött
 * ZBT SSRAM1, nem fizikai flash. A RAM a 4 MiB-os kanonikus SRAM-tartomány;
 * a 0x20400000-tól látszó tükörterületet szándékosan nem használjuk. */
```

```
ENTRY(Reset_Handler)

MEMORY
{
  FLASH (rx)  : ORIGIN = 0x00000000, LENGTH = 4M
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 4M
}

_Min_Stack_Size = 0x1000;          /* ennyi hely mindig maradjon a veremnek */

SECTIONS
{
  .vectors : {
    KEEP(*(.vectors))              /* a 0x00000000-ra; a --gc-sections se dobja el */
  } > FLASH

  .text : {
    *(.text*)
    *(.rodata*)
    . = ALIGN(4);
  } > FLASH

  .ARM.exidx : {                   /* unwind-táblák, ha a fordító készít ilyet */
    *(.ARM.exidx* .gnu.linkonce.armexidx.*)
  } > FLASH

  .data : ALIGN(4) {               /* VMA: RAM, LMA: FLASH */
    _sdata = .;
    *(.data*)
    . = ALIGN(4);                  /* a vég is szóhatár: a szavas másolás nem lép túl */
    _edata = .;
  } > RAM AT> FLASH
  _sidata = LOADADDR(.data);

  .bss (NOLOAD) : ALIGN(4) {
    _sbss = .;
    *(.bss*)
    *(COMMON)
    . = ALIGN(4);
    _ebss = .;
  } > RAM

  _estack = ORIGIN(RAM) + LENGTH(RAM);

  ASSERT(ADDR(.vectors) == ORIGIN(FLASH), "lab-mps2.ld: .vectors is not at the start of FLASH")
  ASSERT(_sidata % 4 == 0 && _sdata % 4 == 0 && _edata % 4 == 0,
         "lab-mps2.ld: .data bounds not word aligned")
  ASSERT(_sbss % 4 == 0 && _ebss % 4 == 0, "lab-mps2.ld: .bss bounds not word aligned")
  ASSERT(_ebss + _Min_Stack_Size <= _estack,
         "lab-mps2.ld: .data/.bss collide with the stack reserve")
  ASSERT(_estack % 8 == 0, "lab-mps2.ld: _estack is not 8-byte aligned")
}
```

Néhány döntés indoklása:

- A `.vectors` saját kimeneti szekció, a `FLASH` elején; az első `ASSERT` ezt linkeléskor ellenőrzi.
- A `.rodata`-t a `.text`-tel egy régióba tesszük, mert konstansokat nem írunk; így nem pazarolunk RAM-ot. Az esetleges `.ARM.exidx` unwind-táblák is a `FLASH`-be kerülnek, nem árva szekcióként a linker által választott helyre.
- A `.data` és a `.bss` **kezdete** (`ALIGN(4)` a szekciófejben) és **vége** (`. = ALIGN(4)` a záró szimbólum előtt) is szóhatárra esik. Így a szekció tartalma akkor is szavas ciklussal másolható, illetve törölhető, ha a benne lévő objektumok mérete 1, 3 vagy 5 bájt: a linker

a végét kitölti a következő szóhatárig. A `_sidata` (LMA) igazítását is ellenőrizzük, mert a másolás forrása is szavanként olvas.

- A `*(COMMON)` a `-fcommon`/`-fno-common` fordítási beállítástól függő, inicializálatlan globálisokat gyűjti be; a `bss`-be sorolásuk biztosítja, hogy a startup nullázza őket.
- A `.bss (NOLOAD)` jelölése deklaratív: dokumentálja, hogy nincs betöltendő tartalom. A `NOBITS` típus miatt a linker egyébként sem írna ki bájtokat a fájlba.
- Az `ASSERT`-ek a szerződést **linkeléskor** kényszerítik ki: ha valamelyik határ nem szóhatár, vagy a `.data` + `.bss` és a legalább `_Min_Stack_Size` bájtos veremtartalék nem fér el a RAM-ban, a linkelés hibával megáll. A verem a RAM tetejéről lefelé nő, ezért a legegyszerűbb ütközésvizsgálat az, hogy a `.bss` vége és az `_estack` között maradjon elég hely. A tényleges veremigény a programtól függ; a `_Min_Stack_Size` egy tervezési érték, nem mért adat.

16.5.3.2 Build: az `ld --script` kapcsoló

Ha közvetlenül az `ld`-t hívjuk, a scriptet a `--script` (egybetűs alakja `-T`) kapcsolóval adjuk át:

```
arm-none-eabi-ld --script=lab-mps2.ld -o lab-firmware.elf \
    lab-startup.o lab-main.o
```

A `-T` a GNU `ld` összes verziójában elérhető, és a `--script` hosszú alak ugyanezt jelenti. Ha a GCC drivert használjuk (ez a szokásos, mert így a `-mcpu`, `-mthumb` és a könyvtárak beállítása egységes marad), a scriptet a `-Wl,` prefixszel adjuk tovább, illetve a `-T` rövidítést a driver közvetlenül is elfogadja és továbbadja a linkernek:

```
CC       := arm-none-eabi-gcc
LDSCRIPT := lab-mps2.ld
CFLAGS   := -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -g3 -O0
LDFLAGS  := -Wl,--script=$(LDSCRIPT) -Wl,-Map=lab-firmware.map \
            -Wl,--gc-sections

OBJS     := lab-startup.o lab-main.o

# A linker script is előfeltétel: ha csak a script változik (MEMORY, ASSERT),
# a Make újralinkel, újrafordítás nélkül. A receptben az explicit $(OBJS) áll,
# nem a $^: az a scriptet egyszerű bemeneti fájlként is átadná a linkernek.
lab-firmware.elf: $(OBJS) $(LDSCRIPT)
        $(CC) $(CFLAGS) $(LDFLAGS) -o $@ $(OBJS)

%.o: %.s
        $(CC) $(CFLAGS) -c -o $@ $<
%.o: %.c
        $(CC) $(CFLAGS) -c -o $@ $<
```

A `$(LDSCRIPT)` előfeltételként azért kell, mert a következő feladatok éppen a scriptet módosítják. Nélküle egy sikeres build után a Make a régi ELF-et késznek tekintené, és egy elrontott `ASSERT` sem állítaná meg. A helyettesítő eszközlánccal kipróbálva: a script puszta módosítása után csak a linkelés futott le, objektumfordítás nélkül. A szándékosan túl nagyra állított `_Min_Stack_Size` mellett a következő `make` a `lab-mps2.ld: .data/.bss collide with the stack reserve` hibával, 2-es állapottal leállt. A régi, csak az objektumoktól függő szabállyal ugyanez a `make up to date` üzenetet adott, 0-s állapottal.

A `-Wl,-Map=` kapcsoló elkészíti a **map fájlt**: ez a szekciók pontos VMA/LMA címeit és méreteit, valamint a szimbólumok értékét listázza. Overlap- vagy mérethibánál gyakorlatilag mindig a

map fájlt olvassuk, nem a forráskódot. A `--gc-sections` és a `KEEP` együttműködése fentebb már előkerült: a `KEEP` nélkül a vektortábla eltűnik.

Segédeszközök a script helyességének ellenőrzésére:

- `arm-none-eabi-ld --verbose` — kiírja a beépített alapértelmezett scriptet; érdemes összevetni a sajátunkkal, hogy lássuk, mit *nem* örököltünk.
- `arm-none-eabi-nm -n lab-firmware.elf` — címsorrendben listázza a szimbólumokat.
- `arm-none-eabi-size lab-firmware.elf` — összegzi a `.text`/`.data`/`.bss` méretét.
- `arm-none-eabi-objdump -h lab-firmware.elf` — a kimeneti szekciók listája VMA- és LMA-oszloppal (az LMA-t az objdump a programfejlécből számolja).
- `arm-none-eabi-readelf -SW` és `-lW lab-firmware.elf` — a szekciófejlécek (`sh_addr`, típus, méret), illetve a `LOAD` szegmensek (`VirtAddr`, `PhysAddr`, `FileSiz`, `MemSiz`).

16.5.3.3 Kimenet (a fő úton mérve: GNU ld 2.42; a lektor mérése)

A kanonikus `lab-mps2.ld`, a Startup kód `lab-startup.s`-e és a firmware build `lab-main.c`-je együtt linkelve (GNU binutils 2.42, `arm-none-eabi-gcc` + `arm-none-eabi-ld`) ezeket a szimbólumértékeket adta. A GNU `nm` a Thumb-bitet levágja, ezért a handlerek címe páros alakban látszik (a páratlan `st_value` a `readelf -s`-ben látható):

```
00000040 T Reset_Handler
0000006a T Default_Handler
00000170 T main
000001d0 A _sidata
20000000 D _sdata
20000014 D _edata
20000014 B _sbss
20000028 B _ebss
20400000 B _estack
```

Az ellenőrzési feltételek: (1) `_sdata` és `_edata` `0x2000....` tartományban van, `_sidata` viszont `0x0000....` (flash) tartományban; (2) `_sidata` `A` (abszolút) típusú, mert a `LOADADDR` abszolút LMA-t ad vissza; (3) minden határ 4-gyel osztható (a `.data` itt 13 bájtnyi objektumot tartalmaz, a szekció eszközláncfüggő kitöltéssel 16 bájt Clang + `ld.lld` mellett, 20 bájt GNU-val, mert a vége szóhatárra van kitöltve); (4) `_ebss + _Min_Stack_Size <= _estack`; (5) a `.vectors` a `0x00000000`-n van, a második szava (`0x00000041`) páratlan, és bit0 nélkül a `.text`-be mut. Az (5) pontot a vektortábla-ellenőrző program gépiesen vizsgálja. A (3) és a (4) pontot az `ASSERT`-ek miatt nem is lehet megsérteni úgy, hogy a linkelés sikeres legyen.

Ellenpróba. Ha a `.data` blokkból kivesszük a záró `. = ALIGN(4);` sort, a linkelés a `lab-mps2.ld: .data bounds not word aligned` hibával megáll (`_edata = 0x2000000d`). Az `ASSERT` kikapcsolásával a hiba futásidőben jelentkezik, és hogy hogyan, az a startup ciklusán múlik: ezt a Startup kód alfejezet mutatja meg.

16.5.4 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés	Egy változtatás
`region 'RAM' overflowed by 1234 bytes`	a RAM-ba szánt tartalom meghaladja a `LENGTH`-et	`arm-none-eabi-size lab-firmware.elf`, `lab-firmware.map`	`LENGTH` növelése (ha a valós hardver engedi) vagy egy nagy statikus tömb `const`-tá alakítása
`section .bss at 0x20000040 overlaps .data at 0x20000040`	hiányzó `ALIGN` vagy elrontott sorrend a SECTIONS blokkban	`objdump -h`	a `.bss` blokk elé `. = ALIGN(4);`, illetve a szekciók sorrendjének ellenőrzése
nincs figyelmeztetés, de a `.data` a flash-ben marad	a `MEMORY` attribútumait a linker csak az árva szekciókra használja; a kifejezett `> FLASH` hozzárendelést nem ellenőrzi	`objdump -h` LMA/VMA, `readelf -lW`	`> RAM AT> FLASH` és `ASSERT` a scriptben
a `.data` változók a flashben maradnak, a RAM-ban szemét van reset után azonnal hard fault	hiányzó `AT> FLASH` vagy hiányzó `LOADADDR`	`objdump -h` LMA-oszlopa; startup másolóciklus	`> RAM AT> FLASH` és `_sidata = LOADADDR(.data);`
	a vektortáblát a `--gc-sections` kidobta	`objdump -s -j .vectors`, `lab-check-vectors.py`	`KEEP(*(.vectors))`
`lab-mps2.ld: .data/.bss collide with the stack reserve`	a `.data` + `.bss` és a veremtartalék nem fér el a RAM-ban	`size -A`, map fájl	nagy statikus tömb `const`-tá alakítása vagy kisebb puffer; a `_Min_Stack_Size` csökkentése csak mért veremigény alapján
`lab-mps2.ld: .data bounds not word aligned`	a `.data` vagy `.bss` vége nincs szóhatárra igazítva	map fájl, `nm -n`	`. = ALIGN(4);` a záró szimbólum elé

A tipikus hiba diagnosztikája mindig ugyanaz a ciklus: **tünet → hipotézis → adat (`map`/`objdump`/`nm`) → egyetlen változtatás → újramérés.** A linker script hibái jellemzően nem fordítási hibaként, hanem futásidejű hard faultként vagy csendesen rossz adatként jelentkeznek, ezért a map fájl olvasása nem luxus, hanem a hibakeresés első lépése.

16.5.5 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd el saját szavaiddal a különbséget a `.data` VMA-ja és LMA-ja között. Miért nem lehet a `.bss`-nek értelmes LMA-ja? Mit ír az `nm` az `_sidata` és a `_sdata` szimbólumok mellett, és miért tér el a betűjelük? Végül a saját firmware-eden keresd meg a `.data` szekció `sh_addr` címét (`readelf -SW`), az őt hordozó `LOAD` szegmens `VirtAddr` és `PhysAddr` értékét (`readelf -lW`), valamint a `.data` VMA- és LMA-oszlopát (`objdump -h`), és mondd meg, melyik érték melyikkel egyezik. **2. (implementációs, középszint — cél: rutinépítés)** Egészítsd ki a `lab-mps2.ld` scriptet egy külön `.rodata` kimeneti szekcióval, amely a `.text` után, de még a flashben helyezkedik el, és exportálj egy `_erodata` szimbólumot a végére. Igazítsd 4 bájtos határra, majd ellenőrizd a

`objdump -h` kimenetén, hogy a `.rodata` LMA-ja egyezik a VMA-jával (mert flashben fut). **3. (hibakeresési/mérési, középszint — cél: rutinépítés)** Csökkentsd a `RAM` régió `LENGTH`-jét `8K`-ra, tegyél a `lab-main.c`-be egy `static volatile uint8_t big[6144];` tömböt, és építsd újra a firmware-t. Olvasd ki a linker hibaüzenetét, majd a map fájlból keresd meg, melyik szekció és melyik szimbólum felelős a túlcsordulásért. Ezután állítsd vissza a `LENGTH`-et, és igazold, hogy a hiba eltűnt. (A laborban ez az egyetlen kontrollált, jelölt memóriarégió-módosítás; más címet ne írj át.) **4. (mérési, alapszint — cél: rutinépítés)** Vedd fel a firmware `.text`, `.data` és `.bss` méretét az `arm-none-eabi-size -A` kimenetéből. Hasonlítsd össze a `-O0` és `-O2` fordítás mellett kapott `.text` méreteket, és fogalmazd meg, miért nem változik a `.bss` mérete érdemben az optimalizálástól.

16.5.6 Megoldások

1. megoldás — Segítség. Gondold át külön, mit jelent "hol fut" (VMA) és "honnan tölt be" (LMA) egy szekció esetén, és hogy a `.bss`-nek van-e egyáltalán betöltendő tartalma.

1. megoldás — Teljes megoldás. A VMA a szekció futásidejű címe: a `.data`-t a RAM-ban olvassa és írja a program. Az LMA a betöltési cím: a `.data` inicializáló tartalma a flashben van, mert a RAM tartalma áramkimaradás után elveszik. Az ELF-fájlban a kettő **nem ugyanabban a fejlécben** van. A szekciófejléc `sh_addr` mezője a szekció futásidejű címe (VMA); külön LMA-mezője nincs. Az LMA a linker által kiszámolt külön tulajdonság, amely a `LOAD` programfejléc `PhysAddr` (`p_paddr`) mezőjében jelenik meg, az `objdump -h` pedig ebből vezeti le a szekciónkénti LMA-oszlopot. A kanonikus firmware (`lab-mps2.ld` + `lab-startup.s` + `lab-main.c`, Clang + `ld.lld` linkelte, GNU `readelf`/`objdump` 2.42 olvasta) a három nézetben, a lényeges sorokra szűkítve:

```
$ readelf -SW lab-firmware.elf
  [Nr] Name              Type            Addr     Off    Size   ES Flg Lk Inf Al
  [ 4] .data             PROGBITS        20000000 020000 000010 00  WA  0   0  4
  [ 5] .bss              NOBITS          20000010 020010 000010 00  WA  0   0  4
$ readelf -lW lab-firmware.elf
  Type           Offset   VirtAddr   PhysAddr   FileSiz MemSiz  Flg Align
  LOAD           0x020000 0x20000000 0x00000214 0x00010 0x00010 RW  0x10000
  LOAD           0x020010 0x20000010 0x20000010 0x00000 0x00010 RW  0x10000
$ objdump -h lab-firmware.elf
Idx Name          Size      VMA       LMA       File off  Algn
  3 .data         00000010  20000000  00000214  00020000  2**2
                  CONTENTS, ALLOC, LOAD, DATA
  4 .bss          00000010  20000010  20000010  00020010  2**2
                  ALLOC
```

A `.data` szekció `sh_addr` értéke `0x20000000`: ez a VMA, a RAM-beli futásidejű cím. A `.data`-t hordozó `LOAD` szegmens `VirtAddr` mezője ugyanez, a `PhysAddr` viszont `0x00000214`: ez a flashbeli betöltési cím, és egyben a `_sidata` értéke. Az `objdump -h` sora a kettőt egymás mellé teszi (VMA `20000000`, LMA `00000214`), az LMA-oszlop azonban nem a szekciófejlécből jön.

A `.bss`-nél két dolgot kell külön kezelni. A szekciónak **van futásidejű címe** (`sh_addr = 0x20000010`), és **memóriát foglal**: a `Size` mező 16 bájt, a szegmensben pedig `MemSiz = 0x10`. Ide kerülnek a változók. **Betöltendő bájtja** viszont nincs: a típusa `NOBITS`, a fájlban nem foglal helyet (`FileSiz = 0`), és a betöltő sem másol belőle semmit. A tartalma csupa nulla, a **nullázási igényt** pedig a startup kód teljesíti a `_sbss`–`_ebss` tartományon. Ezért a `.bss`-nél az LMA-nak nincs gyakorlati szerepe, a `.data`-nál viszont ez a másolás forrása. Az objdump által kiírt `.bss`-LMA is csak a szegmens-elrendezésből adódik, és linkerfüggő. Az `ld.lld` itt külön szegmenst nyit, és `PhysAddr = VirtAddr`. A GNU ld 2.42 egy hasonló scripttel (`.data` VMA `0x20000`, LMA `0x10001`) a `.bss`-t a `.data` szegmensébe tette (`FileSiz 0x4`, `MemSiz 0x14`),

ezért az objdump a `.bss`-hez a semmire nem használt `0x10005` LMA-t írta ki. Az `nm` az `_sidata` mellett `A`-t (abszolút) ír, mert a `LOADADDR` a szerződése szerint abszolút címet ad vissza, akárhol áll a hozzárendelés. A `_sdata` a `.data` blokkon belül, a helyszámlálóból (`.`) kapja az értékét, ezért a szekcióhoz kötött `D` (inicializált adatszekció) jelölést kap.

2. megoldás — Segítség. A `.rodata` a `.text` után, de még FLASH-ben marad — nincs szüksége `AT>` felülírásra, mert a VMA és az LMA itt megegyezik. Gondold át azt is, hogy a kanonikus `.text` blokk már tartalmaz egy `*(.rodata*)` mintát. Egy bemeneti szekció ahhoz a kimeneti szekcióhoz kerül, amelynek mintája a scriptben **elsőként** illeszkedik rá.

2. megoldás — Teljes megoldás. A kanonikus `.text` blokkban szerepel a `*(.rodata*)` minta. Egy bemeneti szekció (a fordító által előállított `.rodata`, `.rodata.str1.1` stb.) ahhoz a kimeneti szekcióhoz kerül, amelynek mintája a scriptben elsőként illeszkedik rá. Ha tehát az új `.rodata` blokkot egyszerűen a `.text` után szúrjuk be, a konstansokat a `.text` már elfogyasztotta, és az új kimeneti szekció **üres** marad. A mintát ezért a `.text`-ből ki kell venni, és át kell vinni az új szekcióba. A módosított `SECTIONS`-részlet:

```
.text : {
  *(.text*)
  /* a *(.rodata*) innen átkerült a saját kimeneti szekciójába */
  . = ALIGN(4);
} > FLASH

.rodata : ALIGN(4) {             /* VMA = LMA: flashben marad, nincs AT> */
  *(.rodata*)
  . = ALIGN(4);
  _erodata = .;                  /* a konstansok vége */
} > FLASH

.ARM.exidx : {                   /* változatlan, a .rodata után */
  *(.ARM.exidx* .gnu.linkonce.armexidx.*)
} > FLASH
```

A **bemeneti** szekció a fordító által egy objektumban előállított rész (például `lab-main.o:(.rodata)`), a **kimeneti** szekció a linker scriptben definiált, az ELF-be kerülő egység (`.rodata`). A map fájlban a kimeneti szekció sora alatt a hozzá rendelt bemeneti szekciók láthatók, ezért ott ellenőrizhető, hová kerültek a konstansok. A helyettesítő eszközlánccal (`ld.lld`) egy `const char msg[] = "lab-rodata";` konstanssal kipróbálva: ha a `.text` megtartja a mintát, az új `.rodata` mérete 0 bájt, és a `msg` a `.text`-be kerül (`nm`: `T`). A javított scripttel a `.rodata` 12 bájt, a `msg` benne van (`nm`: `R`), és az `_erodata` pontosan a szekció végére mutat. A GNU ld ugyanígy viselkedik.

Az `objdump -h`-ban a `.rodata` VMA- és LMA-oszlopa egyezik, mert nincs `AT>` felülírás: a szekció flashben fut, nem másoljuk. A külön kimeneti szekció nem azért kell, mert a `.text`-be gyűjtött konstansok végét nem lehetne jelölni. A `.text` blokkon belül is elhelyezhető szimbólum a minta előtt és után (például `_srodata = .; *(.rodata*) _erodata = .;`). A külön szekció azért hasznos, mert a map fájlban, az `objdump -h`-ban és a `size -A`-ban önálló sorként, saját mérettel jelenik meg.

3. megoldás — Segítség. A linker hibaüzenete megnevezi, melyik régió csordult túl — innen indulj a `map` fájl „Memory Configuration” szakaszával, és nézd meg, melyik szekció viszi a legtöbb helyet.

3. megoldás — Teljes megoldás. A várt hibaüzenet alakja: `region 'RAM' overflowed by N bytes`. A map fájlban a `Memory Configuration` szakasz adja a régiók méretét és a hozzájuk rendelt szekciók listáját; a túlcsordulás tipikusan a `.bss`-ből vagy egy nagy statikus tömbből származik. A helyes diagnózis: először a `.bss` méretét nézzük (`size -A`), majd a legnagyobb

szimbólumot (`nm --size-sort`). A kanonikus scriptben a 8 KiB-os RAM-ban a 6 KiB-os tömb a `.bss`-be még befér, de a 4 KiB-os veremtartalékkal együtt már nem, ezért itt a `lab-mps2.ld: .data/.bss collide with the stack reserve` ASSERT-üzenet jelenik meg; egy 8 KiB-nál nagyobb tömb a `region 'RAM' overflowed` hibát adja. Ha a valós hardveren a RAM ennél nagyobb (a QEMU-gépen 4 MiB), a `LENGTH` visszaállítása a helyes megoldás; ha szűkös a RAM, a nagy tömböt `const`-tá kell alakítani, hogy átkerüljön a flashbe. **Ellenpróba:** a `LENGTH` visszaállítása után a `map` fájlban a `Memory Configuration` soraiban a „used” érték kisebb kell legyen a „size”-nál.

4. megoldás — Segítség. Gondold át, mi okozza a `.text` méretváltozását optimalizáláskor, és miért más a `.bss` tartalmának eredete (deklaráció, nem generált kód).

4. megoldás — Teljes megoldás. A `size -A` táblázatosan listázza a szekciókat. Az `-O2` tipikusan csökkenti a `.text`-et (holt kód eltávolítása, kisebb utasításszekvenciák), esetleg növeli (beágyazott `memcpy`-k, loop unrolling). A `.bss` mérete azért nem változik érdemben, mert a benne lévő objektumok (globális, inicializálatlan tömbök, pufferek) a forráskód deklarációiból származnak, nem a generált kódból; az optimalizálás csak akkor nyúl hozzá, ha a teljes objektumot ki tudja dobni (pl. `static` és soha nem olvasott). Ez a „fordítási szint hatása a generált kódra” kérdéskör, amelyet a Fordítói optimalizálási szintek hatása a generált kódra alfejezet tárgyal részletesen.

16.5.7 Rövid összefoglaló

A GNU `ld` és `as` hibaüzenetei szándékosan angolul szerepelnek, mert az eszközlánc így írja ki őket; a magyarázat a könyv szövegében van. A linker script a MEMORY és SECTIONS blokkból épül fel: a MEMORY deklarálja a régiókat (név, `ORIGIN`, `LENGTH`, attribútumok), a SECTIONS végzi a tényleges helykiosztást. A `.data` esetében a VMA (RAM) és az LMA (flash) szétválik; a `> RAM AT> FLASH` szintaxis és a `LOADADDR(.data)` fejezi ki a kettősséget. A scriptből exportált `_sdata`/`_edata`/`_sbss`/`_ebss`/`_sidata` szimbólumok a startup kód másolási és nullázási szerződése. A scriptet az `ld --script=` (vagy `-T`) kapcsolóval kötjük be, és a map fájllal ellenőrizzük. A tipikus hibák a szekció-átfedés, a régiótúlcsordulás és a `KEEP` hiánya miatt kidobott vektortábla.

Továbbhasznált definíciók:

- **linker script:** a linker helykiosztását leíró szöveges nyelv; MEMORY (régiódeklaráció) és SECTIONS (szekció-elhelyezés) blokkokból áll.
- **VMA és LMA:** a szekció futásidejű, illetve betöltési címe; a `.data` VMA-ja a RAM-ban, LMA-ja a flashben van. Az ELF-ben a VMA a szekciófejléc `sh_addr` mezője, az LMA a `LOAD` programfejléc `PhysAddr` mezőjében látszik.
- **memóriarégió (region):** a MEMORY blokkban deklarált, attribútumokkal ellátott címtartomány, amelybe a SECTIONS blokk `>` operátorral helyez szekciókat.

16.5.8 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Armv7-M Architecture Reference Manual, DDI0403E — a memóriatérkép-tartományok és az M-profile futási modell elsődleges forrása. *Ellenőrizve*: a link él (a `developer.arm.com` → `support.arm.com` átirányítás megerősített), de a JS-renderelt oldal miatt a pontos kiadásbetű automatikus lekéréssel nem volt kinyerhető.

- GNU Binutils as dokumentáció — a `.section` direktíva és a szekciónevek konvenciói. *Ellenőrizve*: a jelenlegi kiadás binutils 2.47.
- GNU Binutils ld dokumentáció — a MEMORY és SECTIONS blokk, a `LOADADDR`, a `KEEP`, az `ENTRY` és a `--script` pontos szemantikája. *Ellenőrizve*: ugyanaz a binutils 2.47-es kiadás tartalmazza.
- QEMU rendszeremulációs dokumentáció — az MPS2-AN385 machine definíciója és memory mapje. *Ellenőrizve*: a `master`ághoz tartozó dokumentáció, nem egy konkrét kiadott verzióhoz kötött; a ténylegesen telepített QEMU-verzió dokumentációja ettől eltérhet.

Ellenőrzési hiányok: (1) A `lab-mps2.ld` scriptet a fő úton, GNU binutils 2.42-vel (`arm-none-eabi-gcc` + `arm-none-eabi-ld`) linkeltük a `lab-startup.s`-sel és a `lab-main.c`-vel; a fenti `nm`-értékek ebből származnak, és minden ellenőrzési feltétel teljesül. A GNU `ld` a `.bss`-t a `.data` LOAD szegmensébe teszi (`FileSiz 0x14`, `MemSiz 0x28`). A firmware a helyettesítő Unicorn-alapú Cortex-M3 emulátorban (`tools/cm-run.py`) `boot ok`-ot írt; az `ASSERT`-ek ellenpróbája (igazítás nélküli `.data`) helyettesítő eszközlánccal készült. (2) A fenti FLASH/RAM memóriatérkép **munkafeltételezés**, amelyet a helyi QEMU-verzió `info mtree` kimenetével vagy az `arm-none-eabi-ld` konkrét alapértelmezett scriptjével kell visszaigazolni. (3) A `region 'RAM' overflowed by N bytes` üzenet és a `section ... overlaps ...` üzenet szövege a GNU `ld` verziójától függően kismértékben eltérhet; a konkrét szöveget a telepített `arm-none-eabi-ld` forrásában vagy a helyi man oldalon kell ellenőrizni. (4) A Makefile-ban szereplő `-Wl,--script=` és a `-T` viszonyát a telepített GCC-verzió dokumentációjával érdemes megerősíteni.

16.6 Startup kód: .data másolása, .bss törlése

16.6.1 Motiváció: miért nem elég a linker script?

Az előző alfejezetben (Linker script alapjai: szekciók memóriába helyezése) láttuk, hogy a `.data` szekció `VMA`-ja (futásidejű címe) RAM-ban van, míg az `LMA`-ja (betöltési címe) a flashben. Ez a szétválás nem esztétikai: a flash memória a Cortex-M-en tipikusan csak olvasható a futó program számára, így a C nyelvű globális változók kezdőértéke egy másik memóriarégióba kerül, mint ahol futásidőben lakniuk kell.

A C programozó ezt soha nem látja: azt írja, hogy

```
int counter = 42;      /* inicializált globális */
int buffer[64];        /* nulla-inicializált globális */
```

és elvárja, hogy a `main()` első utasításakor a `counter` értéke 42, a `buffer` minden eleme 0 legyen. Ez az elvárás **nem az ISA-ból, hanem a C runtime szerződéséből** következik. Az ISA csak annyit mond, hogy bizonyos bájtok bizonyos címen adott értéket tartalmaznak — azt nem, hogy ezt ki és mikor állítja be. Ezt a feladatot a **startup kód** (startup code, néha reset handler) végzi el, mielőtt bármilyen C függvény lefutna. Ebben az alfejezetben megírjuk a `.data` másoló és a `.bss` nullázó ciklust, és megmutatjuk, miért nélkülözhetetlen a linker script és a startup kód összehangolása.

16.6.2 Definíció és működés

Startup kód (startup code): az a legalacsonyabb szintű, kézzel írt assembly rutin, amely a `main()` meghívása előtt lefut, és a C runtime előfeltételeit megteremti. A Cortex-M-en ezt tipikusan a reset vektorba írt címre helyezzük (a vektortábla részleteit lásd Reset és boot-folyamat: vektortábla), így a processzor hardveres újraindításkor automatikusan ide kerül vezérlés.

A startup kód legalább három dolgot végez:

1. **Statikus inicializált adat másolása** a flash-beli kezdőérték-tömbből a RAM-beli `.data` szekcióba.
2. **`.bss` szekció nullázása** — azok a globális és statikus változók, amelyeket C-ben nem inicializáltunk, a C standard szerint 0 értékkel kezdenek. Ehhez nem tárolunk 0-bájtokat a flashben; csak a területet foglaljuk le, és futásidőben nullázzuk.
3. **A `main()` meghívása** — innen veszi át a vezérlést a C világ.

Két külön rutin, két különböző cél. Az első adatot mozgat; a második adat nélkül dolgozik (csak nulláz). Mindkettő ciklushatárait a linker scriptből **exportált szimbólumok** adják.

16.6.3 Miért nem elég a flash-beli inicializálás?

Képzeljünk el egy `.data` szekciót, amely 1 KiB-ot foglal el a RAM-ban. Futásidőben a program írhat bele (hiszen ez RAM). Ha a `.data` LMA-ját és VMA-ját ugyanarra a flash-beli címre tennénk, akkor a `counter = 43;` utasítás egy flash-címre próbálna írni — a Cortex-M-en ez vagy hard fault, vagy (rosszabb esetben) csendben elveszik, és a globális változó minden írás után visszaáll a kezdőértékre. Ezért van az `AT> FLASH` és a `> RAM` kombináció.

A startup kód tehát a linker által kigondolt **LMA→VMA leképezést valósítja meg futásidőben**: bájtonként (szavanként) átmásolja a tartalmat a flash-beli helyéről a RAM-beli helyére.

16.6.4 A linker script exportált szimbólumai

A linker script `SECTIONS` blokkjában szimbólumokat definiálhatunk úgy, hogy egy címkéhez a szekció aktuális helyét kötjük. A Linker script alapjai kanonikus `lab-mps2.ld`-jének vonatkozó része (a teljes scriptet ott találod, ez az alfejezet változatlanul azt használja):

```
.data : ALIGN(4) {                /* VMA: RAM, LMA: FLASH */
  _sdata = .;
  *(.data*)
  . = ALIGN(4);                   /* a vég is szóhatár: a szavas másolás nem lép túl */
  _edata = .;
} > RAM AT> FLASH
_sidata = LOADADDR(.data);

.bss (NOLOAD) : ALIGN(4) {
  _sbss = .;
  *(.bss*)
  *(COMMON)
  . = ALIGN(4);
  _ebss = .;
} > RAM
```

A startup kód és a linker script közös szerződése: **mind az öt határ (`_sidata`, `_sdata`, `_edata`, `_sbss`, `_ebss`) 4 bájtra igazított**, és `_sdata <= _edata`, `_sbss <= _ebss`. A linker script ezt `ASSERT`-tel ellenőrzi, a startup kód pedig erre a szerződésre építve dolgozhat szavanként.

Ezek a szimbólumok **nem** memóriacímet tartalmazó változók: maguk a szimbólumok léteznek, és a címük az, amit képviselnek. Assemblyben ezért így hivatkozunk rájuk:

```
ldr r1, =_sdata
```

A `ldr Rd, =symbol` pszeudo-utasítás a linker relocation során `symbol` címét tölti be `Rd`-be. Nincs szükség `[pc, #offset]` kézi számolgatásra — a linker elvégzi helyettünk.

Fontos: a startup assembly fájlban ezek a szimbólumok **külső** (extern) szimbólumok, ezért nem kell `.global` direktívával deklarálni őket (a linker script globális szimbólumai bármelyik objektumból láthatók). Ha viszont egy assembly vagy C fájl `.extern _sdata`-ként hivatkozik rájuk, az is működik — a fordítás szempontjából az a fontos, hogy **ne definiáljuk újra**.

16.6.5 Kidolgozott példa: teljes startup rutin

Munkakönyvtár: `~/arm-labor/cm/a066/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a066
cd ~/arm-labor/cm/a066
```

Az alábbi `lab-startup.s` a II. rész **kanonikus startup kódja**: a vektortáblát (`.vectors`), a `Reset_Handler`-t és a gyenge aliasokkal bekötött `Default_Handler`-t tartalmazza. A párja a Linker script alapjai `lab-mps2.ld`-je; a firmware build és a későbbi laborok ezt a két fájlt másolják át.

```
/* lab-startup.s - a II. rész kanonikus startup kódja
 * ISA: Thumb-2 / Armv7-M; Target: qemu-system-arm -machine mps2-an385
 * Szerződés a lab-mps2.ld-vel: _sidata, _sdata, _edata, _sbss, _ebss
 * mind 4 bájtra igazított (a linker script ASSERT-tel ellenőrzi). */
    .syntax unified
    .cpu cortex-m3
    .thumb

    .section .vectors, "a", %progbits
    .align 2
    .global __vectors
__vectors:
    .word _estack                /* 0x00: kezdeti SP (MSP)      */
    .word Reset_Handler          /* 0x04: Reset (bit0 = 1)      */
    .word NMI_Handler            /* 0x08: NMI                   */
    .word HardFault_Handler      /* 0x0C: HardFault             */
    .word MemManage_Handler      /* 0x10: MemManage             */
    .word BusFault_Handler       /* 0x14: BusFault              */
    .word UsageFault_Handler     /* 0x18: UsageFault            */
    .word 0, 0, 0, 0             /* 0x1C-0x28: fenntartott      */
    .word SVC_Handler            /* 0x2C: SVCall                */
    .word DebugMon_Handler       /* 0x30: DebugMonitor          */
    .word 0                      /* 0x34: fenntartott           */
    .word PendSV_Handler         /* 0x38: PendSV                */
    .word SysTick_Handler        /* 0x3C: SysTick               */

    .text
    .thumb_func
    .global Reset_Handler
    .type   Reset_Handler, %function
Reset_Handler:
    /* 1. .data másolása FLASH (LMA) -> RAM (VMA), szavanként.
       Elöltesztelő ciklus, előjel nélküli összehasonlítással: üres
       szekciónál egyszer sem fut, és a végcímen pontosan megáll. */
    ldr   r0, =_sidata
    ldr   r1, =_sdata
    ldr   r2, =_edata
1:  cmp   r1, r2
    bhs   2f
```

```
    ldr   r3, [r0], #4
    str   r3, [r1], #4
    b     1b
    /* 2. .bss nullázása, szavanként. */
2:  ldr   r1, =_sbss
    ldr   r2, =_ebss
    movs  r3, #0
3:  cmp   r1, r2
    bhs   4f
    str   r3, [r1], #4
    b     3b
    /* 3. main(); ha visszatérne, végtelen ciklus. */
4:  bl    main
5:  b     5b
    .size Reset_Handler, . - Reset_Handler

    .thumb_func
    .global Default_Handler
    .type   Default_Handler, %function
Default_Handler:
    b     .
    .size Default_Handler, . - Default_Handler

    /* Gyenge aliasok: amit a firmware nem definiál, az a Default_Handlerre fut. */
    .macro lab_default_handler name
    .weak       \name
    .thumb_set  \name, Default_Handler
    .endm
    lab_default_handler NMI_Handler
    lab_default_handler HardFault_Handler
    lab_default_handler MemManage_Handler
    lab_default_handler BusFault_Handler
    lab_default_handler UsageFault_Handler
    lab_default_handler SVC_Handler
    lab_default_handler DebugMon_Handler
    lab_default_handler PendSV_Handler
    lab_default_handler SysTick_Handler
```

Nézzük a szerződéseket és a részleteket:

4 bájtos granularitás és a végcím. A ciklusok `ldr`/`str` szópárokkal dolgoznak, 4 bájtos lépésközzel. Ehhez nem kell, hogy minden C-objektum mérete 4 többszöröse legyen: a linker script a `.data` és a `.bss` kezdetét és végét is szóhatárra igazítja, a köztes réseket és a végét kitölti. Egy 1, 3 vagy 5 bájtos `uint8_t` objektum tehát belefér a szerződésbe; a firmware build `lab-main.c`-je pontosan ilyen objektumokkal ellenőrzi a bootot. Ha a végcím mégsem lenne szóhatár, azt a linker `ASSERT`-je már linkeléskor elutasítja.

Előjel nélküli, elöltesztelő ciklus. Mindkét ciklus a törzs **előtt** vizsgál (`cmp` + `bhs`), ezért üres szekciónál egyszer sem fut. A `bhs` (unsigned higher or same) címekre való összehasonlítás: a ciklus addig fut, amíg a mutató kisebb a végcímnél. Ez **nem** egyenlőségvizsgálat. Egy `cmp` + `bne` ciklus csak akkor állna meg, ha a mutató pontosan elérné a végcímet, és egy nem 4 többszörös hossznál átlépné azt. Az ellenpróbát lent mérjük.

A regiszterek megválasztása. Az `r0..r3` regisztereket használjuk munka-regiszterként. A startup kód itt maga a `Reset_Handler`, ahol még nincs értelmes programállapot, ezért a `main()` hívása előtt szabadon használhatjuk a caller-saved regisztereket. A `main()` hívásakor viszont már betartjuk az AAPCS-t: nem feltételezzük, hogy `r0..r3` érintetlen marad a visszatérés után — pont ezért nem használjuk őket a `main()` előtti és utáni állapot összekapcsolására.

Ellenpróba: mi történik szerződésszegésnél? A kanonikus párt a fő úton (GNU binutils + QEMU 8.2.2) futtatva a kimenet `boot ok`, a kilépési állapot 0 (a lektor mérése). A `.data`

másolásának kihagyásával `boot FAIL`, exit 1 (ugyancsak a fő úton mérve; a részletes indoklás a 16.7 3. megoldásában). A táblázat három szerződésszegő változatát a helyettesítő eszközlánccal (Clang + `ld.lld`, Unicorn-alapú Cortex-M3 emulátor) mértük ugyanabból a `lab-main.c`-ből, amelyben 1, 3 és 5 bájtos `.data`- és `.bss`-objektumok vannak:

Linker script	Startup ciklus	Eredmény
kanonikus (`ALIGN(4)` a végeken, `ASSERT`)	`cmp` + `bhs`	`boot ok` (a fő úton mérve)
a `.data` végéről kivett `ALIGN(4)`, `ASSERT` bekapcsolva	–	a linkelés megáll: `.data bounds not word aligned`
ugyanez, `ASSERT` nélkül (`_edata = 0x2000000d`)	`cmp` + `bhs`	`boot ok`, de a ciklus a `0x2000000d`–`0x2000000f` kitöltő bájtokat is írja; ez csak azért ártalmatlan, mert a `.bss` itt szóhatáron kezdődik
ugyanez, `ASSERT` nélkül	`cmp` + `bne` (gyakori hibás minta)	a mutató a `0x2000000c` → `0x20000010` lépéssel átlépi a `0x2000000d` végcímet, és a ciklus nem áll meg: a RAM nagy részét felülírja, majd az emulátorban a forrásmutató a kódterület végén túl olvasva fault-ot okoz

A tanulság: a helyes viselkedést nem egyetlen elem adja, hanem a szerződés két oldala együtt. Az igazítást a linker script garantálja és ellenőrzi, a ciklus pedig a tartományvégre (`bhs`), nem az egyenlőségre teszteli a mutatót.

A `.bss` nullázás sorrendje. A `.bss` nullázást a `.data` másolás **után** végezzük, mert a két szekció a RAM-ban egymást követi (a linker script így rendezi el őket), és bár a másolás nem érinti a `.bss`-t, a sorrend a szokásos konvenció. Ha `.bss` előbb lenne, sem lenne hiba — a két rutin egymástól független.

A nullázás tesztelése: őrértékkel, nem friss RAM-mal. Az emulátor frissen induló RAM-ja nullázott, ezért a hiányzó `.bss`-nullázás elbújhat: a teszt akkor is `boot ok`-ot mutat. A nullázás vizsgálatához a tesztelő előkészítés (debugger vagy teszt-stub, nem a végleges boot kód) a reset után, a nullázó rutin előtt tölti fel a `.bss`-t nemnulla őrértékkel (pl. `0xA5A5A5A5`), majd a `main()` előtt ellenőrzi az utóállapotot. Például GDB-vel: `break Reset_Handler`, `continue`, a `.bss` feltöltése az őrértékkel, majd `break main`, `continue`, `x/Nxw &_sbss` — mindegyik szó `0x00000000`. A nullázó ciklus kikommentezése így determinisztikusan FAIL (az őrérték megmarad), a visszaállítása PASS; külön próbálj nulla hosszú (`_sbss == _ebss`) és több szavas tartományt is.

16.6.6 Várt futási állapot

A `qemu-system-arm -machine mps2-an385 -kernel lab-firmware.elf -nographic` parancs futtatása után, közvetlenül a `main()` belépése előtt:

Regiszter	Érték (példa)	Jelentés
r0	_sidata + (_edata - _sdata)	a forrás-mutató a .data másolás után, a betöltési kép vége
r1	_ebss = _sbss + .bss méret	a .bss végére mutat
r2	_ebss	a ciklus felső határa
r3	0	a .bss nullázás után
SP	a linker script által definiált _estack	a verem teteje

Ellenőrzés: a `main()` első sora után állítsunk be egy töréspontot GDB-ben (`break main`), és olvassuk ki a `.data` és `.bss` tartalmát. A `.data` értékének meg kell egyeznie a flash-beli LMA-val; a `.bss` minden bájtja 0.

Az `mps2-an385` memóriatérképét a QEMU dokumentációja adja meg; a konkrét címtartományok (a flash-oldali régió és a RAM régió) QEMU-verziófüggők lehetnek, ezért ezeket a linker scriptben **kell** rögzíteni, és a build során a map fájllal ellenőrizni.

16.6.7 Tipikus hibák és ellenpróbák

1. A szimbólumok elgépelése. Ha a linker scriptben `_sdata_` vagy `_sData` szerepel, de az assembly `_sdata`-ra hivatkozik, a linker „undefined symbol” hibát ad. Tünet: linkelési hiba, nem futásidejű. Ellenpróba: `nm lab-firmware.elf | grep -E '_s?data'` — a szimbólumnak meg kell jelennie.

2. A `.data` vagy `.bss` vége nem szóhatár. Ha a linker script a záró `. = ALIGN(4);` nélkül definiálja az `_edata`/`_ebss` szimbólumot, a szavas ciklus a szekció végén túl is ír. A `bhs` ciklus legfeljebb 3 bájttal lépi túl a végcímet, ami a következő szekció elejét írhatja felül; egy `bne` ciklus meg sem áll. Tünet: a `.bss` első bájtjai felülíródnak, vagy a boot fault-tal elszáll. Ellenpróba: a kanonikus scriptben az `ASSERT` ezt linkeléskor jelzi; kézzel `nm -n lab-firmware.elf` — az `_edata` és az `_ebss` értéke legyen 4-gyel osztható.

3. A `.bss` nullázás kimarad. A C kód lefut, de egy „nullázatlan” globális véletlenszerű értéket tartalmaz. Az `mps2-an385` esetében a SRAM tartalma hardver-indításkor nem garantáltan 0 — ezért a C standard megköveteli, hogy a startup kód nullázza. Tünet: a `buffer` elemei nem 0-k, vagy a `counter` kezdőértéke nem az, amit a C kód deklarált. Ellenpróba: állítsuk be a `.bss` régióját előzetesen nem-nulla értékre (QEMU monitorból vagy egyéb debug interfészből), majd futtassuk — a startup után minden 0-nak kell lennie.

4. A `_sidata` és `_sdata` felcserélése. A copy ciklus a flash-ből RAM-ba másol, nem fordítva. Ha tévedésből `_sdata` a forrás és `_sidata` a cél, a ciklus a flash-beli betöltési képre próbál írni. A ciklus a végcímet a **cél** mutatóhoz hasonlítja (`cmp r1, r2`, ahol `r2 = _edata`), ezért a flash-kezdőcímről indulva nem áll meg a `.data` hosszánál, hanem a teljes kód- és vektortartományt felülírja. QEMU-ban ez BusFaulthoz, majd – mert a vektortábla is felülíródott – lockuphoz vezet (`qemu: fatal: Lockup: can't escalate 3 to HardFault`), 134-es kilépési állapottal. Nem csendes `boot FAIL` történik, hanem a firmware összeomlik.

5. A `main()` `bl` helyett `b`. A `b main` nem menti a visszatérési címet, és nem állítja be az `LR`-t. A `main` visszatérésekor a processzor egy véletlenszerű címre ugrik. Tünet: a `main` visszatérése utáni viselkedés kaotikus. Ellenpróba: `objdump -d lab-firmware.elf | grep -A20 '<Reset_Handler>' | grep main` — a `main` hívásnak `bl`-nek kell lennie.

16.6.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd el, miért nem oldható meg a `.data` inicializálás pusztán a linker scripttel, holott a linker ismeri a `.data` VMA-ját és LMA-ját is. Mi az a lépés, amit futásidőben kell megtenni, és miért nem „elég okos” ehhez a linker?

2. (implementációs, középszint — cél: rutinépítés) Módosítsd a startup rutint, hogy **1 bájtos granularitású** legyen (a `.data` és `.bss` méretének 4-gyel való oszthatósága nélkül is működjön). Gondoskodj róla, hogy a ciklusok előbb a maradék 1–3 bájtot másolják/nullázzák, majd az igazított 4 bájtos blokkokat. Ellenőrizd, hogy a rutin továbbra is helyes, ha a méret 0, vagy ha pontosan 4.

3. (hibakeresési/mérési, elmélyült — cél: fejezetek összekapcsolása: Memory-mapped I/O alapelve) A `qemu-system-arm -machine mps2-an385` célon futtasd a startup kódot úgy, hogy a `main()` helyett egy olyan függvény hívódjon, amely kiírja a `.data` és `.bss` tartalmát UART-on (a memory-mapped UART részleteit lásd Memory-mapped I/O alapelve). Készíts egy kis szkriptet, amely megváltoztatja a `_sidata`-t és a `_sdata`-t felcserélő verziót, és figyeld meg, mi történik a `bl main` után. Gyűjtsd össze, milyen regiszterértékek és kivétel-típusok jelennek meg.

16.6.9 Megoldások

1. megoldás — Segítség. A linker egy FORDÍTÁSI IDEJŰ eszköz — fut-e még, amikor a programnak ténylegesen másolnia kellene a bájtokat a flashből a RAM-ba?

1. megoldás — Teljes megoldás. A linker statikus időben dolgozik, és csak a címeket ismeri. A `.data` LMA-ja a flashben van, VMA-ja a RAM-ban. A flash tartalma a programozás után fix; a RAM tartalma futásidőben alakul. A linker nem tudja „átmásolni” a bájtokat, mert a program nem fut, amikor a linker dolgozik. A másolást egy futó processzornak kell elvégeznie — ez a startup kód feladata. Az ISA csak a címzést és az utasításokat adja; a programozó (vagy a C runtime implementációja) dönti el, hogy mit másol hova.

2. megoldás — Segítség. Gondolkodj két fázisban: előbb egy szavas (4 bájtos) ciklus addig fut, amíg még BELEFÉR egy teljes szó a maradék tartományba, utána egy bájtos ciklus takarítja el a maradékot — melyik feltétel dönti el, hogy a szavas ciklus mikor áll le?

2. megoldás — Teljes megoldás. A bájt-granularitású változat:

```
    /* .data másolás bájt-granularitással */
    ldr r0, =_sidata
    ldr r1, =_sdata
    ldr r2, =_edata
    /* először 4-gyel osztható méretig szó, majd maradék bájt */
    b   data_w_check
data_w_copy:
    ldr r3, [r0], #4
    str r3, [r1], #4
data_w_check:
    add r3, r1, #4
    cmp r3, r2
    bls data_w_copy            /* előjel nélkül: r1 + 4 <= r2 */
data_b_copy:
    cmp r1, r2
    bhs data_done
    ldrb r3, [r0], #1
    strb r3, [r1], #1
```

```
    b   data_b_copy
data_done:
```

A `.bss` nullázás hasonlóan: előbb `str` szavakkal amíg a mutató + 4 `_ebss`, majd `strb` a maradékra. Ez a megoldás akkor is helyes, ha a méret 0 (mindkét elsődleges ciklus üresen fut le), vagy pontosan 4 (a bájtos rész azonnal kilép). A lényeg: a feltételek előjel nélküliek (`bls`, `bhs`, címekre ez a helyes), és a bájtos ciklus is tartományvégre tesztel, így a mutató nem csúszhat túl a végen. A kanonikus scriptben erre nincs szükség, mert az `ALIGN(4)` és az `ASSERT` a szavas változatot is biztonságossá teszi; a bájtos farokkezelés akkor kell, ha a linker scriptet nem mi írjuk.

3. megoldás — Segítség. Ha a forrás- és céltartomány-pointer felcserélődik, a program egy RAM-címről MÁR helyesen olvas, de a tárolás célja megváltozik — mi van a flash-címtartomány írási jogosultságával?

3. megoldás — Teljes megoldás. A `_sidata`/`_sdata` felcserélése esetén a `ldr r3, [r0], #4` utasítás egy RAM-címről olvas (ez még működik), majd a `str r3, [r1], #4` a betöltési kép (`FLASH` nevű régió) címére ír. A kimenetel a céltároló tulajdonságán múlik, nem az utasításon. Valódi flash-es eszközön a flash-vezérlő az ilyen írást elutasítja vagy figyelmen kívül hagyja; az előbbi BusFault-ként, eszkalálva HardFault-ként jelenik meg. A QEMU `mps2-an385`-ön a `FLASH` régió írható SSRAM, ezért az írás nem tiltódik le; a ciklus a célmutatót `_edata`-hoz hasonlítva a 0x1d0-től indulva a teljes tartományt írja, felülírja a vektortáblát és a kódot, majd BusFault után lockup következik (`qemu: fatal: Lockup…`, exit 134). A `boot FAIL` azért nem jelenik meg, mert a kód már nem fut: a QEMU nem kivétel nélkül hajtja végre a hibás firmware-t, hanem BusFaultot, majd a felülírt vektortábla miatt lockupot jelez. A lockup-dump egyben a 3. feladatban kért regiszteradatokat is megadja (a lektor mérése, 16.6-01): `LR` = `0xffffffff` (EXC_RETURN: visszatérés Thread módba, MSP-vel, FP-keret nélkül), `XPSR` = `3`, azaz az IPSR a HardFault kivétel száma, valamint `R01` = `0x00400050` (a BusFaultot kiváltó leképezetlen cím) és `R15` = `0`. A mag tehát HardFaulton keresztül jutott a lockupig. Ennek ellenőrzésére GDB-vel érdemes a `str` utasítás után megállni és a RAM- és a betöltési kép tartalmát összevetni.

16.6.10 Rövid összefoglaló

A startup kód felelős a C runtime előfeltételeiért: a `.data` szekciót a flash-beli LMA-ból a RAM-beli VMA-ba másolja, a `.bss` szekciót nullázza, majd a `main()`-t hívja. A ciklushatárok a linker scriptből exportált `_sidata`, `_sdata`, `_edata`, `_sbss`, `_ebss` szimbólumok. A tipikus hibaforrások a szimbólum-elgépelés, a 4 bájtos igazítás megsértése, a `.bss` kihagyása, és a másolási irány fordítottja. A kanonikus pár (`lab-mps2.ld` + `lab-startup.s`) szerződése: szóhatárra igazított határok, amelyeket a linker `ASSERT`-tel ellenőriz, és elöltesztelő, előjel nélküli (`bhs`) ciklusok, amelyek üres szekciónál nem futnak, és a végcímen túl nem írnak.

Továbbhasznált definíciók:

- **startup kód:** assembly rutin, amely a `main()` előtt a C runtime előfeltételeit biztosítja (`.data` másolás, `.bss` nullázás).
- **LMA és VMA:** betöltési és futásidejű cím; a `.data` esetében a LMA flashben, a VMA RAM-ban van.
- **`.bss` szekció:** nulla-inicializált globális/statikus változók szekciója; tartalmát futásidőben nullázni kell.

16.6.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Armv7-M Architecture Reference Manual, DDI0403E — az M-profile futási modell és a memóriatérkép elsődleges forrása; az utasításkódolás (Thumb-2, `ldr/str` immediate offset) itt ellenőrizhető. **Ellenőrizve**: a link él (a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött).
- GNU Binutils `as` kézikönyv — a `.syntax unified`, `.cpu`, `.thumb` direktívák és a `ldr Rd, =symbol` pszeudo-utasítás pontos szemantikája. **Tartalmilag ellenőrizve**: a jelenlegi kiadás binutils 2.47.
- GNU Binutils `ld` kézikönyv — a linker script `SECTIONS`, `LOADADDR`, `AT>` szintaxis, valamint az exportált szimbólumok viselkedése. Ugyanaz a binutils 2.47-es kiadás.
- QEMU System Emulation dokumentáció — az `mps2-an385` machine memóriatérképe és a betöltési konvenciók (`-kernel` opció). **Tartalmilag ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: a kanonikus párt és a `.data` másolásának kihagyását a fő úton (GNU binutils + QEMU 8.2.2) mérte a lektor (`boot ok`, exit 0, illetve `boot FAIL`, exit 1); az ellenpróba-táblázat három szerződésszegő sorát viszont csak a helyettesítő eszközlánccal (Clang + `ld.lld`) és a Unicorn-alapú `tools/cm-run.py` Cortex-M3 emulátorral futtattuk. A fennmaradó tételek: az `mps2-an385` pontos flash-/RAM-címtartománya és a `-kernel` alapértelmezett betöltési címe az aktuális QEMU-verziótól függ; ezt a hallgatónak a saját laborkörnyezetében kell a linker script és a map fájl alapján rögzítenie. A GNU toolchain csomagolt verziója (Ubuntu 24.04) a reprodukálható baseline, de a konkrét `ld`- és `as`-kiadás viselkedése apró részletekben eltérhet — érdemes a `nm`, `readelf -S`, `objdump -d` kimenetét a build során rendszeresen ellenőrizni.

16.7 Firmware build és QEMU-betöltés munkafolyamat

16.7.1 Motiváció

A startup kód és a linker script elkészültével a firmware még mindig csak forrásfájlok és egy leírás halmaza. Ahhoz, hogy a kód ténylegesen fusson, végig kell menni egy olyan láncon, amely a C- és assembly-forrásokból gépi kódot, abból egy betölthető image-et, majd abból egy futó rendszert csinál. Bare-metal környezetben ezt a láncot nem egy operációs rendszer és nem egy disztribúciós csomagkezelő végzi el helyettünk: minden lépést kézzel, explicit módon kell megadni. Ez az alfejezet a teljes `gcc-arm-none-eabi` + `qemu-system-arm` munkafolyamatot építi fel, a fordítási kapcsolóktól a `Makefile`-ig és a GDB-vel összekötött debug-indításig. A cél, hogy egyetlen paranccsal reprodukálható legyen a firmware építése és futtatása, és hogy a hallgató pontosan lássa, hol keletkezik a flash-kép, mit tölt be a QEMU, és hogyan lehet megállítani a processzort a reset után.

16.7.2 Definíció és működés

Definíció (bare-metal build-folyamat): olyan eszközlánc-munkafolyamat, amely forrásfájlokból (`*.c`, `*.s`) tárgykódot, abból linkeléssel egy `ELF` futtatható fájlt, abból pedig `objcopy`-jal egy nyers bináris flash-képet állít elő, majd ezt egy célplatformra (itt QEMU `mps2-an385`) betölti és elindítja. A folyamat minden lépése determinisztikus és parancssorból vezérelhető.

A cross-toolchain neve `arm-none-eabi-`: `arm` a célarchitektúra, `none` a hiányzó operációs rendszer, `eabi` az Application Binary Interface. A `host` x86_64-es Ubuntu 24.04 LTS-en ez a csomag `gcc-arm-none-eabi` néven érhető el. A használt kapcsolók szerepét a következő táblázat foglalja össze.

Kapcsoló	Szerep	Megjegyzés
`-mcpu=cortex-m3`	cél CPU-profil	Thumb-2 utasításkészlet, nincs NEON/VFP
`-mthumb`	Thumb-2 kódgenerálás	Armv7-M-en kötelező
`-ffreestanding`	a fordító nem tételez fel hosted C-környezetet	a `main` nem különleges; a libc kizárása a `-nostdlib` feladata
`-nostdlib`	libc és startup fájlok kihagyása	saját `Reset_Handler` és `main` van
`-nostartfiles`	csak a startup fájlok kihagyása	a `-nostdlib` ezt is tartalmazza
`-T lab-mps2.ld`	linker script	memóriatérkép és szekciók
`-Wl,-Map=...`	linker map	szimbólumok és szekcióméretek
`-O0` / `-Os`	optimalizálás	debug vs. méret

A linkelés bemenete a linker script (16.5), amely megadja a `FLASH` és `RAM` tartományokat, valamint elhelyezi a `.text`, `.data` és `.bss` szekciókat. A `-ffreestanding` önmagában nem zárja ki a libc-t; a GCC freestanding módban is generálhat `memcpy`/`memset` hívást (nagy struktúra-másolásnál), ilyenkor ezeket biztosítani kell. A `-nostdlib` következménye, hogy bizonyos fordító által generált segédfüggvények (például 64 bites osztás, lebegőpontos műveletek) nem lesznek automatikusan elérhetők; ilyenkor a `libgcc`-t explicit módon, `-lgcc` formában hozzá kell linkelni. A mostani példa nem igényel ilyet, de a kapcsoló ismerete később fontos.

A linkelés eredménye egy `ELF` fájl. Ez tartalmazza a szekciókat, a szimbólumtáblát és a program headereket. A QEMU `-kernel` kapcsolója Armv7-M machine esetén a `armv7m_load_kernel` útvonalon tölti be a képet: `.elf` bemenetnél a program headerek fizikai (betöltési, `LMA`) címeit használja, `.bin` bemenetnél a fájlt egyszerűen a `0x00000000`-tól másolja be. Mivel a `.data` `LMA`-ja a flashben, `VMA`-ja a RAM-ban van, mindkét út **ugyanazt** a memóriatartalmat adja: a `.data` kezdőértékei a betöltési képben vannak, és a startup másolja át őket. Ebben az alfejezetben a futtatás és a debug **ugyanazt** az `.elf`-et tölti be, így a kettő pontosan ugyanazt a képet látja; az `.elf` a GDB-nek a szimbólumokat is adja. A `.bin` a valódi eszköz flash-programozójának szóló termék, és QEMU-ban is futtatható.

A flash-kép előállítása `objcopy`-jal történik:

```
arm-none-eabi-objcopy -O binary lab-firmware.elf lab-firmware.bin
```

A `-O binary` a `LOAD` típusú szekciókat folytonos binárisba írja, a `VMA`-k helyett az `LMA`-k sorrendjében. Fontos, hogy a bináris első bájtjai a vektortábla első szavai legyenek: a kezdeti stack pointer és a `Reset_Handler` címe (`Thumb` módban a legalacsonyabb bit 1).

A QEMU indítási minta:

```
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
    -kernel lab-firmware.elf -semihosting -nographic
```

Három futtatási minta. A laborokban a firmware jellegétől függően három parancsminta használható; a stdin kezelése mindegyikben tudatos:

```
# 1) Semihostinggal kilépő firmware: a bemenet nem kell, ezért /dev/null.
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -semihosting \
    -kernel lab-firmware.elf -nographic </dev/null

# 2) UART-os, nem kilépő firmware: a vendég UART0-ja kapja a stdin/stdout-ot.
timeout 5 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
    -display none -monitor none -serial stdio -kernel lab-uart.elf

# 3) GDB-s indítás: a monitor és a soros port ne ossza meg a terminált.
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -S -gdb tcp::3333 \
    -display none -monitor none -serial null -kernel lab-firmware.elf
```

A `-nographic` a terminál bemenetét a QEMU-monitorral multiplexeli, ezért szkriptből vagy egymás utáni parancsoknál elnyelheti a következő parancs bemenetét. UART-os labornál ezért a `-display none -monitor none -serial stdio` a megfelelő minta; semihostingos, magától kilépő firmware-nél a `</dev/null` zárja le a bemenetet. Interaktív terminálon a `Ctrl-A`, majd `X` lép ki a QEMU-ból.

A `-machine mps2-an385` az ARM MPS2+ AN385 referenciakártyát modellezi, amelynek dokumentált memóriatérképén a flash `0x00000000`-tól, a SRAM `0x20000000`-tól indul. A pontos tartományhosszakat és a perifériacímeket a QEMU adott verziójának machine-leírása rögzíti; ezeket a linker scripttel össze kell hangolni. A `-machine help` csak azt igazolja, hogy a `mps2-an385` gép létezik; a tényleges memóriatérképet a QEMU monitor `info mtree` kimenetével (indítás `-S -monitor stdio -display none` mellett, majd `info mtree`: a `0x00000000` és `0x20000000` régió címe/mérete látszik; az `alias`-ok nem új fizikai tárat jelentenek) és a rögzített QEMU-verzió `hw/arm/mps2.c` forrásával kell ellenőrizni. A `-semihosting` engedélyezi az ARM semihosting debug-csatornát, amelyen a firmware a host konzoljára írhat — ez itt az „első kimenet” ellenőrzésének eszköze, nem valódi periféria. A `-nographic` a soros konzolt a standard bemenetre/kimenetre köti. A példa firmware a kiírás után a semihosting `SYS_EXIT` hívásával maga állítja le a QEMU-t (kilépési kód 0 = `boot ok`, 1 = `boot FAIL`); ha egy firmware végtelen ciklusban marad, kilépni `Ctrl-A`, majd `X` kombinációval lehet.

A debug-indítás ugyanezt a képet, **ugyanazzal a semihosting-beállítással** állítja meg a reset után. A `-semihosting` itt is kötelező: a firmware `bkpt 0xab`-ot hajt végre, és semihosting nélkül a QEMU ezt nem kiírásként kezeli, hanem debugkivételként, így a `boot ok` megfigyelés debug alatt elveszne:

```
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
    -kernel lab-firmware.elf -semihosting -S -gdb tcp::3333 -nographic
```

A `-S` hatására a CPU nem hajt végre utasítást, amíg a debugger nem engedi el. A `-gdb tcp::3333` egy GDB szervert nyit a 3333-as TCP-porton. A csatlakozás `gdb-multiarch`-csal történik, mert a host x86-os GDB nem feltétlenül tartalmaz ARM-támogatást:

```
gdb-multiarch lab-firmware.elf \
    -ex "target remote :3333" \
    -ex "break main" \
    -ex "continue"
```

A debug session alatt a `monitor info registers`, `info registers sp pc`, `x/4xw 0x20000000` parancsokkal ellenőrizhető a reset utáni állapot, a `main` elérése és a `.bss` nullázottsága. Egy további `continue` után a QEMU-t futtató terminálban ugyanúgy megjelenik a `boot ok`, mint a `run` célnál, és a QEMU a `SYS_EXIT` hívásnál kilép.

16.7.3 Kidolgozott példa

Munkakönyvtár: `~/arm-labor/cm/a067/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a067
cd ~/arm-labor/cm/a067
```

A példa három saját fájlból és két átvett fájlból áll. A linker script és a startup kód a II. rész kanonikus párja, változatlanul: a `lab-mps2.ld` a Linker script alapjai, a `lab-startup.s` a Startup kód alfejezetből jön (vektortábla: `.vectors`, belépési pont: `Reset_Handler`, szóhatárra igazított és `ASSERT`-tel ellenőrzött `.data`/`.bss` határok).

```
cp ~/arm-labor/cm/a065/lab-mps2.ld ~/arm-labor/cm/a066/lab-startup.s .
```

A `main()` semihosting-kiírással igazolja a `.data` másolását és a `.bss` nullázását, majd a `SYS_EXIT` hívással leállítja a QEMU-t. Az ellenőrzött objektumok között szándékosan vannak 1, 3 és 5 bájtosak is, hogy a `.data` és a `.bss` tartalma ne legyen eleve 4 bájt többszöröse:

```
/* lab-main.c - boot-ellenőrzés semihostinggal */
#include <stdint.h>

volatile uint32_t data_counter = 0x11223344u;   /* .data */
volatile uint32_t bss_counter;                  /* .bss  */

/* Ellenpróba: 1, 3 és 5 bájtos objektumok, hogy a .data és a .bss
   tartalma ne legyen eleve 4 bájt többszöröse. */
volatile uint8_t d1    = 0xA1;
volatile uint8_t d3[3] = { 0xB1, 0xB2, 0xB3 };
volatile uint8_t d5[5] = { 0xC1, 0xC2, 0xC3, 0xC4, 0xC5 };
volatile uint8_t z1, z3[3], z5[5];

static void sh_call(uint32_t op, const void *arg)
{
    register uint32_t r0 asm("r0") = op;
    register const void *r1 asm("r1") = arg;
    asm volatile ("bkpt 0xab" : "+r"(r0) : "r"(r1) : "memory");
}

static int boot_ok(void)
{
    if (data_counter != 0x11223344u || bss_counter != 0u) return 0;
    if (d1 != 0xA1 || z1 != 0) return 0;
    for (int i = 0; i < 3; i++) if (d3[i] != 0xB1 + i || z3[i] != 0) return 0;
    for (int i = 0; i < 5; i++) if (d5[i] != 0xC1 + i || z5[i] != 0) return 0;
    return 1;
}

int main(void)
{
    int ok = boot_ok();
    sh_call(0x04u, ok ? "boot ok\n" : "boot FAIL\n");      /* SYS_WRITE0 */
    sh_call(0x18u, (const void *)(ok ? 0x20026u : 0x20023u));
    /* SYS_EXIT: 0x20026 = ADP_Stopped_ApplicationExit (kilépési kód 0),
       0x20023 = ADP_Stopped_RunTimeErrorUnknown (QEMU: kilépési kód 1). */
    for (;;) { }
}
```

```
# lab-Makefile
CROSS   := arm-none-eabi-
CC      := $(CROSS)gcc
OBJCOPY := $(CROSS)objcopy
```

```
SIZE    := $(CROSS)size
OBJDUMP := $(CROSS)objdump

CPU     := -mcpu=cortex-m3 -mthumb
CFLAGS  := $(CPU) -ffreestanding -nostdlib -nostartfiles -O0 -g3 -Wall
LDFLAGS := $(CPU) -nostdlib -T lab-mps2.ld -Wl,-Map=lab-firmware.map -Wl,--gc-sections
QEMU    := qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -semihosting -nographic

OBJS := lab-startup.o lab-main.o

all: lab-firmware.elf lab-firmware.bin

lab-startup.o: lab-startup.s
        $(CC) $(CFLAGS) -c $< -o $@

lab-main.o: lab-main.c
        $(CC) $(CFLAGS) -c $< -o $@

lab-firmware.elf: $(OBJS) lab-mps2.ld
        $(CC) $(LDFLAGS) $(OBJS) -o $@

lab-firmware.bin: lab-firmware.elf
        $(OBJCOPY) -O binary $< $@

# run és debug ugyanazt a képet és ugyanazt a semihosting-beállítást használja
run: lab-firmware.elf
        $(QEMU) -kernel lab-firmware.elf </dev/null

debug: lab-firmware.elf
        $(QEMU) -kernel lab-firmware.elf -S -gdb tcp::3333 </dev/null

size: lab-firmware.elf
        $(SIZE) -A $<          # szekciónkénti pontos méret (a sima size .text-be vonja a
        ↪ rodata-t)

disasm: lab-firmware.elf
        $(OBJDUMP) -d $< > lab-firmware.dis

clean:
        rm -f lab-startup.o lab-main.o lab-firmware.elf lab-firmware.bin \
              lab-firmware.map lab-firmware.dis
```

A build és futtatás:

```
make -f lab-Makefile
make -f lab-Makefile run; echo "kilépési állapot: $?"
```

Kimenet a fő úton (GNU binutils + QEMU 8.2.2) mérve (a lektor mérése): a `make -f lab-Makefile` hiba és figyelmeztetés nélkül lefut, a `run` cél a QEMU `SYS_EXIT`-jénél `boot ok`-ot ír, és 0-val lép ki. Ugyanezt a kimenetet a helyettesítő eszközlánc (Clang + `ld.lld`, Unicorn-alapú `tools/cm-run.py`) is reprodukálja:

```
boot ok
kilépési állapot: 0
```

A vektortábla-ellenőrző program a kanonikus firmware-en minden sorra `OK` (a lektor mérése).

Ha a `.data` másolás vagy a `.bss` nullázás hibás, a kimenet `boot FAIL` lesz, és a kilépési állapot 1; ez a beépített ellenőrzési feltétel. A `debug` cél ugyanezt a képet ugyanazzal a semihosting-beállítással indítja. GDB-vel a `break main` után `info registers sp pc` mutatja, hogy a stack pointer az `_estack` közelében van (a `main` prológusa már foglalhatott vermet), a PC pedig a `main` címén áll. Egy további `continue` után ugyanaz a `boot ok` jelenik meg, mint a `run` célnál.

16.7.4 Tipikus hibák és ellenpróbák

- **A futtatott és a debugolt kép eltér:** például a `run` egy régi `.bin`-t tölt, a GDB pedig az új `.elf` szimbólumaival dolgozik, így a töréspontok rossz címre kerülnek. Ellenpróba: a `run` és a `debug` cél ugyanazt az `.elf`-et töltse; ha `.bin`-t is használunk, a Makefile függőségként mindig az aktuális `.elf`-ből állítsa elő.
- **Hiányzó `-nostdlib`, vagy fordítva, hiányzó `-lgcc`:** az első esetben a linker libc-szimbólumokat keres; a második ritkább, de 64 bites műveleteknél `undefined reference to __aeabi_ldivmod` tünetként jelentkezik.
- **A vektortábla nem kerül a flash elejére:** a `KEEP(*(.vectors))` hiányzik, vagy a `--gc-sections` kigyomlálja. Tünet: a QEMU a reset után azonnal faultol, vagy a PC 0 körül marad. Ellenpróba: `arm-none-eabi-objdump -s -j .vectors lab-firmware.elf`, illetve a vektortábla-ellenőrző program. A kanonikus `lab-mps2.ld` `ASSERT`-je a rossz helyre került táblát már linkeléskor jelzi.
- **A semihosting nincs engedélyezve (gyakran csak a debug-parancsban):** a `bkpt 0xab` debugkivételt vagy HardFaultot okoz, vagy a QEMU csendben marad. Ellenpróba: a `-semihosting` a `run` és a `debug` parancsban is szerepeljen (a fenti Makefile ezért közös `QEMU` változót használ), illetve GDB-vel a PC és az `xPSR` vizsgálata a `bkpt` után.
- **A GDB a rossz architektúrával csatlakozik:** `gdb` helyett `gdb-multiarch` (vagy `arm-none-eabi-gdb`), különben a `target remote` nem ismeri fel a cél architektúrát.

16.7.5 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd el, miért ad a QEMU `-kernel` kapcsolója ugyanazt a memóriatartalmat `.elf` és `.bin` bemenettel, és miért használja ez az alfejezet mégis az `.elf`-et a futtatáshoz és a debughoz is. Térj ki az `LMA/VMA` különbségre és a szimbólumtábla szerepére. **2. (implementációs, középszint — cél: rutinépítés)** Egészítsd ki a `lab-Makefile`-t egy `map` céllal, amely a `lab-firmware.map`-ből kiírja a `.data`, `.bss` és `.text` szekciók kezdő- és végcímét, valamint egy `verify` céllal, amely ellenőrzi, hogy a `lab-firmware.bin` létezik és nem üres. 3. **(hibakeresési, középszint — cél: fejezetek összekapcsolása: Startup kód: .data másolása, .bss törlése)** Távolítsd el a startup kódból a `.data` másoló ciklust, építsd újra a firmware-t, és GDB-vel (`break main`, majd `x/1xw &data_counter`) állapítsd meg, milyen értéket vesz fel a `data_counter`. Hasonlítsd össze a `boot FAIL` tünettel, és indokold, miért nem a linker script a hiba forrása. **4. (mérési, alapszint — cél: rutinépítés)** Mérd meg a `make -f lab-Makefile size` kimenetét (a szekciónkénti `size -A` formát; alternatíva: `readelf -SW`) `-O0` és `-Os` mellett. Rögzítsd a `.text`, `.rodata`, `.data` és `.bss` méreteket külön, a kezdő- és végcímet a linkerszimbólumokból (`_sdata`, `_edata`, `_sbss`, `_ebss`) vagy ellenőrzött start+size számításból, a `.data` esetén VMA és LMA megkülönböztetésével (a sima `size` a `.text` oszlopba vonja a csak olvasható adatot, ezért egy új `.rodata` szekció nem változtathatja meg a külön jelentett `.text` méretet), és indokold, melyik szekció mérete változik és miért. Az értékeket jelöld mértnek, és tedd hozzá, hogy QEMU-emulációban a méret nem függ az emulációtól, csak a fordító kimenetétől.

16.7.6 Megoldások

1. megoldás — Segítség. A QEMU `-kernel` a betöltendő TARTALMAT konkrét fizikai címekre tölti — melyik címet használja az ELF-ből, és mit tartalmaz a `.bin` a `0x00000000`-tól? A GDB viszont forráskódhoz és nevekhez köti a töréspontokat — melyik fájlformátum hordozza ezt?

1. megoldás — Teljes megoldás. A `.bin` a flash valós tartalmát tükrözi: az `objcopy -O binary` az `LMA`-k sorrendjében ír, így a `.data` inicializáló értékei ott vannak, ahol a valós eszköz flashje tartalmazná őket. Az `.elf` program headerei ugyanezeket a betöltési (fizikai) címeket hordozzák, és a QEMU ezekre tölti a szegmenseket, tehát a memóriatartalom ugyanaz. Ezért mindkét út helyes; a startup mindkét esetben a betöltési képből másol a RAM-ba. Az `.elf` emellett szimbólumtáblát és debuginformációt is tartalmaz, amire a GDB-nek szüksége van. Ha a futtatás és a debug ugyanazt az `.elf`-et tölti, kizárt, hogy a kettő két különböző build képét lássa; a `.bin` a valódi eszköz programozójának szól.

2. megoldás — Segítség. A linkermap soraiban a szekciónevek (`.text`, `.data`, `.bss`) sortörés elején jelennek meg — milyen egyszerű szöveges eszközzel szűrhetők ki ezek a sorok?

2. megoldás — Teljes megoldás. A `map` cél például `grep -E '^\.(text|data|bss)' lab-firmware.map` formában listázza a szekciókat; a `verify` cél `test -s lab-firmware.bin` feltétellel ellenőriz. A szekcióhatárokat a linkermap adja, így a hallgató összevetheti a linker scripttel.

3. megoldás — Segítség. Az 16.6 tárgyalja, mi történik, ha a `.data` másoló ciklus kimarad — a RAM-beli változó ekkor a FLASH-beli kezdőértékét kapja-e meg, vagy valami mást?

3. megoldás — Teljes megoldás. A `.data` másolás elhagyásával a `data_counter` a RAM-ban marad, értéke nem determinisztikus (jellemzően 0, mert a QEMU induláskor nullázza a SRAM-ot, de erre nem szabad építeni). A `boot_ok()` feltétele hamis lesz, a kimenet `boot FAIL`, a kilépési állapot 1. A linker script nem hibás: a `_sidata`/`_sdata`/`_edata` szimbólumok továbbra is definiáltak, csak a startup nem használja őket.

4. megoldás — Segítség. Az `-O0` és `-Os` elsősorban a KÓDOT (utasítás-összevonás, méretoptimalizálás) érinti — ez melyik szekció méretét befolyásolja, és melyiket nem, mert azt a változók száma/típusa határozza meg, nem a fordítói szint?

4. megoldás — Teljes megoldás. `-O0`-nál a `.text` nagyobb, mert nincs utasítás-összevonás; `-Os`-nél kisebb. A `.data` és `.bss` mérete jellemzően nem változik, mert azokat a változók száma és típusa határozza meg. Az értékeket a `size` fejlécében a `text`, `data`, `bss`, `dec` oszlopok adják.

16.7.7 Rövid összefoglaló

A bare-metal build-folyamat forrásból `ELF`-en át nyers flash-képig vezet: a `gcc-arm-none-eabi -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib` kapcsolókkal állítja elő a tárgykódot, a linker script a memóriatérkép szerint helyezi el a szekciókat, az `objcopy -O binary` pedig a betölthető image-et adja. A QEMU `-kernel` az ELF szegmenseit a betöltési címükre tölti (`.bin`-t a `0x00000000`-ra), a `-S -gdb tcp::3333` pedig a reset utáni megállást és GDB-csatlakozást teszi lehetővé. A `run` és a `debug` ugyanazt a képet és ugyanazt a semihosting-beállítást használja, így ugyanaz a `boot ok` megfigyelés mindkettőben elérhető. A `Makefile` ezt a láncot egyetlen paranccsá szervezi.

Továbbhasznált definíciók:

- **bare-metal build-folyamat:** forrás → tárgykód → `ELF` → `objcopy` bináris → QEMU-betöltés.
- **flash-kép (bináris image):** az `LMA`-k sorrendjében előállított nyers gépi kód, amely a valós flash-tartalmat reprezentálja.
- **semihosting:** ARM debug-csatorna (`bkpt 0xab`), amellyel a firmware a host konzoljára írhat; nem valódi periféria.

16.7.8 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblá-ját):

- Armv7-M Architecture Reference Manual, DDI0403E — az M-profile futási modell, a vektortábla és a `BKPT` utasítás elsődleges forrása.
- GNU Binutils `objcopy` kézikönyv — a `-O binary` és a szekciók `LMA` szerinti kiírásának leírása.
- GNU GCC kézikönyv — a `-mcpu`, `-mthumb`, `-ffreestanding`, `-nostdlib` kapcsolók jelentése.
- QEMU dokumentáció — a `-kernel`, `-S`, `-gdb`, `-semihosting` és az `mps2-an385` machine viselkedése.
- ARM semihosting specifikáció — a `SYS_WRITE0` (0x04) műveletkód és a `BKPT 0xAB` konvenció.

Ellenőrzési hiányok: a QEMU `mps2-an385` pontos flash/SRAM címtartománya, a `-semihosting` kapcsoló támogatottsága az adott QEMU-verzióban, valamint az `arm-none-eabi-gcc` Ubuntu 24.04-es csomagverziója a laborban ellenőrizendő. A `make -f lab-Makefile` hibátlan lefutását, a `boot ok` kimenetet és a 0-s kilépési állapotot, valamint a vektortábla-ellenőrző minden sorára `OK` eredményt a fő úton (GNU binutils + QEMU 8.2.2) mérte a lektor; a GDB-lépések ebben a munkafolyamatban nem futottak, a hallgató laborja validálja őket.

17 Kivételek és megszakítások: NVIC

17.1 A Cortex-M kivételmodell: prioritás és vektortábla-bejegyzések

17.1.1 Motiváció: mi dönti el, melyik kezelő fut le?

Egy tipikus Cortex-M firmware-ben a fő ciklus szenzoradatot dolgoz fel, miközben egy UART-vevő megszakításkérés (interrupt request, IRQ) érkezik, egy timer lejár, és egy hibás memóriaelérés miatt buszhiba (BusFault) keletkezik. A processzornak el kell döntenie, melyik eseményt szolgálja ki először, melyik szoríthatja háttérbe a többit, és mi történik a megszakított kód regisztereivel. Ez a döntés nem a C kódban, hanem a Cortex-M kivételmodelljében (exception model) születik meg: a vektortábla (vector table) rögzíti, hova kell ugrani, a prioritásrendszer (priority) pedig azt, hogy egyszerre érkező események közül melyik nyer. A következő alfejezet ezt a két mechanizmust rendszerezi. Az itt tanultakra épül a következő alfejezet NVIC-konfigurációja, és erre támaszkodik az ISR-írás is.

17.1.2 Definíció: kivétel, kivételtípusok, vektortábla-bejegyzés

A Cortex-M szóhasználatában a **kivétel (exception)** minden olyan esemény, amely a processzort arra készteti, hogy a normál utasításfolyamot megszakítsa, és egy előre kijelölt kezelőrutinba (handler) ugorjon. A reset is kivétel: az első utasítás végrehajtása előtt a mag egy kezelőbe kerül. Az Armv7-M architektúra a kivételeket két nagy csoportra osztja:

- **Rendszerkivételek (system exceptions):** számuk és jelentésük fix. Ide tartozik a Reset, az NMI, a HardFault, a MemManage, a BusFault, a UsageFault, az SVCall, a Debug Monitor, a PendSV és a SysTick.
- **Külső megszakítások (external interrupts, IRQ):** számuk implementációfüggő; az NVIC-hez kapcsolódó perifériák generálják őket. A Cortex-M3 például legfeljebb 240 IRQ-t támogat.

A **kivételszám (exception number)** egy egész szám, amely az egyes kivételeket azonosítja. Az 1-es a Reset, a 2-es az NMI, a 3-as a HardFault, a 4–6 a MemManage/BusFault/UsageFault, a 11 az SVCall, a 14 a PendSV, a 15 a SysTick. A 16-tól kezdődnek az IRQ-k: a 16-os az IRQ0, a 17-es az IRQ1, és így tovább. A 0-s kivételszám nem kivételhez tartozik: ez a vektortábla 0. szava, a kezdeti veremmutató.

A **vektortábla (vector table)** a címtér elején elhelyezkedő, 4 bájtos bejegyzésekből álló tömb. A 0. bejegyzés a kezdeti veremmutató (initial stack pointer), az 1. bejegyzés a Reset-hez tartozó kezelő címe, a 2. az NMI-é, és így tovább. A vektortábla **indexe** és a kivételszám közötti kapcsolat egyszerű:

$$\text{vektortábla index} = \text{kivételszám}$$

Ez azért igaz, mert a 0. index foglalja el a veremmutatót, és az 1. indextől kezdődnek a kivételvektorok. Az IRQ0 tehát a 16. indexű bejegyzés, az IRQn a $(16 + n)$-edik. Ez a látszólag

apró eltolás a gyakori hibák egyik fő forrása: ha valaki az IRQ0-t a 0. indexre teszi, akkor a kezdeti veremmutatót írja felül, és a rendszer már a reset után hibás állapotba kerül.

A vektortábla első 16 bejegyzésének fix sorrendjét az előző alfejezet már bemutatta. Itt annyi a kiegészítés, hogy a tábla nem áll meg a 16. bejegyzésnél: az IRQ-k számára fenntartott bejegyzések a 16. indextől kezdődnek, és addig tartanak, ameddig a konkrét implementáció IRQ-t támogat. Egy adott IRQ-hoz tartozó kezelő címét tehát a (16 + IRQ-szám) indexű szó tartalmazza.

A kivételtípusok és vektortábla-pozícióik összefoglalása:

Kivételszám	Név	Vektortábla index	Prioritás típusa
1	Reset	1	fix, -3
2	NMI	2	fix, -2
3	HardFault	3	fix, -1
4	MemManage	4	konfigurálható
5	BusFault	5	konfigurálható
6	UsageFault	6	konfigurálható
7–10	fenntartott	7–10	—
11	SVCall	11	konfigurálható
12	Debug Monitor	12	konfigurálható
13	fenntartott	13	—
14	PendSV	14	konfigurálható
15	SysTick	15	konfigurálható
16 + n	IRQn	16 + n	konfigurálható

A táblázatban a „fenntartott” (reserved) bejegyzések helyére a linkerscriptnek is bejegyzést kell tennie, különben a tábla elcsúszik. A fenntartott helyeket ebben a könyvben **nullával** töltjük ki (a vektortábla-ellenőrző ezt várja), míg a megvalósított, de nem használt kivételek és IRQ-k bejegyzése `Default_Handler` címe legyen; a kettőt nem szabad összekeverni.

17.1.3 Prioritás: a számozás fordított viszonya

A Cortex-M prioritásrendszerének legfontosabb szabálya: **a kisebb prioritásszám magasabb prioritást jelent**. A Reset prioritása -3, az NMI-é -2, a HardFaulté -1. Ezek fix, negatív értékek; nem konfigurálhatók, és minden konfigurálható kivételnél magasabb rendűek. A konfigurálható kivételek prioritásszáma 0 és egy felső határ között mozog. A 0 a legmagasabb konfigurálható prioritás.

A prioritásszám nem feltétlenül használja ki a teljes 8 bites tartományt. Hogy hány bitet valósít meg a mag, az **implementációs választás**: az Armv7-M 3 és 8 bit között engedi meg, és a Cortex-M3-at beépítő gyártó dönti el (sok mikrokontrolleren 3 vagy 4 bit). Ez tehát nem általános Cortex-M3-tulajdonság, hanem a konkrét **futtatókörnyezet** paramétere. CMSIS-ben a `__NVIC_PRIO_BITS`, ebben a könyvben a `LAB_NVIC_PRIO_BITS` laborparaméter rögzíti, és a NVIC-konfiguráció visszaolvasásos próbája méri meg. A könyv két futtatókörnyezete eltér. A QEMU stable-8.2 `mps2-an385` NVIC-modellje Armv7-M-en **8** bitet valósít meg (ott a `0xFF` visszaolvasva `0xFF`); ezt a QEMU forrásából tudjuk, a laborban a visszaolvasás igazolja. A könyv számpéldái és a `tools/cm-run.py` alapértelmezése egy **3** bites modellt követnek, amely sok mikrokontrolleren valós érték. Az alábbi táblázat mindkettőt mutatja. Négy dolgot kell külön kezelni:

Fogalom	Mi ez	Példa (`LAB_NVIC_PRIO_BITS = 3`, 3 bites modell)	Példa (`LAB_NVIC_PRIO_BITS = 8`, QEMU 8.2 `mps2-an385`)
nyers prioritásbájt	a regiszterben tárolt 8 bites érték	`0xA0`	`0xA0`
megvalósított bitek	a bájt felső `LAB_NVIC_PRIO_BITS` bitje; a többi olvasáskor 0, írása hatástalan	a 7:5 bitek; `0xFF` visszaolvasva `0xE0`	mind a 8 bit; `0xFF` visszaolvasva `0xFF`
logikai prioritás	a megvalósított bitek értéke: `nyers >> (8 - LAB_NVIC_PRIO_BITS)`	`0xA0 >> 5 = 5`	`0xA0 >> 0 = 160`
csoport- és alprioritás	az `AIRCR.PRIGROUP` (0–7) kettévágja a bájtot: a `[7 : PRIGROUP+1]` bitek a csoportprioritás, a `[PRIGROUP : 0]` bitek az alprioritás (mindkettő a megvalósított bitekből)	`PRIGROUP = 4` (vagy kisebb): a 3 bit mind csoportprioritás, csoport = 5, alprioritás = 0; `PRIGROUP = 5`: csoport = `0xA0 >> 6 = 2`, alprioritás = `(0xA0 >> 5) & 1 = 1`	`PRIGROUP = 4`: csoport = `0xA0 >> 5 = 5`, alprioritás = `0xA0 & 0x1F = 0`; `PRIGROUP = 5`: csoport = `0xA0 >> 6 = 2`, alprioritás = `0xA0 & 0x3F = 32`
legalacsonyabb konfigurálható prioritás	a visszaolvasott maszk (`0xFF` írása után olvasott érték)	`0xE0`	`0xFF`

A csoportprioritás tehát a két esetben azonos, mert ugyanazokból a felső bitekből jön; az alprioritás és a logikai érték viszont a megvalósított bitek számától függ. A sorrend (melyik kivétel előbb) mindkét esetben ugyanaz, amíg a nyers bájtok csak megvalósított biteket használnak.

A `0xA0` tehát **csak ismert bitszám mellett** jelent „5”-öt: 3 megvalósított bitnél a felső három bit `101`. Ha ugyanaz a mag 8 bitet valósítana meg, a `0xA0` logikai prioritása 160 lenne, 4 bitnél pedig `0xA0 >> 4 = 10`. A nyers bájt és a logikai érték összekeverése a másik gyakori félreértés.

A preemptálásról **csak a csoportprioritás** dönt: egy futó kezelőt az a kivétel szakíthat meg, amelynek csoportprioritása szigorúan magasabb rendű (kisebb szám). Az alprioritás és a kivételszám csak azt dönti el, hogy több egyidejűleg függő kivétel közül melyik kerül sorra először. Reset után `PRIGROUP = 0`, ami 3 megvalósított bit mellett azt jelenti, hogy mindhárom bit csoportprioritás.

A prioritások közötti döntés menete:

1. A mag a legmagasabb prioritású, függőben lévő (pending) kivételt választja.
2. Azonos csoportprioritás esetén az alprioritás, ha az is egyenlő, az alacsonyabb kivételszám nyer (tie-break szabály).
3. Egy futó kezelőt csak **szigorúan magasabb csoportprioritású** kivétel szakíthat meg; azonos csoportprioritású (akár kisebb alprioritású) nem preemptál.
4. A HardFault fix −1 prioritású, minden konfigurálható prioritású kivételnél magasabb rendű, és **nem** konfigurálható: az SCB `SHPR1-3` regiszterei csak a MemManage, BusFault,

UsageFault, SVCall, DebugMonitor, PendSV és SysTick prioritását tartalmazzák. A „mindig kiszolgálható” állításnak azonban vannak korlátai. A `PRIMASK` nem maszkolja a HardFaultot, a `FAULTMASK = 1` viszont a végrehajtási prioritást −1-re emeli, így a HardFault sem tud belépni (csak az NMI). Ha pedig HardFault vagy NMI kezelése közben (−1 vagy −2 prioritáson) újabb szinkron hiba keletkezik, nincs magasabb kezelő, ahová eszkalálódhatna: a mag **lockup** állapotba kerül, és nem hajt végre több utasítást.

A prioritásszám tehát nem „erősség”, hanem rangsor: minél kisebb, annál előrébb van a sorban. Ezt a szemléletet érdemes a C kód prioritásdefinícióiban is következetesen alkalmazni.

17.1.4 Thread mód és Handler mód

A Cortex-M két végrehajtási módot ismer. **Thread módban** fut a normál alkalmazáskód: a fő ciklus, a segédfüggvények, a háttérlogika. **Handler módban** futnak a kivételkezelők. A mag a kivétel belépésekor automatikusan Handler módba vált, és a kivételkezelőből való visszatéréskor tér vissza Thread módba. Handler módban a végrehajtás mindig privilegizált szintű; Thread módban lehet privilegizált vagy nem privilegizált, attól függően, hogyan konfiguráltuk a `CONTROL` regisztert.

A visszatérés mechanizmusa a Cortex-M sajátossága, és két réteget érdemes szétválasztani.

A hardveres réteg: `EXC_RETURN`. A kivétel belépésekor az `LR` (link register) egy speciális **EXC_RETURN** értéket kap (például `0xFFFFFFF9`). Ez nem visszatérési cím, hanem kódolt érték, amely megmondja a magnak, melyik verem keretét kell visszatölteni, és milyen módba kell visszatérni. A kivétel-visszatérést az váltja ki, hogy Handler módban egy **EXC_RETURN-mintájú érték kerül a PC-be** egy erre megengedett utasítással. Ilyen a `BX LR`, de ugyanígy a `POP {..., PC}`, az `LDM` a PC-vel a listában, vagy egy `LDR PC, [...]`, ha a visszatöltött érték az EXC_RETURN.

A fordítói réteg: a C-függvény prológusa és epilógusa. Egy C-ben írt kezelő a fordító szemében **közönséges** `void f(void)` függvény. Ha nem hív mást, jellemzően `BX LR`-rel zárul. Ha hív, a prológus `PUSH {..., LR}`-rel menti az LR-ben lévő EXC_RETURN-t, az epilógus pedig `POP {..., PC}`-vel tölti vissza, és ez maga a kivétel-visszatérés. A kezelőt tehát **nem kell** kézzel `BX LR`-rel zárni, és nem kell külön attribútum sem. A feltétel csak az, hogy az LR-ben kapott értéket a kód ne írja felül végleg, és változatlanul juttassa a PC-be. A GCC `__attribute__((interrupt))` Armv7-M-en érvényes, de mást jelent, mint A-profile-on: a kivételtípust figyelmen kívül hagyja, és azt jelzi, hogy a függvény csak 4 bájtra igazított SP-vel is meghívódhat, ezért a fordító a prológusban 8 bájtra igazítja a vermet. Ha a mag a belépéskor maga igazít (`CCR.STKALIGN = 1`, ami a Cortex-M3 r2p0-tól a reset utáni alapállapot), a sima `void f(void)` is AAPCS-helyes.

17.1.5 Verem és automatikus context-mentés

Amikor egy kivétel belép, a hardver automatikusan elmenti a megszakított program legfontosabb regisztereit a jelenlegi verembe. Ez a **context-mentés (context save)** nyolc szót érint. Az elrendezést az architektúra rögzíti: a belépés után az SP a keret legalacsonyabb címére (R0) mutat, és innen mérve az offsetek mindig ugyanazok:

Offset az új SP-től	Regiszter	Megjegyzés
`+0x00`	R0	
`+0x04`	R1	

Offset az új SP-től	Regiszter	Megjegyzés
`+0x08`	R2	
`+0x0C`	R3	
`+0x10`	R12	
`+0x14`	LR	a megszakított kód link registere
`+0x18`	PC	a **folytatási cím** (lásd lent)
`+0x1C`	xPSR	állapotjelzők; a 9. bit jelzi az igazítási szót
`+0x20`	(igazítási szó, opcionális)	csak ha a belépés előtti SP nem volt 8 bájtra igazított

Ha a belépés előtti SP nem volt 8 bájtra igazított, a mag egy igazítási szót hagy ki. Ez a szó a keret **fölött**, a magasabb címen, az xPSR és a belépés előtti SP között van, nem a keret alatt. A regiszterek offsetjei ezért nem változnak, a visszatérés pedig a mentett xPSR 9. bitjéből tudja, hogy vissza kell-e adnia a +4 bájtot.

A mentett PC általánosan a **folytatási cím**: az az utasítás, amellyel a kivétel-visszatérés után a végrehajtás folytatódik. Aszinkron megszakításnál ez a következő, még végre nem hajtott utasítás. `SVC`-nél az `SVC` utáni utasítás. Szinkron hibáknál (például UsageFault, precíz BusFault) jellemzően maga a hibát okozó utasítás, de nem minden hibatípusnál: egy pontatlan (imprecise) BusFault mentett PC-je egy később végrehajtott utasításra mutathat. A hibakeresésnél tehát a mentett PC a kiindulópont, nem automatikusan a hibás utasítás címe.

A Cortex-M3 nem tartalmaz FPU-t, ezért nála mindig ez a 8 szavas alapkeret jön létre. A lebegőpontos regisztereket is mentő, kiterjesztett keret csak FPU-s magokon (például Cortex-M4F, Cortex-M7) jelenik meg, és külön profilváltást igényel; ebben a fejezetben nem foglalkozunk vele.

A veremmutató a mentés során csökken, és a nyolc szó (szükség esetén az igazítási szóval együtt) bekerül a verembe. A megszakított program regiszterei tehát megőrződnek; a kezelő saját kódja szabadon használhatja őket. A visszatéréskor a hardver a nyolc szót visszatölti, és a megszakított utasításfolyam pontosan ott folytatódik, ahol abbamaradt. Ez a mechanizmus az alapja annak, hogy a C-ben írt ISR-ek látszólag „ingyen” megkapják a regisztermentést.

A context-mentés a jelenleg aktív veremmutatót használja. Thread módban ez általában a fő veremmutató (MSP) vagy a folyamat veremmutatója (PSP); Handler módban mindig az MSP. A veremhasználat és a PSP/MSP váltás részletei a következő alfejezetek témái; itt a fogalmi szint a lényeg: a hardver a kivétel belépésekor automatikusan ment, a visszatéréskor automatikusan visszaállít.

17.1.6 Kidolgozott példa: IRQ prioritásának beállítása és engedélyezése

Munkakönyvtár: `~/arm-labor/cm/a068/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a068
cd ~/arm-labor/cm/a068
```

A következő példa egy külső IRQ-t konfigurál a QEMU `mps2-an385` machine-en. A konkrét IRQ-számot szimbolikusan kezeljük; az AN385-ön a 0-s IRQ a UART0 RX, a 8-as a Timer0, a 12-es pedig a UART0–2 összevont túlcsordulási vonala (QEMU 8.2.2, `hw/arm/mps2.c`). A regisztercímek az Armv7-M System Control Space dokumentált címei.

A NVIC prioritásregiszterei (IPR) bájtonként címezhetők: az IRQ n prioritásbájtja a `0xE000E400 + n` címen található. Az engedélyezés az ISER0 regiszteren keresztül történik, a `0xE000E100` címen.

```
/* Thumb-2 / Armv7-M, Cortex-M3, QEMU mps2-an385 */
.equ NVIC_ISER0, 0xE000E100
.equ NVIC_IPR_BASE, 0xE000E400

    .syntax unified
    .thumb

/* Laborparaméter: a megvalósított prioritásbitek száma (CMSIS: __NVIC_PRIO_BITS).
   A futtatókörnyezethez kell rögzíteni, az a069 visszaolvasásos próbája szerint:
   3 a könyv 3 bites modelljében (tools/cm-run.py alapértelmezés), 8 a QEMU 8.2
   mps2-an385-ön. Az alábbi várható értékek a 3 bites modellre szólnak. */
.equ LAB_NVIC_PRIO_BITS, 3
.equ LAB_IRQ12_PRIO,     5            /* logikai prioritás */

/* Az IRQ12 prioritásának beállítása: a logikai értéket a megvalósított
   felső bitekbe toljuk. 3 bitnél: 5 << 5 = 0xA0. */
    ldr r0, =NVIC_IPR_BASE + 12
    mov r1, #(LAB_IRQ12_PRIO << (8 - LAB_NVIC_PRIO_BITS))
    strb r1, [r0]

/* Az IRQ12 engedélyezése: ISER0 bit 12. */
    ldr r0, =NVIC_ISER0
    mov r1, #(1 << 12)
    str r1, [r0]
```

A hozzá tartozó ISR C-ben, a vektortábla pedig a 16 + 12 = 28. indexű bejegyzést tartalmazza. Az IRQ12 itt szimbolikus példa: az AN385-ön ez a UART0–2 összevont túlcsordulási vonala, nem egy egyszerű perifériakérés (QEMU 8.2.2, `hw/arm/mps2.c`; 17.3 1. feladata a tényleges UART0 RX vonalat használja).

```
/* Thumb-2 / Armv7-M, Cortex-M3 */
void IRQ12_Handler(void)
{
    /* A periféria megszakításjelzőjének törlése itt történik. */
    /* ... */
}
```

A vektortábla megfelelő részlete assemblyben. Ez **rövidített** szemléltetés: a kihagyott bejegyzéseknek valódi szavaknak kell lenniük, különben az `IRQ12_Handler` nem a 28. indexre kerül. A teljes, kitöltött mintát a Megszakítás-vezérelt I/O kidolgozott példája és a Záróprojekt II vektortáblája adja.

```
/* Thumb-2 / Armv7-M, Cortex-M3 */
.section .vectors, "a", %progbits
    .word _estack
    .word Reset_Handler
    .word NMI_Handler
    .word HardFault_Handler
    /* ... a 16. indexig ... */
    .word 0             /* 16: IRQ0 */
    .word 0             /* 17: IRQ1 */
```

```
/* ... */
.word IRQ12_Handler /* 28: IRQ12 */
```

Várható állapot: a fenti konfiguráló részlet önmagában nem futtatható, mert nincs `main`/függvénykeret, és a vektortábla rövidített. A teljes, futtatható változatot a 17.2 laborja (`lab-a069-test.c`, NVIC konfiguráció: engedélyezés és prioritás) adja, és az ottani visszaolvasásos próba igazolja az itt bemutatott értékeket:

- `x/1xb 0xE000E40C` → `0xA0` (`0xE000E400 + 12`) a 3 bites modellben (`LAB_NVIC_PRIO_BITS = 3`);
- `x/1xw 0xE000E100` → `0x00001000` (bit 12);
- a vektortábla 28. szava (`0x70`) az `IRQ12_Handler` címe, páratlan értékkel (a bit0 a Thumb-állapotot jelzi).

A QEMU 8.2 8 bites modelljében ugyanez a kódsor a `LAB_NVIC_PRIO_BITS = 8` mellett `5 << 0 = 0x05`-öt ír be, és a visszaolvasás is `0x05`. A `0xA0` és a `0x05` tehát nem ugyanannak a futtatókörnyezetnek a nyers bájtja: előbbi a 3 bites, utóbbi a 8 bites modellé; a két számot ne keverd össze.

A `0xA0` logikai prioritása `0xA0 >> (8 - 3) = 5`. A reset utáni `PRIGROUP = 0` mellett ez teljes egészében csoportprioritás, tehát egy másik kezelőt akkor preemptál, ha annak csoportprioritása 5-nél nagyobb szám. Ha a paramétert nem igazítjuk a futtatókörnyezethez, és a 8 bites QEMU-n is a 3-as értékkel számolunk, a beírt nyers `0xA0` ott 160-as logikai prioritást jelent; ezért kell a bitszámot a visszaolvasás alapján rögzíteni.

17.1.7 Tipikus hibák és ellenpróbák

- **Rossz vektortábla-index.** Az IRQ0 a 16. indexre kerül, nem a 0-ra. Tünet: a reset után azonnal HardFault vagy hibás SP. Ellenőrzés: `objdump -s -j .vectors` a firmware binárison; a 0. szó SRAM-cím, a 16. szó az IRQ0 kezelő címe.
- **A prioritás fordított vagy rossz bitszámú értelmezése.** A `0xA0` 3 megvalósított bitnél 5-ös, 4 bitnél 10-es, 8 bitnél 160-as logikai prioritás. Tünet: egy „alacsony prioritásúnak” szánt IRQ megelőz egy „magasat”, vagy két, különbözőnek szánt prioritás egybeesik. Ellenőrzés: a `LAB_NVIC_PRIO_BITS` összevetése a visszaolvasásos próbával (17.2), és az `AIRCR.PRIGROUP` értéke: a preemptálásról csak a csoportprioritás dönt.
- **Hiányzó vektortábla-bejegyzés.** Ha egy engedélyezett IRQ vektora 0 vagy egy alapértelmezett hibakezelőre mutat, a megszakítás nem a szándékolt kezelőt futtatja. Tünet: HardFault vagy néma kimaradás. Ellenőrzés: a vektortábla és az `ISER` együttes vizsgálata.
- **Nem engedélyezett IRQ.** Az `ISER` bit beállítása nélkül a függőben lévő kérés nem indul el. Tünet: a periféria jelez, de kezelő nem fut. Ellenőrzés: `ISPR` és `ISER` regiszterek.

A hibakeresés menete mindig ugyanaz: tünet → hipotézis → adatgyűjtés (GDB, `objdump`, NVIC-regiszterek) → egy változtatás → ellenőrzés.

17.1.8 Feladatok

1. **(magyarázó, alapszint — cél: megértés-ellenőrzés)** Sorolj fel négy Cortex-M kivételtípust (Reset, NMI, HardFault, egy külső IRQ), és mindegyikhez add meg a kivételszámot, a vektortábla indexet és a prioritás típusát (fix vagy konfigurálható).

2. **(implementációs, alapszint — cél: rutinépítés)** Írj assembly kódrészletet, amely az IRQ3 prioritását 2-re állítja, majd engedélyezi. Add meg a prioritásregiszter pontos címét és a beírandó bájt értékét `LAB_NVIC_PRIO_BITS = 3` mellett, és azt is, hogy 4 bitnél mi lenne a helyes bájt.
3. **(hibakeresési/mérési, középszint — cél: hipotézisalkotás és regiszterellenőrzés)** Egy IRQ kezelője soha nem fut le, pedig az `ISER` megfelelő bitje be van állítva. Milyen hipotéziseket vizsgálnál? Milyen regisztereket és memóriaterületeket ellenőriznél GDB-vel?

17.1.9 Megoldások

1. megoldás — Segítség. A vektortábla indexe és a kivételszám közötti 1:1 megfelelésből indulj ki, és ne feledd, hogy a 0. index nem kivételhez, hanem a kezdeti veremmutatóhoz tartozik.

1. megoldás — Teljes megoldás. Reset: kivételszám 1, vektortábla index 1, prioritás fix -3. NMI: kivételszám 2, index 2, prioritás fix -2. HardFault: kivételszám 3, index 3, prioritás fix -1. IRQ0: kivételszám 16, index 16, prioritás konfigurálható. Az index minden esetben megegyezik a kivételszámmal, mert a 0. index a kezdeti veremmutatóé.

2. megoldás — Segítség. A prioritásbájt címe `0xE000E400 + IRQn`; a logikai prioritást a megvalósított felső bitekbe kell tolni, a shift mértéke `8 - LAB_NVIC_PRIO_BITS`.

2. megoldás — Teljes megoldás. Az IRQ3 prioritásbájtja a `0xE000E400 + 3 = 0xE000E403` címen van. A 2-es logikai prioritás `LAB_NVIC_PRIO_BITS = 3` mellett `2 << 5 = 0x40` (a felső 3 bit `010`); 4 bitnél `0x20`, 8 bitnél (QEMU 8.2) `0x02`. Az engedélyezés az `ISER0` `0xE000E100` címén, a 3-as bit beállításával történik.

```
/* Thumb-2 / Armv7-M, Cortex-M3 */
ldr r0, =0xE000E403
mov r1, #0x40
strb r1, [r0]
ldr r0, =0xE000E100
mov r1, #(1 << 3)
str r1, [r0]
```

3. megoldás — Segítség. Haladj végig a kiszolgálás feltételein: globális maszk, pending állapot, NVIC-engedélyezés, prioritás, vektortábla-bejegyzés — és a periféria oldalán a saját engedélyező és jelző regisztereken.

3. megoldás — Teljes megoldás. Hipotézisek: (a) a megszakítások maszkolva vannak a `PRIMASK` (vagy `FAULTMASK`, `BASEPRI`) regiszterrel; (b) a kérés függőben van, de egy magasabb prioritású kezelő blokkolja; (c) a periféria nem generál megszakítást, csak státuszbitet állít; (d) a vektortábla bejegyzése rossz címre mutat; (e) a kezelő neve nem egyezik a linkerscriptben használt szimbólummal. Ellenőrzés: `PRIMASK`, `ISPR`, `ISER`, a prioritásregiszter és a vektortábla 16+IRQ indexű szava. A periféria oldalán a megszakítás-engedélyező és jelző regiszterek vizsgálata szükséges.

17.1.10 Rövid összefoglaló

A Cortex-M kivételmodellje a kivételszám és a vektortábla index közötti 1:1 megfelelést használja: a 0. index a kezdeti veremmutató, az 1. a Reset, a 3. a HardFault, a 16. pedig az IRQ0. A prioritásszám fordítottan arányos a tényleges prioritással: a kisebb szám magasabb rendet jelent, a Reset/NMI/HardFault fix negatív prioritású. A megvalósított prioritásbitek száma implementációs paraméter (`LAB_NVIC_PRIO_BITS`), a nyers bájtból ebből adódik a logikai prioritás, a preemptálásról pedig a `PRIGROUP` szerinti csoportprioritás dönt. A kivétel belépésekor a hardver

automatikusan nyolc szót ment a verembe (szükség esetén egy igazítási szóval a keret fölött), és Handler módba vált. A visszatérést egy EXC_RETURN-értéknek a PC-be töltése váltja ki; egy C-függvény epilógusa (`BX LR` vagy `POP {..., PC}`) ezt külön teendő nélkül elvégzi.

Továbbhasznált definíciók:

- **kivételszám (exception number):** a kivételt azonosító egész szám; a vektortábla indexével egyezik meg.
- **prioritásszám (priority number):** a kivétel rangsorát kódoló érték; kisebb szám = magasabb prioritás.
- **automatikus context-mentés (automatic context save):** a hardver által kivétel belépésekor a verembe mentett nyolc regiszter.

17.1.11 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Armv7-M Architecture Reference Manual, DDI0403E — a kivételtípusok, a prioritásrendszer és a context-mentés elsődleges forrása.
- QEMU System Emulation dokumentáció — az `mps2-an385` machine és a CMSDK perifériák leírása; az IRQ-hozzárendelést (IRQ0 = UART0 RX, IRQ8 = Timer0, IRQ12 = UART0–2 összevont túlcsordulás) a QEMU 8.2.2 `hw/arm/mps2.c` forrása rögzíti.
- GNU Binutils as dokumentáció — a `.section`, `.word` és szimbólumkezelés nyelvi eszközei.

Ellenőrzési hiányok: a QEMU `mps2-an385` machine-en az egyes CMSDK perifériákhoz tartozó IRQ-számokat a QEMU 8.2.2 `hw/arm/mps2.c` forrása rögzíti (IRQ0 = UART0 RX, IRQ8 = Timer0, IRQ12 = UART0–2 összevont túlcsordulás); a példában ennek ellenére szimbolikus IRQ-számot használunk, hogy a minta általános maradjon. A megvalósított prioritásbitek száma az Armv7-M implementációs opciója; a könyv laborparamétere (`LAB_NVIC_PRIO_BITS = 3`) munkafeltételezés, amelyet konkrét eszközön a gyártói dokumentáció vagy a 17.2 visszaolvasásos próbája (0xFF írása, visszaolvasás) dönt el. A HardFault-korlátok (`FAULTMASK`, lockup), a keretigazítás és a `CCR.STKALIGN` alapállapota az Armv7-M ARM és a Cortex-M3 műszaki leírás szerinti állítás, itt nem futtattuk le.

17.2 NVIC konfiguráció: engedélyezés és prioritás

17.2.1 Motiváció: két kapu, egy döntéshozó

Tegyük fel, hogy egy 1 kHz-es periodikus timer megszakítására épülő háttérfeladatot és egy UART RX megszakítást kell üzemeltetnünk ugyanazon a Cortex-M3-on. A periféria már konfigurált, a vektortábla megfelelő bejegyzése a kezelőre mutat (lásd Reset és boot-folyamat: vektortábla), a globális megszakítás-maszk nyitva van — mégsem történik semmi. A hiba oka szinte mindig ugyanaz: a periféria *kér*, de a kérést az **NVIC** (Nested Vectored Interrupt Controller) még nem engedélyezte, vagy a prioritás úgy lett beállítva, hogy a kérés örökre várakozik.

Ez a helyzet tipikus mérnöki feladatot ad: két független kaput kell összehangolni. Az egyik kapu a periféria saját megszakítás-engedélyező bitje (ez a periféria-vezérlő regisztereiben van), a másik az NVIC engedélyező bitje. A kettő közül az NVIC az, amelyet assembly szinten, közvetlen regiszterírással ismerni kell: hibakeresésnél nem elég egy CMSIS-függvényt hívni, a regiszter visszaolvasásával kell bizonyítani, hogy az engedélyezés és a prioritás tényleg beállt.

Ráadásul a prioritás beállítása bitmező-művelet egy 32 bites regiszterben — pontosan az a minta, amelyet AArch64-en a `BFI`/`UBFX` kapcsán már láttunk (Bitfield-utasítások: BFI/UBFX/SBFX és csomagolt adatszerkezetek).

17.2.2 Definíció: az NVIC és a System Control Space

> **Definíció (NVIC-engedélyezés és prioritás).** Feltevés (a labor alapállapota): `PRIMASK = FAULTMASK = 0` és `BASEPRI = 0`. Ekkor egy külső IRQ akkor **kiszolgálható** (a mag belép a kezelőjébe), ha egyszerre teljesül: (a) pending (a periféria kérése vagy az `ISPR` miatt), (b) az NVIC **ISER** regiszterének megfelelő bitje 1 (engedélyezett), (c) a releváns maszkok nem tiltják: `PRIMASK`/`FAULTMASK` nulla, és `BASEPRI` nulla vagy az IRQ prioritásértéke kisebb (szigorúan magasabb rendű), mint `BASEPRI`, és (d) az IRQ **preempciós (csoport-) prioritása szigorúan magasabb rendű** (kisebb szám), mint az éppen futó kivételé (vagy nincs futó kivétel, azaz Thread módban vagyunk). Az **azonos** preempciós prioritású pending IRQ nem szakítja meg a futó kezelőt: megvárja a kezelő végét, és az azonos csoporton belül az alprioritás, majd a kisebb kivételszám dönt a sorrendről. A prioritásértéket az **IPR** megfelelő bájtja tárolja; kisebb szám magasabb rendet jelent (lásd A Cortex-M kivételmodell: prioritás és vektortábla-bejegyzések). Három ellenőrizhető eset: azonos preempciós prioritás → vár; magasabb (kisebb szám) → megszakít; `BASEPRI` az IRQ prioritásértéke (és `BASEPRI` `0`) → blokkolt.

Az NVIC az Armv7-M architektúra része, és fix, architekturálisan rögzített címeken érhető el a **System Control Space**-ben (SCS). Az itt használt összes utasítás **Thumb-2/Armv7-M ISA** (a könyv II. része); az I. részben tanult A64 utasítások itt nem érvényesek, a két ISA egyetlen példában sem keverhető.

Regiszter	Szó-cím	Indexelés	1 írásának hatása	Visszaolvasáskor
`NVIC_ISER0-7`	`0xE000E100 + 4·i`	IRQ `32i … 32i+31`	engedélyez	az engedélyezettség
`NVIC_ICER0-7`	`0xE000E180 + 4·i`	ugyanaz	letilt	az engedélyezettség (ugyanaz, mint az `ISER`)
`NVIC_ISPR0-7`	`0xE000E200 + 4·i`	ugyanaz	pending beállít	az aktuális pending állapot
`NVIC_ICPR0-7`	`0xE000E280 + 4·i`	ugyanaz	pending töröl	az aktuális pending állapot (ugyanaz, mint az `ISPR`)
`NVIC_IABR0-7`	`0xE000E300 + 4·i`	ugyanaz	nincs hatása (csak olvasható)	aktív állapot
`NVIC_IPR` bájtok	`0xE000E400 + n`	IRQ `n`	prioritásbájt (nem write-1: a 0 is érték)	prioritásbájt

Hozzáférés típusa: `ISER`/`ISPR` – írás-1-set; `ICER`/`ICPR` – írás-1-clear (0 írása hatástalan); `IABR` – csak olvasható; `IPR` – olvasható/írható mező (a 0 írása is prioritásmódosítás).

A táblázat két szabálya a gyakorlatban a legfontosabb. Egyrészt az `ISER`/`ICER`/`ISPR`/`ICPR` regiszterek **írás-1-hatásúak** (az `IABR` csak olvasható; az `IPR` bájtjai közönséges, olvasható-írható

prioritásmezők, ezekbe a 0 írása prioritásértéket állít be, mert nem set/clear regiszter): a 0 írása a set/clear regiszterekben nem hat, ezért `ISER = (1 << b)` és nem `ISER &= ~(1 << b)` alakban használjuk őket. Másrészt a párok **ugyanazt az állapotot olvassák**: az `ISER` és az `ICER` az engedélyezettséget, az `ISPR` és az `ICPR` az aktuális pending állapotot. Az `ICPR` tehát nem „mindig 0-t” ad, hanem 1-et mutat minden olyan biten, amely éppen függőben van. Emiatt a `|=` és `&=` operátorok ezeken a regisztereken félrevezetők: egy `ICPR |= m` a már függő **összes** megszakítás pending bitjét is visszaírná, és ezzel törölné. Az `ICER` 1-es írása csak tilt, a pending bitet nem törli; arra az `ICPR` való.

17.2.3 Engedélyezés: ISER és ICER bitenként

Az engedélyezés bitkiosztása egyszerű, mert az `ISER`/`ICER` bitek **IRQ-szám szerint** indexeltek:

$$\text{regiszterindex} = \left\lfloor \frac{IRQn}{32} \right\rfloor, \qquad \text{bit} = IRQn \bmod 32, \qquad \text{maszk} = 1 \ll (IRQn \bmod 32)$$

A `str.w r3, [r2, r1, lsl #2]` alakú regiszter-offszetes tárolás pontosan ezt a $4 \cdot i$ bájt-offszetet állítja elő, külön szorzás nélkül. Fontos következmény: a `IRQn = 33` esetén a regiszter az `ISER1` (`0xE000E104`), a bit pedig az 1-es — nem a 33-as, mert a 32 bites szóhatáron túl új regiszter kezdődik. Az AN385 QEMU-modellje ugyanakkor csak 32 IRQ-t valósít meg, ezért az IRQ 33-at csak papíron számold ki; a laborban 0–31 közötti IRQ-val próbáld.

Ugyanez a séma érvényes az `ISPR`-re (pending beállítás) és az `ICPR`-re (pending törlés). Az `ISPR` különösen hasznos **hibakeresésnél**: egy periféria nélkül, tisztán szoftverből pendíthetjük a megszakítást, és ellenőrizhetjük, hogy a kezelő elindul-e. Az `IABR` ezzel szemben csak olvasható: megmutatja, hogy az adott kivétel éppen aktív (fut vagy futott, és még nem fejeződött be) — ez reentrancia-vizsgálatnál és a kezelő „beszorulásának” kimutatásánál hasznos.

17.2.4 Prioritás: IPR és a bitmező-elhelyezés

A prioritásbájtok a `0xE000E400`-tól induló, 240 bájtos tömbben vannak, és itt egy **másik** indexelés él, mint a vektortáblában: a prioritásmező bájtindexe az **IRQ-szám**, nem a kivételszám. Ennek oka, hogy a konfigurálható rendszerkivételek (MemManage, BusFault, UsageFault, SVCall, Debug Monitor, PendSV és SysTick) prioritása nem itt, hanem az SCB `SHPR1-3` regisztereiben (`0xE000ED18-0xE000ED20`) található. A Reset (-3), az NMI (-2) és a HardFault (-1) prioritása fix: ezeknek nincs írható `SHPR`-mezőjük, és nem is vezethető le rájuk beírandó prioritásérték (kivételmodell). Az `NVIC_SetPriority(SysTick_IRQn, …)` CMSIS-hívás ezért nem is az `IPR`-t írja, hanem az `SHPR`-t — a látszat ellenére a két útvonal nem ugyanaz.

Egy IRQ prioritásmezőjének elhelyezése levezetve:

1. szó-index: $w = \lfloor IRQn/4 \rfloor$, mert egy 32 bites `IPR` szó négy prioritásmezőt tartalmaz;
2. szó címe: `0xE000E400 + 4·w`;
3. bájt-sáv a szón belül: $l = IRQn \bmod 4$;
4. bit-offszet a szón belül: $8 \cdot l$, azaz rendre 0, 8, 16 vagy 24.

Például `IRQ 33` esetén $w = 8$, a szó címe `0xE000E420`, $l = 1$, tehát a mező a szó `15:8` bitjein van, a bájt címe pedig `0xE000E421`.

A művelet maga tipikus read–modify–write bitmező-illesztés: maszkkal töröljük a régi mezőt, majd a 8 bites új értéket a helyére tolva `OR`-ral összefésüljük. A 9.3-ban tanult `BFI` itt **nem** használható

közvetlenül, mert a `BFI` az `lsb` és a szélesség operandusát is immediate-ként várja; futásidőben számolt mezőpozícióhoz a maszk+shift minta marad. Ha viszont az IRQ-szám fordítási időben ismert (például egy fix, beégetett konfigurációnál), a fordító jellemzően magától `bfi`/`ubfx` szekvenciát állít elő — ezt a mintát a Tipikus fordítói mintázatok felismerése tárgyalja.

Két gyakorlati buktató tartozik ide. Először: a prioritásmező **nem** azonos a logikai prioritásszámmal. A kivételmodell szerint négy dolgot kell külön kezelni: a nyers prioritásbájtot, a megvalósított bitek számát (CMSIS-ben `__NVIC_PRIO_BITS`, a könyv laborjában `LAB_NVIC_PRIO_BITS`), az ebből adódó logikai prioritást, és az `AIRCR.PRIGROUP` szerinti csoport- és alprioritást. A CMSIS `NVIC_SetPriority()` a kapott logikai értéket balra tolja ($8 - $ ___NVIC_PRIO_BITS) bittel. Ha mi nyers bájtot írunk, a hívó felelőssége a shiftelés. Másodszor: a nem implementált alsó bitek olvasáskor 0-t adnak, ezért a `0x01` és `0x07` ugyanazt a prioritást jelentheti egy 3 bitet megvalósító Cortex-M3-on. Ezt nem lehet kitalálni, csak megmérni: írjunk `0xFF`-et egy szabad IRQ prioritásmezőjébe, olvassuk vissza, és a visszakapott minta megmutatja a megvalósított biteket.

17.2.5 A konfiguráció helyes sorrendje

A megszakítás-vezérlő beállítása nem tetszőleges sorrendben biztonságos. A követendő minta:

1. a periféria megszakítás-kérése legyen letiltva a periféria-vezérlőben;
2. `nvic_disable_irq(n)` — tisztítsuk a korábbi állapotot;
3. `nvic_clear_pending(n)` — ha a periféria már pendített, a bit törlése megakadályozza a „száguldó” kezelőt;
4. `nvic_set_priority(n, prio)`;
5. `nvic_enable_irq(n)`;
6. a globális maszk feloldása (`cpsie i`, PRIMASK) — ez külön mechanizmus, amelyet a ISR írása: C és assembly vegyesen és a Megszakítás-latencia mérése és korlátai tárgyal részletesen;
7. a periféria saját megszakítás-engedélyezése.

A 3. lépés elhagyása a leggyakoribb „első próbálkozás” hiba: ha az engedélyezés előtt egy pending bit már be van állítva, a kezelő azonnal lefut — sokszor még azelőtt, hogy a szoftveres állapot, amire támaszkodik, inicializálódott volna.

17.2.6 Kidolgozott példa: egy timer megszakítás engedélyezése és prioritása

Munkakönyvtár: `~/arm-labor/cm/a069/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a069
cd ~/arm-labor/cm/a069
```

A példa a fejezet laborkörnyezetének CMSDK APB timerét használja `mps2-an385` alatt. A periféria báziscíme és az IRQ-száma a QEMU machine dokumentált memóriatérképéből ellenőrizendő, ezért a kód szimbolikus konstansokat használ; az NVIC-címek architekturálisan rögzítettek.

```
    .syntax unified
    .cpu    cortex-m3
    .thumb

    .equ    NVIC_ISER, 0xE000E100
    .equ    NVIC_ICER, 0xE000E180
    .equ    NVIC_ISPR, 0xE000E200
```

```
    .equ    NVIC_ICPR, 0xE000E280
    .equ    NVIC_IABR, 0xE000E300
    .equ    NVIC_IPR,  0xE000E400

    .text
    .thumb_func
    .global nvic_enable_irq
    @ r0 = IRQn  (0..239)
nvic_enable_irq:
    ubfx    r1, r0, #5, #3              @ r1 = IRQn / 32   -> ISER index
    ubfx    r2, r0, #0, #5              @ r2 = IRQn % 32
    movs    r3, #1
    lsls    r3, r3, r2                  @ maszk = 1 << (IRQn % 32)
    movw    r2, #(NVIC_ISER & 0xFFFF)
    movt    r2, #(NVIC_ISER >> 16)
    str.w   r3, [r2, r1, lsl #2]        @ ISER[IRQn/32] = maszk
    dsb
    isb
    bx      lr

    .thumb_func
    .global nvic_disable_irq
    @ r0 = IRQn ; ugyanaz a minta, csak NVIC_ICER bázissal
nvic_disable_irq:
    ubfx    r1, r0, #5, #3
    ubfx    r2, r0, #0, #5
    movs    r3, #1
    lsls    r3, r3, r2
    movw    r2, #(NVIC_ICER & 0xFFFF)
    movt    r2, #(NVIC_ICER >> 16)
    str.w   r3, [r2, r1, lsl #2]
    dsb
    isb
    bx      lr

    .thumb_func
    .global nvic_set_priority
    @ r0 = IRQn, r1 = prioritásbájt (nyers IPR-bájt, nem logikai prioritásszám)
nvic_set_priority:
    uxtb    r1, r1                      @ csak a 8 bites mező érdekes
    ubfx    r2, r0, #0, #2              @ r2 = IRQn % 4    -> bájt-sáv
    lsls    r2, r2, #3                  @ bit-offszet = 8 * (IRQn % 4)
    bic     r0, r0, #3                  @ r0 = IRQn & ~3   -> szó-offszet
    movw    r3, #(NVIC_IPR & 0xFFFF)
    movt    r3, #(NVIC_IPR >> 16)
    add     r3, r3, r0                  @ r3 = IPR szó címe
    ldr     r0, [r3]                    @ régi szó
    movs    r12, #0xFF
    lsls    r12, r12, r2                @ maszk = 0xFF << bit-offszet
    bic     r0, r0, r12                 @ mező törlése
    lsls    r12, r1, r2                 @ új érték a helyére tolva
    orrs    r0, r0, r12                 @ összefésülés
    str     r0, [r3]                    @ visszaírás
    dsb
    isb
    bx      lr
```

A C oldali megfelelő és a teljes inicializálási sorrend:

```
#include <stdint.h>

#define NVIC_ISER(i)  (*(volatile uint32_t *)(0xE000E100u + 4u * (i)))
#define NVIC_ICER(i)  (*(volatile uint32_t *)(0xE000E180u + 4u * (i)))
```

```c
#define NVIC_ICPR(i)  (*(volatile uint32_t *)(0xE000E280u + 4u * (i)))
#define NVIC_IABR(i)  (*(volatile const uint32_t *)(0xE000E300u + 4u * (i)))
#define NVIC_IPR_BYTE(irq) (*(volatile uint8_t *)(0xE000E400u + (uint32_t)(irq)))

#define IRQ_TIMER0  8u   /* MPS2-AN385 Timer0 */

static void nvic_enable_irq(unsigned irq)  { NVIC_ISER(irq / 32u) = 1u << (irq % 32u); }
static void nvic_disable_irq(unsigned irq) { NVIC_ICER(irq / 32u) = 1u << (irq % 32u); }
static void nvic_clear_pending(unsigned irq){ NVIC_ICPR(irq / 32u) = 1u << (irq % 32u); }
static void nvic_set_priority(unsigned irq, uint8_t raw_prio)
{
    NVIC_IPR_BYTE(irq) = raw_prio;      /* byte-index = IRQ-szám, nem kivételszám! */
}

void timer0_nvic_init(void)
{
    nvic_disable_irq(IRQ_TIMER0);
    nvic_clear_pending(IRQ_TIMER0);          /* maradék pending törlése  */
    nvic_set_priority(IRQ_TIMER0, 0xC0u);    /* nyers 0xC0: 3 bitnél logikai 6, 8 bitnél 192 */
    nvic_enable_irq(IRQ_TIMER0);             /* ISER bit beállítása      */
    __asm volatile ("dsb" ::: "memory");
    __asm volatile ("isb" ::: "memory");
    /* a globális maszk (PRIMASK) és a periféria engedélyezése külön lépés */
}
```

17.2.7 Várható állapot és ellenőrzés

Minden várt cím és bit **ugyanabból a szimbolikus IRQ-ból** (`IRQ_TIMER0 = 8`) származik. Tiszta, csak ezt az IRQ-t engedélyező állapotban:

Ellenőrzés	Cím / művelet	Képlet	Várt érték (`IRQ_TIMER0 = 8`)
engedélyezettség	`x/1xw 0xE000E100`	`ISER[8 / 32]`, bit `8 % 32`	`0x00000100`
prioritás bájt	`x/1xb 0xE000E408`	`0xE000E400 + 8`	`0xC0` (a megvalósított felső bitek szerint maszkolva)
prioritás szó	`x/1xw 0xE000E408`	`0xE000E400 + 4·(8 / 4)`, sáv `8 % 4 = 0`	alsó bájt `0xC0`; a többi bájt a 9–11. IRQ prioritása
pending	`x/1xw 0xE000E280`	`ICPR[0]`, bit 8	a bit **az aktuális pending állapot**: 0, ha nincs függő kérés
aktív állapot	`x/1xw 0xE000E300`	`IABR[0]`, bit 8	0, amíg a kezelő nem fut

A pending regiszterek viselkedését egy rövid teszt igazolja, amely a megszakítást közben **tiltva** tartja, így semmi nem fut le: (1) tiltás és pending törlés; (2) `ISPR`-rel szoftveres pending; (3) visszaolvasás az `ISPR`-ből és az `ICPR`-ből (mindkettő 1-et mutat a biten); (4) törlés `ICPR`-be írt 1-gyel; (5) újabb visszaolvasás (0). A teszt ugyanebben a futásban a megvalósított prioritásbiteket is megméri: `0xFF`-et ír a prioritásbájtba, és visszaolvassa. A kimenet semihostingon megy, a startup és a linker script a kanonikus pár (16.5, 16.6):

```c
/* lab-a069-test.c - NVIC-állapot ellenőrzése egyetlen szimbolikus IRQ-ból.
 * Minden cím és bit a LAB_IRQ-ból származik; a kimenet semihostingon megy. */
```

```c
#include <stdint.h>

#define IRQ_TIMER0  8u                  /* MPS2-AN385 Timer0 */
#define LAB_IRQ     IRQ_TIMER0

#define NVIC_ISER(i)  (*(volatile uint32_t *)(0xE000E100u + 4u * (i)))
#define NVIC_ICER(i)  (*(volatile uint32_t *)(0xE000E180u + 4u * (i)))
#define NVIC_ISPR(i)  (*(volatile uint32_t *)(0xE000E200u + 4u * (i)))
#define NVIC_ICPR(i)  (*(volatile uint32_t *)(0xE000E280u + 4u * (i)))
#define NVIC_IPR_ADDR(irq)  (0xE000E400u + (uint32_t)(irq))
#define NVIC_IPR_BYTE(irq)  (*(volatile uint8_t *)NVIC_IPR_ADDR(irq))

#define IRQ_WORD(irq)  ((irq) / 32u)
#define IRQ_MASK(irq)  (1u << ((irq) % 32u))

static void sh_call(uint32_t op, const void *arg)
{
    register uint32_t r0 asm("r0") = op;
    register const void *r1 asm("r1") = arg;
    asm volatile ("bkpt 0xab" : "+r"(r0) : "r"(r1) : "memory");
}

static void put_hex(const char *label, uint32_t v)
{
    char buf[48];
    char *p = buf;
    while (*label) *p++ = *label++;
    *p++ = '0'; *p++ = 'x';
    for (int sh = 28; sh >= 0; sh -= 4) *p++ = "0123456789abcdef"[(v >> sh) & 0xFu];
    *p++ = '\n'; *p = '\0';
    sh_call(0x04u, buf);                                  /* SYS_WRITE0 */
}

static int fails;
static void expect(const char *label, uint32_t got, uint32_t want)
{
    put_hex(label, got);
    if (got != want) { fails++; put_hex("  HIBA, várt: ", want); }
}

/* GDB-töréspont: noinline + asm volatile + memória-barrier, hogy az -O1/-O2 build se tüntesse
 ↪  el. */
__attribute__((noinline)) void lab_nvic_done(void) { __asm__ volatile("" ::: "memory"); }

int main(void)
{
    const uint32_t w = IRQ_WORD(LAB_IRQ), m = IRQ_MASK(LAB_IRQ);

    __asm volatile ("cpsid i" ::: "memory");            /* a teszt alatt nincs kiszolgálás */
    put_hex("IRQ maszk:          ", m);
    put_hex("IPR bájt címe:      ", NVIC_IPR_ADDR(LAB_IRQ));

    /* 1. kiindulás: tiltva, pending törölve */
    NVIC_ICER(w) = m;
    NVIC_ICPR(w) = m;
    expect("ISER (tiltva):      ", NVIC_ISER(w) & m, 0u);

    /* 2. tiltott IRQ-n szoftveres pending: az ISPR és az ICPR is a pending állapotot olvassa */
    NVIC_ISPR(w) = m;
    expect("ISPR a beállítás után: ", NVIC_ISPR(w) & m, m);
    expect("ICPR a beállítás után: ", NVIC_ICPR(w) & m, m);

    /* 3. törlés write-1-to-clear módon, majd újabb visszaolvasás */
```

```
    NVIC_ICPR(w) = m;
    expect("ICPR a törlés után:    ", NVIC_ICPR(w) & m, 0u);

    /* 4. megvalósított prioritásbitek: 0xFF írása, visszaolvasás */
    NVIC_IPR_BYTE(LAB_IRQ) = 0xFFu;
    put_hex("IPR 0xFF visszaolvasva: ", NVIC_IPR_BYTE(LAB_IRQ));

    /* 5. végleges konfiguráció: prioritás, majd engedélyezés */
    NVIC_IPR_BYTE(LAB_IRQ) = 0xC0u;
    NVIC_ISER(w) = m;
    __asm volatile ("dsb\n isb" ::: "memory");
    expect("ISER (engedélyezve): ", NVIC_ISER(w), m);
    expect("IPR bájt:            ", NVIC_IPR_BYTE(LAB_IRQ), 0xC0u);

    lab_nvic_done();
    sh_call(0x04u, fails ? "nvic FAIL\n" : "nvic ok\n");
    sh_call(0x18u, (const void *)(fails ? 0x20023u : 0x20026u));   /* SYS_EXIT */
    for (;;) { }
}
```

```
cp ~/arm-labor/cm/a065/lab-mps2.ld ~/arm-labor/cm/a066/lab-startup.s .
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 -g \
    -T lab-mps2.ld -o lab-a069.elf lab-startup.s lab-a069-test.c
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -nographic -semihosting \
    -kernel lab-a069.elf; echo "kilépési állapot: $?"
```

Kimenet a fő úton, mérve (GNU + QEMU 8.2.2, a lektor mérése):

```
IRQ maszk:          0x00000100
IPR bájt címe:      0xe000e408
ISER (tiltva):      0x00000000
ISPR a beállítás után: 0x00000100
ICPR a beállítás után: 0x00000100
ICPR a törlés után:    0x00000000
IPR 0xFF visszaolvasva: 0x000000ff
ISER (engedélyezve): 0x00000100
IPR bájt:            0x000000c0
nvic ok
kilépési állapot: 0
```

Az `IPR 0xFF visszaolvasva` sor az egyetlen, amely a futtató környezettől függ: a QEMU 8.2.2 8 bites NVIC-modelljében `0xff`, a könyv 3 bites modelljében (`tools/cm-run.py --prio-bits 3`) `0xe0`, és minden más sor változatlan marad. A QEMU stable-8.2 `mps2-an385` NVIC-modellje Armv7-M-en 8 bitet valósít meg, ott `0xff` a várt érték. A sor mutatja meg, hány prioritásbitet valósít meg a futtatókörnyezet. Ebből kell a `LAB_NVIC_PRIO_BITS`-et rögzíteni: a megvalósított bitek száma a visszaolvasott maszk egyeseinek száma, a legalacsonyabb konfigurálható prioritás pedig maga a maszk (`0xe0`, illetve `0xff`). A `0xC0` nyers érték mindkét esetben érvényes, de csak a 3 bites esetben jelenti a második legalacsonyabb szintet. A várt kimenetet tehát mindig a saját futtatás visszaolvasásából, nem egy másik eszköz adataiból kell levezetni.

GDB-vel az állapotot csak a konfiguráció lefutása **után** érdemes olvasni. A `-S` kapcsolóval indított QEMU a reset után, az első utasítás előtt áll meg; ekkor az NVIC még az indulás előtti állapotot mutatja, engedélyezett bitet még nem láthatunk. A system emulátor GDB-szerverét a `-gdb tcp::PORT` kapcsoló nyitja meg; a II. rész egységesen a `-S -gdb tcp::3333` alakot használja (a `-s` a `-gdb tcp::1234` rövidítése, az is működik, de akkor a GDB-ben is az 1234-es port kell). A qemu-user `-g 1234` kapcsolója a system emulátorba nem vehető át, ezért itt nem használható:

```
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -nographic -semihosting \
    -kernel lab-a069.elf -S -gdb tcp::3333
```

```
# másik terminálban:
gdb-multiarch lab-a069.elf -ex "target remote :3333" \
    -ex "break lab_nvic_done" -ex "continue" \
    -ex "x/1xw 0xE000E100" -ex "x/1xb 0xE000E408" -ex "x/1xw 0xE000E280"
```

Ellenőrizd, hogy a ténylegesen futtatott (-O1) célbináris disassemblyjében szerepel a `bl lab_nvic_done` hívás (`arm-none-eabi-objdump -d lab-a069.elf | grep -A2 lab_nvic_done`), és hogy a GDB batch-futás eléri a töréspontot; a hostos optimalizációs kísérlet ezt nem helyettesíti. A `lab_nvic_done` töréspontnál a konfiguráció már lefutott, ezért ott a fenti táblázat állapota várható. Mérve (GNU + QEMU 8.2.2, GDB 15.1): `0xE000E100` = `0x00000100`, `0xE000E408` = `0xc0`, `0xE000E280` = `0x00000000`. Ha a gdbstub hibát ad az SCS-címek olvasásakor, a firmware saját visszaolvasása (a teszt kimenete) a bizonyíték.

17.2.8 Tipikus hibák és ellenpróbák

- **Tünet:** az engedélyezés „nem hat”. **Hipotézis:** a prioritásmező rossz bájtba került. **Adatgyűjtés:** olvassuk vissza az `IPR` szót, és hasonlítsuk össze a várt `IRQn mod 4` sávval. **Egy változtatás:** írjunk a bájtcímre (`0xE000E400 + IRQn`) `strb`-vel. **Ellenőrzés:** a szó és a bájt ugyanazt tükrözi-e.
- **Tünet:** a kezelő azonnal lefut az engedélyezés után. **Hipotézis:** a pending bit nem lett törölve. **Ellenpróba:** az `ISPR`-rel kézzel pendítsünk, majd töröljük a pending bitet, és igazoljuk, hogy engedélyezett állapotban sem fut le.
- **Tünet:** a letiltás nem működik. **Hipotézis:** `ISER &= ~mask` mintát használtunk. **Ellenpróba:** az `ISER` nem támogatja a 0-írású törlést; a helyes útvonal `ICER = mask`.
- **Tünet:** a prioritás „nem veszi figyelembe” a különbséget. **Hipotézis:** a nem implementált alsó biteket használjuk. **Adatgyűjtés:** írjunk `0xFF`-et, olvassuk vissza.

17.2.9 Feladatok

1. **(magyarázó, alapszint — cél: cím- és bitmező-számítás gyakorlása)** Adott `IRQ 33`. Határozd meg az `ISER` regisztert és bitet, a maszkot, valamint az `IPR` szó címét, bájt-sávját és bit-offszetjét. Indokold, miért nem a kivételszámot használjuk az `IPR` indexelésénél.
2. **(implementációs, alapszint — cél: rutinépítés)** Írj `nvic_get_priority(irq)` függvényt Thumb-2 assemblyben, amely visszaadja egy IRQ prioritásbájtját. Készíts olyan változatot is, amely szóhosszú olvasást és maszkolást használ a bájtolvasás helyett.
3. **(hibakeresési/mérési, középszint — cél: NVIC-állapot megfigyelése és latenciamérés)** Állíts `breakpoint`-ot a timer kezelő első utasítására, és GDB-vel olvasd ki az `IABR`-t a kezelő futása közben, majd a kezelő végén. Magyarázd a különbséget, és mérd meg, mennyi idő telik el az `ISPR`-alapú pendítés és a kezelő első utasítása között (ciklusszámítással vagy `SysTick`-alapú méréssel).

17.2.10 Megoldások

1. megoldás — Segítség. Két külön indexelés van: az `ISER`-nél a 32 bites szó és a bit az IRQ-számból, az `IPR`-nél a szó és a bájtsáv; a kettőt ne keverd össze.

1. megoldás — Teljes megoldás. `exc = IRQ 33 + 16 = 49`, de az `IPR`-t az IRQ-szám indexeli: `w = 33 / 4 = 8` → szó cím `0xE000E400 + 32 = 0xE000E420`; `l = 33 mod 4 = 1` → bit-offszet 8, azaz a szó `15:8` bitjei; bájt cím `0xE000E421`. Az `ISER`-nél `i = 33 / 32 = 1` → `0xE000E104`, bit `33 mod 32 = 1` → maszk `0x00000002`. Azért az IRQ-szám az index, mert a

rendszer-kivételek prioritása az SCB SHPR1–3-ban van; az IPR tömb csak a külső IRQ-kat fedi le.

2. megoldás — Segítség. A bájtcím éppen 0xE000E400 + IRQn; a szóhosszú változatnál a szó címét IRQn & ~3 adja, a shift pedig 8·(IRQn mod 4).

2. megoldás — Teljes megoldás. A bájtolvasó változat kihasználja, hogy a bájtcím éppen 0xE000E400 + IRQn:

```
    .thumb_func
    .global nvic_get_priority
nvic_get_priority:
    movw    r1, #(NVIC_IPR & 0xFFFF)
    movt    r1, #(NVIC_IPR >> 16)
    add     r1, r1, r0              @ r1 = 0xE000E400 + IRQn
    ldrb    r0, [r1]                @ a prioritásbájt
    bx      lr
```

A szóhosszú változat: w = IRQn & ~3 címről ldr, majd lsrs r0, r0, r2 a r2 = 8·(IRQn mod 4) shiftel, végül and r0, r0, #0xFF (a 0xFF modified immediate-ként kódolható). A szóhosszú változat akkor előnyös, ha az implementáció a bájt-elérést megszorítja; a bájtváltozat viszont pontosan azt a CMSIS-mintát tükrözi, amelyet a valós kódbázisok használnak.

3. megoldás — Segítség. A kezelő futása alatt a kivétel aktív; gondold végig, mit mutat az IABR a belépés után és a visszatérés után, és hogy egy szoftveres pendítésnél mi történik a belépési latenciával.

3. megoldás — Teljes megoldás. Az IABR a kezelő futása alatt 1-est mutat a kérdéses biten (a kivétel aktív), a kezelő visszatérése után 0-t. Ez azért hasznos, mert megkülönbözteti a „pending, de még nem futott” és az „aktív, éppen futó” állapotot. A latenciamérésnél az ISPR-be írt pendítés és a kezelő első utasítása közötti ciklusszám a belépési latencia felső korlátját adja; mivel QEMU-emulációban nincs hardverhű ciklusszám, az eredmény csak relatív/illusztratív, és ezt a mérési jegyzőkönyvben jelölni kell.

17.2.11 Rövid összefoglaló

Az NVIC engedélyezését az ISER/ICER végzi IRQ-szám szerinti bitenkénti indexeléssel, a prioritást pedig a 0xE000E400-tól induló bájttömb tárolja, ahol a bájtindex az IRQ-szám (nem a kivételszám), a mezőpozíció pedig $8 \cdot (IRQn \bmod 4)$. Az ISPR/ICPR a pending állapotot állítja és **mindkettő vissza is olvassa**, az IABR az aktív állapotot kérdezi le. A biztonságos sorrend: letiltás, pending törlés, prioritás, engedélyezés, majd a globális maszk és a periféria engedélyezése.

Továbbhasznált definíciók:

- **írás-1-hatású regiszter:** az ISER/ICER/ISPR/ICPR típusú set/clear NVIC-regiszter, amelyben az 1-es írás műveletet vált ki, a 0 írása pedig hatástalan (az IPR nem ilyen).
- **prioritásbájt:** az IPR tömb 0xE000E400 + IRQn címén lévő 8 bites mező, amelynek kisebb értéke magasabb rendet jelent.
- **aktív állapot:** az IABR által mutatott feltétel, amely szerint az adott kivétel éppen kiszolgálás alatt áll.

17.2.12 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblázatát):

- Armv7-M Architecture Reference Manual, DDI0403E — az NVIC `ISER/ICER/ISPR/ICPR/IABR/IPR` regisztereinek bitkiosztása, az írás-1-hatás szemantika és a prioritásmezők elhelyezése.
- Arm M-profile architektúra dokumentáció — a kivétel- és prioritásmodell áttekintése.
- QEMU System Emulation dokumentáció — az `mps2-an385` machine, a CMSDK perifériák báziscímei és IRQ-hozzárendelése.

Ellenőrzési hiányok: (0) a `lab-a069-test.c` tesztet és a könyv assembly NVIC-rutinjait (`nvic_enable_irq`, `nvic_disable_irq`, `nvic_set_priority`) a helyettesítő eszközlánccal fordítottuk, és a `tools/cm-run.py` NVIC-regisztermodelljén futtattuk. Ez a modell az Armv7-M regiszterszemantikát követi (`ISER/ICER` az engedélyezettséget, `ISPR/ICPR` a pending állapotot olvassa), de nem a QEMU NVIC-je; a teszt a megszakítást tiltva tartja. A QEMU alatti futtatás és a GDB-lépések a laborban ellenőrizendők (a közös táblázatot lásd az 16.1 forrásjegyzetében). (1) a CMSDK APB timer báziscíme és pontos IRQ-száma az `mps2-an385` machine-en; (2) a `mps2-an385` (QEMU) és a fizikai MPS2/STM32 által megvalósított prioritásbitek száma — ezt a fenti visszaolvasásos próbával kell meghatározni; (3) a byte/halfword elérés támogatottsága az egyes NVIC-regisztereknél; (4) a `dsb/isb` kötelező volta az NVIC-engedélyezés után. Mindegyik pont a hallgató laborjában, az adott toolchain-verzióval ellenőrizendő.

17.3 ISR írása: C és assembly vegyesen

17.3.1 Motiváció

Tegyük fel, hogy az MPS2-AN385 (Cortex-M3) panelen egy 10 kHz-es mintavevő rutint kell futtatnunk: a periféria időzítője periodikusan megszakítást kér, a kezelőrutin kiolvassa az adatregisztert és beírja egy pufferbe, a főciklus pedig közben a puffert dolgozza fel. A kérdés nem az algoritmus, hanem a *szerződés*: amikor a processzor a főciklus kellős közepén megszakítja önmagát, ki menti el azokat a regisztereket, amelyekben a megszakított kód éppen dolgozott? Ha ezt nekünk kellene megtenni, minden ISR (interrupt service routine, megszakításkezelő rutin) egy assembly trampolinnal kezdődne. A Cortex-M kivételmodellje ezt a munkát hardverre bízza — ennek pontos tartalma és következményei a jelen alfejezet tárgya.

17.3.2 Definíció és a vektortábla-bejegyzés szerződése

Definíció (ISR). Az ISR olyan, a megszakított végrehajtási folyamattól függetlenül lefutó vezérlési szál, amelyet a processzor kivétel-belépés mechanizmusa hív meg, és amelynek visszatérése nem `BL`-hez, hanem egy speciális, hardver által előkészített visszatérési mechanizmushoz kötődik. Thumb-2/Armv7-M ISA kontextusban az ISR „argumentumai” nincsenek: a hívó fél (a megszakított kód) regiszterállapota a verembe kerül, nem paraméterlistába.

Definíció (vektortábla-bejegyzés). A vektortábla egy, a kódtartomány legelején elhelyezkedő, 4 bájtos szavas tömb, amelynek n-edik szava a n sorszámú kivétel belépési pontjának címe. Az `n = 0` slot nem cím, hanem a kezdeti veremmutató (`_estack`) értéke; az `n = 1` a reset-vektor. A rendszerkivételek 1–15 sorszámot kapnak, az externálisan kért megszakítások (IRQ) sorszámozása 16-tól indul, azaz az `IRQn` számú megszakítás táblaindexe $16 + IRQn$, bájt-offszete pedig:

$$\text{offset(IRQn)} = 4 \cdot (16 + \text{IRQn})$$

Ez az összefüggés tisztán levezethető: minden bejegyzés egy 32 bites szó, és előtte 16 rendszerkivétel-slot áll. Például az MPS2 `IRQ8` (CMSDK Timer0) bejegyzése a $4 \cdot 24 = 96 = 0\text{x}60$ bájt-offszetre esik.

A tábla tartalma és a C függvénynév között nincs mágikus kapcsolat: a linker annyit lát, hogy az egyik fordítási egységben (a startup assembly-fájlban) egy `.word TIMER0_IRQHandler` direktíva egy *szimbólumra* hivatkozik, másutt pedig egy C függvény definiálja ugyanezt a szimbólumot. A tábla csak a **szimbólum címe** iránt érdeklődik. Ez a név–cím kapcsolat a regisztrálás teljes mechanizmusa. A jellemző gyakorlat a gyenge szimbólum (weak symbol) és a `Default_Handler`-re mutató alias:

Az alábbi **részlet** csak a `.vectors` szekció, nem önálló fájl. Az IRQ8-ig szóról szóra teljes: minden indexnek valódi szó felel meg. A hozzá tartozó `Reset_Handler`, `Default_Handler` és a gyenge aliasok a kanonikus `lab-startup.s`-ben vannak (Startup kód), és a kidolgozott példa pontosan ezekkel a sorokkal bővíti azt:

```
/* Vektortábla-részlet (csak a .vectors szekció) - Thumb-2 / Armv7-M ISA */
.section .vectors,"a",%progbits
.align 2
    .word   _estack              /* 0  (0x00): kezdeti SP          */
    .word   Reset_Handler        /* 1  (0x04)                      */
    .word   NMI_Handler          /* 2  (0x08)                      */
    .word   HardFault_Handler    /* 3  (0x0C)                      */
    .word   MemManage_Handler    /* 4  (0x10)                      */
    .word   BusFault_Handler     /* 5  (0x14)                      */
    .word   UsageFault_Handler   /* 6  (0x18)                      */
    .word   0, 0, 0, 0           /* 7-10 (0x1C-0x28): fenntartott  */
    .word   SVC_Handler          /* 11 (0x2C)                      */
    .word   DebugMon_Handler     /* 12 (0x30)                      */
    .word   0                    /* 13 (0x34): fenntartott         */
    .word   PendSV_Handler       /* 14 (0x38)                      */
    .word   SysTick_Handler      /* 15 (0x3C)                      */
    .rept   8
    .word   Default_Handler      /* 16-23 (0x40-0x5C): IRQ0-IRQ7   */
    .endr
    .word   TIMER0_IRQHandler    /* 24 (0x60): IRQ8 - ide kerül a C függvény címe */

/* gyenge alapértelmezés: ha a felhasználó nem definiálja, ide köt */
.weak   TIMER0_IRQHandler
.thumb_set TIMER0_IRQHandler, Default_Handler
```

Két részlet magyarázatra szorul. Egyrészt a 4 bájtos szóba kerülő érték nem a függvény „nyers” címe, hanem annak Thumb-bittel jelölt változata; a Thumb-2 szimbólumok címe eleve páratlan (LSB = 1), és a GAS `.thumb_set` illetve a C fordító által előállított szimbólumok ezt automatikusan hordozzák. Ha kézzel írnánk be páros címet, a processzor a kivétel belépésekor hardverhibába (HardFault) futna. Másrészt a `.weak` + `.thumb_set` minta megvalósítja azt, hogy a vektortáblát ne kelljen újrafordítani, amikor valamelyik kezelőt implementáljuk: a linker a gyenge szimbólumot a valódi definícióval cseréli. Az alapértelmezett kezelő tipikusan egy végtelen ciklus, ami rögtön megmutatja a hibát debugging közben.

17.3.3 Az automatikus context-mentés

Kivétel belépésekor a hardver, mielőtt egyetlen utasítást is végrehajtana a kezelőből, nyolc regisztert a verembe ír. A verem a `CONTROL.SPSEL` és a belépéskori mód függvényében az `MSP`

vagy a PSP — a pontos választás az EXC_RETURN értékéből olvasható vissza. A keret elrendezése fix, a veremmutató a legalacsonyabb címre mutat:

Offszet az SP-től	Tartalom	Megjegyzés
+0x00	R0	argumentum / munkaregiszter
+0x04	R1	argumentum / munkaregiszter
+0x08	R2	argumentum / munkaregiszter
+0x0C	R3	argumentum / munkaregiszter
+0x10	R12	scratch (IP)
+0x14	LR (R14)	a megszakított kód visszatérési címe
+0x18	PC (R15)	folytatási cím: ahol a visszatérés után a végrehajtás folytatódik
+0x1C	xPSR	állapotjelzők + IPSR kivételszám

Az SP tehát 32 bájttal csökken. Ha a belépéskori SP nem volt 8 bájt szerint igazítva, a mag további 4 bájtot hagy ki: ez az igazítási szó a keret **fölött**, a magasabb címen (+0x20), az xPSR és a belépés előtti SP között van. A fenti offsetek ezért ettől nem változnak, a kihagyást pedig a mentett xPSR 9. bitje jelzi, hogy a visszatérés pontosan visszaadhassa. Ez az elrendezés architekturális szabály, nem QEMU- vagy hardverfüggő részlet.

A mentett PC általánosan **folytatási cím**. Megszakításnál ez a következő, még végre nem hajtott utasítás. Szinkron hibánál gyakran maga a hibát okozó utasítás, de nem mindig: egy pontatlan (imprecise) BusFault esetén a mentett PC egy később végrehajtott utasításra mutathat. A Cortex-M3-nak nincs FPU-ja, ezért itt mindig ez a 8 szavas keret jön létre. A lebegőpontos regisztereket is tartalmazó kiterjesztett keret csak FPU-s magokon (Cortex-M4F, M7) fordul elő.

Levezetés: miért pontosan ez a nyolc regiszter? Az AAPCS (32 bites) szerint a hívott függvény szabadon felülírhatja az R0–R3 és R12 regisztereket (argumentum- és scratch-regiszterek), valamint az LR-t; a hívó felelőssége ezek megőrzése, ha értékükre a hívás visszatérése után szüksége van. A hardver által mentett halmaz pontosan ez a *caller-saved* halmaz, kiegészítve a PC-vel és az xPSR-rel, amelyek a megszakított folyamat folytatásához elengedhetetlenek. Az R4–R11 (callee-saved) regisztereket nem menti a hardver, mert azokat — az AAPCS szerint — úgyis annak kell megőriznie, aki használja: vagyis az ISR-nek, a fordító által generált szokásos prológusban. Ez a megfigyelés az egész alfejezet kulcsa.

Definíció (automatikus context-mentés, stacking). A kivétel belépésének mikroarchitekturális lépéssorozata, amely a caller-saved regisztereket és a folytatási állapotot a verembe írja, majd az EXC_RETURN kódot az LR-be tölti, és a PC-t a vektortábla bejegyzésére állítja.

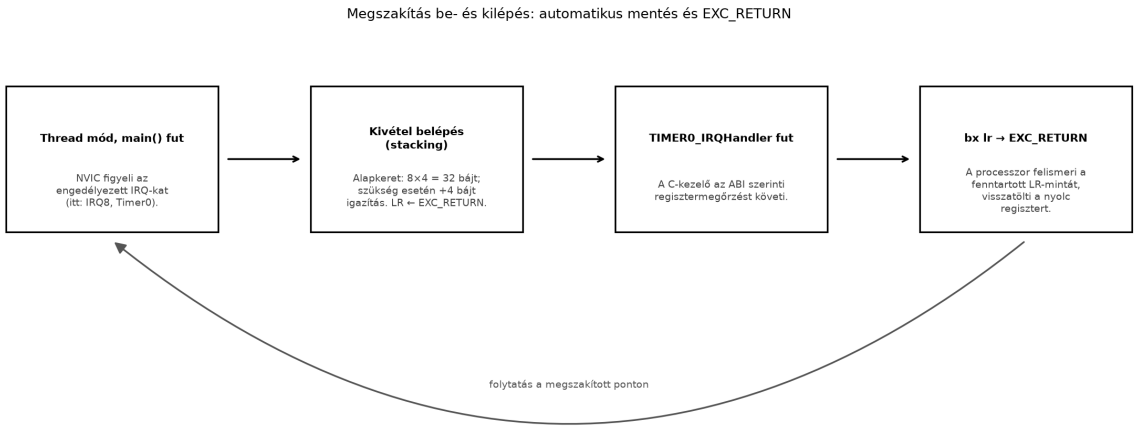
17.3.4 Miért írható a legtöbb ISR tisztán C-ben

Az automatikus mentés miatt a kezelő belépésekor a regiszterállapot már „mentve van” pontosan azon a szerződésen keresztül, amit a C fordító a függvényhívásoknál amúgy is elvár. A C-ben megírt ISR számára a helyzet megegyezik egy közönséges függvényhívással: az R0–R3 és R12 szabadon használható, az R4–R11-et használat előtt meg kell őrizni (ezt a fordító megteszi), az LR-t pedig csak akkor kell menteni, ha az ISR további függvényt hív. Nem kell assembly trampolin, mert a hardver már elvégezte a hívói oldal munkáját. A visszatérést sem kell kézzel

megírni: a fordító szokásos epilógusa az LR-ben kapott **EXC_RETURN** értéket juttatja a PC-be, és ez indítja el a kivétel-kilépést.

Az `EXC_RETURN` fogalmi szinten a következő: kivétel belépésekor az LR nem visszatérési címet, hanem egy fenntartott bitmintát kap, például `0xFFFFFFF9` (Thread mód, `MSP`), `0xFFFFFFFD` (Thread mód, `PSP`) vagy `0xFFFFFFF1` (Handler mód, `MSP`). Amikor a kezelő az LR-en keresztül tér vissza, a processzor felismeri ezt a mintát, és nem ugrást, hanem *kivétel-kilépést* hajt végre: visszatölti a nyolc regisztert, és a megszakított ponton folytatja. Az ISR-t tehát nem kell „különleges” módon megírni. A kivétel-kilépést bármely, Handler módban megengedett PC-író utasítás kiváltja, ha az EXC_RETURN-értéket tölti a PC-be: a `bx lr`, a `pop {..., pc}`, az `ldm` a PC-vel a listában vagy egy `ldr pc, [...]`. A levélfüggvényként fordított kezelő tipikusan `bx lr`-rel zárul, a más függvényt hívó kezelő `push {..., lr}` / `pop {..., pc}` párral; egyiket sem kell kézzel írni. A GCC `__attribute__((interrupt))` Armv7-M-en érvényes, de nem kötelező: a kivételtípust figyelmen kívül hagyja, és azt jelzi, hogy a függvény csak 4 bájtra igazított SP-vel is meghívódhat, ezért a prológus 8 bájtra igazít. A belépéskori hardveres igazítás (`CCR.STKALIGN = 1`, a Cortex-M3 r2p0-tól alapértelmezés) mellett a sima `void f(void)` is helyes.

Az alábbi ábra a teljes belépés–futás–kilépés ciklust foglalja össze, a később bemutatott `TIMER0_IRQHandler` példáján:



17.1. ábra: Megszakítás be- és kilépés: automatikus mentés és EXC_RETURN - A Thumb-2/Armv7-M kivétel-belépés és -kilépés lépéssorozata.

A mentett regiszterek mennyisége jól szemlélteti, mennyit spórolunk. Az alábbi táblázat **levezetett** értékeket mutat: a hardveres keret fix 32 bájtja fölé a fordító csak azt teszi, amit az ISR ténylegesen használ.

Eset	Hardver	Fordító	Összesen
Üres ISR (nem hív más függvényt)	32 B	0 B	32 B
ISR, amely C függvényt hív	32 B	LR + 4 B igazítás = 8 B	40 B
ISR, amely R4–R7-et is használ és hív	32 B	5 szó → 8-ra kerekítve = 24 B	56 B

17.3.5 A pending-jelző törlése: két különböző réteg

Itt rejlik az alfejezet leggyakoribb gyakorlati csapdája, és fontos szétválasztani két fogalmat. Az **NVIC pending bit** (a korábbi alfejezetben tárgyalt `ISPR`/`ICPR` regiszterpár) a megszakításkérésnek a processzor oldali tárolója. Külső, periféria által kért megszakítás esetén ezt a bitet a hardver **automatikusan törli**, amikor a kivételt belépésként elfogadja — tehát az ISR-nek ezzel nem kell foglalkoznia. A **periféria saját megszakításjelzője** viszont egyáltalán nem törlődik magától: amíg a periféria jelzője be van állítva, addig a kéréssor továbbra is aktív, és az NVIC a kezelő visszatérése után azonnal újra pending állapotba kerül.

Definíció (periféria pending-jelző). A periféria saját állapotregiszterében tárolt bit (gyakran `INTSTATUS`/`RIS`/`MIS` néven), amely a kérést addig tartja, amíg azt szoftveresen nem törlik (írás-1-hatású `INTCLR` regiszterbe, vagy a status regiszter olvasásával).

A következmény drámai: elfelejtett törlés esetén az ISR visszatér, a processzor azonnal újra belép ugyanabba a kezelőbe, és a főciklus soha nem kap processzort. Ezt nevezzük **megszakítási viharnak** (interrupt storm). A tünet jellegzetes: a rendszer „él” (az óra jár, a LED villog, ha az ISR-ben van a villogtatás), de a főciklus első utasítása után mindig megszakadás következik, és a megszakításszámláló a valóságosnál nagyságrendekkel gyorsabban nő.

Ezért az ISR szerkezeti vázlata mindig: *adat kiolvasása → a periféria pending-jelzőjének törlése → feldolgozás*. A törlést érdemes a feldolgozás elé tenni (a lehető leghamarabb az adat kiolvasása után), hogy egy hosszabb feldolgozás közben bekövetkező újabb esemény ne vesszen el.

17.3.6 Kidolgozott példa: Timer0 ISR C-ben az MPS2-AN385-ön

Munkakönyvtár: `~/arm-labor/cm/a070/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a070
cd ~/arm-labor/cm/a070
```

A linker script és a startup a II. rész kanonikus párja (Linker script alapjai, Startup kód), a vektortábla ellenőrzője pedig a Vektortábla alfejezet `lab-check-vectors.py`-ja. A linker script változatlan; a startup másolatának vektortábláját egyetlen, gépileg alkalmazható lépéssel bővítjük a Timer0 (IRQ8) bejegyzésével:

```
cp ~/arm-labor/cm/a065/lab-mps2.ld .
cp ~/arm-labor/cm/a066/lab-startup.s lab-a070-startup.s
cp ~/arm-labor/cm/a064/lab-check-vectors.py .
cat > lab-a070-irq.inc <<'EOF'
    /* 16-tól: külső IRQ-k; a Timer0 = IRQ8 a 24. index, előtte 8 helyet ki kell tölteni */
    .rept 8
    .word Default_Handler         /* 0x40-0x5C: IRQ0-IRQ7 (itt nem használt) */
    .endr
    .word TIMER0_IRQHandler       /* 0x60: IRQ8 (Timer0)         */
EOF
sed -i '/\.word SysTick_Handler/r lab-a070-irq.inc' lab-a070-startup.s   # a SysTick sor után
# gyenge alias: kezelő nélkül is linkelhető
echo '    lab_default_handler TIMER0_IRQHandler' >> lab-a070-startup.s
# ellenőrzés: a bővített tábla vége
sed -n '/\.word SysTick_Handler/,/^$/p' lab-a070-startup.s
```

A `sed` mintája csak a vektortábla `.word SysTick_Handler` sorára illeszkedik, a fájl végén lévő `lab_default_handler SysTick_Handler` aliasra nem. A fel nem használt IRQ-helyek a kanonikus `Default_Handler`-re mutatnak. Az `echo` sor a kanonikus `lab_default_handler` makróval a fájl végére egy **gyenge**, `Default_Handler`-re mutató aliast fűz a használt kezelőhöz

is. A C-fájl erős definíciója ezt felülírja, de ha egy build a kezelőt nem tartalmazza, a KEEP-pel megtartott vektortábla hivatkozása akkor sem marad feloldatlan. Ezek **valódi táblaszavak**, nem megjegyzések, mert csak így kerül a használt kezelő a megfelelő indexre (16 + IRQn).

A példa a CMSDK APB Timer0-t használja. A regisztercímek és a CTRL mezőkiosztás mérve QEMU 8.2.2-vel (a lektor mérése) helyesek: CTRL bit0 = ENABLE, bit3 = IRQEN; 0x08 = RELOAD, 0x0C = INTSTATUS/INTCLEAR.

```
/* lab-a070_isr.c  - Thumb-2 / Armv7-M ISA */
#include <stdint.h>

#define CMSDK_TIMER0_BASE     0x40000000UL   /* MPS2-AN385, CMSDK APB0 */
#define CMSDK_TIMER0_CTRL     (*(volatile uint32_t *)(CMSDK_TIMER0_BASE + 0x00))
#define CMSDK_TIMER0_RELOAD   (*(volatile uint32_t *)(CMSDK_TIMER0_BASE + 0x08))
/* Nincs külön INTCLR regiszter: az INTSTATUS-ba írt 1 törli a pendinget (write-1-to-clear). */
#define CMSDK_TIMER0_INTSTATUS (*(volatile uint32_t *)(CMSDK_TIMER0_BASE + 0x0C))

#define NVIC_ISER0            (*(volatile uint32_t *)0xE000E100UL)
#define TIMER0_IRQn           8u

#define TIMER_CTRL_ENABLE     (1u << 0)
#define TIMER_CTRL_IRQEN      (1u << 3)      /* bitpozíció: CMSDK spec. ellenőrizendő */

volatile uint32_t g_tick;       /* kiszolgált megszakítások száma: CSAK az ISR írja, monoton nő
↪  */
/* a főciklus által feldolgozott események; GDB-vel figyelhető */
volatile uint32_t g_events;

/* A szimbólumnév a vektortábla 0x60 offszetén lévő .word operandusa.   */
void TIMER0_IRQHandler(void)    /* nincs kézzel írt assembly trampolin/prológus; a fordító
↪  mentései a disassemblyben láthatók */
{
    CMSDK_TIMER0_INTSTATUS = 1u; /* 1. periféria pending-jelző törlése (write-1-to-clear) */
    g_tick++;                    /* 2. az esemény rögzítése; a főciklus ebből értesül */
}

int main(void)
{
    CMSDK_TIMER0_RELOAD  = 0x00FFFFFFu;         /* lassú periódus */
    CMSDK_TIMER0_CTRL    = TIMER_CTRL_ENABLE | TIMER_CTRL_IRQEN;
    /* esetleges korai pending törlése (write-1-to-clear) */
    CMSDK_TIMER0_INTSTATUS = 1u;
    NVIC_ISER0           = (1u << TIMER0_IRQn); /* engedélyezés, lásd a069 */

    uint32_t seen = 0;          /* a főciklus saját állapota: eddig látott események */
    for (;;) {
        uint32_t now = g_tick;  /* egyetlen igazított 32 bites olvasás */
        if (now != seen) {
            g_events += now - seen; /* az előző kör óta érkezett MINDEN esemény */
            seen = now;
        }
    }
}
```

A build és futtatás:

```
# ~/arm-labor/cm/a070 - target: qemu-system-arm (MPS2+ AN385)
# fájlok: lab-mps2.ld, lab-a070-startup.s (+ lab-a070-irq.inc), lab-a070_isr.c, lab-check-
# vectors.py
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib \
  -Wall -Wextra -g3 -O1 -T lab-mps2.ld \
  lab-a070-startup.s lab-a070_isr.c \
  -o lab-a070.elf
```

```
python3 lab-check-vectors.py lab-a070.elf 1=Reset_Handler 24=TIMER0_IRQHandler
qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -display none -monitor none -serial null \
  -gdb tcp::3333 -S -kernel lab-a070.elf
```

A `-S` és a GDB-kapcsolat lehetővé teszi, hogy a `main`-be állított töréspontон megállva ellenőrizzük a vektortábla tartalmát (`x/32xw 0x00000000`) és a keretet (`info registers`, majd a kezelő belépésekor `x/8xw $sp`). A stackelt keret vizsgálatához **utasításcímre tett töréspont** kell a handler első utasításán (`break *TIMER0_IRQHandler`, a fordítói prológus *előtt*); ha a prológus után állsz meg, a 8 szavas hardveres keret mellé a szoftveresen mentett regisztereket is hozzá kell számolni (`arm-none-eabi-objdump -d` mutatja a tényleges `push {…}`-t). A hardveres kivételkeret (8 szó, 32 bájt) és a fordító szoftveres mentése külön tétel a 20.1 verembecslésében is. A labor feltétele, hogy a thread-kód az `MSP`-t használja; ha egy `PSP`-s thread kap kivételt, a mentett keret nem a handler aktuális `MSP`-jén van, hanem a `PSP`-n. A `g_tick` változó növekedése a `main` ciklusából figyelhető meg — de csak akkor, ha a pending-jelző törlése megtörténik.

Várható viselkedés (ellenőrzési feltétel). A helyes változatban `g_tick` a megszakítási periódusnak megfelelő, kiszámított ütemben nő, és a `main` ciklus rendszeresen lefut (a `g_events` értéke időnként frissül). Hiba esetén: `g_tick` a periódustól függetlenül, nagyságrendekkel gyorsabban nő (a CPU idejének ~100%-a az ISR-ben telik), a `g_events` soha nem frissül — ez a törölt pending-jelző hiányának tipikus tünete. Megjegyzendő, hogy a **ciklusszámra vonatkozó abszolút értékek QEMU-ban nem hardverhűek**, ezért itt csak a relatív viselkedés (rendszeresen fut-e a főciklus) az elfogadási feltétel; a tényleges latenciamérést a Megszakítás-latencia mérése és korlátai tárgyalja.

Ellenőrző lépés GDB-vel. Indítsd a QEMU-t `-S` nélkül, majd néhány másodperc múlva csatlakozz: `gdb-multiarch lab-a070.elf -ex 'target remote :3333' -ex 'p g_tick' -ex 'p g_events'`. A várt érték kb. 1,5 tick/s mellett közel egyenlő; a törlés nélküli változatban `g_tick` milliós, `g_events` 0, és a PC a `TIMER0_IRQHandler`-ben áll. QEMU 8.2.2-n, a lektor mérése szerint, 3 s után `g_tick=4`, `g_events=4`; a vihar-változatban `g_tick=6 768 482`, `g_events=0`.

Mit számol a `g_tick`? A `g_tick` a *kiszolgált megszakítások* számát adja, nem garantáltan a fizikai timerperiódusokét: ha a megszakítás tovább van tiltva (`PRIMASK`, kritikus szakasz) vagy a kezelő tovább fut, mint a periódus, több timeresemény egyetlen pending bitre olvad össze, és a számláló csak eggyel nő. Eltelt periódusok rekonstruálásához vagy bizonyítani kell, hogy a legnagyobb blokkolási idő kisebb a periódusnál, vagy hardveres időalapot (a szabadon futó `TIMER1` számlálót) kell használni. Kísérlet: tartsd tiltva a megszakítást több periódusig (`cpsid i` … `cpsie i`), majd engedélyezd; a `g_tick` ekkor akár több periódus alatt is csak eggyel nő — a leírásnak és a tesztnek ezt a modellkapacitást kell várnia, nem megőrzést.

Miért számláló, és miért nem jelző? Egy `if (g_pending) { g_pending = 0; … }` minta két művelet: olvasás, majd törlés. Ha a kettő közé befut egy újabb megszakítás, és 1-et ír a jelzőbe, a főciklus ezt a frissen beírt 1-et törli: az esemény elveszik. Ez akkor elfogadható, ha a szerződés csak **összevont értesítést** ígér („volt legalább egy esemény”); veszteségmentes eseményátvitelre nem az. A fenti minta ezt kerüli el. A `g_tick`-et egyetlen kontextus, az ISR írja, a főciklus csak olvassa, és a saját `seen` értékével számolja a különbséget. Nincs tehát olyan pont, ahol a főciklus az ISR írását felülírná, és a `now - seen` különbség az átfordulás után is helyes (modulo 2^{32}). Determinisztikus próbával igazoltuk: ha egy második eseményt pontosan az olvasás és a törlés közé injektálunk, a jelzős minta 2 eseményből 1-et dolgoz fel, a számlálós mind a kettőt. A főciklus haladását a `g_events` mutatja; ez külön mérőpont, mert egy csak lokálisan írt, fel nem használt változót a fordító el is hagyhat.

Az igazított 32 bites `g_tick` olvasása Cortex-M3-on egyetlen `LDR`, ezért atomi: a főciklus nem láthat félig frissített értéket. Ez azonban **csak az egyes hozzáférésre** igaz. Egy több műveletből álló protokoll (olvasás–vizsgálat–törlés, vagy több változó együttes frissítése) ettől még nem atomi. A 64 bites vagy több szóból álló adat egyetlen olvasása sem az. Ilyenkor vagy egyetlen író

mintát kell választani, mint itt, vagy rövid, a `PRIMASK`-ot megőrző kritikus szakaszt (Záróprojekt II).

17.3.7 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés	Egy változtatás
A főciklus soha nem fut le	a periféria pending-jelzőjének törlése hiányzik, vagy nem hat (rossz cím vagy bit)	GDB: töréspont az ISR-ben, `g_tick` monotón nő; az `INTSTATUS` visszaolvasása a törlés után	`INTSTATUS = 1` (write-1-to-clear) a kivételből való visszatérés előtt (a példában az ISR elején)
Megszakítás nem indul el	szimbólumnév nem egyezik a vektortábla operandusával, ezért a `Default_Handler` fut	`nm lab-a070.elf | grep TIMER`, `x/32xw 0`	a név és a `.weak`/`.thumb_set` összehangolása
HardFault az első megszakításkor	a vektortáblába páros cím került; a bejegyzések nincsenek szóhatáron; áthelyezett (`VTOR`) táblánál a bázis nem éri el a szükséges igazítást	GDB: `x/2xw 0` (illetve a `VTOR` által mutatott cím) és az `IPSR` értéke; `lab-check-vectors.py`	Thumb-bit és `.align 2` a bejegyzésekre; áthelyezett táblánál a bázis legalább 128 bájtra és a táblaméretnél nem kisebb kettőhatványra igazítva (16.4); a kanonikus `0x00000000` bázis ezt teljesíti
A főciklus nem látja a `g_tick` változást	hiányzó `volatile`, a fordító regiszterben tartja	`objdump -d` a `main`-en, a `g_tick` betöltése	`volatile` minősítő hozzáadása
Ritkán, véletlenszerűen elszállt adat vagy elveszett esemény	több műveletből álló protokoll (pl. jelző olvasása, majd törlése) vagy egy szónál szélesebb megosztott adat megszakítás közben	GDB: az olvasás és az írás közötti utasítások; szándékosan a résbe injektált esemény	egyetlen író, monoton számláló és főciklusbeli `seen` (mint a példában), vagy rövid, `PRIMASK`-ot megőrző kritikus szakasz

17.3.8 Feladatok

1. (magyarázó) Az MPS2-AN385-ön az UART0 **vételi** (RX) megszakítása az `IRQ0` vonalon érkezik; a TX az `IRQ1`, az `IRQ12` pedig a UART0–2 összevont túlcsordulási vonala. Számítsd ki az UART0 RX vektortábla-bejegyzésének indexét és bájt-offszetét, és magyarázd el, milyen érték kerül a szóba akkor, ha a kezelőt `UART0_Handler` néven definiáljuk C-ben, és miért nem kell ehhez assembly kód. **2. (implementációs)** Egészítsd ki a példát úgy, hogy az ISR egy 64 elemű `volatile uint32_t` gyűrűpufferbe írjon mintát, a főciklus pedig a pufferből olvasson. Az ISR maradjon tisztán C, assembly trampolin nélkül. Indokold, hol kell a pending-jelzőt törölni, és miért nem elegendő az `NVIC_ICPR` írása. **3. (hibakeresési/mérési)** Egy kolléga a példa ISR-jében a `CMSDK_TIMER0_INTSTATUS = 1u;` sort az ISR végére tette, és azt állítja, hogy

emiatt a `main` ciklusa soha nem fut le, mert a megszakítás „viharként” újra és újra belép. Döntsd el, igaz-e ez a lassú, `0x00FFFFFF`-es periódus és a háromsoros ISR mellett. Fogalmazz hipotézist a tünet valódi okára, tervezz GDB-alapú adatgyűjtést (mely regisztereket és memóriaterületeket vizsgálod), és javasolj egyetlen változtatást az ellenőrzéssel együtt.

17.3.9 Megoldások

1. megoldás. Táblaindex $16 + 0 = 16$, offszet $4 \cdot 16 = 64 = 0\text{x}40$. Ugyanezt a vonalat használja az UART-driver (`UART0_IRQn = 0`) és a Záróprojekt II vektortáblája (`0x40: IRQ0 (UART0 RX)`); a `lab-check-vectors.py lab-uart.elf 16=UART0_Handler` ellenőrzés ugyanezt a slotot vizsgálja. A szóba a `UART0_Handler` függvény címe kerül (a Thumb-bittel együtt, amit a szimbólum hordoz). Nem kell assembly, mert a hardveres automatikus context-mentés pontosan a caller-saved regiszterhalmazt menti, azaz a C fordító számára az ISR belépése egy szokásos függvényhívásnak látszik; a visszatérést a fordító epilógusa (`bx lr` vagy `pop {..., pc}`) végzi, amely az LR-ben kapott `EXC_RETURN`-t a PC-be juttatja.

2. megoldás. A periféria pending-jelzőjét a kivételből való visszatérés előtt kell törölni; a pontos helyet a periféria eseményprotokollja dönti el. A timernél nincs kiolvasandó adat, a törlés az ISR-ben bárhol állhat a visszatérés előtt (a példában az elején). Az UART vételénél a sorrend számít: előbb az `INTSTATUS` törlése, utána a `DATA` olvasása (UART-driver). Az `NVIC_ICPR` írása azért nem helyettesíti ezt, mert a kérés forrása a periféria jelzője: amíg az be van állítva, a kéréssor továbbra is aktív, és az NVIC pending bitje azonnal újra beáll. A gyűrűpufferhez írás- és olvasásindexet külön kell kezelni (a `volatile` önmagában nem biztosít atomitást több mezőre), ezért az írásindex csak az ISR-ben módosul, az olvasásindex csak a főciklusban.

3. megoldás. Az állítás ebben a formában nem igaz. A CMSDK Timer0 megszakításkérése szintvezérelt: addig áll fenn, amíg az `INTSTATUS` jelző 1. Ha az ISR a jelzőt **a kivételből való visszatérés előtt** bárhol törli, a kérés a visszatéréskor már nem aktív, és az NVIC nem állítja újra pendingbe; a kezelő egyszer fut le eseményenként. Vihar csak akkor keletkezik, ha (1) a törlés egyáltalán nem történik meg vagy nem hat (rossz cím, rossz bit), (2) az ISR futása hosszabb, mint a timer periódusa, így a visszatérés előtt újabb esemény érkezik, vagy (3) a törlő írás a visszatérés pillanatában még nem ért el a perifériáig (írási puffer). Az utolsó esetre egy `dsb` a törlés után a szokásos óvintézkedés. Itt egyik sem áll fenn: a periódus `0x00FFFFFF` ciklus, az ISR néhány utasítás, tehát a késői, de visszatérés előtti nyugtázás szabályos. Hipotézis a tünetre: a törlés nem hat (például rossz regisztercím), vagy hiányzik. Adatgyűjtés: töréspont a `TIMER0_IRQHandler` utolsó utasításán, ott az `INTSTATUS` (`x/1xw 0x4000000C`) és az `IPSR` kiolvasása, a `g_tick` és a `g_events` figyelése. Ha az `INTSTATUS` a törlés után is 1, a törlés nem hat. Egy változtatás: a törlő írás címének és bitjének javítása (szükség esetén `dsb`-vel utána). Ellenőrzés: a `main` ciklus `g_events` értéke nő.

17.3.10 Rövid összefoglaló

Az ISR regisztrálása pusztán szimbólum- és címkérdés: a vektortábla $4 \cdot (16 + \text{IRQn})$ offszetén lévő szó a kezelő szimbólumának címét tartalmazza. Kivétel belépésekor a hardver automatikusan a verembe menti az R0–R3, R12, LR, PC és xPSR regisztereket (32 bájt), és az LR-be `EXC_RETURN` kódot tölt. Mivel ez pontosan a caller-saved halmaz, az egyszerű ISR tisztán C-ben megírható, assembly trampolin nélkül. A periféria pending-jelzőjét viszont az ISR-nek kell törölnie; elmaradása végtelen megszakítási vihart okoz.

Továbbhasznált definíciók:

- **ISR:** hardveresen meghívott, `EXC_RETURN`-nel visszatérő kezelőrutin.

- **automatikus context-mentés:** a caller-saved regiszterek és a folytatási állapot hardveres verembe írása kivétel belépésekor.
- **EXC_RETURN:** az LR-be töltött fenntartott bitminta; ha Handler módban egy megengedett utasítás (`bx lr`, `pop {..., pc}`, `ldm`, `ldr pc`) a PC-be tölti, kivétel-kilépés történik.

17.3.11 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Armv7-M Architecture Reference Manual, DDI0403E — a kivétel-belépés és -kilépés szekvenciája, a mentett keret elrendezése, az `EXC_RETURN` fenntartott értékei, az automatikus mentés regiszterlistája.
- Arm M-profile architektúra dokumentáció — a kivételmodell és a kezelőrutin-szerződés áttekintése.
- QEMU System Emulation dokumentáció — az `mps2-an385` machine és a perifériák modellezése.
- QEMU forráskód: hw/arm/mps2.c, hw/timer/cmsdk-apb-timer.c — a CMSDK APB Timer regiszteroffszetjei és a `CTRL` mezőkiosztás.

A laborkönyvtár összeállítását a helyettesítő eszközlánccal (Clang + `ld.lld`) ellenőriztük: egy üres könyvtárban, kizárólag a Vektortábla, a Linker script alapjai, a Startup kód és ennek az alfejezetnek a fájljaiból, a fenti `cp`/`cat`/`sed` lépésekkel a labor fordul és linkelődik, és a `lab-check-vectors.py` a Reset- és a használt IRQ-bejegyzést is a várt szimbólumra mutatónak találja. A `.rept` blokk nélküli táblánál ugyanez az ellenőrzés `HIBA [24]`-gyel bukik. **Ellenőrzési hiányok:** (1) A CMSDK Timer0 regiszteroffszetek (`INTSTATUS`, `RELOAD`) és a `CTRL` `ENABLE`/`IRQEN` bitpozíciói mérve QEMU 8.2.2-vel (a lektor mérése, `cmsdk-apb-timer`) helyesek: `CTRL` bit0 = ENABLE, bit3 = IRQEN; `0x08` = RELOAD, `0x0C` = INTSTATUS/INTCLEAR. (2) Az `IRQ8` hozzárendelés és a perifériacímek a QEMU `mps2-an385` machine definíciójából ellenőrzendők. (3) A `-nographic` és `-kernel` viselkedése, valamint a UART-alapú kimenet megjelenítése verziófüggő; a bemutatott parancssor mérve QEMU 8.2.2-vel (a lektor mérése) működik. (4) A QEMU-emuláció ciklusviszonyai nem hardverhűek, ezért a példában csak a viselkedésalapú elfogadási feltétel szerepel, és minden ciklusszám-jellegű adat levezetettként, nem mértként jelölt.

17.4 Megszakítás-latencia mérése és korlátai

17.4.1 Motiváció

Egy 10 kHz-es mintavételű motorvezérlőben az enkóder minden ciklusban megszakítást kér. A szabályozási törvény futási ideje kötött, de a bemenet a megszakítás megérkezése és az ISR első utasítása közötti idővel *késik*. Ha ez a késés túllépi a maradék időt, a hurok fázistartaléka elfogy, és a rendszer instabillá válik, a garantált válaszidő pedig elvész. Mérnöki döntéshez két dolog kell: egy **korlát**, amely a rendszer dokumentált feltételeiből (tiltási intervallumok, prioritások, a mag időzítése) következik, és egy **mért érték** a konkrét rendszerből. Ez az alfejezet egy illusztratív költségmodellt és egy mérési módszert ad, és megmutatja, mi kell ahhoz, hogy a modellből valódi felső korlát legyen, és — ami legalább ilyen fontos — megmutatja, miért nem fordítható a QEMU-ban mért ciklusszám hardverhű garanciává. Az NVIC konfiguráció és az ISR írása már megadta a mechanizmust; itt az időbeli viselkedését számszerűsítjük.

17.4.2 Definíció: mit nevezünk megszakítás-latenciának

Megszakítás-latencia (interrupt latency): az az eltelt idő, amely a megszakításkérés processzor számára láthatóvá válásától az ISR első utasításának végrehajtásáig telik el.

Két szintet különböztetünk meg, és a méréseket ehhez kell horgonyozni:

- L_{entry} — **kivétel-belépési latencia**: a kérés felismerésétől (a processzor utasításhatárhoz ér) az ISR első utasításáig.
- L_{total} — **teljes válaszidő**: a külső esemény (pl. GPIO-él) asszertálásától az ISR első utasításáig, azaz $L_{total} = L_{periph} + L_{entry}$, ahol L_{periph} a periféria saját jelzéskésleltetése.

Az alábbi levezetés L_{entry}-t modellezi; a mérés a szoftverben látható L_{total}-t (vagy annak GPIO-változatát) fogja rögzíteni, és a kettő közti különbséget kontrollkísérlettel választjuk szét.

17.4.3 A latencia összetevői

A belépés egy rögzített sorrendű, de részben adatfüggő útvonalon halad. Rétegenként:

#	Összetevő	Jelölés	Keletkezés	Tipikus ciklus
1	Kérés-asszertálás és szinkronizálás	C_{detect}	periféria jele a processzor óratartományába	1–2
2	Utasításhatár és pipeline-ürítés	C_{drain}	az aktuális utasítás befejezése/eldobása	0 – sok
3	Automatikus context-mentés	C_{stack}	8 szó ($R0$–$R3$, $R12$, LR, PC, $xPSR$) verembe írása	kb. 8 + wait state
4	Vektorfetch	C_{vec}	ISR-cím olvasása a vektortáblából	1 + wait state
5	Első utasítás fetch/dekód	C_{fetch}	pipeline feltöltése az ISR belépési pontján	kb. 2
6	Prioritás-arbitráció / preempciós várakozás	C_{arb}	azonos vagy nagyobb prioritás fut, tail-chaining	esetfüggő

A (3) lépés az automatikus context-mentés: a hardver a *caller-saved* halmazt menti, ezért az egyszerű ISR tisztán C-ben megírható. A (2) lépés a legváltozékonyabb, és a mag dokumentációja szerint három esetet kell elkülöníteni. **Megszakítható–folytatható** utasítás a Cortex-M3-on a több szót mozgató `LDM`, `STM`, `PUSH` és `POP`. Ha közben megszakítás érkezik, a mag az utasítást félbehagyja, a haladást (a következő átviendő regiszter sorszámát) az `EPSR` **ICI** (Interrupt-Continuable Instruction) mezőjében menti, és a kivételből visszatérve onnan folytatja. A latenciához tehát nem a teljes regiszterlista ideje adódik, legfeljebb a már elindított egyetlen átvitel befejezése. Az ICI-mező ugyanazokon az `EPSR`-biteken osztozik az IT-blokk állapotával, ezért a kettő egyszerre nem lehet érvényes. Egy IT-blokkon belüli `LDM`/`STM` nem folytatható: megszakításkor eldobódik, és visszatérés után **elölről kezdődik** (újrakezdhető eset). **Valóban nem megszakítható** a már kiadott buszátvitel: egy lassú perifériára vagy wait state-es memóriára induló olvasást vagy

írást a mag kivárja. Ez a C_{drain} valódi, memóriarendszertől függő része. A pontos szabályokat (például a többciklusú osztás kezelését) a konkrét mag TRM-je rögzíti, ezért a modellbe az adott mag szabálya kerüljön. A (6) lépésnél a tail-chaining különleges. Ha a futó kezelő kilépésekor van olyan pending kivétel, amely kiszolgálható (akár a futó ISR alatt, akár a kilépés közben vált pendingé), a mag a vermet nem állítja vissza és nem menti újra, hanem közvetlenül a következő kezelőbe lép. Ez a láncolt útvonal rövidebb, mint egy Thread módból induló teljes belépés.

17.4.4 Levezetés: worst-case becslés

Az alábbi számolás **illusztratív költségmodell**, nem garantált felső korlát. Feltevései:

Feltevés	A modellben	Ha nem teljesül
a kérés nincs maszkolva	nincs `PRIMASK`/`BASEPRI`-szakasz, nem fut azonos vagy magasabb prioritású kezelő	a teljes maszkolt, illetve foglalt idő hozzáadódik
nincs interferáló kivétel	a mérési ablakban nem érkezik magasabb prioritású kérés	minden ilyen kezelő teljes futásideje hozzáadódik
memóriarendszer	zéró-wait-state SRAM a veremnek és a vektortáblának	wait state-ek szavanként, buszkonfliktus
az összetevők additívak	a lenti tagok egyszerűen összeadódnak	a Cortex-M3-on a stacking (adatbusz) és a vektorlekérés (utasításbusz) átfedheti egymást, ezért a valós belépés rövidebb lehet a tagok összegénél
C_{drain}	az éppen futó utasítás befejezése vagy eldobása	`LDM`/`STM` folytatható, nem a teljes listát kell kivárni; egy elindított lassú buszátvitelt igen

A modell tagjai ciklusban:

$$L_{entry}^{modell} = C_{detect} + C_{drain} + 8\,(1 + N_s) + (1 + N_v) + C_{fetch} + C_{arb}$$

ahol N_s a verem memóriájának wait state-száma szavanként, N_v a vektortábla wait state-száma.

Illusztratív számokkal ($f_c = 25\,\text{MHz}$):

- **Kedvező eset** (rövid ALU-utasítás, zéró-wait): $1+0+8+1+2+0 = 12$ ciklus $\Rightarrow 0{,}48\,\mu s$.
- **Kedvezőtlenebb feltételezett eset** ($C_{detect} = 2$, $C_{drain} = 12$, $C_{arb} = 4$): $2 + 12 + 8 + 1 + 2 + 4 = 29$ ciklus $\Rightarrow 1{,}16\,\mu s$.

A 29 ciklus a táblázat feltevései mellett szemléltet, **nem garancia**. Az, hogy a kedvező eset összege 12, egybeesik a Cortex-M3-ra dokumentált idealizált belépési idővel. Ez azonban nem igazolja a modellt, mert a tagok a valóságban részben átfedik egymást. A belépés saját idejét (L_{entry}^{mag}) a konkrét mag TRM-jéből és memóriarendszeréből kell venni.

Mi kell valódi felső korláthoz? A korlátnak a rendszer dokumentált körülményeiből kell következnie:

$$L^{wc} \le T^{max}_{maszk} + \sum_{j \in HP} n_j \cdot C_j + C^{max}_{busz} + L^{mag}_{entry}$$

ahol T^{max}_{maszk} a leghosszabb olyan intervallum, amikor a kérés nem léphet be (`PRIMASK`/`BASEPRI`-szakasz, vagy egy azonos vagy magasabb prioritású kezelő futása), HP a mérési ablakban beérkezhető magasabb prioritású kivételek halmaza, n_j a legnagyobb előfordulásuk, C_j a kezelőjük WCET-je, C^{max}_{busz} a leghosszabb elindított, nem megszakítható buszátvitel. Egy 200 ciklusos `PRIMASK`-szakasz és egyetlen 150 ciklusos, magasabb prioritású ISR esetén a korlát már több mint 350 ciklus. A 29 ciklus ilyen környezetben nem érvényes. A korlát tagjait útvonalanként kell meghatározni, ahogy az ISR-ek WCET-jét a Záróprojekt II auditja is kezeli.

17.4.5 Mérési módszer: trigger- és ISR-időbélyeg

A latenciát két időbélyeg különbségeként mérjük. Időalapnak a trace órajeléhez kötött ciklusszámlálót (`DWT_CYCCNT`) használjuk, amelyet a `DEMCR.TRCENA` és a `DWT_CTRL.CYCCNTENA` bit engedélyez (az időmérés statisztikai módszertanát lásd a 11.4 és a 13.4 alfejezetben). A mérés:

- t_0 — a kiváltó írás **előtt** rögzített ciklusszám,
- t_1 — az ISR első utasításaként rögzített ciklusszám,
- $L_{measured} = t_1 - t_0$.

Fontos, hogy t_1 az ISR lehető legelső művelete legyen. A C-függvény első kifejezése előtt azonban a fordító prológust is generálhat; ez és az időbélyeg kiolvasása a mért érték része, ezért a disassemblyt is vizsgálni kell. $N = 256$ ismétléssel hisztogramot építünk, mert egyetlen minta nem jellemzi a viselkedést.

17.4.6 Kidolgozott példa

Munkakönyvtár: `~/arm-labor/cm/a071/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a071
cd ~/arm-labor/cm/a071
```

A kód a memóriatérkép szerinti SCS-címeket használja. A vektortábla külső IRQ6 bejegyzése a `lab_irq_isr` kezelőre mutat; ezt a vonalat a kontrollkísérlethez fenntartjuk.

A linker script és a startup a II. rész kanonikus párja (Linker script alapjai, Startup kód), a vektortábla ellenőrzője pedig a Vektortábla alfejezet `lab-check-vectors.py`-ja. A linker script változatlan; a startup másolatának vektortábláját egyetlen, gépileg alkalmazható lépéssel bővítjük a tesztvonal (IRQ6) bejegyzésével:

```
cp ~/arm-labor/cm/a065/lab-mps2.ld .
cp ~/arm-labor/cm/a066/lab-startup.s lab-a071-startup.s
cp ~/arm-labor/cm/a064/lab-check-vectors.py .
cat > lab-a071-irq.inc <<'EOF'
    /* 16-tól: külső IRQ-k; a tesztvonal IRQ6 = 22. index, előtte 6 helyet ki kell tölteni */
    .rept 6
    .word Default_Handler         /* 0x40-0x54: IRQ0-IRQ5 (itt nem használt) */
    .endr
    .word lab_irq_isr             /* 0x58: IRQ6 (szoftveres NVIC-pending)    */
EOF
sed -i '/\.word SysTick_Handler/r lab-a071-irq.inc' lab-a071-startup.s   # a SysTick sor után
# gyenge alias: kezelő nélkül is linkelhető
```

```
echo '    lab_default_handler lab_irq_isr' >> lab-a071-startup.s
# ellenőrzés: a bővített tábla vége
sed -n '/\.word SysTick_Handler/,/^$/p' lab-a071-startup.s
```

A `sed` mintája csak a vektortábla `.word SysTick_Handler` sorára illeszkedik, a fájl végén lévő `lab_default_handler SysTick_Handler` aliasra nem. A fel nem használt IRQ-helyek a kanonikus `Default_Handler`-re mutatnak. Az `echo` sor a kanonikus `lab_default_handler` makróval a fájl végére egy **gyenge**, `Default_Handler`-re mutató aliast fűz a használt kezelőhöz is. A C-fájl erős definíciója ezt felülírja, de ha egy build a kezelőt nem tartalmazza, a `KEEP`-pel megtartott vektortábla hivatkozása akkor sem marad feloldatlan. Ezek **valódi táblaszavak**, nem megjegyzések, mert csak így kerül a használt kezelő a megfelelő indexre ($16 + \mathrm{IRQn}$).

A példa szoftveresen állítja pending állapotba az NVIC-et, így nem feltételez GPIO-visszacsatolást. GPIO-éltől mért teljes latenciához külön, valóban implementált periféria és dokumentált bekötés szükséges. A QEMU 8.2.2 AN385 modellje nem implementálja a GPIO-t, és a DWT számlálója sem halad; itt a mérés `SKIP`.

```
/* lab-a071-latency.c - szoftveres NVIC-pending latenciapróba (IRQ6),
 * megfigyelhető PASS/FAIL/SKIP kimenettel (semihosting) */
#include <stdint.h>
#define REG32(a) (*(volatile uint32_t *)(uintptr_t)(a))

#define DEMCR       REG32(0xE000EDFCu)
#define DWT_CTRL    REG32(0xE0001000u)
#define DWT_CYCCNT  REG32(0xE0001004u)
#define NVIC_ISER0  REG32(0xE000E100u)
#define NVIC_ISPR0  REG32(0xE000E200u)
#define TEST_IRQ    6u
#define N           256u

static volatile uint32_t t0, t1, done;
static volatile uint32_t hist[32], above_hist;

void lab_irq_isr(void)
{
    t1 = DWT_CYCCNT;
    uint32_t d = t1 - t0;
    if (d < 32u) hist[d]++;
    else above_hist++;
    done++;
}

static void sh_call(uint32_t op, const void *arg)
{
    register uint32_t r0 asm("r0") = op;
    register const void *r1 asm("r1") = arg;
    asm volatile ("bkpt 0xab" : "+r"(r0) : "r"(r1) : "memory");
}

_Noreturn static void finish(const char *line, int fail)
{
    sh_call(0x04u, line);                                   /* SYS_WRITE0 */
    sh_call(0x18u, (const void *)(fail ? 0x20023u : 0x20026u));    /* SYS_EXIT   */
    for (;;) { }
}

/* Egyetlen szoftveres pending; 1, ha a kezelő a korláton belül lefutott. */
static int trigger_once(void)
{
    uint32_t before = done;
    t0 = DWT_CYCCNT;
```

```
    NVIC_ISPR0 = 1u << TEST_IRQ;
    __asm volatile("dsb\n\tisb" ::: "memory");
    for (uint32_t limit = 1000000u; done == before && limit; --limit) { }
    return done != before;
}

int main(void)
{
    NVIC_ISER0 = 1u << TEST_IRQ;

    /* 1. Funkcionális próba, a DWT-től függetlenül: eléri-e a kérés a kezelőt? */
    if (!trigger_once())
        finish("LATENCY: FAIL (az IRQ6 kérés nem érte el a lab_irq_isr-t)\n", 1);

    /* 2. Halad-e a ciklusszámláló? Ha nem, mérni nem lehet: SKIP. */
    DEMCR |= 1u << 24;                    /* TRCENA    */
    DWT_CYCCNT = 0;
    DWT_CTRL |= 1u;                       /* CYCCNTENA */
    __asm volatile("dsb\n\tisb" ::: "memory");
    uint32_t before = DWT_CYCCNT;
    for (unsigned i = 0; i < 64; ++i) __asm volatile("nop");
    if (DWT_CYCCNT == before)
        finish("LATENCY: SKIP (a DWT CYCCNT nem halad; az IRQ6 kiszolgálása működik)\n", 0);

    /* 3. Mérés: N minta hisztogramba. */
    for (unsigned k = 0; k < 32; ++k) hist[k] = 0;
    above_hist = 0;
    for (uint32_t i = 0; i < N; i++)
        if (!trigger_once())
            finish("LATENCY: FAIL (elmaradt megszakítás a mérés közben)\n", 1);
    uint32_t sum = above_hist;
    for (unsigned k = 0; k < 32; ++k) sum += hist[k];
    if (sum != N)
        finish("LATENCY: FAIL (hist + above_hist != 256)\n", 1);
    finish("LATENCY: PASS (256 minta; a hist[] GDB-vel olvasható)\n", 0);
}
```

Build, a vektortábla gépi ellenőrzése és futtatás (fájlok: `lab-mps2.ld`, a fent bővített `lab-a071-startup.s`, `lab-a071-latency.c`, `lab-check-vectors.py`):

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -nostdlib -ffreestanding \
  -T lab-mps2.ld lab-a071-startup.s lab-a071-latency.c -o lab-a071.elf
python3 lab-check-vectors.py lab-a071.elf 1=Reset_Handler 22=lab_irq_isr
timeout 20 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -nographic -semihosting \
  -kernel lab-a071.elf; echo "kilépési állapot: $?"
```

A labor **szoftveres pending-próba**: a kérést az `NVIC_ISPR0` írása adja, periféria nélkül. A GPIO-éltől mért teljes latencia külön, hardveres kiterjesztés, amelyhez valóban implementált GPIO és dokumentált bekötés kell; a QEMU 8.2.2 AN385 modelljében ez nem érhető el.

A program három kimenetet ad, mindegyiket semihostingos kiírással és szabályos kilépéssel:

Eredmény	Feltétel	Kilépési állapot
`LATENCY: FAIL (az IRQ6 kérés nem érte el a lab_irq_isr-t)`	a funkcionális próba sikertelen: az IRQ nincs engedélyezve, vagy a kezelő nem fut le a korláton belül	1
`LATENCY: SKIP (a DWT CYCCNT nem halad; …)`	a kezelés működik, de a ciklusszámláló nem halad, ezért mérni nem lehet	0

Eredmény	Feltétel	Kilépési állapot
`LATENCY: PASS (256 minta; …)`	működő DWT, 256 minta, `hist + above_hist = 256`	0

A funkcionális próba a DWT-től függetlenül fut, ezért a hiba és a mérhetetlenség nem keveredik össze. Ha a vektor rossz helyre mutat, és a kérés a `Default_Handler` végtelen ciklusába fut, a program nem jut el a kiírásig. Ezt a futtatóparancs `timeout`-ja jelzi 124-es állapottal, a `lab-check-vectors.py` pedig már futtatás előtt kiszűri. A `done` várakozása elkülöníti a mintákat; az egymillió iterációs korlát hiányzó megszakításnál megakadályozza a beragadást.

Eseménysorrend és a mért metrika. A `trigger_once` mintáiban az események sorrendje rögzített: (1) Thread módban kiolvassuk a t_0-t; (2) az `NVIC_ISPR0` írása után az IRQ6 **pending**; (3) a mag kiszolgálja a kérést, és belép a `lab_irq_isr`-be; (4) a kezelő első mérőpontja kiolvassa a t_1-et. A $t_1 - t_0$ tehát egy **Thread módból indított szoftverművelet válaszideje**: benne van a t_0 kiolvasása utáni utasítások, az `NVIC_ISPR0`-írás buszátvitele és maga a belépés. Egyetlen magon a Thread-kód nem fut, amíg egy kezelő aktív. Ezért egy magasabb prioritású kezelő (például egy hosszú `SysTick`) csak két helyen növelheti ezt az intervallumot: ha a kivétel az (1) és a (2) közé ékelődik, vagy ha a (2) és a (3) között vele egy időben pending, és nagyobb prioritása miatt előbb kap sorra. A Thread-trigger tehát **nem** érkezhet egy már futó kezelő belsejébe, és egy aktív ISR hátralévő idejének eloszlását sem méri. Ehhez a kérést a kezelő ismert pontján kell kiadni, ahogy a 2. feladat kontrollált változata teszi.

17.4.7 Várt kimenet és ellenőrzés

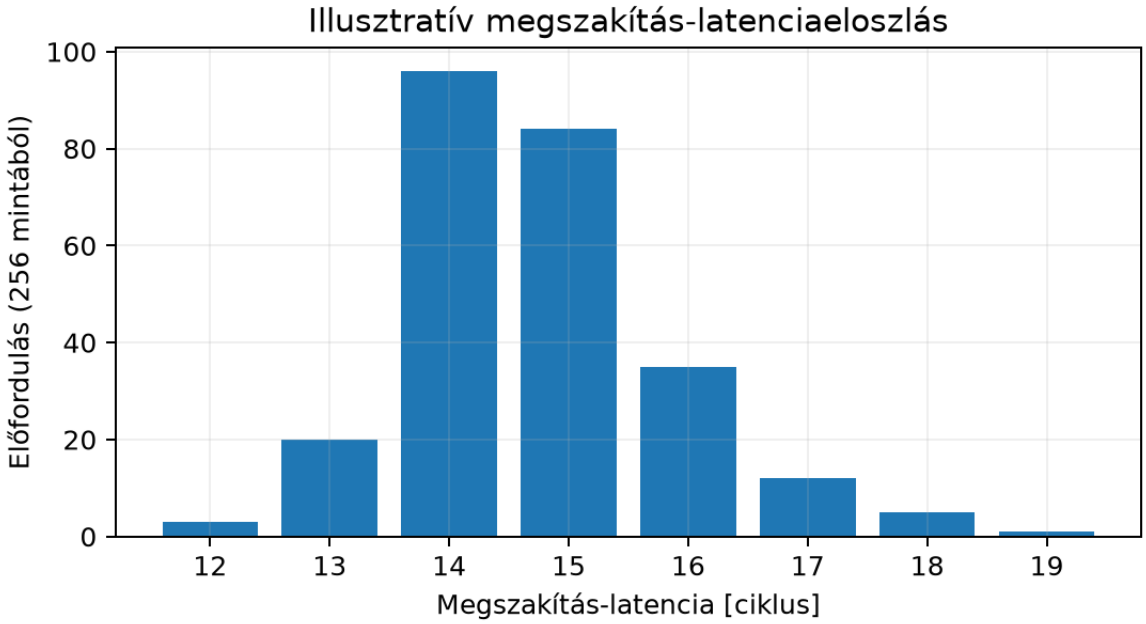
A `lab-a071.elf` a fő úton, mérve (GNU + QEMU 8.2.2, a lektor mérése), ezt adja (a DWT itt nem halad, ezért a ciklusszámok helyett a funkcionális ellenőrzés eredménye látszik):

```
LATENCY: SKIP (a DWT CYCCNT nem halad; az IRQ6 kiszolgálása működik)
kilépési állapot: 0
```

Ellenpróbaként az `NVIC_ISER0` írását kihagyva `LATENCY: FAIL (az IRQ6 kérés nem érte el a lab_irq_isr-t)` és 1-es állapot, a 22. vektort `Default_Handler`-re cserélve az emulátor időkorlátja (124) volt az eredmény.

Működő DWT mellett a `hist[]` tömböt GDB-vel olvassuk ki. Illusztratív, levezetett eloszlás:

Latencia [ciklus]	12	13	14	15	16	17	18	19
Előfordulás	3	20	96	84	35	12	5	1



17.2. ábra: Illusztratív megszakítás-latenciaeloszlás - Hipotetikus, levezetett eloszlás, nem mért adat. QEMU 8.2.2 AN385 alatt a CYCCNT-mérés nem támogatott.

Működő DWT esetén az ellenőrzés: $\sum hist + above_hist = done = 256$. A `above_hist` a 31 ciklusnál nagyobb mintákat is megőrzi darabszámként. A fenti számszerű eloszlás hipotetikus szemléltetés; sem az NVIC-kontroll, sem a GPIO-mérés tényleges eredményét nem állítja.

17.4.8 A QEMU-korlát: miért nem hardverhű a mért érték

A QEMU alapértelmezésben **funkcionális emulátor**, nem cikluspontos szimulátor. A QEMU 8.2.2 AN385 modelljében a `DWT_CYCCNT` nem növekszik, ezért itt érvényes ciklusszámmérés sincs. Más verzión előbb a számláló működését kell ellenőrizni; a működő emulált számláló sem bizonyít cikluspontos pipeline-modellt. `-icount` módban a virtuális óra determinisztikussá tehető, de a modell továbbra sem reprodukálja a busz-várakozási állapotokat, a flash-prefetch viselkedést, a soros hozzáférési konfliktusokat és a valódi pipeline-stallokat. Következmények:

- A mért ciklusszám **relatív** összehasonlításra és a *módszer* validálására alkalmas, abszolút garanciára nem.
- A DWT/CYCCNT támogatás szintje QEMU-verziónként eltér — **ellenőrizendő**, mielőtt belőle következtetünk.
- A fizikai STM32 Nucleo eredménye ettől eltérhet; a kettőt mindig külön kell jelölni, és a QEMU-érték nem helyettesíti a hardvermérést.

17.4.9 Tipikus hibák és ellenpróbák

1. **A `CYCCNT` mindig 0.** Hipotézis: nem állítottuk a `DEMCR.TRCENA`-t vagy a `DWT_CTRL.CYCCNTENA`-t, vagy a QEMU nem modellezi. Adatgyűjtés: GDB-vel olvassuk ki a regisztereket. Egy változtatás: engedélyezzük a biteket; ha így sem nő, a modell a korlát.

2. **(Hardveres GPIO-kiterjesztés) Megszakítási vihar, a minta „beragad”.** A GPIO-forrás pending-jelzőjének törlése (`GPIO_INTCLEAR`) elmaradt (l. 17.3). A szoftveres pending-próbában ez nem fordulhat elő, mert ott nincs periféria-forrás. Ellenpróba: a kezelő a visszatérés előtt törölje a forrás jelzőjét.
3. **Több csúcsú hisztogram.** Több klaszter több okból is keletkezhet: eltérő C_{drain} (az éppen futó utasítás), wait state-ek, a mérési kód saját változékonysága vagy egy láncolt belépés. Egy csúcsot nem lehet pusztán a kisebb értéke miatt tail-chainingnek tekinteni. Az okot kontrollkísérlet választja szét (lásd a 3. feladatot).
4. **(Hardveres GPIO-kiterjesztés) Minden latencia azonos.** A mérés nagy része a kiváltó írás ideje, nem a belépés. Kontrollkísérlet: a GPIO-él helyett a szoftveres `NVIC_ISPR0` pending, mint az alaplaborban. A két eloszlás különbsége izolálja a periféria-asszertálást (L_{periph}).

17.4.10 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Sorold fel a latencia hat összetevőjét, és jelöld meg mindegyiknél, hogy a feltételek függvényében *konstans* vagy *változó*. Indokold, miért a C_{drain} és a C_{arb} a legkevésbé kiszámítható, és miért nem egy 8 regiszteres `LDM` teljes ideje kerül a C_{drain}-be. **2. (implementációs, elmélyült — cél: rutinépítés)** Egészítsd ki a kódot egy nagyobb prioritású `SysTick` ISR-rel, amely ismert mennyiségű munkát végez. A kérést most **ne Thread módból** add ki: a `SysTick` kezelő a munkája 25, 50, illetve 75%-ánál rögzítse a t_0-t, és ott pendingelje az alacsonyabb prioritású IRQ6-ot. Igazold a jegyzőkönyvben a t_0, a pendinggé válás és az IRQ6 első mérőpontjának sorrendjét, mérd meg (működő DWT mellett) a késést a három ponton, és vesd össze a hátralevő munkával, valamint a korlátképlet $\sum n_j C_j$ tagjával. **3. (hibakeresési/mérési, elmélyült — cél: rutinépítés)** Egy működő DWT-vel mért hisztogram három csúcsot mutat (12, 14 és 20 ciklus körül). Fogalmazz **több** hipotézist a csúcsok okára. Tervezz kontrollált, két IRQ-s kísérletet ismert prioritásokkal és pending állapotokkal, amelyben egy láncolásos és egy láncolás nélküli út is dokumentáltan előáll, és döntsd el, melyik csúcs melyik úthoz tartozik.

17.4.11 Megoldások

1. megoldás — Segítség. Nézd meg, mely összetevők függenek az ÉPPEN futó utasítástól vagy az ÉPPEN futó másik kezelőtől, és melyek csak a core felépítésétől.

1. megoldás — Teljes megoldás. Konstans jellegű: C_{detect} és C_{fetch} adott core-on nagyjából állandó. Változó: C_{drain} az aktuális utasítástól függ, C_{stack}/C_{vec} a memória wait state-jeitől, C_{arb} a futó kezelőtől. A C_{drain} azért kiszámíthatatlan, mert a megszakítás bármely utasítás közben érkezhet; a C_{arb} azért, mert a tail-chaining és a preempció időzítésfüggő. Egy `LDM/STM` nem a teljes regiszterlista idejével növeli a C_{drain}-t, mert a Cortex-M3 félbehagyja, a haladást az `EPSR` ICI-mezőjében menti, és a visszatérés után folytatja. Csak az éppen elindított átvitelt kell kivárni. IT-blokkon belül nincs ICI, mert az IT-állapot ugyanazokat a biteket használja: ott az utasítás eldobódik, és elölről kezdődik.

2. megoldás — Segítség. Amíg a magasabb prioritású ISR fut, az IRQ6-kérés nem léphet be. Ha a kérést a `SysTick` kezelő maga adja ki egy ismert ponton, mennyi munka van még hátra a belépésig? És miért nem lehet ugyanezt a Thread módból indított `trigger_once`-szal előállítani?

2. megoldás — Teljes megoldás. A Thread módból indított `trigger_once` erre a kérdésre nem alkalmas. Amíg a `SysTick` fut, a Thread-kód áll, így a kérés nem eshet a kezelő „véletlen pontjára”. A hosszú `SysTick` legfeljebb a t_0 és az `ISPR`-írás közé, vagy a pending és a belépés közé

ékelődik (lásd a Kidolgozott példa eseménysorrendjét), és akkor a mért érték nem a hátralévő ISR-idő. A kontrollált változatban ezért a **kérést a magasabb prioritású kezelő adja ki**, egy ismert ponton. A `SysTick` (nyers prioritás `0x20`) `WORK = 400` munkaegységet végez; a `req_point`-adik egységnél rögzíti a t_0-t, és az `NVIC_ISPR0`-val pendingeli az IRQ6-ot (nyers prioritás `0x80`). Az IRQ6 a `SysTick` alatt pending marad, és csak annak kilépésekor, láncolva lép be. A `SysTick`-et a főprogram az `ICSR.PENDSTSET` bittel indítja, így a kezelő futásának kezdete is a programhoz kötött:

```c
/* lab-a071-systick.c - kontrollált preempciós próba (2. feladat): az IRQ6-kérést
 * a magasabb prioritású SysTick kezelő ismert pontján adjuk ki */
#include <stdint.h>
#define REG32(a) (*(volatile uint32_t *)(uintptr_t)(a))

#define DEMCR       REG32(0xE000EDFCu)
#define DWT_CTRL    REG32(0xE0001000u)
#define DWT_CYCCNT  REG32(0xE0001004u)
#define NVIC_ISER0  REG32(0xE000E100u)
#define NVIC_ISPR0  REG32(0xE000E200u)
#define NVIC_IPR1   REG32(0xE000E404u)    /* IRQ4..IRQ7 prioritásbájtjai     */
#define SCB_ICSR    REG32(0xE000ED04u)    /* bit 26: PENDSTSET               */
#define SCB_SHPR3   REG32(0xE000ED20u)    /* bit 31:24: a SysTick prioritása */
#define TEST_IRQ    6u
#define WORK        400u                  /* a SysTick kezelő munkaegységei  */

static volatile uint32_t work, req_at, entry_at, t0, t1, done, req_point;

void SysTick_Handler(void)                /* nyers prioritás 0x20: magasabb  */
{
    for (uint32_t i = 0; i < WORK; i++) {
        if (i == req_point) {             /* ismert pont: időbélyeg, kérés   */
            req_at = work;
            t0 = DWT_CYCCNT;
            NVIC_ISPR0 = 1u << TEST_IRQ;  /* alacsonyabb prioritás: pending  */
        }
        work++;                           /* egy munkaegység                 */
    }
}

void lab_irq_isr(void)                    /* IRQ6, nyers prioritás 0x80      */
{
    t1 = DWT_CYCCNT;
    entry_at = work;
    done++;
}

static void sh_call(uint32_t op, const void *arg)
{
    register uint32_t r0 asm("r0") = op;
    register const void *r1 asm("r1") = arg;
    asm volatile ("bkpt 0xab" : "+r"(r0) : "r"(r1) : "memory");
}

static void put(const char *s) { sh_call(0x04u, s); }                /* SYS_WRITE0 */

static void put_u32(uint32_t v)
{
    char buf[11], *p = buf + sizeof buf - 1;
    *p = '\0';
    do { *--p = (char)('0' + v % 10u); v /= 10u; } while (v);
    put(p);
}
```

```
_Noreturn static void finish(const char *line, int fail)
{
    put(line);
    sh_call(0x18u, (const void *)(fail ? 0x20023u : 0x20026u));    /* SYS_EXIT   */
    for (;;) { }
}

int main(void)
{
    SCB_SHPR3 = (SCB_SHPR3 & 0x00FFFFFFu) | (0x20u << 24);
    NVIC_IPR1 = (NVIC_IPR1 & ~(0xFFu << 16)) | (0x80u << 16);   /* IRQ6: bit 23:16  */
    NVIC_ISER0 = 1u << TEST_IRQ;

    DEMCR |= 1u << 24;                    /* TRCENA    */
    DWT_CYCCNT = 0;
    DWT_CTRL |= 1u;                       /* CYCCNTENA */
    __asm volatile("dsb\n\tisb" ::: "memory");
    uint32_t c0 = DWT_CYCCNT;
    for (unsigned i = 0; i < 64; ++i) __asm volatile("nop");
    int dwt = DWT_CYCCNT != c0;

    static const uint32_t pct[3] = { 25u, 50u, 75u };
    uint32_t lat[3];
    for (unsigned k = 0; k < 3; k++) {
        req_point = pct[k] * WORK / 100u;
        work = 0;
        uint32_t before = done;
        SCB_ICSR = 1u << 26;              /* SysTick szoftveres pendingelése */
        __asm volatile("dsb\n\tisb" ::: "memory");
        for (uint32_t limit = 1000000u; done == before && limit; --limit) { }
        if (done == before)
            finish("PREEMPT: FAIL (az IRQ6 nem futott le)\n", 1);
        /* Sorrend: t0 és a kérés a SysTick-ben, belépés csak a SysTick után. */
        if (entry_at != WORK || req_at != req_point)
            finish("PREEMPT: FAIL (az IRQ6 a SysTick vége előtt lépett be)\n", 1);
        lat[k] = t1 - t0;
        put("p = ");
        put_u32(pct[k]);
        put("%: hátralevő munka = ");
        put_u32(entry_at - req_at);
        put(" egység, t1 - t0 = ");
        if (dwt) { put_u32(lat[k]); put(" ciklus\n"); }
        else put("SKIP (a DWT CYCCNT nem halad)\n");
    }
    if (!dwt)
        finish("PREEMPT: PASS (sorrend), késés: SKIP (a DWT CYCCNT nem halad)\n", 0);
    if (!(lat[0] > lat[1] && lat[1] > lat[2]))
        finish("PREEMPT: FAIL (a késés nem csökken a hátralevő munkával)\n", 1);
    finish("PREEMPT: PASS (sorrend és csökkenő késés)\n", 0);
}
```

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -nostdlib -ffreestanding \
  -T lab-mps2.ld lab-a071-startup.s lab-a071-systick.c -o lab-a071-systick.elf
python3 lab-check-vectors.py lab-a071-systick.elf \
  1=Reset_Handler 15=SysTick_Handler 22=lab_irq_isr
timeout 20 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -nographic -semihosting \
  -kernel lab-a071-systick.elf; echo "kilépési állapot: $?"
```

A `SysTick_Handler` a kanonikus startupban gyenge alias, ezért a C-fájl erős definíciója kerül a 15. vektorba; a `lab-a071-startup.s` változatlanul az alaplaboré. A **sorrendet** a program a DWT-től függetlenül ellenőrzi: a kérés pillanatában a `req_at` a kérési pontot rögzíti, a belépéskor az `entry_at` pedig a teljes `WORK`-öt. Ha az IRQ6 a `SysTick` vége előtt lépne be (például mert a prioritása magasabb lett), `PREEMPT: FAIL` a kimenet. A sorrend tehát: $t_0 \rightarrow$ pending (a kezelőn

belül) → a `SysTick` hátralevő $(1-p)\cdot$`WORK` egysége → kilépés és láncolt belépés → t_1. Ugyanez a próba a fő úton, mérve (GNU + QEMU 8.2.2, a lektor mérése), ezt adja:

```
p = 25%: hátralevő munka = 300 egység, t1 - t0 = SKIP (a DWT CYCCNT nem halad)
p = 50%: hátralevő munka = 200 egység, t1 - t0 = SKIP (a DWT CYCCNT nem halad)
p = 75%: hátralevő munka = 100 egység, t1 - t0 = SKIP (a DWT CYCCNT nem halad)
PREEMPT: PASS (sorrend), késés: SKIP (a DWT CYCCNT nem halad)
kilépési állapot: 0
```

Ellenpróbaként az IRQ6 nyers prioritását `0x00`-ra (a `SysTick`-nél magasabbra) állítva a kimenet `PREEMPT: FAIL (az IRQ6 a SysTick vége előtt lépett be)`, 1-es állapottal (mérve, GNU + QEMU 8.2.2, a lektor mérése): ilyenkor a kérés azonnal preemptál.

Működő DWT mellett a $t_1 - t_0$ a hátralevő munkával arányosan csökken, ezt a program a 25 → 50 → 75%-os pontok szigorúan csökkenő késéseként ellenőrzi. Közelítőleg $L(p) \approx (1-p)\cdot C_{SysTick} + C_{kilps+lnc}$, ahol $C_{SysTick}$ a kezelő teljes munkarészének futásideje. A legnagyobb érték $p \to 0$ esetén a teljes $C_{SysTick}$-hez közelít: ez a korlátképlet $n_j \cdot C_j$ tagja, $n_j = 1$-gyel. A mérés tehát azt mutatja meg, hogy a 29 ciklusos illusztratív modell ilyen környezetben nem korlát. A tényleges ciklusszámokat a fenti futásban nem mértük (a DWT nem halad), ezért itt csak a sorrend és a hátralevő munka igazolt. Egy aszinkron, a Thread-kódtól független forrás (például egy timer, amely a `SysTick`-kel nem szinkron) véletlen érkezést ad. Ott a kérés a futó kezelő bármely pontjára eshet, és a késés eloszlása a hátralevő idő eloszlását követi. A t_0-t ekkor a forrás eseményéhez kell kötni (például a timer számlálójából visszaszámolva), nem egy Thread-időbélyeghez.

3. megoldás — Segítség. A három szám önmagában nem dönti el az okot. Milyen körülmény hozna létre biztosan láncolt belépést, és milyen biztosan láncolás nélkülit? Melyik kezelő kilépésekor kell a másik kérésnek már pendingnek lennie?

3. megoldás — Teljes megoldás. Lehetséges hipotézisek: (a) eltérő C_{drain}, mert a trigger utáni utasítás (például a várakozó ciklus `ldr`-je vagy egy elindított buszátvitel) más-más ponton van; (b) wait state-ek vagy buszkonfliktus a stacking és a vektorlekérés alatt; (c) a mérési kód változékonysága (a `t0` olvasása és az `ISPR`-írás sorrendje, a kezelő prológusa); (d) láncolt belépés. A (d) csak akkor lehetséges, ha a mért kérés egy **másik** kezelő kilépésekor már pending. Az alappróbában ez nem áll fenn: egyetlen IRQ indul Thread módból, és a főciklus megvárja a kiszolgálását. Kontrollkísérlet két IRQ-val (például IRQ5 és IRQ6, azonos csoportprioritással, hogy egymást ne preemptálják). **Láncolásos út:** az IRQ5 kezelője a visszatérése előtt `NVIC_ISPR0`-val pendingbe teszi az IRQ6-ot, és rögzíti a saját kilépési időbélyegét. Az IRQ6 így a kilépéskor kiszolgálható pending kivétel, és láncolva lép be. **Láncolás nélküli út:** az IRQ6-ot a főciklus pendingeli Thread módból, mint az alappróbában. A két út latenciáját ugyanazzal a módszerrel, külön hisztogramba mérjük. Ha a láncolásos út eloszlása egy meglévő csúcsra esik, a láncolás nélküli pedig nem, az a (d) hipotézist támogatja; ha a két út eloszlása azonos, a csúcsokat más ok (a)–(c) adja. A csúcsot tehát a kísérleti út, nem a viszonylagos értéke azonosítja.

17.4.12 Rövid összefoglaló

A megszakítás-latencia hat összetevőre bontható: kérés-szinkronizálás, pipeline-ürítés, automatikus context-mentés (8 szó), vektorfetch és az első utasítás betöltése, valamint prioritás-arbitráció; ezek összege adja a worst-case korlátot. A mérés két időbélyeg (trigger előtti t_0 és ISR-belépési t_1) különbsége `DWT_CYCCNT`-vel, hisztogramként kiértékelve. A QEMU-érték relatív összehasonlításra jó, hardverhű garanciára nem.

Továbbhasznált definíciók:

- **megszakítás-latencia:** a kérés láthatóvá válásától az ISR első utasításáig eltelt idő.

- L_{entry}**:** a kivétel-belépés latenciája (felismeréstől az első ISR-utasításig).
- **tail-chaining:** a kilépés alatt érkező kivétel újramentés nélküli belépése.

17.4.13 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Armv7-M Architecture Reference Manual, DDI0403E — a kivétel-belépés szekvenciája, a mentett keret, az `EXC_RETURN`, a tail-chaining és a prioritás-arbitráció. *Ellenőrizve: a link él* (a `developer.arm.com` → `support.arm.com` átirányítás megerősített); a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Arm M-profile architektúra dokumentáció — a kivételmodell és a kezelőrutin-szerződés. *Ellenőrizve: a link él*; ugyanaz a JS-renderelési korlát érvényes.
- QEMU System Emulation — a `mps2-an385` machine, az `-icount` mód és a modell korlátjai. **Tartalmilag ellenőrizve** (a user-mode fejezeten keresztül), de verzió-megjegyzéssel: a `master` ág dokumentációja (közös verziókorlát); a hivatalos szöveg nem dokumentálja a cache-hierarchia vagy a busz-várakozási állapotok modellezését, ezért a fejezet QEMU-korlátra vonatkozó állításai a TCG ismert, funkcionális (nem ciluspontos) tervezéséből következnek.

A laborkönyvtár összeállítását a kísérőcsomag v1.0 helyettesítő eszközláncával (Clang + `ld.lld`) ellenőriztük: egy üres könyvtárban, kizárólag a Vektortábla, a Linker script alapjai, a Startup kód és ennek az alfejezetnek a fájljaiból, a fenti `cp`/`cat`/`sed` lépésekkel a labor fordul és linkelődik, és a `lab-check-vectors.py` a Reset- és a használt IRQ-bejegyzést is a várt szimbólumra mutatónak találja.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak: (1) a Cortex-M3 pontos belépési latenciája és a 12 ciklusú referenciaérték a konkrét core-revízióból ellenőrizendő; (2) az `mps2-an385` `DWT_CYCCNT`-támogatása és órafrekvenciája QEMU-verziófüggő; (3) a GPIO0 IRQ-száma és loopback viselkedése a machine definíciótól függ; (4) minden itteni ciklusszám levezetett/illusztratív, nem tényleges mérés; (5) a 2. megoldás kontrollált `SysTick`-próbájában a sorrend és a hátralevő munka mérve (GNU + QEMU 8.2.2, a lektor mérése), a késések csökkenését működő DWT-vel, hardveren kell igazolni.

18 Memory-mapped perifériák és GPIO

18.1 Memory-mapped I/O alapelve

18.1.1 Motiváció

Egy beágyazott rendszerben a szoftver gyakran nem „számol”, hanem eszközökkel beszél: LED-et kapcsol, UART-on küld, timert indít. A Cortex-M-en ezt memory-mapped I/O (MMIO) néven tesszük: a periféria regiszterei a 32 bites címtér részei. Az 16.3 alfejezetben láttuk, hogy a `0x40000000`-tól kezdődő Peripherals régió a perifériáké. Ha azonban ezeket a címeket ugyanúgy kezeljük, mint egy SRAM-változót, hibás kódot kapunk. Egy perifériaregiszter ugyanis nem passzív memóriacella: minden olvasása és írása mellékhatással járhat. Ez az alfejezet megmutatja, miért viselkedik másképp, és milyen programozási fegyelmet kényszerít ez ránk.

Támogatási mátrix (a fejezet és a II. záróprojekt laborjaihoz). Nem minden periféria fut minden platformon. A könyv alapútja a QEMU `mps2-an385`; a GPIO-laborok ezen nem futtathatók:

Periféria	QEMU 8.2.2 `mps2-an385`	Fizikai MPS2+ kártya AN385 FPGA-képpel	A könyv útja
CMSDK UART0 (`0x40004000`)	implementálva	van	**alapút** (QEMU)
CMSDK Timer0 (`0x40000000`)	implementálva	van	**alapút** (QEMU)
NVIC, SysTick (PPB)	implementálva	van	**alapút** (QEMU)
CMSDK GPIO (`0x40010000`–)	**nincs implementálva**	van	csak támogatott hardveren; QEMU-n **SKIP**

A QEMU-n nem implementált perifériánál a regiszter visszaolvasása semmit nem bizonyít: egy nem modellezett tartomány tipikusan 0-t olvas és eldobja az írást (vagy egy egyszerű RAM-modell épp a beírt értéket adja vissza), és egyik sem jelenti, hogy egy láb ténylegesen kapcsolt. A GPIO-t használó laborok (18.2, 18.3) ezért egy `LAB_HAVE_GPIO` build-kapcsolóval futnak. QEMU-n ennek értéke 0, és a futó próba `SKIP`-et ír ki, nem feltételezett sikert. A fordítási és disassembly-ellenőrzések (címek, offsetek, generált utasítások) ettől függetlenül QEMU nélkül is elvégezhetők. A II. záróprojekt QEMU-alapútja ugyanezért UART + timer (20.1).

18.1.2 Definíció: memory-mapped I/O és perifériaregiszter

Definíció (memory-mapped I/O, MMIO): olyan I/O-modell, amelyben a perifériák vezérlő-, állapot- és adatregiszterei a processzor normál adatcímterébe vannak leképezve; a processzor ugyanazokkal a load/store utasításokkal éri el őket, mint a memóriát, de a hozzáférés hatással lehet a periféria állapotára.

Definíció (perifériaregiszter): egy címmel azonosított, tipikusan 32 bites (vagy 8/16 bites nézetű) hardveres tároló, amelynek olvasása/írása nem csak adatcserét, hanem hardveres műveletet is kiválthat.

A kulcs: a cím mint memóriacím viselkedik, de a szemantika nem az. Az 18.2 alfejezetben ezt konkrét GPIO-vezérlésre fordítjuk le, az 18.3 pedig C struktúrával absztrahálja.

18.1.3 Miért nem közönséges memória?

A processzor és a memóriarendszer optimalizál a közönséges memóriára: gyorsítótár (cache), írási puffer, soron kívüli végrehajtás, spekulatív betöltés, hozzáférés-összevonás. Ezek mind feltételezik, hogy egy cím olvasása nem változtat állapotot, és hogy két azonos olvasás ugyanazt adja, amíg senki nem ír. Egy perifériaregiszternél ez nem igaz.

Az Armv7-M architektúra ezért a címtér régióit memóriatípusokba sorolja. A normál SRAM/Flash „Normal” memória, amely cache-elhető, újrarendezhető. A perifériaterület jellemzően „Device” vagy „Strongly-ordered” típusú. A Device memória:

- nem cache-elhető (nincs értelme, mert a periféria állapota a hardvertől függ),
- a hozzáférések sorrendje megmarad a Device területen belül,
- bizonyos összevonások (pl. két egymást követő azonos írás összevonása) tiltottak vagy korlátozottak,
- a spekulatív olvasás nem megengedett, mert mellékhatása lehet.

A Cortex-M3 mag jellemzően nem is tartalmaz adatcache-t, de az írási puffer és a busz miatt a sorrend és a hozzáférések száma akkor is számít. Az architektúra-szintű garancia: a Device/Strongly-ordered minősítés biztosítja, hogy a hozzáférések ténylegesen megtörténjenek, és hogy a Device-hozzáférések **egymáshoz képest** a programsorrendben maradjanak. Ez nem jelenti, hogy egy Device-írás minden más (például Normal memóriába tett) hozzáféréshez képest is sorrendben marad; erre a gát utasítások valók (lásd lent).

Miért tilos a gyorsítótárazás? Ha a periféria állapotregiszterét cache-elnénk, a cache egy régi értéket adna vissza, miközben a hardver már változott. Ha egy írást cache-elnénk, az írás esetleg sosem jutna el a perifériáig, vagy késleltetve, más sorrendben. Ez hibás működés.

Miért tilos az átrendezés? Tegyük fel, hogy előbb beírjuk az adatregisztert, majd beállítjuk a „start” bitet. Ha a processzor a két írást felcseréli, a periféria a régi adatot küldi el. Device memóriánál az architektúra megőrzi a programozási sorrendet erre a területre.

18.1.4 A perifériaregiszter négy tipikus szerepe

A CMSDK-szerű perifériák regiszterei négy csoportba sorolhatók. A pontos nevek perifériánként változnak, de a szerepek visszatérnek.

Szerep	Tipikus név	Olvasás	Írás	Példa
Adat	DATA, RX, TX	bejövő adat	kimenő adat	UART RX/TX, GPIO adat
Engedélyezés	CTRL, ENABLE	engedélyezettség	művelet indítása/engedélyezése	timer start, megszakítás engedélyezés

Szerep	Tipikus név	Olvasás	Írás	Példa
Állapot	STATUS, RIS	aktuális állapot/flagek	flag törlése (gyakran write-1-to-clear)	TX FIFO tele, RX adat elérhető
Konfiguráció	CFG, BAUDDIV, DIR	beállított érték	beállítás	baud rate, GPIO irány

Ez a négyes fontos, mert a programozási minta más-más szerepnél más. Adatregiszternél az érték számít; állapotnál a flag szemantikája (olvasás töröl? írás töröl?); engedélyezésnél a bit beállítása műveletet indíthat; konfigurációnál az érték beállítása a periféria viselkedését változtatja.

18.1.5 volatile C-ben, és miért nincs ilyen probléma assemblyben

A periféria-hozzáférés helyességét **három külön réteg** adja. Egyikből sem következik a másik kettő, és egyik sem ad univerzális sorrendet minden memóriaelérésre.

Réteg	Ki garantálja	Mit garantál	Mit **nem**
`volatile` (fordítói szerződés)	a C fordító	minden `volatile` hozzáférés megtörténik, nem vonható össze, nem hagyható el, és a `volatile` hozzáférések egymáshoz képest programsorrendben maradnak a generált kódban	a nem `volatile` hozzáférések átrendezhetők körülötte; semmit nem mond a hardveres sorrendről, és nem ad atomitást
Device memória-attribútum (architektúra)	a mag és a buszrendszer	a Device-hozzáférések nem spekulatívak, nem vonódnak össze, és egymáshoz képest sorrendben érik el a perifériát	a Normal memóriához (például egy RAM-pufferhez) képesti sorrendet nem garantálja; azt sem, hogy egy írás *befejeződött*, mielőtt a következő utasítás fut

Réteg	Ki garantálja	Mit garantál	Mit **nem**
`DMB` / `DSB` / `ISB` **(gát utasítások)**	a mag, explicit utasításra	`DMB`: a gát előtti memóriaelérések megfigyelhetően a gát utániak előtt történnek; `DSB`: a következő utasítás csak a korábbi elérések **befejeződése** után fut; `ISB`: a pipeline-t üríti, a későbbi utasítások az új állapotot látják (például a rendszerregiszter-írás után)	magától nem keletkezik; a fordító sem rak be ilyet a `volatile` miatt

A C fordító a C absztrakt gépet optimalizálja. Azt feltételezi, hogy egy változó értéke csak akkor változik, ha a program írja, és hogy az ismételt olvasás ugyanazt adja, ha közben nincs írás. Ezért egy `while (*reg & MASK) ;` ciklusból könnyen `if (*reg & MASK) while (1) ;` lesz, vagy a `*reg` bekerül egy regiszterbe, és a ciklus örökké a régi értéket vizsgálja. A `volatile` minősítő az első réteg: minden hozzáférés valódi, nem elhagyható, és nem tartható regiszterben. A `volatile uint32_t *` tehát nem stílus kérdése, hanem funkcionális követelmény.

Assemblyben az első réteg nem kérdés, mert az assembler nem optimalizál: amit leírtunk, az pontosan az utasítássorozat. Egy `LDR r1, [r0]` egy olvasás; ha a ciklusban újra leírjuk, újra megtörténik. A második és a harmadik réteg viszont assemblyben is ugyanúgy érvényes: a Device-szabályok a mag szintjén hatnak, a gátakat pedig nekünk kell kiírni, ahol kellenek. Jellemző eset: egy RAM-ban előkészített puffer után egy periféria „start” regiszterébe írunk. Ekkor a két írás közé `DMB` kell, mert a Normal és a Device hozzáférés közti sorrendet sem a `volatile`, sem a Device-attribútum nem garantálja. Egy NVIC- vagy `VTOR`-írás után pedig `DSB` + `ISB` biztosítja, hogy a következő utasítások már az új beállítással fussanak (NVIC-konfiguráció).

18.1.6 Kidolgozott példa: UART TX flag pollozása

Munkakönyvtár: `~/arm-labor/cm/a072/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a072
cd ~/arm-labor/cm/a072
```

A példa a QEMU `mps2-an385` **tényleges** CMSDK APB UART0 perifériáját használja, nem egy elképzelt eszközt. A CMSDK-nál az állapotregiszter (`STATE`, `+0x004`) 0. bitje a `TXFULL`: 1 jelenti, hogy az adópuffer tele, tehát **amíg 1, várunk**, és 0-nál írhatunk. (Egy elképzelt „TX_READY = 1 kész” bit pont fordított polaritású lenne; a két minta keverése a klasszikus hiba, ezért itt csak a valódi bitet használjuk.) Az adó használatához a `BAUDDIV` és a `CTRL.TXEN` beállítása is kell (előfeltételek).

Assembly (Thumb-2, Armv7-M):

```
@ lab-a072-uart.s - Thumb-2 / Armv7-M, QEMU mps2-an385, CMSDK APB UART0
    .syntax unified
    .thumb

    .equ UART0_BASE,   0x40004000      @ MPS2-AN385 UART0
    .equ UART_DATA,    0x000           @ adatregiszter
    .equ UART_STATE,   0x004           @ bit0: TXFULL, bit1: RXFULL
    .equ UART_CTRL,    0x008           @ bit0: TXEN
    .equ UART_BAUDDIV, 0x010           @ legalább 16
    .equ UART_TXFULL,  (1 << 0)

    .text
    .thumb_func
    .global uart_init
uart_init:
    ldr  r1, =UART0_BASE
    movs r2, #16
    str  r2, [r1, #UART_BAUDDIV]
    movs r2, #1                        @ TXEN
    str  r2, [r1, #UART_CTRL]
    bx   lr

    .thumb_func
    .global uart_putc
    @ r0 = küldendő bájt
uart_putc:
    ldr  r1, =UART0_BASE
wait_tx:
    ldr  r2, [r1, #UART_STATE]         @ állapotregiszter olvasása: minden körben valódi LDR
    tst  r2, #UART_TXFULL
    bne  wait_tx                       @ amíg TXFULL = 1, várunk
    str  r0, [r1, #UART_DATA]          @ adatregiszter írása: elindítja a küldést
    bx   lr
```

Itt a `wait_tx` ciklus minden iterációban valódi LDR-t hajt végre. A STR írás mellékhatása: a periféria megkezdi a bájt kiküldését. Ha az LDR-t kihagynánk, vagy a STR-t az állapotellenőrzés elé tennénk, hibás működés lenne. Az adatregisztert szóhosszú STR-rel írjuk; a regiszter alsó 8 bitje az adat.

C változat volatile-lal:

```c
/* lab-a072-uart.c - ugyanaz C-ben, volatile mutatókkal */
#include <stdint.h>

#define UART0_BASE    0x40004000u
#define UART_DATA     (*(volatile uint32_t *)(UART0_BASE + 0x000u))
#define UART_STATE    (*(volatile uint32_t *)(UART0_BASE + 0x004u))
#define UART_TXFULL   (1u << 0)

void uart_putc_c(uint8_t b)
{
    while (UART_STATE & UART_TXFULL) {
        /* amíg TXFULL = 1, várunk: minden iterációban valódi olvasás */
    }
    UART_DATA = b;   /* írás: elindítja a küldést */
}

void uart_init_c(void)
{
    *(volatile uint32_t *)(UART0_BASE + 0x010u) = 16u;  /* BAUDDIV >= 16 */
    *(volatile uint32_t *)(UART0_BASE + 0x008u) = 1u;   /* CTRL.TXEN */
}
```

A `uart_putc_c` **csak** `uart_init_c()` **után** működik: TXEN nélkül az első írás után a `TXFULL` beáll és nem törlődik, ezért a második karakterre a program örökké vár (QEMU-n timeout, 124-es kilépés, üres kimenet). Ez a várakozó ág QEMU-n is reprodukálható, nem csak hardveren.

Egy futtatható tesztprogram (a kanonikus `lab-mps2.ld` + `lab-startup.s` mellé linkelve):

```
/* lab-a072-main.c - asm és C putc, majd semihosting SYS_EXIT */
#include <stdint.h>
extern void uart_init(void);
extern void uart_putc(uint8_t);
extern void uart_putc_c(uint8_t);

static void sh_exit(int code)
{
    register int r0 __asm__("r0") = 0x18;   /* SYS_EXIT */
    register int r1 __asm__("r1") = code;   /* 0 = OK */
    __asm__ volatile("bkpt 0xab" :: "r"(r0), "r"(r1));
}

int main(void)
{
    uart_init();
    const char *a = "asm\n";
    while (*a) uart_putc((uint8_t)*a++);
    uart_init_c();
    const char *c = "C\n";
    while (*c) uart_putc_c((uint8_t)*c++);
    sh_exit(0);
    for (;;) { }
}
```

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -O1 -nostdlib \
    -T lab-mps2.ld lab-startup.s lab-a072-uart.s lab-a072-uart.c \
    lab-a072-main.c -o lab-a072.elf
timeout 5 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -semihosting \
    -display none -monitor none -serial file:out.txt -kernel lab-a072.elf
cat out.txt        # várt: asm, majd C; a QEMU kilépési állapota 0
```

A `volatile` nélkül a fordító a `while` ciklus olvasását kiemelhetné, és a ciklus egyszer olvasná a `UART_STATE`-et. A `-O2`-vel fordított két változat disassemblyje (mérve, GNU + QEMU 8.2.2, a lektor mérése) ezt meg is mutatja:

```
@ volatile mellett, a wait_tx ciklus törzsében:
ldr   r3, [r2, #4]
@ volatile nélkül a ciklus:
bpl   …                @ feltételes ugrás a ciklus elejére
b.n   .                @ önmagára ugró végtelen ciklus
```

A `volatile` nélküli alak pontosan az `if (…) while (1);` minta: a várakozás nem a regiszter újraolvasásán alapul.

A két változatot a kanonikus startup/linker párral (16.5, 16.6) egy rövid tesztprogramba linkelve a fő úton, mérve (GNU + QEMU 8.2.2, a lektor mérése), mindkét szöveg megjelent a UART0 kimenetén (`asm`, majd `C`, exit 0). A `uart_init_c()` nélküli változatban a második karakterre a program örökké vár (timeout, 124), ami a várakozó ág QEMU-s ellenpróbája.

18.1.7 Tipikus hibák és ellenpróbák

1. **Elfelejtett `volatile`.** Tünet: a program a vártnál előbb kilép a ciklusból, vagy örökké vár. Hipotézis: a fordító kiemelte az olvasást. Adatgyűjtés: `objdump -d` a C objektumon,

a ciklusban keresd a `ldr` utasítást. Ha nincs, ez a hiba. Egy változtatás: `volatile` hozzáadása. Ellenőrzés: a disassemblyben megjelenik a cikluson belüli betöltés.

2. **Írások összevonása.** Tünet: két egymást követő írásból csak az utolsó érvényesül. Hipotézis: a periféria nem Device memóriaként van kezelve, vagy a fordító optimalizálta el az elsőt. C-ben a volatilis írások nem vonhatók össze; assemblyben a hardver Device-szabályai védenek. Ellenpróba: a két írás közé tegyünk egy olvasást, és nézzük meg, megmarad-e mindkét írás.

3. **Nem megfelelő szélesség.** A CMSDK APB perifériák a bájtírást a 32 bites buszon bájtsávokra ismételve kapják meg; a UART `DATA` alsó 8 bitjénél ez működik, de általános szabályként a regiszter dokumentált szélességével (`STR`) érdemes írni. A fenti példa ezért szóhosszú `STR`-t használ. Ellenpróba: a CMSDK dokumentációban ellenőrizni a regiszter szélességét és a támogatott hozzáférést.

4. **Fordított polaritású állapotbit.** Tünet: a küldés minden karaktert eldob, vagy örökké vár. Hipotézis: a kód „ready = 1” bitet feltételez, a CMSDK `STATE.TXFULL` viszont 1-nél jelzi, hogy *nem* lehet írni. Ellenpróba: a ciklus feltétele `bne wait_tx` (amíg TXFULL = 1), nem `beq`.

5. **Hiányzó gát Normal és Device hozzáférés között.** Tünet: a periféria egy RAM-pufferből régi adatot dolgoz fel. Hipotézis: a pufferírás és a „start” regiszter írása között nincs `DMB`. Ellenpróba: `DMB` beszúrása a két írás közé; a `volatile` önmagában ezt nem pótolja.

18.1.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd el, miért nem cache-elhető és miért nem rendezhető át egy perifériaregiszter hozzáférése. Adj példát arra, mi történne, ha a `wait_tx` ciklus `LDR`-jét a processzor egyszer olvasná, és a ciklus a regiszterben tartaná az értéket.

2. (implementációs, alapszint — cél: rutinépítés) Írj Thumb-2 assembly rutint, amely kiolvassa a UART0 `STATE` regiszterét, majd írj C függvényt `volatile` mutatóval, amely ugyanezt teszi. A fő úton ellenőrizhető: olvasd ki a `STATE`-et a TXEN bekapcsolása előtt és után. Kiegészítő feladatként ugyanezt a `GPIO0 DATA` regiszterére is írd meg; a címeket a periféria-struktúra kanonikus `lab-cmsdk-gpio.h` térképéből vedd, és vedd figyelembe, hogy a GPIO QEMU-n nem elérhető, ott csak a fordítás és a disassembly ellenőrizhető.

3. (hibakeresési, alapszint — cél: diagnózis és ellenőrzés) Adott egy C kód, amely `volatile` nélkül polloz egy állapotbitet, és a program a QEMU-ban nem lép ki a ciklusból. Írd le a diagnózis lépéseit: mit néznél meg `objdump -d`-vel, mit várnál a `volatile` hozzáadása után, és hogyan igazolnád az eredményt?

18.1.9 Megoldások

1. megoldás — Segítség. Gondold végig, mihez képest ad sorrendet a `volatile` (a fordító), és mihez képest a Device-attribútum (a mag); a `wait_tx` ciklusnál pedig azt, hogy hányszor kell látnia a flag változását.

1. megoldás — Teljes megoldás. A cache egy gyors másolat, amelyet a hardver állapotváltozásáról nem értesül. Ha a státuszregisztert cache-eljük, a program a régi értéket olvassa, és nem veszi észre, hogy a flag beállt. Az átrendezés akkor okoz gondot, ha az adatírást és a startírást felcseréli a rendszer: a periféria a régi adatot küldi. A `wait_tx` ciklusban, ha egyszer olvasnánk

és regiszterben tartanánk, a ciklus soha nem látná meg a flag változását, mert a regiszter értéke nem frissül; ez végtelen ciklus. Ezért kell minden iterációban valódi olvasás.

2. megoldás — Segítség. A `STATE` regiszter a `0x40004000 + 0x004` címen olvasható; a C-oldalon `volatile` mutató kell, az assembly rutinnál pedig `.thumb_func`, hogy a C-ből hívott cím Thumb-bitje helyes legyen.

2. megoldás — Teljes megoldás. UART `STATE`:

```
    .syntax unified
    .thumb
    .equ UART0_BASE,  0x40004000      @ MPS2-AN385 UART0
    .equ UART_STATE,  0x004           @ bit0: TXFULL

    .text
    .thumb_func
    .global uart_read_state
uart_read_state:
    ldr r0, =UART0_BASE
    ldr r0, [r0, #UART_STATE]
    bx lr
```

```
#include <stdint.h>
#define UART0_BASE  0x40004000u
#define UART_STATE  (*(volatile uint32_t *)(UART0_BASE + 0x004u))

uint32_t uart_read_state(void)
{
    return UART_STATE;
}
```

A `STATE` a `uart_init_c()` (TXEN) előtt és után is olvasható; a `TXFULL` (bit0) értéke a fő úton megfigyelhető. A `.thumb_func` nélkül a C-ből hívott címről hiányozhat a Thumb-bit (16.3). Kiegészítő GPIO-változat:

```
    .equ GPIO0_BASE,  0x40010000      @ MPS2-AN385 GPIO0 (lab-cmsdk-gpio.h)
    .equ GPIO_DATA,   0x000           @ DATA: a lábak bemeneti szintje

    .text
    .thumb_func
    .global gpio_read_data
gpio_read_data:
    ldr r0, =GPIO0_BASE
    ldr r0, [r0, #GPIO_DATA]
    bx lr
```

```
#include <stdint.h>
#define GPIO0_BASE 0x40010000u         /* MPS2-AN385 GPIO0 (lab-cmsdk-gpio.h) */
#define GPIO_DATA  (*(volatile uint32_t *)(GPIO0_BASE + 0x000u))

uint32_t gpio_read_data(void)
{
    return GPIO_DATA;
}
```

A lényeg a `volatile`, és hogy az assemblyben az `LDR` ténylegesen megtörténik.

3. megoldás — Segítség. A kérdés az, hogy a ciklus törzsében van-e `ldr` a periféria címére; ezt `objdump -d`-vel lehet eldönteni.

3. megoldás — Teljes megoldás. Diagnózis: `objdump -d` a lefordított objektumon; keresd a ciklus címét és a benne lévő `ldr` utasítást. Ha a ciklusban nincs `ldr` a periféria-címre, a fordító

kiemelte. Add hozzá a `volatile`-t, fordítsd újra, és a disassemblyben a cikluson belül megjelenik a betöltés. Ellenőrzés QEMU-ban, a valódi UART0-n: a fenti `uart_putc_c` kiírja a karaktert, tehát kilépett a várakozó ciklusból. Fontos: a fő úton a várakozó ág a TXEN kikapcsolásával mutatható meg; a valós baudsebességű várakozás csak hardveren mérhető.

18.1.10 Rövid összefoglaló

A memory-mapped I/O a perifériaregisztereket a normál címtérbe képezi, de a hozzáférés mellékhatással járhat. Három réteg adja a helyességet: a `volatile` a fordítót köti (a `volatile` hozzáférések megtörténnek, sorrendben), a Device-attribútum a hardvert (a Device-hozzáférések egymáshoz képest sorrendben, spekuláció nélkül), a `DMB`/`DSB`/`ISB` pedig ott ad sorrendet vagy befejezést, ahol a másik kettő nem (Normal–Device határ, rendszerregiszter-írás után). Egyik sem ad univerzális sorrendet minden memóriaelérésre. A CMSDK UART `STATE.TXFULL` bitje 1-nél jelzi, hogy várni kell. A GPIO a QEMU-alapúton nem érhető el; ott a GPIO-próba SKIP. A regiszterek négy szerepe: adat, engedélyezés, állapot, konfiguráció.

Továbbhasznált definíciók:

- **memory-mapped I/O (MMIO):** perifériaregiszterek a normál adatcímtérben, load/store hozzáféréssel.
- **perifériaregiszter:** címmel azonosított hardveres tároló, amelynek olvasása/írása mellékhatással járhat.
- **volatile:** C-minősítő; a fordító minden ilyen hozzáférést ténylegesen, a programban leírt számban és a többi volatile hozzáféréshez képest sorrendben végrehajt; az értéket nem tartja processzorregiszterben. Hardveres sorrendet és atomitást nem ad.

18.1.11 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Armv7-M Architecture Reference Manual, DDI0403E — a memóriatípusok (Normal/Device/Strongly-ordered) és a hozzáférési szabályok elsődleges forrása.
- Arm M-profile architecture documentation — kiindulópont.
- QEMU System Emulation dokumentáció — a machine modellek és a perifériaemuláció leírása.

Ellenőrzési hiányok:

- A UART-példa a CMSDK APB UART dokumentált térképét használja (`DATA` 0x000, `STATE` 0x004 – bit0 `TXFULL`, `CTRL` 0x008, `BAUDDIV` 0x010); a futtatás a fő úton, mérve (GNU + QEMU 8.2.2, a lektor mérése) megtörtént, és a GPIO-sor állapota mérve (QEMU 8.2.2, `info mtree`, a lektor mérése). A támogatási mátrix a II. záróprojekt specifikációjának QEMU-korlát megjegyzésén is alapul; más QEMU-verziónál `info mtree`-vel és ténylegesen bekötött eseményforrással kell újraellenőrizni.
- Az Armv7-M Device/Strongly-ordered szabályok részletes, oldalszintű idézése itt nem történt meg.
- A QEMU viselkedése nem hardverhű; a mért latenciák és versenyhelyzetek eltérhetnek a fizikai Cortex-M3-tól.

18.2 GPIO közvetlen vezérlése: bitbanging

A memory-mapped I/O alapelve szerint a perifériaregisztereket ugyanúgy érhetjük el, mint a memóriát: load/store utasításokkal. Most ezt konkrét portára alkalmazzuk. Megnézzük, hogyan lesz egy GPIO-pin kimenet, hogyan állítunk be vagy törlünk egyetlen bitet a port többi bitjének érintetlenül hagyásával, hogyan olvasunk bemenetet és ágazunk el az értékére, és hogyan lesz mindebből **bitbanging** – szoftveresen, ciklusokkal előállított jelsorozat.

18.2.1 Motiváció

A memory-mapped I/O önmagában csak annyit mond, hogy *van* egy cím, amit írni lehet. A gyakorlati hardvervezérlés ennél több: egy 32 bites portán egyszerre 32 pin lakik, és jellemzően **egyetlen** bitet akarunk megváltoztatni, a többit békén hagyni. Ha a `STR`-rel a teljes szót felülírjuk, minden más kimenet is áldozatul esik. Ez az alfejezet megmutatja a helyes read-modify-write (RMW) mintát, összekapcsolva a logikai alapműveletek és bitmaszkolás szakaszban tanult `ORR`/`BIC` maszkolással – azzal a fontos különbséggel, hogy itt **Thumb-2 / Armv7-M** kontextusban dolgozunk, nem A64-ben.

18.2.2 Definíció és működés

GPIO (General-Purpose Input/Output): olyan digitális periféria, amelynek minden csatlakoztatott lábát programozottan bemenetnek vagy kimenetnek lehet kijelölni, és kimenetként magas (1) vagy alacsony (0) szintre lehet hajtani.

Egy GPIO-port regiszterkészlete tipikusan kétféle információt tárol:

- **irányregiszter (direction):** bitenként megmondja, hogy a láb bemenet-e vagy kimenet. Az ARM **CMSDK GPIO** modelljében ez nem egyetlen regiszter, hanem két maszkolt regiszterpár: `OUTENSET` (1-es a beállítandó bitre → kimenet lesz) és `OUTENCLR` (1-es a törlendő bitre → bemenet lesz). Ez eleve megkerüli az RMW-problémát az irányváltásnál.
- **adatregiszter (data):** kimenetnél a kiírt logikai szintet, bemenetnél a láb pillanatnyi szintjét hordozza. A CMSDK-nál külön `DATA` és `DATAOUT` létezik: a `DATA` olvasása a pad tényleges szintjét adja vissza, a `DATAOUT` a legutóbb kiírt kimeneti értéket tárolja.

Bitbanging: olyan technika, amikor egy protokoll vagy jelsorozat időzítését nem dedikált hardverperiféria (UART, SPI, I²C vezérlő), hanem maga a processzor állítja elő, közvetlen GPIO-írásokkal és szoftveres késleltetéssel. Következménye, hogy az időzítés pontossága a processzor ciklusidejétől és a megszakításoktól függ.

18.2.3 A CMSDK GPIO regiszterei az MPS2 AN385-ön

A fejezet célplatformja a `qemu-system-arm -machine mps2-an385`, amely ARM MPS2+ AN385 FPGA-képet modellez, CMSDK perifériákkal. A GPIO-portok báziscíme és regiszteroffszetjei az MPS2 dokumentált példatartományából származnak:

Port	Báziscím
GPIO0	`0x40010000`
GPIO1	`0x40011000`

A regisztertérkép az Arm Cortex-M System Design Kit műszaki leírásának 3-8. táblázatát (3.5.2) követi. A CMSDK AHB GPIO portonként **16 lábat** kezel; a 32 bites regiszter alsó 16 bitje tartozik hozzájuk (más mikrokontrollereken akár 32 láb is juthat egy portra). Ugyanez a térkép a II. rész **egyetlen forrása** a GPIO-offsetekhez: a periféria-struktúra `lab-cmsdk-gpio.h` fejléce és az alábbi assembly-konstansok is ebből készültek, és a záróprojekt is ezt a fejlécet használja.

Offszet	Név	Típus	Szerep
`0x000`	`DATA`	RW	olvasás: a lábak tényleges szintje; írás: kimeneti adat
`0x004`	`DATAOUT`	RW	kimeneti adatregiszter (utolsó kiírt érték)
`0x008`–`0x00C`	—	—	**fenntartott** (két szó)
`0x010`	`OUTENSET`	RW	1 a kívánt bitre → az a láb kimenet lesz
`0x014`	`OUTENCLR`	RW	1 a kívánt bitre → az a láb bemenet lesz
`0x018`	`ALTFUNCSET`	RW	alternatív funkció engedélyezése
`0x01C`	`ALTFUNCCLR`	RW	alternatív funkció letiltása
`0x020`	`INTENSET`	RW	megszakítás engedélyezése bitenként
`0x024`	`INTENCLR`	RW	megszakítás **tiltása** bitenként (nem nyugtázás)
`0x028`	`INTTYPESET`	RW	1 → élvezérelt
`0x02C`	`INTTYPECLR`	RW	1 → szintvezérelt
`0x030`	`INTPOLSET`	RW	1 → felfutó él / magas szint
`0x034`	`INTPOLCLR`	RW	1 → lefutó él / alacsony szint
`0x038`	`INTSTATUS` / `INTCLEAR`	R / W1C	olvasás: függő megszakítások; írás: 1 a bitre → az eseményjelző törlése
`0x400`–`0x7FC`	`MASKLOWBYTE`	W	atomi bájtírás az alsó bájtra; a cím `[9:2]` bitjei a maszk
`0x800`–`0xBFC`	`MASKHIGHBYTE`	W	atomi bájtírás a felső bájtnyi sávra; a cím `[9:2]` bitjei a maszk

Platformtámogatás. A memory-mapped I/O alfejezet támogatási mátrixa szerint a QEMU 8.2.2 `mps2-an385` gépen a GPIO **nincs implementálva**. A regiszterek írása és visszaolvasása ott semmit nem igazol: egy nem modellezett tartomány 0-t ad vagy épp a beírt értéket tükrözi, és egyik sem jelenti, hogy egy láb kapcsolt. Ezért a futó GPIO-próba QEMU-n `SKIP`. Érdemben csak támogatott hardveren (fizikai MPS2+ kártya AN385 FPGA-képpel) futtatható. Egy más gyártói mikrokontroller (például STM32) GPIO-ja más regisztertérképet használ, a fenti offsetek ott nem érvényesek.

18.2.4 Egy bit beállítása és törlése: ORR/BIC

A DATAOUT regiszter egy teljes 32 bites szó. Egy pin megváltoztatásához három lépés kell: olvasás, módosítás, visszaírás:

```
@ Thumb-2 / Armv7-M
ldr  r2, [r0, #GPIO_DATAOUT]   @ 1) olvas: aktuális kimeneti szó
orr  r2, r2, r1                @ 2) a kijelölt bit(ek) beállítása 1-re
str  r2, [r0, #GPIO_DATAOUT]   @ 3) visszaír
```

A törlés ugyanez BIC-kel, ahol `BIC r2, r2, r1` a maszkkal kijelölt biteket **nullázza**, a többit megtartja:

```
ldr  r2, [r0, #GPIO_DATAOUT]
bic  r2, r2, r1
str  r2, [r0, #GPIO_DATAOUT]
```

Ez pontosan a 9.1 alfejezet (Logikai alapműveletek és bitmaszkolás) mintázata: ORR beállít, BIC töröl, a maszk pedig bitpozíciót jelöl. Két gyakorlati korlát azonban M-profilon fokozottan érvényes:

1. **Nem atomi.** Két megszakítás vagy két feladat között ugyanerre a portára írva elveszhet a másik módosítása (*lost update*). Ahol a hardver kínál külön SET/CLR regisztert (irányváltásra), maszkolt írási tartományt (MASKLOWBYTE/MASKHIGHBYTE) vagy bit-band aliast (adatbitre), azt kell használni a LDR/ORR/STR helyett.
2. **Az immediate korlátozott.** Thumb-2-ben az ORR/BIC immediate a *modified immediate* (ThumbExpandImm) családból kódolható; például 0x80000001 nem írható le, ilyenkor regiszterbe töltött maszk kell.

18.2.5 Atomi bitírás RMW nélkül: maszkolt írás és bit-band

A DATAOUT olvasás–módosítás–visszaírása elkerülhető a CMSDK GPIO maszkolt írási tartományaival. A 0x400–0x7FC (MASKLOWBYTE) és 0x800–0xBFC (MASKHIGHBYTE) címeken a cím [9:2] bitjei adják a bájtszintű maszkot: csak a maszkban kijelölt bájtok íródnak, a többi bit változatlan marad. Egyetlen STR elegendő, nincs előzetes LDR:

```
@ pin0 beállítása atomi maszkolt írással (alsó bájt, maszk = 0x01)
movs r2, #1
str  r2, [r0, #0x400 + (0x01 << 2)]   @ 0x404: csak a 0. bájt íródik
```

A Cortex-M3 **bit-band** aliasa szintén atomi bitírást ad. A 0x40000000–0x400FFFFF perifériatartomány bitjei a 0x42000000 + (cím − 0x40000000)·32 + bit·4 címen érhetők el; a GPIO0 0x40010000 is ebben a tartományban van, ezért a DATAOUT 0. bitje a 0x42000000 + 0x10000·32 = 0x42200000 címen írható. A bit-band a QEMU 8.2.2 mps2-an385-ön SRAM-on működik (a 0x22000000 aliasba írt 1 a célszó 3. bitjét állította be); periférián a modell támogatása verziófüggő, ezért hardveren ellenőrizendő.

Bitbanging és bit-banding. A „bitbanging” szoftveres jelgenerálást jelent: a lábak időzítését a program állítja elő. A „bit-banding” a Cortex-M3 bitcímzési aliasa. A két fogalom nem ugyanaz; ennek az alfejezetnek a címe a bitbangingre utal. A példa egyetlen impulzust állít elő, nem teljes protokollt; a 8N1 szoftveres UART TX (start bit, 8 adatbit LSB-first, stop bit, ciklusszámolt késleltetés) és a megszakítás okozta jitter elemzése a fejezet feladatai közé tartozik.

18.2.6 Kidolgozott példa

Munkakönyvtár: `~/arm-labor/cm/a073/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). A példa a Linker script alapjai `lab-mps2.ld` fájljára és a Startup kód `lab-startup.s` fájljára épít; ezeket átmásoljuk, nem a korábbi könyvtárakban dolgozunk tovább.

```
mkdir -p ~/arm-labor/cm/a073
cd ~/arm-labor/cm/a073
cp ~/arm-labor/cm/a065/lab-mps2.ld ~/arm-labor/cm/a066/lab-startup.s .
```

A GPIO-műveletek implementált CMSDK GPIO perifériát és valódi bemenetet feltételeznek. A QEMU 8.2.2 `mps2-an385` modelljében a GPIO nincs implementálva; ott a fizikai be-/kimeneti próba `SKIP`, és a regiszterek írása nem igazolja a lábak működését. A program ezért egy `LAB_HAVE_GPIO` build-kapcsolót kap, amelynek alapértéke 0; a C `main` ezt maga definiálja, ha a build nem adja meg, így az alfejezet önállóan futtatható. A kanonikus fejlécet (a `lab-cmsdk-gpio.h`-t, benne ugyanezzel az alapértékkel) a 18.3 alfejezet vezeti be; a 18.2 assembly-konstansai saját `.equ`-ként tartalmazzák ugyanezt a térképet.

A `lab-gpio-bitbang.s` GPIO0 pin0-t kimenetként használja: megvárja, amíg a pin1 (bemenet) magasra vált, majd egyetlen rövid impulzust ad a pin0-n.

```
@ lab-gpio-bitbang.s - Cortex-M / Thumb-2 (Armv7-M)
.syntax unified
.thumb

@ CMSDK GPIO offsetek - ugyanaz a térkép, mint a lab-cmsdk-gpio.h-ban (a074)
.equ GPIO_DATA,       0x000
.equ GPIO_DATAOUT,    0x004
                                @ 0x008-0x00C: fenntartott
.equ GPIO_OUTENSET,   0x010
.equ GPIO_OUTENCLR,   0x014
.equ GPIO_ALTFUNCSET, 0x018
.equ GPIO_ALTFUNCCLR, 0x01C
.equ GPIO_INTENSET,   0x020
.equ GPIO_INTENCLR,   0x024
.equ GPIO_INTTYPESET, 0x028
.equ GPIO_INTTYPECLR, 0x02C
.equ GPIO_INTPOLSET,  0x030
.equ GPIO_INTPOLCLR,  0x034
.equ GPIO_INTSTATUS,  0x038       @ olvasás: INTSTATUS, írás: INTCLEAR (W1C)
.equ GPIO0_BASE,      0x40010000

.text

@ void gpio_out_enable(uint32_t base, uint32_t pin);   r0=base, r1=pin
.global gpio_out_enable
.thumb_func
gpio_out_enable:
    movs r2, #1
    lsl  r2, r2, r1            @ pinindex -> bitmaszk
    str  r2, [r0, #GPIO_OUTENSET]
    bx   lr

@ void gpio_set(uint32_t base, uint32_t mask);
.global gpio_set
.thumb_func
gpio_set:
    ldr  r2, [r0, #GPIO_DATAOUT]
    orr  r2, r2, r1
    str  r2, [r0, #GPIO_DATAOUT]
```

```
    bx   lr

@ void gpio_clear(uint32_t base, uint32_t mask);
.global gpio_clear
.thumb_func
gpio_clear:
    ldr  r2, [r0, #GPIO_DATAOUT]
    bic  r2, r2, r1
    str  r2, [r0, #GPIO_DATAOUT]
    bx   lr

@ uint32_t gpio_read_input(uint32_t base, uint32_t mask);
.global gpio_read_input
.thumb_func
gpio_read_input:
    ldr  r2, [r0, #GPIO_DATA]  @ a pad tényleges szintje
    and  r0, r2, r1            @ 0 vagy mask
    bx   lr

@ void delay(void);  durva, szoftveres késleltetés (illusztratív)
.global delay
.thumb_func
delay:
    ldr  r1, =0x00080000
delay_loop:
    subs r1, r1, #1
    BNE  delay_loop
    bx   lr

@ void wait_button(uint32_t base, uint32_t mask);  vár magas szintre
.global wait_button
.thumb_func
wait_button:
    push {r4, r5, r6, lr}
    mov  r4, r0
    mov  r5, r1
wait_loop:
    mov  r0, r4
    mov  r1, r5
    bl   gpio_read_input
    cmp  r0, #0
    BEQ  wait_loop             @ 0 -> még alacsony, újraolvasunk
    pop  {r4, r5, r6, pc}

@ void gpio_demo(void);
.global gpio_demo
.thumb_func
gpio_demo:
    push {r4, lr}
    ldr  r4, =GPIO0_BASE

    mov  r0, r4                @ pin0 = kimenet
    movs r1, #0
    bl   gpio_out_enable

    mov  r0, r4                @ pin1 bemenet marad (default)
    movs r1, #2                @ 0b10 -> pin1 maszkja
    bl   wait_button

    mov  r0, r4                @ rövid magas impulzus pin0-n
    movs r1, #1
    bl   gpio_set
    bl   delay
```

```
    mov  r0, r4
    movs r1, #1
    bl   gpio_clear

    pop  {r4, pc}
```

A futtatást egy rövid C `main` vezérli, amely a `gpio_demo`-t csak `LAB_HAVE_GPIO = 1` mellett hívja, egyébként `SKIP`-et ír ki semihostingon:

```
/* lab-gpio-main.c - a gpio_demo futtatása csak támogatott platformon */
#include <stdint.h>
#ifndef LAB_HAVE_GPIO
#define LAB_HAVE_GPIO 0
#endif

void gpio_demo(void);

static void sh_call(uint32_t op, const void *arg)
{
    register uint32_t r0 asm("r0") = op;
    register const void *r1 asm("r1") = arg;
    asm volatile ("bkpt 0xab" : "+r"(r0) : "r"(r1) : "memory");
}

int main(void)
{
#if LAB_HAVE_GPIO
    gpio_demo();                                  /* pin1-re vár, majd impulzus pin0-n */
    sh_call(0x04u, "gpio_demo: kesz\n");
#else
    sh_call(0x04u, "SKIP: GPIO nincs implementalva ezen a platformon (LAB_HAVE_GPIO=0)\n");
#endif
    sh_call(0x18u, (const void *)0x20026u);       /* SYS_EXIT */
    for (;;) { }
}
```

Build és futtatás. A firmware build és QEMU-betöltés munkafolyamat szerint, a startup kód mellé linkelve:

```
# QEMU-alapút: a GPIO-próba SKIP
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 \
  -T lab-mps2.ld -o lab-gpio.elf lab-startup.s lab-gpio-bitbang.s lab-gpio-main.c
timeout 5 qemu-system-arm -M mps2-an385 -cpu cortex-m3 -nographic -semihosting \
  -kernel lab-gpio.elf </dev/null; echo "kilépési állapot: $?"

# Támogatott hardveren (MPS2+ AN385): ugyanez -DLAB_HAVE_GPIO=1 kapcsolóval,
# a debuggeren keresztül, semihostinggal
```

A QEMU-alapút kimenete mérve (GNU + QEMU 8.2.2, a lektor mérése); a modell a GPIO-tartományt nem modellezi (0-t olvas, az írást eldobja):

```
SKIP: GPIO nincs implementalva ezen a platformon (LAB_HAVE_GPIO=0)
kilépési állapot: 0
```

Ellenpróba: miért nem elég „megpróbálni”. Ugyanezt a programot `-DLAB_HAVE_GPIO=1`-gyel a nem modellezett GPIO-n futtatva a `wait_button` örökké vár, mert a `DATA` mindig 0-t ad. A fő úton (QEMU) nincs utasításlimit, ezért a program soha nem áll le magától: `timeout 5 qemu-system-arm` …-tal kell leállítani (a futás 124-es állapottal ér véget), vagy `-nographic` módban Ctrl-A X-szel. A program viselkedéséből nem derül ki, hogy a periféria hiányzik; a QEMU naplójából viszont igen. Ellenpróba: `timeout 5 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -d unimp -D unimp.log -kernel lab-gpio1.elf`

`</dev/null`, majd `head unimp.log` – a QEMU minden nem modellezett elérést naplóz (`cmsdk-ahb-gpio: unimplemented device read/write`). A támogatási mátrix és a build-kapcsoló ettől függetlenül is kezeli a hiányt.

Hardveren, GDB-vel (a II. rész egységes `-S -gdb tcp::3333` / `target remote :3333` mintája helyett ott a debugprobe GDB-szervere a cél):

```
(gdb) break gpio_set
(gdb) continue
(gdb) info registers r0 r1 r2
(gdb) x/2xw $r0
```

Várt állapot támogatott hardveren. A `gpio_set` belépésénél `r0 = 0x40010000`, `r1 = 0x00000001`. A `STR` után a `0x40010004` (`DATAOUT`) címen `0x00000001` olvasható, és a `0x40010010` (`OUTENSET`) visszaolvasva a pin0 bitjét mutatja. A `gpio_read_input` a `DATA` regiszterből (`0x40010000`) olvas, és `AND`-del izolálja a kért bitet; a `CMP r0, #0` utáni `BEQ wait_loop` addig ismétel, amíg a pin1 magas nem lesz. Ezeket az értékeket QEMU-n nem lehet ellenőrizni; ott a próba `SKIP`.

Valódi műveletsor a `gpio_demo` impulzusából (hardveren ellenőrizhető; QEMU-n a GPIO nincs implementálva):

Lépés	Művelet	`DATAOUT`
0	kezdeti érték	`0x00000000`
1	`gpio_set(base, 0x1)` (pin0 magas)	`0x00000001`
2	`gpio_clear(base, 0x1)` (pin0 alacsony)	`0x00000000`

18.2.7 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés	Egy változtatás
rossz pin vált, más nem	pinindexet adtunk át maszk helyett	GDB: `p/x $r1` a `gpio_set`-ben	`LSL`-lel maszkká alakítani, vagy maszkot átadni
semmi nem történik a kimeneten	elmaradt `OUTENSET` (a láb bemenet maradt), vagy rossz offsetre (pl. a fenntartott `0x008`-ra) írtunk	`x/1xw 0x40010010` (hardveren)	`gpio_out_enable` a kiírás előtt, `GPIO_OUTENSET = 0x010`
a QEMU-n a demó „lefagy”	a GPIO nincs implementálva, a `DATA` mindig 0	támogatási mátrix (18.1)	`LAB_HAVE_GPIO = 0`, a próba `SKIP`
az egyik másik kimenet is átbillen	teljes szó felülírása `STR`-rel RMW helyett	olvassuk ki a `DATAOUT`-ot írás előtt/után	`ORR`/`BIC` RMW bevezetése
a beolvasott érték mindig 0	`DATAOUT`-ot olvastunk `DATA` helyett	nézzük a load offszetjét	`[r0, #GPIO_DATA]`
`ORR` nem fordul le	kódolhatatlan immediate	hibaüzenet: »invalid constant (…) after fixup«	maszk betöltése regiszterbe

18.2.8 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Mutasd be lépésenként, miért nem atomi a `LDR/ORR/STR` minta, és hasonlítsd össze a CMSDK GPIO három atomi lehetőségével: `OUTENSET/OUTENCLR` (irány), `MASKLOWBYTE/MASKHIGHBYTE` (adat) és a Cortex-M3 bit-band alias. Mikor melyiket érdemes használni?

2. (implementációs, alapszint — cél: rutinépítés) Írj `void gpio_write_pin(uint32_t base, uint32_t pin, uint32_t value)` függvényt, amely `value != 0` esetén beállítja, különben törli a pinnel jelölt bitet, a többi bitet érintetlenül hagyva.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) A megadott `gpio_demo` verzióban a `wait_button` előtt `MOVS r1, #1` (pin0 maszkja) szerepel a helyes `MOVS r1, #2` helyett:

```
movs r1, #1            @ hibás: pin0 a kimenet, nem a gomb
bl   wait_button
```

Add meg, hogy milyen viselkedést várunk, és javítsd a kódot.

4. (integráló, elmélyült — cél: fejezetek összekapcsolása: ISR írása: C és assembly vegyesen + a jelen fejezet nem-atomi RMW-korlátja) Tegyük fel, hogy a `gpio_set/gpio_clear` rutint a `main` ciklus **és** egy Timer-ISR (a070 `TIMER0_IRQHandler` mintájára) is hívja, ugyanarra a GPIO-portra. Írd le pontosan, milyen utasítássorrend vezet elveszett frissítéshez (lost update), ha a hardveres kivétel-belépés éppen a main() `LDR` és `STR` utasítása közé "ékelődik be". Mivel a Thumb-2 `LDR/ORR/STR` több különálló utasításból áll, bármelyik köztes ponton megszakítható — nincs itt egyetlen atomi RMW-utasítás. Milyen védekezési stratégiát javasolnál? Hasonlítsd össze a kritikus szakaszt a maszkolt írással és a bit-band aliasszal.

5. (integráló, elmélyült — cél: valódi bitbanging) Írj `uart_tx_bitbang(uint32_t base, uint32_t pin, uint8_t byte, uint32_t delay)` rutint, amely egy GPIO-lábon 8N1 szoftveres UART-jelet állít elő: start bit (0), 8 adatbit LSB-first, stop bit (1), bitenként `delay` ciklusos késleltetéssel. Indokold, hogyan változtatja meg a megszakítás okozta jitter a bitidőt, és hogyan lehet ezt csökkenteni.

18.2.9 Megoldások

1. megoldás — Segítség. Gondold át, mi történik, ha két végrehajtási szál *ugyanazt* a pillanatképet olvassa be `LDR`-rel, mielőtt bármelyikük visszaírna — majd nézd meg, hogy az `OUTENSET/OUTENCLR` miért nem igényel előzetes olvasást.

1. megoldás — Teljes megoldás. Az `LDR`-rel beolvasott érték egy pillanatkép. Két konkurens író ugyanazt a pillanatképet kapja, mindkettő módosít, és a későbbi `STR` felülírja az előzőt: elveszett frissítés. Az `OUTENSET/OUTENCLR` éppen azért külön regiszter, hogy a hardver belül végezze el a bitenkénti módosítást, és a szoftver csak a kívánt pozíciókra írjon 1-et; nem kell visszaolvasni. Az adatregiszternél nincs ilyen SET/CLR pár, de van két atomi alternatíva: a `MASKLOWBYTE/MASKHIGHBYTE` maszkolt írás és a Cortex-M3 bit-band alias. Ha ezek használhatók, nem kell RMW; ha nem (például más GPIO-típuson), szoftveres kölcsönös kizárás vagy kritikus szakasz kell.

2. megoldás — Segítség. A meglévő `gpio_set/gpio_clear` rutinok már elvégzik a maszkolt RMW-t; a feladat csak egy elágazás a `value` alapján, a megfelelő meglévő rutin hívásával.

2. megoldás — Teljes megoldás.

```
@ void gpio_write_pin(uint32_t base, uint32_t pin, uint32_t value)
.global gpio_write_pin
.thumb_func
gpio_write_pin:
    push {r4, lr}
    mov  r4, r0
    movs r3, #1
    lsl  r1, r3, r1            @ 1 << pin
    cmp  r2, #0
    BEQ  1f
    mov  r0, r4
    bl   gpio_set
    pop  {r4, pc}
1:  mov  r0, r4
    bl   gpio_clear
    pop  {r4, pc}
```

Az `1 << pin` maszkhoz előbb egy regiszterbe 1-et kell tölteni (`MOVS r3, #1`), majd `LSL r1, r3, r1` a helyes léptetés; a `LSL r1, r1, r1` a `pin << pin` értéket számolná. A lényeg, hogy az ág mindkét esetben a meglévő `ORR`/`BIC` maszkoló rutinra épül, így a többi bit érintetlen marad.

Ellenpróba QEMU-n: ál-GPIO RAM-ban. A `gpio_set`/`gpio_clear`/`gpio_write_pin` báziscímet kap, ezért a logika valódi GPIO nélkül is tesztelhető: a bázist egy RAM-beli tömb elejére állítva a `+0x004` (`DATAOUT`) a tömb második szava. A következő `main` a `lab-gpio-bitbang.s` rutinjaival fut, és PASS/FAIL kilépési állapottal zár:

```c
/* lab-a073-fakegpio.c - az RMW-rutinok tesztje RAM-beli ál-GPIO-n */
#include <stdint.h>

extern void gpio_set(uint32_t base, uint32_t mask);
extern void gpio_clear(uint32_t base, uint32_t mask);
extern void gpio_write_pin(uint32_t base, uint32_t pin, uint32_t value);

static uint32_t fake_gpio[16];   /* [1] = DATAOUT (a bázis + 0x004) */

static void sh_exit(int code)
{
    register int r0 __asm__("r0") = 0x18;   /* SYS_EXIT */
    register int r1 __asm__("r1") = code;
    __asm__ volatile("bkpt 0xab" :: "r"(r0), "r"(r1));
}

int main(void)
{
    uint32_t base = (uint32_t)(uintptr_t)fake_gpio;

    gpio_set(base, 0x1u);
    if (fake_gpio[1] != 0x1u) sh_exit(1);
    gpio_write_pin(base, 2, 1);
    if (fake_gpio[1] != 0x5u) sh_exit(2);
    gpio_write_pin(base, 0, 0);
    if (fake_gpio[1] != 0x4u) sh_exit(3);
    gpio_clear(base, 0x4u);
    if (fake_gpio[1] != 0x0u) sh_exit(4);
    sh_exit(0);                             /* PASS */
    for (;;) { }
}
```

Fordítás és futtatás a fenti kanonikus párral:

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 \
  -T lab-mps2.ld -o lab-a073-fakegpio.elf lab-startup.s lab-gpio-bitbang.s \
```

```
  lab-a073-fakegpio.c
timeout 5 qemu-system-arm -M mps2-an385 -cpu cortex-m3 -nographic -semihosting \
  -kernel lab-a073-fakegpio.elf </dev/null; echo "kilépési állapot: $?"
```

Várt kimenet: `kilépési állapot: 0` (PASS); hiba esetén 1–4. Az ál-GPIO a `gpio_set`/`gpio_clear` RMW-logikáját igazolja, a valódi periféria viselkedését nem.

3. megoldás — Segítség. Nézd meg, melyik pinhez tartozik az átadott maszk, és hogy az a pin éppen bemenet-e vagy kimenet — a `wait_button` csak bemeneti pin változására adhat értelmes választ.

3. megoldás — Teljes megoldás. A `wait_button` `gpio_read_input`-ot hívja `mask = 1`-gyel (pin0). Mivel pin0 épp a kimenet, a `DATA` olvasása a saját kimeneti szintet adja vissza; a ciklus azonnal vagy soha nem teljesül a gomb állapotától függetlenül. A javítás: a gomb maszkja `0b10`, azaz `MOVS r1, #2`. Ez a klasszikus „rossz maszk” hiba, amelyet a pinindex-maszk összekeverése okoz.

4. megoldás — Segítség. Kövesd nyomon lépésről lépésre: main() már elvégezte az `LDR`-t (pillanatkép `r2`-ben), amikor megszakítás érkezik — mit lát és mit ír az ISR a DATAOUT-ban, és mit ír végül vissza main() a saját, már elavult pillanatképéből?

4. megoldás — Teljes megoldás. A `main()` ciklus `LDR r2, [r0, #GPIO_DATAOUT]`-pal beolvassa a `DATAOUT` aktuális értékét egy pillanatképbe (`r2`). Mielőtt az `ORR`/`BIC` + `STR` lefutna, a Timer-ISR megszakítja a folyamot — 17.3 szerint a hardver automatikusan a verembe menti `R0`–`R3`, `R12`, `LR`, `PC`, `xPSR`-t, majd a kezelő fut. Ha az ISR maga is hívja a `gpio_set`/`gpio_clear` rutint ugyanarra a portra, a saját `LDR/ORR/STR` szekvenciája lefut, és sikeresen frissíti a `DATAOUT`-ot. A megszakításból visszatérve a `main()` ciklus **folytatja a saját, már elavult `r2` pillanatképével**: erre alkalmazza a saját `ORR`/`BIC`-jét, majd `STR`-rel visszaírja — ezzel felülírja (eltörli) az ISR által közben elvégzett módosítást. Ez pontosan az elveszett frissítés (lost update), amelyet a fejezet 1. korlátja általánosan már megemlít, itt konkrét ISR-kontextusban. Védekezés: elsőként a hardver atomi útjai jönnek szóba — a `MASKLOWBYTE`/`MASKHIGHBYTE` maszkolt írás vagy a Cortex-M3 bit-band alias `DATAOUT`-ra, mert ezek RMW nélkül, hardveresen atomiak. Ha ilyen nincs (más GPIO-típus), a megszakítások rövid letiltása (`CPSID i`/`CPSIE i`) a `main()` RMW-szekvenciája köré kritikus szakaszként — ugyanaz az elv, mint 17.3 megosztott `g_tick`/`g_pending` változóinak védelme.

5. megoldás — Teljes megoldás. A jelsorozat bitjeit egy maszkolt kiírással állítjuk elő; a késleltetés ciklusszámoló hurka:

```
@ void uart_tx_bitbang(uint32_t base, uint32_t pin, uint8_t byte, uint32_t delay)
@ r0=base, r1=pin, r2=byte, r3=delay
.global uart_tx_bitbang
.thumb_func
uart_tx_bitbang:
    push {r4-r7, lr}
    mov  r4, r0                 @ base
    mov  r5, r1                 @ pin
    mov  r6, r2                 @ byte
    mov  r7, r3                 @ delay
    movs r1, #1
    lsl  r1, r5                 @ maszk = 1 << pin
    @ start bit: 0
    bic  r2, r1
    bl   bitbang_write
    @ 8 adatbit LSB-first
    movs r5, #8
1:  movs r0, r6
    ands r0, r0, #1
    cmp  r0, #0
```

```
    BEQ  2f
    orr  r2, r1
    b    3f
2:  bic  r2, r1
3:  bl   bitbang_write
    LSRS r6, r6, #1
    subs r5, r5, #1
    BNE  1b
    @ stop bit: 1
    orr  r2, r1
    bl   bitbang_write
    pop  {r4-r7, pc}
```

A `bitbang_write` a maszkolt `DATAOUT`-írást végzi, majd `delay` ciklusig vár. A megszakítás a bitidőt megnyújtja: ha az ISR hossza összemérhető a `delay`-jel, a vevő hibás bitet mintavételez. Csökkentés: a bitidőt a megszakítások letiltásával vagy magasabb prioritású, rövid ISR-rel védeni kell; a `delay` értékét a tényleges órajelhez kell kalibrálni, és a jittert mérni kell (pl. a TIMER1 számlálójával).

18.2.10 Rövid összefoglaló

A GPIO-pin vezérlése két regiszterpár köré szerveződik: irány (`OUTENSET`/`OUTENCLR`) és adat (`DATA`/`DATAOUT`); atomi bitíráshoz a maszkolt bájtelérés (`MASKLOWBYTE`/`MASKHIGHBYTE`) szolgál. Egy bit megváltoztatása Cortex-M-en RMW-minta: `LDR` + `ORR` (beállítás) vagy `BIC` (törlés) + `STR`, ami nem atomi. A bemenetet a `DATA` adja vissza, és `CMP` után feltételes elágazással dolgozunk fel. A bitbanging szoftveresen, késleltetéssel állítja elő a jelsorozatot, ezért időzítése nem cikluspontos.

Továbbhasznált definíciók:

- **irányregiszter (direction):** bitenként kijelöli, hogy a láb bemenet- vagy kimenetirányú.
- **bitbanging:** szoftveres, programvezérelt jelforma-előállítás GPIO-pinen, dedikált periféria nélkül.
- **read-modify-write (RMW):** olvasás–módosítás–visszaírás minta egyetlen bit vagy bitcsoport megváltoztatására.

18.2.11 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Arm Cortex-M System Design Kit Technical Reference Manual, 3.5.2, 3-8. táblázat – a CMSDK GPIO regisztertérképe.
- Armv7-M Architecture Reference Manual, DDI0403E – Thumb-2 `ORR`/`BIC`, load/store immediate-korlátok. **Ellenőrizve**: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Arm M-profile architecture documentation – kiindulópont. **Ellenőrizve**: a link él; ugyanaz a JS-renderelési korlát érvényes.
- QEMU System Emulation dokumentáció – a `mps2-an385` machine és a modellezett perifériák leírása; `qemu-system-arm -M mps2-an385 -monitor stdio` + `info mtree` az ellenőrzéshez. **Ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát).

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is fennmaradnak: A regisztertérkép a CMSDK műszaki leírás 3-8. táblázatát követi (a CMSDK műszaki leírás 3-8. táblázata alapján: a `0x008`–`0x00C` fenntartott, az `OUTENSET` a `0x010`-en van). A buildparancs a fő úton, mérve (GNU + QEMU 8.2.2, a lektor mérése) hiba nélkül lefut; a `SKIP`-ág kimenete `SKIP: GPIO nincs implementalva ezen a platformon (LAB_HAVE_GPIO=0)`, a kilépési állapot 0. A `LAB_HAVE_GPIO=1` build a nem modellezett GPIO-n a `wait_button`-ben végtelenül vár, ezért `timeout`-tal leállítva 124-es állapottal ér véget. A hardveres próba (a lábak tényleges kapcsolása, a `DATAOUT`/`OUTENSET` visszaolvasása) támogatott hardveren végzendő, itt nem történt meg. Az Ubuntu 24.04 helyi csomagolt QEMU-verziója ettől eltérhet, és külön jelölést kíván. STM32-n a GPIO-regisztertérkép teljesen más (pl. `BSRR` atomi set/reset regiszter); a labor ott nem futtatható változatlanul.

18.3 Periféria-regiszter absztrakció C struktúrával

18.3.1 Motiváció

Az előző alfejezetben (GPIO közvetlen vezérlése: bitbanging) nyers memóriacímekkel és hexadecimális offsetekkel vezéreltük a GPIO-t. Ez a gép szintjén pontos, de a forráskód szintjén törékeny: egy nagyobb firmware-ben több tíz perifériaregiszter varázsszámként való karbantartása átláthatatlan és hibára hajlamos. Ha egy új perifériát kell kezelni, vagy a regiszterkészlet bővül, a nyers címek sora olvashatatlanná válik.

A mérnöki megoldás a **periféria-struktúra**: a regisztereket egy C `struct` mezőiként írjuk le, a struktúra kezdetét egy fix báziscímre illesztjük, és a továbbiakban nevesített mezőkkel érünk el mindent. Ez az absztrakció nem változtatja meg a generált gépi kódot – ugyanaz a báziscím+offset minta marad –, de a forráskód olvashatóvá, típusellenőrzötté és statikusan verifikálhatóvá válik. Ebben az alfejezetben felépítjük ezt az absztrakciót, és megmutatjuk az assembly-szintű megfelelőjét.

18.3.2 Definíció és működés

Periféria-struktúra (peripheral register struct): olyan C struktúratípus, amelynek mezői egy periféria regisztereit reprezentálják, sorrendben és méretben a hardver regisztertérképének megfelelően, és amelyet egy fordítási idejű konstans báziscímre castolunk.

A definíció három pillére:

1. **Mezők mint regiszterek.** Minden mező egy regisztert jelöl. A mező típusa a regiszter szélességét tükrözze: 32 bites regiszterhez `uint32_t`, 16 biteshez `uint16_t`, 8 biteshez `uint8_t`. A gyakorlatban a legtöbb perifériaregiszter 32 bites, ezért a `uint32_t` a leggyakoribb választás.
2. **Sorrend és offset.** A struktúra mezőinek sorrendje adja a regiszterek offset-sorrendjét. Az első mező a báziscím + 0, a második a báziscím + az első mező mérete (igazítási kitöltéssel), és így tovább. Ez pontosan a Struktúrák, igazítás és mezőelérés alfejezetben tanult mezőoffset-számítás: **mezőcím = báziscím + mezőoffset**.
3. **Fix báziscímre castolás.** A struktúra típusát egy `#define` makróval kötjük egy abszolút címhez, például `((cmsdk_gpio_t *)0x40010000UL)`. Az egész számból pointerré castolás C-ben implementációdefiniált, de a beágyazott programozás bevett mintája: a memóriába leképezett regisztereket pontosan így érünk el.

Az assembly-szintű megfelelő változatlan. A `GPIO0->OUTENSET = érték;` C kifejezésből a fordító `STR` utasítást készít, ahol a bázisregiszter a `0x40010000`, az offset pedig a mező pozíciója. A 7.3 alfejezetben (Struktúrák, igazítás és mezőelérés) tanult összefüggés tehát közvetlenül érvényes: a struktúra-alapú elérés ugyanazt a báziscím+offset mintázatot adja, mint a nyers című változat.

18.3.2.1 A volatile szerepe

A perifériaregiszterek elérése **volatile** (illékony) minősítést igényel. A `volatile` kulcsszó azt mondja a fordítónak, hogy az adott objektum értéke a program akaratán kívül is megváltozhat, ezért:

- minden olvasás és írás ténylegesen végrehajtandó (nem gyomlálható ki, nem vonható össze),
- az elérések relatív sorrendje megmarad,
- az érték nem gyorsítótárazható processzorregiszterben.

E nélkül egy `-O2` szinten optimalizáló fordító a látszólag hatástalan írásokat eltávolíthatja, vagy a read-modify-write sorozatot átrendezheti. A `volatile` azonban nem biztosít atomitást és nem zár ki versenyhelyzetet: a `GPIO0->DATAOUT |= (1u << pin);` továbbra is három gépi lépés (olvas, módosít, ír), amely megszakítás közben megszakítható. A GPIO közvetlen vezérlése: bitbanging RMW-mintája itt is érvényes.

A volatile-t két szinten lehet elhelyezni:

- **mezőnként:** a struktúra minden tagja `volatile uint32_t`; ekkor maga a struktúratípus hordozza a tulajdonságot, és a pointer nem kell volatile legyen;
- **a pointeren:** `volatile cmsdk_gpio_t *`; ekkor a mutatott objektum minden elérése volatile.

Mindkettő helyes. A mezőnkénti változat előnye, hogy a típus önmagában dokumentálja a szándékot, és később a pointer típusát nem kell módosítani. Hátránya, hogy minden meződeklarációt el kell látni a kulcsszóval. A könyv a mezőnkénti változatot használja.

18.3.2.2 Előny és kockázat

Előny: olvashatóság és karbantarthatóság. A `GPIO0->OUTENSET` név önmagát dokumentálja; a `0x40010010` cím nem. A struktúra egy helyen írja le a periféria teljes regisztertérképét, és a mezőnevekhez kötött szemantika csökkenti a téves offset használatát. A fordító típusellenőrzést végez a mezőkön, a `_Static_assert` + `offsetof` párosával pedig a struktúra elrendezése fordítási időben ellenőrizhető. Ez az ellenőrzés azonban csak azt igazolja, hogy a struktúra **azt a táblázatot** írja le, amelyet mi az assertekbe beírtunk. Ha a táblázat maga hibás, a struktúra és az assertek együtt, egymással egyezően hibásak. A hardvertérkép forrása a gyártói leírás; azt a struktúra nem helyettesíti, legfeljebb pontosan átveszi.

Kockázat: a struktúra elrendezése és a tényleges hardver-layout eltérhet. Ha a mezőtípusok nem egyeznek a regiszterek szélességével, a fordító **kitöltést (padding)** szúrhat be, és a további mezők offsetje elcsúszik. Például ha az első mező `uint8_t`, a második `uint32_t`, akkor a második a 4. bájton kezdődik a várt 1. helyett. Ezért kötelező a struktúra méretének és kulcsmező-offsetjeinek statikus ellenőrzése, és a regiszterekhez mindig a hardver szélességének megfelelő típust kell választani. A `__attribute__((packed))` nem javasolt: a fordító a mezőt igazítatlannak tekinti, és a beállítástól függően vagy igazítatlan szóelérést generál (Device memórián ez kiszámíthatatlan), vagy a regisztert bájtonként olvassa és írja (`-mno-unaligned-access`), ami egy 32 bites perifériaregiszternél hibás működés. A Cortex-M3 Normal memórián hardveresen

támogatja az igazítatlan LDR/STR/LDRH/STRH elérést (CCR.UNALIGN_TRP = 0 alapértelmezés); az LDM/STM/LDRD/STRD és az exkluzív elérések igazítatlanul UsageFault-ot adnak.

További kockázat, hogy a struktúra-alapú elérés csak akkor azonos a nyers cíművel, ha a báziscím és az offsetek pontosan egyeznek. Ezt a generált assembly összehasonlításával lehet igazolni.

18.3.3 Kidolgozott példa

Munkakönyvtár: ~/arm-labor/cm/a074/ — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a074
cd ~/arm-labor/cm/a074
```

A GPIO-műveletek implementált CMSDK GPIO perifériát és valódi bemenetet feltételeznek. A QEMU 8.2.2 mps2-an385 modelljében a GPIO nincs implementálva; ott a fizikai be-/kimeneti próba SKIP, és a regiszterek írása nem igazolja a lábak működését.

Tekintsük az MPS2-AN385 CMSDK GPIO regiszterelrendezését. A GPIO0 báziscíme a dokumentált memóriatérképen 0x40010000. A regisztertérkép az Arm Cortex-M System Design Kit műszaki leírásának 3-8. táblázatából (3.5.2) származik:

Offset	Név	Szerep
0x000	DATA	olvasás: a lábak tényleges szintje; írás: kimeneti adat
0x004	DATAOUT	kimeneti adatregiszter
0x008–0x00C	—	fenntartott (két szó)
0x010	OUTENSET	kimenet engedélyezése (set)
0x014	OUTENCLR	kimenet engedélyezése (clear)
0x018	ALTFUNCSET	alternatív funkció engedélyezése (set)
0x01C	ALTFUNCCLR	alternatív funkció engedélyezése (clear)
0x020	INTENSET	megszakítás engedélyezése (set)
0x024	INTENCLR	megszakítás tiltása (clear)
0x028	INTTYPESET	élvezérlés (set)
0x02C	INTTYPECLR	szintvezérlés (clear)
0x030	INTPOLSET	polaritás: felfutó él / magas szint (set)
0x034	INTPOLCLR	polaritás: lefutó él / alacsony szint (clear)
0x038	INTSTATUS / INTCLEAR	olvasás: függő megszakítások; írás: W1C nyugtázás

Ez a táblázat a II. rész **egyetlen forrása** a GPIO-hoz: a lab-cmsdk-gpio.h fejléc ebből készül, az 18.2 assembly-konstansai ugyanezek, és a II. záróprojekt is ezt a fejlécet másolja át, nem saját struktúrát ír. A fenntartott szavakat RESERVED0[2] tömb tölti ki, így az OUTENSET a 0x010-re kerül. A 0x038-on egy union fejezi ki, hogy ugyanazon a címen az olvasás INTSTATUS, az írás INTCLEAR (a const tag miatt az INTSTATUS-ba írni fordítási hiba). A struktúra a 0x038-ig tartó részleges térképet írja le, ezért a teljes mérete 0x03C; a 0x400–0xBFC maszkolt írási tartomány (MASKLOWBYTE/MASKHIGHBYTE) szándékosan nincs a struktúrában, azt a 18.2 külön,

címbittérképként kezeli. A fejléc a CMSDK-modellt írja le; nem állítja, hogy a QEMU 8.2.2 AN385 GPIO-eszközt valósít meg. A nyugtázás a megfelelő maszkot írja az `INTSTATUS`/`INTCLEAR` közös címére; a pin tartománya (`pin < 16`) a függvények rögzített előfeltétele:

```
/* lab-cmsdk-gpio.h - CMSDK AHB GPIO regisztertérkép (részleges, 0x000-0x038)
 * Forrás: Arm Cortex-M System Design Kit TRM, 3.5.2, 3-8. táblázat.
 * Ez a fejléc a II. rész EGYETLEN forrása a GPIO-offsetekhez; az a073
 * assembly-konstansai és a záróprojekt (a078) is ebből származnak.
 * Az offsetof-ellenőrzés csak azt igazolja, hogy a struktúra a fenti
 * táblázatot írja le; a hardver térképét nem bizonyítja. */
#ifndef LAB_CMSDK_GPIO_H
#define LAB_CMSDK_GPIO_H
#include <stddef.h>
#include <stdint.h>

typedef struct {
    volatile uint32_t DATA;          /* 0x000 RW  bemeneti szint olvasás / kimenet írás */
    volatile uint32_t DATAOUT;       /* 0x004 RW  kimeneti adatregiszter               */
    uint32_t          RESERVED0[2];  /* 0x008-0x00C fenntartott                         */
    volatile uint32_t OUTENSET;      /* 0x010 RW  1 → a láb kimenet                     */
    volatile uint32_t OUTENCLR;      /* 0x014 RW  1 → a láb bemenet                     */
    volatile uint32_t ALTFUNCSET;    /* 0x018 RW  1 → alternatív funkció                */
    volatile uint32_t ALTFUNCCLR;    /* 0x01C RW  1 → alternatív funkció ki             */
    volatile uint32_t INTENSET;      /* 0x020 RW  1 → megszakítás engedélyezve          */
    volatile uint32_t INTENCLR;      /* 0x024 RW  1 → megszakítás tiltva (NEM nyugtáz)  */
    volatile uint32_t INTTYPESET;    /* 0x028 RW  1 → élvezérelt                        */
    volatile uint32_t INTTYPECLR;    /* 0x02C RW  1 → szintvezérelt                     */
    volatile uint32_t INTPOLSET;     /* 0x030 RW  1 → felfutó él / magas szint          */
    volatile uint32_t INTPOLCLR;     /* 0x034 RW  1 → lefutó él / alacsony szint        */
    union {                          /* 0x038 ugyanaz a cím:                            */
        volatile const uint32_t INTSTATUS;  /*   olvasás: függő megszakítások          */
        volatile uint32_t       INTCLEAR;   /*   írás: 1 → az eseményjelző törlése (W1C) */
    };
} cmsdk_gpio_t;

_Static_assert(offsetof(cmsdk_gpio_t, DATA)       == 0x000, "DATA");
_Static_assert(offsetof(cmsdk_gpio_t, DATAOUT)    == 0x004, "DATAOUT");
_Static_assert(offsetof(cmsdk_gpio_t, OUTENSET)   == 0x010, "OUTENSET");
_Static_assert(offsetof(cmsdk_gpio_t, OUTENCLR)   == 0x014, "OUTENCLR");
_Static_assert(offsetof(cmsdk_gpio_t, ALTFUNCSET) == 0x018, "ALTFUNCSET");
_Static_assert(offsetof(cmsdk_gpio_t, ALTFUNCCLR) == 0x01C, "ALTFUNCCLR");
_Static_assert(offsetof(cmsdk_gpio_t, INTENSET)   == 0x020, "INTENSET");
_Static_assert(offsetof(cmsdk_gpio_t, INTENCLR)   == 0x024, "INTENCLR");
_Static_assert(offsetof(cmsdk_gpio_t, INTTYPESET) == 0x028, "INTTYPESET");
_Static_assert(offsetof(cmsdk_gpio_t, INTTYPECLR) == 0x02C, "INTTYPECLR");
_Static_assert(offsetof(cmsdk_gpio_t, INTPOLSET)  == 0x030, "INTPOLSET");
_Static_assert(offsetof(cmsdk_gpio_t, INTPOLCLR)  == 0x034, "INTPOLCLR");
_Static_assert(offsetof(cmsdk_gpio_t, INTSTATUS)  == 0x038, "INTSTATUS");
_Static_assert(offsetof(cmsdk_gpio_t, INTCLEAR)   == 0x038, "INTCLEAR");
_Static_assert(sizeof(cmsdk_gpio_t)               == 0x03C, "sizeof");

#define CMSDK_GPIO0_BASE 0x40010000UL      /* MPS2-AN385 */
#define CMSDK_GPIO1_BASE 0x40011000UL
#define CMSDK_GPIO_PINS  16u             /* lábak száma portonként: pin < 16 előfeltétel */
#define CMSDK_GPIO0 ((cmsdk_gpio_t *)CMSDK_GPIO0_BASE)
#define CMSDK_GPIO1 ((cmsdk_gpio_t *)CMSDK_GPIO1_BASE)

/* Platformtámogatás: a QEMU 8.2.2 mps2-an385 gépen a GPIO nincs implementálva.
   QEMU-buildnél LAB_HAVE_GPIO = 0, és a GPIO-próbák SKIP-et jelentenek. */
#ifndef LAB_HAVE_GPIO
#define LAB_HAVE_GPIO 0
#endif
```

```
#endif
```

A struktúra-alapú és a nyers című elérés összevetése, a megszakításkezelő függvényekkel együtt:

```
/* lab-gpio.c - struktúra-alapú és nyers című elérés, megszakítás-kezelés
 * Előfeltétel minden függvénynél: pin < CMSDK_GPIO_PINS (16). A függvények ezt
 * nem ellenőrzik; nagyobb pin érték más láb bitjét vagy (pin >= 32 esetén) C-ben
 * meghatározatlan eltolást adna. */
#include "lab-cmsdk-gpio.h"

void gpio_set_output(uint32_t pin)        { CMSDK_GPIO0->OUTENSET = 1u << pin; }

#define GPIO0_OUTENSET (*(volatile uint32_t *)(CMSDK_GPIO0_BASE + 0x010))
void gpio_set_output_raw(uint32_t pin)    { GPIO0_OUTENSET = 1u << pin; }

void gpio_write(uint32_t pin, int value)
{
    if (value) CMSDK_GPIO0->DATAOUT |= 1u << pin;
    else       CMSDK_GPIO0->DATAOUT &= ~(1u << pin);
}

void gpio_enable_interrupt(uint32_t pin)  { CMSDK_GPIO0->INTENSET = 1u << pin; }
void gpio_disable_interrupt(uint32_t pin) { CMSDK_GPIO0->INTENCLR = 1u << pin; }  /* tiltás */
/* nyugtázás, W1C */
void gpio_clear_interrupt(uint32_t pin)   { CMSDK_GPIO0->INTCLEAR = 1u << pin; }
uint32_t gpio_pending(void)               { return CMSDK_GPIO0->INTSTATUS; }
```

A `gpio_set_output` és a `gpio_set_output_raw` azonos assembly-re kell forduljon. Ezt `-O2` mellett a `arm-none-eabi-gcc -S` kimenetén vagy az objektfájl disassemblyjén ellenőrizzük:

```
arm-none-eabi-gcc -std=c11 -mcpu=cortex-m3 -mthumb -O2 -c lab-gpio.c -o lab-gpio.o
arm-none-eabi-objdump -d lab-gpio.o
```

Kimenet a lektor mérése szerint (arm-none-eabi-gcc 13.2.1, GNU as 2.42, `-O2`):

```
gpio_set_output:
        movs  r3, #1
        ldr   r2, [pc, #8]        @ literal pool: 0x40010000
        lsls  r3, r3, r0
        str   r3, [r2, #16]       @ 0x40010010 = OUTENSET
        bx    lr
        .word 0x40010000

gpio_set_output_raw:              @ bájtra azonos a gpio_set_output-dal

gpio_clear_interrupt:
        …
        str   r3, [r2, #56]       @ 0x40010038 = INTCLEAR
        …
```

A Clang-változat (amely a teljes címet `movs` + `movt` párral rakja össze) ugyanerre a báziscím+offset mintára vezet; a regiszterkiosztás fordítófüggő, ezért itt csak megjegyzésként szerepel.

A két első függvény gépi kódja bájtra azonos, és mindkettő a `0x40010010` címre ír. A GNU kimenet a bázist literal poolból tölti be, ezért az `0x10` eltolás `str r3, [r2, #16]` alakban jelenik meg; a Clang-változat a teljes címet `movs` + `movt` párral rakja össze. A báziscím+offset minta (7.3) mindkét formában ugyanaz. A `gpio_clear_interrupt` a `0x40010038`-ra, a `gpio_disable_interrupt` a `0x40010024`-re ír; a kettő tehát két különböző művelet.

Ellenőrzési feltétel: a struktúra-alapú és a nyers változat disassemblyje egyezzen, és a célcím a gyártói táblázat szerinti legyen. A fordítás QEMU nélkül is ellenőrizhető. A regiszterek tényleges

viselkedése viszont csak támogatott hardveren próbálható ki, mert a QEMU `mps2-an385` gépen a GPIO nincs implementálva (lásd a támogatási mátrixot); ott a futó próba `SKIP`.

18.3.4 Hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés	Javítás
Az írás „eltűnik”, a periféria nem reagál	Hiányzó `volatile`; a fordító elhagyta az írást	`objdump -d`: nincs `str` a várt helyen	Minden regisztermező `volatile`
A második regiszter rossz címen íródik	Padding a mezők között; rossz mezőtípus	`offsetof` kiíratása vagy `_Static_assert`	`uint32_t` minden 32 bites regiszterre; `_Static_assert`
A struktúra mérete nagyobb a vártnál	Záró padding a struktúra igazítása miatt	`sizeof` ellenőrzése	Sorrend/mezőtípusok rendezése; `_Static_assert(sizeof(...))`
Igazítatlan elérés HardFault-ot okoz	`packed` struktúra vagy `uint8_t` mező 32 bites regiszterre	Fault-regiszterek (`CFSR`) vizsgálata	Igazított, szélességhelyes mezők

Ellenpróba: a `gpio_write` ágonként egyetlen RMW-t végez, ezért ott a `volatile` hiánya nem feltétlenül látszik. Olyan próba kell, ahol két egymást követő írás van: `void pulse(void) { CMSDK_GPIO0->DATAOUT = 1; CMSDK_GPIO0->DATAOUT = 0; }`. `-O2` mellett `volatile` nélkül csak egy `str` (a 0) marad, `volatile`-lal kettő. Pollozó ciklusnál (`while (!(CMSDK_GPIO0->DATA & 2));`) a `volatile` nélküli változó olvasása kiemelhető, és a ciklus `b .` végtelen ciklussá válik. Hasonlóképpen cseréljük az `OUTENSET` mezőt `uint8_t`-re, vagy hagyjuk ki a `RESERVED0[2]` tömböt: a `_Static_assert` offset-hibát jelez. (Egy szándékosan elrontott `INTSTATUS == 0x030` assert a fordítást `static assertion failed ... INTSTATUS` hibával állítja meg; a lektor mérése szerint, arm-none-eabi-gcc 13.2.1.)

18.3.5 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd meg, miért nem elegendő magát a pointerváltozót volatile-nak jelölni (`cmsdk_gpio_t *volatile p`), ha a mezők nem azok, és miért elég ezzel szemben a `volatile cmsdk_gpio_t *p`. Melyik forma védi a perifériaregiszterek elérését?

2. (implementációs, alapszint — cél: rutinépítés) Írd meg a 18.1 UART-jának regiszterstruktúráját a `0x000`–`0x010` térképpel (`DATA`, `STATE`, `CTRL`, `INTSTATUS`/`INTCLEAR`, `BAUDDIV`), `_Static_assert` ellenőrzésekkel minden offsetre és a méretre (`0x014`). Készíts `uart_enable_tx`, `uart_disable_tx` és `uart_clear_interrupt` függvényt, és indokold, melyik regiszterbe ír a TX letiltása és melyikbe a nyugtázás.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Adott egy struktúra, amelyben a mezők `uint8_t`, `uint16_t`, `uint32_t` típusúak vegyesen. Írd ki az egyes mezők `offsetof` értékét egy rövid C programmal a gazdagépen (`gcc m.c && ./m`) vagy `qemu-aarch64`-en: ezeknél a típusoknál az elrendezés azonos az AAPCS32-évl. Hasonlítsd össze az értékeket a hardver

regisztertérképével. Azonosítsd, hol keletkezik padding, és javítsd a struktúrát úgy, hogy a `_Static_assert`-ek átmenjenek. Ezután vesd össze a struktúra-alapú és a nyers című elérés generált assemblyjét `arm-none-eabi-objdump -d`-vel.

4. **(integráló, elmélyült — cél: fejezetek összekapcsolása: ISR írása: C és assembly vegyesen)** Írj egy `GPIO0_Handler` ISR-t, amely a struktúra-alapú `CMSDK_GPIO0->INTCLEAR`-be írt maszkkal nyugtázza az eseményt (write-1-to-clear, ugyanaz a `0x038` cím, mint az `INTSTATUS` olvasása), majd egy `volatile uint8_t g_gpio_pending` megosztott jelzőt állít — pontosan 17.3 `TIMER0_IRQHandler` mintáját követve, csak itt a periféria-struktúrán keresztül, nem nyers címmel. Indokold 17.3 alapján, miért nem kell az ISR-ben explicit regisztermentés, és mi történne, ha a `g_gpio_pending`-ről lemaradna a `volatile`. Ellenőrizd `objdump`-pal, hogy a struktúra-alapú `INTCLEAR`-írás ugyanarra a `str`-mintára fordul (a `0x40010038` címre), mint egy nyers címmel írt megfelelője — ez a jelen fejezet saját ellenőrzési módszere, most az ISR kontextusában alkalmazva.

18.3.6 Megoldások

1. megoldás — Segítség. Gondold át külön, mit véd a `volatile` a *mutatott* oldalon, és mit a *pointer* oldalán — a kettő nem ugyanaz a tulajdonság.

1. megoldás — Teljes megoldás. A `volatile cmsdk_gpio_t *p` azt jelenti, hogy a mutatott objektum volatile: minden `p->mezo` elérés volatile lesz, függetlenül attól, hogy a meződeklaráció tartalmaz-e `volatile`-t. A `cmsdk_gpio_t *volatile p` ezzel szemben magát a pointerváltozót védi: a pointer értéke nem gyorsítótárazható, de a mutatott struktúra mezői nem lesznek volatile-ok, ha a mezők nem azok. A perifériaregiszterek védelméhez az első forma (vagy a mezőnkénti `volatile`) kell. A két forma kombinálható is, de a gyakorlatban a mezőnkénti `volatile` a legkevésbé félreérthető.

2. megoldás — Segítség. Indulj a 18.1 UART regisztertérképéből: melyik regiszter melyik offseten van, hol van közös címen olvasás és írás, és mekkora a struktúra a `BAUDDIV`-ig?

2. megoldás — Teljes megoldás. A struktúra a 18.1 UART-ját írja le: `DATA` `0x000`, `STATE` `0x004`, `CTRL` `0x008`, az `INTSTATUS`/`INTCLEAR` union a `0x00C`-en, `BAUDDIV` a `0x010`-en; a méret `0x014`.

```
typedef struct {
    volatile uint32_t DATA;       /* 0x000 */
    volatile uint32_t STATE;      /* 0x004 */
    volatile uint32_t CTRL;       /* 0x008 */
    union {                       /* 0x00C */
        volatile const uint32_t INTSTATUS;
        volatile uint32_t       INTCLEAR;
    };
    volatile uint32_t BAUDDIV;    /* 0x010 */
} cmsdk_uart_t;

_Static_assert(offsetof(cmsdk_uart_t, DATA)      == 0x000, "DATA");
_Static_assert(offsetof(cmsdk_uart_t, STATE)     == 0x004, "STATE");
_Static_assert(offsetof(cmsdk_uart_t, CTRL)      == 0x008, "CTRL");
_Static_assert(offsetof(cmsdk_uart_t, INTSTATUS) == 0x00C, "INTSTATUS");
_Static_assert(offsetof(cmsdk_uart_t, INTCLEAR)  == 0x00C, "INTCLEAR");
_Static_assert(offsetof(cmsdk_uart_t, BAUDDIV)   == 0x010, "BAUDDIV");
_Static_assert(sizeof(cmsdk_uart_t)              == 0x014, "sizeof");

#define CMSDK_UART0_BASE 0x40004000UL
#define CMSDK_UART0 ((cmsdk_uart_t *)CMSDK_UART0_BASE)
```

```
void uart_enable_tx(void)  { CMSDK_UART0->CTRL |= 1u; }       /* CTRL.TXEN = 1 */
void uart_disable_tx(void) { CMSDK_UART0->CTRL &= ~1u; }      /* CTRL.TXEN = 0 */
void uart_clear_interrupt(uint32_t mask) { CMSDK_UART0->INTCLEAR = mask; }
```

A TX letiltása a `CTRL.TXEN` bitet törli (a UART térképén nincs külön tiltóregiszter); a nyugtázás a `0x00C` `INTCLEAR` W1C regiszterbe ír, ugyanarra a címre, ahonnan az `INTSTATUS` olvasható.

Ellenpróba (UART0, a fő úton is): TXEN bekapcsolása előtt a `STATE.TXFULL` beállva marad, TXEN után egy karakter kiírható; az `INTSTATUS`/`INTCLEAR` viselkedése a 18.1 mintáját követi.

3. megoldás. A vegyes típusú struktúra tipikusan így néz ki:

```
typedef struct {
    uint8_t  A;   /* 0x000 */
    uint32_t B;   /* 0x004 - padding 0x001..0x003 */
    uint16_t C;   /* 0x008 */
    uint16_t D;   /* 0x00A - 16 bites regiszter */
    uint32_t E;   /* 0x00C */
} mixed_t;
```

Az `offsetof` értékek: `A`=0, `B`=4, `C`=8, `D`=10 (`0x0A`), `E`=12 (`0x0C`), `sizeof`=16 (`0x10`). Az `A` után 3 bájt kitöltés van; a struktúra végén **nincs** záró kitöltés, mert `E` a `0x0C`–`0x0F` bájtokon a 16. bájtig tart. Záró kitöltésre külön példa kell, például `{ uint32_t a; uint8_t b; }` → `sizeof` 8. A várt térkép szerint `B` a `0x004`-en, `C` a `0x008`-on, a 16 bites `D` pedig a `0x00A`-n van; a fenti elrendezés ezt teljesíti. A javítás elve: minden regiszterhez a hardver szélességének megfelelő típus, és a `_Static_assert`-ek felveszik a várt offseteket. A disassembly-ellenőrzés ezek után azt mutatja, hogy a struktúra-alapú `STR [r0, #offset]` és a nyers `STR [r0, #offset]` azonos kódot ad.

A gazdagépen futtatható ellenőrző program (a `mixed_t` definíciójával együtt):

```
/* lab-a074-offset.c - gazdagépen: gcc -std=c11 lab-a074-offset.c && ./a.out */
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

typedef struct {
    uint8_t  A;
    uint32_t B;
    uint16_t C;
    uint16_t D;
    uint32_t E;
} mixed_t;

int main(void)
{
    printf("A=%zu B=%zu C=%zu D=%zu E=%zu sizeof=%zu\n",
           offsetof(mixed_t, A), offsetof(mixed_t, B),
           offsetof(mixed_t, C), offsetof(mixed_t, D),
           offsetof(mixed_t, E), sizeof(mixed_t));
    return 0;
}
```

Kimenet (mérve a gazdagépen, `gcc -std=c11`-vel): `A=0 B=4 C=8 D=10 E=12 sizeof=16`. Ugyanez a program `qemu-aarch64`-en is lefuttatható; ezeknél a típusoknál az elrendezés az AAPCS32-évelazonos.

4. megoldás — Teljes megoldás. A `GPIO0_Handler` az `INTSTATUS` pillanatképét olvassa, majd ugyanazt a maszkot írja az `INTCLEAR`-be (W1C), és beállítja a megosztott jelzőt:

```c
volatile uint8_t g_gpio_pending;

void GPIO0_Handler(void)
{
    uint32_t s = CMSDK_GPIO0->INTSTATUS;  /* mely bitek pendültek */
    CMSDK_GPIO0->INTCLEAR = s;            /* ugyanaz a maszk: W1C */
    g_gpio_pending = 1;                   /* volatile: a main látja */
}
```

Az ISR-ben nincs szükség explicit regisztermentésre: a hardver belépéskor automatikusan menti az `r0-r3`, `r12`, `lr`, `pc`, `xPSR` regisztereket, a C-fordító pedig a callee-saved regisztereket menti. A `g_gpio_pending` `volatile` nélkül a `main` ciklus kiemelheti az olvasást, és nem látná a jelzőt. Az `objdump`-ban a struktúra-alapú `INTCLEAR`-írás a `0x40010038` címre mutató `str` (a bázis + `0x38` offset), ugyanaz, mint a nyers című változatnál. A GPIO0 vektorszáma az AN385-ön a `lab-check-vectors.py`-jal ellenőrizendő.

18.3.7 Rövid összefoglaló

A periféria-struktúra a memóriába leképezett regisztereket C `struct` mezőiként írja le, és egy fix báziscímre castolja. A mezőoffsetek adják a regisztercímeket, a báziscím+offset minta assembly szinten változatlan. A `volatile` mezők garantálják, hogy a fordító ne optimalizálja el a periféria-hozzáféréseket. Az olvashatóság előnye mellé a `_Static_assert`-ek és az `offsetof` adnak biztonságot a padding és a téves elrendezés ellen. A struktúra-alapú és a nyers című elérés generált kódja azonos kell legyen.

Továbbhasznált definíciók:

- **periféria-struktúra:** C struktúratípus, amelynek mezői egy periféria regisztereit képezik le, fix báziscímre castolva.
- **volatile:** C-minősítő; a fordító minden ilyen hozzáférést ténylegesen, a programban leírt számban és a többi volatile hozzáféréshez képest sorrendben végrehajt; az értéket nem tartja processzorregiszterben. Hardveres sorrendet és atomitást nem ad.
- **mezőoffset:** a struktúramező kezdőcíme a struktúra kezdetéhez képest; a regiszter címének kiszámításához használjuk.

18.3.8 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblázatát):

- Arm Cortex-M System Design Kit Technical Reference Manual, 3.5.2, 3-8. táblázat – a CMSDK GPIO regisztertérképe.
- Armv7-M Architecture Reference Manual, DDI0403E – Thumb-2 load/store utasítások, `STR` immediate-offset kódolás. **Ellenőrizve**: a link él, de a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Arm M-profile architecture documentation – kiindulópont a memóriatérképhez és a perifériákhoz. **Ellenőrizve**: a link él; ugyanaz a JS-renderelési korlát érvényes.
- QEMU System Emulation dokumentáció – a `mps2-an385` machine és a CMSDK GPIO modellezése; a báziscím és az offsetek ellenőrzése `qemu-system-arm -M mps2-an385 -monitor stdio` + `info mtree` paranccsal. **Ellenőrizve**: a `master` ág dokumentációja (közös verziókorlát).
- GNU Compiler Collection kézikönyv – `volatile`, `offsetof`, `_Static_assert`.
- C11 bizottsági tervezet, N1570 – struktúraelrendezés, igazítás, `volatile` szemantika.

Ellenőrzési hiányok: A regisztertérkép a CMSDK műszaki leírás 3-8. táblázatát követi. A `lab-cmsdk-gpio.h` és a `lab-gpio.c` `-std=c11 -Wall -Wextra` mellett hiba és figyelmeztetés nélkül fordul; a lektor mérése szerint (arm-none-eabi-gcc 13.2.1), és a közölt disassembly ebből a fordításból származik. A regiszterek viselkedése (nyugtázás, újbóli megszakításkérés) csak támogatott hardveren próbálható ki; ez itt nem történt meg. A `_Static_assert` offset-ellenőrzések használatához a fordító C11-támogatása szükséges; a projekt build-beállításait ennek megfelelően kell ellenőrizni. Az assembly-kimenetek `-O2` szinten várhatók, de a konkrét regiszterkiosztás fordítóverziófüggő lehet.

19 Megszakítás-vezérelt I/O és driver minta

19.1 Polling versus interrupt-vezérelt I/O

19.1.1 Motiváció

A `qemu-system-arm -machine mps2-an385` gépen futó firmware-ünknek egy soros porton másodpercenként néhány bájtot kell fogadnia, közben 1 ms-os időzítéssel mintát kell vennie, és alkalmanként egy GPIO-lábon gombnyomást kell érzékelnie (fizikai hardveren). A főciklus `while (1)` testében vagy ismételten beolvassuk a periféria állapotregiszterét, vagy a periféria jelez a processzornak, amikor teendő van. Az első megoldás néhány sorból áll, de a CPU-t folyamatosan lefoglalja. A második kevesebb ciklust pazarol, viszont bonyolultabb: megszakítás-prioritás, ISR-szerződés, megosztott állapot és versenyhelyzetek kerülnek a képbe. Ez az alfejezet a két stratégia számszerűsíthető kompromisszumait adja meg, hogy a hallgató egy konkrét feladatról önállóan el tudja dönteni, melyik illik rá — és a fejezet három konkrét esetére konkrét választ kap.

19.1.2 Definíció és működés

Polling (lekérdezéses I/O): a CPU egy ciklusban ismételten beolvassa egy periféria állapotregiszterét (pl. `STATE RXFULL` bitjét), és addig nem lép tovább a cikluson, amíg a várt feltétel nem teljesül. A periféria sosem kezdeményez; kizárólag a szoftver kérdez.

Interrupt-vezérelt I/O: a periféria kérésére az NVIC kivételt vált ki, a processzor az aktuális utasításfolyamot félreállítja, elmenti a szükséges regisztereket, majd az ISR (interrupt service routine) fut le. A vezérlést nem a szoftver kérdező mintája, hanem a hardver eseménye indítja. A belépés latenciáját és összetevőit a Megszakítás-latencia mérése és korlátai tárgyalta.

Hibrid minta: az első bájt érkezéséről megszakítás értesít, majd a szoftver rövid ideig pollozva szedi ki a további, sorozatban érkező bájtokat (FIFO-s perifériánál a FIFO tartalmát), és csak utána tér vissza a megszakításos várakozáshoz. Ez a minta a sorozatos (burst) eseményekre ad jó választ.

CPU-kihasználtság: az az arány, amely a hasznos munkára fordított és az összes eltelt ciklust veti össze. Polling esetén a várakozó ciklusok is elhasználódnak.

Energiafogyasztás: az órajel aktív tartományában a CPU dinamikus teljesítményfelvétele az aktív ciklusok arányával skálázódik; a `WFI`/`WFE` utasításokkal alvó állapotba vitt mag kevesebbet fogyaszt. Az interrupt-vezérelt minta ezt a lehetőséget természetesen kihasználja, a tiszta polling nem.

A polling és a megszakítás nem ellentétpár, hanem a reakcióidő és az erőforrás-fogyasztás két különböző pontja ugyanazon a spektrumon.

19.1.3 A döntést befolyásoló szempontok

1) Latency (reakcióidő). Polling esetén a legrosszabb eset $L_{poll}^{wc} = T_{poll}$, ahol T_{poll} a poll-ciklus periódusa (ideális esetben az az idő, amíg a `while`-test egy teljes iterációja újra a `STATE` olvasásához ér). Az átlagos késleltetés $L_{poll}^{avg} \approx T_{poll}/2$, ha az esemény egyenletesen véletlen. Interrupt-vezérelt esetben a késleltetés lényegében a L_{entry} belépési latencia plusz az ISR első utasításainak futási ideje. A polling ezt a paramétert tehát csak úgy tudja rövidíteni, ha a ciklusunk sűrűbb — ami arányosan több energiát és CPU-t visz el.

2) CPU-kihasználtság. A pollingnak két, külön számolandó modellje van. **Folyamatos busy-poll** esetén (mint a lenti `poll_loop`) a CPU minden várakozó ciklust elhasznál, a teher mindig a teljes órajel. **Időszakos lekérdezés** esetén a CPU T_{poll} időközönként egyszer ellenőriz; a lekérdezéshez ébresztés is kell, ezért a modellben $c_{poll} = c_{wake} + c_{check}$ (például $30 + 6 = 36$ ciklus): T idő alatt nagyjából T/T_{poll} (darab) olvasás történik, akkor is, ha közben nem jött esemény. Interrupt-vezérelt esetben a processzor `WFI`-vel alhat, és csak eseményenként ébred: a pluszköltség eseményenként körülbelül $L_{entry} + L_{ISR} + L_{exit}$, ahol L_{exit} a kilépési szekvencia. Nagy eseménysűrűségnél ($N \gg$ néhány ezer/másodperc) az interrupt-overhead eseményenként dominálhat, és polling olcsóbb lehet; ritka eseményeknél viszont az interrupt nyilvánvalóan nyer.

3) Energiafogyasztás. A két stratégia energiamérlege akkor nyílik szét igazán, ha a CPU-nak van alacsony fogyasztású alvó módja. A tiszta polling ciklus közben a mag aktív órajelet fogyaszt. Az interrupt-vezérelt minta tipikus szerkezete:

```
static inline uint32_t primask_save(void)
{
    uint32_t v;
    __asm__ volatile ("mrs %0, primask" : "=r"(v));
    __asm__ volatile ("cpsid i" ::: "memory");
    return v;
}

static inline void primask_restore(uint32_t v)
{
    __asm__ volatile ("msr primask, %0" :: "r"(v) : "memory");
}

while (1) {
    process_pending_events();
    uint32_t saved = primask_save();
    if (!events_pending())
        __asm volatile ("dsb\n\twfi" ::: "memory");
    primask_restore(saved);
}
```

Az `events_pending()` az alkalmazás megosztott jelzőit vizsgálja; a CMSIS PRIMASK-műveletekkel védett újraellenőrzés és `WFI` megakadályozza az elveszett ébresztést. A főciklusba engedélyezett megszakításokkal lépünk.

Armv7-M-en a `WFI` akkor is felébred, ha a függő megszakítást csak a `PRIMASK` maszkolja; a mag a `WFI` utáni utasítással folytatja, és a kezelő a `PRIMASK` törlésekor fut le. Ezért nem veszhet el az ébresztés a vizsgálat és a `WFI` között. A `WFI` (Wait For Interrupt) és a `WFE` (Wait For Event) részletek: a `WFI` bármely engedélyezett és prioritásban megfelelő megszakításra felébred; a `WFE` egy esemény-regiszter bitjére is (többprocesszoros szinkronizációhoz). A labor `WFI`-je egy *megszakításra váró (alvó) állapot*: a mag leáll, amíg egy engedélyezett megszakítás fel nem ébreszti. Ez nem azonos a gyártófüggő *mélyalvással* (Deep Sleep, `SCR.SLEEPDEEP`, órajel-/tápkapuzás), amelyhez külön konfiguráció kell; a példakód csak a `WFI`-t hajtja végre, mást nem konfigurál, és a

QEMU-ban a várakozásból fizikai fogyasztásra nem következtethetünk. Az alvó mag fogyasztása tipikusan töredéke az aktívénak, így ha az események ritkák, az interrupt-vezérelt minta energia szempontból egyértelműen kedvezőbb. Ha viszont az események sűrűn jönnek, az alvás és az ébredés közötti átmenetek tranzíciós veszteséget okoznak, és a CPU úgyis folyton fut.

4) Tervezési komplexitás. Az interrupt-vezérelt minta több hibalehetőséget hoz: megosztott állapotot kell védeni (kritikus szakasz), a prioritásokat helyesen kell beállítani (lásd NVIC konfiguráció: engedélyezés és prioritás), és figyelni kell arra, hogy az ISR rövid maradjon. A polling ezzel szemben lokálisan olvasható, szimulátorban egyszerűen követhető, és ismert ideig fut.

5) Determinisztikus időzítés. Valós idejű rendszerekben néha fontos, hogy a reakcióidő minden esetben ugyanannyi legyen. A polling ezt fix felső korláttal adja (a ciklus hossza), az interrupt L_{entry}-je pedig worst-case analízist igényel (lásd 17.4). Egyik sem csodaszer: a mérnök feladata a feladat követelményéhez illeszteni.

19.1.4 Döntési táblázat a fejezet három esetére

A fejezet Egyszerű UART driver: inicializálás, küldés/vétel megszakítással és Timer-alapú időzítés és periodikus feladatok szakaszai három tipikus esetet mutatnak be; ezekre a döntés a következő.

Eset	Eseménysűrűség	Tipikus reakcióidő-igény	Választandó stratégia	Indoklás
UART RX, soronkénti bájt	alacsony–közepes (10–10 000 B/s)	néhány tíz µs	interrupt (`RXIE` engedélyezve)	ritka, kiszámíthatatlan; `WFI`-vel alhat a mag, alacsony CPU-teher
UART TX, folyamatos blokk	magas (a vonali sebesség közelében, pl. 115 200 baud 11,5 kB/s folyamatosan)	ütemezett, tűrhető	polling a küldési sor rövid ablakában, vagy DMA (más platformon)	sűrű esemény; az interrupt-overhead eseményenként dominálhat
Timer-tick (pl. 1 kHz)	ismert, periodikus	kicsi, pontosan ismert	interrupt (általában), polling csak ha a főciklus úgyis pontosan ekkor jár	`WFI` integrálható; pontosan periodikus reakció; a főciklus nem garantáltan fut ekkor

A táblázat nem recept, hanem irányadó: a konkrét számok (ciklusidő, ISR-hossz, kívánt latencia) mindig felülírhatják.

19.1.5 Kidolgozott példa: UART RX polling vs interrupt

Munkakönyvtár: `~/arm-labor/cm/a075/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a075
cd ~/arm-labor/cm/a075
```

A példa a QEMU `mps2-an385` machine `CMSDK UART0` perifériáját használja. A báziscíme a QEMU/ARM MPS2 AN385 dokumentációja szerint `0x40004000`; a regiszter-offszetek és a `STATE`/`CTRL` bitmezők a CMSDK UART dokumentációjából valók, és **ellenőrizendők** — a gyakorlatokban a hallgatónak a QEMU `mps2-an385` forrásában vagy a hivatalos MPS2 dokumentációban vissza kell igazolnia. Az itt szereplő konstansok az elv bemutatására szolgálnak.

```
#include <stdint.h>
#include <stddef.h>

#define UART0_BASE  0x40004000u
typedef struct {
    volatile uint32_t DATA;       /* +0x00: olvasás RX adat, írás TX adat */
    volatile uint32_t STATE;      /* +0x04 */
    volatile uint32_t CTRL;       /* +0x08 */
    volatile uint32_t INTSTATUS;  /* +0x0C */
    volatile uint32_t BAUDDIV;    /* +0x10 */
} cmsdk_uart_t;

#define UART0 ((cmsdk_uart_t *)UART0_BASE)
#define STATE_RXFULL (1u << 1)   /* CMSDK RXFULL; bit0: TXFULL */
#define CTRL_RXEN    (1u << 1)
#define CTRL_RXIE    (1u << 3)
```

Polling változat. A főciklus minden iterációban megkérdezi a `STATE`-et; a bájtot a `DATA` beolvasása adja vissza (a `RXFULL` tipikusan ezzel törlődik).

A `process_rx_byte(b)` az alkalmazás bájtfeldolgozó művelete (a teljes, futtatható változat a 19.2 laborjában van). A polling közvetlenül hívja; az interrupt-változatban a főciklus fogyasztja az SPSC körpuffert, a telítettséget pedig `rx_overrun` számolja.

```
void uart_poll_forever(void) {
    while (1) {
        if (UART0->STATE & STATE_RXFULL) {
            uint8_t b = (uint8_t)UART0->DATA;
            process_rx_byte(b);
        }
        /* más feladatok is futhatnak, de a CPU sosem alszik */
    }
}
```

A megfelelő betöltés–teszt–elágazás ciklus assembly-ben (Thumb-2, Cortex-M3):

```
/* Cortex-M3 (Thumb-2), polling wait loop.  A pontos címek ellenőrizendők. */
    .syntax unified
    .thumb
    ldr     r1, =0x40004004      @ UART0 STATE
    ldr     r2, =0x40004000      @ UART0 DATA
poll_loop:
    ldr     r3, [r1]             @ STATE betöltése (volatile olvasás)
    tst     r3, #2               @ RXFULL, a STATE bit1
    beq     poll_loop            @ nincs adat: újra
    ldrb    r3, [r2]             @ adat olvasása; RXFULL törlése
    b       poll_loop
```

Interrupt-vezérelt változat. Az ISR egy soros portonkénti bájtot emel ki, és visszatér. A főciklus `WFI`-vel alhat.

```c
static volatile uint8_t rx_buf[64];
static volatile uint32_t rx_head, rx_tail, rx_overrun;

/* Az attribútum Armv7-M-en érvényes, de nem kötelező (lásd a068): a GCC itt
   a kivételtípust figyelmen kívül hagyja, és csak azt kezeli, hogy a kezelő
   4 bájtra igazított SP-vel is meghívódhat. CCR.STKALIGN = 1 mellett a sima
   void uart0_isr(void) deklaráció is helyes. */
void uart0_isr(void) __attribute__((interrupt("IRQ")));

void uart0_isr(void) {
    if (UART0->INTSTATUS & (1u << 1)) {
        UART0->INTSTATUS = 1u << 1;       /* nyugtázás a DATA olvasása előtt */
        uint8_t b = (uint8_t)UART0->DATA;
        uint32_t next = (rx_head + 1u) & 63u;
        if (next == rx_tail) rx_overrun++;
        else {
            rx_buf[rx_head] = b;
            rx_head = next;
        }
    }
}

void uart_init_irq(void) {
    UART0->INTSTATUS = 1u << 1;
    UART0->CTRL |= CTRL_RXEN | CTRL_RXIE;
    /* NVIC_EnableIRQ(UART0_IRQn); lásd 17.2 */
}

void drain_pending_events(void) {
    while (rx_tail != rx_head) {
        uint8_t b = rx_buf[rx_tail];
        rx_tail = (rx_tail + 1u) & 63u;
        process_rx_byte(b);
    }
}

void main_loop(void) {
    for (;;) {
        drain_pending_events();
        uint32_t saved = __get_PRIMASK();
        __disable_irq();
        if (rx_tail == rx_head)
            __asm volatile ("dsb\n\twfi" ::: "memory");
        primask_restore(saved);
    }
}
```

Elméleti összehasonlítás. A mennyiségek és mértékegységeik:

Jel	Jelentés	Mértékegység
f_{cpu}	órajel-frekvencia	ciklus/s
N	eseménysűrűség (beérkező bájtok)	esemény/s
T_{poll}	két lekérdezés közötti idő (időszakos modell)	s
c_{poll}	egy lekérdezés teljes költsége (ha egy timer ütemezi, az ébresztését is)	ciklus

Jel	Jelentés	Mértékegység
$c_{ISR} = L_{entry} + L_{ISR} + L_{exit}$	az ISR teljes költsége eseményenként: belépés, törzs, kilépés	ciklus/esemény
C	aktív CPU-teher	ciklus/s

A `WFI`-ben töltött idő nem aktív ciklus. A három modell terhe:

$$C_{busy} = f_{cpu}, \qquad C_{poll} = \frac{c_{poll}}{T_{poll}}, \qquad C_{irq} = N \cdot c_{ISR}.$$

Egységvizsgálat: C_{busy} maga az órajel, ciklus/s. C_{poll}: (ciklus/lekérdezés) / (s/lekérdezés) = ciklus/s. C_{irq}: (esemény/s) · (ciklus/esemény) = ciklus/s. A három teher tehát közvetlenül összevethető, és f_{cpu}-val osztva a CPU-idő arányát adja.

Megtérülési pont (időszakos lekérdezés vs. interrupt). A két teher ott egyenlő, ahol $N^* \cdot c_{ISR} = c_{poll}/T_{poll}$, azaz

$$N^* = \frac{c_{poll}}{T_{poll} \cdot c_{ISR}} \qquad \left[\frac{\text{ciklus}}{\text{s} \cdot \text{ciklus/esemény}} = \frac{\text{esemény}}{\text{s}}\right].$$

$N < N^*$ esetén az interrupt, $N > N^*$ esetén az időszakos lekérdezés használ kevesebb aktív ciklust. Az időszakos modell csak addig érvényes, amíg $N \leq 1/T_{poll}$: egy egybájtos vételi regiszternél ennél sűrűbb eseményeknél a lekérdezés bájtot veszít, a latenciakövetelményről nem is beszélve. A **folyamatos busy-poll** külön eset: az mindig f_{cpu} terhet jelent, ezért ciklusszámban csak akkor éri meg, ha az interrupt is a teljes CPU-t kitöltené, azaz $N \geq f_{cpu}/c_{ISR}$. Előnye a legrövidebb reakcióidő, nem a CPU-takarékosság.

Az alábbi illusztratív számok hipotetikusak: $f_{cpu} = 25\,\text{MHz} = 25 \cdot 10^6$ ciklus/s, $T_{poll} = 50\,\mu s$, $c_{poll} = 6$ ciklus, $c_{ISR} = 60$ ciklus/esemény (12 belépés + ~36 törzs + 12 kilépés), $N = 1000$ esemény/s. A busy-poll egy iterációja szintén 6 ciklus. A belépési latencia a Megszakítás-latencia mérése 12 ciklusos referenciaértéke. A 60 ciklus összhangban van a Timer-alapú időzítés és periodikus feladatok 46 ciklusos timer-ISR-becslésével: a UART-ISR több APB-perifériaelérése miatt valamivel hosszabb.

Mutató	Folyamatos busy-poll	Időszakos lekérdezés ($T_{poll} = 50\,\mu s$)	Interrupt
Aktív CPU-teher	$25 \cdot 10^6$ ciklus/s	6 ciklus/$50 \cdot 10^{-6}$ s = $1{,}2 \cdot 10^5$ ciklus/s	1000 esemény/s · 60 ciklus/esemény = $6{,}0 \cdot 10^4$ ciklus/s
CPU-idő aránya (C/f_{cpu})	100 %	$1{,}2 \cdot 10^5 / 25 \cdot 10^6 = 0{,}48$ %	$6{,}0 \cdot 10^4 / 25 \cdot 10^6 = 0{,}24$ %
Átlagos reakcióidő	fél iteráció: 3 ciklus/f_{cpu} = $0{,}12\,\mu s$	$T_{poll}/2 = 25\,\mu s$	belépési latencia: 12 ciklus/$f_{cpu} \approx 0{,}48\,\mu s$
Eseményenkénti feldolgozási CPU-idő	–	–	c_{ISR}/f_{cpu} = $60/25 \cdot 10^6$ s = $2{,}4\,\mu s$ (teljes ISR-költség, nem latencia)

Mutató	Folyamatos busy-poll	Időszakos lekérdezés ($T_{poll} = 50\,\mu s$)	Interrupt
Alvás lehetséges?	nem	csak a lekérdezések között, timer-ébresztéssel	igen (`WFI`)

A 2,4 μs az ISR **teljes** költsége (belépés + törzs + kilépés), ezért nem reakcióidő. A reakcióidőt csak a belépés és az ISR első utasításai határozzák meg.

Stratégia	Aktív CPU-idő	Megjegyzés
Folyamatos busy-poll	100%	a mag a teljes periódusban a `STATE` bitet olvassa
Időszakos lekérdezés 50 µs-onként	0.48%	6 ciklus / 50 µs, 25 MHz-en
Interrupt	0.24%	60 ciklus/esemény, 1000 esemény/s

A táblázat levezetett, nem mért érték; a paraméterek ($f_{cpu} = 25$ MHz, $c_{poll} = 6$ ciklus, $T_{poll} = 50$ µs, $c_{ISR} = 60$ ciklus, $N = 1000$ esemény/s) a fenti jelmagyarázatban szerepelnek.

19.1.6 Tipikus hibák és ellenpróbák

- **Hiányzó `volatile`.** A polling `STATE` olvasását `volatile` nélkül az optimalizáló fordító kiemeli a ciklusból; tünet: végtelen ciklus, a bájt sosem érkezik meg. Ellenpróba: az `-O2` mellett disassemblálva (lásd Disassembly objdump/readelf-fel) a `STATE` betöltésnek a ciklusban kell maradnia.
- **Túl hosszú ISR.** Ha az ISR-ben blokkoló `while` vagy soros portra írás van, a következő megszakítás késik, és `RXOVERRUN` (overrun) hibát kapunk. Szabály: az ISR a lehető legrövidebb, a feldolgozás a főciklusba kerül.
- **Ébresztés elmarad.** `WFI` előtt nem engedélyezett a `RXIE`, ezért a mag sosem ébred fel. Ellenpróba: az `INTSTATUS` és az NVIC `ISPR` regiszter ellenőrzése GDB-vel.
- **Rossz prioritás.** Lásd NVIC konfiguráció: engedélyezés és prioritás; a nem megfelelő prioritás váratlan preempciót, illetve egy fontos megszakítás késleltetett kiszolgálását okozza.

19.1.7 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Sorolj fel három konkrét mérnöki szempontot, amelyben a polling és az interrupt-vezérelt I/O különbözik egy Cortex-M rendszerben, és mindegyikhez adj egy példát, amikor a polling a jobb választás.

2. (implementációs, alapszint — cél: rutinépítés) Az `mps2-an385` `CMSDK UART0` perifériáján írj rövid C programot, amely `lab-uart-poll.c` néven polling alapján fogad bájtokat, és `lab-uart-irq.c` néven ugyanazt interrupt-vezérelten. Mindkettőben tedd láthatóvá a `rx_head` számlálót a UART-on (a 18.1/19.2 `uart_putc`-jével) vagy semihosting `SYS_WRITE0`-val, hogy össze lehessen hasonlítani. A bemenetet csövön át adhatod: `(sleep 1; printf 'ABCDEFGH'; sleep 1) | qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -display none -monitor none -serial stdio -kernel lab-uart.elf`.

3. (mérési/hibakeresési, középszint — cél: számítás és mérési terv) Tegyük fel, hogy $c_{ISR} = 60$ ciklus/esemény, és egy lekérdezés $c_{poll} = 6$ ciklus. Számítsd ki azt az N^* eseménysűrűséget (esemény/s), ahol az **időszakos lekérdezés** ($T_{poll} = 20\,\mu s$) és az interrupt ugyanannyi aktív CPU-ciklust használ másodpercenként, ha $f_{cpu} = 25\,\text{MHz}$. Minden köztes eredmény mellé írd ki a mértékegységet. Mi a helyzet a folyamatos busy-poll-lal? Ezután GDB és `DWT_CYCCNT` (lásd 17.4) segítségével **csak hardveren** igazold, hogy az interrupt-overhead valóban néhány tíz ciklus nagyságrendű; QEMU-n a CYCCNT nem halad, a mérés SKIP. Jegyezd meg, mennyiben tér el a fizikai Cortex-M3-tól.

19.1.8 Megoldások

1. megoldás — Segítség. Gondold végig a három fő szempontot (latencia, CPU-kihasználtság, energiafogyasztás), és mindegyikhez keress olyan helyzetet, ahol a polling egyszerűbb vagy kiszámíthatóbb.

1. megoldás — Teljes megoldás. Szempontok: (a) **latencia** — pollingnál a poll-periódustól függ, interruptnál a belépési latenciától; (b) **CPU-kihasználtság** — polling folyamatosan foglal, interrupt csak eseményenként; (c) **energiafogyasztás** — `WFI` csak interrupt-vezérelt szerkezetben hasznos. A polling jobb például gyors, sorozatos (burst) kommunikációban, ahol a CPU úgyis folyton dolgozik, vagy nagyon rövid, garantáltan T_{poll}-n belül befejeződő szekvenciákban (pl. soros konfigurációs regiszterek kiírása).

2. megoldás — Segítség. A polling változatban a főciklus olvassa a `STATE`-et; az interrupt változatban az ISR és a vektortábla bejegyzése a lényeg. Ellenőrizd, hogy a `DATA` olvasása törli-e a `RXFULL` bitet.

2. megoldás — Teljes megoldás. A polling változat lényege a `while` ciklus; az interrupt változatban `UART0->CTRL |= CTRL_RXEN | CTRL_RXIE`, az ISR neve/címe a vektortáblában legyen, `NVIC_EnableIRQ` a NVIC konfiguráció: engedélyezés és prioritás szerint. Ellenőrzés: `objdump -d` mutassa az ISR-t és a vektorbejegyzést; GDB breakpoint az ISR elején. Ha a `DATA` olvasása után nem törlődik az `RXFULL`, az ISR újra belép — ezt a `STATE`-tel kell igazolni.

3. megoldás — Segítség. Két külön terhet hasonlítasz össze: az időszakos lekérdezés c_{poll}/T_{poll} ciklus/s terhét és az interrupt $N \cdot c_{ISR}$ terhét. Először számold ki a lekérdezés ébresztéssel együtt vett ciklusköltségét ($c_{poll} = c_{wake} + c_{check}$), majd oldd meg a $N^* c_{ISR} = c_{poll}/T_{poll}$ egyenletet N^*-ra, és ellenőrizd, hogy az eredmény az időszakos modell érvényességi tartományában ($N \le 1/T_{poll}$) van-e.

3. megoldás — Teljes megoldás. Az időszakos lekérdezés terhe az ébresztéssel együtt $c_{poll} = c_{wake} + c_{check} = 30 + 6 = 36$ ciklus, így $c_{poll}/T_{poll} = 36/(20 \cdot 10^{-6}) = 1\,800\,000$ ciklus/s, ami f_{cpu}-hoz képest 7,2 %. A megtérülési pont $N^* = (c_{poll}/T_{poll})/c_{ISR} = 1\,800\,000/60 = 30\,000$ esemény/s, ami még az időszakos modell érvényességi határa ($1/T_{poll} = 50\,000$ esemény/s) alatt van. 30 000 esemény/s alatt az interrupt, fölötte a 20 µs-os lekérdezés használ kevesebb aktív ciklust. A f_{cpu} ebben a számításban nem szerepel; csak a százalékos arányhoz kell. A folyamatos busy-poll mindig $25 \cdot 10^6$ ciklus/s, vagyis 100 %. Ciklusszámban csak $N \ge f_{cpu}/c_{ISR} = 25 \cdot 10^6/60 \approx 4{,}2 \cdot 10^5$ esemény/s fölött versenyképes, amikor az interrupt is kitöltené a CPU-t; az előnye a reakcióidő. A számítás után GDB-vel és `DWT_CYCCNT`-vel csak hardveren ellenőrizhető a nagyságrend; QEMU-ban a CYCCNT nem halad, a mérés SKIP.

19.1.9 Rövid összefoglaló

A polling és az interrupt-vezérelt I/O két különböző tervezési pont a reakcióidő–CPU-teher–energiafogyasztás háromszögben. A polling egyszerű és determinisztikus felső korlátú, de aktív ciklusokat emészt, és csak a poll-periódus felbontásával érzékel. Az interrupt-vezérelt I/O alacsony latenciát és `WFI`-alvást tesz lehetővé, cserébe ISR-szerződést, prioritáskezelést és versenyhelyzet-védelmet kíván.

Továbbhasznált definíciók:

- **polling:** a CPU ismételten lekérdezi egy periféria állapotregiszterét, és addig vár, amíg a feltétel nem teljesül.
- **interrupt-vezérelt I/O:** a periféria kérésére az NVIC kivételt vált ki, és az ISR kezeli az eseményt.
- **hybrid minta:** rövid ideig tartó polling egy interrupt-értesítést követő ablakban.

19.1.10 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblázatát):

- Armv7-M Architecture Reference Manual, DDI0403E — a `WFI`/`WFE` utasítások szemantikája, a kivételmodell és az alvó módból való ébredés feltételei.
- Arm M-profile architektúra dokumentáció — az NVIC kivételkezelése, prioritási szabályai.
- QEMU System Emulation — a `mps2-an385` machine és a CMSDK perifériák memóriatérképe; a `-icount` és `WFI`-viselkedés emulációs korlátai.

Ellenőrzési hiányok: (1) a regisztertérkép a 18.1-ben bemutatott CMSDK APB UART-é; a 19.2 laborja a fő úton igazolja a címeket és a biteket; (2) a `WFI`/`WFE` pontos ébredési feltételei és a QEMU ezzel kapcsolatos esetleges eltérései a fizikai Cortex-M3-hoz képest; (3) az `L_{entry}` tipikus ciklusszáma az 17.4-ben jelzett források szerint; (4) a táblázatban szereplő `N=1000 B/s`-os példa és a hozzá tartozó ciklusszámok hipotetikus illusztrációk, nem mért értékek.

19.2 Egyszerű UART driver: inicializálás, küldés/vétel megszakítással

19.2.1 Motiváció

A beágyazott rendszerek többségében a soros kommunikáció az egyik legalapvetőbb perifériakezelési feladat. Egy szenzoradatokat fogadó, parancsokat értelmező vagy egyszerűen csak konzolüzeneteket naplózó firmware gyakran UART-on keresztül kommunikál a külvilággal. Ha a vételt pollinggal valósítjuk meg, a CPU aktívan pörög, és nem tud aludni vagy más feladatot végezni. Ezzel szemben az interrupt-vezérelt vétel lehetővé teszi, hogy a CPU alvó állapotban (`WFI`) vagy más kritikus feladat végrehajtásában legyen, amíg a periféria jelzi, hogy új bájt érkezett. Ez a minta nem csak UART-ra, hanem szinte minden bemeneti perifériára (SPI, I2C, ADC) adaptálható.

Ebben az alfejezetben egy minimális, de teljes UART drivert építünk fel a QEMU `mps2-an385` machine CMSDK APB UART perifériájára. A fókusz a vételi irányon van: a bejövő karaktereket megszakításban fogadjuk, és egy körkörös pufferbe (ring buffer) helyezzük. A küldést egyszerű pollingos `putc`-vel oldjuk meg, hogy a példa ne zsúfolódjon el. Az itt bemutatott regiszterkezelés és ISR-szerződés közvetlenül használható más ARM Cortex-M alapú driver fejlesztésénél is.

19.2.2 Definíció: UART regiszterszerződés

A **UART** (Universal Asynchronous Receiver/Transmitter) egy olyan memóriába leképezett periféria, amely párhuzamos adatokat soros formában továbbít, illetve soros adatokat párhuzamosít. A CMSDK APB UART egy egyszerű, ARM által definiált periféria, amelyet a QEMU `mps2-an385` modellje is implementál. A regisztertérképe öt 32 bites regiszterből áll, amelyek a periféria báziscíméhez képest fix offszeteken érhetők el. Az alábbi táblázat a feltételezett címezést mutatja; a pontos báziscím és offszetek a QEMU machine-definíciójából ellenőrizendők.

Regiszter	Offszet	Írás	Olvasás
`DATA`	0x000	Küldendő bájt	Vett bájt
`STATE`	0x004	—	Állapotbitek: TXFULL, RXFULL, TXOVERRUN, RXOVERRUN
`CTRL`	0x008	Vezérlés: TXEN, RXEN, TXIE, RXIE	Vezérlés visszaolvasása
`INTSTATUS`	0x00C	Write-1-to-clear: a megfelelő bitre írt 1 törli a jelzőt	Aktív megszakítások: TXINT, RXINT
`BAUDDIV`	0x010	Baud osztó értéke	Baud osztó visszaolvasása

A `CTRL` regiszter bitjei a következők:

- bit 0: `TXEN` — adó engedélyezése
- bit 1: `RXEN` — vevő engedélyezése
- bit 2: `TXIE` — adási megszakítás engedélyezése
- bit 3: `RXIE` — vételi megszakítás engedélyezése

A `STATE` regiszter bitjei:

- bit 0: `TXFULL` — adó puffer megtelt
- bit 1: `RXFULL` — vevő puffer megtelt (van olvasható adat)
- bit 2: `TXOVERRUN` — adási túlcsordulás
- bit 3: `RXOVERRUN` — vételi túlcsordulás

Az `INTSTATUS` regiszter bitjei:

- bit 0: `TXINT` — adási megszakítás aktív
- bit 1: `RXINT` — vételi megszakítás aktív

Fontos szerződés: a vételi megszakítás (`RXINT`) akkor válik aktívvá, ha a `RXFULL` bit beáll, és a `RXIE` engedélyezve van. A `DATA` regiszter olvasása törli a `STATE.RXFULL` bitet — ez megakadályozza, hogy *új* RX-megszakítás keletkezzen —, de **nem törli** az `INTSTATUS.RXINT` jelzőt, amely a hardverben attól a pillanattól önálló, “latched” (rögzített) állapotként él. A már megállapított `RXINT`-et külön, **write-1-to-clear** móddal kell törölni: az `INTSTATUS` megfelelő bitjébe 1-et kell írni. Az `INTSTATUS` puszta *olvasása* a jelzőt nem törli. Ha a `DATA` kiolvasása után elmarad az `INTSTATUS`-ba írt törlés, az NVIC a megszakítást továbbra is pendingnek látja,

és a vezérlés az ISR visszatérése után — vagy akár azonnal, mielőtt visszatérne — újra belép ugyanabba az ISR-be: **megszakítási vihart** okozva, amely a CPU-t teljesen lefoglalja. Az adási megszakítás (`TXINT`) nem szintvezérelt: akkor áll be (latched), amikor egy bájt elküldése után az adópuffer kiürül, és a `TXIE` engedélyezett. Üres pufferrel a `TXIE` bekapcsolása önmagában nem kér megszakítást, ezért a TX-megszakításos küldést az első bájt beírásával kell elindítani. Fontos, hogy ennek a fordítottja NEM igaz: a `DATA` regiszter *írása* a puffert feltölti, és ettől a `TXFULL` jellemzően **beáll**, nem törlődik — a `TXFULL` törlését a hardver tényleges adatküldése végzi, amikor a puffer a soros vonalra kiürül, függetlenül attól, hogy a szoftver mikor írt utoljára a `DATA`-ba. A `TXINT` jelzőt ugyanúgy az `INTSTATUS` megfelelő bitjébe írt 1-gyel kell törölni, mint az `RXINT`-et.

A **körkörös puffer** (ring buffer) egy olyan adatszerkezet, amely fix méretű tömbben tárolja az elemeket, és két indexszel (`head`, `tail`) jelöli a következő írási és olvasási pozíciót. Az egyik indexet a termelő (itt az ISR), a másikat a fogyasztó (itt a fő ciklus) módosítja. A puffer akkor üres, ha `head == tail`, és akkor tele, ha `(head + 1) % méret == tail`. A méretet érdemes 2 hatványára választani, hogy a modulo helyett bitmaszkolást használhassunk.

19.2.3 Baud rate konfiguráció

A `BAUDDIV` regiszter a periféria bemeneti órajelét osztja le a kívánt baud rátára. A pontos összefüggés a CMSDK APB UART specifikációjában található; tipikusan a következő alakú:

$$\text{BAUDDIV} = \frac{f_{\text{UARTCLK}}}{\text{baudrate}}$$

ahol f_{UARTCLK} a UART periféria számára biztosított rendszeróra frekvenciája. A QEMU `mps2-an385` esetében ez az érték a machine konfigurációjától függ, és a pontos frekvencia a QEMU forráskódjából vagy az MPS2 AN385 dokumentációból ellenőrizendő. A QEMU CMSDK UART modellje a baud rátát `baudrate = UARTCLK / BAUDDIV` alakban számítja — nincs `-1` eltolás a képletben. A példában 25 MHz-es UARTCLK-t feltételezünk, ami 115200 baud esetén `BAUDDIV = 25000000 / 115200  217` értéket ad. Mivel a QEMU nem valós idejű emulációt végez, a baud rate pontossága a konzol bemenetére nincs hatással, de a regisztert a periféria inicializálásához be kell állítani.

19.2.4 Kidolgozott példa: UART driver

Munkakönyvtár: `~/arm-labor/cm/a076/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a076
cd ~/arm-labor/cm/a076
```

Az alábbi példa egy teljes, fordítható UART driver vázát mutatja. A kód C nyelvű, mert a regiszterelérések és a ring buffer kezelése így olvashatóbb; az assembly szerepe a vektortábla és a startup kód, amelyet a korábbi fejezetekből ismerünk. A futtatáshoz szükséges teljes fájllista: `lab-mps2.ld`, `lab-a076-startup.s` (a kanonikus startup, az alábbi bővítéssel), `lab-uart.h`, `lab-uart.c`, `lab-main.c` és az ellenőrző `lab-check-vectors.py`.

A linker script és a startup a II. rész kanonikus párja (Linker script alapjai, Startup kód), a vektortábla ellenőrzője pedig a Vektortábla alfejezet `lab-check-vectors.py`-ja. A linker script változatlan; a startup másolatának vektortábláját egyetlen, gépileg alkalmazható lépéssel bővítjük az UART0 RX (IRQ0) bejegyzésével:

```
cp ~/arm-labor/cm/a065/lab-mps2.ld .
cp ~/arm-labor/cm/a066/lab-startup.s lab-a076-startup.s
cp ~/arm-labor/cm/a064/lab-check-vectors.py .
cat > lab-a076-irq.inc <<'EOF'
    /* 16-tól: külső IRQ-k; UART0 RX = IRQ0 = 16. index */
    .word UART0_Handler           /* 0x40: IRQ0 (UART0 RX)       */
EOF
sed -i '/\.word SysTick_Handler/r lab-a076-irq.inc' lab-a076-startup.s   # a SysTick sor után
# gyenge alias: kezelő nélkül is linkelhető
echo '    lab_default_handler UART0_Handler' >> lab-a076-startup.s
# ellenőrzés: a bővített tábla vége
sed -n '/\.word SysTick_Handler/,/^$/p' lab-a076-startup.s
```

A `sed` mintája csak a vektortábla `.word SysTick_Handler` sorára illeszkedik, a fájl végén lévő `lab_default_handler SysTick_Handler` aliasra nem. A fel nem használt IRQ-helyek a kanonikus `Default_Handler`-re mutatnak. Az `echo` sor a kanonikus `lab_default_handler` makróval a fájl végére egy **gyenge**, `Default_Handler`-re mutató aliast fűz a használt kezelőhöz is. A C-fájl erős definíciója ezt felülírja, de ha egy build a kezelőt nem tartalmazza, a `KEEP`-pel megtartott vektortábla hivatkozása akkor sem marad feloldatlan. Ezek **valódi táblaszavak**, nem megjegyzések, mert csak így kerül a használt kezelő a megfelelő indexre ($16 + \mathrm{IRQn}$).

19.2.4.1 Regiszterdefiníciók és ring buffer

```
// lab-uart.h
#ifndef LAB_UART_H
#define LAB_UART_H

#include <stdint.h>

#define UART0_BASE 0x40004000UL
#define UART0_IRQn 0  // AN385: UART0 RX = IRQ0, TX = IRQ1 (lásd 18.1 és 20.2)

typedef struct {
    volatile uint32_t DATA;
    volatile uint32_t STATE;
    volatile uint32_t CTRL;
    volatile uint32_t INTSTATUS;
    volatile uint32_t BAUDDIV;
} CMSDK_UART_TypeDef;

#define UART0 ((CMSDK_UART_TypeDef *) UART0_BASE)

#define UART_CTRL_TXEN  (1U << 0)
#define UART_CTRL_RXEN  (1U << 1)
#define UART_CTRL_TXIE  (1U << 2)
#define UART_CTRL_RXIE  (1U << 3)

#define UART_STATE_TXFULL (1U << 0)
#define UART_STATE_RXFULL (1U << 1)

#define UART_INTSTATUS_TXINT (1U << 0)
#define UART_INTSTATUS_RXINT (1U << 1)

/* Előfeltétel: uartclk > 0, baudrate > 0, a BAUDDIV = uartclk / baudrate osztó a CMSDK UART
 * dokumentált tartományában van (minimum 16 a CMSDK TRM szerint - portoláskor ellenőrizendő;
 * a mező 20 bites), és a tényleges baud relatív hibája  3 %. Hibakód: 0 = OK, -1 = nulla
 ↪  paraméter,
 * -2 = osztó a tartományon kívül, -3 = túl nagy baudhiba. (Ez nem általános Cortex-M szabály.)
 ↪  */
int uart_init(uint32_t uartclk, uint32_t baudrate);
```

```c
void uart_putc(char c);
int  uart_getc(void);
int  uart_available(void);

#endif

// lab-uart.c
#include "lab-uart.h"

#define RX_BUF_SIZE 64  // 2 hatványa
static volatile char     rx_buf[RX_BUF_SIZE];
static volatile uint32_t rx_head = 0; // csak az ISR írja
static volatile uint32_t rx_tail = 0; // csak a fő ciklus írja

int uart_init(uint32_t uartclk, uint32_t baudrate) {
    if (uartclk == 0 || baudrate == 0) return -1;
    uint32_t div = uartclk / baudrate;                    // baudrate = UARTCLK / BAUDDIV
    if (div < 16 || div > 0xFFFFFu) return -2;            // 20 bites mező, CMSDK-minimum
    uint32_t actual = uartclk / div;                      // tényleges baud
    uint32_t diff = actual > baudrate ? actual - baudrate : baudrate - actual;
    if ((uint64_t)diff * 100u > (uint64_t)baudrate * 3u) return -3;  // > 3 % relatív hiba
    UART0->CTRL = 0; // kikapcsolás
    UART0->BAUDDIV = div;
    UART0->CTRL = UART_CTRL_TXEN | UART_CTRL_RXEN | UART_CTRL_RXIE;
    return 0;
}

void uart_putc(char c) {
    while (UART0->STATE & UART_STATE_TXFULL) {
        // vár, amíg az adó puffer kiürül
    }
    UART0->DATA = (uint32_t)(uint8_t)c;
}

int uart_getc(void) {
    if (rx_head == rx_tail) {
        return -1;
    }
    uint8_t c = (uint8_t)rx_buf[rx_tail];
    rx_tail = (rx_tail + 1) & (RX_BUF_SIZE - 1);
    return (int)c;
}

int uart_available(void) {
    return (rx_head != rx_tail);
}

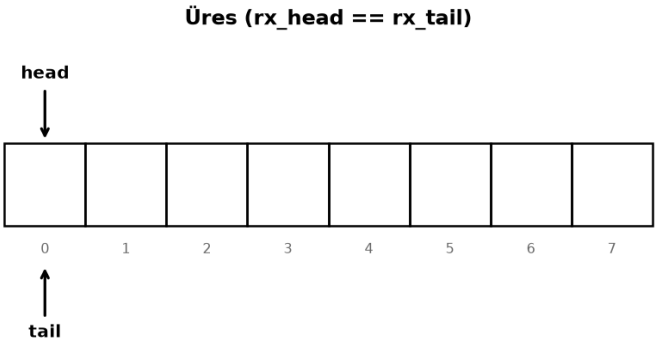
void UART0_Handler(void) {
    uint32_t status = UART0->INTSTATUS;
    if (status & UART_INTSTATUS_RXINT) {
        // write-1-to-clear: torli az INTSTATUS.RXINT-et
        UART0->INTSTATUS = UART_INTSTATUS_RXINT;
        char c = (char)(UART0->DATA & 0xFF);     // olvasás törli a STATE.RXFULL-t
        uint32_t next = (rx_head + 1) & (RX_BUF_SIZE - 1);
        if (next != rx_tail) {
            rx_buf[rx_head] = c;
            rx_head = next;
        }
        // else: a szoftveres ringbe nem fér be, a már kiolvasott bájt elvész (rx_dropped++ a 2.
        ↪  feladatban)
    }
    if (status & UART_INTSTATUS_TXINT) {
        UART0->INTSTATUS = UART_INTSTATUS_TXINT; // write-1-to-clear
        // Adási megszakítás kezelése; itt nem használjuk
```

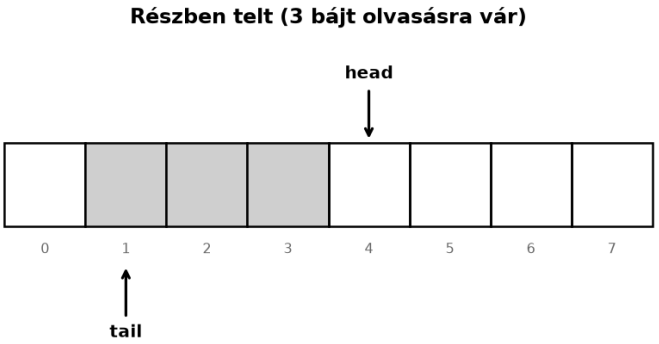
```
    }
}
```

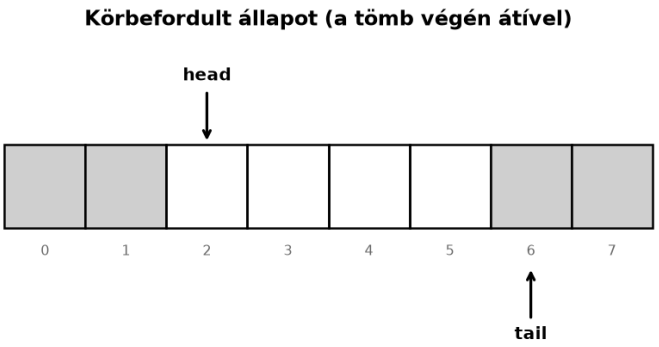
Az alábbi ábra a körpuffer négy jellemző állapotát mutatja — üres, részben telt, körbefordult, tele. A szemléltetéshez 8 szeletet használunk a tényleges 64 helyett, hogy az ábra olvasható maradjon; a `head`/`tail` logika (és a telítettség feltétele) azonban azonos a fenti `rx_buf`-éval.

Az rx_buf körpuffer állapotai: üres, részben telt, körbefordult, tele

N = 8 a szemléltetéshez (a valódi rx_buf mérete RX_BUF_SIZE = 64).









Egy rekesz szabadon marad az üres és teli állapot megkülönböztetéséhez; teli puffernél az újonnan érkező bájtot eldobjuk.

19.1. ábra: Az rx_buf körpuffer állapotai: üres, részben telt, körbefordult, tele - Szemléltetés N=8 szelettel; a valódi rx_buf mérete RX_BUF_SIZE = 64.

19.2.4.2 Fő program és NVIC engedélyezés

```
// lab-main.c
#include "lab-uart.h"

static void nvic_enable_irq(int irqn) {
    volatile uint32_t *nvic_iser = (volatile uint32_t *)0xE000E100;
    nvic_iser[irqn / 32] = (1U << (irqn % 32));
}

int main(void) {
    if (uart_init(25000000, 115200) != 0) {   // feltételezett UARTCLK; hibás paraméternél nem
    ↪  indulunk
        for (;;) { }
    }
    nvic_enable_irq(UART0_IRQn);

    const char *msg = "UART ready\r\n";
    for (const char *p = msg; *p; ++p) {
        uart_putc(*p);
    }

    while (1) {
        int c = uart_getc();
        if (c >= 0) {
            uart_putc((char)c); // echo
        }
    }
}
```

Az egyszerűség kedvéért a főciklus itt nem alszik; az alvó változatot lásd 19.3.

19.2.4.3 Build és futtatás QEMU-ban

```
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 -T lab-mps2.ld \
  -o lab-uart.elf lab-a076-startup.s lab-uart.c lab-main.c
python3 lab-check-vectors.py lab-uart.elf 1=Reset_Handler 16=UART0_Handler
timeout 10 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
  -display none -monitor none -serial stdio -kernel lab-uart.elf
```

Kilépés interaktív terminálon: `Ctrl-A`, majd `X`. Automatikus, megismételhető próba:

```
(sleep 1; printf 'ABCDEFGH'; sleep 1) | timeout 4 qemu-system-arm -machine mps2-an385 \
    -cpu cortex-m3 -display none -monitor none -serial stdio -kernel lab-uart.elf
```

Várt kimenet: `UART ready`, majd `ABCDEFGH`.

A `lab-a076-startup.s` vektortáblájában a `UART0_Handler` az IRQ0 helyén (16. index, `0x40`) áll; ezt a `lab-check-vectors.py` utolsó sora gépileg igazolja. A `-serial stdio` a vendég UART0-ját a standard bemenetre/kimenetre köti; a `-display none -monitor none` kikapcsolja a grafikus ablakot és a QEMU-monitort, így a stdin nem oszlik meg a monitor és a vendég között. A várt kimenet: a konzolon megjelenik az `UART ready` üzenet, majd a beírt karakterek visszaíródnak (echo). Mivel a QEMU emuláció nem valós idejű, a beírt karakterek megjelenési sorrendje és késleltetése eltérhet a fizikai hardveren tapasztalttól; ez nem hiba, hanem az emuláció jellemzője.

19.2.5 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés	Javítás
Az első karakter után több nem érkezik	A `DATA`-t nem olvassuk ki, ezért a `STATE.RXFULL` 1 marad, és a periféria a további bájtokat nem fogadja (overrun)	GDB-vel az ISR után nézzük meg a `STATE` értékét: ha `RXFULL` 1, ez a hiba	Az ISR-ben mindenképpen olvassuk ki a `DATA` regisztert, még akkor is, ha a bájtot eldobjuk
Az ISR azonnal, folytonosan újra belép (megszakítási vihar)	A `DATA` olvasása csak a `STATE.RXFULL`-t törli; az `INTSTATUS.RXINT` jelző „latched”, write-1-to-clear nélkül megmarad	GDB-vel az ISR-ből visszatérés után nézzük meg az `INTSTATUS` értékét: ha `RXINT` még 1, ez a hiba	Írjunk explicit 1-et az `INTSTATUS` megfelelő bitjébe az ISR-ben, **a `DATA` kiolvasása előtt**, ahogy a driver teszi
Gyorsan egymás után érkező bájtoknál a vétel elakad, egy bájt beragad	A nyugtázás a `DATA` olvasása **után** áll: a két művelet között érkező új bájt `RXINT`-jét a késői W1C letörli, a bájt `RXFULL = 1` mellett, megszakítás nélkül marad a regiszterben	`STATE` és `INTSTATUS` az elakadáskor: `RXFULL = 1`, `RXINT = 0`	A sorrend cseréje: előbb W1C, utána `DATA`-olvasás
A beírt karakterek egyáltalán nem jelennek meg	A vételi megszakítás nincs engedélyezve (`RXIE` hiányzik), vagy az NVIC-ben nem engedélyeztük az IRQ-t	Ellenőrizzük a `CTRL` és az NVIC `ISER` regiszterét GDB-vel	Állítsuk be a `RXIE` bitet, és hívjuk meg a `nvic_enable_irq(UART0_IRQn)`-t
A ring buffer tartalma „elavult”, a fő ciklus nem látja a friss adatot	A `rx_head` vagy `rx_tail` változó nem `volatile`, a fordító optimalizálja	Fordítsuk `-O0`-val: ha a hiba eltűnik, és `-O2`-vel visszatér, valószínűleg hiányzó `volatile` az ok; igazolás `objdump`-dal (a ciklusban van-e `ldr`)	Tegyük `volatile`-vá a ring buffer indexeit és a puffert
A beírt karakterek minden második esetben jelennek meg	Az ISR-ben a `DATA` regisztert kétszer olvassuk (pl. a `STATE` ellenőrzése után is), vagy a `rx_head` növelése hibás	Szúrjunk be egy számlálót az ISR-be, és figyeljük a `rx_head` értékét	Egyszer olvassuk a `DATA`-t, és a `rx_head` növelését egyetlen atomi műveletsorként végezzük
A QEMU nem indítja el a firmware-t, vagy hibás címen áll meg	A linker script memóriatérképe nem egyezik a QEMU machine memory mapjével	Ellenőrizzük a `readelf -S lab-uart.elf` kimenetét, és indítsuk a QEMU-t `-S -gdb tcp::3333`-mal; GDB-ben `info registers pc sp` és `x/2xw 0`	Igazítsuk a linker scriptet az MPS2 AN385 flash/RAM tartományához

A **sorrend** a perifériától függő eseményprotokoll része, nem általános szabály. A CMSDK UART-nál az `RXINT` a bájt érkezésekor beáll, és csak W1C törli; a `DATA` olvasása az `RXFULL`-t törli, és ezzel engedi a következő bájt fogadását. Ha a driver előbb nyugtáz, majd olvas, egy közben érkező bájt újra beállítja az `RXINT`-et, és új megszakítást kér. Fordított sorrendnél a `DATA` olvasása után, de a W1C előtt érkező bájt jelzését a W1C letörli: a bájt bent ragad, és mivel

`RXFULL = 1`, további bájt sem érkezik. A helyettesítő eszközlánccal (`tools/cm-run.py`, sűrűn érkező `ABCDEFGH` bemenettel) ez kimutatható. A lektor mérése szerint (GNU + QEMU 8.2.2) a helyes sorrenddel 36 sűrűn érkező bájt mindegyike visszaíródik, a felcserélt sorrenddel viszont csak az első bájt (`A`) jön vissza, utána a vétel elakad; ritkán érkező bájtoknál a két változat nem különbözött.

A felcserélt sorrend futtatható ellenpróbája:

```
cp lab-uart.c lab-uart-swapped.c
cat > swap.sed <<'EOF'
s|UART0->INTSTATUS = UART_INTSTATUS_RXINT;|@@RXINT@@|
s|char c = (char)(UART0->DATA \& 0xFF);.*|UART0->INTSTATUS = UART_INTSTATUS_RXINT;|
s|@@RXINT@@|char c = (char)(UART0->DATA \& 0xFF);|
EOF
sed -i -f swap.sed lab-uart-swapped.c
arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 \
  -T lab-mps2.ld -o lab-uart-swapped.elf \
  lab-a076-startup.s lab-uart-swapped.c lab-main.c
(sleep 1; printf 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'; sleep 1) | \
  timeout 4 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
  -display none -monitor none -serial stdio -kernel lab-uart-swapped.elf
```

A lektor mérése szerint a kimenet ekkor `UART ready`, majd csak `A`; a helyes sorrendű változattal mind a 36 bájt visszaíródik. A timer más eset: ott nincs kiolvasandó adat, a törlésnek csak a visszatérés előtt kell megtörténnie (ISR írása: C és assembly vegyesen).

19.2.6 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Ismertesd a CMSDK UART `STATE` és `INTSTATUS` regisztereinek bitjeit! Magyarázd el, hogyan törlődik az RX és a TX megszakítás, és miért nem elegendő az `INTSTATUS` regiszter olvasása a megszakítás törléséhez!

2. (implementációs, középszint — cél: ring buffer és overrun-számlálás) Egészítsd ki a ring buffert egy `volatile uint32_t rx_dropped` számlálóval, amely a **szoftveres ringbe nem fért, már kiolvasott** bájtokat számolja! Ez nem teljes végponti veszteségszám: a hardveres overrun (a `DATA` időben ki nem olvasása miatt elvesző bájt, `STATE.RXOVERRUN`) külön eseményként jelentendő (és W1C-vel törlendő), mert azt a szoftveres számláló nem látja; teljes veszteségszámhoz a protokollnak sorszámot kell hordoznia. Készíts egy `uart_get_dropped(void)` függvényt, és külön teszteld a ring-full és a periféria-overrun esetet! Ügyelj arra, hogy a számláló írása az ISR-ben, olvasása a fő ciklusban történik, és a 32 bites írás/olvasás atomi a Cortex-M3-on.

3. (hibakeresési/mérési, középszint — cél: determinisztikus hiba és ellenpróba) A `lab-uart.c` alábbi hibás változatában a fő ciklus a `uart_getc()`-et kétszer hívja: `if (uart_getc() >= 0) uart_putc(uart_getc());`. Emiatt a beírt karakterek fele elvész. Azonosítsd a hibát, készíts ellenpróbát (számláló az ISR-ben és a visszaírt karakterek számlálója a fő ciklusban), javítsd a kódot, és QEMU-ban mérd meg a javított és a hibás változat különbségét!

19.2.7 Megoldások

1. megoldás — Segítség. Gondold végig, melyik regiszter melyik állapotot tükrözi, és melyik művelet (olvasás vagy írás) törli a jelzőt. Külön figyelj a `STATE` és az `INTSTATUS` szerepére.

1. megoldás — Teljes megoldás. A `STATE` regiszter bit 0 a `TXFULL`, bit 1 az `RXFULL`. Az `INTSTATUS` bit 0 a `TXINT`, bit 1 az `RXINT`. Az RX megszakítás akkor aktív, ha `RXFULL && RXIE`. A `DATA` regiszter olvasása törli a `STATE.RXFULL`-t, ami megakadályozza, hogy *új* RX-megszakítás

keletkezzen — de az INTSTATUS.RXINT jelző ettől önállóan, “latched” módon megmarad, mert az INTSTATUS **puszta olvasása** csak visszajelzi az állapotot, nem módosítja azt. A már bekövetkezett RXINT-et (és TXINT-et) ezért külön, **write-1-to-clear** móddal kell törölni: 1-et kell írni az INTSTATUS megfelelő bitjébe. A TX megszakítás latched: akkor áll be, amikor egy bájt elküldése után a puffer kiürül, és a TXIE engedélyezett; üres pufferrel a TXIE önmagában nem kér megszakítást. A TXFULL-t a hardver tényleges adatküldése törli, nem a DATA-ba írás — a DATA írása feltölti a puffert, és ettől a TXFULL jellemzően beáll. A TXINT jelzőt emellett ugyanúgy explicit kell törölni az INTSTATUS-ba írt 1-gyel. Ha ez elmarad, az NVIC a megszakítást pendingnek látja, és az ISR azonnal újra belép — megszakítási vihart okozva.

2. megoldás — Segítség. A ring buffer telítettségét a `next == rx_tail` feltétel jelzi, a hardveres overrun pedig a STATE.RXOVERRUN bitjéhez kapcsolódik. A számláló 32 bites, ezért a Cortex-M3 egyetlen LDR/STR utasítással atomian éri el.

2. megoldás — Teljes megoldás.

```
// lab-uart.c kiegészítés
static volatile uint32_t rx_dropped = 0;

void UART0_Handler(void) {
    uint32_t status = UART0->INTSTATUS;
    if (status & UART_INTSTATUS_RXINT) {
        UART0->INTSTATUS = UART_INTSTATUS_RXINT; // write-1-to-clear
        char c = (char)(UART0->DATA & 0xFF);
        uint32_t next = (rx_head + 1) & (RX_BUF_SIZE - 1);
        if (next != rx_tail) {
            rx_buf[rx_head] = c;
            rx_head = next;
        } else {
            rx_dropped++;
        if (status & (1u << 3)) {           /* RXOVERRUN */
            UART0->INTSTATUS = 1u << 3;     /* W1C */
            UART0->STATE = 1u << 3;         /* W1C */
            rx_overrun++;
        }
        }
    }
    // ...
}

uint32_t uart_get_dropped(void) {
    return rx_dropped;
}
```

A `rx_dropped` 32 bites, aligned változó; a Cortex-M3 egyetlen LDR/STR utasítással éri el, ezért az írás és olvasás atomi. Nem kell kritikus szakasz. A hardveres overrunhoz a CTRL.RXORIE (bit5) engedélyezése és az ISR-ben a fenti INTSTATUS/STATE W1C-törlés kell. Ring-full teszt: a főciklus az első 100 ms-ban ne olvasson, közben 100 bájt érkezik csövön át; a várt `rx_dropped` $= 100 - 63 = 37$. Hardveres overrun QEMU-n nem idézhető elő, mert a modell nem ad új bájtot, amíg az RXFULL be nem áll.

3. megoldás — Segítség. A hiba a `uart_getc()` kétszeri hívásában van; gondold végig, melyik hívás melyik bájtot fogyasztja el, és hogyan lehet egyetlen olvasással helyettesíteni.

3. megoldás — Teljes megoldás. A hiba determinisztikus: az `if (uart_getc() >= 0)` feltétel kiértékelése már kiolvassa és eldobja az első karaktert, majd a `uart_putc(uart_getc())` a *következő* karaktert olvassa ki és küldi vissza. Így minden második beírt karakter elvész; 10 beírt karakterből a hibás változat 5-öt ír vissza, a javított 10-et. Az ellenpróba: az ISR növel egy `rx_events` számlálót, a fő ciklus egy `tx_chars` számlálót; a várt `tx_chars == rx_events`

a javított változatban, míg a hibásban `tx_chars  rx_events/2`. A javítás: egyetlen olvasás, eltárolt értékkel:

```
int c = uart_getc();
if (c >= 0)
    uart_putc((uint8_t)c);
```

A QEMU-ban a `-serial stdio` mellett küldött 10 karakterből a hibás változat 5 visszhangot ad, a javított 10-et; a számlálók a `status` üzenetben ellenőrizhetők.

19.2.8 Rövid összefoglaló

Mi vihető át más Cortex-M UART-ra? Átvihető a *módszer*: szerződésből induló MMIO-driver, egyírós/egyolvasós (SPSC) ring buffer és ISR, az ISR rövidsége. A CMSDK-specifikus állítások (regiszterek címei és bitmezői, a `STATE`/`INTSTATUS` szemantika, az ACK-sorrend, a BAUDDIV-minimum) portoláskor újra ellenőrizendők a célperiféria dokumentációjából. Portolási lista: (1) regisztertérkép és hozzáférés-típusok, (2) az interrupt-forrás jellege (szint vagy latch) és a nyugtázás sorrendje, (3) a baudosztó képlete és tartománya, (4) az IRQ-szám és a prioritás, (5) az overrun-jelzés helye. Az UART driver inicializálása a `CTRL`, `BAUDDIV` és `INTSTATUS` regiszterek konfigurációjából áll. A vételi megszakítás az ISR-ben két lépésben törlődik: a `DATA` regiszter olvasása a `STATE.RXFULL` bitet törli, az `INTSTATUS` megfelelő bitjébe írt 1 (write-1-to-clear) pedig a tényleges megszakításjelzőt — mindkettő szükséges, különben megszakítási vihar keletkezik. A karaktert ezután egy körkörös pufferbe írjuk. A ring bufferben az ISR előbb a puffert írja, és csak utána lépteti a `head`-et; a főciklus előbb olvas, és csak utána lépteti a `tail`-t. Mindkét index `volatile`, 32 bites, egy író – így nincs szükség kritikus szakaszra. A küldés polling-alapú egyszerűsítéssel történik. A QEMU `mps2-an385` machine-en a periféria címei és IRQ számai ellenőrizendők.

Továbbhasznált definíciók:

- **UART:** univerzális aszinkron adó-vevő, memóriába leképezett periféria.
- **ring buffer:** fix méretű körkörös puffer `head` és `tail` indexszel.
- **ISR:** hardveresen meghívott megszakításkezelő rutin, amely az eseményt törli és feldolgozza.

19.2.9 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblá-ját):

- Armv7-M Architecture Reference Manual, DDI0403E — a kivételmodell, az NVIC és a memóriába leképezett perifériák általános szerződése.
- Arm M-profile architektúra dokumentáció — az NVIC prioritáskezelése, a megszakítások engedélyezése.
- QEMU System Emulation — a `mps2-an385` machine és a CMSDK perifériák memóriatérképe; a `-nographic` és `-kernel` kapcsolók viselkedése.
- Arm Cortex-M System Design Kit Technical Reference Manual (DDI0479), APB UART fejezet — a `DATA`, `STATE`, `CTRL`, `INTSTATUS`, `BAUDDIV` regiszterek bitmezői és a baud osztó képlete.

A laborkönyvtár összeállítását a helyettesítő eszközlánccal (Clang + `ld.lld`) ellenőriztük: egy üres könyvtárban, kizárólag a Vektortábla, a Linker script alapjai, a Startup kód és ennek az alfejezetnek a fájljaiból, a fenti `cp`/`cat`/`sed` lépésekkel a labor fordul és linkelődik, és a `lab-check-vectors.py` a Reset- és a használt IRQ-bejegyzést is a várt szimbólumra mutatónak

találja. A `tools/cm-run.py` UART0-modelljében (`--uart-stdin`) a program kiírta az `UART ready` sort, és a bemenetet visszaírta. **Mérési állapot:** a lektor mérése szerint (GNU + QEMU 8.2.2) a könyv `cp`/`cat`/`sed`/`echo` lépéseivel a labor fordul, a `lab-check-vectors.py … 16=UART0_Handler` minden sora OK, a UART0 `0x40004000` báziscíme és az IRQ0 a fő úton igazolt (vektorellenőrzés + echo), és a 36 bájtos sűrű bemenet mind a 36 bájtja visszaíródik. A `BAUDDIV` képlete és az UARTCLK frekvenciája a QEMU `mps2-an385` modelljére érvényes, más QEMU-verzión `info mtree`-vel ellenőrizendő.

19.3 Timer-alapú időzítés és periodikus feladatok

19.3.1 Motiváció

Az előző alfejezetben (Egyszerű UART driver: inicializálás, küldés/vétel megszakítással) a vétel már megszakítás-vezérelt volt, de a *kimenet* és a *feladatütemezés* továbbra is polling-alapú maradt. Egy tipikus firmware-ben viszont nemcsak eseményre kell reagálni, hanem **időben is**: szenzort kell periodikusan mintavételezni, LED-et villogtatni, watchdogot rúgni, állapotgépet léptetni. Az első, naiv megoldás a *busy-wait* késleltetés, amely a CPU-t teljesen lefoglalja a várakozás alatt. Ez a mérnöki döntés azonban közvetlenül mérhető költséggel jár: a processzor nem tud más feladatot végezni, az energiafogyasztás nem csökken, és a késleltetés nem skálázódik. A timer-megszakítás-alapú (timer interrupt-driven) időzítés ezzel szemben a periódusos eseményt a hardverre bízza, és a CPU-t csak a megszakításkezelő rutin (ISR) futása idejére köti le.

Ebben az alfejezetben megírjuk a CMSDK APB timer inicializáló rutinját és megszakításkezelőjét, összekötjük a kanonikus startup vektortáblájával, majd levezetjük és numerikusan összevetjük a két késleltetési stratégia CPU-kihasználtságát.

19.3.2 Definíció: timer-periféria, periódus, periodikus feladat, busy-wait

Timer-periféria (timer peripheral): memóriába leképezett (memory-mapped) számlálóegység, amely egy programozható kezdőértékről indulva minden órajel-ciklusban (vagy előosztóval osztott ciklusban) csökkenti a számlálóját; amikor eléri a nullát, a következő órajelre újratölti magát a kezdőértékből, és – ha engedélyezett – megszakítást kér.

Periódus (T): két egymást követő megszakításkérés (illetve számláló-újratöltés) között eltelt idő:

$$T = \frac{\mathrm{RELOAD} + 1}{f_{\mathrm{timer}}}$$

ahol f_{timer} a timer órajel-frekvenciája [Hz], RELOAD pedig a betöltési regiszterbe írt érték. A $+1$ onnan ered, hogy a számláló a RELOAD értéktől nulláig *bezárólag* számol.

Periodikus feladat (periodic task): olyan műveletsor, amelyet a rendszer fix T időközönként, a feladat előző futásától függetlenül hajt végre. A timer-megszakítás ezt a *kiváltó eseményt* szolgáltatja; a feladat maga az ISR-ben vagy egy ISR által jelzett háttérkontextusban fut.

Busy-wait késleltetés: olyan várakozás, amely egy ciklusban aktívan fogyasztja a CPU-ciklusokat, amíg a kívánt idő el nem telik; eközben más munka nem végezhető el.

Fontos terminológiai szétválasztás: a *timer* a hardver objektum, az *időzítés* a periódus számszerű előírása, a *periodikus feladat* pedig a végrehajtandó logika. A QEMU `mps2-an385` machine-en a

timer-periféria a CMSDK (Cortex-M System Design Kit) része, tehát a regiszterszerződés nem az ARM A-profile architektúrából, hanem a platform leírásából származik.

19.3.3 A CMSDK APB timer alapregiszterei

A timer csak néhány regiszterből áll; ezek adják a teljes programozási felületet. A báziscím az ARM MPS2 referenciadokumentációjában rögzített perifériatartományból származik: `TIMER0` = `0x40000000`, IRQ8, 25 MHz APB-óra (mps2-an385; a 17.3 és e labor igazolja).

Offset	Regiszter	Irány	Szerep
`0x000`	`CTRL`	R/W	engedélyezés, megszakítás-engedélyezés
`0x004`	`VALUE`	R/W	aktuális számlálóérték; írása az aktuális értéket állítja be
`0x008`	`RELOAD`	R/W	újratöltési érték; írása **az aktuális értéket is** beállítja
`0x00C`	`INTSTATUS`	R/W	olvasás: megszakítás-státusz (bit 0: függő megszakítás); írás: 1 a megfelelő bitre törli a kérést (write-1-to-clear) — nincs külön `INTCLR` regiszter

A `CTRL` releváns bitjei (a részletes bitmezőket a platform specifikációjából kell visszaigazolni):

Bit	Név	Jelentés
0	`ENABLE`	a számláló fut / áll
1	`EXTIN`	külső engedélyező bemenet; itt 0
2	`EXTCLK`	külső órajel; itt 0
3	`INTEN`	engedélyezi a megszakításkérést a nullaátlépéskor

Ennél a CMSDK APB timernél nincs külön mód-választó bit: a számláló mindig automatikusan újratöltődik a `RELOAD` értékből, amikor eléri a nullát, azaz a viselkedés mindig a fentebb „periodikusnak” nevezett mintát követi, szabadon futó (wrap) mód nélkül. A konkrét periféria fontos mellékhatása (CMSDK műszaki leírás, 4-4. táblázat): **a `RELOAD` írása az aktuális értéket (`VALUE`) is beállítja.** Ezért az inicializálásban a `RELOAD` beírása után az első periódus is pontosan RELOAD + 1 ciklus, külön `VALUE`-írás nélkül. A `VALUE` írása tudatos külön lépés lehet (például ha az első periódust szándékosan rövidebbre akarjuk), de az első periódus helyességéhez nincs rá szükség. Ugyanez a mellékhatás azt is jelenti, hogy egy futó timer `RELOAD`-jának átírása **azonnal** újraindítja az aktuális periódust. Más időzítők (például a SysTick, ahol a `SYST_RVR` írása nem tölti a számlálót) másként viselkednek, ezért a sémát mindig a konkrét periféria leírásából kell venni.

19.3.4 Kidolgozott példa: timer inicializálás, ISR és periodikus feladat

Munkakönyvtár: `~/arm-labor/cm/a077/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le).

```
mkdir -p ~/arm-labor/cm/a077
cd ~/arm-labor/cm/a077
```

A regiszterabsztrakciót a korábban bevezetett struktúra-mintával végezzük (Periféria-regiszter absztrakció C struktúrával). A startup és a linker script a II. rész kanonikus párja (16.5 Linker script alapjai, 16.6 Startup kód); a vektortáblát a Timer0 bejegyzésével bővítjük:

```
cp ~/arm-labor/cm/a065/lab-mps2.ld .
cp ~/arm-labor/cm/a066/lab-startup.s lab-a077-startup.s
cp ~/arm-labor/cm/a064/lab-check-vectors.py .
cat > lab-a077-irq.inc <<'EOF'
    /* 16-tól: külső IRQ-k; a Timer0 = IRQ8 a 24. index, előtte 8 helyet ki kell tölteni */
    .rept 8
    .word Default_Handler         /* 0x40-0x5C: IRQ0-IRQ7 (itt nem használt) */
    .endr
    .word TIMER0_IRQHandler       /* 0x60: IRQ8 (Timer0)         */
EOF
sed -i '/\.word SysTick_Handler/r lab-a077-irq.inc' lab-a077-startup.s   # a SysTick sor után
# gyenge alias: kezelő nélkül is linkelhető
echo '    lab_default_handler TIMER0_IRQHandler' >> lab-a077-startup.s
```

A két változat **ugyanazt** a bővített startupot használja, de `TIMER0_IRQHandler`-t csak az IRQ-s C-fájl definiálja. Ezért az `echo` sor a kanonikus `lab_default_handler` makróval gyenge, `Default_Handler`-re mutató aliast ad hozzá. Az IRQ-s buildben az erős C-definíció felülírja, a busy-wait buildben a vektortábla IRQ8-szava a `Default_Handler`-re mutat. A busy-wait változat a Timer0 megszakítását nem engedélyezi, így ez a vektor ott nem is hívódik. Alias nélkül a busy-wait linkelése `undefined symbol: TIMER0_IRQHandler` hibával állna le, mert a `.vectors` szekciót a linker `KEEP`-pel megtartja.

A két változat teljes forráslistája:

Változat	Források	IRQ8-vektor (24. index)
IRQ-s	`lab-a077-startup.s`, `lab-a077-irq.c`, `lab-a077-common.c` (+ `lab-a077-timer.h`)	`TIMER0_IRQHandler` (erős definíció a `lab-a077-irq.c`-ben)
busy-wait	`lab-a077-startup.s`, `lab-a077-busywait.c`, `lab-a077-common.c` (+ `lab-a077-timer.h`)	`Default_Handler` (a gyenge alias célja)

A két összehasonlítandó program **ugyanazt a feladatot** végzi: 10 ms-onként egy tick; minden tickben ugyanaz a rögzített mennyiségű „hasznos munka” (`lab_work_step`); és minden 100. tickben (1 s) ugyanaz az állapotváltozás és ugyanaz a kimeneti sor (`lab_tick_task`). A különbség csak az, hogy a tick honnan jön: a timer-megszakításból vagy egy busy-wait késleltetésből. A közös rész:

```
/* lab-a077-timer.h - CMSDK APB timer és a két változat közös része */
#include <stdint.h>

typedef struct {
```

```
    volatile uint32_t CTRL;       /* 0x000 */
    volatile uint32_t VALUE;      /* 0x004 - írás: az aktuális érték beállítása */
    volatile uint32_t RELOAD;     /* 0x008 - írás: a RELOAD ÉS az aktuális érték is */
    volatile uint32_t INTSTATUS;  /* 0x00C - olvasás: INTSTATUS; írás: INTCLEAR (W1C) */
} cmsdk_timer_t;

#define TIMER0_BASE  ((uintptr_t)0x40000000u)
#define TIMER0       ((cmsdk_timer_t *)TIMER0_BASE)
#define TIMER0_IRQ   8u

#define TIMER_CTRL_ENABLE   (1u << 0)
#define TIMER_CTRL_INTEN    (1u << 3)

/* A timer órajel-frekvenciája platformfüggő laborparaméter; 25 MHz illusztratív. */
#define TIMER_FREQ_HZ       25000000u
#define LAB_TICK_MS         10u        /* egy tick */
#ifndef LAB_TICKS_PER_LINE
#define LAB_TICKS_PER_LINE  100u       /* kimeneti periódus és mérési ablak: 100 tick = 1 s */
#endif

static inline uint32_t reload_for_ms(uint32_t ms)
{
    return (TIMER_FREQ_HZ / 1000u) * ms - 1u;   /* T = (RELOAD+1)/f */
}

extern volatile uint32_t g_ticks;    /* tickszámláló */
extern volatile uint32_t g_blink;    /* a periodikus feladat állapota */

void lab_uart_init(void);
void lab_uart_puts(const char *s);
void lab_tick_task(uint32_t tick);   /* AZONOS állapotváltozás és kimenet mindkét változatban */
void lab_work_step(void);            /* AZONOS hasznos munka tickenként mindkét változatban */

/* lab-a077-common.c - a két változat közös feladata */
#include "lab-a077-timer.h"

#define UART0_DATA    (*(volatile uint32_t *)0x40004000u)
#define UART0_STATE   (*(volatile uint32_t *)0x40004004u)
#define UART0_CTRL    (*(volatile uint32_t *)0x40004008u)
#define UART0_BAUDDIV (*(volatile uint32_t *)0x40004010u)

volatile uint32_t g_ticks;
volatile uint32_t g_blink;
static volatile uint32_t g_work_acc;

void lab_uart_init(void)
{
    UART0_BAUDDIV = 16u;
    UART0_CTRL    = 1u;                  /* TXEN */
}

void lab_uart_puts(const char *s)
{
    while (*s) {
        while (UART0_STATE & 1u) { }     /* amíg TXFULL = 1, várunk */
        UART0_DATA = (uint8_t)*s++;
    }
}

void lab_tick_task(uint32_t tick)
{
    if (tick % LAB_TICKS_PER_LINE == 0u) {   /* 100 tickenként (1 s): */
        g_blink ^= 1u;                       /*   állapotbillentés    */
        lab_uart_puts("tick\n");             /*   és egy kimeneti sor */
```

```
    }
}

void lab_work_step(void)
{
    uint32_t acc = g_work_acc;              /* rögzített mennyiségű "hasznos munka" */
    for (uint32_t i = 0; i < 64u; i++)
        acc = acc * 1664525u + 1013904223u;
    g_work_acc = acc;
}
```

A timeres változat a polling és interrupt összevetésénél bemutatott versenymentes alvási mintát követi: a tickeket a főciklus dolgozza fel, és ha nincs feldolgozatlan tick, `WFI`-vel alszik. Az ISR a kérést mindenekelőtt az `INTSTATUS/INTCLEAR` címre írt 1-gyel nyugtázza, különben a megszakítás folyamatosan újra kérődik. A `LAB_MEASURE` ág három különböző mennyiséget mér: az ISR-törzset DWT-vel, az ablakot és az alvást TIMER1-gyel (lásd a levezetést):

```
/* lab-a077-irq.c - timer-megszakításos változat, versenymentes alvással.
 * LAB_MEASURE = 1 mellett KÜLÖN méri (1) az ISR törzsidejét a DWT CYCCNT-tel,
 * (2) egy közös, LAB_TICKS_PER_LINE tickes ablak teljes idejét és (3) az
 * ugyanebben az ablakban alvással töltött időt. A (2) és (3) időalapja a
 * szabadon futó TIMER1: ugyanazon az APB-órán jár, mint az ébresztő TIMER0. */
#include "lab-a077-timer.h"

#ifndef LAB_MEASURE
#define LAB_MEASURE 1
#endif

#define DEMCR       (*(volatile uint32_t *)0xE000EDFCu)
#define DWT_CTRL    (*(volatile uint32_t *)0xE0001000u)
#define DWT_CYCCNT  (*(volatile uint32_t *)0xE0001004u)
#define TIMER1      ((cmsdk_timer_t *)(uintptr_t)0x40001000u)   /* időalap, IRQ nélkül */

static int g_have_dwt, g_have_t1;                /* futásidőben eldöntve */
static volatile uint32_t g_isr_body, g_isr_n;    /* ISR-törzs összesen az aktuális ablakban
 ↪  [CYCCNT-ciklus];
                                                    32 bites: ablakonként LAB_TICKS_PER_LINE db
 ↪  ISR, így
                                                    a halmozás < 2^32 amíg ISR-törzs ×
 ↪  LAB_TICKS_PER_LINE < 2^32 */
static uint32_t g_sleep;                         /* alvás a futó ablakban [TIMER1-ciklus] */

void timer0_init_periodic(uint32_t reload)
{
    TIMER0->CTRL      = 0;          /* állj, megszakítás tiltva */
    TIMER0->INTSTATUS = 1;          /* függő kérés törlése (INTCLEAR, W1C) */
    TIMER0->RELOAD    = reload;     /* a RELOAD írása az aktuális értéket is beállítja,
                                       ezért az első periódus is (reload+1) ciklus */
    TIMER0->CTRL      = TIMER_CTRL_ENABLE | TIMER_CTRL_INTEN;
}

void TIMER0_IRQHandler(void)
{
    uint32_t t_in = g_have_dwt ? DWT_CYCCNT : 0u;   /* mérőpont: a törzs eleje */
    TIMER0->INTSTATUS = 1;          /* nyugtázás - ELŐBB, mint a munka */
    g_ticks++;
    if (g_have_dwt) {
        g_isr_body += DWT_CYCCNT - t_in;            /* mérőpont: a törzs vége */
        g_isr_n++;
    }
}

static int dwt_start(void)
```

```
{
    DEMCR     |= 1u << 24;           /* TRCENA */
    DWT_CYCCNT = 0;
    DWT_CTRL |= 1u;                 /* CYCCNTENA */
    __asm volatile ("dsb\n\tisb" ::: "memory");
    uint32_t before = DWT_CYCCNT;
    for (int i = 0; i < 64; i++) __asm volatile ("nop");
    return DWT_CYCCNT != before;    /* aktív futás alatt halad-e? (alvásról nem mond semmit) */
}

static int timebase_start(void)     /* TIMER1: szabadon futó, 32 bites lefelé számláló */
{
    TIMER1->CTRL   = 0;
    TIMER1->RELOAD = 0xFFFFFFFFu;
    TIMER1->CTRL   = TIMER_CTRL_ENABLE;             /* megszakítás nélkül */
    uint32_t before = TIMER1->VALUE;
    for (uint32_t limit = 100000u; limit; --limit)   /* korlátos várakozás az első lépésre */
        if (TIMER1->VALUE != before)
            return 1;
    return 0;
}

static void put_u32(uint32_t v)
{
    char buf[11], *p = buf + 10;
    *p = '\0';
    do { *--p = (char)('0' + v % 10u); v /= 10u; } while (v);
    lab_uart_puts(p);
}

/* Egy teljes ablak jelentése. win és sleep ugyanabból az időalapból és ugyanarra
 * a [kezdet, vég) ablakra vonatkozik, ezért 0 <= sleep <= win. */
static void report(uint32_t win, uint32_t sleep, uint32_t cyc, uint32_t isr_body, uint32_t
↪   isr_n)
{
    const uint32_t expect = LAB_TICKS_PER_LINE * (reload_for_ms(LAB_TICK_MS) + 1u);
    if (win < expect - expect / 10u) {          /* az ismert ébresztési időközhöz képest */
        lab_uart_puts("SKIP: a TIMER1 alvas kozben nem jar, alvasmeres nincs\n");
        return;
    }
    uint32_t per_mille = (win - sleep) / (win / 1000u);
    if (per_mille > 1000u) per_mille = 1000u;
    lab_uart_puts("ablak=");          put_u32(win);
    lab_uart_puts(" alvas=");         put_u32(sleep);
    lab_uart_puts(" aktiv_ezrelek="); put_u32(per_mille);
    if (g_have_dwt) {
        lab_uart_puts(" isr_torzs_atlag="); put_u32(isr_n ? isr_body / isr_n : 0u);
        lab_uart_puts(cyc + win / 10u < win ? " cyccnt_alvasban=megall"
                                            : " cyccnt_alvasban=jar");
    }
    lab_uart_puts("\n");
}

int main(void)
{
    lab_uart_init();
    g_have_dwt = LAB_MEASURE && dwt_start();
    g_have_t1  = LAB_MEASURE && timebase_start();
    if (LAB_MEASURE && !g_have_dwt)
        lab_uart_puts("SKIP: DWT CYCCNT nem halad, ISR-torzsmeres nincs\n");
    if (LAB_MEASURE && !g_have_t1)
        lab_uart_puts("SKIP: a TIMER1 idoalap nem jar, ablak- es alvasmeres nincs\n");
```

```
    timer0_init_periodic(reload_for_ms(LAB_TICK_MS));
    *(volatile uint32_t *)0xE000E100u = 1u << TIMER0_IRQ;   /* NVIC_ISER0 */

    uint32_t seen = 0, win_t1 = 0, win_cyc = 0;
    int have_win = 0;
    for (;;) {
        while (seen != g_ticks) {           /* minden elmaradt tickre ugyanaz a munka */
            seen++;
            lab_tick_task(seen);
            lab_work_step();
            if (g_have_t1 && seen % LAB_TICKS_PER_LINE == 0u) {
                uint32_t now = TIMER1->VALUE;
                uint32_t now_cyc = g_have_dwt ? DWT_CYCCNT : 0u;
                /* Közös pillanatkép és közös nullázás, PRIMASK-ot megőrző rövid kritikus
                   szakaszban: az ISR-ben növelt g_isr_body/g_isr_n pár ugyanahhoz az ablakhoz
                   tartozik. A számolás és kiírás már a szakaszon kívül történik. */
                uint32_t pm, snap_body, snap_n;
                __asm volatile ("mrs %0, primask\n\tcpsid i" : "=r"(pm) :: "memory");
                snap_body = g_isr_body;  snap_n = g_isr_n;
                g_isr_body = 0u;         g_isr_n = 0u;
                __asm volatile ("msr primask, %0" :: "r"(pm) : "memory");
                if (have_win)                /* az első, részleges ablakot eldobjuk */
                    /* lefelé számol; modulo 2^32 */
                    report(win_t1 - now, g_sleep, now_cyc - win_cyc, snap_body, snap_n);
                /* az új ablak kezdete: mindkét mennyiség nulláról */
                win_t1 = now;
                win_cyc = now_cyc;
                g_sleep = 0;
                have_win = 1;
            }
        }
        __asm volatile ("cpsid i" ::: "memory");   /* versenymentes alvás (a075) */
        if (seen == g_ticks) {
            uint32_t t0 = g_have_t1 ? TIMER1->VALUE : 0u;
            __asm volatile ("dsb\n\twfi" ::: "memory");
            if (g_have_t1) g_sleep += t0 - TIMER1->VALUE;   /* még az ISR előtt (PRIMASK = 1) */
        }
        __asm volatile ("cpsie i" ::: "memory");
    }
}
```

A busy-wait változat ugyanazt a ticket, kimenetet és munkát a CPU-val állítja elő:

```
/* lab-a077-busywait.c - busy-wait változat: ugyanaz a tick, kimenet és munka */
#include "lab-a077-timer.h"

/* Iterációk egy tickhez. Az iteráció költsége (subs + bne + nop, plusz a
   betöltések) nem egy ciklus, és fordítófüggő: LAB_DELAY_ITERS = T * f / c,
   ahol c-t a disassemblyből becsüljük, majd hardveren DWT-vel mérjük. */
#ifndef LAB_DELAY_ITERS
#define LAB_DELAY_ITERS  (TIMER_FREQ_HZ / 1000u * LAB_TICK_MS / 4u)   /* c = 4 feltételezés */
#endif

static void delay_busywait(uint32_t n)
{
    while (n--) {
        __asm volatile ("nop");     /* a volatile asm miatt a ciklus nem tűnik el */
    }
}

int main(void)
{
    lab_uart_init();
```

```
    for (;;) {
        delay_busywait(LAB_DELAY_ITERS);     /* ~10 ms, ha c helyes */
        g_ticks++;
        lab_tick_task(g_ticks);              /* ugyanaz, mint az IRQ-változatban */
        lab_work_step();
    }
}
```

A késleltető ciklus egy iterációja nem egy ciklus. A helyettesítő eszközlánccal (Clang `-O1`) a ciklusmag `subs` + `nop` + `bne` lett; Cortex-M3-on ez a vett ág pipeline-újratöltése miatt nagyjából 4 ciklus. Ezért a `LAB_DELAY_ITERS` alapértéke $T \cdot f/4 = 62\,500$. Ez feltételezés, nem mérés: más fordító vagy kapcsoló más ciklusmagot ad, a pontos c értéket hardveren DWT-vel kell megmérni. QEMU-n a busy-wait ciklus a feltételezett 4 ciklusnál sokkal gyorsabban fut, ezért ott a „10 ms” nem érvényes; csak az IRQ-s változat tickel valós időben. A busy-wait periódusa ráadásul nem 10 ms, hanem a késleltetés **plusz** a tick-feladat, a munka és a kiírás ideje, ezért a késleltetés kalibrálásához a teljes ciklushosszt kell mérni, nem a késleltetőét.

Build és futtatás:

```
CF="-mcpu=cortex-m3 -mthumb -ffreestanding -nostdlib -O1 -T lab-mps2.ld"
arm-none-eabi-gcc $CF -o lab-a077-irq.elf \
    lab-a077-startup.s lab-a077-irq.c lab-a077-common.c
arm-none-eabi-gcc $CF -o lab-a077-busywait.elf \
    lab-a077-startup.s lab-a077-busywait.c lab-a077-common.c
python3 lab-check-vectors.py lab-a077-irq.elf      1=Reset_Handler 24=TIMER0_IRQHandler
python3 lab-check-vectors.py lab-a077-busywait.elf 1=Reset_Handler 24=Default_Handler
timeout 5 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
  -display none -monitor none -serial stdio -kernel lab-a077-irq.elf </dev/null
timeout 5 qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
  -display none -monitor none -serial stdio -kernel lab-a077-busywait.elf </dev/null
```

A programok nem állnak le maguktól; a `timeout 5` állítja le őket (kilépési állapot 124), kézzel pedig `Ctrl-A`, majd `X`.

Kimenet. Fizikai hardveren mindkét változat kb. másodpercenként egy `tick` sort ír. QEMU-n csak az IRQ-s: a busy-wait változat ott nagyságrendekkel gyorsabban „tickel” (a lektor mérése szerint ~130 sor/s), mert a QEMU nem cikluspontos. Ez épp azt mutatja, hogy a busy-wait késleltetést mindig a célhardveren kell kalibrálni. A QEMU alapúton a DWT `CYCCNT` nem halad (17.4 Megszakítás-latencia mérése, 20.3 Mérés és validáció), ezért ott az ISR-törzs mérése `SKIP`. Az ablak- és alvásmérés időalapja a TIMER1, amelyet az AN385 modellje tartalmaz; az ilyenkor kiírt értékek azonban a QEMU emulált idejét mutatják, nem hardverhű mérést:

```
SKIP: DWT CYCCNT nem halad, ISR-torzsmeres nincs
tick
tick
ablak=24996414 alvas=24940900 aktiv_ezrelek=2
tick
ablak=25000389 alvas=24956462 aktiv_ezrelek=1
```

A fenti sorok a lektor mérései QEMU 8.2.2-n; az `alvas` és az `aktiv_ezrelek` értéke emulált idő, nem hardverhű mérés. Az első `tick` után még nincs ablak-sor, mert az első, részleges ablakot a program eldobja; a második `tick`-nél zárul az első teljes ablak.

Mit mér a program, és mivel? A `CYCCNT`-et csak az **aktív** ISR-törzs mérésére használjuk. Az, hogy egy NOP-ciklus alatt halad, csak az aktív futásról mond valamit. Alvás közben a mag órajele a megvalósítás és az alvási mód szerint megállhat, és vele a számláló is: ilyenkor a `WFI` előtti és utáni különbség nem a falióra szerinti alvásidő. Ezért az alvást és a mérési ablakot egy olyan időalapból mérjük, amelynek alvás közben is járnia kell. Ez a szabadon futó TIMER1, ugyanazon az APB-órán, mint a TIMER0, amely a magot felébreszti. Hogy valóban jár-e, azt a

program maga ellenőrzi: egy ablak teljes ideje az ismert ébresztési időköz, `LAB_TICKS_PER_LINE · (RELOAD + 1)`. Ha a mért ablak ennél 10%-nál többel rövidebb, az időalap alvás közben nem járt, és a sor `SKIP`, nem 0 alvásidő. Működő DWT mellett a sor végén azt is jelzi, hogy a `CYCCNT` az ablak alatt a TIMER1-gyel együtt járt-e (`cyccnt_alvasban=jar` vagy `megall`); ez a választott számláló alvásbeli viselkedésének külön próbája. A 25 MHz-es AN385-ön a két óra azonos frekvenciájú, más panelen az arányukkal kell számolni.

Közös ablak. Az `ablak` és az `alvas` ugyanarra a `[kezdet, vég)` intervallumra vonatkozik: mindkettő a `LAB_TICKS_PER_LINE`-adik tick feldolgozásakor nullázódik és indul újra, és az első, részleges ablak eredményét a program eldobja. Az alvási szakaszok így mindig az ablak részintervallumai, ezért `0 ≤ alvas ≤ ablak`, és az aktív hányad (`aktiv_ezrelek = 1000 · (ablak − alvas) / ablak`) 0 és 1000 között marad. A TIMER1 lefelé számol, az ablak `kezdet − vég` különbsége egyetlen átfordulásig modulo 2^{32} helyes; 25 MHz-en ez 171 s, egy 1 s-os ablaknál bőven elég. Az aktív hányad a CPU-idő aránya, nem az energiafogyasztás mértéke: az alvási mód áramfelvétele és az ébredés költsége külön, hardveres mérést igényel.

Működő DWT-vel (hardveren) a sorok végén az `isr_torzs_atlag=…` és a `cyccnt_alvasban=…` mező is megjelenik. A helyettesítő eszközlánccal mindkét programot lefordítottuk, és a Unicorn-alapú `tools/cm-run.py` emulátorban futtattuk; ez a TIMER0 megszakítását kézbesíti, a TIMER1-et szabadon futó időalapként modellezi, a DWT viszont nem halad benne. A funkcionális tulajdonságokat így ellenőriztük. Az ablak pontosan `100 · 250 000 = 25 000 000` ciklus lett. `-DLAB_TICKS_PER_LINE=200u`-val kétszer ekkora, a normalizált `aktiv_ezrelek` változatlan. Az alvás közben megállított TIMER1-gyel (`--timer1-stops-in-wfi`) a sor `SKIP: a TIMER1 alvas kozben nem jar…`. Magát az aktív hányad számértékét az emulátor nem méri hardverhűen: utasításcsomagonként lépteti az időt. **Fontos:** a QEMU emulált ideje nem valós idejű; a periódusok tényleges hossza emulációban nem tekinthető hardverhű mérésnek, csak a viselkedés (megszakítás kiváltódik, ISR lefut, számláló nő) igazolható.

19.3.5 Levezetés: CPU-kihasználtság összevetése

A CPU-kihasználtság definíció szerint az adott feladatra fordított idő és a periódus aránya; itt mindkét mennyiség idő:

$$U = \frac{t_{\text{feladat}}}{T} \times 100\%$$

A költségeket a továbbiakban **ciklusban** adjuk meg (C_{entry}, C_{ISR}, C_{exit}, W, $\bar{P}$). A periódusra jutó elérhető ciklusszám $T \cdot f_{\text{cpu}}$ (s · ciklus/s = ciklus), ezért ciklusokkal ugyanez az arány $U = C_{\text{feladat}}/(T \cdot f_{\text{cpu}})$, mert $t_{\text{feladat}} = C_{\text{feladat}}/f_{\text{cpu}}$. Egy képleten belül minden összeadandó azonos egységű: vagy mind ciklus, vagy mind idő. Időben megadott költséget előbb f_{cpu}-val szorozva ciklusra váltunk, és fordítva.

Busy-wait eset: a késleltetés teljes hosszában ciklusokat fogyaszt, tehát a CPU folyamatosan aktív:

$$U_{\text{busy}} = 100\%$$

A tickenkénti hasznos munka (`lab_work_step`) mindkét változatban ugyanannyi, ezért az összevetés a *várakozás* költségéről szól. Busy-wait esetén a várakozás alatt más munka nem végezhető, és a mag nem alhat. Ha egy iteráció c ciklust vesz igénybe, az iterációszám $N = T \cdot f_{\text{cpu}}/c$. A c nem becsülhető egyetlen `NOP`-ból: a fenti ciklusmagban három utasítás és egy vett ág van, és az értéket fordítóverzió és kapcsoló is befolyásolja.

Timer-megszakítás eset: két külön mennyiséget kell számolni. Az **időzítés és a megszakítás többletköltsége** csak a belépést, az ISR magját és a visszatérést tartalmazza:

$$O_{\text{irq}} = \frac{C_{\text{entry}} + C_{\text{ISR}} + C_{\text{exit}}}{T \cdot f_{\text{cpu}}} \times 100\%$$

A **teljes CPU-kihasználtsághoz** hozzá kell adni a főciklus tickenkénti aktív munkáját is: a `lab_work_step` W ciklusát és a tick-feldolgozás, valamint a kiírás tickenkénti átlagát ($\bar{P}$; a `tick` sor csak minden 100. tickben íródik ki, ezért a költségét 100 tickre kell szétosztani):

$$U_{\text{irq}} = \frac{C_{\text{entry}} + C_{\text{ISR}} + C_{\text{exit}} + W + \bar{P}}{T \cdot f_{\text{cpu}}} \times 100\%$$

A számláló ciklus, a nevező ciklus, a hányados mértékegység nélküli arány; a százalékhoz 100-zal szorzunk.

A busy-wait változat ugyanezt a $W + \bar{P}$ munkát végzi, és a periódus maradékát pörgéssel tölti, ezért $U_{\text{busy}} = 100\%$, a várakozás többletköltsége pedig $O_{\text{busy}} = (T f_{\text{cpu}} - W - \bar{P})/(T f_{\text{cpu}})$.

A 12 ciklus a **Cortex-M3 implementáció** adata ideális belépési körülmények között (nulla várakozási ciklusú memória, nincs folyamatban lévő többciklusos utasítás vagy buszhozzáférés). Nem az egész M-profile ISA általános garanciája: más mag, más memóriarendszer vagy tail-chaining más értéket ad. A kilépésre a modell szintén 12 ciklust feltételez; ez egyszerűsítés. Egyik szám sem tartalmazza az ISR utasításait.

Számpélda: megnevezett egyszerű költségmodell (levezetett, nem mért). Feltételezzük: $f_{\text{timer}} = f_{\text{cpu}} = 25\,\text{MHz}$, $T = 10\,\text{ms}$, belépés és kilépés 12–12 ciklus (a fenti modell), az ISR törzse 20–24 ciklus. Egy periódusra $T \cdot f_{\text{cpu}} = 250\,000$ ciklus jut. A középértékkel

$$C_{\text{entry}} + C_{\text{ISR}} + C_{\text{exit}} \approx 12 + 22 + 12 = 46 \text{ ciklus},$$

$$O_{\text{irq}} = \frac{46}{250\,000} \times 100\% = 0{,}0184\%.$$

A tartományt a törzsidő-feltevés adja: 20 ciklusnál $44/250\,000 = 0{,}0176\%$, 24-nél $48/250\,000 = 0{,}0192\%$. A 46 tehát ennek a modellnek az eredménye, nem egy mérésé. Ha egy teljes 10 ms-os várakozás előtt és után olvassuk a ciklusszámlálót, az eltelt ciklusszám ~250 000 lesz, akármilyen rövid az ISR. Az az eseményig eltelt **időt** méri, nem az ISR **CPU-terhelését**. A kettő külön mennyiség, és a fenti program ezért külön méri őket: az ISR-t közvetlenül körülvevő `CYCCNT`-mérőpontok a törzsidőt, a TIMER1 egy közös ablak teljes idejét és az ugyanebben az ablakban `WFI`-ben töltött időt. A törzsidő-mérés a hardveres be- és kilépést nem tartalmazza; azt a modell 12 + 12 ciklusa közelíti.

Ez a 0,0184% tehát a megszakítás modellbeli **többletköltsége**, nem a megszakításos program teljes CPU-terhe. A teljes terhet a közös munka határozza meg. Feltételezve (nem mérve), hogy a `lab_work_step` 64 iterációja iterációnként kb. 4 ciklus ($W \approx 256$), és a tick-feldolgozás a kiírással együtt tickenként átlagosan $\bar{P} \approx 10$ ciklus:

$$U_{\text{irq}} \approx \frac{46 + 256 + 10}{250\,000} \times 100\% = \frac{312}{250\,000} \times 100\% = 0{,}1248\% \approx 0{,}125\%.$$

Időben számolva ugyanez adódik: $312/(25 \cdot 10^6$ ciklus/s$) = 12{,}48\,\mu$s, és $12{,}48\,\mu$s$/10\,$ms $= 0{,}1248\%$.

A busy-wait ugyanezen periódus alatt folyamatosan aktív (100%), a megszakításos változat a várakozás nagy részét alvással tölti. A teljes kihasználtság aránya ebben a feltételezett esetben kb. $100/0{,}125 = 800$, nem a többletköltségek $100/0{,}0184 \approx 5400$-as aránya. Ha a `lab_work_step` iterációszámát százszorosára (6400-ra) növeljük, $W \approx 25\,600$, és $U_{\text{irq}} \approx 10{,}3\%$ lesz, miközben O_{irq} változatlanul 0,0184%. A két metrika tehát mást mér: az O az időzítési mód költsége, az U a program teljes terhe.

Az alábbi táblázat a három értéket veti össze; a numerikus alak azért is hasznos, mert a különbség extrém.

Késleltetési mód	Mennyiség	Ciklus / periódus	Arány
Busy-wait	teljes kihasználtság	250 000	100%
Timer-megszakítás	teljes kihasználtság (feltételezett $W + \bar{P} \approx 266$)	$46 + 266 \approx 312$	kb. 0,125%
Timer-megszakítás	csak a megszakítás többletköltsége	46 (modell; 44–48 a 20–24 ciklusos törzsfeltevéssel)	0,0184% (0,0176–0,0192%)

19.3.6 Mikor éri meg a timer-megszakítás extra komplexitása?

A timer-alapú megoldás nem ingyenes: extra regiszterkonfiguráció, vektortábla-bejegyzés, ISR-kód, megszakítás-latencia és annak szórása (jitter), valamint az osztott állapot (`g_ticks`) miatti versenyhelyzet-veszély. A döntési szempontok:

- **Hasznos munka van a várakozás alatt:** ha a `main` ciklusnak más dolga van (UART, szenzor, állapotgép), a busy-wait kizárt, mert *blokkolja* a haladást. A timer viszont párhuzamosítja az időzítést és a munkát.
- **A periódus hosszú:** hosszú T esetén az ISR költsége elhanyagolható, a busy-wait arányos idővesztesége viszont nem.
- **A periodicitás pontossága számít:** az ISR-alapú időzítés a hardver órajeléhez igazodik, nem a feladat futásidejéhez; busy-wait esetén a periódus a késleltetés *és* a feladat hosszától függ.
- **Energiafogyasztás:** a busy-wait folyamatosan járatja a magot; alacsony fogyasztású, alvó (sleep/wait-for-interrupt) üzemhez a megszakítás-alapú minta szükséges.
- **Nagyon rövid, egyszeri várakozás (néhány száz ciklus, ISR-tiltott szakaszban):** ilyenkor a busy-wait egyszerűbb, és nem növeli a megszakítás-késleltetést.

A Cortex-M magon a `SysTick` a maghoz tartozó periféria, de **memóriába leképzett** regiszterekkel: a System Control Space-ben, a `0xE000E010`–`0xE000E01F` tartományban van (memóriatérkép). Ugyanezt a mintát kínálja beépítve (a `SYST_RVR` írása viszont nem tölti a számlálót, ezért ott a `SYST_CVR` törlése külön lépés); az itt tanult szerkezet (reload, engedélyezés, ISR, nyugtázás) a `SysTick`-re közvetlenül átvihető.

19.3.7 Kapcsolat a 20. fejezet záróprojektjéhez

Ez a mintázat közvetlenül driverré alakul a Projektspecifikáció: periféria, megszakítás-logika és mérési terv keretében: ott a timer-inicializálás, az ISR és a `main` ciklus közötti kapcsolat már a

`lab-` előtagú driver interfészévé szerveződik (pl. `timer_init(period_ms)`, `timer_get_ticks()`), a UART driverrel együtt egy időzített, megszakítás-vezérelt I/O-modulban. A mostani `g_ticks` számláló a projektbeli időbázis (tick clock) alapja lesz.

19.3.8 Tipikus hibák és ellenpróbák

Tünet	Hipotézis	Adatgyűjtés	Egy változtatás	Ellenőrzés
Az ISR folyamatosan, azonnal újra lefut	`INTSTATUS`-ba írt törlés hiányzik vagy a munka előtt/után rossz helyen van	`INTSTATUS` olvasása; breakpoint az ISR elején	`INTSTATUS = 1` (write-1-to-clear) minden ISR-bevezetéskor	Egy megszakítás → egy ISR-futás
Az ISR soha nem fut	NVIC-engedélyezés hiányzik, vagy rossz IRQ-szám	`NVIC_ISER0` státusz; IRQ-szám ellenőrzése a machine-térképen	helyes IRQ-engedélyezés	ISR belép
HardFault az első megszakításkor	a vektortábla bejegyzése nem a Thumb-kezelőre mutat	`objdump -s -j .vectors`; a vektor LSB-je	helyes szimbólum/réteg a vektorban	megszakítás kezelt, nem HardFault
Az első periódus eltér a többitől	a timert a `RELOAD` írása **előtt** engedélyeztük, így a számláló a régi aktuális értékről indult; vagy a `RELOAD` utáni szándékos `VALUE`-írás más értéket adott	a `CTRL` és a `RELOAD` írásának sorrendje; `VALUE` visszaolvasása az engedélyezés előtt	előbb `RELOAD` (ez az aktuális értéket is beállítja), utána `CTRL`	az első periódus is a várt T
A periódus „újraindul”, amikor futás közben módosítjuk	a CMSDK timernél a `RELOAD` írása az aktuális értéket is felülírja	`VALUE` olvasása a `RELOAD`-írás előtt és után	a periódusváltást a tick-határhoz (az ISR-hez) igazítani	a váltás utáni első periódus a teljes új T
A busy-wait ciklus eltűnt	`volatile`/`__asm volatile` hiánya, `-O2`	objdump a `delay_busywait`-en	`volatile` a ciklusváltozón	ciklus megmarad

19.3.9 Feladatok

1. (magyarázó) Vezesd le, mekkora `RELOAD` érték ad 50 ms periódust $f_{\text{timer}} = 25\,\text{MHz}$ esetén. Magyarázd meg, mi történik az első periódussal, ha a `RELOAD` beírása után nem írod külön a `VALUE`-t, és mi történik, ha a timert a `RELOAD` írása előtt engedélyezed. **2. (implementációs)** Módosítsd a timer-inicializálást 1 Hz-es periódusra, és egészítsd ki az ISR-t úgy, hogy egy `g_seconds` számlálót növeljen. Számítsd ki levezetéssel az ISR-hez tartozó CPU-kihasználtságot,

feltételezve ~46 ciklus ISR-költséget. **3. (hibakeresési/mérési)** Az ISR a vártnál kétszer sűrűbben fut. Írd le a tünet→hipotézis→adatgyűjtés→egy változtatás→ellenőrzés lépéseket, és add meg, melyik regisztert olvasnád vissza `gdb`-vel a hipotézis igazolásához. **4. (mérési, összehasonlító)** Készíts mérést, amely a timer-IRQ változatban külön méri az ISR törzsidejét (`DWT CYCCNT`), valamint egy közös ablak teljes idejét és az abban alvással töltött időt (alvás közben is járó időalappal), a busy-wait változatban pedig a teljes ciklushosszt (késleltetés + munka + kiírás). Ha a `CYCCNT` nem halad (QEMU-alapút), a mérés eredménye `SKIP`, nem nulla.

19.3.10 Megoldások

1. $T = 50\,\text{ms} = 0{,}05\,\text{s}$, $T \cdot f = 1\,250\,000$ ciklus, tehát RELOAD $= 1\,250\,000 - 1 = 1\,249\,999$. Ha a `RELOAD` beírása után nem írod külön a `VALUE`-t, **semmi nem változik**: a CMSDK timernél a `RELOAD` írása az aktuális értéket is beállítja, ezért az első periódus is pontosan 1 250 000 ciklus. Ha viszont a timert `INTEN`-nel a `RELOAD` írása **előtt** engedélyezed, és a `VALUE` (például reset után) 0, a két írás között azonnali megszakítás keletkezhet; ezért a helyes sorrend: `CTRL = 0` → `INTCLEAR` → `RELOAD` → `CTRL = ENABLE|INTEN` (ahogy a `timer0_init_periodic` teszi). (Egy olyan időzítőnél, amelynek reload-írása nem tölti a számlálót, mint a SysTick, a `VALUE/CVR` külön kezelése valóban szükséges; a CMSDK timernél nem.)
2. 1 Hz-hez $T \cdot f = 25\,000\,000$ ciklus, RELOAD $= 24\,999\,999$. Az ISR-költség változatlan (~46 ciklus), ezért a megszakítás többletköltsége $O = 46/25\,000\,000 \times 100\% \approx 0{,}00018\%$. A teljes kihasználtsághoz ehhez hozzá kell adni a periódusonként elvégzett munkát. Ha ez is másodpercenként egyszer fut, a terhe hasonlóan elenyésző; ha a munka nem a periódustól függ, a teljes arány a munka mennyiségétől függ. Az ISR-ben `g_seconds++` történik; az `INTSTATUS = 1` (write-1-to-clear) maradjon a visszatérés előtt (a példában a mérőpont utáni első utasítás). A tanulság: hosszabb periódusnál a megszakítás relatív többletköltsége tovább csökken.
3. Tünet: kétszeres ISR-gyakoriság. Hipotézisek sorrendben: (1) a `RELOAD` fele akkora (`print/x *(uint32_t*)0x40000008`, összevetés a `reload_for_ms` értékével); (2) az ISR két vektorhelyen is szerepel (`lab-check-vectors.py`); (3) hiányzó W1C — ez nem kétszerezés, hanem folyamatos megszakítási vihar. Adatgyűjtés: `gdb` alatt breakpoint az ISR-be, majd a `TIMER0->RELOAD` (`0x40000008`) és a `CTRL` visszaolvasása. Egy változtatás: a `RELOAD` újraszámítása (vagy a hiányzó W1C pótlása, ha a tünet vihar). Ellenőrzés: a másodpercenkénti ISR-futások száma a periódus reciprokával egyezik.
4. A mérés a fenti `lab-a077-irq.c` `LAB_MEASURE` ága. Először a program ellenőrzi, hogy a `CYCCNT` (`0xE0001004`) aktív futás alatt halad-e (`DEMCR.TRCENA`, `DWT_CTRL.CYCCNTENA`, majd két olvasás egy rövid ciklus körül); ha nem, az ISR-törzsmérés `SKIP`, ahogy a QEMU-alapúton, összhangban a 17.4 Megszakítás-latencia mérése és a 20.3 Mérés és validáció mintájával. Ez a próba az alvás alatti viselkedésről semmit nem mond, ezért az alvásidőt nem a `CYCCNT`, hanem a TIMER1 adja. Három külön szám születik. (1) Az ISR törzsideje: az ISR elején és végén vett `CYCCNT`-mérőpontok különbségének átlaga, ez a modell 22 ciklusának mért párja, a hardveres be- és kilépés nélkül. (2) Egy `LAB_TICKS_PER_LINE` tickes ablak teljes ideje a TIMER1-gyel; ez az ismert ébresztési időközzel vethető össze, és ha alvás közben nem jár, `SKIP`. (3) Az ugyanebben az ablakban alvással töltött idő, szintén a TIMER1-gyel. Az aktív hányad $1 -$ alvás/ablak, amely ugyanabból az ablakból számolva mindig 0 és 1 közé esik, és nem függ attól, hány tickes az ablak. Egy tickes periódus és egy 100 tickes alvásösszeg hányadosa értelmetlen lenne. A busy-wait változatban a ciklusonként mért teljes hossz (késleltetés + munka + kiírás) mutatja, hogy a periódus a munkával együtt csúszik. A kalibráláshoz ez kell, nem a késleltető ciklus becsült hossza.

19.3.11 Rövid összefoglaló

A timer-megszakítás-alapú periódusos feladatot a timer `CTRL`, `VALUE`, `RELOAD` és `INTSTATUS` regisztereinek konfigurációja, valamint az ISR-ben elvégzett nyugtázás és a tényleges munka adja. A periódus $T = (\text{RELOAD} + 1)/f_{\text{timer}}$. A busy-wait késleltetés 100%-ban lefoglalja a CPU-t, míg a timer-IRQ a periódus ciklusainak csak egy töredékét áldozza az ISR-re, így más munka párhuzamosan folyhat. A nagyobb komplexitás (vektortábla, ISR, jitter) hosszú periódus és párhuzamos feladatok esetén térül meg. Ez a minta a záróprojektbeli időbázis alapja.

Továbbhasznált definíciók:

- **periodikus feladat:** fix T időközönként végrehajtandó műveletsor, amelyet timer-megszakítás vált ki.
- **RELOAD:** a periódust meghatározó újratöltési érték; a számláló innen indul nulláig.
- **busy-wait késleltetés:** CPU-ciklusokat aktívan fogyasztó várakozás, amely alatt más munka nem végezhető.

19.3.12 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Armv7-M Architecture Reference Manual, DDI0403E — kivételbe lépés/visszatérés ciklusköltsége, a megszakítás-késleltetés fogalma és korlátai.
- Arm M-profile architektúra dokumentáció — NVIC-engedélyezés, prioritás, `SysTick`.
- QEMU System Emulation — a `mps2-an385` machine memóriatérképe és a `-nographic`/`-kernel` viselkedése.
- GNU Arm Embedded Toolchain / Binutils — az `arm-none-eabi-gcc`, `objdump`, `gdb` viselkedése és a vektortábla-generálás.

Ellenőrzési hiányok (nyitott):

- TIMER0: `0x40000000`, IRQ8, 25 MHz APB-óra (mps2-an385; a 17.3 és e labor igazolja). A `CTRL` bitmezői a platform dokumentációjából származnak, a `RELOAD`-mellékhatást a QEMU-mérés is megerősíti.
- A 12 ciklusos belépés a Cortex-M3 implementáció ideális körülményekre vonatkozó adata, a 12 ciklusos kilépés és a 20–24 ciklusos ISR-törzs a modell feltevése; a 46 ciklus és a 0,0184% (a megszakítás többletköltsége) ebből levezetett érték, nem mérés. A teljes kihasználtság példájában a `lab_work_step` és a tick-feldolgozás ciklusszáma ($W \approx 256$, $\bar{P} \approx 10$) feltételezés; a hardveren a `LAB_MEASURE` ág aktív hányada méri.
- A közölt programokat (`lab-a077-irq.c`, `lab-a077-busywait.c`, `lab-a077-common.c`, a kibővített kanonikus startup) a helyettesítő eszközlánccal (Clang + `ld.lld`) lefordítottuk, mindkét ELF-et tiszta könyvtárból, pontosan a közölt `cp`/`cat`/`sed`/`echo` lépésekkel és buildparancsokkal linkeltük, és a `lab-check-vectors.py` az IRQ-s változatban `TIMER0_IRQHandler`-t, a busy-wait változatban `Default_Handler`-t találta az IRQ8 helyén. A gyenge alias nélkül a busy-wait linkelése `undefined symbol: TIMER0_IRQHandler` hibával leállt. A `tools/cm-run.py` emulátorban (amely a Timer0-t és az IRQ-kézbesítést is modellezi) mindkét változat `tick` sorokat írt, az IRQ-s változat előtte a DWT-ellenőrzés `SKIP`-sorát. A `qemu-system-arm` alatti futás és a DWT-mérés hardveren ellenőrizendő.

- A `RELOAD`-írás mellékhatását a CMSDK műszaki leírás 4-4. táblázata írja le. A lektor mérése szerint (GNU + QEMU 8.2.2) a `CTRL = 0`, `VALUE = 5`, `RELOAD = 1234` írássor után a `VALUE` visszaolvasva `1234`, vagyis a `RELOAD` írása a `VALUE`-t is beállítja. Mindkét ELF a könyv lépéseivel fordul, a vektorellenőrzés OK (24 = `TIMER0_IRQHandler`, illetve `Default_Handler`).
- A QEMU-emulációban nincs valós idejű órajel; minden periódus- és ciklusszám-adat relatív/illusztratív, fizikai hardveren mérendő.

20 Záróprojekt II: interrupt-vezérelt Cortex-M firmware-modul

20.1 Projektspecifikáció: periféria, megszakítás-logika és mérési terv

20.1.1 Motiváció: a specifikáció mint mérnöki termék

Az előző alfejezetekben külön-külön megismertük a timer-periódusos feladatot (Timer-alapú időzítés és periodikus feladatok), a periféria-struktúrát (Periféria-regiszter absztrakció C struktúrával), az ISR írását (ISR írása: C és assembly vegyesen) és a memóriatérképet (Memóriatérkép és a bare-metal futási modell). A záróprojekt lényege, hogy ezeket **egyetlen, önállóan reprodukálható rendszerbe** illesszük. Az ipari és beágyazott gyakorlatban azonban egy firmware nem ott kezdődik, hogy az ember ISR-t ír. Előbb **specifikáció** készül: pontosan rögzítjük, milyen perifériák vesznek részt, melyik megszakítás mit csinál, mi az interfész a külvilág felé, mennyi erőforrást költünk, és milyen mérés igazolja a helyes működést. Ez az alfejezet ezt a specifikációs munkát végzi el egy konkrét, a QEMU `mps2-an385` platformon megvalósítható feladaton, mielőtt a következő alfejezetekben (Megvalósítás: driver és ISR integrálása a minimal runtime-ba) kódot írnánk.

A mérnöki indok egyszerű: az interrupt-vezérelt rendszerek hibái szinte kivétel nélkül **specifikációs hiányból** erednek — nem tisztázott, melyik ISR módosíthatja a közös állapotot, mekkora a prioritás, mi történik, ha két megszakítás egyszerre érkezik, vagy hogyan igazoljuk, hogy a periódus valóban tartja magát. Az itteni munkafolyamat: **(1)** fixáljuk a periféria-kombinációt és az interfészt, **(2)** felrajzoljuk a megszakítás-logikát és a közös állapotot, **(3)** megtervezzük a mérési és validációs tervet, **(4)** meghatározzuk az erőforrás-budgetet, végül **(5)** megfogalmazzuk az elfogadási kritériumokat. A címek, IRQ-számok és az órajel a QEMU 8.2.2 `mps2-an385`-ön a 20.3 rendszertesztjével igazoltak; más gépre portoláskor újra mérendők. A módszer maga stabil.

20.1.2 Definíció: firmware-projektspecifikáció

Firmware-projektspecifikáció: mérnöki dokumentum, amely egy beágyazott feladatot öt, egymást kiegészítő szerződésben rögzít: **(a)** a felhasznált perifériák és címeik, **(b)** a külső interfész (parancsok, kimenetek, időzítések), **(c)** a megszakítás-logika (melyik ISR, milyen prioritással, milyen állapotot oszt meg), **(d)** az erőforrás-budget (vektortábla, verem, SRAM), valamint **(e)** a mérési és elfogadási terv. A specifikáció **nem** implementáció: NEM tartalmaz konkrét C- vagy assembly-sorokat a viselkedés leírásán túl, és nem írja elő a belső algoritmusokat. A célja az, hogy a megvalósítás (Megvalósítás: driver és ISR integrálása a minimal runtime-ba) és az audit (Auditszempontok bare-metal firmware-hez) egyaránt visszavezethető legyen rá.

A specifikáció **hatóköre** ez a projekt: egyetlen Cortex-M3 mag, egy QEMU `mps2-an385` machine, nulla operációs rendszer, egy `main` és néhány ISR. **Nem** terjed ki RTOS-ütemezőre, dinamikus memóriakezelésre vagy többmagos futásra (lásd Témaköri határok). **Előfeltétel:** az 18.3 periféria-struktúra-minta és az 19.3 timer-periódus-minta ismerete.

20.1.3 1. Periféria-kombináció és feladatválasztás

Három jelölt kombináció jön szóba az `mps2-an385` ismert perifériakészletéből (CMSDK APB UART, Timer, GPIO, Dual Timer):

Kombináció	Tanulság	Hátrány
GPIO + timer	Egyszerű; egy ISR	Nincs stream, kevés versenyhelyzet; QEMU-n nem futtatható
UART + timer	**Periódus + stream; két ISR, valós megosztott állapot**	Nincs külső eseményforrás
GPIO + UART + timer	Esemény + periódus + stream	A GPIO-rész QEMU-n nem futtatható

A választást a platform támogatása dönti el, nem csak a tanulság. A támogatási mátrix szerint a QEMU 8.2.2 `mps2-an385` gépen a UART0 és a Timer0 implementált, a **GPIO nem**. Ezért a projekt **egyetlen** alapfeladata a QEMU-n teljesíthető UART + timer kombináció. Ennek minden követelményét egy kódrész valósítja meg, és egy automatizált teszt ellenőrzi (20.2, 20.3). A GPIO-s eseményszámláló nem része az alapfeladatnak: külön, **hardveres szerződést** kap (lásd a szakasz végén), és kódja nem szerepel a közös projektben.

A feladat neve: `lab-m19` — megszakítás-vezérelt soros terminál időbázissal. (A név a kézirat korábbi számozásából maradt, amikor a záróprojekt még a 19. fejezethez tartozott; a projekt tartalma a 20. fejezeté.)

Követelmények (az alapfeladat szerződése). Minden sor kap egy azonosítót; ugyanez az azonosító szerepel a megvalósítás kommentjeiben és a teszt kimenetében.

Az.	Követelmény	Megvalósítás (20.2)	Teszt (20.3)
K1	Indulás után a UART-on `lab-m19 boot\r\n`	`main`, a megszakítások engedélyezése előtt	`K1 boot`
K2	Minden `S`, `R`, `M`-től különböző bájt változatlanul visszaíródik (echo)	`handle_byte`	`K2 echo`, `F2 burst`
K3	`S` → `status: ticks=<n> overrun=<m> raw=<r>\r\n`, ahol `n` a legutóbbi `R` óta eltelt tickek száma, `m` az azóta tele puffer miatt eldobott bájtoké, `r` a nyers tickszámláló (`n = r - base`)	`status_line`, koherens pillanatkép kritikus szakaszban	`K3 status`

Az.	Követelmény	Megvalósítás (20.2)	Teszt (20.3)
K4	`R` → `reset ok base=<b>\r\n`, ahol `b` a resetkor rögzített új nullpont (a nyers számláló értéke); utána a K3 számlálói erről a bázisról indulnak (`ticks = raw - b`), ismételt `R` után is	`handle_byte`: új nullpont (`tick_base`, `overrun_base`)	`K4 reset`, `K4 ismételt reset`
K5	`M` → `isr_body: n=… min=… max=… mean=…` (ciklus), vagy működő DWT híján `isr_body: SKIP …`	`UART0_Handler` mintavétel, `measure_line`	`K5` (QEMU-n `SKIP`)
K6	Timer0 10 ms-os tick; **minden** tick feldolgozva; minden 100. tickben `tick\r\n`	`TIMER0_Handler` (számláló), fő ciklus (számlálókülönbség)	`K6 tick`, `K6 tickszám`
F2	Kapacitáson belüli (legfeljebb 63 bájtos) bemenet veszteség nélkül, sorrendben	`ring_push`/`ring_pop`	`F2 burst`, `U1`, `U4`, `U6`
F3	Tele RX-puffernél az **új** bájt elveszik, és az `overrun` eggyel nő	`ring_push`	`U3`, `U5`, `U6`

Szándékos egyszerűsítések. Az alapfeladat a két TX-megoldás közül a **blokkoló, pollozó** TX-et választja (`uart_putc` vár, amíg `TXFULL = 1`); a megszakítás-vezérelt TX-gyűrű az 20.2 2. feladata, nem az alapfeladat része. Ennek terhelési korlátja is a szerződés része: amíg a fő program kiír, nem ürít az RX-pufferből. Ezért az átlagos bemeneti bájtsebesség nem haladhatja meg a kimenetit (az echo minden bájtra egy bájtot ír, egy `S` kb. 30-at), és egy kiírás alatt legfeljebb 63 bájt érkezhet. E korlát felett a bájtok az F3 szerint elvesznek, és az `overrun` számolja őket. Az időzítés 25 MHz-es rendszerórajelet feltételez (`LAB_SYSCLK_HZ` laborparaméter).

Hardveres kiterjesztés (külön szerződés, csak támogatott hardveren). Fizikai MPS2+ kártyán, AN385 FPGA-képpel a projekt kiegészíthető egy GPIO-eseménybemenettel. Ez **nem** része a közös kódnak és a QEMU-tesztnek:

Az.	Követelmény	Ellenőrzés
H1	GPIO0 egy kijelölt lába felfutó élre megszakítást kér; a kezelő `INTCLEAR`-rel nyugtáz (18.3)	hardveres próba; QEMU-n `SKIP`
H2	A K3 sor `events=<k>` mezővel bővül	hardveres próba
H3	A H1 kezelő prioritása a timerénél alacsonyabb	NVIC-visszaolvasás, hardveren

A hardveres változat **saját elvárt kimenete és elfogadási tesztje** (csak támogatott hardveren): reset után `S` → `status: ticks=… overrun=0 events=0`; öt felfutó él után `S` → `events=5`; `R` után `S` → `events=0`, és ismételt `R` után ugyanígy. A K1–K6 tesztek ezen a változaton is futnak, változatlan elvárással (a K3 sor egy mezővel bővül). QEMU-n ez a teszt `SKIP`.

Interfész-szerződés (parancsok és kimenetek):

Irány	Elem	Jelentés	Az.
be	`'S'`	azonnali státuszsor	K3
be	`'R'`	a számlálók új nullpontja	K4
be	`'M'`	ISR-törzsidő-statisztika	K5
be	bármely más bájt	echo	K2
ki	`lab-m19 boot\r\n`	indulás	K1
ki	`tick\r\n`	100 tickenként (1 s)	K6
ki	`status: ticks=<n> overrun=<m> raw=<r>\r\n`	az `S` válasza	K3
ki	`reset ok base=<b>\r\n`	az `R` válasza	K4

Ez az interfész-szerződés az elfogadási teszt alapja: minden kimenet determinisztikusan visszavezethető egy bemenetre vagy a periódusra.

20.1.4 2. Megszakítás-logika vázlata

Két ISR és a fő program osztozik az állapoton. A tervezési elv az, hogy **minden megosztott változót pontosan egy kontextus írjon**; így a megosztott állapoton nincs read-modify-write verseny, és a kritikus szakasz csak a koherens pillanatképhez kell.

```
/* lab-m19 közös állapota (a tényleges definíció: lab-m19-isr.c) */
extern ring_t rx_ring;              /* head és overrun: csak az UART-ISR; tail: csak a main */
extern volatile uint32_t tick_count; /* csak a timer-ISR írja                              */
/* a main saját változói: tick_base, overrun_base (az 'R' nullpontja), seen (feldolgozott
 * tick) */
```

ISR-mátrix:

ISR	Kiváltó	Prioritás (nyers IPR-bájt)	Művelet	Ír	Olvas
`TIMER0_Handler` (IRQ8)	Timer0 nullátmenet, 10 ms	`0x40` (magasabb)	nyugtázás, `tick_count++`	`tick_count`	–
`UART0_Handler` (IRQ0)	RX: bájt érkezett	`0x60`	nyugtázás, `DATA` olvasása, `ring_push`	`rx_ring.head`, `rx_ring.buf`, `rx_ring.overrun`	`rx_ring.tail`

Az ARM Cortex-M NVIC-jében **kisebb szám = magasabb prioritás**; a fenti értékek nyers prioritásbájtok, amelyek `LAB_NVIC_PRIO_BITS = 3` mellett a 2 és a 3, a QEMU 8.2 8 bites NVIC-jén a 64 és a 96 logikai prioritást adják; a sorrendjük mindkét esetben ugyanaz (kivételmodell). A ténylegesen megvalósított bitszámot az 17.2 visszaolvasásos próbája méri. A timer magasabb prioritású, mert ő az időbázis.

Számláló, jelző, lemaradás. A tick feldolgozásához nem egy „volt tick” jelző, hanem a **számláló** a megfelelő eszköz. A fő program a saját `seen` változóját hasonlítja a `tick_count`-hoz, és minden különbségre egy-egy ticket dolgoz fel. Így egy lassú fogyasztó sem veszít ticket, csak lemarad, és a lemaradás (`tick_count - seen`) mérhető. Egy egybites jelző ezzel szemben szándékosan **összevon** több eseményt; erre a „valami történt” jelzéshez van szükség, a tickek számlálásához nem (20.2).

Kritikus szakaszok. Csak két helyen kell: a K3 státuszsor a két számlálót egyetlen koherens pillanatképben olvassa ki, a K4 nullpont-állítás pedig ugyanígy rögzíti őket. Mindkettő 3–4 utasítás, függvényhívás és kiírás nélkül. Az alvás előtti feltételvizsgálat (`WFI`) szintén kritikus szakaszban van (versenymentes alvás, 19.1).

20.1.5 3. Erőforrás-budget

Vektortábla. A 16 rendszerbejegyzés után az alapfeladat két külső IRQ-t használ (`UART0` = IRQ0, `TIMER0` = IRQ8). A tábla a kanonikus `lab-startup.s` bővítése, tehát 16 + 9 = 25 szó (100 bájt), a `0x00000000`-tól, a kanonikus linker script szerint (Linker script alapjai).

Verem-mélység. A verem a `_estack` szimbólumtól lefelé nő. A timer (magasabb prioritás) megszakíthatja a futó UART-ISR-t, ezért a legrosszabb esetben két hardveres keret van a veremben. Egy keret 8 szó = 32 bájt, igazítási szóval 36 bájt, plusz az ISR saját mentései. Becslés (levezetett, nem mért): a `main` legmélyebb hívási lánca (`status_line` → `uart_put_u32` → `uart_putc`) kb. 64 bájt, plusz 2 × (36 + 16) bájt, ez kb. 170 bájt. A kanonikus linker script 4 KiB veremtartalékot (`_Min_Stack_Size`) követel meg `ASSERT`-tel. A tartalék tervezési döntés; hogy elég-e, azt a mért veremcsúcs dönti el (20.4).

SRAM és flash. A `.bss` a 64 bájtos RX-puffert, a mintatömböt (64 × 4 bájt) és néhány számlálót tartalmaz. A pontos értéket az `arm-none-eabi-size` adja, ez kerül a jegyzőkönyvbe.

20.1.6 4. Mérési és validációs terv

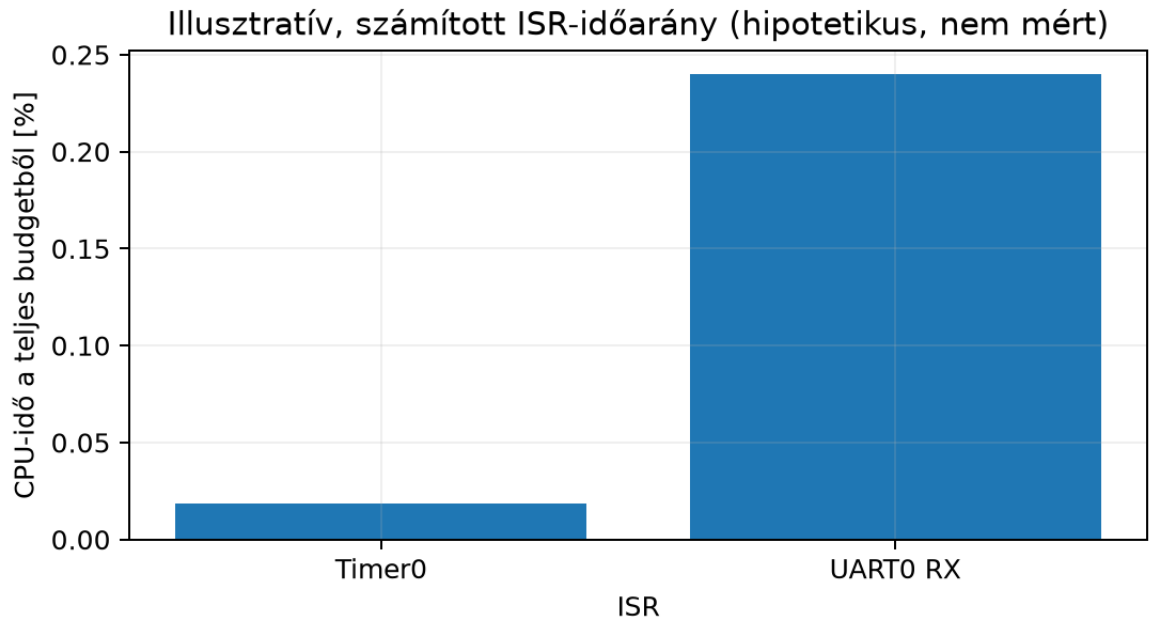
A mérési terv minden eleme egy követelményhez és egy automatizált ellenőrzéshez kötődik (20.3):

Réteg	Mit ellenőriz	Eszköz	Futtatás
egységteszt (U1–U6)	a ring buffer helyessége: alapeset és sorrend, üres puffer, tele + veszteségpolitika, körbefordulás, N−1 elem és az N. elemre előírt overflow, lassú fogyasztó	`lab-m19-ring-test.c`, **hoston** (`gcc`), hardver nélkül	`make ringtest`

Réteg	Mit ellenőriz	Eszköz	Futtatás
rendszerteszt (K1–K6, F2)	a teljes firmware a soros vonalon: boot, echo, kapacitáson belüli burst, `S`, `R` és ismételt `R`, `M`, tick	`lab-m19-systest.py`: host-oldali soros bemenet (stdin) és kimenet (stdout)	`make systest`
ISR-törzsidő (K5)	a UART-ISR törzsének ciklusszáma	DWT `CYCCNT`, mintavétel az ISR-ben, kiértékelés a fő programban	QEMU 8.2.2-n `SKIP`; hardveren mérés
hardveres kiterjesztés (H1–H3)	GPIO-esemény	hardveres próba	QEMU-n `SKIP`

A teszt a megfigyelt viselkedést ellenőrzi, nem jósol meg előre számszerű eredményt. Az F2 veszteségmentesség csak a kapacitáson belüli bemenetre vonatkozik (legfeljebb 63 bájt egyszerre). A tele puffer viselkedését (F3) az egységteszt vizsgálja determinisztikusan, mert egy soros vonalon ezt nem lehet megbízhatóan kikényszeríteni. A DWT-kimenet a QEMU-alapúton `SKIP`; a funkcionális eredményt ettől függetlenül értékeljük.

Az alábbi ábra a két ISR **hipotetikus, számított** CPU-időarányát mutatja az 19.3 költségmodellje alapján (a timer 100 Hz-en, az UART legfeljebb 1000 bájt/s mellett):



20.1. ábra: Illusztratív, számított ISR-időarány (hipotetikus, nem mért) - Hipotetikus, egyszerű költségmodellből levezetett arány (belépés + törzs + kilépés, 25 MHz, Timer0: 100/s × 46 ciklus, a 19.3 modellje szerint 12 + 22 + 12; UART RX: 1000/s × 60 ciklus, a 19.1 c_ISR értéke, mert a W1C nyugtázás és a ring_push hosszabb törzset ad); QEMU-emulációban mért érték nem adható meg hardverhűen.

20.1.7 Kidolgozott példa: a specifikáció vázlata kódban

Munkakönyvtár: `~/arm-labor/cm/zaro2/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez a II. záróprojekt közös könyvtára: mind a négy alfejezete (20.1–20.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/cm/zaro2
cd ~/arm-labor/cm/zaro2
```

A specifikáció legfontosabb kódbeli eleme a ring buffer **szerződése**: ez a fájl változatlanul kerül a megvalósításba (20.2), és hardver nélkül, hoston is tesztelhető (20.3). A periféria-struktúrák (`lab-m19-drivers.h`) és az ISR-ek a megvalósítás részei.

```
/* lab-m19-ring.h - SPSC ring buffer, hardverfüggetlen (hoston is tesztelhető).
 * Szerződés: egyetlen termelő hívja a ring_push-t (az ISR), egyetlen fogyasztó a
 * ring_pop-ot (a fő program). A head-et csak a termelő, a tail-t csak a fogyasztó
 * írja. Kapacitás: RING_CAP - 1 bájt. Tele puffernél a ring_push az ÚJ bájtot
 * eldobja, és az overrun számlálót növeli (veszteségpolitika). */
#ifndef LAB_M19_RING_H
#define LAB_M19_RING_H
#include <stdint.h>

#define RING_CAP 64u                    /* 2 hatványa; használható: 63 */
_Static_assert((RING_CAP & (RING_CAP - 1u)) == 0, "RING_CAP: 2 hatványa");

typedef struct {
    volatile uint8_t  buf[RING_CAP];
    volatile uint16_t head;             /* csak a termelő írja */
    volatile uint16_t tail;             /* csak a fogyasztó írja */
    volatile uint32_t overrun;          /* csak a termelő írja */
} ring_t;

int ring_push(ring_t *r, uint8_t b);    /* 1: betéve, 0: tele, eldobva */
int ring_pop(ring_t *r, uint8_t *out);  /* 1: kivéve, 0: üres */
int ring_empty(const ring_t *r);

#endif
```

A timer periódusa az 19.3 képletével: $T_{\text{tick}} = (\text{RELOAD}+1)/f_{\text{ref}}$. $f_{\text{ref}} = 25\,\text{MHz}$ mellett 10 ms-hoz $\text{RELOAD}+1 = 250\,000$, azaz $\text{RELOAD} = 249\,999$. A referencia-frekvencia QEMU-n és hardveren is ellenőrizendő laborparaméter.

20.1.8 Várható kimenet és ellenőrzési feltétel

Az elfogadás feltétele, hogy `make test` mindkét része `PASS` legyen. A K5 sor a QEMU-alapúton `SKIP`, mert a DWT `CYCCNT` ott nem halad. Hardveren ugyanez a sor mért ciklusszámokat ad. A rendszerteszt kimenetének alakja (20.3, ott a ténylegesen futtatott változat is szerepel):

```
PASS  K1 boot
PASS  K2 echo pontosan "ab"
PASS  F2 burst 50 heterogén bájt: pontos sorrend és hossz
PASS  K3 status formátum, overrun=0  [status: ticks=0 overrun=0 raw=0]
PASS  K6 periodikus tick sor (2 db)
PASS  K6 két tick sor alatt a számláló legalább 100-zal nőtt  [0 -> 200]
PASS  K4 reset: ticks = raw - bázis, monoton  [bázis 200, utána 0/200, később 0/200]
PASS  K4 ismételt reset: ugyanígy  [bázis 300, utána 0/300, később 0/300]
PASS  T1 tickfrekvencia 100 Hz ±20% (host-órához mérve)  [100.0 Hz]
SKIP  K5a ISR-törzsidő formátum  [SKIP (DWT CYCCNT nem halad)]
```

```
SKIP  K5b ISR-törzsidő platformsáv (P1)  [SKIP (DWT CYCCNT nem halad)]
SYSTEST: PASS
```

A hardveres kiterjesztés (H1–H3) eredményei külön jegyzőkönyvbe kerülnek; QEMU-n ezek `SKIP`-ek, nem sikerek. A fenti kimenet `mérve (GNU + QEMU 8.2.2)`; a szögletes zárójelben lévő számok a futtató ütemezésétől függően eltérhetnek, a T1 QEMU-n azért PASS, mert a virtuális óra a host idejét követi.

20.1.9 Tipikus hibák és ellenpróbák

- **Specifikáció és kód elválik.** Tünet: a teszt olyan kimenetet vár (például egy másik státuszformátumot), amelyet a firmware nem ad, vagy a kód olyat tud, amit senki nem ellenőriz. Hipotézis: egy követelményhez nincs kódrész vagy teszt. Adatgyűjtés: a követelménytábla K/F azonosítóinak keresése a forrásban és a teszt kimenetében. Egy változtatás: minden azonosítóhoz egy kódhely és egy tesztsor. Ellenőrzés: a `make test` kimenetében minden azonosító szerepel.
- **A `volatile` elhagyása a közös állapoton.** Tünet: a `main` sosem látja a friss `tick_count`-ot vagy `rx_ring.head`-et. Hipotézis: a fordító regiszterbe emelte. Adatgyűjtés: `arm-none-eabi-objdump -d`, a ciklus törzsének vizsgálata. Egy változtatás: `volatile` minden megosztott mezőn.
- **Rossz prioritás-sorrend.** Tünet: a tick-időbázis csúszik nagy RX-forgalom mellett. Hipotézis: a UART-ISR magasabb prioritású, mint a timer. Adatgyűjtés: az `IPR` bájtok visszaolvasása (17.2). Egy változtatás: a prioritásértékek cseréje.
- **Jelző a számláló helyett.** Tünet: nagy terhelésnél kevesebb `tick` sor jelenik meg, mint ahány tick eltelt. Hipotézis: a fő program egy egybites jelzőt figyel, amely több ticket összevon. Egy változtatás: számlálókülönbség (`seen` vs. `tick_count`). Ellenőrzés: a K6 teszt tickszáma.
- **Nem támogatott periféria a QEMU-tesztben.** Tünet: a GPIO-s teszt QEMU-n „átmegy”, mert a regiszter-visszaolvasás egyezik. Hipotézis: a teszt a támogatás hiányát sikernek veszi. Egy változtatás: a hardveres kiterjesztés tesztjei QEMU-n `SKIP`-et adnak (18.1).

20.1.10 Feladatok

1. (magyarázó) Periféria-kombináció érvelése. Hasonlítsd össze a fenti három kombinációt (GPIO+timer, UART+timer, GPIO+UART+timer) abból a szempontból, hány külön megszakítás-forrást és hány megosztott állapotváltozót hoz létre mindegyik, és melyik teljesíthető a QEMU-alapúton. Indokold, miért a UART+timer az alapfeladat, és miért kap a GPIO külön hardveres szerződést.

2. (implementációs) Mérési terv kiegészítése. Egészítsd ki a K5 mérést egy **levezetett** felső korlát-számítással: feltételezve egy 12 ciklusos kivétel-belépést (a Cortex-M3 ideális körülményekre vonatkozó adata) és egy 20 utasításos ISR-t (átlag 1,5 ciklus/utasítás), mennyi a várható idő az esemény és az ISR vége között $f_{\text{cpu}} = 25\,\text{MHz}$-en? Sorold fel, mely értékek **levezetettek** és melyek **mérendők**, és mi a különbség e mennyiség és a K5 által mért ISR-törzsidő között.

3. (hibakeresési/mérési) Terheléses teszt. Tervezdn olyan reprodukálható teszt-szekvenciát, amely folyamatos UART-RX karaktereket küld, miközben `'R'` és `'S'` parancsokat is ad (QEMU-alapút). Írd le, mit ellenőrizne a kapacitáson belüli és mit a kapacitást meghaladó burst esetén, és melyik követelményhez (F2, F3, K4) köti az egyes ellenőrzéseket.

20.1.11 Megoldások

1. megoldás. A GPIO+timer eset két ISR-t és egy megosztott számlálót ad, de QEMU-n nem futtatható, mert a GPIO ott nincs implementálva. A UART+timer eset két ISR-t (periódus és stream) és több megosztott változót ad (`tick_count`, a ring buffer `head`/`tail`/`overrun` mezői), és mindkét periféria implementált a QEMU-n. A UART+timer nem „a legkisebb" rendszer (egyetlen megszakítás mellett is van valódi konkurencia: ISR és főciklus között), hanem az, amelyik a *legtöbb többlet tanulási célt* adja még QEMU-n automatizáltan tesztelhető áron: két eseményforrás, külön prioritások és az ISR-ek egymás közötti interakciója, több megosztott változóval. Ez összhangban van a 19.2 egy UART-ISR-es ring bufferének versenyhelyzet-elemzésével, amely már az egyetlen megszakítás melletti konkurenciát tárgyalta. A GPIO+UART+timer egy harmadik, eseményalapú forrást ad, de a GPIO-része csak hardveren igazolható. Ezért kap külön szerződést (H1–H3), és nem lehet az alapfeladat feltétele.

2. megoldás. Levezetés: kivétel-belépés 12 ciklus + ISR 20 utasítás × 1,5 ciklus/utasítás = 30 ciklus, összesen **42 ciklus** az eseménytől az ISR utolsó utasításáig — ez egy *egyszerűsített, feltételezett becslés* (ideális belépés + feltételezett átlagos CPI), **nem** bizonyított felső korlát. Felső korláthoz külön kellene: a legnagyobb maszkolási/blokkolási idő, a belépési késleltetés (tail-chaining, busz-várakozás), a memória-várakozási ciklusok és az ISR legrosszabb útvonalának ciklusszáma, mindegyik indokolva. A megoldásban ezért külön osztályozd: *levezetett/ideális* (12 ciklus belépés), *feltételezett* (1,5 CPI, 20 utasítás), *mérendő* (tényleges ISR-törzs) és *bizonyítottan korlátos* (itt nincs). $f_{\text{cpu}} = 25\,\text{MHz}$ mellett a ciklusidő 40 ns, tehát $42 \times 40\,\text{ns} = 1680\,\text{ns} \approx 1{,}7\,\mu\text{s}$. **Levezetett:** a 12, a 30 és az 1,7 µs. **Mérendő:** a tényleges ISR-törzs ciklusszáma és a referenciafrekvencia. A K5 az ISR első és utolsó utasítása között mér, tehát a belépés 12 ciklusát és a fordító prológusának egy részét **nem** tartalmazza. Az eseménytől számított időhöz eseményoldali időbélyeg kellene (20.3).

3. megoldás. (a) Kapacitáson belüli burstök (legfeljebb 63 bájt, a fogyasztó közben ürít): minden bájt visszaíródik, sorrendben, és a K3 `overrun=0`. Ez az F2. (b) A kapacitást meghaladó, a fogyasztónál gyorsabb bemenet a soros vonalon nem kényszeríthető ki megbízhatóan, mert a QEMU a vendég olvasási ütemében adagolja a bájtokat. Ezért az F3-at (az új bájt elveszik, `overrun` nő) a hoston futó egységteszt (U3) ellenőrzi. (c) `R` küldése burst közben: a K4 feltételei (új bázis előző raw, ticks = raw − bázis, raw monoton) hostkéséstől függetlenül igazak; ezért a teszt nem időhöz, hanem ehhez az algebrai viszonyhoz köti a resetet. Az `overrun` a nullpont óta számol. A teszt nem ír elő előre számszerű overrun-értéket; csak azt, ami a szerződésből következik.

20.1.12 Rövid összefoglaló

A `lab-m19` specifikációja egyetlen, QEMU-n teljesíthető alapfeladatot rögzít: Timer0 10 ms-os időbázissal, UART0 megszakítás-vezérelt vétellel, echo-val és az `S`/`R`/`M` parancsokkal. Minden követelmény (K1–K6, F2, F3) egy kódrészhez és egy automatizált teszthez kötődik. A GPIO-eseménybemenet külön hardveres szerződés (H1–H3); QEMU-n `SKIP`. A megosztott állapotot minden változónál egyetlen kontextus írja, a kritikus szakasz csak a koherens pillanatképhez kell, a tickeket számláló, nem jelző viszi át.

Továbbhasznált definíciók:

- **firmware-projektspecifikáció:** a periféria-, interfész-, ISR-, erőforrás- és mérési szerződés együttese; nem implementáció.
- **interfész-szerződés:** a külső bemenetek (parancsok) és kimenetek (üzenetek, időzítések) determinisztikus leírása.

- **ISR-mátrix:** minden megszakítás forrása, prioritása, művelete és a hozzá tartozó megosztott állapot együtt.

20.1.13 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Armv7-M Architecture Reference Manual, DDI0403E — kivételmodell, prioritás, kivételbelépés/visszatérés ciklusköltsége, a megszakítás-latencia fogalma.
- Arm M-profile architektúra dokumentáció — NVIC-engedélyezés, prioritás, vektortábla-elrendezés, DWT.
- QEMU System Emulation — az `mps2-an385` machine memóriatérképe, CMSDK perifériamodellek.

QEMU-korlát: a 8.2.2-es `mps2-an385` modellben az UART0 `0x40004000`, Timer0 `0x40000000` címen működik; a `0x40010000` GPIO-terület nincs implementálva, a DWT CYCCNT pedig nem halad. A GPIO-s eseményvizsgálatot és a ciklusidőmérést itt ki kell hagyni (`SKIP`); ezeket implementált perifériával és működő számlálóval rendelkező hardveren kell elvégezni. A specifikáció ezért a QEMU-alapútra UART + timer kombinációt ír elő, és a GPIO-t külön kiterjesztésként kezeli.

Nyitott ellenőrzési igények:

- A fenti címek az AN385 kiosztását követik; más géptípusra a perifériacímeket, az IRQ-számokat és az implementált perifériákat újra ellenőrizni kell.
- A timer referencia-frekvenciája és a `RELOAD` a QEMU 8.2.2 mps2-an385 modelljén a 20.3 rendszertesztjével igazolt; fizikai MPS2+AN385-ön újra mérendő.
- A `DWT->CYCCNT` a QEMU 8.2.2-n nem halad, ezért a K5 mérés ott `SKIP`; a ciklusszám-alapú mérések csak hardveren értelmezhetők.
- Az NVIC prioritásregiszterek által implementált bitek száma a QEMU-modellben és a fizikai Cortex-M3 magon eltérhet; a prioritási értékek tényleges hatása méréssel igazolandó.

20.2 Megvalósítás: driver és ISR integrálása a minimal runtime-ba

20.2.1 Motiváció: a specifikációtól a működő firmware-ig

A Projektspecifikáció: periféria, megszakítás-logika és mérési terv alfejezetben elkészült a `lab-m19` szerződése: egyetlen, QEMU-n teljesíthető alapfeladat (Timer0 időbázis, UART0 echo és parancsok; a GPIO-eseménybemenet külön hardveres szerződés), a K1–K6, F2, F3 követelmények, az interfész-szerződés, az ISR-mátrix, az erőforrás-budget és a mérési terv. A specifikáció azonban önmagában nem fut. Ebben az alfejezetben a papíron rögzített szerződést alakítjuk át olyan firmware-modullá, amely a Startup kód: .data másolása, .bss törlése alfejezetben épített minimál runtime-ra támaszkodik, és a Firmware build és QEMU-betöltés munkafolyamat lépéssorával reprodukálható módon elindul a `qemu-system-arm -machine mps2-an385` emulátorban.

A mérnöki nehézséget nem az egyes driverek megírása jelenti — a Periféria-regiszter absztrakció C struktúrával, az Egyszerű UART driver és a Timer-alapú időzítés és periodikus feladatok már megadta a mintákat. A valódi feladat kettős: (1) a driverek és a hozzájuk tartozó ISR-ek *illesztése* a vektortáblába, a startup kódba és az NVIC-konfigurációba, valamint (2) azon a ponton kezelni a fő program és az ISR közötti megosztott állapotot, ahol a naiv implementáció

csendben adatot veszít. Ez utóbbi a tipikus „majdnem jó” firmware hibamódja: a kód lefordul, elindul, látszólag működik, majd nagy terhelésnél vagy ritka ütemezésnél elveszít egy ticket vagy egy bemeneti bájtot. Az alábbiakban ezért az integráció mellett kiemelt hangsúlyt kap a versenyhelyzet (race condition) felismerése és a kritikus szakasz (critical section) fegyelmezett használata.

20.2.2 Definíció: mit integrálunk és mi a szerződés

driver-ISR integráció (driver-ISR integration): az a műveletsor, amelynek során egy periféria inicializáló és I/O-függvényei, a hozzá tartozó megszakításkezelő rutin, a vektortábla megfelelő bejegyzése, az NVIC engedélyezési és prioritásbeállítása, valamint a megosztott állapotot védő kritikus szakasz egyetlen, konzisztens egységbe kerül.

Az integrációnak négy, egymástól függetlenül ellenőrizhető szerződéselem felel meg:

Elem	Hol él	Mit garantál
Vektorbejegyzés	vektortábla (`.vectors`)	a kivétel belépési pontja a helyes ISR-re mutat
NVIC-engedélyezés	`NVIC_ISER0`, `NVIC_IPR`	a forrás képes megszakítást kérni, adott prioritással
ISR-törzs	C függvény	a forrás-flag törlése, az adat kiolvasása, a megosztott állapot frissítése
Kritikus szakasz	fő program és ISR	a megosztott állapot nem sérül a nem atomi hozzáférés miatt

Az integráció akkor tekinthető késznek, ha a fenti négy elem konzisztens: a vektorban szereplő szimbólum pontosan az a függvény, amelyet az NVIC engedélyez, és amely a megosztott állapotot ugyanazzal a védelmi szerződéssel használja, mint a fő program. Ha bármelyik elem elcsúszik — például a vektor egy `Default_Handler`-re mutat, miközben az NVIC engedélyezve van —, a hiba tünete „néma”: a megszakítás vagy nem fut le, vagy végtelen ciklusba esik.

20.2.3 A vektortábla kiegészítése

Az M-profile kivételmodellt a A Cortex-M kivételmodell: prioritás és vektortábla-bejegyzések tárgyalta. A vektortábla első szava nem utasítás, hanem az inicializáló veremmutató (`_estack`) értéke; a második szó a reset vektor. Ezután 14 további rendszerkivétel következik (NMI, HardFault, MemManage, BusFault, UsageFault, SVC, DebugMon, PendSV, SysTick és a fenntartott helyek), majd 16-tól kezdődnek a külső megszakítások (IRQ0, IRQ1, …). A `lab-m19` esetében tehát a vektortáblát a 16. indextől kell kiegészíteni a Timer0 és UART0 bejegyzésekkel.

Az ARM MPS2+ AN385 board IRQ-számai board-specifikusak; a pontos értékeket a QEMU `hw/arm/mps2.c` forrása és az MPS2 technikai referenciája adja meg. Az ellenőrzött kiosztás szerint az UART0 RX `IRQ0`, a Timer0 `IRQ8` — ezt a `lab-m19-drivers.h` `UART0_IRQn`/`TIMER0_IRQn` szimbolikus konstansai rögzítik, de a hallgatónak a saját QEMU-verziójával szemben ettől függetlenül meg kell erősítenie. A vektortáblában a bejegyzés helye `16 + IRQn`; a két IRQ-szám közötti **nem** szomszédos pozíciókat (itt IRQ1–IRQ7) explicit kitöltő (`Default_Handler`) bejegyzésekkel kell kitölteni, különben a második kezelő rossz IRQ helyére kerülne.

Fontos részlet, hogy a vektortábla a flash (az AN385 esetében a linker scriptben FLASH régiónak nevezett betöltött SSRAM) legalján van, és a linker garbage collection (GC) hajlamos eldobni, ha nem hivatkozik rá szimbólum. Ezért a linker scriptben `KEEP(*(.vectors))` szerepel, és a tábla `.global` szimbólummal exportált.

20.2.4 Az ISR-ek M-profile szerződése

Az A-profile-tól eltérően az M-profile megszakításkezelő rutinjai **sima C függvények**. A kivétel belépésekor a hardver automatikusan elmenti a hívó által használt regisztereket (R0–R3, R12, LR, PC, xPSR) az aktuális veremre, és az LR-be egy EXC_RETURN-értéket tölt. A visszatérést az váltja ki, hogy ez az érték Handler módban a PC-be kerül. A fordító szokásos epilógusa (`BX LR`, vagy hívó kezelőnél `POP {..., PC}`) ezt külön teendő nélkül elvégzi. Ezért a kezelő szignatúrája `void f(void)`, kézi `BX LR`-zárás nem kell. A GCC `__attribute__((interrupt))` Armv7-M-en érvényes, de nem szükséges: ott csak azt jelzi, hogy a függvény 4 bájtra igazított SP-vel is meghívódhat (17.1). A `CCR.STKALIGN = 1` alapállapot mellett a sima függvény is helyes. Az A-profile-os `__irq` kulcsszó ide nem tartozik.

Két következmény érdemel figyelmet. Egyrészt az ISR-nek nincs visszatérési értéke és nincs paramétere. Másrészt az ISR-ben nem szabad hosszú, blokkoló műveletet végezni: a megszakítás-latencia — amelyet a Megszakítás-latencia mérése és korlátai definiált — a teljes ISR futási idejével nő, és a fő program előremenetelét késlelteti. Az ISR tipikus feladata három lépés: a forrás-flag azonosítása és törlése, az adat kiolvasása/írása, valamint a megosztott állapot minimális frissítése.

20.2.5 Megosztott állapot: az SPSC ring buffer

A specifikáció K2 és F2/F3 követelménye szerint a fő program a beérkező UART-bájtokat dolgozza fel. Mivel a bájtokat az UART0 ISR veszi át, a fő program és az ISR között megosztott puffer kell. A legegyszerűbb helyes szerkezet az egytermelős–egyfogyasztós (single-producer single-consumer, SPSC) ring buffer: az ISR csak a `head` indexet írja, a fő program csak a `tail` indexet. A puffer akkor üres, ha `head == tail`; eggyel kevesebb slot használható, mint a tömb mérete, így a tele/üres állapot megkülönböztethető külön számláló nélkül. A `RING_CAP` 2-hatvány voltát a fejléc `_Static_assert`-tel kényszeríti ki: nem 2-hatványú kapacitásnál a `%` osztásra (`UDIV`) fordulna, ami sokkal drágább, és a 20.4 determinizmus-auditja önmagában nem jelezné.

Az SPSC minta kulcsfelismerése, hogy az indexek külön változók, és mindegyiket pontosan egy kontextus írja. Ekkor a `volatile` minősítő elegendő a láthatósághoz, és nincs szükség kritikus szakaszra a push/pop műveletekhez. Ennek feltétele, hogy az index típusa (itt `uint16_t`) egyetlen, atomi módon olvasható és írható gépi szón belül legyen, és hogy a termelő **előbb** írja az adatot, és csak utána a `head`-et. A puffer kódja hardverfüggetlen külön modul (`lab-m19-ring.h/.c`, a szerződést lásd az 20.1 alfejezetben), ezért a hoston egységtesztelhető (20.3).

20.2.6 A race condition és a kritikus szakasz

Munkakönyvtár: `~/arm-labor/cm/zaro2/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez a II. záróprojekt közös könyvtára: mind a négy alfejezete (20.1–20.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/cm/zaro2
cd ~/arm-labor/cm/zaro2
```

Tekintsük a timer oldalt. Az első, naiv tervben az ISR egy `tick_flag` jelzőt állít, a fő ciklus figyeli és törli:

```
/* HIBÁS példa - szemléltetésre: egybites jelző, a fő program olvassa és törli */
if (tick_flag) {          /* (A) olvasás */
    tick_flag = 0;        /* (B) törlés  */
    process_tick();
}
```

Egyetlen magon az ISR nem fut „párhuzamosan” a fő programmal: vagy a fő program egy utasítása fut, vagy az ISR egésze. Az ISR-t viszont a fő program **bármely két utasítása közé** beékelheti a hardver. A hiba ezért a fő program megszakítható (A) olvasása és (B) törlése közé beszúrt ISR-ből adódik. Végrehajtható idődiagram (minden sor egy lépés, felülről lefelé):

Lépés	Fő program	ISR	`tick_flag`	Eltelt tick	Feldolgozott tick
0	–	1. tick: `tick_flag = 1`	1	1	0
1	(A) `ldr` → 1	–	1	1	0
2	–	2. tick: belépés az (A) és (B) közé, `tick_flag = 1`	1	2	0
3	(B) `str` ← 0	–	0	2	0
4	`process_tick()`	–	0	2	1
5	(A) `ldr` → 0: nincs teendő	–	0	2	1

A 2. lépés eseményét a fő program a 3. lépésben úgy törli, hogy sosem látta: ez a valódi read-modify-write verseny. Ha utána nem jön újabb tick, a fő program erről az eseményről soha nem tud. (A fordított irány, hogy a fő program az ISR `tick_count++`-ának betöltése és visszaírása közé ékelődne, egyetlen magon lehetetlen: a fő program az ISR alatt nem fut.)

A javított jelzőkezelés az olvasást és a törlést **egyetlen** kritikus szakaszba teszi, a jelző tesztjével együtt, így egyetlen koherens pillanatkép keletkezik:

```
uint32_t pm = crit_enter();
uint32_t flag  = tick_flag;       /* teszt ... */
uint32_t ticks = tick_count;
tick_flag = 0;                    /* ... és törlés ugyanabban a szakaszban */
crit_exit(pm);
if (flag)
    process_tick(ticks);
```

Ez koherens pillanatképet vesz, és a verseny ellen véd, de **nem** garantál egy feldolgozást minden tickre, mert **nem** számlál: ha a fő program lassú, és közben két tick történik, a jelző egyszer lesz 1, és a két tick egyetlen feldolgozásba olvad. Ez nem hiba, hanem a jelző természete; egy „valami történt” jelzéshez éppen ez kell. Négy dolgot érdemes tehát szétválasztani:

Fogalom	Mit mutat	Eszköz
eseményszámláló	hány esemény történt összesen	`tick_count`, csak az ISR írja

Fogalom	Mit mutat	Eszköz
eseményjelző	történt-e valami a legutóbbi ellenőrzés óta (összevonással)	`tick_flag`
fogyasztói lemaradás	hány eseményt nem dolgozott még fel a fő program	`tick_count - seen`
RMW-race	egy esemény nyoma elveszik a nem védett olvasás–írás miatt	a fenti idődiagram

A `lab-m19` K6 követelménye szerint **minden** ticket fel kell dolgozni, ezért a megvalósítás nem jelzőt, hanem számlálót használ: a fő program a saját `seen` változóját lépteti a `tick_count`-ig, és minden különbségre egy ticket dolgoz fel. A `tick_count`-ot csak az ISR írja; a K4 `R` parancs nem nullázza, hanem a fő program saját `tick_base` nullpontját állítja. Így a megosztott számlálón nincs második író, tehát nincs RMW-verseny sem. A `seen != now` összehasonlítás és a `seen++` előjel nélküli, modulo 2^{32} aritmetikában dolgozik, ezért a `tick_count` körbefordulása (100 Hz mellett kb. 497 nap után) nem hiba. A feldolgozás addig helyes, amíg a fogyasztói lemaradás kisebb 2^{32} ticknél, és ez a körbefordulási feltétel része a szerződésnek. A kritikus szakasz (`PRIMASK`) csak a maszkolható megszakítások ellen véd: egy NMI- vagy HardFault-kezelő ezalatt is futhat, ezért ezek nem férhetnek hozzá a megosztott állapothoz.

A **volatile önmagában nem oldja meg ezt.** A `volatile` azt garantálja, hogy a fordító minden hozzáféréskor újraolvassa és kiírja a változót, és nem cache-eli regiszterben; nem garantál atomitást és nem zárja ki a kontextusváltást az RMW közben. A helyes megoldás a megszakítások letiltása a kritikus szakaszban. Az M-profile-on ezt a `PRIMASK` speciális regiszter szabályozza; a `cpsid i` utasítás letiltja a maszkolható megszakításokat, a `cpsie i` engedélyezi. A helyes minta a korábbi állapot mentése és visszaállítása, hogy a beágyazott kritikus szakaszok működjenek:

```
/* lab-m19-crit.h - kritikus szakasz M-profile-on, a PRIMASK megőrzésével */
#ifndef LAB_M19_CRIT_H
#define LAB_M19_CRIT_H
#include <stdint.h>

static inline uint32_t crit_enter(void)
{
    uint32_t pm;
    __asm volatile ("mrs %0, primask" : "=r"(pm));
    __asm volatile ("cpsid i" ::: "memory");
    return pm;
}

static inline void crit_exit(uint32_t pm)
{
    __asm volatile ("msr primask, %0\n\tisb" :: "r"(pm) : "memory");
}
#endif
```

A `"memory"` clobber megakadályozza, hogy a fordító memóriahozzáféréseket mozgasson a kritikus szakaszba vagy ki belőle. A mentett `PRIMASK` visszaállítása azért fontos, mert a `cpsie i` feltétel nélküli engedélyezése egy külső kritikus szakasz belsejében hibásan korábban nyitná vissza a megszakításokat. Az `msr primask` után az `ISB` biztosítja, hogy a módosítás a következő utasításoktól érvényes legyen; ezért szerepel a `crit_exit`-ben.

A kritikus szakasz mindössze néhány utasítás: a megszakítás-letiltás ideje rövid, így a megszakítás-latencia növekedése elhanyagolható. A fő szabály: **a kritikus szakaszban nem hívunk függvényt, nem írunk soros portra, nem várunk.** Ha a védendő művelet hosszú, a mintát újra kell tervezni (például dupla pufferrel vagy atomi cserével).

20.2.7 Build-integráció

A `lab-m19` build a korábbi fejezetek elemeit egyetlen projektbe fogja össze. Az alábbi táblázat a forrásokat és a hozzájuk tartozó korábbi alfejezetet rendeli:

Fájl / elem	Szerep	Előzmény
`lab-m19.ld`	szekciók, `_estack`, `_sidata`, `.data`/`.bss`	Linker script alapjai
`lab-m19-startup.S`	reset vektor, `.data` másolás, `.bss` törlés, vektortábla	Startup kód
`lab-m19-drivers.h`/`.c`	UART0, Timer0 regiszterstruktúra, init-függvények, pollozó TX (a `.c`-t a Makefile önállóan fordítja, ezért a megosztott deklarációk a `.h`-ban élnek)	Periféria-regiszter absztrakció, UART driver
`lab-m19-ring.h`/`.c`	hardverfüggetlen SPSC ring buffer (F2, F3)	20.1 szerződése
`lab-m19-crit.h`	kritikus szakasz a `PRIMASK` megőrzésével	Megszakítás-latencia
`lab-m19-isr.h`/`.c`	`TIMER0_Handler`, `UART0_Handler`, a megosztott (nem `static`) ISR-állapot, DWT-mintavétel (K5)	ISR írása
`lab-m19-main.c`	fő ciklus, parancsértelmezés (K1–K6), versenymentes alvás	Polling versus interrupt-vezérelt I/O
`lab-m19-ring-test.c`, `lab-m19-systest.py`	egységteszt (hoston) és rendszerteszt (soros bemenettel)	20.3
`Makefile`	cross-build `-lgcc`-vel, `run`, `ringtest`, `systest`, `test`	Firmware build

20.2.8 Kidolgozott példa: a lab-m19 firmware

20.2.8.1 Linker script és memóriaelrendezés

Az AN385 memóriatérképét a Memóriatérkép és a bare-metal futási modell vezette be. A projekt **nem ír saját linker scriptet**: a `lab-m19.ld` a II. rész kanonikus `lab-mps2.ld`-jének (Linker script alapjai) változatlan másolata. Így ugyanaz a szerződés érvényes, mint a korábbi laborokban: `.vectors` a `0x00000000`-n, 4 MiB `FLASH` (betöltött SSRAM) és a 4 MiB-os kanonikus RAM (a `0x20400000`-tól látszó tükör nélkül), szóhatárra igazított `.data`/`.bss` határok, és `ASSERT` a verem és az adatok ütközésére.

```
cp ~/arm-labor/cm/a065/lab-mps2.ld lab-m19.ld
```

20.2.8.2 Startup és vektortábla

A startup kód a Startup kód kanonikus `lab-startup.s`-e, egyetlen bővítéssel: a vektortábla a 16 rendszerbejegyzés után a projekt két külső megszakítását is tartalmazza. A `Reset_Handler` (előltesztelő, előjel nélküli `bhs` ciklusok), a `Default_Handler` és a gyenge aliasok változatlanok:

```
/* lab-m19-startup.S - a kanonikus lab-startup.s, kiegészítve a lab-m19
 * külső megszakításaival (IRQ0 = UART0 RX, IRQ8 = Timer0). A Reset_Handler,
 * a Default_Handler és a gyenge aliasok változatlanok. */
    .syntax unified
    .cpu cortex-m3
    .thumb

    .section .vectors, "a", %progbits
    .align 2
    .global __vectors
__vectors:
    .word _estack                /* 0x00: kezdeti SP (MSP)       */
    .word Reset_Handler          /* 0x04: Reset (bit0 = 1)       */
    .word NMI_Handler            /* 0x08: NMI                    */
    .word HardFault_Handler      /* 0x0C: HardFault              */
    .word MemManage_Handler      /* 0x10: MemManage              */
    .word BusFault_Handler       /* 0x14: BusFault               */
    .word UsageFault_Handler     /* 0x18: UsageFault             */
    .word 0, 0, 0, 0             /* 0x1C-0x28: fenntartott       */
    .word SVC_Handler            /* 0x2C: SVCall                 */
    .word DebugMon_Handler       /* 0x30: DebugMonitor           */
    .word 0                      /* 0x34: fenntartott            */
    .word PendSV_Handler         /* 0x38: PendSV                 */
    .word SysTick_Handler        /* 0x3C: SysTick                */
    /* 16-tól: külső IRQ-k. Az ellenőrzött AN385-kiosztásban UART0 RX = IRQ0,
       Timer0 = IRQ8; a kettő között 7 helyet (IRQ1-IRQ7) explicit kitöltő
       (Default_Handler) bejegyzésekkel KELL kitölteni, különben a második
       bejegyzés tévesen IRQ1 helyére kerülne, nem IRQ8-ra. */
    .word UART0_Handler          /* 0x40: IRQ0 (UART0 RX)        */
    .rept 7                      /* 0x44-0x5C: IRQ1-IRQ7         */
    .word Default_Handler
    .endr
    .word TIMER0_Handler         /* 0x60: IRQ8 (Timer0)          */

    .text
    .thumb_func
    .global Reset_Handler
    .type   Reset_Handler, %function
Reset_Handler:
    /* 1. .data másolása FLASH (LMA) -> RAM (VMA), szavanként.
       Előltesztelő ciklus, előjel nélküli összehasonlítással: üres
       szekciónál egyszer sem fut, és a végcímen pontosan megáll. */
    ldr   r0, =_sidata
    ldr   r1, =_sdata
    ldr   r2, =_edata
1:  cmp   r1, r2
    bhs   2f
    ldr   r3, [r0], #4
    str   r3, [r1], #4
    b     1b
    /* 2. .bss nullázása, szavanként. */
2:  ldr   r1, =_sbss
    ldr   r2, =_ebss
    movs  r3, #0
3:  cmp   r1, r2
    bhs   4f
    str   r3, [r1], #4
    b     3b
    /* 3. main(); ha visszatérne, végtelen ciklus. */
4:  bl    main
5:  b     5b
    .size Reset_Handler, . - Reset_Handler
```

```
    .thumb_func
    .global Default_Handler
    .type   Default_Handler, %function
Default_Handler:
    b       .
    .size Default_Handler, . - Default_Handler

    /* Gyenge aliasok: amit a firmware nem definiál, az a Default_Handlerre fut. */
    .macro lab_default_handler name
    .weak       \name
    .thumb_set  \name, Default_Handler
    .endm
    lab_default_handler NMI_Handler
    lab_default_handler HardFault_Handler
    lab_default_handler MemManage_Handler
    lab_default_handler BusFault_Handler
    lab_default_handler UsageFault_Handler
    lab_default_handler SVC_Handler
    lab_default_handler DebugMon_Handler
    lab_default_handler PendSV_Handler
    lab_default_handler SysTick_Handler
```

A `TIMER0_Handler` és `UART0_Handler` szimbólumok C függvények, amelyeket a linker old fel; mivel nem `.weak`-ként szerepelnek, ha valamelyik hiányzik, a linker **hibát jelez** (“undefined reference to …”), és a linkelés megszakad — a GNU `ld` alapértelmezett viselkedése a fel nem oldott, nem gyenge szimbólumokra a hibajelzés, *nem* a vektorbejegyzés csendes 0-ra állítása. A rendszer-kivételek (`NMI_Handler`, `HardFault_Handler`, `SVC_Handler`, `PendSV_Handler`, `SysTick_Handler`) ezzel szemben a fent látott `.weak`/`.thumb_set Default_Handler` mintát követik (lásd 16.5): ha a firmware nem definiálja felül őket, a linker a `Default_Handler`-re köti, amely egy végtelen ciklus — ez debuggerben azonnal jelzi, hogy egy kezelt kivétel lefutott, de a tényleges logika hiányzik. Ezért érdemes a vektortáblát `KEEP`-pel védeni, hogy a linker a `--gc-sections` során ne dobja el a futásidőben szükséges, de a kódból közvetlenül nem hivatkozott bejegyzéseket.

20.2.8.3 Driverek

A periféria-regisztereket a Periféria-regiszter absztrakció C struktúrával mintájára struktúrába szervezzük. Az alábbi címek és bitmezők az MPS2 AN385 CMSDK perifériáinak jellemző értékei, de **ellenőrizendők** a QEMU `hw/arm/mps2.c` és a CMSDK dokumentáció alapján.

A struktúrákat és a függvényprototípusokat egy **fejlécbe** tesszük, nem a `.c` fájlba: a `lab-m19` build minden `.c` fájlt önállóan fordít (lásd a Makefile-t), és ha egy `.c` fájlt egy másik `#include`-dal is beemelne, a megosztott szimbólumok (pl. `uart0_init`) kétszer definiálódnának, és a linker „multiple definition” hibával állna el.

```
/* lab-m19-drivers.h */
#ifndef LAB_M19_DRIVERS_H
#define LAB_M19_DRIVERS_H
#include <stdint.h>

typedef struct {
    volatile uint32_t DATA;       /* 0x000 */
    volatile uint32_t STATE;      /* 0x004 bit0 TXFULL, bit1 RXFULL */
    volatile uint32_t CTRL;       /* 0x008 bit0 TXEN, bit1 RXEN, bit3 RXINTEN */
    volatile uint32_t INTSTATUS;  /* 0x00C olvasás: bit0 TXINT, bit1 RXINT; írás: W1C */
    volatile uint32_t BAUDDIV;    /* 0x010 legalább 16 */
} cmsdk_uart_t;
```

```c
typedef struct {
    volatile uint32_t CTRL;       /* 0x000 bit0 ENABLE, bit3 IRQEN */
    volatile uint32_t VALUE;      /* 0x004 */
    volatile uint32_t RELOAD;     /* 0x008 írása a VALUE-t is beállítja */
    volatile uint32_t INTSTATUS;  /* 0x00C olvasás: pending; írás: W1C (INTCLEAR) */
} cmsdk_timer_t;

#define UART0  ((cmsdk_uart_t *)0x40004000u)
#define TIMER0 ((cmsdk_timer_t *)0x40000000u)

#define UART0_IRQn   0u           /* AN385: UART0 RX */
#define TIMER0_IRQn  8u           /* AN385: Timer0   */

#define LAB_SYSCLK_HZ  25000000u  /* QEMU AN385; laborparaméter */
#define LAB_TICK_HZ    100u       /* 10 ms tick */

void uart0_init(uint32_t bauddiv);
void timer0_init(uint32_t reload);
void uart_puts(const char *s);    /* blokkoló, pollozó TX (az alapfeladat szerződése) */
void uart_putc(char c);
void uart_put_u32(uint32_t v);

#endif

/* lab-m19-drivers.c */
#include "lab-m19-drivers.h"

void uart0_init(uint32_t bauddiv)
{
    UART0->BAUDDIV = bauddiv;
    UART0->CTRL    = (1u << 0) | (1u << 1) | (1u << 3);   /* TXEN, RXEN, RXINTEN */
}

void timer0_init(uint32_t reload)
{
    TIMER0->CTRL      = 0;          /* előbb leállít */
    TIMER0->INTSTATUS = 1u;         /* függő kérés törlése (W1C) */
    TIMER0->RELOAD    = reload;     /* a VALUE-t is beállítja: az első periódus is pontos */
    TIMER0->CTRL      = (1u << 0) | (1u << 3);            /* ENABLE, IRQEN */
}

void uart_putc(char c)
{
    while (UART0->STATE & (1u << 0)) { }    /* amíg TXFULL = 1, várunk */
    UART0->DATA = (uint8_t)c;
}

void uart_puts(const char *s)
{
    while (*s) uart_putc(*s++);
}

void uart_put_u32(uint32_t v)
{
    char d[10];
    unsigned n = 0;
    do { d[n++] = (char)('0' + v % 10u); v /= 10u; } while (v);
    while (n) uart_putc(d[--n]);
}
```

A CMSDK UART `DATA` regiszter olvasása a `STATE.RXFULL` bitet üríti, de **nem** törli önmagában az `INTSTATUS.RXINT` megszakításjelzőt — azt külön, write-1-to-clear móddal kell törölni (lásd Egyszerű UART driver). A CMSDK timernek nincs önálló `INTCLR` regisztere: ugyanaz az

`INTSTATUS` regiszter szolgál a pending állapot olvasására és az 1-et író, write-1-to-clear törlésre. A pontos bitmezők (TX/RX enable, interrupt enable) ellenőrzést igényelnek.

20.2.8.4 ISR-ek

Az ISR-ek sima C függvények. Az UART RX bájtot az SPSC ring bufferbe tesszük, a timer számlál. A megosztott állapotot (`rx_ring`, `tick_count`) a fő programnak is látnia kell, ezért ezek **nem** lehetnek `static`: a tényleges definíció itt, a deklarációjuk (`extern`) egy közös fejlécben van, amelyet a `lab-m19-main.c` is `#include`-ol. A DWT-mintavétel (K5) csak elmenti a törzsidőt; a statisztikát és a kiírást a fő program végzi, az ISR-en kívül.

```
/* lab-m19-isr.h */
#ifndef LAB_M19_ISR_H
#define LAB_M19_ISR_H
#include <stdint.h>
#include "lab-m19-ring.h"

extern ring_t rx_ring;                    /* termelő: UART0_Handler */
extern volatile uint32_t tick_count;      /* csak a TIMER0_Handler írja */

/* ISR-törzsidő-minták (DWT); a gyűjtés az ISR-ben, a kiértékelés a fő programban */
#define LAB_SAMPLES 64u
extern int lab_have_dwt;
extern volatile uint32_t isr_body[LAB_SAMPLES];
extern volatile uint32_t isr_body_n;

void UART0_Handler(void);
void TIMER0_Handler(void);
int  lab_dwt_start(void);
#endif

/* lab-m19-ring.c */
#include "lab-m19-ring.h"

int ring_push(ring_t *r, uint8_t b)
{
    uint16_t next = (uint16_t)((r->head + 1u) % RING_CAP);
    if (next == r->tail) {
        r->overrun++;                     /* tele: az új bájt elveszik, de számoljuk */
        return 0;
    }
    r->buf[r->head] = b;                  /* előbb az adat, ... */
    r->head = next;                       /* ... utána a közzététel */
    return 1;
}

int ring_pop(ring_t *r, uint8_t *out)
{
    if (r->tail == r->head) return 0;     /* üres: nem olvas, a tail helyben marad */
    *out = r->buf[r->tail];
    r->tail = (uint16_t)((r->tail + 1u) % RING_CAP);
    return 1;
}

int ring_empty(const ring_t *r) { return r->tail == r->head; }

/* lab-m19-isr.c */
#include "lab-m19-drivers.h"
#include "lab-m19-isr.h"

#define DEMCR      (*(volatile uint32_t *)0xE000EDFCu)
```

```c
#define DWT_CTRL   (*(volatile uint32_t *)0xE0001000u)
#define DWT_CYCCNT (*(volatile uint32_t *)0xE0001004u)

ring_t rx_ring;
volatile uint32_t tick_count;
int lab_have_dwt;
volatile uint32_t isr_body[LAB_SAMPLES];
volatile uint32_t isr_body_n;

int lab_dwt_start(void)
{
    DEMCR |= 1u << 24;              /* TRCENA */
    DWT_CYCCNT = 0;
    DWT_CTRL |= 1u;                 /* CYCCNTENA */
    __asm volatile ("dsb\n\tisb" ::: "memory");
    uint32_t before = DWT_CYCCNT;
    for (int i = 0; i < 64; i++) __asm volatile ("nop");
    return DWT_CYCCNT != before;
}

void UART0_Handler(void)
{
    uint32_t t0 = lab_have_dwt ? DWT_CYCCNT : 0u;   /* ISR-TÖRZS eleje (nem az esemény ideje) */
    if (UART0->INTSTATUS & (1u << 1)) {             /* RXINT */
        UART0->INTSTATUS = 1u << 1;                 /* nyugtázás (W1C) */
        ring_push(&rx_ring, (uint8_t)UART0->DATA);  /* a DATA olvasása üríti az RXFULL-t */
    }
    if (lab_have_dwt && isr_body_n < LAB_SAMPLES)   /* csak mentés; formázás a fő programban */
        isr_body[isr_body_n++] = DWT_CYCCNT - t0;
}

void TIMER0_Handler(void)
{
    TIMER0->INTSTATUS = 1u;         /* nyugtázás (W1C) */
    tick_count++;                   /* egyetlen író: nincs RMW-verseny a fő programmal */
}
```

Az ISR-ben minden megosztott változó `volatile`, és minden változót pontosan egy kontextus ír: az UART-ISR a `head`-et és az `overrun`-t, a timer-ISR a `tick_count`-ot, a fő program a `tail`-t. A DWT-mérés két mérőpontja az ISR **törzsén belül** van, ezért a mért szám az ISR-törzsidő. Nem tartalmazza a hardveres belépést (az esemény és az első utasítás közti időt), és a fordító prológusának a mérőpont előtti részét sem (20.3).

20.2.8.5 NVIC-konfiguráció és prioritás

Az NVIC engedélyezése és prioritásbeállítása a NVIC konfiguráció: engedélyezés és prioritás mintáját követi. A prioritásbitek száma implementációfüggő. A példában a `nvic_enable` már balra igazított, nyers IPR-bájtot kap; így nem feltételezünk négy implementált bitet. A `0x40` timer- és `0x60` UART-prioritás `LAB_NVIC_PRIO_BITS = 3` mellett a 2-es és a 3-as logikai szint (17.1); más hardveren ezt visszaolvasással is ellenőrizni kell.

```c
/* részlet – a teljes fájl lent */
#define NVIC_ISER0 (*(volatile uint32_t *)0xE000E100u)
#define NVIC_IPR   ((volatile uint8_t  *)0xE000E400u)

void nvic_enable(uint32_t irqn, uint8_t raw_prio)
{
    NVIC_IPR[irqn] = raw_prio;              /* balra igazított IPR-bájt */
    NVIC_ISER0 = (1u << irqn);
}
```

A prioritási sorrend a `lab-m19` esetében: a timer (magasabb prioritás, mert ő az időbázis), majd az UART RX (alacsonyabb). A prioritásértékek konkrét száma az implementált prioritásbitektől függ.

20.2.8.6 Fő program

A fő ciklus a ring buffert fogyasztja és a tickeket dolgozza fel; minden követelmény (K1–K6) a kommentekben megjelölt helyen valósul meg. A ciklus végén versenymentes alvás áll (19.1): a feltétel vizsgálata és a `WFI` ugyanabban a kritikus szakaszban van, így egy közben érkező megszakítás nem veszhet el.

```
/* lab-m19-main.c - a lab-m19 alapfeladat (QEMU mps2-an385): UART0 + Timer0 */
#include <stdint.h>
#include "lab-m19-drivers.h"
#include "lab-m19-isr.h"
#include "lab-m19-crit.h"

#define NVIC_ISER0 (*(volatile uint32_t *)0xE000E100u)
#define NVIC_IPR   ((volatile uint8_t  *)0xE000E400u)

static void nvic_enable(uint32_t irqn, uint8_t raw_prio)
{
    NVIC_IPR[irqn] = raw_prio;              /* nyers, balra igazított IPR-bájt */
    NVIC_ISER0 = 1u << irqn;
}

static uint32_t tick_base, overrun_base;    /* 'R' nullpontja: csak a fő program írja */

static void status_line(void)               /* K3 */
{
    uint32_t pm = crit_enter();             /* koherens pillanatkép a két számlálóról */
    uint32_t t  = tick_count;
    uint32_t ov = rx_ring.overrun;
    crit_exit(pm);
    uart_puts("status: ticks=");
    uart_put_u32(t - tick_base);
    uart_puts(" overrun=");
    uart_put_u32(ov - overrun_base);
    uart_puts(" raw=");                     /* K4: a nyers számláló, a bázispróbához */
    uart_put_u32(t);
    uart_puts("\r\n");
}

static void measure_line(void)              /* K5 */
{
    if (!lab_have_dwt) {
        uart_puts("isr_body: SKIP (DWT CYCCNT nem halad)\r\n");
        return;
    }
    uint32_t n = isr_body_n, mn = 0xFFFFFFFFu, mx = 0, q = 0, r = 0;
    for (uint32_t i = 0; i < n; i++) {
        uint32_t v = isr_body[i];
        if (v < mn) mn = v;
        if (v > mx) mx = v;
        q += v / n;                 /* floor(mean) túlcsordulás nélkül: sum(v) = n*sum(v/n) +
↪   sum(v%n) */
        r += v % n;                 /* (32 bites osztás, nincs 64 bites helper) */
    }
    uint32_t mean = n ? q + r / n : 0;   /* n == 0: nincs minta, a sor nem írja ki */
    uart_puts("isr_body: n="); uart_put_u32(n);
    if (n) {
```

```
        uart_puts(" min="); uart_put_u32(mn);
        uart_puts(" max="); uart_put_u32(mx);
        uart_puts(" mean="); uart_put_u32(mean);
    }
    uart_puts("\r\n");
}

static void handle_byte(uint8_t c)
{
    if (c == 'S') {
        status_line();
    /* K4: új nullpont, nem a megosztott számlálók írása */
    } else if (c == 'R') {
        uint32_t pm = crit_enter();
        uint32_t base = tick_count;
        tick_base    = base;
        overrun_base = rx_ring.overrun;
        crit_exit(pm);
        uart_puts("reset ok base=");         /* a válasz a rögzített új bázist hordozza */
        uart_put_u32(base);
        uart_puts("\r\n");
    } else if (c == 'M') {
        measure_line();
    } else {
        uart_putc((char)c);                  /* K2: echo */
    }
}

int main(void)
{
    __asm volatile ("cpsid i" ::: "memory");
    uart0_init(LAB_SYSCLK_HZ / 115200u);
    lab_have_dwt = lab_dwt_start();
    timer0_init(LAB_SYSCLK_HZ / LAB_TICK_HZ - 1u);   /* 10 ms */
    nvic_enable(TIMER0_IRQn, 0x40u);
    nvic_enable(UART0_IRQn,  0x60u);
    uart_puts("lab-m19 boot\r\n");                    /* K1 */
    __asm volatile ("cpsie i" ::: "memory");

    uint32_t seen = 0;
    for (;;) {
        uint8_t c;
        while (ring_pop(&rx_ring, &c))
            handle_byte(c);
        uint32_t now = tick_count;           /* 32 bites, igazított olvasás: atomi */
        /* K6: minden tick feldolgozva, nem csak "volt-e" */
        while (seen != now) {
            seen++;
            if (seen % LAB_TICK_HZ == 0u)
                uart_puts("tick\r\n");
        }
        uint32_t pm = crit_enter();          /* versenymentes alvás */
        if (ring_empty(&rx_ring) && seen == tick_count)
            __asm volatile ("dsb\n\twfi" ::: "memory");
        crit_exit(pm);
    }
}
```

20.2.8.7 Build és reprodukálható QEMU-indítás

A `lab-m19.elf` buildje a GCC-driverrel linkel, és `-lgcc`-t ad hozzá: a fordító a C-kódhoz (például 64 bites osztáshoz) segédfüggvényeket hívhat, amelyeket közvetlen `ld`-hívás nem találna meg. A `test` cél a két tesztréteget futtatja (20.3):

```
# Makefile - lab-m19 (II. záróprojekt): build, futtatás, tesztek
CROSS   ?= arm-none-eabi-
CC      := $(CROSS)gcc
SIZE    := $(CROSS)size
CFLAGS  := -mcpu=cortex-m3 -mthumb -ffreestanding -O1 -g -Wall -Wextra
LDFLAGS := -nostdlib -T lab-m19.ld -Wl,--gc-sections -Wl,-Map=lab-m19.map
LDLIBS  := -lgcc            # pl. 64 bites osztáshoz vagy más fordítói segédfüggvényhez
SRCS    := lab-m19-startup.S lab-m19-drivers.c lab-m19-ring.c lab-m19-isr.c lab-m19-main.c
OBJS    := $(patsubst %.S,%.o,$(SRCS:.c=.o))
HDRS    := $(wildcard lab-m19-*.h)
QEMU    := qemu-system-arm -machine mps2-an385 -cpu cortex-m3 \
           -display none -monitor none -serial stdio

all: lab-m19.elf

lab-m19.elf: $(OBJS) lab-m19.ld
        $(CC) $(CFLAGS) $(LDFLAGS) $(OBJS) $(LDLIBS) -o $@
        $(SIZE) $@

%.o: %.c $(HDRS)
        $(CC) $(CFLAGS) -c $< -o $@

%.o: %.S
        $(CC) $(CFLAGS) -c $< -o $@

run: lab-m19.elf
        $(QEMU) -kernel lab-m19.elf

ringtest: lab-m19-ring-test.c lab-m19-ring.c lab-m19-ring.h
        gcc -std=c11 -Wall -Wextra -o lab-m19-ring-test lab-m19-ring-test.c lab-m19-ring.c
        ./lab-m19-ring-test

systest: lab-m19.elf
        python3 lab-m19-systest.py $(SYSTEST_FLAGS)    # pl. --isr-band LO:HI, --no-timing

test: ringtest systest

clean:
        rm -f $(OBJS) lab-m19.elf lab-m19.map lab-m19-ring-test

.PHONY: all run ringtest systest test clean

cp ~/arm-labor/cm/a065/lab-mps2.ld lab-m19.ld   # ha még nincs meg
make            # lab-m19.elf + méretösszesítő
make run        # interaktív: a UART0 a terminál stdin/stdout-ja (kilépés: Ctrl-C)
make test       # ringtest (hoston) + systest (QEMU, host-oldali soros bemenettel)
```

A `-serial stdio` a UART0-t a QEMU-folyamat standard bemenetére és kimenetére köti; a `-monitor none -display none` miatt a terminálon csak a firmware kimenete jelenik meg. A `qemu-system-arm` az ELF szegmenseit a betöltési címükre tölti, és a resetet a vektortábla `_estack` és `Reset_Handler` bejegyzésével indítja.

20.2.8.8 Várható viselkedés

Indítás után a soros kimeneten megjelenik a `lab-m19 boot`, majd másodpercenként (100 tick) egy `tick`. A beírt bájtok visszaíródnak; az `S` a `status: ticks=… overrun=… raw=…`, az `R` a `reset ok base=…`, az `M` a K5 sort adja. A `raw` a nyers `tick_count`, a `base` a resetkor rögzített új nullpont; a `ticks` mindig `raw - base`. A firmware a fő úton (arm-none-eabi-gcc 13.2.1 + QEMU 8.2.2, a lektor mérése) `make`-kel hibátlanul épül, `make test` PASS. Az `abSRSM` bemenetre a kimenet (a számok az időzítéstől függnek):

```
lab-m19 boot
abstatus: ticks=96 overrun=0 raw=96
reset ok base=96
status: ticks=0 overrun=0 raw=96
isr_body: SKIP (DWT CYCCNT nem halad)
tick
tick
```

A teljes, automatizált ellenőrzést az 20.3 rendszertesztje végzi; a fenti kimenet a fő úton, QEMU 8.2.2 alatt mért érték (a lektor mérése), a szögletes zárójel nélküli számok az időzítéstől függően eltérhetnek.

20.2.9 Tipikus hibák és ellenpróbák

1. A `volatile` elhagyása. Tünet: a fő ciklus soha nem látja a `tick_flag` változást, vagy a `rx_head` értéke beragad. Hipotézis: a fordító a változót regiszterben tartja. Adatgyűjtés: `objdump -d lab-m19.elf`, és keresd a `tick_flag` betöltését a cikluson kívül. Javítás: `volatile` minősítő minden megosztott változóra.

2. Hiányzó vagy nulla vektorbejegyzés. Tünet: egy véletlenül engedélyezett, de a táblában 0-val kitöltött IRQ (pl. IRQ1 = UART0 TX) HardFault-ot (INVSTATE) okoz. Hipotézis: a tábla adott helyén 0 áll. Adatgyűjtés: `lab-check-vectors.py`, `NVIC_ISER0`. Javítás: a kitöltő helyek `Default_Handler`-re mutassanak (ahogy a 19.3 csinálja), ne 0-ra. (A nem gyenge, fel nem oldott szimbólumra a GNU `ld` ezzel szemben „undefined reference” hibát ad, tehát az nem csendes 0-s vektort eredményez.)

3. Két író egy megosztott számlálón. Tünet: `'R'` után a státusz ritkán inkonzisztens (a `ticks` már nullázott, az `overrun` még nem, vagy fordítva). Hipotézis: a fő program több megosztott változót egymás után ír, és egy ISR a két írás közé ékelődik; vagy a fő program is írja azt a számlálót, amelyet az ISR növel. Javítás: a megosztott számlálót csak az ISR írja, a fő program nullpontot tárol (`tick_base`), és a nullpontokat egyetlen kritikus szakaszban rögzíti.

4. Hosszú kritikus szakasz. Tünet: a megszakítás-latencia nő, az UART RX túlcsordul. Hipotézis: a kritikus szakaszon belül `uart_puts`-t hívunk. Javítás: a kritikus szakaszból csak az adatmozgatást tartsuk bent.

5. `cpsie i` a `crit_exit`-ben. Tünet: beágyazott kritikus szakaszban a megszakítások túl korán engedélyeződnek. Javítás: a mentett `PRIMASK` visszaállítása.

20.2.10 Feladatok

1. (magyarázó) Magyarázd el, miért nem elegendő a `volatile` minősítő a naiv `tick_flag`-kezelésnél. Add meg utasításszinten (`ldr`/`str`) azt a sorrendet, amelyben egy tick nyoma elveszik, és jelöld, melyik két utasítás közé ékelődik az ISR. Indokold, miért nem lehetséges a fordított eset, hogy a fő program az ISR `tick_count++`-ának közepére ékelődik.

2. (implementációs) Egészítsd ki a `lab-m19` firmware-t egy TX ring bufferrel. A `uart_puts` ne várjon a TX pufferen, hanem a TX interruptra támaszkodjon. Integráld az új ISR-t a vektortáblába és az NVIC-be, és gondoskodj a termelő/fogyasztó indexek védelméről.

3. (hibakeresési/mérési) Egy jelzőalapú változatban a `tick` sorok száma kevesebb, mint az eltelt tickeké. Tervezz mérést, amely szétválasztja a három lehetséges okot: (a) a jelző szándékos összevonása lassú fogyasztó mellett, (b) a fogyasztói lemaradás, (c) a védtelen olvasás–törlés versenye. Melyik mennyiséget méred melyikhez, és melyik változat (jelző védelem nélkül, jelző kritikus szakaszban, számlálókülönbség) mit mutat?

20.2.11 Megoldások

1. megoldás. A `volatile` csak azt biztosítja, hogy a fordító minden hozzáféréskor ténylegesen olvas és ír; atomitást nem ad. A naiv kód utasításszinten: `ldr r0, [tick_flag]` (A), feltételes ugrás, `str r1(=0), [tick_flag]` (B). Ha az ISR az (A) és a (B) közé ékelődik és `tick_flag = 1`-et ír, a (B) ezt az új értéket írja felül 0-ra, mielőtt a fő program látta volna: a tick nyoma elveszik (lásd az idődiagramot). A fordított eset azért lehetetlen, mert egyetlen magon az ISR a fő programot szakítja meg, és lefut a végéig, mielőtt a fő program folytatódna. Az ISR `ldr/adds/str` hármasa közé a fő program nem ékelődhet. (Egy magasabb prioritású másik ISR igen, de az a `tick_count`-ot nem írja.) Javítás: a teszt és a törlés egy kritikus szakaszban, vagy a jelző helyett számláló, egyetlen íróval.

2. megoldás. A TX oldalon a TXINT akkor áll be, amikor egy bájt elküldése után a puffer kiürül — üres pufferrel a TXIE bekapcsolása önmagában nem kér megszakítást. Ezért a `uart_puts` **priming** lépést végez: ha a TX épp tétlen (`tx_busy == 0`), az első bájtot közvetlenül a `DATA`-ba írja, és beállítja a `tx_busy = 1`-et; a többit a TX ring bufferbe teszi, és engedélyezi a TXIE-t. Az ISR a ringből betölti a következő bájtot, a `DATA`-ba írja; ha a ring kiürül, `tx_busy = 0` és a TXIE letiltása következik. Az indexek SPSC mintát követnek: a `tx_head`-et a fő program, a `tx_tail`-t az ISR írja.

```
/* TX ring: a fő program a head-et, az ISR a tail-t írja (SPSC) */
#define TX_CAP 64u
_Static_assert((TX_CAP & (TX_CAP - 1u)) == 0, "TX_CAP: 2 hatványa");
static volatile uint8_t  tx_buf[TX_CAP];
static volatile uint16_t tx_head, tx_tail;
static volatile uint8_t  tx_busy;

static int tx_ring_push(uint8_t b)
{
    uint16_t next = (uint16_t)((tx_head + 1u) % TX_CAP);
    if (next == tx_tail) return 0;       /* tele: a bájt elveszik */
    tx_buf[tx_head] = b;
    tx_head = next;
    return 1;
}

static int tx_ring_pop(uint8_t *out)
{
    if (tx_tail == tx_head) return 0;    /* üres */
    *out = tx_buf[tx_tail];
    tx_tail = (uint16_t)((tx_tail + 1u) % TX_CAP);
    return 1;
}

void uart_puts_irq(const char *s)
{
    if (!*s) return;
```

```
    if (!tx_busy) {                 /* priming: az első bájtot közvetlenül */
        tx_busy = 1;
        UART0->DATA = (uint8_t)*s++;
    }
    while (*s) tx_ring_push((uint8_t)*s++);
    UART0->CTRL |= (1u << 2);       /* TXIE engedélyezése */
}

void UART0_Handler(void)
{
    uint32_t st = UART0->INTSTATUS;
    if (st & (1u << 1)) {           /* RXINT */
        ring_push(&rx_ring, (uint8_t)UART0->DATA);
        UART0->INTSTATUS = 1u << 1;
    }
    if (st & (1u << 0)) {           /* TXINT: a puffer kiürült */
        uint8_t b;
        if (tx_ring_pop(&b)) UART0->DATA = b;
        else { tx_busy = 0; UART0->CTRL &= ~(1u << 2); }
        UART0->INTSTATUS = 1u << 0;
    }
}
```

A TX-ringes változat a K1–K6 rendszerteszt elvárásait nem változtatja meg: a `make test` ugyanazt a kimenetet adja, mert a protokoll és a válaszok változatlanok. A futtatás eredményét a fő úton (arm-none-eabi-gcc + QEMU) a 20.3 laborja rögzíti; a jelen fejezet csak a kódot adja meg.

3. megoldás. Három külön mennyiséget kell mérni. (1) Az eltelt tickek száma: a `tick_count`, amelyet csak az ISR növel. (2) A feldolgozott tickek száma: a fő program saját számlálója. A kettő különbsége a **lemaradás**. (3) Az összevont események száma: az ISR növelhet egy `merged` számlálót, ha belépéskor a jelző már 1. Ez **nem** bizonyít versenyt, csak azt, hogy a fogyasztó két tick között nem ért oda. A verseny bizonyítéka az, ha védtelen kódnál az eltelt − feldolgozott − összevont különbség pozitív, védett kódnál pedig nulla. Számlálókülönbséges változatnál a feldolgozott tickek száma a lemaradás ledolgozása után mindig megegyezik az eltelttel. A tesztnek ezt az azonosságot kell ellenőriznie, nem azt, hogy az összevonás „nullához közeli”: az összevonás mértéke a terheléstől függ, és nem hiba. QEMU-ban az időzítés nem valós idejű, ezért az arányok csak relatívak.

```
/* a jelzőalapú változat ISR-jében: az összevonás mérése */
if (tick_flag) merged++;      /* az előző ticket még nem dolgozta fel a fő program */
tick_flag = 1;
tick_count++;

/* a mérési változat 'S' válasza: a három mennyiség egy sorban */
status: ticks=<n> seen=<s> merged=<m> overrun=<o> raw=<r>
```

20.2.12 Rövid összefoglaló

A driver és ISR integrációja a vektortábla, az NVIC, az ISR-törzs és a megosztott állapot védelmének konzisztens egysége, és pontosan az 20.1 specifikációját (K1–K6, F2, F3) valósítja meg. Az M-profile ISR sima C függvény; a hardver menti a kontextust. Minden megosztott változót egyetlen kontextus ír; az SPSC ring buffer így kritikus szakasz nélkül működik, a tickeket számláló viszi át, és a kritikus szakasz csak a koherens pillanatképhez és a versenymentes alváshoz kell. A `volatile` láthatóságot ad, atomitást nem. A build a korábbi fejezetek elemeit egyetlen, QEMU-ban reprodukálható firmware-be fűzi.

Továbbhasznált definíciók:

- **driver-ISR integráció:** periféria-driver, ISR, vektorbejegyzés, NVIC-beállítás és kritikus szakasz konzisztens egysége.
- **SPSC ring buffer:** egytermelős–egyfogyasztós puffer, ahol a `head`-et csak az ISR, a `tail`-t csak a fő program írja.
- **kritikus szakasz:** megszakítás-letiltással védett, rövid, nem blokkoló utasítássorozat a megosztott állapot atomi frissítésére.

20.2.13 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblát):

- Armv7-M Architecture Reference Manual, DDI0403E — kivételmodell, vektortábla, `PRIMASK`, kivétel-belépés/visszatérés.
- Arm M-profile architektúra dokumentáció — NVIC-engedélyezés, prioritás, ISR-szerződés.
- QEMU dokumentáció — `mps2-an385` machine, `-kernel` betöltés, `-nographic` soros I/O.

Ellenőrzési hiányok (a hallgatónak kell megerősítenie): a fő úton a könyv blokkjaiból összeállított projekt `make`-kel hibátlanul épül, és a `make test` a 20.3 szerinti kimenettel `PASS` (a lektor mérése: arm-none-eabi-gcc 13.2.1 + QEMU 8.2.2). A helyettesítő eszközlánccal (Clang + `ld.lld`) és a `tools/cm-run.py` Cortex-M3 emulátorban végzett funkcionális próba ezt csak kiegészíti; az emulátor a Timer0-t, a UART0 RX-et és a megszakítás-kézbesítést funkcionálisan, nem cikluspontosan modellezi, ezért abszolút időre nem használható. Ezen felül a hallgatónak meg kell erősítenie: az MPS2 AN385 pontos IRQ-számai (`TIMER0_IRQn`, `UART0_IRQn`); a CMSDK UART és Timer regiszter-offszetek és bitmezők (`DATA`, `STATE`, `CTRL`, `INTSTATUS`); az `NVIC_IPR` prioritásbitszáma; a timer referencia-órajel és az 1 másodperces periódushoz tartozó `RELOAD`; az `UART0` és `TIMER0` báziscímek; valamint a QEMU verziófüggő `-kernel` betöltési és soros viselkedése. A fenti kód ezekre az értékekre nézve **jelölt**, nem ellenőrzött; a QEMU-emulációban mért periódus relatív, nem hardverhű abszolút idő.

20.3 Mérés, validáció és a QEMU-alapú teszt reprodukálhatósága

20.3.1 Motiváció: a „működik nálam” nem bizonyíték

Az előző alfejezetben (driver és ISR integrálása) összeállt egy firmware, amelyben egy periféria-illesztő, az ISR, a vektorbejegyzés, az NVIC konfiguráció és a megosztott SPSC ring buffer egyetlen, QEMU-ban betölthető egésszé áll össze. A fejlesztő fejében innen gyakran az a mondat születik, hogy „kész, működik”. Egy mérnöki projektben ez nem elég: a „működik” egy személyes megfigyelés, amely nem mondja meg, milyen bemenetre, milyen sorrendben, milyen feltételek mellett, és mi az, ami **nem** működik. Ez az alfejezet célja, hogy a modult **mérhető, önállóan reprodukálható** egységgé zárja: olyan funkcionális tesztekkel, amelyeknek van elfogadási kritériumuk, és olyan latenciaméréssel, amelynek van dokumentált módszere és ismert korlátja.

A konkrét mérnöki feladat három, egymástól jól elkülöníthető kérdés megválaszolása. (1) **Funkcionális**: teljesíti-e a firmware az 20.1 követelményeit (K1–K6), és a kapacitáson belüli bemenet helyesen, sorrendhelyesen és veszteség nélkül jut-e a ring bufferből a fogyasztóhoz (F2), illetve a tele puffer a rögzített veszteségpolitika szerint viselkedik-e (F3)? (2) **Teljesítmény**: mennyi ciklust tölt az ISR **törzse** az első mérőponttól az utolsóig, és mekkora ennek a szórása? (3) **Reprodukálhatóság**: ha a jegyzőkönyvbe beírom a pontos parancsot és a bemenetet, egy másik mérnök ugyanazon a gépen ugyanabban az eredmény-tartományban kapja-e vissza ugyanazt? A

fejezet ezt a hármat tanítja egymásra építve, mert a három közül bármelyik hiányában a másik kettő nem áll.

20.3.2 Definíció: funkcionális validáció, mérés, reprodukálhatóság

A fejezet három kulcsfogalmat vezet be, amelyeket élesen megkülönböztetünk.

- **funkcionális validáció:** egy rögzített bemenet-szekvencia és egy hozzá tartozó elvárt kimenet-viselkedés (nem feltétlenül számérték, hanem tulajdonság: sorrend, hossz, nem-elveszés) összevetése; az eredmény bináris: megfelel vagy nem.
- **mérési módszer:** az a meghatározott eljárás, ahogyan egy mennyiséget (itt ciklusszámot) mintavételezünk, beleértve a jelölés helyét, a mintavétel gyakoriságát, a gyűjtött statisztikát (minimum, maximum, átlag, szórás) és a mértékegységet (ciklus, CPU-órajel-ütem).
- **reprodukálhatóság:** az a tulajdonság, hogy a rögzített parancs, rögzített bemenet és rögzített eszközlánc-verzió mellett a kimenet egy dokumentált tartományon belül marad. Nem azt jelenti, hogy bitre ugyanaz (QEMU-emulációban lehet determinisztikus, fizikai hardveren nem), hanem azt, hogy az eltérés **kijelentett és korlátos**.

Ez az alfejezet **hatásköre** szűk: nem írunk formális verifikációt (modell-ellenőrzés, szerződéses bizonyítás), és nem építünk RTOS-ütemezőt vagy kernel-szolgáltatásokat. A validáció célja a meglévő modul viselkedésének dokumentált bemutatása és a mérés megismételhetővé tétele — semmi ennél több.

A funkcionális validáció és a teljesítménymérés két különböző kérdést válaszol meg, és két különböző elfogadási kritériumot kap. Ez azért fontos, mert a gyakorlatban gyakran összekeverik őket: egy gyors, de hibás ISR nem „jobb”, és egy pontos, de lassú ISR nem „sikeres”. Az elfogadási kritérium mindig **mindkét** tengelyen megfogalmazandó.

20.3.3 A funkcionális tesztsorozat QEMU-ban

A funkcionális tesztek célja, hogy a driver-ISR-integrációt nem egyetlen „szép” futásra, hanem **előre megfogalmazott esetekre** próbálja ki. Két, egymástól független réteget használunk, és mindkettő **futtatható**, nem vázlat:

1. **Egységteszt a hoston (U1–U6).** A ring buffer hardverfüggetlen modul (`lab-m19-ring.c`), ezért a host `gcc`-jével, hardver és emulátor nélkül tesztelhető. Esetek: alapeset és sorrendtartás (3 bájt be, ugyanaz ki, sorrendben), üres puffer (a `pop` nem olvas, a `tail` helyben marad), tele puffer (pontosan 63 bájt fér el, a többi az F3 politikája szerint elvész, és az `overrun` számolja), körbefordulás (5000 bájt kis adagokban, kapacitáson belül, veszteség nélkül), határ (az N−1 = 63. elem még befér, az N. elemre az előírt overflow következik be), és lassú fogyasztó (kettőt tesz be, egyet vesz ki: a kivett bájtok sorrendben egyeznek az elfogadottakkal, az `overrun` pontosan az elutasított `push`-ok száma). A tele puffer viselkedése itt determinisztikusan vizsgálható, a soros vonalon nem.
2. **Rendszerteszt soros bemenettel (K1–K6, F2, ismételt reset).** A teljes firmware a QEMU-ban fut, a UART0-ját a `-serial stdio` a QEMU-folyamat stdin/stdout-jára köti. A hostoldali `lab-m19-systest.py` ide ír bájtokat: ez valódi **RX-injektálás** a periférián keresztül, és a firmware UART-ISR-je fogadja őket. A választ a kimeneten olvassa. A firmware a saját `DATA`-írásával nem tudna RX-et szimulálni: az a TX-irány.

20.3.3.1 Elfogadási kritérium megfogalmazása

Egy tesztsorozat csak akkor értékelhető, ha előre rögzítettük, mi számít sikernek. A kritériumokat **a teszt futtatása előtt** fogalmazzuk meg, és mindegyik egy követelményhez tartozik:

#	Tengely	Elfogadási kritérium	Mérési mód
F1	Helyesség	minden egység- és rendszerteszt `PASS` (a `SKIP` nem számít sikernek, de hibának sem)	`make test` kimenete
F2	Veszteségmentesség **a kapacitáson belül**	a legfeljebb 63 bájtos burst bájtjai sorrendben, hiánytalanul visszaíródnak	rendszerteszt `F2`, egységteszt `U1`, `U4`, `U6`
F3	Veszteségpolitika tele puffernél	az új bájt elveszik, az `overrun` pontosan az eldobottak számával nő	egységteszt `U3`, `U5`, `U6`
P1	ISR-törzsidő stabilitása	működő DWT mellett a `min..max` egy előre rögzített, platformonként megadott sávban; DWT nélkül `SKIP`	`M` parancs (K5a formátum, K5b sáv)
T1	Időbázis	a tickfrekvencia 100 Hz ±20% egy valós idejű futtatón; nem valós idejűnél `SKIP`	rendszerteszt `T1`

Tele puffernél tehát **nem** írunk elő veszteségmentességet: az F2 csak a kapacitáson belüli esetekre szól, a tele puffert az F3 szabályozza. A kritérium rögzítése azért kötelező, mert utólag mindig könnyű „hozzáigazítani” az elvárást az eredményhez. Az előzetes kritérium az egyetlen védelem ez ellen.

20.3.4 A megszakítás-latencia mérése a modulon

Az 17.4 alfejezetben megismert módszert itt a **konkrét modulra** alkalmazzuk, de pontosan meg kell nevezni, mit mér. A `lab-m19` UART-ISR-je az első végrehajtható utasításában és a visszatérés előtt is beolvassa a DWT `CYCCNT` számlálót. A különbség az **ISR-törzsidő**: a két mérőpont között eltelt ciklusszám. Ez **nem** belépési latencia:

- nem tartalmazza a hardveres kivétel-belépést (az esemény és az ISR első utasítása közötti időt: prioritás-kiértékelés, keretmentés, vektorolvasás);
- nem feltétlenül tartalmazza a fordító által generált prológus azon részét, amely az első mérőpont elé kerül, és az epilógust sem.

A mintát az ISR csak elmenti egy tömbbe. A statisztikát (`min`, `max`, `mean`) és a kiírást a fő program végzi az `M` parancsra, a mért blokkon kívül, így a formázás és a kiírás nem torzítja a mérést.

A DWT jelenlétét és a CYCCNT haladását futás előtt ellenőrizni kell. A QEMU 8.2.2 `mps2-an385` modelljében a számláló nem halad: itt nincs érvényes ciklusszám, a mérés `SKIP`. Más emulátoron a működő számláló sem jelent automatikusan cikluspontos modellt. Fizikai hardveren a mért ciklusszámot külön jegyzőkönyvezzük.

20.3.4.1 A latencia forrása — levezetés

Az eseménytől az ISR végéig eltelt idő több összetevőből áll:

$$C_{\text{esemény}\to\text{vég}} = C_{\text{belépés}} + C_{\text{prológus előtti rész}} + C_{\text{törzs}} + C_{\text{mérési overhead}}$$

- $C_{\text{belépés}}$ a hardveres kivétel-belépés (verembe mentés, prioritás-kiértékelés, ugrás a vektorra). Függ attól, milyen utasítás fut a megszakítás érkezésekor (például egy hosszú `LDM`/`STM` vagy osztás késleltetheti).
- $C_{\text{törzs}}$ az ISR tényleges munkája a két mérőpont között; ezt méri a K5.
- $C_{\text{mérési overhead}}$ magának a `CYCCNT` beolvasásának és a minta mentésének a költsége; a mérést úgy tervezzük, hogy ez állandó legyen.

A K5 tehát csak a $C_{\text{törzs}} + C_{\text{mérési overhead}}$ részt adja. A teljes, eseménytől számított latenciához **eseményoldali időbélyeg** kell. Timer-megszakításnál ez rendelkezésre áll: a Timer0 `VALUE` regisztere a nullátmenet óta számol vissza, így az ISR elején kiolvasott RELOAD − VALUE az eseménytől eltelt timer-ütemek száma (ha a timer órajele a CPU-órajel). UART-nál a hardver nem ad ilyen időbélyeget; ott külső triggerre (logikai analizátor, GPIO-jel) van szükség, ami csak hardveren végezhető.

20.3.5 Kidolgozott példa: a teszt és a mérés futása

Munkakönyvtár: `~/arm-labor/cm/zaro2/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez a II. záróprojekt közös könyvtára: mind a négy alfejezete (20.1–20.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/cm/zaro2
cd ~/arm-labor/cm/zaro2
```

Az alábbi két program **futtatható**: mindkettőt lefuttattuk (a körülményeket lásd lent). A firmware forrásait és a `Makefile`-t az 20.2 adja; a `make test` mindkét réteget elindítja.

Egységteszt a hoston (`make ringtest`):

```
/* lab-m19-ring-test.c - a ring buffer egységtesztje a HOSTON (gcc), hardver nélkül.
 *   gcc -std=c11 -Wall -Wextra -o lab-m19-ring-test lab-m19-ring-test.c lab-m19-ring.c */
#include <stdio.h>
#include <string.h>
#include "lab-m19-ring.h"

static int fails;
#define CHECK(name, cond) do {                                  \
        int ok_ = (cond);                                       \
        printf("%s  %s\n", ok_ ? "PASS" : "FAIL", name);        \
        fails += !ok_;                                          \
    } while (0)

int main(void)
{
    ring_t r;
    uint8_t b, out[RING_CAP];

    /* U1 alapeset: 3 bájt be, ugyanaz a 3 ki, ugyanabban a sorrendben */
    memset(&r, 0, sizeof r);
    ring_push(&r, 0x41); ring_push(&r, 0x42); ring_push(&r, 0x43);
    int n = 0;
```

```
    while (ring_pop(&r, &b)) out[n++] = b;
    CHECK("U1 alapeset: count = 3, sorrend ABC",
          n == 3 && out[0] == 0x41 && out[1] == 0x42 && out[2] == 0x43);

    /* U2 üres puffer: nem olvas, a tail helyben marad */
    uint16_t tail = r.tail;
    CHECK("U2 üres puffer: pop = 0, tail változatlan", ring_pop(&r, &b) == 0 && r.tail == tail);

    /* U3 tele puffer: RING_CAP-1 fér el; a többi eldobva és számolva */
    memset(&r, 0, sizeof r);
    int accepted = 0;
    for (int i = 0; i < 100; i++) accepted += ring_push(&r, (uint8_t)i);
    CHECK("U3 tele: pontosan RING_CAP-1 = 63 bájt fér el", accepted == RING_CAP - 1);
    CHECK("U3 tele: overrun = 100 - 63 = 37", r.overrun == 100u - (RING_CAP - 1));
    n = 0;
    while (ring_pop(&r, &b)) out[n++] = b;
    int order = n == RING_CAP - 1;
    for (int i = 0; i < n; i++) order = order && out[i] == (uint8_t)i;
    CHECK("U3 tele: a megtartott bájtok az első 63, sorrendben (az újak vesznek el)", order);

    /* U4 körbefordulás: sok kis adag a puffer végén át, veszteség nélkül */
    memset(&r, 0, sizeof r);
    int ok = 1; uint8_t next_in = 0, next_out = 0;
    for (int round = 0; round < 1000; round++) {
        for (int k = 0; k < 5; k++) ok &= ring_push(&r, next_in++);
        for (int k = 0; k < 5; k++) ok &= ring_pop(&r, &b) && b == next_out++;
    }
    CHECK("U4 körbefordulás: 5000 bájt kapacitáson belül, veszteség és csere nélkül",
          ok && r.overrun == 0);

    /* U5 határ: N-1 elem még befér, az N. elemre előírt az overflow */
    memset(&r, 0, sizeof r);
    int all = 1;
    for (unsigned i = 0; i < RING_CAP - 1; i++) all &= ring_push(&r, (uint8_t)i);
    CHECK("U5 határ: az N-1 = 63. elem még befér", all && r.overrun == 0);
    CHECK("U5 határ: az N. elem elutasítva, overrun = 1",
          ring_push(&r, 0xEE) == 0 && r.overrun == 1);

    /* U6 lassú fogyasztó: kettőt tesz be, egyet vesz ki; a sorrend megmarad,
       a veszteség pontosan az elutasított push-ok száma */
    memset(&r, 0, sizeof r);
    unsigned pushed = 0, rejected = 0, popped = 0;
    int inorder = 1;
    uint8_t seq[512]; unsigned nseq = 0;            /* az elfogadott bájtok sorrendje */
    for (int round = 0; round < 200; round++) {
        for (int k = 0; k < 2; k++) {
            uint8_t v = (uint8_t)pushed++;
            if (ring_push(&r, v)) seq[nseq++ % 512] = v; else rejected++;
        }
        if (ring_pop(&r, &b)) { inorder &= b == seq[popped % 512]; popped++; }
    }
    while (ring_pop(&r, &b)) { inorder &= b == seq[popped % 512]; popped++; }
    CHECK("U6 lassú fogyasztó: kivett = elfogadott, sorrendben", inorder && popped == nseq);
    CHECK("U6 lassú fogyasztó: overrun = elutasított push-ok",
          r.overrun == rejected && rejected > 0);

    puts(fails ? "RINGTEST: FAIL" : "RINGTEST: PASS");
    return fails != 0;
}
```

Rendszerteszt host-oldali soros bemenettel (`make systest`). A teszt alapértelmezésben a QEMU-t indítja; a `--runner` kapcsolóval bármely más futtató megadható, amely a UART0-t a stdin/stdout-ra köti. A teszt három szabályt követ.

- **Pontos keretezés.** A `tick` sorok aszinkron, akár két echo-bájt közé is beékelődhetnek. Ezért a teszt minden adatcsomag után egy `S`-t küld, és a státuszsorig visszaérkezett bájtokat, a `tick\r\n` sorok eltávolítása után, **pontosan** az elküldött adattal veti össze. Egy regex-részsztring keresése nem elég, mert 51 visszaküldött bájt között is megtalálja az 50 elvártat.
- **Heterogén adat.** Az F2 nem 50 egyforma bájtot küld, hanem egy determinisztikus, 50 bájtos sorozatot olyan ábécéből, amely a vezérlőparancsokat (`S`, `R`, `M`) és a `tick` részsztringet nem tartalmazza. Így a felcserélt bájt is lebukik, nem csak a hiányzó vagy a többlet.
- **Szétválasztott szintek.** A funkcionális próbák (K1–K4, K6, F2, K5a) a host ütemezésétől függetlenek: a K6 két `tick` sor alatti, legalább 100-as növekedést vár, nem egy abszolút tickszámot. A K4 a reset tényét **a válaszból** ellenőrzi, nem abból, hogy egy későbbi minta kisebb a korábbinál.
- **A reset bizonyítéka a rögzített bázis.** A „reset után kisebb a számláló” feltétel hostidőzítést feltételez. A helyes számláló a reset után tovább nő, amíg a host eljut a következő `S`-ig: 3 másodperces hostkésés után az új érték (300 tick) már nagyobb a reset előttinél (204), és a jó firmware is `FAIL`-t kapna. Ezért az `R` válasza a kritikus szakaszban rögzített új nullpontot is hordozza (`reset ok base=<b>`), a státuszsor pedig a bázishoz mért érték mellett a nyers számlálót is (`ticks=<n> … raw=<r>`). A K4 három feltétele: (1) a bázis nem kisebb a reset előtti nyers értéknél, vagyis a reset a kérés után történt; (2) a reset utáni két státuszsorban `ticks = raw − base`, vagyis a számlálás valóban erről a bázisról indul; (3) a nyers számláló monoton. Ez bármekkora hostkésés mellett igaz. Egy olyan firmware viszont, amely a választ megadja, de a bázist nem állítja át, a (2) feltételen elbukik, akkor is, ha a számlálója egyébként monoton. A `--reset-delay SEC` kapcsoló a reset után szándékos szünetet iktat be: ez a próba saját ellenpróbája. Az időzítési szerződés (T1: a tickfrekvencia a host órájához mérve, ±20% tűréssel) külön eredmény, és nem valós idejű futtatónál (`--no-timing`) `SKIP`. Az ISR-törzsidőnél a K5a a formátumot és a belső összefüggéseket ellenőrzi (`0 < n ≤ 64`, `min ≤ mean ≤ max`), a K5b a platformparaméterként megadott sávot (`--isr-band LO:HI`, a P1 kritérium). Működő számláló nélkül mindkettő `SKIP`.

```
#!/usr/bin/env python3
"""lab-m19-systest.py - a lab-m19 alapfeladat rendszertesztje host-oldali soros bemenettel.

Használat:
  python3 lab-m19-systest.py                       # QEMU (alapértelmezett futtató)
  # más futtató, amely a UART0-t stdin/stdout-ra köti
  python3 lab-m19-systest.py --runner "<parancs>"
  További kapcsolók:
    --no-timing        a T1 frekvenciapróba SKIP (nem valós idejű futtatónál kötelező)
    --isr-band LO:HI   a K5b platformsáv: az ISR-törzsidő min..max ciklusban
    --reset-delay SEC  szünet a reset után (a K4 ütemezésfüggetlenségének ellenpróbája)
A firmware UART0-ját a futtató a folyamat stdin/stdout-jára köti; a teszt ide ír
bájtokat (RX-injektálás) és innen olvassa a választ. Kilépési kód: 0, ha nincs FAIL.

Két szint: a funkcionális próbák (K1-K4, K6, F2, K5a) a host időzítésétől
függetlenek; az időzítési próba (T1) és a hardveres sáv (K5b) külön eredmény.
"""
import queue
import re
import shlex
import subprocess
import sys
import threading
import time

QEMU = ("qemu-system-arm -machine mps2-an385 -cpu cortex-m3 -display none "
        "-monitor none -serial stdio -kernel lab-m19.elf")
```

```
TICK_HZ = 100
LAB_SAMPLES = 64
TICK_LINE = 'tick\r\n'
STATUS = r'status: ticks=(\d+) overrun=(\d+) raw=(\d+)\r\n'
# Heterogén, determinisztikus burst: a vezérlőparancsokat (S, R, M) és a "tick"
# részsztringet nem tartalmazó ábécéből, kapacitáson belül (50 < 63).
ALPHA = '0123456789ABCDEFGHJKLNOPQTUVWXYZ'
BURST = ''.join(ALPHA[(7 * i + 3) % len(ALPHA)] for i in range(50))


class Dut:
    def __init__(self, cmd):
        self.p = subprocess.Popen(shlex.split(cmd), stdin=subprocess.PIPE,
                                  stdout=subprocess.PIPE, stderr=subprocess.DEVNULL)
        self.q = queue.Queue()
        self.buf = ''
        threading.Thread(target=self._reader, daemon=True).start()

    def _reader(self):
        while True:
            b = self.p.stdout.read(1)
            if not b:
                return
            self.q.put(b.decode('latin-1'))

    def send(self, s):
        try:
            self.p.stdin.write(s.encode())
            self.p.stdin.flush()
        except (BrokenPipeError, OSError):
            # a futtató kilépett: a várakozó olvasás bukik el
            pass

    def read_until(self, pattern, timeout=10.0):
        """Vár, amíg a pattern megjelenik; a találat végéig fogyasztja a puffert.
        Visszaadja a (találat előtti szöveg, match) párt; időtúllépésnél match = None."""
        rx = re.compile(pattern)
        end = time.monotonic() + timeout          # monotonic: a faliórás ugrás nem változtatja
↪   a határidőt
        while True:
            m = rx.search(self.buf)
            if m:
                before, self.buf = self.buf[:m.start()], self.buf[m.end():]
                return before, m
            try:
                self.buf += self.q.get(timeout=max(0.0, end - time.monotonic()))
            except queue.Empty:
                return self.buf, None

    def close(self):
        # Leállítás + learatás: a pipe-ok zárása, wait() időkorláttal, kilépett folyamat
        ↪ kezelése.
        try:
            if self.p.poll() is None:
                self.p.kill()
            try:
                self.p.wait(timeout=5.0)
            except subprocess.TimeoutExpired:
                pass
        finally:
            for pipe in (self.p.stdin, self.p.stdout):
                try:
                    if pipe:
```

```
                    pipe.close()
                except OSError:
                    pass


def strip_ticks(text):
    """Az aszinkron tick sorokat a protokoll bárhol beékelheti: ezeket eltávolítjuk."""
    return text.replace(TICK_LINE, '')


def framed_echo(dut, data):
    """Elküldi az adatot, majd egy 'S'-t; a status sorig érkezett bájtoknak (a tick
    sorok nélkül) PONTOSAN az adatnak kell lenniük: ez kizárja a hiányzó,
    duplikált, többlet- és felcserélt bájtot."""
    dut.send(data + 'S')
    before, m = dut.read_until(STATUS)
    got = strip_ticks(before)
    # Siker csak úgy lehet, ha a lezáró státuszkeret határidőn belül megérkezett ÉS a
    # payload pontos; a hiányzó keretet (timeout) nem fedi el a payload-egyezés.
    return m is not None and got == data, got, m


def status(dut):
    dut.send('S')
    _, m = dut.read_until(STATUS)
    return int(m.group(1)) if m else None


def status_raw(dut):
    """(ticks, raw): a bázishoz mért és a nyers számláló ugyanabból a pillanatképből."""
    dut.send('S')
    _, m = dut.read_until(STATUS)
    return (int(m.group(1)), int(m.group(3))) if m else None


def main(argv):
    cmd = argv[argv.index('--runner') + 1] if '--runner' in argv else QEMU
    timing = '--no-timing' not in argv
    delay = float(argv[argv.index('--reset-delay') + 1]) if '--reset-delay' in argv else 0.0
    band = None
    if '--isr-band' in argv:
        lo, hi = argv[argv.index('--isr-band') + 1].split(':')
        band = (int(lo), int(hi))
    dut = Dut(cmd)
    results = []

    def check(name, ok, detail=''):
        results.append((name, 'PASS' if ok else 'FAIL', detail))

    def skip(name, detail):
        results.append((name, 'SKIP', detail))

    try:
        _, m = dut.read_until(r'lab-m19 boot\r\n')
        check('K1 boot', m is not None)

        ok, got, m = framed_echo(dut, 'ab')               # K2: echo, pontosan
        check('K2 echo pontosan "ab"', ok,
              '' if ok else ('a lezáró státuszkeret nem érkezett meg (timeout), payload: ' +
              ↪  repr(got)
                             if m is None else repr(got)))

        ok, got, m = framed_echo(dut, BURST)              # F2: kapacitáson belüli burst
```

```
check('F2 burst 50 heterogén bájt: pontos sorrend és hossz', ok,
      '' if ok else f'{len(got)} bájt érkezett')
check('K3 status formátum, overrun=0', m is not None and m.group(2) == '0',
      m.group(0).strip() if m else 'nincs válasz')

# K6: funkcionális, host-időzítéstől független
t1 = int(m.group(1)) if m else None
lines = 0
for _ in range(2):
    _, mt = dut.read_until(re.escape(TICK_LINE), timeout=5.0)
    lines += mt is not None
check('K6 periodikus tick sor (2 db)', lines == 2)
t2 = status(dut)
check('K6 két tick sor alatt a számláló legalább 100-zal nőtt',
      t1 is not None and t2 is not None and t2 - t1 >= TICK_HZ, f'{t1} -> {t2}')

def reset_probe(name):                          # K4: a reset-válasz bázisa
    before = status_raw(dut)
    dut.send('R')
    _, mr = dut.read_until(r'reset ok base=(\d+)\r\n')
    time.sleep(delay)                           # ellenpróba: hostkésés
    after, later = status_raw(dut), status_raw(dut)
    base = int(mr.group(1)) if mr else None
    # A reset ténye a válaszban rögzített bázisból következik, nem abból, hogy
    # egy későbbi minta kisebb: a ticks = raw - bázis bármekkora késés mellett igaz.
    ok = (None not in (before, after, later, base) and base >= before[1]
          and after[0] == after[1] - base and later[0] == later[1] - base
          and later[1] >= after[1])
    fmt = lambda v: f'{v[0]}/{v[1]}' if v else '-'  # ticks/raw
    check(name, ok, f'bázis {base}, utána {fmt(after)}, később {fmt(later)}')

reset_probe('K4 reset: ticks = raw - bázis, monoton')
dut.read_until(re.escape(TICK_LINE), timeout=5.0)
reset_probe('K4 ismételt reset: ugyanígy')

if timing:                                      # T1: időzítési szerződés, külön
    ta, wa = status(dut), time.monotonic()
    time.sleep(2.0)
    tb, wb = status(dut), time.monotonic()
    rate = (tb - ta) / (wb - wa) if None not in (ta, tb) else 0.0
    check(f'T1 tickfrekvencia {TICK_HZ} Hz ±20% (host-órához mérve)',
          0.8 * TICK_HZ <= rate <= 1.2 * TICK_HZ, f'{rate:.1f} Hz')
else:
    skip('T1 tickfrekvencia', 'nem valós idejű futtató (--no-timing)')

dut.send('M')                                   # K5: ISR-törzsidő
_, m4 = dut.read_until(r'isr_body: ([^\r]*)\r\n')
body = m4.group(1) if m4 else None
if body is not None and body.startswith('SKIP'):
    skip('K5a ISR-törzsidő formátum', body)
    skip('K5b ISR-törzsidő platformsáv (P1)', body)
else:
    mm = re.fullmatch(r'n=(\d+) min=(\d+) max=(\d+) mean=(\d+)'
                      r'(?: median=(\d+))?(?: sd=(\d+))?', body or '')
    ok = bool(mm)
    if ok:
        n, mn, mx, mean = (int(x) for x in mm.groups()[:4])
        med = mm.group(5)                       # a 2. feladat opcionális mezője
        ok = (0 < n <= LAB_SAMPLES and mn <= mean <= mx
              and (med is None or mn <= int(med) <= mx))
    check('K5a ISR-törzsidő formátum: 0 < n <= 64, min <= mean <= max',
          ok, body or 'nincs válasz')
    if band is None:
```

```
                skip('K5b ISR-törzsidő platformsáv (P1)', 'nincs --isr-band megadva')
            else:
                check(f'K5b ISR-törzsidő platformsáv (P1) {band[0]}..{band[1]}',
                      ok and band[0] <= mn and mx <= band[1], body or '')
    finally:
        dut.close()

    for name, res, detail in results:
        print(f'{res:4}  {name}' + (f'  [{detail}]' if detail else ''))
    fails = sum(r == 'FAIL' for _, r, _ in results)
    print('SYSTEST:', 'FAIL' if fails else 'PASS')
    return 1 if fails else 0


if __name__ == '__main__':
    sys.exit(main(sys.argv[1:]))
```

```
make test                        # ringtest + systest
qemu-system-arm --version | head -1   # a jegyzőkönyvbe
sha256sum lab-m19.elf                 # a jegyzőkönyvbe
```

20.3.6 Várható kimenet és ellenőrzési feltétel

A két teszt kimenete. Az egységtesztet a host `gcc`-jével (Ubuntu 24.04, gcc 13.3) futtattuk. A rendszerteszt a fő úton, `make test`-tel, GNU `arm-none-eabi-gcc` 13.2.1 és QEMU 8.2.2 alatt futott (a lektor mérése); a T1 a host órájához mérve 100.0 Hz-cel PASS. A helyettesítő `cm-run.py` futtató csak megjegyzés, nem a mért alapút:

```
PASS  U1 alapeset: count = 3, sorrend ABC
PASS  U2 üres puffer: pop = 0, tail változatlan
PASS  U3 tele: pontosan RING_CAP-1 = 63 bájt fér el
PASS  U3 tele: overrun = 100 - 63 = 37
PASS  U3 tele: a megtartott bájtok az első 63, sorrendben (az újak vesznek el)
PASS  U4 körbefordulás: 5000 bájt kapacitáson belül, veszteség és csere nélkül
PASS  U5 határ: az N-1 = 63. elem még befér
PASS  U5 határ: az N. elem elutasítva, overrun = 1
PASS  U6 lassú fogyasztó: kivett = elfogadott, sorrendben
PASS  U6 lassú fogyasztó: overrun = elutasított push-ok
RINGTEST: PASS
```

```
PASS  K1 boot
PASS  K2 echo pontosan "ab"
PASS  F2 burst 50 heterogén bájt: pontos sorrend és hossz
PASS  K3 status formátum, overrun=0  [status: ticks=0 overrun=0 raw=0]
PASS  K6 periodikus tick sor (2 db)
PASS  K6 két tick sor alatt a számláló legalább 100-zal nőtt  [0 -> 200]
PASS  K4 reset: ticks = raw - bázis, monoton  [bázis 200, utána 0/200, később 0/200]
PASS  K4 ismételt reset: ugyanígy  [bázis 300, utána 0/300, később 0/300]
PASS  T1 tickfrekvencia 100 Hz ±20% (host-órához mérve)  [100.0 Hz]
SKIP  K5a ISR-törzsidő formátum  [SKIP (DWT CYCCNT nem halad)]
SKIP  K5b ISR-törzsidő platformsáv (P1)  [SKIP (DWT CYCCNT nem halad)]
SYSTEST: PASS
```

Ellenpróba: a teszt elkapja-e a hibát? A teszt saját ellenpróbájához egy hamis DUT-ot (a protokollt emuláló Python-programot) futtattunk a `--runner` helyén, szándékos hibákkal:

Hamis DUT hibája	Eredmény
a burst egy bájtját kétszer küldi vissza (51 bájt)	`FAIL  F2 … [51 bájt érkezett]`

Hamis DUT hibája	Eredmény
két szomszédos, eltérő bájtot felcserél	`FAIL F2 … [50 bájt érkezett]`
egy bájtot elhagy	`FAIL F2 … [49 bájt érkezett]`
200 Hz-es tick (kétszeres timerfrekvencia)	`FAIL T1 … [199.9 Hz]`
`isr_body: n=0`	`FAIL K5a …`
`isr_body: n=64 min=30 max=20 mean=25`	`FAIL K5a …`
sávon kívüli mérés (`max=90`, `--isr-band 15:40`)	`FAIL K5b …`
hibátlan, aszinkron `tick` sorokkal	minden funkcionális próba `PASS`

Ellenpélda: egy naiv teszt, amely 50 darab `x`-et küld és regexszel keres, az első két hibás DUT-ra hamis `PASS`-t adna, mert az azonos bájtok cseréjét nem látja, a nem horgonyzott regex pedig az 51. bájtot is eltűri. A firmware egy szándékosan elrontott változatában (az UART-ISR nem nyugtázza az `INTSTATUS.RXINT`-et) a megszakítás a visszatérés után azonnal újra kérődik, és a fő program nem jut processzorhoz. A rendszerteszt erre a `K1 boot` után minden sorban `FAIL`-t adott, `SYSTEST: FAIL` eredménnyel. Ez a futás kb. 2,5 percig tart (próbánként 10 s időtúllépés), ezért a látszólagos megállás nem lefagyás.

Az ellenőrzési feltétel a hallgató számára:

1. minden teszteset `PASS` legyen (F1); a K5 QEMU-n `SKIP`;
2. a kapacitáson belüli burst sorrendben és hiánytalanul visszaérkezzen (F2), a tele puffer viselkedését pedig az U3 igazolja (F3);
3. működő DWT esetén az ISR-törzsidő `min` és `max` értéke egy előre rögzített, platformparaméterként (`--isr-band LO:HI`) megadott sávban legyen (P1, K5b). Nem működő számlálónál a ciklusmérés eredménye `SKIP`; a nullákból képzett statisztika nem érvényes mérés;
4. a tickfrekvencia (T1) csak valós idejű futtatónál (QEMU valós idővel, illetve hardver) értékelhető; a funkcionális próbák eredménye ettől független.

Vázlat (nem futtatott). Az ISR-törzsidő-hisztogram a P1 kiértékelésének javasolt formája: a működő DWT-vel gyűjtött 64 minta eloszlása 1 ciklus felbontású sávokra bontva. Az alábbi táblázat a 4. feladat ellenpéldájának hisztogramja (32 minta 14 és 32 minta 20 cikluson); azt mutatja meg, hogyan kell a formátumot kitölteni, és hogy kétcsúcsú eloszlásnál a `mean` és a `median` is 17. Nem mérési eredmény.

A táblázatos forma (a forrás Markdownban is olvasható módon):

ISR-törzsidő [ciklus]	Minták száma
14	32
20	32
Össz	**64**

20.3.7 Reprodukálhatóság: ugyanaz a parancs, ugyanaz a bemenet, ugyanaz a sáv

A reprodukálhatóság e fejezetben **háromtényezős** szerződés. A jegyzőkönyvben rögzíteni kell:

1. **A pontos parancsot.** A `make test` önmagában nem elég, mert a `Makefile`-ban és a tesztben lévő QEMU-parancssor változhat. A jegyzőkönyvbe kerül a **teljes** hívási sor, vagy a `makefile` és a hívott bináris **verziója**.

2. **A pontos bemenetet.** A teszt-bájtok sorrendje és értéke, valamint a hívások sorrendje. Determinisztikus tesztnél ez fix; ha a bemenet generált, a generátor seedje is rögzül.
3. **Az eredmény-sávot.** Nem az eredmény bitre pontos értéke, hanem egy dokumentált tartomány: a funkcionális teszteknél `PASS`, a latenciánál `min..max` ciklussáv.

A reprodukálhatósághoz rögzítsük a QEMU-verziót, a machine-definíciót, a teljes parancssort, a firmware hash-ét és a beadott eseményeket. Ugyanazon verzió önmagában nem garantál determinisztikus időzítést; az UART-bemenet ütemezése és a virtuális óra beállítása is számít. A nem működő CYCCNT azonos nulla eredménye nem ismételhető ciklusmérés.

A reprodukálhatóság gyakorlati próbája: kétszer egymás után futtatjuk ugyanazt a parancsot (két külön QEMU-indítással), és a naplókat összehasonlítjuk. A naplóban a szerződés szerint változó mezők is vannak: a státuszsorok tickszámai, a reset előtti és utáni értékek, a T1 mért frekvenciája és a K5 számai a futtató ütemezésétől függnek, helyes működés mellett is. Ezeket az összevetés előtt normalizálni kell (például `sed -E 's/[0-9]+/N/g'` a `[…]` részletekre), és a funkcionális rész csak így, a próbák neve és `PASS`/`FAIL`/`SKIP` eredménye szerint egyezzen bitre. A latenciarésznek a **sávon belül** kell maradnia. Ha a második futás más `min`/`max`-ot ad, az gyanús: vagy a QEMU nem determinisztikus (pl. több-szálas végrehajtás, ha a máshol emlegetett konfiguráció így van), vagy a modulban van nem determinisztikus viselkedés (pl. `head` és `tail` verseny egy véletlen ütemezett olvasóval).

20.3.7.1 Ellenőrzési igény

A QEMU `mps2-an385` machine viselkedésének stabilitása verzióközi viszonyításban **itt nem ellenőrzött**. Ezért a jegyzőkönyvben a QEMU-verziót **rögzítettként** kell kezelni: a „reprodukálható” kijelentés csak ahhoz a verzióhoz szól, nem „QEMU általában”. A dokumentációs kiindulópont a QEMU System Emulation kézikönyv, de a konkrét machine-definíció stabilitása ellenőrzést igényel az adott verzióra.

20.3.8 Fizikai STM32 Nucleo (opcionális)

Fizikai Cortex-M3 panelhez a startupot, a linker scriptet, a perifériacímeket és az IRQ-kiosztást a konkrét MCU-hoz kell igazítani, majd külön binárist építeni. Cortex-M3-as Nucleo: NUCLEO-F103RB; a portoláshoz a 16.x linker/startup, a USART2 és a TIM2 kell; a systest a `--runner` kapcsolóval egy soros port (pl. `socat`) felé irányítható. A működő DWT számlálóval kapott eredmény valódi hardvermérés; a QEMU 8.2.2 AN385 esetében a ciklusmérés helyén `SKIP` szerepel. A fizikai értékeknél sem feltételezhető általánosan, hogy nagyobbak vagy kisebbek egy másik modell eredményénél.

A fizikai próba a perifériás működést és az időzítési követelmények teljesülését igazolhatja. A hiányzó emulált ciklusszámhoz nem számolunk gyorsulási vagy latenciaarányt; az emuláció funkcionális eredményeit külön hasonlítjuk össze.

20.3.9 Elfogadási feltétel kiértékelése

Az F1, F2, F3 és P1 kritériumokat a futás után értékeljük ki. Az értékelés a jegyzőkönyv záró szakasza:

Kritérium	Elvárt	Mért	Értékelés
F1	minden teszt `PASS`	(naplóból)	PASS/FAIL

Kritérium	Elvárt	Mért	Értékelés
F2	sorrend- és hosszegyezés a kapacitáson belül	(rendszerteszt, U1, U4)	PASS/FAIL
F3	tele puffer: az új bájt elveszik, `overrun` pontos	(U3)	PASS/FAIL
P1	ISR-törzsidő `min..max` az előre rögzített sávban	(`M` parancs)	PASS/FAIL, DWT nélkül SKIP

A PASS/FAIL nem „igen/nem” jellegű eldöntés a fejlesztő fejében, hanem **dokumentált**, előre rögzített küszöbökkel. Ha valamelyik kritérium nem teljesül, a fejezet vége nem a modul „kudarca”, hanem egy **hipotézis**: a modul vagy a mérési módszer hibás. A tipikus hibák szakasz ennek szétválasztásában segít.

20.3.10 Tipikus hibák és ellenpróbák

- **Tünet:** a funkcionális tesztek zöldek, de a második futás ISR-törzsideje szélesebb sávot ad (hardveren). **Hipotézis:** a mintatömb megtelt, és a minták más terhelési szakaszból származnak, vagy a mérés egy beágyazott megszakítást is tartalmaz. **Adatgyűjtés:** az `M` sor `n=` értéke, és a minták eloszlása. **Egy változtatás:** a mintavétel korlátozása egy rögzített terhelési szakaszra. **Ellenőrzés:** ismételt futás azonos sávot adjon.
- **Tünet:** az ISR-törzsidő „túl alacsony”, pl. 2 ciklus. **Hipotézis:** a `t0` és `t1` beolvasás között nem fut le a valódi ISR-törzs (a fordító átrendezte vagy eltávolította a köztes műveletet). **Adatgyűjtés:** disassemble (`arm-none-eabi-objdump -d`) az ISR-t, nézzük meg, hol van a `DWT_CYCCNT` olvasása. **Egy változtatás:** minden mérési pontnál `volatile` hozzáférést kényszerítünk. **Ellenőrzés:** disassembly visszaadja a kívánt sorrendet.
- **Tünet:** a funkcionális teszt során a `head == tail` ellenére a fogyasztó „olvas” (hamis adat). **Hipotézis:** a `volatile` hiányzik a `head` vagy `tail` deklarációjából, és a fordító cache-elte a régi értéket. **Adatgyűjtés:** `-O1` mellett generált kód az illető hozzáférésre. **Egy változtatás:** `volatile` mindkét indexre. **Ellenőrzés:** a teszt kimenete helyes lesz.
- **Tünet:** a rendszerteszt már a `K1 boot`-nál `FAIL`, pedig a firmware interaktívan működik. **Hipotézis:** a futtató nem a UART0-t köti a stdin/stdout-ra (hiányzik a `-serial stdio`, vagy a `-nographic` a monitort is ugyanoda multiplexeli). **Adatgyűjtés:** a futtató parancssora. **Egy változtatás:** `-display none -monitor none -serial stdio`. **Ellenőrzés:** megjelenik a boot-sor.
- **Tünet:** a rendszerteszt a burst-nél `FAIL`, de rövid bemenetnél `PASS`. **Hipotézis:** a burst meghaladja a puffer kapacitását, vagy a fogyasztó (az echo blokkoló TX-e) lassabb a bemenetnél. **Egy változtatás:** a burst a kapacitáson belül maradjon (F2), a tele puffert az egységteszt vizsgálja (F3).

20.3.11 Feladatok

1. (magyarázó, alapszint — cél: megértés-ellenőrzés) Magyarázd meg, miért nem elég egyetlen PASS a funkcionális tesztből, és miért van szükség a P1 latenciastabilitási kritériumra is. Adj példát egy olyan helyzetre, amikor a funkcionális teszt zöld, de a modul teljesítmény-szempontból nem elfogadható. Mutasd meg a saját példádban, melyik mérési pontok között kellene a latenciát definiálni.

2. (implementációs, középszint — cél: rutinépítés) Egészítsd ki a `lab-m19-main.c` `measure_line()` függvényét úgy, hogy a `min`, `max`, `mean` mellett kiszámítsa a **mediánt** és a **szórást** (populációs szórás az egzakt átlag körül, $\sigma = \sqrt{\frac{1}{n}\sum (v_i - \bar{v})^2}$, lefelé egészre kerekítve, egész aritmetikával), és a meglévő UART-sorban írd ki. A minták az `isr_body[]` tömbben vannak; az UART-ISR (`UART0_Handler`) minden lefutásakor egy minta kerül bele, amíg az `isr_body_n` el nem éri a `LAB_SAMPLES`-t (64), utána a gyűjtés megáll. Oldd meg azt is, hogy az `M` parancs a jelentés után **új mérési ablakot** indítson, úgy, hogy az `isr_body_n`-t ne írja két kontextus versenyben. Add meg a kimenet formátumát, és mutasd meg, hogy a rendszerteszt K5a/K5b próbája elfogadja.

3. (hibakeresési/mérési, középszint — cél: rutinépítés) Egy működő DWT-vel rendelkező futtatón a modul első futásán a mért `min..max` ciklussáv `18..26`, a másodikon `30..52`. Sorold fel, milyen lépéseket tennél annak eldöntésére, hogy a különbség a futtatókörnyezetből, a firmware-binárisból vagy a mérési módszerből (a bemeneti terhelésből és a mintavétel ablakából) ered. Konkrét parancsokat adj, és fogalmazd meg, mit rögzítenél a jegyzőkönyvben a döntés után.

4. (integráló, elmélyült — cél: fejezetek összekapcsolása: 12.5 mérési módszertana + 17.4 többcsúcsú latencia) Az 12.5 alfejezet a NEON-mérésnél szándékosan a **mediánt** használja, mert az kiugró értékekre robusztus. Az 17.4 megmutatta, hogy egy latenciaeloszlásnak több csúcsa is lehet, és hogy a csúcsok okát kontrollkísérlet választja szét. Vizsgáld meg: alkalmas-e a `measure_line()` `min`/`max`/`mean` statisztikája, illetve a 2. feladat `mean` és `median` értékének különbsége arra, hogy egy kétcsúcsú eloszlást felismerjen? Indokolj egy 64 elemű ellenpéldával, és add meg, mivel lehet a kétcsúcsúságot valóban kimutatni, illetve mi kell ahhoz, hogy egy csúcsot egy mechanizmushoz kössünk.

20.3.12 Megoldások

1. megoldás — Segítség. Gondolj egy ISR-re, amely MINDIG helyesen viselkedik (soha nem veszít adatot), de néha sokáig tart — a funkcionális teszt ezt a „néha sokáig” tulajdonságot tudja-e észlelni?

1. megoldás — Teljes megoldás. A funkcionális teszt egy tulajdonságot (sorrend, hossz, nem-elveszés) ellenőriz, nem mond semmit arról, mennyi ideig tart az ISR. Egy helyes, de lassú ISR átengedheti a tesztet, miközben egy valós, kis órajelű rendszerben a következő megszakítást már nem tudja fogadni. A P1 éppen ezt a dimenziót fedi le: a latencia stabilitását. Példa: 115 200 baudon (2170 ciklus/bájt 25 MHz-en) egy olyan ISR, amely általában 60 ciklus, de egy ritka ágon (pl. hibanaplózás) 3000, a következő bájtot overrunnal elveszíti — a funkcionális teszt ezt a ritka ágat nem járja be. A mérési pontok itt: az ISR első utasítása és a `head` írás befejezése.

2. megoldás — Segítség. A mediánhoz rendezett mintasor kell: az `isr_body[]` egy helyi másolatán végezhetsz egyszerű rendezést. Az `isr_body_n`-t az ISR írja; ha a fő program is nullázza, a kettőnek nem szabad versenyben futnia. Melyik eszköz zárja ki, hogy a másolás és a nullázás közben az ISR lefusson?

2. megoldás — Teljes megoldás. A `measure_line()` helyére (a többi függvény változatlan):

```
/* Kiegészítés a lab-m19-main.c-hez (2. feladat): a measure_line() helyett. */
static uint32_t isqrt32(uint32_t v)           /* egész négyzetgyök, lefelé kerekítve */
{
    uint32_t r = 0, bit = 1u << 30;
    while (bit > v) bit >>= 2;
    while (bit) {
        if (v >= r + bit) { v -= r + bit; r = (r >> 1) + bit; }
        else r >>= 1;
        bit >>= 2;
```

```
    }
    return r;
}

static uint32_t isqrt64(uint64_t v)           /* egész négyzetgyök, lefelé kerekítve, osztás
↪  nélkül */
{
    uint64_t r = 0, bit = 1ull << 62;
    while (bit > v) bit >>= 2;
    while (bit) {
        if (v >= r + bit) { v -= r + bit; r = (r >> 1) + bit; }
        else r >>= 1;
        bit >>= 2;
    }
    return (uint32_t)r;
}

static void measure_line(void)               /* K5, mediánnal és szórással */
{
    if (!lab_have_dwt) {
        uart_puts("isr_body: SKIP (DWT CYCCNT nem halad)\r\n");
        return;
    }
    uint32_t v[LAB_SAMPLES];
    uint32_t pm = crit_enter();              /* pillanatkép és új mérési ablak:      */
    uint32_t n = isr_body_n;                 /* az ISR ezalatt nem írhat              */
    for (uint32_t i = 0; i < n; i++)
        v[i] = isr_body[i];
    isr_body_n = 0;                          /* a következő M új ablakot mér          */
    crit_exit(pm);

    uart_puts("isr_body: n="); uart_put_u32(n);
    if (n) {
        for (uint32_t i = 1; i < n; i++) {   /* beszúró rendezés (n <= 64) */
            uint32_t x = v[i], j = i;
            for (; j > 0 && v[j - 1] > x; j--) v[j] = v[j - 1];
            v[j] = x;
        }
        /* floor(mean) a teljes uint32 tartományban, 64 bites osztás nélkül:
           sum(v) = n*sum(v/n) + sum(v%n); sum(v/n) <= max, sum(v%n) < n*n <= 4096 */
        uint32_t q = 0, r = 0;
        for (uint32_t i = 0; i < n; i++) {
            q += v[i] / n;
            r += v[i] % n;
        }
        uint32_t mean = q + r / n;
        /* páros n: a két középső elem egész átlaga lefelé kerekítve, túlcsordulás nélkül */
        uint32_t lo = v[(n - 1) / 2], hi = v[n / 2];
        uint32_t median = lo + (hi - lo) / 2u;
        uart_puts(" min=");    uart_put_u32(v[0]);
        uart_puts(" max=");    uart_put_u32(v[n - 1]);
        uart_puts(" mean=");   uart_put_u32(mean);
        uart_puts(" median="); uart_put_u32(median);
        /* sd= az EGZAKT átlag körüli POPULÁCIÓS szórás egész része (lefelé kerekítve):
           sd = floor(sqrt((n*S2 - S1^2) / n^2)) = floor(isqrt(n*S2 - S1^2) / n),
           ahol S1 = sum(w), S2 = sum(w^2), w = v - min. A szórás eltolásinvariáns, a w < 8192
           mellett S2 < 2^32 (n <= 64); a számláló 64 bites, de osztás csak 32 bites. Az
           eredmény nem a floor(mean) körüli eltérés; felette (max - min >= 8192) az sd=
↪  elmarad. */
        if (v[n - 1] - v[0] < 8192u) {
            uint32_t s1 = 0, s2 = 0;
            for (uint32_t i = 0; i < n; i++) {
                uint32_t w = v[i] - v[0];
```

```
                s1 += w;
                s2 += w * w;
            }
            uint64_t num = (uint64_t)n * s2 - (uint64_t)s1 * s1;
            uart_puts(" sd=");     uart_put_u32(isqrt64(num) / n);
        }
    }
    uart_puts("\r\n");
}
```

A minták átvétele és az `isr_body_n` nullázása **egy** kritikus szakaszban történik (`crit_enter`/`crit_exit`, a `PRIMASK` megőrzésével). Így az ISR nem írhat közben a tömbbe, és nem lesz két, versenyben lévő író; a következő `M` a nullázás óta érkezett, legfeljebb 64 mintát jelenti. A gyűjtés `LAB_SAMPLES`-nél megáll, ezért egy hosszú burst után a minták csak az első 64 UART-megszakításból származnak; az új ablakot ezért a mérendő terhelés **előtt** kell nyitni egy `M`-mel. A statisztika a **teljes `uint32_t` mintatartományban** helyes, és nem kell hozzá 64 bites osztás. Egy naiv `uint32_t sum` 64 darab `0x04000000`-s mintánál éppen 2^{32}-re érne, és nullára fordulna (`mean=0` a helyes 67 108 864 helyett). 64 darab `0x80000000`-s mintánál pedig az átlag és a páros medián `(v[31] + v[32]) / 2` alakja is 0 lenne, ami a K5a `min  mean` feltételét is megsértené. Ezért:

- **Átlag:** $\lfloor \sum v_i/n \rfloor = \sum \lfloor v_i/n \rfloor + \lfloor \sum (v_i \bmod n)/n \rfloor$. Az első összeg legfeljebb a `max`, a második kisebb, mint $n^2 \le 4096$, így egyik sem csordul túl. Az osztások 32 bitesek (`v[i] / n`, `v[i] % n`), ezeket a Cortex-M3 a hardveres `UDIV` utasítással végzi, könyvtári segédrutin nélkül. A széles, 64 bites összegzés is járható út lenne, de a 64 bites osztáshoz az `__aeabi_uldivmod` kellene. A projekt `Makefile`-ja ugyan linkeli a `-lgcc`-t, de ez a megoldás szándékosan nem függ tőle.
- **Páros medián:** a rendezett sor két középső eleméből `lo + (hi - lo) / 2`, amely `lo` `hi` miatt nem csordul túl, és a `lo` és `hi` közé esik.
- **Szórás:** a kimondott definíció a *populációs* szórás az **egzakt** átlag körül (nem a lefelé kerekített `mean` körül: az eltérés $[0,0,2]$ esetén 8/9 helyett 2/3 lenne a négyzetes átlag). Eltolásinvariáns, ezért $w_i = v_i - \min$ értékekkel számolunk: $\sigma = \lfloor \sqrt{nS_2 - S_1^2}\,/\,n \rfloor$, $S_1 = \sum w_i$, $S_2 = \sum w_i^2$ (a számláló 64 bites, az osztás 32 bites). A `max - min < 8192` feltétel mellett $S_2 < 2^{32}$ ($n \le 64$). Regressziós esetek (host-gcc-vel a 64 bites referenciához mérve): $[0,0,2] \to 0$, $[10,10,12] \to 0$ (a pontos érték 0,94), azonos minták $\to 0$, egy minta $\to 0$, páros és páratlan n véletlen esetekre egyezés az egzakt racionális referenciával. Ha a terjedelem ennél nagyobb, az `sd=` mező elmarad: a K5a ezt a mezőt nem követeli meg, a többi statisztika pedig ilyenkor is helyes.

Ebből következik, hogy `min  mean  max` és `min  median  max` mindig teljesül; egyforma mintánál mindhárom egyenlő. A kimenet: `isr_body: n=64 min=… max=… mean=… median=… sd=…`. Hoston (gcc 13.3, `tools/lab-m19/feladat2/`) a kontrollokra és néhány határesetre ezt adta. Utolsó sora 200 000 véletlen mintasort vet össze egy 64 bites referencia-átlaggal:

```
isr_body: n=64 min=14 max=14 mean=14 median=14 sd=0
isr_body: n=64 min=20 max=20 mean=20 median=20 sd=0
isr_body: n=64 min=14 max=20 mean=17 median=17 sd=3
isr_body: n=64 min=67108864 max=67108864 mean=67108864 median=67108864 sd=0
isr_body: n=64 min=2147483648 max=2147483648 mean=2147483648 median=2147483648 sd=0
isr_body: n=64 min=4294967295 max=4294967295 mean=4294967295 median=4294967295 sd=0
isr_body: n=64 min=4294967294 max=4294967295 mean=4294967294 median=4294967294 sd=0
isr_body: n=63 min=1 max=4294967295 mean=2113396605 median=1
isr_body: n=12 min=10 max=60 mean=16 median=10 sd=14
random mean-ellenőrzés: 0 eltérés / 200000
```

A rendszerteszt K5a próbája ezt is elfogadja, a mediánt is a `min..max` tartományban várja; a K5b ugyanúgy a `min`/`max`-ot veti össze a platformsávval. A kiegészítést a helyettesítő eszközlánccal

Thumb-2-re, `-nostdlib` mellett és `-lgcc` nélkül lefordítottuk és linkeltük (`-O0`, `-O1`, `-O2`). Új külső szimbólum nem keletkezett, az osztások `udiv` utasítások lettek, és a rendszerteszt `PASS`-szal futott vele (a K5 a DWT hiánya miatt `SKIP`). A statisztikai részt a fenti hostpróba ellenőrzi; a 4. megoldás értékei is ebből származnak.

3. megoldás — Segítség. Rangsorold a lehetséges okokat aszerint, melyiket a legolcsóbb ellenőrizni (verziószám lekérdezése) és melyiket a legdrágább (kód újraírása) — és haladj ez alapján.

3. megoldás — Teljes megoldás. A döntési fa, a legolcsóbb ellenőrzéssel kezdve. (a) A futtatókörnyezet: a futtató és verziója (`qemu-system-arm --version`, illetve a panel és a mag típusa), és hogy működik-e a DWT; ha bármelyik eltér a jegyzőkönyvben rögzítettől, a különbség innen is eredhet. (b) A bináris azonossága külön, explicit ellenőrzés: `sha256sum lab-m19.elf` a két futás előtt. Ha a hash eltér, az `arm-none-eabi-objdump -d lab-m19.elf` összevetése a `UART0_Handler` két `DWT_CYCCNT`-olvasása közötti utasításokon mutatja meg, mi változott a mért szakaszban (`readelf -h` ehhez nem elég: a fejléc azonos lehet eltérő kód mellett is). (c) Ha a futtató és a bináris is azonos, a mérési módszer a gyanús. Az `isr_body[]` az UART-ISR első 64 lefutását gyűjti a boot vagy az utolsó `M` óta, utána a gyűjtés megáll. Ha a két futás bemenete eltért (más bájtszám, burst vagy egyenként küldött bájtok, eltérő `overrun`), az ISR más útvonalai kerültek a mintába: például tele pufferben a `ring_push` elutasító ága. A minták egy része akár egy korábbi terhelésből is származhat, ha a mérés előtt nem nyitottunk új ablakot. Mintatömb-túlcsordulás nincs: az ISR `isr_body_n < LAB_SAMPLES` mellett ír. A jegyzőkönyvbe a futtatót és verzióját, a firmware `sha256sum`-ját, a pontos bemeneti sorozatot, az `n`-t és a mérési ablak kezdetét rögzítjük, a „sáv eltolódott” megfigyelést a hipotézissel együtt.

4. megoldás — Segítség. Számold ki a mean-t és a mediánt egy olyan 64 elemű sorra, amelynek fele 14, fele 20. Hol van a medián, ha páros mintaszámnál a két középső elem átlagát vesszük? És mi történik egy egycsúcsú, de ferde (néhány nagy kiugró értékkel rendelkező) sornál?

4. megoldás — Teljes megoldás. Egyik összesítő sem bimodalitásjelző. Ellenpélda: 32 darab 14 és 32 darab 20 ciklusos minta. Az eloszlásnak két különálló csúcsa van, mégis `mean = 17` és `median = (14 + 20) / 2 = 17`, a különbség **nulla**; a `min = 14`, `max = 20` pedig egy 14 és 20 közötti egyetlen csúcsnál is ugyanennyi lehetne. Fordítva: egy egycsúcsú, de ferde, 12 elemű sor (tíz darab 10–12 ciklusos érték és két kiugró érték: 30 és 60) `mean = 16`, `median = 10` eredményt ad, vagyis nagy különbséget, kétcsúcsúság nélkül. A 2. megoldás kódja hoston pontosan ezeket az értékeket adta. A `|mean - median|` tehát legfeljebb **aszimmetriára** figyelmeztető leíró jel. A kétcsúcsúságot a **hisztogram** mutatja meg (az `isr_body[]` értékeinek eloszlása ciklusonként, ahogy a P1 vázlata javasolja), vagy egy erre való eloszlásvizsgálat. Hogy egy csúcs mely mechanizmushoz tartozik (például láncolt belépéshez, eltérő kódúthoz vagy wait state-hez), azt egyik összesítő sem dönti el: ehhez az 17.4 3. megoldásának mintájára kontrollkísérlet kell, amelyben a feltételezett mechanizmus dokumentáltan jelen van, illetve hiányzik. Egy ferde, egycsúcsú eloszlás tehát nem bizonyítéka a tail-chainingnek. Az 12.5 azért használja a mediánt, mert az kiugró értékekre robusztus *helyzetmutató*, nem azért, mert a kétcsúcsúságot jelezné.

20.3.13 Rövid összefoglaló

Ez az alfejezet a modult mérhetővé és ellenőrizhetővé zárja. A funkcionális validáció két futtatható rétegből áll: a ring buffer hoston futó egységtesztjéből (alap, üres, tele, körbefordulás) és a teljes firmware soros bemenettel végzett rendszertesztjéből (K1–K6, F2). Az ISR-törzsidő-mérés a `DWT CYCCNT`-jével, a mért blokkon kívüli kiértékeléssel dolgozik, és nem azonos az eseménytől számított latenciával, a reprodukálhatóság pedig a háromtényezős szerződésen (parancs, bemenet, sáv) nyugszik. A működő számláló előfeltétel; QEMU 8.2.2 AN385 alatt a ciklusmérés `SKIP`, a fizikai host eredménye külön oszlop. Az elfogadási kritériumokat a futás előtt rögzítjük, nem

utólag. Az eltérések és nem-determinizmus nem hiba, hanem jelzés: hipotézis, adatgyűjtés, egy változtatás, ellenőrzés.

Továbbhasznált definíciók:

- **funkcionális validáció:** előre rögzített bemenet és elvárt tulajdonság (sorrend, hossz, nem-elveszés) összevetése; bináris eredmény.
- **mérési módszer:** a mintavétel helyének, gyakoriságának, a gyűjtött statisztikának és a mértékegységnek a rögzített eljárása.
- **reprodukálhatóság:** rögzített parancs, bemenet és eszközlánc-verzió mellett az eredmény egy dokumentált sávon belül marad.

20.3.14 Források és ellenőrzési hiányok

A linkek és a dokumentumverziók közös állapota az Olvasói útmutató forrásállapot-táblájában van; itt csak az alfejezet saját megállapításai szerepelnek.

- Armv7-M Architecture Reference Manual, DDI0403E — kivétel-belépés, prioritás, `PRIMASK`, DWT-regiszterek (`DEMCR`, `DWT_CTRL`, `DWT_CYCCNT`). A forrás állapota az Olvasói útmutató táblájában szerepel; a fejezetszám és a kiadásbetű az Arm-oldalak közös korlátja miatt nem ellenőrzött.
- Arm M-profile architektúra dokumentáció — M-profile viselkedés, DWT, vektorbejegyzés. A forrás állapota az Olvasói útmutató táblájában szerepel; a JS-renderelési korlát érvényes.
- QEMU System Emulation dokumentáció — `mps2-an385` machine, `-semihosting-config`, `-nographic`, `-kernel`. A forrás állapota az Olvasói útmutató táblájában szerepel; a hivatalos szöveg nem dokumentálja a cache- vagy időzítésmodellezést, ezért a fejezet QEMU-korlátra vonatkozó állításai a TCG ismert, funkcionális (nem ciluspontos) tervezéséből következnek.
- GNU Arm Embedded Toolchain (arm-none-eabi) dokumentáció — `arm-none-eabi-gcc`, `ld`, `objdump`, `readelf`. A forrás állapota az Olvasói útmutató táblájában szerepel.

Ellenőrzési hiányok, amelyek a fenti ellenőrzés után is ténylegesen fennmaradnak, és amelyeket a jegyzőkönyvben jelölni kell:

- A QEMU `mps2-an385` machine viselkedésének stabilitása verzióközi viszonyításban **nem ellenőrzött itt**; a „reprodukálható” kijelentés a jegyzőkönyvben rögzített verzióra érvényes.
- QEMU 8.2.2 AN385 alatt a `DWT_CYCCNT` nem halad; más verzió DWT-képessége és egy fizikai MCU időzítése külön ellenőrzést igényel.
- A közölt teszteredmények közül az egységteszt a host `gcc`-jével, a rendszerteszt a `tools/cm-run.py` emulátorral (QEMU helyett) és a helyettesítő eszközlánccal épített firmware-rel futott. A rendszerteszt ellenpróbáit egy hamis, a protokollt emuláló DUT-tal végeztük; a T1-et emiatt valós idejű futtatón csak ezzel próbáltuk ki, a firmware-en nem. A QEMU-s `make systest` és a GNU-eszközláncos build a laborban végzendő; a `-serial stdio` viselkedése az adott QEMU-verzióban ellenőrizendő.
- Fizikai STM32 Nucleo-n mért értékek nem részei e fejezet adatainak; az eltérés mértéke hardver- és órajelfüggő.

20.4 Auditszempontok bare-metal firmware-hez

20.4.1 Motiváció: a zöld teszt nem bizonyíték

A 20.3 lezárta a firmware-modult mérhető és reprodukálható formában: funkcionális validáció, latenciamérés, elfogadási kritériumok. Egy átadásra kész modul azonban nem attól lesz megbízható, hogy a tesztesetek átmentek rajta. A validáció azt igazolja, hogy **amit előre rögzítettünk**, az helyesen fut le. Az audit azt keresi, hogy **mi az, amit nem rögzítettünk**, és ami a valós üzem közben hibát okozhat. A kettő nem helyettesíti egymást: a validáció a helyes viselkedést igazolja, az audit a hibamódokat tárja fel.

Egy interrupt-vezérelt Cortex-M firmware tipikus hibamódjai nem determinisztikusak. Egy race condition csak akkor jelenik meg, ha az ISR pont a fő ciklus read-modify-write műveletének közepén fut le. A verem túlcsordulása csak akkor, ha a legmélyebb hívási lánc és a legtöbb beágyazott megszakítás egyszerre jelentkezik. Ezek a hibák a funkcionális teszt „zöld” eredménye mellett is ott maradnak. Az audit ezeket a rejtett feltételeket teszi láthatóvá, még azelőtt, hogy a termék kikerülne.

A mérnöki feladat ebben az alfejezetben az, hogy a modult ne csak működtessük, hanem **strukturáltan megvizsgáljuk**. Az audit nem egyetlen pillanatfelvétel, hanem ismételhető eljárás: checklist, bizonyíték, küszöb, döntés.

20.4.2 Definíció és hatókör: mit vizsgál az audit

Definíció (audit). A *szoftveraudit* egy független, strukturált, bizonyítékalapú vizsgálat (a validációtól módszerében különbözik: a validáció kiválasztott követelményeket és eseteket ellenőriz, az audit a szerződéseket, bizonyítékokat és lefedettségi hiányokat vizsgálja; **mindkettő találhat funkcionális hibát**, és egy korábbi zöld teszt a funkcionális kérdéseket nem zárja le), amelynek célja annak megállapítása, hogy egy rendszer megfelel-e előre rögzített követelményeknek, és hogy milyen hibamódok maradtak feltáratlanul. A beágyazott firmware auditja kiemelten kezeli a megszakításkezelőket, a megosztott állapotot, a veremhasználatot és az erőforráskorlátokat.

Az audit hatóköre szűkebb, mint a teljes rendszer tesztelése. Nem vizsgálja újra a funkcionális helyességet (azt a 20.3 validációja lefedte), hanem a **nem funkcionális** tulajdonságokra fókuszál:

- **determinizmus**: az ISR futásideje adatfüggetlen, korlátos;
- **versenyhelyzet-mentesség**: a megosztott állapot minden hozzáférése védett vagy atomi;
- **erőforráskorlát**: a verem, a CPU-idő és a memória a tervezett kereten belül marad;
- **hibatűrés**: a rendszer egyetlen pont meghibásodása esetén is dokumentált állapotba kerül.

Az audit nem azonos a kódreview-val, bár annak elemeit használja. Az auditnak **bizonyítékra** van szüksége: mérési adatra, disassembly-re, vagy olyan kódrészletre, amely a hibamódot reprodukálhatóan mutatja. A „szemrevételezés alapján jónak tűnik” nem auditmegállapítás.

20.4.3 Az auditchecklist

A strukturált audit első eszköze a checklist. Az alábbi táblázat a jelen alfejezet hatókörébe tartozó kategóriákat, a hozzájuk tartozó kérdéseket és a szükséges bizonyítékokat rögzíti. A checklist nem helyettesíti a mérnöki ítéletet, de biztosítja, hogy egyetlen kategória se maradjon ki.

Kategória	Auditkérdés	Bizonyíték	Küszöb / döntés
ISR-rövidség	Az ISR worst-case futásideje korlátos és dokumentált?	ciklusszám-mérés, disassembly	$t_{ISR} < 0{,}1 \cdot T_{per}$
Determinizmus	Minden ciklusnak van ismert maximális iterációszáma? Van-e korlátlan várakozás?	forráskód, disassembly	minden ciklus korlátos (dokumentált max.), nincs korlátlan `while`, `delay`, allokáció
Race condition	Minden megosztott változó atomi vagy védett?	forráskód, megszakítás-tiltás	nincs védtelen read-modify-write
Verem-mélység	A worst-case stack-használat a linkeren beállított keret alatt marad?	mérés, statikus elemzés	$S_{wc} < S_{limit} - S_{margin}$
Erőforráskorlát	Az ISR CPU-terhelése a periódus alatt korlátos?	ciklusszám, periódus	$U_{ISR} < 0{,}5$
Watchdog	A refresh a fő ciklusban, dokumentált ponton történik?	forráskód, konfiguráció	nincs refresh ISR-ben

A küszöbértékek nem univerzálisak; a konkrét rendszer időzítési követelményeihez kell igazítani őket. Az audit során a küszöböt a mérés **előtt** rögzítjük, ugyanúgy, ahogy az elfogadási kritériumokat a 20.3 validációjában.

20.4.4 ISR-rövidség és determinisztikus futásidő

Az ISR-nek nem kell feltétlenül *gyorsnak* lennie, hanem **determinisztikusan gyorsnak**. Az a megszakításkezelő, amelynek a futásideje a bemenettől függően széles sávban változik, kiszámíthatatlanul késlelteti a többi megszakítást és a fő ciklust. Az audit első kérdése ezért nem az, hogy „milyen gyors”, hanem az, hogy „mennyire korlátos és kiszámítható”.

Az ISR-rövidség ellenőrzésének lépései:

1. **A hívási lánc feltérképezése.** Az ISR-ből hívott összes függvényt fel kell sorolni. Minden hívás stack-használatot és időt jelent.
2. **A ciklusok azonosítása.** Minden `for`, `while`, `do` ciklusnál meg kell adni a **maximális iterációszámot**. A bemenetfüggő ciklus nem eleve korlátlan: egy legfeljebb 64 bájtos puffert feldolgozó ciklus futásideje adatfüggő, de korlátos. Korlátlan az a ciklus, amelynek nincs ismert felső határa, például egy hardverbitre korlát nélkül váró `while`.
3. **A várakozások kiszűrése.** `delay`, `sleep`, buszos várakozás vagy allokáció nem megengedett ISR-kontextusban.
4. **A WCET becslése és a küszöb ellenőrzése.** A worst-case execution time ($WCET$) az útvonalakból adódik: minden lehetséges végrehajtási útvonalra a ciklusok maximális iterációszáma szorozva az iterációnkénti költséggel, plusz az útvonal egyenes szakaszai; az ISR WCET-je ezek maximuma, $C = \max_{\text{út}}(\sum_j n_j^{\max} c_j + c_{\text{egyenes}})$. A mérés ezt nem helyettesíti, csak alátámasztja: egy mért maximum csak a ténylegesen bejárt útvonalakról szól. Végül a WCET és a periódus arányát vetjük össze a küszöbbel.

A CPU-terhelés levezetése: ha az i-edik ISR fenti, ciklusban kifejezett WCET-je C_i, a periódusa T_i (másodperc), az órajel f_{cpu}, akkor az összes ISR együttes terhelése

$$U_{ISR} = \sum_{i=1}^{n} \frac{C_i}{T_i \cdot f_{\text{cpu}}}.$$

Ha a WCET időben adott (C_i/f_{cpu} másodperc), az f_{cpu} szorzó elmarad; a lényeg, hogy a számláló és a nevező azonos egységű legyen.

Ha U_{ISR} megközelíti az 1-et, a fő ciklusra gyakorlatilag nem marad CPU-idő. Az $U_{ISR} < 0{,}5$ ebben a könyvben egy **50%-os projektbudget**, nem általános biztonsági vagy ütemezhetőségi garancia: kis átlagos U mellett is lehet túl hosszú megszakítás-tiltás vagy ISR, amely határidőt sért. Ezért két külön auditpont kell: (1) az összegzett kihasználtság U_{ISR}, és (2) a legrosszabb válaszidő (worst-case response time): a legnagyobb maszkolási idő + belépés + a magasabb prioritású ISR-ek terhelése + a saját WCET határidő. Ellenpélda: egy 1 kHz-es ISR $C = 1000$ ciklus mellett $U = 0{,}04$ (25 MHz-en), de egy 5 ms-os `cpsid i` szakasz egy 1 ms-es határidőt sért — az audit ezt $U < 0{,}5$ esetén is elutasítja. A képlet levezetés, nem mérés: feltételezi, hogy az ISR-ek egymást nem szakítják meg tetszőlegesen, és hogy a periódusok pontosan ismertek.

A QEMU `mps2-an385` emulációjában a ciklusszám relatív/illusztratív, ahogyan a 20.3 is rögzítette. Az auditmegállapítás ilyenkor nem abszolút ciklusszám, hanem a **szerkezeti tulajdonság**: van-e korlátlan ciklus, van-e várakozás.

20.4.5 Race condition és megosztott állapot

A *race condition* (versenyhelyzet) akkor áll elő, ha két végrehajtási kontextus (itt: ISR és fő ciklus) ugyanazt a megosztott állapotot nem atomi módon éri el, és a műveletek sorrendje megváltoztatja az eredményt. A beágyazott firmware-ben a leggyakoribb minta a **read-modify-write**:

```
/* Hibás: a fő ciklus és az ISR ugyanazt a számlálót módosítja */
volatile uint32_t overrun_count;

void UART0_IRQHandler(void) {
    overrun_count++;          /* ISR: load, add, store */
    UART0_INTSTATUS = UART0_RXINT;   /* W1C, bit1 */
}

uint32_t uart_get_overruns(void) {
    uint32_t v = overrun_count;   /* (C) olvasás */
    overrun_count = 0u;           /* (D) törlés - RACE a (C) és (D) között */
    return v;
}
```

A hiba ott van, hogy a fő program (C) olvasása és (D) írása közé az ISR beékelődhet, és növelheti a számlálót. A növelés elveszik, mert a fő ciklus a (D) lépésben felülírja. Ez a klasszikus *check-then-act* hiba.

A `volatile` kulcsszó **nem** nyújt védelmet. A `volatile` csak azt garantálja, hogy a fordító nem optimalizálja el az olvasást/írást, és hogy a művelet a programozott sorrendben történik. Nem garantál atomitást, és nem hoz létre kritikus szakaszt.

A javítási lehetőségek:

1. **Kritikus szakasz megszakítás-tiltással.** A `lab-m19-crit.h` `crit_enter`/`crit_exit` függvényeivel (CMSIS-környezetben ezeknek a `__get_PRIMASK` / `__disable_irq` / `__set_PRIMASK` felel meg, natív assembly-ben `MRS`/`MSR`) a read-modify-write egy megszakíthatatlan szakaszba kerül.
2. **Atomi csere (exchange).** Cortex-M3-on a 32 bites C11 atomikus műveletek lock-free módon `LDREX`/`STREX`-re fordulnak (ellenőrizhető: `objdump`). A kivétel be- és kilépése törli az exkluzív monitort, így egy közbeékelődő ISR a `STREX`-et sikertelenné teszi, és a ciklus újrapróbál.
3. **Az ISR oldaláról elkerülni a módosítást.** Ha az ISR csak egy *flag*-et állít, a race megszűnik, de a flag csak a „legalább egy esemény történt" szerződést viszi át (és az átvétele — olvasás és törlés — maga is atomi műveletet kíván, pl. kritikus szakaszt vagy `LDREX`/`STREX`-et). Pontos darabszámhoz a védett számlálót, az ISR által egyedül írt monoton számlálót és a fő ciklus differenciáját (`g_tick - seen`), vagy eseménysort kell használni. Ha két esemény érkezik egy főciklus-leolvasás előtt, a számláló szerződés 2-t ad, a flag csak „volt esemény"-t — a flages megoldás ne állítsa, hogy megőrizte a darabszámot (lásd 17.3).

Az audit nem azt kéri számon, hogy melyik megoldást választottuk, hanem azt, hogy **minden megosztott hozzáférésre legyen tudatos döntés**. A `volatile` önmagában nem döntés.

20.4.6 Verem-mélység megszakítás-beágyazás mellett

A veremhasználat auditja szorosan kapcsolódik a rekurzióhoz és a stack frame felépítéséhez, amelyet a 8.4 és a 8.1 tárgyalt. A Cortex-M esetében a hardver maga is push-ol regisztereket a veremre, amikor egy kivételbe belép. Az Armv7-M alapértelmezett kivétel-belépési kerete 8 szót (32 byte) tesz a veremre: `xPSR`, `PC`, `LR`, `R12`, `R3`, `R2`, `R1`, `R0`, az új SP-től mérve R0-val a `+0x00` offseten. Ha a belépés előtti SP nem volt 8 bájtra igazított, egy további igazítási szó (4 byte) is elfogy; ez a keret fölött, a magasabb címen van, a regiszteroffseteket nem tolja el. A Cortex-M3 baseline-on ez a teljes hardveres keret.

> **Más magokon (nem a baseline része).** FPU-s magokon (például Cortex-M4F, M7), bekapcsolt FPU és lusta mentés esetén a hardver kiterjesztett keretet is menthet: további 18 szót (`S0`–`S15`, `FPSCR` és egy fenntartott szó), azaz 72 byte-ot. Ez egy másik profil tulajdonsága, és csak külön profilváltással, a konkrét mag dokumentációja szerint kerül elő; a Cortex-M3 baseline veremauditjának nem bizonytalansága.

A worst-case veremmélység levezetése:

$$S_{wc} = S_{main} + \sum_{i=1}^{k} (F_{hw} + P_i + F_{sw,i} + S_{call,i}),$$

ahol S_{main} a fő program legmélyebb, megszakítható hívási láncának veremigénye, k a prioritások szerint egymásba ágyazható ISR-ek száma, $F_{hw} = 32$ byte a hardveres keret, $P_i \in \{0, 4\}$ byte az opcionális igazítási szó, $F_{sw,i}$ az i-edik ISR prológusának szoftveres mentése, $S_{call,i}$ pedig az i-edik ISR-ből induló legmélyebb hívási lánc. Minden szintnél a saját legmélyebb láncát kell venni, nem csak a legfelsőnél, mert bármelyik szint megszakítható a lánca legmélyebb pontján.

A beágyazás lényege: ha egy alacsonyabb prioritású ISR fut, és egy magasabb prioritású megszakítás érkezik, az hardveresen beágyazódik. A verem tetején egyszerre több frame lehet. Az audit során a **legrosszabb kombinációt** kell keresni: a legmélyebb hívási láncot a legtöbb beágyazott megszakítással kombinálva.

A hardveres keret mérete és elrendezése architekturális szabály, ezért a QEMU `mps2-an385` és a fizikai Cortex-M3 között ebben nem lehet eltérés; ami eltérhet, az az időzítés (mikor érkezik a beágyazó megszakítás), ezt a 17.4 már jelölte. Az audit szempontjából a lényeg, hogy a tervezett S_{limit} értékhez **biztonsági margót** rendeljünk. Egy 25–30%-os tartalék a linkeren beállított méret és a levezetett vagy mért csúcs között hasznos **tervezési döntés**, de **nem bizonyíték**: a margó semmit nem igazol arról az útvonalról, amelyet sem a levezetés, sem a mérés nem fedett le. A bizonyíték a fenti összeg, minden szintre a dokumentált legmélyebb lánccal, és a mérés (verem-vízjel) mint ellenpróba.

Az alábbi ábra illusztratív, számított példát mutat a veremmélység alakulására különböző konfigurációkban.

A hozzá tartozó numerikus táblázat:

#	Konfiguráció	Stack-használat [byte]
1	main, megszakítás nélkül	64
2	main + 1 ISR	108
3	main + 2 beágyazott ISR	152
4	main + 2 beágyazott ISR + 64 byte-os hívási lánc az ISR-ből	216

A 2–4. sor ISR-enként 44 byte-tal számol: $F_{hw} = 32 + P = 4$ (a legrosszabb eset) + $F_{sw} = 8$, a 4. sorban pedig ehhez jön egy 64 byte-os hívási lánc (S_{call}). Az ábra és a táblázat értékei nem mérési eredmények, hanem a fenti képlet alapján számított illusztrációk. Az audit során a számított értéket mérési adattal kell összevetni; ha a mért érték megközelíti a linkeren beállított stack-méretet, a margó nem elegendő. A lab-m19 tényleges számított és mért (verem-vízjel) csúcsa ebben a kiadásban nem szerepel: a veremfestéses mérés a fő úton (`arm-none-eabi-gcc -fstack-usage`, illetve kitöltött veremterület és a legalacsonyabb érintett cím kiolvasása) elvégezhető, de itt nem történt meg, ezért a kategória állapota **ellenőrizendő**.

20.4.7 Watchdog mint robusztussági réteg

A *watchdog timer* (őrző időzítő) egy hardveres vagy szoftveres mechanizmus, amely akkor állítja vissza a rendszert, ha a firmware egy meghatározott időn belül nem jelzi, hogy a fő ciklus előrehalad. A watchdog nem javítja a hibát; célja, hogy egy beragadt vagy váratlanul lelassult rendszert ismert állapotba kényszerítsen, és ezzel a rendszer elérhetőségét növelje.

Az elnevezések platformfüggők (a példák STM32-szerűek: IWDG/WWDG; más Cortex-M-en más a periféria neve és konfigurációja, nem minden MCU-n létezik mindkettő). Általános elv: a refresh a főciklusban történik, hogy egy továbbfutó ISR ne fedhesse el a beragadt `main`-t; teszt: beragadt főciklus + tovább futó ISR → reset.

Két alapvető típus terjedt el a Cortex-M világában:

- **Független watchdog (pl. STM32 IWDG).** Saját, a rendszerórajeltől független órajel-forrással (általában belső RC oszcillátor) működik. Akkor is lejár, ha a fő órajel leáll vagy a PLL elveszíti a zárat. Ez a robusztussági réteg legfontosabb tulajdonsága.
- **Ablakos watchdog (pl. STM32 WWDG).** Csak egy meghatározott időablakban engedélyezi a refresh-t („kick”). Ha a refresh túl korán történik, az is resetet okoz. Ez a „runaway loop” ellen véd, amikor a fő ciklus hibásan, de gyorsan pörög.

A watchdog-koncepció audit szempontjából kritikus kérdései:

1. **Hol történik a refresh?** A refresh-nek a fő ciklus **dokumentált, utolsó** pontján kell történnie. Ha az ISR-ben refresh-elünk, a watchdog elveszíti a védőhatását: a fő ciklus beragadhat, miközben az ISR továbbra is „életben tartja” a rendszert.
2. **Milyen periódusban?** A refresh-periódusnak rövidebbnek kell lennie a watchdog időtúllépésénél, de elég hosszúnak ahhoz, hogy a normál végrehajtás beleférjen. A margó itt is számít.
3. **Mi történik reset után?** A watchdog-reset nem hibaüzenet. A firmware-nek a reset után naplóznia kell a watchdog-eseményt (ha van perzisztens tároló), különben a hiba láthatatlan marad.
4. **A watchdog nem helyettesíti a helyes kódot.** Ha a race condition vagy a veremtúlcsordulás egyébként is fennáll, a watchdog csak a tünetet kezeli.

A watchdog fogalmi szinten a robusztussági rétegek egyike. A konkrét regiszterek és a pontos konfiguráció MCU-specifikusak; ezek ellenőrzése külön feladat, és nem része ennek az alfejezetnek.

Fő úti kiegészítés: CMSDK APB watchdog. A QEMU 8.2.2 `mps2-an385 info mtree` kimenete szerint (a lektor mérése) a `0x40008000` címen `cmsdk-apb-watchdog` periféria van (nem `unimplemented-device`), ezért a watchdog a fő úton kipróbálható. Regiszterei: `WDOGLOAD 0x00`, `WDOGCONTROL 0x08` (`INTEN/RESEN`), `WDOGINTCLR 0x0C`, `WDOGLOCK 0xC00` (a `0x1ACCE551` értékkel nyitható). A refresh a fő ciklus végén `WDOGINTCLR = 1`; a „beragadt főciklus + tovább futó ISR” ellenpróba a watchdog-resetet mutatja. Az STM32-s `IWDG/WWDG` nevek csak analógiák; a watchdog kimenetének NMI-re/resetre kötését a QEMU-modellen ellenőrizni kell.

20.4.8 Kidolgozott példa: a UART-modul auditja

Munkakönyvtár: `~/arm-labor/cm/zaro2/` — az alfejezet minden laborfájlja ide kerül, és minden parancs innen futtatandó (a könyvtárrendet a Labor-alapkörnyezet alfejezet írja le). Ez a II. záróprojekt közös könyvtára: mind a négy alfejezete (20.1–20.4) itt dolgozik, és egymás fájljaira épít.

```
mkdir -p ~/arm-labor/cm/zaro2
cd ~/arm-labor/cm/zaro2
```

Legyen az auditálandó objektum a `lab-fw-uart-audit.c` modul, amely a 20.2 implementációjának egyszerűsített változata. A modul egy UART vételt kezel megszakítással, és egy körpuffert tölt.

```
/* lab-fw-uart-audit.c
 * Szándékosan auditálandó UART vétel modul.
 * Cél: race condition és ISR-viselkedés vizsgálata.
 * Memóriacím: MPS2+ AN385 CMSDK UART0, dokumentált QEMU-tartomány.
 * Ellenőrizendő a pontos machine memory map alapján.
 */
#include <stdint.h>

#define RX_SIZE   64u
#define RX_MASK   (RX_SIZE - 1u)

#define UART0_BASE      0x40004000u
#define UART0_DATA      (*(volatile uint32_t *)(UART0_BASE + 0x00u))
#define UART0_STATE     (*(volatile uint32_t *)(UART0_BASE + 0x04u))
#define UART0_CTRL      (*(volatile uint32_t *)(UART0_BASE + 0x08u))
/* Olvasás: TXINT (bit0)/RXINT (bit1) pending; írás: 1 a megfelelő bitre törli (write-1-to-
 * clear). */
#define UART0_INTSTATUS (*(volatile uint32_t *)(UART0_BASE + 0x0Cu))
```

```c
#define UART0_RXINT     (1u << 1)
#define UART0_BAUDDIV   (*(volatile uint32_t *)(UART0_BASE + 0x10u))

static volatile uint8_t  rx_buf[RX_SIZE];
static volatile uint32_t rx_head;
static volatile uint32_t rx_tail;
static volatile uint32_t rx_overrun;

void UART0_IRQHandler(void)
{
    UART0_INTSTATUS = UART0_RXINT;      /* nyugtázás a DATA olvasása előtt */
    uint8_t  d    = (uint8_t)UART0_DATA;
    uint32_t next = (rx_head + 1u) & RX_MASK;

    if (next == (rx_tail & RX_MASK)) {
        rx_overrun++;                   /* (A) csak ISR írja - önmagában OK */
    } else {
        rx_buf[rx_head & RX_MASK] = d;
        rx_head = next;                 /* (B) egyetlen író - OK */
    }
}

/* Fő ciklusból hívva. */
uint32_t uart_get_overruns(void)
{
    uint32_t v = rx_overrun;            /* (C) olvasás */
    rx_overrun = 0u;                    /* (D) törlés - RACE (C) és (D) között */
    return v;
}
```

Az audit megállapításai:

1. **Race condition a `uart_get_overruns`-ban.** A (C) és (D) között az ISR futhat, és növelheti a `rx_overrun` értékét. A növelés elveszik. **Súlyosság: közepes.** A hiba ritkán és nem determinisztikusan jelentkezik, de a hibaszámláló pontosságát tönkreteszi.
2. **`volatile` használat.** A változók `volatile`-ok, ami szükséges, de nem elégséges. Az audit rögzíti, hogy a `volatile` nem ad atomitást.
3. **ISR-rövidség.** Az ISR nem hív függvényt, és nincs benne ciklus, de **nem egyetlen útvonalú**: legalább két ág van (teli puffer → `rx_overrun++`, illetve üres hely → beírás), és az adatfüggetlen futásidő nincs bizonyítva. A vizsgálatnak mindkét ágat le kell fednie (üres és teli pufferes eset), a fordított kód alapján ágankénti utasításszámmal és a legrosszabb ág korlátjával; amíg ez nincs meg, a determinizmus állapota **ellenőrizendő**, a checklist nem adhat konstansidejű PASS-t pusztán a ciklus és a függvényhívás hiánya alapján.
4. **Verem-használat.** Az ISR lokális változói (`d`, `next`) regiszterekbe kerülhetnek; a hardveres keret 32 byte, igazítási szóval 36 byte. Az ISR nem hív függvényt, így $S_{call} = 0$, a szoftveres mentést a disassembly prológusa adja. **Ellenőrizendő**: a 32 (36) byte csak az ISR saját kerete; a teljes csúcs a fő program legmélyebb láncával együtt, valamint a mért vízjel még nincs megadva.
5. **Watchdog.** A modulban nincs watchdog-kezelés. Ez nem hiba önmagában, de a rendszerszintű audit során a watchdog hiánya kockázatot jelent.

A javított `uart_get_overruns` kritikus szakasszal:

```c
/* Javított változat: kritikus szakasz a lab-m19-crit.h-ból.
 * CMSIS-környezetben __get_PRIMASK / __disable_irq / __set_PRIMASK
 * felel meg neki; natív assembly-ben MRS/MSR utasítások.
 */
uint32_t uart_get_overruns(void)
{
```

```
    uint32_t pm = crit_enter();
    uint32_t v  = rx_overrun;
    rx_overrun = 0u;
    crit_exit(pm);
    return v;
}
```

A kritikus szakasz hossza néhány ciklus, ami nem növeli érdemben az ISR-latenciát. Az audit szempontjából a döntés indokolt: a `rx_overrun` frissítése ritka, a kritikus szakasz rövid, a megszakítás-tiltás elfogadható.

A modul auditjának összefoglalása:

Kategória	Állapot	Megjegyzés
ISR-rövidség	ellenőrizendő	nincs ciklus, nincs hívás, de két ág; a legrosszabb ág utasításszáma a disassemblyből még nincs megadva
Determinizmus	ellenőrizendő	két ág; a legrosszabb ág ciklusszáma (a fordított kódból) még nincs megadva
Race condition	**javítva**	bizonyíték: a `uart_get_overruns` kritikus szakaszt kapott (a javított változat fent), a kényszerített interleaving-teszt nem veszít növekményt
Verem-mélység	ellenőrizendő	32 (36) byte hardveres keret + prológus; $S_{call} = 0$; a mért vízjel a fő programmal együtt még nincs megadva
Erőforráskorlát	részben (becslés)	U_{ISR} alacsony *feltételezett* ciklusszámmal; nincs számszerű erőforrás- és latenciabizonyíték, a legrosszabb válaszidő külön auditpont
Watchdog	hiányzik	rendszer-szintű döntés szükséges

Minősítési szabály. „Megfelel” csak akkor áll, ha mellette van a bizonyíték típusa és értéke: számszerű erőforráskorlát (ciklus/periódus, a használt sávszélesség), a legrosszabb latenciára adott korlát vagy mérés (nem a QEMU abszolút ciklusa), a verem legrosszabb esete. Ahol ez hiányzik, a státusz „ellenőrizendő” vagy „részben (becslés)”, és a táblázat a hiányt nevezi meg.

20.4.9 Tipikus hibák és ellenpróbák

Hiba: a `volatile` elégségesnek tekintése. Tünet: a race condition spontán ritkán jelentkezik, de **célzottan reprodukálható**: a `uart_get_overruns` konkrét interleavingje: (1) a fő program beolvassa a számlálót (`v = rx_overrun`), (2) az ISR megnöveli (`rx_overrun++`), (3) a fő program

nullázza (`rx_overrun = 0`) — az ISR növekménye elveszett. Tesztben ezt a három lépést a futtatóból kényszerítjük (töréspont a (C) és (D) között, majd az IRQ kézi pending-be állítása): a régi változat minden kényszerített interleavingnél elveszíti a növekményt, a kritikus szakaszos megőrzi. A teszt tehát nem valószínűségi. Hipotézis: a fordító vagy a hardver sorrendje nem garantált. Adatgyűjtés: a disassembly megmutatja, hogy a read-modify-write három külön utasítás. Egy változtatás: kritikus szakasz bevezetése. Ellenőrzés: a hibaszámláló pontos lesz.

Hiba: a watchdog refresh az ISR-ben. Tünet: a fő ciklus beragad, de a rendszer nem indul újra. Hipotézis: az ISR tartja életben a watchdogot. Adatgyűjtés: a refresh-hívás helyének azonosítása. Egy változtatás: a refresh áthelyezése a fő ciklus végére. Ellenőrzés: a fő ciklus szándékos blokkolása után a watchdog resetel.

Hiba: a verem-margó elhanyagolása. Tünet: ritka HardFault, amely a megszakítás-beágyazás bizonyos kombinációiban jelentkezik. Hipotézis: a worst-case stack-használat meghaladja a linkeren beállított méretet. Adatgyűjtés: a stack pointer mérése a legmélyebb ponton. Egy változtatás: a stack-méret növelése vagy a hívási lánc mélységének csökkentése. Ellenőrzés: a mérés megismétlése.

Hiba: az ISR-ben allokálás vagy `printf`. Tünet: hosszú, kiszámíthatatlan ISR-latencia. Hipotézis: a `printf` vagy `malloc` nem determinisztikus. Adatgyűjtés: a hívási lánc és a ciklusszám mérése. Egy változtatás: a naplózás áthelyezése a fő ciklusba. Ellenőrzés: az ISR WCET-je korlátos lesz.

20.4.10 A két rész együtt: rendszerprogramozási szemlélet

A könyv két része ugyanazt a gondolkodásmódot tanítja két különböző futási modellben. Az I. rész az AArch64 alkalmazásszintű programozását mutatta be: regiszterkészlet, AAPCS64, memóriacímzés, NEON, teljesítmény. A II. rész a Cortex-M bare-metal világát: vektortábla, NVIC, memory-mapped perifériák, interrupt-vezérelt I/O.

A közös alap mindkét részben ugyanaz: az ISA-szintű adatreprezentáció, a verem működése, a hívási konvenció és a hibakeresési módszer. Az 8.4 rekurzióról szóló elemzése ugyanúgy érvényes a Cortex-M verem-auditjára, mint az AArch64 stack frame-jére. Az 14.1 és 14.2 biztonsági mechanizmusai (canary, NX, PAC) a bare-metal világban részben másképp jelennek meg, de a mögöttes elv – a visszaélhető állapot korlátozása – azonos.

A különbség a **privilegizált állapot** és a **hardver közvetlen elérése**. Az AArch64 Linux alatt a rendszermag kezeli a megszakításokat és a perifériákat; a felhasználói program syscall-okon keresztül ér el. A Cortex-M bare-metalon a firmware maga a rendszermag: közvetlenül írja a perifériaregisztereket, és maga konfigurálja az NVIC-et. Az audit ezért a bare-metal világban szélesebb: nem csak a program logikáját, hanem a hardverkonfigurációt is vizsgálja.

Egy valós beágyazott rendszerben a két világ gyakran együtt jelenik meg: egy Cortex-A alkalmazásprocesszor futtatja a Linuxot és a magas szintű logikát, míg egy Cortex-M mikrokontroller végzi a valós idejű, interrupt-vezérelt feladatokat. Az auditmódszer mindkét oldalon ugyanaz: checklist, bizonyíték, küszöb, döntés.

20.4.11 Feladatok

1. (magyarázó, alapszint — cél: a validáció és az audit határának megértése) Ismertesd, mi a különbség a funkcionális validáció és az audit között. Sorolj fel legalább három hibamódot, amelyet a 20.3 validációja önmagában nem feltétlenül fed fel, és indokold, miért.

2. (implementációs, középszint — cél: rutinépítés: race condition kétféle javítása) A `lab-fw-uart-audit.c` modulban a `uart_get_overruns` race condition-t tartalmaz. Javítsd a hibát **két különböző módon**: (a) kritikus szakasszal, (b) atomi cserével vagy olyan átalakítással, amely elkerüli a read-modify-write-ot. Hasonlítsd össze a két megoldást az ISR-latenciára gyakorolt hatás és a hordozhatóság szempontjából.

3. (hibakeresési/mérési, középszint — cél: WCET-mérés tervezése hardveren, QEMU-ellenőrzéssel) Tervezd meg egy ISR worst-case execution time (*WCET*) mérését egy fizikai Cortex-M3-on. Milyen eszközzel méred a ciklusszámot, milyen mintavételi módszerrel, és milyen bizonytalansággal kell számolni? Írd le azt is, mit lehet ebből QEMU-n ellenőrizni (az útvonalak bejárása, az utasításszám a disassemblyből, a TIMER1-es módszer korlátai).

4. (watchdog-tervezés, elmélyült — cél: robusztussági réteg tervezése) Tervezd meg egy watchdog refresh stratégiáját egy interrupt-vezérelt firmware-hez. Hol helyezd el a refresh-hívást, milyen periódusban, és hogyan naplózd a watchdog-reset eseményt? Indokold, miért nem az ISR-ben történik a refresh.

20.4.12 Megoldások

1. megoldás — Segítség. Kérdezd meg, mit rögzít a validáció előre (bemenet, elvárt kimenet), és mit nem; melyik hibamód jelenik meg csak bizonyos időzítési vagy beágyazási kombinációban?

1. megoldás — Teljes megoldás. A funkcionális validáció előre rögzített bemenetekre és elvárt kimenetekre ad PASS/FAIL eredményt; az audit a nem funkcionális tulajdonságokat vizsgálja. Három hibamód, amelyet a validáció nem feltétlenül fed fel: (a) **race condition** – csak akkor jelentkezik, ha az ISR pont a kritikus pillanatban fut le, ami a tesztesetekben nem reprodukálható; (b) **verem-túlcsordulás** – csak a legmélyebb hívási lánc és a legtöbb beágyazott megszakítás kombinációjában; (c) **watchdog-hiány** – a rendszer funkcionálisan helyes, de egy beragadás esetén nem áll helyre. Az audit ezeket a rejtett feltételeket tárja fel.

2. megoldás — Segítség. A két javítás közül az egyik megszakítást tilt, a másik elkerüli a read-modify-write-ot. Melyik hat az ISR-latenciára, és melyik hordozható más magokra?

2. megoldás — Teljes megoldás. (a) **Kritikus szakasz:** a `lab-m19-crit.h` `crit_enter`/`crit_exit` függvénye (CMSIS-ben `__get_PRIMASK` / `__disable_irq` / `__set_PRIMASK`) néhány ciklusra tiltja a megszakításokat. Előnye: egyszerű, hordozható a Cortex-M eszközökön. Hátránya: a megszakítás-tiltás alatt az ISR-latencia nő, ezért a szakasznak rövidnek kell lennie. (b) **Szerkezeti átalakítás: egyetlen író, monoton számláló és bázis.** Ez a Záróprojekt II már használt mintája. Az ISR a `rx_overrun`-t csak növeli, a fő program soha nem írja; a fő program a saját bázisához képest számolja a különbséget:

```
/* csak az ISR írja (rx_overrun++), soha nem nullázódik */
static volatile uint32_t rx_overrun;
static uint32_t overrun_seen;          /* csak a fő program írja */

uint32_t uart_get_overruns(void)       /* az előző hívás óta történt túlcsordulások száma */
{
    uint32_t now = rx_overrun;         /* egyetlen igazított 32 bites olvasás: atomi */
    uint32_t n = now - overrun_seen;   /* modulo 2^32: átforduláskor is helyes */
    overrun_seen = now;
    return n;
}
```

Nincs megszakítás-tiltás, és nincs olyan pont, ahol a fő program az ISR írását felülírná. A pontos eseményszám megmarad: egy determinisztikus próbában, ahol a két hívás közé 2, illetve 10 túlcsordulást injektáltunk (a számláló átfordulása közben is), az API pontosan 2-t, illetve

10-et adott vissza. Egy atomi cserével (`LDREX`/`STREX`, illetve C11 `atomic_exchange`) a számláló olvasás és nullázás egy lépésben is elvégezhető; ilyenkor ellenőrizni kell, hogy a céleszközön a művelet valóban **lock-free** (`ATOMIC_INT_LOCK_FREE == 2`), és a fordító `ldrex`/`strex` sorozatot generál, nem könyvtári hívást, mert ISR-ben zárolásra építő segédfüggvény nem használható. **A számláló átfordulásának szerződése.** A modulo 2^{32} különbség (`now - seen`) addig ad pontos darabszámot, amíg két leolvasás között **kevesebb, mint 2^{32} esemény** történik; a maximális leolvasási intervallum tehát `T_max < 2³² / eseményráta`, ezt a modul dokumentációjában rögzítjük, és több teljes körre vonatkozó garanciát nem vállal. Célzott teszt: tesztstubbal a számlálót `UINT32_MAX`-hoz közeli értékről (pl. `seen = UINT32_MAX - 1`) indítjuk, két esemény után `now = 1`: a különbség `1u - (UINT32_MAX - 1u) = 3` (host-gcc-vel ellenőrizve), és az `R/S` parancsok, valamint a differenciás olvasás is ezen az átmeneten végigmenve helyes. **Nem megoldás** az, hogy az ISR csak egy jelzőbitet állít, a számlálást pedig a fő ciklus végzi. Ha két feldolgozás között több túlcsordulás történik, egyetlen 1-es jelzőből nem derül ki, hány volt: ugyanebben a próbában a jelzőbit 2 és 10 esemény után is 1-et adott. Ez a `uart_get_overruns` számlálási szerződését „történt legalább egy túlcsordulás” szerződésre cserélné, amely más követelmény. Az ISR-latenciára gyakorolt hatás: az (a) változat néhány ciklusra tiltja a megszakításokat, a (b) egyáltalán nem.

A 2/10-es és az átfordulásos próba rövid, hoston futtatható változata (a kimenet: `2`, `10`, `3`):

```c
#include <stdio.h>
#include <stdint.h>

static uint32_t now, seen;              /* now: az ISR írja; seen: a fő program */
static void inject(uint32_t n) { while (n--) now++; }
static uint32_t get(void) { uint32_t n = now; uint32_t d = n - seen; seen = n; return d; }

int main(void)
{
    inject(2);  printf("%u\n", get());          /* 2 */
    inject(10); printf("%u\n", get());          /* 10 */
    now = UINT32_MAX; seen = UINT32_MAX - 1u;   /* átfordulás előtt */
    inject(2);  printf("%u\n", get());          /* 3 */
    return 0;
}
```

3. megoldás — Segítség. A ciklusszámot hardveren a DWT `CYCCNT` adja; a QEMU 8.2.2 `mps2-an385`-ön ez nem halad. Mi marad, amit a disassemblyből és a bejárt útvonalakból ellenőrizni lehet?

3. megoldás — Teljes megoldás. A fizikai Cortex-M3-on a mérési terv: (a) a `DWT CYCCNT` engedélyezése a `DEMCR TRCENA` bitjén és a `DWT_CTRL CYCCNTENA` bitjén keresztül; (b) az ISR belépésénél a `CYCCNT` olvasása egy `volatile` változóba, kilépésnél ismét; (c) a különbség rögzítése egy tömbbe, majd statisztika (minimum, maximum, átlag, szórás); (d) a mérés megismétlése minden útvonalat bejáró bemenetekkel (például üres és tele puffer, a ciklusok maximális iterációszáma). A korlátok: a mérési pontok közötti utasítások is számítanak; a beágyazott megszakítások torzíthatják az eredményt; és a mért maximum csak a bejárt útvonalakról szól. QEMU-n a `CYCCNT` nem halad (a 8.2.2-es `mps2-an385`-ön a mérés `SKIP`, 20.3), ezért ott a mérést három dolog helyettesíti: (1) az útvonalak bejárása és a viselkedés funkcionális ellenőrzése (üres és tele puffer); (2) az utasításszám a disassemblyből (`arm-none-eabi-objdump -d`), áganként, ami a WCET-levezetés alapja; (3) a TIMER1-es módszer korlátai: a TIMER1 a TIMER0-val azonos APB-órajelen fut, ezért a két időzítő aránya értelmes, de a `tools/cm-run.py` időzítése chunk-granulált (az abszolút aktív hányad ott nem mérvadó), és a `CYCCNT`-t nem helyettesíti. Ezért a WCET-et az útvonalankénti levezetés adja, a mérés ezt ellenőrzi.

4. megoldás — Segítség. A watchdog akkor véd, ha a fő ciklus előrehaladását figyeli. Mi történik, ha az ISR tartja életben? Hogyan derül ki utólag, hogy watchdog-reset történt?

4. megoldás — Teljes megoldás. A watchdog refresh a fő ciklus **utolsó** pontján történik, közvetlenül a ciklus újrakezdése előtt. A periódus rövidebb a watchdog időtúllépésénél, de elég hosszú a normál végrehajtáshoz. A refresh nem kerülhet az ISR-be, mert az ISR a fő ciklustól függetlenül fut; ha az ISR tartja életben a watchdogot, a fő ciklus beragadása észrevétlen marad. A watchdog-reset eseményt a következő induláskor egy perzisztens regiszterből vagy nem felejtő memóriából kell kiolvasni és naplózni, különben a hiba láthatatlan.

20.4.13 Rövid összefoglaló

Ez az alfejezet strukturált auditmódszert adott a bare-metal firmware-modulokhoz. Az audit nem helyettesíti a funkcionális validációt, hanem kiegészíti: szerződések és bizonyítékok alapján tárja fel a hibamódokat: ISR-rövidség és determinizmus, race condition és megosztott állapot, verem-mélység megszakítás-beágyazás mellett, erőforráskorlát és watchdog. A checklist biztosítja, hogy egyetlen kategória se maradjon ki; a bizonyíték mérési adat, disassembly vagy reprodukálható kódrészlet. A watchdog fogalmi szinten további robusztussági réteg, amely a fő ciklus előrehaladását figyeli, és nem helyettesíti a helyes kódot. A könyv két része ugyanazt a rendszerprogramozási szemléletet tanítja két futási modellben.

Továbbhasznált definíciók:

- **audit:** független, strukturált, bizonyítékalapú vizsgálat, amely a nem funkcionális hibamódokat tárja fel.
- **worst-case execution time (WCET):** egy kódrészlet legrosszabb esetben vett futásideje; az ISR-rövidség és a CPU-terhelés számításának alapja.
- **kritikus szakasz:** olyan kódrészlet, amely alatt a megosztott állapotot érő versenyző hozzáférések tiltottak vagy atomiak.

20.4.14 Források és ellenőrzési hiányok

Az alábbiak **dokumentációs kiindulópontok** (forrásállapot: lásd az Olvasói útmutató táblázatát):

- Arm M-profile architektúra dokumentáció — M-profile kivételmodell, prioritás, veremkezelés.
- Armv7-M Architecture Reference Manual, DDI0403E — kivétel-belépés, `PRIMASK`, stack frame, `DWT` regiszterek.
- QEMU dokumentáció — `mps2-an385` machine, CMSDK perifériák, memory map.
- Procedure Call Standard for the Arm 64-bit Architecture (AAPCS64) — az I. rész hívási konvenciója.
- GNU Binutils, GCC és GDB kézikönyvek — disassembly, szimbólumkezelés, hibakeresés.

Ellenőrzési hiányok:

- A watchdog konkrét regiszterei és a `WWDG`/`IWDG` konfigurációja MCU-specifikus; a QEMU `mps2-an385` watchdog-perifériája itt nem ellenőrzött.
- A QEMU-emulált ISR-latencia és a fizikai Cortex-M3 latenciája eltér; abszolút ciklusszám nem áll rendelkezésre, csak relatív/illusztratív mérés.

- A `PRIMASK` és az atomi műveletek (`LDREX`/`STREX`, `_Atomic`) QEMU-beli viselkedése nem ellenőrzött; a lock-free fordítás a lektor mérése szerint `arm-none-eabi-gcc` 13.2.1-gyel, `-mcpu=cortex-m3` mellett igazolt (`ldrex`/`strex` sorozat, külső könyvtárhívás nélkül).
- Az `UART0_BASE = 0x40004000` cím a QEMU `mps2-an385` dokumentált memory mapjéből származó kiindulópont; a pontos machine-definíció ellenőrzést igényel.